SANCTIWALK: A MOBILE APP TOUR GUIDE SYSTEM FOR MARY HELP OF CHRISTIANS PARISH AND SAN ROQUE CATHEDRAL PARISH, DIOCESE OF KALOOKAN

Appendix: Complete Source Code

Part 1 — Mobile Application
Part 2 — Backend, Firebase and Administrative Portal
Part 3 — Maps and Map Placer
Part 4 — Unity AR Project
Part 5 — Build, Configuration and Deployment

Generated from the project source on 2026-10-23

Table of Contents

Table of Contents 2

How to Read This Appendix 13

Part 1 — Mobile Application 14

1.1 Application Entry, Routing and Shell 14

1.1.1 APP-001 — main.tsx 14

1.1.2 APP-002 — App.tsx 17

1.1.3 APP-003 — RootGate.tsx 47

1.1.4 APP-004 — PhoneContainer.tsx 49

1.1.5 APP-005 — Landing.tsx 53

1.2 Screens and Components 58

1.2.1 APP-006 — AdminPortal.tsx 58

1.2.2 APP-007 — ApplicationForm.tsx 76

1.2.3 APP-008 — ArPlaceholder.tsx 83

1.2.4 APP-009 — ArScannerPanel.tsx 85

1.2.5 APP-010 — ArTour.tsx 87

1.2.6 APP-011 — AvailabilityBadge.tsx 112

1.2.7 APP-012 — BulletinRail.tsx 114

1.2.8 APP-013 — ChangeParishModal.tsx 119

1.2.9 APP-014 — ChurchHistory.tsx 121

1.2.10 APP-015 — ChurchHistoryCard.tsx 129

1.2.11 APP-016 — CompanionTab.tsx 132

1.2.12 APP-017 — CustomDioceseMap.tsx 137

1.2.13 APP-018 — DailyRosary.tsx 139

1.2.14 APP-019 — Dashboard.tsx 143

1.2.15 APP-020 — DioceseMap.tsx 149

1.2.16 APP-021 — EditProfileCard.tsx 153
1.2.17 APP-022 — ExploreTab.tsx 158
1.2.18 APP-023 — HistoryQuiz.tsx 162
1.2.19 APP-024 — HomeHero.tsx 166
1.2.20 APP-025 — HomeQuickLinks.tsx 171
1.2.21 APP-026 — ItemDetailSections.tsx 173
1.2.22 APP-027 — LoginModal.tsx 176
1.2.23 APP-028 — MapPlaceSheet.tsx 186
1.2.24 APP-029 — MapTab.tsx 191
1.2.25 APP-030 — MassNowBadge.tsx 200
1.2.26 APP-031 — MassSchedule.tsx 202
1.2.27 APP-032 — MassScheduleCard.tsx 207
1.2.28 APP-033 — MeTab.tsx 213
1.2.29 APP-034 — MinistriesTab.tsx 221
1.2.30 APP-035 — MinistrySacramentPair.tsx 229
1.2.31 APP-036 — MyApplications.tsx 232
1.2.32 APP-037 — MyApplicationsPage.tsx 235
1.2.33 APP-038 — NotificationsCard.tsx 239
1.2.34 APP-039 — NotificationsPage.tsx 242
1.2.35 APP-040 — Onboarding.tsx 245
1.2.36 APP-041 — ParishCard.tsx 249
1.2.37 APP-042 — ParishContentEditor.tsx 251
1.2.38 APP-043 — ParishHero.tsx 257
1.2.39 APP-044 — ParishListRow.tsx 262
1.2.40 APP-045 — ParishSectionHeader.tsx 265
1.2.41 APP-046 — ParishWelcomeHeader.tsx 269

1.2.42 APP-047 — PilgrimQuiz.tsx 277
1.2.43 APP-048 — PrayScreen.tsx 282
1.2.44 APP-049 — PresenceBanner.tsx 284
1.2.45 APP-050 — PresenceSheet.tsx 285
1.2.46 APP-051 — PreviewVideo.tsx 288
1.2.47 APP-052 — PwaBanner.tsx 290
1.2.48 APP-053 — RailDots.tsx 294
1.2.49 APP-054 — ReminderSettingsCard.tsx 297
1.2.50 APP-055 — RosarySettingsModal.tsx 302
1.2.51 APP-056 — SacramentsTab.tsx 306
1.2.52 APP-057 — SanctiHost.tsx 311
1.2.53 APP-058 — SanctiSheet.tsx 320
1.2.54 APP-059 — ScannerChrome.tsx 325
1.2.55 APP-060 — SearchScreen.tsx 328
1.2.56 APP-061 — SettingDisclosure.tsx 333
1.2.57 APP-062 — SignInFirst.tsx 335
1.2.58 APP-063 — SimulatorPanel.tsx 337
1.2.59 APP-064 — Spotlight.tsx 340
1.2.60 APP-065 — TripPlanner.tsx 346
1.2.61 APP-066 — TutorialRunner.tsx 350
1.2.62 APP-067 — UpdateBanner.tsx 354
1.2.63 APP-068 — VerseCard.tsx 357
1.2.64 APP-069 — WatchBell.tsx 359
1.3 Application State and Context 362
1.3.1 APP-070 — PresenceContext.tsx 362
1.4 Shared Logic and Utilities 367

1.4.1 APP-071 — activityLog.ts 367
1.4.2 APP-072 — analytics.ts 369
1.4.3 APP-073 — announcementImage.ts 374
1.4.4 APP-074 — announcements.ts 377
1.4.5 APP-075 — apiBase.ts 382
1.4.6 APP-076 — appKey.ts 384
1.4.7 APP-077 — appUpdate.ts 386
1.4.8 APP-078 — applicationFilters.ts 390
1.4.9 APP-079 — applicationForm.ts 395
1.4.10 APP-080 — applicationView.ts 402
1.4.11 APP-081 — applications.ts 406
1.4.12 APP-082 — arApp.ts 410
1.4.13 APP-083 — askSancti.ts 414
1.4.14 APP-084 — availability.ts 417
1.4.15 APP-085 — avatar.ts 420
1.4.16 APP-086 — churches.ts 423
1.4.17 APP-087 — contentTranslations.ts 426
1.4.18 APP-088 — contrast.ts 431
1.4.19 APP-089 — deviceNotifications.ts 434
1.4.20 APP-090 — displayMode.ts 438
1.4.21 APP-091 — exportApplications.ts 440
1.4.22 APP-092 — findScroller.ts 445
1.4.23 APP-093 — followedParishes.ts 447
1.4.24 APP-094 — googleAuth.ts 450
1.4.25 APP-095 — heading.ts 454
1.4.26 APP-096 — historyNarration.ts 458

1.4.27 APP-097 — historyQuiz.ts 461
1.4.28 APP-098 — homeParish.ts 465
1.4.29 APP-099 — idToken.ts 467
1.4.30 APP-100 — itemContent.ts 469
1.4.31 APP-101 — language.ts 474
1.4.32 APP-102 — liturgical.ts 478
1.4.33 APP-103 — liturgicalColours.ts 483
1.4.34 APP-104 — mapAccuracy.ts 485
1.4.35 APP-105 — mapMarkers.ts 488
1.4.36 APP-106 — mapSearch.ts 490
1.4.37 APP-107 — massScheduleWatch.ts 496
1.4.38 APP-108 — ministryApplication.ts 502
1.4.39 APP-109 — ministrySearch.ts 506
1.4.40 APP-110 — modelRetry.ts 508
1.4.41 APP-111 — mysteries.ts 512
1.4.42 APP-112 — navigation.ts 514
1.4.43 APP-113 — notifications.ts 520
1.4.44 APP-114 — openingSubscriptions.ts 525
1.4.45 APP-115 — pageEvents.ts 528
1.4.46 APP-116 — parishIds.ts 533
1.4.47 APP-117 — parishPhotos.ts 536
1.4.48 APP-118 — parishSelection.ts 538
1.4.49 APP-119 — parishTheme.ts 540
1.4.50 APP-120 — pilgrimage.ts 543
1.4.51 APP-121 — presence.ts 547
1.4.52 APP-122 — project.ts 550

1.4.53 APP-123 — qr.ts 552
1.4.54 APP-124 — recordPageEvent.ts 555
1.4.55 APP-125 — recordVisit.ts 557
1.4.56 APP-126 — registerServiceWorker.ts 560
1.4.57 APP-127 — reminderSchedule.ts 562
1.4.58 APP-128 — rosarySettings.ts 569
1.4.59 APP-129 — routing.ts 571
1.4.60 APP-130 — sancti.ts 575
1.4.61 APP-131 — sanctiAnswers.ts 589
1.4.62 APP-132 — sanctiGrounding.ts 596
1.4.63 APP-133 — scanNarration.ts 602
1.4.64 APP-134 — scanQuota.ts 605
1.4.65 APP-135 — scanQuotaStore.ts 608
1.4.66 APP-136 — schedule.ts 612
1.4.67 APP-137 — simulatorMode.ts 616
1.4.68 APP-138 — speech.ts 619
1.4.69 APP-139 — spotlight.ts 624
1.4.70 APP-140 — sundayMasses.ts 628
1.4.71 APP-141 — tutorial.ts 632
1.4.72 APP-142 — ui.ts 636
1.4.73 APP-143 — useAuthUid.ts 640
1.4.74 APP-144 — useCamera.ts 641
1.4.75 APP-145 — useDeviceHeading.ts 647
1.4.76 APP-146 — useDragSafeClicks.ts 652
1.4.77 APP-147 — useLanguage.ts 654
1.4.78 APP-148 — useMassScheduleNotice.ts 656

1.4.79 APP-149 — useNarration.ts 659
1.4.80 APP-150 — useOpeningNotices.ts 662
1.4.81 APP-151 — useParishContent.ts 666
1.4.82 APP-152 — useParishContents.ts 668
1.4.83 APP-153 — useQrScanner.ts 670
1.4.84 APP-154 — useReminders.ts 674
1.4.85 APP-155 — useSimulatorMode.ts 677
1.4.86 APP-156 — userProfile.ts 679
1.4.87 APP-157 — verseBackdrops.ts 682
1.4.88 APP-158 — verses.ts 684
1.4.89 APP-159 — visits.ts 687
1.5 Parish Data 691
1.5.1 APP-160 — data.ts 691
1.5.2 APP-161 — types.ts 704
1.6 Styling 709
1.6.1 APP-162 — index.css 709
Part 2 — Backend, Firebase and Administrative Portal 814
2.1 Backend Server and API 814
2.1.1 API-001 — server.ts 814
2.2 Firebase, Database and Security Rules 834
2.2.1 API-002 — firebase.ts 834
2.2.2 API-003 — authFlow.ts 835
2.2.3 API-004 — parishContent.ts 841
2.2.4 API-005 — parishContentStore.ts 845
2.2.5 API-006 — firestore.rules 847
2.2.6 API-007 — firestore.indexes.json 855

2.3 Administrative Portal 856
2.3.1 API-008 — AdminAnalytics.tsx 856
2.3.2 API-009 — AdminAnnouncements.tsx 866
2.3.3 API-010 — AdminApp.tsx 877
2.3.4 API-011 — AdminApplicationDetail.tsx 880
2.3.5 API-012 — AdminApplications.tsx 886
2.3.6 API-013 — AdminAvailability.tsx 894
2.3.7 API-014 — AdminContent.tsx 898
2.3.8 API-015 — AdminDashboard.tsx 904
2.3.9 API-016 — AdminItems.tsx 909
2.3.10 API-017 — AdminLogin.tsx 913
2.3.11 API-018 — AdminMassSchedule.tsx 916
2.3.12 API-019 — AdminNav.tsx 923
2.3.13 API-020 — AdminTiles.tsx 926
2.3.14 API-021 — ParishStatus.tsx 930
2.3.15 API-022 — StatusBadge.tsx 934
Part 3 — Maps and Map Placer 936
3.1 In-App Map and Navigation 936
3.1.1 MAP-001 — DioceseMapLive.tsx 936
3.1.2 MAP-002 — NavigationOverlay.tsx 966
3.1.3 MAP-003 — CompassControl.tsx 971
3.1.4 MAP-004 — followCamera.ts 975
3.1.5 MAP-005 — geo.ts 979
3.1.6 MAP-006 — travelModes.ts 981
3.2 Station Placer 984
3.2.1 MAP-007 — place.html 984

3.2.2 MAP-008 — index.html 1000
3.2.3 MAP-009 — export.html 1007
3.2.4 MAP-010 — save_server.py 1011
3.3 Placement Pipeline and Survey Tools 1013
3.3.1 MAP-011 — deploy-stations.py 1013
3.3.2 MAP-012 — make-parish-marker.py 1020
3.3.3 MAP-013 — make-marker-pdf.py 1025
3.3.4 MAP-014 — marker-pose.py 1027
3.3.5 MAP-015 — glb-to-obj-indexed.py 1032
3.3.6 MAP-016 — optimise-scan.py 1037
Part 4 — Unity AR Project 1041
4.1 AR Runtime Scripts 1041
4.1.1 AR-001 — AdminAccess.cs 1041
4.1.2 AR-002 — AppLanguage.cs 1044
4.1.3 AR-003 — ArFlowUI.cs 1046
4.1.4 AR-004 — ChurchFrame.cs 1071
4.1.5 AR-005 — ChurchOriginTest.cs 1074
4.1.6 AR-006 — DeviceSpeech.cs 1119
4.1.7 AR-007 — FirstPersonWalkthrough.cs 1123
4.1.8 AR-008 — MarkerScan.cs 1127
4.1.9 AR-009 — MultiSetVps.cs 1129
4.1.10 AR-010 — MuseumStation.cs 1139
4.1.11 AR-011 — MuseumStationsGenerated.cs 1144
4.1.12 AR-012 — MuseumUI.cs 1159
4.1.13 AR-013 — OriginPicker.cs 1180
4.1.14 AR-014 — POIDirectory.cs 1184

4.1.15 AR-015 — POIPanel.cs 1188
4.1.16 AR-016 — PointOfInterest.cs 1191
4.1.17 AR-017 — ScanSimulator.cs 1195
4.1.18 AR-018 — SceneGizmos.cs 1197
4.1.19 AR-019 — SiteReadiness.cs 1199
4.1.20 AR-020 — StationBubble.cs 1205
4.1.21 AR-021 — StationCalibrator.cs 1213
4.1.22 AR-022 — StationMarker.cs 1222
4.1.23 AR-023 — StationRecording.cs 1225
4.2 Unity Editor Tooling 1230
4.2.1 AR-024 — ARProjectSetup.cs 1230
4.2.2 AR-025 — ArDeepLink.cs 1241
4.2.3 AR-026 — AuthoringPreview.cs 1245
4.2.4 AR-027 — DesktopTestBuild.cs 1248
4.2.5 AR-028 — ExportPois.cs 1250
4.2.6 AR-029 — ModelCheck.cs 1255
4.2.7 AR-030 — MultiSetCredentials.cs 1257
4.2.8 AR-031 — MuseumUICapture.cs 1259
4.2.9 AR-032 — ScanFlowDump.cs 1262
4.2.10 AR-033 — StationEditorScene.cs 1265
Part 5 — Build, Configuration and Deployment 1273
5.1 Web Build and Package Configuration 1273
5.1.1 CFG-001 — package.json 1273
5.1.2 CFG-002 — vite.config.ts 1275
5.1.3 CFG-003 — tsconfig.json 1278
5.1.4 CFG-004 — vitest.config.ts 1279

5.1.5 CFG-005 — vercel.json 1280
5.1.6 CFG-006 — capacitor.config.ts 1282
5.2 Android Application Configuration 1284
5.2.1 CFG-007 — build.gradle 1284
5.2.2 CFG-008 — variables.gradle 1287
5.2.3 CFG-009 — AndroidManifest.xml 1288
5.3 Unity Project Configuration 1291
5.3.1 CFG-010 — ChurchMarkers.asset 1291
Master Source Code Index 1293

If the contents above appear blank, open in Word, select all (Ctrl+A) and press F9 to build the table.

How to Read This Appendix

Every listing is the actual file from the project, copied verbatim. No code has been shortened, rewritten or summarised.

Purpose and Explanation are taken from the file's own header comment, written by the authors of the code. Where a file carries no header comment, the entry says so rather than offering a guess.

Functions and Classes are parsed from the source. Related Files are that file's own imports. Where It Is Used lists the files in this project that import it, found by searching the project rather than assumed.

Each file is printed once. Where a file would also have belonged to another part, a cross-reference gives the Code ID it was printed under.

Code IDs carry their part's prefix (APP, API, MAP, AR, CFG) and are unique across the whole appendix. The master index at the end lists every file.

Part 1 — Mobile Application

1.1 Application Entry, Routing and Shell

1.1.1 APP-001 — main.tsx

Code ID	APP-001
File Name	main.tsx
File Path	SanctDemo/src/main.tsx
Language / Type	TypeScript (React, TSX)
Lines	120
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	./components/Spotlight, ./App.tsx, ./routes/FinishSignIn.tsx, ./routes/SignUpFlow.tsx, ./admin/AdminApp.tsx, ./components/RootGate.tsx, ./components/Landing.tsx, ./index.css, ./lib/registerServiceWorker
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/main.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — main.tsx

```
import {StrictMode} from 'react';
import {createRoot} from 'react-dom/client';
import { SpotlightProvider } from './components/Spotlight';
import {BrowserRouter, Routes, Route, Navigate} from 'react-router-dom';
import App from './App.tsx';
import FinishSignIn from './routes/FinishSignIn.tsx';
import SignUpFlow from './routes/SignUpFlow.tsx';
import AdminApp from './admin/AdminApp.tsx';
import RootGate from './components/RootGate.tsx';
import Landing from './components/Landing.tsx';
import './index.css';
import { registerServiceWorker } from './lib/registerServiceWorker';

/**
 * Uncaught errors, with their stack, into the Android log.
 *
 * Capacitor forwards console output to logcat, but an uncaught TypeError
 * arrives there as one line with no file, no line number and no stack -
 * "Cannot read properties of undefined" and nothing to say who read it.
 * That is unusable on a device with no devtools attached.
 *
 * This prints the stack explicitly, tagged, so `adb logcat` can find it.
 * It only listens; it changes no behaviour and swallows nothing.
 */
window.addEventListener('error', event => {
```

```
  const err = event.error as Error | undefined;
  console.error(
    '[SW-ERROR]', event.message,
    '| at', event.filename || '(no file)', event.lineno + ':' + event.colno,
    '| stack:', err?.stack ?? '(none)',
  );
});

window.addEventListener('unhandledrejection', event => {
  const reason = event.reason as { message?: string; stack?: string } | undefined;
  console.error(
    '[SW-REJECT]', reason?.message ?? String(event.reason),
    '| stack:', reason?.stack ?? '(none)',
  );
});


/**
 * Routing was added AROUND the existing app, not through it.
 *
 * App.tsx drives its own screens from a tab held in state, and has done
 * since before there was a router. Converting those tabs into routes would
 * touch every screen in the product to gain nothing a pilgrim would notice,
 * so "/" still renders App exactly as it did and the new routes sit beside
 * it:
 *
 *   /                the pilgrim app, unchanged
 *   /signup          account creation, which is a flow rather than a screen
 *   /finish-sign-in  where the second-factor email link lands
 *   /admin/*         the parish office
 *
 * BrowserRouter rather than HashRouter: the link Firebase emails has to
 * point at a real path on an authorised domain, and Capacitor serves the
 * Android build over http://localhost, where the history API works
 * normally.
 */
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <BrowserRouter>
      <Routes>
        <Route path="/signup" element={<SignUpFlow />} />
        <Route path="/finish-sign-in" element={<FinishSignIn />} />
        {/* The parish office. Not at /admin.

            Moving it is worth almost nothing against anyone who looks -
            the path is in the JavaScript bundle, which is public - and
            it is NOT what keeps the office safe. That is Firebase Auth,
            the admins collection, and the Firestore rules Google
            enforces on their own servers. What it does buy is quiet:
            the bots that sweep the internet for /admin and /wp-admin
            all day stop finding a login form here.

            /admin itself now answers like any other unknown path, so
            the move cannot be undone by guessing the old one. */}
        <Route path="/parish-office-gav8thhe/*" element={<AdminApp />} />

        {/* Both earlier addresses are dead ends now, so neither the
            original guess nor the first move reopens the office. */}
        <Route path="/admin/*" element={<Navigate to="/" replace />} />
```

```
        <Route path="/parish-office/*" element={<Navigate to="/" replace />} />

        {/* The root is two different things.
            A phone gets the app, because somebody on a phone came to use
            it. A laptop gets the page explaining it, because somebody on
            a laptop is deciding whether to look. RootGate renders rather
            than redirects, so the address stays "/" - which matters,
            because that is the PWA's start_url and the service worker's
            cached shell. */}
        <Route path="/" element={<RootGate />} />

        {/* The app, unconditionally. Where "Explore SanctiWalk" goes, and
            the way a laptop reaches the app at all. */}
        <Route path="/app" element={<SpotlightProvider><App /></SpotlightProvider>} />

        {/* The promotional page, unconditionally - so it can be linked to
            from a poster or a post and still be readable on a phone. */}
        <Route path="/welcome" element={<Landing />} />

        {/* Everything else is the app: a catch-all rather than an exact "/"
            so any deep link the pilgrim app already understands still
            reaches it. The printed station posters encode /s/<CODE>, and
            they land here. */}
        {/* The spotlight lives inside the pilgrim app's route only. The
            parish office has no tutorial and no assistant, and a
            provider wrapping both would put a portal on the admin's
            document for no reason. */}
        <Route path="*" element={<SpotlightProvider><App /></SpotlightProvider>} />
      </Routes>
    </BrowserRouter>
  </StrictMode>,
);

// Makes the web build installable on a home screen, and openable
// inside a church with no signal. A no-op in the APK and in dev.
registerServiceWorker();
```

1.1.2 APP-002 — App.tsx

Code ID	APP-002
File Name	App.tsx
File Path	SanctDemo/src/App.tsx
Language / Type	TypeScript (React, TSX)
Lines	1664
Functions / Classes	ParishTheme (internal function); SanctiMount (internal function); PresenceParishSync (internal function); firstLiveParishId (internal function); routeForHomeParish (internal function); App (exported component/function (default))
Related Files (its imports)	./context/PresenceContext, ./components/PhoneContainer, ./components/Onboarding, ./components/SearchScreen, ./components/PrayScreen, ./components/PwaBanner, ./components/Dashboard, ./components/HomeQuickLinks, ./components/MeTab, ./components/ChurchHistory, ./components/MassSchedule, ./components/MinistriesTab, ./components/SacramentsTab, ./components/ArTour, ./components/PilgrimQuiz, ./components/LoginModal, ./lib/displayMode, ./components/AdminPortal, ./components/RosarySettingsModal, ./components/PresenceSheet, ./components/SimulatorPanel, ./components/CustomDioceseMap, ./lib/firebase, ./lib/activityLog
Where It Is Used	src/components/RootGate.tsx, src/main.tsx
How to Recognize It	Open SanctDemo/src/App.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — App.tsx

```
import React, { useState, useEffect, useMemo, useRef, useCallback } from "react";
import { PresenceProvider, usePresence } from "./context/PresenceContext";
import PhoneContainer from "./components/PhoneContainer";
import Onboarding from "./components/Onboarding";
import SearchScreen from "./components/SearchScreen";
import PrayScreen from "./components/PrayScreen";
import PwaBanner from "./components/PwaBanner";

// New Components
import Dashboard from "./components/Dashboard";
import { type HomeSection } from "./components/HomeQuickLinks";
import MeTab from "./components/MeTab";
import ChurchHistory from "./components/ChurchHistory";
import MassSchedule from "./components/MassSchedule";
import MinistriesTab from "./components/MinistriesTab";
import SacramentsTab from "./components/SacramentsTab";
import ArTour from "./components/ArTour";
import PilgrimQuiz from "./components/PilgrimQuiz";
import LoginModal from "./components/LoginModal";
import { useDeviceSurface } from "./lib/displayMode";
import AdminPortal from "./components/AdminPortal";
```

```
import RosarySettingsModal from "./components/RosarySettingsModal";
import PresenceSheet from "./components/PresenceSheet";
import SimulatorPanel from "./components/SimulatorPanel";
import CustomDioceseMap from "./components/CustomDioceseMap";

// Firebase imports
import { auth, db } from "./lib/firebase";
import { logActivity } from "./lib/activityLog";
import { recordChurchVisit, touchActivity } from "./lib/recordVisit";
import { getProfile } from "./lib/userProfile";
import { watchNotifications, unreadCount } from "./lib/notifications";
import { useReminders } from "./lib/useReminders";
import { useParishContents } from "./lib/useParishContents";
import type { ParishReminders } from "./lib/reminderSchedule";
import {
  loadFollowed, saveFollowed, withFollowed, remindableParishes,
} from "./lib/followedParishes";
import { notifyNow } from "./lib/deviceNotifications";
import { announcementsForParish, publishedOnly } from "./lib/announcements";
import ReminderSettingsCard from "./components/ReminderSettingsCard";
import TutorialRunner from "./components/TutorialRunner";
import SanctiHost from "./components/SanctiHost";
import { hasSeenTutorial, type TutorialTab } from "./lib/tutorial";
import { useLanguage } from "./lib/useLanguage";
import { useSimulatorMode } from "./lib/useSimulatorMode";
import { t } from "./lib/ui";
import MyApplicationsPage from "./components/MyApplicationsPage";
import NotificationsPage from "./components/NotificationsPage";
import { onAuthStateChanged, signOut } from "firebase/auth";
import { collection, onSnapshot, doc, setDoc, addDoc, deleteDoc, updateDoc, getDoc, getDocs, writeBatch, query, where } from "firebase/firestore";

import { Route, UserProgress } from "./types";
import { ROUTES, BADGES, MASS_SCHEDULES, MINISTRIES, SACRAMENTS } from "./data";
import { loadHomeParishId, saveHomeParishId } from "./lib/homeParish";
import { tabForParishSelection } from "./lib/parishSelection";
import { parishPhoto, parishPhotoAlt } from "./lib/parishPhotos";
import parishData from "./data/diocese-parishes.json";
import { assertKnownParishIds, routeIdForParish } from "./lib/parishIds";
import UpdateBanner from "./components/UpdateBanner";
import { recordPageEvent } from "./lib/recordPageEvent";
import { useMassScheduleNotice } from "./lib/useMassScheduleNotice";
import { useOpeningNotices } from "./lib/useOpeningNotices";
import { useParishContent } from "./lib/useParishContent";
import { checkParishColour, readableTextOn, parseHex } from "./lib/contrast";

import {
  Compass, Map, Cpu, Sparkles, BookOpen, Clock, Heart,
  Menu, X, Home, Lock, HelpCircle, User, ShieldCheck, HelpCircle as QuizIcon,
  ScanLine as ArIcon, Users as MinistryIcon, MapPin, ChevronRight, Bookmark,
  Settings as SettingsIcon,
  Church, Smartphone, Monitor, Wifi, WifiOff, FlaskConical
} from "lucide-react";

// Watches presence from inside the provider and reports an arrival upward.
// App itself renders PresenceProvider, so it cannot call usePresence(); this
// tiny child can. Only `present` switches the dashboard — `approaching` is
// still just passing by, and switching then would yank the screen around
// while someone walks down the street.
/**
```

```
 * Applies the active parish's admin-chosen colour to the app's tokens.
 *
 * The text colour placed ON that background is derived, never chosen: an
 * admin picks one value and the pairing is computed, so no combination of
 * choices can produce unreadable text. The colour is re-validated here as
 * well as in the editor, because a document written before the validator
 * existed — or edited straight in the Firebase console — would otherwise
 * bypass it entirely.
 */
function ParishTheme({ parishId }: { parishId: string | null }) {
  const managed = useParishContent(parishId);
  const colour = managed?.themeColor;

  useEffect(() => {
    const root = document.documentElement;
    const clear = () => {
      root.style.removeProperty("--color-brand-primary");
      root.style.removeProperty("--color-brand-accent");
      root.style.removeProperty("--color-brand-on-accent");
    };

    if (!colour) {
      clear();
      return;
    }
    const verdict = checkParishColour(colour);
    const rgb = parseHex(colour);
    if (!verdict.ok || !rgb) {
      console.warn(`[SanctiWalk] Ignoring parish colour ${colour}: ${verdict.problem}`);
      clear();
      return;
    }

    root.style.setProperty("--color-brand-primary", colour);
    root.style.setProperty("--color-brand-accent", colour);
    root.style.setProperty("--color-brand-on-accent", readableTextOn(rgb).hex);
    return clear;
  }, [colour]);

  return null;
}

/**
 * Sancti needs the pilgrim's position to answer "how far is it", and
 * App renders PresenceProvider so it cannot read the context itself.
 * This sits inside the provider and hands it down.
 */
function SanctiMount({
  activeParishId, activeParishName, content, language,
  onGo, onShowHomeSection, onOpenItem, onSelectParish, onWalkTo, onFollowParish,
  onReplayTutorial, onSetLanguage,
}: {
  activeParishId: string;
  activeParishName: string;
  content: import("./lib/parishContent").ParishContent | null;
  language: "en" | "fil";
  onGo: (tab: string) => void;
  onShowHomeSection: (section: HomeSection) => void;
  onOpenItem: (tab: "ministries" | "sacraments", itemId: string) => void;
```

```
  onReplayTutorial: () => void;
  onSetLanguage: (language: "en" | "fil") => void;
  onSelectParish: (parishId: string) => void;
  onWalkTo: (parishId: string) => void;
  onFollowParish: (parishId: string) => void;
}) {
  const { position } = usePresence();
  return (
    <SanctiHost
      tools={{
        go: onGo,
        showHomeSection: onShowHomeSection,
        openItem: onOpenItem,
        replayTutorial: onReplayTutorial,
        setLanguage: onSetLanguage,
        selectParish: onSelectParish,
        walkTo: onWalkTo,
        followParish: onFollowParish,
        activeParishId,
        activeParishName,
        content,
        position,
        language,
      }}
    />
  );
}

function PresenceParishSync({ onArrive }: { onArrive: (parishId: string) => void }) {
  const { presence } = usePresence();

  useEffect(() => {
    if (presence.mode === "present" && presence.parishId) {
      onArrive(presence.parishId);
      // The parish's own count of estimated visits.
      //
      // Hung off the presence transition rather than off a new
      // geofence: presence.ts has already decided the pilgrim is
      // inside the parish's radius AND has stayed past the dwell
      // gate, which is the hard half of the problem and is already
      // under test. recordChurchVisit only deduplicates and writes.
      void recordChurchVisit(presence.parishId);
    }
  }, [presence.mode, presence.parishId, onArrive]);

  return null;
}

// The fallback parish used wherever a home parish is needed but none has
// been chosen — the map's own default, and the tour behind a home parish
// that has no tour of its own.
//
// This is also the answer on first run: the app picks a parish rather than
// asking, because the client does not want a welcome screen standing between
// install and the dashboard.
function firstLiveParishId(): string | null {
  return ROUTES.find(r => r.status !== "coming_soon")?.id ?? ROUTES[0]?.id ?? null;
}
```

```
// A home parish may be any of the 31 in the diocese, not just the two with a
// tour behind them — the parish picker lists them all, and a pilgrim's
// own parish is very likely one of the 29 still being documented. The active
// *tour* must still be one of the live routes, so the two are mapped rather
// than conflated.
const DIOCESE_PARISH_IDS = (parishData as { parishes: { id: string }[] }).parishes.map(p => p.id);
const VALID_HOME_IDS = [...ROUTES.map(r => r.id), ...DIOCESE_PARISH_IDS];

/** The tour to open for a chosen home parish, falling back to a live one. */
function routeForHomeParish(homeId: string | null): string | null {
  if (!homeId) return firstLiveParishId();
  if (ROUTES.some(r => r.id === homeId)) return homeId;
  return routeIdForParish(homeId) ?? firstLiveParishId();
}

// Checked once at startup. The bug this guards is a silence, not a crash: a
// stale id makes a documented parish render as though nothing had been
// collected about it, and every screen dutifully says "not collected yet".
//
// Reported rather than thrown — a white screen during a defense demo would
// be a worse failure than the one being guarded against.
try {
  assertKnownParishIds();
} catch (error) {
  console.error("[SanctiWalk]", error instanceof Error ? error.message : error);
}

export default function App() {
  // The pilgrim's home parish, chosen once and remembered. This is the
  // fallback the dashboard shows whenever GPS says they are not at a parish.
  const [homeParishId, setHomeParishId] = useState<string | null>(() =>
    loadHomeParishId(VALID_HOME_IDS) ?? firstLiveParishId()
  );
  /**
   * A request from outside the dashboard for one part of the bulletin.
   *
   * Sancti sets this when the pilgrim accepts "want me to open the Mass
   * schedule?". The id makes a repeat request its own event; see
   * Dashboard's sectionRequest.
   */
  const [homeSectionRequest, setHomeSectionRequest] =
    useState<{ section: HomeSection; id: number } | null>(null);
  /** A request to open one named ministry or sacrament. See itemRequest. */
  const [itemRequest, setItemRequest] = useState<
    { tab: "ministries" | "sacraments"; itemId: string; id: number } | null
  >(null);
  const openItem = useCallback((tab: "ministries" | "sacraments", itemId: string) => {
    setItemRequest(prev => ({ tab, itemId, id: (prev?.id ?? 0) + 1 }));
  }, []);

  const showHomeSection = useCallback((section: HomeSection) => {
    setHomeSectionRequest(prev => ({ section, id: (prev?.id ?? 0) + 1 }));
  }, []);
  const [isChangeParishOpen, setIsChangeParishOpen] = useState(false);
  const [isSearchOpen, setIsSearchOpen] = useState(false);
  const [isSignInOpen, setIsSignInOpen] = useState(false);

  // There is deliberately no first-run parish picker. On a fresh install
  // homeParishId is seeded with firstLiveParishId(), so the app already knows
```

```
// a sensible parish to open on and asking would be a question it can answer
// itself. The picker below still exists, reached from "Change Home Parish"
// in Me, for the pilgrim who wants a different one.

// Navigation & Frame settings. Seeded from the home parish so the app opens
// on that parish's dashboard rather than asking which church to pick — the
// app can work that out, so it should not ask.
const [selectedChurchId, setSelectedChurchId] = useState<string | null>(() =>
  routeForHomeParish(loadHomeParishId(VALID_HOME_IDS))
);
type Tab =
  | "home" | "navigator" | "rosary" | "mass" | "ministries" | "history"
  | "sacraments" | "ar" | "quiz" | "me" | "admin" | "pwa-devkit"
  | "myApplications" | "notifications";

/**
 * The tab, remembered across a restart.
 *
 * Not a convenience. Android's low-memory killer terminates this app
 * while it is in the background - every recorded exit on the test
 * phone was reason=3 LOW_MEMORY, not a crash - and a WebView app
 * carrying a map is a large, attractive target. When that happens the
 * app restarts from scratch, and landing back on Home after having
 * been on the Map is exactly what "the map crashed" looks like from
 * the outside.
 *
 * It cannot stop the kill. It can make coming back feel like coming
 * back rather than starting over.
 *
 * "ar" and "admin" are deliberately NOT restored: reopening straight
 * into the camera, or into the parish office, is not what someone
 * returning to the app expects.
 */
const [activeTab, setActiveTab] = useState<Tab>(() => {
  try {
    const saved = localStorage.getItem("sanctiwalk.tab");
    const restorable: Tab[] = ["home", "navigator", "rosary", "mass", "me"];
    if (saved && (restorable as string[]).includes(saved)) return saved as Tab;
  } catch {
    // Private mode, or storage blocked. Home is the right fallback.
  }
  return "home";
});

// Tells the parish dashboard somebody is using the app.
//
// Driven by the tab, which is the coarsest signal that means "still
// here" - and throttled inside touchActivity to one write per five
// minutes, so a long visit costs a handful of writes rather than one
// per screen. Firestore on the free plan is billed by the document.
useEffect(() => { void touchActivity(); }, [activeTab]);

useEffect(() => {
  try {
    localStorage.setItem("sanctiwalk.tab", activeTab);
  } catch {
    // Nothing to do; the tab simply will not be remembered.
  }
}, [activeTab]);
```

```
// Where "Back" goes from the history page. It is opened from two places -
// the Learn more on Home's Church History card, and the parish's own page
// reached from the map - and a single hardcoded destination would strand
// whoever came from the other one.
// History is reached from the dashboard now that the separate parish
// page is gone, so there is only one place to go back to.
const [historyReturnTab] = useState<"home">("home");

// Whether Scan's viewfinder is open, so the parish band can get out of
// the way of a live camera.
const [scanCameraLive, setScanCameraLive] = useState(false);



const [isOffline, setIsOffline] = useState(false);
const [isMobileOnly, setIsMobileOnly] = useState(true);

// A real phone gets the whole screen. The centring and the 24px of
// breathing room below only make sense around the desktop mockup; applied
// to the installed app they inset it from the edges it should be using,
// and left the fixed-position screens (sign-in, search) looking like a
// different app at a different size.
const deviceSurface = useDeviceSurface();
const [isRosarySettingsOpen, setIsRosarySettingsOpen] = useState(false);
const [isSimulatorOpen, setIsSimulatorOpen] = useState(false);

// Identity state
const [isLoggedIn, setIsLoggedIn] = useState(false);
const [userEmail, setUserEmail] = useState("");
// The greeting name.
//
// The pilgrim's own nickname when they have set one, which is the whole
// point of having one. The email's first word is the fallback, and it
// was the only thing here before - which is how someone came to be
// greeted as "sanctiwalk" on their own home screen.
const [nickname, setNickname] = useState<string | undefined>(undefined);
/** The pilgrim's own parish, from their profile. Home returns here. */
const [myParishId, setMyParishId] = useState<string | null>(null);
const [unreadNotifications, setUnreadNotifications] = useState(0);

const greetingName =
  (isLoggedIn ? nickname?.trim() : "") ||
  (isLoggedIn && userEmail ? userEmail.split("@")[0].split(/[._-]/)[0] : undefined);

const [isAdmin, setIsAdmin] = useState(false);
const [uid, setUid] = useState<string | null>(null);
/**
 * Unread count, and a phone notification for anything new.
 *
 * The seen-set is what stops the second half firing for history. On
 * the first snapshot every existing notification is "new" to this
 * listener, and without recording them first, opening the app would
 * put every decision the parish ever made into the notification tray
 * at once.
 */
const seenNotificationsRef = useRef<Set<string> | null>(null);
useEffect(() => {
  if (!isLoggedIn) {
```

```
    setUnreadNotifications(0);
    seenNotificationsRef.current = null;
    return;
  }
  return watchNotifications(items => {
    setUnreadNotifications(unreadCount(items));

    const ids = new Set(items.map(n => n.id).filter(Boolean) as string[]);
    if (seenNotificationsRef.current === null) {
      // First snapshot: adopt it silently.
      seenNotificationsRef.current = ids;
      return;
    }

    if (!remindersRef.current.settings.applicationUpdates) {
      seenNotificationsRef.current = ids;
      return;
    }

    for (const item of items) {
      if (!item.id || seenNotificationsRef.current.has(item.id)) continue;
      if (item.readAt) continue;
      // Raised on the phone, not only in the app. This is the one
      // category that cannot be scheduled in advance - the parish
      // decides when it decides - so it arrives when the app is next
      // open. See lib/deviceNotifications.ts for why not push.
      void notifyNow(item.title, item.body);
    }
    seenNotificationsRef.current = ids;
  });
}, [isLoggedIn, uid]);
// Read once per sign-in, then kept current by Me's editor rather than
// re-read on every render.
useEffect(() => {
  if (!isLoggedIn || !uid) { setNickname(undefined); return; }
  let live = true;
  void getProfile(uid).then(p => {
    if (!live) return;
    setNickname(p?.nickname ?? undefined);
    // The parish chosen at signup. Home belongs to it: it is the one
    // the pilgrim actually attends, and the one their applications go
    // to. Anything else on screen is a visit.
    if (p?.churchId) {
      setMyParishId(p.churchId);
      setSelectedChurchId(current => current ?? p.churchId);
    }
  });
  return () => { live = false; };
}, [isLoggedIn, uid]);

// The applications collection in Firestore: ministry applications and
// sacrament bookings share it, distinguished by `type`.
//
// The first six fields are what every application has had from the start
// and what the existing lists render. The rest arrive on ministry
// applications (see lib/ministryApplication.ts) and are optional here
// because the sacrament bookings and the older seeded rows do not carry
// them - reading ministry or parish off `details` was the alternative,
// and parsing a sentence is not a data model.
```

```
const [applications, setApplications] = useState<Array<{
  id: string;
  type: string;
  applicant: string;
  details: string;
  date: string;
  status: string;
  email?: string;
  mobile?: string;
  ministryId?: string;
  ministryName?: string;
  parishId?: string;
  parishName?: string;
  message?: string;
  consent?: boolean;
  submittedAt?: string;
}>>([]);

// Dynamic Announcements list
const [announcements, setAnnouncements] = useState<Array<{
  id: string;
  title: string;
  date: string;
  time: string;
  type: string;
}>>([]);

// Dynamic Devotee Stations Comments Database
const [commentsByStation, setCommentsByStation] = useState<Record<string, Array<{
  user: string;
  text: string;
  date: string;
}>>>({
  "mhcp-altar": [
    { user: "Mary Grace", text: "Truly serene altar. It is the best place to pray in Maypajo.", date: "May 14" },
    { user: "Bro. Justine", text: "The wood carvings are outstanding! A true catechism piece.", date: "May 15" }
  ],
  "mhcp-patron": [
    { user: "Nanay Corazon", text: "Maria Auxiliadora, pray for our family during sick days.", date: "May 12" }
  ]
});

const [activeCommentInput, setActiveCommentInput] = useState("");

// User Progress passport status state
const [userProgress, setUserProgress] = useState<UserProgress>({
  completedStations: [],
  completedRoutes: [],
  badges: [],
  steps: 640,
  distanceKm: 0.42,
  points: 120
});

// 1. Listen to Auth State changes in Firebase
useEffect(() => {
  const unsubscribeAuth = onAuthStateChanged(auth, async (user) => {
    if (user) {
      setIsLoggedIn(true);
```

```
      setUserEmail(user.email || "");
      setUid(user.uid);

      // Admin status is derived from membership in the "admins" collection
      // (one document per uid, granted only from the Firebase console),
      // never from anything the client controls like the email address.
      try {
        const adminSnap = await getDoc(doc(db, "admins", user.uid));
        setIsAdmin(adminSnap.exists());
      } catch (err) {
        console.error("Error checking admin status from Firestore:", err);
        setIsAdmin(false);
      }

      // Fetch / sync user profile document in Firestore
      const userDocRef = doc(db, "users", user.uid);
      const unsubscribeUserDoc = onSnapshot(
        userDocRef,
        (docSnap) => {
          if (docSnap.exists()) {
            const data = docSnap.data();
            setUserProgress({
              completedStations: data.completedStations || [],
              completedRoutes: data.completedRoutes || [],
              badges: data.badges || [],
              steps: data.steps || 640,
              distanceKm: data.distanceKm || 0.42,
              points: data.points || 120
            });
          }
        },
        (error) => {
          console.error("Error listening to user profile document (check Firestore rules / sign-in state):", error);
        }
      );

      return () => {
        unsubscribeUserDoc();
      };
    } else {
      setIsLoggedIn(false);
      setUserEmail("");
      setIsAdmin(false);
      setUid(null);
    }
  });

  return () => {
    unsubscribeAuth();
  };
}, []);

// 2. Listen to Applications collection in Firestore (with dynamic seeding if database empty).
// Under the Firestore rules, a signed-out user cannot read this collection
// at all, and a signed-in non-admin can only read documents whose "uid"
// matches their own uid. Subscribe accordingly so we never attempt a read
// the rules will reject.
useEffect(() => {
  if (!isLoggedIn || !uid) {
```

```
    setApplications([]);
    return;
  }

  const applicationsQuery = isAdmin
    ? collection(db, "applications")
    : query(collection(db, "applications"), where("uid", "==", uid));

  const unsub = onSnapshot(
    applicationsQuery,
    (snapshot) => {
      const items: any[] = [];
      snapshot.forEach((doc) => {
        items.push({ id: doc.id, ...doc.data() });
      });

      if (items.length === 0 && isAdmin) {
        // Seed initial applications to Firestore if empty (admin only —
        // a non-admin write here would be rejected by the rules anyway,
        // and every application must carry an owning uid).
        const initialApps = [
          {
            type: "Sacrament Booking",
            applicant: "Christian dela Vega",
            details: "Holy Baptism - May 24, 2026 (Sponsor: Maria Santos)",
            date: "05/12/2026",
            status: "Awaiting Parish Interview",
            uid,
            createdAt: new Date().toISOString()
          },
          {
            type: "Ministry Application",
            applicant: "Justine Valenzuela",
            details: "SOCOM (Social Communications) - Cameraman Volunteer",
            date: "05/14/2026",
            status: "Interview Scheduled",
            uid,
            createdAt: new Date().toISOString()
          }
        ];
        initialApps.forEach(async (app) => {
          await addDoc(collection(db, "applications"), app);
        });
      } else {
        // Sort items by a createdAt timestamp or doc id to keep consistent order
        items.sort((a, b) => b.id.localeCompare(a.id));
        setApplications(items);
      }
    },
    (error) => {
      console.error("Error listening to applications collection (check Firestore rules / sign-in state):", error);
    }
  );

  return () => unsub();
}, [isLoggedIn, isAdmin, uid]);

// 3. Listen to Announcements collection in Firestore (with dynamic seeding if database empty)
useEffect(() => {
```

```
  const unsub = onSnapshot(
    collection(db, "announcements"),
    (snapshot) => {
      const items: any[] = [];
      snapshot.forEach((doc) => {
        items.push({ id: doc.id, ...doc.data() });
      });

      if (items.length === 0 && isAdmin) {
        // Seed initial announcements to Firestore if empty. Announcement
        // writes are admin-only under the rules, so only attempt this
        // once we know the current user is an admin.
        const initialAnns = [
          { title: "Feast of St. Joseph Mass", date: "May 1, 2026", time: "3:00 PM", type: "Mass", createdAt: new Date().toISOString() },
          { title: "Consecration to Maria Auxiliadora", date: "May 13, 2026", time: "3:00 PM", type: "Feast", createdAt: new Date().toISOString() },
          { title: "7th Sunday of Easter Liturgy", date: "May 17, 2026", time: "3:00 PM", type: "Mass", createdAt: new Date().toISOString() }
        ];
        initialAnns.forEach(async (ann) => {
          await addDoc(collection(db, "announcements"), ann);
        });
      } else {
        items.sort((a, b) => (b.createdAt || "").localeCompare(a.createdAt || ""));
        setAnnouncements(items);
      }
    },
    (error) => {
      console.error("Error listening to announcements collection:", error);
    }
  );

  return () => unsub();
}, [isAdmin]);

const activeChurchRoute = ROUTES.find(r => r.id === selectedChurchId) || ROUTES[0];

/**
 * The phone's reminder alarms.
 *
 * Built from the parishes the pilgrim FOLLOWS, not the one they
 * happen to be looking at. Those are different questions, and
 * conflating them meant that glancing at another parish's Mass times
 * silently replaced every reminder for their own — and glancing back
 * replaced them again.
 */
/**
 * The first-run walkthrough.
 *
 * Opened from a lazy initialiser rather than an effect, so it is
 * decided once on mount. An effect would re-evaluate on re-render and
 * could reopen the tour under somebody who had just skipped it.
 */
const [tutorialOpen, setTutorialOpen] = useState(() => !hasSeenTutorial());

/** One reading language for every screen. See lib/language.ts. */
const { language, setLanguage } = useLanguage();
// Whether the app is behaving as it will for a visitor. See
// lib/simulatorMode.ts; the Me screen explains both positions.
const { simulatorMode, setSimulatorMode } = useSimulatorMode();
```

```
const [followedParishes, setFollowedParishes] = useState<string[]>(loadFollowed);
const remindableIds = useMemo(
  () => remindableParishes(followedParishes, homeParishId),
  [followedParishes, homeParishId],
);
/**
 * The parish on screen is watched too, not only the followed ones.
 *
 * Sancti answers about whatever parish you are looking at, and
 * subscribing only to the followed set meant it fell back to the
 * compiled Mass times the moment you asked about a parish you were
 * merely visiting - quoting times the office may have changed.
 */
const watchedParishIds = useMemo(
  () => (remindableIds.includes(activeChurchRoute.id)
    ? remindableIds
    : [...remindableIds, activeChurchRoute.id]),
  [remindableIds, activeChurchRoute.id],
);
const followedContent = useParishContents(watchedParishIds);

/**
 * The parish on screen, as the office has it.
 *
 * Sancti answers from this rather than from the compiled data, so a
 * Mass time the office changed this morning is the one it quotes.
 * Null for a parish nobody has edited, which every answer treats as
 * "fall back to what shipped" rather than as "no information".
 */
const activeParishContent = followedContent[activeChurchRoute.id] ?? null;

const reminderParishes = useMemo<ParishReminders[]>(
  () => remindableIds.map(id => {
    const route = ROUTES.find(r => r.id === id);
    return {
      parishId: id,
      parishName: route?.name.replace(" Guide", "").replace(" Tour", "") ?? id,
      // The parish's own times where an admin has entered them, the
      // compiled ones otherwise — the same precedence the Mass card
      // uses, so a reminder and the card can never disagree.
      massSchedule: followedContent[id]?.massSchedule ?? MASS_SCHEDULES[id]?.schedule ?? [],
      announcements: publishedOnly(announcementsForParish(announcements, id)),
    };
  }),
  [remindableIds, followedContent, announcements],
);

const reminders = useReminders(reminderParishes);

const toggleFollowParish = useCallback((parishId: string, follow: boolean) => {
  setFollowedParishes(current => {
    const next = withFollowed(current, parishId, follow, homeParishId);
    saveFollowed(next);
    return next;
  });
}, [homeParishId]);

// Read by the notification watcher above, which is declared earlier and
// must not re-subscribe every time a setting changes.
```

```
const remindersRef = useRef(reminders);
remindersRef.current = reminders;

// Shown only when the dashboard is on a parish that is not the
// pilgrim's own - which happens two ways, both deliberate: walking near
// another parish, or searching for one. Offering "back" while already
// home would be a button that does nothing.
//
// "Own" means the profile's parish when signed in and the locally chosen
// home parish otherwise. Keying it on the profile alone made the way back
// vanish for everyone signed out: myParishId is null until a profile
// loads, so a visitor who searched their way to another parish was left
// on it with no route home. homeParishId is never null - it is seeded
// with firstLiveParishId() on a fresh install - so the button is always
// there when it is needed.
//
// Both are resolved to a ROUTE id before comparing. They are not stored in
// the same id space - a home parish may be any of the 31 diocese ids,
// while the dashboard always shows one of the two routes - so comparing
// them raw made "parish-mary-help-of-christians-parish" look different
// from "route-mhcp", and the way back was offered to a pilgrim already
// standing in their own parish.
const ownParishId = routeForHomeParish(myParishId ?? homeParishId);

// The Mass schedule is the one thing in the app somebody acts on
// physically: they read it, then they leave the house at a certain
// time. When the parish moves a Mass, anyone who checked last week
// is now wrong and nothing on a screen they are not looking at will
// say so. Home parish only - a notification about a church somebody
// browsed once is spam.
useMassScheduleNotice(
  ownParishId,
  ROUTES.find(r => r.id === ownParishId)?.name.replace(" Guide", "").replace(" Tour", "")
    ?? "Your parish",
);

// One line in the parish office's figures: somebody opened it.
// Deduped to once per browser per half hour inside recordPageEvent,
// because a reload is not a person arriving.
useEffect(() => {
  void recordPageEvent("open");
}, []);

// And the same for anything a pilgrim has put a bell on. A ministry
// that opens for a fortnight a year is the case this exists for:
// without it the only way to catch it is to keep opening the app and
// reading the same "not available" badge until the one day it is not.
useOpeningNotices(
  ownParishId,
  ROUTES.find(r => r.id === ownParishId)?.name.replace(" Guide", "").replace(" Tour", "")
    ?? "Your parish",
  itemId =>
    MINISTRIES.find(m => m.id === itemId)?.name
    ?? SACRAMENTS.find(s => s.id === itemId)?.name
    ?? "A ministry",
);

const viewingAnotherParish =
  ownParishId !== null && activeChurchRoute.id !== ownParishId;
```

```
const backToMyParish = () => {
  if (ownParishId) setSelectedChurchId(ownParishId);
  setActiveTab("home");
};

// Presence sheet actions: the pilgrim may be physically near a parish they
// haven't selected in-app yet (e.g. they came straight from the church
// selector), so opening the tour or AR screen also switches the active
// church context to the one presence detected. "Open Tour" (map icon,
// next to the sheet's own "AR Tour" button) means "show me the diocese
// map" — the pilgrim is already standing at the parish, so jumping to the
// map tab is the correct destination here, distinct from selecting a
// parish elsewhere (see handleSelectParish below).
const [walkToParishId, setWalkToParishId] = useState<string | null>(null);

const handleOpenTourFromPresence = (parishId: string) => {
  setSelectedChurchId(parishId);
  setActiveTab(tabForParishSelection("presence-open-tour"));
};

// "Get directions" on the Home hero: switch to the map and hand it the parish
// to route to, rather than routing from Home and hoping the map picks it up.
// Cleared once the map has consumed it so returning to the tab later does
// not silently redraw a route the pilgrim did not ask for again.
const handleWalkThere = (parishId: string) => {
  setWalkToParishId(parishId);
  setActiveTab("navigator");
};

const handleOpenARFromPresence = (parishId: string) => {
  setSelectedChurchId(parishId);
  setActiveTab("ar");
};

// Selecting a parish from a map pin, a search result, or a parish card —
// anywhere the pilgrim is picking *which parish* to look at — opens that
// parish's own profile (the "home" tab: Dashboard, with its Mass
// schedule/History/Ministries/Sacraments links), not the map tab. Using
// "navigator" here was the bug behind "View parish does nothing": when
// the map pin lives on the map tab itself (App.tsx's "navigator" tab),
// setActiveTab("navigator") is a no-op because that tab is already active.
// Matches the "Start Sanctuary Walk" card button in the church-selector
// screen below, which already does exactly this for the same action.
const handleSelectParish = (parishId: string) => {
  setSelectedChurchId(parishId);
  setActiveTab(tabForParishSelection("pin"));
};

// Global methods to update progress
const addPoints = async (pointsToAdd: number) => {
  setUserProgress(prev => {
    const newProgress = {
      ...prev,
      points: prev.points + pointsToAdd
    };
    if (auth.currentUser) {
      setDoc(doc(db, "users", auth.currentUser.uid), newProgress, { merge: true });
    }
```

```
    return newProgress;
  });
};

const earnBadge = async (badgeId: string) => {
  setUserProgress(prev => {
    if (prev.badges.includes(badgeId)) return prev;
    const newProgress = {
      ...prev,
      badges: [...prev.badges, badgeId]
    };
    if (auth.currentUser) {
      setDoc(doc(db, "users", auth.currentUser.uid), newProgress, { merge: true });
    }
    return newProgress;
  });
};

const handleAddApplication = async (newApp: Omit<typeof applications[0], "id">) => {
  if (!auth.currentUser) {
    // The Firestore rules require every application to carry the uid of
    // its signed-in creator, so an anonymous write would be rejected
    // anyway. Fail loudly here instead of letting it fail silently.
    console.error("Cannot submit application: no signed-in user.");
    alert("Please sign in first before submitting this form.");
    return;
  }
  try {
    await addDoc(collection(db, "applications"), {
      ...newApp,
      uid: auth.currentUser.uid,
      createdAt: new Date().toISOString()
    });
  } catch (err) {
    console.error("Error adding application to Firestore:", err);
  }
};

const handleDeleteApplication = async (id: string) => {
  try {
    await deleteDoc(doc(db, "applications", id));
  } catch (err) {
    console.error("Error deleting application from Firestore:", err);
  }
};

const handleUpdateApplicationStatus = async (id: string, newStatus: string) => {
  try {
    await updateDoc(doc(db, "applications", id), {
      status: newStatus
    });
  } catch (err) {
    console.error("Error updating application status in Firestore:", err);
  }
};

const handleAddAnnouncement = async (newAnn: Omit<typeof announcements[0], "id">) => {
  try {
    await addDoc(collection(db, "announcements"), {
```

```
      ...newAnn,
      createdAt: new Date().toISOString()
    });
  } catch (err) {
    console.error("Error adding announcement to Firestore:", err);
  }
};

const handleDeleteAnnouncement = async (id: string) => {
  try {
    await deleteDoc(doc(db, "announcements", id));
  } catch (err) {
    console.error("Error deleting announcement from Firestore:", err);
  }
};

const handleLoginSuccess = (email: string, adminFlag: boolean) => {
  setIsLoggedIn(true);
  setUserEmail(email);
  setIsAdmin(adminFlag);


  // Earn badge for login
  earnBadge("badge-5"); // Community Active badge represents engagement
  addPoints(200);

  // A sign-in is an event, not an application. It used to be written
  // into the applications collection, which put login records in the
  // queue the parish office works through.
  void logActivity("sign_in", `Signed in as ${adminFlag ? "admin" : "pilgrim"}`);
  void touchActivity();

  if (adminFlag) {
    setActiveTab("admin");
  } else {
    setActiveTab("home");
  }
};

const handleLogout = async () => {
  try {
    await signOut(auth);
  } catch (err) {
    console.error("Error signing out from Firebase:", err);
  }
  setIsLoggedIn(false);
  setUserEmail("");
  setIsAdmin(false);
  setActiveTab("home");
};

// Station check in
const handleStationVisited = (stationId: string, stepsToAdd: number, distanceToAdd: number) => {
  setUserProgress((prev) => {
    if (prev.completedStations.includes(stationId)) return prev;

    const newCompletedStations = [...prev.completedStations, stationId];
    const newSteps = prev.steps + stepsToAdd;
    const newDistance = prev.distanceKm + distanceToAdd;
```

```
    const newPoints = prev.points + 250; // +250 points per stamp

    const newBadges = [...prev.badges];

    // Badge 1: First Stamp inside Maypajo
    if (stationId.startsWith("mhcp-") && !newBadges.includes("badge-1")) {
      newBadges.push("badge-1");
    }

    // Badge 2: Completed all of Cathedral trail
    const srcStationIds = ROUTES.find(r => r.id === "route-src")?.stations.map(s => s.id) || [];
    const completedSrc = srcStationIds.every(id => newCompletedStations.includes(id));
    if (completedSrc && !newBadges.includes("badge-2")) {
      newBadges.push("badge-2");
    }

    void logActivity("station_visit", `Checked in at station ${stationId}`);

    const updatedProgress = {
      ...prev,
      completedStations: newCompletedStations,
      steps: newSteps,
      distanceKm: newDistance,
      points: newPoints,
      badges: newBadges
    };

    if (auth.currentUser) {
      setDoc(doc(db, "users", auth.currentUser.uid), updatedProgress, { merge: true });
    }

    return updatedProgress;
  });
};

// Comments submit
const handlePostComment = (stationId: string) => {
  if (!activeCommentInput.trim()) return;


  const userDisplay = isLoggedIn ? userEmail.split("@")[0] : "Pilgrim Walk";
  const newComment = {
    user: userDisplay,
    text: activeCommentInput,
    date: "Today"
  };

  setCommentsByStation(prev => ({
    ...prev,
    [stationId]: [newComment, ...(prev[stationId] || [])]
  }));

  setActiveCommentInput("");
  addPoints(100); // 100 points for adding a comment
  earnBadge("badge-5"); // Unlock Community Active badge

  void logActivity("station_comment", `Commented at station ${stationId}`);
};
```

```
const liveParishes = ROUTES.filter(r => r.status !== "coming_soon");

const handleChooseHomeParish = (parishId: string) => {
  saveHomeParishId(parishId);
  setHomeParishId(parishId);
  // The home parish may be one of the 29 without a tour. The active route
  // still has to be a live one, so it is mapped rather than set directly —
  // setting a diocese id here would leave activeChurchRoute falling back to
  // ROUTES[0] with no indication why.
  setSelectedChurchId(routeForHomeParish(parishId));
  setActiveTab("home");
  setIsChangeParishOpen(false);
};

return (
  <PresenceProvider>
    <PresenceParishSync onArrive={setSelectedChurchId} />
    <ParishTheme parishId={selectedChurchId} />
    {/* The parish picker, reached only from "Change Home Parish". It is
        not shown on first run: the app opens straight on the dashboard.
        It renders inside PresenceProvider because it orders the list by
        distance and so needs the position. */}
    {isChangeParishOpen && (
      <div className="fixed inset-0 z-50 bg-[var(--color-brand-card)] flex flex-col">
        {/* The inset keeps Cancel clear of the status bar. A `fixed
            inset-0` overlay starts at the very top of the screen, so on a
            phone that draws its status bar over the app this row was
            underneath it and the only way out of the screen could not be
            seen or tapped. */}
        <div
          className="flex justify-end p-3 shrink-0"
          style={{ paddingTop: "calc(0.75rem + env(safe-area-inset-top))" }}
        >
          <button
            type="button"
            onClick={() => setIsChangeParishOpen(false)}
            className="text-[16px] font-semibold text-[var(--color-brand-primary)] px-2"
          >
            Cancel
          </button>
        </div>
        <Onboarding onChoose={handleChooseHomeParish} />
      </div>
    )}

    {/* Sign-in as its own screen rather than embedded at the top of Me,
        where it stacked a second "Pilgrim Profile" header under the first. */}
    {isSignInOpen && (
      <div className="fixed inset-0 z-50 bg-[var(--color-brand-card)] flex flex-col">
        {/* Inset for the status bar, as above. */}
        <div
          className="flex justify-end p-3 shrink-0"
          style={{ paddingTop: "calc(0.75rem + env(safe-area-inset-top))" }}
        >
          <button
            type="button"
            onClick={() => setIsSignInOpen(false)}
            className="text-[16px] font-semibold text-[var(--color-brand-primary)] px-2"
          >
```

```
          Close
        </button>
      </div>
      <LoginModal
        isLoggedIn={isLoggedIn}
        userEmail={userEmail}
        isAdmin={isAdmin}
        activeParishId={activeChurchRoute.id}
        onLoginSuccess={(email, adminFlag) => {
          handleLoginSuccess(email, adminFlag);
          setIsSignInOpen(false);
        }}
        onLogout={handleLogout}
      />
    </div>
  )}

  {isSearchOpen && (
    <div className="fixed inset-0 z-50 bg-[var(--color-brand-card)] flex flex-col">
      <SearchScreen
        onClose={() => setIsSearchOpen(false)}
        onSelectParish={parishId => {
          setIsSearchOpen(false);
          const route = routeIdForParish(parishId);
          // Only the two live parishes have a page to open. Selecting any
          // other is still useful — it centres the map on it — so this
          // routes there instead of doing nothing.
          if (route) {
            handleSelectParish(route);
          } else {
            setWalkToParishId(parishId);
            setActiveTab("navigator");
          }
        }}
      />
    </div>
  )}
<div
  className={
    deviceSurface
      ? "h-[100dvh] bg-[var(--color-brand-card)] text-[var(--color-brand-text)] flex flex-col font-sans overflow-hidden"
      : "min-h-screen bg-[var(--color-brand-card)] text-[var(--color-brand-text)] flex flex-col justify-between font-sans"
  }
>
  {/* There is deliberately no desktop workspace header. It carried a
      "SanctiWalk Core Workspace / Prepared for STI College Capstone
      Defense" title and a "Return to Church Selection" button, which is
      why the app read as a project website on a laptop rather than as the
      app. The button also returned to a selection step the app no longer
      has. */}
  {/* Main Container Workspace */}
  <main
    className={
      deviceSurface
        ? "flex-1 flex flex-col select-none relative min-h-0"
        : "flex-1 flex items-center justify-center py-6 select-none relative"
    }
  >
    <PhoneContainer
```

```
          isOffline={isOffline}
          setIsOffline={setIsOffline}
          isMobileOnly={isMobileOnly}
          setIsMobileOnly={setIsMobileOnly}
        >
          {/* Main App Frame inside simulator */}
          <div className="flex-1 flex flex-col relative bg-[var(--color-brand-card)] overflow-hidden">

            {/* If NO church is selected, show the Church Selector view (Figure 2) */}
            {selectedChurchId === null ? (
              <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] p-5 overflow-y-auto justify-between">
                <div className="space-y-6 pt-4">
                  {/* Stylized App Icon/Header */}
                  <div className="text-center space-y-2">
                    <div className="h-14 w-14 rounded-2xl bg-[var(--color-brand-primary)] border border-[var(--color-brand-gold)] text-white flex items-center
justify-center font-sans text-3xl font-black shadow-md mx-auto">
                      S
                    </div>
                    <div>
                      <h1 className="text-3xl font-black font-sans tracking-tight text-white uppercase">
                        SanctiWalk
                      </h1>
                      <p className="text-sm text-[var(--color-brand-secondary)] font-bold tracking-widest uppercase mt-0.5">
                        Pilgrimage Navigator
                      </p>
                    </div>
                  </div>

                  {/* Instructions */}
                  <div className="text-center space-y-1 px-4">
                    <h2 className="text-[15px] font-bold text-white uppercase tracking-wider font-sans">
                      Choose Your Church Experience
                    </h2>
                    <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed font-sans">
                      Select a historical parish of the Diocese of Kalookan to begin your interactive spiritual walking tour.
                    </p>
                  </div>

                  {/* Diocese map — a geographically calibrated illustration
                      showing both parishes (and the pilgrim's own position,
                      when known) before a church is chosen. Tapping a live
                      pin opens that parish's profile, same as the "Start
                      Sanctuary Walk" card button below does for the same
                      selection action. */}
                  <div className="bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] shadow-xs p-3 h-72">
                    <CustomDioceseMap onSelectParish={handleSelectParish} />
                  </div>

                  {/* Church Selector Cards (Figure 2 / Page 34) */}
                  <div className="space-y-3">
                    {ROUTES.map((route) => (
                      <div
                        key={route.id}
                        className="bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] overflow-hidden shadow-xs hover:border-[var(--
color-brand-primary)] transition-all flex flex-col"
                      >
                        {/* Facade image placeholder */}
                        <div className="relative h-28">
```

```
                        <img
                          src={parishPhoto(route.id)}
                          alt={parishPhotoAlt(route.id, route.name)}
                          className="w-full h-full object-cover"
                        />
                        <div className="absolute inset-0 bg-black/70 flex items-end p-2.5">
                          <span className="text-sm bg-[var(--color-brand-gold)] text-white px-2 py-0.5 rounded font-bold uppercase tracking-wider font-sans">
                            {route.category}
                          </span>
                        </div>
                      </div>

                      {/* Card body */}
                      <div className="p-3.5 space-y-2 text-left font-sans">
                        <div>
                          <h4 className="text-[15px] font-bold text-[var(--color-brand-text)] font-sans">
                            {route.name}
                          </h4>
                          <p className="text-sm text-[var(--color-brand-secondary)] mt-0.5 flex items-center gap-0.5">
                            <MapPin className="w-3 h-3 text-[var(--color-brand-secondary)]" /> {route.location}
                          </p>
                        </div>
                        <p className="text-[15px] text-[var(--color-brand-text)] leading-normal line-clamp-2">
                          {route.description}
                        </p>

                        <button
                          onClick={() => {
                            setSelectedChurchId(route.id);
                            setActiveTab("home");
                          }}
                          className="w-full mt-1 py-2 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white text-sm font-bold
uppercase tracking-wider rounded-xl border border-[var(--color-brand-primary-dark)] transition-colors"
                        >
                          Start Sanctuary Walk
                        </button>
                      </div>
                    </div>
                  ))}
                </div>
              </div>

              <div className="text-center pt-4 border-t border-white/15">
                <span className="text-sm text-[var(--color-brand-secondary)] font-mono block">
                  STI COLLEGE KALOOKAN • CAPSTONE 2026
                </span>
              </div>
            </div>
          ) : (
            /* ACTIVE CHURCH WEB SHELL LAYOUT */
            <div className="flex-1 flex flex-col overflow-hidden">

              {/* The navy header bar is gone. It repeated the parish name
                  already shown on Home, and its two controls moved into Me:
                  Sign In, plus the menu's Change Parish / Rosary Settings /
                  Admin Panel / Location Simulator. Five tabs are now the
                  whole navigation — nothing hidden behind a hamburger. */}
```

```
{/* The slideout sidebar is gone with the header bar that
    opened it. Its four entries live in Me now: Change Parish,
    Rosary Settings, and — for those who have them — Admin
    Panel and the Location Simulator. */}


{/* PRIMARY VIEW CONTENT WORKSPACE */}
{/* data-tab lets the stylesheet tell a scrolling page of
    cards from a full-bleed canvas. Home needs room at the
    bottom so the floating Ask Sancti button never lands on
    the last card; Map and Pray fill their space edge to
    edge, and the same room there is just a dead strip of
    background above the bar. */}
{/* Above the scroller, so it is not scrolled away before
    it has been read. Renders nothing in a browser. */}
<UpdateBanner />

<div
  data-tab={activeTab}
  className="app-scroll flex-1 min-h-0 flex flex-col overflow-y-auto"
>


  {/* TAB 1: Parish Dashboard / Home Tab — TODAY first, then
      the parish grid, then diocese-wide content. Extracted
      into its own component (see src/components/Dashboard.tsx)
      once this block needed a TODAY section on top of the
      existing two; App.tsx was already large. */}
  {activeTab === "home" && (
    <Dashboard
      parish={activeChurchRoute}
      firstName={greetingName}
      announcements={announcements}
      onNavigate={setActiveTab}
      onBackToMyParish={viewingAnotherParish ? backToMyParish : undefined}
      onSelectParish={handleSelectParish}
      onWalkThere={handleWalkThere}
      onOpenSearch={() => setIsSearchOpen(true)}
      sectionRequest={homeSectionRequest ?? undefined}
    />
  )}

  {/* TAB 2: Map / Trail Station Navigator — restyled to match
      the client's earlier prototype (Churches.jsx): a
      centred diocese heading, the map at its full ~374px
      height (not squeezed by a fixed-height card), and one
      tappable card per live parish below it. */}
  {activeTab === "navigator" && (
    <div className="flex-1 flex flex-col overflow-hidden">
      <div className="flex-1 flex flex-col overflow-y-auto">
        {/* The real diocese map replaces the old simulated
            walk. That simulation advanced a timer between
            stations and displayed stored coordinates as if
            they were the pilgrim's own, which was not a walk
            at all. Station-by-station progress returns when
            it is driven by real QR scans at each station. */}
        {/* Full-bleed map with the parish rail floating over
            it, per the redesign. The map is the screen rather
            than a card on it: the header and the rail sit on
```

```
          top as overlays, so nothing steals height from the
          thing people came to this tab to look at. */}
        <div className="map-screen">
          {/* The heading block is gone at the client's
              request — the Map tab is self-evidently the map,
              and the two title lines plus the legend cost
              about a fifth of the screen. */}
          <div className="map-screen__canvas">
            <CustomDioceseMap
              onSelectParish={handleSelectParish}
              walkToParishId={walkToParishId}
              onWalkToConsumed={() => setWalkToParishId(null)}
            />
          </div>

        </div>
      </div>
    </div>
)}

{/* The parish's own page — the redesign's screen 05. */}

{/* TAB 3: Daily Rosary guide */}
{activeTab === "rosary" && (
  <PrayScreen
    onOpenSettings={() => setIsRosarySettingsOpen(true)}
    parish={activeChurchRoute}
  />
)}

{/* TAB 4: Mass schedule table — follows the active parish,
    not hardcoded to any one church (see MassSchedule.tsx). */}
{activeTab === "mass" && (
  <MassSchedule parish={activeChurchRoute} />
)}

{/* TAB 5: Church History archive — same fix: renders the
    active parish's own history rather than an internal tab. */}
{activeTab === "history" && (
  <ChurchHistory
    parish={activeChurchRoute}
    onBack={() => setActiveTab(historyReturnTab)}
  />
)}

{/* TAB 6: Volunteer Guilds list */}
{activeTab === "ministries" && (
  <MinistriesTab
    parish={activeChurchRoute}
    openRequest={itemRequest?.tab === "ministries" ? itemRequest : undefined}
    onAddApplication={handleAddApplication}
    uid={uid}
    userEmail={userEmail}
    onOpenSignIn={() => setIsSignInOpen(true)}
  />
)}

{/* TAB 7: Sacraments office */}
{activeTab === "sacraments" && (
```

```
  <SacramentsTab
    parish={activeChurchRoute}
    onAddApplication={handleAddApplication}
    uid={uid}
    userEmail={userEmail}
    onOpenSignIn={() => setIsSignInOpen(true)}
  />
)}

{/* TAB 8: AR Tour */}
{activeTab === "ar" && (
  <div
    className={
      scanCameraLive
        ? "flex-1 flex flex-col min-h-0"
        : "flex-1 flex flex-col min-h-0 bg-[var(--color-brand-card)]"
    }
  >
    {/* Wrapped out here rather than inside ArTour: that
        component returns early for each of its camera
        states, so a banner added to one of them would
        vanish in the others.

        Gone entirely once the viewfinder is up - the
        screen is the camera then, and the scroller goes
        with it so the picture is not inside something
        that can be scrolled. */}
    {/* Both children are KEYED, and that is load-bearing.
        React matches siblings by position when they have
        no key, so the moment the camera went live and the
        band unmounted, ArTour shifted from the second
        child to the first - React tore it down and built
        a fresh one, which stopped the camera it had just
        started. The banner then cleared, the band came
        back, and it began again. The scanner could never
        stay open for more than a frame. */}
    {/* The "Explore in Augmented Reality / AR Walk"
        band that used to sit here is gone. The AR tour
        has its own application with its own opening
        screen, so this was a title for a thing that
        titles itself, above a card that duplicated the
        scanner's own AR tab. */}
    <ArTour
      key="ar-tour"
      stations={activeChurchRoute.stations}
      parishId={activeChurchRoute.id}
      parishName={activeChurchRoute.name.replace(" Guide", "").replace(" Tour", "")}
      onClose={() => setActiveTab("home")}
      onCameraLiveChange={setScanCameraLive}
    />
  </div>
)}

{/* TAB 9: Pilgrim Catechism Quiz */}
{activeTab === "quiz" && (
  <PilgrimQuiz
    onEarnBadge={earnBadge}
    onAddPoints={addPoints}
    onAddApplication={handleAddApplication}
```

```
  />
)}

{/* TAB 10: "Me" — sign-in state, applications, prayer/visit
    progress, and links to Change Parish and Rosary
    Settings, all assembled from state the app already
    tracks. Replaces the old standalone "Devotee
    Authentication" screen, which is now folded in here. */}
{activeTab === "myApplications" && (
  <MyApplicationsPage
    applications={applications}
    onBack={() => setActiveTab("me")}
  />
)}

{activeTab === "notifications" && (
  <NotificationsPage onBack={() => setActiveTab("me")} />
)}

{activeTab === "me" && (
  <MeTab
    onOpenApplications={() => setActiveTab("myApplications")}
    onOpenNotifications={() => setActiveTab("notifications")}
    unreadNotifications={unreadNotifications}
    onNicknameChange={setNickname}
    isLoggedIn={isLoggedIn}
    userEmail={userEmail}
    isAdmin={isAdmin}
    userProgress={userProgress}
    applications={applications}
    onLoginSuccess={handleLoginSuccess}
    onLogout={handleLogout}
    onOpenChangeParish={() => setIsChangeParishOpen(true)}
    onOpenRosarySettings={() => setIsRosarySettingsOpen(true)}
    parishName={activeChurchRoute?.name.replace(" Guide", "").replace(" Tour", "")}
    onOpenAdmin={() => setActiveTab("admin")}
    onOpenSimulator={() => setIsSimulatorOpen(true)}
    onOpenSignIn={() => setIsSignInOpen(true)}
    onReplayTutorial={() => { setActiveTab("home"); setTutorialOpen(true); }}
    language={language}
    onLanguageChange={setLanguage}
    simulatorMode={simulatorMode}
    onSimulatorModeChange={setSimulatorMode}
    reminders={
      <ReminderSettingsCard
        settings={reminders.settings}
        update={reminders.update}
        permission={reminders.permission}
        enable={reminders.enable}
        scheduled={reminders.scheduled}
        supported={reminders.supported}
        otherParishes={ROUTES
          .filter(r => r.id !== homeParishId)
          .map(r => ({
            id: r.id,
            name: r.name.replace(" Guide", "").replace(" Tour", ""),
            following: followedParishes.includes(r.id),
          }))}
        onToggleParish={toggleFollowParish}
```

```
        />
      }
    />
  )}

  {/* TAB 11: Admin Control Panel */}
  {activeTab === "admin" && (
    <AdminPortal
      userEmail={userEmail}
      applications={applications}
      onDeleteApplication={handleDeleteApplication}
      onUpdateApplicationStatus={handleUpdateApplicationStatus}
      announcements={announcements}
      onAddAnnouncement={handleAddAnnouncement}
      onDeleteAnnouncement={handleDeleteAnnouncement}
    />
  )}

  {/* TAB 12: PWA Workspace — a Demo Tools entry, not a
      pilgrim-facing feature; see the sidebar's "Demo Tools"
      section. */}
  {activeTab === "pwa-devkit" && <PwaBanner />}

  {/* Sancti sits above the tab bar, reachable from every
      screen - which is the point, since most of what it
      does is take you to another one. Hidden while the
      camera has the screen: a floating button over a
      viewfinder covers the thing being scanned. Hidden
      over the sign-in sheet too - that screen has one
      job, and an assistant that cannot help with it was
      sitting on top of the form. */}
  {activeTab !== "ar" && !isSignInOpen && (
    <SanctiMount
      activeParishId={activeChurchRoute.id}
      activeParishName={activeChurchRoute.name.replace(" Guide", "").replace(" Tour", "")}
      content={activeParishContent}
      language={language}
      onGo={tab => setActiveTab(tab as typeof activeTab)}
      onShowHomeSection={showHomeSection}
      onOpenItem={openItem}
      onReplayTutorial={() => { setActiveTab("home"); setTutorialOpen(true); }}
      onSetLanguage={setLanguage}
      onSelectParish={setSelectedChurchId}
      onWalkTo={handleWalkThere}
      onFollowParish={id => toggleFollowParish(id, true)}
    />
  )}
</div>

{/* BOTTOM STICKY PHONE SIM NAVIGATION BAR — five tabs, Scan
    centred and visually raised as the app's signature
    feature, every tab keeping a visible text label. */}
{/* A flex child, not an overlay.


    As `absolute bottom-0` it floated ON TOP of the content:
    every screen then had to reserve 64px for it by hand, the
    outer wrapper reserved only 16px, and screens with their
    own inner scroller (Me, Pray) reserved none at all — so
```

```
                their last rows sat under the bar and could never be
                scrolled clear. Taking part in the layout means the
                content area sizes itself around the bar automatically,
                on every screen, including ones not written yet.

                The safe-area inset keeps the labels clear of the iPhone
                home indicator. */}
              <nav
                className="shrink-0 h-16 bg-[var(--color-brand-card)] border-t border-[var(--color-brand-border)] flex items-center justify-around px-1 z-40
shadow-md"
                style={{ paddingBottom: "env(safe-area-inset-bottom)" }}
              >
                <button
                  onClick={() => setActiveTab("home")}
                  data-spotlight="tab-home"
                  className={`tab-item ${activeTab === "home" ? "tab-item--on" : ""}`}
                  aria-current={activeTab === "home" ? "page" : undefined}
                >
                  <Home className="w-5 h-5" />
                  <span className="text-base font-bold leading-none">{t("tab.home", language)}</span>
                </button>

                <button
                  onClick={() => setActiveTab("navigator")}
                  data-spotlight="tab-map"
                  className={`tab-item ${activeTab === "navigator" ? "tab-item--on" : ""}`}
                  aria-current={activeTab === "navigator" ? "page" : undefined}
                >
                  <Map className="w-5 h-5" />
                  <span className="text-base font-bold leading-none">{t("tab.map", language)}</span>
                </button>

                {/* Scan: the app's signature feature, given a raised,
                    filled treatment so it reads as the visual anchor of
                    the bar — but still carries a text label like every
                    other tab, not an icon-only control. */}
                <button
                  onClick={() => setActiveTab("ar")}
                  data-spotlight="tab-scan"
                  className="flex-1 min-w-0 flex flex-col items-center justify-center gap-0.5 -translate-y-3"
                >
                  <span
                    className={`h-12 w-12 rounded-full flex items-center justify-center shadow-lg border-2 transition-all ${
                      activeTab === "ar"
                        ? "bg-[var(--color-brand-primary)] border-[var(--color-brand-card)] text-white"
                        : "bg-[var(--color-brand-primary)] border-[var(--color-brand-card)] text-white opacity-90"
                    }`}
                  >
                    <ArIcon className="w-5 h-5" />
                  </span>
                  <span className={`text-base font-bold leading-none ${activeTab === "ar" ? "text-[var(--color-brand-secondary)]" : "text-[var(--color-brand-
secondary)]"}`}>
                    {t("tab.scan", language)}
                  </span>
                </button>

                <button
                  onClick={() => setActiveTab("rosary")}
```

```
            data-spotlight="tab-pray"
            className={`tab-item ${activeTab === "rosary" ? "tab-item--on" : ""}`}
            aria-current={activeTab === "rosary" ? "page" : undefined}
          >
            <BookOpen className="w-5 h-5" />
            <span className="text-base font-bold leading-none">{t("tab.pray", language)}</span>
          </button>

          <button
            onClick={() => setActiveTab("me")}
            data-spotlight="tab-me"
            className={`tab-item ${activeTab === "me" ? "tab-item--on" : ""}`}
            aria-current={activeTab === "me" ? "page" : undefined}
          >
            <User className="w-5 h-5" />
            <span className="text-base font-bold leading-none">{t("tab.me", language)}</span>
          </button>
        </nav>

      </div>
    )}

    {/* The walkthrough. Rendered last so its overlay sits above
        the tab bar it points at. */}
    <TutorialRunner
      open={tutorialOpen}
      onClose={() => setTutorialOpen(false)}
      onSwitchTab={(tab: TutorialTab) => setActiveTab(tab)}
    />

    <RosarySettingsModal
      isOpen={isRosarySettingsOpen}
      onClose={() => setIsRosarySettingsOpen(false)}
    />

    <SimulatorPanel
      isOpen={isSimulatorOpen}
      onClose={() => setIsSimulatorOpen(false)}
    />

    {/* The "You are approaching …" banner is gone at the client's
        request, and its own design was the reason.

        It was a floating overlay pinned near the top of whatever
        screen you happened to be on, so it covered content it knew
        nothing about: the turn instruction while navigating, and the
        pilgrim's own name and avatar on Me. Being global was the
        point of it and also its whole problem.

        PresenceSheet stays: it carries the same information plus the
        actions worth taking on arrival, it collapses, and it does not
        sit on top of another screen's heading. */}
    <PresenceSheet onOpenTour={handleOpenTourFromPresence} onOpenAR={handleOpenARFromPresence} />

  </div>
 </PhoneContainer>
</main>

{/* Footer credits bar */}
```

```
    </div>
    </PresenceProvider>
  );
}
```

1.1.3 APP-003 — RootGate.tsx

Code ID	APP-003
File Name	RootGate.tsx
File Path	SanctDemo/src/components/RootGate.tsx
Language / Type	TypeScript (React, TSX)
Lines	70
Functions / Classes	isInstalledApp (internal function); RootGate (exported component/function (default))
Related Files (its imports)	./Spotlight, ./Landing, ../App
Where It Is Used	src/main.tsx
How to Recognize It	Open SanctDemo/src/components/RootGate.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — RootGate.tsx

```
import { useState } from "react";
import { Capacitor } from "@capacitor/core";
import { SpotlightProvider } from "./Spotlight";
import Landing from "./Landing";
import App from "../App";

/**
 * What the root of the site is.
 *
 * The same page for everybody - a phone, a tablet and a laptop all get
 * the page that explains SanctiWalk, with the app one tap away. It used
 * to send phones straight into the app, and that was wrong for a reason
 * worth writing down: somebody shown the site on a laptop and then
 * handed a phone saw two different things and reasonably concluded the
 * phone was out of date. One front door is less clever and much clearer.
 *
 * ## The exception: anyone who already has the app
 *
 * `manifest.webmanifest` sets `start_url: "/"`, so an installed PWA
 * opens this route, and the APK's WebView loads the same bundle at
 * https://localhost/. Showing a page advertising SanctiWalk to
 * somebody who has already installed it and just tapped its icon
 * would be absurd, so both go straight to the app.
 *
 * The Capacitor check is not optional and is listed first. A WebView
 * inside an APK is NOT a PWA: it does not match
 * `(display-mode: standalone)` and sets no `navigator.standalone`, so
 * a display-mode test alone says "browser" and the installed Android
 * app opens on its own advertisement. That is exactly what shipped,
 * and it is the whole reason this function asks three questions
 * instead of one.
 */
```

```
/** True when this is the installed app rather than a browser tab. */
function isInstalledApp(): boolean {
  if (typeof window === "undefined") return false;

  // The APK. Asked first because it is the only one of the three that
  // is a fact rather than an inference.
  try {
    if (Capacitor.isNativePlatform()) return true;
  } catch {
    // No Capacitor bridge: an ordinary browser. Fall through.
  }

  try {
    // iOS Safari predates the standard and uses its own flag.
    const iosStandalone = (window.navigator as { standalone?: boolean }).standalone === true;
    return iosStandalone || window.matchMedia?.("(display-mode: standalone)")?.matches === true;
  } catch {
    return false;
  }
}

export default function RootGate() {
  // Read once. A browser cannot become an installed app mid-session, and
  // swapping the whole page out from under somebody would be a strange
  // thing for a media query to do.
  const [installed] = useState(isInstalledApp);

  if (installed) {
    return (
      <SpotlightProvider>
        <App />
      </SpotlightProvider>
    );
  }

  return <Landing />;
}
```

1.1.4 APP-004 — PhoneContainer.tsx

Code ID	APP-004
File Name	PhoneContainer.tsx
File Path	SanctDemo/src/components/PhoneContainer.tsx
Language / Type	TypeScript (React, TSX)
Lines	193
Functions / Classes	PhoneContainer (exported component/function (default))
Related Files (its imports)	../lib/displayMode
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/PhoneContainer.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PhoneContainer.tsx

```
import React, { useState, useEffect } from "react";
import { Smartphone, Monitor, Wifi, WifiOff, Battery } from "lucide-react";
import { Link } from "react-router-dom";
import { useDeviceSurface } from "../lib/displayMode";

interface PhoneContainerProps {
  children: React.ReactNode;
  isOffline: boolean;
  setIsOffline: (offline: boolean) => void;
  isMobileOnly: boolean;
  setIsMobileOnly: (mobileOnly: boolean) => void;
}

export default function PhoneContainer({
  children,
  isOffline,
  setIsOffline,
  isMobileOnly,
  setIsMobileOnly
}: PhoneContainerProps) {
  const [currentTime, setCurrentTime] = useState("");

  // On a real phone there is nothing to simulate: the device IS the frame.
  // Drawing a mockup there cost most of the screen and made sign-in - which
  // covers the whole display - look like the app changing size.
  const deviceSurface = useDeviceSurface();

  useEffect(() => {
    const updateTime = () => {
      const now = new Date();
      let hours = now.getHours();
      const minutes = now.getMinutes().toString().padStart(2, "0");
      const ampm = hours >= 12 ? "PM" : "AM";
```

```
    hours = hours % 12;
    hours = hours ? hours : 12; // the hour '0' should be '12'
    setCurrentTime(`${hours}:${minutes} ${ampm}`);
  };
  updateTime();
  const interval = setInterval(updateTime, 60000);
  return () => clearInterval(interval);
}, []);

if (deviceSurface) {
  // The app, and only the app, edge to edge. No toolbar, no notch, no
  // invented battery percentage, and no viewport switch - a phone cannot
  // usefully be shown "the desktop version" of a layout built for it.
  return (
    <div
      data-app-shell
      className="flex-1 flex flex-col bg-[var(--color-brand-card)] min-h-0"
    >
      {children}
    </div>
  );
}

return (
  // A centred column: a slim toolbar, then the app. The page used to be a
  // two-column marketing layout — a "SanctiWalk PWA" headline, a paragraph
  // on why the capstone scope changed, and a defense bulletin — with the
  // app itself as a panel beside it. That is a page ABOUT the app, which is
  // why it read as a website rather than as the product on a laptop. The
  // writing has not been thrown away; it belongs in the defense document,
  // not wrapped around the running app.
  <div id="phone-container" className="app-stage">
    {/* The same masthead the landing page wears.

        Before this, a visitor who pressed "Open full screen" landed on
        a bare mockup floating on white, with a viewport switch where
        the branding had been. It read as a different product, which is
        exactly the confusion this removes: one logo, one nav, one
        background, whichever of the two pages you are on.

        The logo is the way back. A wordmark that returns you to the
        front page is a convention every site already taught the
        visitor, and it does the job the separate back button was
        doing less tidily. */}
    <header className="app-stage__bar">
      <Link to="/" aria-label="Back to the SanctiWalk home page">
        <img
          className="app-stage__logo"
          src="/ui/sanctiwalk-lockup-navy.png"
          alt="SanctiWalk"
          width={170}
        />
      </Link>
      <nav className="app-stage__nav">
        <Link to="/">About</Link>
        <a href="/get/">Download</a>
      </nav>
    </header>
```

```
      {/* The viewport switch, and nothing else. Two buttons plus the offline
          simulation, which is a real test of the service worker rather than
          promotional copy. */}
      <div className="flex items-center gap-2 flex-wrap justify-center">
        <div className="flex items-center rounded-full border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-1">
          <button
            id="btn-toggle-mobile"
            onClick={() => setIsMobileOnly(true)}
            aria-pressed={isMobileOnly}
            className={`flex items-center gap-2 py-1.5 px-4 rounded-full text-[15px] font-semibold transition-all ${
              isMobileOnly
                ? "bg-[var(--color-brand-primary)] text-white"
                : "text-[var(--color-brand-secondary)]"
            }`}
          >
            <Smartphone className="w-4 h-4" /> Phone
          </button>
          <button
            id="btn-toggle-responsive"
            onClick={() => setIsMobileOnly(false)}
            aria-pressed={!isMobileOnly}
            className={`flex items-center gap-2 py-1.5 px-4 rounded-full text-[15px] font-semibold transition-all ${
              !isMobileOnly
                ? "bg-[var(--color-brand-primary)] text-white"
                : "text-[var(--color-brand-secondary)]"
            }`}
          >
            <Monitor className="w-4 h-4" /> Full responsive
          </button>
        </div>

        <button
          id="btn-toggle-offline"
          onClick={() => setIsOffline(!isOffline)}
          aria-pressed={isOffline}
          title="Simulate offline, to test the service worker fallback"
          className={`flex items-center gap-1.5 py-2 px-3.5 rounded-full text-[15px] font-semibold transition-all border ${
            isOffline
              ? "bg-[var(--color-brand-gold)]/20 text-[var(--color-brand-gold)] border-[var(--color-brand-gold)]/55"
              : "bg-[var(--color-brand-card)] text-[var(--color-brand-secondary)] border-[var(--color-brand-border)]"
          }`}
        >
          {isOffline ? <WifiOff className="w-3.5 h-3.5" /> : <Wifi className="w-3.5 h-3.5" />}
          {isOffline ? "Offline" : "Online"}
        </button>
      </div>

      {/* Main Container Render */}
      <div className="flex justify-center items-center w-full">
        {isMobileOnly ? (
          /* High Fidelity Smartphone Mockup */
          <div className="relative mx-auto w-[385px] h-[780px] bg-[var(--color-brand-ink-surface)] rounded-[54px] shadow-2xl border-[12px] border-[var(--color-brand-
ink-surface)] flex flex-col overflow-hidden ring-4 ring-[var(--color-brand-ink-surface)]">
            {/* Speaker Notching & Camera Hole */}
            <div className="absolute top-0 left-1/2 -translate-x-1/2 h-[30px] w-[140px] bg-black rounded-b-[18px] z-50 flex items-center justify-center">
              <div className="w-[40px] h-[4px] bg-[#222] rounded-full absolute top-[6px]"></div>
              <div className="w-[8px] h-[8px] bg-[#111] rounded-full absolute right-[24px] top-[4px] ring-2 ring-[#222]"></div>
            </div>
```

```
            {/* Simulated Phone Status Bar */}
            <div className="h-[44px] bg-[var(--color-brand-card)] border-b border-[var(--color-brand-border)]/40 text-[var(--color-brand-text)] px-6 flex items-end
justify-between pb-1.5 text-[15px] font-semibold select-none z-40 shrink-0">
              <span>{currentTime || "10:00 AM"}</span>
              <div className="flex items-center gap-1.5">
                {isOffline ? (
                  <WifiOff className="w-3.5 h-3.5 text-red-600 animate-pulse" />
                ) : (
                  <Wifi className="w-3.5 h-3.5 text-green-700" />
                )}
                <span className="text-sm">LTE</span>
                <Battery className="w-4 h-4 text-gray-800 rotate-90 scale-90 translate-y-[1px]" />
                <span className="text-sm">98%</span>
              </div>
            </div>

            {/* Virtual App Screen Canvas */}
            <div
              data-app-shell
              className="flex-1 bg-[var(--color-brand-card)] flex flex-col overflow-y-auto relative z-30"
            >
              {children}
            </div>

            {/* iOS Bottom Navigation Bar Handle Indicator */}
            <div className="h-[24px] bg-[var(--color-brand-card)] border-t border-[var(--color-brand-border)]/40 flex items-center justify-center pb-2 shrink-0 z-
40">
              <div className="w-[120px] h-[5px] bg-gray-300 rounded-full"></div>
            </div>
          </div>
        ) : (
          /* Full responsive: the app filling the window, with no simulated
             browser chrome around it. It used to be capped at 460px behind a
             navy "SanctiWalk Web View" bar, which is a picture of a web view
             rather than the app actually being responsive. */
          <div className="w-full max-w-[900px] h-[calc(100vh-140px)] min-h-[560px] bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)]
shadow-xl flex flex-col overflow-hidden relative">
            <div data-app-shell className="flex-1 overflow-y-auto bg-[var(--color-brand-card)] flex flex-col">
              {children}
            </div>
          </div>
        )}
      </div>
    </div>
  );
}
```

1.1.5 APP-005 — Landing.tsx

Code ID	APP-005
File Name	Landing.tsx
File Path	SanctDemo/src/components/Landing.tsx
Language / Type	TypeScript (React, TSX)
Lines	228
Functions / Classes	Landing (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/RootGate.tsx, src/main.tsx
How to Recognize It	Open SanctDemo/src/components/Landing.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — Landing.tsx

```
import { Link } from "react-router-dom";
import { Download, ArrowRight } from "lucide-react";

/**
 * The page a laptop sees at the root.
 *
 * ## Why a laptop sees something different at all
 *
 * SanctiWalk is used standing in a church, holding a phone. Somebody on
 * a Windows machine is almost never doing that - they are deciding
 * whether to look, or they are a panel member, or they followed a link
 * from Facebook. Serving them the phone interface stretched across
 * 1440px tells them nothing about what this is. So the root answers the
 * question they actually have.
 *
 * A phone never sees this page. See RootGate: a phone at the root is
 * somebody who came to use the thing, and a promotional page in front
 * of that is a toll gate.
 *
 * ## Why the phone in the picture is a real screenshot
 *
 * It is `/get/shots/home.jpg`, the same capture the download page uses,
 * taken from the running app. A drawn mockup would let the picture
 * drift from the product every time the app changed, and the one thing
 * a promotional page must not do is promise a screen that does not
 * exist.
 */
export default function Landing() {
  return (
    <div className="lp">
      <div className="lp__panel">
        <header className="lp__bar">
          <img
```

```
    className="lp__logo"
    src="/ui/sanctiwalk-lockup-navy.png"
    alt="SanctiWalk"
    width={190}
  />
  <nav className="lp__nav">
    <a href="#what">What it does</a>
    <a href="#parishes">Parishes</a>
    <a href="/get/">Download</a>
  </nav>
</header>

<div className="lp__hero">
  <div className="lp__copy">
    {/* Broken at the comma, which is where the rhyme turns.
        Two lines, not three: the couplet only works if both
        halves are read as halves. */}
    <h1 className="lp__headline">
      The Diocese in your hand,
      <br />
      guided as you planned
    </h1>

    <p className="lp__lede">
      A companion for the churches of the Diocese of Kalookan. Mass times
      kept current by the parish office itself, the story of each church
      read aloud, the Rosary in English or Filipino, and a camera that
      names the statue in front of you.
    </p>

    <div className="lp__actions">
      {/* Straight to the download page, which already carries the
          install steps, the file size and the permissions note. */}
      <a className="lp__cta" href="/get/">
        <Download className="w-4 h-4" aria-hidden />
        Get the Android app
      </a>

      {/* For anyone who wants the running app bigger than the
          frame beside it. Not "Explore" - there is nothing to
          explore elsewhere, it is already running. */}
      <Link className="lp__cta lp__cta--quiet" to="/app">
        {/* "Open full screen" on a laptop, where the app is
            already running in the frame beside this. On a phone
            there is no frame - see below - so the same link is
            the only way in and says so. */}
        <span className="lp__cta-wide">Open full screen</span>
        <span className="lp__cta-narrow">Open SanctiWalk</span>
        <ArrowRight className="w-4 h-4" aria-hidden />
      </Link>
    </div>

    <p className="lp__footnote">
      {/* "beside" on a laptop, "below" on a phone - the frame
          moves, so the sentence pointing at it has to move too. */}
      <span className="lp__cta-wide">The app beside this is live. Tap it. </span>
      <span className="lp__cta-narrow">The app below is live. Try it. </span>
      The Android build adds the AR heritage tour.
    </p>
```

```
    </div>

    <div className="lp__shot">
      {/* The actual application, running, not a picture of it.

          An iframe rather than <App /> rendered inline, for two
          reasons. The app positions things against the viewport -
          the tab bar, the Ask Sancti button, every bottom sheet -
          and inline those would escape into this page and sit at
          the bottom of the browser window. And an iframe at phone
          width makes the app believe it is on a phone, which is
          already how it decides to drop its own desktop mockup
          chrome: see isDeviceSurface in lib/displayMode.ts, which
          is true at 768px and under. So the frame below is the
          only frame, and no new code path was added to the app to
          get that.

          Same origin, so this is the same application with the
          same Firebase session - not a copy. It is the pattern
          the Rosary already uses in DailyRosary.tsx. */}
      <div className="lp__phone">
        {/* The iframe is a real phone viewport - 375 x 812, the
            size the app is designed against - and the wrapper
            scales it down to fit. Setting the iframe itself to
            300px instead made the app lay out AT 300px, which is
            narrower than any handset it was built for: the
            bulletin tiles clipped their own labels and the Ask
            Sancti button sat half off the edge. Scaling shrinks
            the picture; it does not squeeze the layout. */}
        <div className="lp__phone-viewport">
          <iframe
            className="lp__phone-screen"
            /* English, whatever this visitor last chose for themselves.
   The iframe shares the landing page's localStorage, so without
   this the public demo came up in whichever language the last
   person to use the app on this browser had picked. */
src="/app?lang=en"
            title="SanctiWalk, running"
            loading="eager"
            // The app asks for these itself when someone taps Scan
            // or opens the map; without them the iframe would deny
            // them before the app ever saw the request.
            allow="camera; geolocation; microphone"
          />
        </div>
      </div>
      <p className="lp__shot-note">Live — try it</p>
    </div>
  </div>
</div>

<section className="lp__band" id="what">
  <div className="lp__bandinner">
    <h2 className="lp__bandtitle">What you get</h2>

    <dl className="lp__grid">
      <div>
        <dt>Mass times that are actually right</dt>
        <dd>
```

```
          The parish office edits them directly. When they move a Mass it
          moves on your phone, with no update to install.
        </dd>
      </div>
      <div>
        <dt>A guide to what you are looking at</dt>
        <dd>
          Point the camera at a statue, an altar or an image. It names it
          and tells you who they were, drawn from the parish's own records
          where they exist.
        </dd>
      </div>
      <div>
        <dt>History you can listen to</dt>
        <dd>
          Each church's story, read aloud in English or Filipino, so you
          can walk and look rather than read with your head down.
        </dd>
      </div>
      <div>
        <dt>The Rosary, guided</dt>
        <dd>
          All four sets of mysteries in both languages, with an optional
          voice. Works with no signal at all.
        </dd>
      </div>
      <div>
        <dt>Directions that follow the road</dt>
        <dd>
          Real street routing to any parish in the diocese, not a straight
          line drawn across the map.
        </dd>
      </div>
      <div>
        <dt>Ministries and sacraments</dt>
        <dd>
          Requirements, schedules and the application itself, handled in
          the app and answered by the parish.
        </dd>
      </div>
    </dl>
  </div>
</section>

<section className="lp__closing" id="parishes">
  <h2>Two parishes, in full</h2>
  <p>
    Mary Help of Christians Parish in Maypajo and San Roque Cathedral
    Parish are implemented completely, with every station, history and
    sacramental requirement confirmed with the parish itself. The rest of
    the diocese appears on the map and in the directory.
  </p>
  <p className="lp__closingnote">
    Accuracy is the limit here, not the code. A guide that misnames an
    image or invents the meaning of an altar damages the heritage it
    exists to serve.
  </p>
  <Link className="lp__cta" to="/app">
    Explore SanctiWalk
```

```
          <ArrowRight className="w-4 h-4" aria-hidden />
        </Link>
      </section>

      <footer className="lp__foot">
        <p>
          A capstone project for STI College, built in consultation with the
          parishes named above.
        </p>
        <p>
          <a href="https://github.com/Gifeys/SanctDemo">Source code</a>
          <span aria-hidden> · </span>
          <a href="/get/">Download for Android</a>
        </p>
      </footer>
    </div>
  );
}
```

1.2 Screens and Components

1.2.1 APP-006 — AdminPortal.tsx

Code ID	APP-006
File Name	AdminPortal.tsx
File Path	SanctDemo/src/components/AdminPortal.tsx
Language / Type	TypeScript (React, TSX)
Lines	980
Functions / Classes	AdminPortal (exported component/function (default))
Related Files (its imports)	./ParishContentEditor, ../lib/ministryApplication, ../lib/apiBase, ../lib/appKey
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/AdminPortal.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminPortal.tsx

```
import React, { useState } from "react";
import ParishContentEditor from "./ParishContentEditor";
import { ShieldCheck, Sparkles, BarChart2, Database, FileText, Settings, Trash2, Edit, Plus, Check, Mail, CalendarRange, X, Send, Clock, Loader2 } from "lucide-
react";
import { MINISTRY_STATUSES, isMinistryApplication, statusPresentation } from "../lib/ministryApplication";
import { apiUrl } from "../lib/apiBase";
import { withAppKey } from "../lib/appKey";

interface AdminPortalProps {
  applications: Array<{
    id: string;
    type: string;
    applicant: string;
    details: string;
    date: string;
    status: string;
    // Ministry applications carry these; sacrament bookings do not.
    email?: string;
    mobile?: string;
    ministryName?: string;
    parishName?: string;
    message?: string;
  }>;
  onDeleteApplication: (id: string) => void;
  onUpdateApplicationStatus: (id: string, status: string) => void;
  announcements: Array<{ id: string; title: string; date: string; time: string; type: string }>;
  onAddAnnouncement: (announcement: { id: string; title: string; date: string; time: string; type: string }) => void;
  onDeleteAnnouncement: (id: string) => void;
  /** The signed-in admin, recorded against each parish-content change. */
```

```
  userEmail: string;
}

export default function AdminPortal({
  applications,
  onDeleteApplication,
  onUpdateApplicationStatus,
  announcements,
  onAddAnnouncement,
  onDeleteAnnouncement,
  userEmail
}: AdminPortalProps) {
  const [activeTab, setActiveTab] = useState<"dashboard" | "content" | "analytics" | "database">("dashboard");

  // Email and Scheduling Modals State
  const [selectedAppForAction, setSelectedAppForAction] = useState<{
    app: { id: string; type: string; applicant: string; details: string; date: string; status: string };
    actionType: "approve" | "schedule" | "reject";
  } | null>(null);

  const [recipientEmail, setRecipientEmail] = useState("");
  const [emailSubject, setEmailSubject] = useState("");
  const [emailBody, setEmailBody] = useState("");
  const [scheduleDate, setScheduleDate] = useState("");
  const [scheduleTime, setScheduleTime] = useState("");
  const [isSendingEmail, setIsSendingEmail] = useState(false);
  const [emailSentSuccess, setEmailSentSuccess] = useState(false);
  const [isLastEmailSimulated, setIsLastEmailSimulated] = useState(false);
  const [errorMessage, setErrorMessage] = useState<string | null>(null);
  const [smtpConfig, setSmtpConfig] = useState<{ isConfigured: boolean; user: string | null; senderName: string } | null>(null);

  React.useEffect(() => {
    fetch(apiUrl("/api/smtp-status"))
      .then((res) => res.json())
      .then((data) => setSmtpConfig(data))
      .catch((err) => console.error("Error fetching SMTP status:", err));
  }, []);

  const handleOpenApproveModal = (app: { id: string; type: string; applicant: string; details: string; date: string; status: string }) => {
    const emailMatch = app.details.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/);
    const extractedEmail = emailMatch ? emailMatch[0] : "";


    setRecipientEmail(extractedEmail || "applicant@example.com");
    setEmailSubject(`CONGRATULATIONS: Your ${app.type} has been Approved!`);
    setEmailBody(`Dear ${app.applicant},

We are absolutely delighted to inform you that your registration for the "${app.type}" has been officially APPROVED by Father Paul Woo and the parish panel of Mary
Help of Christians Parish!

We would like to formally welcome you and invite you to download your official Congratulatory Form from the parish dashboard.

Best regards,
Parish Office
Mary Help of Christians Parish`);
    setSelectedAppForAction({ app, actionType: "approve" });
    setEmailSentSuccess(false);
    setIsSendingEmail(false);
    setErrorMessage(null);
```

```
  };

  const handleOpenScheduleModal = (app: { id: string; type: string; applicant: string; details: string; date: string; status: string }) => {
    const emailMatch = app.details.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/);
    const extractedEmail = emailMatch ? emailMatch[0] : "";


    const dateMatch = app.details.match(/Scheduled Date: ([^\s\)]+)/);
    const defaultDate = dateMatch ? dateMatch[1] : new Date().toISOString().split('T')[0];


    setRecipientEmail(extractedEmail || "applicant@example.com");
    setScheduleDate(defaultDate);
    setScheduleTime("10:00 AM");


    const initialSubject = `SCHEDULE NOTICE: Appointment for ${app.type}`;
    const initialBody = `Dear ${app.applicant},

This is to confirm that an appointment has been scheduled for your ${app.type}.

Appointment Details:
Date: ${defaultDate}
Time: 10:00 AM
Location: Mary Help of Christians Parish Office

Please bring all required canonical documents with you for this appointment. If you need to reschedule, please contact the Parish Office.

Best regards,
Parish Office
Mary Help of Christians Parish`;

    setEmailSubject(initialSubject);
    setEmailBody(initialBody);
    setSelectedAppForAction({ app, actionType: "schedule" });
    setEmailSentSuccess(false);
    setIsSendingEmail(false);
    setErrorMessage(null);
  };

  const updateScheduleEmailBody = (date: string, time: string) => {
    if (!selectedAppForAction) return;
    const { app } = selectedAppForAction;
    setEmailBody(`Dear ${app.applicant},

This is to confirm that an appointment has been scheduled for your ${app.type}.

Appointment Details:
Date: ${date}
Time: ${time}
Location: Mary Help of Christians Parish Office

Please bring all required canonical documents with you for this appointment. If you need to reschedule, please contact the Parish Office.

Best regards,
Parish Office
Mary Help of Christians Parish`);
  };

  const handleOpenRejectModal = (app: { id: string; type: string; applicant: string; details: string; date: string; status: string }) => {
```

```
    const emailMatch = app.details.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/);
    const extractedEmail = emailMatch ? emailMatch[0] : "";


    setRecipientEmail(extractedEmail || "applicant@example.com");
    setEmailSubject(`UPDATE REQUIRED: ${app.type} Review`);
    setEmailBody(`Dear ${app.applicant},

Thank you for submitting your request for ${app.type}. After review, some required documents or details are incomplete.

Please resubmit your application with the proper guidelines or contact our Maypajo Parish Office for assistance.

Best regards,
Parish Office
Mary Help of Christians Parish`);
    setSelectedAppForAction({ app, actionType: "reject" });
    setEmailSentSuccess(false);
    setIsSendingEmail(false);
    setErrorMessage(null);
  };

  const handleSendEmailAction = async () => {
    if (!selectedAppForAction) return;
    setIsSendingEmail(true);
    setErrorMessage(null);

    let finalStatus = "";
    let isSimulated = false;

    if (selectedAppForAction.actionType === "approve") {
      finalStatus = "Approved (Congratulation Email Sent)";
    } else if (selectedAppForAction.actionType === "schedule") {
      finalStatus = `Scheduled on ${scheduleDate} @ ${scheduleTime} (Invitation Email Sent)`;
    } else if (selectedAppForAction.actionType === "reject") {
      finalStatus = "Rejected - Resubmit (Notice Email Sent)";
    }

    try {
      const response = await fetch(apiUrl("/api/send-email"), {
        method: "POST",
        headers: withAppKey({
          "Content-Type": "application/json"
        }),
        body: JSON.stringify({
          to: recipientEmail,
          subject: emailSubject,
          body: emailBody
        })
      });

      const errData = await response.json().catch(() => ({}));

      if (!response.ok) {
        if (errData.error === "SMTP_NOT_CONFIGURED") {
          isSimulated = true;
        } else {
          throw new Error(errData.message || "Failed to transmit message via SMTP server.");
        }
      }
```

```
      // Update state in firebase
      onUpdateApplicationStatus(
        selectedAppForAction.app.id,
        isSimulated ? `${finalStatus} (Simulated)` : finalStatus
      );

      setIsSendingEmail(false);
      setEmailSentSuccess(true);
      setIsLastEmailSimulated(isSimulated);

      // Auto close modal after showing success state
      setTimeout(() => {
        setSelectedAppForAction(null);
        setIsLastEmailSimulated(false);
      }, 4500);

    } catch (err: any) {
      console.error("SMTP Transmission failed:", err);
      setErrorMessage(err.message || "A connection or authentication error occurred while sending the email.");
      setIsSendingEmail(false);
    }
  };

  // Content edit state
  const [churchName, setChurchName] = useState("Mary Help of Christians Parish");
  const [churchAddress, setChurchNameAddress] = useState("J.P. Rizal Street, Maypajo, Caloocan City");
  const [churchFounded, setChurchFounded] = useState("1952");
  const [churchHistory, setChurchHistory] = useState("Mary Help of Christians Parish in Maypajo was established to serve the growing population of Caloocan and to
provide a center for Marian devotion...");
  const [isSavedContent, setIsSavedContent] = useState(false);

  // New Announcement fields
  const [annTitle, setAnnTitle] = useState("");
  const [annDate, setAnnDate] = useState("");
  const [annTime, setAnnTime] = useState("");
  const [annType, setAnnType] = useState("Mass");

  const handleSaveChurchInfo = (e: React.FormEvent) => {
    e.preventDefault();
    setIsSavedContent(true);
    setTimeout(() => setIsSavedContent(false), 2500);
  };

  const handleAddAnn = (e: React.FormEvent) => {
    e.preventDefault();
    if (!annTitle || !annDate) return;
    onAddAnnouncement({
      id: "ann-" + Date.now(),
      title: annTitle,
      date: annDate,
      time: annTime || "6:00 PM",
      type: annType
    });
    setAnnTitle("");
    setAnnDate("");
    setAnnTime("");
  };
```

```
  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-text)] text-white overflow-hidden font-sans">
      {/* Admin Web Header Bar */}
      <div className="bg-[var(--color-brand-text)] border-b border-[var(--color-brand-primary)]/40 p-4 shrink-0 flex items-center justify-between">
        <div className="flex items-center gap-2">
          <ShieldCheck className="w-5 h-5 text-[var(--color-brand-on-accent)]" />
          <div>
            <h3 className="text-[15px] font-bold font-serif italic tracking-wide text-[var(--color-brand-on-accent)] uppercase">
              SanctiWalk Admin Portal
            </h3>
            <span className="text-sm font-mono text-[var(--color-brand-secondary)] uppercase">Maypajo Church DB Server</span>
          </div>
        </div>
        <span className="text-sm bg-red-800 text-white font-mono px-2 py-0.5 rounded uppercase tracking-wider font-bold">
          LIVE DEMO MODE
        </span>
      </div>

      {/* Main Admin Workspace split */}
      <div className="flex-1 flex overflow-hidden">
        {/* Figure 9: Admin Left Menu sidebar */}
        <div className="w-18 bg-[var(--color-brand-text)] border-r border-[var(--color-brand-primary)]/30 flex flex-col items-center py-4 space-y-3 shrink-0">
          <button
            onClick={() => setActiveTab("dashboard")}
            className={`w-12 h-12 rounded-xl flex flex-col items-center justify-center gap-0.5 transition-all ${
              activeTab === "dashboard" ? "bg-[var(--color-brand-primary)] text-white font-bold" : "text-[var(--color-brand-secondary)] hover:text-[var(--color-
brand-secondary)]"
            }`}
          >
            <BarChart2 className="w-4 h-4" />
            <span className="text-sm uppercase font-bold tracking-wider">Stats</span>
          </button>

          <button
            onClick={() => setActiveTab("content")}
            className={`w-12 h-12 rounded-xl flex flex-col items-center justify-center gap-0.5 transition-all ${
              activeTab === "content" ? "bg-[var(--color-brand-primary)] text-white font-bold" : "text-[var(--color-brand-secondary)] hover:text-[var(--color-brand-
secondary)]"
            }`}
          >
            <Edit className="w-4 h-4" />
            <span className="text-sm uppercase font-bold tracking-wider">Content</span>
          </button>

          <button
            onClick={() => setActiveTab("analytics")}
            className={`w-12 h-12 rounded-xl flex flex-col items-center justify-center gap-0.5 transition-all ${
              activeTab === "analytics" ? "bg-[var(--color-brand-primary)] text-white font-bold" : "text-[var(--color-brand-secondary)] hover:text-[var(--color-
brand-secondary)]"
            }`}
          >
            <Settings className="w-4 h-4" />
            <span className="text-sm uppercase font-bold tracking-wider">Graphs</span>
          </button>

          <button
            onClick={() => setActiveTab("database")}
            className={`w-12 h-12 rounded-xl flex flex-col items-center justify-center gap-0.5 transition-all ${
```

```
              activeTab === "database" ? "bg-[var(--color-brand-primary)] text-white font-bold" : "text-[var(--color-brand-secondary)] hover:text-[var(--color-brand-
secondary)]"
            }`}
          >
            <Database className="w-4 h-4" />
            <span className="text-sm uppercase font-bold tracking-wider">Data</span>
          </button>
        </div>

        {/* Dynamic Admin Sub-view screen */}
        <div className="flex-1 overflow-y-auto p-4 bg-[var(--color-brand-text)]">
          {/* TAB 1: Admin Dashboard (Figure 9) */}
          {activeTab === "dashboard" && (
            <div className="space-y-4 font-sans text-[15px]">
              <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/30 pb-2">
                <h4 className="font-serif italic text-sm font-bold text-[var(--color-brand-on-accent)]">Overview & Stats</h4>
                <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Updated 1 min ago</span>
              </div>

              {/* Grid 4 Stats */}
              <div className="grid grid-cols-2 gap-2 text-center select-none">
                <div className="bg-[var(--color-brand-text)] p-2.5 rounded-xl border border-[var(--color-brand-primary)]/25">
                  <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono">Total Visitors</span>
                  <strong className="text-sm text-white font-bold">300</strong>
                </div>
                <div className="bg-[var(--color-brand-text)] p-2.5 rounded-xl border border-[var(--color-brand-primary)]/25">
                  <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono">AR Usage</span>
                  <strong className="text-sm text-[var(--color-brand-on-accent)] font-bold">70 Scans</strong>
                </div>
                <div className="bg-[var(--color-brand-text)] p-2.5 rounded-xl border border-[var(--color-brand-primary)]/25">
                  <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono">Peak Visiting Hours</span>
                  <strong className="text-sm text-white font-bold font-serif">2:00-4:00 PM</strong>
                </div>
                <div className="bg-[var(--color-brand-text)] p-2.5 rounded-xl border border-[var(--color-brand-primary)]/25">
                  <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono">Engagement Rate</span>
                  <strong className="text-sm text-green-400 font-bold">72.4 %</strong>
                </div>
              </div>

              {/* Figure 9: Most Viewed Sections */}
              <div className="bg-[var(--color-brand-text)] p-3 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-2">
                <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] border-b border-[var(--color-brand-primary)]/20 pb-1">
                  Most Visited Church Zones
                </h5>
                <div className="space-y-1 text-[15px]">
                  <div className="flex justify-between p-1 bg-[var(--color-brand-text)]/40 rounded">
                    <span>1. Crucifix Central</span>
                    <strong className="text-[var(--color-brand-on-accent)]">200 hits</strong>
                  </div>
                  <div className="flex justify-between p-1 bg-[var(--color-brand-text)]/40 rounded">
                    <span>2. Adoration Chapel</span>
                    <strong className="text-[var(--color-brand-on-accent)]">120 hits</strong>
                  </div>
                  <div className="flex justify-between p-1 bg-[var(--color-brand-text)]/40 rounded">
                    <span>3. Altar Table</span>
                    <strong className="text-[var(--color-brand-on-accent)]">100 hits</strong>
                  </div>
                  <div className="flex justify-between p-1 bg-[var(--color-brand-text)]/40 rounded">
                    <span>4. Maria Auxiliadora</span>
```

```
              <strong className="text-[var(--color-brand-on-accent)]">70 hits</strong>
            </div>
          </div>
        </div>

        {/* Dynamic activity log summary */}
        <div className="bg-[var(--color-brand-text)] p-3 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-2">
          <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] border-b border-[var(--color-brand-primary)]/20 pb-1">
            Recent Activities ({applications.length})
          </h5>
          {applications.length === 0 ? (
            <p className="text-sm text-[var(--color-brand-secondary)] italic">No active requests inside current sandbox.</p>
          ) : (
            <div className="space-y-1.5 max-h-[120px] overflow-y-auto">
              {applications.slice(0, 3).map((app) => (
                <div key={app.id} className="p-1.5 bg-[var(--color-brand-text)]/50 rounded text-sm space-y-0.5 border-l-2 border-[var(--color-brand-gold)]">
                  <div className="flex justify-between">
                    <span className="font-bold text-white">{app.type}</span>
                    <span className="text-sm text-[var(--color-brand-secondary)]">{app.date}</span>
                  </div>
                  <p className="text-[var(--color-brand-secondary)] line-clamp-1">{app.details}</p>
                </div>
              ))}
            </div>
          )}
        </div>
      </div>
    )}

    {/* TAB 2: Content Management (Figure 10) */}
    {activeTab === "content" && (
      <div className="space-y-4 text-[15px] font-sans">
        <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/30 pb-2">
          <h4 className="font-serif italic text-sm font-bold text-[var(--color-brand-on-accent)]">Content Management</h4>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Static Content Manager</span>
        </div>

        {/* Parish content the office maintains itself — photograph,
            description, Mass times, colour. Above the older static
            editor because this is the one backed by live data. */}
        <div className="bg-[var(--color-brand-card)] p-4 rounded-2xl border border-[var(--color-brand-border)]">
          <ParishContentEditor editorEmail={userEmail} />
        </div>

        {/* Edit Church Info */}
        <form onSubmit={handleSaveChurchInfo} className="bg-[var(--color-brand-text)] p-3.5 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-
2.5">
          <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] border-b border-[var(--color-brand-primary)]/15 pb-1">
            Edit Parish Details
          </h5>

          {isSavedContent && (
            <div className="p-2 bg-green-950/70 border border-green-700 text-green-200 rounded flex items-center gap-1.5 text-sm">
              <Check className="w-3.5 h-3.5 shrink-0" />
              <span>Church database info updated successfully in current buffer!</span>
            </div>
          )}

          <div className="space-y-1.5">
```

```
                  <span className="text-sm text-[var(--color-brand-secondary)] uppercase tracking-wider font-mono">Church Name</span>
                  <input
                    type="text"
                    value={churchName}
                    onChange={(e) => setChurchName(e.target.value)}
                    className="w-full bg-[var(--color-brand-text)] border border-[var(--color-brand-primary)]/30 rounded-lg p-2 text-[15px] outline-none text-white
font-serif"
                  />
                </div>

                <div className="space-y-1.5">
                  <span className="text-sm text-[var(--color-brand-secondary)] uppercase tracking-wider font-mono">Address location</span>
                  <input
                    type="text"
                    value={churchAddress}
                    onChange={(e) => setChurchNameAddress(e.target.value)}
                    className="w-full bg-[var(--color-brand-text)] border border-[var(--color-brand-primary)]/30 rounded-lg p-2 text-[15px] outline-none text-white"
                  />
                </div>

                <button
                  type="submit"
                  className="py-1.5 px-3 bg-[var(--color-brand-primary)] text-white text-sm font-bold uppercase rounded-md border border-[var(--color-brand-
primary)]"
                >
                  Save Global Church Info
                </button>
              </form>

              {/* Announcements Editor */}
              <div className="bg-[var(--color-brand-text)] p-3.5 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-2.5">
                <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] border-b border-[var(--color-brand-primary)]/15 pb-1">
                  Create Announcement
                </h5>

                <form onSubmit={handleAddAnn} className="space-y-2">
                  <input
                    type="text"
                    placeholder="Announcement Title (e.g. Feast Day Mass)"
                    value={annTitle}
                    onChange={(e) => setAnnTitle(e.target.value)}
                    className="w-full bg-[var(--color-brand-text)] border border-[var(--color-brand-primary)]/30 rounded-lg p-2 text-[15px] outline-none text-white"
                  />

                  <div className="grid grid-cols-2 gap-2">
                    <input
                      type="text"
                      placeholder="Date (e.g. May 13, 2026)"
                      value={annDate}
                      onChange={(e) => setAnnDate(e.target.value)}
                      className="w-full bg-[var(--color-brand-text)] border border-[var(--color-brand-primary)]/30 rounded-lg p-2 text-[15px] outline-none text-
white"
                    />
                    <input
                      type="text"
                      placeholder="Time (e.g. 3:00 PM)"
                      value={annTime}
                      onChange={(e) => setAnnTime(e.target.value)}
```

```
                    className="w-full bg-[var(--color-brand-text)] border border-[var(--color-brand-primary)]/30 rounded-lg p-2 text-[15px] outline-none text-
white"
                  />
                </div>

                <button
                  type="submit"
                  disabled={!annTitle || !annDate}
                  className="w-full py-2 bg-[var(--color-brand-gold)] hover:bg-[var(--color-brand-gold-deep)] disabled:opacity-50 text-black font-bold uppercase
rounded-md text-sm tracking-wide"
                >
                  Publish Announcement to App Bulletin
                </button>
              </form>

              {/* Published List */}
              <div className="space-y-1.5 pt-2 border-t border-[var(--color-brand-text)]">
                <span className="text-sm text-[var(--color-brand-secondary)] uppercase tracking-wider font-mono">Active Announcements
({announcements.length})</span>
                <div className="space-y-1.5 max-h-[140px] overflow-y-auto">
                  {announcements.map((ann) => (
                    <div key={ann.id} className="flex justify-between items-center p-2 bg-[var(--color-brand-text)] rounded-lg border border-[var(--color-brand-
primary)]/20">
                      <div>
                        <span className="font-serif italic font-bold block">{ann.title}</span>
                        <span className="text-sm text-[var(--color-brand-secondary)]">{ann.date} at {ann.time}</span>
                      </div>
                      <button
                        onClick={() => onDeleteAnnouncement(ann.id)}
                        className="p-1 hover:text-red-400 text-gray-400"
                      >
                        <Trash2 className="w-3.5 h-3.5" />
                      </button>
                    </div>
                  ))}
                </div>
              </div>
            </div>
          </div>
        )}

        {/* TAB 3: Analytics Reporting (Figure 11) */}
        {activeTab === "analytics" && (
          <div className="space-y-4 text-[15px] font-sans">
            <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/30 pb-2">
              <h4 className="font-serif italic text-sm font-bold text-[var(--color-brand-on-accent)]">Analytics Reporting</h4>
              <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Statistical Metrics</span>
            </div>

            {/* Feature Usage Pie chart representation */}
            <div className="bg-[var(--color-brand-text)] p-4 rounded-3xl border border-[var(--color-brand-primary)]/25 space-y-3">
              <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] text-center border-b border-[var(--color-brand-
primary)]/15 pb-1.5">
                Feature Usage Distribution
              </h5>

              {/* Horizontal Bar Chart visualization */}
              <div className="space-y-2.5">
                <div>
```

```
                  <div className="flex justify-between text-sm mb-1">
                    <span>Augmented Reality AR Scans</span>
                    <strong className="text-cyan-400">902 usages (59%)</strong>
                  </div>
                  <div className="w-full h-2 bg-[var(--color-brand-text)] rounded-full overflow-hidden">
                    <div className="bg-cyan-400 h-full rounded-full" style={{ width: "59%" }}></div>
                  </div>
                </div>

                <div>
                  <div className="flex justify-between text-sm mb-1">
                    <span>Santo Rosary Guide</span>
                    <strong className="text-rose-400">366 usages (24%)</strong>
                  </div>
                  <div className="w-full h-2 bg-[var(--color-brand-text)] rounded-full overflow-hidden">
                    <div className="bg-rose-400 h-full rounded-full" style={{ width: "24%" }}></div>
                  </div>
                </div>

                <div>
                  <div className="flex justify-between text-sm mb-1">
                    <span>History of Church</span>
                    <strong className="text-amber-400">138 usages (9%)</strong>
                  </div>
                  <div className="w-full h-2 bg-[var(--color-brand-text)] rounded-full overflow-hidden">
                    <div className="bg-amber-400 h-full rounded-full" style={{ width: "9%" }}></div>
                  </div>
                </div>

                <div>
                  <div className="flex justify-between text-sm mb-1">
                    <span>Ministries & Sacraments Form</span>
                    <strong className="text-purple-400">128 usages (8%)</strong>
                  </div>
                  <div className="w-full h-2 bg-[var(--color-brand-text)] rounded-full overflow-hidden">
                    <div className="bg-purple-400 h-full rounded-full" style={{ width: "8%" }}></div>
                  </div>
                </div>
              </div>
            </div>

            {/* Weekly Visitor Frequency (Figure 11 graph representation) */}
            <div className="bg-[var(--color-brand-text)] p-4 rounded-3xl border border-[var(--color-brand-primary)]/25 space-y-3">
              <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] text-center border-b border-[var(--color-brand-primary)]/15 pb-1.5">
                Weekly Visitor Frequency
              </h5>

              {/* Simulated vertical bar graph */}
              <div className="flex items-end justify-between h-28 pt-2 select-none px-2 border-b border-[var(--color-brand-primary)]/40">
                <div className="flex flex-col items-center gap-1 w-6">
                  <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "20px" }}></div>
                  <span className="text-sm font-mono text-[var(--color-brand-secondary)]">M</span>
                </div>
                <div className="flex flex-col items-center gap-1 w-6">
                  <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "45px" }}></div>
                  <span className="text-sm font-mono text-[var(--color-brand-secondary)]">T</span>
                </div>
```

```
        <div className="flex flex-col items-center gap-1 w-6">
          <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "40px" }}></div>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)]">W</span>
        </div>
        <div className="flex flex-col items-center gap-1 w-6">
          <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "30px" }}></div>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Th</span>
        </div>
        <div className="flex flex-col items-center gap-1 w-6">
          <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "65px" }}></div>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)]">F</span>
        </div>
        <div className="flex flex-col items-center gap-1 w-6">
          <div className="bg-[var(--color-brand-gold)] rounded-t w-3" style={{ height: "80px" }}></div>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Sa</span>
        </div>
        <div className="flex flex-col items-center gap-1 w-6">
          <div className="bg-green-500 rounded-t w-3 animate-pulse" style={{ height: "98px" }}></div>
          <span className="text-sm font-mono text-[var(--color-brand-secondary)] font-bold text-green-400">Su</span>
        </div>
      </div>
    </div>
  </div>
)}

{/* TAB 4: Database Management & Visitor Logs (Figure 12) */}
{activeTab === "database" && (
  <div className="space-y-4 text-[15px] font-sans">
    <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/30 pb-2">
      <h4 className="font-serif italic text-sm font-bold text-[var(--color-brand-on-accent)]">Database Management</h4>
      <span className="text-sm font-mono text-[var(--color-brand-secondary)]">Live Query Table logs</span>
    </div>

    {/* Historical assets table */}
    <div className="bg-[var(--color-brand-text)] p-3 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-2">
      <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)] border-b border-[var(--color-brand-primary)]/15 pb-1">
        Historical Church Schemas (Stored Local)
      </h5>
      <div className="space-y-1 text-sm font-mono text-[var(--color-brand-secondary)]">
        <div className="p-1 bg-[var(--color-brand-text)] rounded flex justify-between">
          <span>1. Main Altar Content description (Maypajo)</span>
          <span className="text-green-400">[ACTIVE]</span>
        </div>
        <div className="p-1 bg-[var(--color-brand-text)] rounded flex justify-between">
          <span>2. Maria Auxiliadora 3D coordinates</span>
          <span className="text-green-400">[ACTIVE]</span>
        </div>
        <div className="p-1 bg-[var(--color-brand-text)] rounded flex justify-between">
          <span>3. Cathedral Sanctuary (San Roque) assets</span>
          <span className="text-green-400">[ACTIVE]</span>
        </div>
      </div>
    </div>

    {/* Figure 12 style: Visitor Log storage */}
    <div className="bg-[var(--color-brand-text)] p-3 rounded-2xl border border-[var(--color-brand-primary)]/25 space-y-2">
      <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/15 pb-1">
        <h5 className="font-serif italic text-[15px] font-bold text-[var(--color-brand-on-accent)]">
          Visitor Logs & Transactions
```

```
                  </h5>
                  <span className="text-sm font-mono bg-[var(--color-brand-primary)] text-white px-1.5 py-0.5 rounded">
                    {applications.length} Records
                  </span>
                </div>

                {applications.length === 0 ? (
                  <p className="text-sm text-[var(--color-brand-secondary)] italic text-center py-4">No data logged. Test forms/scans inside smartphone emulator
first!</p>
                ) : (
                  <div className="space-y-1.5 max-h-[220px] overflow-y-auto">
                    {applications.map((app) => (
                      <div key={app.id} className="p-2.5 bg-[var(--color-brand-text)] rounded border border-[var(--color-brand-primary)]/15 text-sm space-y-2">
                        <div className="flex justify-between items-center border-b border-[var(--color-brand-primary)]/10 pb-0.5">
                          <span className="font-bold text-[var(--color-brand-on-accent)] uppercase tracking-wider text-sm">{app.type}</span>
                          <button
                            onClick={() => onDeleteApplication(app.id)}
                            className="text-red-400 hover:text-red-500 font-mono text-sm"
                          >
                            DELETE
                          </button>
                        </div>
                        <div>
                          <span className="text-white font-serif italic block">Applicant: {app.applicant}</span>

                          {/* A ministry application has its own fields, so
                              the office reads them rather than picking them
                              out of one run-on line. Anything without them
                              (a sacrament booking, or a row seeded before
                              they existed) still shows its summary. */}
                          {isMinistryApplication(app) && app.ministryName ? (
                            <div className="text-sm font-mono text-[var(--color-brand-secondary)] space-y-0.5 mt-1">
                              <div>Ministry: <span className="text-white">{app.ministryName}</span></div>
                              {app.parishName && <div>Parish: <span className="text-white">{app.parishName}</span></div>}
                              {app.email && <div className="break-all">Email: <span className="text-white">{app.email}</span></div>}
                              <div>Mobile: <span className="text-white">{app.mobile || "not given"}</span></div>
                              {app.message && <div className="leading-normal">Message: <span className="text-white">{app.message}</span></div>}
                            </div>
                          ) : (
                            <p className="text-[var(--color-brand-secondary)] font-mono leading-normal break-all text-sm">{app.details}</p>
                          )}
                        </div>
                        <div className="flex justify-between items-center pt-0.5 text-sm font-mono text-[var(--color-brand-secondary)]">
                          <span>Logged: {app.date}</span>
                          <span className="text-green-400 font-bold uppercase tracking-wider bg-[var(--color-brand-primary)]/25 px-1.5 py-0.5
rounded">{app.status}</span>
                        </div>

                        {/* Status update/Review panel */}
                        <div className="pt-1.5 border-t border-[var(--color-brand-primary)]/10 flex flex-wrap items-center gap-1">
                          <span className="text-[7.5px] text-[var(--color-brand-secondary)] uppercase tracking-wider block mr-1 font-bold">Review Status:</span>
                          <button
                            onClick={() => handleOpenApproveModal(app)}
                            className="bg-green-900/40 hover:bg-green-900/70 text-green-300 font-mono text-[7.5px] px-2 py-0.5 rounded border border-green-700/40
transition-all uppercase font-bold"
                          >
                            Approve
                          </button>
                          <button
```

```
                          onClick={() => handleOpenScheduleModal(app)}
                          className="bg-blue-900/40 hover:bg-blue-900/70 text-blue-300 font-mono text-[7.5px] px-2 py-0.5 rounded border border-blue-700/40
transition-all uppercase font-bold"
                        >
                          Schedule
                        </button>
                        <button
                          onClick={() => handleOpenRejectModal(app)}
                          className="bg-rose-900/40 hover:bg-rose-900/70 text-rose-300 font-mono text-[7.5px] px-2 py-0.5 rounded border border-rose-700/40
transition-all uppercase font-bold"
                        >
                          Reject
                        </button>
                      </div>

                      {/* Setting the status directly, with no email to
                          compose. The three buttons above open the
                          email modals and still do; this is for the
                          common case of moving an application along.
                          The existing flow is untouched because some
                          offices do want to write to the applicant. */}
                      {isMinistryApplication(app) && (
                        <div className="pt-1.5 border-t border-[var(--color-brand-primary)]/10 flex items-center gap-2">
                          <label
                            htmlFor={`status-${app.id}`}
                            className="text-[7.5px] text-[var(--color-brand-secondary)] uppercase tracking-wider font-bold shrink-0"
                          >
                            Set status
                          </label>
                          <select
                            id={`status-${app.id}`}
                            value={MINISTRY_STATUSES.includes(app.status as never) ? app.status : ""}
                            onChange={(e) => {
                              if (e.target.value) onUpdateApplicationStatus(app.id, e.target.value);
                            }}
                            className="flex-1 bg-[var(--color-brand-text)] text-white font-mono text-sm px-2 py-1 rounded border border-[var(--color-brand-
primary)]/40 outline-none"
                          >
                            {/* A status from the older email flow is not
                                one of the four, so it is offered as the
                                current value rather than silently
                                displaying "Pending". */}
                            {!MINISTRY_STATUSES.includes(app.status as never) && (
                              <option value="">{app.status} (current)</option>
                            )}
                            {MINISTRY_STATUSES.map((status) => (
                              <option key={status} value={status}>
                                {statusPresentation(status).dot} {status}
                              </option>
                            ))}
                          </select>
                        </div>
                      )}
                    </div>
                  ))}
                </div>
              )}
            </div>
          </div>
```

```
          )}
        </div>
      </div>

      {/* Interactive Email/Scheduling Modal Overlay */}
      {selectedAppForAction && (
        <div className="fixed inset-0 bg-black/75 backdrop-blur-xs flex items-center justify-center p-4 z-50 text-[var(--color-brand-text)] font-sans">
          {/* Height-safe: capped at the overlay, header fixed, body scrolls.
              This one compiles a whole email, so it was the tallest card in
              the app and the first to push its own close button off the top
              of a short screen - and `overflow-hidden` meant nothing
              scrolled to bring it back. */}
          <div className="bg-[var(--color-brand-card)] rounded-3xl w-full max-w-md max-h-full flex flex-col overflow-hidden shadow-2xl border border-[var(--color-
brand-border)] animate-in fade-in zoom-in-95 duration-200">
            {/* Modal Header */}
            <div className="shrink-0 bg-[var(--color-brand-primary)] text-white p-4 flex items-center justify-between border-b border-[var(--color-brand-
border)]/30">
              <div className="flex items-center gap-1.5">
                <Mail className="w-4 h-4 text-[var(--color-brand-on-accent)]" />
                <h3 className="text-[15px] font-bold tracking-wider uppercase font-serif italic text-white">
                  {selectedAppForAction.actionType === "approve" && "Draft Approval & Congratulatory Form"}
                  {selectedAppForAction.actionType === "schedule" && "Schedule Appointment & Draft Invitation"}
                  {selectedAppForAction.actionType === "reject" && "Compose Revision / Rejection Notice"}
                </h3>
              </div>
              <button
                disabled={isSendingEmail || emailSentSuccess}
                onClick={() => setSelectedAppForAction(null)}
                className="text-[var(--color-brand-secondary)] hover:text-white transition-colors"
              >
                <X className="w-4 h-4" />
              </button>
            </div>

            {/* Modal Body */}
            <div className="min-h-0 flex-1 overflow-y-auto p-4 space-y-3.5">
              {emailSentSuccess ? (
                <div className="py-6 text-center space-y-3">
                  <div className={`w-12 h-12 rounded-full flex items-center justify-center mx-auto ${isLastEmailSimulated ? 'bg-amber-100 text-amber-700' : 'bg-
green-100 text-green-700'}`}>
                    {isLastEmailSimulated ? (
                      <Clock className="w-6 h-6 stroke-[2.5]" />
                    ) : (
                      <Check className="w-6 h-6 stroke-[3]" />
                    )}
                  </div>
                  <div>
                    <h4 className="text-[15px] font-bold font-serif italic text-gray-900">
                      {isLastEmailSimulated ? "Database Committed (Simulated Email)" : "Real Email Transmitted Successfully!"}
                    </h4>
                    <p className="text-[9.5px] text-gray-600 max-w-xs mx-auto mt-1 leading-relaxed">
                      {isLastEmailSimulated ? (
                        <>
                          The database state has successfully transitioned to <span className="font-bold text-[var(--color-brand-secondary)]">Approved /
Scheduled</span> in Firestore!
                          <span className="block mt-1.5 text-[8.5px] bg-amber-50 p-2 rounded-lg border border-amber-200/50 text-amber-800">
                            <strong>Note:</strong> Since <strong>SMTP_USER</strong> and <strong>SMTP_PASS</strong> secrets are not configured in AI Studio, a real
physical email could not be delivered to <span className="font-mono">{recipientEmail}</span>. Add your credentials in Secrets to trigger actual emails!
                          </span>
```

```
                      </>
                    ) : (
                      <>
                        The actual email correspondence was dispatched using your SMTP mail server straight to <strong className="font-mono text-gray-
800">{recipientEmail}</strong>.
                      </>
                    )}
                  </p>
                </div>
              </div>
            ) : (
              <div className="space-y-3">
                {/* SMTP Status banner */}
                {smtpConfig && (
                  <div className={`p-2 rounded-xl text-[8.5px] leading-normal font-mono border ${smtpConfig.isConfigured ? 'bg-green-50 text-green-800 border-
green-200' : 'bg-amber-50 text-amber-800 border-amber-200'}`}>
                    {smtpConfig.isConfigured ? (
                      <span>● Real Email Delivery is ACTIVE via <strong className="underline">{smtpConfig.user}</strong></span>
                    ) : (
                      <span>
                        ⚠️ Real Delivery INACTIVE (SMTP Credentials Missing). Click <strong>"Send Email & Commit"</strong> to simulate database logging & status
updates!
                      </span>
                    )}
                  </div>
                )}

                {/* SMTP Error Banner */}
                {errorMessage && (
                  <div className="p-2.5 rounded-xl bg-red-50 border border-red-200 text-red-800 text-sm leading-relaxed font-sans">
                    <strong className="block font-bold mb-0.5">⚠️ Email Dispatch Failed:</strong>
                    <span className="font-mono text-sm break-words">{errorMessage}</span>
                    <p className="mt-1.5 text-[8.5px] text-red-600 font-serif italic">
                      <strong>Troubleshooting Tips:</strong>
                      <span className="block mt-0.5 ml-1">• If using Gmail/Google Workspace, you <strong>MUST</strong> use an <strong className="underline">App
Password</strong> (16 characters) instead of your primary password.</span>
                      <span className="block ml-1">• Double-check that your Secrets variable names (e.g., <span className="font-mono">SMTP_USER</span>, <span
className="font-mono">SMTP_PASS</span>) match exactly in AI Studio settings.</span>
                    </p>
                  </div>
                )}

                {/* Recipient Details */}
                <div className="bg-[var(--color-brand-card)] p-2.5 rounded-xl border border-[var(--color-brand-border)]/45 space-y-1">
                  <div className="flex justify-between items-center text-[8.5px] text-[var(--color-brand-secondary)] uppercase font-mono">
                    <span>Recipient Applicant</span>
                    <span>ID: {selectedAppForAction.app.id}</span>
                  </div>
                  <strong className="block text-[15px] text-[var(--color-brand-text)] font-serif italic">{selectedAppForAction.app.applicant}</strong>
                  <span className="block text-sm text-[var(--color-brand-secondary)] truncate">{selectedAppForAction.app.type}</span>
                </div>

                {/* Recipient Email Address Input */}
                <div className="space-y-1">
                  <label className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
                    Destination Email Address
                  </label>
                  <input
```

```
                    type="email"
                    value={recipientEmail}
                    onChange={(e) => setRecipientEmail(e.target.value)}
                    className="w-full bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-xl p-2 text-[15px] outline-none text-[var(--
color-brand-text)] font-mono"
                    placeholder="e.g. applicant@email.com"
                  />
                </div>

                {/* Scheduling Fields if actionType is "schedule" */}
                {selectedAppForAction.actionType === "schedule" && (
                  <div className="grid grid-cols-2 gap-2 bg-blue-50/50 p-2.5 rounded-xl border border-blue-200/50">
                    <div className="space-y-1">
                      <label className="text-[8.5px] font-bold text-blue-900 uppercase tracking-wider font-serif italic flex items-center gap-1">
                        <CalendarRange className="w-3 h-3" /> Date
                      </label>
                      <input
                        type="date"
                        value={scheduleDate}
                        onChange={(e) => {
                          setScheduleDate(e.target.value);
                          updateScheduleEmailBody(e.target.value, scheduleTime);
                        }}
                        className="w-full bg-[var(--color-brand-card)] border border-blue-300 rounded-lg p-1.5 text-[15px] outline-none text-[var(--color-brand-
text)] font-mono"
                      />
                    </div>
                    <div className="space-y-1">
                      <label className="text-[8.5px] font-bold text-blue-900 uppercase tracking-wider font-serif italic flex items-center gap-1">
                        <Clock className="w-3 h-3" /> Time
                      </label>
                      <input
                        type="text"
                        value={scheduleTime}
                        onChange={(e) => {
                          setScheduleTime(e.target.value);
                          updateScheduleEmailBody(scheduleDate, e.target.value);
                        }}
                        placeholder="e.g. 10:00 AM"
                        className="w-full bg-[var(--color-brand-card)] border border-blue-300 rounded-lg p-1.5 text-[15px] outline-none text-[var(--color-brand-
text)] font-mono"
                      />
                    </div>
                  </div>
                )}

                {/* Subject Line */}
                <div className="space-y-1">
                  <label className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
                    Email Subject
                  </label>
                  <input
                    type="text"
                    value={emailSubject}
                    onChange={(e) => setEmailSubject(e.target.value)}
                    className="w-full bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-xl p-2 text-[15px] outline-none text-[var(--
color-brand-text)] font-sans"
                  />
                </div>
```

```
                {/* Email Message Composer */}
                <div className="space-y-1">
                  <label className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
                    Email Message Copy
                  </label>
                  <textarea
                    rows={5}
                    value={emailBody}
                    onChange={(e) => setEmailBody(e.target.value)}
                    className="w-full bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-xl p-2.5 text-[15px] outline-none text-[var(--
color-brand-text)] font-sans resize-none leading-normal"
                  />
                </div>

                {/* Actions */}
                <div className="pt-2 flex gap-2">
                  <button
                    type="button"
                    disabled={isSendingEmail}
                    onClick={() => setSelectedAppForAction(null)}
                    className="flex-1 py-2 bg-gray-100 hover:bg-gray-200 text-gray-700 text-sm font-bold uppercase tracking-wider rounded-xl transition-colors"
                  >
                    Cancel
                  </button>
                  <button
                    type="button"
                    disabled={isSendingEmail || !recipientEmail}
                    onClick={handleSendEmailAction}
                    className="flex-1 py-2 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] disabled:opacity-50 text-white text-sm font-
bold uppercase tracking-wider rounded-xl transition-colors flex items-center justify-center gap-1.5"
                  >
                    {isSendingEmail ? (
                      <>
                        <Loader2 className="w-3.5 h-3.5 animate-spin" />
                        <span>Transmitting...</span>
                      </>
                    ) : (
                      <>
                        <Send className="w-3.5 h-3.5" />
                        <span>Send Email & Commit</span>
                      </>
                    )}
                  </button>
                </div>
              </div>
            )}
          </div>
        </div>
      )}
    </div>
  );
}
```

1.2.2 APP-007 — ApplicationForm.tsx

Code ID	APP-007
File Name	ApplicationForm.tsx
File Path	SanctDemo/src/components/ApplicationForm.tsx
Language / Type	TypeScript (React, TSX)
Lines	376
Functions / Classes	ApplicationForm (exported component/function (default)); FieldInput (internal function); FieldError (internal function); Review (internal function); Submitted (internal function)
Related Files (its imports)	../lib/applications
Where It Is Used	src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/ApplicationForm.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ApplicationForm.tsx

```
import { useEffect, useMemo, useState } from "react";
import { ArrowLeft, Check, CheckCircle2, Loader2 } from "lucide-react";
import {
  ministrySteps, sacramentSteps, reviewRows, toFormData, validateStep,
  type Answers, type Field, type Step,
} from "../lib/applicationForm";
import { submitApplication, SubmitError } from "../lib/applications";

/**
 * Applying, on a screen of its own.
 *
 * ## Why this is not a form under the description
 *
 * It was, for both ministries and sacraments, and for sacraments it was
 * worse than that - Apply scrolled you to a form at the bottom of the
 * page that asked, again, which sacrament you wanted. Reading about
 * something and applying for it are two different jobs, and a page that
 * does both makes the second one feel like a footnote to the first.
 *
 * Taking over the screen also gives the form somewhere to put its steps,
 * its validation and its review, none of which fit underneath a
 * description without burying the thing you were reading.
 *
 * ## Why a review step
 *
 * The parish acts on what is submitted, and an applicant cannot edit it
 * afterwards - the rules forbid it, on purpose. So the last chance to
 * catch a mistyped number has to be here.
 */
export default function ApplicationForm({
  kind,
```

```
  itemId,
  itemName,
  parishName,
  defaults,
  onClose,
  onSubmitted,
}: {
  kind: "ministry" | "sacrament";
  itemId: string;
  itemName: string;
  parishName: string;
  /** From the signed-in account, so nobody retypes what the app knows. */
  defaults?: { fullName?: string; email?: string; mobile?: string };
  onClose: () => void;
  /** Lets the host tab refresh its own list. */
  onSubmitted?: () => void;
}) {
  const steps = useMemo(
    () => (kind === "ministry" ? ministrySteps(itemId, itemName) : sacramentSteps(itemName)),
    [kind, itemId, itemName],
  );

  const [index, setIndex] = useState(0);
  const [answers, setAnswers] = useState<Answers>(() => ({
    fullName: defaults?.fullName ?? "",
    mobile: defaults?.mobile ?? "",
  }));
  const [errors, setErrors] = useState<Record<string, string>>({});
  const [busy, setBusy] = useState(false);
  const [failed, setFailed] = useState<string | null>(null);
  const [reference, setReference] = useState<string | null>(null);

  // Each step starts at the top. Without this, step three opens scrolled
  // to wherever step two's last field was.
  useEffect(() => { window.scrollTo?.({ top: 0 }); }, [index]);

  const reviewing = index === steps.length;
  const step = steps[index];

  function set(name: string, value: string | string[] | boolean) {
    setAnswers(a => ({ ...a, [name]: value }));
    // Clearing as they fix it, rather than making them press Continue
    // again to find out whether they have.
    setErrors(e => (e[name] ? { ...e, [name]: "" } : e));
  }

  function next() {
    if (!step) return;
    const found = validateStep(step, answers);
    if (Object.keys(found).length > 0) {
      setErrors(found);
      return;
    }
    setErrors({});
    setIndex(i => i + 1);
  }

  async function submit() {
    setBusy(true);
```

```
  setFailed(null);
  try {
    const saved = await submitApplication({
      kind,
      itemId,
      type: itemName,
      applicantName: String(answers.fullName ?? "").trim(),
      formData: {
        ...toFormData(steps, answers),
        // Kept alongside the answers, as the old forms did, so the
        // parish's export and detail view keep reading what they read.
        [kind === "ministry" ? "ministryId" : "sacramentId"]: itemId,
        [kind === "ministry" ? "ministryName" : "sacramentName"]: itemName,
        parishName,
      },
    });
    setReference(saved.referenceNumber);
    onSubmitted?.();
  } catch (err) {
    setFailed(
      err instanceof SubmitError
        ? err.message
        : "That could not be sent. Check your connection and try again.",
    );
  } finally {
    setBusy(false);
  }
}

if (reference) {
  return <Submitted reference={reference} itemName={itemName} parishName={parishName} onClose={onClose} />;
}

return (
  <div className="apply-screen">
    <header className="apply-screen__bar">
      <button
        type="button"
        onClick={() => (index === 0 ? onClose() : setIndex(i => i - 1))}
        className="apply-screen__back"
      >
        <ArrowLeft className="w-4 h-4" />
        {index === 0 ? "Back" : "Previous"}
      </button>
      <p className="apply-screen__count">
        {reviewing ? "Review" : `Step ${index + 1} of ${steps.length + 1}`}
      </p>
    </header>

    <div className="apply-screen__body">
      <p className="apply-screen__eyebrow">
        {kind === "ministry" ? "Ministry application" : "Sacrament application"}
      </p>
      <h1 className="apply-screen__title">{itemName}</h1>
      <p className="apply-screen__parish">{parishName}</p>

      {/* A bar, not a number alone. Four steps is short, but not
          knowing whether it is four or fourteen is what makes people
          abandon a form on the first screen. */}
```

```
        <div className="apply-progress" aria-hidden>
          {steps.map((_, i) => (
            <span key={i} className={`apply-progress__seg${i <= index ? " is-done" : ""}`} />
          ))}
          <span className={`apply-progress__seg${reviewing ? " is-done" : ""}`} />
        </div>

        {reviewing ? (
          <Review
            steps={steps}
            answers={answers}
            busy={busy}
            failed={failed}
            onEdit={() => setIndex(0)}
            onSubmit={() => void submit()}
          />
        ) : (
          <>
            <h2 className="apply-step__title">{step!.title}</h2>
            {step!.blurb && <p className="apply-step__blurb">{step!.blurb}</p>}

            <div className="mt-4 space-y-5">
              {step!.fields.map(field => (
                <FieldInput
                  key={field.name}
                  field={field}
                  value={answers[field.name]}
                  error={errors[field.name]}
                  onChange={v => set(field.name, v)}
                />
              ))}
            </div>

            <button type="button" onClick={next} className="apply-primary mt-7">
              {index === steps.length - 1 ? "Review your application" : "Continue"}
            </button>
          </>
        )}
      </div>
    </div>
  );
}

function FieldInput({
  field, value, error, onChange,
}: {
  field: Field;
  value: string | string[] | boolean | undefined;
  error?: string;
  onChange: (value: string | string[] | boolean) => void;
}) {
  const id = `f-${field.name}`;
  const described = [field.help ? `${id}-help` : null, error ? `${id}-err` : null]
    .filter(Boolean).join(" ") || undefined;

  if (field.kind === "confirm") {
    return (
      <div>
        <label className="flex items-start gap-3 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
```

```
        <input
          type="checkbox"
          checked={value === true}
          aria-describedby={described}
          onChange={e => onChange(e.target.checked)}
          className="mt-0.5 w-5 h-5 shrink-0 accent-[var(--color-brand-primary)]"
        />
        <span>{field.label}</span>
      </label>
      {error && <FieldError id={`${id}-err`}>{error}</FieldError>}
    </div>
  );
}

if (field.kind === "choices") {
  const chosen = Array.isArray(value) ? value : [];
  return (
    <fieldset>
      <legend className="apply-label">
        {field.label}
        {!field.required && <span className="apply-optional"> — optional</span>}
      </legend>
      {field.help && <p id={`${id}-help`} className="apply-help">{field.help}</p>}
      <div className="mt-2 flex flex-wrap gap-2">
        {(field.options ?? []).map(option => {
          const on = chosen.includes(option);
          return (
            <button
              key={option}
              type="button"
              aria-pressed={on}
              onClick={() =>
                onChange(on ? chosen.filter(c => c !== option) : [...chosen, option])
              }
              className={`apply-chip${on ? " is-on" : ""}`}
            >
              {on && <Check className="w-3.5 h-3.5" />}
              {option}
            </button>
          );
        })}
      </div>
      {error && <FieldError id={`${id}-err`}>{error}</FieldError>}
    </fieldset>
  );
}

const common = {
  id,
  value: typeof value === "string" ? value : "",
  placeholder: field.placeholder,
  maxLength: field.maxLength,
  "aria-describedby": described,
  "aria-invalid": error ? true : undefined,
  onChange: (e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) =>
    onChange(e.target.value),
  className: `apply-input${error ? " is-bad" : ""}`,
};
```

```
  return (
    <div>
      <label htmlFor={id} className="apply-label">
        {field.label}
        {!field.required && <span className="apply-optional"> — optional</span>}
      </label>
      {field.help && <p id={`${id}-help`} className="apply-help">{field.help}</p>}
      {field.kind === "textarea"
        ? <textarea {...common} rows={4} />
        : <input {...common} type={field.kind === "tel" ? "tel" : field.kind === "date" ? "date" : "text"} />}
      {error && <FieldError id={`${id}-err`}>{error}</FieldError>}
    </div>
  );
}

function FieldError({ id, children }: { id: string; children: React.ReactNode }) {
  return (
    <p id={id} role="alert" className="mt-1.5 text-[14px] font-semibold text-[var(--color-brand-error)]">
      {children}
    </p>
  );
}

function Review({
  steps, answers, busy, failed, onEdit, onSubmit,
}: {
  steps: Step[];
  answers: Answers;
  busy: boolean;
  failed: string | null;
  onEdit: () => void;
  onSubmit: () => void;
}) {
  const rows = reviewRows(steps, answers);
  return (
    <>
      <h2 className="apply-step__title">Check this over</h2>
      <p className="apply-step__blurb">
        Once it is sent you cannot change it yourself — the parish is working from
        it. Ring the office if something needs correcting.
      </p>

      {failed && (
        <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {failed}
        </p>
      )}

      <dl className="mt-4 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] divide-y divide-[var(--color-brand-border)]">
        {rows.map(row => (
          <div key={row.label} className="px-4 py-3">
            <dt className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
              {row.label}
            </dt>
            <dd className="mt-0.5 text-[16px] leading-relaxed text-[var(--color-brand-text)] break-words">
              {row.value}
            </dd>
          </div>
        ))}
```

```
      </dl>

      <button type="button" onClick={onSubmit} disabled={busy} className="apply-primary mt-6">
        {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : null}
        {busy ? "Sending…" : "Submit application"}
      </button>
      <button type="button" onClick={onEdit} disabled={busy} className="apply-secondary mt-2.5">
        Change something
      </button>
    </>
  );
}

function Submitted({
  reference, itemName, parishName, onClose,
}: { reference: string; itemName: string; parishName: string; onClose: () => void }) {
  return (
    <div className="apply-screen">
      <div className="apply-screen__body text-center">
        <CheckCircle2 className="mx-auto w-14 h-14 text-[var(--color-brand-success)]" />
        <h1 className="mt-3 text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
          Application sent
        </h1>
        <p className="mt-2 text-[16px] leading-relaxed text-[var(--color-brand-text)]">
          Your application for <strong>{itemName}</strong> is with {parishName}.
        </p>

        <div className="mt-5 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-5">
          <p className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
            Your reference
          </p>
          {/* Big and monospaced because its job is to be read down a
              telephone to a parish secretary. */}
          <p className="mt-1 font-mono text-[22px] font-bold tracking-wide text-[var(--color-brand-text)]">
            {reference}
          </p>
          <p className="mt-3 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
            Status: <strong className="text-[var(--color-brand-text)]">Pending</strong>. You
            will be told here when the parish has looked at it — it is under
            My Applications on the Me tab.
          </p>
        </div>

        <button type="button" onClick={onClose} className="apply-primary mt-6">
          Done
        </button>
      </div>
    </div>
  );
}
```

1.2.3 APP-008 — ArPlaceholder.tsx

Code ID	APP-008
File Name	ArPlaceholder.tsx
File Path	SanctDemo/src/components/ArPlaceholder.tsx
Language / Type	TypeScript (React, TSX)
Lines	75
Functions / Classes	ArPlaceholder (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ArPlaceholder.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ArPlaceholder.tsx

```
import React from "react";
import { Sparkles, ScanLine, MapPin, BookOpenText, Hourglass } from "lucide-react";

export default function ArPlaceholder() {
  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto">
      {/* Page Header */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden shrink-0 border-b border-[var(--color-
brand-border)]">
        <div className="absolute right-0 top-0 opacity-10 translate-x-4 -translate-y-4">
          <ScanLine className="w-32 h-32 text-white" />
        </div>
        <div className="flex items-center gap-1.5 text-[var(--color-brand-on-accent)] font-bold text-[15px] tracking-wider uppercase font-serif italic">
          <Sparkles className="w-3.5 h-3.5" /> Coming Soon
        </div>
        <div>
          <h2 className="text-2xl font-bold font-serif italic tracking-tight">
            AR Tour
          </h2>
          <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-95 mt-1 max-w-xs leading-relaxed font-sans">
            An augmented-reality guide to the parish, built directly into SanctiWalk.
          </p>
        </div>
      </div>

      <div className="p-4 space-y-4">
        {/* Status card */}
        <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-5 shadow-xs space-y-3 text-center">
          <div className="h-14 w-14 rounded-2xl bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] flex items-center justify-center mx-auto text-
[var(--color-brand-accent)]">
            <Hourglass className="w-7 h-7" />
          </div>
          <h3 className="text-base font-bold text-[var(--color-brand-text)] font-serif italic">
```

```
            The AR Tour is still being built
          </h3>
          <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans">
            This is a placeholder for our real-world augmented reality feature, which is
            still in development. It will let pilgrims explore the church through their
            camera, right from this app.
          </p>
        </div>

        {/* What it will do */}
        <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-4.5 shadow-xs space-y-3">
          <h4 className="text-sm font-bold text-[var(--color-brand-text)] uppercase tracking-wider font-sans border-b border-[var(--color-brand-border)]/45 pb-1.5">
            What the AR Tour will do
          </h4>

          <div className="flex gap-3 items-start">
            <div className="h-9 w-9 rounded-xl bg-[var(--color-brand-card)] flex items-center justify-center shrink-0 text-[var(--color-brand-accent)]">
              <MapPin className="w-4.5 h-4.5" />
            </div>
            <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans pt-1">
              Point your phone's camera at a marked feature inside the church — an altar,
              a statue, a historical marker.
            </p>
          </div>

          <div className="flex gap-3 items-start">
            <div className="h-9 w-9 rounded-xl bg-[var(--color-brand-card)] flex items-center justify-center shrink-0 text-[var(--color-brand-accent)]">
              <BookOpenText className="w-4.5 h-4.5" />
            </div>
            <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans pt-1">
              Its history, meaning, and significance to the parish will appear on screen,
              turning a self-guided walk into a living tour.
            </p>
          </div>
        </div>

        <p className="text-sm text-[var(--color-brand-secondary)] text-center leading-relaxed font-sans px-2">
          We're building this feature for real church spaces. Check back in a future
          update once it's ready.
        </p>
      </div>
    </div>
  );
}
```

1.2.4 APP-009 — ArScannerPanel.tsx

Code ID	APP-009
File Name	ArScannerPanel.tsx
File Path	SanctDemo/src/components/ArScannerPanel.tsx
Language / Type	TypeScript (React, TSX)
Lines	83
Functions / Classes	ArStage (exported type); ArScannerPanel (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/components/ArScannerPanel.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ArScannerPanel.tsx

```
import { Check } from "lucide-react";

/**
 * The AR half of the scanner.
 *
 * ## Why it stops at the door
 *
 * It used to run a tour of its own once the handover happened: stop 1
 * of 3, the station's name, its narration read aloud, End tour, Next
 * stop. All of it over a camera view that was no longer doing anything,
 * underneath an AR app that was already showing the station's name and
 * already reading it aloud — and the two could not agree, because only
 * the AR app knows which statue the pilgrim is standing in front of.
 * SanctiWalk was guessing and the guess was on top.
 *
 * So the tour belongs to the AR app, whole. This is the door to it and
 * the way back in, and nothing else.
 *
 * ## Why it no longer asks the pilgrim to find the marker first
 *
 * It used to: "I can see the marker", pressed before the handover. That
 * made the pilgrim do the marker step twice, because the AR app opens on
 * its own marker screen and reads the sheet itself - so the honest
 * answer to "are you at a marker" was always "the next app will tell
 * you". Two screens asking the same question, and the first one could
 * only guess.
 *
 * So this is now a door, not a checkpoint: it says what is about to
 * happen and opens the tour. Finding the marker belongs to the app that
 * can actually see it.
 */

export type ArStage = "find" | "reading" | "live";
```

```
export default function ArScannerPanel({
  stage,
  error,
  onFoundMarker,
  parishName,
}: {
  stage: ArStage;
  /** Why the handover to the AR app did not happen, when it did not. */
  error?: string | null;
  onFoundMarker: () => void;
  parishName: string;
}) {
  if (stage === "reading") {
    return (
      <div className="scan-panel">
        <h3 className="scan-panel__title">Opening the AR tour</h3>
        <div className="scan-progress" role="progressbar" aria-label="Opening the AR tour" />
        <p className="scan-caption">Starting the AR app for {parishName}.</p>
      </div>
    );
  }

  // "live" means the AR app took the screen and the pilgrim has come
  // back to this one. The tour is still theirs to walk over there, so
  // the only thing worth offering is the way back.
  const live = stage === "live";

  return (
    <div className="scan-panel">
      <h3 className="scan-panel__title">
        {live ? "AR tour is open" : "AR Museum tour"}
      </h3>
      {error ? (
        <p className="scan-panel__body" role="alert" style={{ color: "#ffd9a8" }}>
          {error}
        </p>
      ) : (
        <p className="scan-panel__body">
          {live
            ? "The tour is running in the SanctiWalk AR app. Switch back to it to carry on walking."
            : "Walk the church and hear each statue, altar and image tell its own story. The tour opens in the SanctiWalk AR app and starts by reading the printed
marker."}
        </p>
      )}
      <button type="button" onClick={onFoundMarker} className="scan-action">
        <Check className="w-4 h-4" /> {live ? "Back to the AR tour" : "Open AR tour"}
      </button>
    </div>
  );
}
```

1.2.5 APP-010 — ArTour.tsx

Code ID	APP-010
File Name	ArTour.tsx
File Path	SanctDemo/src/components/ArTour.tsx
Language / Type	TypeScript (React, TSX)
Lines	1392
Functions / Classes	stationToRecognition (internal function); ArTour (exported component/function (default))
Related Files (its imports)	../lib/useCamera, ../lib/useQrScanner, ../lib/qr, ../types, ../data, ../lib/arApp, ./PreviewVideo, ../lib/useNarration, ../lib/scanNarration, ../lib/idToken, ../lib/scanQuota, ../lib/useAuthUid, ../lib/simulatorMode, ../lib/useSimulatorMode, ../context/PresenceContext, ../lib/apiBase, ./ArScannerPanel, ../lib/useLanguage, ../lib/contentTranslations, ../lib/language, ../lib/appKey
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/ArTour.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ArTour.tsx

```
import { useCallback, useEffect, useState } from "react";
import {
  Camera,
  ScanLine,
  SwitchCamera,
  X,
  Sparkles,
  AlertTriangle,
  Loader2,
  ListChecks,
  PlayCircle,
  Download,
  ChevronRight,
  ChevronLeft,
  WifiOff,
  QrCode,
  MapPin,
  Volume2,
  VolumeX,
} from "lucide-react";
import { useCamera, captureFailed, type CameraStatus, type CaptureFailure } from "../lib/useCamera";
import { useQrScanner } from "../lib/useQrScanner";
import type { QrScanResult } from "../lib/qr";
import type { Station } from "../types";
import { PARISH_PATRON_IMAGES, PARISH_HEADER_IMAGES } from "../data";

const PLACEHOLDER_PHOTO = "/parish/placeholder-photo.svg";
```

```
/** The church itself, for the card about pointing a camera around it. */
const PARISH_CHURCH_PHOTOS: Record<string, string> = {
  "route-mhcp": "/parish/mary-help-history.jpg",
  "route-src": "/parish/san-roque-church.jpg",
};
import { Capacitor } from "@capacitor/core";
import { arAvailability, arLaunchParish, arTourUrl } from "../lib/arApp";
import PreviewVideo from "./PreviewVideo";
import { useNarration } from "../lib/useNarration";
import { scanScript } from "../lib/scanNarration";
import { idToken } from "../lib/idToken";
import { SCANS_PER_PARISH_PER_DAY } from "../lib/scanQuota";
import { useAuthUid } from "../lib/useAuthUid";
import { arAccess } from "../lib/simulatorMode";
import { useSimulatorMode } from "../lib/useSimulatorMode";
import { usePresence } from "../context/PresenceContext";
import { apiUrl } from "../lib/apiBase";
import {
  ScannerHeader, ScannerSheet, ScannerTabs, ScannerReticle, type ScannerMode,
} from "./ScannerChrome";
import ArScannerPanel, { type ArStage } from "./ArScannerPanel";
import { useLanguage } from "../lib/useLanguage";
import { stationText } from "../lib/contentTranslations";
import type { Language } from "../lib/language";
import { withAppKey } from "../lib/appKey";

/**
 * ArTour — the camera experience behind the AR Tour tab.
 *
 * The camera fills the tab and stays visible; recognised information rises over
 * it as a card rather than replacing it. That is the decision recorded in
 * docs/ar-and-pilgrim-tour-decisions.md — an information card anchored over the
 * camera view, not a 3D model.
 *
 * `stations` is the accuracy lever for the live scan (names become the
 * `candidates` shortlist — see /api/identify in server.ts) AND the source
 * list for manual selection below, so the two paths describe the same set of
 * places.
 *
 * The camera can only open on a secure origin (https, or localhost) — a
 * browser rule, not a bug this app can route around. Most real-world use is
 * a phone on the parish wifi at a bare LAN IP, which is insecure by default.
 * Rather than dead-ending there, every non-ready state offers "choose your
 * station manually" as a fallback that always works — see docs/CAMERA-SETUP.md
 * for how to get the camera itself working (HTTPS).
 */

interface Recognition {
  recognized: boolean;
  title: string;
  category: string;
  /** Omitted for manual selections — there's no vision-model confidence to show. */
  confidence?: number;
  summary: string;
  highlights: string[];
  /** "ai" = the vision model recognised this. "qr" = the pilgrim scanned the
   *  parish's own printed code, which is the most certain of the three: the
   *  code names exactly one station and nothing was inferred. "manual" = they
   *  picked it from the list because the camera couldn't open or couldn't
```

```
   *  tell. Mirrors the coordinatesVerified / scheduleVerified pattern in
   *  data.ts: unverified data is shown, but never labelled as confirmed. */
  source: "ai" | "manual" | "qr";
  /** False when the subject was recognised but is not one of this parish's
   *  own stations — the camera still says what it is, it just isn't part of
   *  the tour. Undefined for manual picks and for open recognition. */
  matchedStation?: boolean;
  /** Which station this is, when it is one. Lets the card show that
   *  station's photograph and offer the next stop on the tour — the AI path
   *  has no station behind it, so it carries no id. */
  stationId?: string;

  /* Who the subject is, when it depicts a saint, a blessed, or Our Lady
   * under a title.
   *
   * `summary` answers "why do you think that" - it describes the object,
   * which is the honest way to let somebody check the identification
   * against what is in front of them. But a person standing before a
   * statue can already see the statue. What they cannot see is who it is
   * of, and that is the thing worth learning. These carry it.
   *
   * General Catholic knowledge, not parish record, and labelled as such
   * on the card: the same line this app draws everywhere else between
   * what a parish has verified and what is simply true of the Church.
   * feastDay and patronage are left empty by the model rather than
   * guessed - a wrong feast day sends somebody to church on the wrong
   * morning. */
  saintName?: string;
  feastDay?: string;
  patronage?: string;
  aboutSaint?: string;
}

/**
 * How long to wait for a recognition before giving up.
 *
 * Must stay ABOVE the server's own ceiling (OVERALL_DEADLINE_MS, 40s) or the
 * retrying is wasted: aborting first replaces the honest "the service is busy"
 * with a bare timeout, and the pilgrim loses the one piece of information that
 * tells them trying again will work.
 *
 * Long, and deliberately so. Measured: a congested primary model refuses in
 * ~3s and the fallback then answers in 7-18s, so the common recovery is
 * 10-20s. This is the outer bound for the bad case, not the expected wait.
 */
const SCAN_TIMEOUT_MS = 45000;

/**
 * This visitor's allowance at this parish today — see /api/identify.
 *
 * It used to be shared: the server counted against the caller's IP, so
 * everyone on a church's Wi-Fi drew from one pot of twenty. It is per
 * account and per parish now, which is why the token below travels with
 * every scan.
 */
interface ScanBudget {
  limit: number;
  used: number;
  remaining: number;
```

```
  day?: string;
  parishId?: string;
}

type Phase = "idle" | "scanning" | "done";

const STATUS_COPY: Partial<Record<CameraStatus, { title: string; icon: "warning" | "offline" }>> = {
  denied: { title: "Camera permission needed", icon: "warning" },
  insecure: { title: "Camera needs a secure connection", icon: "offline" },
  unavailable: { title: "Camera unavailable", icon: "warning" },
  "in-use": { title: "Camera is busy", icon: "warning" },
  error: { title: "Camera unavailable", icon: "warning" },
};

function stationToRecognition(
  station: Station,
  source: Recognition["source"] = "manual",
  /**
   * The reader's language. The station text compiled into data.ts is
   * English only, so a pilgrim reading in Tagalog scanned a statue and
   * got an English card back - which is the complaint this fixes.
   *
   * The history stays as written: it is the parish's own account, and
   * only the description and the reflection have been translated.
   */
  language: Language = "en",
): Recognition {
  const words = stationText(station, language);
  const highlights = [
    station.history && `History: ${station.history}`,
    words.reflection && `Reflection: ${words.reflection}`,
  ].filter((v): v is string => Boolean(v));

  return {
    recognized: true,
    title: station.name,
    category: "Station",
    summary: words.description,
    highlights,
    source,
    stationId: station.id,
  };
}


export default function ArTour({
  stations = [], parishId, parishName = "your parish", onClose, onCameraLiveChange,
}: {
  stations?: Station[];
  parishId?: string;
  /** Named in the AR handover, so the pilgrim knows which tour is opening. */
  parishName?: string;
  /**
   * Leaves the scanner. The reference design puts a close in the corner
   * of the viewfinder, and on a screen that has taken over the display
   * there has to be a visible way back that is not the system gesture.
   */
  onClose?: () => void;
  /**
```

```
   * Fires when the viewfinder opens or closes. The parish band lives above
   * this component, and once the camera is up the screen is the camera -
   * a header over a live viewfinder is covering the thing being scanned.
   */
  onCameraLiveChange?: (live: boolean) => void;
}) {
  // This parish's own pictures, so the choice looks like this church's
  // rather than the software's. The patron fronts the museum because the
  // museum is largely its statues; the church interior fronts the scanner
  // because that is what the camera will be pointed at.
  // The scanner's picture is the church, not the patron: the patron already
  // fronts the museum card directly above, and two photographs of the same
  // statue made the two choices look like one thing listed twice.
  const scannerImage = PARISH_CHURCH_PHOTOS[parishId ?? ""] ?? PARISH_HEADER_IMAGES[parishId ?? ""] ?? PLACEHOLDER_PHOTO;

  const camera = useCamera();

  // "ready" is the only status with a picture on screen; every other one
  // is a card explaining why there is not.
  const cameraLive = camera.status === "ready";

  /**
   * Open the camera as soon as the tab does.
   *
   * Tapping Scan used to land on a menu of cards with a "Scan now"
   * button on one of them, so reaching the scanner took two taps and
   * the first one showed something nobody came for. A scanner tab
   * should be a viewfinder; the AI/AR switch inside it already offers
   * the choice those cards were offering.
   *
   * Only from "idle". Re-running after a denial would reopen the
   * permission prompt on every render, and after a grant it would stop
   * and restart a camera that is already running.
   */
  useEffect(() => {
    if (camera.status === "idle") void camera.start();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [camera.status]);
  useEffect(() => {
    onCameraLiveChange?.(cameraLive);
    // Leaving the tab with the camera open must put the band back.
    return () => onCameraLiveChange?.(false);
  }, [cameraLive, onCameraLiveChange]);
  const { language } = useLanguage();
  const stationNames = stations.map((s) => s.name);
  const [phase, setPhase] = useState<Phase>("idle");
  const [result, setResult] = useState<Recognition | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [sheetOpen, setSheetOpen] = useState(false);

  /*
   * The result, read aloud.
   *
   * The same voice the history page uses - the phone's own
   * text-to-speech in the app, the browser's on a laptop, asking for
   * fil-PH or en-US to match the language on screen.
   *
   * This is the screen that needs it most. The others are read sitting
   * down; a scan result is read standing in front of a statue, often in
```

```
 * poor light, by somebody who would rather be looking at the statue
 * than at the phone.
 */
const narration = useNarration(language);

// The feature, recorded. Shown when it cannot be demonstrated
// live - during a Mass, or to a survey respondent who is not
// standing in the church.
// Which clip is playing, if any. Not a boolean: the no-camera
// screen offers both recordings side by side, and there the scanner
// has no current mode to borrow from.
const [previewFor, setPreviewFor] = useState<"ar" | "ai" | null>(null);
const [pickerOpen, setPickerOpen] = useState(false);
const [budget, setBudget] = useState<ScanBudget | null>(null);

// The allowance belongs to an account, so the counter has to follow
// signing in and out rather than being read once at mount.
const signedIn = useAuthUid() !== null;

/**
 * Which scanner is showing, and how far the AR one has got.
 *
 * They share the camera and the chrome and differ only in what they
 * do with a frame, so this is one screen with a switch rather than
 * two screens with a duplicate camera between them.
 */
const [mode, setMode] = useState<ScannerMode>("ai");

// Whether the AR tour may open here. Three facts decide it and the
// component already holds two of them; see lib/simulatorMode.ts for
// why the switch exists and which way round it reads.
const { simulatorMode } = useSimulatorMode();
const { presence } = usePresence();
const arGate = arAccess({
  simulatorMode,
  atParish: presence.mode === "present",
  android: arAvailability(navigator.userAgent, Capacitor.getPlatform()).kind === "ready",
});
const [arStage, setArStage] = useState<ArStage>("find");
const [arStop, setArStop] = useState(0);
const [arError, setArError] = useState<string | null>(null);

// Fetched so the remaining count is visible before anyone spends one.
// Failure is silent: the counter just doesn't appear, rather than
// blocking a scan over a number that is only informational.
//
// Re-run when the parish or the signed-in user changes, because the
// allowance belongs to that pair and not to the device.
useEffect(() => {
  let live = true;
  const parish = parishId ?? "route-mhcp";

  void (async () => {
    const token = await idToken();
    try {
      const r = await fetch(
        `${apiUrl("/api/identify/budget")}?parishId=${encodeURIComponent(parish)}`,
        { headers: withAppKey(token ? { Authorization: `Bearer ${token}` } : {}) },
      );
```

```
      const b = r.ok ? await r.json() : null;
      if (live && b) setBudget(b);
    } catch {
      /* informational only */
    }
  })();

  return () => {
    live = false;
  };
}, [parishId, signedIn]);

const scan = useCallback(async () => {
  const capture = camera.captureFrame();

  if (captureFailed(capture)) {
    // Each reason gets its own words. "It didn't work" is the same message
    // for a camera that has not painted a frame yet and for one that never
    // will, and those need different actions from the pilgrim.
    // Record<CaptureFailure, …>, not Record<typeof capture.reason, …>:
    // a `typeof` in a TYPE position reads the declared type, not the
    // one narrowed by the `if` above, so it saw the whole union and
    // the success branch has no `reason`.
    const message: Record<CaptureFailure, string> = {
      "no-video": "The camera view is not open. Close and reopen the scanner.",
      "not-ready": "The camera is still warming up. Give it a second and tap again.",
      "no-canvas": "This browser could not prepare the image. Try reloading the app.",
      "encode-failed": "The captured frame was empty. Try again, and make sure the camera is not covered.",
    };
    console.error(`[Scanner] Scanner error: capture failed — ${capture.reason}`);
    setError(message[capture.reason]);
    return;
  }

  const frame = capture.frame;

  // A frame this dark carries nothing for the model to work from, and
  // spending one of the day's twenty recognitions to be told so is worse
  // than saying it here. -1 means brightness could not be measured, which
  // is not a reason to block.
  if (frame.brightness >= 0 && frame.brightness < 18) {
    console.warn(`[Scanner] Scanner error: frame too dark (brightness ${frame.brightness.toFixed(0)}/255)`);
    setError("Too dark to read. Move closer, turn on more light, and keep the object inside the frame.");
    return;
  }

  setPhase("scanning");
  setError(null);
  console.log(
    `[Scanner] Sending image to AI: ${frame.width}x${frame.height}, ` +
      `${(frame.bytes / 1024).toFixed(0)} KB, ${stationNames.length} station candidates`,
  );

  const startedAt = Date.now();

  // A hard ceiling on the wait. The server now retries a congested model
  // three times before falling back to a second one, so a slow answer is
  // expected — but without an abort a genuinely hung request left the
  // scanner spinning on "scanning" with no error and no way back.
```

```
const controller = new AbortController();
const timeout = window.setTimeout(() => controller.abort(), SCAN_TIMEOUT_MS);

try {
  // The token is what makes the allowance this person's rather
  // than this Wi-Fi's — but whether one is REQUIRED is the
  // server's decision, not this component's.
  //
  // It used to refuse to send without one, which duplicated the
  // server's policy in the client and got it wrong: a development
  // server running uncounted does not want a token, and the
  // scanner was dead on localhost for want of a sign-in nobody
  // was going to check. The server answers 401 when it does want
  // one, and that answer is handled below.
  const token = await idToken();

  const response = await fetch(apiUrl("/api/identify"), {
    method: "POST",
    headers: withAppKey({
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    }),
    signal: controller.signal,
    body: JSON.stringify({
      imageBase64: frame.base64,
      mimeType: frame.mimeType,
      candidates: stationNames,
      parishId: parishId ?? "route-mhcp",
    }),
  });

  console.log(`[Scanner] API response: ${response.status} in ${Date.now() - startedAt}ms`);

  // A 413 used to arrive as the dev server's HTML error page, which threw
  // on .json() and surfaced as "Could not reach the server" — a size
  // limit reported as a network fault. Read the content type first.
  const contentType = response.headers.get("content-type") ?? "";
  if (!contentType.includes("application/json")) {
    console.error(
      `[Scanner] Scanner error: expected JSON, got "${contentType}" (status ${response.status})`,
    );
    setError(
      response.status === 413
        ? "That photo was too large to send. Try again from a little further back."
        : `The server replied unexpectedly (status ${response.status}). Please try again.`,
    );
    setPhase("idle");
    return;
  }

  const data = await response.json();
  if (data?.budget) setBudget(data.budget);

  if (!response.ok) {
    console.error(`[Scanner] Scanner error: ${response.status} — ${data?.error ?? "no message"}`);
    setError(data?.error ?? "Recognition failed. Please try again.");
    setPhase("idle");
    return;
  }
```

```
    if (!data.recognized) {
      console.log("[Scanner] AI result: nothing identifiable in frame");
      setError(
        data?.advice ??
          "Nothing recognisable in view. Move closer, improve the lighting, and keep the object inside the frame.",
      );
      setPhase("idle");
      return;
    }

    console.log(
      `[Scanner] AI result: "${data.title}" (${Math.round((data.confidence ?? 0) * 100)}% confidence` +
        `${data.matchedStation === false ? ", not a listed station" : ""})`,
    );
    setResult({ ...(data as Omit<Recognition, "source">), source: "ai" });
    // Deliberately NOT opening the full card. A scan used to
    // cover the whole screen with text the moment it returned,
    // which buries the thing the camera is pointed at. The name
    // lands in the slim bar below and a tap opens the rest.
    setSheetOpen(false);
    setPhase("done");
  } catch (err) {
    console.error("[Scanner] Scanner error:", err);
    // An abort is our own timeout firing, not a network fault, and saying
    // "check your connection" for it sends the pilgrim after the wrong
    // problem — their connection was fine, the answer just never came.
    const timedOut = err instanceof DOMException && err.name === "AbortError";
    setError(
      timedOut
        ? `The recognition service did not answer within ${Math.round(SCAN_TIMEOUT_MS / 1000)} seconds. Tap scan to try again.`
        : err instanceof TypeError
          ? "Could not reach the server. Check your connection and try again."
          : "Something went wrong reading the result. Please try again.",
    );
    setPhase("idle");
  } finally {
    window.clearTimeout(timeout);
  }
}, [camera, stationNames]);

// A voice still describing the last statue while the pilgrim has
// closed the card and walked on is the kind of bug people close an
// app over. Closing the card, or scanning something new, silences it.
const stopNarration = narration.stop;
useEffect(() => {
  if (!sheetOpen) stopNarration();
}, [sheetOpen, result?.title, stopNarration]);

const pickStation = useCallback((station: Station) => {
  setResult(stationToRecognition(station, "manual", language));
  setSheetOpen(true);
  setPhase("done");
  setPickerOpen(false);
}, []);

// The station currently on the card, when the card is showing one. The AI
// path recognises things that are not stations at all, so this is often
// null and every use of it has to allow for that.
```

```
const currentStation = result?.stationId
  ? stations.find(s => s.id === result.stationId) ?? null
  : null;

// The next stop on the tour, so a pilgrim can walk the church in order
// without going back to the picker between every station. Wraps at the end
// rather than dead-ending on the last one.
const nextStation =
  currentStation && stations.length > 1
    ? stations[(stations.findIndex(s => s.id === currentStation.id) + 1) % stations.length]
    : null;

/* ---------------------------------------------------------------------- QR */

/**
 * The parish's printed codes, read continuously while the camera is idle.
 *
 * No mode to choose and no button to press, deliberately: a pilgrim
 * standing in front of a poster should not first have to work out that the
 * app has a separate QR mode. It costs nothing to look — decoding runs on a
 * downscaled frame a few times a second — and every code found names
 * exactly one station, so unlike the vision model there is nothing to get
 * wrong.
 *
 * Paused while a result card is open or a recognition is in flight, so a
 * code still in frame cannot yank the screen out from under a reader.
 */
const onQrResult = useCallback(
  (scan: QrScanResult) => {
    if (scan.kind === "station") {
      const { station, route } = scan.match;
      console.log(`[Scanner] QR matched station "${station.name}" in ${route.id}`);
      setError(null);
      setResult(stationToRecognition(station, "qr", language));
      setSheetOpen(true);
      setPhase("done");
      return;
    }

    // Both misses get words that place the blame accurately. A pilgrim who
    // scanned the parish's own poster and is told "that is not a SanctiWalk
    // code" will reasonably conclude the app is broken.
    setError(
      scan.kind === "unknown-station"
        ? `Code ${scan.code} is a SanctiWalk code, but this parish's content has not been added yet.`
        : "That code is not a SanctiWalk station code. Point the camera at a station instead, or use the scan button.",
    );
  },
  [],
);

useQrScanner(camera.videoRef, {
  active: camera.isReady && !sheetOpen && phase !== "scanning",
  onResult: onQrResult,
});

/* ---------------------------------------------------------- permission */

if (camera.status !== "ready") {
```

```
    const statusCopy = STATUS_COPY[camera.status];
    const showManualFallback = camera.status !== "idle" && camera.status !== "requesting";

    // A manual pick was made while the camera was unavailable — show the
    // station's info card right here, same shape as a real scan would, with
    // a way back to either try the camera again or pick a different station.
    if (result && result.source === "manual" && sheetOpen) {
      return (
        <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto">
          <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden shrink-0 border-b border-[var(--
color-brand-border)]">
            <button
              onClick={() => setSheetOpen(false)}
              aria-label="Back"
              className="flex items-center gap-1 text-[15px] font-bold text-white/90 font-sans mb-2"
            >
              <ChevronLeft className="w-4 h-4" /> Back
            </button>
            <div className="flex items-center gap-2 mb-1">
              <span className="text-[14px] font-bold uppercase tracking-[0.09em] text-[var(--color-brand-on-accent)] font-sans">
                {result.category}
              </span>
            </div>
            <h2 className="text-2xl font-bold font-serif italic tracking-tight">{result.title}</h2>
          </div>

          <div className="p-4 space-y-4">
            {/* Unverified badge — mirrors scheduleVerified's amber "sample,
                unconfirmed" treatment in MassSchedule.tsx / Dashboard.tsx. A
                manual pick is not a scan, and must never look like one. */}
            <div className="flex items-center gap-2 p-2.5 bg-amber-50 border border-amber-300 rounded-xl">
              <AlertTriangle className="w-4 h-4 text-amber-700 shrink-0" />
              <span className="text-[15px] font-bold text-amber-900 leading-snug font-sans">
                Selected manually — not confirmed by a camera scan.
              </span>
            </div>

            <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-5 shadow-xs">
              {/* The station's own photograph, when one exists — nothing when
                  it does not. Most stations have none, and an empty space is
                  more honest than another church's altar. */}
              {currentStation?.imageUrl && (
                <img
                  src={currentStation.imageUrl}
                  alt={currentStation.name}
                  className="mb-4 w-full h-40 object-cover rounded-2xl border border-[var(--color-brand-border)]"
                  loading="lazy"
                />
              )}

              {/* Read it aloud. Above the text rather than buried under it:
                somebody who wants to listen rather than read should not
                have to read to the bottom to find out they could have
                listened. Absent when no voice is installed, and absent
                when there is too little to be worth hearing. */}
              {narration.supported && scanScript(result, language) && (
                <button
                  type="button"
                  onClick={() => narration.toggle(scanScript(result, language))}
```

```
    className="scan-listen"
    aria-pressed={narration.speaking}
    aria-label={
      narration.speaking
        ? "Stop reading this aloud"
        : `Read about ${result.title} aloud`
    }
  >
    {narration.speaking ? (
      <>
        <VolumeX className="w-4 h-4" aria-hidden />
        <span>{language === "fil" ? "Ihinto" : "Stop"}</span>
      </>
    ) : (
      <>
        <Volume2 className="w-4 h-4" aria-hidden />
        <span>{language === "fil" ? "Pakinggan" : "Listen"}</span>
      </>
    )}
  </button>
)}

<p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans">{result.summary}</p>

{/* Who they were. Kept visually distinct from the summary above
    it, because the two answer different questions and come from
    different kinds of knowing: the summary is what the camera
    saw, this is what the Church remembers. */}
{result.aboutSaint?.trim() && (
  <section className="saint-note">
    <h4 className="saint-note__head">
      About {result.saintName?.trim() || "this saint"}
    </h4>

    {(result.feastDay?.trim() || result.patronage?.trim()) && (
      <dl className="saint-note__facts">
        {result.feastDay?.trim() && (
          <div className="saint-note__fact">
            <dt>Feast day</dt>
            <dd>{result.feastDay}</dd>
          </div>
        )}
        {result.patronage?.trim() && (
          <div className="saint-note__fact">
            <dt>Patron of</dt>
            <dd>{result.patronage}</dd>
          </div>
        )}
      </dl>
    )}

    <p className="saint-note__body">{result.aboutSaint}</p>

    {/* Said plainly, for the same reason Ask Sancti says it: this
        is the Church's general record, not something this parish
        has checked and published. The link goes out rather than
        the text coming in - that site's writing is its own. */}
    <p className="saint-note__source">
      General Catholic information, not parish record.{" "}
```

```
                <a
                  href="https://mycatholic.life/saints/"
                  target="_blank"
                  rel="noreferrer noopener"
                  className="saint-note__link"
                >
                  Read more at My Catholic Life
                </a>
              </p>
            </section>
          )}

            {result.highlights.length > 0 && (
              <ul className="mt-4 space-y-2">
                {result.highlights.map((fact) => (
                  <li key={fact} className="flex gap-2.5 items-start">
                    <span className="w-1.5 h-1.5 rounded-full bg-[var(--color-brand-accent)] mt-2 shrink-0" />
                    <span className="text-[14px] text-[var(--color-brand-text)] leading-relaxed font-sans">
                      {fact}
                    </span>
                  </li>
                ))}
              </ul>
            )}
          </div>

          {/* Walks the church in order, naming the next stop rather than
              saying "Next" — knowing you are being sent to the Baptismal
              Font is what makes it worth following. */}
          {nextStation && (
            <button
              type="button"
              onClick={() => pickStation(nextStation)}
              className="w-full flex items-center justify-between gap-3 rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4
text-left active:scale-[0.99] transition-transform"
            >
              <span className="min-w-0">
                <span className="block text-[14px] font-bold uppercase tracking-[0.1em] text-[var(--color-brand-secondary)]">
                  Next location
                </span>
                <span className="mt-0.5 block text-[16px] font-semibold text-[var(--color-brand-text)] leading-snug">
                  {nextStation.name}
                </span>
              </span>
              <ChevronRight className="w-5 h-5 shrink-0 text-[var(--color-brand-primary)]" />
            </button>
          )}

          <button
            onClick={() => setPickerOpen(true)}
            className="w-full bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] text-[var(--color-brand-text)] rounded-2xl py-3 font-bold
text-[15px] font-sans active:scale-[0.98] transition-transform"
          >
            Choose a different station
          </button>
        </div>
      </div>
    );
  }
```

```
    // The manual station picker — the fallback that "always works" per the
    // parish's request, since the camera itself is a browser permission the
    // app cannot force.
    if (pickerOpen) {
      return (
        <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto">
          <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden shrink-0 border-b border-[var(--
color-brand-border)]">
            <button
              onClick={() => setPickerOpen(false)}
              aria-label="Back"
              className="flex items-center gap-1 text-[15px] font-bold text-white/90 font-sans mb-2"
            >
              <ChevronLeft className="w-4 h-4" /> Back
            </button>
            <div className="flex items-center gap-1.5 text-[var(--color-brand-on-accent)] font-bold text-[15px] tracking-wider uppercase font-serif italic">
              <ListChecks className="w-3.5 h-3.5" /> Choose your station
            </div>
            <h2 className="text-2xl font-bold font-serif italic tracking-tight">
              Where are you standing?
            </h2>
            <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-95 mt-1 max-w-xs leading-relaxed font-sans">
              Pick the station in front of you and we&rsquo;ll show what the scan would have.
            </p>
          </div>

          <div className="p-4 space-y-2.5">
            {stations.length === 0 ? (
              <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-5 shadow-xs text-center">
                <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans">
                  No stations are listed for this parish yet.
                </p>
              </div>
            ) : (
              stations.map((station) => (
                <button
                  key={station.id}
                  onClick={() => pickStation(station)}
                  className="w-full flex items-center justify-between gap-3 bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] p-4
shadow-xs text-left active:scale-[0.98] transition-transform"
                >
                  <span className="min-w-0">
                    <span className="block text-[16px] font-bold text-[var(--color-brand-text)] font-serif italic truncate">
                      {station.name}
                    </span>
                    <span className="block text-[14px] text-[var(--color-brand-text)]/70 font-sans truncate">
                      {stationText(station, language).description}
                    </span>
                  </span>
                  <ChevronRight className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
                </button>
              ))
            )}
          </div>
        </div>
      );
    }
```

```
return (
  /* No overflow here: App's .app-scroll is the page's one scroller, and a
     second one inside it would scroll the cards while leaving the parish
     band - which sits outside this component - stationary above them. */
  <div className="flex-1 flex flex-col bg-[var(--color-brand-card)]">
    {/* The flat navy panel with the faded clip-art scan icon used to sit
        here. The parish band above this screen says the same thing in
        the parish's own photography, so a second header was two titles
        for one tab. */}

    <div className="p-4 space-y-3">
      {/* Two ways in, offered as a choice. The museum walk and the
          scanner are different things - one is a guided tour of the
          parish's own pieces, the other identifies whatever is in
          front of you - and the old screen buried the first under a
          link beneath the second. */}

      {/* The "AR Museum" card stood here. It opened the same
          companion application the scanner's own AR tab opens, so
          the screen offered one thing twice - and the card could not
          say whether the tour would actually open, because it did
          not know where the pilgrim was. The AR tab does, and says
          so. */}
      <section className="scan-card">
        <span className="scan-card__media">
          <img src={scannerImage} alt="" loading="lazy" />
          <span className="scan-card__scrim" aria-hidden />
          <span className="scan-card__title">Scanner</span>
        </span>
        <div className="scan-card__body">
          <p className="scan-card__blurb">
            {camera.status === "requesting"
              ? "Opening the camera…"
              : statusCopy?.title
                ? statusCopy.title
                : "Point your camera at an altar, statue or marker and the app will tell you what it is."}
          </p>

          {/* The privacy line stays with the button that asks for the
              camera - it is the answer to the permission prompt the
              pilgrim is about to see. */}
          <p className={`scan-card__note${camera.error ? " scan-card__note--error" : ""}`}>
            {camera.error ?? "Nothing is recorded — frames are analysed and discarded."}
          </p>

          {camera.status !== "insecure" && (
            <button
              type="button"
              onClick={() => void camera.start()}
              disabled={camera.status === "requesting"}
              className="scan-card__action"
            >
              {camera.status === "requesting" ? (
                <>
                  <Loader2 className="w-4 h-4 animate-spin" /> Opening…
                </>
              ) : statusCopy?.icon === "offline" ? (
                <>
                  <WifiOff className="w-4 h-4" /> Try again
```

```
                  </>
                ) : camera.error ? (
                  <>
                    <AlertTriangle className="w-4 h-4" /> Try again
                  </>
                ) : (
                  <>
                    <Camera className="w-4 h-4" /> Scan now
                  </>
                )}
              </button>
            )}
          </div>
        </section>

        {result && !sheetOpen && (
          <button
            onClick={() => setSheetOpen(true)}
            className="w-full flex items-center justify-between gap-2 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] text-[var(--color-
brand-text)] rounded-2xl py-3 px-4 font-sans active:scale-[0.98] transition-transform"
          >
            <span className="min-w-0 text-left">
              <span className="block text-[12.5px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
                Tap to read
              </span>
              <span className="block text-[15px] font-bold truncate">{result.title}</span>
            </span>
            <ChevronRight className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          </button>
        )}

        {/* The fallback that "always works": the camera is a browser
            permission the app cannot force open on an insecure origin, so
            offer the same station information another way rather than
            leaving a dead end. */}
        {/* Without a camera the scanner cannot be shown by doing it,
            and the AI/AR switch lives inside the camera view - so from
            here both the AR museum and the AI scanner were simply
            missing, which reads as the AR scanner being broken. The
            recordings are the one thing that still works on a laptop,
            at a survey table, or behind a denied permission, so they
            are offered here rather than only past the gate. */}
        <section className="scan-card scan-card--previews">
          <div className="scan-card__body">
            <p className="scan-card__blurb">See how it works</p>
            <p className="scan-card__note">
              Short recordings of both scanners. No camera needed.
            </p>
            <button
              type="button"
              onClick={() => setPreviewFor("ai")}
              className="scan-card__action"
            >
              <PlayCircle className="w-4 h-4" /> Preview AI Scanner
            </button>
            <button
              type="button"
              onClick={() => setPreviewFor("ar")}
              className="scan-card__action scan-card__action--ghost"
```

```
              >
                <PlayCircle className="w-4 h-4" /> Preview AR Museum
              </button>
            </div>
          </section>

          {showManualFallback && (
            <button
              onClick={() => setPickerOpen(true)}
              className="w-full flex items-center justify-center gap-2 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] text-[var(--color-
brand-text)] rounded-2xl py-3 font-bold text-[15px] font-sans active:scale-[0.98] transition-transform"
            >
              <ListChecks className="w-4 h-4" /> Choose your station manually instead
            </button>
          )}
        </div>

        {previewFor && (
          <PreviewVideo
            src={previewFor === "ar" ? "/preview/ar.mp4" : "/preview/scanner.mp4"}
            title={previewFor === "ar" ? "AR Museum tour" : "AI Scanner"}
            onClose={() => setPreviewFor(null)}
          />
        )}
      </div>
    );
  }

  /* ------------------------------------------------------------ camera view */

  const busy = phase === "scanning";

  /* ---- what the overlay says, per mode and stage ---- */

  const arStation = stations[arStop] ?? null;

  const headline =
    mode === "ai"
      ? busy
        ? { title: "Reading what you see", sub: "Hold steady" }
        // One line at phone width. The longer version wrapped and shoved
        // the subtitle down over the viewfinder.
        : { title: "Point at a statue or marker", sub: "The AI scanner names it and tells you about it" }
      : arStage === "find"
        ? { title: "AR Museum", sub: "Opens the SanctiWalk AR app, which reads the printed marker for you" }
        : arStage === "reading"
          ? { title: "Hold the marker in the frame", sub: "Keep steady while it is read" }
          : { title: arStation?.name ?? "AR tour running", sub: "Follow the markers around the church" };

  function startAr() {
    setArError(null);
    setArStage("reading");

    // The marker itself is read by the Unity app, not by this WebView.
    // The pause is the handover, not a fake detection: it gives the
    // pilgrim a moment to see that something happened before another
    // app takes the screen.
    window.setTimeout(() => {
      const left = () => document.visibilityState === "hidden";
```

```
    // Which parish, not just which URL: away from a church with the
    // simulator off this hands over the sala instead, so Start AR
    // opens a room the phone can actually localize in.
    window.location.href = arTourUrl(arLaunchParish({
      parishId: parishId ?? "route-mhcp",
      simulatorMode,
      atParish: presence.mode === "present",
    }));

    /*
     * Nothing happens when the AR app is not installed.
     *
     * A custom scheme fails silently: no error, no navigation, the
     * page simply stays put. Without this check the pilgrim would be
     * left looking at "Reading the marker" for ever, with the app
     * apparently frozen. If we are still here and still visible a
     * moment later, the handover did not take.
     */
    window.setTimeout(() => {
      if (left()) {
        setArStage("live");
      } else {
        setArStage("find");
        setArError(
          "The SanctiWalk AR app did not open. It is a separate Android app — install it, then try again.",
        );
      }
    }, 1200);
  }, 1200);
}

return (
  <div className="scan flex-1 relative bg-black overflow-hidden">
    <video
      ref={camera.videoRef}
      className={`absolute inset-0 w-full h-full object-cover ${
        camera.facingMode === "user" ? "scale-x-[-1]" : ""
      }`}
      playsInline
      muted
      autoPlay
    />

    {/* Keeps the controls legible over a bright window or a dark nave. */}
    <div
      className="absolute inset-0 pointer-events-none"
      style={{
        background:
          "linear-gradient(to bottom, rgba(0,0,0,0.45), transparent 22%, transparent 55%, rgba(0,0,0,0.6))",
      }}
    />

    {/* The instruction, over the feed. */}
    {!sheetOpen && (
      <ScannerHeader
        title={headline.title}
        subtitle={headline.sub}
        onClose={onClose}
```

```
        />
      )}

      {/* The target. In AI mode it frames whatever is being pointed at;
          in AR it frames the marker while it is read. */}
      {!sheetOpen && mode !== "ar" && <ScannerReticle tight={busy} />}

      {/* Status pill */}
      {/* Status pill */}
      {(busy || error) && (
        <div className="absolute top-4 left-1/2 -translate-x-1/2 max-w-[85%] px-4 py-2 rounded-full bg-black/65 backdrop-blur-md border border-white/15 flex items-
center gap-2">
          {busy && <Loader2 className="w-3.5 h-3.5 text-[var(--color-brand-on-accent)] animate-spin shrink-0" />}
          <span
            className={`text-[15px] font-sans font-medium ${error ? "text-[var(--color-brand-error-soft)]" : "text-white"}`}
          >
            {busy ? "Looking…" : error}
          </span>
        </div>
      )}


      {/* What the scan found, before it takes over the screen.
          A result is deliberately shown collapsed first - the name only,
          over the live camera - because a full card the instant the answer
          lands buries the thing the camera is pointed at. That was the
          intent; what was missing is this bar. The collapsed state existed
          only on the no-camera fallback screen, so in the camera view a
          successful scan set a result that nothing rendered: the scan was
          spent, the answer arrived, and the screen showed nothing. */}
      {/* The sheet: the switch, and whatever the chosen scanner needs. */}
      {!sheetOpen && (
        <ScannerSheet>
          {/* What the scan found, before it takes over the screen.
              A result is deliberately shown collapsed first - the name
              only - because a full card the instant the answer lands
              buries the thing the camera is pointed at. That was always
              the intent; the bar itself was missing. It existed only on
              the no-camera fallback screen, so in the camera view a
              successful scan set a result that nothing rendered: the
              scan was spent, the answer came back, and the screen showed
              nothing at all.

              Inside the sheet rather than floating above it, so it needs
              no hand-measured offset to clear the tabs. */}
          {result && (
            <button
              type="button"
              onClick={() => setSheetOpen(true)}
              className="scan-result-peek"
              aria-label={`Read about ${result.title}`}
            >
              <span className="scan-result-peek__text">
                <span className="scan-result-peek__label">Tap to read</span>
                <span className="scan-result-peek__title">{result.title}</span>
              </span>
              <ChevronRight className="w-4 h-4 shrink-0" aria-hidden />
            </button>
          )}
```

```
<button
  type="button"
  onClick={() => setPreviewFor(mode)}
  className="scan-preview-btn"
  aria-label={`Watch how the ${mode === "ar" ? "AR tour" : "AI Scanner"} works`}
>
  Preview
</button>
<ScannerTabs
  mode={mode}
  onChange={next => {
    setMode(next);
    setError(null);
    // Coming back to AR should ask again where you are, not
    // resume a tour you walked away from.
    if (next === "ar") setArStage("find");
  }}
/>

{mode === "ai" ? (
  <>
    <button
      type="button"
      onClick={() => void scan()}
      disabled={busy}
      aria-label="Identify what the camera is pointed at"
      className="scan-shutter"
    >
      <span className={`scan-shutter__core${busy ? " is-busy" : ""}`} />
    </button>

    {/* The allowance is this person's at this parish, so the
        caption says whose and where rather than implying one
        shared pot. Signed out there is no allowance to report,
        and saying so here is kinder than letting them tap and
        be refused. */}
    <p className="scan-caption">
      {!signedIn
        ? `Sign in to use the AI scanner — ${SCANS_PER_PARISH_PER_DAY} scans a day at ${parishName}.`
        : budget && budget.remaining === 0
          ? `No scans left for ${parishName} today. The count resets at midnight UTC.`
          : budget
            ? `Tap to identify. ${budget.remaining} of ${budget.limit} scans left here today.`
            : "Tap to identify. The parish code is read automatically."}
    </p>
  </>
) : !arGate.allowed ? (
  /* The AR tour cannot open here, and the panel below would
     offer a button that does nothing. The notice says which
     of the two reasons it is, and both are things the pilgrim
     can act on: change phone, or come to the parish. */
  <div className="scan-gate" role="status">
    <span className="scan-gate__icon" aria-hidden>
      <MapPin className="w-5 h-5" />
    </span>
    <p className="scan-gate__title">
      {arGate.reason === "device"
        ? "Not available on this phone"
```

```
                : "Open this at the parish"}
            </p>
            <p className="scan-gate__body">{arGate.message}</p>

            {/* A dead end is what this was: the notice said the phone
                could not run the tour and stopped there. Both of these
                are real and already in the build - the recording the
                AR preview button plays is the same file the Preview
                control uses, and the download page is the one that
                carries the install steps. Shown only for the device
                reason: somebody standing in the wrong place needs to
                move, not to download anything. */}
            {arGate.reason === "device" && (
              <div className="scan-gate__actions">
                <button
                  type="button"
                  onClick={() => setPreviewFor("ar")}
                  className="scan-gate__btn"
                >
                  <PlayCircle className="w-4 h-4" aria-hidden />
                  Watch AR preview
                </button>
                <a href="/get/" className="scan-gate__btn scan-gate__btn--quiet">
                  <Download className="w-4 h-4" aria-hidden />
                  Get the Android app
                </a>
              </div>
            )}
          </div>
        ) : (
          <ArScannerPanel
            stage={arStage}
            error={arError}
            onFoundMarker={startAr}
            parishName={parishName}
          />
        )}
      </ScannerSheet>
    )}

    {previewFor && (
      <PreviewVideo
        src={previewFor === "ar" ? "/preview/ar.mp4" : "/preview/scanner.mp4"}
        title={previewFor === "ar" ? "AR Museum tour" : "AI Scanner"}
        onClose={() => setPreviewFor(null)}
      />
    )}

    {/* Information card */}
    {result && (
      <div
        className={`absolute inset-x-0 bottom-0 bg-[var(--color-brand-card)] rounded-t-[2rem] border-t border-[var(--color-brand-border)] shadow-2xl transition-
transform duration-500 ease-out ${
          sheetOpen ? "translate-y-0" : "translate-y-full"
        }`}
        style={{ maxHeight: "72%" }}
      >
        <div className="flex items-start gap-3 p-5 pb-3">
          <div className="flex-1 min-w-0">
```

```
              <div className="flex items-center gap-2 mb-1 flex-wrap">
                <span className="text-[14px] font-bold uppercase tracking-[0.09em] text-[var(--color-brand-accent)] font-sans">
                  {result.category}
                </span>
                {result.confidence != null && (
                  <span className="text-[14px] text-[var(--color-brand-secondary)] font-sans">
                    {Math.round(result.confidence * 100)}% match
                  </span>
                )}
              </div>
              <h3 className="text-xl font-bold text-[var(--color-brand-text)] font-serif italic leading-tight">
                {result.title}
              </h3>
              {/* Recognised, but not one of this parish's stations. Saying so
                  keeps the tour's own list meaningful while still answering
                  the question the pilgrim actually asked. */}
              {result.matchedStation === false && (
                <p className="mt-1.5 text-[15px] leading-snug text-[var(--color-brand-secondary)]">
                  Not one of this parish's tour stations — identified from the camera.
                </p>
              )}
            </div>
            <button
              onClick={() => setSheetOpen(false)}
              aria-label="Close"
              className="h-9 w-9 rounded-xl bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] flex items-center justify-center text-[var(--
color-brand-text)] shrink-0 active:scale-95 transition-transform"
            >
              <X className="w-4 h-4" />
            </button>
          </div>

          <div className="px-5 pb-6 overflow-y-auto" style={{ maxHeight: "calc(72vh - 90px)" }}>
            {/* The station's own photograph, when one exists. Deliberately
                nothing when it does not: four of these carried stock images
                of unrelated churches until this pass, and an empty space is
                more honest than another parish's altar. */}
            {currentStation?.imageUrl && (
              <img
                src={currentStation.imageUrl}
                alt={currentStation.name}
                className="mb-4 w-full h-40 object-cover rounded-2xl border border-[var(--color-brand-border)]"
                loading="lazy"
              />
            )}

            {/* Read it aloud. Above the text rather than buried under it:
                somebody who wants to listen rather than read should not
                have to read to the bottom to find out they could have
                listened. Absent when no voice is installed, and absent
                when there is too little to be worth hearing. */}
            {narration.supported && scanScript(result, language) && (
              <button
                type="button"
                onClick={() => narration.toggle(scanScript(result, language))}
                className="scan-listen"
                aria-pressed={narration.speaking}
                aria-label={
                  narration.speaking
```

```
        ? "Stop reading this aloud"
        : `Read about ${result.title} aloud`
    }
  >
    {narration.speaking ? (
      <>
        <VolumeX className="w-4 h-4" aria-hidden />
        <span>{language === "fil" ? "Ihinto" : "Stop"}</span>
      </>
    ) : (
      <>
        <Volume2 className="w-4 h-4" aria-hidden />
        <span>{language === "fil" ? "Pakinggan" : "Listen"}</span>
      </>
    )}
  </button>
)}

<p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans">{result.summary}</p>

{/* Who they were. Kept visually distinct from the summary above
    it, because the two answer different questions and come from
    different kinds of knowing: the summary is what the camera
    saw, this is what the Church remembers. */}
{result.aboutSaint?.trim() && (
  <section className="saint-note">
    <h4 className="saint-note__head">
      About {result.saintName?.trim() || "this saint"}
    </h4>

    {(result.feastDay?.trim() || result.patronage?.trim()) && (
      <dl className="saint-note__facts">
        {result.feastDay?.trim() && (
          <div className="saint-note__fact">
            <dt>Feast day</dt>
            <dd>{result.feastDay}</dd>
          </div>
        )}
        {result.patronage?.trim() && (
          <div className="saint-note__fact">
            <dt>Patron of</dt>
            <dd>{result.patronage}</dd>
          </div>
        )}
      </dl>
    )}

    <p className="saint-note__body">{result.aboutSaint}</p>

    {/* Said plainly, for the same reason Ask Sancti says it: this
        is the Church's general record, not something this parish
        has checked and published. The link goes out rather than
        the text coming in - that site's writing is its own. */}
    <p className="saint-note__source">
      General Catholic information, not parish record.{" "}
      <a
        href="https://mycatholic.life/saints/"
        target="_blank"
        rel="noreferrer noopener"
```

```
                  className="saint-note__link"
                >
                  Read more at My Catholic Life
                </a>
              </p>
            </section>
          )}

          {result.highlights?.length > 0 && (
            <ul className="mt-4 space-y-2">
              {result.highlights.map((fact) => (
                <li key={fact} className="flex gap-2.5 items-start">
                  <span className="w-1.5 h-1.5 rounded-full bg-[var(--color-brand-accent)] mt-2 shrink-0" />
                  <span className="text-[14px] text-[var(--color-brand-text)] leading-relaxed font-sans">
                    {fact}
                  </span>
                </li>
              ))}
            </ul>
          )}

          {/* Three ways to arrive at this card, three different degrees of
              certainty, said plainly. A pilgrim should know which text is
              parish-verified and which is not — the same principle as
              coordinatesVerified and scheduleVerified in data.ts. */}
          {result.source === "manual" ? (
            <div className="mt-5 flex items-center gap-2 p-2.5 bg-amber-50 border border-amber-300 rounded-xl">
              <AlertTriangle className="w-4 h-4 text-amber-700 shrink-0" />
              <span className="text-sm font-bold text-amber-900 leading-snug font-sans">
                Selected manually — not confirmed by a camera scan.
              </span>
            </div>
          ) : result.source === "qr" ? (
            // The most certain of the three, and it should read that way.
            // The parish printed this code beside this station; nothing was
            // inferred and there is nothing here for a model to get wrong.
            <div className="mt-5 flex items-center gap-2 p-2.5 bg-emerald-50 border border-emerald-300 rounded-xl">
              <QrCode className="w-4 h-4 text-emerald-800 shrink-0" />
              <span className="text-sm font-bold text-emerald-900 leading-snug font-sans">
                Confirmed by the parish's own code — this is the station in front of you.
              </span>
            </div>
          ) : (
            <p className="mt-5 text-[15px] text-[var(--color-brand-secondary)] leading-relaxed font-sans bg-[var(--color-brand-card)]/70 border border-[var(--color-brand-border)] rounded-xl p-3">
              Identified by AI from your camera. Details may be incomplete — the parish record
              is the authority.
            </p>
          )}

          {/* Walks the church in order, so a pilgrim does not have to go
              back to the picker between every station. It names the next
              stop rather than saying "Next" — knowing you are being sent to
              the Baptismal Font is what makes it worth following. */}
          {nextStation && (
            <button
              type="button"
              onClick={() => pickStation(nextStation)}
```

```
              className="mt-5 w-full flex items-center justify-between gap-3 rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card-
sunk)] p-4 text-left active:scale-[0.99] transition-transform"
            >
              <span className="min-w-0">
                <span className="block text-[14px] font-bold uppercase tracking-[0.1em] text-[var(--color-brand-secondary)]">
                  Next location
                </span>
                <span className="mt-0.5 block text-[16px] font-semibold text-[var(--color-brand-text)] leading-snug">
                  {nextStation.name}
                </span>
              </span>
              <ChevronRight className="w-5 h-5 shrink-0 text-[var(--color-brand-primary)]" />
            </button>
          )}
        </div>
      </div>
    )}
  </div>
 );
}
```

1.2.6 APP-011 — AvailabilityBadge.tsx

Code ID	APP-011
File Name	AvailabilityBadge.tsx
File Path	SanctDemo/src/components/AvailabilityBadge.tsx
Language / Type	TypeScript (React, TSX)
Lines	38
Functions / Classes	AvailabilityBadge (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/AvailabilityBadge.tsx. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> "Accepting applications" / "Not available", as a chip.
>
> ## Why the closed state is grey and not red
>
> A parish closing the choir to new members for Lent is housekeeping, not an error. Red is what the app uses for a rejected application and for a form that would not send, and borrowing it here would tell a pilgrim that something had gone wrong with THEM.
>
> ## Why the dot is not the whole signal
>
> A coloured dot alone is invisible to a screen reader and ambiguous to the roughly one man in twelve who cannot separate the two colours. The words carry the meaning; the dot only makes it quicker to scan.

Complete Source Code — AvailabilityBadge.tsx

```
/**
 * "Accepting applications" / "Not available", as a chip.
 *
 * ## Why the closed state is grey and not red
 *
 * A parish closing the choir to new members for Lent is housekeeping, not
 * an error. Red is what the app uses for a rejected application and for a
 * form that would not send, and borrowing it here would tell a pilgrim
 * that something had gone wrong with THEM.
 *
 * ## Why the dot is not the whole signal
 *
 * A coloured dot alone is invisible to a screen reader and ambiguous to
 * the roughly one man in twelve who cannot separate the two colours. The
 * words carry the meaning; the dot only makes it quicker to scan.
 */
export default function AvailabilityBadge({
```

```
  open,
  kind,
  className = "",
}: {
  open: boolean;
  kind: "ministry" | "sacrament";
  className?: string;
}) {
  const label = open
    ? kind === "ministry" ? "Accepting applications" : "Applications open"
    : "Not available";

  return (
    <span
      className={`availability-badge${open ? "" : " availability-badge--closed"} ${className}`}
    >
      <span className="availability-badge__dot" aria-hidden />
      {label}
    </span>
  );
}
```

1.2.7 APP-012 — BulletinRail.tsx

Code ID	APP-012
File Name	BulletinRail.tsx
File Path	SanctDemo/src/components/BulletinRail.tsx
Language / Type	TypeScript (React, TSX)
Lines	232
Functions / Classes	Announcement (exported type); BulletinRail (exported component/function (default)); AnnouncementDeck (internal function); SectionCard (internal function)
Related Files (its imports)	../lib/announcements
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/BulletinRail.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — BulletinRail.tsx

```
import { useEffect, useRef, useState, type ReactNode } from "react";
import { CalendarDays, ChevronLeft, ChevronRight, Users, Sparkles } from "lucide-react";
import { formatWhen, upcomingAnnouncements, type AnnouncementDoc } from "../lib/announcements";

/**
 * The parish's own photographs, from the design the parish supplied: the
 * altar servers in the sanctuary, and a christening.
 */
const MINISTRY_PHOTO = "/parish/ministry-altar-servers.jpg";
const SACRAMENT_PHOTO = "/parish/sacraments-christening.jpg";

export type Announcement = AnnouncementDoc;

interface BulletinRailProps {
  announcements: AnnouncementDoc[];
  onNavigate: (tab: "ministries" | "sacraments" | "mass") => void;
}

/**
 * The Parish Bulletin, below the Verse of the Day.
 *
 * ## Why nothing here slides any more
 *
 * It was one horizontal rail carrying the verse, the announcements and the
 * two section cards together, and that was wrong in two different ways at
 * once. The announcements are a stack of the same kind of thing, so moving
 * between them should be a change of content, not a change of place - they
 * CROSSFADE. The two section cards are a fixed pair that fit side by side
 * on a phone, so swiping to reach the second one hid a card that was never
 * off screen to begin with - they are simply BOTH THERE.
 *
```

```
 * A rail is for a list whose length you do not know. Neither of these is
 * that, and treating them as one made the only genuinely variable thing on
 * the screen - the verse - swipeable too.
 */
export default function BulletinRail({ announcements, onNavigate }: BulletinRailProps) {
  const now = new Date();
  const upcoming = upcomingAnnouncements(announcements, now);

  return (
    <section className="bulletin">
      <AnnouncementDeck items={upcoming} now={now} />

      {/* Ministries and Sacraments used to be two photo cards here.
          They are buttons at the top of Home now, directly under the
          parish's name, where the four most-asked questions belong -
          and keeping them here as well would have put each of them on
          the screen twice. See HomeQuickLinks. */}
    </section>
  );
}

/**
 * The announcements, one at a time, crossfading.
 *
 * Every slide is rendered into the SAME grid cell rather than laid out in
 * a row, which is what makes the fade possible and also what keeps the
 * deck's height fixed: the cell is as tall as the tallest announcement, so
 * moving between a short notice and a long one does not make the page
 * below it jump.
 *
 * The inactive slides are inert, not merely transparent - `visibility`
 * and `aria-hidden` together, or a screen reader reads all four at once
 * and a tab lands on a card nobody can see.
 */
function AnnouncementDeck({
  items,
  now,
}: {
  items: Array<AnnouncementDoc & { when: Date | null }>;
  now: Date;
}) {
  const [index, setIndex] = useState(0);
  const touchX = useRef<number | null>(null);

  // A parish retiring the announcement you were looking at must not leave
  // the deck pointing past the end of the list.
  useEffect(() => {
    setIndex(i => (i > items.length - 1 ? Math.max(0, items.length - 1) : i));
  }, [items.length]);

  if (items.length === 0) {
    return (
      <p className="bulletin-note">
        Nothing from the parish office this week. Announcements, feast days and
        schedule changes appear here first.
      </p>
    );
  }
```

```
const count = items.length;
const go = (next: number) => setIndex(((next % count) + count) % count);

return (
  <div className="bulletin-deck-frame">
    <div
      className="bulletin-deck"
      onTouchStart={e => { touchX.current = e.touches[0]?.clientX ?? null; }}
      onTouchEnd={e => {
        const from = touchX.current;
        touchX.current = null;
        if (from === null || count < 2) return;
        const dx = (e.changedTouches[0]?.clientX ?? from) - from;
        // Generous enough that a slightly diagonal vertical scroll does
        // not count as a swipe.
        if (Math.abs(dx) > 45) go(index + (dx < 0 ? 1 : -1));
      }}
    >
      {items.map((item, i) => (
        <article
          key={item.id}
          className={
            "bulletin-card"
            + (item.imageUrl ? "" : " bulletin-card--text")
            + (i === index ? " is-current" : "")
          }
          aria-hidden={i === index ? undefined : true}
          // The deck is one region that changes; without this a reader
          // announces the whole stack on every move.
          aria-roledescription="announcement"
        >
          {item.imageUrl && (
            <span className="bulletin-card__photo">
              <img src={item.imageUrl} alt="" loading="lazy" />
            </span>
          )}
          <span className="bulletin-card__body">
            <span className="bulletin-card__chip">
              <CalendarDays className="w-3.5 h-3.5" aria-hidden />
              {item.type}
            </span>
            <h3 className="bulletin-card__title">{item.title}</h3>
            <p className="bulletin-card__when">
              {item.when ? formatWhen(item.when, now) : "From the parish office"}
              {item.time ? ` · ${item.time}` : ""}
            </p>
            {item.body && <p className="bulletin-card__body-text">{item.body}</p>}
          </span>
        </article>
      ))}
    </div>

    {count > 1 && (
      <div className="bulletin-deck__nav">
        <button
          type="button"
          onClick={() => go(index - 1)}
          aria-label="Previous announcement"
          className="bulletin-deck__arrow"
```

```
          >
            <ChevronLeft className="w-4 h-4" />
          </button>

          <span className="bulletin-deck__dots">
            {items.map((item, i) => (
              <button
                key={item.id}
                type="button"
                onClick={() => go(i)}
                aria-label={`Announcement ${i + 1} of ${count}`}
                aria-current={i === index ? "true" : undefined}
                className={`bulletin-deck__dot${i === index ? " is-current" : ""}`}
              />
            ))}
          </span>

          <button
            type="button"
            onClick={() => go(index + 1)}
            aria-label="Next announcement"
            className="bulletin-deck__arrow"
          >
            <ChevronRight className="w-4 h-4" />
          </button>
        </div>
      )}
    </div>
  );
}

/**
 * One of the two section cards. The description lives INSIDE the card
 * rather than in a panel underneath: on a phone that panel sits below the
 * fold, so the thing the tap revealed is the one thing you cannot see.
 */
function SectionCard({
  icon, label, hint, caption, imageUrl, onClick, spotlight,
}: {
  /** Name the tutorial and Sancti can point at. */
  spotlight?: string;
  icon: ReactNode;
  label: string;
  hint: string;
  /** Named over the photograph — an example of what is inside. */
  caption?: string;
  imageUrl?: string;
  onClick: () => void;
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      data-spotlight={spotlight}
      className="bulletin-section"
    >
      {imageUrl && (
        <span className="bulletin-section__photo">
          <img src={imageUrl} alt="" loading="lazy" />
```

```
          {caption && (
            <>
              {/* A scrim, not a shadow: what sits behind the caption is a
                  photograph whose brightness cannot be assumed. */}
              <span className="bulletin-section__scrim" aria-hidden />
              <span className="bulletin-section__caption">{caption}</span>
            </>
          )}
        </span>
      )}
      <span className="bulletin-section__body">
        <span className="bulletin-section__label">
          <span className="bulletin-section__icon">{icon}</span>
          {/* No chevron. At half a phone's width "Sacraments" plus its
              icon already fills the row, and a chevron pushed to the end
              was simply clipped - the card is a button, which is what
              says it opens. */}
          {label}
        </span>
        <span className="bulletin-section__hint">{hint}</span>
      </span>
    </button>
  );
}
```

1.2.8 APP-013 — ChangeParishModal.tsx

Code ID	APP-013
File Name	ChangeParishModal.tsx
File Path	SanctDemo/src/components/ChangeParishModal.tsx
Language / Type	TypeScript (React, TSX)
Lines	80
Functions / Classes	ChangeParishModal (exported component/function (default))
Related Files (its imports)	../types
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ChangeParishModal.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChangeParishModal.tsx

```
import { X, Church, MapPin, Check } from "lucide-react";
import { Route } from "../types";

interface ChangeParishModalProps {
  isOpen: boolean;
  onClose: () => void;
  parishes: Route[];
  currentParishId: string | null;
  onChoose: (parishId: string) => void;
}

// The "change it later" surface promised by the one-time HomeParishChooser.
// Reachable from the sidebar drawer (see App.tsx's "Change Home Parish"
// entry). Picking a parish here updates the stored home parish immediately
// and closes the sheet; it does not require re-running onboarding.
export default function ChangeParishModal({ isOpen, onClose, parishes, currentParishId, onChoose }: ChangeParishModalProps) {
  if (!isOpen) return null;

  return (
    <div
      className="absolute inset-0 z-[70] flex items-center justify-center p-5 bg-black/60 transition-opacity duration-300 ease-out"
      onClick={onClose}
    >
      <div
        className="w-full max-w-sm rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] text-[var(--color-brand-text)] shadow-2xl
transition-all duration-300 ease-out font-sans"
        onClick={(e) => e.stopPropagation()}
      >
        <div className="flex items-center justify-between px-5 pt-5 pb-3 border-b border-[var(--color-brand-border)]">
          <div className="flex items-center gap-1.5">
            <Church className="w-4 h-4 text-[var(--color-brand-primary)]" />
            <h2 className="text-base font-bold tracking-tight">Home Parish</h2>
          </div>
```

```
          <button
            onClick={onClose}
            aria-label="Close change parish"
            className="flex items-center justify-center w-11 h-11 -mr-2 rounded-full text-[var(--color-brand-secondary)] hover:bg-[var(--color-brand-card)]
transition-colors duration-300"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        <div className="px-5 py-4 space-y-2.5">
          <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed pb-1">
            We'll show this parish whenever you're not near a parish in person.
          </p>
          {parishes.map((route) => {
            const isCurrent = route.id === currentParishId;
            return (
              <button
                key={route.id}
                onClick={() => {
                  onChoose(route.id);
                  onClose();
                }}
                className={`w-full rounded-2xl border p-3.5 flex items-center gap-3 text-left transition-all ${
                  isCurrent
                    ? "bg-[var(--color-brand-card)] border-[var(--color-brand-primary)]"
                    : "bg-[var(--color-brand-card)] border-[var(--color-brand-border)] hover:border-[var(--color-brand-primary)]"
                }`}
              >
                <div className="h-10 w-10 rounded-xl bg-[var(--color-brand-card)] text-[var(--color-brand-primary)] flex items-center justify-center shrink-0">
                  <Church className="w-4.5 h-4.5" />
                </div>
                <div className="flex-1 min-w-0">
                  <h3 className="text-base font-bold text-[var(--color-brand-text)] truncate">
                    {route.name.replace(" Guide", "").replace(" Tour", "")}
                  </h3>
                  <p className="text-sm text-[var(--color-brand-secondary)] mt-0.5 flex items-center gap-1">
                    <MapPin className="w-3.5 h-3.5 shrink-0" /> {route.location}
                  </p>
                </div>
                {isCurrent && <Check className="w-4 h-4 text-[var(--color-brand-primary)] shrink-0" />}
              </button>
            );
          })}
        </div>
      </div>
    </div>
  );
}
```

1.2.9 APP-014 — ChurchHistory.tsx

Code ID	APP-014
File Name	ChurchHistory.tsx
File Path	SanctDemo/src/components/ChurchHistory.tsx
Language / Type	TypeScript (React, TSX)
Lines	423
Functions / Classes	ParishFigure (exported type); ParishHistorySection (exported type); parishHistorySections (exported function); hasParishHistory (exported function); parishHistoryLede (exported function); parishHeroPhoto (exported function); parishAccent (exported function); ChurchHistory (exported component/function (default))
Related Files (its imports)	./HistoryQuiz, ../types, ../data, ../lib/useParishContent, ../lib/useLanguage, ../lib/language, ../lib/parishTheme, ../data/mhcpHistory, ../lib/useNarration, ../lib/historyNarration
Where It Is Used	src/App.tsx, src/components/ChurchHistoryCard.tsx, src/components/SanctiHost.tsx, src/data/mhcpHistory.ts
How to Recognize It	Open SanctDemo/src/components/ChurchHistory.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChurchHistory.tsx

```
import React, { type CSSProperties } from "react";
import HistoryQuiz from "./HistoryQuiz";
import { ArrowLeft, Volume2, VolumeX } from "lucide-react";
import type { Route } from "../types";
import { PARISH_PATRON_IMAGES, PARISH_PATRON_SAINTS } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { useLanguage } from "../lib/useLanguage";
import { pick, type Bilingual, type Language } from "../lib/language";
import { parishTheme } from "../lib/parishTheme";
import { MHCP_HISTORY } from "../data/mhcpHistory";
import { useNarration } from "../lib/useNarration";
import { historyScript, spokenLength } from "../lib/historyNarration";

/** The heading the parish's own Filipino account starts at. */
const FIRST_OWN_SECTION = pick(MHCP_HISTORY[0]?.heading, "fil");

/**
 * A photograph from the parish's own record, with what the parish
 * called it.
 *
 * `srcs` is a list because some captions cover a set - "Ilan sa mga
 * Lumang Larawan ng Iba't Ibang Samahan" names three photographs at
 * once, and splitting them would invent three captions the parish did
 * not write.
 *
 * `caption` is optional. Two of the photographs in the parish's
```

```
 * document carry none, and naming the people in them from the
 * surrounding text would be a guess printed under a face.
 */
export interface ParishFigure {
  srcs: string[];
  caption?: Bilingual | string;
  /** Where the parish got it: the souvenir programme, a photographer. */
  source: string;
}

export interface ParishHistorySection {
  /**
   * Sits above the paragraphs in small caps, as "OUR HISTORY" does.
   *
   * A plain string where only one language exists - the compiled
   * English sections and San Roque's - and a pair where the parish's
   * own text has been translated. `pick` takes either.
   */
  heading: Bilingual | string;
  paragraphs: (Bilingual | string)[];
  figures?: ParishFigure[];
}

interface ParishHistoryContent {
  sections: ParishHistorySection[];
}

// Real parish-specific history text, keyed by route id. Both of these came
// from the diocese's own material — nothing here is invented. A parish with
// no entry falls back to the honest "not yet documented" panel below rather
// than showing either of these.
const PARISH_HISTORY: Record<string, ParishHistoryContent> = {
  "route-mhcp": {
    sections: [
      {
        heading: "Our History",
        paragraphs: [
          "Mary Help of Christians Parish in Maypajo, Caloocan City was established in 1952 to serve the rapidly expanding population of Southern Caloocan and to
provide a sacred sanctuary dedicated to Marian devotion. Under the zealous pastoral leadership of the parish priests and active community builders, it grew from a
simple chapel of wood and nipa into a beautiful parish that stands as a beacon of Christian faith and Salesian spirit.",
          "Over the decades, the parish has nurtured thousands of families, active lay organizations, and youth ministries, becoming a vital hub for prayer,
catechesis, sacraments, and social services.",
        ],
      },
      {
        heading: "Our Patroness",
        paragraphs: [
          "The beloved patroness is the Blessed Virgin Mary under the title Mary, Help of Christians (Maria Auxiliadora). This title highlights her powerful maternal
role as a helper, protector, and defender of Christians in times of severe trial, hardship, or spiritual battle.",
          "Venerated highly within the Salesian family of St. John Bosco, her image depicts her holding the Child Jesus with both arms open, inviting the faithful to
surrender their cares, seek grace, and find victory over life's daily storms through Christ.",
        ],
      },
      // The parish's own full account, in its own words and its own
      // language, after the two compiled sections above. See
      // data/mhcpHistory.ts.
      ...MHCP_HISTORY,
    ],
  },
```

```
  "route-src": {
    sections: [
      {
        heading: "Our History",
        paragraphs: [
          "The historic San Roque Cathedral was originally founded as a parish in 1815 under the Archdiocese of Manila. The church stood witness to the turbulent
days of the Philippine Revolution, serving as a military post and safe haven for local revolutionaries. It was repeatedly damaged during wars but rose each time
through the absolute devotion of the Caloocan parishioners.",
          "In 2003, His Holiness Pope John Paul II established the Diocese of Kalookan, and San Roque Parish was officially elevated to the dignity of a Cathedral,
serving as the central seat of the Bishop, guiding the spiritual flock of Caloocan, Malabon, and Navotas.",
        ],
      },
      {
        heading: "Our Patron",
        paragraphs: [
          "The Cathedral is named in honor of San Roque (Saint Roch), a 14th-century lay saint who traveled across Europe nursing patients afflicted with the
horrific bubonic plague. He is depicted pointing to a plague sore on his thigh, flanked by his faithful dog who brought him daily loaves of bread when he was
isolated in the forest.",
          "San Roque is beloved as the patron saint of the sick, plague victims, healthcare workers, and epidemics, standing as a testament of pure Christian
charity, selfless volunteerism, and healing.",
        ],
      },
    ],
  },
};

/**
 * True when the history page has something to show for this parish — either
 * the compiled text above or, at runtime, something the parish has written
 * itself.
 *
 * Exported because the Church History card on Home has to decide whether to
 * offer "Learn more", and the only honest basis for that is whether there is
 * a page behind it. The card used to key that button off admin-written text
 * alone, so both parishes with a real compiled history showed "not been
 * written up yet" and no way through to the history they already had.
 */
/**
 * A parish's compiled history, for anything that has to read it rather
 * than render it.
 *
 * Exported for the quiz's test, which asserts that every correct answer
 * really is a sentence on this page. Without that check a question
 * could drift away from the history it claims to be about, or be
 * invented outright - which is the one thing this feature must not do.
 */
export function parishHistorySections(routeId: string): ParishHistorySection[] {
  return PARISH_HISTORY[routeId]?.sections ?? [];
}

export function hasParishHistory(routeId: string, managedBody?: string | null): boolean {
  return Boolean(managedBody?.trim()) || (PARISH_HISTORY[routeId]?.sections.length ?? 0) > 0;
}

/**
 * The opening of a parish's compiled history, for the card that links
 * to it. Takes the reader's language, since a parish's own sections
 * now carry both.
 */
```

```
export function parishHistoryLede(
  routeId: string,
  language: Language = "en",
): string | undefined {
  const first = PARISH_HISTORY[routeId]?.sections[0]?.paragraphs[0];
  return first === undefined ? undefined : pick(first, language);
}



/**
 * The photograph the history page opens on, taken from the parish's own
 * artboard in the PSD and cropped clear of the mockup's status bar.
 *
 * Separate from PARISH_PATRON_IMAGES on purpose. Those are wide shots of the
 * patron in its niche, which is right for a card on Home but wrong filling
 * half a screen - at that size the figure is small and the frame is mostly
 * altar. The design opens on the face, and these are the frames the design
 * opens on. A parish with no entry falls back to its niche photograph, which
 * is still better than an empty panel.
 */
const PARISH_HISTORY_PHOTOS: Record<string, string> = {
  "route-mhcp": "/parish/mary-help-history.jpg",
  "route-src": "/parish/san-roque-history.jpg",
};

/** Where the face sits in each of those frames, as a fraction of the height. */
const PARISH_HISTORY_FACES: Record<string, number> = {
  "route-mhcp": 0.38,
  "route-src": 0.3,
};

/**
 * The hero photograph for a parish, and where to crop it.
 *
 * Exported so the parish's own page can open on the same frame as its
 * history rather than on a different photograph of the same statue.
 */
export function parishHeroPhoto(routeId: string, managedPhoto?: string | null): {
  src: string | undefined;
  /** An object-position value, already in the form CSS wants. */
  position: string;
} {
  // An admin-uploaded photograph wins, but nobody has said where its subject
  // is, so it gets the neutral crop rather than another parish's measurement.
  if (managedPhoto) return { src: managedPhoto, position: "center 30%" };

  const designed = PARISH_HISTORY_PHOTOS[routeId];
  if (designed) {
    return { src: designed, position: "center " + Math.round(PARISH_HISTORY_FACES[routeId] * 100) + "%" };
  }
  return { src: PARISH_PATRON_IMAGES[routeId], position: "center 30%" };
}

/**
 * A parish's two panel colours, from the one table that holds every
 * parish's livery. Kept as a named export because three call sites read the
 * panel gradient specifically, and "the history panel's colours" is a
 * clearer thing to ask for than two fields of a theme.
```

```
 */
export function parishAccent(routeId: string): { top: string; bottom: string } {
  const theme = parishTheme(routeId);
  return { top: theme.panelTop, bottom: theme.panelBottom };
}

interface ChurchHistoryProps {
  parish: Route;
  /** Omitted when this page is reached from the tab bar rather than from a parish. */
  onBack?: () => void;
}

/**
 * A parish's history — the "MHCP HISTORY PAGE" and "SRCP HISTORY PAGE"
 * artboards from the client's PSD.
 *
 * The design is one idea: the patron fills the top of the screen, and the
 * history rises over the bottom of that photograph on a panel coloured from
 * the image itself. There is no card, no border and no white page — the words
 * sit on the parish's own colour, which is also why the panel carries a faint
 * second copy of the patron behind the text instead of a flat fill.
 *
 * The title is the patron's name rather than the parish's. That is what the
 * PSD has ("Maria Auxiliadora", "San Roque"), and it is right: this is a
 * devotional history, and the parish name is on every screen that leads here.
 */
export default function ChurchHistory({ parish, onBack }: ChurchHistoryProps) {
  const { language } = useLanguage();
  const managed = useParishContent(parish.id);
  const parishName = parish.name.replace(" Guide", "").replace(" Tour", "");

  // The stored patron is "Maria Auxiliadora (Mary Help of Christians)". The
  // gloss repeats the parish name that led the pilgrim here, and at display
  // size it turned a two-line title into five.
  const patron = PARISH_PATRON_SAINTS[parish.id]?.replace(/\s*\([^)]*\)\s*$/, "").trim();
  const title = managed?.historyTitle?.trim() || patron || parishName;
  const hero = parishHeroPhoto(parish.id, managed?.historyPhotoUrl);
  const photo = hero.src;
  const accent = parishAccent(parish.id);

  // Admin-written history replaces the compiled text rather than joining it.
  // A parish that has written its own history has said what it wants said,
  // and running the two together would publish a version nobody approved.
  const managedBody = managed?.historyBody?.trim();
  const sections: ParishHistorySection[] = managedBody
    ? [
        {
          heading: "Our History",
          paragraphs: managedBody
            .split(/\n{2,}/)
            .map(paragraph => paragraph.trim())
            .filter(Boolean),
        },
      ]
    : PARISH_HISTORY[parish.id]?.sections ?? [];

  /*
   * The history, read aloud.
   *
```

```
 * The same voice the rest of SanctiWalk uses - native text-to-speech
 * on the phone, the browser's own on a laptop - asking for fil-PH or
 * en-US to match the language on screen. It reads the language the
 * reader is looking at, never a fixed one: a Filipino page narrated
 * in English would be the app talking over the parish.
 *
 * The engine is not everywhere. A browser with no installed voice
 * reports none, and `supported` is false there, so the control is
 * absent rather than present and dead.
 */
const narration = useNarration(language);
const script = historyScript(title, sections, language);
const length = spokenLength(script, language);

// Switching language mid-sentence would leave the previous language
// playing under the new text. Stopping is the honest reaction; the
// reader can tap again and hear it in the language they just chose.
const stopNarration = narration.stop;
React.useEffect(() => {
  stopNarration();
}, [language, stopNarration]);

const style = {
  "--ph-top": accent.top,
  "--ph-bottom": accent.bottom,
  "--ph-face": hero.position,
} as CSSProperties;

return (
  <div className="parish-history" style={style}>
    <div className="parish-history__hero">
      {photo ? (
        <img
          className="parish-history__photo"
          src={photo}
          alt={patron ? `${patron}, the patron of ${parishName}` : parishName}
        />
      ) : (
        <div className="parish-history__photo parish-history__photo--empty" aria-hidden />
      )}

      {onBack && (
        // Floating over the photograph rather than sitting in a bar above
        // it. The artboard has no header here at all, and a bar would spend
        // the top of the patron's image saying one word.
        <button type="button" onClick={onBack} className="parish-history__back" aria-label="Back">
          <ArrowLeft className="w-5 h-5" />
        </button>
      )}
    </div>

    <article className="parish-history__panel">
      {/* The patron again, very faint, behind the words. It is what gives
          the panel its depth in the PSD; a flat fill reads as a coloured
          box. Hidden from assistive tech because the same photograph is
          already described directly above it. */}
      {photo && <img className="parish-history__watermark" src={photo} alt="" aria-hidden />}

      <div className="parish-history__body">
```

```
          <h1 className="parish-history__title">{title}</h1>

          {/* Directly under the title, where someone decides whether to
              read or to listen - not floating over the photograph, which
              is where a media control goes when it belongs to the image
              rather than to the words. */}
          {narration.supported && script && (
            <button
              type="button"
              onClick={() => narration.toggle(script)}
              className="parish-history__listen"
              aria-pressed={narration.speaking}
              aria-label={
                narration.speaking
                  ? "Stop reading this history aloud"
                  : `Read this history aloud${length ? `, ${length}` : ""}`
              }
            >
              {narration.speaking ? (
                <>
                  <VolumeX className="w-4 h-4" aria-hidden />
                  <span>{language === "fil" ? "Ihinto" : "Stop"}</span>
                </>
              ) : (
                <>
                  <Volume2 className="w-4 h-4" aria-hidden />
                  <span>{language === "fil" ? "Pakinggan" : "Listen"}</span>
                  {length && <span className="parish-history__listen-time">{length}</span>}
                </>
              )}
            </button>
          )}

          {sections.length > 0 ? (
            sections.map(section => (
              <React.Fragment key={pick(section.heading, "fil")}>
              {/* Said once, at the point the parish's own words begin.
                  Ten sections of Filipino arriving with no explanation
                  would read to an English reader as a translation the
                  app had failed to do, rather than as the parish's own
                  record left as the parish wrote it. */}
              {pick(section.heading, "fil") === FIRST_OWN_SECTION && (
                <p className="parish-history__language-note">
                  {language === "fil"
                    ? "Ang sumusunod ay ang buong kasaysayan na isinulat mismo ng parokya."
                    : "What follows is the parish's own history, written in Filipino and translated here for readers. The Filipino is the original; where the two
differ, the parish's own words are the ones that stand."}
                </p>
              )}
              <section className="parish-history__section">
                <h2 className="parish-history__eyebrow">
                  {pick(section.heading, language)}
                </h2>
                {section.paragraphs.map(paragraph => {
                  const text = pick(paragraph, language);
                  return (
                    <p key={text.slice(0, 48)} className="parish-history__paragraph">
                      {text}
                    </p>
```

```
              );
            })}

            {section.figures?.map(figure => (
              <figure key={figure.srcs[0]} className="parish-figure">
                <div
                  className={`parish-figure__frame${
                    figure.srcs.length > 1 ? " is-set" : ""}`}
                >
                  {figure.srcs.map(src => (
                    <img
                      key={src}
                      src={src}
                      /* The caption is directly beneath and says the
                         same thing, so repeating it here would have a
                         screen reader read every photograph twice. */
                      alt=""
                      loading="lazy"
                      className="parish-figure__img"
                    />
                  ))}
                </div>
                <figcaption className="parish-figure__caption">
                  {figure.caption && <span>{pick(figure.caption, language)}</span>}
                  <span className="parish-figure__source">
                    Sanggunian: {figure.source}
                  </span>
                </figcaption>
              </figure>
            ))}
          </section>
          </React.Fragment>
        ))
      ) : (
        // No compiled text, and nothing written by the parish. Inventing
        // a history would put words in a church's mouth, and a devotional
        // history that is subtly wrong is worse than an honest absence.
        <section className="parish-history__section">
          <h2 className="parish-history__eyebrow">Our History</h2>
          <p className="parish-history__paragraph">
            {parishName} has not had its history written up yet. The parish office can add it
            from the admin portal, and it will appear here.
          </p>
        </section>
      )}

      {/* At the very bottom, after the history has been read. See
          HistoryQuiz: it renders nothing for a parish whose
          history is not written up. */}
      <HistoryQuiz routeId={parish.id} />
    </div>
  </article>
 </div>
 );
}
```

1.2.10 APP-015 — ChurchHistoryCard.tsx

Code ID	APP-015
File Name	ChurchHistoryCard.tsx
File Path	SanctDemo/src/components/ChurchHistoryCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	115
Functions / Classes	ChurchHistoryCard (exported component/function (default))
Related Files (its imports)	../data, ../lib/useParishContent, ./ChurchHistory
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/ChurchHistoryCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChurchHistoryCard.tsx

```
import type { CSSProperties } from "react";
import { PARISH_PATRON_IMAGES, PARISH_PATRON_SAINTS } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { hasParishHistory, parishAccent, parishHistoryLede } from "./ChurchHistory";

interface ChurchHistoryCardProps {
  routeId: string;
  parishName: string;
  onOpenHistory: () => void;
}

/**
 * The navy Church History card on Home.
 *
 * Nothing here is invented for a parish that has no history: inventing one
 * would put words in a church's mouth, and a devotional history that is
 * subtly wrong is worse than an honest absence. What the card WILL show is
 * the history the app already carries — this used to look only at
 * admin-written text, so both parishes with a real compiled history were
 * told their history "has not been written up yet" while the history page
 * behind the card had two sections of it.
 */
export default function ChurchHistoryCard({
  routeId,
  parishName,
  onOpenHistory,
}: ChurchHistoryCardProps) {
  const managed = useParishContent(routeId);

  // The patron's name is the natural title, so an admin who writes only the
  // body still gets a headed card.
  //
  // The trailing gloss is dropped: the stored patron is "Maria Auxiliadora
```

```
// (Mary Help of Christians)", and the parish name directly above already
// says Mary Help of Christians. Kept whole it wrapped to five lines and
// repeated the header.
const patron = PARISH_PATRON_SAINTS[routeId]?.replace(/\s*\([^)]*\)\s*$/, "").trim();
const title = managed?.historyTitle?.trim() || patron || parishName;
const managedBody = managed?.historyBody?.trim();
// The card shows the opening of whatever the history page will show, so the
// two never disagree about whether this parish has a history.
const body = managedBody || parishHistoryLede(routeId);
const canLearnMore = hasParishHistory(routeId, managedBody);
const photo = managed?.historyPhotoUrl ?? PARISH_PATRON_IMAGES[routeId];

// The card wears the parish's own colours, not the app's navy. That is what
// the PSD has - the Mary Help card is the blue of her mantle, and its
// gradient is the same one the history page opens with - and it is what
// makes the card and the page it opens read as one thing. It also stops
// this card arriving on San Roque's page as a navy rectangle on orange.
const accent = parishAccent(routeId);
const style = {
  background: "linear-gradient(160deg, " + accent.top + " 0%, " + accent.bottom + " 78%)",
  "--chc-accent-bottom": accent.bottom,
} as CSSProperties;

return (
  <section className="relative overflow-hidden rounded-[26px] shadow-lg" style={style}>
    {photo && (
      <>
        {/* The photograph sits to the right and the text runs over the
            left, as in the design. A scrim rather than a shadow: these are
            gilded statues under warm light, so brightness under the text is
            unpredictable — a shadow holds over dark vestments and vanishes
            over gold. */}
        <img
          src={photo}
          alt=""
          aria-hidden
          className="absolute inset-y-0 right-0 h-full w-[52%] object-cover"
          style={{ objectPosition: "center 30%" }}
          loading="lazy"
        />
        <span
          aria-hidden
          className="absolute inset-0"
          style={{
            background:
              "linear-gradient(90deg, var(--chc-accent-bottom) 46%, color-mix(in srgb, var(--chc-accent-bottom) 55%, transparent) 68%, transparent 100%)",
          }}
        />
      </>
    )}

    <div
      className="relative px-5 pt-5 pb-4 text-white"
      style={{ maxWidth: photo ? "62%" : "100%" }}
    >
      <h3 className="church-history-card__title">{title}</h3>

      {body ? (
        // Clamped, not truncated by hand. This is the opening of the
```

```
          // history page's first paragraph, and it is meant to run out mid
          // thought - that is what "Learn more" is answering.
          <p className="church-history-card__body">{body}</p>
        ) : (
          <p className="mt-2.5 text-[15px] leading-relaxed text-white/75">
            This parish's history has not been written up yet. The parish office can add it
            from the admin portal.
          </p>
        )}
      </div>

      {/* Centred at the foot of the card with a short rule under it, as the
          PSD has it - not a left-aligned chevron link. It spans the whole
          card rather than the text column, because in the design it sits
          under the photograph too. */}
      {canLearnMore && (
        <button type="button" onClick={onOpenHistory} className="church-history-card__more">
          Learn more
        </button>
      )}
    </section>
  );
}
```

1.2.11 APP-016 — CompanionTab.tsx

Code ID	APP-016
File Name	CompanionTab.tsx
File Path	SanctDemo/src/components/CompanionTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	248
Functions / Classes	CompanionTab (exported component/function (default))
Related Files (its imports)	../types, ../lib/apiBase, ../lib/appKey
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/CompanionTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — CompanionTab.tsx

```
import React, { useState } from "react";
import { Route, Station } from "../types";
import { Sparkles, MapPin, Clock, BrainCircuit, Play, Compass, Loader2 } from "lucide-react";
import { apiUrl } from "../lib/apiBase";
import { withAppKey } from "../lib/appKey";

interface CompanionTabProps {
  onLoadCustomRoute: (customRoute: Route) => void;
  isOffline: boolean;
}

export default function CompanionTab({ onLoadCustomRoute, isOffline }: CompanionTabProps) {
  const [location, setLocation] = useState("Antipolo, Rizal");
  const [interest, setInterest] = useState("Quiet Contemplation & Nature");
  const [duration, setDuration] = useState("30");
  const [loading, setLoading] = useState(false);
  const [generatedRoute, setGeneratedRoute] = useState<Route | null>(null);
  const [loadingMessage, setLoadingMessage] = useState("");

  const loadingPhrases = [
    "Consulting historical parish archives...",
    "Calculating optimal pedestrian walking paths...",
    "Drafting reflective pilgrim mindfulness prompts...",
    "Preparing your personalized audio guide transcript...",
    "Structuring heritage routes near you..."
  ];

  const handleGenerate = async () => {
    if (isOffline) {
      alert("AI generation is offline. Please enable Online Sim mode to connect with Gemini.");
      return;
    }
```

```
setLoading(true);
setGeneratedRoute(null);

// Cycle loading phrases for rich UI feedback
let phraseIdx = 0;
setLoadingMessage(loadingPhrases[0]);
const messageInterval = setInterval(() => {
  phraseIdx = (phraseIdx + 1) % loadingPhrases.length;
  setLoadingMessage(loadingPhrases[phraseIdx]);
}, 1500);

try {
  const res = await fetch(apiUrl("/api/generate-walk"), {
    method: "POST",
    headers: withAppKey({
      "Content-Type": "application/json"
    }),
    body: JSON.stringify({
      location,
      interest,
      durationMinutes: parseInt(duration)
    })
  });

  const data = await res.json();

  // Structure the returned schema to fit our standard Route format
  const customRoute: Route = {
    id: `custom-${Date.now()}`,
    name: data.name,
    description: data.description,
    location: location,
    distanceKm: data.distanceKm || 1.5,
    durationMins: data.durationMins || parseInt(duration),
    difficulty: "Moderate",
    estimatedSteps: data.estimatedSteps || 2500,
    category: "Spiritual Retreat",
    accentColor: "from-purple-600 to-purple-800",
    stations: (data.stations || []).map((s: any, idx: number) => ({
      id: `custom-st-${idx}`,
      name: s.name,
      description: s.description,
      history: s.history,
      reflection: s.reflection,
      coordinates: {
        lat: 14.5 + (Math.random() - 0.5) * 0.1,
        lng: 121.0 + (Math.random() - 0.5) * 0.1
      },
      qrCode: `SW-AI-QR-${idx}`
    }))
  };

  setGeneratedRoute(customRoute);
} catch (err) {
  console.error(err);
} finally {
  clearInterval(messageInterval);
  setLoading(false);
```

```
    }
  };

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] pb-20 select-none">
      {/* Hero Header */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm border-b border-[var(--color-brand-border)]">
        <h3 className="text-[15px] font-bold text-[var(--color-brand-gold)] font-serif italic uppercase tracking-wider flex items-center gap-1.5">
          <BrainCircuit className="w-3.5 h-3.5 text-[var(--color-brand-gold)]" /> AI Walk Companion
        </h3>
        <h2 className="text-xl font-bold font-serif italic tracking-tight mt-0.5">
          Custom Pilgrim Route Planner
        </h2>
        <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-90 mt-1 max-w-sm font-sans leading-relaxed">
          Fulfill complex capstone requirements! Leverage server-side Gemini to construct a fully custom heritage walk based on your precise interest.
        </p>
      </div>

      <div className="p-4 space-y-4">
        {loading ? (
          /* High-quality animated loading panel */
          <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-8 text-center space-y-4 shadow-xs">
            <Loader2 className="w-8 h-8 text-[var(--color-brand-secondary)] animate-spin mx-auto" />
            <div className="space-y-1">
              <h4 className="text-sm font-bold text-[var(--color-brand-text)] font-serif italic">Generating Your SanctiWalk...</h4>
              <p className="text-[15px] text-[var(--color-brand-secondary)] italic animate-pulse">{loadingMessage}</p>
            </div>
          </div>
        ) : generatedRoute ? (
          /* Generated Custom Route View */
          <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-4 shadow-sm space-y-4">
            <div className="bg-[var(--color-brand-card)] text-[var(--color-brand-secondary)] text-sm font-bold px-2.5 py-1 rounded-full uppercase tracking-wider w-
fit flex items-center gap-1 border border-[var(--color-brand-border)]">
              <Sparkles className="w-3.5 h-3.5 text-[var(--color-brand-gold)]" /> Custom Gemini Gen
            </div>

            <div>
              <h3 className="text-sm font-bold text-[var(--color-brand-text)] font-serif italic leading-tight">{generatedRoute.name}</h3>
              <p className="text-[15px] text-[var(--color-brand-secondary)] mt-1 leading-normal font-sans">{generatedRoute.description}</p>
            </div>

            {/* Custom walk stats */}
            <div className="grid grid-cols-3 gap-2 border-y border-[var(--color-brand-border)] py-3 text-center">
              <div>
                <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono font-bold">Distance</span>
                <strong className="text-[15px] text-[var(--color-brand-text)] font-bold">{generatedRoute.distanceKm} km</strong>
              </div>
              <div>
                <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono font-bold">Time</span>
                <strong className="text-[15px] text-[var(--color-brand-text)] font-bold">{generatedRoute.durationMins} mins</strong>
              </div>
              <div>
                <span className="block text-sm text-[var(--color-brand-secondary)] uppercase font-mono font-bold">Est. Steps</span>
                <strong className="text-[15px] text-[var(--color-brand-text)] font-bold">{generatedRoute.estimatedSteps.toLocaleString()}</strong>
              </div>
            </div>

            {/* Stations Summary List */}
            <div className="space-y-2.5">
```

```
              <h4 className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">Stations along Route</h4>
              <div className="space-y-2">
                {generatedRoute.stations.map((st, idx) => (
                  <div key={st.id} className="p-3 bg-[var(--color-brand-card)]/30 rounded-2xl border border-[var(--color-brand-border)] flex gap-2.5 items-start">
                    <span className="h-5 w-5 rounded-full bg-[var(--color-brand-primary)] text-white flex items-center justify-center text-sm font-bold font-mono
shrink-0 border border-[var(--color-brand-primary-dark)]">
                      0{idx + 1}
                    </span>
                    <div className="space-y-0.5">
                      <h5 className="text-[15px] font-bold text-[var(--color-brand-text)] font-serif italic leading-tight">{st.name}</h5>
                      <p className="text-[15px] text-[var(--color-brand-secondary)] leading-normal line-clamp-1 font-sans">{st.description}</p>
                    </div>
                  </div>
                ))}
              </div>
            </div>

            <button
              onClick={() => onLoadCustomRoute(generatedRoute)}
              className="w-full py-2.5 px-4 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white font-bold text-[15px] rounded-full
flex items-center justify-center gap-2 shadow-sm border border-[var(--color-brand-primary-dark)] uppercase tracking-wider transition-all hover:scale-[1.01]"
            >
              <Play className="w-4 h-4 fill-current ml-0.5 text-white" /> Start AI Guide Now
            </button>
          </div>
        ) : (
          /* Setup / Customizer Form */
          <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-4 shadow-xs space-y-4">
            <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed font-sans">
              Define your surroundings and preferences below. The Gemini AI engine will synthesize architectural history, custom routes, and wellness meditations
dynamically.
            </p>

            {/* Location Input */}
            <div className="space-y-1">
              <label htmlFor="input-location" className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
                Where are you walking?
              </label>
              <div className="bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-2xl p-2.5 flex items-center gap-2">
                <MapPin className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
                <input
                  id="input-location"
                  type="text"
                  placeholder="e.g. Antipolo Rizal / Quiapo Manila / Cebu City"
                  value={location}
                  onChange={(e) => setLocation(e.target.value)}
                  className="w-full bg-transparent text-[15px] text-[var(--color-brand-text)] outline-none placeholder-[var(--color-brand-secondary)] font-sans"
                />
              </div>
            </div>

            {/* Interest Input */}
            <div className="space-y-1">
              <label htmlFor="input-interest" className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
                What is your walk interest/focus?
              </label>
              <div className="bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-2xl p-2.5 flex items-center gap-2">
                <Compass className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
                <input
```

```
                id="input-interest"
                type="text"
                placeholder="e.g. Baroque Carvings / Mental Peace / Local folklore"
                value={interest}
                onChange={(e) => setInterest(e.target.value)}
                className="w-full bg-transparent text-[15px] text-[var(--color-brand-text)] outline-none placeholder-[var(--color-brand-secondary)] font-sans"
              />
            </div>
          </div>

          {/* Duration Selector */}
          <div className="space-y-1">
            <label htmlFor="select-duration" className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
              Target Walking Time (Minutes)
            </label>
            <div className="bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-2xl p-2.5 flex items-center gap-2">
              <Clock className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
              <select
                id="select-duration"
                value={duration}
                onChange={(e) => setDuration(e.target.value)}
                className="w-full bg-transparent text-[15px] text-[var(--color-brand-text)] outline-none font-bold cursor-pointer font-serif italic"
              >
                <option value="20" className="font-serif">20 Minutes Walk</option>
                <option value="30" className="font-serif">30 Minutes Walk</option>
                <option value="45" className="font-serif">45 Minutes Walk</option>
                <option value="60" className="font-serif">60 Minutes Walk</option>
              </select>
            </div>
          </div>

          <button
            onClick={handleGenerate}
            className="w-full py-2.5 px-4 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white font-bold text-[15px] rounded-full
flex items-center justify-center gap-2 shadow-sm border border-[var(--color-brand-primary-dark)] uppercase tracking-wider transition-all hover:scale-[1.01]"
          >
            <Sparkles className="w-4 h-4 text-[var(--color-brand-gold)]" /> Generate Custom AI Walk
          </button>
        </div>
      )}
    </div>
  </div>
);
}
```

CompassControl.tsx — listed in full as MAP-003.

1.2.12 APP-017 — CustomDioceseMap.tsx

Code ID	APP-017
File Name	CustomDioceseMap.tsx
File Path	SanctDemo/src/components/CustomDioceseMap.tsx
Language / Type	TypeScript (React, TSX)
Lines	54
Functions / Classes	CustomDioceseMap (exported component/function (default))
Related Files (its imports)	./DioceseMapLive
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/CustomDioceseMap.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — CustomDioceseMap.tsx

```
import { ExternalLink } from "lucide-react";
import DioceseMapLive, { OPEN_IN_GOOGLE_MAPS_URL } from "./DioceseMapLive";

// One map: the app's own interactive MapLibre map (DioceseMapLive), restyled
// and shaded to match the client's Google My Map ("SanctDemoMap") — same
// dark base, teal pins (the client's orange, swapped per their request so
// pins match the app), and the client's own 7-point study-area polygon
// reproduced as a MapLibre fill layer. This used to be a two-tab switcher
// between an embedded copy of their My Map and this live map; the embed is
// gone because a My Map embed is a sealed Google iframe that can never show
// the pilgrim's own position — the entire point of merging the two designs
// into one map instead.
//
// The "open in Google Maps" link is kept below: harmless, and genuinely
// useful for sharing the client's original map outside the app.
interface CustomDioceseMapProps {
  onSelectParish: (parishId: string) => void;
  /** A parish to draw a walking route to as soon as the map is ready. */
  walkToParishId?: string | null;
  onWalkToConsumed?: () => void;
  // See DioceseMapLive's heightPx: when set, the map gets this fixed pixel
  // height and the legend/link below it grow the card instead of being
  // squeezed inside it. Omitted by the two callers that still live inside a
  // fixed-height card (Dashboard's mini map, the church selector).
  mapHeight?: number;
}

export default function CustomDioceseMap({ onSelectParish, mapHeight, walkToParishId, onWalkToConsumed }: CustomDioceseMapProps) {
  const wrapClassName = mapHeight != null ? "flex flex-col gap-2" : "flex flex-col gap-2 h-full min-h-0";
  const mapSlotClassName = mapHeight != null ? "" : "flex-1 min-h-0";
```

```
  return (
    <div className={wrapClassName}>
      <div className={mapSlotClassName}>
        <DioceseMapLive
          onSelectParish={onSelectParish}
          heightPx={mapHeight}
          walkToParishId={walkToParishId}
          onWalkToConsumed={onWalkToConsumed}
        />
      </div>

      <a
        href={OPEN_IN_GOOGLE_MAPS_URL}
        target="_blank"
        rel="noopener noreferrer"
        className="dmap-live__open-link"
      >
        <ExternalLink className="w-4 h-4" />
        Open full map in Google Maps
      </a>
    </div>
  );
}
```

1.2.13 APP-018 — DailyRosary.tsx

Code ID	APP-018
File Name	DailyRosary.tsx
File Path	SanctDemo/src/components/DailyRosary.tsx
Language / Type	TypeScript (React, TSX)
Lines	203
Functions / Classes	DailyRosary (exported component/function (default))
Related Files (its imports)	../lib/speech
Where It Is Used	src/components/PrayScreen.tsx
How to Recognize It	Open SanctDemo/src/components/DailyRosary.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — DailyRosary.tsx

```
import { useEffect, useRef } from "react";

import { Capacitor } from "@capacitor/core";
import { SpeechRecognition } from "@capacitor-community/speech-recognition";

import { speakAloud, stopAloud } from "../lib/speech";

// The client's own hand-built rosary app is mounted here as-is, via iframe,
// rather than ported to React. See public/rosary/index.html (plus the
// public/rosary/20 Mysteries/ image set) for the source. It is intentionally
// left unstyled to match the rest of this app; it keeps its own look.
//
// Layout note: this component renders inside PhoneContainer's scrollable
// "children" slot (a flex flex-col container with overflow-y-auto and a
// fixed pixel height set by the phone/desktop frame). Using 100vh here would
// overflow that frame on desktop, so instead we stretch to fill the parent
// via flex-1 + h-full, letting the iframe's own height:100% fill that.

/** Both ends agree on this so other postMessage traffic is ignored. */
const CHANNEL = "sanctiwalk-rosary";

/**
 * Speaks for the Rosary, because inside the app it cannot speak for
 * itself.
 *
 * The Rosary page narrates through window.speechSynthesis. In a desktop
 * browser that works. In the Android build the page is inside a WebView,
 * whose speechSynthesis is a stub - it accepts an utterance, reports no
 * error, has no voices behind it and makes no sound. The guided Rosary
 * was therefore silent on the one device the pilgrim actually prays
 * with, which is the only device that matters here.
 *
 * Everything else in the app avoids that stub by going through
```

```
 * lib/speech, which uses the platform's text-to-speech on a phone. An
 * iframe cannot reach a Capacitor plugin, so it asks us instead: it
 * says hello, we answer, and after that each line arrives as a message
 * and we read it aloud with the same voice the rest of the app uses.
 *
 * With no answer the page falls back to Web Speech, which is what a
 * laptop wants anyway.
 */
export default function DailyRosary() {
  const frame = useRef<HTMLIFrameElement | null>(null);

  /** Announce us again once the page is up, in case its hello raced us. */
  function announce() {
    frame.current?.contentWindow?.postMessage(
      { channel: CHANNEL, type: "host-ready" },
      window.location.origin,
    );
  }

  useEffect(() => {
    function reply(message: Record<string, unknown>) {
      const win = frame.current?.contentWindow;
      if (!win) return;
      win.postMessage({ channel: CHANNEL, ...message }, window.location.origin);
    }

    async function onMessage(event: MessageEvent) {
      // Same page, same frame, or it is not the Rosary talking.
      if (event.origin !== window.location.origin) return;
      if (frame.current && event.source !== frame.current.contentWindow) return;

      const data = event.data as
        | { channel?: string; type?: string; text?: string; lang?: string }
        | null;
      if (!data || data.channel !== CHANNEL) return;

      if (data.type === "hello") {
        reply({ type: "host-ready" });
        return;
      }

      if (data.type === "stop") {
        await stopAloud();
        return;
      }

      if (data.type === "speak" && typeof data.text === "string") {
        await speakAloud(data.text, data.lang === "fil" ? "fil" : "en");
        // Resolves when the voice stops, so this is the moment the page
        // can light its next bead rather than guess at a duration.
        reply({ type: "spoke-end" });
        return;
      }

      // ---- listening, for the same reason as speaking -----------------
      //
      // The Rosary follows along as you pray through the Web Speech API's
      // SpeechRecognition. That exists in desktop Chrome and does not
      // exist at all in the Android WebView - window.SpeechRecognition and
```

```
      // window.webkitSpeechRecognition are both undefined there - so on
      // the phone the page simply never heard anything. It is the same
      // gap as the voice, in the other direction, and it has the same
      // answer: the app around it has a native recogniser and lends it.

      if (data.type === "listen-available") {
        reply({ type: "listen-availability", available: await listenAvailable() });
        return;
      }

      if (data.type === "listen-start") {
        await startListening(data.lang === "fil" ? "fil" : "en", reply);
        return;
      }

      if (data.type === "listen-stop") {
        await stopListening();
        return;
      }
    }

    window.addEventListener("message", onMessage);
    return () => {
      window.removeEventListener("message", onMessage);
      // Leaving the Pray tab must not leave a voice praying on behind it.
      void stopAloud();
    };
  }, []);

  return (
    <div className="flex-1 flex flex-col h-full min-h-[600px] w-full">
      <iframe
        ref={frame}
        src="/rosary/index.html"
        title="Holy Rosary"
        /* Without this the WebView refuses getUserMedia inside the
           frame, which also kills the loudness glow that lights the
           bead even where recognition is unavailable. */
        allow="microphone"
        onLoad={announce}
        style={{ border: 0 }}
        className="flex-1 w-full h-full"
      />
    </div>
  );
}

/** Native recognition, where there is any. */
async function listenAvailable(): Promise<boolean> {
  try {
    if (!Capacitor.isNativePlatform()) return false;
    const { available } = await SpeechRecognition.available();
    if (!available) return false;
    const perm = await SpeechRecognition.checkPermissions();
    if (perm.speechRecognition !== "granted") {
      const asked = await SpeechRecognition.requestPermissions();
      if (asked.speechRecognition !== "granted") return false;
    }
    return true;
```

```
  } catch {
    return false;
  }
}

let listening = false;

async function stopListening(): Promise<void> {
  if (!listening) return;
  listening = false;
  try {
    await SpeechRecognition.removeAllListeners();
    await SpeechRecognition.stop();
  } catch {
    /* nothing was listening; stopping silence is not an error */
  }
}

/**
 * Listens and relays what it hears, phrase by phrase.
 *
 * partialResults, because the Rosary matches continuously as somebody
 * prays rather than waiting for them to finish a decade. popup is off:
 * Android's own dialog would cover the prayer the pilgrim is reading.
 */
async function startListening(
  language: "fil" | "en",
  reply: (m: Record<string, unknown>) => void,
): Promise<void> {
  if (!(await listenAvailable())) {
    reply({ type: "listen-availability", available: false });
    return;
  }
  await stopListening();
  listening = true;

  try {
    await SpeechRecognition.addListener("partialResults", (result: { matches?: string[] }) => {
      const heard = result?.matches?.[0];
      if (heard) reply({ type: "heard", text: heard });
    });
    await SpeechRecognition.start({
      language: language === "fil" ? "fil-PH" : "en-US",
      maxResults: 1,
      partialResults: true,
      popup: false,
    });
  } catch {
    listening = false;
    reply({ type: "listen-availability", available: false });
  }
}
```

1.2.14 APP-019 — Dashboard.tsx

Code ID	APP-019
File Name	Dashboard.tsx
File Path	SanctDemo/src/components/Dashboard.tsx
Language / Type	TypeScript (React, TSX)
Lines	271
Functions / Classes	parishDisplayName (internal function); formatCountdown (exported function); Dashboard (exported component/function (default))
Related Files (its imports)	./HomeQuickLinks, ./MinistrySacramentPair, ../lib/useLanguage, ../lib/ui, ../types, ../data, ./BulletinRail, ./VerseCard, ./ParishWelcomeHeader, ./MassScheduleCard, ./ChurchHistoryCard, ../lib/parishTheme, ../lib/useParishContent, ../lib/liturgical, ../lib/announcements
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/Dashboard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — Dashboard.tsx

```
import { useEffect, useState } from "react";
import HomeQuickLinks, { type HomeSection } from "./HomeQuickLinks";
import MinistrySacramentPair from "./MinistrySacramentPair";
import { useLanguage } from "../lib/useLanguage";
import { t } from "../lib/ui";
import { Route } from "../types";
import { PARISH_PATRON_IMAGES, PARISH_HEADER_IMAGES, PARISH_HEADER_FACES } from "../data";
import BulletinRail from "./BulletinRail";
import VerseCard from "./VerseCard";
import ParishWelcomeHeader from "./ParishWelcomeHeader";
import MassScheduleCard from "./MassScheduleCard";
import ChurchHistoryCard from "./ChurchHistoryCard";
import { parishThemeStyle } from "../lib/parishTheme";
import { useParishContent } from "../lib/useParishContent";
import { liturgicalDay } from "../lib/liturgical";
import { announcementsForParish, publishedOnly, type AnnouncementDoc } from "../lib/announcements";


interface DashboardProps {
  parish: Route;
  announcements: AnnouncementDoc[];
  onNavigate: (
    tab: "mass" | "history" | "ministries" | "sacraments" | "ar" | "navigator" | "rosary" | "me"
  ) => void;
  onSelectParish: (parishId: string) => void;
  /** Opens the Map tab with a walking route already drawn to this parish. */
  onWalkThere: (parishId: string) => void;
  /** Opens the full-screen parish search. */
```

```
  onOpenSearch: () => void;
  /**
   * Given only when this dashboard is showing a parish that is not the
   * pilgrim's own, which is what puts "Back to my parish" on the band.
   */
  onBackToMyParish?: () => void;
  /** First name of the signed-in pilgrim; the greeting omits it when absent. */
  firstName?: string;
  /**
   * Which part of the bulletin to show, when something outside the
   * dashboard needs to choose.
   *
   * Sancti needs this. Offering "want me to open the Mass schedule?" and
   * then landing the pilgrim on the bulletin with the ministries showing
   * is not opening anything - the section they said yes to is one of
   * three, and only one is on screen at a time.
   *
   * Carries an `id` because the same section can be asked for twice:
   * ask Sancti for the Mass schedule, tap History yourself, ask again,
   * and a bare section value would be unchanged and the request would
   * do nothing. The id makes each request its own event.
   *
   * Undefined means nobody outside has an opinion and the dashboard
   * keeps its own.
   */
  sectionRequest?: { section: HomeSection; id: number };
}

function parishDisplayName(parish: Route): string {
  return parish.name.replace(" Guide", "").replace(" Tour", "");
}

// "in 3 hours", "in 25 minutes" — coarse enough to stay readable, fine
// enough that a pilgrim glancing at the dashboard can tell whether to
// hurry. Recomputed every 30s by the caller so it never goes stale while
// the dashboard is left open.
export function formatCountdown(target: Date, now: Date): string {
  const diffMs = target.getTime() - now.getTime();
  if (diffMs <= 0) return "starting now";

  const totalMinutes = Math.round(diffMs / 60000);
  if (totalMinutes < 1) return "in under a minute";
  if (totalMinutes < 60) return `in ${totalMinutes} minute${totalMinutes === 1 ? "" : "s"}`;

  const hours = Math.round(totalMinutes / 60);
  if (hours < 24) return `in ${hours} hour${hours === 1 ? "" : "s"}`;

  const days = Math.round(hours / 24);
  return `in ${days} day${days === 1 ? "" : "s"}`;
}

export default function Dashboard({
  parish, announcements, onNavigate, firstName, onOpenSearch, onBackToMyParish,
  sectionRequest,
}: DashboardProps) {
  const parishName = parishDisplayName(parish);
  const [now, setNow] = useState(() => new Date());

  // Keeps the date and the Mass card fresh without needing the pilgrim to
```

```
// reopen the tab.
useEffect(() => {
  const id = setInterval(() => setNow(new Date()), 30000);
  return () => clearInterval(id);
}, []);

// Admin-managed overlay: photo, description, Mass times, history, colour.
// Null for every parish nobody has edited, which is all of them until
// someone does — and the compiled data in data.ts carries those.
const managed = useParishContent(parish.id);
const { language } = useLanguage();

/**
 * Which part of the bulletin is showing.
 *
 * Ministries first: it is the one most visitors are looking for, and
 * it is the only one of the three that is a door rather than a
 * notice.
 */
const [section, setSection] = useState<HomeSection>("ministries");

// A request from outside wins, until the pilgrim taps a different one
// themselves. Held as state rather than read straight from the prop so
// the three buttons keep working afterwards: a prop read directly would
// pin the bulletin to whatever Sancti last asked for and make the
// chooser look broken.
useEffect(() => {
  if (sectionRequest) setSection(sectionRequest.section);
}, [sectionRequest?.id, sectionRequest?.section]);
const today = liturgicalDay(now);

// The parish's own words win; the calendar's named feast is the fallback.
// Both may be absent — most days of the year are not a feast — and that is
// a reason to show nothing, not to invent something.
const commemorates = managed?.commemoratesText?.trim() || today.feast;

return (
  <div
    className="home-bright home-bright__ground flex-1 flex flex-col text-left"
    style={parishThemeStyle(parish.id)}
  >
    {/* The bright theme is scoped to THIS element, not to :root. Every
        component below — including shared ones like BulletinRail — picks up
        the new palette through the same role tokens, while Map, Pray, Me
        and Scan stay warm paper. Rolling the theme app-wide later means
        moving the block in index.css to :root, not editing components.

        The parish's own three colours ride in on the same element, so the
        whole screen wears its livery: the navy was never the app's colour,
        it was Mary Help's, and San Roque's screen was wearing another
        parish's blue. */}

    {/* The welcome band from the design: the patron image behind the
        greeting, the date, the parish name and where it is. It replaces a
        date row plus a collapsing photo card, which read as a list item
        rather than as arriving at the parish's own page. */}
    <ParishWelcomeHeader
      parishName={parishName}
      location={parish.location}
```

```
  imageUrl={managed?.photoUrl ?? PARISH_HEADER_IMAGES[parish.id] ?? PARISH_PATRON_IMAGES[parish.id]}
  // Only for the parish's own photograph. An admin-uploaded one has
  // nobody to say where the face is, so it keeps the plain crop.
  imageFaceY={managed?.photoUrl ? undefined : PARISH_HEADER_FACES[parish.id]}
  firstName={firstName}
  onSearch={onOpenSearch}
  onBackToMyParish={onBackToMyParish}
  now={now}
/>

{managed?.description && (
  <div className="px-5 pt-1.5">
    <p className="text-[16px] leading-relaxed text-[var(--color-brand-text)]">
      {managed.description}
    </p>
  </div>
)}

<div className="home-sheet px-4 pb-4">
  {/* The whole sheet is the bulletin, so its name goes at the top
      rather than over one section of it. */}
  <h2 className="home-sheet__title">{t("home.bulletin", language)}</h2>

  <HomeQuickLinks
    language={language}
    active={section}
    onSelect={setSection}
  />

  {/* One section at a time, and it fades. Keeping all three on the
      page is what sent Mass and History below the fold in the
      first place; swapping them means the answer to the tap is
      the only thing under the buttons. The key restarts the fade
      so the change is visible rather than silent. */}
  <div key={section} className="home-panel">
  {section === "ministries" && (
    <section className="home-section">
      <MinistrySacramentPair language={language} onNavigate={onNavigate} parishId={parish.id} />
    </section>
  )}

  {section === "mass" && (
  <div className="home-motif home-motif--calendar">
  <div data-spotlight="mass-schedule">
  <MassScheduleCard
    routeId={parish.id}
    now={now}
  />
  </div>

  {/* What the day commemorates — the parish's own words if they have
      written any, otherwise the named solemnity or feast the calendar
      supplies.

      Shown only when there is something to say. Falling back to the
      liturgical day NAME printed the Mass card's own title a second
      time, word for word, which is noise rather than information; on an
      ordinary Wednesday with no feast the card simply is not there. */}
  {commemorates && (
```

```
  <div className="mt-3 rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-5">
    <h4 className="text-[19px] font-bold italic text-[var(--color-brand-text)]">
      {t("home.commemorates", language)}
    </h4>
    <p className="mt-1.5 text-[16px] leading-relaxed text-[var(--color-brand-secondary)]">
      {commemorates}
    </p>
  </div>
)}

</div>
)}

{section === "history" && (
<div className="home-motif home-motif--cross">
<div data-spotlight="church-info">
<ChurchHistoryCard
  routeId={parish.id}
  parishName={parishName}
  onOpenHistory={() => onNavigate("history")}
/>
</div>
</div>
)}
</div>

{/* The parish's standing notices. Not one of the three choices —
    they are not alternatives to a Mass schedule — so they sit
    below whatever is chosen and never go away. */}
<div className="home-motif home-motif--pin">

{/* The bulletin reads top to bottom: the verse, then what the
    parish has announced, then the two ways in. The verse is the
    thing that changes every day, so it leads and stays put —
    inside the old rail it was one card among four and had to be
    swiped away to reach the others. */}
<div className="home-verse">
  <VerseCard />
</div>

{/* Scoped to the parish being shown, not to the pilgrim's own.
    Searching your way onto San Roque's dashboard and reading Mary
    Help's announcements there was the bug this closes. The legacy
    diocese-wide ones - the documents with no churchId at all -
    still appear everywhere; see lib/announcements.ts.

    Published only - a draft the parish is still writing, and
    anything they have archived, must never reach a home screen. */}
<BulletinRail
  announcements={publishedOnly(announcementsForParish(announcements, parish.id))}
  onNavigate={onNavigate}
/>

</div>


{/* The Verse of the Day card now lives on Pray, at the client's
    request. It was the last thing on a long Home scroll, where it
    sat under the church history and was rarely reached; on Pray it
```

```
            is beside the day's mysteries, which is the screen a pilgrim
            opens to read something. verses.ts is unchanged and is now
            imported by PrayScreen instead. */}
      </div>
    </div>
  );
}
```

1.2.15 APP-020 — DioceseMap.tsx

Code ID	APP-020
File Name	DioceseMap.tsx
File Path	SanctDemo/src/components/DioceseMap.tsx
Language / Type	TypeScript (React, TSX)
Lines	200
Functions / Classes	ChurchGlyph (internal function); shortLabel (internal function); wrapLabel (internal function); PinLabel (internal function); DioceseMap (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/project, ../data
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/components/DioceseMap.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — DioceseMap.tsx

```
import { usePresence } from "../context/PresenceContext";
import { projectToMap, DIOCESE_BOUNDS } from "../lib/project";
import { ROUTES } from "../data";

const WIDTH = 300;
const HEIGHT = 400;

interface DioceseMapProps {
  onSelectParish: (parishId: string) => void;
}

// A tiny hand-drawn church glyph: a square body, a triangular roof, and a
// cross above it. `scale` shrinks the whole glyph for coming-soon pins. No
// fill/stroke are set on the children — they inherit from the parent <g>'s
// class (`.dmap__pin--live` / `.dmap__pin--soon`), which is how this stays
// token-only without repeating colours per shape.
function ChurchGlyph({ scale = 1 }: { scale?: number }) {
  return (
    <>
      <rect x={-5 * scale} y={-2 * scale} width={10 * scale} height={8 * scale} rx={1 * scale} />
      <polygon points={`${-6 * scale},${-2 * scale} 0,${-10 * scale} ${6 * scale},${-2 * scale}`} />
      <line x1="0" y1={-10 * scale} x2="0" y2={-14 * scale} strokeWidth={1.5 * scale} />
      <line
        x1={-2.5 * scale}
        y1={-12 * scale}
        x2={2.5 * scale}
        y2={-12 * scale}
        strokeWidth={1.5 * scale}
      />
    </>
```

```
  );
}

// Drop trailing " Guide"/" Tour"/" Parish" wording so labels fit the
// 300-unit-wide frame — matches the trimming App.tsx already does elsewhere
// for the same names.
function shortLabel(name: string): string {
  return name.replace(/ Guide$/, "").replace(/ Tour$/, "").replace(/ Parish$/, "");
}

// The map is displayed inside a height-capped card, so the SVG is scaled
// down from its 300x400 viewBox on screen. Text set in the SVG's own
// coordinate system scales down with it, so to keep the *rendered* text at
// or above the 14px floor at the narrowest supported viewport, the nominal
// font-size (set in index.css) is inflated well past 14. A single line of a
// full parish name no longer fits at that size, so names wrap onto at most
// two lines instead of shrinking below the floor.
const LINE_HEIGHT = 24;

// Split a label into at most two roughly-balanced lines, breaking on the
// word boundary that minimises the length difference between the lines.
function wrapLabel(name: string): string[] {
  const clean = shortLabel(name);
  const words = clean.split(" ");
  if (words.length <= 1) return [clean];

  let bestSplit = 1;
  let bestDiff = Infinity;
  for (let i = 1; i < words.length; i++) {
    const line1 = words.slice(0, i).join(" ");
    const line2 = words.slice(i).join(" ");
    const diff = Math.abs(line1.length - line2.length);
    if (diff < bestDiff) {
      bestDiff = diff;
      bestSplit = i;
    }
  }
  return [words.slice(0, bestSplit).join(" "), words.slice(bestSplit).join(" ")];
}

// Renders a (possibly two-line) pin label. `anchorAbove` stacks the lines
// upward from just above the pin instead of downward from just below it,
// which is how nearby live pins avoid colliding.
function PinLabel({
  x,
  y,
  name,
  anchorAbove = false,
}: {
  x: number;
  y: number;
  name: string;
  anchorAbove?: boolean;
}) {
  const lines = wrapLabel(name);
  const startY = anchorAbove ? y - 18 - LINE_HEIGHT * (lines.length - 1) : y + 26;
  return (
    <text x={x} y={startY} className="dmap__pin-label">
      {lines.map((line, i) => (
```

```
        <tspan key={i} x={x} dy={i === 0 ? 0 : LINE_HEIGHT}>
          {line}
        </tspan>
      ))}
    </text>
  );
}

// The diocese-level map shown before a parish is chosen. A geographically
// calibrated illustration (not a street map — see the plan for why), with
// both parishes plotted at their real projected positions and the pilgrim's
// own position when known. This is a different feature from `MapTab`, which
// is a walking tour of stations *inside* one already-selected church.
export default function DioceseMap({ onSelectParish }: DioceseMapProps) {
  const { position } = usePresence();

  const live = ROUTES.filter(r => r.status === "live");
  const comingSoon = ROUTES.filter(r => r.status === "coming_soon");

  const you = projectToMap(position, DIOCESE_BOUNDS, WIDTH, HEIGHT);

  return (
    <svg
      viewBox={`0 0 ${WIDTH} ${HEIGHT}`}
      preserveAspectRatio="xMidYMid meet"
      role="img"
      className="dmap rounded-2xl"
    >
      <title>Map of the Diocese of Kalookan</title>

      {/* Manila Bay along the west edge */}
      <rect x="0" y="0" width="58" height="400" rx="28" ry="60" className="dmap__water" />

      {/* Three land areas — suggestive shapes, not a traced coastline */}
      <rect x="55" y="15" width="200" height="195" rx="55" className="dmap__land" />
      <rect x="38" y="225" width="130" height="175" rx="50" className="dmap__land" />
      <rect x="150" y="70" width="145" height="320" rx="45" className="dmap__land" />

      {/* Tullahan river, dividing Malabon from Caloocan */}
      <path
        d="M148,10 Q168,90 142,170 Q116,250 152,330 Q176,375 166,400"
        className="dmap__river"
        strokeWidth="3"
      />

      {/* A few arterial roads, for orientation only */}
      <path d="M60,340 L280,120" className="dmap__road" strokeWidth="2" />
      <path d="M70,55 L235,375" className="dmap__road" strokeWidth="2" />
      <path d="M20,150 L292,210" className="dmap__road" strokeWidth="2" />

      <text x="118" y="48" className="dmap__area-label">MALABON</text>
      {/* Nudged up from the land shape's visual centre so it clears Mary
          Help of Christians' pin label below it — measured with getBBox()
          (see Task 9 report). */}
      {/* Moved 298 -> 248 after Mary Help's coordinates were corrected to the
          diocese's real value, which shifted its pin label up into this one.
          Measured collision was 34.9 x 20.4 units. */}
      <text x="103" y="248" className="dmap__area-label">NAVOTAS</text>
      <text x="225" y="100" className="dmap__area-label">CALOOCAN</text>
```

```
      {live.map((route, i) => {
        if (!route.coordinates) return null;
        const p = projectToMap(route.coordinates, DIOCESE_BOUNDS, WIDTH, HEIGHT);
        if (!p) return null;
        // Alternate the label above/below the pin so nearby live pins never
        // collide with each other.
        const anchorAbove = i % 2 === 1;
        return (
          <g
            key={route.id}
            className="dmap__hit"
            role="button"
            tabIndex={0}
            aria-label={`Open ${shortLabel(route.name)}`}
            onClick={() => onSelectParish(route.id)}
            onKeyDown={e => {
              if (e.key === "Enter" || e.key === " ") {
                e.preventDefault();
                onSelectParish(route.id);
              }
            }}
          >
            <g transform={`translate(${p.x},${p.y})`} className="dmap__pin--live">
              <ChurchGlyph scale={1} />
            </g>
            <PinLabel x={p.x} y={p.y} name={route.name} anchorAbove={anchorAbove} />
          </g>
        );
      })}

      {comingSoon.map(route => {
        if (!route.coordinates) return null;
        const p = projectToMap(route.coordinates, DIOCESE_BOUNDS, WIDTH, HEIGHT);
        if (!p) return null;
        return (
          <g key={route.id} transform={`translate(${p.x},${p.y})`} className="dmap__pin--soon">
            <title>{shortLabel(route.name)}</title>
            <ChurchGlyph scale={0.7} />
          </g>
        );
      })}

      {you && (
        <g>
          <circle cx={you.x} cy={you.y} r="8" className="dmap__you-ring" />
          <circle cx={you.x} cy={you.y} r="5" className="dmap__you" />
        </g>
      )}
    </svg>
  );
}
```

DioceseMapLive.tsx — listed in full as MAP-001.

1.2.16 APP-021 — EditProfileCard.tsx

Code ID	APP-021
File Name	EditProfileCard.tsx
File Path	SanctDemo/src/components/EditProfileCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	234
Functions / Classes	EditProfileCard (exported component/function (default))
Related Files (its imports)	../lib/firebase, ../lib/userProfile, ../lib/avatar, ../types
Where It Is Used	src/components/MeTab.tsx
How to Recognize It	Open SanctDemo/src/components/EditProfileCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — EditProfileCard.tsx

```
import { useEffect, useRef, useState } from "react";
import { Camera, Check, Loader2, Pencil, Trash2, X } from "lucide-react";
import { auth } from "../lib/firebase";
import { getProfile, updateProfile } from "../lib/userProfile";
import { fileToAvatarDataUrl, initialsFor, AvatarError } from "../lib/avatar";
import type { UserProfile } from "../types";

/**
 * Your own profile: the picture and the name the app calls you.
 *
 * ## Why only these two
 *
 * They are the two things that are yours to decide. The full name on your
 * applications is what the parish office records, your parish cannot be
 * changed once chosen, and your email is your sign-in - each of those is
 * either someone else's to change or a different conversation. Putting
 * them in the same card as "what should we call you" would invite edits
 * the rules will refuse.
 *
 * ## Why the picture is applied immediately and the name is not
 *
 * Choosing a photograph is already a deliberate act with a file picker in
 * front of it; making someone then press Save is a second confirmation of
 * something they just confirmed. Typing is different - a half-typed name
 * is not an intention - so the nickname waits for Save.
 */
export default function EditProfileCard({
  isLoggedIn,
  onNicknameChange,
  onPhotoChange,
```

```
}: {
  isLoggedIn: boolean;
  /** Lets Home's greeting update without waiting for a reload. */
  onNicknameChange?: (nickname: string) => void;
  /** Keeps the profile header's avatar in step with this card's. */
  onPhotoChange?: (photoUrl: string) => void;
}) {
  const [profile, setProfile] = useState<UserProfile | null>(null);
  const [editing, setEditing] = useState(false);
  const [draft, setDraft] = useState("");
  const [busy, setBusy] = useState<"photo" | "name" | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState(false);
  const fileRef = useRef<HTMLInputElement | null>(null);

  useEffect(() => {
    if (!isLoggedIn || !auth.currentUser) { setProfile(null); return; }
    let live = true;
    void getProfile(auth.currentUser.uid).then(p => { if (live) setProfile(p); });
    return () => { live = false; };
  }, [isLoggedIn]);

  if (!isLoggedIn || !profile) return null;

  const shownName = profile.nickname?.trim() || profile.fullName || "Pilgrim";

  async function choosePhoto(file: File) {
    setBusy("photo");
    setError(null);
    try {
      const photoUrl = await fileToAvatarDataUrl(file);
      await updateProfile(profile!.uid, { photoUrl });
      setProfile(p => (p ? { ...p, photoUrl } : p));
      onPhotoChange?.(photoUrl);
    } catch (err) {
      setError(err instanceof AvatarError ? err.message : "That photo could not be saved.");
    } finally {
      setBusy(null);
    }
  }

  async function removePhoto() {
    setBusy("photo");
    try {
      await updateProfile(profile!.uid, { photoUrl: "" });
      setProfile(p => (p ? { ...p, photoUrl: "" } : p));
      onPhotoChange?.("");
    } finally {
      setBusy(null);
    }
  }

  async function saveNickname() {
    const next = draft.trim().slice(0, 24);
    setBusy("name");
    setError(null);
    try {
      await updateProfile(profile!.uid, { nickname: next });
      setProfile(p => (p ? { ...p, nickname: next } : p));
```

```
      onNicknameChange?.(next);
      setEditing(false);
      setSaved(true);
      window.setTimeout(() => setSaved(false), 2200);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(null);
    }
  }

  return (
    <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5">
      <h4 className="mb-3 text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-sans">
        My Profile
      </h4>

      {error && (
        <p role="alert" className="mb-3 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-3.5 py-2.5 text-[14px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}

      <div className="flex items-center gap-4">
        <div className="relative shrink-0">
          {profile.photoUrl ? (
            <img
              src={profile.photoUrl}
              alt=""
              className="w-[72px] h-[72px] rounded-full object-cover border border-[var(--color-brand-border)]"
            />
          ) : (
            <span
              aria-hidden
              className="w-[72px] h-[72px] rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] flex items-center justify-center text-
[24px] font-bold"
            >
              {initialsFor(shownName)}
            </span>
          )}

          <button
            type="button"
            onClick={() => fileRef.current?.click()}
            disabled={busy === "photo"}
            aria-label={profile.photoUrl ? "Change your photo" : "Add a photo"}
            className="absolute -bottom-1 -right-1 w-8 h-8 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] border-2 border-[var(--
color-brand-card-sunk)] flex items-center justify-center"
          >
            {busy === "photo" ? <Loader2 className="w-4 h-4 animate-spin" /> : <Camera className="w-4 h-4" />}
          </button>

          <input
            ref={fileRef}
            type="file"
            accept="image/*"
            className="hidden"
            onChange={e => {
              const file = e.target.files?.[0];
```

```
              // Cleared so choosing the same file twice still fires.
              e.target.value = "";
              if (file) void choosePhoto(file);
            }}
          />
        </div>

        <div className="min-w-0 flex-1">
          {editing ? (
            <>
              <label htmlFor="nick" className="block text-[13px] font-bold text-[var(--color-brand-secondary)] mb-1">
                Nickname
              </label>
              <input
                id="nick"
                value={draft}
                maxLength={24}
                autoFocus
                onChange={e => setDraft(e.target.value)}
                onKeyDown={e => {
                  if (e.key === "Enter") void saveNickname();
                  if (e.key === "Escape") setEditing(false);
                }}
                className="w-full rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-3 py-2 text-[16px]"
              />
              <div className="mt-2 flex gap-2">
                <button
                  type="button"
                  onClick={() => void saveNickname()}
                  disabled={busy === "name"}
                  className="inline-flex items-center gap-1.5 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-3.5 py-2 text-
[14px] font-bold disabled:opacity-50"
                >
                  {busy === "name" ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Check className="w-3.5 h-3.5" />}
                  Save
                </button>
                <button
                  type="button"
                  onClick={() => setEditing(false)}
                  className="inline-flex items-center gap-1.5 rounded-full border border-[var(--color-brand-border)] px-3.5 py-2 text-[14px] font-bold text-[var(--
color-brand-secondary)]"
                >
                  <X className="w-3.5 h-3.5" /> Cancel
                </button>
                {/* Removing the photo is an edit, so it lives in the
                    editing state. On the resting card it sat beside the
                    name as though it were as ordinary as reading it. */}
                {profile.photoUrl && (
                  <button
                    type="button"
                    onClick={() => void removePhoto()}
                    className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-error)]"
                  >
                    <Trash2 className="w-3.5 h-3.5" /> Remove photo
                  </button>
                )}
              </div>
            </>
          ) : (
```

```
            <>
              <p className="text-[20px] font-bold text-[var(--color-brand-text)] truncate">{shownName}</p>
              <p className="text-[14px] text-[var(--color-brand-secondary)] truncate">{profile.email}</p>
              <div className="mt-1.5 flex flex-wrap gap-2">
                <button
                  type="button"
                  onClick={() => { setDraft(profile.nickname ?? ""); setEditing(true); }}
                  className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-primary)]"
                >
                  <Pencil className="w-3.5 h-3.5" /> Edit nickname
                </button>

              </div>
            </>
          )}
        </div>
      </div>

      {saved && (
        <p role="status" className="mt-3 text-[14px] text-[var(--color-brand-success)]">
          Saved. Your home screen will greet you by this name.
        </p>
      )}

      <p className="mt-3 text-[13px] leading-relaxed text-[var(--color-brand-secondary)]">
        Your nickname is what the app calls you. The parish still records applications
        under your full name, {profile.fullName || "—"}.
      </p>
    </div>
  );
}
```

1.2.17 APP-022 — ExploreTab.tsx

Code ID	APP-022
File Name	ExploreTab.tsx
File Path	SanctDemo/src/components/ExploreTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	156
Functions / Classes	ExploreTab (exported component/function (default))
Related Files (its imports)	../types, ../data
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ExploreTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ExploreTab.tsx

```
import React, { useState } from "react";
import { Route } from "../types";
import { ROUTES } from "../data";
import { Search, MapPin, Compass, Play, Star, ChevronRight, Sparkles } from "lucide-react";

interface ExploreTabProps {
  onSelectRoute: (routeId: string) => void;
}

export default function ExploreTab({ onSelectRoute }: ExploreTabProps) {
  const [searchQuery, setSearchQuery] = useState("");
  const [selectedCategory, setSelectedCategory] = useState<string>("All");

  const categories = ["All", "Historic Heritage", "Spiritual Retreat", "Marian Church"];

  const filteredRoutes = ROUTES.filter((route) => {
    const matchesSearch =
      route.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
      route.location.toLowerCase().includes(searchQuery.toLowerCase());
    const matchesCategory = selectedCategory === "All" || route.category === selectedCategory;
    return matchesSearch && matchesCategory;
  });

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] pb-20">
      {/* Header Banner */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden select-none border-b border-[var(--
color-brand-border)]">
        <div className="absolute right-0 top-0 opacity-10 translate-x-4 -translate-y-4">
          <Compass className="w-32 h-32 text-white" />
        </div>
        <div className="relative z-10 space-y-3">
          <div className="flex items-center gap-1.5 text-[var(--color-brand-on-accent)] font-bold text-[15px] tracking-wider uppercase font-serif italic">
```

```
      <Sparkles className="w-3.5 h-3.5" /> Philippines Heritage Guide
    </div>
    <div>
      <h2 className="text-2xl font-bold font-serif italic tracking-tight">
        Begin Your Walk
      </h2>
      <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-90 mt-1 max-w-xs leading-relaxed font-sans">
        Rediscover historical churches, scenic trails, and wellness milestones on foot.
      </p>
    </div>
  </div>
</div>

<div className="px-4 -mt-5">
  {/* Search Bar Input */}
  <div className="bg-[var(--color-brand-card)] rounded-xl shadow-xs border border-[var(--color-brand-border)] p-2 flex items-center gap-2">
    <Search className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0 ml-1" />
    <input
      id="search-routes"
      type="text"
      placeholder="Search historic trails, cities..."
      value={searchQuery}
      onChange={(e) => setSearchQuery(e.target.value)}
      className="w-full bg-transparent text-[15px] text-[var(--color-brand-text)] outline-none placeholder-[var(--color-brand-secondary)]"
    />
  </div>
</div>

{/* Category Pills */}
<div className="px-4 mt-5">
  <div className="flex items-center gap-2 overflow-x-auto pb-1 no-scrollbar">
    {categories.map((cat) => (
      <button
        key={cat}
        onClick={() => setSelectedCategory(cat)}
        className={`text-[15px] px-3.5 py-1.5 rounded-full font-bold uppercase tracking-wider shrink-0 transition-all border ${
          selectedCategory === cat
            ? "bg-[var(--color-brand-primary)] text-white border-[var(--color-brand-primary)] shadow-xs"
            : "bg-[var(--color-brand-card)] text-[var(--color-brand-secondary)] border-[var(--color-brand-border)] hover:bg-[var(--color-brand-border)]/45"
        }`}
      >
        {cat}
      </button>
    ))}
  </div>
</div>

{/* Trail Cards */}
<div className="px-4 mt-5 space-y-4 flex-1">
  <div className="flex items-center justify-between">
    <h3 className="text-[15px] font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
      Recommended Routes ({filteredRoutes.length})
    </h3>
  </div>

  {filteredRoutes.length > 0 ? (
    filteredRoutes.map((route) => (
      <div
        key={route.id}
```

```
              onClick={() => onSelectRoute(route.id)}
              className="bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] overflow-hidden shadow-xs hover:shadow-sm hover:border-
[var(--color-brand-primary)] transition-all cursor-pointer group flex flex-col"
            >
              {/* Image banner */}
              <div className="relative h-32 bg-[var(--color-brand-card)] overflow-hidden">
                <img
                  src={route.stations[0]?.imageUrl || "https://dioceseofkalookan.ph/wp-content/uploads/2020/12/maryhelpPic1-1-1024x680.jpg"}
                  alt={route.name}
                  className="w-full h-full object-cover group-hover:scale-105 transition-transform duration-500"
                  referrerPolicy="no-referrer"
                />
                <div className="absolute top-2.5 left-2.5 bg-black/70 backdrop-blur-xs px-2.5 py-0.5 rounded-full text-sm font-bold text-white uppercase tracking-
wider">
                  {route.category}
                </div>
                <div className="absolute bottom-2 left-2.5 flex items-center gap-1 bg-[var(--color-brand-primary)]/95 backdrop-blur-xs text-sm text-white px-2.5 py-
0.5 rounded-full font-medium border border-[var(--color-brand-border)]/40">
                  <MapPin className="w-3 h-3 text-[var(--color-brand-gold)]" /> {route.location}
                </div>
              </div>

              {/* Card Body */}
              <div className="p-4 space-y-2 flex-1 flex flex-col justify-between">
                <div>
                  <h4 className="font-bold text-sm text-[var(--color-brand-text)] font-serif italic group-hover:text-[var(--color-brand-secondary)] transition-colors
leading-tight">
                    {route.name}
                  </h4>
                  <p className="text-[15px] text-[var(--color-brand-secondary)] line-clamp-2 mt-1 leading-normal font-sans">
                    {route.description}
                  </p>
                </div>

                {/* Trail Details */}
                <div className="flex items-center justify-between border-t border-[var(--color-brand-border)]/40 pt-2.5 mt-2 text-sm text-[var(--color-brand-
secondary)] font-mono">
                  <div className="flex gap-4">
                    <div>
                      <span className="block text-[var(--color-brand-secondary)] uppercase text-sm font-bold">Distance</span>
                      <strong className="text-[var(--color-brand-text)] text-[15px] font-semibold">{route.distanceKm} km</strong>
                    </div>
                    <div>
                      <span className="block text-[var(--color-brand-secondary)] uppercase text-sm font-bold">Duration</span>
                      <strong className="text-[var(--color-brand-text)] text-[15px] font-semibold">{route.durationMins} mins</strong>
                    </div>
                    <div>
                      <span className="block text-[var(--color-brand-secondary)] uppercase text-sm font-bold">Difficulty</span>
                      <span className={`inline-block text-sm font-bold ${
                        route.difficulty === "Easy" ? "text-emerald-700" : "text-[var(--color-brand-gold)]"
                      }`}>{route.difficulty}</span>
                    </div>
                  </div>

                  <button className="h-7 w-7 rounded-full bg-[var(--color-brand-card)] text-[var(--color-brand-secondary)] flex items-center justify-center border
border-[var(--color-brand-border)] hover:bg-[var(--color-brand-primary)] hover:text-white hover:border-[var(--color-brand-primary)] transition-colors group-
hover:scale-105">
                    <Play className="w-3.5 h-3.5 fill-current ml-0.5" />
                  </button>
```

```
              </div>
            </div>
          </div>
        ))
      ) : (
        <div className="p-8 text-center bg-[var(--color-brand-card)] rounded-xl border border-[var(--color-brand-border)] shadow-xs">
          <p className="text-[15px] text-[var(--color-brand-secondary)]">No walking routes found for your filters.</p>
        </div>
      )}
    </div>
  </div>
 );
}
```

1.2.18 APP-023 — HistoryQuiz.tsx

Code ID	APP-023
File Name	HistoryQuiz.tsx
File Path	SanctDemo/src/components/HistoryQuiz.tsx
Language / Type	TypeScript (React, TSX)
Lines	200
Functions / Classes	HistoryQuiz (exported component/function (default)); Question (internal function)
Related Files (its imports)	../lib/historyQuiz
Where It Is Used	src/components/ChurchHistory.tsx
How to Recognize It	Open SanctDemo/src/components/HistoryQuiz.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — HistoryQuiz.tsx

```
import { useMemo, useState } from "react";
import { Check, RefreshCw, X } from "lucide-react";
import { gradeQuiz, quizFor, unanswered, type QuizItem } from "../lib/historyQuiz";

/**
 * Five questions at the foot of a parish's history.
 *
 * ## Why it is at the very bottom
 *
 * The history is the point of the page and the quiz is a way of
 * looking back at it. Put between the prose and the stations it would
 * interrupt the reading to test it, which is the wrong way round: a
 * visitor should meet the questions having already read the answers.
 *
 * ## Why nothing is scored until Submit
 *
 * Marking each answer as it is chosen turns five questions into five
 * verdicts, and somebody who gets the first wrong stops. One submission
 * gives a score and then the explanations, which is where the teaching
 * is - the explanation says what the history page said, so a wrong
 * answer sends the reader back up the page rather than away.
 *
 * ## Where there is no quiz
 *
 * Only two parishes have a written history. For the rest this renders
 * nothing at all - no heading, no empty panel. Questions invented to
 * fill the space would be the one thing this app refuses everywhere.
 */
export default function HistoryQuiz({ routeId }: { routeId: string }) {
  const quiz = useMemo(() => quizFor(routeId), [routeId]);
  /**
   * Closed until asked for.
   *
```

```
 * Five questions printed open under the history made the page end in
 * a test rather than in its history, and pushed the last of the
 * prose a long way up the screen. Closed, the section is one line
 * and a button, and the reader decides whether the page ends in a
 * quiz.
 */
const [open, setOpen] = useState(false);
const [answers, setAnswers] = useState<(number | null)[]>([]);
const [submitted, setSubmitted] = useState(false);
const [missing, setMissing] = useState<number[]>([]);

if (!quiz) return null;

const result = submitted ? gradeQuiz(quiz, answers) : null;

function choose(qIndex: number, option: number) {
  if (submitted) return;
  setAnswers(prev => {
    const next = [...prev];
    next[qIndex] = option;
    return next;
  });
  // Clear the "you missed these" note as soon as it stops being true.
  setMissing(prev => prev.filter(n => n !== qIndex + 1));
}

function submit() {
  const blank = unanswered(answers, quiz!.length);
  if (blank.length > 0) {
    // Named, not just counted. "Answer all five" leaves the reader
    // hunting for which one they skipped.
    setMissing(blank);
    return;
  }
  setMissing([]);
  setSubmitted(true);
}

function tryAgain() {
  setAnswers([]);
  setSubmitted(false);
  setMissing([]);
  // Stays open. Asking to try again and being handed the button
  // again would be a tap for nothing.
}

return (
  <section className="history-quiz" aria-labelledby="history-quiz-title">
    <h2 id="history-quiz-title" className="history-quiz__title">
      Test Sancti Knowledge
    </h2>
    <p className="history-quiz__lede">
      Five questions about this parish. Every answer is somewhere on
      this page.
    </p>

    {!open && (
      <button
        type="button"
```

```
          onClick={() => setOpen(true)}
          aria-expanded={false}
          className="history-quiz__start"
        >
          Start the quiz
        </button>
      )}

      {open && quiz.map((item, qi) => (
        <Question
          key={item.question}
          item={item}
          index={qi}
          chosen={answers[qi] ?? null}
          correct={result ? result.marks[qi] : null}
          revealed={submitted}
          flagged={missing.includes(qi + 1)}
          onChoose={option => choose(qi, option)}
        />
      ))}

      {open && !submitted && (
        <>
          {missing.length > 0 && (
            <p role="alert" className="history-quiz__missing">
              {missing.length === 1
                ? `Question ${missing[0]} has not been answered yet.`
                : `Questions ${missing.join(", ")} have not been answered yet.`}
            </p>
          )}
          <button type="button" onClick={submit} className="history-quiz__submit">
            Submit quiz
          </button>
        </>
      )}

      {open && submitted && result && (
        <div className="history-quiz__result" role="status">
          <p className="history-quiz__score">
            {result.score} out of {result.total}
          </p>
          <p className="history-quiz__verdict">
            {result.score === result.total
              ? "Every one correct. You read it well."
              : result.score === 0
                ? "The answers are all on this page — the explanations say where."
                : "The explanations above say where each answer came from."}
          </p>
          <button type="button" onClick={tryAgain} className="history-quiz__again">
            <RefreshCw className="w-4 h-4" aria-hidden /> Try again
          </button>
        </div>
      )}
    </section>
  );
}

function Question({
  item, index, chosen, correct, revealed, flagged, onChoose,
```

```
}: {
  item: QuizItem;
  index: number;
  chosen: number | null;
  correct: boolean | null;
  revealed: boolean;
  flagged: boolean;
  onChoose: (option: number) => void;
}) {
  return (
    <div className={`history-quiz__q${flagged ? " is-flagged" : ""}`}>
      <p className="history-quiz__question">
        <span className="history-quiz__number">{index + 1}.</span> {item.question}
      </p>

      <div role="radiogroup" aria-label={item.question} className="history-quiz__options">
        {item.options.map((option, oi) => {
          const picked = chosen === oi;
          const isAnswer = revealed && oi === item.correct;
          const isWrongPick = revealed && picked && oi !== item.correct;
          return (
            <button
              key={option}
              type="button"
              role="radio"
              aria-checked={picked}
              disabled={revealed}
              onClick={() => onChoose(oi)}
              className={`history-quiz__option${picked ? " is-picked" : ""}${
                isAnswer ? " is-answer" : ""}${isWrongPick ? " is-wrong" : ""}`}
            >
              <span className="history-quiz__mark" aria-hidden>
                {isAnswer ? <Check className="w-4 h-4" />
                  : isWrongPick ? <X className="w-4 h-4" /> : null}
              </span>
              <span>{option}</span>
            </button>
          );
        })}
      </div>

      {revealed && (
        <p className={`history-quiz__explain${correct ? " is-right" : ""}`}>
          {item.explanation}
        </p>
      )}
    </div>
  );
}
```

1.2.19 APP-024 — HomeHero.tsx

Code ID	APP-024
File Name	HomeHero.tsx
File Path	SanctDemo/src/components/HomeHero.tsx
Language / Type	TypeScript (React, TSX)
Lines	211
Functions / Classes	HomeHero (exported component/function (default)); relativeMass (internal function); Stat (internal function)
Related Files (its imports)	../context/PresenceContext, ../lib/geo, ../lib/routing, ../data, ../lib/schedule, ../data/diocese-parishes.json, ../lib/parishIds
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/HomeHero.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — HomeHero.tsx

```
import { useEffect, useState, type ReactNode } from "react";
import { Footprints, Heart, Ruler, Clock } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import { haversineMeters, type Coordinates } from "../lib/geo";
import { fetchWalkingRoute, formatDistance, formatWalkingMinutes } from "../lib/routing";
import { MASS_SCHEDULES } from "../data";
import { nextMass } from "../lib/schedule";
import parishData from "../data/diocese-parishes.json";
import { routeIdForParish } from "../lib/parishIds";

interface DioceseParish {
  id: string;
  name: string;
  vicariate: string;
  coordinates: Coordinates;
  status: string;
}

const PARISHES = (parishData as { parishes: DioceseParish[] }).parishes;

interface HomeHeroProps {
  /** Opens the Map tab and draws the walking route to this parish. */
  onWalkThere: (parishId: string) => void;
  /** Opens a parish's own page, for the two that have one. */
  onOpenParish: (routeId: string) => void;
}

/**
 * "Nearest to you now" — the redesign's Home hero.
 *
```

```
 * It features whichever of the 31 parishes is actually closest, not whichever
 * has the most content. That means the card frequently lands on a parish with
 * no Mass times collected yet, and when it does the Mass line and the
 * open/closed chip are simply absent rather than faked or filled with a
 * placeholder. A hero that invented a Mass time would send someone to a
 * locked church.
 */
export default function HomeHero({ onWalkThere, onOpenParish }: HomeHeroProps) {
  const { position, accuracyMeters, gpsStatus } = usePresence();
  const [route, setRoute] = useState<{ kind: string; distanceMeters: number; durationMinutes: number } | null>(null);

  const nearest = position
    ? PARISHES.reduce<{ parish: DioceseParish; metres: number } | null>((best, parish) => {
        const metres = haversineMeters(position, parish.coordinates);
        return !best || metres < best.metres ? { parish, metres } : best;
      }, null)
    : null;

  // One routing call, for the one parish being featured. The straight-line
  // distance shows immediately and is replaced when (or if) OSRM answers, so
  // the card is never blank waiting on the network.
  useEffect(() => {
    if (!position || !nearest) {
      setRoute(null);
      return;
    }
    let live = true;
    const target = nearest.parish.coordinates;
    void fetchWalkingRoute(position, target).then(result => {
      if (live) setRoute(result);
    });
    return () => {
      live = false;
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [position?.lat, position?.lng, nearest?.parish.id]);

  if (!position || !nearest) {
    return (
      <section className="rounded-[22px] bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] p-5">
        <p className="text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
          Nearest to you now
        </p>
        <p className="mt-3 text-[16px] leading-relaxed text-[var(--color-brand-text)]">
          {gpsStatus === "denied"
            ? "Location is switched off, so the app cannot tell which parish you are nearest. Turn it on to see distances and walking times."
            : gpsStatus === "unavailable"
              ? "This device cannot report a location, so distances and walking times are unavailable. Browse the parishes on the map instead."
              : "Finding your location…"}
        </p>
      </section>
    );
  }

  const { parish, metres } = nearest;
  const routeId = routeIdForParish(parish.id);
  const schedule = routeId ? MASS_SCHEDULES[routeId] : undefined;
  const upcoming = schedule?.schedule ? nextMass(schedule.schedule, new Date()) : null;
```

```
// OSRM's distance is trustworthy; its duration assumes a vehicle, so the
// walking figure always comes from the app's own 5 km/h (see routing.ts).
const shownMetres = route ? route.distanceMeters : metres;
const minutes = route ? route.durationMinutes : metres / (5000 / 60);
const routed = route?.kind === "routed";

return (
  <section className="rounded-[22px] bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] overflow-hidden">
    <div className="p-5">
      <div className="flex items-center justify-between gap-3">
        <p className="text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
          Nearest to you now
        </p>
        {/* The chip only exists when a real schedule backs it. */}
        {upcoming && (
          <span className="text-[14px] font-semibold px-2.5 py-1 rounded-md bg-[var(--color-brand-success)] text-[var(--color-brand-on-accent)]">
            Mass {relativeMass(upcoming.date, new Date())}
          </span>
        )}
      </div>

      <h2 className="mt-2 text-[24px] font-bold leading-tight tracking-tight text-[var(--color-brand-text)]">
        {parish.name}
      </h2>

      <div className="mt-4 grid grid-cols-3 gap-3">
        <Stat
          icon={<Ruler className="w-4 h-4" />}
          value={formatDistance(shownMetres)}
          label={routed ? "walking route" : "direct"}
        />
        <Stat
          icon={<Footprints className="w-4 h-4" />}
          value={formatWalkingMinutes(minutes)}
          label="on foot"
        />
        <Stat
          icon={<Clock className="w-4 h-4" />}
          value={upcoming ? upcoming.time : "—"}
          label={upcoming ? "next Mass" : "no times yet"}
        />
      </div>

      {/* Said plainly rather than hidden: this is the state most of the
          diocese is in, and naming it is what tells the field team which
          parish still needs visiting. */}
      {!schedule && (
        <p className="mt-3 text-[15px] leading-snug text-[var(--color-brand-secondary)]">
          Mass times for this parish have not been collected yet.
        </p>
      )}
      {schedule && !schedule.scheduleVerified && (
        <p className="mt-3 text-[15px] leading-snug text-[var(--color-brand-error)]">
          These Mass times are unconfirmed — check with the parish before travelling.
        </p>
      )}
      {accuracyMeters !== null && accuracyMeters > 60 && (
        <p className="mt-2 text-[15px] text-[var(--color-brand-secondary)]">
          Your position is only accurate to ±{Math.round(accuracyMeters)} m, so this distance is rough.
```

```
          </p>
        )}
      </div>

      <div className="flex items-stretch border-t border-[var(--color-brand-border)]">
        <button
          type="button"
          onClick={() => onWalkThere(parish.id)}
          className="flex-1 py-3.5 text-[16px] font-semibold bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)]"
        >
          Get directions
        </button>
        {routeId && (
          <button
            type="button"
            onClick={() => onOpenParish(routeId)}
            className="px-5 text-[16px] font-semibold text-[var(--color-brand-primary)] border-l border-[var(--color-brand-border)]"
          >
            Open
          </button>
        )}
        <button
          type="button"
          aria-label="Save this parish"
          className="px-5 text-[var(--color-brand-secondary)] border-l border-[var(--color-brand-border)]"
        >
          <Heart className="w-5 h-5" />
        </button>
      </div>
    </section>
  );
}

/**
 * "in 40 minutes" / "in 3 hours" / "tomorrow 6:00 AM" / "Friday".
 *
 * The mockup's chip reads "Mass in 3 hours", which only means anything if it
 * is counted from now — a fixed string would be wrong within the hour.
 */
function relativeMass(when: Date, now: Date): string {
  const minutes = Math.round((when.getTime() - now.getTime()) / 60000);
  if (minutes <= 0) return "now";
  if (minutes < 60) return `in ${minutes} min`;
  const hours = Math.round(minutes / 60);
  if (hours < 12) return `in ${hours} hour${hours === 1 ? "" : "s"}`;
  const sameDay = when.toDateString() === now.toDateString();
  if (sameDay) return `at ${when.toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit" })}`;
  const tomorrow = new Date(now.getTime() + 86400000);
  if (when.toDateString() === tomorrow.toDateString()) return "tomorrow";
  return when.toLocaleDateString("en-US", { weekday: "long" });
}

function Stat({ icon, value, label }: { icon: ReactNode; value: string; label: string }) {
  return (
    <div>
      <div className="flex items-center gap-1.5 text-[var(--color-brand-primary)]">
        {icon}
        <span className="text-[20px] font-bold tabular-nums leading-none">{value}</span>
      </div>
```

```
      <p className="mt-1 text-[15px] text-[var(--color-brand-secondary)]">{label}</p>
    </div>
  );
}
```

1.2.20 APP-025 — HomeQuickLinks.tsx

Code ID	APP-025
File Name	HomeQuickLinks.tsx
File Path	SanctDemo/src/components/HomeQuickLinks.tsx
Language / Type	TypeScript (React, TSX)
Lines	86
Functions / Classes	HomeSection (exported type); HomeQuickLinks (exported component/function (default))
Related Files (its imports)	../lib/ui, ../lib/language
Where It Is Used	src/App.tsx, src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/HomeQuickLinks.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — HomeQuickLinks.tsx

```
import { BookOpen, Church, Users } from "lucide-react";
import { t } from "../lib/ui";
import type { Language } from "../lib/language";

/**
 * The three parts of the Parish Bulletin, as a chooser.
 *
 * ## Why it filters rather than scrolls
 *
 * It scrolled to the section for one version, and that was the wrong
 * answer to the right problem: the sections were below the fold, so a
 * smooth scroll moved the page and left the pilgrim to work out which
 * of the three things now on screen was the one they asked for.
 *
 * Choosing shows that section and only that section. Nothing has to be
 * scrolled past, and there is never a second section underneath to be
 * mistaken for the answer.
 *
 * ## Why the verse and the announcements are not in here
 *
 * They are not alternatives to each other - they are the parish's
 * standing notices, and they stay below whatever is chosen. The
 * chooser only governs the part that changes.
 */

export type HomeSection = "ministries" | "mass" | "history";

const SECTIONS: Array<{
  id: HomeSection;
  icon: React.ReactNode;
  label: { en: string; fil: string };
}> = [
  {
```

```
    id: "mass",
    icon: <Church className="w-[17px] h-[17px]" />,
    label: { en: "Mass\nSchedule", fil: "Oras ng\nMisa" },
  },
  {
    id: "history",
    icon: <BookOpen className="w-[17px] h-[17px]" />,
    label: { en: "History", fil: "Kasaysayan" },
  },
  {
    id: "ministries",
    icon: <Users className="w-[17px] h-[17px]" />,
    // One button for both: they sit side by side and a visitor who is
    // not sure which word covers their errand should not have to guess.
    label: { en: "Ministry &\nSacraments", fil: "Ministeryo at\nSakramento" },
  },
];

export default function HomeQuickLinks({
  language,
  active,
  onSelect,
}: {
  language: Language;
  active: HomeSection;
  onSelect: (section: HomeSection) => void;
}) {
  return (
    <div
      className="quick"
      role="tablist"
      aria-label={t("quick.heading", language)}
    >
      {SECTIONS.map(section => {
        const on = section.id === active;
        return (
          <button
            key={section.id}
            type="button"
            role="tab"
            aria-selected={on}
            onClick={() => onSelect(section.id)}
            data-spotlight={`home-${section.id}`}
            className={`quick__btn${on ? " is-on" : ""}`}
          >
            <span className="quick__icon">{section.icon}</span>
            <span className="quick__label">{section.label[language]}</span>
          </button>
        );
      })}
    </div>
  );
}
```

1.2.21 APP-026 — ItemDetailSections.tsx

Code ID	APP-026
File Name	ItemDetailSections.tsx
File Path	SanctDemo/src/components/ItemDetailSections.tsx
Language / Type	TypeScript (React, TSX)
Lines	112
Functions / Classes	ItemDetailSections (exported component/function (default)); Section (internal function); Bullets (internal function)
Related Files (its imports)	../lib/itemContent
Where It Is Used	src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/ItemDetailSections.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ItemDetailSections.tsx

```
import { Check, Clock, Info, ListOrdered, Phone } from "lucide-react";
import type { ResolvedItem } from "../lib/itemContent";

/**
 * The parts of a ministry or sacrament page beyond the description.
 *
 * ## Why every section can be absent
 *
 * The parish writes these; most have written none of them yet. A
 * heading over an empty list reads as a page that failed to load, so
 * each section renders only when there is something in it. A sacrament
 * page that is just a description and its requirements is a correct
 * page, not a broken one.
 *
 * ## Why cards and not one block of text
 *
 * Somebody reads this standing in a church porch deciding whether they
 * have the right papers. "Requirements" has to be findable by scanning,
 * which means it has to be its own thing with its own heading, not the
 * fourth paragraph.
 */
export default function ItemDetailSections({ item }: { item: ResolvedItem }) {
  return (
    <>
      {item.responsibilities.length > 0 && (
        <Section title="What members do" icon={<Check className="w-3.5 h-3.5" />}>
          <Bullets items={item.responsibilities} />
        </Section>
      )}

      {item.requirements.length > 0 && (
```

```
        <Section title="Requirements" icon={<Check className="w-3.5 h-3.5" />}>
          <ul className="space-y-1.5">
            {item.requirements.map(req => (
              <li key={req} className="flex gap-2 items-start text-[15px] leading-relaxed text-[var(--color-brand-text)]">
                <Check className="w-3.5 h-3.5 mt-1 shrink-0 text-[var(--color-brand-success)]" />
                <span>{req}</span>
              </li>
            ))}
          </ul>
        </Section>
      )}

      {item.schedule && (
        <Section title="Schedule" icon={<Clock className="w-3.5 h-3.5" />}>
          <p className="text-[15px] leading-relaxed text-[var(--color-brand-text)]">
            {item.schedule}
          </p>
        </Section>
      )}

      {item.process.length > 0 && (
        <Section title="How applying works" icon={<ListOrdered className="w-3.5 h-3.5" />}>
          <ol className="space-y-2">
            {item.process.map((step, i) => (
              <li key={step} className="flex gap-2.5 items-start text-[15px] leading-relaxed text-[var(--color-brand-text)]">
                <span
                  aria-hidden
                  className="mt-0.5 w-5 h-5 shrink-0 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] text-[12px] font-bold grid
place-items-center"
                >
                  {i + 1}
                </span>
                <span>{step}</span>
              </li>
            ))}
          </ol>
        </Section>
      )}

      {item.reminders.length > 0 && (
        <Section title="Important reminders" icon={<Info className="w-3.5 h-3.5" />}>
          <Bullets items={item.reminders} />
        </Section>
      )}

      {item.contact && (
        <Section title="Contact the parish" icon={<Phone className="w-3.5 h-3.5" />}>
          <p className="text-[15px] leading-relaxed text-[var(--color-brand-text)] break-words">
            {item.contact}
          </p>
        </Section>
      )}
    </>
  );
}

function Section({
  title, icon, children,
}: { title: string; icon: React.ReactNode; children: React.ReactNode }) {
```

```
  return (
    <section className="rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-4">
      <h5 className="mb-2 flex items-center gap-1.5 text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        <span className="text-[var(--color-brand-primary)]">{icon}</span>
        {title}
      </h5>
      {children}
    </section>
  );
}

function Bullets({ items }: { items: string[] }) {
  return (
    <ul className="space-y-1.5">
      {items.map(text => (
        <li key={text} className="flex gap-2 items-start text-[15px] leading-relaxed text-[var(--color-brand-text)]">
          <span aria-hidden className="mt-2 w-1.5 h-1.5 shrink-0 rounded-full bg-[var(--color-brand-secondary)]" />
          <span>{text}</span>
        </li>
      ))}
    </ul>
  );
}
```

Landing.tsx — listed in full as APP-005.

1.2.22 APP-027 — LoginModal.tsx

Code ID	APP-027
File Name	LoginModal.tsx
File Path	SanctDemo/src/components/LoginModal.tsx
Language / Type	TypeScript (React, TSX)
Lines	491
Functions / Classes	Field (internal function); LoginModal (exported component/function (default)); GoogleMark (internal function)
Related Files (its imports)	../lib/firebase, ../lib/googleAuth, ../lib/authFlow
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/LoginModal.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — LoginModal.tsx

```
import React, { useRef, useState } from "react";
import { ShieldAlert, CheckCircle, ShieldCheck, UserPlus } from "lucide-react";
import {
  signInWithEmailAndPassword,
  createUserWithEmailAndPassword,
  sendPasswordResetEmail,
  setPersistence,
  browserLocalPersistence,
  browserSessionPersistence,
} from "firebase/auth";
import { auth, db } from "../lib/firebase";
import { doc, setDoc, getDoc } from "firebase/firestore";
import { ensureGoogleProfile, signInWithGoogle } from "../lib/googleAuth";
import { friendlyAuthError } from "../lib/authFlow";

// Label above the box, which is how the reference layout does it: the
// caption sits on its own line in muted type and the field below carries
// nothing but the value. It replaces a label punched through the field's
// top border - that notch is fussy at this size, and it put the caption
// and its field on the same visual line, so a column of three of them
// read as one block rather than three labelled things.
function Field({
  label,
  type,
  value,
  onChange,
  placeholder,
  autoComplete,
}: {
```

```
  label: string;
  type: string;
  value: string;
  onChange: (value: string) => void;
  placeholder: string;
  autoComplete?: string;
}) {
  const id = `field-${label.toLowerCase().replace(/[^a-z]+/g, "-")}`;
  return (
    <div>
      <label
        htmlFor={id}
        className="block mb-1.5 text-[14px] font-semibold text-[var(--color-brand-text)]"
      >
        {label}
      </label>
      <input
        id={id}
        type={type}
        value={value}
        autoComplete={autoComplete}
        onChange={(e) => onChange(e.target.value)}
        placeholder={placeholder}
        className="w-full bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)] rounded-xl px-4 py-3.5 text-[16px] text-[var(--color-brand-
text)] placeholder:text-[var(--color-brand-secondary)]/70 outline-none transition-colors focus:border-[var(--color-brand-primary)] focus:bg-[var(--color-brand-
card)]"
      />
    </div>
  );
}

interface LoginModalProps {
  onLoginSuccess: (email: string, isAdmin: boolean) => void;
  /**
   * The parish the app is currently showing.
   *
   * Recorded as the parish of a profile created by Google sign-in.
   * Google supplies a name and an email and cannot know which parish
   * somebody belongs to, and the rules refuse an application from a
   * profile with none. Me > Change Parish moves it.
   */
  activeParishId?: string;
  onLogout: () => void;
  isLoggedIn: boolean;
  userEmail: string;
  isAdmin: boolean;
}

export default function LoginModal({ onLoginSuccess, onLogout, isLoggedIn, userEmail, isAdmin, activeParishId }: LoginModalProps) {
  const [activeMode, setActiveMode] = useState<"signin" | "signup">("signin");
  const [googleBusy, setGoogleBusy] = useState(false);

  /**
   * Sign in with a Google account.
   *
   * Kept apart from handleAuth because it shares none of its steps:
   * there is no password to validate, no verification email to send,
   * and the profile is written from what Google already knows rather
   * than from a form.
```

```
 */
async function handleGoogle() {
  setErrorMsg("");
  setSuccessMsg("");
  setGoogleBusy(true);
  try {
    const user = await signInWithGoogle();
    await ensureGoogleProfile(user, activeParishId ?? "");
    const adminSnap = await getDoc(doc(db, "admins", user.uid));
    onLoginSuccess(user.email ?? "", adminSnap.exists());
  } catch (err) {
    const code = (err as { code?: string })?.code ?? "";
    // Closing the picker is not a failure and must not be reported as
    // one; a red error for changing your mind is its own small insult.
    if (!code.includes("popup-closed") && !code.includes("cancelled")) {
      setErrorMsg(friendlyAuthError(err).message);
    }
  } finally {
    setGoogleBusy(false);
  }
}

const [email, setEmail] = useState("");
const [password, setPassword] = useState("");
const [fullName, setFullName] = useState("");
const [errorMsg, setErrorMsg] = useState("");
const [successMsg, setSuccessMsg] = useState("");
const [isLoading, setIsLoading] = useState(false);

// Switching mode adds or removes a field, so the form changes height
// under a button that has just taken focus - and the browser then
// scrolls to keep that button in view, which pushed the emblem and the
// heading off the top of the screen. Switching forms should show the top
// of the new one.
const scrollRef = useRef<HTMLDivElement | null>(null);

// Checked: the session survives closing the app. Unchecked: it ends with
// it. This is Firebase's own persistence setting, not a stored password.
const [rememberMe, setRememberMe] = useState(true);

const handleForgotPassword = async () => {
  setErrorMsg("");
  setSuccessMsg("");
  if (!email) {
    setErrorMsg("Enter your e-mail above first, and we will send a reset link to it.");
    return;
  }
  try {
    await sendPasswordResetEmail(auth, email);
    // Deliberately the same message whether or not the address is
    // registered: saying "no such account" would tell anyone who asks
    // which e-mails have accounts here.
    setSuccessMsg(`If ${email} has an account, a password reset link is on its way.`);
  } catch (error: any) {
    console.error("Password reset error:", error);
    setErrorMsg(
      error?.code === "auth/invalid-email"
        ? "Please enter a valid e-mail address."
        : "Could not send the reset e-mail. Please try again.",
```

```
    );
  }
};

const handleAuth = async (e: React.FormEvent) => {
  e.preventDefault();
  setErrorMsg("");
  setSuccessMsg("");

  if (!email || !password) {
    setErrorMsg("Please fill out all fields.");
    return;
  }

  if (password.length < 6) {
    setErrorMsg("Password must be at least 6 characters.");
    return;
  }

  setIsLoading(true);

  try {
    // Set before signing in, so the very first session honours the choice.
    await setPersistence(auth, rememberMe ? browserLocalPersistence : browserSessionPersistence);

    if (activeMode === "signup") {
      if (!fullName) {
        setErrorMsg("Please enter your full name.");
        setIsLoading(false);
        return;
      }

      // 1. Create firebase auth user
      const userCredential = await createUserWithEmailAndPassword(auth, email, password);
      const user = userCredential.user;

      // 2. Create profile document in Firestore
      await setDoc(doc(db, "users", user.uid), {
        uid: user.uid,
        email: user.email,
        fullName: fullName,
        points: 120,
        steps: 640,
        completedStations: [],
        badges: [],
        createdAt: new Date().toISOString()
      });

      setSuccessMsg("Profile registered successfully! Logging you in...");
      // Admin status comes from the "admins" collection (one document per
      // uid, granted only from the Firebase console) — never from the
      // email address, which the person creating the account controls.
      const signupAdminSnap = await getDoc(doc(db, "admins", user.uid));
      const signupIsAdmin = signupAdminSnap.exists();
      setTimeout(() => {
        onLoginSuccess(email, signupIsAdmin);
        setIsLoading(false);
      }, 1500);
```

```
    } else {
      // Sign In
      const userCredential = await signInWithEmailAndPassword(auth, email, password);
      const user = userCredential.user;

      // Load profile from Firestore if it exists
      const docRef = doc(db, "users", user.uid);
      const docSnap = await getDoc(docRef);

      if (!docSnap.exists()) {
        // If no profile, initialize one
        await setDoc(docRef, {
          uid: user.uid,
          email: user.email,
          fullName: email.split("@")[0],
          points: 120,
          steps: 640,
          completedStations: [],
          badges: [],
          createdAt: new Date().toISOString()
        });
      }

      // Admin status comes from the "admins" collection (one document per
      // uid, granted only from the Firebase console) — never from the
      // email address.
      const signinAdminSnap = await getDoc(doc(db, "admins", user.uid));
      const signinIsAdmin = signinAdminSnap.exists();
      setSuccessMsg(`Welcome back, ${signinIsAdmin ? "Parish Administrator" : "Faithful Pilgrim"}!`);
      setTimeout(() => {
        onLoginSuccess(email, signinIsAdmin);
        setIsLoading(false);
      }, 1500);
    }
  } catch (error: any) {
    console.error("Authentication error:", error);
    let friendlyMessage = "Authentication failed. Please check your credentials.";
    if (error.code === "auth/email-already-in-use") {
      friendlyMessage = "This email is already registered. Please Sign In instead.";
    } else if (error.code === "auth/invalid-credential" || error.code === "auth/wrong-password" || error.code === "auth/user-not-found") {
      friendlyMessage = "Invalid email or password. Please try again.";
    } else if (error.code === "auth/invalid-email") {
      friendlyMessage = "Please enter a valid e-mail address.";
    } else if (error.code === "auth/configuration-not-found") {
      // Nothing the person can do about this one, and "check your
      // credentials" blamed them for it. It means e-mail/password
      // sign-in has not been switched on in the Firebase console.
      friendlyMessage =
        "Sign-in is not switched on for this parish app yet. This is a setting on our side, not a problem with your details.";
    } else if (error.code === "auth/network-request-failed") {
      friendlyMessage = "No connection. Check your internet and try again.";
    } else if (error.code === "auth/too-many-requests") {
      friendlyMessage = "Too many attempts. Please wait a moment and try again.";
    }
    setErrorMsg(friendlyMessage);
    setIsLoading(false);
  }
};
```

```
  // Signed in: the account summary, unchanged in substance.
  if (isLoggedIn) {
    return (
      <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto">
        <div className="px-6 pt-6 pb-10 max-w-[420px] w-full mx-auto space-y-6">
          <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-6 text-center space-y-5 shadow-xs">
            <div className="h-16 w-16 bg-[var(--color-brand-primary)]/10 rounded-full flex items-center justify-center mx-auto text-[var(--color-brand-primary)]">
              <ShieldCheck className="w-8 h-8" />
            </div>

            <div className="space-y-1">
              <h3 className="text-sm font-bold text-[var(--color-brand-text)] uppercase tracking-wider">
                Signed in
              </h3>
              <p className="text-[15px] text-[var(--color-brand-secondary)]">
                You are signed in as
              </p>
              <span className="inline-block text-[15px] font-mono font-bold text-[var(--color-brand-text)] bg-[var(--color-brand-card-sunk)] px-4 py-1.5 rounded-full
border border-[var(--color-brand-border)] mt-2 break-all">
                {userEmail}
              </span>
            </div>

            {isAdmin && (
              <div className="p-4 bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)] text-[var(--color-brand-text)] rounded-2xl text-[15px]
leading-relaxed text-left space-y-1.5">
                <span className="font-bold block uppercase tracking-wider text-sm">Admin Privilege Unlocked</span>
                <p className="text-[15px] text-[var(--color-brand-secondary)]">
                  You now have authorization to edit parish history details, add or remove parish bulletin
                  announcements, and moderate pilgrim logs in the <strong>Admin Portal</strong>.
                </p>
              </div>
            )}

            <button
              onClick={onLogout}
              className="w-full py-3 bg-red-50 hover:bg-red-100 text-red-600 active:scale-98 transition-all text-[15px] font-bold uppercase tracking-wider rounded-
full border border-red-200"
            >
              Sign out
            </button>
          </div>
        </div>
      </div>
    );
  }

  const signingUp = activeMode === "signup";

  return (
    <div
      ref={scrollRef}
      className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto"
    >
      <div className="px-6 pt-4 pb-10 max-w-[420px] w-full mx-auto">
        {/* Emblem, then a short heading, then one line under it - the
            order the reference layout uses. The emblem is the diocese
            crest rather than a padlock or a shield: this is the way into
            one particular diocese's app, and its own mark says that,
```

```
            where a generic security glyph would only say "a form". */}
        <header className="text-center pt-6 pb-7">
          <div className="mx-auto mb-5 h-[72px] w-[72px] rounded-full bg-[var(--color-brand-primary)]/8 border border-[var(--color-brand-border)] grid place-items-
center">
            {signingUp ? (
              <UserPlus className="w-8 h-8 text-[var(--color-brand-primary)]" />
            ) : (
              <img
                src="/ui/sanctiwalk-mark-navy.png"
                alt="SanctiWalk"
                className="w-10 h-10 object-contain"
              />
            )}
          </div>

          <h2 className="text-[30px] leading-[1.15] font-extrabold tracking-tight text-[var(--color-brand-text)]">
            {signingUp ? "Create account" : "Welcome back"}
          </h2>
          <p className="mt-2 text-[15px] text-[var(--color-brand-secondary)]">
            {signingUp
              ? "Set up your pilgrim profile to get started."
              : "Sign in to continue to your parish."}
          </p>
        </header>

        {/* Above the form, not below it.
            Somebody who has a Google account should not read past a
            password field they will never fill in to find the button
            that saves them the trouble. */}
        <button
          type="button"
          onClick={() => void handleGoogle()}
          disabled={googleBusy || isLoading}
          className="w-full mb-4 flex items-center justify-center gap-3 rounded-full border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-5 py-
3.5 text-[16px] font-bold text-[var(--color-brand-text)] transition-colors hover:border-[var(--color-brand-primary)] disabled:opacity-60"
        >
          <GoogleMark />
          {googleBusy ? "Opening Google…" : "Continue with Google"}
        </button>

        <div className="mb-4 flex items-center gap-3" aria-hidden>
          <span className="h-px flex-1 bg-[var(--color-brand-border)]" />
          <span className="text-[14px] text-[var(--color-brand-secondary)]">or</span>
          <span className="h-px flex-1 bg-[var(--color-brand-border)]" />
        </div>

        <form onSubmit={handleAuth} className="space-y-5">
          {errorMsg && (
            <div className="p-3 bg-red-50 border border-red-200 text-red-800 rounded-xl flex items-start gap-2 text-[15px]">
              <ShieldAlert className="w-4 h-4 text-red-600 shrink-0 mt-0.5" />
              <span>{errorMsg}</span>
            </div>
          )}

          {successMsg && (
            <div className="p-3 bg-green-50 border border-green-200 text-green-800 rounded-xl flex items-start gap-2 text-[15px]">
              <CheckCircle className="w-4 h-4 text-green-600 shrink-0 mt-0.5" />
              <span>{successMsg}</span>
            </div>
```

```
          )}

          {signingUp && (
            <Field
              label="Full name"
              type="text"
              value={fullName}
              onChange={setFullName}
              placeholder="Juan dela Cruz"
              autoComplete="name"
            />
          )}

          <Field
            label="E-mail"
            type="email"
            value={email}
            onChange={setEmail}
            placeholder="example@email.com"
            autoComplete="email"
          />

          <Field
            label="Password"
            type="password"
            value={password}
            onChange={setPassword}
            placeholder="Your Password"
            autoComplete={signingUp ? "new-password" : "current-password"}
          />

          <div className="flex items-center justify-between gap-3 text-[15px]">
            <label className="flex items-center gap-2 text-[var(--color-brand-secondary)] select-none">
              <input
                type="checkbox"
                checked={rememberMe}
                onChange={(e) => setRememberMe(e.target.checked)}
                className="w-4 h-4 accent-[var(--color-brand-primary)]"
              />
              <span>Remember me</span>
            </label>

            {!signingUp && (
              <button
                type="button"
                onClick={handleForgotPassword}
                className="text-[var(--color-brand-secondary)] hover:text-[var(--color-brand-primary)] transition-colors"
              >
                Forgot Password?
              </button>
            )}
          </div>

          <button
            type="submit"
            disabled={isLoading}
            className="w-full py-4 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] disabled:opacity-60 text-white text-[16px] font-bold
rounded-full transition-colors active:scale-[0.99]"
          >
```

```
            {isLoading ? "Please wait…" : signingUp ? "Create account" : "Sign in"}
          </button>
        </form>

        {/* "Or", then a full-width outlined alternative - the reference
            layout's own pattern for a second way in.

            In the reference that slot holds Google, Apple and Facebook.
            Those need providers enabled and configured in Firebase and
            none are, so buttons there would be three things that look
            tappable and do nothing. What genuinely exists is the other
            mode, so that is what the slot carries. */}
        <div className="flex items-center gap-3 my-6">
          <span className="h-px flex-1 bg-[var(--color-brand-border)]" />
          <span className="text-[14px] text-[var(--color-brand-secondary)]">Or</span>
          <span className="h-px flex-1 bg-[var(--color-brand-border)]" />
        </div>

        <button
          type="button"
          onClick={() => {
            setActiveMode(signingUp ? "signin" : "signup");
            setErrorMsg("");
            setSuccessMsg("");
            scrollRef.current?.scrollTo({ top: 0 });
          }}
          className="w-full py-3.5 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-full text-[15px] font-semibold text-[var(--color-
brand-text)] hover:border-[var(--color-brand-primary)] transition-colors"
        >
          {signingUp ? "Sign in to an existing account" : "Create a new account"}
        </button>

        <p className="mt-6 text-[14px] leading-relaxed text-[var(--color-brand-secondary)] text-center">
          Admin access to the parish office portal is granted by parish staff and
          cannot be self-assigned.
        </p>
      </div>
    </div>
  );
}

/**
 * Google's mark, inline.
 *
 * Drawn rather than linked. A remote image fails on a handset with a
 * poor connection, and a sign-in button with a missing logo looks like
 * a broken page at the moment somebody is deciding whether to trust it.
 */
function GoogleMark() {
  return (
    <svg width="19" height="19" viewBox="0 0 48 48" aria-hidden focusable="false">
      <path fill="#FFC107" d="M43.6 20.5H42V20H24v8h11.3C33.7 32.7 29.3 36 24 36c-6.6 0-12-5.4-12-12s5.4-12 12-12c3.1 0 5.9 1.2 8 3.1l5.7-5.7C34.0 6.1 29.3 4 24 4
12.9 4 4 12.9 4 24s8.9 20 20 20 20-8.9 20-20c0-1.3-.1-2.4-.4-3.5z"/>
      <path fill="#FF3D00" d="M6.3 14.7l6.6 4.8C14.7 15.1 19 12 24 12c3.1 0 5.9 1.2 8 3.1l5.7-5.7C34.0 6.1 29.3 4 24 4 16.3 4 9.7 8.3 6.3 14.7z"/>
      <path fill="#4CAF50" d="M24 44c5.2 0 9.9-2 13.4-5.2l-6.2-5.2C29.2 35.1 26.7 36 24 36c-5.3 0-9.7-3.3-11.3-8l-6.5 5C9.6 39.6 16.2 44 24 44z"/>
      <path fill="#1976D2" d="M43.6 20.5H42V20H24v8h11.3c-.8 2.2-2.2 4.1-4.1 5.6l6.2 5.2C37.0 40.2 44 35 44 24c0-1.3-.1-2.4-.4-3.5z"/>
    </svg>
  );
}
```

1.2.23 APP-028 — MapPlaceSheet.tsx

Code ID	APP-028
File Name	MapPlaceSheet.tsx
File Path	SanctDemo/src/components/MapPlaceSheet.tsx
Language / Type	TypeScript (React, TSX)
Lines	237
Functions / Classes	MapPlaceSheet (exported component/function (default))
Related Files (its imports)	../data, ./MassNowBadge, ../lib/routing
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/components/MapPlaceSheet.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MapPlaceSheet.tsx

```
import { useState } from "react";
import { Navigation, X, ChevronRight, Footprints, Bike, Car, Check, Plus } from "lucide-react";
import {
  TRAVEL_MODES,
  formatModeDuration,
  minutesFor,
  travelMode,
  type TravelModeId,
} from "../lib/travelModes";
import { PARISH_PATRON_IMAGES, PARISH_PATRON_SAINTS } from "../data";
import MassNowBadge from "./MassNowBadge";
import { formatDistance } from "../lib/routing";

interface MapPlaceSheetProps {
  /** Display name of the tapped parish. */
  name: string;
  /** "Vicariate of San Roque (unconfirmed)" — as shown on the pin. */
  location?: string;
  /** The tour route id, when this parish has one. Null for the other 29. */
  routeId: string | null;
  /** Straight-line metres from the pilgrim, or null with no fix. */
  straightLineMetres: number | null;
  /** Progress text for the directions request: "" while idle. */
  directionsStatus: string;
  /** Road distance of the drawn route, once there is one. */
  routeMetres: number | null;
  directionsBusy: boolean;
  /** True once a route to this parish is drawn, which turns Directions into Start. */
  hasRoute: boolean;
  /**
   * True while the road route is being measured.
   *
   * The card says "Measuring…" rather than showing a straight-line figure
```

```
   * about to be replaced by a longer road one. The client saw exactly that
   * swap — 8 min becoming 10 min for a walk that had not changed — and
   * reported it as the same journey giving two different answers.
   */
  measuring?: boolean;
  /** True when this church is already on the visit being planned. */
  inTrip: boolean;
  /** Adds this church to the visit, or takes it back off. */
  onToggleTrip: () => void;
  onDirections: () => void;
  onStartWalking: () => void;
  onOpenParish: () => void;
  onClose: () => void;
}


const MODE_ICONS: Record<TravelModeId, typeof Footprints> = {
  walk: Footprints,
  motorcycle: Bike,
  car: Car,
};

/**
 * The card that opens when a pin is tapped.
 *
 * It replaces a 220px MapLibre popup that carried three stacked buttons —
 * View parish, Get directions, Start walking — where two of the three were
 * the same errand. This is the place card the client asked for: the parish's
 * details first, and ONE action.
 *
 * Start walking is not a fourth button competing with Directions. It appears
 * only after a route has been drawn, which is the order the errand actually
 * happens in: see the way, then set off.
 */
export default function MapPlaceSheet({
  name,
  location,
  routeId,
  straightLineMetres,
  directionsStatus,
  routeMetres,
  directionsBusy,
  hasRoute,
  measuring = false,
  inTrip,
  onToggleTrip,
  onDirections,
  onStartWalking,
  onOpenParish,
  onClose,
}: MapPlaceSheetProps) {
  const [modeId, setModeId] = useState<TravelModeId>("walk");
  const photo = routeId ? PARISH_PATRON_IMAGES[routeId] : undefined;

  // The road distance once routed, the straight line before that. Both are
  // real measurements; which one is in hand is said plainly below rather
  // than blurred into a single confident-looking number.
  const metres = routeMetres ?? straightLineMetres;
  const selected = travelMode(modeId);
```

```
const patron = routeId ? PARISH_PATRON_SAINTS[routeId] : undefined;

return (
  <div className="map-sheet" role="dialog" aria-label={name}>
    <button type="button" className="map-sheet__close" onClick={onClose} aria-label="Close">
      <X className="w-4 h-4" />
    </button>

    {photo && (
      <img
        src={photo}
        alt={patron ? `${patron}, ${name}` : name}
        className="map-sheet__photo"
        style={{ objectPosition: "center 32%" }}
      />
    )}

    <div className="map-sheet__body">
      <h3 className="map-sheet__name">{name}</h3>
      <p className="map-sheet__meta">Catholic parish{location ? ` · ${location}` : ""}</p>

      {routeId && <MassNowBadge routeId={routeId} className="mt-2" />}

      {patron && <p className="map-sheet__patron">{patron}</p>}

      {/* Google's mode row, with the three ways people actually reach a
          parish here. Every time is an estimate from ONE distance — the
          routing service serves a single road profile and ignores the mode
          asked of it (see travelModes.ts) — so the row says so underneath
          instead of implying three separate routings. */}
      {/* Travel times for EVERY parish, not only the two with a tour.
          Distance and routing need coordinates, and all 31 have verified
          ones — the tour route was never what made this work, it was just
          what the code happened to key off. */}
      {/* While the road route is being measured, nothing numeric is shown.
          A straight-line figure here would be replaced by a longer road one
          a second later, which is the swap the client reported: 8 min
          becoming 10 min for a walk that had not changed. Better to wait a
          moment and show one number that stays put. */}
      {measuring && metres !== null && (
        <p className="map-sheet__estimate mt-3">Measuring the walking route…</p>
      )}

      {!measuring && metres !== null && (
        <>
          <div className="map-sheet__modes" role="tablist" aria-label="How you are travelling">
            {TRAVEL_MODES.map(mode => {
              const Icon = MODE_ICONS[mode.id];
              return (
                <button
                  key={mode.id}
                  type="button"
                  role="tab"
                  aria-selected={mode.id === modeId}
                  className="map-sheet__mode"
                  data-selected={mode.id === modeId}
                  onClick={() => setModeId(mode.id)}
                >
                  <Icon className="w-[18px] h-[18px]" />
```

```
            <span className="map-sheet__mode-time">
              {formatModeDuration(minutesFor(metres, mode))}
            </span>
          </button>
        );
      })}
    </div>

    <p className="map-sheet__estimate">
      {formatDistance(metres)}
      {routeMetres === null && " direct"} · {selected.note}
    </p>
  </>
)}

{/* Directions for every parish in the diocese.
    A pilgrim looking for their OWN parish is served by this long
    before anyone writes its history — and 29 of the 31 are somebody's
    own parish. Gating it on having a tour route made the other 29 a
    dead end that said "coming soon" and offered nothing. */}
<>
  {/* One button, and which one depends on where the errand has
      got to. Before a route exists the only useful thing is
      Directions; once it is drawn, asking for the same directions
      again is the redundancy the client spotted — the button
      becomes Start, exactly as Google Maps does it. */}
  <div className="map-sheet__actions">
      {hasRoute ? (
        <button type="button" className="map-sheet__action" onClick={onStartWalking}>
          <Navigation className="w-4 h-4" />
          Start
        </button>
      ) : (
        <button
          type="button"
          className="map-sheet__action"
          onClick={onDirections}
          disabled={directionsBusy}
        >
          <Navigation className="w-4 h-4" />
          {directionsBusy ? "Finding a route…" : "Directions"}
        </button>
      )}

      {/* Beside Directions, not instead of it. Directions answers
          "take me to this one"; this answers "and this one too" -
          a Bisita Iglesia is seven churches, and having to leave
          the map and come back for each one is the errand the
          planner exists to remove. */}
      <button
        type="button"
        className={inTrip ? "map-sheet__add map-sheet__add--on" : "map-sheet__add"}
        onClick={onToggleTrip}
        aria-pressed={inTrip}
      >
        {inTrip ? <Check className="w-4 h-4" /> : <Plus className="w-4 h-4" />}
        {inTrip ? "On the visit" : "Add to visit"}
      </button>
    </div>
```

```
            {/* Only while there is no route. Once one is drawn the mode row
                above already carries the distance and the time for the mode
                actually selected, and this line always said "walk" — so with
                Car chosen the sheet contradicted itself. */}
            {!hasRoute && directionsStatus && (
              <p className="map-sheet__status">{directionsStatus}</p>
            )}

            {/* The parish page stays gated, because only these two HAVE one.
                Sending someone to an empty screen would be worse than not
                offering it. The note below says so rather than leaving the
                absence unexplained. */}
            {routeId ? (
              <button type="button" className="map-sheet__link" onClick={onOpenParish}>
                Parish page <ChevronRight className="w-4 h-4" />
              </button>
            ) : (
              <p className="map-sheet__soon">
                Mass times, history and the guided tour are still being collected for this parish.
              </p>
            )}
          </>
        </div>
      </div>
    );
}
```

1.2.24 APP-029 — MapTab.tsx

Code ID	APP-029
File Name	MapTab.tsx
File Path	SanctDemo/src/components/MapTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	440
Functions / Classes	MapTab (exported component/function (default))
Related Files (its imports)	../types, ../data
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/MapTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MapTab.tsx

```
import React, { useState, useEffect, useRef } from "react";
import { Route, Station } from "../types";
import { ROUTES } from "../data";
import { MapPin, Navigation, Volume2, CheckCircle2, Play, Pause, ChevronLeft, Award, BookOpen, AlertCircle, Sparkles } from "lucide-react";

interface MapTabProps {
  routeId: string | null;
  onSelectRoute: (routeId: string | null) => void;
  isOffline: boolean;
  onStationVisited: (stationId: string, stepsToAdd: number, distanceToAdd: number) => void;
  visitedStations: string[];
}

export default function MapTab({
  routeId,
  onSelectRoute,
  isOffline,
  onStationVisited,
  visitedStations
}: MapTabProps) {
  const currentRoute = ROUTES.find((r) => r.id === routeId) || ROUTES[0];

  // Simulation States
  const [activeStationIndex, setActiveStationIndex] = useState(0);
  const [isPlayingAudio, setIsPlayingAudio] = useState(false);
  const [audioProgress, setAudioProgress] = useState(0);
  const [isSimulatingWalk, setIsSimulatingWalk] = useState(false);
  const [walkProgress, setWalkProgress] = useState(0); // 0 to 100 between stations
  const [diaryNote, setDiaryNote] = useState("");
  const [savedNotes, setSavedNotes] = useState<Record<string, string>>({});
  const [simLat, setSimLat] = useState(currentRoute.stations[0]?.coordinates.lat);
  const [simLng, setSimLng] = useState(currentRoute.stations[0]?.coordinates.lng);
```

```
const activeStation = currentRoute.stations[activeStationIndex];

// Reset indices on route change
useEffect(() => {
  setActiveStationIndex(0);
  setWalkProgress(0);
  setIsSimulatingWalk(false);
  setIsPlayingAudio(false);
  setAudioProgress(0);
  if (currentRoute.stations[0]) {
    setSimLat(currentRoute.stations[0].coordinates.lat);
    setSimLng(currentRoute.stations[0].coordinates.lng);
  }
}, [routeId]);

// Audio Playback Simulation Interval
useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlayingAudio) {
    timer = setInterval(() => {
      setAudioProgress((prev) => {
        if (prev >= 100) {
          setIsPlayingAudio(false);
          return 0;
        }
        return prev + 5;
      });
    }, 500);
  }
  return () => clearInterval(timer);
}, [isPlayingAudio]);

// Simulated Speech Synthesis option
const speakReflection = () => {
  if ("speechSynthesis" in window) {
    window.speechSynthesis.cancel();
    const utterance = new SpeechSynthesisUtterance(activeStation.reflection);
    utterance.rate = 0.95;
    window.speechSynthesis.speak(utterance);
    setIsPlayingAudio(true);
  } else {
    setIsPlayingAudio(true);
  }
};

const stopSpeaking = () => {
  if ("speechSynthesis" in window) {
    window.speechSynthesis.cancel();
  }
  setIsPlayingAudio(false);
};

// Walk Simulation Interval
useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isSimulatingWalk) {
    timer = setInterval(() => {
      setWalkProgress((prev) => {
```

```
        if (prev >= 100) {
          setIsSimulatingWalk(false);
          // Arrived at next station!
          const nextIndex = (activeStationIndex + 1) % currentRoute.stations.length;
          setActiveStationIndex(nextIndex);

          // Set coordinates
          const targetStation = currentRoute.stations[nextIndex];
          setSimLat(targetStation.coordinates.lat);
          setSimLng(targetStation.coordinates.lng);

          return 0;
        }

        // Interpolate coordinates
        const currentIndex = activeStationIndex;
        const nextIndex = (activeStationIndex + 1) % currentRoute.stations.length;
        const start = currentRoute.stations[currentIndex].coordinates;
        const end = currentRoute.stations[nextIndex].coordinates;
        const pct = prev / 100;
        setSimLat(start.lat + (end.lat - start.lat) * pct);
        setSimLng(start.lng + (end.lng - start.lng) * pct);

        return prev + 10;
      });
    }, 300);
  }
  return () => clearInterval(timer);
}, [isSimulatingWalk, activeStationIndex, currentRoute]);

const handleNextStationSim = () => {
  if (currentRoute.stations.length <= 1) return;
  setIsSimulatingWalk(true);
  setWalkProgress(0);
};

const handleCheckIn = () => {
  // Generate reward and add progress
  onStationVisited(activeStation.id, 800, 0.6);
};

const handleSaveDiary = () => {
  if (!diaryNote.trim()) return;
  setSavedNotes((prev) => ({
    ...prev,
    [activeStation.id]: diaryNote
  }));
  setDiaryNote("");
};

// SVG coordinate calculator for map trail representation
const stationsCount = currentRoute.stations.length;
const mapWidth = 320;
const mapHeight = 180;

return (
  <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] pb-20 select-none">
```

```
      {/* Route Navigation Header */}
      <div className="bg-[var(--color-brand-primary)] text-white px-4 py-3 flex items-center gap-3 shrink-0 border-b border-[var(--color-brand-border)]">
        <button
          onClick={() => onSelectRoute(null)}
          className="p-1 hover:bg-white/10 rounded-full transition-colors"
        >
          <ChevronLeft className="w-5 h-5" />
        </button>
        <div>
          <h3 className="text-[15px] font-bold text-[var(--color-brand-on-accent)] font-serif italic uppercase tracking-wider">Active SanctiWalk Guide</h3>
          <h2 className="text-sm font-bold tracking-tight line-clamp-1 font-serif">{currentRoute.name}</h2>
        </div>
      </div>

      {/* Geolocation Telemetry Display */}
      <div className="bg-[var(--color-brand-text)] text-[var(--color-brand-secondary)] font-mono text-sm px-4 py-2 flex items-center justify-between shadow-inner
shrink-0 border-b border-[var(--color-brand-border)]/45">
        <span className="flex items-center gap-1">
          <Navigation className="w-3 h-3 text-[var(--color-brand-on-accent)] animate-pulse" />
          LAT: <span className="text-[var(--color-brand-on-accent)] font-bold">{simLat.toFixed(6)}</span>
            LNG: <span className="text-[var(--color-brand-on-accent)] font-bold">{simLng.toFixed(6)}</span>
        </span>
        <span className="text-sm bg-[var(--color-brand-primary-dark)] text-[var(--color-brand-secondary)] px-2 py-0.5 rounded font-bold uppercase tracking-wider">
          GPS Sim
        </span>
      </div>

      {/* Interactive Map Section */}
      <div className="p-4 shrink-0">
        <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-3.5 shadow-xs relative overflow-hidden">
          <div className="flex items-center justify-between mb-2">
            <span className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic">
              Sancti Trail Visualizer
            </span>
            {isOffline && (
              <span className="flex items-center gap-1 text-sm font-bold text-[var(--color-brand-text)] bg-[var(--color-brand-card)] px-2 py-0.5 rounded-full border
border-[var(--color-brand-border)]">
                <AlertCircle className="w-2.5 h-2.5" /> Cached Offline Map
              </span>
            )}
          </div>

          {/* SVG Map Canvas */}
          <div className="relative h-[160px] bg-[var(--color-brand-border)] rounded-2xl border border-[var(--color-brand-border)] flex items-center justify-center
overflow-hidden">
            {/* Background elements representing parks & roads */}
            <div className="absolute inset-0 opacity-20 bg-[radial-gradient(var(--color-brand-primary)_1px,transparent_1px)] [background-size:20px_20px]">
              <div className="absolute top-1/4 left-1/12 w-16 h-16 bg-[var(--color-brand-primary)]/40 rounded-full blur-md"></div>
              <div className="absolute bottom-1/4 right-1/10 w-24 h-24 bg-[var(--color-brand-primary)]/30 rounded-full blur-lg"></div>
              <div className="absolute top-1/2 left-1/2 w-40 h-2 bg-[var(--color-brand-border)] rotate-12"></div>
              <div className="absolute top-1/3 left-1/3 w-2 bg-[var(--color-brand-border)] h-40 rotate-45"></div>
            </div>

            <svg viewBox={`0 0 ${mapWidth} ${mapHeight}`} className="w-full h-full relative z-10">
              {/* Draw walk path/trail */}
              {stationsCount > 1 && (
                <path
                  d={`M 40 ${mapHeight / 2} Q 160 ${mapHeight / 5} 280 ${mapHeight / 2}`}
                  fill="none"
```

```
    stroke="var(--color-brand-primary)"
    strokeWidth="3"
    strokeDasharray="6,4"
    strokeOpacity="0.6"
  />
)}

{/* Station Markers */}
{currentRoute.stations.map((st, idx) => {
  const ratio = idx / (stationsCount - 1 || 1);
  const x = 40 + ratio * 240;
  // Simple quadratic bezier path height estimation for positioning points
  const t = ratio;
  const y = (1 - t) * (1 - t) * (mapHeight / 2) + 2 * (1 - t) * t * (mapHeight / 5) + t * t * (mapHeight / 2);


  const isCurrent = idx === activeStationIndex;
  const isVisited = visitedStations.includes(st.id);

  return (
    <g key={st.id}>
      {/* Ring glow for current active stop */}
      {isCurrent && (
        <circle
          cx={x}
          cy={y}
          r="14"
          fill="none"
          stroke="var(--color-brand-primary)"
          strokeWidth="1.5"
          className="animate-ping"
          style={{ transformOrigin: `${x}px ${y}px` }}
        />
      )}
      {/* Outer border */}
      <circle
        cx={x}
        cy={y}
        r="9"
        fill={isCurrent ? "var(--color-brand-primary)" : isVisited ? "var(--color-brand-success)" : "var(--color-brand-card)"}
        stroke={isCurrent ? "var(--color-brand-accent)" : "var(--color-brand-border)"}
        strokeWidth="2.5"
      />
      {/* Inner gold core */}
      <circle
        cx={x}
        cy={y}
        r="4"
        fill={isCurrent ? "var(--color-brand-accent)" : "var(--color-brand-card)"}
      />
      {/* Station Number Text */}
      <text
        x={x}
        y={y - 12}
        textAnchor="middle"
        fill="var(--color-brand-text)"
        fontSize="14"
        fontWeight="bold"
        fontFamily="Lora, serif"
```

```
                >
                  Stop {idx + 1}
                </text>
              </g>
            );
          })}

          {/* Avatar position simulation along bezier trail */}
          {stationsCount > 1 && !isSimulatingWalk && (
            (() => {
              const ratio = activeStationIndex / (stationsCount - 1);
              const x = 40 + ratio * 240;
              const t = ratio;
              const y = (1 - t) * (1 - t) * (mapHeight / 2) + 2 * (1 - t) * t * (mapHeight / 5) + t * t * (mapHeight / 2);
              return (
                <g>
                  <circle cx={x} cy={y} r="15" fill="var(--color-brand-primary)" fillOpacity="0.2" />
                  <g transform={`translate(${x - 7}, ${y - 18})`}>
                    <MapPin className="w-3.5 h-3.5 text-[var(--color-brand-secondary)] fill-[var(--color-brand-gold)]" />
                  </g>
                </g>
              );
            })()
          )}

          {/* Walking simulation avatar */}
          {isSimulatingWalk && (
            (() => {
              const nextIndex = (activeStationIndex + 1) % stationsCount;
              const currentRatio = activeStationIndex / (stationsCount - 1);
              const nextRatio = nextIndex / (stationsCount - 1);
              const stepRatio = currentRatio + (nextRatio - currentRatio) * (walkProgress / 100);
              const x = 40 + stepRatio * 240;
              const t = stepRatio;
              const y = (1 - t) * (1 - t) * (mapHeight / 2) + 2 * (1 - t) * t * (mapHeight / 5) + t * t * (mapHeight / 2);
              return (
                <g className="animate-bounce">
                  <circle cx={x} cy={y} r="8" fill="var(--color-brand-accent)" stroke="var(--color-brand-primary)" strokeWidth="2" />
                </g>
              );
            })()
          )}
        </svg>

        {/* Float simulation guide button overlay */}
        <div className="absolute bottom-2.5 right-2.5 z-20">
          <button
            id="btn-simulate-walk"
            disabled={isSimulatingWalk || stationsCount <= 1}
            onClick={handleNextStationSim}
            className="bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white font-bold uppercase tracking-wider text-sm px-3 py-
1.5 rounded-full flex items-center gap-1.5 shadow-md disabled:bg-[var(--color-brand-border)] disabled:text-[var(--color-brand-secondary)] disabled:shadow-none
transition-colors border border-[var(--color-brand-primary-dark)]"
          >
            <Navigation className="w-3 h-3 rotate-45 fill-current" />
            {isSimulatingWalk ? "Walking..." : "Simulate Walk"}
          </button>
        </div>
      </div>
```

```
        </div>
      </div>

      {/* Active Station Detail Drawer */}
      <div className="px-4 flex-1 overflow-y-auto space-y-4">
        <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] p-4 shadow-xs space-y-4">
          <div className="flex gap-3">
            <img
              src={activeStation.imageUrl}
              alt={activeStation.name}
              className="w-20 h-20 rounded-2xl object-cover border border-[var(--color-brand-border)]/50 shrink-0"
              referrerPolicy="no-referrer"
            />
            <div className="space-y-1">
              <span className="text-sm font-bold text-[var(--color-brand-gold)] uppercase tracking-widest block font-serif italic">
                Current Station {activeStationIndex + 1} of {stationsCount}
              </span>
              <h4 className="text-[15px] font-bold text-[var(--color-brand-text)] font-serif italic leading-tight">
                {activeStation.name}
              </h4>
              <p className="text-[15px] text-[var(--color-brand-secondary)] leading-normal line-clamp-2">
                {activeStation.description}
              </p>
            </div>
          </div>

          {/* Historical Details */}
          <div className="p-3 bg-[var(--color-brand-card)]/60 rounded-2xl space-y-1 border border-[var(--color-brand-border)]/50">
            <h5 className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic flex items-center gap-1">
              <BookOpen className="w-3.5 h-3.5 text-[var(--color-brand-secondary)]" /> Heritage & History
            </h5>
            <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed font-sans">
              {activeStation.history}
            </p>
          </div>

          {/* Spiritual Reflection Segment */}
          <div className="p-3.5 bg-[var(--color-brand-card)]/30 rounded-2xl border border-[var(--color-brand-border)] space-y-2">
            <div className="flex items-center justify-between">
              <h5 className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-serif italic flex items-center gap-1.5">
                <Sparkles className="w-3.5 h-3.5 text-[var(--color-brand-gold)]" /> Pilgrim Reflection Prompt
              </h5>


              {/* Voice Guided Audio Button */}
              <button
                onClick={isPlayingAudio ? stopSpeaking : speakReflection}
                className="flex items-center gap-1 bg-[var(--color-brand-primary)] text-white px-2.5 py-0.5 rounded-full text-sm font-bold border border-[var(--
color-brand-primary-dark)] uppercase tracking-wider hover:bg-[var(--color-brand-primary-dark)] transition-all"
              >
                <Volume2 className="w-3 h-3" />
                {isPlayingAudio ? "Stop Guide" : "Listen Guide"}
              </button>
            </div>

            <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed italic font-medium font-sans">
              "{activeStation.reflection}"
            </p>
```

```
            {/* Audio guide animated sound wave */}
            {isPlayingAudio && (
              <div className="flex items-center gap-1 pt-1.5">
                <span className="w-1.5 h-3 bg-[var(--color-brand-primary)] rounded-full animate-bounce"></span>
                <span className="w-1.5 h-5 bg-[var(--color-brand-primary)] rounded-full animate-bounce" style={{ animationDelay: "150ms" }}></span>
                <span className="w-1.5 h-2 bg-[var(--color-brand-primary)] rounded-full animate-bounce" style={{ animationDelay: "300ms" }}></span>
                <span className="w-1.5 h-4 bg-[var(--color-brand-primary)] rounded-full animate-bounce" style={{ animationDelay: "450ms" }}></span>
                <span className="text-sm font-mono text-[var(--color-brand-secondary)] ml-1.5">Playing Audio Guide...</span>
              </div>
            )}

            {/* Local Notes/Reflections Diary input */}
            <div className="space-y-2 pt-2 border-t border-[var(--color-brand-border)]/45">
              {savedNotes[activeStation.id] ? (
                <div className="p-2.5 bg-[var(--color-brand-card)] rounded-xl border border-[var(--color-brand-border)] text-[15px] text-gray-700 space-y-1">
                  <span className="text-sm text-[var(--color-brand-secondary)] font-mono block font-bold uppercase">Your Saved Note:</span>
                  <p className="italic font-medium text-[var(--color-brand-text)]">"{savedNotes[activeStation.id]}"</p>
                </div>
              ) : (
                <div className="flex gap-2">
                  <input
                    id="diary-note-input"
                    type="text"
                    placeholder="Write down your thoughts here..."
                    value={diaryNote}
                    onChange={(e) => setDiaryNote(e.target.value)}
                    className="flex-1 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] text-[15px] px-2.5 py-1.5 rounded-xl outline-none
placeholder-[var(--color-brand-secondary)] text-[var(--color-brand-text)]"
                  />
                  <button
                    onClick={handleSaveDiary}
                    className="bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white font-bold uppercase tracking-wider text-sm px-3
py-1.5 rounded-xl shrink-0"
                  >
                    Save
                  </button>
                </div>
              )}
            </div>
          </div>

          {/* Action Check In */}
          <div className="pt-2 flex items-center justify-between">
            <span className="text-sm text-[var(--color-brand-secondary)] font-mono flex items-center gap-1 font-bold">
              QR Code: <strong className="text-[var(--color-brand-text)]">{activeStation.qrCode}</strong>
            </span>

            {visitedStations.includes(activeStation.id) ? (
              <span className="flex items-center gap-1.5 text-[15px] text-green-700 bg-green-50 px-3.5 py-1.5 rounded-full font-bold border border-green-200">
                <CheckCircle2 className="w-4 h-4 text-green-600" /> Arrived & Checked-In
              </span>
            ) : (
              <button
                id="btn-station-checkin"
                onClick={handleCheckIn}
                className="bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white font-bold text-[15px] px-4 py-2.5 rounded-full flex
items-center gap-1.5 shadow-sm border border-[var(--color-brand-primary-dark)] transition-all uppercase tracking-wider hover:scale-[1.02]"
              >
                <Award className="w-4 h-4" /> Stamp Passport
```

```
              </button>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}
```

1.2.25 APP-030 — MassNowBadge.tsx

Code ID	APP-030
File Name	MassNowBadge.tsx
File Path	SanctDemo/src/components/MassNowBadge.tsx
Language / Type	TypeScript (React, TSX)
Lines	63
Functions / Classes	MassNowBadge (exported component/function (default))
Related Files (its imports)	../data, ../lib/schedule
Where It Is Used	src/components/MapPlaceSheet.tsx, src/components/ParishCard.tsx
How to Recognize It	Open SanctDemo/src/components/MassNowBadge.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MassNowBadge.tsx

```
import { useEffect, useState } from "react";
import { MASS_SCHEDULES } from "../data";
import { massStatus, type MassStatus } from "../lib/schedule";

interface MassNowBadgeProps {
  /** A route id — the two parishes whose schedule has been collected. */
  routeId: string | null | undefined;
  className?: string;
}

/**
 * "Mass now" / "Mass in 15 min" — live, or nothing at all.
 *
 * This is the question a mapping app structurally cannot answer and a parish
 * app must: not "when is the next Mass" but "can I still walk in". It renders
 * nothing outside those windows, so it reads as an event rather than as a
 * label that is always present.
 *
 * Only the two parishes with a collected schedule can show it. The other 29
 * render nothing rather than "no Mass", which would state as fact something
 * nobody has checked.
 */
export default function MassNowBadge({ routeId, className = "" }: MassNowBadgeProps) {
  const [status, setStatus] = useState<MassStatus>({ state: "none", time: null, minutes: 0 });

  useEffect(() => {
    const schedule = routeId ? MASS_SCHEDULES[routeId]?.schedule : undefined;
    if (!schedule) {
      setStatus({ state: "none", time: null, minutes: 0 });
      return;
    }

    // Re-checked every half minute so the badge appears and clears on its own
```

```
    // while the screen is left open — a pilgrim standing outside at 5:58 should
    // not have to reopen the app to see it turn over.
    const tick = () => setStatus(massStatus(schedule, new Date()));
    tick();
    const id = setInterval(tick, 30_000);
    return () => clearInterval(id);
  }, [routeId]);

  if (status.state === "none") return null;

  const live = status.state === "in-progress";

  return (
    <span
      className={`inline-flex items-center gap-1.5 text-[14px] font-semibold px-2.5 py-1 rounded-md ${
        live
          ? "bg-[var(--color-brand-error)] text-[var(--color-brand-on-accent)]"
          : "bg-[var(--color-brand-success)] text-[var(--color-brand-on-accent)]"
      } ${className}`}
    >
      {live && (
        <span
          aria-hidden
          className="w-2 h-2 rounded-full bg-current animate-pulse motion-reduce:animate-none"
        />
      )}
      {live ? `Mass now · ${status.time}` : `Mass in ${status.minutes} min`}
    </span>
  );
}
```

1.2.26 APP-031 — MassSchedule.tsx

Code ID	APP-031
File Name	MassSchedule.tsx
File Path	SanctDemo/src/components/MassSchedule.tsx
Language / Type	TypeScript (React, TSX)
Lines	209
Functions / Classes	groupWeekdayRows (internal function); MassSchedule (exported component/function (default))
Related Files (its imports)	../lib/parishPhotos, ../data, ../types, ../lib/schedule
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/MassSchedule.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MassSchedule.tsx

```
import React from "react";
import { parishPhoto, parishPhotoAlt } from "../lib/parishPhotos";
import { Clock, Phone, Mail, MapPin, Calendar, Sparkles, AlertTriangle } from "lucide-react";
import { MASS_SCHEDULES, PARISH_CONTACTS } from "../data";
import { Route } from "../types";
import { parseTimes } from "../lib/schedule";

const WEEKDAY_ORDER = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];

// Facade photographs now come from lib/parishPhotos.ts. This file used to
// keep its own copy of the roster, which is how San Roque ended up
// illustrated by a stock photograph of an unrelated church in three separate
// screens — each with its own list, each pasted from the last.

// Groups consecutive weekdays that share the exact same Mass time string into
// a single display row (e.g. "Monday & Tuesday", "Wednesday - Saturday"),
// mirroring how the parish bulletin itself groups days.
function groupWeekdayRows(schedule: { day: string; time: string }[]) {
  const byDay = new Map(schedule.map((s) => [s.day, s.time]));
  const groups: { label: string; time: string }[] = [];
  let i = 0;
  while (i < WEEKDAY_ORDER.length) {
    const day = WEEKDAY_ORDER[i];
    const time = byDay.get(day);
    if (!time) {
      i++;
      continue;
    }
    let j = i;
    while (j + 1 < WEEKDAY_ORDER.length && byDay.get(WEEKDAY_ORDER[j + 1]) === time) j++;
    const label =
```

```
      j === i
        ? day
        : j === i + 1
        ? `${day} & ${WEEKDAY_ORDER[j]}`
        : `${day} - ${WEEKDAY_ORDER[j]}`;
    groups.push({ label, time });
    i = j + 1;
  }
  return groups;
}

interface MassScheduleProps {
  parish: Route;
}

export default function MassSchedule({ parish }: MassScheduleProps) {
  const parishName = parish.name.replace(" Guide", "").replace(" Tour", "");
  const parishSchedule = MASS_SCHEDULES[parish.id];
  const schedule = parishSchedule?.schedule ?? [];
  const scheduleVerified = parishSchedule?.scheduleVerified ?? false;
  const weekdayRows = groupWeekdayRows(schedule);
  const sundayEntry = schedule.find((s) => s.day === "Sunday");
  const sundayTimes = sundayEntry ? parseTimes(sundayEntry.time) : [];
  const contact = PARISH_CONTACTS[parish.id];
  const photo = parishPhoto(parish.id);
  const photoAlt = parishPhotoAlt(parish.id, parishName);

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto scroll-safe">
      {/* Page Header */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden shrink-0 border-b border-[var(--color-
brand-border)]">
        <div className="absolute right-0 top-0 opacity-10 translate-x-4 -translate-y-4">
          <Clock className="w-32 h-32 text-white" />
        </div>
        <div className="flex items-center gap-1.5 text-[var(--color-brand-on-accent)] font-bold text-[15px] tracking-wider uppercase font-serif italic">
          <Sparkles className="w-3.5 h-3.5" /> Liturgical Hours
        </div>
        <div>
          <h2 className="text-2xl font-bold font-serif italic tracking-tight">
            Mass & Sacraments
          </h2>
          <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-95 mt-1 max-w-xs leading-relaxed font-sans">
            Plan your visitation and sacramental prayers around {parishName}'s daily liturgical schedule.
          </p>
        </div>
      </div>

      <div className="p-4 space-y-4">
        {/* Main Schedule Card */}
        <div className="bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] overflow-hidden shadow-xs p-5 space-y-4">
          {/* Facade photo representation */}
          <div className="rounded-2xl overflow-hidden h-28 border border-[var(--color-brand-border)]/40 relative">
            {photo && (
              <img
                src={photo}
                alt={photoAlt}
                className="w-full h-full object-cover"
              />
```

```
  )}
  <div className="absolute inset-0 bg-black/70 flex items-center justify-center">
    <span className="text-white text-[15px] font-bold font-serif italic tracking-wider text-center uppercase px-4">
      {parishName}
    </span>
  </div>
</div>

<h3 className="text-base font-bold text-center text-[var(--color-brand-text)] font-serif italic border-b border-[var(--color-brand-border)]/45 pb-1.5">
  Holy Mass Schedule
</h3>

{/* Unverified-schedule badge — must sit right next to the times
    it qualifies, not buried below them, so nobody reads the mock
    times as confirmed. See scheduleVerified in src/data.ts. */}
{schedule.length > 0 && !scheduleVerified && (
  <div className="flex items-center gap-2 p-2.5 bg-amber-50 border border-amber-300 rounded-xl">
    <AlertTriangle className="w-4 h-4 text-amber-700 shrink-0" />
    <span className="text-sm font-bold text-amber-900 leading-snug">
      Sample schedule — not yet confirmed with the parish. Please call ahead before you go.
    </span>
  </div>
)}

{schedule.length === 0 ? (
  <div className="p-4 bg-[var(--color-brand-card)]/60 rounded-xl border border-[var(--color-brand-border)]/40 text-center">
    <p className="text-[15px] text-[var(--color-brand-text)] font-bold font-sans">
      Mass schedule not yet published for this parish.
    </p>
    <p className="text-sm text-[var(--color-brand-secondary)] font-sans mt-1">
      Please contact the parish office directly to confirm Mass times before visiting.
    </p>
  </div>
) : (
  <div className="space-y-3 font-sans">
    {weekdayRows.map((row) => (
      <div
        key={row.label}
        className="flex justify-between items-center p-2.5 bg-[var(--color-brand-card)]/60 rounded-xl border border-[var(--color-brand-border)]/30"
      >
        <span className="text-[15px] font-bold text-[var(--color-brand-text)]">{row.label}</span>
        <span className="text-[15px] font-mono font-bold text-[var(--color-brand-text)]">
          {parseTimes(row.time).join(" / ")}
        </span>
      </div>
    ))}

    {/* Sunday */}
    {sundayTimes.length > 0 && (
      <div className="p-2.5 bg-[var(--color-brand-card)]/50 rounded-xl border border-[var(--color-brand-border)]/60 space-y-1.5">
        <div className="flex justify-between items-center">
          <span className="text-[15px] font-bold text-[var(--color-brand-secondary)]">Sunday Masses</span>
          <span className="text-sm bg-[var(--color-brand-primary)] text-white px-2 py-0.5 rounded-full font-bold uppercase tracking-wider">
            Lord's Day
          </span>
        </div>
        <div className="grid grid-cols-2 gap-2 text-center text-[15px] font-mono font-bold text-[var(--color-brand-text)] pt-1">
          {sundayTimes.map((time) => (
            <div key={time} className="p-1.5 bg-[var(--color-brand-card)] rounded border border-[var(--color-brand-border)]/40">
```

```
                    {time}
                  </div>
                ))}
              </div>
            </div>
          )}
        </div>
      )}

      {/* Confessions block — a diocese-wide practice, not specific to
          this parish's own published schedule. */}
      <div className="p-3 bg-amber-50 border border-amber-200 rounded-2xl space-y-1">
        <h4 className="text-[15px] font-bold text-amber-800 font-serif italic">
          Sacrament of Reconciliation
        </h4>
        <p className="text-[15px] text-amber-900 leading-normal font-sans">
          Confessions are available every first Friday of the month, or you can inquire at the Parish office to request an advanced schedule with our priest.
This is a diocese-wide practice — confirm the exact time with this parish's office.
        </p>
      </div>
    </div>

    {/* Parish info card */}
    <div className="p-4 bg-[var(--color-brand-card)] rounded-3xl border border-[var(--color-brand-border)] space-y-2 text-[15px] text-[var(--color-brand-text)]">
      <h4 className="font-bold text-[var(--color-brand-text)] font-serif italic uppercase tracking-wider text-sm flex items-center gap-1">
        <MapPin className="w-3.5 h-3.5" /> Contact & Administration
      </h4>

      {contact ? (
        <div className="space-y-1.5 font-sans text-[var(--color-brand-secondary)] font-medium pl-1.5">
          <div className="flex items-center gap-2">
            <MapPin className="w-3.5 h-3.5 shrink-0" />
            <span>{contact.address}</span>
          </div>
          {contact.phone && (
            <div className="flex items-center gap-2">
              <Phone className="w-3.5 h-3.5 shrink-0" />
              <span>{contact.phone}</span>
            </div>
          )}
          {contact.email && (
            <div className="flex items-center gap-2">
              <Mail className="w-3.5 h-3.5 shrink-0" />
              <span className="break-all">{contact.email}</span>
            </div>
          )}
          {(!contact.phone || !contact.email) && (
            <p className="text-sm text-[var(--color-brand-secondary)] pt-1">
              Phone/email not yet available for this parish office.
            </p>
          )}
        </div>
      ) : (
        <p className="text-[15px] text-[var(--color-brand-secondary)] font-medium pl-1.5 font-sans">
          Contact details not yet available for this parish.
        </p>
      )}
    </div>
  </div>
```

```
    </div>
  );
}
```

1.2.27 APP-032 — MassScheduleCard.tsx

Code ID	APP-032
File Name	MassScheduleCard.tsx
File Path	SanctDemo/src/components/MassScheduleCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	309
Functions / Classes	MassScheduleCard (exported component/function (default)); MassRow (internal function); WeekRow (internal function)
Related Files (its imports)	../data, ../lib/liturgical, ../lib/useParishContent, ./WatchBell
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/MassScheduleCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MassScheduleCard.tsx

```
import { useState } from "react";
import { AlertTriangle, CalendarDays, ChevronUp, Clock } from "lucide-react";
import { MASS_SCHEDULES } from "../data";
import { liturgicalDay } from "../lib/liturgical";
import {
  dayOfMonth,
  sundayMassCard,
  weekAhead,
  type MassDay,
  type MassTime,
} from "../lib/sundayMasses";
import { useParishContent } from "../lib/useParishContent";
import WatchBell from "./WatchBell";

interface MassScheduleCardProps {
  /** The active tour route id, e.g. "route-mhcp". */
  routeId: string;
  now: Date;
}

/**
 * The green Mass panel on Home.
 *
 * Every value is derived, not written in: the liturgical title comes from
 * liturgicalDay(), the times from the parish's own schedule (or an admin's
 * override), and the dates from the coming Sunday. The mockup's "DATE 00"
 * placeholders become the real day of the month, which is the difference
 * between a card that is decoration and one a pilgrim can plan around — for
 * six days out of seven, "Sunday Masses" with no date is ambiguous.
 *
 * The green is deliberately darker than the mockup's. White body text on that
```

```
 * lighter green measures 2.78:1, and what sits on it is the Mass TIMES; the
 * gradient here holds 4.7:1 at its lightest point. See --home-mass-from.
 */
export default function MassScheduleCard({
  routeId,
  now,
}: MassScheduleCardProps) {
  const [expanded, setExpanded] = useState(true);
  const [showWeek, setShowWeek] = useState(false);

  const managed = useParishContent(routeId);
  const compiled = MASS_SCHEDULES[routeId];

  // An admin's schedule wins over the one compiled into data.ts. A schedule
  // typed by the parish office is confirmed by definition, which is why it
  // also clears the unverified flag.
  const schedule = managed?.massSchedule ?? compiled?.schedule ?? [];
  const verified = managed?.massSchedule ? true : compiled?.scheduleVerified !== false;

  const card = sundayMassCard(schedule, now, { verified });
  const today = liturgicalDay(now);

  const hasAnything = card.anticipated !== null || card.sunday.times.length > 0;

  // Every day of the coming week, today first.
  const week = weekAhead(schedule, now);

  // Any Mass the parish has marked off this week.
  const suspended =
    (card.anticipated !== null && !card.anticipated.available) ||
    card.sunday.times.some(t => !t.available);

  return (
    <section
      className="rounded-[26px] overflow-hidden shadow-lg"
      style={{
        background: `linear-gradient(160deg, var(--home-mass-from), var(--home-mass-to))`,
      }}
    >
      {managed?.massImageUrl && (
        <img
          src={managed.massImageUrl}
          alt=""
          className="w-full h-[168px] object-cover"
          loading="lazy"
        />
      )}

      <div className="p-5 text-white">
        {/* The season sits above as an eyebrow rather than beside the title.
            Squeezed alongside, it forced a long liturgical name like
            "Wednesday of the 23rd Week in Ordinary Time" onto four cramped
            lines. It is also DROPPED when the title already contains it,
            which for most of the year it does — "Ordinary Time · Wednesday
            of the 23rd Week in Ordinary Time" says it twice. */}
        {!today.name.toLowerCase().includes(today.season.toLowerCase()) && (
          <p className="text-[13px] font-bold uppercase tracking-[0.14em] text-white/75">
            {today.season}
          </p>
```

```
)}
<div className="flex items-start justify-between gap-3">
  <h3 className="text-[21px] font-bold leading-[1.2] tracking-tight text-balance">
    {today.name}
  </h3>

  {/* Only where a Mass is actually off. A bell on a schedule with
      nothing wrong with it is an invitation to subscribe to
      silence. The icon carries it alone: a sentence explaining
      the bell took more room than the times it was about. */}
  {suspended && (
    <WatchBell
      watch={{ kind: "mass", parishId: routeId }}
      label="this Mass schedule"
      className="watch-bell--onDark"
      iconOnly
    />
  )}
</div>

{!hasAnything ? (
  // 29 of the 31 parishes have no collected schedule. An empty green
  // panel reads as broken; this says which it is.
  <p className="mt-3 text-[16px] leading-relaxed text-white/90">
    Mass times for this parish have not been collected yet.
  </p>
) : (
  <>
    {card.unverified && (
      <div className="mt-3 flex items-start gap-2 rounded-xl bg-black/25 px-3 py-2">
        <AlertTriangle className="w-4 h-4 shrink-0 mt-0.5" />
        <span className="text-[15px] font-semibold leading-snug">
          Sample times, not yet confirmed by the parish.
        </span>
      </div>
    )}

    {expanded && (
      <div className="mt-4 space-y-4">
        {card.anticipated && (
          <MassRow
            label="Anticipated Mass"
            times={[card.anticipated]}
            date={card.anticipated.date}
          />
        )}

        {card.sunday.times.length > 0 && (
          <MassRow
            label="Sunday Masses"
            times={card.sunday.times}
            date={card.sunday.date}
          />
        )}

        {/* The rest of the week.
            The card showed the anticipated Mass and Sunday and
            nothing else, while every parish also holds weekday
            Masses - so somebody looking for Tuesday's 6:00 AM
```

```
                was told in effect that it did not exist. It is
                folded away rather than listed, because Sunday is
                what most people open this for. */}
              <div className="pt-1">
                <button
                  type="button"
                  onClick={() => setShowWeek(v => !v)}
                  aria-expanded={showWeek}
                  className="flex w-full items-center justify-center gap-1.5 rounded-xl bg-black/20 px-3 py-2.5 text-[14px] font-bold text-white/90"
                >
                  <CalendarDays className="w-4 h-4" aria-hidden />
                  {showWeek ? "Hide the full schedule" : "See the full schedule"}
                </button>

                {showWeek && (
                  <div className="mt-3 space-y-3 border-t border-white/20 pt-3">
                    <p className="text-[13px] font-bold uppercase tracking-[0.12em] text-white/70">
                      The week ahead
                    </p>
                    {week.map(day => (
                      <WeekRow key={day.date.toISOString()} day={day} />
                    ))}
                  </div>
                )}
              </div>
            </div>
          )}

          <button
            type="button"
            onClick={() => setExpanded(v => !v)}
            className="mt-4 w-full flex items-center justify-center gap-1.5 text-[14px] font-bold uppercase tracking-[0.12em] text-white/85"
            aria-expanded={expanded}
          >
            <ChevronUp
              className={`w-4 h-4 transition-transform duration-200 ${expanded ? "" : "rotate-180"}`}
            />
            {expanded ? "Show less" : "Show times"}
          </button>
        </>
      )}
    </div>
  </section>
 );
}

/**
 * One labelled group of times with the date it falls on.
 *
 * The times wrap into two columns the way the mockup has them, but as a grid
 * rather than fixed columns — a parish with three Sunday Masses should not
 * leave a ragged hole, and one with eight should not overflow.
 */
/**
 * A suspended Mass is struck through and labelled, not removed.
 *
 * Someone who comes at 8:00 every Sunday needs to see that the 8:00
 * exists and is off this week. An absence tells them nothing - they
 * assume they misremembered and turn up anyway.
```

```
 */
function MassRow({ label, times, date }: { label: string; times: MassTime[]; date: Date }) {
  return (
    <div className="flex items-start justify-between gap-4">
      <div className="min-w-0 flex-1">
        <p className="text-[17px] font-bold">{label}</p>
        <ul className="mt-1.5 grid grid-cols-2 gap-x-4 gap-y-1">
          {times.map(({ time, available }) => (
            <li
              key={time}
              /* A struck-through time is given the whole row.
                 "6:00 AM NOT AVAILABLE" does not fit one half of a
                 two-column grid on a phone, and whitespace-nowrap meant
                 it did not wrap either - it ran straight over the time
                 in the next column. Full width, and the label can sit
                 beside it with room to spare. */
              className={`flex items-center gap-1.5 text-[16px] ${
                available ? "text-white/95" : "col-span-2 text-white/55"
              }`}
            >
              <Clock className="w-3.5 h-3.5 shrink-0 opacity-80" />
              <span className={`font-variant-numeric tabular-nums${available ? "" : " line-through"}`}>
                {time}
              </span>
              {!available && (
                <span className="text-[12px] font-bold uppercase tracking-wider">
                  Not available
                </span>
              )}
            </li>
          ))}
        </ul>
      </div>

      <div className="shrink-0 text-right">
        <p className="text-[13px] font-bold uppercase tracking-[0.12em] text-white/70">
          {date.toLocaleDateString("en-US", { month: "short" })}
        </p>
        <p className="text-[30px] font-black leading-none tabular-nums">{dayOfMonth(date)}</p>
      </div>
    </div>
  );
}

/**
 * One day of the week ahead.
 *
 * Laid out flat rather than as the two-column grid the Sunday row
 * uses: a weekday usually has one or two Masses, and a grid built for
 * six leaves a hole beside every one of them.
 *
 * A day with no Mass says so. Leaving the row out would make the
 * reader notice an absence between Sunday and Tuesday, which is
 * exactly the kind of thing people get wrong when deciding whether to
 * leave the house.
 */
function WeekRow({ day }: { day: MassDay }) {
  return (
    <div className={`flex items-start gap-3 ${day.isToday ? "" : "opacity-95"}`}>
```

```
      <div className="w-[88px] shrink-0">
        <p className="text-[14px] font-bold leading-tight">
          {day.isToday ? "Today" : day.day}
        </p>
        <p className="text-[12px] font-semibold uppercase tracking-wider text-white/60">
          {day.date.toLocaleDateString("en-US", { month: "short" })} {dayOfMonth(day.date)}
        </p>
      </div>

      <div className="min-w-0 flex-1">
        {day.times.length === 0 ? (
          <p className="text-[14px] text-white/55">No Mass</p>
        ) : (
          <ul className="flex flex-wrap gap-x-3 gap-y-1">
            {day.times.map(({ time, available }) => (
              <li
                key={time}
                className={`flex items-center gap-1.5 text-[15px] ${
                  available ? "text-white/95" : "text-white/55"
                }`}
              >
                <Clock className="w-3.5 h-3.5 shrink-0 opacity-70" aria-hidden />
                <span className={`tabular-nums${available ? "" : " line-through"}`}>
                  {time}
                </span>
                {!available && (
                  <span className="text-[11px] font-bold uppercase tracking-wider">
                    Off
                  </span>
                )}
              </li>
            ))}
          </ul>
        )}

        {day.note && (
          <p className="mt-1 text-[13px] leading-snug text-white/70">{day.note}</p>
        )}
      </div>
    </div>
  );
}
```

1.2.28 APP-033 — MeTab.tsx

Code ID	APP-033
File Name	MeTab.tsx
File Path	SanctDemo/src/components/MeTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	429
Functions / Classes	MeTab (exported component/function (default)); SummaryRow (internal function)
Related Files (its imports)	../types, ../lib/ui, ../data, ./MyApplications, ./EditProfileCard, ./SettingDisclosure, ../lib/firebase, ../lib/userProfile, ./NotificationsCard
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/MeTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MeTab.tsx

```
import React, { useEffect, useState } from "react";
import { Church, Settings as SettingsIcon, Footprints, Ruler, Star, Award, ClipboardList, FlaskConical, ShieldCheck, User, Bell, ChevronRight, Compass, Languages }
from "lucide-react";
import { UserProgress } from "../types";
import { t } from "../lib/ui";
import { BADGES } from "../data";
import MyApplications from "./MyApplications";
import EditProfileCard from "./EditProfileCard";
import SettingDisclosure from "./SettingDisclosure";
import { auth } from "../lib/firebase";
import { getProfile } from "../lib/userProfile";
import NotificationsCard from "./NotificationsCard";

type Application = {
  id: string;
  type: string;
  applicant: string;
  details: string;
  date: string;
  status: string;
  // Present on ministry applications only. See lib/ministryApplication.ts.
  ministryName?: string;
  parishName?: string;
  submittedAt?: string;
};

interface MeTabProps {
  /** Lets Home's greeting refresh the moment the nickname changes. */
  onNicknameChange?: (nickname: string) => void;
  onOpenApplications: () => void;
  onOpenNotifications: () => void;
```

```
  /** For the badge; the page itself owns the list. */
  unreadNotifications: number;
  isLoggedIn: boolean;
  userEmail: string;
  isAdmin: boolean;
  userProgress: UserProgress;
  applications: Application[];
  onLoginSuccess: (email: string, isAdmin: boolean) => void;
  onLogout: () => void;
  onOpenChangeParish: () => void;
  onOpenRosarySettings: () => void;
  onOpenAdmin: () => void;
  onOpenSimulator: () => void;
  /** Opens the sign-in screen. */
  onOpenSignIn: () => void;
  /** Runs the first-time walkthrough again. */
  onReplayTutorial?: () => void;
  /** The reading language for the whole app. */
  language?: "en" | "fil";
  onLanguageChange?: (language: "en" | "fil") => void;
  /**
   * Whether the app is behaving as it will for a visitor.
   *
   * On, the AR tour opens only inside a parish. Off, everything is
   * reachable from anywhere, which is how the app is tested away from
   * the churches. See lib/simulatorMode.ts.
   */
  simulatorMode?: boolean;
  onSimulatorModeChange?: (on: boolean) => void;
  /**
   * The reminder settings card, passed in rather than built here.
   *
   * It needs the parish's Mass schedule and announcements, which this
   * tab has no business knowing about. App already holds both for the
   * scheduler, so it hands the finished card down.
   */
  reminders?: React.ReactNode;
  /** The pilgrim's home parish, shown under their name in the header. */
  parishName?: string;
}

// The bottom nav's fifth tab — assembled entirely from state the app
// already tracks (sign-in, applications, prayer/visit progress). No new
// account system, just one place to find what used to be scattered across
// the hamburger menu.
export default function MeTab({
  isLoggedIn,
  userEmail,
  isAdmin,
  userProgress,
  applications,
  onLoginSuccess,
  onLogout,
  onOpenChangeParish,
  onOpenRosarySettings,
  onOpenAdmin,
  onOpenSimulator,
  onOpenSignIn,
  parishName,
```

```
  onNicknameChange,
  onOpenApplications,
  onOpenNotifications,
  unreadNotifications,
  reminders,
  onReplayTutorial,
  language,
  onLanguageChange,
  simulatorMode = false,
  onSimulatorModeChange,
}: MeTabProps) {
  // The reader's language. Falls back to English so the component still
  // renders if a caller has not been updated to pass it.
  const lang = language ?? "en";
  // The design shows a name and initials. Signed out there is no name to
  // show, so the header says "Pilgrim" rather than an empty avatar — the app
  // works fully without an account and should not imply otherwise.
  const displayName = isLoggedIn && userEmail ? userEmail.split("@")[0] : "Pilgrim";
  // Read once, then kept current by the card below rather than re-read.
  const [photoUrl, setPhotoUrl] = useState<string>("");
  useEffect(() => {
    if (!isLoggedIn || !auth.currentUser) { setPhotoUrl(""); return; }
    let live = true;
    void getProfile(auth.currentUser.uid).then(p => {
      if (live) setPhotoUrl(p?.photoUrl ?? "");
    });
    return () => { live = false; };
  }, [isLoggedIn]);

  const initials = displayName
    .split(/[.\s_-]+/)
    .filter(Boolean)
    .slice(0, 2)
    .map(part => part[0]?.toUpperCase() ?? "")
    .join("") || "P";

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto scroll-safe">

      <div className="px-5 pt-6 pb-4 flex items-center gap-4">
        <div className="flex-1 min-w-0">
          <p className="text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
            Pilgrim profile
          </p>
          <h1 className="mt-1.5 text-[24px] font-bold tracking-tight text-[var(--color-brand-text)] capitalize truncate">
            {displayName}
          </h1>
          {parishName && (
            <p className="mt-0.5 text-[16px] text-[var(--color-brand-secondary)] truncate">{parishName}</p>
          )}
        </div>
        {/* The same photograph the profile card below shows. Two avatars
            on one screen disagreeing about who you are reads as a bug -
            and it was one: changing the picture changed the card and left
            this circle on its initials. */}
        {photoUrl ? (
          <img
            src={photoUrl}
            alt=""
```

```
            className="shrink-0 w-14 h-14 rounded-full object-cover border border-[var(--color-brand-border)]"
          />
        ) : (
          <span
            aria-hidden
            className="shrink-0 w-14 h-14 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] flex items-center justify-center text-
[20px] font-bold"
          >
            {initials}
          </span>
        )}
      </div>

      <div className="px-4 pb-4 space-y-4 font-sans">
        {/* The "Your Pilgrimage" card — steps, km walked, points and stamp
            badges — is gone at the client's request.

            Worth recording why it is no loss. "640 steps · 0.42 km" were the
            literal seed values in App.tsx, and nothing could ever change
            them: the only code that adds steps is handleStationVisited, whose
            sole caller is MapTab — a component no longer rendered anywhere in
            the app. So every pilgrim on every device saw exactly 640 steps
            and 0.42 km, for ever, however far they walked.

            A progress card that cannot track progress is worse than no card,
            and this one was stating two measurements of a walk that never
            happened. If step tracking is built later, it returns with real
            numbers behind it.

            userProgress is untouched — badges and points are still awarded by
            the quiz and still stored. */}
        {/* Applications — the pilgrim's own submissions (sacrament bookings,
            ministry sign-ups, station stamps). Firestore rules already scope
            this list to the signed-in uid (or everything, for an admin), so
            it renders as-is. */}
        {/* One list, newest first, covering both generations of document
            in the collection: the new ones with a reference number and a
            status from the fixed set, and the older rows whose status is
            free text. MyApplications normalises them rather than this
            screen deciding which is which. */}
        {/* Your picture and the name the app calls you. Above the
            applications, because it is about who you are rather than what
            you have asked the parish for. */}
        {/* Signing in is the first thing on this screen now. It was
            four rows down among the settings, so somebody opening Me to
            sign in had to read past the language, the reminders and the
            rosary to find it - and signed-out is the state a new
            pilgrim arrives in. */}
        <button
          onClick={isLoggedIn ? onLogout : onOpenSignIn}
          className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-[var(--
color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
        >
          <User className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <span className="flex-1 truncate">
            {isLoggedIn ? t("me.signOut", lang) : t("me.signIn", lang)}
          </span>
          {isLoggedIn && (
            <span className="text-[14px] font-normal text-[var(--color-brand-secondary)] truncate max-w-[45%]">
```

```
      {userEmail}
    </span>
  )}
</button>

<EditProfileCard
  isLoggedIn={isLoggedIn}
  onNicknameChange={onNicknameChange}
  onPhotoChange={setPhotoUrl}
/>

{/* Two rows rather than two lists. Both of these grow without
    limit - an application is never removed, a notification
    arrives on every status change - and inline they turned the
    one screen that is meant to be "your account" into something
    you scroll past. The count is the part worth seeing here;
    the list gets a page. */}
{isLoggedIn && (
  <div className="space-y-2.5">
    <SummaryRow
      icon={<ClipboardList className="w-5 h-5" />}
      label="My Applications"
      detail={
        applications.length === 0
          ? "Nothing submitted yet"
          : `${applications.length} application${applications.length === 1 ? "" : "s"}`
      }
      onClick={onOpenApplications}
    />
    <SummaryRow
      icon={<Bell className="w-5 h-5" />}
      label={t("me.notifications", lang)}
      detail={unreadNotifications > 0
        ? `${unreadNotifications} ${t("me.unread", lang)}`
        : t("me.nothingNew", lang)}
      badge={unreadNotifications}
      onClick={onOpenNotifications}
    />
  </div>
)}

{/* Everything else, under one heading. They were a loose stack
    of rows in the order they happened to be added, which read as
    a list of leftovers rather than as settings. */}
<h4 className="pt-1 text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-sans">
  {t("me.settings", lang)}
</h4>
<div className="space-y-2">
  {/* Language and Reminders fold.
      Both were printed open above this heading: two paragraphs,
      a pair of language buttons and seven reminder switches,
      stacked over the settings a pilgrim actually came for. They
      are rows now, like everything else on the tab, and what you
      tapped is the only thing open. */}
  {language && onLanguageChange && (
    <SettingDisclosure
      icon={<Languages className="w-4 h-4" />}
      label={t("me.language", lang)}
      bare
```

```
  >
    <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5">
      <p className="mb-3 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
        {t("me.languageHelp", lang)}
      </p>
      <div className="flex gap-2">
        {([["en", "English"], ["fil", "Tagalog"]] as const).map(([value, label]) => (
          <button
            key={value}
            type="button"
            aria-pressed={language === value}
            onClick={() => onLanguageChange(value)}
            className={`flex-1 rounded-full py-3 text-[15px] font-bold border-[1.5px] transition-colors ${
              language === value
                ? "bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] border-[var(--color-brand-primary)]"
                : "border-[var(--color-brand-border)] text-[var(--color-brand-text)]"
            }`}
          >
            {label}
          </button>
        ))}
      </div>
    </div>
  </SettingDisclosure>
)}

{/* Simulator mode.
    The labels say what each position DOES, because the name
    alone is ambiguous: "simulator on" could reasonably mean
    either "pretend to be a visitor" or "pretend to be at the
    church". One of those locks the AR tour and the other
    unlocks it, so the row spells it out rather than leaving
    the reader to guess from a word. */}
<SettingDisclosure
  icon={<FlaskConical className="w-4 h-4" />}
  label={t("me.simulator", lang)}
  bare
>
  <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5">
    <p className="mb-3 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
      {simulatorMode
        ? t("me.simulatorOnHelp", lang)
        : t("me.simulatorOffHelp", lang)}
    </p>
    <div className="flex gap-2">
      {([[false, t("me.simulatorOff", lang)], [true, t("me.simulatorOn", lang)]] as const).map(
        ([value, label]) => (
          <button
            key={String(value)}
            type="button"
            aria-pressed={simulatorMode === value}
            onClick={() => onSimulatorModeChange?.(value)}
            className={`flex-1 rounded-full py-3 text-[15px] font-bold border-[1.5px] transition-colors ${
              simulatorMode === value
                ? "bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] border-[var(--color-brand-primary)]"
                : "border-[var(--color-brand-border)] text-[var(--color-brand-text)]"
            }`}
          >
            {label}
```

```
                  </button>
                ),
              )}
            </div>
          </div>
        </SettingDisclosure>

        {reminders && (
          <SettingDisclosure
            icon={<Bell className="w-4 h-4" />}
            label={t("rem.title", lang)}
            bare
          >
            {reminders}
          </SettingDisclosure>
        )}

        {/* Sign-in is a row here rather than a whole screen embedded at
            the top of Me. Inline, LoginModal brought its own "Pilgrim
            Profile" header, so Me showed two competing profile headers
            stacked on each other. */}
        {onReplayTutorial && (
          <button
            onClick={onReplayTutorial}
            className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-
[var(--color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
          >
            <Compass className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
            <span className="flex-1 truncate">{t("me.replayTutorial", lang)}</span>
          </button>
        )}

        <button
          onClick={onOpenChangeParish}
          className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-[var(-
-color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
        >
          <Church className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <span>{t("me.changeParish", lang)}</span>
        </button>
        <button
          onClick={onOpenRosarySettings}
          className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-[var(-
-color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
        >
          <SettingsIcon className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <span>{t("me.rosarySettings", lang)}</span>
        </button>

        {/* Moved here from the removed sidebar. The simulator is a demo
            tool and says so, rather than sitting unlabelled among the
            pilgrim's own settings. */}
        <button
          onClick={onOpenSimulator}
          className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-[var(-
-color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
        >
          <FlaskConical className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <span className="flex-1">Location Simulator</span>
```

```
            <span className="text-[14px] font-normal text-[var(--color-brand-secondary)]">Demo tool</span>
          </button>

          {isAdmin && (
            <button
              onClick={onOpenAdmin}
              className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-
[var(--color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
            >
              <ShieldCheck className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
              <span>Admin Panel</span>
            </button>
          )}
        </div>
      </div>
    </div>
  );
}

/**
 * One line that says how many, and opens the list.
 *
 * A chevron and a count, not a preview: a half-shown list invites you to
 * read it here, which is the thing these rows exist to stop.
 */
function SummaryRow({ icon, label, detail, badge, onClick }: {
  icon: React.ReactNode;
  label: string;
  detail: string;
  badge?: number;
  onClick: () => void;
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      className="w-full flex items-center gap-3 rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] px-5 py-4 text-left"
    >
      <span className="shrink-0 text-[var(--color-brand-primary)]">{icon}</span>
      <span className="min-w-0 flex-1">
        <span className="block text-[16px] font-bold text-[var(--color-brand-text)]">{label}</span>
        <span className="block text-[14px] text-[var(--color-brand-secondary)]">{detail}</span>
      </span>
      {badge ? (
        <span className="shrink-0 rounded-full bg-[var(--color-brand-primary)] px-2 py-0.5 text-[13px] font-bold text-[var(--color-brand-on-accent)]">
          {badge}
        </span>
      ) : null}
      <ChevronRight className="w-5 h-5 shrink-0 text-[var(--color-brand-secondary)]" aria-hidden />
    </button>
  );
}
```

1.2.29 APP-034 — MinistriesTab.tsx

Code ID	APP-034
File Name	MinistriesTab.tsx
File Path	SanctDemo/src/components/MinistriesTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	412
Functions / Classes	MinistriesTab (exported component/function (default))
Related Files (its imports)	./ParishSectionHeader, ../data, ../lib/ministrySearch, ../types, ../lib/ministryApplication, ./ApplicationForm, ./SignInFirst, ./ItemDetailSections, ../lib/itemContent, ../lib/useLanguage, ../lib/language, ../lib/contentTranslations, ../lib/applications, ../lib/useParishContent, ../lib/availability, ./AvailabilityBadge, ./WatchBell
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/MinistriesTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MinistriesTab.tsx

```
import React, { useEffect, useState } from "react";
import ParishSectionHeader from "./ParishSectionHeader";
import { MINISTRY_IMAGES, ministriesForParish } from "../data";
import { searchMinistries, suggestionsFor } from "../lib/ministrySearch";
import { Users, ChevronDown, ChevronUp, Check, CheckCircle, Sparkles, AlertCircle, Info, X, ArrowLeft, ChevronRight, Search } from "lucide-react";
import { Route } from "../types";
import { type MinistryApplicationDoc } from "../lib/ministryApplication";
import ApplicationForm from "./ApplicationForm";
import SignInFirst from "./SignInFirst";
import ItemDetailSections from "./ItemDetailSections";
import { resolveMinistry } from "../lib/itemContent";
import { useLanguage } from "../lib/useLanguage";
import { pick } from "../lib/language";
import { MINISTRY_DESCRIPTION_EN, bilingualFor } from "../lib/contentTranslations";
import { submitApplication } from "../lib/applications";
import { useParishContent } from "../lib/useParishContent";
import { isOpenForApplications, closedMessage } from "../lib/availability";
import AvailabilityBadge from "./AvailabilityBadge";
import WatchBell from "./WatchBell";

interface MinistriesTabProps {
  parish: Route;
  onAddApplication: (app: MinistryApplicationDoc) => void;
  /** The signed-in pilgrim's uid, or null when signed out. */
  uid: string | null;
  /** Their account email. The form never asks for what the account knows. */
  userEmail: string;
  onOpenSignIn: () => void;
  /**
```

```
   * A request from outside to open one ministry to its own screen.
   *
   * Sancti sets this when a pilgrim names a ministry: "open Teatro
   * Pilipino" should open Teatro Pilipino, not a list of fifteen with
   * it somewhere inside. The id makes a repeat request its own event -
   * ask twice with a look at the list in between and a bare itemId
   * would be unchanged and the second ask would do nothing.
   */
  openRequest?: { itemId: string; id: number };
}

/**
 * Stands in until the parish photographs each ministry. An illustration of
 * nothing in particular rather than a stock photo of somebody else's choir,
 * which in this card would read as a photo of THIS parish's choir.
 */
const PLACEHOLDER_MINISTRY = "/parish/placeholder-photo.svg";

/**
 * The parishes whose own ministry photographs we actually hold.
 *
 * MINISTRY_IMAGES came out of Mary Help of Christians' PDF - its
 * altar servers, its choirs, its lectors. Keyed by ministry id alone,
 * they were shown to every parish, so San Roque's page carried
 * photographs of another parish's people. The ministry ids are shared
 * between parishes; the congregations in the pictures are not.
 */
const PARISHES_WITH_OWN_PHOTOS = new Set(["route-mhcp"]);

export default function MinistriesTab({ parish, onAddApplication, uid, userEmail, onOpenSignIn, openRequest }: MinistriesTabProps) {
  const parishName = parish.name.replace(" Guide", "").replace(" Tour", "");

  // A photograph only when it is this parish's own.
  const photoFor = (ministryId: string) =>
    PARISHES_WITH_OWN_PHOTOS.has(parish.id)
      ? MINISTRY_IMAGES[ministryId] ?? PLACEHOLDER_MINISTRY
      : PLACEHOLDER_MINISTRY;

  // Which ministries this parish is currently taking applications for.
  // Parish-scoped, not global: the choir may be full at Mary Help and
  // short-handed at San Roque, and the ministries themselves are shared.
  const managed = useParishContent(parish.id);
  const { language } = useLanguage();

  const [expandedId, setExpandedId] = useState<string | null>(null);

  /// Which ministry has been opened to its own full screen, or null for the
  /// list. Separate from expandedId, which the accordion still uses: a
  /// ministry is either one of several being browsed or the only thing on
  /// the screen, and conflating those is what made the old page a list of
  /// rows that grew downwards while you read them.
  const [openId, setOpenId] = useState<string | null>(null);

  // Seeds the local state rather than replacing it, so the back button
  // and the list still work after Sancti has opened something.
  useEffect(() => {
    if (openRequest) setOpenId(openRequest.itemId);
  }, [openRequest?.id, openRequest?.itemId]);
```

```
/// What has been typed into the search box. Fifteen ministries is more
/// than anyone scrolls through looking for one they half-remember the
/// name of.
const [query, setQuery] = useState("");

// This parish's ministries, not the app's. The list used to be the
// same fifteen everywhere, which offered a pilgrim at San Roque
// groups the cathedral had never been asked about.
const ministries = ministriesForParish(parish?.id);
const matches = searchMinistries(ministries, query);
const suggestions = suggestionsFor(ministries);

/// Which ministry's application screen is open, or null.
///
/// Applying now takes over the whole screen rather than unfolding a
/// form under the card. Reading about a ministry and applying to it
/// are two different jobs, and the form needs room for its steps, its
/// validation and its review - none of which fit under a description
/// without burying the thing you were reading.
const [applyingToId, setApplyingToId] = useState<string | null>(null);

/// What the pilgrim last sent, so the list can say so when they come
/// back out of the application screen.
const [submittedMinistry, setSubmittedMinistry] = useState<string>("");
const [isSubmitted, setIsSubmitted] = useState(false);

const applyingTo = ministries.find(m => m.id === applyingToId) ?? null;

const toggleExpand = (id: string) => {
  setExpandedId(expandedId === id ? null : id);
};

// The application screen replaces everything, including the tab's own
// header. Rendering it alongside would leave "Parish Ministries" and a
// search box above a form, which is the half-measure this replaced.
if (applyingTo) {
  if (!uid) {
    return (
      <SignInFirst
        what={applyingTo.name}
        onSignIn={onOpenSignIn}
        onBack={() => setApplyingToId(null)}
      />
    );
  }
  return (
    <ApplicationForm
      kind="ministry"
      itemId={applyingTo.id}
      itemName={applyingTo.name}
      parishName={parishName}
      defaults={{ email: userEmail }}
      onClose={() => setApplyingToId(null)}
      onSubmitted={() => {
        setSubmittedMinistry(applyingTo.name);
        setIsSubmitted(true);
        // Deliberately NOT onAddApplication. That prop writes a second
        // document straight to the applications collection, and
        // ApplicationForm has already written the real one through
```

```
        // lib/applications - the one with the reference number, the
        // history entry and the notification. Calling both would file
        // every application twice.
      }}
    />
  );
}

return (
  <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto scroll-safe">
    <ParishSectionHeader
      routeId={parish.id}
      eyebrow="Serve and Volunteer"
      title="Parish Ministries"
      blurb="Join our lay ministries to serve the parish community."
      icon={<Sparkles className="w-3.5 h-3.5" />}
      collapsing
    />

    <div className="p-4 space-y-4">
      {/* These ministry types are the same across the diocese — this is
          not {parishName}'s own private list, and the app should say so
          rather than implying otherwise. */}
      <div
        className="flex items-start gap-2 p-3 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-2xl"
        hidden={openId !== null}
      >
        <Info className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0 mt-0.5" />
        <p className="text-[15px] text-[var(--color-brand-text)] font-sans leading-snug">
          These are the ministries of <strong>{parishName}</strong>. Applying below sends your application to that parish office.
        </p>
      </div>

      {/* The list: one photo card per ministry, the same card the bulletin
          rail uses on Home. It was a stack of text rows, which read as a
          form to fill in rather than as people to join. */}
      {openId === null && (
        <div className="space-y-2.5">
          <div className="flex items-baseline justify-between gap-3 pl-1">
            <h3 className="text-[15px] font-bold text-[var(--color-brand-secondary)] uppercase tracking-widest font-serif italic">
              Available Ministries
            </h3>
            <span className="text-[14px] text-[var(--color-brand-secondary)] shrink-0 tabular-nums">
              {query.trim() ? `${matches.length} of ${ministries.length}` : ministries.length}
            </span>
          </div>

          {/* In the flow, not floating over it. An overlay here would sit
              on top of the very cards it filters, which is the thing it
              exists to help you read. */}
          <div className="ministry-search">
            <Search className="w-4 h-4 shrink-0 text-[var(--color-brand-secondary)]" />
            <input
              type="search"
              value={query}
              onChange={e => setQuery(e.target.value)}
              placeholder="Search a ministry"
              aria-label="Search the ministries"
            />
```

```
  {query && (
    <button
      type="button"
      onClick={() => setQuery("")}
      aria-label="Clear the search"
      className="shrink-0 text-[var(--color-brand-secondary)]"
    >
      <X className="w-4 h-4" />
    </button>
  )}
</div>

{/* Shown only before anything is typed. Each one is checked
    against the ministries actually present, so a suggestion can
    never lead to an empty screen. */}
{!query.trim() && suggestions.length > 0 && (
  <div className="ministry-chips">
    {suggestions.map(chip => (
      <button key={chip} type="button" onClick={() => setQuery(chip)}>
        {chip}
      </button>
    ))}
  </div>
)}

{query.trim() && matches.length === 0 && (
  <p className="px-1 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
    No ministry matches &ldquo;{query.trim()}&rdquo;. Try a shorter word, or the
    short name the parish uses &mdash; MAS, EMHC, SOCCOM.
  </p>
)}

<div className="space-y-3">
  {matches.map(min => (
    <button
      key={min.id}
      type="button"
      onClick={() => {
        setOpenId(min.id);
        setExpandedId(min.id);
        setApplyingToId(null);
        setIsSubmitted(false);
      }}
      className={`ministry-card${isOpenForApplications(managed, min.id) ? "" : " is-unavailable"}`}
    >
      <span className="ministry-card__media">
        <img
          src={photoFor(min.id)}
          alt=""
          loading="lazy"
          onError={e => { (e.currentTarget as HTMLImageElement).src = PLACEHOLDER_MINISTRY; }}
        />
        <span className="ministry-card__scrim" aria-hidden />
        <span className="ministry-card__caption">{min.name}</span>
      </span>
      <span className="ministry-card__foot">
        <Users className="w-5 h-5 text-[var(--color-brand-primary)] shrink-0" />
        <span className="ministry-card__hint">{pick(bilingualFor(min.description, MINISTRY_DESCRIPTION_EN[min.id], "fil"), language)}</span>
        <ChevronRight className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
```

```
          </span>
          {/* On the list too, not only inside. Otherwise the only
              way to find out the choir is closed is to open it. */}
          {!isOpenForApplications(managed, min.id) && (
            <span className="block px-3 pb-3">
              <AvailabilityBadge open={false} kind="ministry" />
            </span>
          )}
        </button>
      ))}
    </div>
  </div>
)}

{/* One ministry, on its own screen. Only the opened one is rendered,
    so the accordion below is always the expanded state - there is
    nothing else on the page to collapse against. */}
<div className="space-y-2.5">
  <div className="space-y-2">
    {(openId ? ministries.filter(m => m.id === openId) : []).map((min) => {
      const isExpanded = true;
      const resolved = resolveMinistry(managed, min.id, language);
      return (
        <div
          key={min.id}
          className={`bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] overflow-hidden shadow-xs transition-all${
            isOpenForApplications(managed, min.id) ? "" : " is-unavailable"
          }`}
        >
          <div className="ministry-hero">
            <img
              src={photoFor(min.id)}
              alt=""
              onError={e => { (e.currentTarget as HTMLImageElement).src = PLACEHOLDER_MINISTRY; }}
            />
            <span className="ministry-hero__scrim" aria-hidden />
            <button
              type="button"
              onClick={() => { setOpenId(null); setApplyingToId(null); }}
              aria-label="Back to the ministries"
              className="ministry-hero__back"
            >
              <ArrowLeft className="w-5 h-5" />
            </button>
            <h4 className="ministry-hero__name">{min.name}</h4>
          </div>

          {isExpanded && (
            <div className="px-4 pb-4 pt-1 border-t border-[var(--color-brand-card)] bg-[var(--color-brand-card)]/30 space-y-3 font-sans">
              {/* Closed today is not closed forever. Without
                  this the only way to find out ministry reopened
                  was to keep coming back and looking at the same
                  badge. */}
              <div className="flex flex-wrap items-center gap-2">
                <AvailabilityBadge
                  open={isOpenForApplications(managed, min.id)}
                  kind="ministry"
                />
                {!isOpenForApplications(managed, min.id) && (
```

```
                        <WatchBell
                          watch={{ kind: "ministry", parishId: parish.id, itemId: min.id }}
                          label={min.name}
                        />
                      )}
                    </div>

                    {/* The parish's own words where they have written
                        any, the compiled description where they have
                        not. Every section below renders only when it
                        has something in it - a heading over an empty
                        list reads as a page that failed to load. */}
                    <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed">
                      {resolved?.about ?? min.description}
                    </p>

                    {resolved && <ItemDetailSections item={resolved} />}

                    {/* Closed ministries keep their card, their
                        description and their requirements. Hiding them
                        would leave a pilgrim wondering whether the
                        ministry had been disbanded or whether the app
                        was broken; greyed out, the answer is on the
                        card. */}
                    {isOpenForApplications(managed, min.id) ? (
                      <button
                        onClick={() => {
                          setApplyingToId(min.id);
                          setIsSubmitted(false);
                        }}
                        className="py-1.5 px-3 bg-[var(--color-brand-primary)] text-white text-sm font-bold uppercase tracking-wider rounded-full hover:bg-[var(--
color-brand-primary-dark)] transition-colors"
                      >
                        Apply now
                      </button>
                    ) : (
                      <div className="space-y-1.5">
                        <button
                          type="button"
                          disabled
                          className="py-1.5 px-3 bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)] text-[var(--color-brand-secondary)]
text-sm font-bold uppercase tracking-wider rounded-full cursor-not-allowed"
                        >
                          Not available
                        </button>
                        <p className="text-sm leading-relaxed text-[var(--color-brand-secondary)]">
                          {closedMessage("ministry", min.name)}
                        </p>
                      </div>
                    )}

                    {/* The form for THIS ministry, opened by the button
                        above. Nothing asks which ministry, and nothing
                        asks which parish: tapping Apply answered the
                        first and the dashboard answers the second. */}
                  </div>
                )}
              </div>
            );
```

```
            })}
          </div>
        </div>

        {/* Confirmation. The form lives inside the ministry card now, and
            that card may well be collapsed by the time this shows, so the
            confirmation stands on its own rather than inside it. */}
        {isSubmitted && (
          <div className="p-4 bg-green-50 border border-green-200 rounded-2xl text-center space-y-2">
            <CheckCircle className="w-8 h-8 text-green-700 mx-auto" />
            <div className="space-y-0.5">
              <h4 className="text-[15px] font-bold text-green-900 font-serif italic">Application submitted</h4>
              <p className="text-[15px] text-green-800 leading-relaxed font-sans">
                Your application to {submittedMinistry} at {parishName} is now
                <strong> Pending</strong>. We will contact you through your
                registered email regarding your application. You can follow its
                status under <strong>Me &rarr; My Application</strong>.
              </p>
            </div>
            <button
              onClick={() => setIsSubmitted(false)}
              className="mt-1.5 text-sm bg-[var(--color-brand-primary)] text-white px-3 py-1 rounded-full font-bold uppercase tracking-wide"
            >
              Done
            </button>
          </div>
        )}
      </div>
    </div>
  );
}
```

1.2.30 APP-035 — MinistrySacramentPair.tsx

Code ID	APP-035
File Name	MinistrySacramentPair.tsx
File Path	SanctDemo/src/components/MinistrySacramentPair.tsx
Language / Type	TypeScript (React, TSX)
Lines	117
Functions / Classes	MinistrySacramentPair (exported component/function (default)); Card (internal function)
Related Files (its imports)	../data, ../lib/ui, ../lib/language
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/MinistrySacramentPair.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MinistrySacramentPair.tsx

```
import type { ReactNode } from "react";
import { Sparkles, Users } from "lucide-react";
import { MINISTRIES, SACRAMENTS } from "../data";
import { t } from "../lib/ui";
import type { Language } from "../lib/language";

/**
 * These two photographs are Mary Help of Christians' own.
 *
 * They came out of that parish's PDF: its altar servers, a christening
 * in its church. They were shown on every parish's Home, so somebody
 * opening San Roque was looking at a photograph of another parish's
 * people and reasonably taking it for their own. For a guide whose
 * whole claim is that it tells you the truth about the church you are
 * standing in, that is the worst kind of wrong - confident and
 * plausible.
 *
 * San Roque's own photographs have not been collected yet. Until they
 * are, it gets the mark instead of somebody else's congregation.
 */
const PARISH_PHOTOS: Record<string, { ministry: string; sacrament: string }> = {
  "route-mhcp": {
    ministry: "/parish/ministry-altar-servers.jpg",
    sacrament: "/parish/sacraments-christening.jpg",
  },
};

/**
 * The two ways in, as their own section on Home.
 *
 * They used to live inside the bulletin, between the verse and the
 * announcements. The bulletin is now the last thing on Home — it is
 * the part that changes daily, not the part people come looking for —
```

```
 * and these two are a destination the directory at the top jumps to,
 * so they need to be a section of their own rather than a passenger
 * inside another one.
 */
export default function MinistrySacramentPair({
  language,
  onNavigate,
  parishId,
}: {
  language: Language;
  onNavigate: (tab: "ministries" | "sacraments") => void;
  /** Whose Home this is. Decides whether there are photographs to show. */
  parishId?: string;
}) {
  const photos = parishId ? PARISH_PHOTOS[parishId] : undefined;

  return (
    <div className="bulletin-pair">
      <Card
        icon={<Users className="w-4 h-4" />}
        label={t("quick.ministries", language)}
        hint={language === "fil" ? "Sumali sa isang grupo sa parokyang ito." : "Join a group in this parish."}
        caption={MINISTRIES[0]?.name}
        imageUrl={photos?.ministry}
        onClick={() => onNavigate("ministries")}
      />
      <Card
        icon={<Sparkles className="w-4 h-4" />}
        label={t("quick.sacraments", language)}
        hint={language === "fil" ? "Inaayos kasama ang tanggapan ng parokya." : "Arranged with the parish office."}
        caption={SACRAMENTS[0]?.name}
        imageUrl={photos?.sacrament}
        onClick={() => onNavigate("sacraments")}
      />
    </div>
  );
}

function Card({
  icon, label, hint, caption, imageUrl, onClick,
}: {
  icon: ReactNode;
  label: string;
  hint: string;
  /** Named over the photograph — an example of what is inside. */
  caption?: string;
  imageUrl?: string;
  onClick: () => void;
}) {
  return (
    <button type="button" onClick={onClick} className="bulletin-section">
      {!imageUrl && (
        /* No photograph for this parish yet. The mark, on the parish's
           own tint, rather than a stock picture or another church's
           congregation - it says "nothing here yet" without pretending
           otherwise. */
        <span className="bulletin-section__photo bulletin-section__photo--mark">
          <img src="/ui/sanctiwalk-icon-512.png" alt="" loading="lazy" />
        </span>
```

```
      )}

      {imageUrl && (
        <span className="bulletin-section__photo">
          <img src={imageUrl} alt="" loading="lazy" />
          {caption && (
            <>
              {/* A scrim, not a shadow: what sits behind the caption is a
                  photograph whose brightness cannot be assumed. */}
              <span className="bulletin-section__scrim" aria-hidden />
              <span className="bulletin-section__caption">{caption}</span>
            </>
          )}
        </span>
      )}
      <span className="bulletin-section__body">
        <span className="bulletin-section__label">
          <span className="bulletin-section__icon">{icon}</span>
          {label}
        </span>
        <span className="bulletin-section__hint">{hint}</span>
      </span>
    </button>
  );
}
```

1.2.31 APP-036 — MyApplications.tsx

Code ID	APP-036
File Name	MyApplications.tsx
File Path	SanctDemo/src/components/MyApplications.tsx
Language / Type	TypeScript (React, TSX)
Lines	124
Functions / Classes	MyApplications (exported component/function (default)); Card (internal function)
Related Files (its imports)	../lib/applicationView
Where It Is Used	src/components/MeTab.tsx
How to Recognize It	Open SanctDemo/src/components/MyApplications.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MyApplications.tsx

```
import { ClipboardList, Church, Users, Clock } from "lucide-react";
import { toViewApplication, sortByNewest, type ViewApplication } from "../lib/applicationView";

/**
 * "My Applications" — what the pilgrim submitted and where it has got to.
 *
 * ## Why the reference number is prominent
 *
 * It is the only thing a parish office can look someone up by over the
 * phone. Buried in small print it may as well not exist, so it sits under
 * the title in a monospaced face that survives being read aloud.
 *
 * ## Why the status is a word and a tint, never a tint alone
 *
 * "Approved" and "Rejected" are the two most consequential words the app
 * says to anyone. A colour-blind reader, a printed page or a bright day in
 * a church porch all take the colour away; the word has to carry it.
 */

const TONE: Record<string, { bg: string; fg: string }> = {
  pending:      { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-text)" },
  under_review: { bg: "#E8EEF8", fg: "#1C2C56" },
  approved:     { bg: "#E4F0E8", fg: "#2F5A41" },
  rejected:     { bg: "#FBE9E4", fg: "#8E3F2C" },
  completed:    { bg: "#EDE7F6", fg: "#4A3B70" },
  unknown:      { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-secondary)" },
};

export default function MyApplications({ applications }: { applications: unknown[] }) {
  const items = sortByNewest(
    applications.map(a => toViewApplication(a as Record<string, unknown>)),
  );
```

```
  return (
    <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5 space-y-3">
      <h4 className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-sans flex items-center gap-1.5">
        <ClipboardList className="w-4 h-4" /> My Applications
      </h4>

      {items.length === 0 ? (
        <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          Nothing submitted yet. Sacraments and ministries are on your parish's
          dashboard, and anything you apply for appears here with its progress.
        </p>
      ) : (
        <>
          <ul className="space-y-3">
            {items.map(app => <Card key={app.id} app={app} />)}
          </ul>
          <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
            The parish will contact you at your registered email. Quote your reference
            number if you ring them.
          </p>
        </>
      )}
    </div>
  );
}

function Card({ app }: { app: ViewApplication }) {
  const tone = TONE[app.status] ?? TONE.unknown;
  const Icon = app.kind === "ministry" ? Users : app.kind === "sacrament" ? Church : ClipboardList;

  return (
    <li className="bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] p-3.5">
      <div className="flex items-start justify-between gap-2">
        <span className="min-w-0 flex items-start gap-2">
          <Icon className="w-4 h-4 mt-0.5 shrink-0 text-[var(--color-brand-secondary)]" aria-hidden />
          <span className="min-w-0">
            <span className="block font-bold text-[15px] leading-snug text-[var(--color-brand-text)]">
              {app.title}
            </span>
            {app.referenceNumber && (
              <span className="block font-mono text-[13px] text-[var(--color-brand-secondary)]">
                {app.referenceNumber}
              </span>
            )}
          </span>
        </span>

        <span
          className="shrink-0 rounded-full px-2.5 py-1 text-[13px] font-bold whitespace-nowrap"
          style={{ background: tone.bg, color: tone.fg }}
        >
          {app.statusLabel}
        </span>
      </div>

      <dl className="mt-2 text-[14px] space-y-0.5">
        {app.submitted && (
          <div className="flex gap-2">
            <dt className="text-[var(--color-brand-secondary)] w-[7.5rem] shrink-0">Submitted</dt>
```

```
            <dd className="text-[var(--color-brand-text)]">{app.submitted}</dd>
          </div>
        )}
        {app.lastUpdate && (
          <div className="flex gap-2">
            <dt className="text-[var(--color-brand-secondary)] w-[7.5rem] shrink-0">Last update</dt>
            <dd className="text-[var(--color-brand-text)] inline-flex items-center gap-1.5">
              <Clock className="w-3.5 h-3.5 text-[var(--color-brand-secondary)]" aria-hidden />
              {app.lastUpdate}
            </dd>
          </div>
        )}
      </dl>

      {app.note && (
        <p className="mt-2 rounded-xl bg-[var(--color-brand-card-sunk)] px-3 py-2 text-[14px] leading-relaxed text-[var(--color-brand-text)]">
          <span className="font-bold">From the parish: </span>{app.note}
        </p>
      )}

      {/* Only when there is nothing structured to show. New submissions
          keep their answers in formData; this is the old flattened
          sentence, and printing it alongside real fields would repeat
          them. */}
      {!app.referenceNumber && app.details && app.details !== app.title && (
        <p className="mt-2 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
          {app.details}
        </p>
      )}
    </li>
  );
}
```

1.2.32 APP-037 — MyApplicationsPage.tsx

Code ID	APP-037
File Name	MyApplicationsPage.tsx
File Path	SanctDemo/src/components/MyApplicationsPage.tsx
Language / Type	TypeScript (React, TSX)
Lines	203
Functions / Classes	MyApplicationsPage (exported component/function (default))
Related Files (its imports)	../lib/firebase, ../lib/applicationView
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/MyApplicationsPage.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MyApplicationsPage.tsx

```
import { useState } from "react";
import { ArrowLeft, Church, ClipboardList, Clock, Trash2, Users } from "lucide-react";
import { doc, deleteDoc } from "firebase/firestore";
import { db } from "../lib/firebase";
import { toViewApplication, sortByNewest, type ViewApplication } from "../lib/applicationView";

/**
 * "My Applications", on a screen of its own.
 *
 * It was a card on the Me tab, under the profile and above the
 * notifications, and it grew with every application a pilgrim ever made -
 * so the one screen that is meant to be "your account" became a list you
 * had to scroll past to reach anything. A row that says how many, opening
 * its own page, keeps Me short and gives the list room.
 */

const TONE: Record<string, { bg: string; fg: string }> = {
  pending:      { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-text)" },
  under_review: { bg: "#E8EEF8", fg: "#1C2C56" },
  approved:     { bg: "#E4F0E8", fg: "#2F5A41" },
  rejected:     { bg: "#FBE9E4", fg: "#8E3F2C" },
  completed:    { bg: "#EDE7F6", fg: "#4A3B70" },
  unknown:      { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-secondary)" },
};

export default function MyApplicationsPage({
  applications,
  onBack,
}: {
  applications: unknown[];
  onBack: () => void;
}) {
  const [withdrawing, setWithdrawing] = useState<ViewApplication | null>(null);
```

```
const [busy, setBusy] = useState(false);
const [error, setError] = useState<string | null>(null);

const items = sortByNewest(
  applications.map(a => toViewApplication(a as Record<string, unknown>)),
);

async function withdraw(app: ViewApplication) {
  setBusy(true);
  setError(null);
  try {
    await deleteDoc(doc(db, "applications", app.id));
    setWithdrawing(null);
  } catch {
    // The rules refuse once the parish has acted, which is the common
    // case for anything not pending - say that rather than "failed".
    setError(
      "That could not be withdrawn. Your parish has already started on it — " +
      "please contact the office instead.",
    );
  } finally {
    setBusy(false);
  }
}

return (
  <div className="flex-1 flex flex-col min-h-0 overflow-y-auto scroll-safe bg-[var(--color-brand-card)]">
    <div className="px-5 pt-5 pb-3">
      <button
        onClick={onBack}
        className="inline-flex items-center gap-1.5 text-[15px] font-semibold text-[var(--color-brand-primary)]"
      >
        <ArrowLeft className="w-4 h-4" /> Back
      </button>
      <h1 className="mt-3 text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
        My Applications
      </h1>
      <p className="text-[15px] text-[var(--color-brand-secondary)]">
        {items.length === 0
          ? "Nothing submitted yet."
          : `${items.length} application${items.length === 1 ? "" : "s"}`}
      </p>
    </div>

    {error && (
      <p role="alert" className="mx-4 mb-3 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
        {error}
      </p>
    )}

    <div className="px-4 pb-6">
      {items.length === 0 ? (
        <div className="rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-6 text-center">
          <ClipboardList className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
          <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
            Sacraments and ministries are on your parish's dashboard. Anything you
            apply for appears here with its progress.
          </p>
        </div>
```

```
) : (
  <ul className="space-y-3">
    {items.map(app => {
      const tone = TONE[app.status] ?? TONE.unknown;
      const Icon = app.kind === "ministry" ? Users : app.kind === "sacrament" ? Church : ClipboardList;
      return (
        <li key={app.id} className="rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-4">
          <div className="flex items-start justify-between gap-2">
            <span className="min-w-0 flex items-start gap-2">
              <Icon className="w-4 h-4 mt-0.5 shrink-0 text-[var(--color-brand-secondary)]" aria-hidden />
              <span className="min-w-0">
                <span className="block font-bold text-[16px] leading-snug text-[var(--color-brand-text)]">
                  {app.title}
                </span>
                {app.referenceNumber && (
                  <span className="block font-mono text-[13px] text-[var(--color-brand-secondary)]">
                    {app.referenceNumber}
                  </span>
                )}
              </span>
            </span>
            <span className="shrink-0 rounded-full px-2.5 py-1 text-[13px] font-bold whitespace-nowrap"
                  style={{ background: tone.bg, color: tone.fg }}>
              {app.statusLabel}
            </span>
          </div>

          <dl className="mt-2 text-[14px] space-y-0.5">
            {app.submitted && (
              <div className="flex gap-2">
                <dt className="text-[var(--color-brand-secondary)] w-[7.5rem] shrink-0">Submitted</dt>
                <dd className="text-[var(--color-brand-text)]">{app.submitted}</dd>
              </div>
            )}
            {app.lastUpdate && (
              <div className="flex gap-2">
                <dt className="text-[var(--color-brand-secondary)] w-[7.5rem] shrink-0">Last update</dt>
                <dd className="text-[var(--color-brand-text)] inline-flex items-center gap-1.5">
                  <Clock className="w-3.5 h-3.5 text-[var(--color-brand-secondary)]" aria-hidden />
                  {app.lastUpdate}
                </dd>
              </div>
            )}
          </dl>

          {app.note && (
            <p className="mt-2 rounded-xl bg-[var(--color-brand-card)] px-3 py-2 text-[14px] leading-relaxed text-[var(--color-brand-text)]">
              <span className="font-bold">From the parish: </span>{app.note}
            </p>
          )}

          {/* Only when there is nothing structured to show. New
              submissions keep their answers in formData; this is
              the old flattened sentence, and printing it beside
              real fields would repeat them. */}
          {!app.referenceNumber && app.details && app.details !== app.title && (
            <p className="mt-2 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
              {app.details}
            </p>
```

```
                )}

                {/* Withdraw, not delete, and only before the parish has
                    started. Once it is under review the record belongs
                    to their work as much as yours. */}
                {app.status === "pending" && (
                  <button
                    onClick={() => setWithdrawing(app)}
                    className="mt-3 inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-error)]"
                  >
                    <Trash2 className="w-3.5 h-3.5" /> Withdraw
                  </button>
                )}
              </li>
            );
          })}
        </ul>
      )}

      {items.length > 0 && (
        <p className="mt-4 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          The parish will contact you at your registered email. Quote your reference
          number if you ring them.
        </p>
      )}
    </div>

    {withdrawing && (
      <div role="dialog" aria-modal="true" aria-label="Withdraw application"
           className="fixed inset-0 z-50 flex items-center justify-center bg-black/40 px-5">
        <div className="w-full max-w-[380px] rounded-[22px] bg-[var(--color-brand-card)] p-6">
          <h3 className="text-[18px] font-bold font-serif italic">Withdraw this application?</h3>
          <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
            Your application for <strong>{withdrawing.title}</strong> will be removed and
            your parish will no longer see it. You can apply again later.
          </p>
          <div className="mt-5 flex gap-2">
            <button onClick={() => setWithdrawing(null)} disabled={busy}
                    className="flex-1 rounded-full border-[1.5px] border-[var(--color-brand-border)] py-3 text-[15px] font-bold">
              Keep it
            </button>
            <button onClick={() => void withdraw(withdrawing)} disabled={busy}
                    className="flex-1 rounded-full bg-[var(--color-brand-error)] text-white py-3 text-[15px] font-bold disabled:opacity-60">
              {busy ? "Withdrawing…" : "Yes, withdraw"}
            </button>
          </div>
        </div>
      </div>
    )}
  </div>
);
}
```

NavigationOverlay.tsx — listed in full as MAP-002.

1.2.33 APP-038 — NotificationsCard.tsx

Code ID	APP-038
File Name	NotificationsCard.tsx
File Path	SanctDemo/src/components/NotificationsCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	93
Functions / Classes	NotificationsCard (exported component/function (default))
Related Files (its imports)	../lib/notifications, ../types
Where It Is Used	src/components/MeTab.tsx
How to Recognize It	Open SanctDemo/src/components/NotificationsCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — NotificationsCard.tsx

```
import { useEffect, useState } from "react";
import { Bell, BellOff, Check } from "lucide-react";
import { watchNotifications, markRead, unreadCount } from "../lib/notifications";
import type { NotificationDoc } from "../types";

/**
 * The pilgrim's notifications.
 *
 * ## Why unread is a weight and a dot, not a dot alone
 *
 * The unread state has to survive a printed page, a bright porch and a
 * reader who cannot separate the hues, so the row carries a bolder title
 * as well as the marker.
 *
 * ## Why reading one is an explicit tap
 *
 * Marking everything read on open is the usual shortcut and it loses
 * things: a person opening the tab to check something else should not come
 * back later to find the parish's decision already dismissed. The tap is
 * the acknowledgement.
 */
export default function NotificationsCard({ isLoggedIn }: { isLoggedIn: boolean }) {
  const [items, setItems] = useState<NotificationDoc[]>([]);

  useEffect(() => {
    if (!isLoggedIn) { setItems([]); return; }
    return watchNotifications(setItems);
  }, [isLoggedIn]);

  if (!isLoggedIn) return null;
```

```
  const unread = unreadCount(items);

  return (
    <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5 space-y-3">
      <h4 className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-sans flex items-center gap-1.5">
        <Bell className="w-4 h-4" /> Notifications
        {unread > 0 && (
          <span className="ml-1 rounded-full bg-[var(--color-brand-primary)] px-2 py-0.5 text-[12px] text-[var(--color-brand-on-accent)]">
            {unread} new
          </span>
        )}
      </h4>

      {items.length === 0 ? (
        <p className="flex items-start gap-2 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          <BellOff className="w-4 h-4 mt-0.5 shrink-0" aria-hidden />
          Nothing yet. When you apply for something, and when your parish responds,
          it appears here.
        </p>
      ) : (
        <ul className="space-y-2.5">
          {items.map(n => (
            <li
              key={n.id}
              className="rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-3.5"
            >
              <div className="flex items-start justify-between gap-2">
                <p className={`text-[15px] leading-snug text-[var(--color-brand-text)] ${n.readAt ? "font-semibold" : "font-bold"}`}>
                  {!n.readAt && (
                    <span
                      aria-label="Unread"
                      className="mr-1.5 inline-block h-2 w-2 rounded-full align-middle bg-[var(--color-brand-primary)]"
                    />
                  )}
                  {n.title}
                </p>
                {!n.readAt && n.id && (
                  <button
                    type="button"
                    onClick={() => void markRead(n.id!)}
                    aria-label={`Mark "${n.title}" as read`}
                    className="shrink-0 inline-flex items-center gap-1 rounded-full border border-[var(--color-brand-border)] px-2.5 py-1 text-[13px] font-bold text-
[var(--color-brand-secondary)]"
                  >
                    <Check className="w-3.5 h-3.5" /> Read
                  </button>
                )}
              </div>
              <p className="mt-1 text-[14px] leading-relaxed text-[var(--color-brand-text)]">{n.body}</p>
              {n.createdAt && (
                <p className="mt-1 text-[13px] text-[var(--color-brand-secondary)]">
                  {new Date(n.createdAt).toLocaleString(undefined, {
                    month: "long", day: "numeric", hour: "numeric", minute: "2-digit",
                  })}
                </p>
              )}
            </li>
          ))}
```

```
        </ul>
      )}
    </div>
  );
}
```

1.2.34 APP-039 — NotificationsPage.tsx

Code ID	APP-039
File Name	NotificationsPage.tsx
File Path	SanctDemo/src/components/NotificationsPage.tsx
Language / Type	TypeScript (React, TSX)
Lines	128
Functions / Classes	NotificationsPage (exported component/function (default))
Related Files (its imports)	../lib/notifications, ../lib/firebase, ../types
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/NotificationsPage.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — NotificationsPage.tsx

```
import { useEffect, useState } from "react";
import { ArrowLeft, Bell, BellOff, Check, CheckCheck, Trash2 } from "lucide-react";
import { watchNotifications, markRead, unreadCount } from "../lib/notifications";
import { doc, deleteDoc } from "firebase/firestore";
import { db } from "../lib/firebase";
import type { NotificationDoc } from "../types";

/**
 * Notifications, on their own screen.
 *
 * Same reasoning as My Applications: it was a card on the Me tab that
 * grew without limit, pushing everything else down. A row with a count,
 * opening a page, keeps Me readable and lets this list breathe.
 */
export default function NotificationsPage({ onBack }: { onBack: () => void }) {
  const [items, setItems] = useState<NotificationDoc[]>([]);

  useEffect(() => watchNotifications(setItems), []);

  const unread = unreadCount(items);

  async function remove(id: string) {
    try {
      await deleteDoc(doc(db, "notifications", id));
    } catch (err) {
      // Yours to delete by the rules, so a failure here is the network.
      console.debug("[notifications] delete failed:", err);
    }
  }

  async function markAllRead() {
    // Sequential rather than parallel: a parish list is short, and a
    // burst of writes is more likely to trip a rate limit than to feel
```

```
    // faster.
    for (const n of items) {
      if (!n.readAt && n.id) await markRead(n.id);
    }
  }

  return (
    <div className="flex-1 flex flex-col min-h-0 overflow-y-auto scroll-safe bg-[var(--color-brand-card)]">
      <div className="px-5 pt-5 pb-3">
        <button
          onClick={onBack}
          className="inline-flex items-center gap-1.5 text-[15px] font-semibold text-[var(--color-brand-primary)]"
        >
          <ArrowLeft className="w-4 h-4" /> Back
        </button>

        <div className="mt-3 flex items-end justify-between gap-3">
          <div className="min-w-0">
            <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
              Notifications
            </h1>
            <p className="text-[15px] text-[var(--color-brand-secondary)]">
              {items.length === 0
                ? "Nothing yet."
                : unread > 0 ? `${unread} unread` : "All caught up"}
            </p>
          </div>
          {unread > 0 && (
            <button
              onClick={() => void markAllRead()}
              className="shrink-0 inline-flex items-center gap-1.5 rounded-full border border-[var(--color-brand-border)] px-3 py-2 text-[14px] font-bold text-[var(-
-color-brand-secondary)]"
            >
              <CheckCheck className="w-3.5 h-3.5" /> Mark all read
            </button>
          )}
        </div>
      </div>

      <div className="px-4 pb-6">
        {items.length === 0 ? (
          <div className="rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-6 text-center">
            <BellOff className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
            <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
              When you apply for something, and when your parish responds, it appears here.
            </p>
          </div>
        ) : (
          <ul className="space-y-2.5">
            {items.map(n => (
              <li key={n.id}
                  className="rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card-sunk)] p-4">
                <div className="flex items-start justify-between gap-2">
                  <p className={`text-[16px] leading-snug text-[var(--color-brand-text)] ${n.readAt ? "font-semibold" : "font-bold"}`}>
                    {!n.readAt && (
                      <span aria-label="Unread"
                            className="mr-1.5 inline-block h-2 w-2 rounded-full align-middle bg-[var(--color-brand-primary)]" />
                    )}
                    {n.title}
```

```
                  </p>
                  <Bell className="w-4 h-4 shrink-0 mt-0.5 text-[var(--color-brand-secondary)]" aria-hidden />
                </div>

                <p className="mt-1 text-[15px] leading-relaxed text-[var(--color-brand-text)]">{n.body}</p>

                {n.createdAt && (
                  <p className="mt-1 text-[13px] text-[var(--color-brand-secondary)]">
                    {new Date(n.createdAt).toLocaleString(undefined, {
                      month: "long", day: "numeric", hour: "numeric", minute: "2-digit",
                    })}
                  </p>
                )}

                <div className="mt-2.5 flex flex-wrap gap-3">
                  {!n.readAt && n.id && (
                    <button onClick={() => void markRead(n.id!)}
                            className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-primary)]">
                      <Check className="w-3.5 h-3.5" /> Mark read
                    </button>
                  )}
                  {n.id && (
                    <button onClick={() => void remove(n.id!)}
                            aria-label={`Delete "${n.title}"`}
                            className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-secondary)]">
                      <Trash2 className="w-3.5 h-3.5" /> Delete
                    </button>
                  )}
                </div>
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
}
```

1.2.35 APP-040 — Onboarding.tsx

Code ID	APP-040
File Name	Onboarding.tsx
File Path	SanctDemo/src/components/Onboarding.tsx
Language / Type	TypeScript (React, TSX)
Lines	154
Functions / Classes	Onboarding (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/mapSearch, ../lib/geo, ./ParishListRow, ../data/diocese-parishes.json
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/Onboarding.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — Onboarding.tsx

```
import { useMemo, useState } from "react";
import { Search } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import { searchParishesScored } from "../lib/mapSearch";
import { haversineMeters, type Coordinates } from "../lib/geo";
import ParishListRow from "./ParishListRow";
import parishData from "../data/diocese-parishes.json";

interface DioceseParish {
  id: string;
  name: string;
  vicariate: string;
  vicariateVerified?: boolean;
  coordinates: Coordinates;
  status: string;
}

const PARISHES = (parishData as { parishes: DioceseParish[] }).parishes;

interface OnboardingProps {
  onChoose: (parishId: string) => void;
}

/**
 * "Choose your home parish" — the redesign's first screen.
 *
 * The mockup says 47 parishes; this diocese has 31 in the dataset, and the
 * count is read from the data rather than written into the copy so it cannot
 * drift from what the list actually shows.
 *
 * Parishes are ordered by distance when there is a GPS fix, because the one
```

```
 * you are standing in is almost always the one you want. Without a fix they
 * fall back to alphabetical rather than an arbitrary file order.
 */
export default function Onboarding({ onChoose }: OnboardingProps) {
  const { position, gpsStatus } = usePresence();
  const [query, setQuery] = useState("");
  const [selected, setSelected] = useState<string | null>(null);

  const rows = useMemo<{ parish: DioceseParish; metres: number | null }[]>(() => {
    const withDistance = PARISHES.map(p => ({
      parish: p,
      metres: position ? haversineMeters(position, p.coordinates) : null,
    }));

    if (query.trim()) {
      const searchable = PARISHES.map(p => ({
        id: p.id,
        name: p.name,
        location: p.vicariate ?? "",
        coordinates: p.coordinates,
      }));
      const hits = searchParishesScored(query, searchable, { origin: position });
      return hits.map(h => ({
        parish: PARISHES.find(p => p.id === h.parish.id)!,
        metres: h.distanceMeters,
      }));
    }

    return withDistance.sort((a, b) => {
      // The two finished parishes first, always.
      //
      // This list is somebody's first screen, and alphabetical order
      // opened it on Birhen ng Lourdes - a parish whose page is a name
      // and nothing else yet. The only two that lead anywhere were
      // buried halfway down thirty-one rows. Distance and the alphabet
      // still order everything after them.
      const aLive = a.parish.status === "live" ? 0 : 1;
      const bLive = b.parish.status === "live" ? 0 : 1;
      if (aLive !== bLive) return aLive - bLive;

      if (a.metres !== null && b.metres !== null) return a.metres - b.metres;
      return a.parish.name.localeCompare(b.parish.name);
    });
  }, [query, position?.lat, position?.lng]);

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] min-h-0">
      <div className="px-5 pt-8 pb-4 shrink-0">
        <p className="text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
          SanctiWalk
        </p>
        <h1 className="mt-3 text-[30px] font-bold leading-[1.15] tracking-tight text-[var(--color-brand-text)]">
          Choose your
          <br />
          home parish.
        </h1>
        <p className="mt-3 text-[16px] leading-relaxed text-[var(--color-brand-secondary)]">
          Mass times, the day's mystery and your walking distances are all measured from here.
          You can change it any time.
```

```
        </p>

        <div className="mt-5 flex items-center gap-2.5 px-4 py-3 rounded-full bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)]">
          <Search className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <input
            type="search"
            value={query}
            onChange={e => setQuery(e.target.value)}
            placeholder={`Search ${PARISHES.length} parishes`}
            aria-label={`Search ${PARISHES.length} parishes`}
            className="flex-1 min-w-0 bg-transparent border-0 outline-none text-[16px] text-[var(--color-brand-text)] placeholder:text-[var(--color-brand-
secondary)]"
          />
        </div>

        {/* Said once, here, rather than repeated on every row. */}
        {gpsStatus !== "granted" && (
          <p className="mt-3 text-[15px] text-[var(--color-brand-secondary)]">
            Turn on location to see which parishes are nearest.
          </p>
        )}
      </div>

      <div className="flex-1 min-h-0 overflow-y-auto px-5 pb-4 space-y-2">
        {rows.length === 0 && (
          <p className="py-8 text-center text-[16px] text-[var(--color-brand-secondary)]">
            No parish matches "{query.trim()}".
          </p>
        )}
        {rows.map(({ parish, metres }) => (
          <div key={parish.id}>
          <ParishListRow
            parish={{
              id: parish.id,
              name: parish.name,
              // Flagged, because these assignments are inferred from the
              // nearest vicariate seat, not taken from the diocese.
              meta: parish.vicariate ? `${parish.vicariate} (unconfirmed)` : undefined,
              coordinates: parish.coordinates,
              isLive: parish.status === "live",
            }}
            distanceMeters={metres}
            selected={selected === parish.id}
            onSelect={setSelected}
          />
          </div>
        ))}
      </div>

      <div className="shrink-0 px-5 pt-3 pb-5 border-t border-[var(--color-brand-border)] bg-[var(--color-brand-card)]">
        <button
          type="button"
          disabled={!selected}
          onClick={() => selected && onChoose(selected)}
          className="w-full py-3.5 rounded-full text-[16px] font-semibold bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] disabled:opacity-40"
        >
          Continue
        </button>
        <p className="mt-2.5 text-center text-[15px] text-[var(--color-brand-secondary)]">
```

```
          Diocese of Kalookan · {PARISHES.length} parishes
        </p>
      </div>
    </div>
  );
}
```

1.2.36 APP-041 — ParishCard.tsx

Code ID	APP-041
File Name	ParishCard.tsx
File Path	SanctDemo/src/components/ParishCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	64
Functions / Classes	parishDisplayName (internal function); nextMassLine (internal function); ParishCard (exported component/function (default))
Related Files (its imports)	../types, ../data, ../lib/schedule, ../context/PresenceContext, ./MassNowBadge, ../lib/geo, ../lib/routing
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ParishCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishCard.tsx

```
import { Route } from "../types";
import { MASS_SCHEDULES, PARISH_PATRON_SAINTS } from "../data";
import { nextMass } from "../lib/schedule";
import { usePresence } from "../context/PresenceContext";
import MassNowBadge from "./MassNowBadge";
import { haversineMeters } from "../lib/geo";
import { formatDistance, formatWalkingMinutes, WALK_SPEED_MPS } from "../lib/routing";

interface ParishCardProps {
  parish: Route;
  onSelect: (parishId: string) => void;
}

function parishDisplayName(parish: Route): string {
  return parish.name.replace(" Guide", "").replace(" Tour", "");
}

// "Next Mass · Wednesday 6:00 AM" — matches the prototype's line verbatim.
// A parish whose schedule is unverified (see data.ts's scheduleVerified
// flag) still shows the sample time but flags it, same as PresenceSheet
// already does elsewhere — a card must never state a placeholder Mass time
// as if it were confirmed.
function nextMassLine(parishId: string): string {
  const parishSchedule = MASS_SCHEDULES[parishId];
  const upcoming = nextMass(parishSchedule?.schedule ?? [], new Date());
  if (!upcoming) return "Mass schedule coming soon";

  const label = `Next Mass · ${upcoming.day} ${upcoming.time}`;
  return parishSchedule?.scheduleVerified === false ? `${label} (sample, unconfirmed)` : label;
}
```

```
export default function ParishCard({ parish, onSelect }: ParishCardProps) {
  const patron = PARISH_PATRON_SAINTS[parish.id] ?? "Patron saint not yet listed";
  const { position } = usePresence();

  // Straight-line, and labelled "direct" so it is never mistaken for a
  // walking distance — the card is a summary, and firing an OSRM request per
  // card just to fill a caption is what used to draw unasked-for routes
  // across the map. "Get directions" on the pin is where the real route
  // lives. Absent entirely when there is no fix, rather than showing a dash.
  const straightLine =
    position && parish.coordinates ? haversineMeters(position, parish.coordinates) : null;

  return (
    <button type="button" className="parish-card" onClick={() => onSelect(parish.id)}>
      <div className="parish-card__row">
        <h3 className="parish-card__name">{parishDisplayName(parish)}</h3>
        <span className="chip--live">Live</span>
      </div>
      <MassNowBadge routeId={parish.id} className="mt-1.5" />
      {straightLine !== null && (
        <p className="parish-card__distance">
          {formatDistance(straightLine)} direct
          <span className="parish-card__walk">
            · about {formatWalkingMinutes(straightLine / WALK_SPEED_MPS / 60)} on foot
          </span>
        </p>
      )}
      <p className="parish-card__patron">{patron}</p>
      <hr className="parish-card__rule" />
      <p className="parish-card__meta">{nextMassLine(parish.id)}</p>
    </button>
  );
}
```

1.2.37 APP-042 — ParishContentEditor.tsx

Code ID	APP-042
File Name	ParishContentEditor.tsx
File Path	SanctDemo/src/components/ParishContentEditor.tsx
Language / Type	TypeScript (React, TSX)
Lines	307
Functions / Classes	ParishContentEditor (exported component/function (default))
Related Files (its imports)	../data, ../lib/contrast, ../lib/parishContent, ../lib/useParishContent, ../lib/schedule
Where It Is Used	src/components/AdminPortal.tsx
How to Recognize It	Open SanctDemo/src/components/ParishContentEditor.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishContentEditor.tsx

```
import { useEffect, useRef, useState } from "react";
import { Upload, Check, AlertTriangle, Loader2 } from "lucide-react";
import { ROUTES, MASS_SCHEDULES } from "../data";
import { checkParishColour, readableTextOn, parseHex } from "../lib/contrast";
import { saveParishContent, uploadParishPhoto, type ParishContent } from "../lib/parishContent";
import { useParishContent } from "../lib/useParishContent";
import { parseTimes } from "../lib/schedule";

const DAYS = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];

interface ParishContentEditorProps {
  /** The signed-in admin, recorded against each change. */
  editorEmail: string;
}

/**
 * The parish office's own editor: patron photograph, description, Mass times
 * and brand colour, without a developer.
 *
 * Mass times are the reason this is worth building. Twenty-nine of the
 * thirty-one parishes have none, and every one collected so far had to be
 * typed into data.ts by hand — so the field team could gather them but not
 * enter them. Here they can.
 */
export default function ParishContentEditor({ editorEmail }: ParishContentEditorProps) {
  const [parishId, setParishId] = useState(ROUTES[0]?.id ?? "");
  const managed = useParishContent(parishId);

  const [description, setDescription] = useState("");
  const [themeColor, setThemeColor] = useState("");
  const [schedule, setSchedule] = useState<Record<string, string>>({});
  const [status, setStatus] = useState<{ kind: "idle" | "busy" | "ok" | "error"; message: string }>({
    kind: "idle",
```

```
  message: "",
});
const fileRef = useRef<HTMLInputElement | null>(null);

// Reload the form whenever the parish changes or someone else saves.
useEffect(() => {
  const fallback = MASS_SCHEDULES[parishId]?.schedule ?? [];
  const asMap: Record<string, string> = {};
  for (const day of DAYS) {
    asMap[day] =
      managed?.massSchedule?.find(e => e.day === day)?.time ??
      fallback.find(e => e.day === day)?.time ??
      "";
  }
  setSchedule(asMap);
  setDescription(managed?.description ?? "");
  setThemeColor(managed?.themeColor ?? "");
}, [parishId, managed]);

const colourVerdict = themeColor.trim() ? checkParishColour(themeColor.trim()) : null;

const save = async (patch: Partial<ParishContent>) => {
  setStatus({ kind: "busy", message: "Saving…" });
  try {
    await saveParishContent(parishId, patch, editorEmail);
    setStatus({ kind: "ok", message: "Saved. Every device updates within a second." });
  } catch (error) {
    const message = error instanceof Error ? error.message : String(error);
    console.error("[ParishContentEditor]", message);
    setStatus({
      kind: "error",
      message: /permission|insufficient/i.test(message)
        ? "Saving was refused. Your account is not in the admins collection, or the Firestore rules are not deployed."
        : "Could not save. Check your connection and try again.",
    });
  }
};

const onPickPhoto = async (file: File) => {
  setStatus({ kind: "busy", message: "Uploading photo…" });
  const result = await uploadParishPhoto(parishId, file);
  if (result.url) {
    await save({ photoUrl: result.url });
  } else {
    setStatus({ kind: "error", message: result.error ?? "The photo could not be uploaded." });
  }
};

const saveSchedule = () => {
  // Empty days are dropped rather than stored blank — "no Mass listed" and
  // "Mass at nothing o'clock" are different things downstream.
  const entries = DAYS.filter(day => schedule[day]?.trim()).map(day => ({
    day,
    time: schedule[day].trim(),
  }));
  void save({ massSchedule: entries });
};

const invalidDays = DAYS.filter(day => {
```

```
    const value = schedule[day]?.trim();
    if (!value) return false;
    return parseTimes(value).some(t => !/^\d{1,2}:\d{2}\s*(AM|PM)$/i.test(t.trim()));
  });

  return (
    <div className="space-y-5">
      <div>
        <label className="block text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)] mb-1.5">
          Parish
        </label>
        <select
          value={parishId}
          onChange={e => setParishId(e.target.value)}
          className="w-full p-3 rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] text-[16px] text-[var(--color-brand-text)]"
        >
          {ROUTES.map(route => (
            <option key={route.id} value={route.id}>
              {route.name.replace(" Guide", "").replace(" Tour", "")}
            </option>
          ))}
        </select>
      </div>

      {/* Patron photograph */}
      <section className="rounded-[22px] border border-[var(--color-brand-border)] p-4 space-y-3">
        <h4 className="text-[16px] font-semibold text-[var(--color-brand-text)]">Patron photograph</h4>
        {managed?.photoUrl ? (
          <img
            src={managed.photoUrl}
            alt="Current patron photograph"
            className="w-full h-40 object-cover rounded-xl border border-[var(--color-brand-border)]"
          />
        ) : (
          <p className="text-[15px] text-[var(--color-brand-secondary)]">
            No photograph yet. The home screen shows the parish name alone until one is added.
          </p>
        )}
        <input
          ref={fileRef}
          type="file"
          accept="image/*"
          className="hidden"
          onChange={e => {
            const file = e.target.files?.[0];
            if (file) void onPickPhoto(file);
            e.target.value = "";
          }}
        />
        <button
          type="button"
          onClick={() => fileRef.current?.click()}
          className="w-full py-3 rounded-full text-[16px] font-semibold bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] flex items-center
justify-center gap-2"
        >
          <Upload className="w-4 h-4" />
          {managed?.photoUrl ? "Replace photograph" : "Upload photograph"}
        </button>
        <p className="text-[15px] text-[var(--color-brand-secondary)]">
```

```
    A photograph of this parish's own patron image. Under 5 MB — most phones offer to resize
    when sharing. Please do not use a picture of a different church.
  </p>
</section>

{/* Description */}
<section className="rounded-[22px] border border-[var(--color-brand-border)] p-4 space-y-3">
  <h4 className="text-[16px] font-semibold text-[var(--color-brand-text)]">Parish description</h4>
  <textarea
    value={description}
    onChange={e => setDescription(e.target.value)}
    rows={4}
    placeholder="Mary Help of Christians Parish in Maypajo is a vibrant Catholic community…"
    className="w-full p-3 rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] text-[16px] text-[var(--color-brand-text)]"
  />
  <button
    type="button"
    onClick={() => void save({ description: description.trim() })}
    className="w-full py-3 rounded-full text-[16px] font-semibold border border-[var(--color-brand-primary)] text-[var(--color-brand-primary)]"
  >
    Save description
  </button>
</section>

{/* Mass schedule */}
<section className="rounded-[22px] border border-[var(--color-brand-border)] p-4 space-y-3">
  <h4 className="text-[16px] font-semibold text-[var(--color-brand-text)]">Mass schedule</h4>
  <p className="text-[15px] text-[var(--color-brand-secondary)]">
    One line per day. Separate multiple Masses with commas, e.g. <strong>6:00 AM, 6:00 PM</strong>.
    Leave a day blank if there is no Mass.
  </p>
  {DAYS.map(day => (
    <div key={day} className="flex items-center gap-3">
      <span className="w-[92px] shrink-0 text-[15px] text-[var(--color-brand-secondary)]">{day}</span>
      <input
        value={schedule[day] ?? ""}
        onChange={e => setSchedule(s => ({ ...s, [day]: e.target.value }))}
        placeholder="—"
        className={`flex-1 min-w-0 p-2.5 rounded-xl border bg-[var(--color-brand-card)] text-[16px] text-[var(--color-brand-text)] ${
          invalidDays.includes(day)
            ? "border-[var(--color-brand-error)]"
            : "border-[var(--color-brand-border)]"
        }`}
      />
    </div>
  ))}
  {invalidDays.length > 0 && (
    <p className="text-[15px] text-[var(--color-brand-error)]">
      {invalidDays.join(", ")}: times must look like <strong>6:00 AM</strong>. A time the app
      cannot read is skipped, which would silently hide a Mass.
    </p>
  )}
  <button
    type="button"
    disabled={invalidDays.length > 0}
    onClick={saveSchedule}
    className="w-full py-3 rounded-full text-[16px] font-semibold bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] disabled:opacity-40"
  >
    Save Mass schedule
```

```
        </button>
      </section>

      {/* Theme colour */}
      <section className="rounded-[22px] border border-[var(--color-brand-border)] p-4 space-y-3">
        <h4 className="text-[16px] font-semibold text-[var(--color-brand-text)]">Parish colour</h4>
        <div className="flex items-center gap-3">
          <input
            type="color"
            value={parseHex(themeColor) ? themeColor : "#1C2C56"}
            onChange={e => setThemeColor(e.target.value.toUpperCase())}
            className="w-14 h-12 rounded-xl border border-[var(--color-brand-border)] bg-transparent"
            aria-label="Pick a parish colour"
          />
          <input
            value={themeColor}
            onChange={e => setThemeColor(e.target.value.toUpperCase())}
            placeholder="#1C2C56"
            className="flex-1 min-w-0 p-3 rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] text-[16px] font-mono text-[var(--color-
brand-text)]"
          />
        </div>

        {/* The whole point of validating: an admin sees the consequence before
            a congregation does. */}
        {colourVerdict && (
          <div
            className={`rounded-xl p-3 text-[15px] leading-relaxed ${
              colourVerdict.ok
                ? "bg-[var(--color-brand-card-sunk)] text-[var(--color-brand-text)]"
                : "bg-[var(--color-brand-error)] text-[var(--color-brand-on-accent)]"
            }`}
          >
            {colourVerdict.ok ? (
              <>
                <span className="flex items-center gap-1.5 font-semibold">
                  <Check className="w-4 h-4" /> Readable
                </span>
                <span
                  className="mt-2 inline-block px-3 py-1.5 rounded-md text-[15px] font-semibold"
                  style={{
                    background: themeColor,
                    color: readableTextOn(parseHex(themeColor)!).hex,
                  }}
                >
                  Button preview
                </span>
              </>
            ) : (
              <span className="flex items-start gap-1.5">
                <AlertTriangle className="w-4 h-4 shrink-0 mt-0.5" />
                {colourVerdict.problem}
              </span>
            )}
          </div>
        )}

        <button
          type="button"
```

```
          disabled={!colourVerdict?.ok}
          onClick={() => void save({ themeColor: themeColor.trim() })}
          className="w-full py-3 rounded-full text-[16px] font-semibold bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] disabled:opacity-40"
        >
          Save colour
        </button>
        <button
          type="button"
          onClick={() => {
            setThemeColor("");
            void save({ themeColor: "" });
          }}
          className="w-full py-2.5 text-[16px] font-semibold text-[var(--color-brand-secondary)]"
        >
          Reset to the app's own colour
        </button>
      </section>

      {status.kind !== "idle" && (
        <p
          className={`flex items-center gap-2 text-[16px] ${
            status.kind === "error"
              ? "text-[var(--color-brand-error)]"
              : "text-[var(--color-brand-text)]"
          }`}
        >
          {status.kind === "busy" && <Loader2 className="w-4 h-4 animate-spin shrink-0" />}
          {status.message}
        </p>
      )}

      {managed?.updatedBy && (
        <p className="text-[15px] text-[var(--color-brand-secondary)]">
          Last changed by {managed.updatedBy}
          {managed.updatedAt ? ` on ${new Date(managed.updatedAt).toLocaleString()}` : ""}.
        </p>
      )}
    </div>
  );
}
```

1.2.38 APP-043 — ParishHero.tsx

Code ID	APP-043
File Name	ParishHero.tsx
File Path	SanctDemo/src/components/ParishHero.tsx
Language / Type	TypeScript (React, TSX)
Lines	220
Functions / Classes	ParishHero (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ParishHero.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishHero.tsx

```
import { useEffect, useRef } from "react";
import { MapPin } from "lucide-react";

interface ParishHeroProps {
  parishName: string;
  /** "Maypajo, Caloocan City" — shown under the name. */
  location?: string;
  /** The patron saint, shown over the image when there is one. */
  patron?: string;
  /**
   * A photograph of this parish's patron image. Absent for every parish whose
   * photography has not been collected, which is currently all of them — see
   * the note in Dashboard.
   */
  imageUrl?: string;
}

/** How tall the image is at rest, and over how much scroll it collapses. */
const HERO_HEIGHT_PX = 220;
const COLLAPSE_OVER_PX = 180;
/** The gap under the photograph, which collapses with it. */
const HERO_MARGIN_PX = 14;

/**
 * The collapsing parish header.
 *
 * The image shrinks in step with the scroll position rather than on a timer,
 * so it tracks the finger exactly and reverses when the finger does — a
 * duration-based animation always lags a scroll and feels detached from it.
 *
 * The name and location never collapse. They are the answer to "which parish
 * am I looking at", which stays true no matter how far down the screen is,
 * and a header that takes its own title away with the image leaves the rest
```

```
 * of the page unlabelled.
 *
 * When there is no photograph, this renders the name block alone — which is
 * simply the collapsed state, so nothing has to be designed twice and no
 * placeholder or stock image stands in for a parish it is not.
 */
export default function ParishHero({ parishName, location, patron, imageUrl }: ParishHeroProps) {
  const frameRef = useRef<HTMLDivElement | null>(null);
  const stickyRef = useRef<HTMLDivElement | null>(null);
  // A zero-height marker above the photograph. See `apply` for why the
  // collapse is measured from this rather than from scrollTop.
  const sentinelRef = useRef<HTMLDivElement | null>(null);

  useEffect(() => {
    const frame = frameRef.current;
    const sticky = stickyRef.current;
    if (!sticky) return;

    // The scroller is an ancestor (the tab content area in App.tsx), not this
    // component, so it has to be found rather than assumed.
    //
    // It must be one that ACTUALLY scrolls, not merely one declaring
    // overflow-y: auto. Several ancestors here declare it while comfortably
    // fitting their content, and stopping at the first of those pinned
    // scrollTop to a permanent zero: the listener fired, the maths ran, and
    // the photograph sat at full height at every scroll position.
    let scroller: HTMLElement | null = sticky.parentElement;
    while (scroller && scroller !== document.documentElement) {
      const overflowY = getComputedStyle(scroller).overflowY;
      const declaresScroll = overflowY === "auto" || overflowY === "scroll";
      if (declaresScroll && scroller.scrollHeight > scroller.clientHeight + 4) break;
      scroller = scroller.parentElement;
    }
    if (!scroller || scroller === document.documentElement) return;
    const view = scroller;

    // Whatever padding the scroller already had. The compensation below adds
    // to this rather than replacing it.
    const basePaddingBottom = parseFloat(getComputedStyle(view).paddingBottom) || 0;
    let ticking = false;

    const apply = () => {
      ticking = false;

      // Progress is measured from a sentinel ABOVE the photograph rather than
      // from the frame itself, so the reading cannot be moved by the very
      // collapse it is driving.
      const sentinel = sentinelRef.current;
      const travelled = sentinel
        ? view.getBoundingClientRect().top - sentinel.getBoundingClientRect().top
        : view.scrollTop;

      // The hairline under the pinned name, on only once it has actually
      // reached the top. Measured against the scroller, not the viewport:
      // the scroller starts partway down the window inside the phone frame,
      // so an IntersectionObserver against the viewport called it unstuck at
      // every position.
      sticky.dataset.stuck = String(
        sticky.getBoundingClientRect().top - view.getBoundingClientRect().top <= 1,
```

```
    );

    if (!frame || !imageUrl) return;

    const t = Math.min(1, Math.max(0, travelled / COLLAPSE_OVER_PX));
    const height = Math.round(HERO_HEIGHT_PX * (1 - t));
    const marginBottom = Math.round(HERO_MARGIN_PX * (1 - t));
    frame.style.height = `${height}px`;
    // Fades slightly ahead of the collapse so the last few pixels do not
    // pop out of existence.
    frame.style.opacity = String(Math.max(0, 1 - t * 1.15));
    // Margin collapses with it, or a gap is left behind where it was.
    frame.style.marginBottom = `${marginBottom}px`;
    // The 1px border top and bottom survives a zero height and leaves a
    // 2px hairline sitting where the photo was. Dropped once collapsed.
    frame.style.borderWidth = height === 0 ? "0" : "1px";

    // Pad the scroller only by however much is needed to stop the finger
    // outrunning the page — usually nothing.
    //
    // Shrinking the header shortens the page. On a SHORT page that pushes
    // the maximum scroll position below where the finger already is, the
    // browser clamps scrollTop, the progress falls, and the photograph
    // grows back — which lengthens the page again. The two settle against
    // each other with the photo stuck part-open.
    //
    // The first fix padded by the full reclaimed height unconditionally.
    // That works, but on a long page — which Home became once the Mass and
    // History cards arrived — the loop cannot happen at all, and the
    // padding was simply 234px of dead space under the last card. So the
    // shortfall is measured instead: how far past the natural end of the
    // page the current position would fall. On a long page that is zero.
    const currentPad = parseFloat(view.style.paddingBottom) || 0;
    const naturalRange = view.scrollHeight - currentPad - view.clientHeight;
    const shortfall = Math.max(0, view.scrollTop - naturalRange);
    view.style.paddingBottom = shortfall > 0 ? `${basePaddingBottom + shortfall}px` : "";
  };

  const onScroll = () => {
    // rAF-throttled and written straight to style: driving this through
    // React state would re-render the whole dashboard on every scroll frame.
    if (ticking) return;
    ticking = true;
    requestAnimationFrame(apply);
  };

  apply();
  view.addEventListener("scroll", onScroll, { passive: true });
  return () => {
    view.removeEventListener("scroll", onScroll);
    view.style.paddingBottom = "";
  };
}, [imageUrl]);

// A fragment, not a wrapper element, and this matters more than it looks.
// `position: sticky` is confined to its PARENT's box, so any wrapper here
// would bound the pinned name to the height of the header rather than the
// page — it would unstick and scroll away the moment the header block
// ended, which is exactly what it did when this rendered a <header>. With
```

```
  // no wrapper, the caller's own container becomes the containing block.
  return (
    <>
      <div ref={sentinelRef} aria-hidden className="h-0" />

      {imageUrl && (
        <div
          ref={frameRef}
          // overflow-hidden is what makes the collapse read as a shrinking
          // window onto the photo; the img inside keeps its own aspect ratio
          // via object-cover, so nothing squashes as the frame loses height.
          className="relative mx-5 overflow-hidden rounded-[22px] border border-[var(--color-brand-border)] will-change-[height,opacity]"
          style={{ height: HERO_HEIGHT_PX, marginBottom: HERO_MARGIN_PX }}
        >
          <img
            src={imageUrl}
            alt={patron ? `${patron}, ${parishName}` : parishName}
            className="w-full h-full object-cover"
            // Devotional photographs are almost always shot upright with the
            // figure in the upper half — a crowned statue in its niche, a
            // retablo, a facade. Cropping to the vertical centre of a portrait
            // photo in a landscape frame therefore lands on vestments rather
            // than the face. Biasing the focus upward frames the subject.
            style={{ objectPosition: "center 32%" }}
            loading="eager"
          />
          {patron && (
            <>
              {/* A gradient scrim rather than a drop-shadow alone. These are
                  photographs of gilded statues under warm light, so the
                  brightness under the caption is unpredictable — a shadow
                  holds up over dark vestments and disappears over gold. The
                  scrim guarantees the contrast whatever the photo does, and
                  reads as a caption band rather than text dropped on a face. */}
              <span
                aria-hidden
                className="absolute inset-x-0 bottom-0 h-20 bg-gradient-to-t from-black/75 via-black/35 to-transparent"
              />
              <span className="absolute left-4 bottom-3 right-4 text-[15px] font-semibold text-white leading-snug">
                {patron}
              </span>
            </>
          )}
        </div>
      )}

      {/* Pinned to the top of the scroller, so once the photograph has
          collapsed the parish name is still on screen — which is the whole
          point of a collapsing header rather than one that simply scrolls
          away. It carries the page background and a hairline that appears
          only once it is actually stuck, so at rest it reads as ordinary
          text and while scrolling it reads as a bar. */}
      <div
        ref={stickyRef}
        className="sticky top-0 z-10 bg-[var(--color-brand-bg)] pt-1.5 pb-2.5 px-5 transition-[border-color] duration-200 border-b border-transparent data-
[stuck=true]:border-[var(--color-brand-border)]"
      >
        <h1 className="text-[24px] font-bold leading-tight tracking-tight text-[var(--color-brand-text)]">
          {parishName}
```

```
        </h1>
        {location && (
          <p className="mt-0.5 flex items-center gap-1.5 text-[16px] text-[var(--color-brand-secondary)]">
            <MapPin className="w-4 h-4 shrink-0 text-[var(--color-brand-primary)]" />
            {location}
          </p>
        )}
      </div>
    </>
  );
}
```

1.2.39 APP-044 — ParishListRow.tsx

Code ID	APP-044
File Name	ParishListRow.tsx
File Path	SanctDemo/src/components/ParishListRow.tsx
Language / Type	TypeScript (React, TSX)
Lines	118
Functions / Classes	ParishRowData (exported type); ParishListRow (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/geo, ../lib/routing
Where It Is Used	src/components/Onboarding.tsx, src/components/SearchScreen.tsx
How to Recognize It	Open SanctDemo/src/components/ParishListRow.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishListRow.tsx

```
import { Check } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import { haversineMeters, type Coordinates } from "../lib/geo";
import { formatDistance, formatWalkingMinutes, WALK_SPEED_MPS } from "../lib/routing";

export interface ParishRowData {
  id: string;
  name: string;
  /** Vicariate, town, or whatever secondary line the caller has. */
  meta?: string;
  coordinates?: Coordinates;
  /** True for the two parishes with a tour, schedule and history behind them. */
  isLive?: boolean;
}

interface ParishListRowProps {
  parish: ParishRowData;
  onSelect: (parishId: string) => void;
  selected?: boolean;
  /** Pre-computed distance, when the caller already has it (e.g. from search). */
  distanceMeters?: number | null;
}

/**
 * One parish in a list — the row shared by onboarding, search and anywhere
 * else the diocese is enumerated.
 *
 * The design's rows read "Vicariate of San Roque · 0.7 km away". Both halves
 * are conditional here for the same reason: the vicariate in this dataset is
 * inferred rather than official (flagged vicariateVerified: false), and the
 * distance needs a GPS fix. Neither is invented when missing — the row simply
 * shows less.
 */
```

```
export default function ParishListRow({ parish, onSelect, selected, distanceMeters }: ParishListRowProps) {
  const { position } = usePresence();

  const metres =
    distanceMeters !== undefined
      ? distanceMeters
      : position && parish.coordinates
        ? haversineMeters(position, parish.coordinates)
        : null;

  /*
   * Only the two parishes with content behind them can be opened.
   *
   * The other twenty-nine were tappable and led to a page with a name
   * and nothing else - no Mass times, no history, no ministries,
   * because nobody has collected them yet. A row that opens an empty
   * page does not read as "not ready", it reads as broken, and it is
   * the kind of thing a visitor finds during a demonstration.
   *
   * Disabled rather than hidden. The diocese has thirty-one parishes
   * and the directory should show thirty-one; what it should not do is
   * promise that all of them are finished.
   */
  const openable = Boolean(parish.isLive);

  return (
    <button
      type="button"
      onClick={() => openable && onSelect(parish.id)}
      disabled={!openable}
      aria-pressed={selected}
      aria-label={openable ? undefined : `${parish.name} — details not published yet`}
      className={`w-full text-left flex items-center gap-3 px-4 py-3.5 rounded-2xl border transition-colors ${
        selected
          ? "bg-[var(--color-brand-card-sunk)] border-[var(--color-brand-primary)]"
          : parish.isLive
            // The two parishes with a tour, a schedule and a history behind
            // them. In a list of thirty-one they were one badge apart from
            // thirty rows that open to almost nothing, so the row itself
            // now carries the difference.
            ? "bg-[var(--color-brand-card-sunk)] border-[var(--color-brand-primary)] shadow-[0_1px_3px_rgba(16,34,86,0.10)]"
            : "bg-[var(--color-brand-card)] border-[var(--color-brand-border)] opacity-60 cursor-default"
      }`}
    >
      <div className="flex-1 min-w-0">
        <div className="flex items-center gap-2">
          <p
            className={`text-[16px] leading-snug text-[var(--color-brand-text)] ${
              parish.isLive ? "font-bold" : "font-semibold"
            }`}
          >
            {parish.name}
          </p>
          {parish.isLive && (
            <span className="text-[14px] font-semibold px-2 py-0.5 rounded-md bg-[var(--color-brand-success)] text-[var(--color-brand-on-accent)]">
              Live
            </span>
          )}
        </div>
```

```
        <p className="mt-0.5 text-[15px] text-[var(--color-brand-secondary)] leading-snug">
          {[
            parish.meta,
            metres !== null ? `${formatDistance(metres)} away` : null,
          ]
            .filter(Boolean)
            .join(" · ") || "Distance unknown — location is off"}
        </p>
        {metres !== null && (
          <p className="mt-0.5 text-[15px] text-[var(--color-brand-primary)] font-medium tabular-nums">
            about {formatWalkingMinutes(metres / WALK_SPEED_MPS / 60)} on foot
          </p>
        )}
      </div>

      {selected && (
        <span
          aria-hidden
          className="shrink-0 w-7 h-7 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] flex items-center justify-center"
        >
          <Check className="w-4 h-4" />
        </span>
      )}
    </button>
  );
}
```

1.2.40 APP-045 — ParishSectionHeader.tsx

Code ID	APP-045
File Name	ParishSectionHeader.tsx
File Path	SanctDemo/src/components/ParishSectionHeader.tsx
Language / Type	TypeScript (React, TSX)
Lines	201
Functions / Classes	scrollerFor (internal function); ParishSectionHeader (exported component/function (default))
Related Files (its imports)	../lib/findScroller, ../data, ../lib/parishTheme
Where It Is Used	src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/ParishSectionHeader.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishSectionHeader.tsx

```
import { useEffect, useRef, type CSSProperties, type ReactNode } from "react";
import { findScroller } from "../lib/findScroller";
import { PARISH_HEADER_FACES, PARISH_HEADER_IMAGES, PARISH_PATRON_IMAGES } from "../data";
import { parishThemeStyle } from "../lib/parishTheme";

interface ParishSectionHeaderProps {
  /** Route id, so the photograph and the colours are this parish's. */
  routeId: string;
  /** The small line above the title: "Serve and Volunteer". */
  eyebrow: string;
  title: string;
  blurb?: string;
  /** Sits in the eyebrow, before the words. */
  icon?: ReactNode;
  /**
   * Collapses on scroll the way Home's welcome band does: the eyebrow and
   * blurb leave, the title shrinks and pins. Opt-in, because Ministries and
   * Sacraments are short pages where a band that moves would only twitch.
   */
  collapsing?: boolean;
}

/**
 * The element this band actually scrolls inside.
 *
 * findScroller looks for App's named .app-scroll first, which is right for
 * Home - but Ministries, Sacraments and Pray each declare their own
 * overflow-y-auto, and the content scrolls in THAT while .app-scroll never
 * moves. A listener on the wrong one reads the same number at every scroll
 * position, so the band measures a collapse and then never collapses.
 *
```

```
 * So: nearest ancestor that declares a scroll, whatever its current
 * content height; only then fall back to the named one.
 */
function scrollerFor(el: HTMLElement): HTMLElement | null {
  let node = el.parentElement;
  while (node && node !== document.documentElement) {
    const overflowY = getComputedStyle(node).overflowY;
    if (overflowY === "auto" || overflowY === "scroll") return node;
    node = node.parentElement;
  }
  return findScroller(el);
}

/** The title's size once collapsed, over its size at rest. */
const PINNED_SCALE = 0.72;
/** Room above and below the title in the collapsed bar. */
const PINNED_PAD_TOP = 10;
const PINNED_PAD_BOTTOM = 10;

/**
 * The header on Ministries, Sacraments and the section pages like them.
 *
 * It used to be a flat navy panel with a giant faded outline icon in the
 * corner — the same panel on every screen, for every parish. Two things were
 * wrong with that. The colour was Mary Help's blue, so San Roque's ministries
 * were introduced in another parish's livery; and the decoration was a
 * clip-art icon where the app everywhere else uses the parish's own
 * photography.
 *
 * This is the welcome band from Home, reused: the patron's photograph bled in
 * from the right behind the words, the parish's own ink underneath, and the
 * one italic serif the design uses for a parish's name. A pilgrim moving
 * from Home into Ministries should not feel they have changed apps.
 */
export default function ParishSectionHeader({
  routeId,
  eyebrow,
  title,
  blurb,
  icon,
  collapsing = false,
}: ParishSectionHeaderProps) {
  // Where the patron's face sits in that photograph, as a fraction of its
  // height. Read before the collapse effect, which needs it to work out how
  // far to slide the picture.
  const face = PARISH_HEADER_FACES[routeId];

  const bandRef = useRef<HTMLElement | null>(null);
  const titleRef = useRef<HTMLHeadingElement | null>(null);

  useEffect(() => {
    if (!collapsing) return;
    const band = bandRef.current;
    const titleEl = titleRef.current;
    if (!band || !titleEl) return;

    const view = scrollerFor(band);
    if (!view) return;
```

```
let collapse = 0;
let ticking = false;

const measure = () => {
  // offsetTop, not a rect: the title already carries a transform by the
  // time this re-runs, and a rect would report the moved, scaled box -
  // so the maths driving the transform would read its own output.
  let titleTop = 0;
  let node: HTMLElement | null = titleEl;
  while (node && node !== band) {
    titleTop += node.offsetTop;
    node = node.offsetParent as HTMLElement | null;
  }
  const titleHeight = titleEl.offsetHeight;
  const collapsedHeight = PINNED_PAD_TOP + titleHeight * PINNED_SCALE + PINNED_PAD_BOTTOM;
  const bandHeight = band.offsetHeight;
  collapse = Math.max(0, bandHeight - collapsedHeight);

  band.style.setProperty("--ps-collapse", `${collapse}px`);
  band.style.setProperty("--ps-title-dy", `${PINNED_PAD_TOP - titleTop + collapse}px`);
  band.style.setProperty("--ps-scale-delta", String(1 - PINNED_SCALE));

  // How far to slide the photograph DOWN so the patron's face lands in
  // the collapsed strip.
  //
  // Sticky with a negative top takes the band's TOP off screen, so what
  // survives is its bottom edge - and an untransformed photograph shows
  // its own bottom there, which on these frames is a hem. A fixed
  // parallax was the first attempt and cropped the face off at some
  // heights and not others, because the right amount depends on how
  // tall the band is and where the face sits in the picture.
  //
  // Clamped to [0, collapse] so the picture still covers the strip:
  // any further and its own edge would leave bare navy behind it.
  const faceFraction = face ?? 0.3;
  const photoDy = Math.min(
    collapse,
    Math.max(0, collapse - faceFraction * bandHeight + collapsedHeight / 2),
  );
  band.style.setProperty("--ps-photo-dy", `${photoDy}px`);
};

const apply = () => {
  ticking = false;
  if (collapse <= 0) {
    band.style.setProperty("--ps-t", "0");
    return;
  }
  const pushed = view.getBoundingClientRect().top - band.getBoundingClientRect().top;
  band.style.setProperty("--ps-t", Math.min(1, Math.max(0, pushed / collapse)).toFixed(4));
};

const onScroll = () => {
  if (ticking) return;
  ticking = true;
  requestAnimationFrame(apply);
};
const remeasure = () => { measure(); apply(); };
```

```
    remeasure();
    view.addEventListener("scroll", onScroll, { passive: true });
    const resize = new ResizeObserver(remeasure);
    resize.observe(view);
    resize.observe(band);
    return () => {
      view.removeEventListener("scroll", onScroll);
      resize.disconnect();
    };
  }, [collapsing, routeId, eyebrow, title, blurb, face]);
  const photo = PARISH_HEADER_IMAGES[routeId] ?? PARISH_PATRON_IMAGES[routeId];

  // Where the patron's face sits in that photograph, as a fraction of its
  // height. Without it the crop is top-aligned and Mary's face ends up
  // above the band while the band shows her hem.

  const style = {
    ...parishThemeStyle(routeId),
    ...(face != null ? { "--psh-face": `${Math.round(face * 100)}%` } : {}),
  } as CSSProperties;

  return (
    <header
      className={`parish-section${collapsing ? " parish-section--collapsing" : ""}`}
      style={style}
      ref={bandRef}
    >
      {photo && (
        <>
          <div
            className="parish-section__photo"
            style={{ backgroundImage: `url(${photo})` }}
            aria-hidden
          />
          {/* A scrim, not a shadow. These are gilded statues under warm
              light, so the brightness behind the text cannot be assumed: a
              shadow holds over dark vestments and vanishes over gold. */}
          <div className="parish-section__scrim" aria-hidden />
        </>
      )}

      <div className="parish-section__content">
        <p className="parish-section__eyebrow">
          {icon}
          {eyebrow}
        </p>
        <h2 className="parish-section__title" ref={titleRef}>{title}</h2>
        {blurb && <p className="parish-section__blurb">{blurb}</p>}
      </div>
    </header>
  );
}
```

1.2.41 APP-046 — ParishWelcomeHeader.tsx

Code ID	APP-046
File Name	ParishWelcomeHeader.tsx
File Path	SanctDemo/src/components/ParishWelcomeHeader.tsx
Language / Type	TypeScript (React, TSX)
Lines	394
Functions / Classes	ParishWelcomeHeader (exported component/function (default)); faceOffset (internal function); clamp (internal function); offsetWithin (internal function)
Related Files (its imports)	../lib/useLanguage, ../lib/ui, ../lib/findScroller
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/ParishWelcomeHeader.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishWelcomeHeader.tsx

```
import { useEffect, useRef } from "react";
import { useLanguage } from "../lib/useLanguage";
import { t } from "../lib/ui";
import { ArrowLeft, MapPin, Search } from "lucide-react";
import { findScroller } from "../lib/findScroller";

interface ParishWelcomeHeaderProps {
  parishName: string;
  location?: string;
  /** The parish's patron image, bled in behind the text from the right. */
  imageUrl?: string;
  /**
   * How far down that photograph the patron's face sits, 0 to 1. Given, the
   * collapse keeps the face in the bar; omitted, the picture stays
   * top-aligned and whatever survives the crop survives it.
   */
  imageFaceY?: number;
  /** First name of the signed-in pilgrim, or undefined when signed out. */
  firstName?: string;
  /** Opens parish search, from the top-right corner. */
  onSearch: () => void;
  /**
   * Given only when the band is showing a parish that is NOT the
   * pilgrim's own. Its presence is what puts the way back on screen.
   */
  onBackToMyParish?: () => void;
  /** Rendered so the date matches the rest of the screen's clock. */
  now: Date;
}

/** The parish name's size once collapsed, over its size at rest: 21 of 27px. */
```

```
const PINNED_SCALE = 21 / 27;
/** Room above and below the name in the collapsed bar. */
const PINNED_PAD_TOP = 11;
const PINNED_PAD_BOTTOM = 9;

/**
 * How much of the scroll the photograph absorbs. At 0 it travels with the
 * band; at 1 it is nailed to the screen. Between them it drifts, which is
 * what reads as depth. Under reduced motion it becomes 1, so nothing moves
 * at a rate the finger did not ask for.
 */
const PHOTO_PARALLAX = 0.62;

/**
 * The welcome header from the design: a dark navy band with the parish's
 * patron image bleeding in from the right, the SanctiWalk mark, a greeting,
 * the date, the parish name and where it is.
 *
 * It replaces a date row plus a collapsing photograph. That earlier header
 * put the picture in a card with the name underneath, so the top of the
 * screen read as a list item rather than as arriving somewhere; the design
 * has the image behind everything, which is what makes it feel like the
 * parish's own page.
 *
 * The greeting degrades rather than inventing a name: signed out it reads
 * "Welcome" alone. The design shows "WELCOME, ADRICH" because the designer
 * was signed in.
 *
 * ## The collapse
 *
 * Scrolling must leave the parish name, its location and the patron picture
 * on screen — the answer to "which parish am I looking at" stays true no
 * matter how far down the page goes, and a header that takes its own title
 * away leaves everything below it unlabelled.
 *
 * It gets there by travelling, not by swapping. Every value tracks the
 * scroll position: the band slides up, the crest row, greeting and date fade
 * while sliding out of the way, the name shrinks to its smaller size, and
 * the photograph drifts up more slowly than the page so the collapse reads
 * as depth rather than as a picture being dragged off.
 *
 * ### Why it costs the page no height
 *
 * The obvious way — shrinking the band's height — was tried in an earlier
 * header and fights itself. A shorter band is a shorter page, and on a page
 * that is barely scrollable that pushes the maximum scroll position above
 * where the finger already is; the browser clamps scrollTop, the progress
 * falls, the band grows back, and the two settle against each other with the
 * band stuck half open.
 *
 * So nothing here changes layout. The band keeps its full height and is
 * `position: sticky` with a NEGATIVE top, which lets it scroll up by exactly
 * the amount that should disappear and then pin. Everything else is a
 * transform or an opacity, which the compositor can do without laying the
 * page out again. Picking the sticky offset as `height - collapsedHeight`
 * is what makes the band's own bottom edge arrive in the right place with no
 * height animation at all.
 *
 * JavaScript writes one custom property per frame, `--pw-t`, running 0 to 1.
```

```
 * The stylesheet derives every movement from it. If the script never runs,
 * `--pw-t` stays 0 and the band simply scrolls away like any other header.
 */
export default function ParishWelcomeHeader({
  parishName,
  location,
  imageUrl,
  imageFaceY,
  firstName,
  onSearch,
  onBackToMyParish,
  now,
}: ParishWelcomeHeaderProps) {
  const { language } = useLanguage();
  const bandRef = useRef<HTMLElement | null>(null);
  const titleRef = useRef<HTMLDivElement | null>(null);
  const imageRef = useRef<HTMLDivElement | null>(null);
  /** The photograph's own pixel size, once the browser has told us. */
  const natural = useRef<{ w: number; h: number } | null>(null);

  useEffect(() => {
    const band = bandRef.current;
    const title = titleRef.current;
    if (!band || !title) return;

    const view = findScroller(band);
    if (!view) return;

    let collapse = 0;
    let ticking = false;
    const reduce = window.matchMedia("(prefers-reduced-motion: reduce)");

    const measure = () => {
      // offsetTop and offsetHeight, not rects: the title already carries a
      // transform by the time this re-runs, and a rect would report the
      // moved, scaled box — so the maths that drives the transform would be
      // reading its own output.
      const titleTop = offsetWithin(title, band);
      const titleHeight = title.offsetHeight;

      // The rounded lip along the band's bottom edge is the top of the
      // content sheet, drawn by the band so it travels with it. It covers
      // the band's last pixels, so the collapsed bar has to be tall enough
      // to clear it or the street address ends up underneath it. Read from
      // the stylesheet rather than repeated here.
      const lip = parseFloat(getComputedStyle(band, "::after").height) || 0;

      const collapsedHeight =
        PINNED_PAD_TOP + titleHeight * PINNED_SCALE + PINNED_PAD_BOTTOM + lip;
      collapse = Math.max(0, band.offsetHeight - collapsedHeight);

      band.style.setProperty("--pw-collapse", `${collapse}px`);
      // Where the title has to travel to land at PINNED_PAD_TOP from the top
      // of the screen once the band has slid up by `collapse`.
      band.style.setProperty("--pw-title-dy", `${PINNED_PAD_TOP - titleTop + collapse}px`);
      band.style.setProperty("--pw-scale-delta", String(1 - PINNED_SCALE));

      // The photograph's parallax. Reduced motion takes it to 1, which
      // holds the picture still rather than drifting it.
```

```
  const parallax = reduce.matches ? 1 : PHOTO_PARALLAX;
  band.style.setProperty("--pw-media-dy", `${parallax * collapse}px`);

  band.style.setProperty(
    "--pw-image-dy",
    `${faceOffset(imageRef.current, natural.current, imageFaceY, {
      collapse,
      collapsedHeight,
      lip,
      parallax,
    })}px`,
  );
};

const apply = () => {
  ticking = false;
  if (collapse <= 0) {
    band.style.setProperty("--pw-t", "0");
    return;
  }
  // How far the band has been pushed up past the top of the scroller.
  // Measured against the scroller rather than the window, because the
  // scroller starts partway down the page inside the desktop phone frame
  // and a window-relative reading is wrong by that offset.
  const pushed = view.getBoundingClientRect().top - band.getBoundingClientRect().top;
  const t = Math.min(1, Math.max(0, pushed / collapse));
  band.style.setProperty("--pw-t", t.toFixed(4));
};

const onScroll = () => {
  if (ticking) return;
  ticking = true;
  requestAnimationFrame(apply);
};

const remeasure = () => {
  measure();
  apply();
};

// The photograph is a CSS background, so its proportions are not on the
// element. Asking for them separately reads the browser's cache — the
// picture is already on screen — and remeasures once they arrive.
let probe: HTMLImageElement | null = null;
if (imageUrl && imageFaceY !== undefined && !natural.current) {
  probe = new window.Image();
  probe.onload = () => {
    natural.current = { w: probe!.naturalWidth, h: probe!.naturalHeight };
    remeasure();
  };
  probe.src = imageUrl;
}

remeasure();
view.addEventListener("scroll", onScroll, { passive: true });
reduce.addEventListener("change", remeasure);
// The band's height moves with how many lines the parish name takes, and
// the scroller's with the keyboard and the address bar.
const resize = new ResizeObserver(remeasure);
```

```
  resize.observe(view);
  resize.observe(band);

  return () => {
    view.removeEventListener("scroll", onScroll);
    reduce.removeEventListener("change", remeasure);
    resize.disconnect();
    if (probe) probe.onload = null;
  };
}, [parishName, location, imageUrl, imageFaceY, firstName]);

const date = now
  .toLocaleDateString("en-US", { weekday: "long", month: "long", day: "numeric" })
  .toUpperCase();

return (
  <header className="parish-welcome" ref={bandRef}>
    {/* Image and scrim share one wrapper so a single transform gives them
        the same parallax. Two transforms would drift apart under rounding
        and let the gradient slide off the text it exists to protect. */}
    <div className="parish-welcome__media" aria-hidden="true">
      {imageUrl && (
        <div
          className="parish-welcome__image"
          ref={imageRef}
          style={{ backgroundImage: `url("${imageUrl}")` }}
        />
      )}
      {/* Two scrims, not one. A single left-to-right fade left the parish
          name legible but washed the top row out; this darkens the whole
          band a little and the text side a lot. */}
      <div className="parish-welcome__scrim" />
    </div>

    <div className="parish-welcome__content">
      {/* Leaves with the scroll, so it is grouped: one transform and one
          opacity move the crest, the greeting and the date together. */}
      <div className="parish-welcome__intro">
        <div className="parish-welcome__top">
          {/* The horizontal lockup: the emblem with "SanctiWalk" beside
              it on one line.

              Composed rather than lifted whole, because the brand file
              has no such variant - its Primary stacks "Sancti / Walk"
              and its Secondary sets the word underneath the emblem. Both
              pieces are the file's own artwork, taken from the Secondary
              and set side by side; nothing here is retyped in a
              substitute font. */}
          {/* Away from your own parish, the brand slot becomes the way
              back. Nothing else on this screen says you are looking at
              somewhere other than your parish - the band wears that
              parish's photograph and colours, which is exactly what
              makes it convincing - so the way home has to be where your
              eye already goes. */}
          {onBackToMyParish ? (
            <button
              type="button"
              onClick={onBackToMyParish}
```

```
          className="parish-welcome__back"
        >
          <ArrowLeft className="w-[15px] h-[15px] shrink-0" aria-hidden />
          {t("home.backToMyParish", language)}
        </button>
      ) : (
        <span className="parish-welcome__brand">
          <img
            src="/ui/sanctiwalk-lockup-white.png"
            alt="SanctiWalk"
            className="parish-welcome__lockup"
          />
        </span>
      )}

      {/* Search, not the profile. Your account already lives behind
          the Me tab; what this corner is for is finding the other
          parish. */}
      <button
        type="button"
        onClick={onSearch}
data-spotlight="pick-church"
        className="parish-welcome__avatar"
        aria-label="Search parishes"
      >
        <Search className="w-[18px] h-[18px]" />
      </button>
    </div>

    {/* "Welcome," is the same word every day; the name is the part
        worth reading. They were set at one size, so the greeting
        shouted the greeting. Now the salutation is small and quiet
        and the name carries the line. */}
    <p className="parish-welcome__greeting">
      {firstName ? (
        <>
          <span className="parish-welcome__salutation">Welcome,</span>
          <span className="parish-welcome__name">{firstName}</span>
        </>
      ) : (
        <span className="parish-welcome__salutation parish-welcome__salutation--alone">
          Welcome
        </span>
      )}
    </p>

    <p className="parish-welcome__date">{date}</p>
  </div>

  {/* Stays, so it is grouped separately: it travels down inside the
      band by as much as the band travels up, which is what holds it on
      screen while everything above it leaves. */}
  <div className="parish-welcome__title" ref={titleRef}>
    <h1 className="parish-welcome__parish">{parishName}</h1>

    {location && (
      <p className="parish-welcome__location">
        <MapPin className="w-[15px] h-[15px] shrink-0" />
        {location}
```

```
            </p>
          )}
        </div>
      </div>
    </header>
  );
}

/**
 * How far to shift the photograph, at full collapse, to put the patron's
 * face in the middle of the collapsed bar's clear area. Negative moves the
 * picture up, which brings a lower part of it into the bar.
 *
 * The band's visible remainder is `collapsedHeight`, and the sheet's rounded
 * lip covers its last pixels, so the area a face can actually be seen in is
 * what is left above that lip.
 *
 * `background-size: cover` scales the picture by whichever axis is short,
 * and `background-position: 50% 0%` aligns its top with the box's top — so
 * the face's row inside the box is simply its fraction of the scaled height.
 * Everything else here is bookkeeping: the band will have travelled up by
 * `collapse` and the picture back down by `parallax * collapse`, leaving
 * `slack` still to account for.
 *
 * Returns 0 when there is nothing to work with — no photograph, no measured
 * size, or no recorded face position — which leaves the plain top-aligned
 * crop the band has always had.
 */
function faceOffset(
  image: HTMLElement | null,
  natural: { w: number; h: number } | null,
  faceY: number | undefined,
  band: { collapse: number; collapsedHeight: number; lip: number; parallax: number },
): number {
  if (!image || !natural || faceY === undefined || band.collapse <= 0) return 0;
  if (!natural.w || !natural.h) return 0;

  const scale = Math.max(image.offsetWidth / natural.w, image.offsetHeight / natural.h);
  const scaledHeight = natural.h * scale;
  const faceRow = faceY * scaledHeight;
  const clearCentre = (band.collapsedHeight - band.lip) / 2;
  const slack = band.collapse * (1 - band.parallax);

  // Clamped so the photograph still covers the bar: any further down and its
  // top edge would leave a band of bare navy above it, any further up and
  // its bottom edge would leave one below.
  const lowest = band.collapsedHeight - scaledHeight + slack;
  return clamp(clearCentre - faceRow + slack, lowest, slack);
}

function clamp(value: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, value));
}

/**
 * How far `el` sits below `ancestor`, in layout pixels.
 *
 * Walks offsetTop rather than subtracting bounding rects because offsetTop
 * ignores transforms, and everything here is transformed.
```

```
 */
function offsetWithin(el: HTMLElement, ancestor: HTMLElement): number {
  let top = 0;
  let node: HTMLElement | null = el;
  while (node && node !== ancestor) {
    top += node.offsetTop;
    node = node.offsetParent as HTMLElement | null;
  }
  return top;
}
```

PhoneContainer.tsx — listed in full as APP-004.

1.2.42 APP-047 — PilgrimQuiz.tsx

Code ID	APP-047
File Name	PilgrimQuiz.tsx
File Path	SanctDemo/src/components/PilgrimQuiz.tsx
Language / Type	TypeScript (React, TSX)
Lines	220
Functions / Classes	PilgrimQuiz (exported component/function (default))
Related Files (its imports)	../data
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/PilgrimQuiz.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PilgrimQuiz.tsx

```
import React, { useState } from "react";
import { PILGRIM_QUIZ, QuizQuestion } from "../data";
import { Award, Check, X, ArrowRight, RefreshCw, Sparkles, HelpCircle, ShieldCheck } from "lucide-react";

interface PilgrimQuizProps {
  onEarnBadge: (badgeId: string) => void;
  onAddPoints: (points: number) => void;
  onAddApplication: (app: { id: string; type: string; applicant: string; details: string; date: string; status: string }) => void;
}

export default function PilgrimQuiz({ onEarnBadge, onAddPoints, onAddApplication }: PilgrimQuizProps) {
  const [currentQuestionIndex, setCurrentQuestionIndex] = useState(0);
  const [selectedOptionIndex, setSelectedOptionIndex] = useState<number | null>(null);
  const [isSubmitted, setIsSubmitted] = useState(false);
  const [score, setScore] = useState(0);
  const [isQuizCompleted, setIsQuizCompleted] = useState(false);

  const activeQuestion = PILGRIM_QUIZ[currentQuestionIndex];

  const handleOptionSelect = (idx: number) => {
    if (isSubmitted) return;
    setSelectedOptionIndex(idx);
  };

  const handleSubmitAnswer = () => {
    if (selectedOptionIndex === null) return;

    const isCorrect = selectedOptionIndex === activeQuestion.correctAnswerIndex;
    if (isCorrect) {
      setScore((s) => s + 1);
```

```
      onAddPoints(100); // 100 points per correct answer
    }
    setIsSubmitted(true);
  };

  const handleNextQuestion = () => {
    setSelectedOptionIndex(null);
    setIsSubmitted(false);

    if (currentQuestionIndex < PILGRIM_QUIZ.length - 1) {
      setCurrentQuestionIndex((prev) => prev + 1);
    } else {
      setIsQuizCompleted(true);
      // If perfect score, award the Faith Defender Badge
      if (score + (selectedOptionIndex === activeQuestion.correctAnswerIndex ? 1 : 0) === 5) {
        onEarnBadge("badge-4");
      }

      // Log quiz result to simulated database
      onAddApplication({
        id: "quiz-log-" + Date.now(),
        type: "Pilgrim Quiz Completed",
        applicant: "Public Student",
        details: `Completed Pilgrim Quiz - Final Score: ${score + (selectedOptionIndex === activeQuestion.correctAnswerIndex ? 1 : 0)}/5`,
        date: new Date().toLocaleDateString(),
        status: "Database Synced"
      });
    }
  };

  const handleResetQuiz = () => {
    setCurrentQuestionIndex(0);
    setSelectedOptionIndex(null);
    setIsSubmitted(false);
    setScore(0);
    setIsQuizCompleted(false);
  };

  return (
    <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto">
      {/* Page Header */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 pt-6 rounded-b-[2rem] shadow-sm relative overflow-hidden shrink-0 border-b border-[var(--color-
brand-border)]">
        <div className="absolute right-0 top-0 opacity-10 translate-x-4 -translate-y-4">
          <HelpCircle className="w-32 h-32 text-white" />
        </div>
        <div className="flex items-center gap-1.5 text-[var(--color-brand-on-accent)] font-bold text-[15px] tracking-wider uppercase font-serif italic">
          <Sparkles className="w-3.5 h-3.5" /> Interactive Catechesis
        </div>
        <div>
          <h2 className="text-2xl font-bold font-serif italic tracking-tight">
            Pilgrim History Quiz
          </h2>
          <p className="text-[15px] text-[var(--color-brand-secondary)] opacity-95 mt-1 max-w-xs leading-relaxed font-sans">
            Test your knowledge of the Diocese of Kalookan history and PWA technology to earn the <strong>Faith Defender Badge</strong>!
          </p>
        </div>
      </div>
```

```
      <div className="p-4 flex-1 flex flex-col justify-between">
        {!isQuizCompleted ? (
          <div className="space-y-4 flex-1 flex flex-col justify-between">
            {/* Question card */}
            <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5 space-y-4">
              <div className="flex justify-between items-center text-sm text-[var(--color-brand-secondary)] font-mono border-b border-[var(--color-brand-card)] pb-2
font-bold">
                <span>QUESTION {currentQuestionIndex + 1} OF {PILGRIM_QUIZ.length}</span>
                <span>SCORE: {score}</span>
              </div>

              <h3 className="text-sm font-bold text-[var(--color-brand-text)] font-serif italic leading-relaxed">
                {activeQuestion.question}
              </h3>

              {/* Options */}
              <div className="space-y-2 select-none font-sans">
                {activeQuestion.options.map((option, idx) => {
                  let optionStyle = "border-[var(--color-brand-border)] bg-[var(--color-brand-card)]/30 hover:bg-[var(--color-brand-card)]/35";

                  if (selectedOptionIndex === idx) {
                    optionStyle = "border-[var(--color-brand-primary)] bg-[var(--color-brand-card)] font-bold";
                  }

                  if (isSubmitted) {
                    if (idx === activeQuestion.correctAnswerIndex) {
                      optionStyle = "border-green-600 bg-green-50 text-green-950 font-bold";
                    } else if (selectedOptionIndex === idx) {
                      optionStyle = "border-red-600 bg-red-50 text-red-950 line-through";
                    } else {
                      optionStyle = "border-gray-200 bg-gray-50/50 opacity-60";
                    }
                  }

                  return (
                    <button
                      key={idx}
                      onClick={() => handleOptionSelect(idx)}
                      disabled={isSubmitted}
                      className={`w-full p-3.5 text-[15px] rounded-xl border text-left flex items-center justify-between transition-all ${optionStyle}`}
                    >
                      <span className="leading-normal">{option}</span>
                      {isSubmitted && idx === activeQuestion.correctAnswerIndex && (
                        <Check className="w-4 h-4 text-green-700 shrink-0" />
                      )}
                      {isSubmitted && selectedOptionIndex === idx && idx !== activeQuestion.correctAnswerIndex && (
                        <X className="w-4 h-4 text-red-700 shrink-0" />
                      )}
                    </button>
                  );
                })}
              </div>
            </div>

            {/* Explanation & Action button */}
            <div className="space-y-3">
              {isSubmitted && (
```

```
              <div className="p-4 bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] text-[15px] leading-relaxed space-y-1">
                <span className="text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-widest font-serif italic block">
                  Historical Explanation:
                </span>
                <p className="text-[var(--color-brand-text)] font-sans text-justify italic">
                  {activeQuestion.explanation}
                </p>
              </div>
            )}

            {!isSubmitted ? (
              <button
                onClick={handleSubmitAnswer}
                disabled={selectedOptionIndex === null}
                className="w-full py-3 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] disabled:opacity-50 text-white text-[15px] font-bold uppercase tracking-wider rounded-full border border-[var(--color-brand-primary-dark)] transition-all shadow-xs"
              >
                Confirm Answer Selection
              </button>
            ) : (
              <button
                onClick={handleNextQuestion}
                className="w-full py-3 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white text-[15px] font-bold uppercase tracking-wider rounded-full flex items-center justify-center gap-1 border border-[var(--color-brand-primary-dark)] transition-all shadow-xs"
              >
                {currentQuestionIndex === PILGRIM_QUIZ.length - 1 ? "Complete Quiz" : "Next Question"} <ArrowRight className="w-4 h-4" />
              </button>
            )}
          </div>
        </div>
      ) : (
        /* Finished Quiz Screen */
        <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-6 text-center space-y-5 flex-1 flex flex-col justify-center">
          <div className="h-14 w-12 bg-[var(--color-brand-card)] rounded-2xl flex items-center justify-center mx-auto text-[var(--color-brand-gold)]">
            <Award className="w-8 h-8 fill-current" />
          </div>

          <div className="space-y-1">
            <h3 className="text-lg font-bold text-[var(--color-brand-text)] font-serif italic">
              Catechism Quiz Finished!
            </h3>
            <p className="text-[15px] text-[var(--color-brand-secondary)] font-sans">
              You scored a total of:
            </p>
            <div className="text-3xl font-bold text-[var(--color-brand-secondary)] font-serif italic tracking-tight pt-1">
              {score} / 5 Correct
            </div>
          </div>

          <div className="p-4 bg-[var(--color-brand-card)]/70 rounded-2xl border border-[var(--color-brand-border)]/45 space-y-1 text-[15px]">
            {score === 5 ? (
              <>
                <p className="font-bold text-green-800 flex items-center justify-center gap-1.5">
                  <ShieldCheck className="w-4 h-4" /> Perfect Score Achievement!
                </p>
                <p className="text-[var(--color-brand-text)] font-sans leading-relaxed text-center">
                  You have unlocked the prestigious <strong>"Faith Defender"</strong> passport badge, and earned a bonus of +500 points! Excellent defense preparation!
```

```
                </p>
              </>
            ) : (
              <p className="text-[var(--color-brand-text)] font-sans leading-relaxed text-center">
                You earned +{score * 100} points. Get a perfect 5/5 to unlock the rare **Faith Defender** badge on your Pilgrim Stamp Passport!
              </p>
            )}
          </div>

          <button
            onClick={handleResetQuiz}
            className="w-full py-3 bg-[var(--color-brand-primary)] hover:bg-[var(--color-brand-primary-dark)] text-white text-[15px] font-bold uppercase tracking-
widest rounded-full flex items-center justify-center gap-1.5 border border-[var(--color-brand-primary-dark)] transition-all shadow-xs"
          >
            <RefreshCw className="w-3.5 h-3.5" /> Retake Pilgrim Quiz
          </button>
        </div>
      )}
    </div>
  </div>
);
}
```

1.2.43 APP-048 — PrayScreen.tsx

Code ID	APP-048
File Name	PrayScreen.tsx
File Path	SanctDemo/src/components/PrayScreen.tsx
Language / Type	TypeScript (React, TSX)
Lines	42
Functions / Classes	PrayScreen (exported component/function (default))
Related Files (its imports)	./DailyRosary, ../types
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/PrayScreen.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PrayScreen.tsx

```
import DailyRosary from "./DailyRosary";
import type { Route } from "../types";

interface PrayScreenProps {
  /**
   * Kept so the tab's call site does not have to change, and because the
   * rosary will want to know whose parish it is the moment it carries a
   * parish's own intentions. Unused today.
   */
  parish?: Route;
  /**
   * Also unused now. The rosary app brings its own settings control - the
   * gear in its top corner - so a second one above it was two ways into
   * the same panel.
   */
  onOpenSettings?: () => void;
}

/**
 * Pray opens straight into the rosary.
 *
 * It used to land on a page ABOUT the rosary: a parish band, the verse of
 * the day, the day's mysteries written out, and a button that finally
 * started the thing. Four screens' worth of preamble in front of the one
 * thing the tab is named after.
 *
 * The verse moved to Home's bulletin rail, where someone is already
 * browsing and reading is the point. The rest is gone. Tapping Pray is a
 * decision to pray, not a request for a table of contents.
 *
 * DailyRosary is the client's own rosary app, mounted in an iframe. It has
 * its own heading, its own settings gear and its own "Begin today's
 * Rosary" - which is exactly why a second header above it made this feel
```

```
 * like a page rather than a prayer.
 */
export default function PrayScreen(_props: PrayScreenProps) {
  return (
    <div className="flex-1 flex flex-col min-h-0 bg-[var(--color-brand-card)]">
      <DailyRosary />
    </div>
  );
}
```

1.2.44 APP-049 — PresenceBanner.tsx

Code ID	APP-049
File Name	PresenceBanner.tsx
File Path	SanctDemo/src/components/PresenceBanner.tsx
Language / Type	TypeScript (React, TSX)
Lines	26
Functions / Classes	PresenceBanner (exported component/function (default))
Related Files (its imports)	../context/PresenceContext
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/PresenceBanner.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PresenceBanner.tsx

```
import { MapPin } from "lucide-react";
import { usePresence } from "../context/PresenceContext";

// Sits just below the app's own h-14 (56px) header bar so it never covers the
// screen heading beneath it — that exact overlap bug was found in the source
// app. Positioned `absolute` against the nearest positioned ancestor (the
// relative "Main App Frame inside simulator" div in App.tsx), which is bounded
// by PhoneContainer's screen canvas — never `fixed`, or it would escape the
// phone frame and float over the whole browser window on desktop.
export default function PresenceBanner() {
  const { presence, parish } = usePresence();

  if (presence.mode !== "approaching" || !parish) return null;

  return (
    <div
      className="absolute top-14 inset-x-0 z-40 px-4 py-2.5 flex items-center gap-2 shadow-sm transition-all duration-300 ease-out bg-[var(--color-brand-primary)]
border-b border-[var(--color-brand-border)]"
      role="status"
    >
      <MapPin className="w-4 h-4 text-[var(--color-brand-card)] shrink-0" />
      <p className="text-[15px] leading-snug text-[var(--color-brand-card)] font-sans">
        You are approaching <strong className="font-bold text-white">{parish.name.replace(" Guide", "").replace(" Tour", "")}</strong>
      </p>
    </div>
  );
}
```

1.2.45 APP-050 — PresenceSheet.tsx

Code ID	APP-050
File Name	PresenceSheet.tsx
File Path	SanctDemo/src/components/PresenceSheet.tsx
Language / Type	TypeScript (React, TSX)
Lines	142
Functions / Classes	nextMassLabel (internal function); PresenceSheet (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/schedule, ../data
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/PresenceSheet.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PresenceSheet.tsx

```
import { useEffect, useState } from "react";
import { ChevronUp, ScanLine, Map as MapIcon, X } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import { nextMass } from "../lib/schedule";
import { MASS_SCHEDULES } from "../data";

interface PresenceSheetProps {
  onOpenTour: (parishId: string) => void;
  onOpenAR: (parishId: string) => void;
}

function nextMassLabel(parishId: string): string {
  const parishSchedule = MASS_SCHEDULES[parishId];
  const upcoming = nextMass(parishSchedule?.schedule ?? [], new Date());

  // A parish with no schedule at all (real or placeholder) reads as a calm,
  // complete sentence rather than a blank or broken line.
  if (!upcoming) return "Mass schedule coming soon";

  const today = new Date();
  const isToday = upcoming.date.toDateString() === today.toDateString();
  const label = `Next Mass: ${isToday ? "Today" : upcoming.day}, ${upcoming.time}`;

  // The times behind this label may be an unverified placeholder (see
  // data.ts's scheduleVerified flag) — this quick-glance sheet must not
  // state a Mass time as fact when it hasn't been confirmed by the parish.
  return parishSchedule?.scheduleVerified === false ? `${label} (sample, unconfirmed)` : label;
}

// Rendered only in `present`. Anchored `absolute` (never `fixed`) against the
// same relative "phone screen" container the bottom nav bar uses, offset by
// exactly the nav bar's own height (h-16 / 64px) so the sheet sits above it
// rather than covering it. No backdrop/scrim behind it — the design
```

```
// deliberately avoids that commercial-app idiom.
export default function PresenceSheet({ onOpenTour, onOpenAR }: PresenceSheetProps) {
  const { presence, parish } = usePresence();
  const [collapsed, setCollapsed] = useState(false);
  /**
   * Dismissed outright, not just collapsed.
   *
   * Collapsing leaves a bar across the bottom of every screen. Someone
   * who is standing in their own parish all morning does not need to be
   * told so all morning, so the X takes it away entirely until they
   * arrive somewhere else.
   */
  const [dismissed, setDismissed] = useState(false);

  // Arriving at a different parish (or leaving/re-entering `present`) must
  // re-open the sheet — a pilgrim who hid it at one church shouldn't find it
  // silently hidden at the next one.
  useEffect(() => {
    setCollapsed(false);
    setDismissed(false);
  }, [presence.parishId, presence.mode]);

  if (presence.mode !== "present" || !parish || dismissed) return null;

  const displayName = parish.name.replace(" Guide", "").replace(" Tour", "");

  if (collapsed) {
    return (
      <div className="absolute bottom-16 inset-x-0 z-40 px-3 pb-2">
        <div className="w-full flex items-center gap-1 px-2 py-1 rounded-2xl shadow-md bg-[var(--color-brand-primary)] border border-[var(--color-brand-border)]">
          <button
            onClick={() => setCollapsed(false)}
            className="flex-1 min-w-0 flex items-center justify-between gap-2 px-2 py-2 text-left"
          >
            <span className="text-[16px] font-bold text-white font-sans truncate">
              You are near {displayName}
            </span>
            <ChevronUp className="w-4 h-4 text-[var(--color-brand-card)] shrink-0" />
          </button>

          {/* Its own button, outside the expand target: a dismiss nested
              inside "tap to expand" is a tap that does the opposite of
              what the row says. */}
          <button
            onClick={() => setDismissed(true)}
            aria-label="Hide this until I reach another parish"
            className="shrink-0 w-9 h-9 rounded-full flex items-center justify-center text-white/80 hover:text-white"
          >
            <X className="w-4 h-4" />
          </button>
        </div>
      </div>
    );
  }

  return (
    <div className="absolute bottom-16 inset-x-0 z-40 px-3 pb-2">
      <div className="rounded-3xl shadow-lg transition-all duration-300 ease-out bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] p-4 space-y-
3">
```

```
        <div className="flex items-start justify-between gap-2">
          <div>
            <span className="text-sm font-bold uppercase tracking-widest text-[var(--color-brand-secondary)] font-sans">
              You are near
            </span>
            <h3 className="text-lg font-bold text-[var(--color-brand-text)] font-serif italic leading-tight mt-0.5">
              {displayName}
            </h3>
          </div>
          <div className="flex items-center gap-1 shrink-0">
            {/* Two different things, so two buttons. "Hide" leaves the
                thin bar you can tap to bring this back; the X means you
                do not want to be told again at this parish. */}
            <button
              onClick={() => setCollapsed(true)}
              className="text-[15px] font-bold text-[var(--color-brand-secondary)] px-2 py-1 rounded-lg hover:bg-[var(--color-brand-card)] transition-colors"
            >
              Hide
            </button>
            <button
              onClick={() => setDismissed(true)}
              aria-label="Dismiss until I reach another parish"
              className="w-8 h-8 rounded-full flex items-center justify-center text-[var(--color-brand-secondary)] hover:bg-[var(--color-brand-card)]"
            >
              <X className="w-4 h-4" />
            </button>
          </div>
        </div>

        <p className="text-[15px] text-[var(--color-brand-text)] font-sans">
          {nextMassLabel(parish.id)}
        </p>

        <div className="grid grid-cols-2 gap-2">
          <button
            onClick={() => onOpenTour(parish.id)}
            className="flex items-center justify-center gap-1.5 py-2.5 px-3 rounded-xl text-[16px] font-bold text-white bg-[var(--color-brand-primary)] hover:bg-
[var(--color-brand-primary-dark)] transition-colors font-sans"
          >
            <MapIcon className="w-4 h-4" /> Open Tour
          </button>
          <button
            onClick={() => onOpenAR(parish.id)}
            className="flex items-center justify-center gap-1.5 py-2.5 px-3 rounded-xl text-[16px] font-bold text-[var(--color-brand-primary)] bg-[var(--color-brand-
card)] border border-[var(--color-brand-border)] hover:bg-[var(--color-brand-card)] transition-colors font-sans"
          >
            <ScanLine className="w-4 h-4" /> AR Tour
          </button>
        </div>
      </div>
    </div>
  );
}
```

1.2.46 APP-051 — PreviewVideo.tsx

Code ID	APP-051
File Name	PreviewVideo.tsx
File Path	SanctDemo/src/components/PreviewVideo.tsx
Language / Type	TypeScript (React, TSX)
Lines	81
Functions / Classes	PreviewVideo (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/components/PreviewVideo.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PreviewVideo.tsx

```
import { useEffect, useRef } from "react";
import { X } from "lucide-react";

/**
 * A short recording of a feature, played over the screen.
 *
 * ## Why this exists
 *
 * The scanner and the AR tour can only be shown by doing them, and
 * doing them means standing in the church pointing a phone at the
 * altar. During a Mass that is not available, and a survey respondent
 * cannot be asked to imagine the feature from a description. So each
 * one carries a recording of itself.
 *
 * ## Why the file is bundled rather than streamed
 *
 * It is shown to respondents in a church hall on whatever signal the
 * building has. A video that buffers is worse than no video when
 * somebody is standing over your shoulder waiting, so both clips ship
 * inside the app and play with the network off.
 */
export default function PreviewVideo({
  src,
  title,
  onClose,
}: {
  src: string;
  title: string;
  onClose: () => void;
}) {
  const video = useRef<HTMLVideoElement>(null);

  // Escape closes it, as it would any dialog. Without this the only
```

```
  // way out on a desktop browser is the mouse.
  useEffect(() => {
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") onClose();
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, [onClose]);

  // Paused on the way out. A video left playing behind a closed dialog
  // keeps its audio going, which during a demonstration is the worst
  // possible moment for it.
  useEffect(() => () => video.current?.pause(), []);

  return (
    <div
      className="preview-backdrop"
      role="dialog"
      aria-modal="true"
      aria-label={title}
      onClick={onClose}
    >
      {/* Stops a tap inside the player from closing it. */}
      <div className="preview-box" onClick={e => e.stopPropagation()}>
        <div className="preview-bar">
          <span className="preview-title">{title}</span>
          <button type="button" onClick={onClose} aria-label="Close preview" className="preview-close">
            <X className="w-4 h-4" />
          </button>
        </div>

        <video
          ref={video}
          src={src}
          controls
          autoPlay
          playsInline
          controlsList="nodownload"
          className="preview-video"
        />

        <p className="preview-note">
          A recording of the feature. Tap play, pause or full screen as you would any video.
        </p>
      </div>
    </div>
  );
}
```

1.2.47 APP-052 — PwaBanner.tsx

Code ID	APP-052
File Name	PwaBanner.tsx
File Path	SanctDemo/src/components/PwaBanner.tsx
Language / Type	TypeScript (React, TSX)
Lines	182
Functions / Classes	PwaBanner (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/PwaBanner.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PwaBanner.tsx

```
import React, { useState } from "react";
import { Download, FileCode, CheckCircle, HelpCircle, Layers, Clipboard, Globe } from "lucide-react";

export default function PwaBanner() {
  const [copiedText, setCopiedText] = useState<string | null>(null);

  const manifestJson = `{
  "short_name": "SanctiWalk",
  "name": "SanctiWalk Pilgrim Guide",
  "icons": [
    {
      "src": "/icons/icon-192.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "/icons/icon-512.png",
      "type": "image/png",
      "sizes": "512x512"
    }
  ],
  "start_url": "/?source=pwa",
  "background_color": "var(--color-brand-primary)",
  "theme_color": "var(--color-brand-primary)",
  "display": "standalone",
  "orientation": "portrait"
}`;

  const serviceWorkerCode = `// public/sw.js
const CACHE_NAME = 'sanctiwalk-cache-v1';
const ASSETS = [
  '/',
  '/index.html',
```

```
  '/src/main.tsx',
  '/src/index.css',
  '/src/data.ts',
  '/favicon.ico'
];

self.addEventListener('install', (e) => {
  e.waitUntil(
    caches.open(CACHE_NAME).then((cache) => {
      return cache.addAll(ASSETS);
    })
  );
});

self.addEventListener('fetch', (e) => {
  e.respondWith(
    caches.match(e.request).then((cachedResponse) => {
      return cachedResponse || fetch(e.request);
    })
  );
});`;

  const copyToClipboard = (text: string, type: string) => {
    navigator.clipboard.writeText(text);
    setCopiedText(type);
    setTimeout(() => setCopiedText(null), 2500);
  };

  return (
    <div className="p-4 bg-[var(--color-brand-card)] space-y-6">
      {/* Overview Card */}
      <div className="bg-[var(--color-brand-primary)] text-white p-5 rounded-3xl shadow-sm relative overflow-hidden border border-[var(--color-brand-border)]">
        <div className="absolute right-0 bottom-0 opacity-10 translate-y-4 translate-x-4">
          <Layers className="w-48 h-48" />
        </div>
        <div className="relative z-10 space-y-2">
          <div className="text-sm uppercase font-bold tracking-widest text-[var(--color-brand-gold)] font-serif italic">
            STI College - Capstone 2 Toolkit
          </div>
          <h2 className="text-xl font-bold font-serif italic tracking-tight">
            How to Pitch PWA for Capstone 2
          </h2>
          <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed max-w-sm font-sans">
            Presenting a PWA solves the cross-platform mobile limitation elegantly. You maintain a single React codebase, deploy instantly, and fulfill all
requirements of a high-grade software thesis!
          </p>
        </div>
      </div>

      {/* Slideout FAQ Section */}
      <div className="space-y-3">
        <h3 className="text-sm font-bold text-[var(--color-brand-text)] flex items-center gap-1.5 font-serif italic">
          <HelpCircle className="w-4 h-4 text-[var(--color-brand-secondary)]" /> STI Panel Q&A Defense Prep
        </h3>


        <div className="space-y-2.5">
          <div className="p-3 bg-[var(--color-brand-card)]/30 rounded-2xl border border-[var(--color-brand-border)] space-y-1">
            <h4 className="text-[15px] font-bold text-[var(--color-brand-text)] font-serif italic">
```

```
              Q: Why build a PWA instead of native iOS & Android apps?
            </h4>
            <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed font-sans">
              <strong>Answer:</strong> Native iOS requires Xcode and macOS (not accessible with Windows laptops). Developing separate Kotlin and Swift apps doubles
development overhead. A PWA provides a single, cost-effective codebase that runs cross-platform natively, utilizing Service Workers to work offline.
            </p>
          </div>

          <div className="p-3 bg-[var(--color-brand-card)]/30 rounded-2xl border border-[var(--color-brand-border)] space-y-1">
            <h4 className="text-[15px] font-bold text-[var(--color-brand-text)] font-serif italic">
              Q: Can a PWA use hardware like Geolocation & Cameras?
            </h4>
            <p className="text-[15px] text-[var(--color-brand-secondary)] leading-relaxed font-sans">
              <strong>Answer:</strong> Yes, modern browsers grant PWAs full native-like hardware access via standardized Web APIs (such as `navigator.geolocation`
and `MediaDevices`). No App Store compilation required!
            </p>
          </div>
        </div>
      </div>

      {/* Code assets */}
      <div className="space-y-4 border-t border-[var(--color-brand-border)] pt-4">
        <h3 className="text-sm font-bold text-[var(--color-brand-text)] flex items-center gap-1.5 font-serif italic">
          <FileCode className="w-4 h-4 text-[var(--color-brand-secondary)]" /> PWA Blueprint Code Files
        </h3>
        <p className="text-[15px] text-[var(--color-brand-secondary)] font-sans">
          Copy these directly into your project directory for full desktop and mobile installation capability.
        </p>

        {/* manifest.json */}
        <div className="space-y-2">
          <div className="flex items-center justify-between text-[15px]">
            <span className="font-mono text-sm font-bold text-[var(--color-brand-secondary)] bg-[var(--color-brand-card)] px-2.5 py-1 rounded-md border border-[var(-
-color-brand-border)]">
              public/manifest.json
            </span>
            <button
              onClick={() => copyToClipboard(manifestJson, "manifest")}
              className="flex items-center gap-1 text-[var(--color-brand-secondary)] hover:underline font-bold text-[15px] uppercase tracking-wider"
            >
              {copiedText === "manifest" ? (
                <>
                  <CheckCircle className="w-3 h-3 text-green-700" /> Copied!
                </>
              ) : (
                <>
                  <Clipboard className="w-3 h-3" /> Copy Schema
                </>
              )}
            </button>
          </div>
          <pre className="p-3 bg-[var(--color-brand-text)] text-[var(--color-brand-gold)] font-mono text-sm rounded-2xl overflow-x-auto max-h-[160px] border border-
[var(--color-brand-border)]/40">
            {manifestJson}
          </pre>
        </div>

        {/* sw.js */}
        <div className="space-y-2">
```

```
          <div className="flex items-center justify-between text-[15px]">
            <span className="font-mono text-sm font-bold text-[var(--color-brand-secondary)] bg-[var(--color-brand-card)] px-2.5 py-1 rounded-md border border-[var(-
--color-brand-border)]">
              public/sw.js (Service Worker)
            </span>
            <button
              onClick={() => copyToClipboard(serviceWorkerCode, "sw")}
              className="flex items-center gap-1 text-[var(--color-brand-secondary)] hover:underline font-bold text-[15px] uppercase tracking-wider"
            >
              {copiedText === "sw" ? (
                <>
                  <CheckCircle className="w-3 h-3 text-green-700" /> Copied!
                </>
              ) : (
                <>
                  <Clipboard className="w-3 h-3" /> Copy Code
                </>
              )}
            </button>
          </div>
          <pre className="p-3 bg-[var(--color-brand-text)] text-[var(--color-brand-secondary)] font-mono text-sm rounded-2xl overflow-x-auto max-h-[160px] border
border-[var(--color-brand-border)]/40">
            {serviceWorkerCode}
          </pre>
        </div>
      </div>

      <div className="p-4 bg-[var(--color-brand-border)] border border-[var(--color-brand-border)] rounded-2xl flex items-start gap-2.5">
        <Globe className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0 mt-0.5" />
        <div className="text-[15px] text-[var(--color-brand-text)] space-y-1">
          <p className="font-bold font-serif italic">Instant Cloud Deployment Ready!</p>
          <p className="leading-relaxed font-sans text-[15px] text-[var(--color-brand-secondary)]">
            Because this is deployed as a secure full-stack web application, you can test it on your actual iPhone or Android phone immediately by visiting the
**Shared App URL** at the top right of your screen! No Xcode or Windows emulators needed.
          </p>
        </div>
      </div>
    </div>
  );
}
```

1.2.48 APP-053 — RailDots.tsx

Code ID	APP-053
File Name	RailDots.tsx
File Path	SanctDemo/src/components/RailDots.tsx
Language / Type	TypeScript (React, TSX)
Lines	122
Functions / Classes	RailDots (exported component/function (default)); offsetOf (internal function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/RailDots.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — RailDots.tsx

```
import { useEffect, useRef, useState } from "react";
import type { RefObject } from "react";

interface RailDotsProps {
  /** The horizontally scrolling rail these dots belong to. */
  railRef: RefObject<HTMLElement | null>;
  /** Re-count the cards when this changes. */
  refreshKey: string;
}

/**
 * The dots under a swiping card rail: how many cards there are, which one
 * is in view, and a tap to go to any of them.
 *
 * A rail cut off at the screen edge gives no sign that it scrolls at all,
 * so the card hanging half off the right reads as a layout fault rather
 * than as an invitation to swipe. The dots say how much there is.
 *
 * The cards are read off the rail's own children rather than passed in, so
 * the count stays right as announcements come and go without the caller
 * having to tell this component anything about them.
 */
export default function RailDots({ railRef, refreshKey }: RailDotsProps) {
  const [count, setCount] = useState(0);
  const [active, setActive] = useState(0);
  const cardsRef = useRef<HTMLElement[]>([]);

  useEffect(() => {
    const rail = railRef.current;
    if (!rail) return;

    const cards = [...rail.children].filter((el): el is HTMLElement => el instanceof HTMLElement);
    cardsRef.current = cards;
```

```
  setCount(cards.length);
  if (cards.length < 2) return;

  let ticking = false;

  const apply = () => {
    ticking = false;
    let nearest = 0;
    let best = Infinity;
    cardsRef.current.forEach((card, i) => {
      const distance = Math.abs(offsetOf(rail, card));
      if (distance < best) {
        best = distance;
        nearest = i;
      }
    });
    // Only ever set a different value: this runs on every scroll frame and
    // React would otherwise re-render the dots sixty times a second.
    setActive((current) => (current === nearest ? current : nearest));
  };

  const onScroll = () => {
    if (ticking) return;
    ticking = true;
    requestAnimationFrame(apply);
  };

  apply();
  rail.addEventListener("scroll", onScroll, { passive: true });
  // Cards are sized in percentages of the rail, so every card moves when
  // the rail does — on rotation, or when a photograph finishes loading.
  const resize = new ResizeObserver(apply);
  resize.observe(rail);

  return () => {
    rail.removeEventListener("scroll", onScroll);
    resize.disconnect();
  };
}, [railRef, refreshKey]);

if (count < 2) return null;

const goTo = (index: number) => {
  const rail = railRef.current;
  const card = cardsRef.current[index];
  if (!rail || !card) return;
  // Only the rail's own scrollLeft is touched. scrollIntoView would have
  // been shorter and can scroll the vertical ancestor too, jumping the
  // page out from under the finger that only asked for a card.
  rail.scrollTo({ left: rail.scrollLeft + offsetOf(rail, card), behavior: "smooth" });
};

return (
  // Rendered from the count rather than from the ref: a ref holds no
  // value on the first render and reading one during render is a lie about
  // what React has committed. `tablist` was wrong here too — these select
  // nothing and reveal no panel, they scroll a strip of cards.
  <div className="rail-dots" aria-label="Bulletin cards">
    {Array.from({ length: count }, (_, i) => (
```

```
        <button
          key={i}
          type="button"
          className="rail-dots__hit"
          aria-current={i === active ? "true" : undefined}
          aria-label={`Card ${i + 1} of ${count}`}
          onClick={() => goTo(i)}
        >
          <span className="rail-dots__dot" />
        </button>
      ))}
    </div>
  );
}

/**
 * How far the rail would have to scroll to bring `card` to its resting
 * place. Zero means the card is already there.
 *
 * The rail snaps cards to its CENTRE, so "there" is the middle of the
 * scrollport. Measuring centre against centre needs nothing from the
 * computed style: the previous version read scroll-padding-left, which was
 * the resting place only while cards aligned to the start, and would now
 * mark a card as arrived while it was still half a card short.
 */
function offsetOf(rail: HTMLElement, card: HTMLElement): number {
  const railBox = rail.getBoundingClientRect();
  const cardBox = card.getBoundingClientRect();
  return cardBox.left + cardBox.width / 2 - (railBox.left + railBox.width / 2);
}
```

1.2.49 APP-054 — ReminderSettingsCard.tsx

Code ID	APP-054
File Name	ReminderSettingsCard.tsx
File Path	SanctDemo/src/components/ReminderSettingsCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	241
Functions / Classes	ReminderSettingsCard (exported component/function (default)); Row (internal function)
Related Files (its imports)	../lib/useLanguage, ../lib/ui, ../lib/deviceNotifications, ../lib/reminderSchedule
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ReminderSettingsCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ReminderSettingsCard.tsx

```
import { useState } from "react";
import { useLanguage } from "../lib/useLanguage";
import { t } from "../lib/ui";
import { Bell, Church, PartyPopper, CalendarDays, Megaphone, ClipboardList, MapPin } from "lucide-react";
import { notifyNow } from "../lib/deviceNotifications";
import { LEAD_CHOICES, type ReminderSettings } from "../lib/reminderSchedule";
import type { PermissionState } from "../lib/deviceNotifications";

/**
 * Which reminders this phone should show.
 *
 * ## Why the permission ask is a button and not automatic
 *
 * Android 13 and later require the user to grant notifications, and a
 * prompt thrown at someone the second they open an app is the one they
 * refuse — after which it cannot be asked again from inside the app at
 * all. Asked from a screen that has just explained what the reminders
 * are for, the answer means something.
 *
 * ## Why the count is shown
 *
 * "Reminders are on" is a claim. "12 reminders set" is the app showing
 * its work, and it is the difference between a pilgrim trusting this to
 * wake them for the 6 AM Mass and checking their own alarm anyway.
 */
export default function ReminderSettingsCard({
  settings, update, permission, enable, scheduled, supported,
  otherParishes = [], onToggleParish,
}: {
  settings: ReminderSettings;
  update: (patch: Partial<ReminderSettings>) => void;
  permission: PermissionState;
  enable: () => Promise<PermissionState>;
```

```
  scheduled: number;
  supported: boolean;
  /**
   * Every parish that can be reminded about, besides the pilgrim's own.
   *
   * Their own is always on and is deliberately not in this list — a
   * switch that cannot be turned off is not a switch.
   *
   * This started as a card on the parish's own dashboard, which was
   * wrong twice over: it put one reminder control on Home while every
   * other one lived here, and it meant the only way to follow a parish
   * was to go and look at it first.
   */
  otherParishes?: Array<{ id: string; name: string; following: boolean }>;
  onToggleParish?: (parishId: string, follow: boolean) => void;
}) {
  const { language } = useLanguage();
  const [tested, setTested] = useState(false);

  return (
    <div className="bg-[var(--color-brand-card-sunk)] rounded-[22px] border border-[var(--color-brand-border)] p-5">
      <h4 className="mb-1 flex items-center gap-2 text-sm font-bold text-[var(--color-brand-secondary)] uppercase tracking-wider font-sans">
        <Bell className="w-4 h-4" /> {t("rem.title", language)}
      </h4>

      {!supported ? (
        // The browser has no alarms to set. Said plainly rather than
        // showing switches that would quietly do nothing.
        <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          Reminders work in the installed SanctiWalk app on your phone. In a web
          browser there is nothing to set an alarm with.
        </p>
      ) : permission !== "granted" ? (
        <>
          <p className="text-[15px] leading-relaxed text-[var(--color-brand-text)]">
            Let SanctiWalk remind you before Mass, and tell you about feast days,
            parish news and your applications — even when the app is closed.
          </p>
          {permission === "denied" ? (
            <p className="mt-3 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
              Notifications are switched off for SanctiWalk. Android only asks once,
              so this has to be turned back on in{" "}
              <strong>Settings → Apps → SanctiWalk → Notifications</strong>.
            </p>
          ) : (
            <button
              type="button"
              onClick={() => void enable()}
              className="mt-4 w-full rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] py-3 text-[15px] font-bold"
            >
              {t("rem.turnOn", language)}
            </button>
          )}
        </>
      ) : (
        <>
          <p className="mb-4 text-[15px] text-[var(--color-brand-secondary)]">
            {scheduled > 0
              ? `${scheduled} reminder${scheduled === 1 ? "" : "s"} set on this phone.`
```

```
    : "Nothing to remind you about just yet."}
</p>

<Row
  icon={<Church className="w-4 h-4" />}
  label={t("rem.beforeMass", language)}
  hint={t("rem.beforeMassHint", language)}
  on={settings.massReminders}
  onChange={v => update({ massReminders: v })}
/>

{settings.massReminders && (
  <div className="mb-4 pl-7">
    <p className="mb-2 text-[14px] font-bold text-[var(--color-brand-secondary)]">
      {t("rem.howLongBefore", language)}
    </p>
    <div className="flex flex-wrap gap-2">
      {LEAD_CHOICES.map(minutes => (
        <button
          key={minutes}
          type="button"
          aria-pressed={settings.massLeadMinutes === minutes}
          onClick={() => update({ massLeadMinutes: minutes })}
          className={`apply-chip${settings.massLeadMinutes === minutes ? " is-on" : ""}`}
        >
          {minutes < 60 ? `${minutes} min` : `${minutes / 60} hour${minutes === 60 ? "" : "s"}`}
        </button>
      ))}
    </div>
  </div>
)}

<Row
  icon={<PartyPopper className="w-4 h-4" />}
  label={t("rem.feastDays", language)}
  hint={t("rem.feastHint", language)}
  on={settings.feastReminders}
  onChange={v => update({ feastReminders: v })}
/>
<Row
  icon={<CalendarDays className="w-4 h-4" />}
  label={t("rem.events", language)}
  on={settings.eventReminders}
  onChange={v => update({ eventReminders: v })}
/>
<Row
  icon={<Megaphone className="w-4 h-4" />}
  label={t("rem.announcements", language)}
  on={settings.announcementReminders}
  onChange={v => update({ announcementReminders: v })}
/>
<Row
  icon={<ClipboardList className="w-4 h-4" />}
  label={t("rem.applications", language)}
  hint={t("rem.applicationsHint", language)}
  on={settings.applicationUpdates}
  onChange={v => update({ applicationUpdates: v })}
  last
/>
```

```
        {/* Parishes besides their own. Always shown when there are
            any, following or not: this is the only place the choice
            is offered now, so hiding it until something is already
            followed would make the feature unreachable. */}
        {otherParishes.length > 0 && onToggleParish && (
          <div className="mt-4 pt-4 border-t border-[var(--color-brand-border)]">
            <p className="mb-1 flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-secondary)]">
              <MapPin className="w-4 h-4" /> {t("rem.otherParishes", language)}
            </p>
            <p className="mb-3 text-[13px] leading-relaxed text-[var(--color-brand-secondary)]">
              {t("rem.otherParishesHint", language)}
            </p>
            {otherParishes.map(parish => (
              <Row
                key={parish.id}
                icon={<Church className="w-4 h-4" />}
                label={parish.name}
                on={parish.following}
                onChange={next => onToggleParish(parish.id, next)}
                last
              />
            ))}
          </div>
        )}

        {/* Proof, on demand.
            "Reminders are on" is a claim somebody has to take on
            trust until the next Mass. One tap that puts a real
            notification in the tray settles it in a second, and it is
            also the quickest way to find out that Android's own
            per-app switch is off. */}
        <button
          type="button"
          onClick={() => {
            setTested(true);
            void notifyNow(
              "Reminders are working",
              "This is what a parish reminder looks like.",
            );
            window.setTimeout(() => setTested(false), 4000);
          }}
          className="mt-4 w-full rounded-full border-[1.5px] border-[var(--color-brand-border)] py-3 text-[15px] font-bold text-[var(--color-brand-text)]"
        >
          {tested ? t("rem.testSent", language) : t("rem.test", language)}
        </button>

        {/* The honest limit, said once rather than discovered. */}
        <p className="mt-4 text-[13px] leading-relaxed text-[var(--color-brand-secondary)]">
          Mass, feast and event reminders are set on this phone, so they arrive
          even with no signal. News about your applications reaches you when you
          next open the app.
        </p>
      </>
    )}
  </div>
);
}
```

```
function Row({
  icon, label, hint, on, onChange, last,
}: {
  icon: React.ReactNode;
  label: string;
  hint?: string;
  on: boolean;
  onChange: (on: boolean) => void;
  last?: boolean;
}) {
  return (
    <label className={`flex items-center gap-3 ${last ? "" : "mb-4"} cursor-pointer`}>
      <span className="text-[var(--color-brand-primary)] shrink-0">{icon}</span>
      <span className="min-w-0 flex-1">
        <span className="block text-[16px] font-bold leading-snug text-[var(--color-brand-text)]">
          {label}
        </span>
        {hint && (
          <span className="block text-[14px] text-[var(--color-brand-secondary)]">{hint}</span>
        )}
      </span>
      <input
        type="checkbox"
        className="availability-switch"
        aria-label={label}
        checked={on}
        onChange={e => onChange(e.target.checked)}
      />
    </label>
  );
}
```

RootGate.tsx — listed in full as APP-003.

1.2.50 APP-055 — RosarySettingsModal.tsx

Code ID	APP-055
File Name	RosarySettingsModal.tsx
File Path	SanctDemo/src/components/RosarySettingsModal.tsx
Language / Type	TypeScript (React, TSX)
Lines	202
Functions / Classes	RosarySettingsModal (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/RosarySettingsModal.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — RosarySettingsModal.tsx

```
import React, { useEffect, useState } from "react";
import { X, Languages, Sparkles, Music } from "lucide-react";
import {
  RosaryLanguage,
  RosaryMysterySet,
  RosaryMusic,
  RosarySettings,
  ROSARY_SETTINGS_KEY,
  loadRosarySettings,
  saveRosarySettings,
  parseRosarySettingsEventValue,
} from "../lib/rosarySettings";

interface RosarySettingsModalProps {
  isOpen: boolean;
  onClose: () => void;
}

const MYSTERY_OPTIONS: { key: RosaryMysterySet; label: string }[] = [
  { key: "today", label: "Today's Mystery" },
  { key: "joyful", label: "Joyful" },
  { key: "sorrowful", label: "Sorrowful" },
  { key: "glorious", label: "Glorious" },
  { key: "luminous", label: "Luminous" },
];

// Kept in sync with public/rosary/index.html via the shared
// `sanctiwalk.rosary.settings` localStorage key (see src/lib/rosarySettings.ts).
// Both this modal and the rosary iframe read on mount, write on change, and
// listen for the `storage` event so a change made in one place is reflected
```

```
// live in the other, without any postMessage bridge.
export default function RosarySettingsModal({ isOpen, onClose }: RosarySettingsModalProps) {
  const [settings, setSettings] = useState<RosarySettings>(() => loadRosarySettings());

  // Pick up whatever is in storage each time the modal opens (e.g. changed
  // from the rosary tab while this modal was closed).
  useEffect(() => {
    if (isOpen) setSettings(loadRosarySettings());
  }, [isOpen]);

  // Live sync: reflect changes made elsewhere (the rosary iframe, or this
  // same app open in another tab) while the modal is open. The `storage`
  // event never fires in the context that made the change, so the setters
  // below also apply the change directly at the point of interaction.
  useEffect(() => {
    function onStorage(e: StorageEvent) {
      if (e.key !== ROSARY_SETTINGS_KEY) return;
      setSettings(parseRosarySettingsEventValue(e.newValue));
    }
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, []);

  // Functional updates: two settings changes made in quick succession (e.g.
  // the same click handler, or React 18 batching two synthetic events into
  // one render) must not have the second overwrite the first by spreading
  // a stale `settings` closure.
  function updateSettings(updater: (prev: RosarySettings) => RosarySettings) {
    setSettings((prev) => {
      const next = updater(prev);
      saveRosarySettings(next);
      return next;
    });
  }

  function setLanguage(language: RosaryLanguage) {
    updateSettings((prev) => ({ ...prev, language }));
  }

  function setMysterySet(mysterySet: RosaryMysterySet) {
    updateSettings((prev) => ({ ...prev, mysterySet }));
  }

  function setMusic(music: RosaryMusic) {
    updateSettings((prev) => ({ ...prev, music }));
  }

  if (!isOpen) return null;

  return (
    <div
      className="absolute inset-0 z-[70] flex items-center justify-center p-5 bg-black/60 transition-opacity duration-300 ease-out"
      onClick={onClose}
    >
      {/* Height-safe: the card is capped at the overlay and split into a
          header that stays and a body that scrolls. Without the cap a card
          taller than the screen was centred, so it overflowed equally at
          both ends and carried its own close button off the top - and
          nothing scrolled, so there was no way to reach it or to shut the
```

```
          modal except by guessing that the backdrop closes it. It happened
          on any short viewport: a rotated phone, a small one, or a tall
          card with the keyboard up. */}
        <div
          className="w-full max-w-sm max-h-full flex flex-col rounded-2xl border border-[var(--color-brand-on-accent)]/30 bg-[var(--color-brand-primary)] text-white
shadow-2xl transition-all duration-300 ease-out font-secondary"
          onClick={(e) => e.stopPropagation()}
        >
          {/* Header */}
          <div className="shrink-0 flex items-center justify-between px-5 pt-5 pb-3 border-b border-white/10">
            <h2 className="text-base font-bold tracking-tight font-primary">Rosary Settings</h2>
            <button
              onClick={onClose}
              aria-label="Close rosary settings"
              className="flex items-center justify-center w-11 h-11 -mr-2 rounded-full text-white/80 hover:bg-white/10 hover:text-white transition-colors duration-300"
            >
              <X className="w-5 h-5" />
            </button>
          </div>

          <div className="min-h-0 flex-1 overflow-y-auto px-5 py-4 space-y-5">
            {/* Language */}
            <div>
              <div className="flex items-center gap-1.5 mb-2 text-sm font-semibold uppercase tracking-wider text-white/90">
                <Languages className="w-3.5 h-3.5" />
                <span>Language</span>
              </div>
              <div className="grid grid-cols-2 gap-2">
                <button
                  onClick={() => setLanguage("en")}
                  className={`min-h-11 rounded-xl border text-base font-bold transition-colors duration-300 ${
                    settings.language === "en"
                      ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                      : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
                  }`}
                >
                  English
                </button>
                <button
                  onClick={() => setLanguage("fil")}
                  className={`min-h-11 rounded-xl border text-base font-bold transition-colors duration-300 ${
                    settings.language === "fil"
                      ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                      : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
                  }`}
                >
                  Filipino
                </button>
              </div>
            </div>

            {/* Mystery */}
            <div>
              <div className="flex items-center gap-1.5 mb-2 text-sm font-semibold uppercase tracking-wider text-white/90">
                <Sparkles className="w-3.5 h-3.5" />
                <span>Mystery</span>
              </div>
              <div className="grid grid-cols-2 gap-2">
                {MYSTERY_OPTIONS.map((opt) => (
```

```
              <button
                key={opt.key}
                onClick={() => setMysterySet(opt.key)}
                className={`min-h-11 px-2 py-2 rounded-xl border text-base font-bold transition-colors duration-300 ${
                  opt.key === "today" ? "col-span-2" : ""
                } ${
                  settings.mysterySet === opt.key
                    ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                    : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
                }`}
              >
                {opt.label}
              </button>
            ))}
          </div>
        </div>

        {/* Background music */}
        <div>
          <div className="flex items-center gap-1.5 mb-2 text-sm font-semibold uppercase tracking-wider text-white/90">
            <Music className="w-3.5 h-3.5" />
            <span>Background Music</span>
          </div>
          <div className="grid grid-cols-2 gap-2">
            <button
              onClick={() => setMusic("on")}
              className={`min-h-11 rounded-xl border text-base font-bold transition-colors duration-300 ${
                settings.music === "on"
                  ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                  : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
              }`}
            >
              On
            </button>
            <button
              onClick={() => setMusic("off")}
              className={`min-h-11 rounded-xl border text-base font-bold transition-colors duration-300 ${
                settings.music === "off"
                  ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                  : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
              }`}
            >
              Off
            </button>
          </div>
        </div>

        <p className="text-sm text-white/70 leading-snug">
          These apply the next time the rosary is opened or started, and stay in sync with the Rosary tab.
        </p>
      </div>
    </div>
  </div>
  );
}
```

1.2.51 APP-056 — SacramentsTab.tsx

Code ID	APP-056
File Name	SacramentsTab.tsx
File Path	SanctDemo/src/components/SacramentsTab.tsx
Language / Type	TypeScript (React, TSX)
Lines	245
Functions / Classes	SacramentsTab (exported component/function (default))
Related Files (its imports)	./ParishSectionHeader, ../data, ../types, ../lib/applications, ../lib/useParishContent, ./ApplicationForm, ./SignInFirst, ./ItemDetailSections, ../lib/itemContent, ../lib/useLanguage, ../lib/language, ../lib/contentTranslations, ../lib/availability, ./AvailabilityBadge, ./WatchBell
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/SacramentsTab.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SacramentsTab.tsx

```
import React, { useState } from "react";
import ParishSectionHeader from "./ParishSectionHeader";
import { SACRAMENTS, SACRAMENT_IMAGES } from "../data";
import { Sparkles, Calendar, BookOpen, ChevronDown, ChevronUp, Check, CheckCircle, AlertCircle, Bookmark, Info, ArrowLeft, ChevronRight } from "lucide-react";
import { Route } from "../types";
import { submitApplication } from "../lib/applications";
import { useParishContent } from "../lib/useParishContent";
import ApplicationForm from "./ApplicationForm";
import SignInFirst from "./SignInFirst";
import ItemDetailSections from "./ItemDetailSections";
import { resolveSacrament } from "../lib/itemContent";
import { useLanguage } from "../lib/useLanguage";
import { pick } from "../lib/language";
import { SACRAMENT_DESCRIPTION_FIL, bilingualFor } from "../lib/contentTranslations";
import { isOpenForApplications, closedMessage } from "../lib/availability";
import AvailabilityBadge from "./AvailabilityBadge";
import WatchBell from "./WatchBell";

interface SacramentsTabProps {
  parish: Route;
  onAddApplication: (app: { id: string; type: string; applicant: string; details: string; date: string; status: string }) => void;
  /** The signed-in pilgrim. Applying without one is refused by the rules. */
  uid?: string | null;
  userEmail?: string;
  onOpenSignIn?: () => void;
}

/** Stands in until the parish photographs each sacrament. */
const PLACEHOLDER_SACRAMENT = "/parish/placeholder-photo.svg";
```

```
export default function SacramentsTab({
  parish, onAddApplication, uid, userEmail, onOpenSignIn,
}: SacramentsTabProps) {
  const parishName = parish.name.replace(" Guide", "").replace(" Tour", "");

  // Which sacraments this parish is currently taking applications for.
  const managed = useParishContent(parish.id);
  const { language } = useLanguage();

  const [expandedId, setExpandedId] = useState<string | null>(null);

  /// Which sacrament has the whole screen, or null for the list. The same
  /// split as Ministries: a sacrament is either one of several being browsed
  /// or the only thing on the page, and the old accordion made every row
  /// grow downwards while you were reading it.
  const [openId, setOpenId] = useState<string | null>(null);
  /// Which sacrament's application screen is open, or null.
  ///
  /// Apply used to scroll to a form at the bottom of this page - a form
  /// which then asked, again, which sacrament you wanted. Tapping Apply
  /// had already answered that. It is now its own screen, and the
  /// sacrament is the one whose page you were reading.
  const [applyingToId, setApplyingToId] = useState<string | null>(null);
  const [lastSent, setLastSent] = useState<string>("");

  const applyingTo = SACRAMENTS.find(s => s.id === applyingToId) ?? null;

  const toggleExpand = (id: string) => {
    setExpandedId(expandedId === id ? null : id);
  };

  if (applyingTo) {
    if (!uid) {
      return (
        <SignInFirst
          what={applyingTo.name}
          onSignIn={onOpenSignIn ?? (() => {})}
          onBack={() => setApplyingToId(null)}
        />
      );
    }
    // The parish can close a sacrament while this screen is open - the
    // content is live. Checked here as well as on the button, so a
    // screen left open overnight cannot post into a closed queue. The
    // security rules refuse it too; this makes the refusal a sentence.
    if (!isOpenForApplications(managed, applyingTo.id)) {
      setApplyingToId(null);
      return null;
    }
    return (
      <ApplicationForm
        kind="sacrament"
        itemId={applyingTo.id}
        itemName={applyingTo.name}
        parishName={parishName}
        defaults={{ email: userEmail }}
        onClose={() => setApplyingToId(null)}
        onSubmitted={() => setLastSent(applyingTo.name)}
```

```
    />
  );
}

return (
  <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] overflow-y-auto scroll-safe">
    <div hidden={openId !== null}>
    <ParishSectionHeader
      routeId={parish.id}
      eyebrow="Holy Sacraments"
      title="Sacraments Office"
      blurb="Review the guidelines, prepare the documents, and arrange a sacrament with the parish."
      icon={<Sparkles className="w-3.5 h-3.5" />}
      collapsing
    />
    </div>

    <div className="p-4 space-y-4">
      {/* Canon law requirements for these sacraments do not change from
          parish to parish — this note keeps the app honest about that
          rather than implying the list below is {parishName}-specific. */}
      <div className="flex items-start gap-2 p-3 bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] rounded-2xl"
        hidden={openId !== null}
      >
        <Info className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0 mt-0.5" />
        <p className="text-[15px] text-[var(--color-brand-text)] font-sans leading-snug">
          Canonical requirements are the same diocese-wide. Booking below will be handled by <strong>{parishName}</strong>, your current parish.
        </p>
      </div>

      {/* Sacraments Guide Accordion */}
      <div className="space-y-2.5">
        <h3 className="text-[15px] font-bold text-[var(--color-brand-secondary)] uppercase tracking-widest font-serif italic pl-1">
          Canonical Requirements
        </h3>

        <div className="space-y-2">
          {/* The list: one photo card per sacrament, the same card Ministries
              and the bulletin rail already use. */}
          {openId === null && SACRAMENTS.map(sac => (
            <button
              key={sac.id}
              type="button"
              onClick={() => { setOpenId(sac.id); setExpandedId(sac.id); }}
              className={`ministry-card${isOpenForApplications(managed, sac.id) ? "" : " is-unavailable"}`}
            >
              <span className="ministry-card__media">
                <img
                  src={SACRAMENT_IMAGES[sac.id] ?? PLACEHOLDER_SACRAMENT}
                  alt=""
                  loading="lazy"
                  onError={e => { (e.currentTarget as HTMLImageElement).src = PLACEHOLDER_SACRAMENT; }}
                />
                <span className="ministry-card__scrim" aria-hidden />
                <span className="ministry-card__caption">{sac.name}</span>
              </span>
              <span className="ministry-card__foot">
                <Bookmark className="w-5 h-5 text-[var(--color-brand-primary)] shrink-0" />
                <span className="ministry-card__hint">{pick(bilingualFor(sac.description, SACRAMENT_DESCRIPTION_FIL[sac.id], "en"), language)}</span>
```

```
      <ChevronRight className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
    </span>
  </button>
))}

{(openId ? SACRAMENTS.filter(x => x.id === openId) : []).map((sac) => {
  const isExpanded = true;
  const resolved = resolveSacrament(managed, sac.id, language);
  return (
    <div
      key={sac.id}
      className="bg-[var(--color-brand-card)] rounded-2xl border border-[var(--color-brand-border)] overflow-hidden shadow-xs transition-all"
    >
      <div className="ministry-hero">
        <img
          src={SACRAMENT_IMAGES[sac.id] ?? PLACEHOLDER_SACRAMENT}
          alt=""
          onError={e => { (e.currentTarget as HTMLImageElement).src = PLACEHOLDER_SACRAMENT; }}
        />
        <span className="ministry-hero__scrim" aria-hidden />
        <button
          type="button"
          onClick={() => setOpenId(null)}
          aria-label="Back to the sacraments"
          className="ministry-hero__back"
        >
          <ArrowLeft className="w-5 h-5" />
        </button>
        <h4 className="ministry-hero__name">{sac.name}</h4>
      </div>

      {isExpanded && (
        <div className="px-4 pb-4 pt-1 border-t border-[var(--color-brand-card)] bg-[var(--color-brand-card)]/30 space-y-3 font-sans">
          {/* Closed today is not closed forever. Without
              this the only way to find out sacrament reopened
              was to keep coming back and looking at the same
              badge. */}
          <div className="flex flex-wrap items-center gap-2">
            <AvailabilityBadge
              open={isOpenForApplications(managed, sac.id)}
              kind="sacrament"
            />
            {!isOpenForApplications(managed, sac.id) && (
              <WatchBell
                watch={{ kind: "sacrament", parishId: parish.id, itemId: sac.id }}
                label={sac.name}
              />
            )}
          </div>

          {/* The parish's own words where they have written
              any; the compiled text where they have not.
              Requirements, schedule, process and reminders
              each render only when there is something in
              them - see ItemDetailSections. */}
          <p className="text-[15px] text-[var(--color-brand-text)] leading-relaxed">
            {resolved?.about ?? sac.description}
          </p>
```

```
                    {resolved && <ItemDetailSections item={resolved} />}

                    {/* The information stays readable either way. A
                        pilgrim whose parish has paused Confirmation
                        still needs the requirements, so they can have
                        the papers ready when it reopens. */}
                    {isOpenForApplications(managed, sac.id) ? (
                      <button
                        onClick={() => setApplyingToId(sac.id)}
                        className="py-1.5 px-3 bg-[var(--color-brand-primary)] text-white text-sm font-bold uppercase tracking-wider rounded-full hover:bg-[var(--
color-brand-primary-dark)] transition-colors"
                      >
                        Apply now
                      </button>
                    ) : (
                      <div className="space-y-1.5">
                        <button
                          type="button"
                          disabled
                          className="py-1.5 px-3 bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)] text-[var(--color-brand-secondary)]
text-sm font-bold uppercase tracking-wider rounded-full cursor-not-allowed"
                        >
                          Not available
                        </button>
                        <p className="text-sm leading-relaxed text-[var(--color-brand-secondary)]">
                          {closedMessage("sacrament", sac.name)}
                        </p>
                      </div>
                    )}
                  </div>
                )}
              </div>
            );
          })}
        </div>
      </div>

    </div>
  </div>
);
}
```

1.2.52 APP-057 — SanctiHost.tsx

Code ID	APP-057
File Name	SanctiHost.tsx
File Path	SanctDemo/src/components/SanctiHost.tsx
Language / Type	TypeScript (React, TSX)
Lines	490
Functions / Classes	SanctiTools (exported type); parishLabel (internal function); SanctiHost (exported component/function (default))
Related Files (its imports)	./SanctiSheet, ./Spotlight, ../lib/itemContent, ../lib/availability, ../lib/geo, ../data, ./ChurchHistory, ../lib/parishContent, ../lib/askSancti, ../lib/language
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/SanctiHost.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SanctiHost.tsx

```
import { useCallback, useRef, useState } from "react";
import SanctiSheet, { SanctiButton, type SanctiMessage } from "./SanctiSheet";
import { useSpotlight } from "./Spotlight";
import {
  understand, aliasesFor, itemAliases, needsConfirmation, readAnswer,
  type SanctiAction, type PriorTurn,
} from "../lib/sancti";
import {
  massAnswer, historyAnswer, itemAnswer, distanceAnswer, contactAnswer,
  reminderAnswer, helpAnswer, unknownAnswer, offerFor, offerForItem, withOffer,
  declinedAnswer,
  openingLine, type Reply,
} from "../lib/sanctiAnswers";
import { resolveSacrament, resolveMinistry } from "../lib/itemContent";
import { isOpenForApplications } from "../lib/availability";
import { haversineMeters, type Coordinates } from "../lib/geo";
import { ROUTES, MASS_SCHEDULES, MINISTRIES, SACRAMENTS } from "../data";
import { parishHistoryLede } from "./ChurchHistory";
import type { ParishContent } from "../lib/parishContent";
import { askSancti } from "../lib/askSancti";
import type { Language } from "../lib/language";

/**
 * Sancti, wired to the app.
 *
 * ## Why the plumbing is separate from the chat
 *
 * SanctiSheet draws bubbles and knows nothing else. This decides what a
 * sentence means, fetches the parish's real data, writes the answer and
 * then does the thing — switching tabs, pointing at a button, setting a
```

```
 * reminder. Keeping those apart means the chat can be restyled without
 * touching the behaviour, and the behaviour can be reasoned about
 * without reading any JSX.
 *
 * ## Why answers are late and actions are later
 *
 * Sancti replies, and only then navigates. Acting first would slide the
 * screen away before the pilgrim had read why — which is the difference
 * between a guide and a jump-cut.
 */

export interface SanctiTools {
  /** Switch tabs. */
  go: (tab: string) => void;
  /**
   * Show one part of the Parish Bulletin on Home.
   *
   * Home shows one of its three sections at a time, so going to Home is
   * not the same as showing the thing on Home that was asked for.
   */
  showHomeSection: (section: "ministries" | "mass" | "history") => void;
  /** Open one named ministry or sacrament, rather than the list of them. */
  openItem: (tab: "ministries" | "sacraments", itemId: string) => void;
  /** Run the walkthrough again, from the start. */
  replayTutorial: () => void;
  /** Switch the reading language for the whole app. */
  setLanguage: (language: "en" | "fil") => void;
  /** Make this parish the one on screen. */
  selectParish: (parishId: string) => void;
  /** Open the map focused on a parish. */
  walkTo: (parishId: string) => void;
  /** Turn a Mass reminder on for a parish the pilgrim does not belong to. */
  followParish: (parishId: string) => void;
  /** The parish currently on screen. */
  activeParishId: string;
  activeParishName: string;
  /** Admin-managed content for the active parish, if any. */
  content: ParishContent | null;
  /** Where the pilgrim is, when they have allowed it. */
  position: Coordinates | null;
  /**
   * The reading language. Sancti ANSWERS in this, but understands both
   * whatever it is set to - a pilgrim reading in English still types
   * "anong oras ang misa", and refusing that would be the app telling
   * someone their own language is the wrong one.
   */
  language: Language;
}

const PARISH_NAMES = ROUTES.map(r => aliasesFor(r.id, r.name));

/**
 * Every ministry and sacrament, by name.
 *
 * Built once. These are the same fifteen ministries and five sacraments
 * at both parishes - the parish decides which are open, not which
 * exist - so the list does not depend on which parish is on screen.
 */
const ITEM_NAMES = [
```

```
  ...MINISTRIES.map(m => itemAliases(m.id, m.name, "ministry")),
  ...SACRAMENTS.map(s => itemAliases(s.id, s.name, "sacrament")),
];

function parishLabel(parishId: string): string {
  const route = ROUTES.find(r => r.id === parishId);
  return route?.name.replace(" Guide", "").replace(" Tour", "") ?? "this parish";
}

export default function SanctiHost({ tools }: { tools: SanctiTools }) {
  const spotlight = useSpotlight();
  const [open, setOpen] = useState(false);
  const [messages, setMessages] = useState<SanctiMessage[]>([]);
  /**
   * The transcript, for the model's benefit only.
   *
   * A ref beside the state because `ask` must keep a stable identity -
   * it holds a pending offer across a turn, and rebuilding it whenever
   * a message arrives would drop that offer mid-conversation.
   */
  const messagesRef = useRef<SanctiMessage[]>([]);
  messagesRef.current = messages;
  const [thinking, setThinking] = useState(false);

  const toolsRef = useRef(tools);
  toolsRef.current = tools;

  const say = useCallback((from: SanctiMessage["from"], reply: Reply | string) => {
    const body = typeof reply === "string" ? { text: reply } : reply;
    setMessages(m => [
      ...m,
      { id: `${Date.now()}-${m.length}`, from, text: body.text, followUp: body.followUp },
    ]);
  }, []);

  /**
   * Points at something in the app, after closing the sheet.
   *
   * The same spotlight the tutorial uses, so "the Map is here" looks
   * like the tour the pilgrim already saw rather than a second,
   * unfamiliar highlight. It times out by itself: a highlight that
   * waits for a tap is a modal, and nobody asked for one.
   */
  const point = useCallback((target: string, body: string) => {
    setOpen(false);
    window.setTimeout(() => {
      spotlight.show({ target, card: { body }, autoHideMs: 4200, dismissOnTapOutside: true });
    }, 360);
  }, [spotlight]);

  /**
   * The thing Sancti has offered to open and is waiting on a yes for.
   *
   * A ref rather than state: it is read and written inside `ask`, which
   * must not be rebuilt between the question and the answer - a new
   * `ask` identity mid-conversation would lose the offer it is holding.
   */
  /**
   * What the last exchange was about, so a fragment can continue it.
```

```
 * A ref, not state: it is read on the next message rather than
 * rendered, and re-rendering the chat to remember a topic would
 * repaint the whole sheet for nothing visible.
 */
const priorRef = useRef<PriorTurn | undefined>(undefined);
const pendingRef = useRef<{
  action: SanctiAction; parishId?: string; itemId?: string; language?: "en" | "fil";
} | null>(null);

const act = useCallback((
  action: SanctiAction,
  parishId?: string,
  itemId?: string,
  language?: "en" | "fil",
) => {
  const t = toolsRef.current;
  const target = parishId ?? t.activeParishId;

  switch (action) {
    case "OPEN_MAP":
      t.go("navigator");
      point("tab-map", "This is the Map — every parish in the diocese, with walking directions.");
      break;
    case "SHOW_CHURCH_LOCATION":
      t.walkTo(target);
      setOpen(false);
      break;
    case "OPEN_CHURCH":
      t.selectParish(target);
      t.go("home");
      setOpen(false);
      break;
    case "OPEN_AR":
      t.go("ar");
      point("tab-scan", "Point your camera at a statue or marker and the scanner will name it.");
      break;
    case "OPEN_MASS_SCHEDULE":
      if (parishId) t.selectParish(parishId);
      // Home, with the bulletin already turned to the Mass schedule.
      //
      // Going to Home was not enough once Home became a chooser: it
      // shows one of its three sections at a time, so accepting "open
      // the Mass schedule" landed the pilgrim on the ministries with
      // a highlight pointing at a card that was not rendered. The
      // section has to be chosen for them, which is the whole
      // difference between arriving at a screen and arriving at the
      // thing they asked for.
      t.go("home");
      t.showHomeSection("mass");
      point("mass-schedule", "The Mass times for this parish, kept by the parish office.");
      break;
    case "OPEN_CHURCH_HISTORY":
      if (parishId) t.selectParish(parishId);
      t.go("history");
      setOpen(false);
      break;
    case "OPEN_SACRAMENTS":
    case "OPEN_BAPTISM":
    case "OPEN_WEDDING":
```

```
      // Named one, opened to that one. Landing the pilgrim on a list
      // with the thing they named somewhere inside it is not opening
      // what they asked for.
      if (itemId) t.openItem("sacraments", itemId);
      t.go("sacraments");
      setOpen(false);
      break;
    case "OPEN_MINISTRIES":
      if (itemId) t.openItem("ministries", itemId);
      t.go("ministries");
      setOpen(false);
      break;
    case "OPEN_SETTINGS":
      t.go("me");
      point("tab-me", "Reminders, your profile and your applications live here.");
      break;
    case "REPLAY_TUTORIAL":
      // Closes the sheet first: the walkthrough points at the tab bar
      // and the sheet is sitting on top of it.
      setOpen(false);
      window.setTimeout(() => t.replayTutorial(), 260);
      break;
    case "CHANGE_LANGUAGE":
      // Only when the pilgrim named one. "Change the language" with
      // no target is answered in the chat instead - switching the
      // whole app to a language nobody asked for is not a guess worth
      // making.
      if (language) t.setLanguage(language);
      break;
    case "CREATE_REMINDER":
      // Following the parish is what actually schedules the alarms;
      // the pilgrim's own parish is always followed already.
      if (parishId && parishId !== t.activeParishId) t.followParish(parishId);
      break;
    default:
      break;
  }
}, [point]);

/**
 * One question, answered.
 *
 * The pause is deliberate and short. Everything here is local, so a
 * reply could land in the same frame as the question — which reads as
 * the app ignoring you and printing a canned line. A beat makes it
 * read as having been considered.
 */
const ask = useCallback((text: string) => {
  say("you", text);
  setThinking(true);

  // Async only for the one branch that reaches the network; every
  // other answer is still produced synchronously inside this tick.
  // askSancti never throws, so there is nothing here to catch.
  window.setTimeout(async () => {
    const t = toolsRef.current;

    // A bare yes or no, when Sancti has an offer open.
    //
```

```
// Checked before understanding the sentence, because "open it" and
// "sige" carry no intent of their own - they only mean anything as
// an answer to what was just asked.
const pending = pendingRef.current;
if (pending) {
  const answer = readAnswer(text);
  if (answer === "yes") {
    pendingRef.current = null;
    setThinking(false);
    act(pending.action, pending.parishId, pending.itemId, pending.language);
    return;
  }
  if (answer === "no") {
    pendingRef.current = null;
    setThinking(false);
    say("sancti", declinedAnswer());
    return;
  }
  // Anything else is a new question, and the offer lapses rather
  // than waiting around to be answered by an unrelated sentence.
  pendingRef.current = null;
}

const heard = understand(text, PARISH_NAMES, ITEM_NAMES, priorRef.current);

// Remembered before the answer is built, so a reply that throws
// still leaves the conversation somewhere sensible. Only an
// understood turn becomes context: carrying UNKNOWN forward would
// let one confusing sentence poison every fragment after it.
if (heard.action !== "UNKNOWN") {
  priorRef.current = {
    action: heard.action,
    ...(heard.parishId ? { parishId: heard.parishId } : {}),
    ...(heard.itemId ? { itemId: heard.itemId } : {}),
  };
}
const parishId = heard.parishId ?? t.activeParishId;
const name = parishLabel(parishId);

// The parish's own times where the office has entered them, the
// compiled ones otherwise — the same precedence the Mass card
// uses, so Sancti and the card can never disagree.
const schedule =
  (parishId === t.activeParishId ? t.content?.massSchedule : undefined)
  ?? MASS_SCHEDULES[parishId]?.schedule
  ?? [];

let reply: Reply;
switch (heard.action) {
  case "OPEN_MASS_SCHEDULE":
    reply = massAnswer(name, schedule);
    break;
  case "CREATE_REMINDER":
    reply = reminderAnswer(name, schedule, heard.time);
    break;
  case "OPEN_CHURCH_HISTORY": {
    // The parish's own words where an admin has written them, the
    // compiled opening otherwise - the same order the history
    // card uses, so the two can never disagree about whether this
```

```
  // parish has a history at all.
  const managed = parishId === t.activeParishId ? t.content?.historyBody?.trim() : undefined;
  reply = historyAnswer(name, managed || parishHistoryLede(parishId));
  break;
}
case "OPEN_BAPTISM": {
  const item = resolveSacrament(t.content, "sac-baptism", t.language);
  reply = itemAnswer(item, isOpenForApplications(t.content, "sac-baptism"));
  break;
}
case "OPEN_WEDDING": {
  const item = resolveSacrament(t.content, "sac-matrimony", t.language);
  reply = itemAnswer(item, isOpenForApplications(t.content, "sac-matrimony"));
  break;
}
case "OPEN_SACRAMENTS":
  reply = { text: `${name} has a page for each sacrament, with what the office needs for it.` };
  break;
case "OPEN_MINISTRIES": {
  const first = resolveMinistry(t.content, "min-altar-servers", t.language);
  reply = first
    ? { text: `${name} has lay ministries you can join.` }
    : unknownAnswer();
  break;
}
case "ANSWER_DISTANCE": {
  const route = ROUTES.find(r => r.id === parishId);
  const metres =
    t.position && route
      ? haversineMeters(t.position, route.coordinates)
      : null;
  reply = distanceAnswer(name, metres, t.position !== null);
  break;
}
case "ANSWER_CONTACT":
  reply = contactAnswer(name, resolveSacrament(t.content, "sac-baptism", t.language)?.contact);
  break;
// These say WHERE the thing is rather than announcing that
// Sancti is about to go there. The offer underneath does the
// asking, and a reply that already said "I'll show you" would
// be promising something the pilgrim has not agreed to yet.
case "OPEN_MAP":
  reply = { text: "The map is on the Map tab — every parish in the diocese, with walking directions." };
  break;
case "SHOW_CHURCH_LOCATION":
  reply = { text: `${name} is on the map, and I can walk you there from where you are.` };
  break;
case "OPEN_CHURCH":
  reply = { text: `${name} has its own page — Mass times, history and what the office offers.` };
  break;
case "OPEN_AR":
  reply = { text: "The scanner is on the Scan tab. Point it at a statue, an image or a marker and it will tell you what it is." };
  break;
case "REPLAY_TUTORIAL":
  reply = { text: "I'll show you around. It takes about a minute, and you can stop at any point." };
  break;
case "CHANGE_LANGUAGE":
  reply = heard.language
    ? {
```

```
              text: heard.language === "fil"
                ? "Sige — nakalipat na sa Tagalog ang buong app."
                : "Done — the whole app is in English now.",
            }
            : {
              // No language named. Rather than guess, say where the
              // setting is and what it covers.
              text: "You can read SanctiWalk in English or Tagalog. The setting is in Me, under Language, and it changes everything including the station and
ministry descriptions. Tell me which you want and I'll switch it.",
              followUp: "Change to Tagalog",
            };
          break;
        case "OPEN_SETTINGS":
          reply = { text: "Your reminders, your profile and your applications are under Me." };
          break;
        case "HELP":
          reply = helpAnswer(name);
          break;
        default: {
          /*
           * The one case worth sending to the model.
           *
           * Everything above is a question SanctiWalk can answer from
           * its own data, instantly and with no signal - and routing
           * those through a network call would make a working feature
           * slower and less reliable to no end. What lands here is the
           * question the matcher could not place, which until now got
           * the same shrug however reasonable it was. That shrug is
           * what made Sancti feel generic.
           *
           * The model sees only facts retrieved from the parish's own
           * records, server-side; it is never asked to recall a Mass
           * time. If it cannot be reached, the shrug is still there.
           */
          const aiReply = await askSancti({
            question: text,
            parishId,
            parishName: name,
            history: messagesRef.current.slice(-6).map(m => ({ role: m.from, text: m.text })),
          });
          reply = aiReply ? { text: aiReply.answer } : unknownAnswer();
        }
      }

      setThinking(false);

      // Ask before leaving the screen.
      //
      // Sancti used to answer and then navigate, every time: "where is
      // the map" opened the map, and the pilgrim lost the conversation
      // to an answer they had not asked to be taken to. A question now
      // gets its answer in full, with the screen offered underneath as
      // one tap. An instruction - "open the map" - is consent already
      // given and still goes straight there.
      // A named ministry or sacrament is offered BY NAME. "Want me to
      // open the ministries?" is the wrong question to ask somebody who
      // just typed the name of one - and naming it back is how they can
      // tell Sancti understood which, especially after a misspelling.
      const namedItem = heard.itemId
```

```
        ? (resolveMinistry(t.content, heard.itemId, t.language)
          ?? resolveSacrament(t.content, heard.itemId, t.language))
        : null;
      if (namedItem) reply = itemAnswer(namedItem, isOpenForApplications(t.content, heard.itemId!));

      const offer = !needsConfirmation(heard)
        ? null
        : namedItem
          ? offerForItem(namedItem.name)
          : offerFor(heard.action);
      if (offer) {
        pendingRef.current = {
          action: heard.action, parishId: heard.parishId,
          itemId: heard.itemId, language: heard.language,
        };
        say("sancti", withOffer(reply, offer));
        return;
      }

      // Told to, rather than asked: acknowledge and go. Repeating the
      // full answer under an instruction to open it reads as not having
      // been heard, and the screen carrying that answer is already on
      // its way.
      const opening = heard.imperative ? openingLine(heard.action) : null;
      say("sancti", opening ? { text: opening } : reply);

      // Said first, then done. See the note at the top.
      if (heard.action !== "UNKNOWN" && heard.action !== "HELP") {
        window.setTimeout(() => act(heard.action, heard.parishId, heard.itemId, heard.language), 620);
      }
    }, 420);
  }, [say, act]);

  return (
    <>
      <SanctiButton onClick={() => setOpen(true)} />
      <SanctiSheet
        open={open}
        onClose={() => setOpen(false)}
        messages={messages}
        thinking={thinking}
        parishName={tools.activeParishName}
        onSend={ask}
        onFollowUp={m => ask(m.followUp ?? "")}
      />
    </>
  );
}
```

1.2.53 APP-058 — SanctiSheet.tsx

Code ID	APP-058
File Name	SanctiSheet.tsx
File Path	SanctDemo/src/components/SanctiSheet.tsx
Language / Type	TypeScript (React, TSX)
Lines	249
Functions / Classes	SanctiMessage (exported type); SanctiSheet (exported component/function (default)); loadTucked (internal function); SanctiButton (exported function)
Related Files (its imports)	../lib/sancti, ../lib/useLanguage, ../lib/ui, ../lib/sanctiAnswers
Where It Is Used	src/components/SanctiHost.tsx
How to Recognize It	Open SanctDemo/src/components/SanctiSheet.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SanctiSheet.tsx

```
import { useEffect, useRef, useState } from "react";
import { Send, X } from "lucide-react";
import { SUGGESTIONS } from "../lib/sancti";
import { useLanguage } from "../lib/useLanguage";
import { t } from "../lib/ui";
import type { Reply } from "../lib/sanctiAnswers";

/**
 * Sancti, as a sheet over whatever you were looking at.
 *
 * ## Why a sheet and not a screen
 *
 * Most of what Sancti does is take you somewhere. A full screen would
 * have to close itself to do that, and the pilgrim would lose the
 * conversation at the moment it paid off. Over the app, it can answer,
 * slide away to show you the thing, and still be there when you come
 * back.
 *
 * ## Why it does not look like a chat app
 *
 * The brief is explicit, and it is right: this is for people who have
 * never used one. So there is no typing indicator pretending somebody
 * is at the other end, no avatars on every line, no timestamps. Two
 * kinds of bubble, the parish's own navy and card colours, and the
 * suggestions sitting in the empty state where a new user will look.
 */

export interface SanctiMessage {
  id: string;
  from: "you" | "sancti";
  text: string;
```

```
  /** A tappable follow-up Sancti offered with this answer. */
  followUp?: string;
}

export default function SanctiSheet({
  open, onClose, messages, thinking, onSend, onFollowUp, parishName,
}: {
  open: boolean;
  onClose: () => void;
  messages: SanctiMessage[];
  thinking: boolean;
  onSend: (text: string) => void;
  onFollowUp: (message: SanctiMessage) => void;
  parishName: string;
}) {
  const { language } = useLanguage();
  const [draft, setDraft] = useState("");
  const endRef = useRef<HTMLDivElement | null>(null);
  const inputRef = useRef<HTMLInputElement | null>(null);

  // New answers scroll into view; the input keeps focus so a follow-up
  // question does not need the keyboard summoned again.
  useEffect(() => {
    if (open) endRef.current?.scrollIntoView({ block: "end" });
  }, [open, messages.length, thinking]);

  useEffect(() => {
    if (open) window.setTimeout(() => inputRef.current?.focus(), 250);
  }, [open]);

  if (!open) return null;

  function send(text: string) {
    const clean = text.trim();
    if (!clean) return;
    onSend(clean);
    setDraft("");
  }

  return (
    <div className="sancti" role="dialog" aria-modal="true" aria-label="Ask Sancti">
      <button type="button" className="sancti__scrim" onClick={onClose} aria-label="Close Sancti" />

      <div className="sancti__sheet">
        <header className="sancti__head">
          <span className="sancti__avatar" aria-hidden>
            <img src="/ui/sanctiwalk-mark-white.png" alt="" />
          </span>
          <span className="sancti__who">
            <span className="sancti__name">Sancti</span>
            <span className="sancti__sub">{t("sancti.subtitle", language)} {parishName}</span>
          </span>
          <button type="button" onClick={onClose} aria-label="Close" className="sancti__x">
            <X className="w-5 h-5" />
          </button>
        </header>

        <div className="sancti__log">
          {messages.length === 0 && (
```

```
    <div className="sancti__empty">
      <p className="sancti__greeting">
        {t("sancti.greeting", language)}
      </p>
      <p className="sancti__try">{t("sancti.tryThese", language)}</p>
      <div className="sancti__chips">
        {SUGGESTIONS.map(s => (
          <button key={s} type="button" className="sancti__chip" onClick={() => send(s)}>
            {s}
          </button>
        ))}
      </div>
    </div>
  )}

  {messages.map(m => (
    <div key={m.id} className={`sancti__row sancti__row--${m.from}`}>
      <div className={`sancti__bubble sancti__bubble--${m.from}`}>
        {/* Answers carry real line breaks - a Mass schedule is a
            list, not a paragraph - so they are split rather than
            run together. */}
        {m.text.split("\n").map((line, i) => (
          <span key={i} className="sancti__line">{line}</span>
        ))}
      </div>
      {m.followUp && (
        <button type="button" className="sancti__followup" onClick={() => onFollowUp(m)}>
          {m.followUp}
        </button>
      )}
    </div>
  ))}

  {thinking && (
    <div className="sancti__row sancti__row--sancti">
      <div className="sancti__bubble sancti__bubble--sancti sancti__thinking" aria-label="Sancti is looking that up">
        <span /><span /><span />
      </div>
    </div>
  )}

  <div ref={endRef} />
</div>

<form
  className="sancti__compose"
  onSubmit={e => { e.preventDefault(); send(draft); }}
>
  <input
    ref={inputRef}
    value={draft}
    onChange={e => setDraft(e.target.value)}
    placeholder={t("sancti.placeholder", language)}
    aria-label="Ask Sancti a question"
    className="sancti__input"
  />
  <button
    type="submit"
    className="sancti__send"
```

```
            disabled={!draft.trim()}
            aria-label="Send"
          >
            <Send className="w-4 h-4" />
          </button>
        </form>
      </div>
    </div>
  );
}

/** Where the tucked/untucked choice is remembered. */
const TUCK_KEY = "sanctiwalk.sanctiTucked";

function loadTucked(): boolean {
  try { return localStorage.getItem(TUCK_KEY) === "1"; } catch { return false; }
}

/**
 * The way in: a floating button above the tab bar.
 *
 * Here rather than in the tab bar because Sancti's job is taking you
 * places, which means it has to be reachable from the place you are
 * stuck on — and because a sixth tab would have pushed five labels
 * into truncating.
 *
 * ## Why it can be pushed aside
 *
 * A pill that says "Ask Sancti" is wide, and it floats over the bottom
 * right of every screen — which on Home is where the bulletin's own
 * announcements sit. Sancti was covering the parish's notices, and a
 * helper that hides the thing you opened the app to read is worse than
 * no helper.
 *
 * Dragging it sideways tucks it to a small mark at the edge, and the
 * choice is remembered because somebody who moved it once meant it.
 *
 * Tucked, a tap OPENS IT BACK OUT rather than opening Sancti. A bare
 * mark at the edge of the screen does not say "Ask Sancti" to anybody
 * who has not been told, and a control nobody recognises that jumps
 * straight into a chat is worse than one that is merely small. So the
 * first tap brings the words back and the second one asks. Nothing is
 * ever more than two taps away, and the thing you tap always says what
 * it is before it does anything.
 *
 * Tap and drag are told apart by distance, not by timing: under eight
 * pixels of travel is a tap, which is forgiving of the thumb-roll that
 * every real tap on a phone has in it.
 */
export function SanctiButton({ onClick }: { onClick: () => void }) {
  const { language } = useLanguage();
  const [tucked, setTucked] = useState(loadTucked);
  const drag = useRef<{ x: number; moved: boolean } | null>(null);

  function remember(next: boolean) {
    setTucked(next);
    try { localStorage.setItem(TUCK_KEY, next ? "1" : "0"); } catch { /* private mode */ }
  }
```

```
  const DRAG_THRESHOLD = 8;

  return (
    <button
      type="button"
      onPointerDown={e => {
        drag.current = { x: e.clientX, moved: false };
        // Captured, or a quick swipe stops reporting the moment the
        // finger leaves the pill - which on a small target is most of
        // them, and the gesture would simply not work.
        try { e.currentTarget.setPointerCapture(e.pointerId); } catch { /* older webview */ }
      }}
      onPointerMove={e => {
        if (!drag.current) return;
        const dx = e.clientX - drag.current.x;
        if (Math.abs(dx) < DRAG_THRESHOLD) return;
        drag.current.moved = true;
        // Rightwards pushes it away, leftwards pulls it back: the
        // direction matches where it goes.
        remember(dx > 0);
      }}
      onPointerUp={() => {
        const wasDrag = drag.current?.moved ?? false;
        drag.current = null;
        if (wasDrag) return;
        // Tucked: the tap is "show me what this is", not "open it".
        if (tucked) remember(false);
        else onClick();
      }}
      onPointerCancel={() => { drag.current = null; }}
      data-spotlight="sancti-button"
      className={`sancti-fab${tucked ? " sancti-fab--tucked" : ""}`}
      aria-label={t("sancti.ask", language)}
      aria-expanded={!tucked}
      title={t("sancti.ask", language)}
    >
      <img src="/ui/sanctiwalk-mark-white.png" alt="" aria-hidden />
      <span className="sancti-fab__label">{t("sancti.ask", language)}</span>
    </button>
  );
}
```

1.2.54 APP-059 — ScannerChrome.tsx

Code ID	APP-059
File Name	ScannerChrome.tsx
File Path	SanctDemo/src/components/ScannerChrome.tsx
Language / Type	TypeScript (React, TSX)
Lines	130
Functions / Classes	ScannerMode (exported type); ScannerHeader (exported function); ScannerSheet (exported function); ScannerTabs (exported function); ScannerReticle (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/ScannerChrome.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ScannerChrome.tsx

```
import type { ReactNode } from "react";
import { X } from "lucide-react";

/**
 * The scanner's furniture: the instruction over the camera, the dark
 * sheet at the bottom, and the control that switches between the two
 * scanners.
 *
 * ## Why these are separate from ArTour
 *
 * ArTour is 900 lines of camera handling, recognition and error states.
 * The chrome is the part the client is actually looking at, it is the
 * same in every state, and keeping it here means a change to the look
 * is one file rather than a hunt through the logic.
 *
 * ## The look
 *
 * Near-black is not a style choice here — it is a darkened camera feed,
 * and anything placed on it has to survive a bright window behind a
 * statue or an unlit side chapel. So: white type at two weights,
 * everything else held back, and exactly one accent.
 *
 * That accent is a candle gold rather than the usual interface blue.
 * The parish's navy disappears against a dark nave, and this is the one
 * colour in the building that is already associated with "look here" -
 * gilt on a retablo, a lit taper. It is spent on the AR states only,
 * where the pilgrim is being asked to find something.
 */

export type ScannerMode = "ai" | "ar";
```

```
/**
 * The instruction, centred over the camera.
 *
 * Two lines and no more: what to point at, and what will happen. Anyone
 * reading this is holding a phone up in a church, which is not the
 * posture for a paragraph.
 */
export function ScannerHeader({
  title,
  subtitle,
  onClose,
}: {
  title: string;
  subtitle?: string;
  onClose?: () => void;
}) {
  return (
    <div className="scan-head">
      <div className="scan-head__text">
        <h2 className="scan-head__title">{title}</h2>
        {subtitle && <p className="scan-head__sub">{subtitle}</p>}
      </div>
      {onClose && (
        <button
          type="button"
          onClick={onClose}
          aria-label="Close the scanner"
          className="scan-head__close"
        >
          <X className="w-5 h-5" />
        </button>
      )}
    </div>
  );
}

/** The dark panel the controls live in. */
export function ScannerSheet({ children }: { children: ReactNode }) {
  return <div className="scan-sheet">{children}</div>;
}

/**
 * AI or AR.
 *
 * Two radio buttons wearing a segmented control, rather than two
 * `<button>`s with an `aria-pressed` each. They are one choice with two
 * answers, and a radiogroup is the only shape that says so to a screen
 * reader — and it brings arrow-key movement between them for free.
 */
export function ScannerTabs({
  mode,
  onChange,
}: {
  mode: ScannerMode;
  onChange: (mode: ScannerMode) => void;
}) {
  const options: Array<{ value: ScannerMode; label: string; hint: string }> = [
    { value: "ai", label: "AI Scanner", hint: "Identify anything with the camera" },
    { value: "ar", label: "AR Scanner", hint: "Start the AR tour at a marker" },
```

```
  ];

  return (
    <div className="scan-tabs" role="radiogroup" aria-label="Which scanner">
      {options.map(option => (
        <button
          key={option.value}
          type="button"
          role="radio"
          aria-checked={mode === option.value}
          aria-label={`${option.label}. ${option.hint}`}
          onClick={() => onChange(option.value)}
          className={`scan-tab${mode === option.value ? " is-on" : ""}`}
        >
          {option.label}
        </button>
      ))}
    </div>
  );
}

/**
 * The corner brackets that say "put it here".
 *
 * Drawn as four corners rather than a full rectangle: a closed frame
 * reads as a border around the whole picture, while corners read as a
 * target to fill. `tight` pulls them in and warms them to gold while
 * something is being read, so the state change is visible without any
 * text having to announce it.
 */
export function ScannerReticle({ tight }: { tight?: boolean }) {
  return (
    <div className={`scan-reticle${tight ? " is-reading" : ""}`} aria-hidden>
      <span className="scan-reticle__c scan-reticle__c--tl" />
      <span className="scan-reticle__c scan-reticle__c--tr" />
      <span className="scan-reticle__c scan-reticle__c--bl" />
      <span className="scan-reticle__c scan-reticle__c--br" />
    </div>
  );
}
```

1.2.55 APP-060 — SearchScreen.tsx

Code ID	APP-060
File Name	SearchScreen.tsx
File Path	SanctDemo/src/components/SearchScreen.tsx
Language / Type	TypeScript (React, TSX)
Lines	250
Functions / Classes	loadRecent (internal function); pushRecent (internal function); SearchScreen (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/mapSearch, ../data, ../lib/schedule, ../lib/geo, ./ParishListRow, ../data/diocese-parishes.json, ../lib/parishIds
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/SearchScreen.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SearchScreen.tsx

```
import { useEffect, useMemo, useState } from "react";
import { Search as SearchIcon, Clock } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import { searchParishesScored } from "../lib/mapSearch";
import { MASS_SCHEDULES } from "../data";
import { nextMass } from "../lib/schedule";
import { type Coordinates } from "../lib/geo";
import ParishListRow from "./ParishListRow";
import parishData from "../data/diocese-parishes.json";
import { routeIdForParish } from "../lib/parishIds";

interface DioceseParish {
  id: string;
  name: string;
  vicariate: string;
  coordinates: Coordinates;
  status: string;
}

const PARISHES = (parishData as { parishes: DioceseParish[] }).parishes;

/** The two parishes with a tour, a schedule and a history behind them. */
const LIVE_IDS = new Set(PARISHES.filter(p => p.status === "live").map(p => p.id));

const RECENT_KEY = "sanctiwalk.recentSearches";
const MAX_RECENT = 4;

function loadRecent(): string[] {
  try {
    const raw = localStorage.getItem(RECENT_KEY);
```

```
    return raw ? (JSON.parse(raw) as string[]).slice(0, MAX_RECENT) : [];
  } catch {
    // Storage may be unavailable (private mode, quota). Recents are a
    // convenience; losing them must never break search.
    return [];
  }
}

function pushRecent(parishId: string): string[] {
  const next = [parishId, ...loadRecent().filter(id => id !== parishId)].slice(0, MAX_RECENT);
  try {
    localStorage.setItem(RECENT_KEY, JSON.stringify(next));
  } catch {
    /* ignore */
  }
  return next;
}

/**
 * The design offers four filters: Parishes, Mass soon, Confession, Places.
 *
 * Only two of those have data behind them. Confession times were never
 * collected — they are a line on the parish sheet the field team is still
 * filling in — and "Places" means non-parish landmarks, of which this app has
 * none. Rather than render two chips that quietly return nothing, they are
 * absent, and the screen says what is missing.
 */
type Filter = "parishes" | "mass-soon";

const FILTERS: { id: Filter; label: string }[] = [
  { id: "parishes", label: "Parishes" },
  { id: "mass-soon", label: "Mass soon" },
];

interface SearchScreenProps {
  onSelectParish: (parishId: string) => void;
  onClose: () => void;
}

export default function SearchScreen({ onSelectParish, onClose }: SearchScreenProps) {
  const { position } = usePresence();
  const [query, setQuery] = useState("");
  const [filter, setFilter] = useState<Filter>("parishes");
  const [recent, setRecent] = useState<string[]>(() => loadRecent());
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 60000);
    return () => clearInterval(id);
  }, []);

  const searchable = useMemo(
    () =>
      PARISHES.map(p => ({
        id: p.id,
        name: p.name,
        location: p.vicariate ?? "",
        coordinates: p.coordinates,
      })),
```

```
  [],
);

const results = useMemo<{ parish: { id: string; name: string }; distanceMeters: number | null }[]>(() => {
  const hits = query.trim()
    ? searchParishesScored(query, searchable, { origin: position })
    : searchable.map(p => ({ parish: p, score: 0, distanceMeters: null as number | null }));

  // The live parishes first, always.
  //
  // Without this they sat wherever the alphabet or the GPS put them -
  // San Roque Cathedral fell past the fold on an unfiltered list, so the
  // two parishes the app can actually show you were the two hardest to
  // find in it. The sort is stable, so within each group the scoring and
  // the distance ordering are untouched.
  const ranked = [...hits].sort(
    (a, b) => Number(LIVE_IDS.has(b.parish.id)) - Number(LIVE_IDS.has(a.parish.id)),
  );

  if (filter !== "mass-soon") return ranked;

  // "Mass soon" can only mean something for the parishes whose schedule has
  // actually been collected — two of thirty-one today.
  return ranked.filter(h => {
    const routeId = routeIdForParish(h.parish.id);
    const schedule = routeId ? MASS_SCHEDULES[routeId] : undefined;
    if (!schedule?.schedule) return false;
    const upcoming = nextMass(schedule.schedule, now);
    if (!upcoming) return false;
    const hours = (upcoming.date.getTime() - now.getTime()) / 3_600_000;
    return hours >= 0 && hours <= 6;
  });
}, [query, filter, position?.lat, position?.lng, searchable, now]);

const choose = (parishId: string) => {
  setRecent(pushRecent(parishId));
  onSelectParish(parishId);
};

const showRecent = !query.trim() && filter === "parishes" && recent.length > 0;

return (
  <div className="flex-1 flex flex-col bg-[var(--color-brand-card)] min-h-0">
    {/* The inset keeps the field and Cancel clear of the status bar: this
        screen is rendered into a `fixed inset-0` overlay, which starts at
        the very top of the phone. */}
    <div
      className="px-4 pt-5 pb-3 shrink-0 space-y-3"
      style={{ paddingTop: "calc(1.25rem + env(safe-area-inset-top))" }}
    >
      <div className="flex items-center gap-3">
        <div className="flex-1 flex items-center gap-2.5 px-4 py-3 rounded-full bg-[var(--color-brand-card-sunk)] border border-[var(--color-brand-border)]">
          <SearchIcon className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
          <input
            autoFocus
            type="search"
            value={query}
            onChange={e => setQuery(e.target.value)}
            placeholder="Search parishes"
```

```
            aria-label="Search parishes"
            className="flex-1 min-w-0 bg-transparent border-0 outline-none text-[16px] text-[var(--color-brand-text)] placeholder:text-[var(--color-brand-
secondary)]"
          />
        </div>
        <button
          type="button"
          onClick={onClose}
          className="text-[16px] font-semibold text-[var(--color-brand-primary)] shrink-0"
        >
          Cancel
        </button>
      </div>

      <div className="flex gap-2">
        {FILTERS.map(f => (
          <button
            key={f.id}
            type="button"
            onClick={() => setFilter(f.id)}
            aria-pressed={filter === f.id}
            className={`text-[15px] font-semibold px-3.5 py-1.5 rounded-full border ${
              filter === f.id
                ? "bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] border-[var(--color-brand-primary)]"
                : "bg-[var(--color-brand-card)] text-[var(--color-brand-secondary)] border-[var(--color-brand-border)]"
            }`}
          >
            {f.label}
          </button>
        ))}
      </div>
    </div>

    <div className="flex-1 min-h-0 overflow-y-auto px-4 pb-6 space-y-2">
      {showRecent && (
        <>
          <p className="pt-1 pb-1 text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
            Recent
          </p>
          {recent
            .map(id => PARISHES.find(p => p.id === id))
            .filter((p): p is DioceseParish => Boolean(p))
            .map(p => (
              <button
                key={p.id}
                type="button"
                onClick={() => choose(p.id)}
                className="w-full flex items-center gap-3 px-4 py-3 rounded-2xl bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] text-left"
              >
                <Clock className="w-4 h-4 text-[var(--color-brand-secondary)] shrink-0" />
                <span className="text-[16px] text-[var(--color-brand-text)]">{p.name}</span>
              </button>
            ))}
          <p className="pt-4 pb-1 text-[14px] font-mono uppercase tracking-[0.14em] text-[var(--color-brand-secondary)]">
            All parishes
          </p>
        </>
      )}
```

```
        {results.length === 0 && (
          <p className="py-8 text-center text-[16px] leading-relaxed text-[var(--color-brand-secondary)]">
            {filter === "mass-soon"
              ? "No Mass in the next six hours at a parish whose schedule has been collected. Only 2 of 31 have times so far."
              : `No parish matches "${query.trim()}".`}
          </p>
        )}

        {results.map(({ parish, distanceMeters }) => {
          const full = PARISHES.find(p => p.id === parish.id)!;
          return (
            <div key={parish.id}>
            <ParishListRow
              parish={{
                id: parish.id,
                name: parish.name,
                // "(unconfirmed)" did not explain why the row cannot be
                // opened. Saying the details are not published yet does,
                // and it is the truth: the field team has not collected
                // them for these parishes.
                meta: full.status === "live"
                  ? full.vicariate
                  : [full.vicariate, "details coming soon"].filter(Boolean).join(" · "),
                coordinates: full.coordinates,
                isLive: full.status === "live",
              }}
              distanceMeters={distanceMeters}
              onSelect={choose}
            />
            </div>
          );
        })}

        {/* The two filters the design shows that this data cannot answer yet.
            Named rather than silently dropped, so it reads as a known gap
            being worked on and not as a feature that was forgotten. */}
        <p className="pt-5 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          Confession times and non-parish landmarks are not searchable yet — both are still
          being collected by the field team.
        </p>
      </div>
    </div>
  );
}
```

1.2.56 APP-061 — SettingDisclosure.tsx

Code ID	APP-061
File Name	SettingDisclosure.tsx
File Path	SanctDemo/src/components/SettingDisclosure.tsx
Language / Type	TypeScript (React, TSX)
Lines	73
Functions / Classes	SettingDisclosure (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/MeTab.tsx
How to Recognize It	Open SanctDemo/src/components/SettingDisclosure.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SettingDisclosure.tsx

```
import { useId, useState, type ReactNode } from "react";
import { ChevronDown } from "lucide-react";

/**
 * A setting that opens when you ask it to.
 *
 * ## Why these fold
 *
 * Language and Reminders were both printed open on the Me tab: two
 * explanatory paragraphs, a pair of language buttons and seven reminder
 * switches, all unfolded above a heading that said "Settings" and a
 * tidy list of rows underneath. The settings a pilgrim came for were
 * below two panels nobody had asked to see, and the page read as a
 * dumping ground rather than a list.
 *
 * Folded, every row on the tab is the same shape - a label you tap -
 * and what you tapped is the only thing open.
 *
 * ## Why the row carries the heading
 *
 * The panels inside already print their own "LANGUAGE" and "REMINDERS"
 * headings, so a wrapper that repeated them would say everything twice.
 * `bare` drops the row's own chrome for a child that brings its own
 * card, leaving this responsible only for the folding.
 */
export default function SettingDisclosure({
  icon,
  label,
  children,
  bare = false,
  defaultOpen = false,
}: {
  icon: ReactNode;
```

```
  label: string;
  children: ReactNode;
  /** True when the child renders its own card and heading. */
  bare?: boolean;
  defaultOpen?: boolean;
}) {
  const [open, setOpen] = useState(defaultOpen);
  const panelId = useId();

  return (
    <div>
      <button
        type="button"
        onClick={() => setOpen(o => !o)}
        aria-expanded={open}
        aria-controls={panelId}
        className="w-full bg-[var(--color-brand-card-sunk)] p-4 rounded-2xl border border-[var(--color-brand-border)] flex items-center gap-3 hover:border-[var(--
color-brand-primary)] text-left transition-colors text-[16px] font-semibold text-[var(--color-brand-text)]"
      >
        <span className="text-[var(--color-brand-secondary)] shrink-0">{icon}</span>
        <span className="flex-1 truncate">{label}</span>
        <ChevronDown
          aria-hidden
          className={`w-4 h-4 shrink-0 text-[var(--color-brand-secondary)] transition-transform duration-200 ${
            open ? "rotate-180" : ""
          }`}
        />
      </button>

      {/* Unmounted rather than hidden. The reminder panel reads the
          device's notification permission when it mounts, and keeping a
          closed panel alive would have it do that on a tab the pilgrim
          never opened. */}
      {open && (
        <div id={panelId} className={bare ? "mt-2" : "mt-2 px-1"}>
          {children}
        </div>
      )}
    </div>
  );
}
```

1.2.57 APP-062 — SignInFirst.tsx

Code ID	APP-062
File Name	SignInFirst.tsx
File Path	SanctDemo/src/components/SignInFirst.tsx
Language / Type	TypeScript (React, TSX)
Lines	53
Functions / Classes	SignInFirst (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/SignInFirst.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SignInFirst.tsx

```
import { ArrowLeft, LogIn } from "lucide-react";

/**
 * The wall in front of an application for someone who is not signed in.
 *
 * ## Why a screen and not a line of red text
 *
 * The security rules refuse an application that carries no uid, and My
 * Applications has nowhere to show one either. So this is not a
 * validation failure to be reported under a field - it is a thing that
 * has to happen first, and it deserves the same screen the form would
 * have had.
 *
 * It says why, too. "Please sign in" with no reason reads as the app
 * collecting accounts; the real reason is that the parish replies to
 * you, and you need somewhere to read the reply.
 */
export default function SignInFirst({
  what,
  onSignIn,
  onBack,
}: {
  /** The ministry or sacrament they were trying to apply for. */
  what: string;
  onSignIn: () => void;
  onBack: () => void;
}) {
  return (
    <div className="apply-screen">
      <header className="apply-screen__bar">
        <button type="button" onClick={onBack} className="apply-screen__back">
          <ArrowLeft className="w-4 h-4" /> Back
        </button>
```

```
      </header>

      <div className="apply-screen__body">
        <h1 className="apply-screen__title">Sign in to apply</h1>
        <p className="mt-3 text-[16px] leading-relaxed text-[var(--color-brand-text)]">
          Applying for <strong>{what}</strong> needs an account, for two reasons: the
          parish replies to you by email, and your application appears under
          <strong> My Applications</strong> so you can follow what happens to it.
        </p>
        <p className="mt-3 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          It takes a moment, and everything you have read so far stays where it is.
        </p>

        <button type="button" onClick={onSignIn} className="apply-primary mt-6">
          <LogIn className="w-4 h-4" /> Sign in
        </button>
      </div>
    </div>
  );
}
```

1.2.58 APP-063 — SimulatorPanel.tsx

Code ID	APP-063
File Name	SimulatorPanel.tsx
File Path	SanctDemo/src/components/SimulatorPanel.tsx
Language / Type	TypeScript (React, TSX)
Lines	133
Functions / Classes	SimulatorPanel (exported component/function (default))
Related Files (its imports)	../context/PresenceContext
Where It Is Used	src/App.tsx
How to Recognize It	Open SanctDemo/src/components/SimulatorPanel.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SimulatorPanel.tsx

```
import { X, Navigation, MapPin, Satellite, Ruler } from "lucide-react";
import { usePresence, SIMULATIONS } from "../context/PresenceContext";

interface SimulatorPanelProps {
  isOpen: boolean;
  onClose: () => void;
}

// This panel is deliberately reachable from the app's own navigation, not
// hidden behind a debug flag — it is how presence gets demonstrated in a
// defense room where real GPS is nowhere near either parish.
export default function SimulatorPanel({ isOpen, onClose }: SimulatorPanelProps) {
  const { presence, parish, position, accuracyMeters, gpsStatus, simulation, setSimulation } = usePresence();

  if (!isOpen) return null;

  return (
    <div
      className="absolute inset-0 z-[70] flex items-center justify-center p-5 bg-black/60 transition-opacity duration-300 ease-out"
      onClick={onClose}
    >
      {/* Height-safe: the card is capped at the overlay and split into a
          header that stays and a body that scrolls. Without the cap a card
          taller than the screen was centred, so it overflowed equally at
          both ends and carried its own close button off the top - and
          nothing scrolled, so there was no way to reach it or to shut the
          modal except by guessing that the backdrop closes it. It happened
          on any short viewport: a rotated phone, a small one, or a tall
          card with the keyboard up. */}
      <div
        className="w-full max-w-sm max-h-full flex flex-col rounded-2xl border border-[var(--color-brand-on-accent)]/30 bg-[var(--color-brand-primary)] text-white
shadow-2xl transition-all duration-300 ease-out font-secondary"
        onClick={(e) => e.stopPropagation()}
```

```
>
  {/* Header */}
  <div className="shrink-0 flex items-center justify-between px-5 pt-5 pb-3 border-b border-white/10">
    <div className="flex items-center gap-1.5">
      <Navigation className="w-4 h-4 text-[var(--color-brand-on-accent)]" />
      <h2 className="text-base font-bold tracking-tight font-primary">Location Simulator</h2>
    </div>
    <button
      onClick={onClose}
      aria-label="Close location simulator"
      className="flex items-center justify-center w-11 h-11 -mr-2 rounded-full text-white/80 hover:bg-white/10 hover:text-white transition-colors duration-300"
    >
      <X className="w-5 h-5" />
    </button>
  </div>

  <div className="min-h-0 flex-1 overflow-y-auto px-5 py-4 space-y-5">
    {/* Simulation options */}
    <div>
      <div className="flex items-center gap-1.5 mb-2 text-sm font-semibold uppercase tracking-wider text-white/90">
        <MapPin className="w-3.5 h-3.5" />
        <span>Simulated Position</span>
      </div>
      <div role="radiogroup" aria-label="Simulated position" className="space-y-2">
        {SIMULATIONS.map((opt) => (
          <label
            key={opt.value}
            className={`flex items-center gap-2.5 min-h-11 px-3 py-2 rounded-xl border cursor-pointer transition-colors duration-300 ${
              simulation === opt.value
                ? "bg-[var(--color-brand-primary-dark)] border-[var(--color-brand-on-accent)]/80 text-white shadow-[0_0_20px_rgba(24,134,160,0.4)]"
                : "bg-transparent border-white/25 text-white/90 hover:bg-white/5"
            }`}
          >
            <input
              type="radio"
              name="presence-simulation"
              value={opt.value}
              checked={simulation === opt.value}
              onChange={() => setSimulation(opt.value)}
              className="w-4 h-4 accent-[var(--color-brand-on-accent)] shrink-0"
            />
            <span className="text-base font-bold">{opt.label}</span>
          </label>
        ))}
      </div>
    </div>

    {/* Live readout */}
    <div>
      <div className="flex items-center gap-1.5 mb-2 text-sm font-semibold uppercase tracking-wider text-white/90">
        <Satellite className="w-3.5 h-3.5" />
        <span>Live Readout</span>
      </div>
      <div className="rounded-xl border border-white/25 bg-white/5 p-3 space-y-1.5 text-[15px] font-mono">
        <div className="flex justify-between gap-2">
          <span className="text-white/70">Mode</span>
          <span className="font-bold text-white uppercase">{presence.mode}</span>
        </div>
        <div className="flex justify-between gap-2">
```

```
              <span className="text-white/70">Parish</span>
              <span className="font-bold text-white text-right">{parish ? parish.name.replace(" Guide", "").replace(" Tour", "") : "—"}</span>
            </div>
            <div className="flex justify-between gap-2">
              <span className="text-white/70 flex items-center gap-1"><Ruler className="w-3 h-3" /> Distance</span>
              <span className="font-bold text-white">
                {presence.distance != null ? `${Math.round(presence.distance)} m` : "—"}
              </span>
            </div>
            <div className="flex justify-between gap-2">
              <span className="text-white/70">GPS Status</span>
              <span className="font-bold text-white uppercase">{gpsStatus}</span>
            </div>
            <div className="flex justify-between gap-2">
              <span className="text-white/70">Position</span>
              <span className="font-bold text-white text-right">
                {position ? `${position.lat.toFixed(4)}, ${position.lng.toFixed(4)}` : "—"}
              </span>
            </div>
            <div className="flex justify-between gap-2">
              <span className="text-white/70">Accuracy</span>
              <span className="font-bold text-white text-right">
                {simulation !== "off"
                  ? "Simulated — not a real fix"
                  : accuracyMeters != null
                    ? `±${Math.round(accuracyMeters)} m`
                    : "—"}
              </span>
            </div>
          </div>
        </div>

        <p className="text-sm text-white/70 leading-snug">
          Real GPS is used automatically whenever simulation is "Off". This panel exists for defense-room
          demonstrations where the diocese's parishes are out of reach. A simulated position is clearly
          labelled as such wherever it appears on screen — never shown as a real GPS fix.
        </p>
      </div>
    </div>
  </div>
  );
}
```

1.2.59 APP-064 — Spotlight.tsx

Code ID	APP-064
File Name	Spotlight.tsx
File Path	SanctDemo/src/components/Spotlight.tsx
Language / Type	TypeScript (React, TSX)
Lines	269
Functions / Classes	SpotlightCard (exported type); useSpotlight (exported function); SPOTLIGHT_ATTR (exported constant); appShell (internal function); findTarget (internal function); shellViewport (internal function); SpotlightProvider (exported function); Overlay (internal function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/RootGate.tsx, src/components/SanctiHost.tsx, src/components/TutorialRunner.tsx, src/main.tsx
How to Recognize It	Open SanctDemo/src/components/Spotlight.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — Spotlight.tsx

```
import {
  createContext, useCallback, useContext, useEffect, useLayoutEffect,
  useRef, useState, type ReactNode,
} from "react";
import { createPortal } from "react-dom";
import {
  placeSpotlight, isUsableTarget, type Placement, type Rect,
} from "../lib/spotlight";

/**
 * Putting a hole in the screen around one thing and explaining it.
 *
 * Used by two features that would otherwise each build their own: the
 * first-run tutorial walks its steps through this, and Sancti uses it
 * to point at whatever a pilgrim just asked about. One implementation
 * means one set of edge cases, and the tutorial and the assistant
 * cannot drift into looking like different apps.
 *
 * ## Why targets are found by attribute, not by ref
 *
 * A ref would have to be threaded from the tab bar, the map and half a
 * dozen screens up to a provider at the root, and every new target
 * would mean touching every component in between. A component opts in
 * by putting `data-spotlight="map-tab"` on itself and nothing else
 * changes.
 *
 * The cost is that a mistyped name fails silently, so `show()` reports
 * whether it found anything and the tutorial uses that to skip a step
```

```
 * whose target is not on screen rather than freezing on it.
 */

export interface SpotlightCard {
  title?: string;
  body: string;
  /** Rendered under the body: Next / Back / Skip, or nothing. */
  footer?: ReactNode;
  /** Progress dots, as [current, total]. Omitted for a one-off. */
  progress?: [number, number];
}

interface SpotlightRequest {
  target: string;
  card?: SpotlightCard;
  /** Fades itself out after this long. Omitted means it waits. */
  autoHideMs?: number;
  /** Tapping the dimmed area dismisses. Off during the tutorial. */
  dismissOnTapOutside?: boolean;
  onDismiss?: () => void;
}

interface SpotlightApi {
  /** Returns false when nothing on screen carries that name. */
  show: (request: SpotlightRequest) => boolean;
  hide: () => void;
  /** True while something is being highlighted. */
  active: boolean;
}

const Ctx = createContext<SpotlightApi | null>(null);

export function useSpotlight(): SpotlightApi {
  const api = useContext(Ctx);
  if (!api) throw new Error("useSpotlight must be used inside <SpotlightProvider>");
  return api;
}

/** Put this on anything the tutorial or Sancti should be able to point at. */
export const SPOTLIGHT_ATTR = "data-spotlight";

/**
 * The box the overlay should cover and measure against.
 *
 * On a phone that is the window, and this returns it. In the browser
 * preview the app runs inside a phone mock a few hundred pixels wide,
 * and measuring the window there puts the dimming over the whole
 * laptop screen and lets the card hang off the side of the mock. The
 * shell is tagged in every one of PhoneContainer's layouts, so
 * whichever is rendering, this finds it.
 */
function appShell(): HTMLElement | null {
  if (typeof document === "undefined") return null;
  return document.querySelector<HTMLElement>("[data-app-shell]");
}

function findTarget(name: string): Rect | null {
  if (typeof document === "undefined") return null;
  const el = document.querySelector(`[${SPOTLIGHT_ATTR}="${CSS.escape(name)}"]`);
```

```
  if (!el) return null;
  const r = el.getBoundingClientRect();
  // Relative to the shell, because that is the box the overlay is
  // positioned inside. On a phone the shell starts at 0,0 and this is
  // the same number either way.
  const host = appShell()?.getBoundingClientRect();
  return {
    top: r.top - (host?.top ?? 0),
    left: r.left - (host?.left ?? 0),
    width: r.width,
    height: r.height,
  };
}

function shellViewport(): { width: number; height: number } {
  const host = appShell()?.getBoundingClientRect();
  if (!host || host.width < 1) {
    return { width: window.innerWidth, height: window.innerHeight };
  }
  return { width: host.width, height: host.height };
}

export function SpotlightProvider({ children }: { children: ReactNode }) {
  const [request, setRequest] = useState<SpotlightRequest | null>(null);
  const [placement, setPlacement] = useState<Placement | null>(null);
  const cardRef = useRef<HTMLDivElement | null>(null);

  const hide = useCallback(() => {
    setRequest(null);
    setPlacement(null);
  }, []);

  const show = useCallback((next: SpotlightRequest) => {
    if (!isUsableTarget(findTarget(next.target))) return false;
    setRequest(next);
    return true;
  }, []);

  // Measured after paint, not during: the card's height depends on how
  // much text it holds, and the placement depends on that height.
  useLayoutEffect(() => {
    if (!request) return;

    const reposition = () => {
      const target = findTarget(request.target);
      if (!isUsableTarget(target)) {
        // Whatever it was pointing at has gone - a tab changed under it.
        // Better to stop than to keep a hole over nothing.
        hide();
        return;
      }
      const cardHeight = cardRef.current?.offsetHeight ?? 150;
      setPlacement(placeSpotlight(target!, shellViewport(), cardHeight));
    };

    reposition();

    // The tab bar moves when the keyboard opens and the map's controls
    // move when it is rotated; a hole left behind is worse than none.
```

```
    window.addEventListener("resize", reposition);
    window.addEventListener("scroll", reposition, true);
    return () => {
      window.removeEventListener("resize", reposition);
      window.removeEventListener("scroll", reposition, true);
    };
  }, [request, hide]);

  // Re-measure once the card's real height is known.
  useEffect(() => {
    if (!request || !cardRef.current) return;
    const observer = new ResizeObserver(() => {
      const target = findTarget(request.target);
      if (!isUsableTarget(target)) return;
      setPlacement(
        placeSpotlight(target!, shellViewport(), cardRef.current?.offsetHeight ?? 150),
      );
    });
    observer.observe(cardRef.current);
    return () => observer.disconnect();
  }, [request, placement !== null]);

  useEffect(() => {
    if (!request?.autoHideMs) return;
    const timer = window.setTimeout(() => {
      request.onDismiss?.();
      hide();
    }, request.autoHideMs);
    return () => window.clearTimeout(timer);
  }, [request, hide]);

  // Escape closes anything that a tap outside would close.
  useEffect(() => {
    if (!request?.dismissOnTapOutside) return;
    const onKey = (e: KeyboardEvent) => {
      if (e.key === "Escape") { request.onDismiss?.(); hide(); }
    };
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown", onKey);
  }, [request, hide]);

  const api: SpotlightApi = { show, hide, active: request !== null };

  return (
    <Ctx.Provider value={api}>
      {children}
      {request && placement && typeof document !== "undefined" &&
        createPortal(
          <Overlay
            request={request}
            placement={placement}
            cardRef={cardRef}
            onDismiss={() => { request.onDismiss?.(); hide(); }}
          />,
          appShell() ?? document.body,
        )}
    </Ctx.Provider>
  );
}
```

```
function Overlay({
  request, placement, cardRef, onDismiss,
}: {
  request: SpotlightRequest;
  placement: Placement;
  cardRef: React.MutableRefObject<HTMLDivElement | null>;
  onDismiss: () => void;
}) {
  const { hole, card, arrow, arrowOffset } = placement;

  return (
    <div
      className="spot"
      role="dialog"
      aria-modal="true"
      aria-label={request.card?.title ?? "Guide"}
    >
      {/*
        The dimming and the hole are one element. A giant box-shadow
        spreading outwards from the hole darkens everything around it in
        a single paint, which is both cheaper than four panels around
        the target and immune to the hairline seams four panels leave
        between them on fractional pixel positions.
      */}
      <div
        className="spot__hole"
        style={{ top: hole.top, left: hole.left, width: hole.width, height: hole.height }}
        onClick={request.dismissOnTapOutside ? onDismiss : undefined}
      />

      {request.card && (
        <div
          ref={cardRef}
          className={`spot__card spot__card--arrow-${arrow}`}
          style={{ top: card.top, left: card.left, width: card.width }}
        >
          {arrow !== "none" && (
            <span className="spot__arrow" style={{ left: arrowOffset }} aria-hidden />
          )}

          {request.card.title && <h2 className="spot__title">{request.card.title}</h2>}
          <p className="spot__body">{request.card.body}</p>

          {(request.card.progress || request.card.footer) && (
            <div className="spot__foot">
              {request.card.progress && (
                <span className="spot__dots" aria-hidden>
                  {Array.from({ length: request.card.progress[1] }, (_, i) => (
                    <span
                      key={i}
                      className={`spot__dot${i === request.card!.progress![0] ? " is-on" : ""}`}
                    />
                  ))}
                </span>
              )}
              {request.card.footer}
            </div>
          )}
```

```
        </div>
      )}
    </div>
  );
}
```

1.2.60 APP-065 — TripPlanner.tsx

Code ID	APP-065
File Name	TripPlanner.tsx
File Path	SanctDemo/src/components/TripPlanner.tsx
Language / Type	TypeScript (React, TSX)
Lines	160
Functions / Classes	TripPlanner (exported component/function (default))
Related Files (its imports)	../lib/routing
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/components/TripPlanner.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — TripPlanner.tsx

```
import { ArrowLeft, Plus, Sparkles, Trash2, X } from "lucide-react";
import {
  MAX_TRIP_STOPS,
  formatTripDuration,
  stopLabel,
  type TripRoute,
  type TripStop,
} from "../lib/pilgrimage";
import { formatDistance } from "../lib/routing";

interface TripPlannerProps {
  stops: TripStop[];
  /** The measured trip, or null before the first measurement returns. */
  trip: TripRoute | null;
  measuring: boolean;
  /** False when the order could still be shortened, which is what enables Nearest first. */
  optimal: boolean;
  /** No GPS fix: the plan still works, but nothing can be ordered or measured from here. */
  hasPosition: boolean;
  onAddStop: () => void;
  onRemoveStop: (id: string) => void;
  onOptimise: () => void;
  onClear: () => void;
  onClose: () => void;
}

/**
 * Planning a visit to several churches in one journey.
 *
 * The tradition is seven churches on Maundy Thursday, and the hard part has
 * never been finding them; it is deciding the order. A group standing in
 * Maypajo with a list of seven parishes cannot tell which to walk to first,
 * and getting it wrong is the difference between a six-kilometre evening and
```

```
 * an eleven-kilometre one.
 *
 * So the planner does the one thing a map can do that a list cannot: it
 * works out the order. "Nearest first" is offered rather than applied, and
 * only when it would actually shorten the walk — a parish may want to reach
 * a particular church at a particular hour, and a planner that silently
 * reshuffled their evening would be answering a question nobody asked.
 *
 * The layout is a maps app's: the stops in order down the left with their
 * letters, each removable, an Add stop row at the bottom, and the total
 * across the footer.
 */
export default function TripPlanner({
  stops,
  trip,
  measuring,
  optimal,
  hasPosition,
  onAddStop,
  onRemoveStop,
  onOptimise,
  onClear,
  onClose,
}: TripPlannerProps) {
  const full = stops.length >= MAX_TRIP_STOPS;

  return (
    <div className="trip" role="dialog" aria-label="Plan a church visit">
      <div className="trip__head">
        <button type="button" className="trip__icon" onClick={onClose} aria-label="Back to the map">
          <ArrowLeft className="w-5 h-5" />
        </button>
        <div className="trip__heading">
          <p className="trip__title">Add church</p>
          <p className="trip__sub">
            {stops.length === 0
              ? "Add the churches you want to visit"
              : `${stops.length} church${stops.length === 1 ? "" : "es"}`}
          </p>
        </div>
        {stops.length > 0 && (
          <button type="button" className="trip__icon" onClick={onClear} aria-label="Clear this visit">
            <Trash2 className="w-5 h-5" />
          </button>
        )}
      </div>

      <ol className="trip__stops">
        {/* Where the pilgrim is, as the first row. It is not a stop and
            cannot be removed, but leaving it out made the list read as if
            the walk began at the first church rather than at their feet. */}
        <li className="trip__stop trip__stop--start">
          <span className="trip__pip" aria-hidden />
          <span className="trip__name">{hasPosition ? "Your location" : "Location off"}</span>
        </li>

        {stops.map((stop, index) => (
          <li key={stop.id} className="trip__stop">
            <span className="trip__letter" aria-hidden>
```

```
        {stopLabel(index)}
      </span>
      <span className="trip__name">{stop.name}</span>
      <button
        type="button"
        className="trip__remove"
        onClick={() => onRemoveStop(stop.id)}
        aria-label={`Remove ${stop.name} from this visit`}
      >
        <X className="w-4 h-4" />
      </button>
    </li>
  ))}

  <li className="trip__stop trip__stop--add">
    <span className="trip__letter trip__letter--ghost" aria-hidden>
      <Plus className="w-4 h-4" />
    </span>
    <button type="button" className="trip__add" onClick={onAddStop} disabled={full}>
      {full ? `That is ${MAX_TRIP_STOPS} churches — enough for one day` : "Add a church"}
    </button>
  </li>
</ol>

{/* Offered only when it would change something. A button that promises
    a shorter route and then reorders nothing is worse than no button. */}
{!optimal && hasPosition && (
  <button type="button" className="trip__optimise" onClick={onOptimise}>
    <Sparkles className="w-4 h-4" />
    Put them in the nearest order
  </button>
)}

<div className="trip__foot">
  <div className="trip__total">
    {measuring ? (
      <span className="trip__measuring">Measuring the walk…</span>
    ) : /* A measured trip of zero metres is not a walk, it is the
           absence of one: one church with no known starting point has
           nothing between it and anywhere. Printing "Total trip: 0 min"
           for that reads as an answer when it is the lack of one. */
    trip && trip.distanceMeters > 0 ? (
      <>
        <span className="trip__time">Total trip: {formatTripDuration(trip.durationMinutes)}</span>
        <span className="trip__distance">
          {formatDistance(trip.distanceMeters)}
          {/* Said plainly rather than quietly rolled into the total.
              A leg the router could not solve is a straight line
              across the map, which is always shorter than the street
              it stands in for. */}
          {trip.hasDirectLeg && " · some legs straight-line only"}
        </span>
      </>
    ) : (
      <span className="trip__measuring">
        {hasPosition
          ? "Add churches to see the walk"
          : "Turn on location to measure the walk"}
      </span>
```

```
          )}
        </div>
        <button type="button" className="trip__done" onClick={onClose}>
          Done
        </button>
      </div>
    </div>
  );
}
```

1.2.61 APP-066 — TutorialRunner.tsx

Code ID	APP-066
File Name	TutorialRunner.tsx
File Path	SanctDemo/src/components/TutorialRunner.tsx
Language / Type	TypeScript (React, TSX)
Lines	157
Functions / Classes	TutorialRunner (exported component/function (default))
Related Files (its imports)	./Spotlight
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/TutorialRunner.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — TutorialRunner.tsx

```
import { useCallback, useEffect, useRef, useState } from "react";
import { useSpotlight, SPOTLIGHT_ATTR } from "./Spotlight";
import {
  TUTORIAL_STEPS, markTutorialSeen, nextUsableStep,
  type TutorialTab,
} from "../lib/tutorial";

/**
 * Walks the pilgrim through the app once.
 *
 * ## Why this drives the spotlight rather than drawing its own
 *
 * Sancti points things out with the same hole and the same card. One
 * implementation means the assistant saying "the Map is here" looks
 * like the app that showed you round on your first morning, instead of
 * like a second feature bolted on beside the first.
 *
 * ## Why each step waits before measuring
 *
 * A step can ask for a different tab, and the target does not exist
 * until that tab has rendered. Measuring in the same tick finds
 * nothing and the step gets skipped as missing. One frame of delay is
 * the difference between a tutorial that works and one that silently
 * drops half its steps.
 */
export default function TutorialRunner({
  open,
  onClose,
  onSwitchTab,
}: {
  open: boolean;
  onClose: () => void;
  onSwitchTab: (tab: TutorialTab) => void;
```

```
}) {
  const spotlight = useSpotlight();
  const [index, setIndex] = useState(0);
  const [finished, setFinished] = useState(false);

  // So move() can read where it is without putting a side effect in a
  // state updater. See move() below.
  const indexRef = useRef(0);
  indexRef.current = index;

  // Held in a ref so the effect that shows a step does not re-run every
  // time one of these functions is re-created.
  const api = useRef({ spotlight, onSwitchTab, onClose });
  api.current = { spotlight, onSwitchTab, onClose };

  const present = useCallback(
    (target: string) =>
      !!document.querySelector(`[${SPOTLIGHT_ATTR}="${CSS.escape(target)}"]`),
    [],
  );

  const finish = useCallback(() => {
    markTutorialSeen();
    api.current.spotlight.hide();
    setFinished(true);
  }, []);

  const close = useCallback(() => {
    markTutorialSeen();
    api.current.spotlight.hide();
    setFinished(false);
    setIndex(0);
    api.current.onClose();
  }, []);

  /**
   * Step forward or back, skipping anything not on screen.
   *
   * The index is read from a ref rather than from inside a setState
   * updater. It was in the updater, and that was a real bug: finishing
   * the tour is a side effect, React calls an updater twice under
   * StrictMode, and the tour marked itself seen and vanished the
   * instant it opened. Updaters have to be pure.
   */
  const move = useCallback((direction: 1 | -1) => {
    const candidate = indexRef.current + direction;
    if (candidate < 0) return;
    if (candidate >= TUTORIAL_STEPS.length) { finish(); return; }

    const found = nextUsableStep(candidate, direction, present);
    if (found === null) {
      // Nothing usable left that way. Forward means we are done; back
      // means stay put rather than closing the tour behind them.
      if (direction === 1) finish();
      return;
    }
    setIndex(found);
  }, [finish, present]);
```

```
useEffect(() => {
  if (!open) { setIndex(0); setFinished(false); }
}, [open]);

useEffect(() => {
  if (!open || finished) return;
  const step = TUTORIAL_STEPS[index];
  if (!step) return;

  if (step.tab) api.current.onSwitchTab(step.tab);

  // Two frames: one for the tab switch to commit, one for layout.
  let raf2 = 0;
  const raf1 = requestAnimationFrame(() => {
    raf2 = requestAnimationFrame(() => {
      const shown = api.current.spotlight.show({
        target: step.target,
        card: {
          title: step.title,
          body: step.body,
          progress: [index, TUTORIAL_STEPS.length],
          footer: (
            <>
              <button type="button" className="spot__btn--quiet" onClick={close}>
                Skip
              </button>
              {index > 0 && (
                <button type="button" className="spot__btn--quiet" onClick={() => move(-1)}>
                  Back
                </button>
              )}
              <button type="button" className="spot__btn" onClick={() => move(1)}>
                {index === TUTORIAL_STEPS.length - 1 ? "Finish" : "Next"}
              </button>
            </>
          ),
        },
      });

      // The target is not on this screen after all. Skip rather than
      // sit on a step that cannot draw.
      if (!shown) move(1);
    });
  });

  return () => { cancelAnimationFrame(raf1); cancelAnimationFrame(raf2); };
}, [open, finished, index, close, move]);

if (!open || !finished) return null;

return (
  <div className="tut-done" role="dialog" aria-modal="true" aria-label="Tutorial finished">
    <div className="tut-done__card">
      <h2 className="tut-done__title">You&rsquo;re ready to explore SanctiWalk</h2>
      <p className="tut-done__body">
        Ask Sancti whenever you are not sure where something is. You can run
        this tour again from <strong>Me</strong> at any time.
      </p>
      <button type="button" className="tut-done__go" onClick={close}>
```

```
          Get started
        </button>
      </div>
    </div>
  );
}
```

1.2.62 APP-067 — UpdateBanner.tsx

Code ID	APP-067
File Name	UpdateBanner.tsx
File Path	SanctDemo/src/components/UpdateBanner.tsx
Language / Type	TypeScript (React, TSX)
Lines	105
Functions / Classes	UpdateBanner (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/components/UpdateBanner.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — UpdateBanner.tsx

```
import { useEffect, useState } from "react";
import { Download, X } from "lucide-react";
import { Capacitor } from "@capacitor/core";

import {
  dismiss,
  fetchLatest,
  isNewer,
  wasDismissed,
  type LatestRelease,
} from "../lib/appUpdate";

/** Where the published release is described. Written at build time. */
const LATEST_URL = "https://sanct-demo-vxzb.vercel.app/download/latest.json";

/**
 * Tells an installed SanctiWalk that a newer one is available.
 *
 * ## Only in the installed app
 *
 * In a browser there is nothing to update - the page IS the latest
 * version, and the service worker has already seen to it. Showing this
 * on the web would be offering somebody a download of the thing they
 * are currently looking at.
 *
 * ## Why it does not install the update itself
 *
 * Tapping Update opens the APK's own URL, which hands the download to
 * Android. The alternative - fetching 66MB inside the app and firing an
 * install intent through a FileProvider - needs native code, a
 * permission most users will decline, and careful handling of a
 * half-finished download. Android's own downloader already does all of
 * that correctly, and the pilgrim then taps the finished file once.
```

```
 *
 * Because the app is signed with a stable release key, that install
 * goes over the top: the saved parish, reminders and subscriptions all
 * survive. That was not true while it was signed with a debug key.
 */
export default function UpdateBanner() {
  const [release, setRelease] = useState<LatestRelease | null>(null);

  useEffect(() => {
    // The browser updates itself; see the note above.
    let installed = false;
    try {
      installed = Capacitor.isNativePlatform();
    } catch {
      installed = false;
    }
    if (!installed) return;

    let live = true;

    void (async () => {
      const latest = await fetchLatest(LATEST_URL);
      if (!live || !latest) return;
      if (!isNewer(__APP_VERSION__, latest)) return;
      // Asked once per release, not once per launch.
      if (wasDismissed(latest.versionName)) return;
      setRelease(latest);
    })();

    return () => {
      live = false;
    };
  }, []);

  if (!release) return null;

  return (
    <div className="update-banner" role="status">
      <Download className="w-[18px] h-[18px] shrink-0" aria-hidden />

      <span className="update-banner__text">
        <span className="update-banner__title">
          SanctiWalk {release.versionName} is available
        </span>
        {release.notes && (
          <span className="update-banner__notes">{release.notes}</span>
        )}
      </span>

      <a
        className="update-banner__go"
        href={release.url}
        // Android takes over the download from here.
        download
      >
        Update
      </a>

      <button
```

```
        type="button"
        className="update-banner__close"
        aria-label="Not now"
        onClick={() => {
          dismiss(release.versionName);
          setRelease(null);
        }}
      >
        <X className="w-4 h-4" aria-hidden />
      </button>
    </div>
  );
}
```

1.2.63 APP-068 — VerseCard.tsx

Code ID	APP-068
File Name	VerseCard.tsx
File Path	SanctDemo/src/components/VerseCard.tsx
Language / Type	TypeScript (React, TSX)
Lines	54
Functions / Classes	VerseCard (exported component/function (default))
Related Files (its imports)	../lib/liturgical, ../lib/verses, ../lib/liturgicalColours
Where It Is Used	src/components/Dashboard.tsx
How to Recognize It	Open SanctDemo/src/components/VerseCard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — VerseCard.tsx

```
import { liturgicalDay } from "../lib/liturgical";
import { verseForDate } from "../lib/verses";
import { seasonAccent } from "../lib/liturgicalColours";

/**
 * Verse of the Day, in the colour of the season.
 *
 * ## Why a tinted card and not a photograph
 *
 * It was set over the parish's own interiors for a while. The client's
 * verdict was plain - the backdrop was ugly - and they are right about
 * what it cost: a photograph behind text needs a scrim, the scrim dulls
 * the photograph, and the liturgical colour that is the whole point of
 * the card gets reduced to a 3px rule along the top edge. Flat, the
 * season IS the card: green through Ordinary Time, violet in Advent and
 * Lent, gold at Christmas and Easter, red at Pentecost.
 *
 * The three values come from liturgicalColours.ts, which picks an ink
 * dark enough to clear WCAG AA on its own wash - so this stays readable
 * in every season without a per-season override here.
 */
export default function VerseCard({ className = "" }: { className?: string }) {
  const now = new Date();
  const today = liturgicalDay(now);
  const verse = verseForDate(now);
  const accent = seasonAccent(today.colour);

  return (
    <article
      className={`verse-card ${className}`}
      style={{
        ["--verse-tint" as string]: accent.tint,
        ["--verse-border" as string]: accent.border,
```

```
        ["--verse-ink" as string]: accent.ink,
      }}
    >
      <div className="verse-card__body">
        <h4 className="verse-card__eyebrow">Verse of the Day</h4>

        <blockquote className="verse-card__text">
          &ldquo;{verse.text}&rdquo;
        </blockquote>

        <div className="verse-card__foot">
          <cite className="verse-card__ref">{verse.reference}</cite>
          {/* "Ordinary Time" and the other seasons stay on the card. It is
              the one word saying which season's prayers these are, and the
              brief is explicit that it must not disappear. */}
          <span className="verse-card__season">{verse.season}</span>
        </div>
      </div>
    </article>
  );
}
```

1.2.64 APP-069 — WatchBell.tsx

Code ID	APP-069
File Name	WatchBell.tsx
File Path	SanctDemo/src/components/WatchBell.tsx
Language / Type	TypeScript (React, TSX)
Lines	93
Functions / Classes	WatchBell (exported component/function (default))
Related Files (its imports)	../lib/deviceNotifications, ../lib/openingSubscriptions
Where It Is Used	src/components/MassScheduleCard.tsx, src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/components/WatchBell.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — WatchBell.tsx

```
import { useCallback, useEffect, useState } from "react";
import { Bell, BellRing } from "lucide-react";

import { permissionState, requestPermission } from "../lib/deviceNotifications";
import { isWatched, onWatchChange, setWatched, type Watch } from "../lib/openingSubscriptions";

/**
 * The bell beside something that is closed.
 *
 * Tapping it is the pilgrim saying "tell me when this changes", and it
 * is the only place in the app that asks for notification permission.
 * That is on purpose. Asking on first launch, alongside location, gets
 * a reflexive no from somebody who has not yet seen anything worth
 * being told about; asking here, one tap after they have said they want
 * to be told, is the same question with its answer already given.
 *
 * It stays useful when the answer is no. The watch is stored either
 * way, and lib/useOpeningNotices still shows the change in the app -
 * what permission buys is being told without opening it.
 */
export default function WatchBell({
  watch,
  label,
  className = "",
  iconOnly = false,
}: {
  watch: Watch;
  /** What is being watched, for the screen reader: "Confirmation". */
  label: string;
  className?: string;
  /** The bell on its own, with no words beside it. */
```

```
  iconOnly?: boolean;
}) {
  const [on, setOn] = useState(() => isWatched(watch));
  const [note, setNote] = useState<string | null>(null);

  // Another bell on the same thing may have been tapped; see
  // onWatchChange in openingSubscriptions.
  useEffect(() => onWatchChange(() => setOn(isWatched(watch))), [watch.kind, watch.parishId, watch.itemId]);

  const toggle = useCallback(async () => {
    const next = !isWatched(watch);
    setWatched(watch, next);
    setOn(next);

    if (!next) {
      setNote(null);
      return;
    }

    // Only now, and only the first time.
    const state = await permissionState();
    const granted = state === "granted" ? state : await requestPermission();

    setNote(
      granted === "granted"
        ? "You'll be notified."
        : // Not an error, and not a nag to go to Settings. The watch is
          // kept and the notice will be waiting in the app.
          "Saved. You'll see it here next time you open the app.",
    );
  }, [watch.kind, watch.parishId, watch.itemId]);

  return (
    <span className="watch-bell__wrap">
      <button
        type="button"
        onClick={toggle}
        aria-pressed={on}
        aria-label={on ? `Stop reminders about ${label}` : `Remind me when ${label} opens`}
        className={`watch-bell${on ? " is-on" : ""}${iconOnly ? " watch-bell--icon" : ""} ${className}`.trim()}
      >
        {on ? (
          <BellRing className="w-[18px] h-[18px]" aria-hidden />
        ) : (
          <Bell className="w-[18px] h-[18px]" aria-hidden />
        )}
        {/* The aria-label carries the whole meaning when the words are
            gone, which is why it names the thing rather than the icon. */}
        {!iconOnly && (
          <span className="watch-bell__text">{on ? "Reminding you" : "Notify me"}</span>
        )}
      </button>
      {/* The bell alone has no room beside it for a sentence; its own
          filled state is the acknowledgement there. */}
      {note && !iconOnly && (
        <span className="watch-bell__note" role="status">
          {note}
        </span>
      )}
```

```
    </span>
  );
}
```

1.3 Application State and Context

1.3.1 APP-070 — PresenceContext.tsx

Code ID	APP-070
File Name	PresenceContext.tsx
File Path	SanctDemo/src/context/PresenceContext.tsx
Language / Type	TypeScript (React, TSX)
Lines	213
Functions / Classes	SimulationValue (exported type); GpsStatus (exported type); SIMULATIONS (exported constant); toGeoParish (internal function); buildParishes (internal function); simulatedPosition (internal function); PresenceProvider (exported function); usePresence (exported function)
Related Files (its imports)	../lib/presence, ../lib/geo, ../data, ../types
Where It Is Used	src/App.tsx, src/components/ArTour.tsx, src/components/DioceseMap.tsx, src/components/DioceseMapLive.tsx, src/components/HomeHero.tsx, src/components/NavigationOverlay.tsx, src/components/Onboarding.tsx, src/components/ParishCard.tsx, src/components/ParishListRow.tsx, src/components/PresenceBanner.tsx, src/components/PresenceSheet.tsx, src/components/SearchScreen.tsx, src/components/SimulatorPanel.tsx
How to Recognize It	Open SanctDemo/src/context/PresenceContext.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PresenceContext.tsx

```
import { createContext, useContext, useEffect, useMemo, useRef, useState, type ReactNode } from 'react'
import { INITIAL_PRESENCE, nextPresence, FORCE_TRANSITION_AT, type GeoParish, type PresenceState } from '../lib/presence'
import type { Coordinates } from '../lib/geo'
import { ROUTES } from '../data'
import type { Route } from '../types'

export type SimulationValue = 'off' | 'approaching_mhcp' | 'at_mhcp' | 'at_src'
export type GpsStatus =
  | 'idle'
  | 'granted'
  | 'denied' // the user refused, or the OS blocked it - not recoverable here
  | 'unavailable' // no geolocation API at all
  | 'searching' // a fix has not arrived yet, or one timed out; still trying
  | 'lost' // had a fix, then the device stopped producing one

export const SIMULATIONS: { value: SimulationValue; label: string }[] = [
  { value: 'off', label: 'Off — use real GPS' },
  { value: 'approaching_mhcp', label: 'Approaching Mary Help of Christians' },
  { value: 'at_mhcp', label: 'At Mary Help of Christians' },
```

```
  { value: 'at_src', label: 'At San Roque Cathedral' },
]

interface PresenceContextValue {
  presence: PresenceState
  parish: Route | null
  position: Coordinates | null
  // The browser's own reported accuracy radius for `position`, in metres —
  // straight from GeolocationCoordinates.accuracy. Null whenever the
  // position isn't a real GPS fix (no position yet, or the simulator is
  // engaged and the position is fabricated) — a simulated position has no
  // honest accuracy figure to report, so it must not display one. On
  // desktop, wifi-based geolocation is routinely accurate only to within a
  // few hundred metres; there is no code fix for that, only surfacing it
  // instead of asserting a precise dot.
  accuracyMeters: number | null
  gpsStatus: GpsStatus
  simulation: SimulationValue
  setSimulation: (value: SimulationValue) => void
}

const PresenceCtx = createContext<PresenceContextValue | null>(null)

// Route's geography fields are optional because CompanionTab builds
// AI-generated custom routes with no location. GeoParish requires them, so
// this conversion is the only place a parish can silently lose its
// coordinates — it must be strict. A route missing coordinates is skipped,
// never defaulted to {lat: 0, lng: 0}, which sits in the Atlantic Ocean and
// would quietly make the parish unreachable rather than obviously absent.
function toGeoParish(route: Route): GeoParish | null {
  if (!route.coordinates || route.geofenceRadius == null || !route.status) return null
  return {
    id: route.id,
    coordinates: route.coordinates,
    geofenceRadius: route.geofenceRadius,
    status: route.status,
  }
}

function buildParishes(routes: Route[]): GeoParish[] {
  const parishes: GeoParish[] = []
  for (const route of routes) {
    const parish = toGeoParish(route)
    if (!parish) {
      console.warn(`PresenceContext: skipping route "${route.id}" — missing coordinates/geofenceRadius/status`)
      continue
    }
    parishes.push(parish)
  }
  return parishes
}

// The simulator bypasses the dwell debounce so a demo responds instantly.
function simulatedPosition(simulation: SimulationValue, parishes: GeoParish[]): Coordinates | null {
  if (simulation === 'off') return null

  const mhcp = parishes.find(p => p.id === 'route-mhcp')
  const src = parishes.find(p => p.id === 'route-src')
```

```
  if (simulation === 'approaching_mhcp') {
    if (!mhcp) return null
    return { lat: mhcp.coordinates.lat + 0.0027, lng: mhcp.coordinates.lng }
  }
  if (simulation === 'at_mhcp') return mhcp ? mhcp.coordinates : null
  if (simulation === 'at_src') return src ? src.coordinates : null
  return null
}

export function PresenceProvider({ children }: { children: ReactNode }) {
  const parishes = useMemo(() => buildParishes(ROUTES), [])

  const [presence, setPresence] = useState<PresenceState>(INITIAL_PRESENCE)
  const [gpsStatus, setGpsStatus] = useState<GpsStatus>('idle')
  const [simulation, setSimulation] = useState<SimulationValue>('off')
  const [position, setPosition] = useState<Coordinates | null>(null)
  const [accuracyMeters, setAccuracyMeters] = useState<number | null>(null)
  const presenceRef = useRef<PresenceState>(INITIAL_PRESENCE)

  function applyPosition(pos: Coordinates | null, options: { instant?: boolean } = {}) {
    const { instant = false } = options
    // FORCE_TRANSITION_AT is presence.ts's documented "skip the dwell gate"
    // sentinel — a simulator switch must respond instantly, not in 10s.
    const prev: PresenceState = instant
      ? { ...presenceRef.current, changedAt: FORCE_TRANSITION_AT }
      : presenceRef.current
    const next = nextPresence(prev, pos, Date.now(), parishes)
    presenceRef.current = next
    setPresence(next)
    setPosition(pos ? { lat: pos.lat, lng: pos.lng } : null)
  }

  // Simulated location wins over real GPS whenever it is switched on.
  // Switching it off means "we no longer know where the user is" until a
  // real position arrives, so it applies a null position instantly instead
  // of leaving the last simulated presence frozen forever.
  useEffect(() => {
    if (simulation === 'off') {
      applyPosition(null, { instant: true })
      return
    }
    // A simulated position is fabricated, not measured — it never carries
    // an honest accuracy figure, so any previous real-GPS accuracy reading
    // must not linger and be shown alongside it.
    setAccuracyMeters(null)
    applyPosition(simulatedPosition(simulation, parishes), { instant: true })
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [simulation])

  // Real GPS, only while the simulator is off. Real GPS always takes
  // precedence over the simulator whenever the simulator is off — this
  // effect and the one above are mutually exclusive on `simulation`, so a
  // real fix can never be silently overridden by a stale simulated one.
  useEffect(() => {
    if (simulation !== 'off') return
    if (!('geolocation' in navigator)) {
      setGpsStatus('unavailable')
      setAccuracyMeters(null)
      applyPosition(null, { instant: true })
```

```
    return
  }

  setGpsStatus(current => (current === 'granted' ? current : 'searching'))
  let everFixed = false

  const id = navigator.geolocation.watchPosition(
    p => {
      everFixed = true
      setGpsStatus('granted')
      setAccuracyMeters(p.coords.accuracy ?? null)
      applyPosition({ lat: p.coords.latitude, lng: p.coords.longitude })
    },
    err => {
      // Three very different failures used to be collapsed into 'denied',
      // and the position was wiped for all of them. That was the bug
      // behind "recentre stops working after I move": indoors or on a cold
      // start a fix routinely takes longer than the old 15s timeout, the
      // TIMEOUT error fired, the app declared the user had refused
      // permission and threw away a perfectly good last-known position -
      // which also disabled the Recentre button, since it is disabled
      // whenever position is null.
      //
      // Only PERMISSION_DENIED is a refusal. The other two are transient,
      // and watchPosition keeps trying after them, so the last known
      // position is kept and the status says we are still looking.
      // Defensive: browsers always pass a GeolocationPositionError, but
      // treat a malformed one as transient rather than as a refusal.
      // Falsely reporting denial is the bug this whole branch exists to
      // fix, so that is the safer way to be wrong.
      if (err?.code === 1) {
        setGpsStatus('denied')
        setAccuracyMeters(null)
        applyPosition(null, { instant: true })
        return
      }

      setGpsStatus(everFixed ? 'lost' : 'searching')
    },
    // No `timeout`. On a watch it does not mean "give up after this" - it
    // fires an error and keeps going, so all it achieved was the false
    // denial above. maximumAge: 0 forbids reusing a cached fix at all:
    // this app's whole premise is where the pilgrim is standing NOW, and a
    // cached fix is exactly how a map ends up showing where they were.
    { enableHighAccuracy: true, maximumAge: 0 },
  )
  return () => navigator.geolocation.clearWatch(id)
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [simulation])

const parish = useMemo(
  () => (presence.parishId ? ROUTES.find(route => route.id === presence.parishId) ?? null : null),
  [presence.parishId],
)

const value = useMemo<PresenceContextValue>(
  () => ({
    presence,
    parish,
```

```
      position,
      accuracyMeters,
      gpsStatus: simulation === 'off' ? gpsStatus : 'granted',
      simulation,
      setSimulation,
    }),
    [presence, parish, position, accuracyMeters, gpsStatus, simulation],
  )

  return <PresenceCtx.Provider value={value}>{children}</PresenceCtx.Provider>
}

export function usePresence(): PresenceContextValue {
  const ctx = useContext(PresenceCtx)
  if (!ctx) throw new Error('usePresence must be used inside a PresenceProvider')
  return ctx
}
```

1.4 Shared Logic and Utilities

1.4.1 APP-071 — activityLog.ts

Code ID	APP-071
File Name	activityLog.ts
File Path	SanctDemo/src/lib/activityLog.ts
Language / Type	TypeScript
Lines	47
Functions / Classes	ActivityKind (exported type); logActivity (exported function)
Related Files (its imports)	./firebase
Where It Is Used	src/App.tsx, src/lib/analytics.ts
How to Recognize It	Open SanctDemo/src/lib/activityLog.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — activityLog.ts

```
import { addDoc, collection } from "firebase/firestore";
import { auth, db } from "./firebase";

/**
 * Session and visit events, kept out of the applications collection.
 *
 * ## Why this exists
 *
 * Three call sites were writing "User Authenticated", "Station Visited" and
 * "Station Comment Posted" into `applications`, each inventing its own
 * status - "Session Authorized", "Stamp Awarded", "Moderated Approval".
 * That had two costs. The parish office's queue, which is meant to be the
 * list of things needing a decision, filled up with login records; and the
 * security rules that make an application an application - owner's uid,
 * status "pending", a real church - cannot hold for a row that is not one.
 *
 * So they moved here. The parish can still see them; they are simply not
 * pretending to be sacrament applications.
 *
 * ## Why a failure here is swallowed
 *
 * This is telemetry. A pilgrim who scans a station and loses the log entry
 * has lost nothing they can see; a pilgrim whose check-in throws because
 * the log write failed has lost the thing they came for. Errors go to the
 * console, not to the person.
 */
export type ActivityKind = "sign_in" | "station_visit" | "station_comment";

export async function logActivity(
  kind: ActivityKind,
  summary: string,
```

```
  churchId?: string,
): Promise<void> {
  const user = auth.currentUser;
  if (!user) return;
  try {
    await addDoc(collection(db, "activity"), {
      uid: user.uid,
      kind,
      summary,
      churchId: churchId ?? "",
      createdAt: new Date().toISOString(),
    });
  } catch (err) {
    console.debug("[activity] not recorded:", err);
  }
}
```

1.4.2 APP-072 — analytics.ts

Code ID	APP-072
File Name	analytics.ts
File Path	SanctDemo/src/lib/analytics.ts
Language / Type	TypeScript
Lines	243
Functions / Classes	ActivityEntry (exported type); HourBucket (exported type); WeekdayBucket (exported type); DayBucket (exported type); StationCount (exported type); ACTIVITY_KINDS (exported constant); stationIdFrom (exported function); at (internal function); uniqueVisitors (exported function); countsByKind (exported function); visitsByHour (exported function); peakHour (exported function); visitsByWeekday (exported function); dateKey (internal function); dailyVisits (exported function); topStations (exported function); AnalyticsSummary (exported type); summariseActivity (exported function); formatHour (exported function)
Related Files (its imports)	./activityLog
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/analytics.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — analytics.ts

```
import type { ActivityKind } from './activityLog'

/**
 * Reading the activity log back.
 *
 * ## Why this exists
 *
 * `activityLog.ts` has been writing sign-ins, station visits and station
 * comments to Firestore since the check-in feature was built, and until
 * now nothing read any of it. The parish was being recorded and shown
 * nothing - which is the worst of both arrangements, since the data is
 * collected whether or not anybody benefits from it.
 *
 * ## Why the maths is out here
 *
 * It is arithmetic over timestamps, and arithmetic belongs where it can
 * be tested. Inside the dashboard component it could only be checked by
 * signing in as an administrator of a parish with months of real
 * activity behind it, which is not a thing that exists during
 * development.
 *
 * ## One deliberate omission
 *
 * Nothing here identifies a person. The log carries a uid, and it is
 * used only to count how many DISTINCT people appear - never to list
```

```
 * them, never to show what any one of them did. A parish asking "how
 * many came on Sunday" is asking a different question from "who came on
 * Sunday", and only the first is answered.
 */

export interface ActivityEntry {
  uid: string
  kind: ActivityKind
  summary: string
  churchId: string
  /** ISO 8601, as written by activityLog. */
  createdAt: string
}

export interface HourBucket {
  hour: number
  count: number
}

export interface WeekdayBucket {
  weekday: number
  label: string
  count: number
}

export interface DayBucket {
  /** Local calendar date, YYYY-MM-DD. */
  date: string
  count: number
  visitors: number
}

export interface StationCount {
  stationId: string
  count: number
}

export const ACTIVITY_KINDS: ActivityKind[] = [
  'sign_in', 'station_visit', 'station_comment',
]

const WEEKDAY_LABELS = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat']

/**
 * The station a logged sentence refers to.
 *
 * The log was written as prose - "Checked in at station st-altar" - and
 * the station id was never stored in a field of its own. Rather than
 * leave the existing rows unreadable, the id is recovered from the
 * sentence. New rows carry a `stationId` field as well; this remains the
 * fallback for everything written before that.
 */
export function stationIdFrom(summary: string): string | null {
  const match = /\bstation\s+(\S+)/i.exec(summary ?? '')
  return match ? match[1] : null
}

/** A timestamp, or null when the row carries something unparseable. */
function at(entry: ActivityEntry): Date | null {
```

```
  const date = new Date(entry.createdAt)
  return Number.isNaN(date.getTime()) ? null : date
}

/** How many distinct people appear, however often each one does. */
export function uniqueVisitors(entries: ActivityEntry[]): number {
  return new Set(entries.map(e => e.uid)).size
}

/** How often each feature was used. Every kind is named, including zero. */
export function countsByKind(entries: ActivityEntry[]): Record<ActivityKind, number> {
  const counts = Object.fromEntries(
    ACTIVITY_KINDS.map(k => [k, 0]),
  ) as Record<ActivityKind, number>

  for (const entry of entries) {
    if (entry.kind in counts) counts[entry.kind] += 1
  }
  return counts
}

/** Activity by hour of the day, all twenty-four of them. */
export function visitsByHour(entries: ActivityEntry[]): HourBucket[] {
  const buckets: HourBucket[] = Array.from({ length: 24 }, (_, hour) => ({ hour, count: 0 }))
  for (const entry of entries) {
    const when = at(entry)
    if (when) buckets[when.getHours()].count += 1
  }
  return buckets
}

/**
 * The busiest hour, or null when nothing has been recorded.
 *
 * Null rather than 0, because 0 is midnight: reporting midnight as a
 * parish's peak visiting hour on the strength of no data at all would be
 * a confident falsehood.
 */
export function peakHour(entries: ActivityEntry[]): number | null {
  const buckets = visitsByHour(entries)
  const busiest = buckets.reduce((best, b) => (b.count > best.count ? b : best))
  return busiest.count === 0 ? null : busiest.hour
}

/** Activity by day of the week, Sunday first. */
export function visitsByWeekday(entries: ActivityEntry[]): WeekdayBucket[] {
  const buckets: WeekdayBucket[] = WEEKDAY_LABELS.map((label, weekday) => ({
    weekday, label, count: 0,
  }))
  for (const entry of entries) {
    const when = at(entry)
    if (when) buckets[when.getDay()].count += 1
  }
  return buckets
}

/** Local calendar date as YYYY-MM-DD. */
function dateKey(date: Date): string {
  const month = `${date.getMonth() + 1}`.padStart(2, '0')
```

```
  const day = `${date.getDate()}`.padStart(2, '0')
  return `${date.getFullYear()}-${month}-${day}`
}

/**
 * The last `days` days up to and including today, oldest first.
 *
 * Both the number of events and the number of distinct people are
 * carried, because they answer different questions: twenty visits from
 * one pilgrim walking a tour is not twenty pilgrims.
 */
export function dailyVisits(
  entries: ActivityEntry[],
  days: number,
  now: Date,
): DayBucket[] {
  const order: string[] = []
  const counts = new Map<string, { count: number; visitors: Set<string> }>()

  for (let back = days - 1; back >= 0; back--) {
    const day = new Date(now.getFullYear(), now.getMonth(), now.getDate() - back)
    const key = dateKey(day)
    order.push(key)
    counts.set(key, { count: 0, visitors: new Set() })
  }

  for (const entry of entries) {
    const when = at(entry)
    if (!when) continue
    const bucket = counts.get(dateKey(when))
    if (!bucket) continue // Outside the window.
    bucket.count += 1
    bucket.visitors.add(entry.uid)
  }

  return order.map(date => {
    const bucket = counts.get(date)!
    return { date, count: bucket.count, visitors: bucket.visitors.size }
  })
}

/**
 * The most engaged-with stations.
 *
 * A comment counts alongside a visit. Both are someone stopping at that
 * station and doing something there, which is what "most visited area"
 * is asking about.
 */
export function topStations(entries: ActivityEntry[], limit: number): StationCount[] {
  const counts = new Map<string, number>()

  for (const entry of entries) {
    const stationId = stationIdFrom(entry.summary)
    if (!stationId) continue
    counts.set(stationId, (counts.get(stationId) ?? 0) + 1)
  }

  return [...counts.entries()]
    .map(([stationId, count]) => ({ stationId, count }))
```

```
    .sort((a, b) => b.count - a.count || a.stationId.localeCompare(b.stationId))
    .slice(0, limit)
}

export interface AnalyticsSummary {
  totalEvents: number
  uniqueVisitors: number
  peakHour: number | null
  byKind: Record<ActivityKind, number>
  byHour: HourBucket[]
  byWeekday: WeekdayBucket[]
  daily: DayBucket[]
  topStations: StationCount[]
}

/** Everything the dashboard shows, from one pass over the log. */
export function summariseActivity(
  entries: ActivityEntry[],
  now: Date,
  days = 7,
  stations = 5,
): AnalyticsSummary {
  return {
    totalEvents: entries.length,
    uniqueVisitors: uniqueVisitors(entries),
    peakHour: peakHour(entries),
    byKind: countsByKind(entries),
    byHour: visitsByHour(entries),
    byWeekday: visitsByWeekday(entries),
    daily: dailyVisits(entries, days, now),
    topStations: topStations(entries, stations),
  }
}

/** "2 PM", for an hour number. Midnight and noon named rather than "12". */
export function formatHour(hour: number | null): string {
  if (hour === null) return "—"
  if (hour === 0) return "midnight"
  if (hour === 12) return "noon"
  return hour < 12 ? `${hour} AM` : `${hour - 12} PM`
}
```

1.4.3 APP-073 — announcementImage.ts

Code ID	APP-073
File Name	announcementImage.ts
File Path	SanctDemo/src/lib/announcementImage.ts
Language / Type	TypeScript
Lines	85
Functions / Classes	BANNER_W (exported constant); BANNER_H (exported constant); MAX_STORED_BYTES (exported constant); AnnouncementImageError (exported class); fileToBannerDataUrl (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/admin/AdminAnnouncements.tsx
How to Recognize It	Open SanctDemo/src/lib/announcementImage.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turning a chosen photo into an announcement banner.
>
> Same stopgap as lib/avatar.ts and for the same reason: Firebase Storage is not set up on this project, so an upload would be a feature that fails for every parish. The picture is resized hard here and carried in the announcement document as a data URL.
>
> Unlike an avatar this crops to 16:9, not to a square. An announcement photo is a group of altar servers or a church interior - a centre-square crop of either loses the ends of the group and the aisle.

Complete Source Code — announcementImage.ts

```
/**
 * Turning a chosen photo into an announcement banner.
 *
 * Same stopgap as lib/avatar.ts and for the same reason: Firebase Storage
 * is not set up on this project, so an upload would be a feature that
 * fails for every parish. The picture is resized hard here and carried in
 * the announcement document as a data URL.
 *
 * Unlike an avatar this crops to 16:9, not to a square. An announcement
 * photo is a group of altar servers or a church interior - a centre-square
 * crop of either loses the ends of the group and the aisle.
 */

/** The stored banner's width. Retina-sharp at the ~330px it displays. */
export const BANNER_W = 720;
export const BANNER_H = 405; // 16:9

/**
 * Refused above this. A Firestore document cannot exceed 1 MB in total,
```

```
 * and the bulletin may hold several of these on screen at once.
 */
export const MAX_STORED_BYTES = 220_000;

export class AnnouncementImageError extends Error {}

export async function fileToBannerDataUrl(file: File): Promise<string> {
  if (!file.type.startsWith("image/")) {
    throw new AnnouncementImageError("That file is not an image. Please choose a photo.");
  }

  const bitmap = await loadImage(file);

  // The largest 16:9 rectangle that fits, centred. Squashing a photograph
  // to the frame is worse than cropping it.
  const targetRatio = BANNER_W / BANNER_H;
  let sw = bitmap.width;
  let sh = Math.round(sw / targetRatio);
  if (sh > bitmap.height) {
    sh = bitmap.height;
    sw = Math.round(sh * targetRatio);
  }
  const sx = Math.round((bitmap.width - sw) / 2);
  const sy = Math.round((bitmap.height - sh) / 2);

  const canvas = document.createElement("canvas");
  canvas.width = BANNER_W;
  canvas.height = BANNER_H;
  const ctx = canvas.getContext("2d");
  if (!ctx) throw new AnnouncementImageError("This browser could not process the image.");

  ctx.drawImage(bitmap, sx, sy, sw, sh, 0, 0, BANNER_W, BANNER_H);
  if ("close" in bitmap) (bitmap as ImageBitmap).close?.();

  let quality = 0.8;
  let url = canvas.toDataURL("image/jpeg", quality);
  while (url.length > MAX_STORED_BYTES && quality > 0.4) {
    quality -= 0.1;
    url = canvas.toDataURL("image/jpeg", quality);
  }
  if (url.length > MAX_STORED_BYTES) {
    throw new AnnouncementImageError("That photo is too detailed to store. Please try another.");
  }
  return url;
}

async function loadImage(file: File): Promise<ImageBitmap | HTMLImageElement> {
  if (typeof createImageBitmap === "function") {
    try {
      return await createImageBitmap(file);
    } catch {
      // Safari has refused some JPEGs here; the <img> path below handles them.
    }
  }
  const url = URL.createObjectURL(file);
  try {
    return await new Promise<HTMLImageElement>((resolve, reject) => {
      const img = new Image();
      img.onload = () => resolve(img);
```

```
      img.onerror = () => reject(new AnnouncementImageError("That image could not be read."));
      img.src = url;
    });
  } finally {
    URL.revokeObjectURL(url);
  }
}
```

1.4.4 APP-074 — announcements.ts

Code ID	APP-074
File Name	announcements.ts
File Path	SanctDemo/src/lib/announcements.ts
Language / Type	TypeScript
Lines	235
Functions / Classes	AnnouncementStatus (exported type); AnnouncementDoc (exported type); ANNOUNCEMENT_TYPES (exported constant); FEAST_TYPE (exported constant); EVENT_TYPE (exported constant); Board (exported type); onBoard (exported function); BOARD_LABEL (exported constant); statusOf (exported function); STATUS_LABEL (exported constant); publishedOnly (exported function); startOfDay (exported function); announcementsForParish (exported function); upcomingAnnouncements (exported function); formatWhen (exported function); NewAnnouncement (exported type); createAnnouncement (exported function); updateAnnouncement (exported function); deleteAnnouncement (exported function)
Related Files (its imports)	./firebase
Where It Is Used	src/components/BulletinRail.tsx, src/components/Dashboard.tsx, src/lib/reminderSchedule.ts
How to Recognize It	Open SanctDemo/src/lib/announcements.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — announcements.ts

```
import { addDoc, collection, deleteDoc, doc, updateDoc } from "firebase/firestore";
import { db } from "./firebase";

/**
 * Parish announcements — the middle of the Parish Bulletin.
 *
 * ## Why churchId is optional
 *
 * Announcements existed before parishes were isolated, and those documents
 * have no churchId at all. Treating a missing churchId as "deleted" would
 * empty every bulletin the day this shipped; treating it as diocese-wide is
 * what it actually meant when it was written, so that is what it stays.
 * Anything the new admin writes carries a churchId and belongs to one
 * parish.
 *
 * ## Why the image is a data URL
 *
 * Same reason as lib/avatar.ts: Firebase Storage is not set up on this
 * project, so an upload path would be a feature that silently fails for
 * every parish. announcementImage.ts resizes hard enough that the picture
 * fits in the document, and the field is called imageUrl rather than
```

```
 * imageData so swapping to Storage later costs nothing above this module.
 */

/**
 * Where an announcement is in its life.
 *
 * Absent means published. Everything written before this field existed is
 * already on people's home screens, and reading those as drafts would
 * empty every bulletin in the diocese the moment this shipped.
 */
export type AnnouncementStatus = "published" | "draft" | "archived";

export interface AnnouncementDoc {
  id: string;
  /** Absent on the legacy diocese-wide announcements; see above. */
  churchId?: string;
  title: string;
  /** The parish's own sentence or two. Optional — a title can be enough. */
  body?: string;
  /** When the thing happens. Any string Date can parse; "" means undated. */
  date: string;
  time?: string;
  /**
   * "Mass", "Feast", "Event"… shown as the chip on the card, and what
   * separates the Feast Days and Events screens from the rest.
   *
   * Reused rather than joined by a second `category` field: this one
   * already holds exactly these values on every existing document.
   */
  type: string;
  status?: AnnouncementStatus;
  /** Feast days and events happen somewhere. Announcements usually do not. */
  location?: string;
  imageUrl?: string;
  createdAt?: string;
  createdBy?: string;
  updatedAt?: string;
  updatedBy?: string;
}

/** What the admin form offers. Free text would give every parish its own. */
export const ANNOUNCEMENT_TYPES = [
  "Mass",
  "Feast",
  "Event",
  "Notice",
  "Schedule change",
] as const;

/**
 * The three screens in the admin, and what each one holds.
 *
 * Feast Days and Events are not their own collections. They are the same
 * document with a different chip on it - same title, date, picture,
 * published/archived life - and three collections would have meant three
 * sets of rules, three admin pages and three places for the next bug.
 */
export const FEAST_TYPE = "Feast";
export const EVENT_TYPE = "Event";
```

```
export type Board = "announcements" | "feasts" | "events";

export function onBoard(item: AnnouncementDoc, board: Board): boolean {
  if (board === "feasts") return item.type === FEAST_TYPE;
  if (board === "events") return item.type === EVENT_TYPE;
  // The announcements board is everything, feasts and events included.
  // A parish secretary looking for "the thing I posted on Tuesday" should
  // not have to remember which chip they gave it.
  return true;
}

export const BOARD_LABEL: Record<Board, string> = {
  announcements: "Announcements",
  feasts: "Feast Days",
  events: "Events",
};

/** Absent means published; see AnnouncementStatus. */
export function statusOf(item: AnnouncementDoc): AnnouncementStatus {
  return item.status ?? "published";
}

export const STATUS_LABEL: Record<AnnouncementStatus, string> = {
  published: "Published",
  draft: "Draft",
  archived: "Archived",
};

/**
 * What a pilgrim may see: published only.
 *
 * Drafts and archived items are filtered here, in one place, so no screen
 * can forget. The bulletin, the feast list and anything added later all
 * come through this.
 */
export function publishedOnly(items: AnnouncementDoc[]): AnnouncementDoc[] {
  return items.filter(a => statusOf(a) === "published");
}

const COLLECTION = "announcements";

export function startOfDay(date: Date): number {
  return new Date(date.getFullYear(), date.getMonth(), date.getDate()).getTime();
}

/**
 * The ones this parish's bulletin should show: its own, plus the legacy
 * diocese-wide ones that belong to nobody.
 */
export function announcementsForParish(
  all: AnnouncementDoc[],
  parishId: string,
): AnnouncementDoc[] {
  return all.filter(a => !a.churchId || a.churchId === parishId);
}

/**
 * Soonest first, with anything already past dropped.
```

```
 *
 * A bulletin still showing last month's fiesta is worse than a short one.
 * Undated announcements are kept rather than guessed at — a parish notice
 * with no event date is still a notice — and they sort to the end.
 */
export function upcomingAnnouncements(
  items: AnnouncementDoc[],
  now: Date = new Date(),
): Array<AnnouncementDoc & { when: Date | null }> {
  const today = startOfDay(now);
  return items
    .map(a => {
      const parsed = a.date ? new Date(a.date) : null;
      const when = parsed && !Number.isNaN(parsed.getTime()) ? parsed : null;
      return { ...a, when };
    })
    .filter(a => a.when === null || a.when.getTime() >= today)
    .sort((a, b) => {
      if (a.when === null) return b.when === null ? 0 : 1;
      if (b.when === null) return -1;
      return a.when.getTime() - b.when.getTime();
    });
}

/** "Today", "Tomorrow", "Saturday", or a date once it is more than a week off. */
export function formatWhen(when: Date, now: Date = new Date()): string {
  const days = Math.round((startOfDay(when) - startOfDay(now)) / 86_400_000);
  if (days <= 0) return "Today";
  if (days === 1) return "Tomorrow";
  if (days < 7) return when.toLocaleDateString("en-US", { weekday: "long" });
  return when.toLocaleDateString("en-US", { month: "long", day: "numeric" });
}

export interface NewAnnouncement {
  churchId: string;
  title: string;
  body?: string;
  date: string;
  time?: string;
  type: string;
  location?: string;
  imageUrl?: string;
  status?: AnnouncementStatus;
}

export async function createAnnouncement(
  input: NewAnnouncement,
  authorEmail: string,
): Promise<void> {
  // Undefined fields are stripped rather than sent: Firestore rejects an
  // undefined value outright, and an optional field the parish left blank
  // is the ordinary case here, not an error.
  const payload: Record<string, string> = {
    churchId: input.churchId,
    title: input.title.trim(),
    date: input.date,
    type: input.type,
    createdAt: new Date().toISOString(),
    createdBy: authorEmail,
```

```
  };
  payload.status = input.status ?? "published";
  if (input.body?.trim()) payload.body = input.body.trim();
  if (input.time) payload.time = input.time;
  if (input.location?.trim()) payload.location = input.location.trim();
  if (input.imageUrl) payload.imageUrl = input.imageUrl;

  await addDoc(collection(db, COLLECTION), payload);
}

/**
 * Edit, publish, archive and restore - all one write.
 *
 * Archiving is a status change, not a delete. A parish that archived last
 * year's fiesta can still look it up, which is the whole point of having
 * an archive rather than a confirmation dialog.
 */
export async function updateAnnouncement(
  id: string,
  patch: Partial<NewAnnouncement> & { status?: AnnouncementStatus },
  editorEmail: string,
): Promise<void> {
  const clean: Record<string, unknown> = {
    updatedAt: new Date().toISOString(),
    updatedBy: editorEmail,
  };
  for (const [key, value] of Object.entries(patch)) {
    // Firestore rejects undefined outright, and a field the parish
    // cleared has to be written as "" rather than left behind.
    if (value !== undefined) clean[key] = typeof value === "string" ? value.trim() : value;
  }
  await updateDoc(doc(db, COLLECTION, id), clean);
}

export async function deleteAnnouncement(id: string): Promise<void> {
  await deleteDoc(doc(db, COLLECTION, id));
}
```

1.4.5 APP-075 — apiBase.ts

Code ID	APP-075
File Name	apiBase.ts
File Path	SanctDemo/src/lib/apiBase.ts
Language / Type	TypeScript
Lines	41
Functions / Classes	apiBase (exported function); apiUrl (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/AdminPortal.tsx, src/components/ArTour.tsx, src/components/CcompanionTab.tsx, src/lib/askSancti.ts
How to Recognize It	Open SanctDemo/src/lib/apiBase.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Where the Express backend lives.
>
> On the web, SanctiWalk is served BY that backend, so "/api/identify" is a relative path to the same origin and everything just works. Packaged as an Android app it is not: the APK serves its own files from a local scheme, there is no server at its origin, and every relative /api call 404s.
>
> So the base URL becomes a build-time setting. Set VITE_API_BASE when building the APK and leave it unset for the web build.
>
> Worth being clear about what this does and does not affect. The features behind these calls - AI recognition, the emailer - need the internet regardless, because they reach Gemini and an SMTP server. Bundling the app into an APK makes the app shell, the parish data, the Rosary and the station content work offline; it was never going to make the AI work offline. This setting only decides WHERE the server is, not whether one is needed.
>
> Pure and dependency-free so the joining rules can be tested.

Complete Source Code — apiBase.ts

```
// Where the Express backend lives.
//
// On the web, SanctiWalk is served BY that backend, so "/api/identify" is a
// relative path to the same origin and everything just works. Packaged as an
// Android app it is not: the APK serves its own files from a local scheme,
// there is no server at its origin, and every relative /api call 404s.
//
// So the base URL becomes a build-time setting. Set VITE_API_BASE when
// building the APK and leave it unset for the web build.
//
// Worth being clear about what this does and does not affect. The features
// behind these calls - AI recognition, the emailer - need the internet
// regardless, because they reach Gemini and an SMTP server. Bundling the app
// into an APK makes the app shell, the parish data, the Rosary and the
```

```
// station content work offline; it was never going to make the AI work
// offline. This setting only decides WHERE the server is, not whether one is
// needed.
//
// Pure and dependency-free so the joining rules can be tested.

/**
 * Base URL for backend calls. Empty string means "same origin as this page",
 * which is correct for the web build.
 */
export function apiBase(): string {
  const configured = import.meta.env?.VITE_API_BASE
  return typeof configured === 'string' ? configured.trim().replace(/\/+$/, '') : ''
}

/**
 * Joins a backend path onto the base.
 *
 * Takes the path exactly as it is written at the call site ("/api/identify")
 * so the change is invisible on the web and every call site reads the same as
 * before.
 */
export function apiUrl(path: string): string {
  const base = apiBase()
  if (!base) return path
  return `${base}${path.startsWith('/') ? path : `/${path}`}`
}
```

1.4.6 APP-076 — appKey.ts

Code ID	APP-076
File Name	appKey.ts
File Path	SanctDemo/src/lib/appKey.ts
Language / Type	TypeScript
Lines	37
Functions / Classes	APP_KEY_HEADER (exported constant); appKey (exported function); withAppKey (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/AdminPortal.tsx, src/components/ArTour.tsx, src/components/CcompanionTab.tsx, src/lib/askSancti.ts
How to Recognize It	Open SanctDemo/src/lib/appKey.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The shared key the app sends with every backend call.
>
> Once the Express server is deployed it is reachable by anyone who finds the URL, and /api/identify spends a real Gemini quota. This header is what stops a bot or a passer-by draining it.
>
> Be clear about what this is worth. The key is compiled into the app, so anyone willing to unpack the APK can read it. It is a speed bump, not authentication: it stops opportunistic and automated abuse, which is the realistic threat for a student project with a published URL. Real per-user auth would mean signing in before scanning, which the app deliberately does not require.
>
> Paired with the per-IP daily cap on the server, the damage one determined person can do is bounded.

Complete Source Code — appKey.ts

```
// The shared key the app sends with every backend call.
//
// Once the Express server is deployed it is reachable by anyone who finds the
// URL, and /api/identify spends a real Gemini quota. This header is what
// stops a bot or a passer-by draining it.
//
// Be clear about what this is worth. The key is compiled into the app, so
// anyone willing to unpack the APK can read it. It is a speed bump, not
// authentication: it stops opportunistic and automated abuse, which is the
// realistic threat for a student project with a published URL. Real
// per-user auth would mean signing in before scanning, which the app
// deliberately does not require.
//
// Paired with the per-IP daily cap on the server, the damage one determined
// person can do is bounded.
```

```
export const APP_KEY_HEADER = 'X-SanctiWalk-Key'

/**
 * The key for this build, or empty when unset.
 *
 * Unset is the correct state for local development: the server only enforces
 * the key when APP_KEY is set in its own environment, so a developer running
 * both halves locally needs no configuration at all.
 */
export function appKey(): string {
  const configured = import.meta.env?.VITE_APP_KEY
  return typeof configured === 'string' ? configured.trim() : ''
}

/**
 * Adds the key to request headers, leaving them untouched when none is set.
 */
export function withAppKey(headers: Record<string, string> = {}): Record<string, string> {
  const key = appKey()
  return key ? { ...headers, [APP_KEY_HEADER]: key } : { ...headers }
}
```

1.4.7 APP-077 — appUpdate.ts

Code ID	APP-077
File Name	appUpdate.ts
File Path	SanctDemo/src/lib/appUpdate.ts
Language / Type	TypeScript
Lines	134
Functions / Classes	LatestRelease (exported type); versionCodeOf (exported function); isNewer (exported function); fetchLatest (exported function); wasDismissed (exported function); dismiss (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/appUpdate.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Telling an installed SanctiWalk that a newer one exists.
>
> ## Why this is needed at all
>
> SanctiWalk is handed out as an APK from its own download page, not through Google Play, so nothing updates it. Parish content does not need this - Mass times, announcements and availability stream from Firestore and change under the running app - but a code change does, and until now the only way anyone would learn of one is if somebody told them to go and download it again.
>
> ## How the version is known on both sides
>
> One number, from package.json, used twice. Vite bakes it into the web bundle as __APP_VERSION__, and the Android build reads the same field for versionName and derives versionCode from it. They cannot drift, because there is only one of them.
>
> The published copy is a small JSON file written beside the APK at build time. A static file rather than an endpoint on the backend: the backend sleeps when idle and takes several seconds to wake, and an update check is the last thing that should keep somebody waiting at a church door.

Complete Source Code — appUpdate.ts

```
/**
 * Telling an installed SanctiWalk that a newer one exists.
 *
 * ## Why this is needed at all
 *
 * SanctiWalk is handed out as an APK from its own download page, not
 * through Google Play, so nothing updates it. Parish content does not
 * need this - Mass times, announcements and availability stream from
```

```
 * Firestore and change under the running app - but a code change does,
 * and until now the only way anyone would learn of one is if somebody
 * told them to go and download it again.
 *
 * ## How the version is known on both sides
 *
 * One number, from package.json, used twice. Vite bakes it into the web
 * bundle as __APP_VERSION__, and the Android build reads the same field
 * for versionName and derives versionCode from it. They cannot drift,
 * because there is only one of them.
 *
 * The published copy is a small JSON file written beside the APK at
 * build time. A static file rather than an endpoint on the backend:
 * the backend sleeps when idle and takes several seconds to wake, and
 * an update check is the last thing that should keep somebody waiting
 * at a church door.
 */

/** Shape of /download/latest.json, written by tools/write-latest-version. */
export interface LatestRelease {
  versionName: string
  versionCode: number
  /** Where the APK is, as an absolute or site-relative URL. */
  url: string
  /** Optional one-line summary shown in the banner. */
  notes?: string
}

/**
 * major.minor.patch as one integer, the same way build.gradle does it.
 *
 * Comparing version STRINGS is the classic way to get this wrong: by
 * string order "1.10.0" sorts before "1.9.0", so the update that
 * mattered would be the one never offered.
 */
export function versionCodeOf(versionName: string): number {
  const parts = versionName.trim().split('.')
  if (parts.length !== 3) return 0

  const nums = parts.map(p => (/^\d+$/.test(p) ? Number(p) : NaN))
  if (nums.some(n => !Number.isFinite(n))) return 0

  return nums[0] * 10000 + nums[1] * 100 + nums[2]
}

/**
 * Whether what is published is newer than what is running.
 *
 * Strictly newer. Equal is not an update, and a published version OLDER
 * than the installed one is not either - that happens while a deploy is
 * half-rolled-out, and offering a downgrade would be worse than silence
 * because Android will refuse to install it.
 */
export function isNewer(installed: string, latest: LatestRelease | null): boolean {
  if (!latest) return false

  const here = versionCodeOf(installed)
  // Trust the published code where there is one; fall back to its name.
  const there = Number.isFinite(latest.versionCode) && latest.versionCode > 0
```

```
    ? latest.versionCode
    : versionCodeOf(latest.versionName ?? '')

  if (here === 0 || there === 0) return false
  return there > here
}

/**
 * Reads the published release, or null.
 *
 * Every failure is null and none of them are reported to the pilgrim.
 * An update check is a courtesy; a church with bad signal must not show
 * somebody an error about one.
 */
export async function fetchLatest(
  url: string,
  fetchImpl: typeof fetch = fetch,
): Promise<LatestRelease | null> {
  try {
    const response = await fetchImpl(url, { cache: 'no-store' })
    if (!response.ok) return null

    const data: unknown = await response.json()
    if (!data || typeof data !== 'object') return null

    const record = data as Record<string, unknown>
    const versionName = typeof record.versionName === 'string' ? record.versionName : ''
    const apkUrl = typeof record.url === 'string' ? record.url : ''
    if (!versionName || !apkUrl) return null

    return {
      versionName,
      versionCode: typeof record.versionCode === 'number' ? record.versionCode : 0,
      url: apkUrl,
      notes: typeof record.notes === 'string' ? record.notes : undefined,
    }
  } catch {
    return null
  }
}

/** Key for the version a pilgrim has already been offered and declined. */
const DISMISSED_KEY = 'sanctiwalk.update.dismissed'

/**
 * Whether this exact version has already been turned down.
 *
 * Asked once per release, not once per launch. Somebody who taps "Later"
 * has answered the question, and asking again every time they open the
 * app is how a helpful prompt becomes the reason they stop opening it.
 * A NEWER release asks again, because that is a different question.
 */
export function wasDismissed(versionName: string): boolean {
  try {
    return localStorage.getItem(DISMISSED_KEY) === versionName
  } catch {
    return false
  }
}
```

```
export function dismiss(versionName: string): void {
  try {
    localStorage.setItem(DISMISSED_KEY, versionName)
  } catch {
    /* private mode; it will ask again, which is the harmless failure */
  }
}
```

1.4.8 APP-078 — applicationFilters.ts

Code ID	APP-078
File Name	applicationFilters.ts
File Path	SanctDemo/src/lib/applicationFilters.ts
Language / Type	TypeScript
Lines	225
Functions / Classes	Scope (exported type); ARCHIVED_STATUSES (exported constant); isArchived (exported function); Filters (exported type); NO_FILTERS (exported constant); MONTHS (exported constant); submittedAt (exported function); matchesSearch (internal function); applyFilters (exported function); startOfDay (internal function); endOfDay (internal function); SortKey (exported type); sortApplications (exported function); typesPresent (exported function); yearsPresent (exported function); activeFilterCount (exported function)
Related Files (its imports)	../types
Where It Is Used	src/admin/AdminDashboard.tsx, src/lib/applications.ts, src/lib/exportApplications.ts
How to Recognize It	Open SanctDemo/src/lib/applicationFilters.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — applicationFilters.ts

```
import type { ApplicationDoc, ApplicationStatus } from "../types";

/**
 * Filtering and sorting the parish's applications.
 *
 * Pure functions on an array, deliberately. The alternative is a Firestore
 * query per filter combination, and that needs a composite index for every
 * one of them - churchId + kind + status + a createdAt range is a different
 * index from churchId + type + status, and an index that has not been
 * deployed makes the page throw rather than render.
 *
 * A parish's application list is hundreds of rows, not millions. The whole
 * parish arrives in one churchId query - which the security rules already
 * scope - and the filtering happens here, where it is instant, needs no
 * index, and can be unit-tested without a database.
 *
 * If a parish ever outgrows that, the fix is pagination on createdAt, not a
 * matrix of indexes.
 */

/**
 * Whether a view is showing live work or the archive.
 *
 * "Archived" is not a field on the document and deliberately so. An
 * application is finished when the parish has finished with it - rejected
 * or completed - and that is already recorded in `status`. A separate
```

```
 * `archived: true` would be a second source of truth for the same fact,
 * and the two would drift the first time anyone reopened something.
 */
export type Scope = "open" | "archived" | "all";

/** The statuses that take an application out of the parish's in-tray. */
export const ARCHIVED_STATUSES: ApplicationStatus[] = ["rejected", "completed"];

export function isArchived(status: ApplicationStatus): boolean {
  return ARCHIVED_STATUSES.includes(status);
}

export interface Filters {
  /**
   * Defaults to "open". A parish office asking "how many applications do
   * we have" means the ones still needing something done, not every one
   * ever filed - the finished ones were inflating every count on the
   * dashboard and every list under it.
   */
  scope: Scope;
  /** 1-12, or null for any month. */
  month: number | null;
  year: number | null;
  /** "ministry" | "sacrament", or null for both. */
  kind: "ministry" | "sacrament" | null;
  /** The exact ministry or sacrament name, or null for all of them. */
  type: string | null;
  status: ApplicationStatus | null;
  /** ISO dates, inclusive. Independent of month/year. */
  from: string | null;
  to: string | null;
  /** Matched against name, email, reference and type. */
  search: string;
}

/**
 * The resting state of the table: no filters chosen, and the archive put
 * away. "Clear filters" returns here, which is why scope lives in this
 * object rather than beside it.
 */
export const NO_FILTERS: Filters = {
  scope: "open",
  month: null, year: null, kind: null, type: null,
  status: null, from: null, to: null, search: "",
};

export const MONTHS = [
  "January", "February", "March", "April", "May", "June",
  "July", "August", "September", "October", "November", "December",
];

/** The date an application is filed under: when it was submitted. */
export function submittedAt(app: ApplicationDoc): Date | null {
  const raw = app.createdAt;
  if (!raw) return null;
  const d = new Date(raw);
  return Number.isNaN(d.getTime()) ? null : d;
}
```

```
function matchesSearch(app: ApplicationDoc, needle: string): boolean {
  const q = needle.trim().toLowerCase();
  if (!q) return true;
  // Reference last: it is the least likely thing typed, but the most
  // decisive when it is.
  return (
    (app.applicantName ?? "").toLowerCase().includes(q) ||
    (app.applicantEmail ?? "").toLowerCase().includes(q) ||
    (app.type ?? "").toLowerCase().includes(q) ||
    (app.referenceNumber ?? "").toLowerCase().includes(q)
  );
}

export function applyFilters(apps: ApplicationDoc[], f: Filters): ApplicationDoc[] {
  return apps.filter(app => {
    if (f.kind && app.kind !== f.kind) return false;
    if (f.type && app.type !== f.type) return false;
    if (f.status && app.status !== f.status) return false;

    // Scope yields to an explicit status. Asking for "Completed" while the
    // view is scoped to open work would otherwise return nothing at all,
    // and an empty table is read as "there are none" rather than as "those
    // two filters cannot both be true".
    if (!f.status && f.scope !== "all") {
      const archived = isArchived(app.status);
      if (f.scope === "open" && archived) return false;
      if (f.scope === "archived" && !archived) return false;
    }
    if (!matchesSearch(app, f.search)) return false;

    const when = submittedAt(app);

    // A row with no usable date cannot honestly be said to fall inside a
    // month or a range, so it drops out of a dated view rather than being
    // shown under a month it might not belong to.
    if (f.month !== null || f.year !== null || f.from || f.to) {
      if (!when) return false;
    }

    if (f.year !== null && when!.getFullYear() !== f.year) return false;
    if (f.month !== null && when!.getMonth() + 1 !== f.month) return false;

    if (f.from) {
      const from = new Date(f.from);
      if (!Number.isNaN(from.getTime()) && when! < startOfDay(from)) return false;
    }
    if (f.to) {
      const to = new Date(f.to);
      // Inclusive: "to 31 August" means the whole of the 31st, not midnight.
      if (!Number.isNaN(to.getTime()) && when! > endOfDay(to)) return false;
    }

    return true;
  });
}

function startOfDay(d: Date): Date {
  return new Date(d.getFullYear(), d.getMonth(), d.getDate(), 0, 0, 0, 0);
}
```

```
function endOfDay(d: Date): Date {
  return new Date(d.getFullYear(), d.getMonth(), d.getDate(), 23, 59, 59, 999);
}

export type SortKey = "submitted" | "name" | "type" | "status" | "reference";

export function sortApplications(
  apps: ApplicationDoc[],
  key: SortKey,
  direction: "asc" | "desc",
): ApplicationDoc[] {
  const sign = direction === "asc" ? 1 : -1;
  return [...apps].sort((a, b) => {
    let cmp = 0;
    if (key === "submitted") {
      const ta = submittedAt(a)?.getTime() ?? 0;
      const tb = submittedAt(b)?.getTime() ?? 0;
      cmp = ta - tb;
    } else if (key === "name") {
      cmp = (a.applicantName ?? "").localeCompare(b.applicantName ?? "");
    } else if (key === "type") {
      cmp = (a.type ?? "").localeCompare(b.type ?? "");
    } else if (key === "status") {
      cmp = (a.status ?? "").localeCompare(b.status ?? "");
    } else {
      cmp = (a.referenceNumber ?? "").localeCompare(b.referenceNumber ?? "");
    }
    return cmp * sign;
  });
}

/**
 * The ministry and sacrament names actually present in this parish's
 * applications.
 *
 * Read from the data rather than from the MINISTRIES catalogue, so the
 * dropdown offers what someone has really applied for. A list of thirty
 * ministries where twenty-eight return nothing is a list you have to read
 * twice to use once - and a ministry that was renamed still appears here
 * for the old applications that carry the old name.
 */
export function typesPresent(apps: ApplicationDoc[], kind?: "ministry" | "sacrament"): string[] {
  const seen = new Set<string>();
  for (const a of apps) {
    if (kind && a.kind !== kind) continue;
    if (a.type) seen.add(a.type);
  }
  return [...seen].sort((a, b) => a.localeCompare(b));
}

/** The years with at least one application, newest first. */
export function yearsPresent(apps: ApplicationDoc[]): number[] {
  const seen = new Set<number>();
  for (const a of apps) {
    const d = submittedAt(a);
    if (d) seen.add(d.getFullYear());
  }
  return [...seen].sort((a, b) => b - a);
}
```

```
/**
 * How many filters the admin has chosen, for the "Clear filters" button.
 *
 * The default scope does not count: it is the resting state of the table,
 * and offering to clear a filter nobody set is confusing. Switching to the
 * archive or to everything does count, because that IS a choice and it is
 * the one most likely to explain a surprising list.
 */
export function activeFilterCount(f: Filters): number {
  let n = 0;
  if (f.scope !== "open") n++;
  if (f.month !== null) n++;
  if (f.year !== null) n++;
  if (f.kind) n++;
  if (f.type) n++;
  if (f.status) n++;
  if (f.from) n++;
  if (f.to) n++;
  if (f.search.trim()) n++;
  return n;
}
```

1.4.9 APP-079 — applicationForm.ts

Code ID	APP-079
File Name	applicationForm.ts
File Path	SanctDemo/src/lib/applicationForm.ts
Language / Type	TypeScript
Lines	358
Functions / Classes	FieldKind (exported type); Field (exported type); Step (exported type); Answers (exported type); personalStep (internal function); ministrySpecific (internal function); ministrySteps (exported function); sacramentSteps (exported function); validateStep (exported function); isPlausiblePhone (exported function); toFormData (exported function); reviewRows (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/applicationForm.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The questions an application asks, as data.
>
> ## Why a schema and not fifteen hand-written forms
>
> The brief is explicit that different ministries need different questions - a choir asks what you sing, altar servers ask whether you have served before - and equally explicit that the same questions must not be forced on all of them. Hard-coding that would be fifteen forms to keep in step, and the validation would drift between them within a month.
>
> As data it is one form component, one validator, and one place to add a question. It is also the shape a parish admin could eventually edit: the step from "this array lives in the code" to "this array lives in parishContent" is a loader, not a rewrite.
>
> ## Why nothing here invents a requirement
>
> Every question below is administrative - who you are, how to reach you, when you are free. Nothing asks for a certificate, a seminar or a fee, because this codebase has a standing rule against inventing parish policy, and the real requirements are already in data.ts where the parish put them.

Complete Source Code — applicationForm.ts

```
/**
 * The questions an application asks, as data.
 *
 * ## Why a schema and not fifteen hand-written forms
 *
 * The brief is explicit that different ministries need different
 * questions - a choir asks what you sing, altar servers ask whether you
```

```
 * have served before - and equally explicit that the same questions must
 * not be forced on all of them. Hard-coding that would be fifteen forms
 * to keep in step, and the validation would drift between them within a
 * month.
 *
 * As data it is one form component, one validator, and one place to add
 * a question. It is also the shape a parish admin could eventually edit:
 * the step from "this array lives in the code" to "this array lives in
 * parishContent" is a loader, not a rewrite.
 *
 * ## Why nothing here invents a requirement
 *
 * Every question below is administrative - who you are, how to reach
 * you, when you are free. Nothing asks for a certificate, a seminar or a
 * fee, because this codebase has a standing rule against inventing
 * parish policy, and the real requirements are already in data.ts where
 * the parish put them.
 */

export type FieldKind = "text" | "tel" | "email" | "date" | "textarea" | "choices" | "confirm";

export interface Field {
  /** Key in the application's formData. Stable - the parish reads these. */
  name: string;
  label: string;
  kind: FieldKind;
  required?: boolean;
  /** Shown under the label; a hint, not a placeholder. */
  help?: string;
  placeholder?: string;
  /** For "choices": the options, chosen as many as apply. */
  options?: string[];
  maxLength?: number;
}

export interface Step {
  title: string;
  /** One sentence on why this step is being asked. */
  blurb?: string;
  fields: Field[];
}

/** Everything a person types, before it becomes formData. */
export type Answers = Record<string, string | string[] | boolean>;

const AVAILABILITY: Field = {
  name: "availability",
  label: "When are you usually free?",
  kind: "choices",
  required: true,
  options: ["Weekday mornings", "Weekday evenings", "Saturday", "Sunday"],
};

/** Asked of everyone, ministry or sacrament. */
function personalStep(kind: "ministry" | "sacrament"): Step {
  return {
    title: "Your details",
    blurb: kind === "ministry"
      ? "So the ministry coordinator can reach you."
```

```
      : "So the parish office can reach you about the arrangements.",
    fields: [
      {
        name: "fullName",
        label: "Full name",
        kind: "text",
        required: true,
        help: "As it should appear on parish records.",
        maxLength: 80,
      },
      { name: "mobile", label: "Contact number", kind: "tel", required: true, placeholder: "09XX XXX XXXX" },
      { name: "address", label: "Address", kind: "text", required: false, maxLength: 120 },
      { name: "birthDate", label: "Date of birth", kind: "date", required: false },
    ],
  };
}

const EMERGENCY_STEP: Step = {
  title: "Someone we can call",
  blurb: "Only used if something happens while you are serving.",
  fields: [
    { name: "emergencyName", label: "Name", kind: "text", required: false, maxLength: 80 },
    { name: "emergencyRelation", label: "Relationship to you", kind: "text", required: false, maxLength: 40 },
    { name: "emergencyMobile", label: "Contact number", kind: "tel", required: false },
  ],
};

/**
 * The questions that belong to one ministry rather than to all of them.
 *
 * Keyed by id, so renaming a ministry cannot silently drop its questions.
 * A ministry with no entry gets the common step and nothing more, which
 * is the right default - an empty object here means "nothing special to
 * ask", not "unfinished".
 */
const MINISTRY_FIELDS: Record<string, Field[]> = {
  "min-altar-servers": [
    {
      name: "servedBefore",
      label: "Have you served at the altar before?",
      kind: "textarea",
      required: false,
      help: "Where, and for roughly how long. Leave blank if this would be your first time.",
      maxLength: 300,
    },
  ],
  "min-lectors": [
    {
      name: "languages",
      label: "Which languages can you read aloud confidently?",
      kind: "choices",
      required: true,
      options: ["Filipino", "English"],
    },
    {
      name: "readingExperience",
      label: "Any experience reading or speaking in public?",
      kind: "textarea",
      required: false,
```

```
      maxLength: 300,
    },
  ],
  "min-emhc": [
    {
      name: "homeVisits",
      label: "Could you bring Communion to the sick at home?",
      kind: "choices",
      required: false,
      options: ["Yes", "Not at the moment"],
    },
  ],
};

/** Choirs ask the same things, and there are several of them. */
const CHOIR_FIELDS: Field[] = [
  {
    name: "voicePart",
    label: "Which part do you usually sing?",
    kind: "choices",
    required: false,
    options: ["Soprano", "Alto", "Tenor", "Bass", "Not sure"],
  },
  {
    name: "instrument",
    label: "Do you play an instrument?",
    kind: "text",
    required: false,
    placeholder: "Guitar, keyboard, none…",
    maxLength: 60,
  },
  {
    name: "singingExperience",
    label: "Any singing or musical experience?",
    kind: "textarea",
    required: false,
    maxLength: 300,
  },
];

function ministrySpecific(ministryId: string, ministryName: string): Field[] {
  if (MINISTRY_FIELDS[ministryId]) return MINISTRY_FIELDS[ministryId];
  // Matched on the name as well as the id, because the parish has
  // several choirs and they were added at different times under
  // different id conventions.
  if (/cho(ir|rale)/i.test(ministryName)) return CHOIR_FIELDS;
  return [];
}

export function ministrySteps(ministryId: string, ministryName: string): Step[] {
  return [
    personalStep("ministry"),
    {
      title: `About joining ${ministryName}`,
      fields: [
        {
          name: "message",
          label: "Why would you like to join?",
          kind: "textarea",
```

```
          required: true,
          help: "A sentence or two is plenty.",
          maxLength: 500,
        },
        ...ministrySpecific(ministryId, ministryName),
        AVAILABILITY,
      ],
    },
    EMERGENCY_STEP,
    {
      title: "Before you send it",
      fields: [
        {
          name: "consent",
          label: "The parish may keep these details and contact me about this ministry.",
          kind: "confirm",
          required: true,
        },
      ],
    },
  ];
}

export function sacramentSteps(sacramentName: string): Step[] {
  return [
    personalStep("sacrament"),
    {
      title: `About the ${sacramentName}`,
      blurb: "The parish office will confirm everything with you before anything is fixed.",
      fields: [
        {
          name: "preferredDate",
          label: "Preferred date",
          kind: "date",
          required: true,
          help: "A request, not a booking — the office will confirm what is possible.",
        },
        {
          name: "sponsorOrParent",
          label: "Parent, sponsor or the other party",
          kind: "text",
          required: false,
          help: "Whoever else this concerns, if anyone.",
          maxLength: 120,
        },
        {
          name: "notes",
          label: "Anything the office should know?",
          kind: "textarea",
          required: false,
          maxLength: 500,
        },
      ],
    },
    {
      title: "Documents",
      blurb: "You will bring the originals to the parish office. This only tells them what you already have.",
      fields: [
        {
```

```
          name: "documentsReady",
          label: "Which of these do you already have?",
          kind: "choices",
          required: false,
          options: [
            "PSA birth certificate",
            "Baptismal certificate",
            "Confirmation certificate",
            "Marriage contract",
            "None of these yet",
          ],
        },
      ],
    },
    {
      title: "Before you send it",
      fields: [
        {
          name: "consent",
          label: "The parish may keep these details and contact me about this application.",
          kind: "confirm",
          required: true,
        },
      ],
    },
  ];
}

/**
 * What is missing from one step, keyed by field name.
 *
 * Returns per-field messages rather than one summary, so each one can sit
 * under the field it is about. A list at the top of a four-field form
 * makes someone count down the form to find which box is wrong.
 */
export function validateStep(step: Step, answers: Answers): Record<string, string> {
  const errors: Record<string, string> = {};
  for (const field of step.fields) {
    if (!field.required) continue;
    const value = answers[field.name];

    if (field.kind === "confirm") {
      if (value !== true) errors[field.name] = "Please tick this to continue.";
      continue;
    }
    if (field.kind === "choices") {
      if (!Array.isArray(value) || value.length === 0) {
        errors[field.name] = `Please choose at least one.`;
      }
      continue;
    }
    if (typeof value !== "string" || value.trim() === "") {
      errors[field.name] = `Please enter your ${field.label.toLowerCase()}.`;
      continue;
    }
    if (field.kind === "tel" && !isPlausiblePhone(value)) {
      errors[field.name] = "That does not look like a contact number.";
    }
  }
```

```
  return errors;
}

/**
 * Deliberately loose.
 *
 * Philippine mobiles are 11 digits starting 09, but people write them
 * with +63, with spaces, with dashes, and a parish would rather have a
 * landline than nothing. This rejects what is obviously not a number and
 * waves through everything else - a form that refuses a real person's
 * real number is worse than one that accepts an odd one.
 */
export function isPlausiblePhone(value: string): boolean {
  const digits = value.replace(/\D/g, "");
  return digits.length >= 7 && digits.length <= 15;
}

/** Every answer, flattened the way the parish's export expects to read it. */
export function toFormData(steps: Step[], answers: Answers): Record<string, unknown> {
  const out: Record<string, unknown> = {};
  for (const step of steps) {
    for (const field of step.fields) {
      const value = answers[field.name];
      if (value === undefined) continue;
      if (Array.isArray(value)) {
        if (value.length > 0) out[field.name] = value.join(", ");
        continue;
      }
      if (typeof value === "string") {
        if (value.trim()) out[field.name] = value.trim();
        continue;
      }
      out[field.name] = value;
    }
  }
  return out;
}

/** The answers as label/value pairs, for the review screen. */
export function reviewRows(steps: Step[], answers: Answers): Array<{ label: string; value: string }> {
  const rows: Array<{ label: string; value: string }> = [];
  for (const step of steps) {
    for (const field of step.fields) {
      if (field.kind === "confirm") continue;
      const value = answers[field.name];
      const text = Array.isArray(value) ? value.join(", ")
        : typeof value === "string" ? value.trim()
        : "";
      // Blank optional answers are left out rather than shown as "—".
      // A review screen is for checking what you said, not for counting
      // what you did not.
      if (text) rows.push({ label: field.label, value: text });
    }
  }
  return rows;
}
```

1.4.10 APP-080 — applicationView.ts

Code ID	APP-080
File Name	applicationView.ts
File Path	SanctDemo/src/lib/applicationView.ts
Language / Type	TypeScript
Lines	146
Functions / Classes	ViewApplication (exported type); normaliseStatus (exported function); readableDate (internal function); toViewApplication (exported function); sortByNewest (exported function)
Related Files (its imports)	../types, ./applications
Where It Is Used	src/components/MyApplications.tsx, src/components/MyApplicationsPage.tsx
How to Recognize It	Open SanctDemo/src/lib/applicationView.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — applicationView.ts

```
import type { ApplicationStatus } from "../types";
import { STATUS_LABEL } from "./applications";

/**
 * One shape for a pilgrim's submissions, old and new.
 *
 * ## Why a normaliser exists at all
 *
 * The applications collection has two generations in it. The new ones carry
 * `kind`, `referenceNumber`, a lowercase status from a fixed set, and
 * structured `formData`. The ones already in the database carry `type:
 * "Ministry Application"`, a free-text status like "Awaiting Parish
 * Interview", and the answers flattened into a `details` sentence.
 *
 * Migrating them would mean a server pass over documents belonging to real
 * people, on a plan with no Cloud Functions, to change rows a parish may
 * already have acted on. Reading both is cheaper and loses nothing: the old
 * rows are finite and stop appearing once they are closed.
 */

export interface ViewApplication {
  id: string;
  /** "Baptism", or the ministry's name. */
  title: string;
  kind: "sacrament" | "ministry" | "other";
  status: ApplicationStatus | "unknown";
  /** What to print for the status, including an old free-text one. */
  statusLabel: string;
  referenceNumber?: string;
  /** Already formatted for reading. */
```

```
  submitted: string;
  /** When the parish last touched it, if they have. */
  lastUpdate?: string;
  /** The office's note, when there is one. */
  note?: string;
  /** Legacy summary line, shown only when there is nothing better. */
  details?: string;
}

/** A record as it may arrive from Firestore: either generation. */
type RawApplication = Record<string, unknown> & { id?: string };

const KNOWN: ApplicationStatus[] = [
  "pending", "under_review", "approved", "rejected", "completed",
];

/**
 * A free-text status as one of the five, where that is honest.
 *
 * "Awaiting" is tested before "interview" on purpose: "Awaiting Parish
 * Interview" means the parish has not acted, which is pending. Matching
 * "interview" first would report it as already under review and tell the
 * applicant something untrue.
 *
 * Anything unrecognised stays "unknown" and is printed as written, rather
 * than being guessed into a colour that implies an outcome.
 */
export function normaliseStatus(raw: unknown): { status: ApplicationStatus | "unknown"; label: string } {
  const text = String(raw ?? "").trim();
  if (!text) return { status: "unknown", label: "Submitted" };

  const exact = text.toLowerCase().replace(/[\s-]+/g, "_");
  if ((KNOWN as string[]).includes(exact)) {
    return { status: exact as ApplicationStatus, label: STATUS_LABEL[exact as ApplicationStatus] };
  }

  const lower = text.toLowerCase();
  if (lower.includes("declin") || lower.includes("reject")) return { status: "rejected", label: text };
  if (lower.includes("complet")) return { status: "completed", label: text };
  if (lower.includes("approve")) return { status: "approved", label: text };
  if (lower.includes("pending") || lower.includes("awaiting")) return { status: "pending", label: text };
  if (lower.includes("review") || lower.includes("interview") || lower.includes("schedul")) {
    return { status: "under_review", label: text };
  }
  return { status: "unknown", label: text };
}

function readableDate(value: unknown): string {
  const text = String(value ?? "").trim();
  if (!text) return "";
  const d = new Date(text);
  // Old rows store a localised date string, which is already readable and
  // does not survive being parsed and reformatted.
  if (Number.isNaN(d.getTime())) return text;
  return d.toLocaleDateString(undefined, { year: "numeric", month: "long", day: "numeric" });
}

export function toViewApplication(raw: RawApplication): ViewApplication {
  const kindField = raw.kind;
```

```
  const kind: ViewApplication["kind"] =
    kindField === "sacrament" || kindField === "ministry"
      ? kindField
      : raw.type === "Ministry Application"
        ? "ministry"
        : raw.type === "Sacrament Booking"
          ? "sacrament"
          : "other";

  const { status, label } = normaliseStatus(raw.status);

  // New rows name the thing in `type`; the oldest ministry rows put the
  // literal words "Ministry Application" there and the real name beside it.
  const title =
    (typeof raw.ministryName === "string" && raw.ministryName) ||
    (typeof raw.type === "string" && raw.type !== "Ministry Application" && raw.type) ||
    (typeof raw.details === "string" && raw.details) ||
    "Application";

  const history = Array.isArray(raw.history) ? raw.history : [];
  const last = history.length > 1 ? history[history.length - 1] : undefined;

  return {
    id: String(raw.id ?? ""),
    title: String(title),
    kind,
    status,
    statusLabel: label,
    referenceNumber: typeof raw.referenceNumber === "string" ? raw.referenceNumber : undefined,
    submitted: readableDate(raw.createdAt ?? raw.submittedAt ?? raw.date),
    lastUpdate: typeof raw.reviewedAt === "string"
      ? readableDate(raw.reviewedAt)
      : last && typeof (last as { at?: unknown }).at === "string"
        ? readableDate((last as { at: string }).at)
        : undefined,
    note: typeof raw.adminNote === "string" ? raw.adminNote : undefined,
    details: typeof raw.details === "string" ? raw.details : undefined,
  };
}

/**
 * Newest first, by the date shown.
 *
 * Old rows carry a localised date string that cannot be compared, so they
 * fall to the end rather than being interleaved wrongly - which is the
 * honest ordering when the data cannot say.
 */
export function sortByNewest(list: ViewApplication[]): ViewApplication[] {
  return [...list].sort((a, b) => {
    const da = Date.parse(a.submitted);
    const dbb = Date.parse(b.submitted);
    if (Number.isNaN(da) && Number.isNaN(dbb)) return 0;
    if (Number.isNaN(da)) return 1;
    if (Number.isNaN(dbb)) return -1;
    return dbb - da;
  });
}
```

1.4.11 APP-081 — applications.ts

Code ID	APP-081
File Name	applications.ts
File Path	SanctDemo/src/lib/applications.ts
Language / Type	TypeScript
Lines	190
Functions / Classes	STATUS_ORDER (exported constant); STATUS_LABEL (exported constant); referenceFor (exported function); SubmitError (exported class); SubmitInput (exported type); submitApplication (exported function); setApplicationStatus (exported function); isClosed (exported function)
Related Files (its imports)	./firebase, ./userProfile, ./notifications, ./applicationFilters, ../types
Where It Is Used	src/admin/AdminApplicationDetail.tsx, src/admin/AdminApplications.tsx, src/admin/StatusBadge.tsx, src/components/ApplicationForm.tsx, src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx, src/lib/applicationView.ts, src/lib/exportApplications.ts, src/lib/notifications.ts
How to Recognize It	Open SanctDemo/src/lib/applications.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — applications.ts

```
import { addDoc, collection, doc, updateDoc, arrayUnion } from "firebase/firestore";
import { auth, db } from "./firebase";
import { getProfile } from "./userProfile";
import { notifySubmitted, notifyStatusChanged } from "./notifications";
import { isArchived } from "./applicationFilters";
import type { ApplicationDoc, ApplicationStatus } from "../types";

/**
 * Submitting and deciding applications.
 *
 * ## Why every submission goes through here
 *
 * The security rules refuse an application unless it carries the submitter's
 * own uid, a status of exactly "pending", a kind of "sacrament" or
 * "ministry", and a churchId equal to the one on the submitter's profile.
 * Four conditions, and a form that gets any of them wrong fails at the last
 * moment in front of the person filling it in. One place builds the
 * document so there is one place that has to be right.
 *
 * ## Why churchId is read from the profile and not passed in
 *
 * The rules compare it against the profile anyway, so a caller that passed
 * a different one would simply be refused. Reading it here means the caller
 * cannot be wrong, rather than being wrong and finding out late.
 */
```

```
/** Statuses, in the order the parish works through them. */
export const STATUS_ORDER: ApplicationStatus[] = [
  "pending", "under_review", "approved", "rejected", "completed",
];

export const STATUS_LABEL: Record<ApplicationStatus, string> = {
  pending: "Pending",
  under_review: "Under review",
  approved: "Approved",
  rejected: "Rejected",
  completed: "Completed",
};

/**
 * A reference the applicant can quote on the phone.
 *
 * Six random characters, not a running number. A sequence like 000123 needs
 * a counter that two submissions cannot both read at once, which means a
 * transaction on a shared document or a server - and the server is Cloud
 * Functions, which needs the Blaze plan this project is not on. A random
 * tail cannot collide in practice at parish volumes and cannot be guessed
 * at, which matters more here than being able to tell that yours was the
 * hundred-and-twenty-third.
 *
 * Ambiguous characters are left out so the reference survives being read
 * aloud and written on paper.
 */
const ALPHABET = "ACDEFGHJKLMNPQRTUVWXY349";

export function referenceFor(kind: "sacrament" | "ministry", when: Date = new Date()): string {
  const prefix = kind === "sacrament" ? "SAC" : "MIN";
  let tail = "";
  const bytes = new Uint8Array(6);
  crypto.getRandomValues(bytes);
  for (const b of bytes) tail += ALPHABET[b % ALPHABET.length];
  return `${prefix}-${when.getFullYear()}-${tail}`;
}

export class SubmitError extends Error {}

export interface SubmitInput {
  kind: "sacrament" | "ministry";
  /**
   * Which ministry or sacrament, by id - "min-altar-servers",
   * "sac-baptism".
   *
   * Required, and required by the security rules too: it is what the rule
   * checks against the parish's closed list, and an application that
   * could omit it could be submitted to a ministry the parish had shut.
   * The display name cannot do this job - it is editable text, and
   * renaming a ministry would silently reopen it.
   */
  itemId: string;
  /** "Baptism", or the ministry's name. Shown in the parish's list. */
  type: string;
  applicantName: string;
  /** Everything the form asked that is not one of the fields above. */
  formData?: Record<string, unknown>;
}
```

```
/**
 * Writes an application for the signed-in user, to their own parish.
 *
 * Throws rather than returning an error object: every caller here is a
 * form submit handler that has to stop and say something, and a thrown
 * error is harder to forget to check than a returned one.
 */
export async function submitApplication(input: SubmitInput): Promise<ApplicationDoc> {
  const user = auth.currentUser;
  if (!user) {
    throw new SubmitError("Please sign in first, so the parish can reply to you.");
  }

  const profile = await getProfile(user.uid);
  if (!profile?.churchId) {
    throw new SubmitError(
      "Your account is not linked to a parish yet, so there is nowhere to send this. " +
      "Please contact your parish office.",
    );
  }

  if (!input.itemId) {
    // A caller bug, not something a pilgrim can cause. Caught here with a
    // sentence rather than at the database with "permission denied".
    throw new SubmitError("This application form is misconfigured. Please tell the parish office.");
  }

  const now = new Date();
  const document: ApplicationDoc = {
    uid: user.uid,
    itemId: input.itemId,
    applicantName: input.applicantName.trim() || profile.fullName || "",
    applicantEmail: profile.email || user.email || "",
    churchId: profile.churchId,
    kind: input.kind,
    type: input.type,
    status: "pending",
    referenceNumber: referenceFor(input.kind, now),
    formData: input.formData ?? {},
    createdAt: now.toISOString(),
    history: [{ status: "pending", at: now.toISOString() }],
  };

  const ref = await addDoc(collection(db, "applications"), document);
  // A record of the submission, for the pilgrim's own list. Written after
  // the application so a failure here cannot lose the application itself.
  await notifySubmitted(document.churchId, ref.id, document.type, document.referenceNumber);
  return { ...document, id: ref.id };
}

/**
 * Records a parish's decision.
 *
 * Appends to `history` rather than replacing it, with arrayUnion so two
 * reviewers acting at once cannot erase each other's entry. The rules allow
 * exactly these fields and nothing else, so an attempt to "correct" the
 * applicant's answers while approving is refused by the database, not by
 * this function.
```

```
 */
export async function setApplicationStatus(
  applicationId: string,
  status: ApplicationStatus,
  note?: string,
  /**
   * Who to tell, and about what. Passed in rather than re-read here: the
   * caller is already holding the document, and a second read would be a
   * second chance for the two to disagree.
   */
  applicant?: { uid: string; churchId: string; what: string },
): Promise<void> {
  const user = auth.currentUser;
  if (!user) throw new SubmitError("You are not signed in.");

  const at = new Date().toISOString();
  await updateDoc(doc(db, "applications", applicationId), {
    status,
    reviewedBy: user.uid,
    reviewedAt: at,
    updatedAt: at,
    ...(note ? { adminNote: note } : {}),
    history: arrayUnion({ status, at, by: user.uid, ...(note ? { note } : {}) }),
  });

  if (applicant) {
    await notifyStatusChanged(
      applicant.uid, applicant.churchId, applicationId, applicant.what, status, note,
    );
  }
}

/**
 * True for the statuses that end an application's life.
 *
 * Used to decide whether to offer the decision buttons at all. Re-approving
 * something already completed is not a thing the parish means to do.
 *
 * Delegates rather than repeating the list. The same two statuses are what
 * moves an application into the admin's archive, and two copies of that
 * rule would eventually disagree about one of them - leaving a row the
 * table files as finished but the detail page still offers to decide.
 */
export function isClosed(status: ApplicationStatus): boolean {
  return isArchived(status);
}
```

1.4.12 APP-082 — arApp.ts

Code ID	APP-082
File Name	arApp.ts
File Path	SanctDemo/src/lib/arApp.ts
Language / Type	TypeScript
Lines	136
Functions / Classes	AR_APP_SCHEME (exported constant); AR_APP_HELP (exported constant); ArAvailability (exported type); arAvailability (exported function); arTourUrl (exported function); AR_SALA_PREVIEW (exported constant); ArLaunchTarget (exported type); arLaunchParish (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/arApp.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Handoff from SanctiWalk to the native AR tour app.
>
> The AR tour is a separate Unity/ARCore application, because the browser cannot do what it does: ARCore's world tracking keeps content anchored to a place after the trigger image has left the camera, and no web API on iOS or Android offers that today. SanctiWalk stays a web app and launches it.
>
> Pure and dependency-free so the URL building and platform rules can be tested without a browser.

Complete Source Code — arApp.ts

```
// Handoff from SanctiWalk to the native AR tour app.
//
// The AR tour is a separate Unity/ARCore application, because the browser
// cannot do what it does: ARCore's world tracking keeps content anchored to a
// place after the trigger image has left the camera, and no web API on iOS or
// Android offers that today. SanctiWalk stays a web app and launches it.
//
// Pure and dependency-free so the URL building and platform rules can be
// tested without a browser.

/** Custom scheme registered by the AR app's Android manifest. */
export const AR_APP_SCHEME = 'sanctiwalkar'

/** Where to send someone whose phone cannot run it. */
export const AR_APP_HELP = 'https://github.com/Gifeys/SanctDemo'

export type ArAvailability =
  | { kind: 'ready' }
  | { kind: 'ios'; reason: string }
  | { kind: 'desktop'; reason: string }
```

```
/**
 * Whether the AR app can be launched from this device.
 *
 * Deliberately honest about iOS. The AR app is an Android build: an iOS
 * version needs Xcode on macOS to compile, which the project does not have
 * yet. Offering the button on an iPhone would open nothing and look broken,
 * so it says why instead.
 */
export function arAvailability(
  userAgent: string,
  /**
   * Capacitor's own answer, when there is one: 'android', 'ios' or 'web'.
   *
   * The user agent is a guess at what the device is, and inside the
   * Android WebView it is a guess that can be wrong - a shell that
   * trims or rewrites it leaves no 'android' to find, and SanctiWalk
   * then tells somebody holding an Android phone that their Android
   * phone cannot run the Android app. Capacitor is not guessing: it
   * knows which shell it is running in. The user agent stays as the
   * fallback for the browser, where there is no Capacitor.
   */
  platform?: string,
): ArAvailability {
  if (platform === 'android') return { kind: 'ready' }
  if (platform === 'ios') {
    return {
      kind: 'ios',
      reason: 'The AR tour is an Android app for now. An iPhone version needs a Mac to build.',
    }
  }

  const ua = userAgent.toLowerCase()

  if (/iphone|ipad|ipod/.test(ua)) {
    return {
      kind: 'ios',
      reason: 'The AR tour is an Android app for now. An iPhone version needs a Mac to build.',
    }
  }

  if (!/android/.test(ua)) {
    return {
      kind: 'desktop',
      reason: 'Open SanctiWalk on an Android phone to start the AR tour.',
    }
  }

  return { kind: 'ready' }
}

/**
 * The launch URL for a parish's AR tour.
 *
 * The parish id travels with the link so the AR app knows whose stations to
 * load — the same id used everywhere else in SanctiWalk, so nothing has to be
 * mapped or translated between the two applications.
 */
export function arTourUrl(parishId: string): string {
```

```
  return `${AR_APP_SCHEME}://tour?church=${encodeURIComponent(parishId)}`
}

/**
 * The sala stand-in the AR app ships for testing.
 *
 * It is the same four models on the scan of the proponents' sala, and the
 * AR app already knows this id; nothing new is installed to use it.
 */
export const AR_SALA_PREVIEW = 'route-mhcp-preview'

export interface ArLaunchTarget {
  parishId: string
  /** The simulator setting, from simulatorMode.ts. */
  simulatorMode: boolean
  /** True when presence says the pilgrim is inside a parish's geofence. */
  atParish: boolean
}

/**
 * The parish whose stations the AR app should load.
 *
 * The one the pilgrim chose. Always.
 *
 * ## What this used to do, and why it was wrong
 *
 * It substituted the sala preview whenever the simulator was off and
 * presence could not confirm the pilgrim was inside a geofence. The
 * intention was kind - opening the tour at a desk localizes against a
 * church kilometres away, which made Start AR look broken while we
 * were building it.
 *
 * The cost landed on the person it was meant to help. GPS indoors, in
 * a concrete church, is exactly where a geofence fails; so somebody
 * standing in Mary Help of Christians, in front of the marker, tapped
 * Start AR and was handed a scan of the proponents' living room. At
 * San Roque the same thing, and the id it substituted was the MHCP
 * preview - so the wrong parish, not merely the wrong room.
 *
 * ## Why no substitution is also the safer failure
 *
 * The AR app cannot start until it reads a printed marker. Away from
 * the parish there is no marker, so the honest outcome of launching
 * the real parish at a desk is an app that waits and says it is
 * waiting. That is self-explanatory. Being shown a different church's
 * stations is not, and it cannot be told apart from the app being
 * broken - which is precisely how it was read.
 *
 * The sala is still reachable: AR_SALA_PREVIEW is a parish id like any
 * other and can be chosen on purpose. What it can no longer do is
 * arrive uninvited in place of the parish somebody picked.
 */
export function arLaunchParish(
  { parishId }: ArLaunchTarget,
): string {
  return parishId
}
```

1.4.13 APP-083 — askSancti.ts

Code ID	APP-083
File Name	askSancti.ts
File Path	SanctDemo/src/lib/askSancti.ts
Language / Type	TypeScript
Lines	84
Functions / Classes	AskRequest (exported type); AskReply (exported type); askSancti (exported function)
Related Files (its imports)	./apiBase, ./appKey
Where It Is Used	src/components/SanctiHost.tsx
How to Recognize It	Open SanctDemo/src/lib/askSancti.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The client's side of the grounded assistant.
>
> ## Why this can always fail, and why that is fine
>
> Sancti was built to run on the device precisely because the Express backend is not deployed, and an assistant whose first act is to fail at a dead URL is worse than no assistant. That reasoning has not changed; this does not replace the on-device matcher, it is only reached for the questions the matcher could not place, and every failure here returns null so the caller falls back to what it had.
>
> So: a short timeout, no retries, no error shown. A pilgrim asking a question should never be left watching a spinner because a server is asleep, and should never be told about a backend they did not know existed.

Complete Source Code — askSancti.ts

```
/**
 * The client's side of the grounded assistant.
 *
 * ## Why this can always fail, and why that is fine
 *
 * Sancti was built to run on the device precisely because the Express
 * backend is not deployed, and an assistant whose first act is to fail
 * at a dead URL is worse than no assistant. That reasoning has not
 * changed; this does not replace the on-device matcher, it is only
 * reached for the questions the matcher could not place, and every
 * failure here returns null so the caller falls back to what it had.
 *
 * So: a short timeout, no retries, no error shown. A pilgrim asking a
 * question should never be left watching a spinner because a server is
 * asleep, and should never be told about a backend they did not know
 * existed.
 */
import { apiUrl } from './apiBase'
```

```
import { withAppKey } from './appKey'

/**
 * Short on purpose. Past a few seconds the pilgrim has already decided
 * the app is broken, and the on-device answer was available the whole
 * time.
 */
const TIMEOUT_MS = 8000

export interface AskRequest {
  question: string
  parishId?: string
  parishName?: string
  history?: Array<{ role: string; text: string }>
}

export interface AskReply {
  answer: string
  /** True when the answer rested on at least one verified parish fact. */
  grounded: boolean
  sources: Array<{ topic: string; source: 'parish' | 'app'; updatedAt: string | null }>
  gaps: string[]
}

/**
 * Ask the grounded assistant, or return null.
 *
 * Null covers every failure - no backend, a timeout, a 503, a malformed
 * body - because the caller does the same thing in all of them. It
 * never throws and never returns a fabricated answer.
 */
export async function askSancti(
  req: AskRequest,
  fetchImpl: typeof fetch = fetch,
): Promise<AskReply | null> {
  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), TIMEOUT_MS)

  try {
    const res = await fetchImpl(apiUrl('/api/ask'), {
      method: 'POST',
      headers: withAppKey({ 'Content-Type': 'application/json' }),
      body: JSON.stringify(req),
      signal: controller.signal,
    })
    if (!res.ok) return null

    const data = await res.json()
    const answer = typeof data?.answer === 'string' ? data.answer.trim() : ''
    // An empty answer is a failure wearing a 200. Falling back to the
    // on-device reply is better than showing a blank message bubble.
    if (!answer) return null

    return {
      answer,
      grounded: Boolean(data?.grounded),
      sources: Array.isArray(data?.sources) ? data.sources : [],
      gaps: Array.isArray(data?.gaps) ? data.gaps : [],
    }
```

```
  } catch {
    // Offline, aborted, non-JSON body - all the same outcome.
    return null
  } finally {
    clearTimeout(timer)
  }
}
```

authFlow.ts — listed in full as API-003.

1.4.14 APP-084 — availability.ts

Code ID	APP-084
File Name	availability.ts
File Path	SanctDemo/src/lib/availability.ts
Language / Type	TypeScript
Lines	102
Functions / Classes	closedIds (exported function); isOpenForApplications (exported function); withAvailability (exported function); AvailabilitySummary (exported type); summarise (exported function); closedMessage (exported function)
Related Files (its imports)	./parishContent
Where It Is Used	src/admin/AdminAvailability.tsx, src/admin/AdminItems.tsx, src/admin/ParishStatus.tsx, src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx, src/components/SanctiHost.tsx, src/lib/useOpeningNotices.ts
How to Recognize It	Open SanctDemo/src/lib/availability.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — availability.ts

```
import type { ParishContent } from "./parishContent";

/**
 * Whether a parish is currently accepting applications for a ministry or
 * a sacrament.
 *
 * ## Why a list of CLOSED ids and not a flag on each one
 *
 * Three reasons, and they all point the same way.
 *
 * Absent means open. The ministries and sacraments are compiled into
 * data.ts and 29 of the 31 parishes have no parishContent document at
 * all. A `Record<id, boolean>` would have to be seeded for every parish
 * before anything worked, and a parish nobody had seeded would read as
 * "false everywhere" - every ministry closed, on the day this shipped.
 * A list of exceptions is empty by default, and empty means normal.
 *
 * It is one field, so the security rules can enforce it. The rule on
 * creating an application has to answer "is this one closed" in a single
 * expression over one document; `itemId in closedApplications` does that.
 * Two maps would need two gets and a branch on kind.
 *
 * It says what the parish actually did. A parish closes the choir for
 * Lent; it does not "set thirty booleans, one of them false".
 *
 * ## Why ids and not names
```

```
 *
 * A ministry's name is displayed text and has been edited before -
 * "Lectors" became "Ministry of Lectors and Commentators (MLC)". Keying
 * off the name would have silently reopened it.
 */

/** Nothing is closed unless the parish has said so. */
export function closedIds(content: ParishContent | null | undefined): string[] {
  return content?.closedApplications ?? [];
}

/**
 * True when a pilgrim may apply for this ministry or sacrament.
 *
 * The one function the whole feature turns on. The Apply button, the
 * badge, the admin's toggle and the security rule all have to agree, so
 * they all come through here.
 */
export function isOpenForApplications(
  content: ParishContent | null | undefined,
  itemId: string,
): boolean {
  return !closedIds(content).includes(itemId);
}

/**
 * The list with one id closed or opened, for the admin's toggle.
 *
 * Returns a new array and keeps it sorted and free of duplicates: it is
 * written straight to Firestore, and a field that grows a second copy of
 * the same id every time someone double-taps is a field that eventually
 * cannot be read back.
 */
export function withAvailability(
  current: string[],
  itemId: string,
  open: boolean,
): string[] {
  const next = new Set(current);
  if (open) next.delete(itemId);
  else next.add(itemId);
  return [...next].sort();
}

export interface AvailabilitySummary {
  open: number;
  closed: number;
}

/** For the admin dashboard's parish-status panel. */
export function summarise(
  content: ParishContent | null | undefined,
  itemIds: string[],
): AvailabilitySummary {
  let open = 0;
  let closed = 0;
  for (const id of itemIds) {
    if (isOpenForApplications(content, id)) open++;
    else closed++;
```

```
  }
  return { open, closed };
}

/**
 * What the user app says in place of "Apply".
 *
 * Deliberately not "Disabled" or "Unavailable" on its own. A pilgrim
 * reading "Not available" on a ministry needs to know it is the
 * APPLICATIONS that are closed and that this is temporary - otherwise it
 * reads as the ministry having been disbanded.
 */
export function closedMessage(kind: "ministry" | "sacrament", name: string): string {
  return kind === "ministry"
    ? `Applications for ${name} are currently closed. Please check again later, or contact the parish office.`
    : `${name} applications are currently unavailable. Please check again later, or contact the parish office.`;
}
```

1.4.15 APP-085 — avatar.ts

Code ID	APP-085
File Name	avatar.ts
File Path	SanctDemo/src/lib/avatar.ts
Language / Type	TypeScript
Lines	119
Functions / Classes	AVATAR_PX (exported constant); MAX_STORED_BYTES (exported constant); AvatarError (exported class); fileToAvatarDataUrl (exported function); loadImage (internal function); loadViaElement (internal function); initialsFor (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/EditProfileCard.tsx
How to Recognize It	Open SanctDemo/src/lib/avatar.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turning a chosen photo into a small square avatar.
>
> ## Why this produces a data URL and not a Storage upload
>
> Firebase Storage is not set up on this project - `firebase deploy --only storage` refuses with "Firebase Storage has not been set up", and nothing can be uploaded until someone presses Get Started in the console. Waiting for that would mean shipping an avatar feature that silently fails for everyone.
>
> So the image is resized here, hard, and kept as a data URL on the user's own document. A 256px JPEG at quality 0.82 lands around 15-25 KB, comfortably inside Firestore's 1 MB document limit, and it is only ever read by the person it belongs to and their parish admin.
>
> This is a deliberate stopgap, not a design. When Storage is enabled the right shape is an upload to `avatars/{uid}` and a real URL in `photoUrl` - which is why the field is called photoUrl rather than photoData, so the swap costs nothing above this module.
>
> ## Why it resizes rather than validating and rejecting
>
> A phone camera produces a 4 MB, 4000px JPEG. Telling someone their own photograph is too big, when the app could simply make it smaller, is work handed back to the person for the app's convenience.

Complete Source Code — avatar.ts

```
/**
 * Turning a chosen photo into a small square avatar.
 *
 * ## Why this produces a data URL and not a Storage upload
 *
 * Firebase Storage is not set up on this project - `firebase deploy
```

```
 * --only storage` refuses with "Firebase Storage has not been set up",
 * and nothing can be uploaded until someone presses Get Started in the
 * console. Waiting for that would mean shipping an avatar feature that
 * silently fails for everyone.
 *
 * So the image is resized here, hard, and kept as a data URL on the
 * user's own document. A 256px JPEG at quality 0.82 lands around 15-25 KB,
 * comfortably inside Firestore's 1 MB document limit, and it is only ever
 * read by the person it belongs to and their parish admin.
 *
 * This is a deliberate stopgap, not a design. When Storage is enabled the
 * right shape is an upload to `avatars/{uid}` and a real URL in
 * `photoUrl` - which is why the field is called photoUrl rather than
 * photoData, so the swap costs nothing above this module.
 *
 * ## Why it resizes rather than validating and rejecting
 *
 * A phone camera produces a 4 MB, 4000px JPEG. Telling someone their own
 * photograph is too big, when the app could simply make it smaller, is
 * work handed back to the person for the app's convenience.
 */

/** The stored square's edge, in pixels. Retina-sharp at the 64px it displays. */
export const AVATAR_PX = 256;

/** Refused above this. A Firestore document cannot exceed 1 MB in total. */
export const MAX_STORED_BYTES = 180_000;

export class AvatarError extends Error {}

/**
 * Reads an image file and returns a square JPEG data URL.
 *
 * The crop is centred and takes the largest square that fits, rather than
 * squashing the picture to fit a square box - a stretched face is worse
 * than a cropped one.
 */
export async function fileToAvatarDataUrl(file: File): Promise<string> {
  if (!file.type.startsWith("image/")) {
    throw new AvatarError("That file is not an image. Please choose a photo.");
  }

  const bitmap = await loadImage(file);
  const side = Math.min(bitmap.width, bitmap.height);
  const sx = (bitmap.width - side) / 2;
  const sy = (bitmap.height - side) / 2;

  const canvas = document.createElement("canvas");
  canvas.width = AVATAR_PX;
  canvas.height = AVATAR_PX;
  const ctx = canvas.getContext("2d");
  if (!ctx) throw new AvatarError("This browser could not process the image.");

  ctx.drawImage(bitmap, sx, sy, side, side, 0, 0, AVATAR_PX, AVATAR_PX);
  if ("close" in bitmap) (bitmap as ImageBitmap).close?.();

  // JPEG, not PNG: a photograph as PNG is several times larger for no
  // visible gain, and the budget here is a document field.
  let quality = 0.82;
```

```
  let url = canvas.toDataURL("image/jpeg", quality);

  // Busy photographs compress worse. Step down rather than refusing a
  // picture that is only slightly over.
  while (url.length > MAX_STORED_BYTES && quality > 0.4) {
    quality -= 0.12;
    url = canvas.toDataURL("image/jpeg", quality);
  }

  if (url.length > MAX_STORED_BYTES) {
    throw new AvatarError("That photo is too detailed to store. Please try another.");
  }
  return url;
}

function loadImage(file: File): Promise<ImageBitmap | HTMLImageElement> {
  // createImageBitmap handles EXIF orientation, so a photo taken in
  // portrait does not arrive on its side.
  if (typeof createImageBitmap === "function") {
    return createImageBitmap(file, { imageOrientation: "from-image" }).catch(() =>
      loadViaElement(file),
    );
  }
  return loadViaElement(file);
}

function loadViaElement(file: File): Promise<HTMLImageElement> {
  return new Promise((resolve, reject) => {
    const url = URL.createObjectURL(file);
    const img = new Image();
    img.onload = () => {
      URL.revokeObjectURL(url);
      resolve(img);
    };
    img.onerror = () => {
      URL.revokeObjectURL(url);
      reject(new AvatarError("That image could not be opened."));
    };
    img.src = url;
  });
}

/**
 * The initials to show when there is no photograph.
 *
 * Takes the first letter of the first two words, so "Juan Dela Cruz" is
 * JD rather than JDC, and a single name gives one letter rather than a
 * repeated one.
 */
export function initialsFor(name: string): string {
  const parts = name.trim().split(/[.\s_-]+/).filter(Boolean);
  if (parts.length === 0) return "P";
  return parts.slice(0, 2).map(p => p[0]!.toUpperCase()).join("");
}
```

1.4.16 APP-086 — churches.ts

Code ID	APP-086
File Name	churches.ts
File Path	SanctDemo/src/lib/churches.ts
Language / Type	TypeScript
Lines	91
Functions / Classes	LIVE_CHURCHES (exported constant); bundledChurches (internal function); listChurches (exported function); getChurch (exported function); churchName (exported function)
Related Files (its imports)	./firebase, ../types, ../data/diocese-parishes.json
Where It Is Used	src/admin/AdminApp.tsx, src/routes/SignUpFlow.tsx
How to Recognize It	Open SanctDemo/src/lib/churches.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — churches.ts

```
import { collection, doc, getDoc, getDocs } from "firebase/firestore";
import { db } from "./firebase";
import type { Church } from "../types";
import parishData from "../data/diocese-parishes.json";

/**
 * The churches an account can belong to.
 *
 * ## Why this reads Firestore and falls back to the bundled list
 *
 * The signup screen has to show a church list before anyone has signed in,
 * and the rules make churches/ publicly readable for exactly that reason.
 * But a brand-new project has an empty collection, and a signup screen with
 * no churches in it is a dead end that looks like a bug. So the diocese list
 * that already ships with the app is the floor: Firestore can add to it and
 * correct it, and until someone does, signup still works.
 *
 * The fallback carries no adminUid. Only a real Firestore document can say
 * who administers a church, because that is the field the whole
 * one-admin-per-church rule rests on, and a value invented client-side would
 * be a value the rules then refuse to honour.
 */

interface DioceseParish {
  id: string;
  name: string;
  vicariate: string;
  status: string;
}

const PARISHES = (parishData as { parishes: DioceseParish[] }).parishes;

/**
```

```
 * The two parishes with a route id, which is what a churchId has to be.
 *
 * parishContent is keyed by route id and the rules authorise that write with
 * `isAdminOf(parishId)`, so a church whose id is not the route id would have
 * an admin who cannot edit their own parish page.
 */
export const LIVE_CHURCHES: Church[] = [
  { id: "route-mhcp", name: "Mary Help of Christians Parish", location: "Maypajo, Caloocan City" },
  { id: "route-src", name: "San Roque Cathedral Parish", location: "Caloocan City" },
];

/** The bundled diocese list, as churches, for parishes with no route yet. */
function bundledChurches(): Church[] {
  const live = new Set(LIVE_CHURCHES.map(c => c.id));
  const rest = PARISHES
    .filter(p => !live.has(p.id))
    .map(p => ({ id: p.id, name: p.name, location: p.vicariate }));
  return [...LIVE_CHURCHES, ...rest];
}

/**
 * Every church a new account may choose from, Firestore first.
 *
 * A failed read is not an error here: churches/ is world-readable, so a
 * failure means the network, and the bundled list is a better answer than an
 * empty dropdown.
 */
export async function listChurches(): Promise<Church[]> {
  try {
    const snap = await getDocs(collection(db, "churches"));
    if (snap.empty) return bundledChurches();

    const fromDb = snap.docs.map(d => ({ id: d.id, ...(d.data() as Omit<Church, "id">) }));
    const seen = new Set(fromDb.map(c => c.id));
    // Bundled entries fill the gaps rather than replacing anything: a church
    // that exists in Firestore is the authoritative version of itself.
    return [...fromDb, ...bundledChurches().filter(c => !seen.has(c.id))];
  } catch {
    return bundledChurches();
  }
}

/** One church, or null. Used to show which parish an account belongs to. */
export async function getChurch(churchId: string): Promise<Church | null> {
  if (!churchId) return null;
  try {
    const snap = await getDoc(doc(db, "churches", churchId));
    if (snap.exists()) return { id: snap.id, ...(snap.data() as Omit<Church, "id">) };
  } catch {
    /* fall through to the bundled list */
  }
  return bundledChurches().find(c => c.id === churchId) ?? null;
}

/** The display name for a churchId, falling back to the id itself. */
export function churchName(churchId: string, churches: Church[]): string {
  return churches.find(c => c.id === churchId)?.name ?? churchId;
}
```

1.4.17 APP-087 — contentTranslations.ts

Code ID	APP-087
File Name	contentTranslations.ts
File Path	SanctDemo/src/lib/contentTranslations.ts
Language / Type	TypeScript
Lines	184
Functions / Classes	MINISTRY_DESCRIPTION_EN (exported constant); SACRAMENT_DESCRIPTION_FIL (exported constant); STATION_FIL (exported constant); bilingualFor (exported function); stationText (exported function)
Related Files (its imports)	./language
Where It Is Used	src/components/ArTour.tsx, src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/lib/contentTranslations.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — contentTranslations.ts

```
import type { Bilingual } from "./language";

/**
 * The second language for content that only ever had one.
 *
 * ## Why these live here and not in data.ts
 *
 * data.ts is what the parish supplied. These are translations OF that,
 * and the distinction matters: when the parish rewrites a ministry
 * description, the new text goes in data.ts and the stale translation
 * here is visibly next to it rather than buried in the same object.
 *
 * ## What a translation is and is not
 *
 * Every entry below carries the same claims as its source, in the other
 * language. Nothing is added, softened or explained - a description
 * that does not mention a requirement does not acquire one in
 * translation. Where a term is the Church's own and is used in Filipino
 * parishes in English ("Holy Communion", "liturgy"), it stays, because
 * translating it would be less faithful rather than more.
 *
 * The ministry names themselves are not translated at all. "Ministry of
 * Altar Servers (MAS)" is what the parish calls it on its own noticeboard
 * in either language.
 */

/** English for the fifteen ministry descriptions, which were Tagalog only. */
export const MINISTRY_DESCRIPTION_EN: Record<string, string> = {
  "min-altar-servers":
```

```
    "The Ministry of Altar Servers (MAS) answers the honourable duty of serving at the altar of the Lord. It is made up of young people of open heart and steady faith who serve willingly in the orderly and reverent celebration of the Sacred Liturgy and the other sacramental works of the Church. As servants of the altar, the ministry forms their discipline, holiness and proper bearing, so that they become living witnesses of order, respect and reverence within the church.",

  "min-emhc":
    "The Extraordinary Ministers of Holy Communion (EMHC) support the priest, especially at Masses with large congregations and when Communion must be brought to the sick, the elderly, or those confined at home. EMHCs are chosen carefully on the basis of faith, moral character and active sharing in the life of the Church, and their ministry is rooted in humility and service. As representatives of the community, they help ensure that the faithful receive the Body and Blood of Christ with dignity and devotion, and they extend the pastoral care of the Church to those who cannot attend Mass.",

  "min-lectors":
    "The Ministry of Lectors and Commentators (MLC) is devoted to the clear, dignified and meaningful proclamation of the Word of God and of the invitations within the sacred celebration. Through the careful reading of Holy Scripture and disciplined guidance of the assembly as commentators, this ministry helps the community take full part in the liturgy. In their preparation, discipline and respect for the holiness of the work, they become a channel of order, understanding, and deeper listening to the voice of God.",

  "min-greeters":
    "The Ministry of Greeters and Collectors (MGC) serves as the first face of the Christian community through a courteous welcome, orderly guidance, and a caring presence towards every one of the faithful. Alongside this they are entrusted with the honest, disciplined and accountable gathering of the offerings, which serve as a concrete expression of the community's sacrifice for the continuing mission of the Church. In their humility, integrity and respect for the holiness of the work, this ministry strengthens order, unity and the spirit of service within the worshipping community.",

  "min-soccom":
    "The Social Communications Ministry (SOCCOM) spreads the message of the Gospel through the accurate, responsible and modern use of media and communications. In documenting, reporting and making known the works of the Church, they serve as the voice and the image of the community's living faith. They uphold truth and dignity in every message they carry.",

  "min-apostolado":
    "The Apostolado ng Panalangin (AnP) seeks to deepen the believer's relationship with God through earnest, shared and unceasing prayer. In offering self, work and suffering to the Lord, this ministry becomes a spiritual pillar of the Church that supports its mission through prayer.",

  "min-mother-butlers":
    "The Mother Butlers Guild (MBG) is a ministry devoted to the careful and attentive keeping of the sacred vessels and to the good order of the altar and the church. In their quiet but essential service, they express respect for the holiness of the liturgy and the spirit of a mother who serves with love.",

  "min-youth":
    "The Ministry on Youth Affairs serves as guide and companion to young people in their formation as responsible Christians and active members of the Church and of society. Through formation, participation and service, this ministry prepares the young as leaders of the Church of today and of the future.",

  "min-catechetical":
    "The Catechetical Ministry carries the responsibility of proclaiming and teaching the foundations of the Catholic faith in a clear, faithful and systematic way. In their teaching and guidance, they deepen the community's understanding of doctrine and form a Christian life rooted in the Gospel.",
};

/**
 * The six choirs share one description, word for word, in data.ts.
 *
 * Kept as a single string rather than copied six times: a change to the
 * Tagalog would otherwise need six matching edits here, and the first
 * one missed would leave two choirs describing themselves differently
 * in English while describing themselves identically in Tagalog.
 */
const CHOIR_EN =
  "Oversees the fitting use of music in the liturgy, to support prayer and deepen the participation of the faithful. It is responsible for choosing and preparing the hymns and liturgical songs for Holy Mass and the other celebrations of the Church. Through reverent and well-ordered music, it serves for the greater glory of God.";

for (const id of [
  "min-choir-cfc",
  "min-choir-teatro",
```

```
  "min-choir-singing-servants",
  "min-choir-charismatic",
  "min-choir-marian-youth",
  "min-choir-apostleship",
]) {
  MINISTRY_DESCRIPTION_EN[id] = CHOIR_EN;
}

/** Tagalog for the five sacrament descriptions, which were English only. */
export const SACRAMENT_DESCRIPTION_FIL: Record<string, string> = {
  "sac-baptism":
    "Ang pintuan ng buhay sa Espiritu, na naghuhugas ng orihinal na kasalanan at tumatanggap sa kaluluwa sa Katawan ni Kristo.",
  "sac-confirmation":
    "Ipinagkakaloob nito ang natatanging lakas ng Espiritu Santo upang aktibong magpatotoo sa pananampalatayang Kristiyano.",
  "sac-matrimony":
    "Isang tipan kung saan ang lalaki at babae ay nagtatatag ng panghabambuhay na pagsasama na nakatuon sa kabutihan ng mag-asawa.",
  "sac-eucharist":
    "Ang unang pagtanggap kay Kristo sa Banal na Komunyon — ang kanyang tunay na Katawan at Dugo — na nagbubuklod sa mga bata sa buhay ng Simbahan.",
  // The id is sac-confession, not sac-reconciliation. A test that
  // checks every sacrament has a translation caught the wrong guess;
  // without it this one would simply have stayed English for Tagalog
  // readers and nobody would have been told.
  "sac-confession":
    "Ang maawaing sakramento ng pagpapagaling, kung saan pinatatawad ang mga kasalanan at naibabalik ang kaluluwa sa biyaya.",
};

/**
 * Tagalog for the five station descriptions and reflections.
 *
 * These are the words the AR tour and the scanner read out and show on
 * the result card. They were English only, so a pilgrim reading the
 * app in Tagalog scanned a statue and got an English answer - which is
 * exactly the complaint that started this.
 *
 * The reflections are prayers addressed to the reader, so they are
 * translated as prayers rather than as sentences: the questions stay
 * questions, and the second person stays second person.
 */
export const STATION_FIL: Record<string, { description: string; reflection: string }> = {
  "mhcp-altar": {
    description:
      "Ang gitnang santuwaryo ng Mary Help of Christians Parish, itinayo para sa malalim na pagsamba at sa pagdiriwang ng Eukaristiya.",
    reflection:
      "Patahimikin ang iyong puso habang nakatayo sa harap ng Dambana. Sa katahimikan, pakinggan ang bulong ng Diyos. Ano ang pinakapinasasalamatan mo ngayong
araw?",
  },
  "mhcp-patron": {
    description:
      "Ang marangyang koronadong imahen ng Mary Help of Christians na karga ang Batang Hesus nang nakabukas ang mga bisig.",
    reflection:
      "Karga ni Maria si Hesus at iniaalay siya sa atin bilang ating pinakamataas na saklolo. Saan sa iyong buhay kailangan mo ng katulong ngayon? Hindi binabalewala
ang taimtim na panalangin.",
  },
  "mhcp-bapt": {
    description:
      "Ang banal na batisan kung saan ang mga sanggol at nakatatanda ay pumapasok sa pamilya ng Diyos, binyagan sa tubig at sa Espiritu.",
    reflection:
      "Ang tubig ay nagdadala ng buhay at naglilinis. Tandaan na may banal na layunin ang iyong buhay. Paano ka makapagdadala ng malinis na simula at pag-asa sa
isang nanlulumo ngayong araw?",
```

```
  },
  "src-sanctuary": {
    description:
      "Ang taimtim na dambana at luklukan ng Obispo ng Kalookan, kung saan ginaganap ang mga malalaking kapistahan at makasaysayang pagtitipon.",
    reflection:
      "Pagnilayan ang tatag ng katedral na ito, nakatindig sa gitna ng mga digmaan at sakuna. Gaano katatag ang iyong espiritu sa panahon ng pagsubok?",
  },
  "src-statue": {
    description:
      "Ang lubos na iginagalang na imahen ni San Roque, larawan ng kanyang buhay ng sakripisyo at pagpapagaling.",
    reflection:
      "Isinapanganib ni San Roque ang kanyang buhay upang alagaan ang iba. Sino sa iyong maysakit na kapitbahay, kaibigan, o kapamilya ang maipagdarasal o madadalaw
mo ngayong araw?",
  },
};

/**
 * Everything the app already holds in both languages, by id.
 *
 * `bilingualFor` is what call sites use; it returns the source text
 * paired with its translation, or the source twice when no translation
 * exists yet. Twice, rather than empty: a missing translation shows the
 * original, which is readable and visibly wrong, instead of a blank
 * paragraph nobody can report.
 */
export function bilingualFor(
  source: string,
  translation: string | undefined,
  sourceLanguage: "en" | "fil",
): Bilingual {
  if (sourceLanguage === "fil") {
    return { fil: source, en: translation ?? source };
  }
  return { en: source, fil: translation ?? source };
}

/**
 * One station, in the reader's language.
 *
 * Every screen that shows a station goes through this: the scanner's
 * result card, the AR tour's stop, the narration that reads it aloud.
 * Three call sites each doing their own fallback is three chances for
 * one of them to stay English after the setting is changed.
 */
export function stationText(
  station: { id: string; description: string; reflection: string },
  language: "en" | "fil",
): { description: string; reflection: string } {
  if (language === "en") {
    return { description: station.description, reflection: station.reflection };
  }
  const fil = STATION_FIL[station.id];
  // No translation yet means the English, which is readable and
  // visibly wrong, rather than a blank card nobody can report.
  return {
    description: fil?.description || station.description,
    reflection: fil?.reflection || station.reflection,
  };
}
```

1.4.18 APP-088 — contrast.ts

Code ID	APP-088
File Name	contrast.ts
File Path	SanctDemo/src/lib/contrast.ts
Language / Type	TypeScript
Lines	130
Functions / Classes	Rgb (exported type); parseHex (exported function); luminance (exported function); contrastRatio (exported function); readableTextOn (exported function); ColourVerdict (exported type); checkParishColour (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ParishContentEditor.tsx
How to Recognize It	Open SanctDemo/src/lib/contrast.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Contrast maths, so an admin choosing a parish colour cannot accidentally make the app unreadable.
>
> The palette in index.css was tuned pairing by pairing to clear WCAG AA. Letting someone pick an arbitrary colour throws that away unless the consequences are computed rather than hoped for — a pale gold that looks lovely in a colour picker renders white-on-white in a nave, to a congregation that includes people in their seventies.
>
> Pure and dependency-free so it can be tested directly.

Complete Source Code — contrast.ts

```
// Contrast maths, so an admin choosing a parish colour cannot accidentally
// make the app unreadable.
//
// The palette in index.css was tuned pairing by pairing to clear WCAG AA.
// Letting someone pick an arbitrary colour throws that away unless the
// consequences are computed rather than hoped for — a pale gold that looks
// lovely in a colour picker renders white-on-white in a nave, to a
// congregation that includes people in their seventies.
//
// Pure and dependency-free so it can be tested directly.

export interface Rgb {
  r: number
  g: number
  b: number
}

/** Parses "#RGB" or "#RRGGBB". Returns null for anything else. */
export function parseHex(hex: string): Rgb | null {
  const value = hex.trim().replace(/^#/, '')
  if (!/^[0-9a-fA-F]{3}$|^[0-9a-fA-F]{6}$/.test(value)) return null
```

```
  const full =
    value.length === 3
      ? value
          .split('')
          .map((c) => c + c)
          .join('')
      : value

  return {
    r: parseInt(full.slice(0, 2), 16),
    g: parseInt(full.slice(2, 4), 16),
    b: parseInt(full.slice(4, 6), 16),
  }
}

/** Relative luminance, per WCAG 2.1. */
export function luminance({ r, g, b }: Rgb): number {
  const channel = (v: number) => {
    const s = v / 255
    return s <= 0.03928 ? s / 12.92 : Math.pow((s + 0.055) / 1.055, 2.4)
  }
  return 0.2126 * channel(r) + 0.7152 * channel(g) + 0.0722 * channel(b)
}

/** Contrast ratio between two colours, 1 to 21. */
export function contrastRatio(a: Rgb, b: Rgb): number {
  const la = luminance(a)
  const lb = luminance(b)
  return (Math.max(la, lb) + 0.05) / (Math.min(la, lb) + 0.05)
}

const WHITE: Rgb = { r: 250, g: 248, b: 244 } // --color-brand-on-accent
const INK: Rgb = { r: 20, g: 23, b: 31 } // --color-brand-text
const PAPER: Rgb = { r: 250, g: 248, b: 244 } // --color-brand-card

/**
 * The text colour to place ON a chosen background — whichever of ink or paper
 * reads better against it.
 *
 * This is what makes an admin colour picker safe rather than a hazard: the
 * admin chooses the background, the app chooses the text, and the pairing is
 * never left to chance.
 */
export function readableTextOn(background: Rgb): { hex: string; ratio: number } {
  const onInk = contrastRatio(background, INK)
  const onWhite = contrastRatio(background, WHITE)
  return onInk >= onWhite
    ? { hex: '#14171F', ratio: onInk }
    : { hex: '#FAF8F4', ratio: onWhite }
}

export interface ColourVerdict {
  ok: boolean
  /** Best achievable contrast for text placed on this colour. */
  onColourRatio: number
  /** Contrast of this colour used as text on the app's paper background. */
  asTextRatio: number
  /** Plain-language reason, or null when the colour is fine. */
```

```
  problem: string | null
}

/**
 * Whether a parish colour is safe to use.
 *
 * Two separate jobs are checked, because a brand colour does both in this app
 * and a colour can pass one and fail the other. A deep navy makes a fine
 * button but is barely distinguishable as a heading on paper; a pale gold
 * reads as a heading but gives a button no readable label.
 */
export function checkParishColour(hex: string): ColourVerdict {
  const rgb = parseHex(hex)
  if (!rgb) {
    return {
      ok: false,
      onColourRatio: 0,
      asTextRatio: 0,
      problem: 'That is not a valid colour. Use a hex value such as #1C2C56.',
    }
  }

  const onColour = readableTextOn(rgb).ratio
  const asText = contrastRatio(rgb, PAPER)

  // 4.5:1 is AA for normal text — the floor this app already holds itself to.
  if (onColour < 4.5) {
    return {
      ok: false,
      onColourRatio: onColour,
      asTextRatio: asText,
      problem: `Buttons in this colour would have hard-to-read labels (${onColour.toFixed(
        1,
      )}:1, needs 4.5:1). Try a darker or more saturated shade.`,
    }
  }

  if (asText < 4.5) {
    return {
      ok: false,
      onColourRatio: onColour,
      asTextRatio: asText,
      problem: `Headings in this colour would be too faint on the page background (${asText.toFixed(
        1,
      )}:1, needs 4.5:1). Try a darker shade.`,
    }
  }

  return { ok: true, onColourRatio: onColour, asTextRatio: asText, problem: null }
}
```

1.4.19 APP-089 — deviceNotifications.ts

Code ID	APP-089
File Name	deviceNotifications.ts
File Path	SanctDemo/src/lib/deviceNotifications.ts
Language / Type	TypeScript
Lines	194
Functions / Classes	notificationsSupported (exported function); PermissionState (exported type); permissionState (exported function); ensureChannel (exported function); requestPermission (exported function); applySchedule (exported function); notifyNow (exported function); pendingCount (exported function)
Related Files (its imports)	./reminderSchedule
Where It Is Used	src/components/ReminderSettingsCard.tsx, src/components/WatchBell.tsx, src/lib/useMassScheduleNotice.ts, src/lib/useOpeningNotices.ts
How to Recognize It	Open SanctDemo/src/lib/deviceNotifications.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — deviceNotifications.ts

```
import { LocalNotifications } from "@capacitor/local-notifications";
import { Capacitor } from "@capacitor/core";
import type { Reminder } from "./reminderSchedule";

/**
 * Handing reminders to Android, so they appear when the app is closed.
 *
 * ## Why nothing reached the phone before
 *
 * The existing notification system writes a document to Firestore and
 * shows it on the Notifications screen. That is a message inside the
 * app, and the phone was never told about any of it — which is exactly
 * why none ever arrived. This module is the missing half.
 *
 * ## What works with the app closed, and what does not
 *
 * A local notification is an alarm set ON the device. Once scheduled it
 * fires with the app closed, the screen off and no signal. Mass times,
 * feast days and dated announcements are all known in advance, so they
 * are scheduled ahead and behave exactly as a reminder should.
 *
 * What CANNOT work that way is anything the parish decides while the app
 * is closed — an application approved an hour ago. Delivering that needs
 * push (FCM), which needs a server holding a service-account key, which
 * needs the Blaze plan. On Spark the honest behaviour is to raise it the
 * moment the app is next opened, which `notifyNow` does.
 *
 * ## Why every call is guarded
```

```
 *
 * On the web there is no plugin. Every function here returns quietly
 * rather than throwing, so the same code runs in a browser during
 * development and simply does nothing.
 */

const CHANNEL_ID = "sanctiwalk-parish";

export function notificationsSupported(): boolean {
  return Capacitor.isNativePlatform() && Capacitor.isPluginAvailable("LocalNotifications");
}

export type PermissionState = "granted" | "denied" | "unsupported" | "prompt";

export async function permissionState(): Promise<PermissionState> {
  if (!notificationsSupported()) return "unsupported";
  try {
    const { display } = await LocalNotifications.checkPermissions();
    if (display === "granted") return "granted";
    if (display === "denied") return "denied";
    return "prompt";
  } catch {
    return "unsupported";
  }
}

/**
 * Makes sure the notification channel exists.
 *
 * ## Why this is not part of asking for permission
 *
 * It was, and that was a bug that silently threw every notification
 * away. Android 8 and later drop — with no error, no log, nothing — any
 * notification posted to a channel that does not exist. Creating the
 * channel only inside requestPermission() meant it was created only on
 * the run where the user first said yes. On every run after that,
 * permission was already granted, the request was never made, the
 * channel was never created, and both the test notification and twelve
 * scheduled Mass reminders went nowhere. Reinstalling made it worse,
 * because the OS remembers the grant while the app forgets the channel.
 *
 * So: the channel is a precondition for posting, not a consequence of
 * being granted permission, and every path that posts calls this first.
 *
 * Memoised because creating it is idempotent but not free, and the
 * scheduler calls it on every resume.
 */
let channelReady: Promise<void> | null = null;

export function ensureChannel(): Promise<void> {
  if (!notificationsSupported()) return Promise.resolve();
  if (channelReady) return channelReady;

  channelReady = LocalNotifications.createChannel({
    id: CHANNEL_ID,
    name: "Parish reminders",
    description: "Mass times, feast days and news from your parish.",
    // Default-high: a Mass reminder should appear and make a sound, not
    // take over the screen the way an alarm or a call does.
```

```
    importance: 4,
    visibility: 1,
  })
    .then(() => undefined)
    .catch(error => {
      // Let the next call try again rather than caching the failure.
      channelReady = null;
      console.warn("[notifications] channel not created:", error);
    });

  return channelReady;
}

export async function requestPermission(): Promise<PermissionState> {
  if (!notificationsSupported()) return "unsupported";
  try {
    const { display } = await LocalNotifications.requestPermissions();
    if (display !== "granted") return display === "denied" ? "denied" : "prompt";
    await ensureChannel();
    return "granted";
  } catch {
    return "unsupported";
  }
}

/**
 * Replaces everything pending with this list.
 *
 * Cancel-then-schedule rather than diffing. The reminder ids are derived
 * from what each reminder IS, so rescheduling the same Mass reuses its
 * id and Android replaces the alarm — but a Mass the parish has since
 * suspended simply would not appear in the new list, and without the
 * cancel its old alarm would still be sitting there waiting to fire.
 * Sending someone to a cancelled Mass is the failure that matters most.
 */
export async function applySchedule(reminders: Reminder[]): Promise<number> {
  if (!notificationsSupported()) return 0;

  try {
    // Before anything is queued: a reminder scheduled against a missing
    // channel fires into nothing, hours later, with no sign of why.
    await ensureChannel();

    const pending = await LocalNotifications.getPending();
    if (pending.notifications.length > 0) {
      await LocalNotifications.cancel({ notifications: pending.notifications });
    }

    if (reminders.length === 0) return 0;

    await LocalNotifications.schedule({
      notifications: reminders.map(r => ({
        id: r.id,
        title: r.title,
        body: r.body,
        channelId: CHANNEL_ID,
        schedule: { at: r.at, allowWhileIdle: true },
        smallIcon: "ic_stat_sanctiwalk",
        extra: { kind: r.kind },
```

```
      })),
    });
    return reminders.length;
  } catch (error) {
    // Never fatal. A phone that refuses to schedule reminders is a phone
    // without reminders, not a broken app.
    console.warn("[notifications] could not schedule:", error);
    return 0;
  }
}

/**
 * Shows something immediately.
 *
 * For the things that cannot be known in advance — an application the
 * parish has just decided on. Fires while the app is open, and on next
 * open for anything decided while it was closed.
 */
export async function notifyNow(title: string, body: string): Promise<void> {
  if (!notificationsSupported()) return;
  try {
    await ensureChannel();
    await LocalNotifications.schedule({
      notifications: [{
        // Time-based so two notifications in the same second cannot
        // collide, and so it never clashes with a scheduled reminder's
        // content-derived id.
        id: Math.floor(Date.now() % 2_000_000_000),
        title,
        body,
        channelId: CHANNEL_ID,
        smallIcon: "ic_stat_sanctiwalk",
      }],
    });
  } catch (error) {
    console.warn("[notifications] could not show:", error);
  }
}

/** For the settings screen: how many reminders are actually queued. */
export async function pendingCount(): Promise<number> {
  if (!notificationsSupported()) return 0;
  try {
    return (await LocalNotifications.getPending()).notifications.length;
  } catch {
    return 0;
  }
}
```

1.4.20 APP-090 — displayMode.ts

Code ID	APP-090
File Name	displayMode.ts
File Path	SanctDemo/src/lib/displayMode.ts
Language / Type	TypeScript
Lines	74
Functions / Classes	DEVICE_WIDTH_BREAKPOINT (exported constant); isDeviceSurface (exported function); readNative (internal function); currentWidth (internal function); useDeviceSurface (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/App.tsx, src/components/PhoneContainer.tsx
How to Recognize It	Open SanctDemo/src/lib/displayMode.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — displayMode.ts

```
import { useEffect, useState } from 'react'
import { Capacitor } from '@capacitor/core'

// Is this screen a real device surface, or a desktop showing the app off?
//
// App.tsx wraps everything in PhoneContainer, which draws a 385x780 phone
// mockup: a notch, a painted status bar reading "LTE 98%", an iOS home
// indicator, and a Phone / Full responsive toggle above it. On a laptop
// that is useful - it is how the app is demonstrated during the defense.
//
// On an actual phone it is absurd. The installed app rendered a picture of
// a phone inside the real one, so the app itself occupied a fraction of the
// screen, behind a fake battery percentage and beside buttons offering to
// switch viewport. It also made sign-in look like a bug: that screen is
// `fixed inset-0`, so it escaped the mockup and filled the real display,
// and the app appeared to change size on signing in. It did.
//
// Two things count as a real device surface:
//   - the installed Android app, which is definitionally a phone;
//   - any narrow viewport, because a phone mockup inside a phone browser is
//     the same mistake by another route.

/** The width at or below which the viewport is a phone, not a desk. */
export const DEVICE_WIDTH_BREAKPOINT = 768

/**
 * Pure so the rule can be tested without a browser.
 *
 * `native` is passed in rather than read here because Capacitor decides it
 * from the bridge, which does not exist under test.
 */
```

```
export function isDeviceSurface(native: boolean, viewportWidth: number): boolean {
  return native || viewportWidth <= DEVICE_WIDTH_BREAKPOINT
}

function readNative(): boolean {
  try {
    return Capacitor.isNativePlatform()
  } catch {
    // The bridge is absent in a plain browser and in tests. Not native.
    return false
  }
}

function currentWidth(): number {
  // Server-side or a stubbed environment: assume a desk, which keeps the
  // existing presentation behaviour rather than silently changing it.
  if (typeof window === 'undefined') return Number.POSITIVE_INFINITY
  return window.innerWidth
}

/**
 * Live answer, so rotating the phone or dragging a desktop window narrow
 * takes effect immediately instead of at the next reload.
 */
export function useDeviceSurface(): boolean {
  const [deviceSurface, setDeviceSurface] = useState(() => isDeviceSurface(readNative(), currentWidth()))

  useEffect(() => {
    if (typeof window === 'undefined') return
    const native = readNative()
    const update = () => setDeviceSurface(isDeviceSurface(native, window.innerWidth))
    update()
    window.addEventListener('resize', update)
    // Rotation does not always fire resize on Android WebView.
    window.addEventListener('orientationchange', update)
    return () => {
      window.removeEventListener('resize', update)
      window.removeEventListener('orientationchange', update)
    }
  }, [])

  return deviceSurface
}
```

1.4.21 APP-091 — exportApplications.ts

Code ID	APP-091
File Name	exportApplications.ts
File Path	SanctDemo/src/lib/exportApplications.ts
Language / Type	TypeScript
Lines	211
Functions / Classes	dateOnly (internal function); field (internal function); ministryRows (internal function); sacramentRows (internal function); mixedRows (internal function); rowsFor (exported function); exportFilename (exported function); download (internal function); exportExcel (exported function); csvSafeCell (exported function); exportCsv (exported function)
Related Files (its imports)	../types, ./applications, ./applicationFilters
Where It Is Used	src/admin/AdminApplications.tsx
How to Recognize It	Open SanctDemo/src/lib/exportApplications.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — exportApplications.ts

```
import * as XLSX from "xlsx";
import type { ApplicationDoc } from "../types";
import { STATUS_LABEL } from "./applications";
import { MONTHS, type Filters } from "./applicationFilters";

/**
 * Exporting the filtered list, for sending to a coordinator.
 *
 * ## What goes in the file
 *
 * Only fields the forms actually collect. The brief asked for an Age
 * column; nothing in SanctiWalk ever asks anyone's age, and a column of
 * blanks in a file going to a ministry coordinator is worse than no
 * column - it reads as data that was lost rather than data never asked
 * for.
 *
 * Ministry and sacrament applications get DIFFERENT columns, because they
 * collect different things. One merged sheet would be half empty whichever
 * way it was filtered.
 */

function dateOnly(iso?: string): string {
  if (!iso) return "";
  const d = new Date(iso);
  if (Number.isNaN(d.getTime())) return iso;
  return d.toLocaleDateString("en-CA"); // YYYY-MM-DD: sorts correctly in a spreadsheet
}

function field(app: ApplicationDoc, key: string): string {
```

```
  const v = app.formData?.[key];
  if (v === undefined || v === null || v === "") return "";
  if (typeof v === "boolean") return v ? "Yes" : "No";
  return String(v);
}

type Row = Record<string, string | number>;

function ministryRows(apps: ApplicationDoc[]): Row[] {
  return apps.map((a, i) => ({
    "No.": i + 1,
    "Reference": a.referenceNumber ?? "",
    "Applicant Name": a.applicantName ?? "",
    "Ministry": a.type ?? "",
    "Contact Number": field(a, "mobile"),
    "Email": a.applicantEmail ?? "",
    "Message": field(a, "message"),
    "Date Applied": dateOnly(a.createdAt),
    "Status": STATUS_LABEL[a.status] ?? a.status,
    "Last Updated": dateOnly(a.reviewedAt ?? a.updatedAt),
    "Parish Note": a.adminNote ?? "",
  }));
}

function sacramentRows(apps: ApplicationDoc[]): Row[] {
  return apps.map((a, i) => ({
    "No.": i + 1,
    "Reference": a.referenceNumber ?? "",
    "Applicant Name": a.applicantName ?? "",
    "Sacrament": a.type ?? "",
    "Preferred Date": field(a, "preferredDate"),
    "Sponsor / Parent": field(a, "sponsorOrParent"),
    "PSA Prepared": field(a, "hasPSA"),
    "Baptismal Prepared": field(a, "hasBaptismal"),
    "Email": a.applicantEmail ?? "",
    "Date Applied": dateOnly(a.createdAt),
    "Status": STATUS_LABEL[a.status] ?? a.status,
    "Last Updated": dateOnly(a.reviewedAt ?? a.updatedAt),
    "Parish Note": a.adminNote ?? "",
  }));
}

/** Both kinds together, on the columns they share. */
function mixedRows(apps: ApplicationDoc[]): Row[] {
  return apps.map((a, i) => ({
    "No.": i + 1,
    "Reference": a.referenceNumber ?? "",
    "Applicant Name": a.applicantName ?? "",
    "Type": a.kind === "ministry" ? "Ministry" : "Sacrament",
    "Ministry / Sacrament": a.type ?? "",
    "Contact Number": field(a, "mobile"),
    "Email": a.applicantEmail ?? "",
    "Preferred Date": field(a, "preferredDate"),
    "Date Applied": dateOnly(a.createdAt),
    "Status": STATUS_LABEL[a.status] ?? a.status,
    "Last Updated": dateOnly(a.reviewedAt ?? a.updatedAt),
  }));
}
```

```
export function rowsFor(apps: ApplicationDoc[], kind: Filters["kind"]): Row[] {
  if (kind === "ministry") return ministryRows(apps);
  if (kind === "sacrament") return sacramentRows(apps);
  return mixedRows(apps);
}

/**
 * A filename that says what is in the file.
 *
 * "Altar_Servers_August_2026" rather than "applications(3)". The file
 * leaves the app and lands in someone's inbox beside a dozen others, so
 * the name has to survive on its own.
 */
export function exportFilename(f: Filters, extension: string): string {
  const parts: string[] = [];

  if (f.type) parts.push(f.type);
  else if (f.kind === "ministry") parts.push("Ministry Applications");
  else if (f.kind === "sacrament") parts.push("Sacrament Applications");
  else parts.push("Applications");

  // Which set this was taken from, when it is not the ordinary one. A
  // file of finished applications landing in a coordinator's inbox named
  // like a file of live ones is how the wrong list gets worked through.
  if (f.scope === "archived") parts.push("Archive");
  else if (f.scope === "all") parts.push("All");

  if (f.status) parts.push(STATUS_LABEL[f.status] ?? f.status);
  if (f.month !== null) parts.push(MONTHS[f.month - 1]!);
  if (f.year !== null) parts.push(String(f.year));
  if (f.month === null && f.year === null && (f.from || f.to)) {
    parts.push(`${f.from || "start"}_to_${f.to || "today"}`);
  }

  const safe = parts
    .join("_")
    .replace(/[^A-Za-z0-9 _-]/g, "")   // parentheses and slashes break downloads
    .replace(/\s+/g, "_")
    .replace(/_+/g, "_");
  return `${safe}.${extension}`;
}

function download(blob: Blob, filename: string): void {
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  document.body.appendChild(a);
  a.click();
  a.remove();
  // Freed on the next tick: revoking immediately cancels the download in
  // some browsers.
  setTimeout(() => URL.revokeObjectURL(url), 1000);
}

export function exportExcel(apps: ApplicationDoc[], f: Filters): void {
  const rows = rowsFor(apps, f.kind);
  const sheet = XLSX.utils.json_to_sheet(rows);
```

```
  // Column widths from the content. Without these every column is the
  // same narrow default and the coordinator's first action is dragging
  // eleven borders.
  const headers = Object.keys(rows[0] ?? {});
  sheet["!cols"] = headers.map(h => {
    const longest = rows.reduce(
      (max, r) => Math.max(max, String(r[h] ?? "").length),
      h.length,
    );
    return { wch: Math.min(Math.max(longest + 2, 10), 48) };
  });

  const book = XLSX.utils.book_new();
  // Sheet names cannot exceed 31 characters or contain : \ / ? * [ ]
  const sheetName = (f.type || (f.kind === "ministry" ? "Ministry" : f.kind === "sacrament" ? "Sacraments" : "Applications"))
    .replace(/[:\\/?*[\]]/g, "")
    .slice(0, 31);
  XLSX.utils.book_append_sheet(book, sheet, sheetName);

  const out = XLSX.write(book, { bookType: "xlsx", type: "array" });
  download(
    new Blob([out], { type: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet" }),
    exportFilename(f, "xlsx"),
  );
}

/**
 * Defuses a cell that a spreadsheet would run as a formula.
 *
 * Everything in these files is typed by an applicant - their name, their
 * message to the parish - and the whole point of the export is to email
 * it to a coordinator who opens it in Excel. A message of
 * `=cmd|'/c calc'!A1` is a formula on that coordinator's machine, not
 * text. CSV carries no types, so the first character decides.
 *
 * A leading apostrophe is Excel's own "this is text" marker: it is not
 * shown in the cell and the value reads back unchanged. The tab and
 * carriage return are in the list because they let a cell smuggle a
 * second one past the delimiter.
 *
 * Only the CSV path needs this. json_to_sheet writes .xlsx cells with an
 * explicit string type, so Excel never parses those as formulas, and an
 * apostrophe there would be a literal character in the coordinator's
 * spreadsheet.
 */
/** Characters that make a spreadsheet treat a cell as a formula. */
const FORMULA_START = /^[=+\-@\t\r]/;

export function csvSafeCell(value: string | number): string | number {
  if (typeof value !== "string") return value;
  return FORMULA_START.test(value) ? `'${value}` : value;
}
export function exportCsv(apps: ApplicationDoc[], f: Filters): void {
  const rows = rowsFor(apps, f.kind).map(row => {
    const safe: Row = {};
    for (const [key, value] of Object.entries(row)) safe[key] = csvSafeCell(value);
    return safe;
  });
  const sheet = XLSX.utils.json_to_sheet(rows);
```

```
  const csv = XLSX.utils.sheet_to_csv(sheet);
  // BOM, so Excel opens UTF-8 correctly: without it "Niño" arrives as
  // "NiÃ±o", which matters in a list of Filipino names.
  download(new Blob(["" + csv], { type: "text/csv;charset=utf-8" }), exportFilename(f, "csv"));
}
```

1.4.22 APP-092 — findScroller.ts

Code ID	APP-092
File Name	findScroller.ts
File Path	SanctDemo/src/lib/findScroller.ts
Language / Type	TypeScript
Lines	36
Functions / Classes	findScroller (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ParishSectionHeader.tsx, src/components/ParishWelcomeHeader.tsx
How to Recognize It	Open SanctDemo/src/lib/findScroller.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The class App.tsx puts on the one element that scrolls tab content.

Complete Source Code — findScroller.ts

```
/** The class App.tsx puts on the one element that scrolls tab content. */
const SCROLLER_CLASS = "app-scroll";

/**
 * The element that scrolls around `el`.
 *
 * Normally that is the named tab-content scroller in App.tsx. Finding it by
 * name rather than by measurement matters: the measurement below can only
 * recognise a scroller that already has somewhere to scroll to, so on the
 * first commit — before images have loaded and the page is still shorter
 * than the screen — it finds nothing, and a caller that runs once on mount
 * then never gets a second chance.
 *
 * The measurement is kept as a fallback for any scroller without the class.
 * It insists on an element that ACTUALLY scrolls, because several ancestors
 * here declare `overflow-y: auto` while comfortably fitting their content,
 * and stopping at the first of those returns an element whose scrollTop is
 * a permanent zero — a listener on it fires and every reading comes back
 * the same at every scroll position.
 *
 * Returns null when nothing scrolls. Callers should treat that as "no
 * scroll behaviour to attach" rather than as an error.
 */
export function findScroller(el: HTMLElement): HTMLElement | null {
  const named = el.closest<HTMLElement>(`.${SCROLLER_CLASS}`);
  if (named) return named;

  let node = el.parentElement;
  while (node && node !== document.documentElement) {
    const overflowY = getComputedStyle(node).overflowY;
```

```
    const declaresScroll = overflowY === "auto" || overflowY === "scroll";
    if (declaresScroll && node.scrollHeight > node.clientHeight + 4) return node;
    node = node.parentElement;
  }
  return null;
}
```

firebase.ts — listed in full as API-002.

followCamera.ts — listed in full as MAP-004.

1.4.23 APP-093 — followedParishes.ts

Code ID	APP-093
File Name	followedParishes.ts
File Path	SanctDemo/src/lib/followedParishes.ts
Language / Type	TypeScript
Lines	100
Functions / Classes	MAX_FOLLOWED (exported constant); loadFollowed (exported function); saveFollowed (exported function); isFollowing (exported function); withFollowed (exported function); remindableParishes (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/followedParishes.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Parishes you want reminders from, besides your own.
>
> ## The problem this solves
>
> Reminders were built from whichever parish the dashboard happened to be showing. So looking up San Roque's Mass times quietly replaced every reminder for your own parish, and looking back replaced them again — the pilgrim never asked for either, and nothing on screen said it had happened.
>
> Which parish you are READING about and which parishes you want to be REMINDED about are two different questions. This answers the second one, and only ever changes when somebody presses a switch.
>
> ## Why your own parish is not in the list
>
> It is always included, and it cannot be removed here. Storing it would mean two places could disagree about where you belong, and the first bug would be a pilgrim who changed parish and kept being woken for the old one's 6am Mass. The home parish comes from the profile; this list is strictly the extras.
>
> ## Why it lives on the device
>
> Same reasoning as the reminder settings themselves: these are the alarms THIS phone has queued. A shared account should not mean a shared alarm clock.

Complete Source Code — followedParishes.ts

```
/**
 * Parishes you want reminders from, besides your own.
 *
 * ## The problem this solves
 *
 * Reminders were built from whichever parish the dashboard happened to
 * be showing. So looking up San Roque's Mass times quietly replaced
 * every reminder for your own parish, and looking back replaced them
 * again — the pilgrim never asked for either, and nothing on screen
 * said it had happened.
 *
 * Which parish you are READING about and which parishes you want to be
 * REMINDED about are two different questions. This answers the second
 * one, and only ever changes when somebody presses a switch.
 *
 * ## Why your own parish is not in the list
 *
 * It is always included, and it cannot be removed here. Storing it
 * would mean two places could disagree about where you belong, and the
 * first bug would be a pilgrim who changed parish and kept being woken
 * for the old one's 6am Mass. The home parish comes from the profile;
 * this list is strictly the extras.
 *
 * ## Why it lives on the device
 *
 * Same reasoning as the reminder settings themselves: these are the
 * alarms THIS phone has queued. A shared account should not mean a
 * shared alarm clock.
 */

const STORAGE_KEY = "sanctiwalk.followedParishes";

/** Enough for a pilgrim doing a Visita Iglesia; far short of the alarm cap. */
export const MAX_FOLLOWED = 5;

export function loadFollowed(): string[] {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return [];
    const parsed: unknown = JSON.parse(raw);
    if (!Array.isArray(parsed)) return [];
    // Filtered on the way in: a malformed entry here becomes a Firestore
    // listener on a junk id later, which is a harder failure to trace.
    return parsed.filter((id): id is string => typeof id === "string" && id.length > 0);
  } catch {
    return [];
  }
}

export function saveFollowed(ids: string[]): void {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(ids));
  } catch {
    // Private mode. The choice holds for this run and no longer.
  }
}

export function isFollowing(followed: string[], parishId: string): boolean {
  return followed.includes(parishId);
```

```
}

/**
 * Adds or removes one parish.
 *
 * Returns a new array, capped, and never containing the home parish —
 * that one is always on, so letting it in here would make the switch
 * for it look meaningful when it is not.
 */
export function withFollowed(
  followed: string[],
  parishId: string,
  follow: boolean,
  homeParishId?: string | null,
): string[] {
  if (parishId === homeParishId) return followed;

  if (!follow) return followed.filter(id => id !== parishId);
  if (followed.includes(parishId)) return followed;
  // Oldest out when full, rather than refusing. Somebody adding a sixth
  // parish means it, and a silent "no" from a switch that visibly moved
  // is worse than quietly retiring the one they chose longest ago.
  return [...followed, parishId].slice(-MAX_FOLLOWED);
}

/**
 * Every parish to build reminders for: yours first, then the extras.
 *
 * Yours leads because its reminders are the ones that matter most when
 * the cap trims the list, and the cap keeps the order it is given.
 */
export function remindableParishes(
  followed: string[],
  homeParishId?: string | null,
): string[] {
  const out = homeParishId ? [homeParishId] : [];
  for (const id of followed) {
    if (id !== homeParishId && !out.includes(id)) out.push(id);
  }
  return out;
}
```

geo.ts — listed in full as MAP-005.

1.4.24 APP-094 — googleAuth.ts

Code ID	APP-094
File Name	googleAuth.ts
File Path	SanctDemo/src/lib/googleAuth.ts
Language / Type	TypeScript
Lines	151
Functions / Classes	SignInRoute (exported type); signInRoute (exported function); ProfileLike (exported type); needsProfile (exported function); GoogleUserLike (exported type); googleProfileFrom (exported function); signInWithGoogle (exported function); ensureGoogleProfile (exported function); signOutGoogle (exported function)
Related Files (its imports)	./firebase, ./speech
Where It Is Used	src/components/LoginModal.tsx
How to Recognize It	Open SanctDemo/src/lib/googleAuth.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — googleAuth.ts

```
import { FirebaseAuthentication } from '@capacitor-firebase/authentication'
import { GoogleAuthProvider, signInWithCredential, signInWithPopup, type User } from 'firebase/auth'
import { doc, getDoc, setDoc } from 'firebase/firestore'
import { auth, db } from './firebase'
import { isNativeApp } from './speech'

/**
 * Signing in with a Google account.
 *
 * ## Why there are two routes and not one
 *
 * Google refuses OAuth requests that come from an embedded webview and
 * answers `disallowed_useragent`. Capacitor runs the app inside exactly
 * such a webview, so `signInWithPopup` in the APK does not degrade
 * gracefully - it shows the pilgrim a Google error page.
 *
 * So the installed app uses Android's own account picker through the
 * native plugin and then hands the resulting credential to the
 * JavaScript SDK, which is the SDK the rest of the app and every
 * Firestore rule actually reads. A browser keeps the popup, which is
 * the right flow there.
 *
 * ## Why a parish still has to be chosen
 *
 * Google supplies an email address and a name. It does not know which
 * parish somebody belongs to, and the security rules refuse an
 * application whose submitter has no `churchId`. A Google account
 * therefore cannot be a complete profile on its own: the parish the
```

```
 * app is currently showing is recorded as a starting point, exactly as
 * it is for a pilgrim who never signs in, and Me > Change Parish moves
 * it. That is the same deliberate decision as the absent first-run
 * picker, not a new guess.
 */

export type SignInRoute = 'native' | 'popup'

/** Which flow this device must use. */
export function signInRoute(native: boolean): SignInRoute {
  return native ? 'native' : 'popup'
}

/** The fields of a profile this module reads to decide if it is usable. */
export interface ProfileLike {
  uid: string
  churchId?: string
}

/**
 * Whether this account still needs a profile written for it.
 *
 * A profile with no parish counts as missing. It exists, but the rules
 * will refuse every application made from it, so treating it as
 * complete would leave the pilgrim unable to apply for anything with no
 * explanation.
 */
export function needsProfile(profile: ProfileLike | null): boolean {
  if (!profile) return true
  return !profile.churchId
}

export interface GoogleUserLike {
  uid: string
  email: string | null
  displayName: string | null
}

/** The profile to write for somebody whose first sign-in was Google. */
export function googleProfileFrom(user: GoogleUserLike, churchId: string) {
  const email = user.email ?? ''
  // The email's first word is the fallback, which is how somebody came
  // to be greeted as "sanctiwalk" on their own home screen once.
  const fullName = user.displayName?.trim() || email.split('@')[0] || 'Pilgrim'
  return {
    uid: user.uid,
    email,
    fullName,
    // Home greets you by one name, not three.
    nickname: fullName.split(/\s+/)[0],
    churchId,
    role: 'user' as const,
    // Google verifies an email address. It does not verify a phone, and
    // the rules reject a profile that claims otherwise.
    phoneNumber: '',
    phoneVerified: false,
    emailVerified: true,
    createdAt: new Date().toISOString(),
  }
```

```
}

/**
 * Signs in with Google and returns the Firebase user.
 *
 * The native branch deliberately signs in TWICE: once natively, which
 * is what opens the account picker, and once in the JavaScript SDK with
 * the credential that returns. Without the second call `auth.currentUser`
 * stays null, and every Firestore read in the app is made as a signed-out
 * visitor while the pilgrim looks at their own name on screen.
 */
export async function signInWithGoogle(): Promise<User> {
  if (signInRoute(isNativeApp()) === 'native') {
    const result = await FirebaseAuthentication.signInWithGoogle()
    const idToken = result.credential?.idToken
    if (!idToken) {
      throw new Error('Google did not return a sign-in token. Please try again.')
    }
    const credential = GoogleAuthProvider.credential(idToken)
    const signed = await signInWithCredential(auth, credential)
    return signed.user
  }

  const provider = new GoogleAuthProvider()
  // Always ask which account. Silently reusing the last one is wrong on
  // a shared phone, and parish devices are shared.
  provider.setCustomParameters({ prompt: 'select_account' })
  const signed = await signInWithPopup(auth, provider)
  return signed.user
}

/**
 * Writes the profile for a first Google sign-in, and leaves an existing
 * one alone.
 *
 * Returns true when a profile was created, so the caller can tell a new
 * pilgrim which parish they have been placed in rather than letting
 * them discover it.
 */
export async function ensureGoogleProfile(
  user: User,
  churchId: string,
): Promise<boolean> {
  const ref = doc(db, 'users', user.uid)
  const snap = await getDoc(ref)
  const existing = snap.exists() ? (snap.data() as ProfileLike) : null
  if (!needsProfile(existing)) return false

  // merge, not overwrite: a profile that exists but lacks a parish keeps
  // whatever else it already holds.
  await setDoc(ref, googleProfileFrom(user, churchId), { merge: true })
  return true
}

/** Signs out of both SDKs, so the native session does not outlive the web one. */
export async function signOutGoogle(): Promise<void> {
  if (isNativeApp()) {
    try {
      await FirebaseAuthentication.signOut()
```

```
    } catch {
      // Not signed in natively. Nothing to undo.
    }
  }
}
```

1.4.25 APP-095 — heading.ts

Code ID	APP-095
File Name	heading.ts
File Path	SanctDemo/src/lib/heading.ts
Language / Type	TypeScript
Lines	154
Functions / Classes	CompassPoint (exported type); COMPASS_POINTS_8 (exported constant); COMPASS_POINTS_4 (exported constant); normalizeDegrees (exported function); shortestAngleDelta (exported function); classifyHeading (exported function); headingToDirectionName (exported function); formatHeading (exported function); DEFAULT_SMOOTHING_ALPHA (exported constant); DEFAULT_HEADING_DEADBAND_DEGREES (exported constant); SmoothHeadingOptions (exported type); smoothHeading (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/CompassControl.tsx, src/components/DioceseMapLive.tsx, src/lib/followCamera.ts, src/lib/navigation.ts, src/lib/useDeviceHeading.ts
How to Recognize It	Open SanctDemo/src/lib/heading.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Compass heading: classification, smoothing and formatting.
>
> Deliberately pure and sensor-free. The device sensors that feed this only exist on a phone — a laptop has no magnetometer, so nothing downstream of here can be exercised in a dev browser at all. Keeping the maths in a module with no DOM dependency is what makes steps 2–6 of this feature verifiable without walking outside with a handset.
>
> Angles throughout are compass bearings in degrees: 0 = north, increasing clockwise, so 90 = east. That is the convention both DeviceOrientationEvent and MapLibre's `bearing` use, so no conversion is needed at either edge.

Complete Source Code — heading.ts

```
// Compass heading: classification, smoothing and formatting.
//
// Deliberately pure and sensor-free. The device sensors that feed this only
// exist on a phone — a laptop has no magnetometer, so nothing downstream of
// here can be exercised in a dev browser at all. Keeping the maths in a
// module with no DOM dependency is what makes steps 2–6 of this feature
// verifiable without walking outside with a handset.
//
// Angles throughout are compass bearings in degrees: 0 = north, increasing
// clockwise, so 90 = east. That is the convention both DeviceOrientationEvent
// and MapLibre's `bearing` use, so no conversion is needed at either edge.

/** A named compass sector. `centre` is the bearing the name points at. */
```

```
export interface CompassPoint {
  name: string
  abbreviation: string
  centre: number
}

/**
 * The 8-point rose, as the client specified: N/NE/E/SE/S/SW/W/NW.
 *
 * Exported as data rather than baked into a switch so the sector set is
 * configurable — `classifyHeading` derives sector width from the array's
 * length, so passing a 4- or 16-point rose works with no code change.
 */
export const COMPASS_POINTS_8: readonly CompassPoint[] = [
  { name: 'North', abbreviation: 'N', centre: 0 },
  { name: 'Northeast', abbreviation: 'NE', centre: 45 },
  { name: 'East', abbreviation: 'E', centre: 90 },
  { name: 'Southeast', abbreviation: 'SE', centre: 135 },
  { name: 'South', abbreviation: 'S', centre: 180 },
  { name: 'Southwest', abbreviation: 'SW', centre: 225 },
  { name: 'West', abbreviation: 'W', centre: 270 },
  { name: 'Northwest', abbreviation: 'NW', centre: 315 },
]

/** The cardinal-only rose, for UI too small to carry eight labels. */
export const COMPASS_POINTS_4: readonly CompassPoint[] = [
  { name: 'North', abbreviation: 'N', centre: 0 },
  { name: 'East', abbreviation: 'E', centre: 90 },
  { name: 'South', abbreviation: 'S', centre: 180 },
  { name: 'West', abbreviation: 'W', centre: 270 },
]

/**
 * Wraps any angle into [0, 360).
 *
 * Sensor readings and accumulated smoothing both drift outside that range —
 * a heading of -5° and one of 355° are the same direction, and every
 * comparison below assumes a single canonical form.
 */
export function normalizeDegrees(degrees: number): number {
  if (!Number.isFinite(degrees)) return 0
  return ((degrees % 360) + 360) % 360
}

/**
 * The signed shortest rotation from `from` to `to`, in (-180, 180].
 *
 * Naive subtraction takes the long way round the discontinuity at north:
 * turning from 350° to 10° is a 20° turn right, not a 340° turn left. Every
 * piece of smoothing and animation here has to go the short way or the map
 * spins almost all the way around whenever the pilgrim crosses north.
 */
export function shortestAngleDelta(from: number, to: number): number {
  const diff = (normalizeDegrees(to) - normalizeDegrees(from) + 540) % 360 - 180
  // `% 360` yields -180 for an exact half turn; normalise to +180 so the
  // result is consistently in (-180, 180] and a half turn always rotates
  // the same way rather than depending on float sign.
  return diff === -180 ? 180 : diff
}
```

```
/**
 * Names the sector a bearing falls in.
 *
 * Sector width comes from `points.length`, so the same function serves a
 * 4-, 8- or 16-point rose. Each name owns the band centred on its bearing:
 * with 8 points that is ±22.5°, so 337.5°–22.5° is North.
 */
export function classifyHeading(
  degrees: number,
  points: readonly CompassPoint[] = COMPASS_POINTS_8,
): CompassPoint {
  const sector = 360 / points.length
  const index = Math.round(normalizeDegrees(degrees) / sector) % points.length
  return points[index]
}

/** "Southeast" — the sector name alone, for prose like "Head Southeast". */
export function headingToDirectionName(
  degrees: number,
  points: readonly CompassPoint[] = COMPASS_POINTS_8,
): string {
  return classifyHeading(degrees, points).name
}

/** "142° SE" — for a compact readout beside the compass. */
export function formatHeading(
  degrees: number,
  points: readonly CompassPoint[] = COMPASS_POINTS_8,
): string {
  const normalized = normalizeDegrees(degrees)
  return `${Math.round(normalized)}° ${classifyHeading(normalized, points).abbreviation}`
}

/**
 * How much of a new reading to accept per update, 0–1. Lower is smoother and
 * laggier. 0.15 settles a turn in roughly half a second at the ~60Hz these
 * sensor events arrive at, while flattening the several-degree jitter a phone
 * magnetometer produces when held still.
 */
export const DEFAULT_SMOOTHING_ALPHA = 0.15

/**
 * Below this, a new reading is treated as noise and ignored outright.
 *
 * Exponential smoothing alone still creeps: a magnetometer sitting still
 * wanders a degree or two, and feeding that through a filter produces a
 * marker that drifts perpetually instead of resting. A deadband makes
 * "not moving" an actual state.
 */
export const DEFAULT_HEADING_DEADBAND_DEGREES = 1.5

export interface SmoothHeadingOptions {
  alpha?: number
  deadbandDegrees?: number
}

/**
 * One step of exponential smoothing, wraparound-safe.
```

```
 *
 * Interpolating raw values would sweep the wrong way across north — from
 * 350° toward 10° the arithmetic mean is 180°, pointing due south. Smoothing
 * the *delta* instead and adding it back keeps every turn on the short arc.
 *
 * `previous` of null means this is the first reading: adopt it exactly, so
 * the marker appears already pointing the right way rather than swinging up
 * from north.
 */
export function smoothHeading(
  previous: number | null,
  next: number,
  { alpha = DEFAULT_SMOOTHING_ALPHA, deadbandDegrees = DEFAULT_HEADING_DEADBAND_DEGREES }: SmoothHeadingOptions = {},
): number {
  const target = normalizeDegrees(next)
  if (previous === null) return target

  const current = normalizeDegrees(previous)
  const delta = shortestAngleDelta(current, target)
  if (Math.abs(delta) < deadbandDegrees) return current

  return normalizeDegrees(current + delta * alpha)
}
```

1.4.26 APP-096 — historyNarration.ts

Code ID	APP-096
File Name	historyNarration.ts
File Path	SanctDemo/src/lib/historyNarration.ts
Language / Type	TypeScript
Lines	114
Functions / Classes	NarratableSection (exported type); asSentence (internal function); historyScript (exported function); spokenSeconds (exported function); spokenLength (exported function)
Related Files (its imports)	./language
Where It Is Used	src/components/ChurchHistory.tsx
How to Recognize It	Open SanctDemo/src/lib/historyNarration.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turning a parish history into something that can be read aloud.
>
> ## Why the script is not just the page text
>
> The history page carries more than the history. There is a note explaining that the Filipino is the original, captions under the photographs, and a "Sanggunian:" credit on each one. All of that is there for someone looking at the page. Read aloud it becomes "source colon National Historical Commission" every few sentences, and the listener loses the thread of the story the parish is actually telling. So the script is the title, the headings and the prose - nothing that only makes sense next to a photograph.
>
> ## Why it is assembled rather than read from the DOM
>
> Reading the rendered text would pick up whatever is on screen, including the quiz at the bottom and any future furniture. Building it from the same data the page renders keeps the two honest without tying narration to the markup.
>
> Pure, so what gets spoken can be tested without a speech engine.

Complete Source Code — historyNarration.ts

```
/**
 * Turning a parish history into something that can be read aloud.
 *
 * ## Why the script is not just the page text
 *
 * The history page carries more than the history. There is a note
 * explaining that the Filipino is the original, captions under the
 * photographs, and a "Sanggunian:" credit on each one. All of that is
 * there for someone looking at the page. Read aloud it becomes "source
 * colon National Historical Commission" every few sentences, and the
 * listener loses the thread of the story the parish is actually
```

```
 * telling. So the script is the title, the headings and the prose -
 * nothing that only makes sense next to a photograph.
 *
 * ## Why it is assembled rather than read from the DOM
 *
 * Reading the rendered text would pick up whatever is on screen,
 * including the quiz at the bottom and any future furniture. Building
 * it from the same data the page renders keeps the two honest without
 * tying narration to the markup.
 *
 * Pure, so what gets spoken can be tested without a speech engine.
 */
import { pick, type Bilingual, type Language } from './language'

/** The shape ChurchHistory already uses; repeated here to stay dependency-free. */
export interface NarratableSection {
  heading: Bilingual | string
  paragraphs: (Bilingual | string)[]
}

/**
 * A heading ends with a full stop so the voice drops and pauses.
 *
 * Without it the engine runs the heading straight into the first
 * sentence - "Early Years the parish was founded" - which is the
 * clearest sign a page is being read by a machine rather than read
 * out.
 */
function asSentence(text: string): string {
  const trimmed = text.trim()
  if (!trimmed) return ''
  return /[.!?:]$/.test(trimmed) ? trimmed : `${trimmed}.`
}

/**
 * What the speaker button reads, in the language on screen.
 *
 * Returns an empty string when there is nothing to read, which is the
 * caller's signal to show no button at all: a speaker that speaks
 * silence is worse than no speaker.
 */
export function historyScript(
  title: string,
  sections: NarratableSection[],
  language: Language,
): string {
  const parts: string[] = []

  const spokenTitle = asSentence(title)
  if (spokenTitle) parts.push(spokenTitle)

  for (const section of sections) {
    const heading = asSentence(pick(section.heading, language))
    // A heading with nothing under it is a label, not a chapter.
    const paragraphs = section.paragraphs
      .map(p => pick(p, language).trim())
      .filter(Boolean)

    if (paragraphs.length === 0) continue
```

```
    if (heading) parts.push(heading)
    parts.push(...paragraphs)
  }

  // Blank lines between parts: every engine treats a paragraph break as
  // a longer pause than a full stop, which is the difference between
  // listening to a history and listening to a list.
  return parts.join('\n\n')
}

/**
 * Roughly how long the reading will take, in seconds.
 *
 * Only ever used to tell the listener what they are committing to
 * before they tap. Deliberately approximate - the engines differ, and
 * a precise-looking number that is wrong is worse than a round one.
 * 140 words per minute is a shade under conversational pace, matching
 * the RATE the speech module asks for.
 */
export function spokenSeconds(script: string): number {
  const words = script.split(/\s+/).filter(Boolean).length
  if (words === 0) return 0
  return Math.round((words / 140) * 60)
}

/**
 * "about 3 min", in the language on screen, or null when there is
 * nothing to read.
 *
 * Translated because it sits inside a Tagalog button, beside a Tagalog
 * verb. An English duration there is the seam showing.
 */
export function spokenLength(script: string, language: Language = 'en'): string | null {
  const seconds = spokenSeconds(script)
  if (seconds === 0) return null

  if (language === 'fil') {
    if (seconds < 60) return 'wala pang isang minuto'
    return `mga ${Math.round(seconds / 60)} min`
  }

  if (seconds < 60) return 'under a minute'
  return `about ${Math.round(seconds / 60)} min`
}
```

1.4.27 APP-097 — historyQuiz.ts

Code ID	APP-097
File Name	historyQuiz.ts
File Path	SanctDemo/src/lib/historyQuiz.ts
Language / Type	TypeScript
Lines	183
Functions / Classes	QuizItem (exported type); HISTORY_QUIZZES (exported constant); quizFor (exported function); unanswered (exported function); QuizResult (exported type); gradeQuiz (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/HistoryQuiz.tsx
How to Recognize It	Open SanctDemo/src/lib/historyQuiz.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Five questions about a parish's own history.
>
> ## The one rule
>
> Every correct answer is a sentence on the parish's history page. Nothing here is invented, nothing is drawn from outside the diocese's own material, and nothing is generated at runtime by a model.
>
> `evidence` is the phrase from that page which the answer comes from, and a test asserts that it really does appear in the prose. That test is the guard: if somebody rewrites a parish's history and a question stops matching, the suite fails rather than the quiz quietly asking about something the page no longer says.
>
> ## Why only two parishes
>
> Only two have a written history. For the other twenty-nine the quiz does not appear at all, which is the honest answer - questions invented to fill the section would be exactly the thing this app refuses everywhere else.

Complete Source Code — historyQuiz.ts

```
/**
 * Five questions about a parish's own history.
 *
 * ## The one rule
 *
 * Every correct answer is a sentence on the parish's history page.
 * Nothing here is invented, nothing is drawn from outside the diocese's
 * own material, and nothing is generated at runtime by a model.
 *
 * `evidence` is the phrase from that page which the answer comes from,
```

```
 * and a test asserts that it really does appear in the prose. That test
 * is the guard: if somebody rewrites a parish's history and a question
 * stops matching, the suite fails rather than the quiz quietly asking
 * about something the page no longer says.
 *
 * ## Why only two parishes
 *
 * Only two have a written history. For the other twenty-nine the quiz
 * does not appear at all, which is the honest answer - questions
 * invented to fill the section would be exactly the thing this app
 * refuses everywhere else.
 */

export interface QuizItem {
  question: string
  options: string[]
  /** Index into `options`. */
  correct: number
  explanation: string
  /** The phrase on the history page the answer comes from. */
  evidence: string
}

export const HISTORY_QUIZZES: Record<string, QuizItem[]> = {
  "route-mhcp": [
    {
      question: "In what year was Mary Help of Christians Parish established?",
      options: ["1815", "1952", "1898", "2003"],
      correct: 1,
      explanation:
        "The parish was established in 1952, to serve the growing population of Southern Caloocan.",
      evidence: "established in 1952",
    },
    {
      question: "What did the parish grow from?",
      options: [
        "A stone cathedral",
        "A Spanish-era convent",
        "A simple chapel of wood and nipa",
        "A school building",
      ],
      correct: 2,
      explanation:
        "It began as a simple chapel of wood and nipa before becoming the parish that stands today.",
      evidence: "simple chapel of wood and nipa",
    },
    {
      question: "By what Spanish title is the parish's patroness also known?",
      options: ["Nuestra Señora de Guia", "Maria Auxiliadora", "La Naval", "Santa Maria"],
      correct: 1,
      explanation:
        "Mary, Help of Christians is known as Maria Auxiliadora — her role as helper and protector of Christians.",
      evidence: "maria auxiliadora",
    },
    {
      question: "Which saint's spiritual family venerates this title most highly?",
      options: ["St. John Bosco", "St. Francis of Assisi", "St. Dominic", "St. Ignatius of Loyola"],
      correct: 0,
      explanation:
```

```
      "The title is venerated within the Salesian family of St. John Bosco, whose spirit the parish carries.",
    evidence: "st. john bosco",
  },
  {
    question: "How is the patroness depicted in her image?",
    options: [
      "Crowned alone with twelve stars",
      "Standing on a crescent moon",
      "Holding the Child Jesus with both arms open",
      "Seated with an open book",
    ],
    correct: 2,
    explanation:
      "Her image shows her holding the Child Jesus with both arms open, inviting the faithful to bring her their cares.",
    evidence: "holding the child jesus with both arms open",
  },
],

"route-src": [
  {
    question: "In what year was San Roque originally founded as a parish?",
    options: ["1815", "1952", "2003", "1762"],
    correct: 0,
    explanation:
      "San Roque was founded as a parish in 1815, under the Archdiocese of Manila.",
    evidence: "founded as a parish in 1815",
  },
  {
    question: "What role did the church play during the Philippine Revolution?",
    options: [
      "It was closed for the duration",
      "A military post and safe haven for local revolutionaries",
      "A hospital for Spanish soldiers",
      "A printing house for propaganda",
    ],
    correct: 1,
    explanation:
      "The church stood witness to the revolution, serving as a military post and a safe haven for local revolutionaries.",
    evidence: "military post and safe haven for local revolutionaries",
  },
  {
    question: "What happened to San Roque Parish in 2003?",
    options: [
      "It was rebuilt after a fire",
      "It was transferred to another diocese",
      "It was elevated to the dignity of a Cathedral",
      "It was declared a national shrine",
    ],
    correct: 2,
    explanation:
      "When Pope John Paul II established the Diocese of Kalookan in 2003, San Roque was elevated to the dignity of a Cathedral and became the seat of the
Bishop.",
    evidence: "elevated to the dignity of a cathedral",
  },
  {
    question: "Which three cities does the Diocese of Kalookan serve?",
    options: [
      "Caloocan, Malabon, and Navotas",
      "Caloocan, Quezon City, and Manila",
```

```
        "Malabon, Valenzuela, and Navotas",
        "Caloocan, Marikina, and Pasig",
      ],
      correct: 0,
      explanation:
        "The Cathedral guides the spiritual flock of Caloocan, Malabon, and Navotas.",
      evidence: "caloocan, malabon, and navotas",
    },
    {
      question: "Who is San Roque the patron saint of?",
      options: [
        "Farmers and harvesters",
        "Students and teachers",
        "Travellers and sailors",
        "The sick, plague victims, healthcare workers",
      ],
      correct: 3,
      explanation:
        "A 14th-century layman who nursed plague sufferers across Europe, San Roque is the patron of the sick, plague victims, healthcare workers, and epidemics.",
      evidence: "the sick, plague victims, healthcare workers",
    },
  ],
}

/** The quiz for a parish, or null where its history is not written up. */
export function quizFor(routeId: string): QuizItem[] | null {
  return HISTORY_QUIZZES[routeId] ?? null
}

/**
 * Which questions are still blank, numbered from one as the reader sees
 * them. Returned rather than a bare count so the screen can say which.
 */
export function unanswered(answers: (number | null)[], total: number): number[] {
  const blank: number[] = []
  for (let i = 0; i < total; i++) {
    if (answers[i] === null || answers[i] === undefined) blank.push(i + 1)
  }
  return blank
}

export interface QuizResult {
  score: number
  total: number
  /** True where the answer given was right, per question. */
  marks: boolean[]
}

export function gradeQuiz(quiz: QuizItem[], answers: (number | null)[]): QuizResult {
  const marks = quiz.map((q, i) => answers[i] === q.correct)
  return {
    score: marks.filter(Boolean).length,
    total: quiz.length,
    marks,
  }
}
```

1.4.28 APP-098 — homeParish.ts

Code ID	APP-098
File Name	homeParish.ts
File Path	SanctDemo/src/lib/homeParish.ts
Language / Type	TypeScript
Lines	38
Functions / Classes	HOME_PARISH_KEY (exported constant); loadHomeParishId (exported function); saveHomeParishId (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/homeParish.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The pilgrim's home parish, chosen once on first run and remembered thereafter. Follows the same shape as src/lib/rosarySettings.ts (typed module, single localStorage key, safe fallback on missing/corrupt data), but deliberately uses a *different* key — this is not rosary data and must not collide with it.
>
> Unlike rosary settings, the set of valid values isn't a fixed enum: it's whatever live parish ids exist in ROUTES at the time of validation. So the loader takes the caller's current list of valid ids rather than hardcoding them here, keeping this module decoupled from ../data.

Complete Source Code — homeParish.ts

```
// The pilgrim's home parish, chosen once on first run and remembered
// thereafter. Follows the same shape as src/lib/rosarySettings.ts (typed
// module, single localStorage key, safe fallback on missing/corrupt data),
// but deliberately uses a *different* key — this is not rosary data and
// must not collide with it.
//
// Unlike rosary settings, the set of valid values isn't a fixed enum: it's
// whatever live parish ids exist in ROUTES at the time of validation. So the
// loader takes the caller's current list of valid ids rather than hardcoding
// them here, keeping this module decoupled from ../data.

export const HOME_PARISH_KEY = "sanctiwalk.homeParish";

/**
 * Never throws. Returns null when nothing is stored, the stored value is
 * malformed, or it no longer matches a known parish id (e.g. diocese data
 * changed underneath a stale localStorage entry).
 */
export function loadHomeParishId(validParishIds: readonly string[]): string | null {
  try {
    const raw = localStorage.getItem(HOME_PARISH_KEY);
    if (!raw) return null;
    const parsed = JSON.parse(raw);
```

```
    if (typeof parsed !== "string" || !parsed) return null;
    return validParishIds.indexOf(parsed) !== -1 ? parsed : null;
  } catch {
    return null;
  }
}

export function saveHomeParishId(parishId: string): void {
  try {
    localStorage.setItem(HOME_PARISH_KEY, JSON.stringify(parishId));
  } catch {
    // Storage may be unavailable (private mode, quota, etc). The choice
    // still applies for this session via React state.
  }
}
```

1.4.29 APP-099 — idToken.ts

Code ID	APP-099
File Name	idToken.ts
File Path	SanctDemo/src/lib/idToken.ts
Language / Type	TypeScript
Lines	34
Functions / Classes	idToken (exported function); isSignedIn (exported function)
Related Files (its imports)	./firebase
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/idToken.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The signed-in user's identity token, for calls the backend must trust.
>
> The scanner's allowance is twenty a day per person per parish, and the server has to know which person without taking the client's word for it. A Firebase ID token is signed by Google and verified by the Admin SDK on the other side, so a forged one cannot pass and a user cannot mint themselves a second identity to get a second twenty.
>
> Deliberately not cached here. The SDK already caches the token and refreshes it when it is close to expiring; caching it again in app code is how a request ends up presenting a token that expired four minutes ago and gets a 401 nobody can explain.

Complete Source Code — idToken.ts

```
/**
 * The signed-in user's identity token, for calls the backend must trust.
 *
 * The scanner's allowance is twenty a day per person per parish, and
 * the server has to know which person without taking the client's word
 * for it. A Firebase ID token is signed by Google and verified by the
 * Admin SDK on the other side, so a forged one cannot pass and a user
 * cannot mint themselves a second identity to get a second twenty.
 *
 * Deliberately not cached here. The SDK already caches the token and
 * refreshes it when it is close to expiring; caching it again in app
 * code is how a request ends up presenting a token that expired four
 * minutes ago and gets a 401 nobody can explain.
 */
import { auth } from './firebase'

/** The current user's token, or null when nobody is signed in. */
export async function idToken(): Promise<string | null> {
  const user = auth.currentUser
  if (!user) return null
  try {
    return await user.getIdToken()
```

```
  } catch {
    // Offline, or the account was disabled while the app was open.
    // Null reads as "not signed in", which is the right thing for the
    // caller to do about it either way.
    return null
  }
}

/** Whether anyone is signed in at all, for deciding what to show. */
export function isSignedIn(): boolean {
  return auth.currentUser !== null
}
```

1.4.30 APP-100 — itemContent.ts

Code ID	APP-100
File Name	itemContent.ts
File Path	SanctDemo/src/lib/itemContent.ts
Language / Type	TypeScript
Lines	204
Functions / Classes	ItemContent (exported type); ResolvedItem (exported type); managedFor (internal function); listOrFallback (internal function); textOrFallback (internal function); resolveMinistry (exported function); resolveSacrament (exported function); linesToList (exported function); listToLines (exported function); mergeItemContent (exported function); hasParishContent (exported function)
Related Files (its imports)	./parishContent, ../data, ./language
Where It Is Used	src/components/ItemDetailSections.tsx, src/components/MinistriesTab.tsx, src/components/SacramentsTab.tsx, src/components/SanctiHost.tsx, src/lib/sanctiAnswers.ts
How to Recognize It	Open SanctDemo/src/lib/itemContent.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — itemContent.ts

```
import type { ParishContent } from "./parishContent";
import { MINISTRIES, SACRAMENTS } from "../data";
import { pick, type Language } from "./language";
import {
  MINISTRY_DESCRIPTION_EN, SACRAMENT_DESCRIPTION_FIL, bilingualFor,
} from "./contentTranslations";

/**
 * What a parish has written about one ministry or sacrament.
 *
 * ## Why this overlays data.ts rather than replacing it
 *
 * The descriptions and canonical requirements in data.ts came from the
 * parish and are correct. 29 of the 31 parishes have written nothing
 * here, and must keep showing what they have. So every field is optional
 * and falls back: a parish that edits only the schedule keeps the
 * description the app shipped with.
 *
 * ## Why this is not the application record
 *
 * This is the page a pilgrim reads before applying. The application is
 * what they send afterwards. Keeping them apart means a parish can
 * correct a requirement without touching anybody's submission, which is
 * the whole reason the brief asks for them to be separate.
 *
 * ## Why nothing here is invented
```

```
 *
 * Every field starts empty. The app never writes a requirement, a fee or
 * a schedule that a parish did not type - the standing rule in this
 * codebase is that a plausible-looking wrong answer about a sacrament is
 * worse than no answer.
 */
export interface ItemContent {
  /** Replaces the compiled description when the parish writes one. */
  about?: string;
  /** Replaces the compiled requirements list entirely when non-empty. */
  requirements?: string[];
  /** "Saturdays, 9:00 AM" - free text, because parishes phrase it their way. */
  schedule?: string;
  /** The steps, in order, from applying to the day itself. */
  process?: string[];
  /** "Bring the originals", "Arrive fifteen minutes early". */
  reminders?: string[];
  /** Who to ring, and when the office is open. */
  contact?: string;
  /** Responsibilities, for ministries. */
  responsibilities?: string[];
}

/** The content as it should be displayed: parish's words over the app's. */
export interface ResolvedItem {
  id: string;
  name: string;
  about: string;
  requirements: string[];
  schedule: string;
  process: string[];
  reminders: string[];
  responsibilities: string[];
  contact: string;
}

function managedFor(
  content: ParishContent | null | undefined,
  itemId: string,
): ItemContent {
  return content?.itemContent?.[itemId] ?? {};
}

/**
 * A list wins only when it has something in it.
 *
 * An admin who opens the editor, saves, and types nothing writes `[]`.
 * Treating that as "the parish says there are no requirements" would
 * blank a correct list that came from the parish in the first place.
 */
function listOrFallback(managed: string[] | undefined, compiled: string[]): string[] {
  return managed && managed.length > 0 ? managed : compiled;
}

function textOrFallback(managed: string | undefined, compiled: string): string {
  return managed?.trim() ? managed.trim() : compiled;
}

export function resolveMinistry(
```

```
  content: ParishContent | null | undefined,
  ministryId: string,
  /**
   * The reader's language. The compiled ministry descriptions are all
   * Tagalog, so an English reader got fifteen blocks of Tagalog until
   * this was threaded through.
   *
   * A parish's OWN text is never translated. If the office wrote it,
   * it is shown as written - guessing at a translation of something a
   * parish typed this morning is exactly the kind of invention this
   * codebase refuses everywhere else.
   */
  language: Language = "en",
): ResolvedItem | null {
  const base = MINISTRIES.find(m => m.id === ministryId);
  if (!base) return null;
  const managed = managedFor(content, ministryId);
  const described = pick(
    bilingualFor(base.description, MINISTRY_DESCRIPTION_EN[base.id], "fil"),
    language,
  );
  return {
    id: base.id,
    name: base.name,
    about: textOrFallback(managed.about, described),
    requirements: listOrFallback(managed.requirements, base.requirements),
    schedule: managed.schedule?.trim() ?? "",
    process: managed.process ?? [],
    reminders: managed.reminders ?? [],
    responsibilities: managed.responsibilities ?? [],
    contact: managed.contact?.trim() ?? "",
  };
}

export function resolveSacrament(
  content: ParishContent | null | undefined,
  sacramentId: string,
  /** See resolveMinistry. The compiled sacrament text is English only. */
  language: Language = "en",
): ResolvedItem | null {
  const base = SACRAMENTS.find(s => s.id === sacramentId);
  if (!base) return null;
  const managed = managedFor(content, sacramentId);
  const described = pick(
    bilingualFor(base.description, SACRAMENT_DESCRIPTION_FIL[base.id], "en"),
    language,
  );
  return {
    id: base.id,
    name: base.name,
    about: textOrFallback(managed.about, described),
    requirements: listOrFallback(managed.requirements, base.requirements),
    // The compiled scheduleDetails is the parish's own line and stays
    // the fallback - a sacrament page with no schedule at all is the
    // most-asked question in a parish office.
    schedule: textOrFallback(managed.schedule, base.scheduleDetails),
    process: managed.process ?? [],
    reminders: managed.reminders ?? [],
    responsibilities: [],
```

```
    contact: managed.contact?.trim() ?? "",
  };
}

/**
 * Turns a textarea into a list, one item per line.
 *
 * Blank lines are dropped rather than becoming empty bullets - people
 * leave one between paragraphs out of habit, and a list with a gap in it
 * looks broken.
 */
export function linesToList(value: string): string[] {
  return value.split("\n").map(line => line.trim()).filter(Boolean);
}

export function listToLines(value: string[] | undefined): string {
  return (value ?? []).join("\n");
}

/**
 * One item's content merged into the parish's document.
 *
 * Fields the admin left empty are REMOVED rather than written as "" or
 * [], so the fallback to data.ts keeps working. Writing an empty string
 * would mean "the parish says this is blank", which is a different and
 * wrong claim.
 */
export function mergeItemContent(
  existing: ParishContent | null | undefined,
  itemId: string,
  patch: ItemContent,
): Record<string, ItemContent> {
  const all = { ...(existing?.itemContent ?? {}) };
  const clean: ItemContent = {};

  if (patch.about?.trim()) clean.about = patch.about.trim();
  if (patch.schedule?.trim()) clean.schedule = patch.schedule.trim();
  if (patch.contact?.trim()) clean.contact = patch.contact.trim();
  if (patch.requirements?.length) clean.requirements = patch.requirements;
  if (patch.process?.length) clean.process = patch.process;
  if (patch.reminders?.length) clean.reminders = patch.reminders;
  if (patch.responsibilities?.length) clean.responsibilities = patch.responsibilities;

  if (Object.keys(clean).length === 0) {
    // Nothing written: drop the key entirely rather than leaving {}
    // behind for every item an admin ever opened.
    delete all[itemId];
  } else {
    all[itemId] = clean;
  }
  return all;
}

/** True when the parish has written anything at all about this one. */
export function hasParishContent(
  content: ParishContent | null | undefined,
  itemId: string,
): boolean {
  return Object.keys(managedFor(content, itemId)).length > 0;
```

```
}
```

1.4.31 APP-101 — language.ts

Code ID	APP-101
File Name	language.ts
File Path	SanctDemo/src/lib/language.ts
Language / Type	TypeScript
Lines	145
Functions / Classes	Language (exported type); LANGUAGE_KEY (exported constant); LANGUAGE_NAMES (exported constant); Bilingual (exported type); isBilingual (exported function); pick (exported function); languageFromUrl (internal function); loadLanguage (exported function); LANGUAGE_EVENT (exported constant); saveLanguage (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx, src/components/ChurchHistory.tsx, src/components/HomeQuickLinks.tsx, src/components/MinistriesTab.tsx, src/components/MinistrySacramentPair.tsx, src/components/SacramentsTab.tsx, src/components/SanctiHost.tsx, src/lib/contentTranslations.ts, src/lib/historyNarration.ts, src/lib/itemContent.ts, src/lib/speech.ts, src/lib/ui.ts, src/lib/useNarration.ts
How to Recognize It	Open SanctDemo/src/lib/language.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — language.ts

```
import {
  loadRosarySettings, saveRosarySettings, type RosaryLanguage,
} from "./rosarySettings";

/**
 * One language for the whole app.
 *
 * ## The problem
 *
 * SanctiWalk's content was written in whichever language each piece was
 * written in: all fifteen ministry descriptions are Tagalog, every
 * sacrament and every station description is English. A pilgrim reading
 * in English hits fifteen blocks of Tagalog; a pilgrim reading in
 * Tagalog hits English everywhere else. Neither is a choice anybody
 * made - it is just how the content arrived.
 *
 * So the language stops being a property of each paragraph and becomes
 * a property of the reader. Every piece of content carries both, and
 * `pick()` chooses.
 *
 * ## Why this does not replace the rosary's own setting
 *
 * The rosary's language lives in localStorage under a contract that is
 * duplicated, deliberately, inside public/rosary/index.html - a static
 * file with its own inline JavaScript that cannot import from here.
```

```
 * Rewriting that contract would mean changing two things that have no
 * way of checking each other.
 *
 * Instead this is the master, and setting it writes the rosary's field
 * too. The two can never disagree, and the static file keeps reading
 * exactly what it always read.
 */

export type Language = RosaryLanguage; // "en" | "fil"

export const LANGUAGE_KEY = "sanctiwalk.language";

export const LANGUAGE_NAMES: Record<Language, string> = {
  en: "English",
  fil: "Tagalog",
};

/**
 * Text that exists in both languages.
 *
 * Both are required, not optional. An optional second language is how
 * the app got into this state: it lets a contributor add one side and
 * move on, and the gap is only found by a reader.
 */
export interface Bilingual {
  en: string;
  fil: string;
}

export function isBilingual(value: unknown): value is Bilingual {
  return (
    !!value &&
    typeof value === "object" &&
    typeof (value as Bilingual).en === "string" &&
    typeof (value as Bilingual).fil === "string"
  );
}

/**
 * The reader's side of a bilingual string.
 *
 * Accepts a plain string too, so a call site can be converted before
 * its data is, and so anything still monolingual keeps rendering rather
 * than disappearing mid-migration.
 */
export function pick(value: Bilingual | string | undefined, language: Language): string {
  if (value === undefined) return "";
  if (typeof value === "string") return value;
  // Falls back to the other language rather than showing nothing. A
  // paragraph in the wrong language is worth more than a blank space,
  // and it is visible, which is how the gap gets reported.
  return value[language] || value[language === "en" ? "fil" : "en"] || "";
}

/**
 * A language pinned by the URL, as in /app?lang=en.
 *
 * This exists for the landing page, whose phone mockup is a real
 * SanctiWalk in an iframe. The iframe is same-origin, so it reads the
```

```
 * same localStorage as the app - which means a visitor who had once
 * set Tagalog saw the marketing demo in Tagalog, and so did anyone
 * looking over their shoulder. The demo is a showcase for strangers
 * and has to be legible to all of them.
 *
 * It deliberately does NOT persist. Pinning the preview must not
 * silently rewrite the language of the pilgrim's own app, which is the
 * same storage.
 */
function languageFromUrl(): Language | null {
  try {
    const value = new URLSearchParams(window.location.search).get("lang");
    return value === "en" || value === "fil" ? value : null;
  } catch {
    return null;
  }
}

export function loadLanguage(): Language {
  try {
    const pinned = languageFromUrl();
    if (pinned) return pinned;

    const stored = localStorage.getItem(LANGUAGE_KEY);
    if (stored === "en" || stored === "fil") return stored;
    // Nothing chosen yet: inherit whatever the rosary was set to, so a
    // pilgrim who already picked Tagalog there is not asked twice.
    return loadRosarySettings().language;
  } catch {
    return "en";
  }
}

/**
 * Fired when the language changes, for every hook in this tab.
 *
 * `storage` only reaches OTHER tabs, so without this each
 * `useLanguage()` kept its own stale copy: changing the language in Me
 * updated Me, and the tab bar and Sancti's button stayed in the old
 * language until a reload. One broadcast keeps them all in step.
 */
export const LANGUAGE_EVENT = "sanctiwalk:language";

export function saveLanguage(language: Language): void {
  try {
    localStorage.setItem(LANGUAGE_KEY, language);
    // Keep the rosary in step. See the note at the top.
    const rosary = loadRosarySettings();
    if (rosary.language !== language) {
      saveRosarySettings({ ...rosary, language });
    }
  } catch {
    // Storage is blocked; the broadcast below still updates this run.
  }

  try {
    window.dispatchEvent(new CustomEvent(LANGUAGE_EVENT, { detail: language }));
  } catch {
    // Private mode. The choice holds for this run.
```

```
  }
}
```

1.4.32 APP-102 — liturgical.ts

Code ID	APP-102
File Name	liturgical.ts
File Path	SanctDemo/src/lib/liturgical.ts
Language / Type	TypeScript
Lines	195
Functions / Classes	LiturgicalSeason (exported type); LiturgicalColour (exported type); LiturgicalDay (exported type); utc (internal function); dayOf (internal function); addDays (internal function); daysBetween (internal function); sundayOnOrBefore (internal function); easterSunday (exported function); adventStart (exported function); baptismOfTheLord (exported function); ordinal (internal function); liturgicalDay (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/Dashboard.tsx, src/components/MassScheduleCard.tsx, src/components/VerseCard.tsx, src/lib/liturgicalColours.ts, src/lib/verses.ts
How to Recognize It	Open SanctDemo/src/lib/liturgical.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The liturgical day, computed rather than stored.
>
> The redesign's Home screen names the day the way a parish bulletin does — "Wednesday of the 21st Week in Ordinary Time" — which no data file here carries and which changes every day. Everything below derives from one fixed point, the date of Easter, so it stays correct in any year without anyone maintaining a table.
>
> This is the Roman Rite as celebrated in the Philippines. Two local details matter and are handled: Epiphany is transferred to the Sunday between 2 and 8 January rather than fixed to the 6th, and the Baptism of the Lord falls on the Sunday after that.
>
> Deliberately free of clock access — every function takes the date it should reason about, which is what makes a calendar spanning a whole year testable.

Complete Source Code — liturgical.ts

```
// The liturgical day, computed rather than stored.
//
// The redesign's Home screen names the day the way a parish bulletin does —
// "Wednesday of the 21st Week in Ordinary Time" — which no data file here
// carries and which changes every day. Everything below derives from one
// fixed point, the date of Easter, so it stays correct in any year without
// anyone maintaining a table.
//
// This is the Roman Rite as celebrated in the Philippines. Two local details
// matter and are handled: Epiphany is transferred to the Sunday between 2 and
// 8 January rather than fixed to the 6th, and the Baptism of the Lord falls
// on the Sunday after that.
//
```

```
// Deliberately free of clock access — every function takes the date it should
// reason about, which is what makes a calendar spanning a whole year testable.

export type LiturgicalSeason =
  | 'Advent'
  | 'Christmas'
  | 'Ordinary Time'
  | 'Lent'
  | 'Triduum'
  | 'Easter'

/** The vesture colour of the day, for the band the redesign puts on Home. */
export type LiturgicalColour = 'violet' | 'white' | 'green' | 'red' | 'rose'

export interface LiturgicalDay {
  season: LiturgicalSeason
  /** Week within the season, 1-based. Null where the season isn't counted in weeks. */
  week: number | null
  colour: LiturgicalColour
  /** "Wednesday of the 21st Week in Ordinary Time" */
  name: string
  /** A named solemnity or feast that outranks the day, when there is one. */
  feast: string | null
}

const DAY_MS = 86_400_000
const WEEKDAYS = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']

/** Midnight UTC for a Y/M/D, so arithmetic never trips over local DST. */
function utc(year: number, month: number, day: number): Date {
  return new Date(Date.UTC(year, month, day))
}

/** Strips the time from a date, keeping its calendar day. */
function dayOf(date: Date): Date {
  return utc(date.getFullYear(), date.getMonth(), date.getDate())
}

function addDays(date: Date, days: number): Date {
  return new Date(date.getTime() + days * DAY_MS)
}

function daysBetween(from: Date, to: Date): number {
  return Math.round((to.getTime() - from.getTime()) / DAY_MS)
}

/** The Sunday on or before `date`. Liturgical weeks run Sunday to Saturday. */
function sundayOnOrBefore(date: Date): Date {
  return addDays(date, -date.getUTCDay())
}

/**
 * Easter Sunday, by the Anonymous Gregorian ("Meeus/Jones/Butcher") algorithm.
 *
 * The whole calendar hangs off this one date: Ash Wednesday, Pentecost and
 * both stretches of Ordinary Time are all measured from it.
 */
export function easterSunday(year: number): Date {
  const a = year % 19
```

```
  const b = Math.floor(year / 100)
  const c = year % 100
  const d = Math.floor(b / 4)
  const e = b % 4
  const f = Math.floor((b + 8) / 25)
  const g = Math.floor((b - f + 1) / 3)
  const h = (19 * a + b - d - g + 15) % 30
  const i = Math.floor(c / 4)
  const k = c % 4
  const l = (32 + 2 * e + 2 * i - h - k) % 7
  const m = Math.floor((a + 11 * h + 22 * l) / 451)
  const month = Math.floor((h + l - 7 * m + 114) / 31)
  const day = ((h + l - 7 * m + 114) % 31) + 1
  return utc(year, month - 1, day)
}

/** The First Sunday of Advent — the fourth Sunday before Christmas Day. */
export function adventStart(year: number): Date {
  const christmas = utc(year, 11, 25)
  // The Sunday on or before Christmas is the fourth Sunday of Advent unless
  // Christmas *is* a Sunday, in which case that Sunday belongs to Christmas.
  const fourth = christmas.getUTCDay() === 0 ? addDays(christmas, -7) : sundayOnOrBefore(christmas)
  return addDays(fourth, -21)
}

/**
 * The Baptism of the Lord, which closes Christmastide.
 *
 * In the Philippines Epiphany is kept on the Sunday falling between 2 and 8
 * January, and the Baptism on the Sunday after. When Epiphany lands on 7 or 8
 * January the Baptism is moved to the following Monday so the two do not
 * collide.
 */
export function baptismOfTheLord(year: number): Date {
  const jan2 = utc(year, 0, 2)
  const offset = (7 - jan2.getUTCDay()) % 7
  const epiphany = addDays(jan2, offset) // the Sunday between 2 and 8 January
  const date = epiphany.getUTCDate()
  return date >= 7 ? addDays(epiphany, 1) : addDays(epiphany, 7)
}

function ordinal(n: number): string {
  const suffix =
    n % 100 >= 11 && n % 100 <= 13 ? 'th' : ['th', 'st', 'nd', 'rd'][n % 10] ?? 'th'
  return `${n}${suffix}`
}

/**
 * Names the liturgical day for `date`.
 *
 * Ordinary Time is the awkward part, and the reason it is worth writing out.
 * It comes in two stretches with one continuous numbering: the first runs
 * from the day after the Baptism to the day before Ash Wednesday, the second
 * from the day after Pentecost to the day before Advent. The second stretch
 * is numbered *backwards* from the end — the last week before Advent is
 * always the 34th — because the first stretch's length varies with the date
 * of Easter, so counting forwards would leave a gap or an overlap.
 */
export function liturgicalDay(date: Date): LiturgicalDay {
```

```
const today = dayOf(date)
const year = today.getUTCFullYear()
const weekday = WEEKDAYS[today.getUTCDay()]

const easter = easterSunday(year)
const ashWednesday = addDays(easter, -46)
const holyThursday = addDays(easter, -3)
const pentecost = addDays(easter, 49)
const baptism = baptismOfTheLord(year)
const advent = adventStart(year)
const christmasDay = utc(year, 11, 25)

const named = (season: LiturgicalSeason, week: number | null, colour: LiturgicalColour, name: string, feast: string | null = null): LiturgicalDay =>
  ({ season, week, colour, name, feast })

// Fixed points that outrank whatever season they land in.
if (today.getTime() === easter.getTime()) return named('Easter', 1, 'white', 'Easter Sunday', 'Easter Sunday')
if (today.getTime() === pentecost.getTime()) return named('Easter', 8, 'red', 'Pentecost Sunday', 'Pentecost Sunday')
if (today.getTime() === christmasDay.getTime()) return named('Christmas', null, 'white', 'The Nativity of the Lord', 'Christmas Day')
if (today.getTime() === ashWednesday.getTime()) return named('Lent', 1, 'violet', 'Ash Wednesday', 'Ash Wednesday')

// Advent through to the end of the civil year.
if (today >= advent && today < christmasDay) {
  const week = Math.floor(daysBetween(advent, sundayOnOrBefore(today)) / 7) + 1
  return named('Advent', week, week === 3 ? 'rose' : 'violet', `${weekday} of the ${ordinal(week)} Week of Advent`)
}

// Christmastide spans the new year, so it is two ranges, not one.
if (today >= christmasDay || today <= baptism) {
  return named('Christmas', null, 'white', `${weekday} of the Christmas Season`)
}

if (today >= holyThursday && today < easter) {
  return named('Triduum', null, 'white', `${weekday} of the Sacred Triduum`)
}

if (today > ashWednesday && today < holyThursday) {
  // Lent's weeks are counted from the First Sunday of Lent, which is the
  // Sunday after Ash Wednesday — the days between belong to "after Ashes".
  const firstSunday = addDays(easter, -42)
  if (today < firstSunday) return named('Lent', null, 'violet', `${weekday} after Ash Wednesday`)
  const week = Math.floor(daysBetween(firstSunday, sundayOnOrBefore(today)) / 7) + 1
  return named('Lent', week, week === 4 ? 'rose' : 'violet', `${weekday} of the ${ordinal(week)} Week of Lent`)
}

if (today > easter && today < pentecost) {
  const week = Math.floor(daysBetween(easter, sundayOnOrBefore(today)) / 7) + 1
  return named('Easter', week, 'white', `${weekday} of the ${ordinal(week)} Week of Easter`)
}

// Everything left is Ordinary Time.
const sunday = sundayOnOrBefore(today)
let week: number
if (today > baptism && today < ashWednesday) {
  // Counted forwards: the week containing the Baptism is the first.
  week = Math.floor(daysBetween(sundayOnOrBefore(baptism), sunday) / 7) + 1
} else {
  // Counted backwards from Christ the King, always the 34th and always the
  // Sunday before Advent.
```

```
    const christTheKing = addDays(advent, -7)
    week = 34 - Math.floor(daysBetween(sunday, christTheKing) / 7)
  }
  return named('Ordinary Time', week, 'green', `${weekday} of the ${ordinal(week)} Week in Ordinary Time`)
}
```

1.4.33 APP-103 — liturgicalColours.ts

Code ID	APP-103
File Name	liturgicalColours.ts
File Path	SanctDemo/src/lib/liturgicalColours.ts
Language / Type	TypeScript
Lines	53
Functions / Classes	SeasonAccent (exported type); seasonAccent (exported function); ACCENTED_COLOURS (exported constant)
Related Files (its imports)	./liturgical
Where It Is Used	src/components/VerseCard.tsx
How to Recognize It	Open SanctDemo/src/lib/liturgicalColours.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The liturgical colour, turned into something a card can actually be painted with.
>
> liturgical.ts already decides the colour of the day — the vesture colour the priest wears: green through Ordinary Time, violet in Advent and Lent, white at Christmas and Easter, red at Pentecost, rose on Gaudete and Laetare. That is the authority for this; nothing here re-derives the calendar.
>
> Each colour becomes three values rather than one, because one hex cannot do the job: a wash light enough to read body text on, a border a shade firmer, and an ink dark enough to clear WCAG AA on that wash. Painting the card in the vesture colour itself would put white text on Christmas and near-black on rose, which is how a seasonal accent turns into an unreadable card.
>
> One deliberate substitution: WHITE becomes GOLD. White vestments are worn with gold, and the app's own card is already warm off-white (#FAF8F4), so a white wash on a white card is no wash at all. Gold is what a missal or a set of white-and-gold vestments actually looks like, and it is visible.
>
> Pure and dependency-free, like contrast.ts, so the pairings can be tested rather than eyeballed.

Complete Source Code — liturgicalColours.ts

```
// The liturgical colour, turned into something a card can actually be painted
// with.
//
// liturgical.ts already decides the colour of the day — the vesture colour the
// priest wears: green through Ordinary Time, violet in Advent and Lent, white
// at Christmas and Easter, red at Pentecost, rose on Gaudete and Laetare.
// That is the authority for this; nothing here re-derives the calendar.
//
// Each colour becomes three values rather than one, because one hex cannot do
// the job: a wash light enough to read body text on, a border a shade firmer,
// and an ink dark enough to clear WCAG AA on that wash. Painting the card in
// the vesture colour itself would put white text on Christmas and near-black
```

```
// on rose, which is how a seasonal accent turns into an unreadable card.
//
// One deliberate substitution: WHITE becomes GOLD. White vestments are worn
// with gold, and the app's own card is already warm off-white (#FAF8F4), so a
// white wash on a white card is no wash at all. Gold is what a missal or a
// set of white-and-gold vestments actually looks like, and it is visible.
//
// Pure and dependency-free, like contrast.ts, so the pairings can be tested
// rather than eyeballed.

import type { LiturgicalColour } from './liturgical'

export interface SeasonAccent {
  /** A light wash for the card's background. */
  tint: string
  /** A firmer shade of the same hue for its border. */
  border: string
  /** Text dark enough to clear WCAG AA on `tint`. */
  ink: string
}

const ACCENTS: Record<LiturgicalColour, SeasonAccent> = {
  // Ordinary Time runs for most of the year, so this is the wash the card
  // wears most days - and at #EAF2EC it read as off-white rather than as a
  // season. Pulled towards the app's own success green (#4C7A5C) until the
  // card is recognisably green at a glance, with the ink darkened to keep
  // body text clear of AA on the firmer wash.
  green: { tint: '#D8EADE', border: '#9CC6AC', ink: '#17512F' },
  violet: { tint: '#EFEBF6', border: '#D0C5E5', ink: '#4B2A80' },
  white: { tint: '#F6F0E2', border: '#E0D2AE', ink: '#6E5010' },
  red: { tint: '#F7EAEA', border: '#E5C4C4', ink: '#8B1A20' },
  rose: { tint: '#F9ECF1', border: '#EBCBD9', ink: '#9A2C56' },
}

/** The wash, border and ink for a liturgical colour. */
export function seasonAccent(colour: LiturgicalColour): SeasonAccent {
  return ACCENTS[colour]
}

/** Every colour the calendar can return, for tests and for iteration. */
export const ACCENTED_COLOURS = Object.keys(ACCENTS) as LiturgicalColour[]
```

1.4.34 APP-104 — mapAccuracy.ts

Code ID	APP-104
File Name	mapAccuracy.ts
File Path	SanctDemo/src/lib/mapAccuracy.ts
Language / Type	TypeScript
Lines	97
Functions / Classes	EQUATOR_METRES_PER_PIXEL (exported constant); metresPerPixelAtZoomZero (exported function); accuracyRadiusPixels (exported function); accuracyRadiusExpression (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/lib/mapAccuracy.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The accuracy disc's radius, in pixels, as a MapLibre expression.
>
> The GPS reports its confidence as a distance on the ground - "somewhere within 20 metres". MapLibre's `circle-radius` is in SCREEN pixels, and the two only line up at one zoom level. Hardcoding a pixel radius would draw a disc that claims 20 m when zoomed in and 200 m when zoomed out while looking identical, which is worse than drawing nothing: a confident picture of the wrong thing.
>
> Web Mercator gives the conversion:
>
> metresPerPixel = 156543.03392 * cos(latitude) / 2^zoom
>
> so
>
> pixels = metres * 2^zoom / (156543.03392 * cos(latitude))
>
> The zoom term stays inside the expression, where MapLibre re-evaluates it every frame as the map zooms. cos(latitude) is folded in here as a constant: MapLibre expressions have no cos(), and the map never spans enough latitude for the error to be visible - across the whole diocese it is under a tenth of a percent.

Complete Source Code — mapAccuracy.ts

```
/**
 * The accuracy disc's radius, in pixels, as a MapLibre expression.
 *
 * The GPS reports its confidence as a distance on the ground - "somewhere
 * within 20 metres". MapLibre's `circle-radius` is in SCREEN pixels, and
 * the two only line up at one zoom level. Hardcoding a pixel radius would
 * draw a disc that claims 20 m when zoomed in and 200 m when zoomed out
```

```
 * while looking identical, which is worse than drawing nothing: a
 * confident picture of the wrong thing.
 *
 * Web Mercator gives the conversion:
 *
 *     metresPerPixel = 156543.03392 * cos(latitude) / 2^zoom
 *
 * so
 *
 *     pixels = metres * 2^zoom / (156543.03392 * cos(latitude))
 *
 * The zoom term stays inside the expression, where MapLibre re-evaluates
 * it every frame as the map zooms. cos(latitude) is folded in here as a
 * constant: MapLibre expressions have no cos(), and the map never spans
 * enough latitude for the error to be visible - across the whole diocese
 * it is under a tenth of a percent.
 */
import type { DataDrivenPropertyValueSpecification } from "maplibre-gl";

/** Metres per pixel at zoom 0 on the equator, the Web Mercator constant. */
export const EQUATOR_METRES_PER_PIXEL = 156543.03392;

export function metresPerPixelAtZoomZero(latitude: number): number {
  return EQUATOR_METRES_PER_PIXEL * Math.cos((latitude * Math.PI) / 180);
}

/** What `pixels` the expression will produce at a given zoom. For tests. */
export function accuracyRadiusPixels(
  metres: number,
  latitude: number,
  zoom: number,
): number {
  return (metres * 2 ** zoom) / metresPerPixelAtZoomZero(latitude);
}

/**
 * The zoom range the stops span. MapLibre's own maximum is 24; 22 is past
 * anything this map allows and keeps the numbers small.
 */
const MIN_ZOOM = 0;
const MAX_ZOOM = 22;

/**
 * The radius as a top-level `interpolate` on zoom.
 *
 * ## Why not the arithmetic version
 *
 * This used to be the formula written straight out:
 *
 *     ["/", ["*", metres, ["^", 2, ["zoom"]]], metresPerPixel0]
 *
 * which is correct arithmetic and an illegal expression. MapLibre allows
 * `["zoom"]` ONLY as the direct input to a top-level `step` or
 * `interpolate`; nested inside `^` it is rejected with
 *
 *     "zoom" expression may only be used as input to a top-level "step"
 *     or "interpolate" expression
 *
 * on every single layer update. That had two consequences, and the second
```

```
 * was much worse than the first: the accuracy disc never drew at all, and
 * the stream of errors tripped the live map's error threshold and sent
 * the whole screen to the schematic fallback as soon as GPS produced a
 * fix. On a desktop with no GPS neither happened, which is why it looked
 * fine in a browser and broke on every phone.
 *
 * ## Why this is exact, not an approximation
 *
 * Interpolating with `["exponential", 2]` between two stops gives
 *
 *     r = r0 + (2^(z-z0) - 1) / (2^(z1-z0) - 1) * (r1 - r0)
 *
 * and substituting r0 = k·2^z0, r1 = k·2^z1 collapses it to exactly
 * k·2^z. So two stops reproduce the formula at every zoom between them,
 * not merely near them - there is no error to trade off against more
 * stops.
 */
export function accuracyRadiusExpression(
  metres: number,
  latitude: number,
): DataDrivenPropertyValueSpecification<number> {
  return [
    "interpolate",
    ["exponential", 2],
    ["zoom"],
    MIN_ZOOM,
    accuracyRadiusPixels(metres, latitude, MIN_ZOOM),
    MAX_ZOOM,
    accuracyRadiusPixels(metres, latitude, MAX_ZOOM),
  ] as DataDrivenPropertyValueSpecification<number>;
}
```

1.4.35 APP-105 — mapMarkers.ts

Code ID	APP-105
File Name	mapMarkers.ts
File Path	SanctDemo/src/lib/mapMarkers.ts
Language / Type	TypeScript
Lines	72
Functions / Classes	MarkerParish (exported type); buildChurchPinElement (exported function); churchGlyph (internal function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/lib/mapMarkers.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Church-glyph pin and popup-card DOM builders for the live diocese map (DioceseMapLive.tsx). Extracted into pure functions — MapLibre markers and popups live outside React's tree (they're detached DOM nodes handed to MapLibre, not JSX), and this environment's headless browser can't paint the MapLibre WebGL canvas at all (see docs/reports/ map-prototype-design-swap.md), so this is the only way the pin markup and its live-vs-coming-soon branching can be verified — by building the DOM directly in a unit test rather than by screenshot.
>
> The glyph itself (a body, a roof and a cross) is the client's own earlier prototype's design (DioceseMap.jsx), reused verbatim.

Complete Source Code — mapMarkers.ts

```
// Church-glyph pin and popup-card DOM builders for the live diocese map
// (DioceseMapLive.tsx). Extracted into pure functions — MapLibre markers and
// popups live outside React's tree (they're detached DOM nodes handed to
// MapLibre, not JSX), and this environment's headless browser can't paint
// the MapLibre WebGL canvas at all (see docs/reports/
// map-prototype-design-swap.md), so this is the only way the pin markup
// and its live-vs-coming-soon branching can be verified — by
// building the DOM directly in a unit test rather than by screenshot.
//
// The glyph itself (a body, a roof and a cross) is the client's own earlier
// prototype's design (DioceseMap.jsx), reused verbatim.

export interface MarkerParish {
  name: string
  location?: string
  isLive: boolean
  /**
   * The parish's patron photograph. Given, the pin becomes that picture in
   * a navy ring instead of the generic church glyph.
   *
   * Only a handful of parishes have one. The rest keep the glyph rather
   * than showing a stand-in: 31 identical placeholder discs would read as
```

```
   * 31 photographs nobody can tell apart, which is worse than a symbol
   * that is honestly a symbol.
   */
  photoUrl?: string
}

export function buildChurchPinElement(parish: MarkerParish): HTMLButtonElement {
  const el = document.createElement('button')
  el.type = 'button'
  // Every pin is activatable, including coming-soon ones: tapping is how
  // you find out which parish a pin is, so silent pins would be a dead end.
  el.setAttribute('aria-label', parish.isLive ? parish.name : `${parish.name} — coming soon`)

  if (parish.photoUrl) {
    el.className = 'dmap__pin dmap__pin--photo'
    const img = document.createElement('img')
    img.src = parish.photoUrl
    img.alt = ''
    img.loading = 'lazy'
    // A photograph that 404s would leave a torn-image icon pinned to the
    // map, so the glyph takes over instead.
    img.addEventListener('error', () => {
      el.className = parish.isLive ? 'dmap__pin dmap__pin--live' : 'dmap__pin dmap__pin--soon'
      el.innerHTML = churchGlyph()
    })
    const disc = document.createElement('span')
    disc.className = 'dmap__pin-disc'
    disc.appendChild(img)
    const stem = document.createElement('span')
    stem.className = 'dmap__pin-stem'
    stem.setAttribute('aria-hidden', 'true')
    el.append(disc, stem)
    return el
  }

  el.className = parish.isLive ? 'dmap__pin dmap__pin--live' : 'dmap__pin dmap__pin--soon'
  el.innerHTML = churchGlyph()
  return el
}

/** A body, a roof and a cross — the client's own earlier prototype's glyph. */
function churchGlyph(): string {
  return (
    '<svg viewBox="-16 -18 32 32" aria-hidden="true">' +
    '<polygon points="-6,-2 0,-10 6,-2" />' +
    '<rect x="-5" y="-2" width="10" height="8" rx="1" />' +
    '<line x1="0" y1="-10" x2="0" y2="-14" stroke-width="1.5" />' +
    '<line x1="-2.5" y1="-12" x2="2.5" y2="-12" stroke-width="1.5" />' +
    '</svg>')
}
```

1.4.36 APP-106 — mapSearch.ts

Code ID	APP-106
File Name	mapSearch.ts
File Path	SanctDemo/src/lib/mapSearch.ts
Language / Type	TypeScript
Lines	250
Functions / Classes	SearchableParish (exported type); MAX_SEARCH_RESULTS (exported constant); normalizeText (exported function); tokenize (internal function); acronymOf (exported function); editDistance (exported function); typoBudget (internal function); SearchOptions (exported type); ScoredParish (exported type); isNearMeQuery (exported function); scoreParish (internal function); searchParishes (exported function); searchParishesScored (exported function)
Related Files (its imports)	./geo
Where It Is Used	src/components/DioceseMapLive.tsx, src/components/Onboarding.tsx, src/components/SearchScreen.tsx
How to Recognize It	Open SanctDemo/src/lib/mapSearch.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Search for the diocese map's floating search bar.
>
> Pure and map-free so it is unit-testable without a MapLibre instance (the map canvas can't be inspected in this environment's headless browser — see docs/reports/map-prototype-design-swap.md).
>
> This replaced a plain substring filter, which failed the moment anyone typed the parish's name in a different order, abbreviated it, or misspelled it — and, because results were capped in *input order* rather than by relevance, could push the parish someone actually wanted off the visible list entirely. Everything here is scored first and truncated second.
>
> 31 records means a full scan per keystroke costs nothing, so there is no index to build or keep in sync, and no network dependency.

Complete Source Code — mapSearch.ts

```
// Search for the diocese map's floating search bar.
//
// Pure and map-free so it is unit-testable without a MapLibre instance (the
// map canvas can't be inspected in this environment's headless browser — see
// docs/reports/map-prototype-design-swap.md).
//
// This replaced a plain substring filter, which failed the moment anyone
// typed the parish's name in a different order, abbreviated it, or misspelled
// it — and, because results were capped in *input order* rather than by
// relevance, could push the parish someone actually wanted off the visible
```

```
// list entirely. Everything here is scored first and truncated second.
//
// 31 records means a full scan per keystroke costs nothing, so there is no
// index to build or keep in sync, and no network dependency.

import { haversineMeters, type Coordinates } from './geo'

export interface SearchableParish {
  id: string
  name: string
  location: string
  coordinates?: Coordinates
}

export const MAX_SEARCH_RESULTS = 6

/**
 * Words skipped when building an acronym. "Mary Help of Christians Parish"
 * has to yield MHCP, not MHOCP — nobody writes the "of".
 */
const ACRONYM_STOPWORDS = new Set(['of', 'the', 'and', 'de', 'del', 'la', 'las', 'los', 'ng', 'sa'])

/** Recognises "churches near me" and its common phrasings, including Tagalog. */
const NEAR_ME_PATTERN = /\b(near|nearby|nearest|closest|malapit)\b/i

/**
 * Strips case, accents and punctuation so "Sto. Niño" and "sto nino" compare
 * equal. Parish names in this dataset carry both, and a pilgrim typing on a
 * phone keyboard will produce neither consistently.
 */
export function normalizeText(value: string): string {
  return value
    .toLowerCase()
    .normalize('NFD')
    .replace(/\p{Diacritic}/gu, '')
    .replace(/[^a-z0-9\s]/g, ' ')
    .replace(/\s+/g, ' ')
    .trim()
}

function tokenize(value: string): string[] {
  const normalized = normalizeText(value)
  return normalized === '' ? [] : normalized.split(' ')
}

/** "Mary Help of Christians Parish" → "mhcp". */
export function acronymOf(name: string): string {
  return tokenize(name)
    .filter(token => !ACRONYM_STOPWORDS.has(token))
    .map(token => token[0])
    .join('')
}

/**
 * Levenshtein distance, capped.
 *
 * `max` lets the row scan bail as soon as every cell exceeds the budget,
 * which matters because this runs across every parish on every keystroke and
 * most comparisons are hopeless from the second character.
```

```
 */
export function editDistance(a: string, b: string, max = Number.POSITIVE_INFINITY): number {
  if (a === b) return 0
  if (a.length === 0) return b.length
  if (b.length === 0) return a.length
  if (Math.abs(a.length - b.length) > max) return max + 1

  let previous = Array.from({ length: b.length + 1 }, (_, i) => i)
  let current = new Array<number>(b.length + 1)

  for (let i = 1; i <= a.length; i++) {
    current[0] = i
    let rowMin = current[0]
    for (let j = 1; j <= b.length; j++) {
      const substitution = previous[j - 1] + (a[i - 1] === b[j - 1] ? 0 : 1)
      current[j] = Math.min(current[j - 1] + 1, previous[j] + 1, substitution)
      if (current[j] < rowMin) rowMin = current[j]
    }
    if (rowMin > max) return max + 1
    const swap = previous
    previous = current
    current = swap
  }

  return previous[b.length]
}

/**
 * How far a typo may stray before a token stops counting as a match.
 *
 * Scaled to the word's length: a short word gets no latitude, a long one gets
 * two edits. A flat threshold either rejects "chirstians" or lets "mary"
 * match "mass".
 */
function typoBudget(token: string): number {
  if (token.length <= 3) return 0
  if (token.length <= 6) return 1
  return 2
}

// Score bands, spaced so a weaker band can never outrank a stronger one no
// matter how many bonuses accumulate inside it. This is what stops an
// unrelated parish that fuzzily matches a couple of tokens from displacing
// the one whose name literally starts with what was typed.
const SCORE_EXACT_NAME = 10000
const SCORE_ACRONYM = 8000
const SCORE_NAME_PREFIX = 6000
const SCORE_NAME_SUBSTRING = 4000
const SCORE_ALL_TOKENS = 2000
const SCORE_FUZZY = 500

export interface SearchOptions {
  /** The pilgrim's position, used to rank ties and to answer "near me". */
  origin?: Coordinates | null
}

export interface ScoredParish<T> {
  parish: T
  score: number
```

```
  distanceMeters: number | null
}

/** True when the query is asking for proximity rather than naming a parish. */
export function isNearMeQuery(query: string): boolean {
  return NEAR_ME_PATTERN.test(query)
}

function scoreParish<T extends SearchableParish>(query: string, parish: T): number {
  const q = normalizeText(query)
  if (q === '') return 0

  const name = normalizeText(parish.name)
  const location = normalizeText(parish.location ?? '')
  const queryTokens = tokenize(query)
  const nameTokens = tokenize(parish.name)
  const haystackTokens = [...nameTokens, ...tokenize(parish.location ?? '')]

  if (name === q) return SCORE_EXACT_NAME

  // Checked before substring: "MHCP" is not a substring of the name, and a
  // pilgrim typing an abbreviation knows exactly which parish they want.
  if (acronymOf(parish.name) === q.replace(/\s/g, '')) return SCORE_ACRONYM

  // A prefix that lands on a word boundary beats one that cuts a word in
  // half. Typing "San" means San Roque, not Santo Niño — both names start
  // with those three letters, but only one of them starts with that *word*,
  // and without this the shorter name wins on the length bonus alone.
  if (name.startsWith(q)) {
    const endsOnBoundary = name.length === q.length || name[q.length] === ' '
    const band = endsOnBoundary ? SCORE_NAME_PREFIX + 500 : SCORE_NAME_PREFIX
    return band + Math.round((q.length / name.length) * 400)
  }
  if (name.includes(q)) {
    const at = name.indexOf(q)
    const startsWord = at === 0 || name[at - 1] === ' '
    const band = startsWord ? SCORE_NAME_SUBSTRING + 500 : SCORE_NAME_SUBSTRING
    return band + Math.round((q.length / name.length) * 400)
  }

  // Every typed word appears somewhere, in any order — "Christians Parish"
  // and "Mary Help Caloocan" both land here. Name hits are weighted above
  // location hits so a parish matched only by its town never outranks one
  // matched by name.
  const tokenHits = queryTokens.map(token => {
    if (nameTokens.some(word => word.startsWith(token))) return 2
    if (name.includes(token)) return 2
    if (location.includes(token)) return 1
    return 0
  })
  if (tokenHits.length > 0 && tokenHits.every(hit => hit > 0)) {
    return SCORE_ALL_TOKENS + tokenHits.reduce((sum, hit) => sum + hit, 0) * 100
  }

  // Last resort: typo tolerance. Every query token has to land near some word
  // in the record, and the score falls off with total edit distance.
  let totalDistance = 0
  for (const token of queryTokens) {
    const budget = typoBudget(token)
```

```
    let best = Number.POSITIVE_INFINITY
    for (const word of haystackTokens) {
      const distance = editDistance(token, word, budget)
      if (distance < best) best = distance
      if (best === 0) break
    }
    if (best > budget) return 0
    totalDistance += best
  }
  return SCORE_FUZZY - totalDistance * 50
}

/**
 * Ranked search. Returns at most MAX_SEARCH_RESULTS, best first.
 */
export function searchParishes<T extends SearchableParish>(
  query: string,
  parishes: T[],
  options: SearchOptions = {},
): T[] {
  return searchParishesScored(query, parishes, options).map(result => result.parish)
}

/**
 * As `searchParishes`, but keeps each result's score and distance so the UI
 * can show "0.8 km away".
 *
 * "churches near me" (and similar) short-circuits name scoring entirely and
 * ranks by real geographic distance — but only when a position is known.
 * Without one there is nothing honest to sort by, so it falls through to
 * ordinary matching rather than returning an arbitrary order.
 */
export function searchParishesScored<T extends SearchableParish>(
  query: string,
  parishes: T[],
  { origin }: SearchOptions = {},
): ScoredParish<T>[] {
  if (normalizeText(query) === '') return []

  const withDistance = (parish: T): number | null =>
    origin && parish.coordinates ? haversineMeters(origin, parish.coordinates) : null

  if (isNearMeQuery(query) && origin) {
    return parishes
      .map(parish => ({ parish, score: SCORE_ALL_TOKENS, distanceMeters: withDistance(parish) }))
      .filter((result): result is ScoredParish<T> & { distanceMeters: number } => result.distanceMeters !== null)
      .sort((a, b) => a.distanceMeters - b.distanceMeters)
      .slice(0, MAX_SEARCH_RESULTS)
  }

  return parishes
    .map(parish => ({ parish, score: scoreParish(query, parish), distanceMeters: withDistance(parish) }))
    .filter(result => result.score > 0)
    .sort((a, b) => {
      if (b.score !== a.score) return b.score - a.score
      // Equal relevance: the nearer parish is the more useful answer.
      if (a.distanceMeters !== null && b.distanceMeters !== null) {
        return a.distanceMeters - b.distanceMeters
      }
```

```
      return a.parish.name.localeCompare(b.parish.name)
    })
    .slice(0, MAX_SEARCH_RESULTS)
}
```

1.4.37 APP-107 — massScheduleWatch.ts

Code ID	APP-107
File Name	massScheduleWatch.ts
File Path	SanctDemo/src/lib/massScheduleWatch.ts
Language / Type	TypeScript
Lines	243
Functions / Classes	MassEntryLike (exported type); partsOf (internal function); MassChange (exported type); massFingerprintKey (exported function); massFingerprint (exported function); MassMoved (exported type); massDiff (exported function); spoken (internal function); classifyMassChange (exported function); nextStoredFingerprint (exported function); MassNotice (exported type); massNotice (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/massScheduleWatch.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Telling somebody their parish changed the Mass times.
>
> ## Why a notification at all
>
> The Mass schedule is the one thing in the app a person acts on physically: they read it, and then they leave the house at a particular time. When the parish moves a Mass, anyone who checked last week is now wrong, and nothing in an app they are not looking at will tell them. So the phone does.
>
> ## The two cases, and why they read differently
>
> Updated - the times changed. That is news the reader must act on.
>
> Unavailable - the parish has taken the schedule down, or has not published one. That is also worth saying, because the alternative is a person opening the app on Sunday morning, finding nothing, and not knowing whether the app is broken or the parish simply has not said.
>
> ## Why a fingerprint rather than a timestamp
>
> An admin saving the page without changing a time bumps updatedAt, and a notification for "the Mass times changed" when they have not is how people learn to ignore notifications. The fingerprint is of the times themselves, so only a real change speaks.

Complete Source Code — massScheduleWatch.ts

```
/**
 * Telling somebody their parish changed the Mass times.
```

```
 *
 * ## Why a notification at all
 *
 * The Mass schedule is the one thing in the app a person acts on
 * physically: they read it, and then they leave the house at a
 * particular time. When the parish moves a Mass, anyone who checked
 * last week is now wrong, and nothing in an app they are not looking
 * at will tell them. So the phone does.
 *
 * ## The two cases, and why they read differently
 *
 * Updated - the times changed. That is news the reader must act on.
 *
 * Unavailable - the parish has taken the schedule down, or has not
 * published one. That is also worth saying, because the alternative is
 * a person opening the app on Sunday morning, finding nothing, and not
 * knowing whether the app is broken or the parish simply has not said.
 *
 * ## Why a fingerprint rather than a timestamp
 *
 * An admin saving the page without changing a time bumps updatedAt,
 * and a notification for "the Mass times changed" when they have not
 * is how people learn to ignore notifications. The fingerprint is of
 * the times themselves, so only a real change speaks.
 *
 * Pure and dependency-free so the rules can be tested without a
 * device, a network or a Firestore.
 */

/**
 * One day of the stored schedule.
 *
 * `time` is a COMMA-SEPARATED STRING, not an array, because that is
 * what MassScheduleEntry in lib/schedule actually holds and what the
 * admin writes to Firestore.
 *
 * This type used to declare `times?: string[]`, which no caller ever
 * supplied. The field was therefore always undefined, every day
 * fingerprinted as "sunday|", and the fingerprint encoded nothing but
 * the LIST OF DAY NAMES - so moving a Mass, adding one, or suspending
 * one all produced an identical fingerprint and no notification was
 * ever sent for any of them. The tests passed because they were
 * written against this same wrong shape.
 */
export interface MassEntryLike {
  day?: string
  /** "6:00 AM, 9:00 AM" */
  time?: string
  /** Times the parish has suspended. Exact matches against `time`. */
  unavailableTimes?: string[]
  note?: string
}

/** Splits a stored "6:00 AM, 9:00 AM" into its parts, normalised. */
function partsOf(value: string | undefined): string[] {
  return (value ?? '')
    .split(',')
    .map(t => t.trim().toLowerCase())
    .filter(Boolean)
```

```
}

export type MassChange = 'none' | 'first-seen' | 'updated' | 'unavailable' | 'restored'

/** Where the last-seen schedule is remembered, per parish. */
export function massFingerprintKey(parishId: string): string {
  return `sanctiwalk.massSeen.${parishId}`
}

/**
 * A stable summary of the schedule's content.
 *
 * Only the day and the times, in order, lowercased and trimmed:
 * whitespace and capitalisation are the admin's typing, not a change
 * to when Mass is. Returns null when there is no schedule at all,
 * which is a different thing from an empty one.
 */
export function massFingerprint(schedule: MassEntryLike[] | null | undefined): string | null {
  if (!Array.isArray(schedule) || schedule.length === 0) return null
  const parts = schedule.map(e => {
    const day = (e.day ?? '').trim().toLowerCase()
    const times = partsOf(e.time).join(',')
    // Suspensions belong in the fingerprint. A parish calling off this
    // Sunday's 6:00 AM has not changed a single time in `time`, so
    // without this the schedule looks untouched and the one person who
    // most needs telling - the one who goes at six - is told nothing.
    const off = (e.unavailableTimes ?? [])
      .map(t => t.trim().toLowerCase())
      .filter(Boolean)
      .sort()
      .join(',')
    return `${day}|${times}|${off}`
  })
  return parts.join(';')
}

/** One suspended or restored Mass, for the wording below. */
export interface MassMoved {
  day: string
  time: string
}

/**
 * Which Masses were called off, and which came back.
 *
 * Read back out of the two fingerprints rather than the schedules
 * themselves, because the fingerprint is the only thing kept between
 * runs. That is also why its shape above is parseable.
 */
export function massDiff(
  previous: string | null,
  current: string | null,
): { suspended: MassMoved[]; restored: MassMoved[] } {
  const none = { suspended: [], restored: [] }
  if (!previous || !current || previous === 'UNAVAILABLE') return none

  const read = (fp: string): Map<string, Set<string>> => {
    const map = new Map<string, Set<string>>()
    for (const part of fp.split(';')) {
```

```
      const [day = '', , off = ''] = part.split('|')
      map.set(day, new Set(off.split(',').filter(Boolean)))
    }
    return map
  }

  const before = read(previous)
  const after = read(current)

  const suspended: MassMoved[] = []
  const restored: MassMoved[] = []

  for (const [day, nowOff] of after) {
    const wasOff = before.get(day) ?? new Set<string>()
    for (const time of nowOff) if (!wasOff.has(time)) suspended.push({ day, time })
    for (const time of wasOff) if (!nowOff.has(time)) restored.push({ day, time })
  }

  return { suspended, restored }
}

/** "sunday" + "6:00 am" -> "6:00 AM Sunday". */
function spoken({ day, time }: MassMoved): string {
  const Day = day.charAt(0).toUpperCase() + day.slice(1)
  return `${time.toUpperCase()} ${Day}`
}

/**
 * What happened between the schedule last seen and the one now.
 *
 * `first-seen` is deliberately not a notification. A fresh install has
 * no baseline, and announcing "the Mass times changed" to somebody who
 * has never seen them is both untrue and the worst possible first
 * impression of the app's notifications.
 */
export function classifyMassChange(
  previous: string | null,
  current: string | null,
): MassChange {
  if (previous === null && current === null) return 'none'
  if (previous === null) return 'first-seen'
  if (current === null) return 'unavailable'
  if (previous === 'UNAVAILABLE') return 'restored'
  return previous === current ? 'none' : 'updated'
}

/** What gets stored, given what just happened. */
export function nextStoredFingerprint(current: string | null): string {
  return current === null ? 'UNAVAILABLE' : current
}

export interface MassNotice {
  title: string
  body: string
}

/** The words for a change, or null when there is nothing to say. */
export function massNotice(
  change: MassChange,
```

```
  parishName: string,
  /**
   * What exactly moved, from massDiff.
   *
   * Optional, and the generic wording below is used without it. With
   * it the notice can name the Mass, which is the difference between
   * something the reader must open the app to act on and something
   * they can act on from the lock screen: "the 6:00 AM Sunday Mass is
   * suspended" either does or does not concern you, and you know
   * which without tapping anything.
   */
  moved?: { suspended: MassMoved[]; restored: MassMoved[] },
): MassNotice | null {
  switch (change) {
    case 'updated': {
      const suspended = moved?.suspended ?? []
      const restored = moved?.restored ?? []

      if (suspended.length === 1 && restored.length === 0) {
        return {
          title: `${spoken(suspended[0])} Mass is suspended`,
          body: `${parishName} has called off the ${spoken(suspended[0])} Mass. The rest of the schedule is unchanged.`,
        }
      }
      if (suspended.length > 1 && restored.length === 0) {
        return {
          title: `${suspended.length} Masses suspended`,
          body: `${parishName} has called off ${suspended.map(spoken).join(', ')}.`,
        }
      }
      if (restored.length === 1 && suspended.length === 0) {
        return {
          title: `${spoken(restored[0])} Mass is back`,
          body: `${parishName} is celebrating the ${spoken(restored[0])} Mass again.`,
        }
      }
      if (restored.length > 1 && suspended.length === 0) {
        return {
          title: `${restored.length} Masses are back`,
          body: `${parishName} is celebrating ${restored.map(spoken).join(', ')} again.`,
        }
      }

      // Times themselves moved, or both happened at once. Nothing
      // short and true can be said, so it says the true vague thing.
      return {
        title: 'Mass schedule updated',
        body: `${parishName} has changed its Mass times. Open SanctiWalk to see the new schedule.`,
      }
    }
    case 'unavailable':
      return {
        title: 'Mass schedule unavailable',
        body: `${parishName} has taken its Mass schedule down. Contact the parish office for times.`,
      }
    case 'restored':
      return {
        title: 'Mass schedule is back',
        body: `${parishName} has published its Mass times again.`,
```

```
      }
    default:
      return null
  }
}
```

1.4.38 APP-108 — ministryApplication.ts

Code ID	APP-108
File Name	ministryApplication.ts
File Path	SanctDemo/src/lib/ministryApplication.ts
Language / Type	TypeScript
Lines	149
Functions / Classes	MINISTRY_APPLICATION_TYPE (exported constant); MINISTRY_STATUSES (exported constant); MinistryStatus (exported type); DEFAULT_MINISTRY_STATUS (exported constant); MinistryApplicationInput (exported type); MinistryApplicationDoc (exported type); buildMinistryApplication (exported function); isMinistryApplication (exported function); StatusPresentation (exported type); statusPresentation (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/AdminPortal.tsx, src/components/MinistriesTab.tsx
How to Recognize It	Open SanctDemo/src/lib/ministryApplication.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Ministry applications: the document shape, and how a status is shown.
>
> This reuses the existing `applications` collection rather than adding a second one. That collection already carries sacrament bookings, already has Firestore rules scoping reads to the owning uid, and is already listened to in App.tsx and rendered in both Me and the admin portal. A parallel collection would mean duplicating all of it.
>
> So a ministry application is an application with `type` set to MINISTRY_APPLICATION_TYPE and some extra fields. The old fields (applicant, details, date, status) are still written, because the admin portal and the pilgrim's own list read them - the new structured fields sit alongside so both views can be precise about ministry and parish instead of parsing them back out of a sentence.

Complete Source Code — ministryApplication.ts

```
// Ministry applications: the document shape, and how a status is shown.
//
// This reuses the existing `applications` collection rather than adding a
// second one. That collection already carries sacrament bookings, already
// has Firestore rules scoping reads to the owning uid, and is already
// listened to in App.tsx and rendered in both Me and the admin portal. A
// parallel collection would mean duplicating all of it.
//
// So a ministry application is an application with `type` set to
// MINISTRY_APPLICATION_TYPE and some extra fields. The old fields
// (applicant, details, date, status) are still written, because the admin
// portal and the pilgrim's own list read them - the new structured fields
// sit alongside so both views can be precise about ministry and parish
// instead of parsing them back out of a sentence.
```

```
export const MINISTRY_APPLICATION_TYPE = 'Ministry Application'

/** The only four statuses a ministry application moves through. */
export const MINISTRY_STATUSES = ['Pending', 'Under Review', 'Approved', 'Declined'] as const

export type MinistryStatus = (typeof MINISTRY_STATUSES)[number]

/** Every application starts here. The parish office moves it along. */
export const DEFAULT_MINISTRY_STATUS: MinistryStatus = 'Pending'

export interface MinistryApplicationInput {
  uid: string
  fullName: string
  email: string
  mobile: string
  ministryId: string
  ministryName: string
  parishId: string
  parishName: string
  message: string
  /** The applicant ticked the box agreeing to be contacted. */
  consent: boolean
  /** Injected so the document is testable without mocking the clock. */
  now?: Date
}

export interface MinistryApplicationDoc {
  type: typeof MINISTRY_APPLICATION_TYPE
  uid: string
  applicant: string
  email: string
  mobile: string
  ministryId: string
  ministryName: string
  parishId: string
  parishName: string
  message: string
  consent: boolean
  details: string
  date: string
  submittedAt: string
  status: MinistryStatus
}

/**
 * Builds the document to store. `id` is Firestore's to assign, and `uid`
 * must be the signed-in user's or the rules reject the write.
 */
export function buildMinistryApplication(input: MinistryApplicationInput): MinistryApplicationDoc {
  const now = input.now ?? new Date()
  const mobile = input.mobile.trim()
  const message = input.message.trim()

  return {
    type: MINISTRY_APPLICATION_TYPE,
    uid: input.uid,
    applicant: input.fullName.trim(),
    email: input.email.trim(),
    mobile,
```

```
    ministryId: input.ministryId,
    ministryName: input.ministryName,
    parishId: input.parishId,
    parishName: input.parishName,
    message,
    consent: input.consent,

    // Kept for the views that predate the structured fields. It is a
    // summary, never the source of truth - anything reading ministry or
    // parish should read those fields.
    details: `${input.ministryName} — ${input.parishName}`,

    // `date` is what the existing lists display; submittedAt is the exact
    // instant, kept separately because a localised date string cannot be
    // sorted or compared reliably.
    date: now.toLocaleDateString(),
    submittedAt: now.toISOString(),

    status: DEFAULT_MINISTRY_STATUS,
  }
}

export function isMinistryApplication(app: { type?: string }): boolean {
  return app.type === MINISTRY_APPLICATION_TYPE
}

export interface StatusPresentation {
  label: string
  /** The coloured dot the brief asks for. */
  dot: string
  /** Tailwind classes for the pill. */
  className: string
}

const PRESENTATIONS: Record<MinistryStatus, StatusPresentation> = {
  Pending: { label: 'Pending', dot: '🟡', className: 'bg-amber-50 text-amber-900 border-amber-200' },
  'Under Review': { label: 'Under Review', dot: '🔵', className: 'bg-blue-50 text-blue-900 border-blue-200' },
  Approved: { label: 'Approved', dot: '🟢', className: 'bg-green-50 text-green-900 border-green-200' },
  Declined: { label: 'Declined', dot: '🔴', className: 'bg-red-50 text-red-900 border-red-200' },
}

/**
 * How to show a status, including ones written before these four existed.
 *
 * The collection already holds statuses like "Interview Scheduled" and
 * "Approved (Simulated)" from the sacrament flow and the email modals.
 * Rewriting those rows would be a migration nobody asked for, and showing
 * them as "Pending" would be a lie, so they are matched by meaning and
 * shown with their own words.
 */
export function statusPresentation(status: string): StatusPresentation {
  const exact = PRESENTATIONS[status as MinistryStatus]
  if (exact) return exact

  const lower = status.toLowerCase()
  if (lower.includes('declin') || lower.includes('reject')) {
    return { ...PRESENTATIONS.Declined, label: status }
  }
```

```
  if (lower.includes('approve')) {
    return { ...PRESENTATIONS.Approved, label: status }
  }
  // "Awaiting" is checked before "interview" deliberately. "Awaiting Parish
  // Interview" says the parish has not acted yet, which is Pending; reading
  // the word "interview" first would report it as already under review.
  if (lower.includes('pending') || lower.includes('awaiting')) {
    return { ...PRESENTATIONS.Pending, label: status }
  }
  if (lower.includes('review') || lower.includes('interview') || lower.includes('schedul')) {
    return { ...PRESENTATIONS['Under Review'], label: status }
  }

  // Unrecognised: show it plainly rather than guess a colour that implies
  // an outcome.
  return { label: status, dot: '◌', className: 'bg-[var(--color-brand-card)] text-[var(--color-brand-text)] border-[var(--color-brand-border)]' }
}
```

1.4.39 APP-109 — ministrySearch.ts

Code ID	APP-109
File Name	ministrySearch.ts
File Path	SanctDemo/src/lib/ministrySearch.ts
Language / Type	TypeScript
Lines	83
Functions / Classes	normalize (exported function); acronymOf (exported function); haystack (internal function); searchMinistries (exported function); suggestionsFor (exported function)
Related Files (its imports)	../data
Where It Is Used	src/components/MinistriesTab.tsx
How to Recognize It	Open SanctDemo/src/lib/ministrySearch.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ministrySearch.ts

```
import type { Ministry } from '../data'

/**
 * Finding a ministry among fifteen, by typing what you actually remember.
 *
 * People do not remember "Ministry of Lectors and Commentators (MLC)". They
 * remember "lector", or "MLC", or "yung nagbabasa" — so matching only the
 * start of the official name finds almost nothing. What this handles, in the
 * order it matters:
 *
 *  - **Any word, anywhere.** "servants" finds Singing Servants of Christ.
 *  - **Acronyms.** The parish's own names carry them in brackets — MAS,
 *    EMHC, SOCCOM — and they are what a member says out loud.
 *  - **The description too**, because "musika" should find the choirs even
 *    though none of them has that word in its name.
 *  - **Accents and case ignored**, so "nino" finds "Niño" and nobody has to
 *    find the tilde on a phone keyboard.
 *
 * Every term must match something, but they may match different fields, so
 * "youth choir" finds Marian Youth Choir.
 */

/** Lower case, accents stripped, punctuation flattened to spaces. */
export function normalize(text: string): string {
  return text
    .normalize('NFD')
    .replace(/[̀-ͯ]/g, '')
    .toLowerCase()
    .replace(/[^a-z0-9]+/g, ' ')
    .trim()
}
```

```
/** The bracketed short name a parish actually says: "(EMHC)" -> "emhc". */
export function acronymOf(name: string): string {
  const bracketed = name.match(/\(([^)]+)\)/)
  if (!bracketed) return ''

  const raw = bracketed[1].trim()
  const letters = raw.replace(/[^A-Za-z]/g, '')
  if (letters.length === 0 || letters.length > 8) return ''

  // Mostly capitals is what makes it an acronym rather than a word.
  //
  // Length alone is not enough: "(choir)" is short and spaceless and would
  // pass, which would make every choir answer to a word that describes a
  // category rather than naming one ministry. Capitalisation separates them
  // cleanly, and the threshold is 0.6 rather than 1.0 so that genuinely
  // mixed forms like "(AnP)" still count.
  const capitals = letters.replace(/[^A-Z]/g, '').length
  if (capitals / letters.length < 0.6) return ''

  return normalize(raw)
}

function haystack(m: Ministry): string {
  return [normalize(m.name), acronymOf(m.name), normalize(m.description)]
    .filter(Boolean)
    .join(' ')
}

export function searchMinistries(ministries: Ministry[], query: string): Ministry[] {
  const terms = normalize(query).split(' ').filter(Boolean)
  if (terms.length === 0) return ministries

  return ministries.filter(m => {
    const hay = haystack(m)
    return terms.every(term => hay.includes(term))
  })
}

/**
 * The chips shown before anyone types.
 *
 * Not a fixed list: each one is checked against the ministries actually
 * present and dropped if it would find nothing, so a suggestion can never
 * lead to an empty screen. A parish with no choir does not get a "Choir"
 * chip.
 */
const CANDIDATES = ['Choir', 'Kabataan', 'Liturhiya', 'Panalangin', 'Altar', 'Musika']

export function suggestionsFor(ministries: Ministry[]): string[] {
  return CANDIDATES.filter(c => searchMinistries(ministries, c).length > 0).slice(0, 5)
}
```

1.4.40 APP-110 — modelRetry.ts

Code ID	APP-110
File Name	modelRetry.ts
File Path	SanctDemo/src/lib/modelRetry.ts
Language / Type	TypeScript
Lines	180
Functions / Classes	RETRY_BACKOFF_MS (exported constant); ATTEMPT_TIMEOUT_MS (exported constant); OVERALL_DEADLINE_MS (exported constant); isModelBusy (exported function); isTimeout (exported function); isRetryable (exported function); RetryPlanStep (exported type); retryPlan (exported function); AttemptOutcome (exported type); askWithRetry (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	server.ts
How to Recognize It	Open SanctDemo/src/lib/modelRetry.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Retrying a congested vision model.
>
> Extracted from server.ts so the sequencing can be tested. It earned that: six consecutive scans from a real phone were logged failing with `503 UNAVAILABLE — "This model is currently experiencing high demand"`, and because there was no retry at all, every one surfaced to the pilgrim as a flat "Recognition failed. Please try again." The feature looked broken when nothing about it was.

Complete Source Code — modelRetry.ts

```
/**
 * Retrying a congested vision model.
 *
 * Extracted from server.ts so the sequencing can be tested. It earned that:
 * six consecutive scans from a real phone were logged failing with
 * `503 UNAVAILABLE — "This model is currently experiencing high demand"`,
 * and because there was no retry at all, every one surfaced to the pilgrim as
 * a flat "Recognition failed. Please try again." The feature looked broken
 * when nothing about it was.
 */

/** Backoff before the second and third attempts. */
export const RETRY_BACKOFF_MS = [400, 1500]

/**
 * How long any single attempt may run before it is abandoned.
 *
 * Set from measurement, not taste. Against the app's real 208 KB / 960px
 * capture:
 *
```

```
 *   gemini-3.5-flash-lite   503 after 108s, then 503 after 3.3s
 *   gemini-3.5-flash        answered in 7.3s, then in 18.5s
 *
 * The 108s is the important one. The model was congested, and the SDK retries
 * a 503 internally with its own long backoff before surfacing it — so what
 * looks like a hang is really the SDK still trying. This cap is what cuts
 * that short. It sits above the slowest observed SUCCESS (18.5s) so a genuine
 * answer is never thrown away to save a few seconds.
 */
export const ATTEMPT_TIMEOUT_MS = 20000

/**
 * The ceiling on the WHOLE sequence — every attempt and every backoff.
 *
 * This must stay below the client's own abort (SCAN_TIMEOUT_MS in
 * ArTour.tsx) or the retries are worse than useless: the pilgrim is shown a
 * timeout while the server is still working towards an answer it can no
 * longer deliver, and the honest "the service is busy" — the more useful
 * message, and the entire point of retrying — never reaches them. That is
 * not hypothetical: an earlier 21s deadline let a fourth attempt start at
 * 19.9s and return at 26.3s, past the client's 25s.
 */
export const OVERALL_DEADLINE_MS = 40000

/**
 * True for the errors worth trying again: this model's serving capacity is
 * full right now, which is a temporary property of the provider's fleet
 * rather than anything about the request.
 *
 * A 429 is deliberately NOT busy. That is a quota decision — the allowance is
 * spent, and asking again cannot succeed, it only spends the remainder
 * faster. Neither is a bad key or a malformed request: those fail identically
 * however many times they are asked, so retrying them only delays the honest
 * error by three seconds.
 */
export function isModelBusy(message: string): boolean {
  return /\b503\b|UNAVAILABLE|overloaded|high demand/i.test(message)
}

/**
 * True for an attempt that was abandoned rather than answered.
 *
 * Retried for the same reason congestion is: a call that hangs says nothing
 * about whether the next one will, and the fallback is a different service
 * that may well answer at once.
 */
export function isTimeout(message: string): boolean {
  return /abort|timed? ?out|ETIMEDOUT|deadline/i.test(message)
}

/** Every failure worth another attempt. Everything else is rethrown at once. */
export function isRetryable(message: string): boolean {
  return isModelBusy(message) || isTimeout(message)
}

export interface RetryPlanStep {
  /** Milliseconds to wait before this attempt. 0 for the first. */
  waitMs: number
  /** Which model to ask. */
```

```
  model: string
  /** True once the primary has been given up on. */
  isFallback: boolean
}

/**
 * The sequence of attempts: the primary once, then the fallback twice.
 *
 * The primary gets ONE try, not three. Measurement drove this: while
 * flash-lite was congested it refused every request, and the fallback
 * answered correctly first time. Asking a saturated pool three times spends
 * the pilgrim's patience to learn what the first refusal already said.
 *
 * The fallback must not be another model from the same congested family — a
 * 503 means that pool is saturated, so retrying inside it is the one thing
 * guaranteed not to help. Separate models also hold separate free-tier
 * quotas, so falling back neither borrows nor exhausts the primary's.
 */
export function retryPlan(primary: string, fallback: string): RetryPlanStep[] {
  return [
    { waitMs: 0, model: primary, isFallback: false },
    { waitMs: RETRY_BACKOFF_MS[0], model: fallback, isFallback: true },
    { waitMs: RETRY_BACKOFF_MS[1], model: fallback, isFallback: true },
  ]
}

const sleep = (ms: number) => new Promise(resolve => setTimeout(resolve, ms))

export interface AttemptOutcome<T> {
  value: T
  model: string
  /** 1-based, so a log line reads the way a person counts. */
  attempt: number
}

/**
 * Runs `ask` against each step of the plan until one succeeds.
 *
 * Rethrows a non-busy error IMMEDIATELY rather than working through the
 * remaining attempts — a wrong API key is not going to become right, and
 * making the pilgrim wait three seconds to be told so is worse than telling
 * them at once.
 */
export async function askWithRetry<T>(
  plan: RetryPlanStep[],
  ask: (model: string, timeoutMs: number) => Promise<T>,
  hooks: {
    onBusy?: (model: string, attempt: number) => void
    wait?: (ms: number) => Promise<void>
    now?: () => number
    deadlineMs?: number
  } = {},
): Promise<AttemptOutcome<T>> {
  const wait = hooks.wait ?? sleep
  const now = hooks.now ?? Date.now
  const deadlineMs = hooks.deadlineMs ?? OVERALL_DEADLINE_MS
  const startedAt = now()
  let lastError: unknown = null
```

```
  for (let i = 0; i < plan.length; i++) {
    const step = plan[i]

    // The deadline only ever cancels a RETRY. The first attempt always runs:
    // a deadline already spent is a reason to stop trying again, never a
    // reason to have asked nothing at all and report a failure that no call
    // was ever made to earn.
    if (i > 0) {
      // Checked before the backoff as well as after it, so a sequence that
      // has already run long does not sleep two more seconds only to start a
      // call it has no time left to finish.
      if (now() - startedAt + step.waitMs >= deadlineMs) break
      if (step.waitMs > 0) await wait(step.waitMs)
      if (now() - startedAt >= deadlineMs) break
    }

    // Each attempt gets whatever is LEFT of the deadline, capped at the
    // per-attempt limit — never a flat 12s.
    //
    // A flat limit overruns: measured end to end, four attempts of up to 12s
    // plus backoff returned at 26.3s, past the client's own 25s abort. The
    // pilgrim was then shown "no answer in 25 seconds" instead of the honest
    // "the service is busy", which is the more useful of the two and the
    // whole reason for the retry. Handing each attempt only the remaining
    // time makes the total impossible to exceed.
    const remaining = deadlineMs - (now() - startedAt)
    const attemptTimeout = Math.min(ATTEMPT_TIMEOUT_MS, Math.max(1000, remaining))

    try {
      return { value: await ask(step.model, attemptTimeout), model: step.model, attempt: i + 1 }
    } catch (error) {
      const message = String((error as { message?: unknown })?.message ?? error)
      if (!isRetryable(message)) throw error
      hooks.onBusy?.(step.model, i + 1)
      lastError = error
    }
  }

  // Only reachable having failed at least once: the first attempt is never
  // skipped by the deadline check above.
  throw lastError
}
```

1.4.41 APP-111 — mysteries.ts

Code ID	APP-111
File Name	mysteries.ts
File Path	SanctDemo/src/lib/mysteries.ts
Language / Type	TypeScript
Lines	39
Functions / Classes	MysterySet (exported type); MYSTERY_BY_WEEKDAY (exported constant); mysteryForDate (exported function); daysForMystery (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/mysteries.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Which set of Mysteries is prayed on which day.
>
> Diocese-wide, not tied to any parish, and shared by the Home dashboard and the Pray screen — it lived only in Dashboard.tsx before, and a second copy on Pray would have been a second thing to keep in step.

Complete Source Code — mysteries.ts

```
// Which set of Mysteries is prayed on which day.
//
// Diocese-wide, not tied to any parish, and shared by the Home dashboard and
// the Pray screen — it lived only in Dashboard.tsx before, and a second copy
// on Pray would have been a second thing to keep in step.

export type MysterySet = "Joyful" | "Sorrowful" | "Glorious" | "Luminous"

/** The standard weekday cycle. Sunday and Wednesday both carry the Glorious. */
export const MYSTERY_BY_WEEKDAY: Record<number, MysterySet> = {
  0: "Glorious", // Sunday
  1: "Joyful", // Monday
  2: "Sorrowful", // Tuesday
  3: "Glorious", // Wednesday
  4: "Luminous", // Thursday
  5: "Sorrowful", // Friday
  6: "Joyful", // Saturday
}

export function mysteryForDate(date: Date): MysterySet {
  return MYSTERY_BY_WEEKDAY[date.getDay()]
}

/**
 * "Prayed on Wednesdays and Sundays" — the days a given set comes round,
```

```
 * written out for the Pray screen rather than hardcoded per set, so it stays
 * true if the cycle above is ever adjusted.
 */
export function daysForMystery(set: MysterySet): string {
  const names = ["Sundays", "Mondays", "Tuesdays", "Wednesdays", "Thursdays", "Fridays", "Saturdays"]
  const days = Object.entries(MYSTERY_BY_WEEKDAY)
    .filter(([, value]) => value === set)
    .map(([day]) => names[Number(day)])
    .sort()

  if (days.length === 0) return ""
  if (days.length === 1) return `Prayed on ${days[0]}`
  return `Prayed on ${days.slice(0, -1).join(", ")} and ${days[days.length - 1]}`
}
```

1.4.42 APP-112 — navigation.ts

Code ID	APP-112
File Name	navigation.ts
File Path	SanctDemo/src/lib/navigation.ts
Language / Type	TypeScript
Lines	255
Functions / Classes	RouteStep (exported type); OFF_ROUTE_METERS (exported constant); OFF_ROUTE_STRIKES (exported constant); STEP_REACHED_METERS (exported constant); toLocalMeters (internal function); distanceToSegment (exported function); distanceToPath (exported function); instructionFor (exported function); NavigationProgress (exported type); progressAlong (exported function); OffRouteState (exported type); nextOffRouteState (exported function); announcementFor (exported function)
Related Files (its imports)	./heading, ./geo
Where It Is Used	src/lib/routing.ts
How to Recognize It	Open SanctDemo/src/lib/navigation.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turn-by-turn navigation: instructions, progress along a route, and knowing when the pilgrim has left it.
>
> Pure and sensor-free, like heading.ts and for the same reason — the only honest way to test navigation without walking a kilometre with a handset is to feed it positions and assert on what it says.
>
> Distances here use a local flat-earth approximation rather than haversine for the point-to-segment maths. Over the tens of metres these calculations span, inside one diocese, the error is under a centimetre, and projecting to metres first is what makes "how far am I from this line" expressible at all.

Complete Source Code — navigation.ts

```
// Turn-by-turn navigation: instructions, progress along a route, and knowing
// when the pilgrim has left it.
//
// Pure and sensor-free, like heading.ts and for the same reason — the only
// honest way to test navigation without walking a kilometre with a handset is
// to feed it positions and assert on what it says.
//
// Distances here use a local flat-earth approximation rather than haversine
// for the point-to-segment maths. Over the tens of metres these calculations
// span, inside one diocese, the error is under a centimetre, and projecting to
// metres first is what makes "how far am I from this line" expressible at all.

import { classifyHeading, COMPASS_POINTS_8 } from './heading'
import { haversineMeters, type Coordinates } from './geo'

/** One OSRM step, narrowed to the fields this app actually uses. */
```

```
export interface RouteStep {
  /** Street name, empty for unnamed paths. */
  name: string
  /** Metres covered by this step. */
  distanceMeters: number
  maneuver: {
    type: string
    modifier?: string
    /** [lng, lat] — where the manoeuvre happens. */
    location: [number, number]
    bearingAfter?: number
  }
}

/**
 * How far off the line the pilgrim may drift before it counts as off-route.
 *
 * Generous on purpose. A phone in a street of low buildings routinely reports
 * 20–30m of error, and a threshold tight enough to catch a genuine wrong turn
 * quickly is also tight enough to fire constantly while standing still.
 */
export const OFF_ROUTE_METERS = 35

/**
 * Consecutive off-route fixes before a reroute is triggered.
 *
 * The distance threshold alone is not enough: GPS jumps. Requiring several
 * fixes in a row means a single bad reading cannot throw away a good route,
 * at the cost of a second or two before a real wrong turn is noticed — the
 * right trade, since rerouting mid-street is far more disorienting than a
 * moment's delay.
 */
export const OFF_ROUTE_STRIKES = 3

/** Within this distance of a manoeuvre, it counts as reached. */
export const STEP_REACHED_METERS = 18

/** Metres per degree of latitude. Constant enough anywhere on Earth. */
const METERS_PER_DEG_LAT = 111_320

function toLocalMeters(point: Coordinates, origin: Coordinates): { x: number; y: number } {
  const latRadians = (origin.lat * Math.PI) / 180
  return {
    x: (point.lng - origin.lng) * METERS_PER_DEG_LAT * Math.cos(latRadians),
    y: (point.lat - origin.lat) * METERS_PER_DEG_LAT,
  }
}

/**
 * Shortest distance in metres from a point to the segment a–b.
 *
 * Segment, not infinite line: past either end the nearest point is the
 * endpoint itself. Using an infinite line would report a pilgrim standing
 * beyond the end of a street as being on it.
 */
export function distanceToSegment(point: Coordinates, a: Coordinates, b: Coordinates): number {
  const p = toLocalMeters(point, a)
  const q = toLocalMeters(b, a)
  const lengthSquared = q.x * q.x + q.y * q.y
```

```
  if (lengthSquared === 0) return Math.hypot(p.x, p.y)

  // How far along a→b the perpendicular lands, clamped into the segment.
  const t = Math.max(0, Math.min(1, (p.x * q.x + p.y * q.y) / lengthSquared))
  return Math.hypot(p.x - t * q.x, p.y - t * q.y)
}

/** Shortest distance from a point to a whole path, in metres. */
export function distanceToPath(point: Coordinates, path: Coordinates[]): number {
  if (path.length === 0) return Number.POSITIVE_INFINITY
  if (path.length === 1) return haversineMeters(point, path[0])

  let best = Number.POSITIVE_INFINITY
  for (let i = 0; i < path.length - 1; i++) {
    const d = distanceToSegment(point, path[i], path[i + 1])
    if (d < best) best = d
  }
  return best
}

/**
 * Turns one OSRM manoeuvre into something a walker can act on.
 *
 * OSRM describes a manoeuvre as a type plus a modifier ("turn"/"left"); this
 * is where that becomes English. Two details matter:
 *
 * - An unnamed path is common on foot — alleys, church grounds, cut-throughs.
 *   "Turn left onto " with nothing after it is worse than "Turn left", so the
 *   street name is appended only when there is one.
 * - The departure instruction uses the compass, not a turn: at the start
 *   there is no previous heading to turn from, so "Head northeast" is the
 *   only thing that can be said. That reuses the same eight-point rose the
 *   compass UI shows, so the words match what the dial is displaying.
 */
export function instructionFor(step: RouteStep): string {
  const { type, modifier, bearingAfter } = step.maneuver
  const onto = step.name ? ` onto ${step.name}` : ''
  const on = step.name ? ` on ${step.name}` : ''

  const turn = (verb: string) => {
    switch (modifier) {
      case 'left':
        return `${verb} left${onto}`
      case 'right':
        return `${verb} right${onto}`
      case 'slight left':
        return `${verb} slightly left${onto}`
      case 'slight right':
        return `${verb} slightly right${onto}`
      case 'sharp left':
        return `${verb} sharply left${onto}`
      case 'sharp right':
        return `${verb} sharply right${onto}`
      case 'uturn':
        return `Turn around${onto}`
      case 'straight':
      default:
        return `Continue straight${onto}`
```

```
    }
  }

  switch (type) {
    case 'depart':
      return bearingAfter != null
        ? `Head ${classifyHeading(bearingAfter, COMPASS_POINTS_8).name.toLowerCase()}${on}`
        : `Set off${on}`
    case 'arrive':
      return 'Arrive at your destination'
    case 'turn':
    case 'end of road':
      return turn('Turn')
    case 'fork':
      return turn('Keep')
    case 'merge':
      return turn('Merge')
    case 'new name':
      return step.name ? `Continue onto ${step.name}` : 'Continue straight'
    case 'continue':
      return `Continue${on}`
    case 'roundabout':
    case 'rotary':
      return `Take the roundabout${onto}`
    default:
      return turn('Turn')
  }
}

export interface NavigationProgress {
  /** Index of the step currently being followed. */
  stepIndex: number
  /** Metres to the next manoeuvre. */
  metersToManeuver: number
  /** Metres remaining to the destination. */
  metersRemaining: number
  /** True once the destination has been reached. */
  arrived: boolean
}

/**
 * Where the pilgrim is along a route.
 *
 * `stepIndex` only ever moves forward. Walking back past a manoeuvre — which
 * happens constantly from GPS noise near a corner — must not rewind the
 * instruction, or the app spends the whole junction flipping between "turn
 * left" and "continue".
 */
export function progressAlong(
  position: Coordinates,
  steps: RouteStep[],
  destination: Coordinates,
  previousStepIndex = 0,
): NavigationProgress {
  if (steps.length === 0) {
    const metersRemaining = haversineMeters(position, destination)
    return { stepIndex: 0, metersToManeuver: metersRemaining, metersRemaining, arrived: metersRemaining <= STEP_REACHED_METERS }
  }
```

```
  let stepIndex = Math.min(previousStepIndex, steps.length - 1)

  // Scan forward for the LAST manoeuvre already reached, rather than
  // advancing only while the current one is in reach.
  //
  // The difference matters when a fix arrives late, or when steps are short
  // enough that two are passed between updates: the current step's manoeuvre
  // is then already behind the pilgrim and far away, a "stop as soon as the
  // current one is out of reach" loop breaks immediately, and the app keeps
  // showing an instruction for a turn that was taken two corners ago.
  let lastReached = -1
  for (let i = stepIndex; i < steps.length; i++) {
    const [lng, lat] = steps[i].maneuver.location
    if (haversineMeters(position, { lat, lng }) <= STEP_REACHED_METERS) lastReached = i
  }
  if (lastReached >= 0) stepIndex = Math.min(lastReached + 1, steps.length - 1)

  const [lng, lat] = steps[stepIndex].maneuver.location
  const metersToManeuver = haversineMeters(position, { lat, lng })
  const metersRemaining = haversineMeters(position, destination)

  return {
    stepIndex,
    metersToManeuver,
    metersRemaining,
    arrived: metersRemaining <= STEP_REACHED_METERS,
  }
}

export interface OffRouteState {
  strikes: number
  offRoute: boolean
}

/**
 * Tracks whether the pilgrim has genuinely left the route.
 *
 * Returns the next state; the caller keeps it between fixes. A single fix
 * inside the corridor resets the count completely — a wrong turn produces a
 * run of off-route fixes, whereas noise produces one or two among good ones.
 */
export function nextOffRouteState(
  position: Coordinates,
  path: Coordinates[],
  previous: OffRouteState = { strikes: 0, offRoute: false },
  { thresholdMeters = OFF_ROUTE_METERS, strikesNeeded = OFF_ROUTE_STRIKES } = {},
): OffRouteState {
  const distance = distanceToPath(position, path)

  if (distance <= thresholdMeters) return { strikes: 0, offRoute: false }

  const strikes = previous.strikes + 1
  return { strikes, offRoute: strikes >= strikesNeeded }
}

/** "In 40 m, turn left onto A. Mabini Street" — the spoken-style line. */
export function announcementFor(step: RouteStep, metersToManeuver: number): string {
  const instruction = instructionFor(step)
  if (step.maneuver.type === 'arrive') return instruction
```

```
  if (metersToManeuver <= STEP_REACHED_METERS) return instruction
  return `In ${Math.round(metersToManeuver / 10) * 10} m, ${instruction[0].toLowerCase()}${instruction.slice(1)}`
}
```

1.4.43 APP-113 — notifications.ts

Code ID	APP-113
File Name	notifications.ts
File Path	SanctDemo/src/lib/notifications.ts
Language / Type	TypeScript
Lines	211
Functions / Classes	notifySubmitted (exported function); notifyStatusChanged (exported function); watchNotifications (exported function); markRead (exported function); unreadCount (exported function); MAX_PARISH_NOTIFICATIONS (exported constant); BroadcastResult (exported type); notifyParish (exported function)
Related Files (its imports)	./firebase, ./applications, ../types
Where It Is Used	src/admin/AdminAnnouncements.tsx, src/admin/AdminMassSchedule.tsx, src/components/NotificationsCard.tsx, src/components/NotificationsPage.tsx, src/lib/applications.ts
How to Recognize It	Open SanctDemo/src/lib/notifications.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — notifications.ts

```
import { addDoc, collection, doc, getDocs, onSnapshot, query, updateDoc, where } from "firebase/firestore";
import { auth, db } from "./firebase";
import { STATUS_LABEL } from "./applications";
import type { ApplicationStatus, NotificationDoc } from "../types";

/**
 * In-app notifications.
 *
 * ## Why these are not emails
 *
 * The spec asks for email on every status change. Sending mail from a
 * trusted place means Cloud Functions, which means the Blaze plan, and this
 * project is on Spark. Sending it from the browser would mean an email
 * provider's API key in client code, where anyone can read it and send mail
 * as the parish - which is worse than not sending the email.
 *
 * So the record is written here, where it is real, and the person sees it
 * next time they open the app. Firebase's own verification and
 * password-reset mail still goes out, because Firebase sends that itself.
 *
 * ## Who is allowed to write one
 *
 * The rules permit a notification addressed to yourself, or one written by
 * the admin of the church it names. That is exactly the two cases: a
 * pilgrim recording their own submission, and a parish telling a pilgrim
 * what they decided. Nobody can write a notification into a stranger's
 * list.
 *
```

```
 * ## Why failures are swallowed
 *
 * A notification is a copy of something that already happened. If writing
 * it fails, the application was still submitted and the decision was still
 * recorded; throwing here would roll back nothing and would tell the
 * person their application failed when it did not.
 */

async function write(entry: Omit<NotificationDoc, "id" | "createdAt">): Promise<void> {
  try {
    await addDoc(collection(db, "notifications"), {
      ...entry,
      createdAt: new Date().toISOString(),
      readAt: null,
    });
  } catch (err) {
    console.debug("[notifications] not recorded:", err);
  }
}

/** Written by the applicant, about their own submission. */
export async function notifySubmitted(
  churchId: string,
  applicationId: string,
  what: string,
  reference: string,
): Promise<void> {
  const user = auth.currentUser;
  if (!user) return;
  await write({
    userId: user.uid,
    churchId,
    applicationId,
    title: "Application submitted",
    body: `Your application for ${what} has been sent to your parish. Reference ${reference}.`,
  });
}

/**
 * Written by the parish, to the applicant.
 *
 * The wording differs per outcome rather than being one templated line,
 * because "approved" and "rejected" are the two most consequential words
 * this app says to anyone and a shared sentence makes both sound
 * administrative.
 */
export async function notifyStatusChanged(
  applicantUid: string,
  churchId: string,
  applicationId: string,
  what: string,
  status: ApplicationStatus,
  note?: string,
): Promise<void> {
  const body: Record<ApplicationStatus, string> = {
    pending: `Your application for ${what} is back in the queue.`,
    under_review: `Your parish is now reviewing your application for ${what}.`,
    approved: `Good news - your application for ${what} has been approved.`,
    rejected: `Your application for ${what} was not approved.`,
```

```
    completed: `Your application for ${what} is complete.`,
  };

  await write({
    userId: applicantUid,
    churchId,
    applicationId,
    title: STATUS_LABEL[status] ?? status,
    body: note ? `${body[status]} ${note}` : body[status],
  });
}

/**
 * Live notifications for the signed-in user, newest first.
 *
 * Sorted here rather than with orderBy because a composite query on
 * userId + createdAt needs an index, and an index that has not been
 * deployed makes the list throw instead of render. The volume per person
 * is small enough that the cost is nothing.
 */
export function watchNotifications(
  onChange: (items: NotificationDoc[]) => void,
): () => void {
  const user = auth.currentUser;
  if (!user) {
    onChange([]);
    return () => {};
  }

  const q = query(collection(db, "notifications"), where("userId", "==", user.uid));
  return onSnapshot(
    q,
    snap => {
      const items = snap.docs.map(d => ({ id: d.id, ...(d.data() as NotificationDoc) }));
      items.sort((a, b) => (b.createdAt ?? "").localeCompare(a.createdAt ?? ""));
      onChange(items);
    },
    err => {
      console.debug("[notifications] subscription failed:", err);
      onChange([]);
    },
  );
}

/** The only change a reader may make. The rules allow this field alone. */
export async function markRead(notificationId: string): Promise<void> {
  try {
    await updateDoc(doc(db, "notifications", notificationId), {
      readAt: new Date().toISOString(),
    });
  } catch (err) {
    console.debug("[notifications] could not mark read:", err);
  }
}

export function unreadCount(items: NotificationDoc[]): number {
  return items.filter(n => !n.readAt).length;
}
```

```
/**
 * Telling the whole parish about something.
 *
 * ## Why this is a fan-out from the browser, and what that costs
 *
 * A notification is a document addressed to one person, so "tell the
 * parish" means one write per parishioner. The usual way is a Cloud
 * Function triggered by the change; that needs the Blaze plan, and this
 * project is on Spark. So the admin's own browser does it, one write at
 * a time, under the rule that lets a parish admin write to their own
 * parishioners.
 *
 * That is fine for a parish of a few hundred and would not be for a
 * diocese. It is capped rather than left to run: a change that would
 * notify more people than the cap tells the admin so instead of locking
 * their browser up for a minute. When this project moves to Blaze the
 * right shape is a Function on parishContent and announcements, and this
 * becomes a few lines calling it.
 *
 * ## Why it is never automatic
 *
 * Every caller passes an explicit opt-in from a checkbox the admin
 * ticked. A parish office correcting a typo in a Mass time must not send
 * four hundred phones a notification, and the surest way to make people
 * ignore notifications is to send them one that did not matter.
 */

/** Above this, the admin is told to use an announcement instead. */
export const MAX_PARISH_NOTIFICATIONS = 400;

export interface BroadcastResult {
  sent: number;
  /** Set when nothing was sent because the parish is larger than the cap. */
  tooManyFor?: number;
}

export async function notifyParish(
  churchId: string,
  title: string,
  body: string,
): Promise<BroadcastResult> {
  const snap = await getDocs(
    query(collection(db, "users"), where("churchId", "==", churchId)),
  );

  if (snap.size > MAX_PARISH_NOTIFICATIONS) {
    return { sent: 0, tooManyFor: snap.size };
  }

  let sent = 0;
  for (const person of snap.docs) {
    // Sequential, not Promise.all: a few hundred simultaneous writes from
    // one browser is how a connection gets throttled and half of them
    // fail. write() already swallows its own failures, so one bad
    // document cannot stop the rest.
    await write({
      userId: person.id,
      churchId,
      title,
```

```
      body,
    });
    sent++;
  }
  return { sent };
}
```

1.4.44 APP-114 — openingSubscriptions.ts

Code ID	APP-114
File Name	openingSubscriptions.ts
File Path	SanctDemo/src/lib/openingSubscriptions.ts
Language / Type	TypeScript
Lines	118
Functions / Classes	WatchKind (exported type); Watch (exported type); watchKey (exported function); readAll (internal function); writeAll (internal function); isWatched (exported function); setWatched (exported function); watchedKeys (exported function); notifyListeners (internal function); onWatchChange (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/WatchBell.tsx, src/lib/useOpeningNotices.ts
How to Recognize It	Open SanctDemo/src/lib/openingSubscriptions.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> "Tell me when this opens."
>
> Most of what a parish offers is closed most of the time. Confirmation runs once a year, the choir takes members when somebody leaves, and a Mass marked unavailable goes back on the schedule when the priest is back. The app could say "not available" and leave it there - and did - which puts the burden on the pilgrim to keep checking a screen that will look identical on all but one of the days they open it.
>
> A subscription is that check, moved off the pilgrim. It is kept on the device, not in Firestore: the thing being watched is already streaming in for the parish page, so the only state that needs to persist is which of those things this person cares about. That also means no account is required to be told, which matters for a parish app whose visitors mostly never sign in.
>
> What it deliberately is NOT is a push subscription. Nothing is delivered while the app is closed; see lib/deviceNotifications for why that stayed out of scope. The notice arrives the first time the app is open and sees the change, which for a ministry that opens for a fortnight is in good time, and for a Mass restored this Sunday is not. That is a real limit and it is why the bell's own wording promises a notice, not an instant one.

Complete Source Code — openingSubscriptions.ts

```
/**
 * "Tell me when this opens."
 *
 * Most of what a parish offers is closed most of the time. Confirmation
 * runs once a year, the choir takes members when somebody leaves, and a
 * Mass marked unavailable goes back on the schedule when the priest is
 * back. The app could say "not available" and leave it there - and did
 * - which puts the burden on the pilgrim to keep checking a screen that
 * will look identical on all but one of the days they open it.
```

```
 *
 * A subscription is that check, moved off the pilgrim. It is kept on the
 * device, not in Firestore: the thing being watched is already streaming
 * in for the parish page, so the only state that needs to persist is
 * which of those things this person cares about. That also means no
 * account is required to be told, which matters for a parish app whose
 * visitors mostly never sign in.
 *
 * What it deliberately is NOT is a push subscription. Nothing is
 * delivered while the app is closed; see lib/deviceNotifications for
 * why that stayed out of scope. The notice arrives the first time the
 * app is open and sees the change, which for a ministry that opens for
 * a fortnight is in good time, and for a Mass restored this Sunday is
 * not. That is a real limit and it is why the bell's own wording
 * promises a notice, not an instant one.
 */

/** What a pilgrim can ask to be told about. */
export type WatchKind = "mass" | "ministry" | "sacrament";

export interface Watch {
  kind: WatchKind;
  parishId: string;
  /** The ministry or sacrament id. Omitted for a parish's Mass schedule. */
  itemId?: string;
}

const STORE_KEY = "sanctiwalk.watch.v1";

/**
 * One line per watch, stable across reloads and orderings.
 *
 * Built from the parts rather than stored as typed, so two components
 * asking about the same ministry always agree.
 */
export function watchKey(watch: Watch): string {
  return watch.itemId
    ? `${watch.kind}:${watch.parishId}:${watch.itemId}`
    : `${watch.kind}:${watch.parishId}`;
}

function readAll(): Set<string> {
  try {
    const raw = localStorage.getItem(STORE_KEY);
    if (!raw) return new Set();
    const parsed: unknown = JSON.parse(raw);
    // Anything else in that slot is somebody else's data or a half
    // written value. An empty set is the safe reading of both.
    return Array.isArray(parsed) ? new Set(parsed.filter(v => typeof v === "string")) : new Set();
  } catch {
    // Private mode, or a quota refusal. Watching simply does not
    // persist; nothing else should break because of it.
    return new Set();
  }
}

function writeAll(keys: Set<string>): void {
  try {
    localStorage.setItem(STORE_KEY, JSON.stringify([...keys]));
```

```
  } catch {
    /* as above */
  }
}

export function isWatched(watch: Watch): boolean {
  return readAll().has(watchKey(watch));
}

/** Returns what the watch is after the change, so callers can render it. */
export function setWatched(watch: Watch, on: boolean): boolean {
  const keys = readAll();
  if (on) keys.add(watchKey(watch));
  else keys.delete(watchKey(watch));
  writeAll(keys);
  notifyListeners();
  return on;
}

export function watchedKeys(): ReadonlySet<string> {
  return readAll();
}

/**
 * Lets every bell on screen agree with every other one.
 *
 * Two bells can point at the same thing - a sacrament has one on its
 * card and another on its detail page - and localStorage fires no event
 * in the tab that wrote it, so without this the one you did not tap
 * keeps the old face until something else re-renders it.
 */
type Listener = () => void;
const listeners = new Set<Listener>();

function notifyListeners(): void {
  for (const fn of [...listeners]) {
    try {
      fn();
    } catch {
      /* one bad subscriber must not silence the rest */
    }
  }
}

export function onWatchChange(fn: Listener): () => void {
  listeners.add(fn);
  return () => {
    listeners.delete(fn);
  };
}
```

1.4.45 APP-115 — pageEvents.ts

Code ID	APP-115
File Name	pageEvents.ts
File Path	SanctDemo/src/lib/pageEvents.ts
Language / Type	TypeScript
Lines	185
Functions / Classes	PageEventKind (exported type); PageEvent (exported type); visitorId (exported function); randomId (internal function); OPEN_DEDUPE_MS (exported constant); lastKey (internal function); shouldRecord (exported function); safeGet (internal function); markRecorded (exported function); PageEventTotals (exported type); summarisePageEvents (exported function); recentEvents (exported function); byDay (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/pageEvents.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Counting who opened SanctiWalk and who downloaded it.
>
> ## Why this is separate from `visits`
>
> `visits` answers "who came to the church" - it needs a signed-in uid and a parish, because its question is about a building. This answers "how many people found the app at all", which is a question about everyone, most of whom never sign in and are not standing anywhere near a parish.
>
> ## What is stored, and what deliberately is not
>
> A kind, a timestamp, and a random id generated in the browser. No account, no name, no address, nothing that reaches a person. The id exists only so that one visitor opening the app ten times on Sunday is not reported as ten visitors; it is meaningless outside this collection and is never shown.
>
> The parish office asked to see who downloads the app. It gets counts instead, which is the same decision already made for Visitor Activity: "how many came on Sunday" and "who came on Sunday" are different questions, and only the first has been answered here.
>
> ## Why the count can only ever start today
>
> Nothing recorded this before, so there is no history to recover. A figure for last month cannot be produced from data that was never written, and inventing one would be worse than the gap.

Complete Source Code — pageEvents.ts

```
/**
```

```
 * Counting who opened SanctiWalk and who downloaded it.
 *
 * ## Why this is separate from `visits`
 *
 * `visits` answers "who came to the church" - it needs a signed-in uid
 * and a parish, because its question is about a building. This answers
 * "how many people found the app at all", which is a question about
 * everyone, most of whom never sign in and are not standing anywhere
 * near a parish.
 *
 * ## What is stored, and what deliberately is not
 *
 * A kind, a timestamp, and a random id generated in the browser. No
 * account, no name, no address, nothing that reaches a person. The id
 * exists only so that one visitor opening the app ten times on Sunday
 * is not reported as ten visitors; it is meaningless outside this
 * collection and is never shown.
 *
 * The parish office asked to see who downloads the app. It gets counts
 * instead, which is the same decision already made for Visitor
 * Activity: "how many came on Sunday" and "who came on Sunday" are
 * different questions, and only the first has been answered here.
 *
 * ## Why the count can only ever start today
 *
 * Nothing recorded this before, so there is no history to recover. A
 * figure for last month cannot be produced from data that was never
 * written, and inventing one would be worse than the gap.
 */

/** What happened. Two kinds, because two were asked for. */
export type PageEventKind = 'open' | 'download'

export interface PageEvent {
  kind: PageEventKind
  /** Random per-browser token. See the note above. */
  visitorId: string
  /** ISO 8601, from the server's clock where there is one. */
  at: string
}

const VISITOR_KEY = 'sanctiwalk.visitor'

/**
 * A stable random id for this browser.
 *
 * Not a fingerprint and not derived from anything about the device or
 * the person: it is random, kept in this browser's own storage, and
 * cleared whenever they clear site data. Private mode gets a fresh one
 * each session, which overcounts slightly - the honest direction for a
 * number nobody acts on.
 */
export function visitorId(): string {
  try {
    const existing = localStorage.getItem(VISITOR_KEY)
    if (existing) return existing
    const made = randomId()
    localStorage.setItem(VISITOR_KEY, made)
    return made
```

```
  } catch {
    // Storage refused. A per-session id still separates this visitor
    // from the others in the same window.
    return randomId()
  }
}

function randomId(): string {
  try {
    const bytes = new Uint8Array(16)
    crypto.getRandomValues(bytes)
    return Array.from(bytes, b => b.toString(16).padStart(2, '0')).join('')
  } catch {
    return `${Date.now().toString(16)}${Math.random().toString(16).slice(2, 10)}`
  }
}

/** How long before the same browser's "open" counts again. */
export const OPEN_DEDUPE_MS = 30 * 60 * 1000

function lastKey(kind: PageEventKind): string {
  return `sanctiwalk.lastEvent.${kind}`
}

/**
 * Whether this event is worth writing.
 *
 * An "open" fires on every mount, and React remounts for reasons that
 * have nothing to do with a person arriving - a reload, a tab coming
 * back, a route change in some future version. Half an hour of silence
 * per browser turns that into something closer to a visit.
 *
 * A download is not deduped. Somebody downloading twice has in fact
 * downloaded twice, usually because the first one failed, and that is
 * worth seeing.
 */
export function shouldRecord(
  kind: PageEventKind,
  now: number = Date.now(),
  read: (key: string) => string | null = safeGet,
): boolean {
  if (kind === 'download') return true
  const last = Number(read(lastKey(kind)) ?? '')
  if (!Number.isFinite(last) || last <= 0) return true
  return now - last >= OPEN_DEDUPE_MS
}

function safeGet(key: string): string | null {
  try {
    return localStorage.getItem(key)
  } catch {
    return null
  }
}

export function markRecorded(kind: PageEventKind, now: number = Date.now()): void {
  try {
    localStorage.setItem(lastKey(kind), String(now))
  } catch {
```

```
    /* private mode; it will simply record again next time */
  }
}

// ---------------------------------------------------------------- reading

export interface PageEventTotals {
  opens: number
  downloads: number
  /** Distinct browsers that opened it, not distinct opens. */
  people: number
  /** Distinct browsers that downloaded it. */
  downloaders: number
}

/**
 * The totals, from whatever rows the admin screen was given.
 *
 * Counting distinct ids as well as rows because the two answer
 * different questions and the difference matters: forty opens from
 * four browsers is a very different Sunday from forty from forty.
 */
export function summarisePageEvents(events: PageEvent[]): PageEventTotals {
  const openers = new Set<string>()
  const downloaders = new Set<string>()
  let opens = 0
  let downloads = 0

  for (const e of events) {
    if (e.kind === 'open') {
      opens += 1
      if (e.visitorId) openers.add(e.visitorId)
    } else if (e.kind === 'download') {
      downloads += 1
      if (e.visitorId) downloaders.add(e.visitorId)
    }
  }

  return { opens, downloads, people: openers.size, downloaders: downloaders.size }
}

/** Rows from the last `days` days, newest first. */
export function recentEvents(events: PageEvent[], days: number, now: Date = new Date()): PageEvent[] {
  const cutoff = now.getTime() - days * 24 * 60 * 60 * 1000
  return events
    .filter(e => {
      const t = Date.parse(e.at)
      return Number.isFinite(t) && t >= cutoff
    })
    .sort((a, b) => Date.parse(b.at) - Date.parse(a.at))
}

/** A count per calendar day, oldest first, for a small bar chart. */
export function byDay(events: PageEvent[], kind: PageEventKind): { day: string; count: number }[] {
  const tally = new Map<string, number>()
  for (const e of events) {
    if (e.kind !== kind) continue
    const t = Date.parse(e.at)
    if (!Number.isFinite(t)) continue
```

```
    const day = new Date(t).toISOString().slice(0, 10)
    tally.set(day, (tally.get(day) ?? 0) + 1)
  }
  return [...tally.entries()]
    .map(([day, count]) => ({ day, count }))
    .sort((a, b) => a.day.localeCompare(b.day))
}
```

parishContent.ts — listed in full as API-004.

parishContentStore.ts — listed in full as API-005.

1.4.46 APP-116 — parishIds.ts

Code ID	APP-116
File Name	parishIds.ts
File Path	SanctDemo/src/lib/parishIds.ts
Language / Type	TypeScript
Lines	74
Functions / Classes	PARISH_TO_ROUTE (exported constant); ROUTE_TO_PARISH (exported constant); routeIdForParish (exported function); parishIdForRoute (exported function); assertKnownParishIds (exported function)
Related Files (its imports)	../data/diocese-parishes.json
Where It Is Used	src/components/HomeHero.tsx, src/components/SearchScreen.tsx
How to Recognize It	Open SanctDemo/src/lib/parishIds.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The link between the two id spaces this app carries.
>
> `diocese-parishes.json` identifies all 31 parishes as `parish-<slug>`. `data.ts` identifies the two with a tour, schedule and history as `route-mhcp` / `route-src`. Nothing in either file points at the other, so the correspondence has to be written down — and it belongs in exactly one place.
>
> It was previously written out inline in four components, each with the `parish-` prefix missing. Every lookup silently missed: the Home hero never found a Mass time even standing at Mary Help, search's "Mass soon" filter never matched anything, and the church page lost its vicariate and founding date. Nothing threw — the code simply behaved as though the two documented parishes had no data, which is indistinguishable from the state the other 29 are genuinely in.
>
> Hence assertKnownParishIds below: a typo here now fails loudly in development instead of quietly degrading into "no data collected yet".

Complete Source Code — parishIds.ts

```
// The link between the two id spaces this app carries.
//
// `diocese-parishes.json` identifies all 31 parishes as `parish-<slug>`.
// `data.ts` identifies the two with a tour, schedule and history as
// `route-mhcp` / `route-src`. Nothing in either file points at the other, so
// the correspondence has to be written down — and it belongs in exactly one
// place.
//
// It was previously written out inline in four components, each with the
// `parish-` prefix missing. Every lookup silently missed: the Home hero
```

```
// never found a Mass time even standing at Mary Help, search's "Mass soon"
// filter never matched anything, and the church page lost its vicariate and
// founding date. Nothing threw — the code simply behaved as though the two
// documented parishes had no data, which is indistinguishable from the state
// the other 29 are genuinely in.
//
// Hence assertKnownParishIds below: a typo here now fails loudly in
// development instead of quietly degrading into "no data collected yet".

import parishData from '../data/diocese-parishes.json'

const PARISHES = (parishData as { parishes: { id: string; status?: string }[] }).parishes

/** Diocese parish id → the id of its tour in data.ts. */
export const PARISH_TO_ROUTE: Record<string, string> = {
  'parish-mary-help-of-christians-parish': 'route-mhcp',
  'parish-san-roque-cathedral': 'route-src',
}

/** The reverse: tour id → diocese parish id. */
export const ROUTE_TO_PARISH: Record<string, string> = Object.fromEntries(
  Object.entries(PARISH_TO_ROUTE).map(([parishId, routeId]) => [routeId, parishId]),
)

/** The tour for a diocese parish, or null for the 29 without one. */
export function routeIdForParish(parishId: string | null | undefined): string | null {
  return parishId ? (PARISH_TO_ROUTE[parishId] ?? null) : null
}

/** The diocese record's id for a tour. */
export function parishIdForRoute(routeId: string | null | undefined): string | null {
  return routeId ? (ROUTE_TO_PARISH[routeId] ?? null) : null
}

/**
 * Fails loudly when a key above does not exist in the parish data, or when a
 * parish marked live has no tour mapped to it.
 *
 * Called once at startup in development. The failure this guards is not a
 * crash but a silence: a stale or mistyped id makes a documented parish look
 * undocumented, and every screen dutifully renders "not collected yet".
 */
export function assertKnownParishIds(): void {
  const known = new Set(PARISHES.map(p => p.id))

  for (const parishId of Object.keys(PARISH_TO_ROUTE)) {
    if (!known.has(parishId)) {
      throw new Error(
        `parishIds: "${parishId}" is not a parish in diocese-parishes.json. ` +
          `Mass times, distances and the parish page will all silently show as uncollected. ` +
          `Ids look like "parish-san-roque-cathedral".`,
      )
    }
  }

  for (const parish of PARISHES) {
    if (parish.status === 'live' && !PARISH_TO_ROUTE[parish.id]) {
      throw new Error(
        `parishIds: "${parish.id}" is marked live in diocese-parishes.json but has no tour ` +
```

```
          `mapped to it, so its schedule and history will never be found.`,
      )
    }
  }
}
```

1.4.47 APP-117 — parishPhotos.ts

Code ID	APP-117
File Name	parishPhotos.ts
File Path	SanctDemo/src/lib/parishPhotos.ts
Language / Type	TypeScript
Lines	56
Functions / Classes	PLACEHOLDER_PHOTO (exported constant); PARISH_FACADE_PHOTOS (exported constant); parishPhoto (exported function); hasRealPhoto (exported function); parishPhotoAlt (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/MassSchedule.tsx
How to Recognize It	Open SanctDemo/src/lib/parishPhotos.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Photographs of parishes, and the one rule about them.
>
> A parish's photograph is a CLAIM about that place. Before this module the app made that claim falsely in four separate files: the same Unsplash photograph of an unrelated church stood in for San Roque Cathedral on the Mass schedule, in the history screen — captioned "San Roque Cathedral Caloocan" — and on the church-selector cards, while four of the five tour stations carried stock photographs of other churches' altars and fonts.
>
> Anyone looking at those screens would reasonably believe they were seeing San Roque. They were not. This is the same principle as `scheduleVerified` in data.ts and the "sample, unconfirmed" Mass times: show the placeholder, never the plausible-looking wrong thing.
>
> Centralised so a stock URL cannot be pasted back in one screen at a time. To add a real photograph, put it in PARISH_FACADE_PHOTOS below — and only if it is genuinely a photograph OF THAT PARISH.

Complete Source Code — parishPhotos.ts

```
/**
 * Photographs of parishes, and the one rule about them.
 *
 * A parish's photograph is a CLAIM about that place. Before this module the
 * app made that claim falsely in four separate files: the same Unsplash
 * photograph of an unrelated church stood in for San Roque Cathedral on the
 * Mass schedule, in the history screen — captioned "San Roque Cathedral
 * Caloocan" — and on the church-selector cards, while four of the five tour
 * stations carried stock photographs of other churches' altars and fonts.
 *
 * Anyone looking at those screens would reasonably believe they were seeing
 * San Roque. They were not. This is the same principle as `scheduleVerified`
 * in data.ts and the "sample, unconfirmed" Mass times: show the placeholder,
 * never the plausible-looking wrong thing.
```

```
 *
 * Centralised so a stock URL cannot be pasted back in one screen at a time.
 * To add a real photograph, put it in PARISH_FACADE_PHOTOS below — and only
 * if it is genuinely a photograph OF THAT PARISH.
 */

/** Shown wherever a real photograph does not exist. Says so on its face. */
export const PLACEHOLDER_PHOTO = '/parish/placeholder-photo.svg'

/**
 * Real, verified photographs, keyed by tour route id.
 *
 * Only Mary Help of Christians has one: the Diocese of Kalookan's own image
 * of the parish. Every other parish resolves to the placeholder until
 * somebody supplies a photograph of the actual building.
 */
export const PARISH_FACADE_PHOTOS: Record<string, string> = {
  'route-mhcp':
    'https://dioceseofkalookan.ph/wp-content/uploads/2020/12/maryhelpPic1-1-1024x680.jpg',
}

/** The facade photograph for a parish, or the placeholder. Never a stand-in. */
export function parishPhoto(routeId: string | null | undefined): string {
  return (routeId && PARISH_FACADE_PHOTOS[routeId]) || PLACEHOLDER_PHOTO
}

/** True when this is a real photograph rather than the placeholder. */
export function hasRealPhoto(routeId: string | null | undefined): boolean {
  return Boolean(routeId && PARISH_FACADE_PHOTOS[routeId])
}

/**
 * Alt text that does not lie.
 *
 * The placeholder is decorative — it depicts no particular church — so it
 * takes an empty alt rather than a parish name. Captioning a drawing "San
 * Roque Cathedral Caloocan" is exactly the bug this module exists to stop,
 * and a screen reader would repeat it as fact.
 */
export function parishPhotoAlt(routeId: string | null | undefined, parishName: string): string {
  return hasRealPhoto(routeId) ? parishName : ''
}
```

1.4.48 APP-118 — parishSelection.ts

Code ID	APP-118
File Name	parishSelection.ts
File Path	SanctDemo/src/lib/parishSelection.ts
Language / Type	TypeScript
Lines	49
Functions / Classes	Tab (exported type); ParishSelectionSource (exported type); tabForParishSelection (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/parishSelection.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Which app tab a parish selection should land on, extracted as a pure function so the exact defect behind "View parish does nothing" can be unit-tested without rendering the full App tree (which needs Firebase auth/Firestore and a MapLibre instance, neither mocked in this project's test setup — see mapSearch.ts/mapMarkers.ts for the same extraction pattern used for the same reason).
>
> The bug: App.tsx's handleOpenTourFromPresence set activeTab to "navigator" — the map tab itself — and was reused as the click handler for the map popup's "View parish" button. When that popup lives on the map tab (which it does, in the diocese-wide map screen), setting the active tab to the tab already active is a no-op: nothing visibly happens, which reads as "the button isn't working" even though the click handler fires correctly.
>
> 'pin' covers every place a pilgrim picks *which* parish to look at — a map pin/popup, a search result, a parish card — and opens that parish's dashboard.
>
> It briefly opened a separate parish screen instead, carrying the distance, today's Masses and links out to History/Ministries/ Sacraments/Scan. That screen is gone: the dashboard already shows all of it for whichever parish is active, so the extra page was a second version of Home that you had to leave before you could use anything on it. Picking a parish now simply makes it the active one.
>
> 'presence-open-tour' is the presence sheet's own "Open Tour" button (paired with its "AR Tour" button) — the pilgrim is already standing at the parish, so jumping to the diocese map is the one case where that destination is actually correct, and stays unchanged.

Complete Source Code — parishSelection.ts

```
// Which app tab a parish selection should land on, extracted as a pure
// function so the exact defect behind "View parish does nothing" can be
// unit-tested without rendering the full App tree (which needs Firebase
// auth/Firestore and a MapLibre instance, neither mocked in this project's
// test setup — see mapSearch.ts/mapMarkers.ts for the same extraction
// pattern used for the same reason).
//
// The bug: App.tsx's handleOpenTourFromPresence set activeTab to
```

```
// "navigator" — the map tab itself — and was reused as the click handler
// for the map popup's "View parish" button. When that popup lives on the
// map tab (which it does, in the diocese-wide map screen), setting the
// active tab to the tab already active is a no-op: nothing visibly happens,
// which reads as "the button isn't working" even though the click handler
// fires correctly.
//
// 'pin' covers every place a pilgrim picks *which* parish to look at — a
// map pin/popup, a search result, a parish card — and opens that parish's
// dashboard.
//
// It briefly opened a separate parish screen instead, carrying the
// distance, today's Masses and links out to History/Ministries/
// Sacraments/Scan. That screen is gone: the dashboard already shows all
// of it for whichever parish is active, so the extra page was a second
// version of Home that you had to leave before you could use anything on
// it. Picking a parish now simply makes it the active one.
//
// 'presence-open-tour' is the presence sheet's own "Open Tour" button
// (paired with its "AR Tour" button) — the pilgrim is already standing at
// the parish, so jumping to the diocese map is the one case where that
// destination is actually correct, and stays unchanged.
export type Tab =
  | "home"
  | "navigator"
  | "rosary"
  | "mass"
  | "ministries"
  | "history"
  | "sacraments"
  | "ar"
  | "quiz"
  | "me"
  | "admin"
  | "pwa-devkit"

export type ParishSelectionSource = "pin" | "presence-open-tour"

export function tabForParishSelection(source: ParishSelectionSource): Tab {
  return source === "presence-open-tour" ? "navigator" : "home"
}
```

1.4.49 APP-119 — parishTheme.ts

Code ID	APP-119
File Name	parishTheme.ts
File Path	SanctDemo/src/lib/parishTheme.ts
Language / Type	TypeScript
Lines	90
Functions / Classes	ParishTheme (exported type); parishTheme (exported function); parishThemeStyle (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ChurchHistory.tsx, src/components/Dashboard.tsx, src/components/ParishSectionHeader.tsx
How to Recognize It	Open SanctDemo/src/lib/parishTheme.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Each parish's own colours.
>
> SanctiWalk used to be navy everywhere. That navy is not the app's colour, it is Mary Help of Christians' — the blue of Maria Auxiliadora's mantle, taken off her page in "SANCTIWALK UI (1).psd" — and wearing it on San Roque's screens said the wrong thing about whose church you were standing in. Every parish gets its own livery, and Home, the parish page and the history page all read from this one table.
>
> Three colours describe a parish, and everything else on screen is mixed from them in CSS:
>
> ink the deep brand colour — headings, buttons, the primary token band the near-black behind the full-bleed welcome photograph tint the pale ground the content sheet sits on
>
> The pale steps (sunk surfaces, borders, the watermark behind each section) are `color-mix` of ink into tint, which is why they are not listed: mixing keeps them in the parish's hue automatically, where eight hand-picked browns would drift. See `.home-bright` in index.css for the mixes.
>
> The panel pair is separate because it is doing a different job — it is the gradient the history page's panel is painted with, sampled from the artboard rather than derived, and it runs bright-to-dark rather than pale.
>
> Contrast was checked for every derived pair, not assumed. The worst case across both parishes is the secondary text colour on its own ground at 6.1:1, comfortably past AA; white on ink is 13.2:1 for Mary Help and 9.4:1 for San Roque.

Complete Source Code — parishTheme.ts

```
/**
 * Each parish's own colours.
 *
```

```
 * SanctiWalk used to be navy everywhere. That navy is not the app's colour,
 * it is Mary Help of Christians' — the blue of Maria Auxiliadora's mantle,
 * taken off her page in "SANCTIWALK UI (1).psd" — and wearing it on San
 * Roque's screens said the wrong thing about whose church you were standing
 * in. Every parish gets its own livery, and Home, the parish page and the
 * history page all read from this one table.
 *
 * Three colours describe a parish, and everything else on screen is mixed
 * from them in CSS:
 *
 *   ink    the deep brand colour — headings, buttons, the primary token
 *   band   the near-black behind the full-bleed welcome photograph
 *   tint   the pale ground the content sheet sits on
 *
 * The pale steps (sunk surfaces, borders, the watermark behind each section)
 * are `color-mix` of ink into tint, which is why they are not listed: mixing
 * keeps them in the parish's hue automatically, where eight hand-picked
 * browns would drift. See `.home-bright` in index.css for the mixes.
 *
 * The panel pair is separate because it is doing a different job — it is the
 * gradient the history page's panel is painted with, sampled from the
 * artboard rather than derived, and it runs bright-to-dark rather than pale.
 *
 * Contrast was checked for every derived pair, not assumed. The worst case
 * across both parishes is the secondary text colour on its own ground at
 * 6.1:1, comfortably past AA; white on ink is 13.2:1 for Mary Help and
 * 9.4:1 for San Roque.
 */
export interface ParishTheme {
  ink: string;
  band: string;
  tint: string;
  /** The history panel's gradient: bright at the top, deep at the bottom. */
  panelTop: string;
  panelBottom: string;
}

const NAVY: ParishTheme = {
  ink: "#1B2A6B",
  band: "#101B4A",
  tint: "#EEF3FC",
  panelTop: "#3DA8EA",
  panelBottom: "#00204C",
};

const PARISH_THEMES: Record<string, ParishTheme> = {
  // Mary Help of Christians — the design's own navy, unchanged. Its derived
  // tints land within a few points of the hand-picked ones they replace.
  "route-mhcp": NAVY,

  // San Roque — the burnt orange of his habit and the warm cream of the
  // cathedral wall behind him, both off his artboard.
  "route-src": {
    ink: "#6B3A16",
    band: "#3A1F0C",
    tint: "#FAF1E8",
    panelTop: "#D37931",
    panelBottom: "#46280F",
  },
```

```
};

/**
 * A parish's livery. Parishes nobody has designed a page for keep the navy,
 * which is the app's existing appearance rather than a guess at theirs.
 */
export function parishTheme(routeId: string | null | undefined): ParishTheme {
  return (routeId && PARISH_THEMES[routeId]) || NAVY;
}

/**
 * The same theme as the custom properties the stylesheet reads.
 *
 * Spread onto the element that scopes the theme — Home's `.home-bright`, the
 * parish page's header, the history page. Everything below it recolours
 * through the tokens, so no component has to know which parish it is
 * rendering.
 */
export function parishThemeStyle(routeId: string | null | undefined): Record<string, string> {
  const theme = parishTheme(routeId);
  return {
    "--parish-ink": theme.ink,
    "--parish-band": theme.band,
    "--parish-tint": theme.tint,
    "--ph-top": theme.panelTop,
    "--ph-bottom": theme.panelBottom,
  };
}
```

1.4.50 APP-120 — pilgrimage.ts

Code ID	APP-120
File Name	pilgrimage.ts
File Path	SanctDemo/src/lib/pilgrimage.ts
Language / Type	TypeScript
Lines	203
Functions / Classes	TripStop (exported type); TripLeg (exported type); MAX_TRIP_STOPS (exported constant); tripLengthMetres (exported function); orderStops (exported function); isOptimalOrder (exported function); TripRoute (exported type); measureTrip (exported function); formatTripDuration (exported function); stopLabel (exported function)
Related Files (its imports)	./geo, ./routing
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/pilgrimage.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — pilgrimage.ts

```
import { haversineMeters, type Coordinates } from './geo'
import { fetchWalkingRoute, walkingMinutesFor, type WalkingRoute } from './routing'

/**
 * Planning a visit to several churches in one go — a Bisita Iglesia.
 *
 * The tradition is seven churches on Maundy Thursday, and the app already
 * knows where all 31 parishes in the diocese are, so the useful thing it can
 * do is answer the question nobody can answer standing in the street with a
 * list: which one first, and which one after that.
 *
 * Everything here is pure. The ordering is what decides whether a group
 * walks 6km or 11km, so it is worth being able to test without a map, a
 * network or a browser.
 */

export interface TripStop {
  /** The diocese parish id, so a church cannot be added twice. */
  id: string
  name: string
  coordinates: Coordinates
  /** The tour route id when the parish has one, for the parish page link. */
  routeId?: string | null
}

/** One church-to-church hop, once the order has been decided. */
export interface TripLeg {
  from: Coordinates
  to: Coordinates
  /** The stop this leg ARRIVES at. The first leg starts at the pilgrim. */
```

```
  stop: TripStop
  route: WalkingRoute | null
}

export const MAX_TRIP_STOPS = 14

/**
 * The total length of a visit in a given order, as the crow flies.
 *
 * Straight-line, deliberately. Ordering compares thousands of candidate
 * orders against each other, and asking a public routing server for each
 * comparison would take minutes and hammer someone else's demo service.
 * Street distances are strictly longer than straight-line ones but they are
 * longer in much the same proportion across a few square kilometres of the
 * same city, so the ORDER this produces is the order the roads produce.
 * The distances shown to the pilgrim are always the measured road ones.
 */
export function tripLengthMetres(start: Coordinates | null, stops: readonly TripStop[]): number {
  let total = 0
  let cursor = start
  for (const stop of stops) {
    if (cursor) total += haversineMeters(cursor, stop.coordinates)
    cursor = stop.coordinates
  }
  return total
}

/**
 * The order to visit them in: nearest first, then improved.
 *
 * Two passes, because one is not enough:
 *
 * 1. **Nearest neighbour.** Start where the pilgrim is, repeatedly walk to
 *    the closest church not yet visited. This is what "which one first" means
 *    to someone standing in the street, and it is a good route most of the
 *    time.
 * 2. **2-opt.** Nearest neighbour has one well-known failure: it takes every
 *    easy church first and leaves the far one for last, so the route ends
 *    with a long doubling back across everything it already walked. 2-opt
 *    repeatedly reverses a section of the order whenever doing so shortens
 *    the total, which is exactly the move that untangles that crossing. It
 *    runs until no reversal helps.
 *
 * With no known position there is no "first", so the order given is kept —
 * reordering a list against an unknown starting point would just shuffle it.
 *
 * The cost is O(n^2) per improvement sweep, which at MAX_TRIP_STOPS is a few
 * thousand comparisons: instant, and bounded.
 */
export function orderStops(start: Coordinates | null, stops: readonly TripStop[]): TripStop[] {
  if (!start || stops.length < 2) return [...stops]

  // 1. Nearest neighbour.
  const remaining = [...stops]
  const ordered: TripStop[] = []
  let cursor = start

  while (remaining.length > 0) {
    let best = 0
```

```
    let bestMetres = Infinity
    remaining.forEach((stop, index) => {
      const metres = haversineMeters(cursor, stop.coordinates)
      if (metres < bestMetres) {
        bestMetres = metres
        best = index
      }
    })
    const [next] = remaining.splice(best, 1)
    ordered.push(next)
    cursor = next.coordinates
  }

  // 2. 2-opt. An open path, not a loop: the pilgrim is not required to come
  // back to where they started, so the last stop's position is free and the
  // reversal window runs to the end of the list.
  let improved = true
  let guard = 0
  while (improved && guard < 64) {
    improved = false
    guard += 1
    for (let i = 0; i < ordered.length - 1; i++) {
      for (let k = i + 1; k < ordered.length; k++) {
        const candidate = [
          ...ordered.slice(0, i),
          ...ordered.slice(i, k + 1).reverse(),
          ...ordered.slice(k + 1),
        ]
        if (tripLengthMetres(start, candidate) < tripLengthMetres(start, ordered) - 0.5) {
          ordered.splice(0, ordered.length, ...candidate)
          improved = true
        }
      }
    }
  }

  return ordered
}

/** True when this order is already the one orderStops would produce. */
export function isOptimalOrder(start: Coordinates | null, stops: readonly TripStop[]): boolean {
  if (!start || stops.length < 2) return true
  const best = orderStops(start, stops)
  // Compared on total length, not on identity: two orders can be the same
  // length (a pair of churches side by side), and telling someone their plan
  // can be improved when it cannot would be a lie the button then fails to
  // deliver on.
  return tripLengthMetres(start, stops) <= tripLengthMetres(start, best) + 0.5
}

export interface TripRoute {
  legs: TripLeg[]
  distanceMeters: number
  durationMinutes: number
  /** True when any leg fell back to a straight line — the UI must say so. */
  hasDirectLeg: boolean
}

/**
```

```
 * Measures every hop of the visit on real streets.
 *
 * Sequential rather than parallel. These go to OSRM's public demo server,
 * which rate-limits, and a seven-church visit firing seven simultaneous
 * requests is how a planner gets itself throttled into seven straight lines.
 * Seven requests at roughly a second each is a wait worth having for
 * distances that are true.
 *
 * A leg that cannot be routed comes back as a straight line rather than
 * failing the trip — fetchWalkingRoute already guarantees that — and
 * hasDirectLeg is how the UI knows not to call the total a walking distance.
 */
export async function measureTrip(
  start: Coordinates | null,
  stops: readonly TripStop[],
  fetchRoute: typeof fetchWalkingRoute = fetchWalkingRoute,
): Promise<TripRoute> {
  const legs: TripLeg[] = []
  let cursor = start

  for (const stop of stops) {
    if (!cursor) {
      // No fix yet: the first church has no measurable approach, but every
      // hop after it does, so the trip is still worth measuring.
      legs.push({ from: stop.coordinates, to: stop.coordinates, stop, route: null })
      cursor = stop.coordinates
      continue
    }
    const route = await fetchRoute(cursor, stop.coordinates)
    legs.push({ from: cursor, to: stop.coordinates, stop, route })
    cursor = stop.coordinates
  }

  const distanceMeters = legs.reduce((sum, leg) => sum + (leg.route?.distanceMeters ?? 0), 0)
  return {
    legs,
    distanceMeters,
    durationMinutes: walkingMinutesFor(distanceMeters),
    hasDirectLeg: legs.some(leg => leg.route?.kind === 'direct'),
  }
}

/** "5 hr 12 min", "48 min" — the form the trip footer uses. */
export function formatTripDuration(minutes: number): string {
  const total = Math.round(minutes)
  if (total < 60) return `${total} min`
  const hours = Math.floor(total / 60)
  const rest = total % 60
  return rest === 0 ? `${hours} hr` : `${hours} hr ${rest} min`
}

/** A, B, C … for the stop markers, as a maps app labels them. */
export function stopLabel(index: number): string {
  return String.fromCharCode(65 + (index % 26))
}
```

1.4.51 APP-121 — presence.ts

Code ID	APP-121
File Name	presence.ts
File Path	SanctDemo/src/lib/presence.ts
Language / Type	TypeScript
Lines	114
Functions / Classes	PresenceMode (exported type); GeoParish (exported type); PresenceState (exported type); APPROACH_ENTER_M (exported constant); APPROACH_LEAVE_M (exported constant); PRESENT_LEAVE_FACTOR (exported constant); MIN_DWELL_MS (exported constant); FORCE_TRANSITION_AT (exported constant); INITIAL_PRESENCE (exported constant); nearestLiveParish (internal function); rawMode (internal function); nextPresence (exported function)
Related Files (its imports)	./geo
Where It Is Used	src/context/PresenceContext.tsx, src/lib/geo.ts
How to Recognize It	Open SanctDemo/src/lib/presence.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — presence.ts

```
import { haversineMeters, type Coordinates } from './geo'

export type PresenceMode = 'diocese' | 'approaching' | 'present'

export interface GeoParish {
  id: string
  coordinates: Coordinates
  geofenceRadius: number
  status: 'live' | 'coming_soon'
}

export interface PresenceState {
  mode: PresenceMode
  parishId: string | null
  distance: number | null
  changedAt: number
}

export const APPROACH_ENTER_M = 500
export const APPROACH_LEAVE_M = 600
export const PRESENT_LEAVE_FACTOR = 1.5
export const MIN_DWELL_MS = 10000

// A changedAt of exactly 0 means "never transitioned, or a caller is forcing
// an immediate transition" — it always clears the dwell gate. The demo
// location simulator passes { ...prev, changedAt: FORCE_TRANSITION_AT } so a
// switch responds instantly. This is public contract, not an accident of
// clock magnitude.
```

```
export const FORCE_TRANSITION_AT = 0

export const INITIAL_PRESENCE: PresenceState = {
  mode: 'diocese',
  parishId: null,
  distance: null,
  changedAt: FORCE_TRANSITION_AT,
}

function nearestLiveParish(
  pos: Coordinates | null,
  parishes: GeoParish[],
): { parish: GeoParish; distance: number } | null {
  if (!pos) return null

  let best: { parish: GeoParish; distance: number } | null = null
  for (const parish of parishes) {
    if (parish.status !== 'live') continue
    const distance = haversineMeters(pos, parish.coordinates)
    if (!best || distance < best.distance) {
      best = { parish, distance }
    }
  }
  return best
}

// Works out which mode the raw position implies, taking the previous mode
// into account so the boundaries are sticky rather than knife-edge.
function rawMode(
  prev: PresenceState,
  nearest: { parish: GeoParish; distance: number } | null,
): { mode: PresenceMode; parishId: string | null } {
  if (!nearest) return { mode: 'diocese', parishId: null }

  const { parish, distance } = nearest
  const wasPresentHere = prev.mode === 'present' && prev.parishId === parish.id
  const wasApproachingHere = prev.mode === 'approaching' && prev.parishId === parish.id

  const presentLeave = parish.geofenceRadius * PRESENT_LEAVE_FACTOR

  if (wasPresentHere ? distance <= presentLeave : distance <= parish.geofenceRadius) {
    return { mode: 'present', parishId: parish.id }
  }

  const approachLeave =
    wasApproachingHere || wasPresentHere ? APPROACH_LEAVE_M : APPROACH_ENTER_M

  if (distance <= approachLeave) {
    return { mode: 'approaching', parishId: parish.id }
  }

  return { mode: 'diocese', parishId: null }
}

export function nextPresence(
  prev: PresenceState,
  pos: Coordinates | null,
  now: number,
  parishes: GeoParish[],
```

```
): PresenceState {
  const nearest = nearestLiveParish(pos, parishes)
  const distance = nearest ? nearest.distance : null
  const candidate = rawMode(prev, nearest)

  const unchanged = candidate.mode === prev.mode && candidate.parishId === prev.parishId

  // Distance always tracks reality, even when a mode change is suppressed.
  if (unchanged) {
    return { ...prev, distance }
  }

  const dwellElapsed =
    prev.changedAt === FORCE_TRANSITION_AT || now - prev.changedAt >= MIN_DWELL_MS
  if (!dwellElapsed) {
    return { ...prev, distance }
  }

  return {
    mode: candidate.mode,
    parishId: candidate.parishId,
    distance,
    // Never record 0 as a real transition time — that would leave the force
    // sentinel armed and let the very next tick skip the debounce.
    changedAt: now === FORCE_TRANSITION_AT ? 1 : now,
  }
}
```

1.4.52 APP-122 — project.ts

Code ID	APP-122
File Name	project.ts
File Path	SanctDemo/src/lib/project.ts
Language / Type	TypeScript
Lines	43
Functions / Classes	MapBounds (exported type); MapPoint (exported type); DIOCESE_BOUNDS (exported constant); projectToMap (exported function)
Related Files (its imports)	./geo
Where It Is Used	src/components/DioceseMap.tsx, src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/lib/project.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — project.ts

```
import type { Coordinates } from './geo'

export interface MapBounds {
  latMin: number
  latMax: number
  lngMin: number
  lngMax: number
}

export interface MapPoint {
  x: number
  y: number
}

// The diocese's territory: southern Caloocan, Malabon and Navotas.
export const DIOCESE_BOUNDS: MapBounds = {
  latMin: 14.610,
  latMax: 14.700,
  lngMin: 120.925,
  lngMax: 121.000,
}

// Linear equirectangular projection. Over a ~10km box the distortion is far
// below the width of a map pin, so the extra maths of a real projection buys
// nothing here.
export function projectToMap(
  coords: Coordinates | null | undefined,
  bounds: MapBounds,
  width: number,
  height: number
): MapPoint | null {
  if (!coords) return null
```

```
  const { lat, lng } = coords
  if (lat < bounds.latMin || lat > bounds.latMax) return null
  if (lng < bounds.lngMin || lng > bounds.lngMax) return null

  const x = ((lng - bounds.lngMin) / (bounds.lngMax - bounds.lngMin)) * width
  // SVG y grows downward, so higher latitude means smaller y.
  const y = ((bounds.latMax - lat) / (bounds.latMax - bounds.latMin)) * height

  return { x, y }
}
```

1.4.53 APP-123 — qr.ts

Code ID	APP-123
File Name	qr.ts
File Path	SanctDemo/src/lib/qr.ts
Language / Type	TypeScript
Lines	122
Functions / Classes	StationMatch (exported type); QrScanResult (exported type); extractStationCode (exported function); looksLikeStationCode (exported function); findStationByCode (exported function); resolveScannedQr (exported function); allStationCodes (exported function)
Related Files (its imports)	../data, ../types
Where It Is Used	src/components/ArTour.tsx, src/lib/useQrScanner.ts
How to Recognize It	Open SanctDemo/src/lib/qr.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — qr.ts

```
import { ROUTES } from '../data'
import type { Route, Station } from '../types'

/**
 * Turning a scanned QR code into the station it names.
 *
 * The codes themselves already existed — `Station.qrCode` in types.ts, with a
 * value on all five stations in data.ts — but nothing ever read one. MapTab
 * printed the code on screen as text and that was the whole of it. This is the
 * part that was missing.
 *
 * Kept separate from the camera and the decoder so the matching rules can be
 * tested without either, the same way mapSearch.ts and parishIds.ts are.
 */

export interface StationMatch {
  station: Station
  /** The tour this station belongs to, e.g. "route-mhcp". */
  routeId: string
  route: Route
}

/**
 * What a scanned code turned out to be.
 *
 * `unknown` is deliberately distinct from `not-a-sanctiwalk-code`: one is a
 * SanctiWalk code for a station this build does not carry (a poster printed
 * for a parish whose content has not shipped yet), the other is somebody's
 * wifi password or a shop's payment code. A pilgrim who scanned the parish's
 * own poster and is told "that isn't a SanctiWalk code" will reasonably
 * conclude the app is broken.
```

```
 */
export type QrScanResult =
  | { kind: 'station'; match: StationMatch }
  | { kind: 'unknown-station'; code: string }
  | { kind: 'not-ours'; raw: string }

/**
 * Pulls the station code out of whatever the QR actually encoded.
 *
 * Parish posters should carry a URL rather than a bare code, because a bare
 * code does nothing at all in a phone's built-in camera app — the overwhelming
 * majority of scans. A URL opens the app (or the web app) and still works for
 * someone who has never installed anything. So both forms are accepted:
 *
 *   MHCP-ALTAR
 *   https://sanctiwalk.app/s/MHCP-ALTAR
 *   sanctiwalk://station/MHCP-ALTAR
 *
 * Case and surrounding whitespace are ignored: printers, laminators and
 * hand-typed codes are not careful about either.
 */
export function extractStationCode(raw: string): string | null {
  const trimmed = (raw ?? '').trim()
  if (!trimmed) return null

  // The last non-empty path segment of a URL, or the whole string otherwise.
  // Query strings and fragments are dropped — a poster's code may well carry
  // a campaign parameter appended by whoever generated the QR.
  const withoutQuery = trimmed.split(/[?#]/)[0]
  const segments = withoutQuery.split('/').filter(Boolean)
  const candidate = segments.length > 0 ? segments[segments.length - 1] : withoutQuery

  const normalized = candidate.trim().toUpperCase()

  // Station codes are letters, digits and hyphens. Anything else — a wifi
  // string, a payment code, a sentence — is not one of ours.
  return /^[A-Z0-9][A-Z0-9-]*$/.test(normalized) ? normalized : null
}

/** True for a code shaped like one of ours, whether or not it is known. */
export function looksLikeStationCode(code: string): boolean {
  // Two hyphen-separated parts: a parish prefix and a station name.
  return /^[A-Z0-9]+-[A-Z0-9-]+$/.test(code)
}

/** Finds the station a code names, across every route. */
export function findStationByCode(code: string): StationMatch | null {
  const wanted = code.trim().toUpperCase()

  for (const route of ROUTES) {
    for (const station of route.stations ?? []) {
      if ((station.qrCode ?? '').trim().toUpperCase() === wanted) {
        return { station, routeId: route.id, route }
      }
    }
  }

  return null
}
```

```
/**
 * The whole job: a raw decoded QR payload in, a decision out.
 *
 * Never throws and never guesses. A code it cannot place is reported as
 * exactly that, so the scanner can say something true rather than opening the
 * nearest station and hoping.
 */
export function resolveScannedQr(raw: string): QrScanResult {
  const code = extractStationCode(raw)
  if (!code) return { kind: 'not-ours', raw: (raw ?? '').trim() }

  const match = findStationByCode(code)
  if (match) return { kind: 'station', match }

  return looksLikeStationCode(code)
    ? { kind: 'unknown-station', code }
    : { kind: 'not-ours', raw: (raw ?? '').trim() }
}

/** Every station code this build knows, for the admin's poster sheet. */
export function allStationCodes(): { code: string; station: string; parish: string }[] {
  return ROUTES.flatMap(route =>
    (route.stations ?? [])
      .filter(station => station.qrCode)
      .map(station => ({
        code: station.qrCode,
        station: station.name,
        parish: route.name.replace(' Guide', '').replace(' Tour', ''),
      })),
  )
}
```

1.4.54 APP-124 — recordPageEvent.ts

Code ID	APP-124
File Name	recordPageEvent.ts
File Path	SanctDemo/src/lib/recordPageEvent.ts
Language / Type	TypeScript
Lines	34
Functions / Classes	recordPageEvent (exported function)
Related Files (its imports)	./firebase
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/recordPageEvent.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — recordPageEvent.ts

```
import { addDoc, collection, serverTimestamp } from "firebase/firestore";

import { db } from "./firebase";
import {
  markRecorded, shouldRecord, visitorId, type PageEventKind,
} from "./pageEvents";

/**
 * Writes one page event, or quietly does not.
 *
 * Separated from pageEvents.ts so the rules there stay testable
 * without a Firestore: this file is the only part that talks to one.
 *
 * Every failure is swallowed. A pilgrim opening the app must never see
 * an error because a counter did not save, and the count is a courtesy
 * to the parish office rather than something the app depends on.
 */
export async function recordPageEvent(kind: PageEventKind): Promise<void> {
  try {
    if (!shouldRecord(kind)) return;
    // Marked before the write, not after. A write that fails must not
    // be retried on every render for the rest of the session.
    markRecorded(kind);
    await addDoc(collection(db, "pageEvents"), {
      kind,
      visitorId: visitorId(),
      // The server's clock. A handset with a wrong date cannot file
      // itself into last month.
      createdAt: serverTimestamp(),
    });
  } catch {
    /* counting is not worth interrupting anybody for */
  }
```

```
}
```

1.4.55 APP-125 — recordVisit.ts

Code ID	APP-125
File Name	recordVisit.ts
File Path	SanctDemo/src/lib/recordVisit.ts
Language / Type	TypeScript
Lines	127
Functions / Classes	lastVisitAt (internal function); rememberVisit (internal function); recordChurchVisit (exported function); HEARTBEAT_INTERVAL_MS (exported constant); touchActivity (exported function); resetHeartbeat (exported function)
Related Files (its imports)	./firebase, ./visits
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/recordVisit.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — recordVisit.ts

```
import { addDoc, collection, doc, serverTimestamp, updateDoc } from 'firebase/firestore'
import { auth, db } from './firebase'
import { ACTIVE_WINDOW_MS, shouldRecordVisit, visitKey } from './visits'

/**
 * Writing the two things the parish dashboard counts.
 *
 * Both are deliberately cheap. Firestore on the free plan is billed by
 * the document, and a tour guide that spends a parish's quota counting
 * its own users would be a poor trade.
 */

/** Reads the device's note of when it last recorded a visit here. */
function lastVisitAt(uid: string, churchId: string): number | null {
  try {
    const raw = localStorage.getItem(visitKey(uid, churchId))
    if (!raw) return null
    const n = Number(raw)
    return Number.isFinite(n) ? n : null
  } catch {
    // Storage blocked. Treated as "never", which at worst records one
    // extra visit per session rather than losing the feature.
    return null
  }
}

function rememberVisit(uid: string, churchId: string, at: number): void {
  try {
    localStorage.setItem(visitKey(uid, churchId), String(at))
  } catch {
    /* ignore */
```

```
  }
}

/**
 * Records that a signed-in pilgrim was inside a parish's geofence.
 *
 * Called when presence.ts moves to 'present', which already requires
 * the pilgrim to be within the parish's radius and to have stayed
 * there past the dwell gate. This adds only the deduplication and the
 * write.
 *
 * Signed out, nothing is recorded. An anonymous visit could not be
 * deduplicated, could not be attributed to a parish's own
 * congregation, and would need a rule that let anyone write anything
 * into the collection.
 *
 * Failures are swallowed on purpose. A pilgrim who walks into a church
 * and gets an error about analytics has been given a worse experience
 * so that a statistic could be collected, which is the wrong way round.
 */
export async function recordChurchVisit(churchId: string): Promise<boolean> {
  const user = auth.currentUser
  if (!user || !churchId) return false

  const now = Date.now()
  if (!shouldRecordVisit(lastVisitAt(user.uid, churchId), now)) return false

  // Written before the request, not after. Two geofence events can
  // arrive in the same tick, and a cooldown recorded on success would
  // let both through.
  rememberVisit(user.uid, churchId, now)

  try {
    await addDoc(collection(db, 'visits'), {
      uid: user.uid,
      churchId,
      at: new Date(now).toISOString(),
      // The server's own clock, for anything that must not trust the
      // handset's. The rules check it.
      createdAt: serverTimestamp(),
    })
    return true
  } catch (err) {
    console.debug('[visits] not recorded:', err)
    return false
  }
}

/**
 * The last time this device told Firestore the pilgrim was using the app.
 *
 * Module scope rather than storage: it only has to survive the session,
 * and reading storage on every heartbeat is more work than the write it
 * is trying to avoid.
 */
let lastHeartbeat = 0

/**
 * How often a heartbeat may be written.
```

```
 *
 * A third of the active window. Often enough that somebody using the
 * app continuously is never counted as having left, rare enough that an
 * hour of use costs twelve writes rather than one per screen.
 */
export const HEARTBEAT_INTERVAL_MS = ACTIVE_WINDOW_MS / 3

/**
 * Marks the signed-in pilgrim as active, at most once per interval.
 *
 * Throttled hard. The naive version - a write per navigation - turns a
 * ten-minute visit into dozens of writes per user, and the figure it
 * produces is no better for it.
 */
export async function touchActivity(): Promise<void> {
  const user = auth.currentUser
  if (!user) return

  const now = Date.now()
  if (now - lastHeartbeat < HEARTBEAT_INTERVAL_MS) return
  lastHeartbeat = now

  try {
    await updateDoc(doc(db, 'users', user.uid), {
      lastActiveAt: new Date(now).toISOString(),
    })
  } catch (err) {
    // A profile that does not exist yet, or rules refusing it. Neither
    // is worth interrupting anybody over.
    console.debug('[activity] heartbeat not written:', err)
  }
}

/** Lets a sign-out reset the throttle, so the next account is seen at once. */
export function resetHeartbeat(): void {
  lastHeartbeat = 0
}
```

1.4.56 APP-126 — registerServiceWorker.ts

Code ID	APP-126
File Name	registerServiceWorker.ts
File Path	SanctDemo/src/lib/registerServiceWorker.ts
Language / Type	TypeScript
Lines	46
Functions / Classes	insideNativeApp (internal function); registerServiceWorker (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/main.tsx
How to Recognize It	Open SanctDemo/src/lib/registerServiceWorker.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turning the web build into something an iPhone can keep.
>
> ## Why only here, and only sometimes
>
> The Android app is a real APK and serves its own bundle from inside itself. A service worker there would be a second cache in front of files that are already local, and the first thing it would do is make an update stick. So it is registered only in a browser.
>
> It is also registered only in a production build. In development Vite serves modules it expects to replace on every save, and a worker caching them turns hot reload into a puzzle.

Complete Source Code — registerServiceWorker.ts

```
/**
 * Turning the web build into something an iPhone can keep.
 *
 * ## Why only here, and only sometimes
 *
 * The Android app is a real APK and serves its own bundle from inside
 * itself. A service worker there would be a second cache in front of
 * files that are already local, and the first thing it would do is make
 * an update stick. So it is registered only in a browser.
 *
 * It is also registered only in a production build. In development Vite
 * serves modules it expects to replace on every save, and a worker
 * caching them turns hot reload into a puzzle.
 */

/** True inside the Capacitor shell, where the bundle is already local. */
function insideNativeApp(): boolean {
  if (typeof window === 'undefined') return false
  // Read from the global rather than importing Capacitor: this runs
```

```
  // before the app mounts, and it must not drag the native bridge into
  // the first chunk a browser downloads.
  const cap = (window as { Capacitor?: { isNativePlatform?: () => boolean } }).Capacitor
  try {
    return cap?.isNativePlatform?.() === true
  } catch {
    return false
  }
}

export function registerServiceWorker(): void {
  if (typeof window === 'undefined') return
  if (!('serviceWorker' in navigator)) return
  if (insideNativeApp()) return
  if (!import.meta.env.PROD) return

  // After load, not during. Registering while the page is still fetching
  // its own assets makes the worker compete with the thing it exists to
  // speed up, and the first visit is the one that matters.
  window.addEventListener('load', () => {
    void navigator.serviceWorker.register('/sw.js').catch(error => {
      // A refused registration is not worth an error screen. The app
      // works without it; it simply will not open offline.
      console.warn('[SanctiWalk] offline support unavailable:', error)
    })
  })
}
```

1.4.57 APP-127 — reminderSchedule.ts

Code ID	APP-127
File Name	reminderSchedule.ts
File Path	SanctDemo/src/lib/reminderSchedule.ts
Language / Type	TypeScript
Lines	337
Functions / Classes	MAX_SCHEDULED (exported constant); MASS_HORIZON_DAYS (exported constant); parseAnyTime (exported function); ReminderKind (exported type); Reminder (exported type); ReminderSettings (exported type); DEFAULT_REMINDER_SETTINGS (exported constant); LEAD_CHOICES (exported constant); reminderId (exported function); upcomingMasses (exported function); massReminders (exported function); announcementReminders (exported function); ParishReminders (exported type); buildScheduleForParishes (exported function); buildSchedule (exported function)
Related Files (its imports)	./schedule, ./announcements
Where It Is Used	src/components/ReminderSettingsCard.tsx, src/lib/deviceNotifications.ts
How to Recognize It	Open SanctDemo/src/lib/reminderSchedule.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — reminderSchedule.ts

```
import { availableTimes, toMinutes, type MassScheduleEntry } from "./schedule";
import { statusOf, type AnnouncementDoc } from "./announcements";

/**
 * What the phone should be told to show, and when.
 *
 * ## Why this is pure
 *
 * Everything here is "given a schedule and a clock, which notifications
 * exist" — no plugin, no Capacitor, no Date.now(). The whole reason
 * reminders are hard is the timing: an hour before a 6:00 AM Mass is
 * 5:00 AM *the previous evening's* tomorrow, a Mass that has already
 * started is not a reminder, and rescheduling must not fire a second
 * copy of something already queued. None of that needs a device to test,
 * and all of it is wrong at least once if it is written inside a
 * component.
 *
 * ## Why local notifications and not push
 *
 * Push means FCM, which means a server holding a service-account key,
 * which means Cloud Functions and the Blaze plan. This project is on
 * Spark. A local notification is scheduled ON the phone and fires
 * whether or not the app is running, whether or not there is signal —
 * which for "Mass starts in an hour" is better than push would be
 * anyway: it works in a church basement with no bars.
```

```
 *
 * The honest limit is that only things the phone already KNOWS can be
 * scheduled this way. A Mass time, a feast date, an announcement it has
 * synced: yes. A parish decision made five minutes ago while the app is
 * closed: no, not until the app next opens. That gap is real and is
 * documented where it bites, in notifyOnOpen below.
 */

/** Android allows a limited number of pending alarms; stay well inside it. */
export const MAX_SCHEDULED = 48;

/** How far ahead to schedule Mass reminders. */
export const MASS_HORIZON_DAYS = 7;

const DAYS = [
  "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
];

/**
 * Minutes past midnight, from either clock format the app produces.
 *
 * The two halves of this app write times differently and nothing made
 * them agree. The Mass schedule is hand-typed as "6:00 AM"; the
 * announcement form uses `<input type="time">`, which always yields
 * 24-hour "15:00". `toMinutes` only ever understood the first, so every
 * feast and event the parish gave a time to silently fell back to 8am —
 * a reminder at the wrong hour, which is worse than none.
 *
 * Returns NaN for anything it cannot read, and the caller treats that as
 * "no time given" rather than guessing.
 */
export function parseAnyTime(value: string): number {
  const twelveHour = toMinutes(value);
  if (!Number.isNaN(twelveHour)) return twelveHour;

  const match = value.trim().match(/^(\d{1,2}):(\d{2})$/);
  if (!match) return NaN;

  const hours = Number(match[1]);
  const minutes = Number(match[2]);
  if (hours > 23 || minutes > 59) return NaN;
  return hours * 60 + minutes;
}

export type ReminderKind = "mass" | "feast" | "event" | "announcement";

export interface Reminder {
  /**
   * Stable across reschedules, and numeric because Android identifies a
   * pending alarm by int. Rescheduling the same Mass must replace its
   * alarm rather than add a second one, so the id is derived from what
   * the reminder IS, not from when it was created.
   */
  id: number;
  kind: ReminderKind;
  title: string;
  body: string;
  /** When the notification should appear. */
  at: Date;
```

```
}

export interface ReminderSettings {
  massReminders: boolean;
  /** Minutes before the Mass. 60 is the default the brief asks for. */
  massLeadMinutes: number;
  feastReminders: boolean;
  eventReminders: boolean;
  announcementReminders: boolean;
  /** Status changes on your own applications. */
  applicationUpdates: boolean;
}

export const DEFAULT_REMINDER_SETTINGS: ReminderSettings = {
  massReminders: true,
  massLeadMinutes: 60,
  feastReminders: true,
  eventReminders: true,
  announcementReminders: true,
  applicationUpdates: true,
};

/** Lead times the settings screen offers, in minutes. */
export const LEAD_CHOICES = [15, 30, 60, 120] as const;

/**
 * A stable 31-bit id from a string.
 *
 * Android's notification id is a signed int, so the hash is masked into
 * that range. Collisions are possible in principle; with a few dozen
 * reminders whose keys are "mass:Sunday:6:00 AM" they are not a
 * practical concern, and the cost of one would be a single reminder
 * replacing another rather than anything worse.
 */
export function reminderId(key: string): number {
  let hash = 5381;
  for (let i = 0; i < key.length; i++) {
    hash = ((hash << 5) + hash + key.charCodeAt(i)) | 0;
  }
  return Math.abs(hash) % 2_000_000_000;
}

/**
 * Every Mass in the next week, as real dates.
 *
 * Suspended times are already excluded by availableTimes — a reminder
 * for a Mass the parish has called off is the single worst notification
 * this app could send.
 */
export function upcomingMasses(
  schedule: MassScheduleEntry[],
  now: Date,
  horizonDays: number = MASS_HORIZON_DAYS,
): Array<{ day: string; time: string; at: Date }> {
  const out: Array<{ day: string; time: string; at: Date }> = [];

  for (let offset = 0; offset <= horizonDays; offset++) {
    const date = new Date(now);
    date.setDate(date.getDate() + offset);
```

```
    const dayName = DAYS[date.getDay()];
    const entry = schedule.find(e => e.day === dayName);
    if (!entry) continue;

    for (const time of availableTimes(entry)) {
      const minutes = toMinutes(time);
      if (Number.isNaN(minutes)) continue;

      const at = new Date(date);
      at.setHours(Math.floor(minutes / 60), minutes % 60, 0, 0);
      // A Mass that has already begun is not upcoming. Without this the
      // first entry every morning is one that started hours ago.
      if (at.getTime() <= now.getTime()) continue;
      out.push({ day: dayName, time, at });
    }
  }

  return out.sort((a, b) => a.at.getTime() - b.at.getTime());
}

export function massReminders(
  schedule: MassScheduleEntry[],
  parishName: string,
  now: Date,
  settings: ReminderSettings,
  /**
   * Which parish these belong to.
   *
   * Part of the id, and it has to be: the id was keyed on the Mass time
   * alone, so two parishes that both have a 6:00 AM Sunday Mass
   * produced the SAME id and Android replaced one alarm with the other.
   * Following a second parish silently cost you a reminder from the
   * first. Optional only so the single-parish callers and their tests
   * read unchanged.
   */
  parishId = "",
): Reminder[] {
  if (!settings.massReminders) return [];
  const lead = settings.massLeadMinutes;

  return upcomingMasses(schedule, now)
    .map(mass => {
      const at = new Date(mass.at.getTime() - lead * 60_000);
      return {
        // Keyed on the parish AND the moment, so tomorrow's 6 AM
        // reminder keeps its id across every reschedule between now and
        // then, and two parishes at the same hour stay two reminders.
        id: reminderId(`mass:${parishId}:${mass.at.toISOString()}`),
        kind: "mass" as const,
        title: lead >= 60
          ? `Mass in ${lead / 60} hour${lead === 60 ? "" : "s"}`
          : `Mass in ${lead} minutes`,
        body: `${mass.time} Mass at ${parishName}.`,
        at,
      };
    })
    // A reminder whose moment has passed cannot be scheduled. This is the
    // ordinary case for the next Mass when the app is opened inside the
    // lead window - the Mass is still upcoming, its reminder is not.
```

```
    .filter(r => r.at.getTime() > now.getTime());
}

/**
 * The morning of a feast or event, and the evening before.
 *
 * Two reminders rather than one: the evening before is when somebody can
 * still change their plans, and the morning of is when they act on it.
 */
export function announcementReminders(
  announcements: AnnouncementDoc[],
  now: Date,
  settings: ReminderSettings,
): Reminder[] {
  const out: Reminder[] = [];

  for (const item of announcements) {
    if (statusOf(item) !== "published") continue;

    const kind: ReminderKind =
      item.type === "Feast" ? "feast" : item.type === "Event" ? "event" : "announcement";

    if (kind === "feast" && !settings.feastReminders) continue;
    if (kind === "event" && !settings.eventReminders) continue;
    if (kind === "announcement" && !settings.announcementReminders) continue;

    const when = item.date ? new Date(item.date) : null;
    if (!when || Number.isNaN(when.getTime())) continue;

    // The time the parish gave, or 8am when they gave none. An all-day
    // notice firing at midnight is a notification nobody reads.
    const minutes = item.time ? parseAnyTime(item.time) : NaN;
    const start = new Date(when);
    if (Number.isFinite(minutes)) {
      start.setHours(Math.floor(minutes / 60), minutes % 60, 0, 0);
    } else {
      start.setHours(8, 0, 0, 0);
    }

    const label = kind === "feast" ? "Feast day" : kind === "event" ? "Parish event" : "Parish notice";

    const dayBefore = new Date(start.getTime() - 24 * 60 * 60_000);
    if (dayBefore.getTime() > now.getTime()) {
      out.push({
        id: reminderId(`ann-eve:${item.id}`),
        kind,
        title: `${label} tomorrow`,
        body: item.title,
        at: dayBefore,
      });
    }

    if (start.getTime() > now.getTime()) {
      out.push({
        id: reminderId(`ann-day:${item.id}`),
        kind,
        title: kind === "announcement" ? label : `${label} today`,
        body: item.location ? `${item.title} — ${item.location}` : item.title,
        at: start,
```

```
      });
    }
  }

  return out;
}

/**
 * Everything to hand the phone, soonest first and capped.
 *
 * Android limits how many alarms one app may have pending, and a parish
 * with a busy week plus a full Mass schedule can produce more than the
 * cap. Sorting by time before cutting means what survives is the next
 * few days — which is what a reminder is for. The list is rebuilt every
 * time the app opens, so the ones trimmed off the end get scheduled long
 * before they matter.
 */
/**
 * One parish's contribution to the reminder queue.
 *
 * A parish is its times and its notices; the caller is responsible for
 * having fetched both. Bundling them means adding a parish is adding an
 * item to an array rather than threading two more parallel lists
 * through every function below.
 */
export interface ParishReminders {
  parishId: string;
  parishName: string;
  massSchedule: MassScheduleEntry[];
  announcements: AnnouncementDoc[];
}

/**
 * The queue across every parish the pilgrim follows.
 *
 * Merged, then sorted by time, then capped — in that order, and the
 * order matters. Capping each parish separately would hold a slot for
 * next Sunday at a parish across the city while dropping tonight's Mass
 * at their own. What the phone is holding should be whatever happens
 * soonest, wherever it is.
 */
export function buildScheduleForParishes(
  parishes: ParishReminders[],
  now: Date,
  settings: ReminderSettings,
): Reminder[] {
  const all: Reminder[] = [];

  for (const parish of parishes) {
    all.push(
      ...massReminders(parish.massSchedule, parish.parishName, now, settings, parish.parishId),
      ...announcementReminders(parish.announcements, now, settings),
    );
  }

  return all
    .filter(r => r.at.getTime() > now.getTime())
    .sort((a, b) => a.at.getTime() - b.at.getTime())
    .slice(0, MAX_SCHEDULED);
```

```
}

export function buildSchedule(
  massSchedule: MassScheduleEntry[],
  announcements: AnnouncementDoc[],
  parishName: string,
  now: Date,
  settings: ReminderSettings,
): Reminder[] {
  return [
    ...massReminders(massSchedule, parishName, now, settings),
    ...announcementReminders(announcements, now, settings),
  ]
    .filter(r => r.at.getTime() > now.getTime())
    .sort((a, b) => a.at.getTime() - b.at.getTime())
    .slice(0, MAX_SCHEDULED);
}
```

1.4.58 APP-128 — rosarySettings.ts

Code ID	APP-128
File Name	rosarySettings.ts
File Path	SanctDemo/src/lib/rosarySettings.ts
Language / Type	TypeScript
Lines	71
Functions / Classes	RosaryLanguage (exported type); RosaryMysterySet (exported type); RosaryMusic (exported type); RosarySettings (exported type); ROSARY_SETTINGS_KEY (exported constant); DEFAULT_ROSARY_SETTINGS (exported constant); isValidRosarySettings (internal function); loadRosarySettings (exported function); saveRosarySettings (exported function); parseRosarySettingsEventValue (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/rosarySettings.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Shared rosary settings, kept in sync between the app's own "Rosary settings" surface (src/components/RosarySettingsModal.tsx) and the client's standalone rosary (public/rosary/index.html), which is mounted same-origin in an <iframe>. Both sides read/write this one localStorage key and react to the `storage` event, so no postMessage bridge is needed.
>
> IMPORTANT: this file's shape (key name, field names/values) is duplicated intentionally inside public/rosary/index.html, which is a static HTML file with its own inline JS and cannot import from here. If you change this contract, update both places.

Complete Source Code — rosarySettings.ts

```
// Shared rosary settings, kept in sync between the app's own "Rosary
// settings" surface (src/components/RosarySettingsModal.tsx) and the
// client's standalone rosary (public/rosary/index.html), which is mounted
// same-origin in an <iframe>. Both sides read/write this one localStorage
// key and react to the `storage` event, so no postMessage bridge is needed.
//
// IMPORTANT: this file's shape (key name, field names/values) is duplicated
// intentionally inside public/rosary/index.html, which is a static HTML
// file with its own inline JS and cannot import from here. If you change
// this contract, update both places.

export type RosaryLanguage = "en" | "fil";
export type RosaryMysterySet = "today" | "joyful" | "sorrowful" | "glorious" | "luminous";
export type RosaryMusic = "on" | "off";

export interface RosarySettings {
  language: RosaryLanguage;
  mysterySet: RosaryMysterySet;
```

```
  music: RosaryMusic;
}

export const ROSARY_SETTINGS_KEY = "sanctiwalk.rosary.settings";

export const DEFAULT_ROSARY_SETTINGS: RosarySettings = {
  language: "en",
  mysterySet: "today",
  music: "on",
};

const VALID_MYSTERY_SETS: RosaryMysterySet[] = ["today", "joyful", "sorrowful", "glorious", "luminous"];

function isValidRosarySettings(value: unknown): value is RosarySettings {
  if (!value || typeof value !== "object") return false;
  const v = value as Record<string, unknown>;
  return (v.language === "en" || v.language === "fil") &&
    typeof v.mysterySet === "string" &&
    VALID_MYSTERY_SETS.indexOf(v.mysterySet as RosaryMysterySet) !== -1 &&
    (v.music === "on" || v.music === "off");
}

/** Never throws. Falls back to the default on missing or corrupted data. */
export function loadRosarySettings(): RosarySettings {
  try {
    const raw = localStorage.getItem(ROSARY_SETTINGS_KEY);
    if (!raw) return { ...DEFAULT_ROSARY_SETTINGS };
    const parsed = JSON.parse(raw);
    return isValidRosarySettings(parsed) ? parsed : { ...DEFAULT_ROSARY_SETTINGS };
  } catch {
    return { ...DEFAULT_ROSARY_SETTINGS };
  }
}

export function saveRosarySettings(settings: RosarySettings): void {
  try {
    localStorage.setItem(ROSARY_SETTINGS_KEY, JSON.stringify(settings));
  } catch {
    // Storage may be unavailable (private mode, quota, etc). The setting
    // still applies for this session via React state.
  }
}

/** Parses a `storage` event's newValue for this key. Never throws. */
export function parseRosarySettingsEventValue(newValue: string | null): RosarySettings {
  if (!newValue) return { ...DEFAULT_ROSARY_SETTINGS };
  try {
    const parsed = JSON.parse(newValue);
    return isValidRosarySettings(parsed) ? parsed : { ...DEFAULT_ROSARY_SETTINGS };
  } catch {
    return { ...DEFAULT_ROSARY_SETTINGS };
  }
}
```

1.4.59 APP-129 — routing.ts

Code ID	APP-129
File Name	routing.ts
File Path	SanctDemo/src/lib/routing.ts
Language / Type	TypeScript
Lines	195
Functions / Classes	WALK_SPEED_MPS (exported constant); WalkingRoute (exported type); parseSteps (internal function); walkingMinutes (internal function); walkingMinutesFor (exported function); directRoute (internal function); fetchWalkingRoute (exported function); formatDistance (exported function); formatWalkingMinutes (exported function); DirectionsResult (exported type); getWalkingDirections (exported function)
Related Files (its imports)	./geo, ./navigation
Where It Is Used	src/components/DioceseMapLive.tsx, src/components/HomeHero.tsx, src/components/MapPlaceSheet.tsx, src/components/NavigationOverlay.tsx, src/components/ParishCard.tsx, src/components/ParishListRow.tsx, src/components/TripPlanner.tsx, src/lib/pilgrimage.ts
How to Recognize It	Open SanctDemo/src/lib/routing.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — routing.ts

```
import { haversineMeters, type Coordinates } from './geo'
import type { RouteStep } from './navigation'

// OSRM's public demo server — free, keyless, no billing account (the same
// requirement that ruled out Google/Mapbox for the base map itself). It
// carries no uptime guarantee and rate-limits under load, so every call
// here is time-boxed and backed by a straight-line fallback: routing must
// never leave the UI on a spinner or an error where a route should be.
const OSRM_BASE = 'https://router.project-osrm.org/route/v1/foot'
const DEFAULT_TIMEOUT_MS = 5000

// A brisk, sustainable adult walking pace on a paved urban route.
//
// OSRM's own `duration` field is NEVER used, for either a routed or a
// direct result. Verified against the two live parishes: OSRM's demo
// server returned duration: 240s (4 min) for a 1850m foot route — that's
// ~28 km/h, the server's default vehicle-profile speed, not a walking
// pace corrected for the "foot" profile it was asked for. Only
// `distance` from OSRM is trustworthy; duration is always derived from
// distance at WALK_SPEED_MPS instead.
export const WALK_SPEED_MPS = 5000 / 3600 // 5 km/h

export interface WalkingRoute {
  // 'routed': geometry follows OSRM's street path. 'direct': OSRM was
  // unreachable, timed out, or returned no usable route — geometry is a
```

```
  // straight line between the two points and callers must label the
  // distance as a direct/straight-line distance, never as "walking route".
  kind: 'routed' | 'direct'
  path: Coordinates[]
  distanceMeters: number
  durationMinutes: number
  /**
   * Turn-by-turn manoeuvres, when they were asked for and OSRM supplied them.
   * Always empty for a 'direct' route: a straight line across the map has no
   * turns, and inventing "head north, then arrive" for one would be worse
   * than saying nothing.
   */
  steps: RouteStep[]
}

/**
 * Narrows OSRM's steps to the fields this app uses.
 *
 * Defensive about shape rather than trusting the response: this is a public
 * demo server, the legs/steps nesting is easy to get wrong, and a malformed
 * step would otherwise crash navigation mid-walk. Anything unparseable is
 * dropped, and an empty list simply means no turn-by-turn — which the UI
 * already has to handle for direct routes.
 */
function parseSteps(route: any): RouteStep[] {
  const legs = Array.isArray(route?.legs) ? route.legs : []
  const steps: RouteStep[] = []

  for (const leg of legs) {
    for (const raw of Array.isArray(leg?.steps) ? leg.steps : []) {
      const location = raw?.maneuver?.location
      if (!Array.isArray(location) || location.length < 2) continue
      if (typeof raw?.maneuver?.type !== 'string') continue

      steps.push({
        name: typeof raw.name === 'string' ? raw.name : '',
        distanceMeters: typeof raw.distance === 'number' ? raw.distance : 0,
        maneuver: {
          type: raw.maneuver.type,
          modifier: typeof raw.maneuver.modifier === 'string' ? raw.maneuver.modifier : undefined,
          location: [location[0], location[1]],
          bearingAfter:
            typeof raw.maneuver.bearing_after === 'number' ? raw.maneuver.bearing_after : undefined,
        },
      })
    }
  }

  return steps
}

function walkingMinutes(distanceMeters: number): number {
  return distanceMeters / WALK_SPEED_MPS / 60
}

/**
 * The same conversion, for callers that have a distance from somewhere else
 * - a multi-leg trip summing its legs, say. Exported so nothing outside this
 * file has to know WALK_SPEED_MPS or repeat the arithmetic, which is how two
```

```
 * screens end up quoting different times for the same walk.
 */
export function walkingMinutesFor(distanceMeters: number): number {
  return walkingMinutes(distanceMeters)
}

function directRoute(from: Coordinates, to: Coordinates): WalkingRoute {
  const distanceMeters = haversineMeters(from, to)
  return {
    kind: 'direct',
    path: [from, to],
    distanceMeters,
    durationMinutes: walkingMinutes(distanceMeters),
    steps: [],
  }
}

interface FetchWalkingRouteOptions {
  fetchImpl?: typeof fetch
  timeoutMs?: number
  /**
   * Ask OSRM for turn-by-turn steps. Off by default: the map's own route
   * line and the distance readouts do not need them, and the response is
   * substantially larger with them.
   */
  withSteps?: boolean
}

export async function fetchWalkingRoute(
  from: Coordinates,
  to: Coordinates,
  options: FetchWalkingRouteOptions = {},
): Promise<WalkingRoute> {
  const { fetchImpl = fetch, timeoutMs = DEFAULT_TIMEOUT_MS, withSteps = false } = options

  const controller = new AbortController()
  const timer = setTimeout(() => controller.abort(), timeoutMs)

  try {
    const url =
      `${OSRM_BASE}/${from.lng},${from.lat};${to.lng},${to.lat}` +
      `?overview=full&geometries=geojson${withSteps ? '&steps=true' : ''}`
    const res = await fetchImpl(url, { signal: controller.signal })
    if (!res.ok) return directRoute(from, to)

    const data = await res.json()
    const coords: [number, number][] | undefined = data?.routes?.[0]?.geometry?.coordinates
    if (data?.code !== 'Ok' || !coords || coords.length < 2) {
      return directRoute(from, to)
    }

    const path: Coordinates[] = coords.map(([lng, lat]) => ({ lat, lng }))
    const distanceMeters: number = data.routes[0].distance
    return {
      kind: 'routed',
      path,
      distanceMeters,
      durationMinutes: walkingMinutes(distanceMeters),
      steps: withSteps ? parseSteps(data.routes[0]) : [],
```

```
    }
  } catch {
    // Network failure, non-JSON body, or the abort() above firing on
    // timeout — all collapse to the same safe outcome.
    return directRoute(from, to)
  } finally {
    clearTimeout(timer)
  }
}

// Metres under 1km, one decimal km above — matches how the app already
// reads distances elsewhere (geofence radii, MapTab station distances).
export function formatDistance(meters: number): string {
  if (meters < 1000) return `${Math.round(meters)} m`
  return `${(meters / 1000).toFixed(1)} km`
}

export function formatWalkingMinutes(minutes: number): string {
  if (minutes < 1) return '<1 min walk'
  return `${Math.round(minutes)} min walk`
}

// Pure result + label logic behind the map popup's "Get directions" action,
// extracted so it's unit-testable without MapLibre/DOM (mirrors mapSearch.ts
// and mapMarkers.ts, extracted for the same reason). The caller (
// DioceseMapLive.tsx) is responsible for everything this can't express as a
// pure function: reading the pilgrim's *current* position at click time,
// superseding a stale in-flight request with a newer one, drawing/updating
// the map layer, and fitting the camera to the route.
export type DirectionsResult =
  // No known position yet — the caller must say why it can't route rather
  // than failing silently (no position, no button spinner that never ends).
  | { status: 'no-position' }
  | { status: 'ok'; route: WalkingRoute; label: string }

export async function getWalkingDirections(
  from: Coordinates | null,
  to: Coordinates,
  options: FetchWalkingRouteOptions = {},
): Promise<DirectionsResult> {
  if (!from) return { status: 'no-position' }

  const route = await fetchWalkingRoute(from, to, options)
  const label =
    route.kind === 'routed'
      ? `${formatDistance(route.distanceMeters)} walk · ${formatWalkingMinutes(route.durationMinutes)}`
      // OSRM was unreachable/timed out/returned nothing usable — this is a
      // straight-line distance, never presented as a walking route.
      : `${formatDistance(route.distanceMeters)} direct (straight-line — walking route unavailable)`

  return { status: 'ok', route, label }
}
```

1.4.60 APP-130 — sancti.ts

Code ID	APP-130
File Name	sancti.ts
File Path	SanctDemo/src/lib/sancti.ts
Language / Type	TypeScript
Lines	749
Functions / Classes	SanctiAction (exported type); Understanding (exported type); CONFIDENCE_FLOOR (exported constant); normalise (exported function); isImperative (exported function); hasPhrase (internal function); readAnswer (exported function); NAVIGATING_ACTIONS (exported constant); needsConfirmation (exported function); findLanguage (exported function); NamedItem (exported type); spellingVariants (internal function); itemAliases (exported function); withinEdits (internal function); wordPresent (internal function); findNamedItem (exported function); ParishName (exported type); aliasesFor (exported function); findParish (exported function); findTime (exported function); PriorTurn (exported type); isFollowUp (exported function); understand (exported function); SUGGESTIONS (exported constant)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/SanctiSheet.tsx
How to Recognize It	Open SanctDemo/src/lib/sancti.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Working out what a pilgrim is asking Sancti for.
>
> ## Why this is not a language model
>
> It was going to be Gemini, through the Express backend. That backend is not deployed, which is exactly why the AI scanner does not work on the phone today — and an assistant whose first act is to fail at a dead URL is worse than no assistant. Everything here runs on the device: it answers with no signal, costs nothing, and cannot be taken down by a server being asleep.
>
> The trade is honest and worth stating: this understands the things SanctiWalk can actually DO, in many phrasings, and says so plainly when a question is outside that. It does not hold a conversation. For a guide whose job is "take me to the map" and "what time is Mass", that is the right shape.
>
> ## Why the triggers are bilingual and the setting is not consulted
>
> The language setting decides what Sancti SAYS. It has no say in what it understands. Somebody reading the app in English still types "anong oras ang misa", and a guide that refused that would be telling a pilgrim their own language is the wrong one. Every intent below carries its English, its Tagalog and the Taglish in between, and all of them are live whatever the setting says.
>
> ## Why scoring, not a chain of `if (text.includes(...))`

"Where is the map?" and "take me to Mary Help" both contain "ma". A first-match chain answers whichever rule happened to be written first, which is how these things end up confidently wrong. Every intent scores itself against the sentence and the best score wins, with a floor below which Sancti admits it did not understand rather than guessing.

Complete Source Code — sancti.ts

```
/**
 * Working out what a pilgrim is asking Sancti for.
 *
 * ## Why this is not a language model
 *
 * It was going to be Gemini, through the Express backend. That backend
 * is not deployed, which is exactly why the AI scanner does not work on
 * the phone today — and an assistant whose first act is to fail at a
 * dead URL is worse than no assistant. Everything here runs on the
 * device: it answers with no signal, costs nothing, and cannot be taken
 * down by a server being asleep.
 *
 * The trade is honest and worth stating: this understands the things
 * SanctiWalk can actually DO, in many phrasings, and says so plainly
 * when a question is outside that. It does not hold a conversation.
 * For a guide whose job is "take me to the map" and "what time is
 * Mass", that is the right shape.
 *
 * ## Why the triggers are bilingual and the setting is not consulted
 *
 * The language setting decides what Sancti SAYS. It has no say in what
 * it understands. Somebody reading the app in English still types
 * "anong oras ang misa", and a guide that refused that would be telling
 * a pilgrim their own language is the wrong one. Every intent below
 * carries its English, its Tagalog and the Taglish in between, and all
 * of them are live whatever the setting says.
 *
 * ## Why scoring, not a chain of `if (text.includes(...))`
 *
 * "Where is the map?" and "take me to Mary Help" both contain "ma".
 * A first-match chain answers whichever rule happened to be written
 * first, which is how these things end up confidently wrong. Every
 * intent scores itself against the sentence and the best score wins,
 * with a floor below which Sancti admits it did not understand rather
 * than guessing.
 */

export type SanctiAction =
  | "OPEN_MAP"
  | "OPEN_CHURCH"
  | "OPEN_AR"
  | "OPEN_MASS_SCHEDULE"
  | "OPEN_SACRAMENTS"
  | "OPEN_BAPTISM"
  | "OPEN_WEDDING"
  | "OPEN_MINISTRIES"
  | "OPEN_CHURCH_HISTORY"
  | "OPEN_SETTINGS"
  | "CREATE_REMINDER"
  | "REPLAY_TUTORIAL"
```

```
  | "CHANGE_LANGUAGE"
  | "SHOW_CHURCH_LOCATION"
  | "ANSWER_DISTANCE"
  | "ANSWER_CONTACT"
  | "HELP"
  | "UNKNOWN";

export interface Understanding {
  action: SanctiAction;
  /**
   * Whether the pilgrim told Sancti to do it, or asked about it.
   *
   * "Open the map" is an instruction. "Where is the map?" is a
   * question, and answering a question by throwing the pilgrim onto
   * another screen is how an assistant becomes something people stop
   * tapping. Same action, different consent.
   */
  imperative: boolean;
  /** The parish the sentence named, if it named one. */
  parishId?: string;
  /**
   * The individual ministry or sacrament the sentence named.
   *
   * "Open the ministries" is a list. "Open Teatro Pilipino" is one
   * ministry, and taking the pilgrim to a list of fifteen with theirs
   * somewhere in it is not what they asked for.
   */
  itemId?: string;
  /** For CREATE_REMINDER: the Mass time asked about, as written. */
  time?: string;
  /**
   * For CHANGE_LANGUAGE: the language the sentence named, if it named
   * one. Absent means "change the language" with no target, which is a
   * question rather than an instruction - switching the whole app to a
   * language nobody asked for is not a reasonable guess.
   */
  language?: "en" | "fil";
  /** How sure we are. Below CONFIDENCE_FLOOR the action is UNKNOWN. */
  score: number;
}

/** Below this, Sancti says it did not understand instead of guessing. */
export const CONFIDENCE_FLOOR = 2;

/**
 * Normalises a sentence for matching.
 *
 * Punctuation goes, case goes, and runs of whitespace collapse. Tagalog
 * and English are both left alone otherwise - pilgrims here type both,
 * often in the same sentence, and stripping either would make half the
 * questions unmatchable.
 */
export function normalise(text: string): string {
  return text
    .toLowerCase()
    .replace(/[^\p{L}\p{N}\s:]/gu, " ")
    .replace(/\s+/g, " ")
    .trim();
}
```

```
/**
 * The phrases that point at each action.
 *
 * Longer phrases score higher than single words on purpose: "mass
 * schedule" should beat the bare "mass" that also appears in "where is
 * the mass" — and "take me to" should outrank the "map" that
 * SHOW_CHURCH_LOCATION and OPEN_MAP both answer to.
 */
const TRIGGERS: Array<{ action: SanctiAction; phrases: string[] }> = [
  {
    action: "OPEN_MASS_SCHEDULE",
    phrases: [
      "mass schedule", "schedule of mass", "what time is mass", "time of mass",
      "misa schedule", "oras ng misa", "anong oras ang misa", "next mass",
      "mass time", "mass times", "when is mass", "sunday mass",
      // Taglish, which is how people here actually type.
      "anong oras ang mass", "oras ng mass", "kailan ang misa", "kailan ang mass",
      "schedule ng misa", "schedule ng mass", "may misa ba", "anong oras misa",
    ],
  },
  {
    action: "CREATE_REMINDER",
    phrases: [
      "remind me", "set a reminder", "reminder", "alarm", "notify me",
      "paalala", "alalahanin", "paalalahanan mo ako", "remind mo ako",
      "paalalahanan", "i remind mo",
    ],
  },
  {
    action: "OPEN_BAPTISM",
    phrases: ["baptism", "binyag", "christening", "baptismal",
      "pabinyag", "magpabinyag", "paano magpabinyag"],
  },
  {
    action: "OPEN_WEDDING",
    phrases: ["wedding", "matrimony", "kasal", "marriage", "get married",
      "ikakasal", "magpakasal", "paano magpakasal"],
  },
  {
    action: "OPEN_SACRAMENTS",
    phrases: [
      "sacrament", "sacraments", "sakramento", "confirmation", "kumpil",
      "confession", "communion",
    ],
  },
  {
    action: "OPEN_MINISTRIES",
    phrases: [
      "ministry", "ministries", "ministeryo", "volunteer", "join a group",
      "altar server", "choir", "lector", "serve the parish",
      "sumali", "paano sumali", "maglingkod", "gusto kong sumali",
    ],
  },
  {
    action: "OPEN_CHURCH_HISTORY",
    phrases: [
      "history", "kasaysayan", "story of", "tell me about", "background",
      "when was it built", "who built",
```

```
    "kwento ng", "kuwento ng", "ano ang kasaysayan", "sino ang nagtayo",
  ],
},
{
  action: "OPEN_AR",
  phrases: [
    "ar", "augmented reality", "scanner", "scan", "camera", "how do i use ar",
    "ar tour", "point my camera", "paano gamitin ang scanner", "paano mag scan",
  ],
},
{
  action: "ANSWER_DISTANCE",
  phrases: [
    "how far", "distance", "gaano kalayo", "how long to get", "how many minutes",
    "how long does it take", "ilang minuto", "malayo ba", "gaano katagal",
  ],
},
{
  action: "SHOW_CHURCH_LOCATION",
  phrases: [
    "take me to", "directions", "navigate", "how do i get to", "bring me to",
    "paano pumunta", "show me on the map", "where is the church",
    "dalhin mo ako", "punta tayo", "saan ang simbahan", "paano papunta",
  ],
},
{
  action: "OPEN_MAP",
  phrases: ["map", "mapa", "where is the map", "open the map", "show the map",
    // "show me the map" used to lose to OPEN_CHURCH's generic "show
    // me", which is two words against one and outscored the "map"
    // that says what is actually wanted.
    "show me the map", "take me to the map",
    "nasaan ang map", "nasaan ang mapa", "buksan ang mapa"],
},
{
  action: "ANSWER_CONTACT",
  phrases: [
    "contact", "phone number", "email", "parish office", "call the parish",
    "office hours",
  ],
},
{
  action: "REPLAY_TUTORIAL",
  phrases: [
    "show me around", "show me around again", "tour of the app", "app tour",
    "tutorial", "walkthrough", "walk me through", "guide me through",
    "how do i use the app", "how to use the app", "show me how",
    "ipakita mo sa akin ang app", "paano gamitin ang app", "turuan mo ako",
    "gabayan mo ako", "paano ito gamitin",
  ],
},
{
  action: "CHANGE_LANGUAGE",
  phrases: [
    "change the language", "change language", "switch language",
    "language setting", "language settings", "where is the language",
    "set the language", "change to english", "change to tagalog",
    "change to filipino", "in english", "in tagalog", "language",
    "palitan ang wika", "palitan ang language", "wika", "gawing tagalog",
```

```
      "gawing english", "saan ang wika",
      "i want english", "i want tagalog", "i want filipino",
      "english please", "tagalog please", "use english", "use tagalog",
      "read in english", "read in tagalog",
    ],
  },
  {
    action: "OPEN_SETTINGS",
    phrases: ["settings", "preferences", "my account", "notifications settings"],
  },
  {
    action: "OPEN_CHURCH",
    phrases: ["show me", "open", "go to", "information about", "tell me about"],
  },
  {
    action: "HELP",
    phrases: [
      "help", "what can you do", "who are you", "tulong", "what can i ask",
      // "pwede ba" is deliberately NOT here. It opens half the questions
      // people ask - "pwede ba mag apply for baptism" - so as a help
      // trigger it outscored the real intent and answered the wrong
      // question confidently. A test caught it.
      "how does this work", "ano ang kaya mo", "sino ka", "ano ang magagawa mo",
    ],
  },
];

/**
 * Phrases that make a sentence an instruction rather than a question.
 *
 * Kept apart from the intent triggers because they are orthogonal: any
 * intent can arrive either way, and the pilgrim who types "open the mass
 * schedule" wants exactly what the pilgrim who types "what time is mass"
 * does NOT want - the screen, now, without being asked.
 *
 * "Show me on the map" counts. It names the destination and the verb;
 * there is nothing left to confirm.
 */
const IMPERATIVES = [
  "open", "show me", "take me", "bring me", "go to", "navigate", "directions",
  "launch", "start", "let me see", "i want to see", "view",
  "buksan", "ipakita", "dalhin mo", "punta tayo", "pumunta", "tara",
  "gusto kong makita", "pakita", "pakibuksan", "sige",
];

/** Yes, in the languages and spellings people here actually type. */
const AFFIRMATIVES = [
  "yes", "yeah", "yep", "yup", "sure", "ok", "okay", "okey", "go", "go ahead",
  "please", "please do", "do it", "open it", "sige", "oo", "opo", "oo nga",
  "sige na", "payag", "game", "tara",
];

/** No. Short, because a refusal usually is. */
const NEGATIVES = [
  "no", "nope", "not now", "later", "nah", "cancel", "never mind", "nevermind",
  "hindi", "ayaw", "ayoko", "wag", "huwag", "mamaya", "hindi muna",
];

/**
```

```
 * Whether the sentence is telling Sancti to do something.
 *
 * Matched on word boundaries, not substrings: "open" inside "opening
 * hours" is not an instruction, and treating it as one would make
 * "what are the opening hours" fling the pilgrim at a screen.
 */
export function isImperative(text: string): boolean {
  const hay = normalise(text);
  return IMPERATIVES.some(phrase => hasPhrase(hay, phrase));
}

/** True when `phrase` appears in `hay` as whole words. */
function hasPhrase(hay: string, phrase: string): boolean {
  const at = hay.indexOf(phrase);
  if (at === -1) return false;
  const before = at === 0 ? " " : hay[at - 1];
  const after = at + phrase.length >= hay.length ? " " : hay[at + phrase.length];
  return before === " " && after === " ";
}

/**
 * A bare yes or no, when Sancti has just asked something.
 *
 * Only consulted while an offer is open. Out of that context "no" is
 * part of a sentence rather than an answer, and reading it as consent -
 * or as a refusal - would be guessing.
 *
 * Returns null for anything that is not plainly one or the other, which
 * is how "yes but what time" stays a question rather than becoming a
 * yes to something else.
 */
export function readAnswer(text: string): "yes" | "no" | null {
  const hay = normalise(text).trim();
  if (!hay) return null;
  // Whole-sentence match. A yes buried in a longer sentence is that
  // sentence's business, not an answer to the offer.
  if (AFFIRMATIVES.includes(hay)) return "yes";
  if (NEGATIVES.includes(hay)) return "no";
  return null;
}

/**
 * The actions that take the pilgrim off this screen.
 *
 * These are the ones that need a yes when the sentence was a question.
 * Everything else either answers in place or changes nothing the
 * pilgrim can see, and stopping to ask about those would be a second
 * tap for nothing.
 */
export const NAVIGATING_ACTIONS: readonly SanctiAction[] = [
  "OPEN_MAP", "OPEN_CHURCH", "OPEN_AR", "OPEN_MASS_SCHEDULE", "OPEN_SACRAMENTS",
  "OPEN_BAPTISM", "OPEN_WEDDING", "OPEN_MINISTRIES", "OPEN_CHURCH_HISTORY",
  "OPEN_SETTINGS", "SHOW_CHURCH_LOCATION",
  // REPLAY_TUTORIAL and CHANGE_LANGUAGE are deliberately absent. Asking
  // to be shown around IS the instruction, and a tutorial that stopped
  // to ask whether you wanted the tutorial would be absurd. A language
  // change is one tap to undo and visible the instant it happens.
];
```

```
/**
 * Whether Sancti should offer rather than act.
 *
 * An instruction is consent already given. A question is not.
 */
export function needsConfirmation(heard: Understanding): boolean {
  if (heard.imperative) return false;
  return NAVIGATING_ACTIONS.includes(heard.action);
}

/**
 * One ministry or sacrament, by every name somebody might type for it.
 *
 * ## Why aliases rather than the name
 *
 * The compiled names are the parish's formal ones - "Ministry of Altar
 * Servers (MAS)", "Extraordinary Ministers of Holy Communion (EMHC)" -
 * and nobody types those. People type "altar servers", or "emhc", or
 * the choir's name with the ministry word left off.
 */
/**
 * The language a sentence asked for, if it asked for one.
 *
 * Only consulted for CHANGE_LANGUAGE. Elsewhere the word "Tagalog" is
 * part of a question, not a setting - and a pilgrim asking "may misa
 * ba sa Tagalog" is asking about a Mass, not about the interface.
 */
export function findLanguage(text: string): "en" | "fil" | undefined {
  const hay = normalise(text);
  if (/\b(tagalog|filipino|pilipino|wika)\b/.test(hay)) return "fil";
  if (/\b(english|ingles|inglis)\b/.test(hay)) return "en";
  return undefined;
}

export interface NamedItem {
  id: string;
  kind: "ministry" | "sacrament";
  /** Already normalised. Longest match wins. */
  aliases: string[];
}

/**
 * Words that carry no identity.
 *
 * Dropping them is what lets "altar servers" find "Ministry of Altar
 * Servers": the alias becomes the distinctive part, and a sentence
 * containing those words alone matches it.
 */
const GENERIC_WORDS = new Set([
  "ministry", "ministries", "ministers", "ministeryo", "of", "the", "and",
  "on", "for", "to", "a", "ng", "sa", "at", "mga",
]);

/**
 * The shortest acronym worth matching.
 *
 * Three letters collide with real words - "MAS" is Tagalog for "more",
 * and as an alias it hijacked "mas maaga ba ang misa". Four is long
 * enough that a collision is somebody actually naming the ministry.
```

```
 */
const MIN_ACRONYM = 4;

/** Filipino and Pilipino are the same word to everyone who types it. */
function spellingVariants(text: string): string[] {
  const swapped = text.replace(/\bfilipino\b/g, "pilipino");
  const back = text.replace(/\bpilipino\b/g, "filipino");
  return [...new Set([text, swapped, back])];
}

/** Every way somebody might name this ministry or sacrament. */
export function itemAliases(
  id: string,
  displayName: string,
  kind: "ministry" | "sacrament",
): NamedItem {
  const full = normalise(displayName);
  const aliases = new Set<string>();

  // The parenthesised acronym, when it is long enough to be safe.
  const acronym = /\(([^)]+)\)/.exec(displayName)?.[1];
  if (acronym && acronym.length >= MIN_ACRONYM) aliases.add(normalise(acronym));

  const withoutAcronym = normalise(displayName.replace(/\([^)]*\)/g, ""));
  const distinctive = withoutAcronym
    .split(" ")
    .filter(word => word && !GENERIC_WORDS.has(word))
    .join(" ");

  for (const base of [full, withoutAcronym, distinctive]) {
    for (const variant of spellingVariants(base)) {
      if (variant.trim()) aliases.add(variant.trim());
    }
  }

  return { id, kind, aliases: [...aliases] };
}

/** Levenshtein distance, capped: anything past `max` is simply "too far". */
function withinEdits(a: string, b: string, max: number): boolean {
  if (a === b) return true;
  if (Math.abs(a.length - b.length) > max) return false;

  let previous = Array.from({ length: b.length + 1 }, (_, i) => i);
  for (let i = 1; i <= a.length; i++) {
    const current = [i];
    for (let j = 1; j <= b.length; j++) {
      current[j] = Math.min(
        previous[j] + 1,
        current[j - 1] + 1,
        previous[j - 1] + (a[i - 1] === b[j - 1] ? 0 : 1),
      );
    }
    previous = current;
  }
  return previous[b.length] <= max;
}

/**
```

```
 * Whether a word of the alias appears in the sentence.
 *
 * A single mistyped letter is forgiven on words long enough for that to
 * be unambiguous - "teatri" for "teatro". Short words are matched
 * exactly, because at four letters an edit of one turns half the
 * dictionary into half the rest of it.
 */
const MIN_FUZZY = 5;

function wordPresent(word: string, haystack: string[]): boolean {
  if (haystack.includes(word)) return true;
  if (word.length < MIN_FUZZY) return false;
  return haystack.some(token =>
    token.length >= MIN_FUZZY && withinEdits(word, token, 1));
}

/**
 * The ministry or sacrament the sentence names, if it names one.
 *
 * Two of the alias's words are enough, and the alias that matches the
 * most words wins.
 *
 * Not all of them, because nobody says the whole name: "Teatro Pilipino
 * Choir" is asked for as "teatro pilipino", and requiring every word
 * meant the one ministry anybody names by name was the one Sancti could
 * not find. Two, because a single shared word - "choir", which six of
 * these have, or "ministry", which is stripped anyway - identifies
 * nothing. A one-word alias is an acronym, and those match alone.
 */
const MIN_ALIAS_WORDS = 2;

export function findNamedItem(
  text: string,
  items: NamedItem[],
): NamedItem | undefined {
  const tokens = normalise(text).split(" ").filter(Boolean);
  if (tokens.length === 0) return undefined;

  let best: { item: NamedItem; matched: number } | null = null;

  for (const item of items) {
    for (const alias of item.aliases) {
      const words = alias.split(" ").filter(Boolean);
      if (words.length === 0) continue;

      const matched = words.filter(word => wordPresent(word, tokens)).length;
      if (matched < Math.min(MIN_ALIAS_WORDS, words.length)) continue;
      if (!best || matched > best.matched) best = { item, matched };
    }
  }

  return best?.item;
}

export interface ParishName {
  id: string;
  /** Every way someone might type it, already normalised. */
  aliases: string[];
}
```

```
/**
 * Builds the alias list for a parish from its display name.
 *
 * The compiled names carry suffixes nobody says out loud - "Mary Help
 * of Christians Parish Guide" - so the suffixes are stripped, and the
 * distinctive words are kept as a shorter alias. Nobody types the whole
 * thing into a chat box.
 */
export function aliasesFor(id: string, displayName: string): ParishName {
  const clean = normalise(
    displayName.replace(/\b(guide|tour)\b/gi, ""),
  );
  const withoutParish = clean.replace(/\b(parish|cathedral)\b/g, "").replace(/\s+/g, " ").trim();

  const aliases = new Set<string>([clean, withoutParish]);

  // The initials people actually use for these two.
  if (/mary help/.test(clean)) { aliases.add("mary help"); aliases.add("mhcp"); }
  if (/san roque/.test(clean)) { aliases.add("san roque"); aliases.add("src"); }

  return { id, aliases: [...aliases].filter(Boolean) };
}

/** Which parish the sentence named, if any. Longest alias wins. */
export function findParish(text: string, parishes: ParishName[]): string | undefined {
  const hay = normalise(text);
  let best: { id: string; length: number } | null = null;

  for (const parish of parishes) {
    for (const alias of parish.aliases) {
      if (alias.length >= 3 && hay.includes(alias)) {
        if (!best || alias.length > best.length) best = { id: parish.id, length: alias.length };
      }
    }
  }
  return best?.id;
}

/**
 * A clock time written in any of the ways people write one.
 *
 * Returned as it would be typed on a schedule - "6:00 PM" - because
 * that is what it has to be matched against.
 */
export function findTime(text: string): string | undefined {
  const m = normalise(text).match(/\b(\d{1,2})(?::(\d{2}))?\s*(am|pm|nn|hapon|umaga)?\b/);
  if (!m) return undefined;

  const hour = Number(m[1]);
  if (hour < 1 || hour > 24) return undefined;
  const minutes = m[2] ?? "00";

  let suffix = m[3];
  if (suffix === "hapon" || suffix === "nn") suffix = "pm";
  if (suffix === "umaga") suffix = "am";

  if (!suffix) {
    // 24-hour, or a bare number. Anything from 13 up is unambiguous;
```

```
    // below that it is a guess, and the caller checks the real schedule
    // before acting on it anyway.
    if (hour > 12) {
      const h12 = hour - 12;
      return `${h12}:${minutes} PM`;
    }
    return undefined;
  }

  return `${hour}:${minutes} ${suffix.toUpperCase()}`;
}

/**
 * What the sentence is asking for.
 *
 * Scores every intent and takes the best. A phrase scores its own word
 * count, so a three-word match beats a one-word match that happens to
 * sit inside it.
 */
/**
 * What the last exchange was about.
 *
 * Sancti had no memory at all: every sentence was understood from
 * scratch, so "what about tomorrow?" scored nothing and came back as
 * "I did not understand". The fragment is not a failure of phrasing,
 * it is how people speak once a subject is already on the table.
 */
export interface PriorTurn {
  action: SanctiAction;
  parishId?: string;
  itemId?: string;
}

/**
 * Sentences that continue the last question instead of starting one.
 *
 * Kept deliberately tight. A follow-up inherits the previous intent
 * wholesale, so a loose matcher here would answer an unrelated question
 * with the previous topic - a worse failure than admitting it did not
 * understand. Bilingual, like every other trigger in this file.
 */
const FOLLOW_UP = [
  "what about", "how about", "and tomorrow", "what time tomorrow",
  "tomorrow", "bukas", "kahapon", "yesterday", "next week", "sa susunod",
  "paano papunta", "how do i get there", "how to get there", "get there",
  "doon", "dun", "that one", "it", "there",
];

/**
 * Whether this sentence is a continuation rather than a new question.
 *
 * Requires a prior turn AND a short sentence. "There" inside a long
 * sentence is ordinary English; "there?" on its own is a follow-up.
 */
export function isFollowUp(text: string, prior?: PriorTurn): boolean {
  if (!prior || prior.action === "UNKNOWN") return false;
  const hay = normalise(text);
  if (!hay) return false;
  // Six words is a fragment. Beyond that the sentence is carrying
```

```
  // enough of its own meaning to be understood on its own terms.
  if (hay.split(" ").length > 6) return false;
  return FOLLOW_UP.some(f => hay.includes(f));
}

export function understand(
  text: string,
  parishes: ParishName[],
  items: NamedItem[] = [],
  /**
   * The previous exchange, when there was one. Only ever used to fill
   * a blank - a sentence that names its own parish or intent is always
   * taken at its word.
   */
  prior?: PriorTurn,
): Understanding {
  const hay = normalise(text);
  if (!hay) return { action: "UNKNOWN", score: 0, imperative: false };

  const imperative = isImperative(hay);

  let best: { action: SanctiAction; score: number } = { action: "UNKNOWN", score: 0 };

  for (const { action, phrases } of TRIGGERS) {
    let score = 0;
    for (const phrase of phrases) {
      if (!hay.includes(phrase)) continue;
      // Word count, so a longer phrase outranks a shorter one inside it.
      score = Math.max(score, phrase.split(" ").length + 1);
    }
    if (score > best.score) best = { action, score };
  }

  // The sentence's own parish always wins; the previous one fills a
  // blank. "What time is Mass?" after asking about San Roque means San
  // Roque, which is the whole of requirement 4.
  const namedHere = findParish(hay, parishes);
  const parishId = namedHere ?? prior?.parishId;

  // A named ministry or sacrament outranks everything.
  //
  // "Open Teatro Pilipino" matched OPEN_MINISTRIES on the word
  // "ministry" and nothing else, so it opened a list of fifteen with
  // the pilgrim's somewhere in it. Naming one is a request for that
  // one, and it is a stronger signal than any generic trigger - which
  // is why it is checked before the score floor rather than after.
  const named = findNamedItem(hay, items);
  if (named) {
    const result: Understanding = {
      action: named.kind === "ministry" ? "OPEN_MINISTRIES" : "OPEN_SACRAMENTS",
      itemId: named.id,
      score: CONFIDENCE_FLOOR + 1,
      imperative,
    };
    if (parishId) result.parishId = parishId;
    return result;
  }

  // A fragment continuing the last question. Checked only after every
```

```
  // real trigger has had its chance, so "what about the map?" is still
  // understood as the map rather than as more of the previous topic.
  if (best.score < CONFIDENCE_FLOOR && isFollowUp(text, prior)) {
    const carried: Understanding = {
      action: prior!.action,
      score: CONFIDENCE_FLOOR,
      imperative,
    };
    if (parishId) carried.parishId = parishId;
    if (prior!.itemId) carried.itemId = prior!.itemId;
    if (prior!.action === "CREATE_REMINDER") {
      const time = findTime(text);
      if (time) carried.time = time;
    }
    return carried;
  }

  // Naming a parish and nothing else is a request to open it. Only a
  // parish named in THIS sentence: an inherited one means the pilgrim
  // is still asking about it, not asking to open it again.
  if (best.score < CONFIDENCE_FLOOR && namedHere) {
    return { action: "OPEN_CHURCH", parishId: namedHere, score: CONFIDENCE_FLOOR, imperative };
  }

  if (best.score < CONFIDENCE_FLOOR) return { action: "UNKNOWN", score: best.score, imperative };

  // "Tell me about Mary Help" reads as history; "open Mary Help" does
  // not. Both match OPEN_CHURCH's generic openers, so the more specific
  // intent is preferred whenever one also matched.
  const result: Understanding = { action: best.action, score: best.score, imperative };
  if (parishId) result.parishId = parishId;
  if (best.action === "CREATE_REMINDER") {
    const time = findTime(text);
    if (time) result.time = time;
  }
  if (best.action === "CHANGE_LANGUAGE") {
    const language = findLanguage(text);
    if (language) result.language = language;
  }
  return result;
}

/** What the chat offers before anyone has typed anything. */
export const SUGGESTIONS = [
  "What time is Mass?",
  "Where is the map?",
  "Is baptism available?",
  "How do I use the scanner?",
  "Tell me the history",
] as const;
```

1.4.61 APP-131 — sanctiAnswers.ts

Code ID	APP-131
File Name	sanctiAnswers.ts
File Path	SanctDemo/src/lib/sanctiAnswers.ts
Language / Type	TypeScript
Lines	370
Functions / Classes	NO_INFORMATION (exported constant); Offer (exported type); openingLine (exported function); offerForItem (exported function); offerFor (exported function); withOffer (exported function); declinedAnswer (exported function); Reply (exported type); massAnswer (exported function); historyAnswer (exported function); itemAnswer (exported function); distanceAnswer (exported function); contactAnswer (exported function); findMassTime (exported function); reminderAnswer (exported function); unknownAnswer (exported function); helpAnswer (exported function)
Related Files (its imports)	./schedule, ./itemContent
Where It Is Used	src/components/SanctiSheet.tsx
How to Recognize It	Open SanctDemo/src/lib/sanctiAnswers.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — sanctiAnswers.ts

```
import { availableTimes, type MassScheduleEntry } from "./schedule";
import type { ResolvedItem } from "./itemContent";

/**
 * Turning what Sancti understood into something it may actually say.
 *
 * ## The one rule
 *
 * Sancti repeats what the parish wrote and nothing else. Every answer
 * below is assembled from data passed in; where the data is absent the
 * reply says so, in the words the brief asked for, rather than filling
 * the gap. A guide that invents a Mass time sends somebody to a locked
 * church, and a guide that invents a baptismal requirement sends them
 * away from the office without the papers.
 *
 * That is why none of these functions reach for a default. The absence
 * of a schedule is itself the answer.
 */

/** The sentence the brief specifies for anything SanctiWalk does not hold. */
export const NO_INFORMATION =
  "I don't have that information in SanctiWalk yet. Please check with the parish office for the most accurate information.";

/**
 * What Sancti offers to open, once it has already answered.
```

```
 *
 * ## Why the answer comes first and the offer second
 *
 * "Where is the map?" used to open the map. The pilgrim got an answer
 * they did not ask for, delivered as a screen they did not choose, and
 * lost the conversation to get it. Asking "what time is Mass" and being
 * thrown at a schedule is the same discourtesy: the times were the
 * answer, and the screen is at most a follow-up.
 *
 * So the reply carries the answer in full and the offer underneath, as
 * one tap. Saying nothing more is a perfectly good outcome.
 *
 * `followUp` is phrased as an INSTRUCTION ("Open the map"), not as
 * "yes". It is sent back through the same understanding Sancti applies
 * to anything typed, so the imperative is what makes the second pass
 * act instead of asking again - and a pilgrim who types "yes" rather
 * than tapping gets there by the other road.
 */
export interface Offer {
  /** The question Sancti appends to its answer. */
  question: string;
  /** The tappable instruction underneath it. */
  followUp: string;
  /**
   * What Sancti says while doing it.
   *
   * Written out rather than derived from `followUp`. Turning "Open the
   * map" into "Opening the map" by swapping the verb works until the
   * instruction contains a pronoun: "Show me on the map" became
   * "Showing me on the map", which is Sancti talking about itself.
   */
  acting: string;
}

const OFFERS: Record<string, Offer> = {
  OPEN_MAP: {
    question: "Want me to open the map?", followUp: "Open the map",
    acting: "Opening the map.",
  },
  SHOW_CHURCH_LOCATION: {
    question: "Want me to open the map and show you the way?",
    followUp: "Show me on the map",
    acting: "Showing you on the map.",
  },
  OPEN_MASS_SCHEDULE: {
    question: "Want me to open the Mass schedule?",
    followUp: "Open the Mass schedule",
    acting: "Opening the Mass schedule.",
  },
  OPEN_CHURCH_HISTORY: {
    question: "Want me to open the full history?",
    followUp: "Open the history",
    acting: "Opening the history.",
  },
  OPEN_SACRAMENTS: {
    question: "Want me to open the sacraments?", followUp: "Open the sacraments",
    acting: "Opening the sacraments.",
  },
  OPEN_BAPTISM: {
```

```
    question: "Want me to open baptism?", followUp: "Open baptism",
    acting: "Opening baptism.",
  },
  OPEN_WEDDING: {
    question: "Want me to open weddings?", followUp: "Open the wedding page",
    acting: "Opening the wedding page.",
  },
  OPEN_MINISTRIES: {
    question: "Want me to open the ministries?", followUp: "Open the ministries",
    acting: "Opening the ministries.",
  },
  OPEN_AR: {
    question: "Want me to open the scanner?", followUp: "Open the scanner",
    acting: "Opening the scanner.",
  },
  OPEN_SETTINGS: {
    question: "Want me to open your settings?", followUp: "Open settings",
    acting: "Opening your settings.",
  },
  OPEN_CHURCH: {
    question: "Want me to open it?", followUp: "Open it",
    acting: "Opening it.",
  },
};

/**
 * What Sancti says while doing as it was told.
 *
 * An instruction gets an acknowledgement, not the answer again. Tapping
 * "Open the Mass schedule" under a list of Mass times and being handed
 * the same list a second time reads as not having been heard - and the
 * schedule itself is already arriving on screen.
 */
export function openingLine(action: string): string | null {
  return OFFERS[action]?.acting ?? null;
}

/**
 * The offer for one named thing, rather than for the list it is in.
 *
 * "Want me to open the ministries?" is the wrong question to ask
 * somebody who just typed the name of a ministry. They did not ask for
 * the list; naming theirs back to them is also how they can tell Sancti
 * understood which one they meant - particularly when they misspelt it.
 */
export function offerForItem(name: string): Offer {
  return {
    question: `Want me to open ${name}?`,
    followUp: `Open ${name}`,
    acting: `Opening ${name}.`,
  };
}

/** The offer for an action, or null where there is nothing to open. */
export function offerFor(action: string): Offer | null {
  return OFFERS[action] ?? null;
}

/**
```

```
 * An answer with the offer appended.
 *
 * A blank line between them, so the offer reads as a separate thought
 * rather than as the last sentence of the answer.
 */
export function withOffer(reply: Reply, offer: Offer): Reply {
  return { text: `${reply.text}

${offer.question}`, followUp: offer.followUp };
}

/** What Sancti says when the pilgrim declines. Never a sulk, never a retry. */
export function declinedAnswer(): Reply {
  return { text: "No problem. Ask me anything else." };
}

export interface Reply {
  text: string;
  /** Shown under the message as a tappable follow-up. */
  followUp?: string;
}

/**
 * The Mass times, read back as a sentence.
 *
 * Suspended Masses are named as suspended rather than dropped. Leaving
 * them out would have Sancti answer "there is no 8am" to somebody who
 * goes to the 8am every week, when the truth is that it is off this
 * month.
 */
export function massAnswer(
  parishName: string,
  schedule: MassScheduleEntry[],
): Reply {
  if (schedule.length === 0) {
    return {
      text: `I don't have the Mass schedule for ${parishName} in SanctiWalk yet. Please check with the parish office for the most accurate information.`,
    };
  }

  const lines: string[] = [];
  const suspended: string[] = [];

  for (const entry of schedule) {
    const on = availableTimes(entry);
    if (on.length > 0) lines.push(`${entry.day}: ${on.join(", ")}`);
    for (const time of entry.unavailableTimes ?? []) {
      suspended.push(`${entry.day} ${time}`);
    }
  }

  if (lines.length === 0) {
    return {
      text: `Every Mass at ${parishName} is suspended at the moment. Please check with the parish office.`,
    };
  }

  let text = `Mass at ${parishName}:\n${lines.join("\n")}`;
  if (suspended.length > 0) {
```

```
    text += `\n\nSuspended just now: ${suspended.join(", ")}.`;
  }
  return { text, followUp: "Remind me before Mass" };
}

/** The parish's own history, never a summary of one. */
export function historyAnswer(parishName: string, history?: string): Reply {
  if (!history?.trim()) {
    return {
      text: `${parishName} hasn't added its history to SanctiWalk yet. Please check with the parish office for the most accurate information.`,
    };
  }
  return { text: history.trim(), followUp: "Open the full history" };
}

/**
 * A sacrament or ministry, as the parish described it.
 *
 * Requirements are listed only when the parish listed them. An empty
 * list is reported as empty, because "no requirements" and "we have not
 * written them down" are different claims and only one of them is ours
 * to make.
 */
export function itemAnswer(item: ResolvedItem | null, open: boolean): Reply {
  if (!item) return { text: NO_INFORMATION };

  const parts: string[] = [item.about.trim()];

  if (item.requirements.length > 0) {
    parts.push(`What to bring:\n${item.requirements.map(r => `• ${r}`).join("\n")}`);
  } else {
    parts.push("The parish hasn't listed the requirements in SanctiWalk yet — the office can tell you.");
  }

  if (item.schedule) parts.push(`When: ${item.schedule}`);

  parts.push(
    open
      ? "Applications are open — I can take you to the form."
      : "Applications are closed at the moment, so the form isn't available. The information stays here for when it reopens.",
  );

  return {
    text: parts.join("\n\n"),
    followUp: open ? `Open ${item.name}` : undefined,
  };
}

/**
 * How far away a parish is.
 *
 * The walking figure is deliberately coarse. A minute-level estimate
 * from a straight-line distance would be a precise-looking number built
 * on a guess about roads nobody checked.
 */
export function distanceAnswer(
  parishName: string,
  metres: number | null,
  hasLocation: boolean,
```

```
): Reply {
  if (!hasLocation) {
    return {
      text: `I need your location to measure that. Turn on location for SanctiWalk and ask me again.`,
    };
  }
  if (metres === null) {
    return { text: `I couldn't work out where you are just now. Try again in a moment.` };
  }

  const km = metres / 1000;
  const distance = km < 1 ? `${Math.round(metres)} m` : `${km.toFixed(1)} km`;
  // About 5 km/h, rounded to five minutes. Straight-line, and said so.
  const minutes = Math.max(1, Math.round((km / 5) * 60 / 5) * 5);
  const walk = km < 3 ? ` That is roughly ${minutes} minutes' walk, going directly.` : "";

  return {
    text: `${parishName} is about ${distance} away.${walk}`,
    followUp: "Show it on the map",
  };
}

/** Contact details, only when the parish gave them. */
export function contactAnswer(parishName: string, contact?: string): Reply {
  if (!contact?.trim()) {
    return {
      text: `I don't have contact details for ${parishName} in SanctiWalk yet. Please check with the parish office for the most accurate information.`,
    };
  }
  return { text: `${parishName}: ${contact.trim()}` };
}

/**
 * Whether a Mass at that time actually exists before promising a reminder.
 *
 * The brief is explicit that a reminder is only created if the Mass is
 * real. Asked for "the 7pm Mass" at a parish whose last Mass is 6pm,
 * Sancti says what the real times are instead of setting an alarm for
 * something that will not happen.
 */
export function findMassTime(
  schedule: MassScheduleEntry[],
  wanted: string,
): { day: string; time: string } | null {
  const target = wanted.trim().toUpperCase();
  for (const entry of schedule) {
    for (const time of availableTimes(entry)) {
      if (time.trim().toUpperCase() === target) return { day: entry.day, time };
    }
  }
  return null;
}

export function reminderAnswer(
  parishName: string,
  schedule: MassScheduleEntry[],
  wanted?: string,
): Reply {
  if (schedule.length === 0) {
```

```
    return {
      text: `I don't have the Mass schedule for ${parishName} yet, so I can't set that reminder. Please check with the parish office.`,
    };
  }

  if (!wanted) {
    return {
      text: `Which Mass would you like reminding about? ${parishName} has:\n${schedule
        .map(e => `${e.day}: ${availableTimes(e).join(", ")}`)
        .filter(line => !line.endsWith(": "))
        .join("\n")}`,
    };
  }

  const found = findMassTime(schedule, wanted);
  if (!found) {
    return {
      text: `There isn't a ${wanted} Mass at ${parishName}. The times are:\n${schedule
        .map(e => `${e.day}: ${availableTimes(e).join(", ")}`)
        .filter(line => !line.endsWith(": "))
        .join("\n")}`,
    };
  }

  return {
    text: `Done — I'll remind you before the ${found.day} ${found.time} Mass at ${parishName}. You can change how early under Me.`,
  };
}

/** What Sancti says when it did not follow the question. */
export function unknownAnswer(): Reply {
  return {
    text:
      "I didn't quite follow that. I can show you the map, a church's history, Mass times, sacraments and ministries, set a Mass reminder, or open the scanner. Try
asking in a few words — “what time is Mass?”",
  };
}

export function helpAnswer(parishName: string): Reply {
  return {
    text:
      `I'm Sancti, your guide around ${parishName} and the rest of the diocese.\n\n` +
      "Ask me things like:\n" +
      "• What time is Mass?\n" +
      "• Take me to San Roque\n" +
      "• Is baptism available?\n" +
      "• Tell me the history\n" +
      "• Remind me about the 6 PM Mass\n\n" +
      "For anything pastoral, your parish priest is the one to ask.",
  };
}
```

1.4.62 APP-132 — sanctiGrounding.ts

Code ID	APP-132
File Name	sanctiGrounding.ts
File Path	SanctDemo/src/lib/sanctiGrounding.ts
Language / Type	TypeScript
Lines	244
Functions / Classes	Fact (exported type); Gap (exported type); Grounding (exported type); liveTimesFor (exported function); massFact (exported function); isGap (exported function); itemFacts (exported function); offersItem (exported function); freshnessNote (exported function); groundingBlock (exported function)
Related Files (its imports)	./parishContent, ./schedule
Where It Is Used	server.ts
How to Recognize It	Open SanctDemo/src/lib/sanctiGrounding.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The facts Sancti is allowed to state, and where each one came from.
>
> ## Why this exists
>
> Sancti's answers were only ever as current as whatever happened to be compiled into the bundle. Three separate things made that inaccurate, and all three are structural rather than wording:
>
> 1. Live admin-managed content was consulted ONLY for the parish the pilgrim currently had open. Ask about any other parish and the answer silently fell back to the compiled schedule - so a parish office could move its Mass in the admin portal and Sancti would keep quoting the old time to everybody not standing in that parish. 2. Nothing ever said how old a fact was, though `updatedAt` has been on every parish document all along. 3. "Missing" and "nothing published" were indistinguishable, so an empty requirements list read as "there are no requirements".
>
> ## The contract
>
> Everything in here is VERIFIED - it came from the parish's own document or the app's compiled data, never from a language model. A `Fact` is something Sancti may state. A `Gap` is something it was asked for and does not have, and must say it does not have. Anything that is neither is not an answer about this parish.
>
> Pure and dependency-free, so the rules can be tested without Firebase, a network or a model.

Complete Source Code — sanctiGrounding.ts

```
/**
 * The facts Sancti is allowed to state, and where each one came from.
 *
```

```
 * ## Why this exists
 *
 * Sancti's answers were only ever as current as whatever happened to be
 * compiled into the bundle. Three separate things made that inaccurate,
 * and all three are structural rather than wording:
 *
 * 1. Live admin-managed content was consulted ONLY for the parish the
 *    pilgrim currently had open. Ask about any other parish and the
 *    answer silently fell back to the compiled schedule - so a parish
 *    office could move its Mass in the admin portal and Sancti would
 *    keep quoting the old time to everybody not standing in that parish.
 * 2. Nothing ever said how old a fact was, though `updatedAt` has been
 *    on every parish document all along.
 * 3. "Missing" and "nothing published" were indistinguishable, so an
 *    empty requirements list read as "there are no requirements".
 *
 * ## The contract
 *
 * Everything in here is VERIFIED - it came from the parish's own
 * document or the app's compiled data, never from a language model.
 * A `Fact` is something Sancti may state. A `Gap` is something it was
 * asked for and does not have, and must say it does not have. Anything
 * that is neither is not an answer about this parish.
 *
 * Pure and dependency-free, so the rules can be tested without Firebase,
 * a network or a model.
 */
import type { ParishContent } from './parishContent'
import type { MassScheduleEntry } from './schedule'

/** A single verified statement, with its provenance. */
export interface Fact {
  /** What this is about: 'mass', 'history', 'contact', an item id. */
  topic: string
  /** The fact itself, already in words. */
  value: string
  /**
   * Where it came from. 'parish' is the office's own entry through the
   * admin portal; 'app' is SanctiWalk's compiled data, which is correct
   * but not maintained by that parish.
   */
  source: 'parish' | 'app'
  /** ISO timestamp of the parish's last edit, when the document carries one. */
  updatedAt?: string
}

/** Something asked for that genuinely is not known. */
export interface Gap {
  topic: string
  /** Said to the pilgrim verbatim, so the wording is reviewed here. */
  reason: string
}

export interface Grounding {
  parishId: string
  parishName: string
  facts: Fact[]
  gaps: Gap[]
}
```

```
/** Readable times for one day, skipping any the parish has suspended. */
export function liveTimesFor(entry: MassScheduleEntry): string[] {
  const suspended = new Set(
    (((entry as { suspended?: string[] }).suspended) ?? []).map(t => t.trim().toLowerCase()),
  )
  return entry.time
    .split(',')
    .map(t => t.trim())
    .filter(t => t.length > 0 && !suspended.has(t.toLowerCase()))
}

/**
 * The Mass schedule as a fact, or a gap when there is none.
 *
 * The parish's own entry always wins over the compiled one - that is the
 * precedence the Mass card already uses, and Sancti disagreeing with the
 * card on the same screen is its own kind of wrong. Unlike before, this
 * applies to EVERY parish, not only the one currently open.
 */
export function massFact(
  parishContent: ParishContent | null | undefined,
  compiled: MassScheduleEntry[] | null | undefined,
): Fact | Gap {
  const parish = parishContent?.massSchedule
  const usingParish = Boolean(parish && parish.length > 0)
  const chosen = usingParish ? parish : compiled
  const source: 'parish' | 'app' = usingParish ? 'parish' : 'app'

  if (!chosen || chosen.length === 0) {
    return {
      topic: 'mass',
      reason: 'No Mass schedule has been published for this parish.',
    }
  }

  const lines = chosen
    .map(entry => {
      const times = liveTimesFor(entry)
      if (times.length === 0) return `${entry.day}: no Mass currently scheduled`
      return `${entry.day}: ${times.join(', ')}`
    })
    .join('; ')

  return {
    topic: 'mass',
    value: lines,
    source,
    updatedAt: source === 'parish' ? parishContent?.updatedAt : undefined,
  }
}

/** Whether a value is a Gap rather than a Fact. */
export function isGap(v: Fact | Gap): v is Gap {
  return (v as Gap).reason !== undefined
}

/**
 * What the parish has published about one ministry or sacrament.
```

```
 *
 * Each sub-field is weighed separately. A parish that has written an
 * "about" but no requirements has published one thing and not the other,
 * and collapsing that into a single yes/no is how "requirements not
 * listed" turns into "there are no requirements".
 */
export function itemFacts(
  parishContent: ParishContent | null | undefined,
  itemId: string,
  itemName: string,
): { facts: Fact[]; gaps: Gap[] } {
  const content = parishContent?.itemContent?.[itemId]
  const facts: Fact[] = []
  const gaps: Gap[] = []
  const at = parishContent?.updatedAt

  const add = (topic: string, value: string | undefined) => {
    const trimmed = value?.trim()
    if (trimmed) facts.push({ topic, value: trimmed, source: 'parish', updatedAt: at })
  }
  const addList = (topic: string, values: string[] | undefined) => {
    const cleaned = (values ?? []).map(v => v.trim()).filter(Boolean)
    if (cleaned.length > 0) {
      facts.push({ topic, value: cleaned.join('; '), source: 'parish', updatedAt: at })
    }
  }

  add(`${itemId}.about`, content?.about)
  addList(`${itemId}.requirements`, content?.requirements)
  add(`${itemId}.schedule`, content?.schedule)
  addList(`${itemId}.process`, content?.process)
  add(`${itemId}.contact`, content?.contact)

  if (!content?.requirements?.some(r => r.trim())) {
    gaps.push({
      topic: `${itemId}.requirements`,
      reason: `${itemName} requirements have not been published by this parish.`,
    })
  }
  if (!content?.schedule?.trim()) {
    gaps.push({
      topic: `${itemId}.schedule`,
      reason: `No ${itemName.toLowerCase()} schedule has been published by this parish.`,
    })
  }

  return { facts, gaps }
}

/**
 * Whether this parish can be said to OFFER something.
 *
 * Deliberately strict, and deliberately not the inverse of
 * closedApplications. A parish with no document at all is open for
 * applications - that is the rule availability.ts encodes and it is the
 * right one for a form. It is the wrong one for a sentence: "yes, they
 * do baptisms" said on the strength of no data is exactly the claim
 * requirement 3 forbids. Silence here means "not stated", and the only
 * honest answer is to send the pilgrim to the office.
```

```
 */
export function offersItem(
  parishContent: ParishContent | null | undefined,
  itemId: string,
): 'yes' | 'no' | 'unstated' {
  if (parishContent?.closedApplications?.includes(itemId)) return 'no'
  const content = parishContent?.itemContent?.[itemId]
  const published = Boolean(
    content?.about?.trim() ||
      content?.schedule?.trim() ||
      content?.requirements?.some(r => r.trim()) ||
      content?.process?.some(p => p.trim()),
  )
  return published ? 'yes' : 'unstated'
}

/** How old a fact is, in plain words, or null when it carries no date. */
export function freshnessNote(fact: Fact, now: Date = new Date()): string | null {
  if (fact.source !== 'parish' || !fact.updatedAt) return null
  const then = new Date(fact.updatedAt)
  if (Number.isNaN(then.getTime())) return null

  const days = Math.floor((now.getTime() - then.getTime()) / 86_400_000)
  if (days < 0) return null
  if (days === 0) return 'updated by the parish today'
  if (days === 1) return 'updated by the parish yesterday'
  if (days < 30) return `updated by the parish ${days} days ago`
  if (days < 365) return `last updated by the parish ${Math.floor(days / 30)} months ago`
  return 'last updated by the parish over a year ago'
}

/**
 * The grounding block handed to the model.
 *
 * Facts and gaps are both listed, and the gaps are not padding: a model
 * given only facts will fill a silence, and the thing it fills it with
 * is a plausible Mass time. Naming the gap explicitly is what turns
 * an admission of ignorance into the expected answer rather than a
 * failure to answer.
 */
export function groundingBlock(g: Grounding, now: Date = new Date()): string {
  const lines: string[] = [`PARISH: ${g.parishName}`]

  if (g.facts.length === 0) {
    lines.push('VERIFIED FACTS: none available for this parish.')
  } else {
    lines.push('VERIFIED FACTS (you may state these):')
    for (const f of g.facts) {
      const fresh = freshnessNote(f, now)
      const origin =
        f.source === 'parish'
          ? `parish office${fresh ? `, ${fresh}` : ''}`
          : 'SanctiWalk app data, not maintained by this parish'
      lines.push(`- [${f.topic}] ${f.value} (source: ${origin})`)
    }
  }

  if (g.gaps.length > 0) {
    lines.push('NOT AVAILABLE (you must say these are not available, never guess):')
```

```
    for (const gap of g.gaps) lines.push(`- [${gap.topic}] ${gap.reason}`)
  }

  return lines.join('\n')
}
```

1.4.63 APP-133 — scanNarration.ts

Code ID	APP-133
File Name	scanNarration.ts
File Path	SanctDemo/src/lib/scanNarration.ts
Language / Type	TypeScript
Lines	102
Functions / Classes	NarratableResult (exported type); asSentence (internal function); scanScript (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/scanNarration.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Reading a scan result aloud.
>
> ## Why a scan especially needs a voice
>
> The other places this app speaks are places you sit down to read. A scan result is read standing in front of a statue, often in low light, frequently by somebody older than the phone they are holding. It is the screen most likely to be squinted at and the one where looking up at the thing itself matters more than looking down.
>
> ## What is read, and in what order
>
> Name, then what was seen, then who the saint was, then the short facts. That is the order the card itself uses, and the order a guide would use out loud: say what it is before explaining it.
>
> The provenance line is NOT read. On the page it is a quiet caveat the eye can take in at a glance; spoken, "general Catholic information, not parish record" lands with the same weight as the history itself and interrupts the one part worth hearing. The label stays visible for anyone checking.
>
> Pure, so what gets spoken can be tested without a speech engine.

Complete Source Code — scanNarration.ts

```
/**
 * Reading a scan result aloud.
 *
 * ## Why a scan especially needs a voice
 *
 * The other places this app speaks are places you sit down to read. A
 * scan result is read standing in front of a statue, often in low light,
 * frequently by somebody older than the phone they are holding. It is
 * the screen most likely to be squinted at and the one where looking up
```

```
 * at the thing itself matters more than looking down.
 *
 * ## What is read, and in what order
 *
 * Name, then what was seen, then who the saint was, then the short
 * facts. That is the order the card itself uses, and the order a guide
 * would use out loud: say what it is before explaining it.
 *
 * The provenance line is NOT read. On the page it is a quiet caveat the
 * eye can take in at a glance; spoken, "general Catholic information,
 * not parish record" lands with the same weight as the history itself
 * and interrupts the one part worth hearing. The label stays visible
 * for anyone checking.
 *
 * Pure, so what gets spoken can be tested without a speech engine.
 */

/** The fields of a scan result that are worth hearing. */
export interface NarratableResult {
  title?: string
  summary?: string
  saintName?: string
  feastDay?: string
  patronage?: string
  aboutSaint?: string
  highlights?: string[]
}

/**
 * A heading ends with a full stop so the voice drops and pauses.
 *
 * Without it the engine runs one part into the next - "Feast day October
 * 4 patron of animals" - which is the clearest sign a page is being read
 * by a machine rather than read out.
 */
function asSentence(text: string | undefined): string {
  const trimmed = (text ?? '').trim()
  if (!trimmed) return ''
  return /[.!?]$/.test(trimmed) ? trimmed : `${trimmed}.`
}

/**
 * What the speaker button reads for a scan result.
 *
 * Returns an empty string when there is nothing worth hearing, which is
 * the caller's signal to show no button: a speaker that speaks a title
 * and stops is worse than no speaker.
 */
export function scanScript(result: NarratableResult, language: 'en' | 'fil' = 'en'): string {
  const parts: string[] = []

  const title = asSentence(result.title)
  if (title) parts.push(title)

  const summary = (result.summary ?? '').trim()
  if (summary) parts.push(summary)

  // Who they were, introduced so the shift from "what I saw" to "who
  // this is" is audible. Without the lead-in the two run together as
```

```
  // one undifferentiated paragraph.
  const about = (result.aboutSaint ?? '').trim()
  if (about) {
    const who = (result.saintName ?? '').trim()
    parts.push(language === 'fil' ? `Tungkol kay ${who || 'kanya'}.` : `About ${who || 'this saint'}.`)

    const feast = (result.feastDay ?? '').trim()
    if (feast) {
      parts.push(language === 'fil' ? `Kapistahan: ${asSentence(feast)}` : `Feast day: ${asSentence(feast)}`)
    }

    const patron = (result.patronage ?? '').trim()
    if (patron) {
      parts.push(
        language === 'fil' ? `Patron ng ${asSentence(patron)}` : `Patron of ${asSentence(patron)}`,
      )
    }

    parts.push(about)
  }

  const highlights = (result.highlights ?? []).map(h => h.trim()).filter(Boolean)
  if (highlights.length > 0) {
    parts.push(...highlights.map(h => asSentence(h)))
  }

  // Nothing but a name is not worth a button.
  if (parts.length < 2) return ''

  // Blank lines between parts: every engine treats a paragraph break as
  // a longer pause than a full stop, which is the difference between
  // being told about something and being read a list.
  return parts.join('\n\n')
}
```

1.4.64 APP-134 — scanQuota.ts

Code ID	APP-134
File Name	scanQuota.ts
File Path	SanctDemo/src/lib/scanQuota.ts
Language / Type	TypeScript
Lines	104
Functions / Classes	SCANS_PER_PARISH_PER_DAY (exported constant); SCANNABLE_PARISHES (exported constant); ScannableParish (exported type); ScanBudget (exported type); utcDay (exported function); isScannableParish (exported function); scanQuotaDocId (exported function); budgetFrom (exported function); emptyBudget (exported function); spentMessage (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	server.ts, src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/scanQuota.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Who may spend a recognition, and how many are left.
>
> ## Why this replaced the per-IP cap
>
> The old ceiling counted scans against the caller's IP address. Inside a church, a school or a house, every phone on the same Wi-Fi shares one public address — so twenty scans were twenty for the whole room, and the first person through the door could spend everybody's. The opposite failed too: a visitor who walked outside and dropped to mobile data became a new IP and got a fresh twenty.
>
> An address is not a person. An account is, so the allowance is counted against the signed-in user.
>
> ## Why per parish
>
> A pilgrim visiting both churches is on two separate visits and should not arrive at the second one with an allowance already spent at the first. Each parish keeps its own counter.
>
> ## Why the day is UTC
>
> The counter has to reset on a boundary the server and the phone agree on without being told each other's timezone. Manila is UTC+8, so the reset lands at 08:00 local - inconvenient to explain, but a counter that resets at a predictable wrong-looking hour is better than one that resets twice for a traveller or not at all.

Complete Source Code — scanQuota.ts

```
/**
```

```
 * Who may spend a recognition, and how many are left.
 *
 * ## Why this replaced the per-IP cap
 *
 * The old ceiling counted scans against the caller's IP address. Inside
 * a church, a school or a house, every phone on the same Wi-Fi shares
 * one public address — so twenty scans were twenty for the whole room,
 * and the first person through the door could spend everybody's. The
 * opposite failed too: a visitor who walked outside and dropped to
 * mobile data became a new IP and got a fresh twenty.
 *
 * An address is not a person. An account is, so the allowance is
 * counted against the signed-in user.
 *
 * ## Why per parish
 *
 * A pilgrim visiting both churches is on two separate visits and should
 * not arrive at the second one with an allowance already spent at the
 * first. Each parish keeps its own counter.
 *
 * ## Why the day is UTC
 *
 * The counter has to reset on a boundary the server and the phone agree
 * on without being told each other's timezone. Manila is UTC+8, so the
 * reset lands at 08:00 local - inconvenient to explain, but a counter
 * that resets at a predictable wrong-looking hour is better than one
 * that resets twice for a traveller or not at all.
 *
 * Pure and dependency-free: the server enforces these rules and the app
 * only displays them, so both can import the same file and neither can
 * drift from the other.
 */

/** Recognitions per user, per parish, per day. */
export const SCANS_PER_PARISH_PER_DAY = 20

/** The parishes that have a scanner at all. */
export const SCANNABLE_PARISHES = ['route-mhcp', 'route-src'] as const

export type ScannableParish = (typeof SCANNABLE_PARISHES)[number]

export interface ScanBudget {
  limit: number
  used: number
  remaining: number
  /** The UTC day this count belongs to, as YYYY-MM-DD. */
  day: string
  parishId: string
}

/** The UTC day a moment falls in. */
export function utcDay(at: Date = new Date()): string {
  return at.toISOString().slice(0, 10)
}

export function isScannableParish(parishId: unknown): parishId is ScannableParish {
  return typeof parishId === 'string'
    && (SCANNABLE_PARISHES as readonly string[]).includes(parishId)
}
```

```
/**
 * The Firestore document a user's allowance for one parish-day lives in.
 *
 * One document per user per parish per day, rather than one growing
 * document per user. A day's counter is then a single read, old days
 * fall out of the way on their own, and two parishes cannot contend for
 * the same document while someone scans in both.
 *
 * The id is built from values the server already has, never from
 * anything the client sends unchecked: the uid comes from a verified
 * token and the parish from a fixed list.
 */
export function scanQuotaDocId(uid: string, parishId: string, day: string): string {
  return `${uid}__${parishId}__${day}`
}

export function budgetFrom(used: number, parishId: string, day: string): ScanBudget {
  const safe = Number.isFinite(used) && used > 0 ? Math.floor(used) : 0
  return {
    limit: SCANS_PER_PARISH_PER_DAY,
    used: safe,
    remaining: Math.max(0, SCANS_PER_PARISH_PER_DAY - safe),
    day,
    parishId,
  }
}

/** An untouched allowance, for a user who has not scanned here today. */
export function emptyBudget(parishId: string, day: string = utcDay()): ScanBudget {
  return budgetFrom(0, parishId, day)
}

/**
 * What to tell somebody who has none left.
 *
 * Names the parish, because the other one still has its own twenty and
 * a visitor standing in the second church should not be told the app is
 * finished with them for the day.
 */
export function spentMessage(parishName: string): string {
  return `You have used all ${SCANS_PER_PARISH_PER_DAY} scans for ${parishName} today. `
    + `The count resets at midnight UTC, and other parishes have their own.`
}
```

1.4.65 APP-135 — scanQuotaStore.ts

Code ID	APP-135
File Name	scanQuotaStore.ts
File Path	SanctDemo/src/lib/scanQuotaStore.ts
Language / Type	TypeScript
Lines	177
Functions / Classes	initQuotaStore (exported function); quotaStoreReady (exported function); uidFromToken (exported function); readBudget (exported function); SpendResult (exported type); spendScan (exported function); refundScan (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/lib/parishContentStore.ts
How to Recognize It	Open SanctDemo/src/lib/scanQuotaStore.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The server's side of the scan allowance: who is asking, and whether they have one left.
>
> Server-only. It uses the Firebase Admin SDK, which holds credentials that must never reach a browser, so nothing under src/components may import it. The pure rules live in scanQuota.ts and are shared.
>
> ## Why a transaction and not read-then-write
>
> Two scans fired at once - a double tap, or a retry arriving beside the original - both read "19 used" and both write 20, and the user has spent two scans for one increment. Firestore's transaction retries on contention, so the second attempt sees the first's write. The allowance is small enough that one free scan matters.
>
> ## Why the allowance is spent before the model is asked
>
> A scan that is paid for and then fails is unfair; a scan that is free when it fails is an exploit, because a client that aborts every request has an unlimited allowance. The middle course, and the one taken here: reserve first, and refund only when the failure is ours (the model timed out, the key is wrong), never when it is simply a photograph the model could not name.

Complete Source Code — scanQuotaStore.ts

```
/**
 * The server's side of the scan allowance: who is asking, and whether
 * they have one left.
 *
 * Server-only. It uses the Firebase Admin SDK, which holds credentials
 * that must never reach a browser, so nothing under src/components may
 * import it. The pure rules live in scanQuota.ts and are shared.
 *
```

```
 * ## Why a transaction and not read-then-write
 *
 * Two scans fired at once - a double tap, or a retry arriving beside
 * the original - both read "19 used" and both write 20, and the user
 * has spent two scans for one increment. Firestore's transaction
 * retries on contention, so the second attempt sees the first's write.
 * The allowance is small enough that one free scan matters.
 *
 * ## Why the allowance is spent before the model is asked
 *
 * A scan that is paid for and then fails is unfair; a scan that is free
 * when it fails is an exploit, because a client that aborts every
 * request has an unlimited allowance. The middle course, and the one
 * taken here: reserve first, and refund only when the failure is ours
 * (the model timed out, the key is wrong), never when it is simply a
 * photograph the model could not name.
 */
import { applicationDefault, cert, getApps, initializeApp } from 'firebase-admin/app'
import { FieldValue, getFirestore } from 'firebase-admin/firestore'
import { getAuth } from 'firebase-admin/auth'

import {
  SCANS_PER_PARISH_PER_DAY,
  budgetFrom,
  scanQuotaDocId,
  utcDay,
  type ScanBudget,
} from './scanQuota'

const COLLECTION = 'scanQuotas'

let ready = false

/**
 * Starts the Admin SDK once, from whichever credential the host gives.
 *
 * On Render the service account arrives as one JSON blob in an
 * environment variable, because that is the only shape a dashboard can
 * hold. Locally, GOOGLE_APPLICATION_CREDENTIALS pointing at a file is
 * the usual route. Either is fine; neither is ever committed.
 */
export function initQuotaStore(): boolean {
  if (ready) return true
  if (getApps().length) {
    ready = true
    return true
  }

  const raw = process.env.FIREBASE_SERVICE_ACCOUNT
  try {
    if (raw && raw.trim().startsWith('{')) {
      initializeApp({ credential: cert(JSON.parse(raw)) })
    } else if (process.env.GOOGLE_APPLICATION_CREDENTIALS) {
      initializeApp({ credential: applicationDefault() })
    } else {
      return false
    }
    ready = true
    return true
```

```
  } catch (err) {
    console.error('[Quota] could not start the Admin SDK:', (err as Error).message)
    return false
  }
}

/** True when the server can verify tokens and count scans. */
export function quotaStoreReady(): boolean {
  return ready || initQuotaStore()
}

/**
 * The uid behind a bearer token, or null.
 *
 * Verified by the Admin SDK against Google's keys, so a forged token
 * cannot pass. This is the whole reason the allowance is trustworthy:
 * the client says which parish, but never who it is.
 */
export async function uidFromToken(authorization: string | undefined): Promise<string | null> {
  if (!quotaStoreReady()) return null
  const token = (authorization ?? '').replace(/^Bearer\s+/i, '').trim()
  if (!token) return null
  try {
    const decoded = await getAuth().verifyIdToken(token)
    return decoded.uid
  } catch (err) {
    console.warn('[Quota] rejected a token:', (err as Error).message)
    return null
  }
}

/** What this user has left at this parish today, without spending any. */
export async function readBudget(uid: string, parishId: string): Promise<ScanBudget> {
  const day = utcDay()
  if (!quotaStoreReady()) return budgetFrom(0, parishId, day)

  const snap = await getFirestore()
    .collection(COLLECTION)
    .doc(scanQuotaDocId(uid, parishId, day))
    .get()

  return budgetFrom((snap.data()?.used as number) ?? 0, parishId, day)
}

export interface SpendResult {
  allowed: boolean
  budget: ScanBudget
}

/**
 * Takes one scan from the allowance, or refuses.
 *
 * The read and the write are one transaction, so two requests racing
 * cannot both be told they were the twentieth.
 */
export async function spendScan(uid: string, parishId: string): Promise<SpendResult> {
  const day = utcDay()
  if (!quotaStoreReady()) {
    // Fail closed. This used to allow the scan and log a warning, which
```

```
    // made a forgotten environment variable into an unlimited public
    // scanner. Callers must not reach here - requireUser answers 503
    // first - so this is the belt to that braces, and it throws rather
    // than quietly returning a budget nobody is counting.
    throw new Error('scan quota store is not configured')
  }

  const ref = getFirestore().collection(COLLECTION).doc(scanQuotaDocId(uid, parishId, day))

  return getFirestore().runTransaction(async tx => {
    const snap = await tx.get(ref)
    const used = (snap.data()?.used as number) ?? 0

    if (used >= SCANS_PER_PARISH_PER_DAY) {
      return { allowed: false, budget: budgetFrom(used, parishId, day) }
    }

    tx.set(ref, {
      uid,
      parishId,
      day,
      used: used + 1,
      updatedAt: FieldValue.serverTimestamp(),
    }, { merge: true })

    return { allowed: true, budget: budgetFrom(used + 1, parishId, day) }
  })
}

/**
 * Gives a reserved scan back, after a failure that was not the user's.
 *
 * Never below zero: a refund arriving after the daily reset must not
 * leave tomorrow's counter negative and hand out twenty-one.
 */
export async function refundScan(uid: string, parishId: string): Promise<void> {
  const day = utcDay()
  if (!quotaStoreReady()) return

  const ref = getFirestore().collection(COLLECTION).doc(scanQuotaDocId(uid, parishId, day))
  try {
    await getFirestore().runTransaction(async tx => {
      const snap = await tx.get(ref)
      const used = (snap.data()?.used as number) ?? 0
      if (used <= 0) return
      tx.set(ref, { used: used - 1, updatedAt: FieldValue.serverTimestamp() }, { merge: true })
    })
  } catch (err) {
    console.warn('[Quota] refund failed:', (err as Error).message)
  }
}
```

1.4.66 APP-136 — schedule.ts

Code ID	APP-136
File Name	schedule.ts
File Path	SanctDemo/src/lib/schedule.ts
Language / Type	TypeScript
Lines	177
Functions / Classes	MassScheduleEntry (exported type); isTimeAvailable (exported function); availableTimes (exported function); withTimeAvailability (exported function); NextMass (exported type); parseTimes (exported function); toMinutes (exported function); nextMass (exported function); MASS_DURATION_MINUTES (exported constant); MASS_STARTING_SOON_MINUTES (exported constant); MassState (exported type); MassStatus (exported type); massStatus (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/admin/ParishStatus.tsx, src/components/HomeHero.tsx, src/components/MassNowBadge.tsx, src/components/MassSchedule.tsx, src/components/ParishCard.tsx, src/components/ParishContentEditor.tsx, src/components/PresenceSheet.tsx, src/components/SearchScreen.tsx, src/lib/parishContent.ts, src/lib/reminderSchedule.ts, src/lib/sanctiAnswers.ts, src/lib/sanctiGrounding.ts, src/lib/sundayMasses.ts
How to Recognize It	Open SanctDemo/src/lib/schedule.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — schedule.ts

```
const DAYS = ['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday']

export interface MassScheduleEntry {
  day: string
  /** One or more times for this day, comma separated: "6:00 AM, 8:00 AM". */
  time: string
  /**
   * Times on this day the parish has temporarily suspended.
   *
   * A list of exceptions, not a flag per time, for the same reason
   * closedApplications is: absent means everything runs, so every parish
   * that has never touched this keeps its full schedule. The strings
   * match the entries in `time` exactly.
   *
   * Suspended, not deleted: the app still SHOWS a 8:00 AM Mass marked
   * "Not available", because a parishioner who turns up at 8 needs to
   * know the Mass exists and is off, not to find no mention of it and
   * assume they misremembered.
   */
  unavailableTimes?: string[]
  /** "No 6pm Mass during the renovation" - shown with the day. */
```

```
  note?: string
}

/** True unless the parish has suspended this particular Mass. */
export function isTimeAvailable(entry: MassScheduleEntry, time: string): boolean {
  return !(entry.unavailableTimes ?? []).includes(time.trim())
}

/** The times on this day that are actually being celebrated. */
export function availableTimes(entry: MassScheduleEntry): string[] {
  return parseTimes(entry.time).filter((t) => isTimeAvailable(entry, t))
}

/** Adds or removes one time from a day's suspended list. */
export function withTimeAvailability(
  entry: MassScheduleEntry,
  time: string,
  available: boolean,
): MassScheduleEntry {
  const next = new Set(entry.unavailableTimes ?? [])
  if (available) next.delete(time.trim())
  else next.add(time.trim())
  const list = [...next]
  // Dropped entirely when empty rather than written as [], so a parish
  // that never suspends anything keeps the document it always had.
  const { unavailableTimes: _old, ...rest } = entry
  return list.length > 0 ? { ...rest, unavailableTimes: list } : rest
}

export interface NextMass {
  day: string
  time: string
  date: Date
}

export function parseTimes(value: string): string[] {
  return value
    .split(',')
    .map((s) => s.trim())
    .filter(Boolean)
}

export function toMinutes(time: string): number {
  const match = time.trim().match(/^(\d{1,2}):(\d{2})\s*(AM|PM)$/i)
  if (!match) return NaN

  let hours = Number(match[1]) % 12
  const minutes = Number(match[2])
  if (match[3].toUpperCase() === 'PM') hours += 12

  return hours * 60 + minutes
}

export function nextMass(schedule: MassScheduleEntry[], now: Date): NextMass | null {
  if (schedule.length === 0) return null

  // Suspended Masses are not candidates for "the next Mass". Telling
  // someone to come at 8 for a Mass the parish has called off is the one
  // failure this whole feature exists to prevent.
```

```
  const byDay = new Map(schedule.map((s) => [s.day, availableTimes(s)]))
  const nowMinutes = now.getHours() * 60 + now.getMinutes()

  for (let offset = 0; offset < 8; offset++) {
    const dayName = DAYS[(now.getDay() + offset) % 7]
    const times = byDay.get(dayName)
    if (!times) continue

    // Sort by clock time rather than trusting the data's order — parish data
    // is hand-entered, and "4:30 PM, 6:00 AM" would otherwise report the
    // afternoon Mass as the next one.
    const ordered = times
      .filter((t) => !Number.isNaN(toMinutes(t)))
      .sort((a, b) => toMinutes(a) - toMinutes(b))

    for (const time of ordered) {
      const minutes = toMinutes(time)
      if (offset === 0 && minutes <= nowMinutes) continue

      const date = new Date(now)
      date.setDate(date.getDate() + offset)
      date.setHours(Math.floor(minutes / 60), minutes % 60, 0, 0)

      return { day: dayName, time, date }
    }
  }

  return null
}

/**
 * How long a Mass is assumed to run, in minutes.
 *
 * An ordinary Sunday or weekday Mass in a Philippine parish runs roughly an
 * hour; a sung or Tagalog Mass often runs longer. Sixty is the honest middle,
 * and the consequence of being wrong is small in one direction and large in
 * the other: saying "in progress" a few minutes after it ended simply sends
 * someone into a quiet church, whereas saying "not on" while it is still
 * being celebrated turns them away from the door.
 */
export const MASS_DURATION_MINUTES = 60

/** Within this many minutes of the hour, a Mass counts as about to start. */
export const MASS_STARTING_SOON_MINUTES = 30

export type MassState = 'in-progress' | 'starting-soon' | 'none'

export interface MassStatus {
  state: MassState
  /** The Mass this refers to, e.g. "6:00 AM". Null when state is 'none'. */
  time: string | null
  /** Minutes until it starts (starting-soon), or since it began (in-progress). */
  minutes: number
}

/**
 * Whether a Mass is happening right now, or about to.
 *
 * This is the thing a mapping app cannot answer and a parish app must: not
```

```
 * "when is the next Mass" but "can I still walk in". It reads only today's
 * entry, because a Mass tomorrow is not happening now — `nextMass` already
 * answers the forward-looking question.
 */
export function massStatus(
  schedule: MassScheduleEntry[],
  now: Date,
  { durationMinutes = MASS_DURATION_MINUTES, soonMinutes = MASS_STARTING_SOON_MINUTES } = {},
): MassStatus {
  const today = schedule.find((entry) => entry.day === DAYS[now.getDay()])
  if (!today) return { state: 'none', time: null, minutes: 0 }

  const nowMinutes = now.getHours() * 60 + now.getMinutes()
  const times = parseTimes(today.time)
    .map((time) => ({ time, at: toMinutes(time) }))
    .filter((t) => Number.isFinite(t.at))
    // Hand-entered data arrives in whatever order someone typed it, so
    // "4:30 PM, 6:00 AM" must not report the afternoon Mass as the earlier.
    .sort((a, b) => a.at - b.at)

  // In progress wins over starting soon: standing outside at 6:05 with a
  // 6:00 and a 6:30 Mass, the useful fact is that one is already under way.
  for (const { time, at } of times) {
    const since = nowMinutes - at
    if (since >= 0 && since < durationMinutes) {
      return { state: 'in-progress', time, minutes: since }
    }
  }

  for (const { time, at } of times) {
    const until = at - nowMinutes
    if (until > 0 && until <= soonMinutes) {
      return { state: 'starting-soon', time, minutes: until }
    }
  }

  return { state: 'none', time: null, minutes: 0 }
}
```

1.4.67 APP-137 — simulatorMode.ts

Code ID	APP-137
File Name	simulatorMode.ts
File Path	SanctDemo/src/lib/simulatorMode.ts
Language / Type	TypeScript
Lines	103
Functions / Classes	SIMULATOR_KEY (exported constant); SIMULATOR_EVENT (exported constant); loadSimulatorMode (exported function); saveSimulatorMode (exported function); ArBlockedReason (exported type); ArAccess (exported type); ArConditions (exported type); arAccess (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/simulatorMode.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Simulator mode, and who may open the AR tour.
>
> ## What the switch is for
>
> The AR tour is only meaningful inside a parish: it stands the parish's own statues where they really are, read off a scan of that building. Opened anywhere else it has nothing to anchor to, so for a visitor it should say "this works at the parish" rather than opening onto an empty room.
>
> But the proponents cannot be in the church every time they change something, and a gate they cannot pass at home would mean no testing at all. So it is a switch.
>
> OFF - everything is reachable from anywhere. This is the testing state, and it is the default, because a default that locked the tour would make a fresh install look broken.
>
> ON - the app behaves as it will for a visitor: the AR tour opens only at the parish, and the AI scanner still works anywhere.
>
> The names are the other way round from the usual sense of "simulator", so nothing here relies on the name: the setting screen spells out what each position does, and so does the notice the scanner shows.

Complete Source Code — simulatorMode.ts

```
/**
 * Simulator mode, and who may open the AR tour.
 *
 * ## What the switch is for
 *
 * The AR tour is only meaningful inside a parish: it stands the parish's
 * own statues where they really are, read off a scan of that building.
 * Opened anywhere else it has nothing to anchor to, so for a visitor it
```

```
 * should say "this works at the parish" rather than opening onto an
 * empty room.
 *
 * But the proponents cannot be in the church every time they change
 * something, and a gate they cannot pass at home would mean no testing
 * at all. So it is a switch.
 *
 *   OFF - everything is reachable from anywhere. This is the testing
 *         state, and it is the default, because a default that locked
 *         the tour would make a fresh install look broken.
 *
 *   ON  - the app behaves as it will for a visitor: the AR tour opens
 *         only at the parish, and the AI scanner still works anywhere.
 *
 * The names are the other way round from the usual sense of
 * "simulator", so nothing here relies on the name: the setting screen
 * spells out what each position does, and so does the notice the
 * scanner shows.
 */

export const SIMULATOR_KEY = 'sanctiwalk.simulatorMode'

/** Fired when the setting changes, for the hooks in this tab. */
export const SIMULATOR_EVENT = 'sanctiwalk:simulator'

export function loadSimulatorMode(): boolean {
  try {
    return localStorage.getItem(SIMULATOR_KEY) === 'on'
  } catch {
    // Private mode. Off is the state that leaves everything reachable.
    return false
  }
}

export function saveSimulatorMode(on: boolean): void {
  try {
    localStorage.setItem(SIMULATOR_KEY, on ? 'on' : 'off')
  } catch {
    // The choice still holds for this run, through the event below.
  }
  try {
    window.dispatchEvent(new CustomEvent(SIMULATOR_EVENT, { detail: on }))
  } catch {
    /* ignore */
  }
}

export type ArBlockedReason = 'device' | 'away'

export interface ArAccess {
  allowed: boolean
  reason: ArBlockedReason | null
  /** What to tell the visitor. Empty when they may go ahead. */
  message: string
}

export interface ArConditions {
  simulatorMode: boolean
  /** True when presence says the pilgrim is inside a parish's geofence. */
```

```
  atParish: boolean
  /** True on a handset that can run the companion AR application. */
  android: boolean
}

/**
 * Whether the AR tour may be opened, and what to say when it may not.
 *
 * The device is checked before the location. Telling somebody on an
 * iPhone to walk to the church would send them on a journey that cannot
 * help them when they arrive.
 */
export function arAccess({ simulatorMode, atParish, android }: ArConditions): ArAccess {
  if (!android) {
    return {
      allowed: false,
      reason: 'device',
      message:
        'The AR tour runs on an Android phone. Everything else in '
        + 'SanctiWalk works here, including the scanner.',
    }
  }

  if (simulatorMode && !atParish) {
    return {
      allowed: false,
      reason: 'away',
      message:
        'The AR tour opens inside the parish. It places the parish’s '
        + 'own statues where they really stand, so it needs you to be '
        + 'there. The scanner below works anywhere.',
    }
  }

  return { allowed: true, reason: null, message: '' }
}
```

1.4.68 APP-138 — speech.ts

Code ID	APP-138
File Name	speech.ts
File Path	SanctDemo/src/lib/speech.ts
Language / Type	TypeScript
Lines	227
Functions / Classes	Engine (exported type); speechLocale (exported function); webSpeechUsable (exported function); chooseEngine (exported function); isNativeApp (exported function); currentEngine (exported function); voiceFor (exported function); canSpeak (exported function); speakAloud (exported function); stopAloud (exported function)
Related Files (its imports)	./language
Where It Is Used	src/components/DailyRosary.tsx, src/lib/googleAuth.ts, src/lib/useNarration.ts
How to Recognize It	Open SanctDemo/src/lib/speech.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — speech.ts

```
import { Capacitor } from '@capacitor/core'
import { TextToSpeech } from '@capacitor-community/text-to-speech'
import type { Language } from './language'

/**
 * Reading a station aloud, on a phone as well as in a browser.
 *
 * ## The bug this exists to fix
 *
 * Narration was `window.speechSynthesis` and nothing else. That works
 * in Chrome and is silent in the installed app, because Capacitor runs
 * the app inside Android System WebView, which has never implemented
 * the Web Speech API - a gap open in Chromium's tracker for years.
 *
 * The failure was the bad kind. WebView exposes `speechSynthesis` as a
 * stub often enough that `"speechSynthesis" in window` returns true, so
 * the app offered a narration button, the pilgrim pressed it, and
 * nothing happened. An audio tour guide that is silent on the phone it
 * was installed on, while appearing to work, is worse than one that
 * says it cannot speak here.
 *
 * ## What it does instead
 *
 * Inside the installed app it speaks through Android's own
 * text-to-speech engine, which is what every other Android application
 * uses and which works without a network once a voice is installed. In
 * a browser it keeps using Web Speech. Where neither can make a sound
 * it reports itself unavailable, and the narration button goes away
 * rather than lying.
 */
```

```
export type Engine = 'native' | 'web' | 'none'

/**
 * The voice to ask for.
 *
 * `fil-PH` rather than a bare `fil`: Android matches voices by full
 * BCP-47 tag, and the Filipino voice on a Philippine handset is
 * registered against the region.
 */
export function speechLocale(language: Language): string {
  return language === 'fil' ? 'fil-PH' : 'en-US'
}

/**
 * Whether the browser's own speech can actually be heard.
 *
 * Presence of the object is not the test - that is exactly what WebView
 * gets wrong. An engine with no voices speaks nothing, so an empty
 * voice list counts as unusable.
 */
export function webSpeechUsable(win: Window | undefined): boolean {
  const synth = (win as (Window & { speechSynthesis?: SpeechSynthesis }) | undefined)
    ?.speechSynthesis
  if (!synth || typeof synth.getVoices !== 'function') return false
  try {
    return synth.getVoices().length > 0
  } catch {
    return false
  }
}

/** Which engine to use. Native wins on a phone; it is the only one there. */
export function chooseEngine(
  { native, webUsable }: { native: boolean; webUsable: boolean },
): Engine {
  if (native) return 'native'
  return webUsable ? 'web' : 'none'
}

/** True inside the installed Android or iOS app, false in a browser. */
export function isNativeApp(): boolean {
  try {
    return Capacitor.isNativePlatform()
  } catch {
    return false
  }
}

/**
 * The engine for this run.
 *
 * Voices arrive asynchronously in some browsers, so this is read at the
 * point of speaking rather than cached at module load - a cached "none"
 * taken before the voice list populated would silence the web build for
 * the whole session.
 */
export function currentEngine(): Engine {
  return chooseEngine({
```

```
    native: isNativeApp(),
    webUsable: typeof window !== 'undefined' && webSpeechUsable(window),
  })
}

/**
 * The installed voice that actually speaks this language, or null.
 *
 * ## Why setting `lang` is not enough
 *
 * `utterance.lang` is a REQUEST. When no installed voice matches it the
 * engine does not fail and does not fall silent - it reads the text in
 * whatever its default voice is, which on almost every machine is
 * English. So Tagalog prayers came out in an English voice, pronounced
 * as though they were English words, with nothing anywhere saying why.
 *
 * Picking the voice object explicitly is the only way to be sure, and
 * returning null when there is none is what lets the caller say so
 * instead of quietly doing the wrong thing.
 *
 * Filipino is matched on both `fil` and `tl`: the two tags are the same
 * language and engines disagree about which to publish.
 */
export function voiceFor(
  language: Language,
  win: Window | undefined = typeof window === 'undefined' ? undefined : window,
): SpeechSynthesisVoice | null {
  const synth = (win as (Window & { speechSynthesis?: SpeechSynthesis }) | undefined)?.speechSynthesis
  if (!synth || typeof synth.getVoices !== 'function') return null

  let voices: SpeechSynthesisVoice[]
  try {
    voices = synth.getVoices()
  } catch {
    return null
  }
  if (!voices || voices.length === 0) return null

  const wanted = language === 'fil' ? /^(fil|tl)\b/i : /^en\b/i
  const tagOf = (v: SpeechSynthesisVoice) => (v.lang ?? '').replace('_', '-')

  // An exact regional match first, then the bare language. A Philippine
  // Filipino voice is preferred over any other Filipino voice, but any
  // Filipino voice beats falling back to English.
  const exact = language === 'fil' ? 'fil-ph' : 'en-us'
  return (
    voices.find(v => tagOf(v).toLowerCase() === exact) ??
    voices.find(v => wanted.test(tagOf(v))) ??
    null
  )
}

/**
 * Whether this device can actually speak this language.
 *
 * The UI uses it to tell somebody their phone has no Filipino voice
 * rather than reading Tagalog to them in English and leaving them to
 * work out what went wrong.
 */
```

```
export function canSpeak(
  language: Language,
  win: Window | undefined = typeof window === 'undefined' ? undefined : window,
): boolean {
  if (isNativeApp()) return true // the device engine decides; asked at speak time
  return voiceFor(language, win) !== null
}

/** A shade under natural pace; the default rattles through a reflection. */
const RATE = 0.95

export async function speakAloud(text: string, language: Language): Promise<void> {
  const engine = currentEngine()
  if (engine === 'none' || !text.trim()) return

  if (engine === 'native') {
    // Stop first. The plugin queues otherwise, and the pilgrim waits out
    // the previous station before hearing this one.
    await stopAloud()
    await TextToSpeech.speak({
      text,
      lang: speechLocale(language),
      rate: RATE,
      // Read through the media channel so the phone's volume keys do
      // what the pilgrim expects while the tour is talking.
      category: 'playback',
    })
    return
  }

  window.speechSynthesis.cancel()
  const utterance = new SpeechSynthesisUtterance(text)
  utterance.lang = speechLocale(language)
  // The voice object, not only the tag. Without this the engine reads
  // Tagalog in whatever its default voice is, which is English.
  const picked = voiceFor(language)
  if (picked) utterance.voice = picked
  utterance.rate = RATE

  // Resolve when the voice actually stops, not when it starts.
  //
  // `speechSynthesis.speak` is fire-and-forget: it returns the instant
  // the utterance is queued. Callers await this to know when to put a
  // speaker button back to "play", so returning early told them the
  // reading had finished before a word of it was spoken - the button
  // flipped straight back and a second tap started a second voice over
  // the first instead of stopping it. The native branch above already
  // resolves on completion; this makes the web branch keep the same
  // promise.
  //
  // `cancel()` fires `end` too, so stopping resolves this as well.
  await new Promise<void>(resolve => {
    let settled = false
    const done = () => {
      if (settled) return
      settled = true
      resolve()
    }
    utterance.onend = done
```

```
    // An engine that fails mid-sentence must not leave the caller
    // waiting forever on a promise that will never settle.
    utterance.onerror = done
    window.speechSynthesis.speak(utterance)
  })
}

export async function stopAloud(): Promise<void> {
  if (isNativeApp()) {
    try {
      await TextToSpeech.stop()
    } catch {
      // Nothing was speaking. Stopping silence is not an error.
    }
    return
  }
  if (typeof window !== 'undefined' && 'speechSynthesis' in window) {
    window.speechSynthesis.cancel()
  }
}
```

1.4.69 APP-139 — spotlight.ts

Code ID	APP-139
File Name	spotlight.ts
File Path	SanctDemo/src/lib/spotlight.ts
Language / Type	TypeScript
Lines	137
Functions / Classes	Rect (exported type); Viewport (exported type); HOLE_PADDING (exported constant); CARD_GAP (exported constant); SCREEN_MARGIN (exported constant); ARROW_INSET (exported constant); Placement (exported type); clamp (internal function); holeFor (exported function); placeSpotlight (exported function); isUsableTarget (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/spotlight.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Where to put a spotlight cut-out and the card that explains it.
>
> ## Why this is separate from the component
>
> Placing a callout next to a highlighted button is almost entirely arithmetic, and all of it is wrong at least once: the card runs off the bottom when the target is the tab bar, off the top when it is the header, off the side when the target is in a corner, and the little arrow ends up pointing at nothing. None of that needs a browser to get right, and none of it is testable once it is tangled up in a render.
>
> Everything here takes plain rectangles and returns plain numbers.

Complete Source Code — spotlight.ts

```
/**
 * Where to put a spotlight cut-out and the card that explains it.
 *
 * ## Why this is separate from the component
 *
 * Placing a callout next to a highlighted button is almost entirely
 * arithmetic, and all of it is wrong at least once: the card runs off
 * the bottom when the target is the tab bar, off the top when it is the
 * header, off the side when the target is in a corner, and the little
 * arrow ends up pointing at nothing. None of that needs a browser to
 * get right, and none of it is testable once it is tangled up in a
 * render.
 *
 * Everything here takes plain rectangles and returns plain numbers.
 */
```

```
export interface Rect {
  top: number;
  left: number;
  width: number;
  height: number;
}

export interface Viewport {
  width: number;
  height: number;
}

/** Breathing room between the cut-out and the thing it is cutting around. */
export const HOLE_PADDING = 8;

/** Gap between the cut-out and the card. */
export const CARD_GAP = 14;

/** Closest the card may come to the edge of the screen. */
export const SCREEN_MARGIN = 12;

/** The arrow's width, used to keep it inside the card's rounded corners. */
export const ARROW_INSET = 22;

export interface Placement {
  /** The hole, already padded. */
  hole: Rect;
  /** Where the card goes. */
  card: { top: number; left: number; width: number };
  /** Which side of the card the arrow sits on. */
  arrow: "top" | "bottom" | "none";
  /** How far along the card's edge the arrow points, in px from its left. */
  arrowOffset: number;
}

function clamp(value: number, min: number, max: number): number {
  return Math.min(Math.max(value, min), max);
}

/**
 * Grows the target rect by the padding, clipped to the screen.
 *
 * Clipped because a target flush against an edge — the tab bar is — would
 * otherwise produce a hole hanging off the screen, and the dimming
 * around it stops looking like a hole and starts looking like a seam.
 */
export function holeFor(target: Rect, viewport: Viewport): Rect {
  const top = Math.max(0, target.top - HOLE_PADDING);
  const left = Math.max(0, target.left - HOLE_PADDING);
  const right = Math.min(viewport.width, target.left + target.width + HOLE_PADDING);
  const bottom = Math.min(viewport.height, target.top + target.height + HOLE_PADDING);
  return { top, left, width: right - left, height: bottom - top };
}

/**
 * Works out where the card goes relative to the hole.
 *
 * Below the target when it fits, above when it does not. Preferring
 * below is not arbitrary: most of what gets highlighted here is in the
```

```
 * upper half of the screen, and a card under the target leaves the
 * target itself unobscured by the hand holding the phone.
 *
 * When neither side fits — a target taller than the gap on both sides,
 * which the full-screen map is — the card is pinned to the bottom and
 * loses its arrow rather than pointing at something it is covering.
 */
export function placeSpotlight(
  target: Rect,
  viewport: Viewport,
  cardHeight: number,
): Placement {
  const hole = holeFor(target, viewport);

  const width = Math.min(viewport.width - SCREEN_MARGIN * 2, 360);
  const left = clamp(
    hole.left + hole.width / 2 - width / 2,
    SCREEN_MARGIN,
    Math.max(SCREEN_MARGIN, viewport.width - width - SCREEN_MARGIN),
  );

  const below = hole.top + hole.height + CARD_GAP;
  const above = hole.top - CARD_GAP - cardHeight;

  let top: number;
  let arrow: Placement["arrow"];

  if (below + cardHeight + SCREEN_MARGIN <= viewport.height) {
    top = below;
    arrow = "top"; // the arrow is on the card's top edge, pointing up
  } else if (above >= SCREEN_MARGIN) {
    top = above;
    arrow = "bottom";
  } else {
    top = Math.max(SCREEN_MARGIN, viewport.height - cardHeight - SCREEN_MARGIN);
    arrow = "none";
  }

  // Pointing at the centre of the hole, but never so close to a corner
  // that it collides with the card's radius.
  const arrowOffset = clamp(
    hole.left + hole.width / 2 - left,
    ARROW_INSET,
    Math.max(ARROW_INSET, width - ARROW_INSET),
  );

  return { hole, card: { top, left, width }, arrow, arrowOffset };
}

/**
 * True when the element is actually on screen.
 *
 * A target that is scrolled out of view, or hidden because its tab is
 * not the active one, still has a rect — an empty one at the origin.
 * Spotlighting that puts a hole in the top-left corner and a card
 * pointing at nothing, which is how a tutorial step looks broken.
 */
export function isUsableTarget(rect: Rect | null | undefined): boolean {
  if (!rect) return false;
```

```
  if (rect.width <= 1 || rect.height <= 1) return false;
  return true;
}
```

1.4.70 APP-140 — sundayMasses.ts

Code ID	APP-140
File Name	sundayMasses.ts
File Path	SanctDemo/src/lib/sundayMasses.ts
Language / Type	TypeScript
Lines	199
Functions / Classes	MassTime (exported type); SundayMassCard (exported type); nextSunday (exported function); saturdayBefore (exported function); anticipatedTime (exported function); sundayMassCard (exported function); dayOfMonth (exported function); MassDay (exported type); weekAhead (exported function)
Related Files (its imports)	./schedule
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/sundayMasses.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — sundayMasses.ts

```
import { isTimeAvailable, parseTimes, type MassScheduleEntry } from './schedule'

/**
 * The Sunday Mass card's data: the anticipated Mass and the Sunday Masses,
 * with the real dates each falls on.
 *
 * The mockup carries "DATE 00" placeholders next to both. Those are not
 * decoration — a pilgrim looking at a Mass card on a Tuesday needs to know
 * WHICH Sunday it is describing, and a card that says "Sunday Masses" with no
 * date is ambiguous for six days out of seven.
 *
 * Everything here is derived from the parish's own schedule in data.ts (or an
 * admin's override), never hardcoded, so all 31 parishes get a correct card
 * from the moment someone enters their times.
 */

/**
 * One Mass, and whether it is actually being celebrated.
 *
 * A suspended Mass stays in this list rather than being filtered out.
 * Dropping it would leave a parishioner who comes every week at 8:00
 * seeing no 8:00 Mass at all, which reads as "I misremembered" rather
 * than "the parish has called it off this week".
 */
export interface MassTime {
  time: string
  available: boolean
}
```

```
export interface SundayMassCard {
  /** The Saturday evening Mass that fulfils the Sunday obligation. */
  anticipated: (MassTime & { date: Date }) | null
  /** Every Mass on the Sunday itself, in the order the parish lists them. */
  sunday: { times: MassTime[]; date: Date }
  /** True when this parish's times are placeholders, not parish-confirmed. */
  unverified: boolean
  /** The parish's own note for the Sunday, if they have written one. */
  note?: string
}

/** The next Sunday on or after `from`. Sunday itself counts as today. */
export function nextSunday(from: Date): Date {
  const date = new Date(from)
  date.setHours(0, 0, 0, 0)
  date.setDate(date.getDate() + ((7 - date.getDay()) % 7))
  return date
}

/**
 * The Saturday belonging to a given Sunday — the day before it.
 *
 * Not "the next Saturday": on a Sunday those are six days apart, and the
 * anticipated Mass shown beside today's Sunday Masses must be the one that
 * has just happened, not one a week out.
 */
export function saturdayBefore(sunday: Date): Date {
  const date = new Date(sunday)
  date.setDate(date.getDate() - 1)
  return date
}

/**
 * The LAST time listed on Saturday — the anticipated (vigil) Mass.
 *
 * A parish that has both a 6:00 AM and a 6:00 PM Saturday Mass lists them in
 * that order, and only the evening one fulfils the Sunday obligation. Taking
 * the first would put a 6 AM weekday Mass under the heading "Anticipated",
 * which is wrong in a way a parishioner would notice immediately.
 *
 * Returns null when the parish lists no Saturday Mass at all, rather than
 * inventing one.
 */
export function anticipatedTime(schedule: MassScheduleEntry[]): string | null {
  const saturday = schedule.find(entry => entry.day === 'Saturday')
  if (!saturday) return null

  const times = parseTimes(saturday.time)
  if (times.length === 0) return null

  const evening = times.filter(t => /pm/i.test(t))
  // Prefer the last evening Mass; fall back to the last listed time only when
  // the parish lists no evening Mass, and let the caller's own labelling make
  // clear what it is.
  return (evening.length > 0 ? evening : times)[Math.max(0, (evening.length > 0 ? evening : times).length - 1)]
}

export function sundayMassCard(
  schedule: MassScheduleEntry[],
```

```
  now: Date,
  { verified = true }: { verified?: boolean } = {},
): SundayMassCard {
  const sunday = nextSunday(now)
  const anticipated = anticipatedTime(schedule)
  const saturdayEntry = schedule.find(entry => entry.day === 'Saturday')
  const sundayEntry = schedule.find(entry => entry.day === 'Sunday')

  return {
    anticipated: anticipated
      ? {
          time: anticipated,
          available: saturdayEntry ? isTimeAvailable(saturdayEntry, anticipated) : true,
          date: saturdayBefore(sunday),
        }
      : null,
    sunday: {
      times: sundayEntry
        ? parseTimes(sundayEntry.time).map(time => ({
            time,
            available: isTimeAvailable(sundayEntry, time),
          }))
        : [],
      date: sunday,
    },
    unverified: !verified,
    ...(sundayEntry?.note ? { note: sundayEntry.note } : {}),
  }
}

/** "07" — the day of the month, zero-padded, as the mockup's "DATE 00" slot. */
export function dayOfMonth(date: Date): string {
  return String(date.getDate()).padStart(2, '0')
}

/** One day of the week ahead, with the date it falls on. */
export interface MassDay {
  /** "Monday" — as the parish wrote it. */
  day: string
  date: Date
  times: MassTime[]
  /** The parish's own note for this day, if they wrote one. */
  note?: string
  /** True for the day the pilgrim is actually standing in. */
  isToday: boolean
}

const DAY_NAMES = [
  'Sunday',
  'Monday',
  'Tuesday',
  'Wednesday',
  'Thursday',
  'Friday',
  'Saturday',
]

/**
 * The next seven days, today first.
```

```
 *
 * ## Why the card needed this
 *
 * The Mass card showed the anticipated Mass and Sunday, and nothing
 * else. Every parish in the data also lists weekday Masses - Mary Help
 * has one every morning and an evening Mass from Wednesday - and none
 * of them appeared anywhere in the app. Somebody looking for Tuesday's
 * 6:00 AM was told, in effect, that it did not exist.
 *
 * ## Why it starts today rather than on Monday
 *
 * A calendar week is an administrator's unit. A pilgrim's question is
 * "when is the next Mass I can get to", and the answer is almost always
 * today or tomorrow. Starting on Monday buries today's remaining Masses
 * four rows down on a Thursday.
 *
 * ## Why days with no Mass are kept
 *
 * A parish with no Monday Mass is answering the question. Dropping the
 * row leaves the reader to notice an absence between Sunday and
 * Tuesday, which is the kind of thing people get wrong when they are
 * deciding whether to leave the house.
 */
export function weekAhead(schedule: MassScheduleEntry[], now: Date): MassDay[] {
  const start = new Date(now)
  start.setHours(0, 0, 0, 0)

  const days: MassDay[] = []

  for (let offset = 0; offset < 7; offset++) {
    const date = new Date(start)
    date.setDate(date.getDate() + offset)

    const name = DAY_NAMES[date.getDay()]
    const entry = schedule.find(e => e.day === name)

    days.push({
      day: name,
      date,
      times: entry
        ? parseTimes(entry.time).map(time => ({
            time,
            available: isTimeAvailable(entry, time),
          }))
        : [],
      ...(entry?.note ? { note: entry.note } : {}),
      isToday: offset === 0,
    })
  }

  return days
}
```

travelModes.ts — listed in full as MAP-006.

1.4.71 APP-141 — tutorial.ts

Code ID	APP-141
File Name	tutorial.ts
File Path	SanctDemo/src/lib/tutorial.ts
Language / Type	TypeScript
Lines	151
Functions / Classes	TutorialTab (exported type); TutorialStep (exported type); TUTORIAL_STEPS (exported constant); hasSeenTutorial (exported function); markTutorialSeen (exported function); forgetTutorial (exported function); nextUsableStep (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/tutorial.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The first-run walkthrough, as data.
>
> ## Why the steps are a list and not a component each
>
> Ten steps written as ten pieces of JSX is ten places to change the wording, ten chances for one card to look unlike the others, and no way to answer "how many steps are there" without counting them by hand. As data, the progress dots, the Back button's bounds and the skipping below all fall out of the array's length.
>
> ## Why a step can name a tab
>
> Half of what the tutorial points at is not on the screen it starts on. A step that needs the Me tab says so, and the runner switches before it looks for the target - rather than the tutorial quietly failing on step eight because the thing it wants is one tap away.

Complete Source Code — tutorial.ts

```
/**
 * The first-run walkthrough, as data.
 *
 * ## Why the steps are a list and not a component each
 *
 * Ten steps written as ten pieces of JSX is ten places to change the
 * wording, ten chances for one card to look unlike the others, and no
 * way to answer "how many steps are there" without counting them by
 * hand. As data, the progress dots, the Back button's bounds and the
 * skipping below all fall out of the array's length.
 *
```

```
 * ## Why a step can name a tab
 *
 * Half of what the tutorial points at is not on the screen it starts
 * on. A step that needs the Me tab says so, and the runner switches
 * before it looks for the target - rather than the tutorial quietly
 * failing on step eight because the thing it wants is one tap away.
 */

export type TutorialTab = "home" | "navigator" | "ar" | "rosary" | "me";

export interface TutorialStep {
  /** The `data-spotlight` name to put the hole around. */
  target: string;
  title: string;
  body: string;
  /** Switch here first. Omitted means stay where we are. */
  tab?: TutorialTab;
}

export const TUTORIAL_STEPS: TutorialStep[] = [
  {
    target: "pick-church",
    tab: "home",
    title: "Pick a church",
    body: "Tap here to search any parish in the diocese. Everything else on this screen then belongs to the church you picked.",
  },
  {
    target: "tab-home",
    tab: "home",
    title: "Home",
    body: "Your parish's front page — the verse of the day, what the office has announced, and the next Mass.",
  },
  {
    target: "tab-map",
    tab: "home",
    title: "Map",
    body: "Every parish on a map, with walking directions. Plan a Visita Iglesia and it will measure the walk between stops.",
  },
  {
    target: "tab-scan",
    tab: "home",
    title: "Scan",
    body: "Point your camera at a statue, an image or a marker and the app will tell you what it is. The AR tour starts from here too.",
  },
  {
    target: "church-info",
    tab: "home",
    title: "Church information",
    body: "The parish's own history and story, written by the parish office rather than by us.",
  },
  {
    target: "mass-schedule",
    tab: "home",
    title: "Mass schedule",
    body: "The times the parish keeps, updated by the office. A Mass they have suspended stays listed and says so, so nobody turns up for one that is off.",
  },
  {
    target: "sacraments",
    tab: "home",
```

```
    title: "Sacraments",
    body: "Baptism, matrimony, confirmation — the requirements, the schedule, and how to apply.",
  },
  {
    target: "ministries",
    tab: "home",
    title: "Ministries",
    body: "The parish's lay ministries, what each one does, and how to join.",
  },
  {
    target: "sancti-button",
    tab: "home",
    title: "Ask Sancti",
    body: "Your guide. Ask for anything in plain words — “what time is Mass?”, “take me to Mary Help” — and Sancti will answer or take you there.",
  },
  {
    target: "tab-me",
    tab: "home",
    title: "Reminders",
    body: "Under Me you can be reminded before Mass, on feast days, and when the parish decides on your application. You can replay this tour from there too.",
  },
];

const SEEN_KEY = "sanctiwalk.tutorialSeen";

/**
 * Whether the walkthrough has already run on this phone.
 *
 * Device-local, not on the account: the tour is about finding your way
 * round THIS screen, and someone who has used the app for a year should
 * still get it when they pick up a new phone.
 *
 * Read defensively - a blocked localStorage throws rather than
 * returning null, and the honest fallback there is "treat it as seen".
 * Showing a full-screen tutorial on every single launch, to someone who
 * cannot dismiss it permanently, is far worse than never showing it.
 */
export function hasSeenTutorial(): boolean {
  try {
    return localStorage.getItem(SEEN_KEY) === "yes";
  } catch {
    return true;
  }
}

export function markTutorialSeen(): void {
  try {
    localStorage.setItem(SEEN_KEY, "yes");
  } catch {
    // Nothing to do. It will offer itself again next launch.
  }
}

export function forgetTutorial(): void {
  try {
    localStorage.removeItem(SEEN_KEY);
  } catch {
    // Replaying works from the Settings button regardless.
  }
```

```
}

/**
 * The next step whose target is actually on screen.
 *
 * Steps are skipped rather than stalled on. A target can be missing
 * for honest reasons — a parish with no ministries listed has no
 * ministries card — and a tutorial that freezes on step eight is worse
 * than one that is nine steps long that day. `direction` is +1 going
 * forward and -1 going back, so Back skips the same gaps Next did.
 */
export function nextUsableStep(
  from: number,
  direction: 1 | -1,
  isPresent: (target: string) => boolean,
  steps: TutorialStep[] = TUTORIAL_STEPS,
): number | null {
  for (let i = from; i >= 0 && i < steps.length; i += direction) {
    if (isPresent(steps[i]!.target)) return i;
  }
  return null;
}
```

1.4.72 APP-142 — ui.ts

Code ID	APP-142
File Name	ui.ts
File Path	SanctDemo/src/lib/ui.ts
Language / Type	TypeScript
Lines	147
Functions / Classes	UiKey (exported type); t (exported function); translator (exported function); UI_KEYS (exported constant)
Related Files (its imports)	./language
Where It Is Used	src/components/Dashboard.tsx, src/components/HomeQuickLinks.tsx, src/components/MeTab.tsx, src/components/MinistrySacramentPair.tsx, src/components/ParishWelcomeHeader.tsx, src/components/ReminderSettingsCard.tsx, src/components/SanctiSheet.tsx
How to Recognize It	Open SanctDemo/src/lib/ui.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ui.ts

```
import type { Language } from "./language";

/**
 * The app's own words, in both languages.
 *
 * ## Why a table and not a library
 *
 * i18next and its relatives bring plurals, interpolation, namespaces
 * and lazy-loaded bundles. SanctiWalk has two languages, one of them
 * default, and a few hundred short labels that all ship with the app —
 * none of that machinery would ever be used, and all of it would be in
 * the bundle a pilgrim downloads over parish wifi.
 *
 * A frozen object and a lookup is the whole requirement. TypeScript
 * makes the key set exhaustive, so a label added in English without its
 * Tagalog fails the build rather than appearing untranslated on
 * somebody's phone.
 *
 * ## What belongs here and what does not
 *
 * Interface labels: tabs, headings, buttons, the words the app says
 * about itself. NOT the parish's content — descriptions, Mass times,
 * announcements and histories come from data.ts or from Firestore and
 * are handled by contentTranslations.ts, because translating what a
 * parish wrote is a different act from translating a button.
 */

const STRINGS = {
  // ---- the tab bar ----
```

```
"tab.home": { en: "Home", fil: "Bahay" },
"tab.map": { en: "Map", fil: "Mapa" },
"tab.scan": { en: "Scan", fil: "I-scan" },
"tab.pray": { en: "Pray", fil: "Dasal" },
"tab.me": { en: "Me", fil: "Ako" },

// ---- Home ----
"home.welcome": { en: "Welcome", fil: "Maligayang pagdating" },
"home.bulletin": { en: "Parish Bulletin", fil: "Patalastas ng Parokya" },
"home.verseOfTheDay": { en: "Verse of the Day", fil: "Talata ng Araw" },
"home.massSchedule": { en: "Mass Schedule", fil: "Oras ng Misa" },
"home.churchHistory": { en: "Church History", fil: "Kasaysayan ng Simbahan" },
"home.commemorates": { en: "Commemorates", fil: "Ginugunita" },
"home.learnMore": { en: "Learn more", fil: "Alamin pa" },
"home.backToMyParish": { en: "Back to my parish", fil: "Balik sa aking parokya" },
"home.nothingThisWeek": {
  en: "Nothing from the parish office this week. Announcements, feast days and schedule changes appear here first.",
  fil: "Walang bago mula sa tanggapan ng parokya ngayong linggo. Dito unang lalabas ang mga patalastas, kapistahan at pagbabago sa oras ng Misa.",
},

// ---- the quick buttons ----
"quick.ministries": { en: "Ministries", fil: "Mga Ministeryo" },
"quick.sacraments": { en: "Sacraments", fil: "Mga Sakramento" },
"quick.mass": { en: "Mass Schedule", fil: "Oras ng Misa" },
"quick.history": { en: "History", fil: "Kasaysayan" },
"quick.heading": { en: "What are you looking for?", fil: "Ano ang hinahanap mo?" },

// ---- Me ----
"me.title": { en: "Pilgrim Profile", fil: "Profile ng Peregrino" },
"me.account": { en: "Account", fil: "Account" },
"me.signIn": { en: "Sign in", fil: "Mag-sign in" },
"me.signOut": { en: "Sign out", fil: "Mag-sign out" },
"me.myApplications": { en: "My Applications", fil: "Aking mga Aplikasyon" },
"me.notifications": { en: "Notifications", fil: "Mga Abiso" },
"me.nothingNew": { en: "Nothing new", fil: "Walang bago" },
"me.unread": { en: "unread", fil: "hindi pa nababasa" },
"me.settings": { en: "Settings", fil: "Mga Setting" },
"me.language": { en: "Language", fil: "Wika" },
"me.languageHelp": {
  en: "Everything in SanctiWalk is shown in this language, including the station and ministry descriptions.",
  fil: "Lahat ng nasa SanctiWalk ay ipapakita sa wikang ito, pati na ang mga paglalarawan ng istasyon at ministeryo.",
},
"me.changeParish": { en: "Change Parish", fil: "Palitan ang Parokya" },
"me.rosarySettings": { en: "Rosary Settings", fil: "Setting ng Rosaryo" },
"me.replayTutorial": { en: "Show me around again", fil: "Ipakita muli ang gabay" },
"me.helpAndGuide": { en: "Help & guide", fil: "Tulong at gabay" },
"me.simulator": { en: "Simulator mode", fil: "Simulator mode" },
"me.simulatorOn": { en: "On", fil: "Naka-on" },
"me.simulatorOff": { en: "Off", fil: "Naka-off" },
"me.simulatorOnHelp": {
  en: "On. The app behaves as it will for a visitor: the AR tour opens only inside the parish. The scanner still works anywhere.",
  fil: "Naka-on. Kumikilos ang app gaya ng para sa bisita: bubukas lang ang AR tour sa loob ng parokya. Gumagana pa rin ang scanner kahit saan.",
},
"me.simulatorOffHelp": {
  en: "Off. Everything is reachable from anywhere, for testing away from the churches.",
  fil: "Naka-off. Maaabot ang lahat kahit saan, para sa pagsubok na malayo sa mga simbahan.",
},

// ---- reminders ----
```

```
  "rem.title": { en: "Reminders", fil: "Mga Paalala" },
  "rem.beforeMass": { en: "Before Mass", fil: "Bago ang Misa" },
  "rem.beforeMassHint": { en: "Your parish's Mass times.", fil: "Ang oras ng Misa sa iyong parokya." },
  "rem.howLongBefore": { en: "How long before?", fil: "Gaano katagal bago mag-Misa?" },
  "rem.feastDays": { en: "Feast days", fil: "Mga Kapistahan" },
  "rem.feastHint": { en: "The evening before, and on the day.", fil: "Gabi bago ang araw, at sa mismong araw." },
  "rem.events": { en: "Parish events", fil: "Mga Gawain ng Parokya" },
  "rem.announcements": { en: "Parish announcements", fil: "Mga Patalastas ng Parokya" },
  "rem.applications": { en: "My applications", fil: "Aking mga aplikasyon" },
  "rem.applicationsHint": { en: "When the parish decides on one.", fil: "Kapag may pasya na ang parokya." },
  "rem.otherParishes": { en: "Other parishes", fil: "Ibang mga Parokya" },
  "rem.otherParishesHint": {
    en: "Be reminded before Mass at another parish as well as your own.",
    fil: "Maaari ka ring paalalahanan bago ang Misa sa ibang parokya bukod sa iyong sarili.",
  },
  "rem.turnOn": { en: "Turn on reminders", fil: "Buksan ang mga paalala" },
  "rem.test": { en: "Send a test reminder", fil: "Magpadala ng pansubok na paalala" },
  "rem.testSent": { en: "Sent — check your notifications", fil: "Naipadala — tingnan ang iyong mga abiso" },

  // ---- Sancti ----
  "sancti.ask": { en: "Ask Sancti", fil: "Tanungin si Sancti" },
  "sancti.subtitle": { en: "Your guide around", fil: "Iyong gabay sa" },
  "sancti.placeholder": { en: "Ask Sancti…", fil: "Magtanong kay Sancti…" },
  "sancti.greeting": {
    en: "Good day! Ask me anything about the parish — where things are, when Mass is, or how to apply for a sacrament.",
    fil: "Magandang araw! Itanong mo sa akin ang anuman tungkol sa parokya — kung nasaan ang mga bagay, kailan ang Misa, o paano mag-apply para sa isang
sakramento.",
  },
  "sancti.tryThese": { en: "Try one of these:", fil: "Subukan ang isa sa mga ito:" },

  // ---- shared words ----
  "common.back": { en: "Back", fil: "Bumalik" },
  "common.next": { en: "Next", fil: "Susunod" },
  "common.skip": { en: "Skip", fil: "Laktawan" },
  "common.finish": { en: "Finish", fil: "Tapusin" },
  "common.close": { en: "Close", fil: "Isara" },
  "common.search": { en: "Search", fil: "Maghanap" },
} as const satisfies Record<string, { en: string; fil: string }>;

export type UiKey = keyof typeof STRINGS;

/**
 * One interface label.
 *
 * Returns the key itself for anything unknown, which cannot happen
 * through the type but can through a cast. A visible "me.title" on
 * screen is reported in minutes; a silent empty string is not.
 */
export function t(key: UiKey, language: Language): string {
  const entry = STRINGS[key];
  if (!entry) return key;
  return entry[language] || entry.en;
}

/** A `t` already bound to a language, for components that use many. */
export function translator(language: Language): (key: UiKey) => string {
  return key => t(key, language);
}
```

```
export const UI_KEYS = Object.keys(STRINGS) as UiKey[];
```

1.4.73 APP-143 — useAuthUid.ts

Code ID	APP-143
File Name	useAuthUid.ts
File Path	SanctDemo/src/lib/useAuthUid.ts
Language / Type	TypeScript
Lines	19
Functions / Classes	useAuthUid (exported function)
Related Files (its imports)	./firebase
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/useAuthUid.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The signed-in user's id, as a value a component can re-render on.
>
> `auth.currentUser` is a snapshot: read it during render and the component never hears about signing in, so the scanner would keep showing "sign in to scan" to somebody who just had. This subscribes, which is the difference between a value and a reading.

Complete Source Code — useAuthUid.ts

```
/**
 * The signed-in user's id, as a value a component can re-render on.
 *
 * `auth.currentUser` is a snapshot: read it during render and the
 * component never hears about signing in, so the scanner would keep
 * showing "sign in to scan" to somebody who just had. This subscribes,
 * which is the difference between a value and a reading.
 */
import { useEffect, useState } from 'react'
import { onAuthStateChanged } from 'firebase/auth'
import { auth } from './firebase'

export function useAuthUid(): string | null {
  const [uid, setUid] = useState<string | null>(() => auth.currentUser?.uid ?? null)

  useEffect(() => onAuthStateChanged(auth, user => setUid(user?.uid ?? null)), [])

  return uid
}
```

1.4.74 APP-144 — useCamera.ts

Code ID	APP-144
File Name	useCamera.ts
File Path	SanctDemo/src/lib/useCamera.ts
Language / Type	TypeScript
Lines	322
Functions / Classes	CameraStatus (exported type); CapturedFrame (exported type); CaptureFailure (exported type); CaptureResult (exported type); captureFailed (exported function); isCameraSecureContext (exported function); insecureContextMessage (internal function); useCamera (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/useCamera.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useCamera.ts

```
import { useCallback, useEffect, useRef, useState } from "react";

/**
 * useCamera — owns the MediaStream lifecycle for the AR Tour.
 *
 * Returns a ref for a <video>, permission/error state, a front/back flip, and
 * captureFrame() which grabs the current frame as base64 JPEG for the vision
 * endpoint.
 *
 * Note the camera only works on a secure origin: https, or localhost. On a bare
 * LAN IP the browser refuses regardless of permission, which is the usual cause
 * of "it works on my laptop but not my phone".
 */

export type CameraStatus =
  | "idle"
  | "requesting"
  | "ready"
  | "denied"
  | "unavailable"
  | "insecure"
  | "in-use"
  | "error";

export interface CapturedFrame {
  base64: string;
  mimeType: string;
  width: number;
  height: number;
  /** Encoded size in bytes, before base64 inflation. */
```

```
  bytes: number;
  /** Mean luminance 0-255, used to catch a frame taken in the dark. */
  brightness: number;
}

/**
 * Why a capture failed. `captureFrame` used to return plain null, which the
 * caller could only translate into a single vague message — or, as it did,
 * into nothing at all.
 */
export type CaptureFailure =
  | "no-video"       // the <video> element is not mounted
  | "not-ready"      // stream is live but no frame has painted yet
  | "no-canvas"      // 2D context unavailable
  | "encode-failed"; // toDataURL produced nothing usable

export type CaptureResult =
  | { ok: true; frame: CapturedFrame }
  | { ok: false; reason: CaptureFailure };

/**
 * Narrows a CaptureResult to its failure branch.
 *
 * `if (!result.ok)` ought to be enough, and in a strict project it is. This
 * one compiles without `strictNullChecks`, and without it TypeScript does
 * not narrow a union by a boolean discriminant - so `result.reason` inside
 * that branch was an error, and the four distinct failure messages below
 * were unreachable as far as the compiler was concerned.
 *
 * A user-defined type guard narrows regardless of that setting. Turning
 * `strict` on across the project would be the real fix and is a bigger
 * decision than this file.
 */
export function captureFailed(
  result: CaptureResult,
): result is { ok: false; reason: CaptureFailure } {
  return !result.ok;
}

/**
 * True when getUserMedia can even be attempted here. False on a bare LAN IP
 * over plain HTTP (e.g. http://192.168.1.7:5173) — the browser hides
 * mediaDevices entirely rather than prompting and denying, so this has to be
 * checked before calling start(), not inferred from a caught error.
 */
export function isCameraSecureContext(): boolean {
  return typeof window !== "undefined" && window.isSecureContext && !!navigator.mediaDevices;
}

function insecureContextMessage(): string {
  const origin = typeof window !== "undefined" ? window.location.origin : "this address";
  return (
    `The camera needs a secure connection, and ${origin} is not one. ` +
    "Open the app's HTTPS address instead (ask whoever set this up for the " +
    "https:// link), or use the app's deployed URL. On a self-signed HTTPS " +
    "address your phone will show a one-time security warning — that is " +
    "expected, tap through it."
  );
}
```

```
export function useCamera({ autoStart = false }: { autoStart?: boolean } = {}) {
  const videoRef = useRef<HTMLVideoElement | null>(null);
  const canvasRef = useRef<HTMLCanvasElement | null>(null);
  const streamRef = useRef<MediaStream | null>(null);
  // Guards against getUserMedia resolving after teardown or a newer start().
  const startTokenRef = useRef(0);

  const [status, setStatus] = useState<CameraStatus>("idle");
  const [error, setError] = useState<string | null>(null);
  const [facingMode, setFacingMode] = useState<"environment" | "user">("environment");
  const [hasMultipleCameras, setHasMultipleCameras] = useState(false);

  const stop = useCallback(() => {
    startTokenRef.current++;
    streamRef.current?.getTracks().forEach((track) => track.stop());
    streamRef.current = null;
    if (videoRef.current) videoRef.current.srcObject = null;
    setStatus("idle");
  }, []);

  const start = useCallback(
    async (mode: "environment" | "user" = facingMode) => {
      if (!isCameraSecureContext() || !navigator.mediaDevices?.getUserMedia) {
        setStatus("insecure");
        setError(insecureContextMessage());
        return;
      }

      console.log(`[Scanner] Camera initialized: requesting ${mode} camera`);
      setStatus("requesting");
      setError(null);

      startTokenRef.current++;
      streamRef.current?.getTracks().forEach((track) => track.stop());
      streamRef.current = null;
      const token = startTokenRef.current;

      try {
        const stream = await navigator.mediaDevices.getUserMedia({
          // `ideal` rather than `exact`, so a laptop with one webcam still works.
          video: { facingMode: { ideal: mode }, width: { ideal: 1920 }, height: { ideal: 1080 } },
          audio: false,
        });

        if (token !== startTokenRef.current) {
          stream.getTracks().forEach((track) => track.stop());
          return;
        }

        streamRef.current = stream;

        // Deliberately NOT attaching the stream here.
        //
        // The <video> element only exists once status is "ready" — ArTour
        // returns a different screen for every other status. So at this point
        // videoRef.current is still null, the assignment would be skipped
        // silently, and the user would get a live camera behind a black
        // rectangle with no error to explain it. The attach happens in the
```

```
      // effect below, once React has actually mounted the element.
      console.log("[Scanner] Camera permission: granted");
      setStatus("ready");

      const devices = await navigator.mediaDevices.enumerateDevices();
      setHasMultipleCameras(devices.filter((d) => d.kind === "videoinput").length > 1);
    } catch (err) {
      if (token !== startTokenRef.current) return;
      const e = err as DOMException;
      console.error(`[Scanner] Scanner error: getUserMedia failed — ${e.name}: ${e.message}`);
      if (e.name === "NotAllowedError" || e.name === "SecurityError") {
        setStatus("denied");
        setError(
          "Camera permission was denied. Re-enable it from the camera icon in your " +
            "browser's address bar (or Settings → Site settings → Camera on mobile), " +
            "then try again.",
        );
      } else if (e.name === "NotFoundError" || e.name === "OverconstrainedError") {
        setStatus("unavailable");
        setError("No camera was found on this device.");
      } else if (e.name === "NotReadableError" || e.name === "TrackStartError") {
        setStatus("in-use");
        setError(
          "The camera is already in use by another app. Close any other camera app " +
            "(or browser tab using the camera) and try again.",
        );
      } else {
        setStatus("error");
        setError(e.message || "The camera could not be started.");
      }
    }
  },
  [facingMode],
);

const switchCamera = useCallback(() => {
  setFacingMode((current) => {
    const next = current === "environment" ? "user" : "environment";
    void start(next);
    return next;
  });
}, [start]);

// Attach the live stream once the <video> is actually on the page.
//
// This runs after the render that mounts the element, which is the whole
// point: start() acquires the stream while the video element does not yet
// exist, so the attach cannot happen there. Keyed on `status` so it fires
// exactly when the element appears, and re-runs after a camera switch.
useEffect(() => {
  const video = videoRef.current;
  const stream = streamRef.current;
  if (status !== "ready" || !video || !stream) return;

  if (video.srcObject !== stream) {
    video.srcObject = stream;
  }

  const track = stream.getVideoTracks()[0];
```

```
  const settings = track?.getSettings?.();
  console.log(
    `[Scanner] Camera is ready: ${settings?.width ?? "?"}x${settings?.height ?? "?"} ` +
      `facing ${settings?.facingMode ?? "?"}`,
  );

  // play() can reject if the browser wants a fresher gesture; the stream is
  // still live and the element will usually paint anyway, so this must never
  // throw into the UI.
  void video.play().catch(() => {});
}, [status]);

/**
 * The current frame as a base64 JPEG, with the checks that make a failure
 * explainable instead of silent.
 *
 * Capture width is 960, not the sensor's. A larger frame is not a better
 * answer: measured against the same photograph of a crowned Marian statue,
 * 640px produced "Statue of Our Lady with the Child Jesus" and 1280px
 * produced "Our Lady of the Rosary" — more specific, and wrong. Resolution
 * is not what limits recognition here, so 960 is chosen as headroom for a
 * genuinely distant subject rather than as an accuracy fix.
 */
const captureFrame = useCallback((): CaptureResult => {
  const video = videoRef.current;
  if (!video) {
    console.warn("[Scanner] Capturing image: no <video> element mounted");
    return { ok: false, reason: "no-video" };
  }

  // readyState < 2 means metadata may exist but no frame has been decoded,
  // so drawImage would paint a blank rectangle rather than the scene.
  if (!video.videoWidth || !video.videoHeight || video.readyState < 2) {
    console.warn(
      `[Scanner] Capturing image: video not ready (${video.videoWidth}x${video.videoHeight}, readyState ${video.readyState})`,
    );
    return { ok: false, reason: "not-ready" };
  }

  const canvas =
    canvasRef.current ?? (canvasRef.current = document.createElement("canvas"));

  const maxWidth = 960;
  const scale = Math.min(1, maxWidth / video.videoWidth);
  canvas.width = Math.round(video.videoWidth * scale);
  canvas.height = Math.round(video.videoHeight * scale);

  const ctx = canvas.getContext("2d", { willReadFrequently: true });
  if (!ctx) {
    console.error("[Scanner] Capturing image: no 2D context");
    return { ok: false, reason: "no-canvas" };
  }
  ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
  console.log(`[Scanner] Image dimensions: ${canvas.width}x${canvas.height} (source ${video.videoWidth}x${video.videoHeight})`);

  // Mean luminance over a sparse grid. Cheap, and it distinguishes "the
  // lens cap is on / the church is dark" from "the model could not tell",
  // which are the same message to a pilgrim otherwise.
  let brightness = 0;
```

```
    try {
      const sample = ctx.getImageData(0, 0, canvas.width, canvas.height).data;
      let total = 0;
      let count = 0;
      for (let i = 0; i < sample.length; i += 4 * 97) {
        total += 0.2126 * sample[i] + 0.7152 * sample[i + 1] + 0.0722 * sample[i + 2];
        count++;
      }
      brightness = count ? total / count : 0;
    } catch {
      // A tainted canvas cannot be read. Not fatal — brightness is advice,
      // not a gate.
      brightness = -1;
    }

    const dataUrl = canvas.toDataURL("image/jpeg", 0.8);
    const base64 = dataUrl.split(",")[1] ?? "";
    if (base64.length < 512) {
      console.error(`[Scanner] Image encoding: produced only ${base64.length} chars — treating as failed`);
      return { ok: false, reason: "encode-failed" };
    }

    const bytes = Math.round(base64.length * 0.75);
    console.log(`[Scanner] Encoded image size: ${(bytes / 1024).toFixed(0)} KB, brightness ${brightness.toFixed(0)}/255`);

    return {
      ok: true,
      frame: { base64, mimeType: "image/jpeg", width: canvas.width, height: canvas.height, bytes, brightness },
    };
  }, []);

  // Always release the camera when the tab unmounts — a live green dot after
  // leaving the AR tab reads as the app spying on you.
  useEffect(() => {
    if (autoStart) void start();
    return () => {
      startTokenRef.current++;
      streamRef.current?.getTracks().forEach((track) => track.stop());
      streamRef.current = null;
    };
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  return {
    videoRef,
    status,
    error,
    facingMode,
    hasMultipleCameras,
    isReady: status === "ready",
    start,
    stop,
    switchCamera,
    captureFrame,
  };
}
```

1.4.75 APP-145 — useDeviceHeading.ts

Code ID	APP-145
File Name	useDeviceHeading.ts
File Path	SanctDemo/src/lib/useDeviceHeading.ts
Language / Type	TypeScript
Lines	231
Functions / Classes	HeadingStatus (exported type); isHeadingSupported (exported function); headingNeedsPermission (exported function); readEventHeading (exported function); screenAngle (internal function); DeviceHeading (exported type); useDeviceHeading (exported function)
Related Files (its imports)	./heading
Where It Is Used	src/components/CompassControl.tsx, src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/lib/useDeviceHeading.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useDeviceHeading.ts

```
import { useCallback, useEffect, useRef, useState } from 'react'
import { DEFAULT_SMOOTHING_ALPHA, normalizeDegrees, smoothHeading } from './heading'

/**
 * useDeviceHeading — the device compass, as a smoothed bearing in degrees.
 *
 * This is the browser's stand-in for Unity's `Input.compass`, and it carries
 * three constraints that do not exist in a native Android build:
 *
 *  1. **It needs a secure origin.** On `http://<LAN-IP>` the orientation
 *     events simply never fire — no error, no prompt, nothing. Same rule the
 *     camera lives under (see useCamera.ts), and the same trap: it works on
 *     the laptop and looks broken on the phone.
 *  2. **iOS requires an explicit gesture.** `DeviceOrientationEvent
 *     .requestPermission()` only resolves from inside a real tap handler, so
 *     the compass cannot start itself — hence `requestPermission` being
 *     returned for a button to call.
 *  3. **Desktops have no magnetometer.** A laptop reports `unavailable`
 *     forever. That is the expected state in a dev browser, not a bug, and
 *     everything downstream must render without a heading.
 *
 * What comes back is *magnetic* north, not true north. In Metro Manila the
 * declination is well under a degree, far below this sensor's own error, so
 * no correction is applied — worth knowing before anyone ports this further
 * afield, where the gap reaches tens of degrees.
 */

export type HeadingStatus =
  | 'unsupported' // no DeviceOrientationEvent at all
```

```
  | 'insecure' // not a secure origin, so events would never fire
  | 'prompt' // iOS: permission needed, awaiting a user gesture
  | 'granted' // listening
  | 'denied' // iOS: permission refused
  | 'unavailable' // listening, but no absolute heading is being produced

interface IOSDeviceOrientationEvent {
  requestPermission?: () => Promise<'granted' | 'denied'>
}

interface AbsoluteOrientationEvent extends DeviceOrientationEvent {
  webkitCompassHeading?: number
  webkitCompassAccuracy?: number
}

/** True when the orientation sensors could fire here at all. */
export function isHeadingSupported(): boolean {
  return typeof window !== 'undefined' && 'DeviceOrientationEvent' in window
}

/** iOS gates the sensor behind a permission call; Android and desktop do not. */
export function headingNeedsPermission(): boolean {
  if (!isHeadingSupported()) return false
  const ctor = window.DeviceOrientationEvent as unknown as IOSDeviceOrientationEvent
  return typeof ctor.requestPermission === 'function'
}

/**
 * Turns one orientation event into a compass bearing, or null if it carries
 * no absolute reference.
 *
 * Two incompatible sources, and the difference matters:
 *
 * - iOS exposes `webkitCompassHeading`, already a clockwise bearing from
 *   north — usable as-is.
 * - Everyone else exposes `alpha`, which runs *counter*-clockwise, so the
 *   bearing is `360 - alpha`. Getting this backwards produces a compass that
 *   mirrors every turn, which reads as "the compass is broken" rather than
 *   as a sign error.
 *
 * A non-absolute event is rejected outright. `deviceorientation` without
 * `absolute` is relative to wherever the device happened to be when the page
 * loaded — a plausible-looking number with no relationship to north, which
 * is worse than showing nothing.
 */
export function readEventHeading(event: AbsoluteOrientationEvent): number | null {
  if (typeof event.webkitCompassHeading === 'number' && Number.isFinite(event.webkitCompassHeading)) {
    return normalizeDegrees(event.webkitCompassHeading)
  }
  if (!event.absolute) return null
  if (typeof event.alpha !== 'number' || !Number.isFinite(event.alpha)) return null
  return normalizeDegrees(360 - event.alpha)
}

/**
 * The sensor frame is fixed to the *device*, not the screen. Turning a phone
 * to landscape rotates what the user sees without changing the sensor's own
 * axes, so the heading has to be offset by the screen's rotation or the
 * compass reads 90° out whenever the phone is sideways.
```

```
 */
function screenAngle(): number {
  if (typeof window === 'undefined') return 0
  const angle = window.screen?.orientation?.angle
  return typeof angle === 'number' ? angle : 0
}

export interface DeviceHeading {
  /** Smoothed bearing, 0–360, or null when no heading is available. */
  heading: number | null
  /** Unsmoothed most recent reading — for diagnostics, not for display. */
  rawHeading: number | null
  status: HeadingStatus
  /** iOS only: call from a tap handler to request sensor access. */
  requestPermission: () => Promise<void>
  /**
   * Reported heading error in degrees, where the platform provides one
   * (iOS `webkitCompassAccuracy`). Null on Android, which exposes no such
   * figure through DeviceOrientationEvent. A negative value means iOS
   * considers the reading unreliable and the magnetometer needs calibrating.
   */
  accuracyDegrees: number | null
  /** True when the reading is known to be untrustworthy - prompt a figure-of-eight. */
  needsCalibration: boolean
}

export function useDeviceHeading({ alpha = DEFAULT_SMOOTHING_ALPHA }: { alpha?: number } = {}): DeviceHeading {
  const [heading, setHeading] = useState<number | null>(null)
  const [rawHeading, setRawHeading] = useState<number | null>(null)
  const [accuracyDegrees, setAccuracyDegrees] = useState<number | null>(null)
  const [status, setStatus] = useState<HeadingStatus>(() => {
    if (!isHeadingSupported()) return 'unsupported'
    if (typeof window !== 'undefined' && !window.isSecureContext) return 'insecure'
    return headingNeedsPermission() ? 'prompt' : 'granted'
  })

  // Smoothing is a running value, and re-rendering on every sensor event at
  // ~60Hz to read it back out of state would be its own performance problem.
  const smoothedRef = useRef<number | null>(null)

  const requestPermission = useCallback(async () => {
    if (!headingNeedsPermission()) return
    const ctor = window.DeviceOrientationEvent as unknown as IOSDeviceOrientationEvent
    try {
      const result = await ctor.requestPermission!()
      setStatus(result === 'granted' ? 'granted' : 'denied')
    } catch {
      // requestPermission throws when called outside a user gesture. That is
      // a wiring mistake rather than a refusal, but from here the two are
      // indistinguishable and both leave the compass off.
      setStatus('denied')
    }
  }, [])

  // Listening is gated on PERMISSION, not on `status`.
  //
  // This used to be `if (status !== 'granted') return`, which made the
  // "no magnetometer" timer below self-fulfilling: after 3 seconds with no
  // reading it set status to 'unavailable', the effect re-ran, and the
```

```
// listeners were torn down - so a sensor that simply took four seconds to
// produce its first absolute reading could never recover. The compass was
// dead for the rest of the session and the UI said the hardware was
// missing. Magnetometers are routinely slow to settle, especially
// uncalibrated ones indoors, which is exactly where this app is used.
const listening = status === 'granted' || status === 'unavailable'

useEffect(() => {
  if (!listening) return

  let sawAbsoluteReading = false

  const onOrientation = (event: Event) => {
    const reading = readEventHeading(event as AbsoluteOrientationEvent)
    if (reading === null) return

    // A reading arriving after the timer already gave up puts the
    // compass back into service instead of leaving it wrongly reported
    // as missing hardware.
    if (!sawAbsoluteReading) setStatus('granted')
    sawAbsoluteReading = true

    const accuracy = (event as AbsoluteOrientationEvent).webkitCompassAccuracy
    setAccuracyDegrees(typeof accuracy === 'number' ? accuracy : null)

    // The screen's rotation, not the device's. Adding it is correct:
    // rotating the phone 90 degrees anticlockwise makes the device's top
    // point 90 degrees further anticlockwise while screen.orientation.angle
    // reports 90, and the two cancel to give the bearing the user is
    // actually looking along.
    const corrected = normalizeDegrees(reading + screenAngle())
    setRawHeading(corrected)
    smoothedRef.current = smoothHeading(smoothedRef.current, corrected, { alpha })
    setHeading(smoothedRef.current)
  }

  // `deviceorientationabsolute` is the one that carries a true north
  // reference on Android. Plain `deviceorientation` is listened to as well
  // because that is where iOS delivers webkitCompassHeading; readings
  // without an absolute reference are discarded inside the handler.
  window.addEventListener('deviceorientationabsolute', onOrientation)
  window.addEventListener('deviceorientation', onOrientation)

  // A device with no magnetometer fires nothing at all, so silence is the
  // only signal that the sensor isn't there. Without this the UI would sit
  // on "granted" forever showing no heading and no explanation.
  const noSensorTimer = window.setTimeout(() => {
    if (!sawAbsoluteReading) setStatus('unavailable')
  }, 3000)

  return () => {
    window.removeEventListener('deviceorientationabsolute', onOrientation)
    window.removeEventListener('deviceorientation', onOrientation)
    window.clearTimeout(noSensorTimer)
  }
}, [listening, alpha])

// Start the compass on the first touch, rather than on a button.
//
```

```
  // `prompt` is a platform quirk, not a decision the pilgrim made: iOS will
  // not hand over the magnetometer except from inside a real gesture, so the
  // sensor cannot start itself on load. That constraint used to surface as an
  // "Enable compass" pill on the map - one more control to find and tap
  // before a compass that the pilgrim never asked to be off would work.
  //
  // Any gesture satisfies iOS, so the first one anywhere in the app is taken.
  // `pointerdown` fires before the tap it belongs to is over, so the request
  // is inside the gesture window; `once` and the `prompt` guard together mean
  // this asks exactly once and never again after an answer. A refusal lands
  // in `denied` and is not re-asked, because iOS would not show the sheet a
  // second time anyway and pestering would achieve nothing.
  useEffect(() => {
    if (status !== 'prompt') return
    const start = () => void requestPermission()
    window.addEventListener('pointerdown', start, { once: true })
    return () => window.removeEventListener('pointerdown', start)
  }, [status, requestPermission])

  // iOS reports -1 when it cannot trust the compass at all, and anything
  // past ~25 degrees is too coarse to point someone down a nave with.
  const needsCalibration = accuracyDegrees !== null && (accuracyDegrees < 0 || accuracyDegrees > 25)

  return { heading, rawHeading, status, requestPermission, accuracyDegrees, needsCalibration }
}
```

1.4.76 APP-146 — useDragSafeClicks.ts

Code ID	APP-146
File Name	useDragSafeClicks.ts
File Path	SanctDemo/src/lib/useDragSafeClicks.ts
Language / Type	TypeScript
Lines	58
Functions / Classes	useDragSafeClicks (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/useDragSafeClicks.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useDragSafeClicks.ts

```
import { useCallback, useRef, type MouseEvent, type PointerEvent } from "react";

/**
 * Stops a swipe across a horizontal rail from firing the click of whatever
 * card the finger happened to start on.
 *
 * The cards in these rails are buttons, so a drag beginning on one is
 * ambiguous: the browser sees a pointer go down on a button and come up on
 * the same button, and reports a click — even though the rail scrolled
 * hundreds of pixels in between. Swiping the bulletin therefore opened
 * Ministries instead of moving the rail.
 *
 * CSS alone cannot fix this. `touch-action: pan-x` on the rail is overridden
 * by the button's own `auto`, and even with the panning corrected the click
 * still fires on release. So the movement is measured and the click is
 * cancelled in the capture phase, before it reaches the button's handler.
 *
 * Threshold is in pixels of travel, not time: a slow deliberate drag is still
 * a drag, and a fast tap is still a tap.
 */
const DRAG_THRESHOLD_PX = 8;

export function useDragSafeClicks(thresholdPx: number = DRAG_THRESHOLD_PX) {
  const startX = useRef(0);
  const startY = useRef(0);
  const dragged = useRef(false);

  const onPointerDown = useCallback((event: PointerEvent) => {
    startX.current = event.clientX;
    startY.current = event.clientY;
    dragged.current = false;
  }, []);
```

```
  const onPointerMove = useCallback(
    (event: PointerEvent) => {
      if (dragged.current) return;
      // Either axis counts. A diagonal flick that scrolls the page is just as
      // much "not a tap" as a horizontal one.
      if (
        Math.abs(event.clientX - startX.current) > thresholdPx ||
        Math.abs(event.clientY - startY.current) > thresholdPx
      ) {
        dragged.current = true;
      }
    },
    [thresholdPx],
  );

  const onClickCapture = useCallback((event: MouseEvent) => {
    if (!dragged.current) return;
    // Capture phase, so this runs before the card's own onClick.
    event.preventDefault();
    event.stopPropagation();
    dragged.current = false;
  }, []);

  return { onPointerDown, onPointerMove, onClickCapture };
}
```

1.4.77 APP-147 — useLanguage.ts

Code ID	APP-147
File Name	useLanguage.ts
File Path	SanctDemo/src/lib/useLanguage.ts
Language / Type	TypeScript
Lines	53
Functions / Classes	useLanguage (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx, src/components/ChurchHistory.tsx, src/components/Dashboard.tsx, src/components/MinistriesTab.tsx, src/components/ParishWelcomeHeader.tsx, src/components/ReminderSettingsCard.tsx, src/components/SacramentsTab.tsx, src/components/SanctiSheet.tsx
How to Recognize It	Open SanctDemo/src/lib/useLanguage.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useLanguage.ts

```
import { useCallback, useEffect, useState } from "react";
import {
  LANGUAGE_EVENT, LANGUAGE_KEY, loadLanguage, saveLanguage, type Language,
} from "./language";

/**
 * The reader's language, shared across every screen.
 *
 * ## Why a storage listener and not a context
 *
 * A context would mean a provider wrapping the app and a prop drilled
 * into the rosary's iframe, which is same-origin and keeps its own copy
 * of this setting in localStorage. Both sides already agree through
 * that key; listening to `storage` means changing the language in the
 * rosary updates the app and the other way round, with nothing between
 * them to keep in sync.
 *
 * The `storage` event does not fire in the tab that made the change, so
 * the setter updates local state itself.
 */
export function useLanguage(): {
  language: Language;
  setLanguage: (next: Language) => void;
} {
  const [language, setLanguageState] = useState<Language>(loadLanguage);

  useEffect(() => {
    const onStorage = (e: StorageEvent) => {
      // A null key means the whole store was cleared.
      if (e.key !== null && e.key !== LANGUAGE_KEY) return;
```

```
      setLanguageState(loadLanguage());
    };
    // `storage` reaches other tabs; the custom event reaches the other
    // hooks in this one. Both are needed: without the first the rosary
    // iframe and the app drift apart, without the second the tab bar
    // and Sancti's button keep the old language until a reload.
    const onLocal = () => setLanguageState(loadLanguage());

    window.addEventListener("storage", onStorage);
    window.addEventListener(LANGUAGE_EVENT, onLocal);
    return () => {
      window.removeEventListener("storage", onStorage);
      window.removeEventListener(LANGUAGE_EVENT, onLocal);
    };
  }, []);

  const setLanguage = useCallback((next: Language) => {
    saveLanguage(next);
    setLanguageState(next);
  }, []);

  return { language, setLanguage };
}
```

1.4.78 APP-148 — useMassScheduleNotice.ts

Code ID	APP-148
File Name	useMassScheduleNotice.ts
File Path	SanctDemo/src/lib/useMassScheduleNotice.ts
Language / Type	TypeScript
Lines	90
Functions / Classes	readSeen (internal function); writeSeen (internal function); useMassScheduleNotice (exported function)
Related Files (its imports)	./deviceNotifications, ./parishContent
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/useMassScheduleNotice.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Watches the home parish's Mass schedule and tells the phone when it moves.
>
> The rules live in massScheduleWatch.ts and are tested there; this is only the plumbing — subscribe, compare against what was last seen, and hand the words to the notification layer.
>
> Only the HOME parish. Somebody browsing another parish is reading, not planning to attend, and a notification about a church they looked at once is spam.

Complete Source Code — useMassScheduleNotice.ts

```
/**
 * Watches the home parish's Mass schedule and tells the phone when it
 * moves.
 *
 * The rules live in massScheduleWatch.ts and are tested there; this is
 * only the plumbing — subscribe, compare against what was last seen,
 * and hand the words to the notification layer.
 *
 * Only the HOME parish. Somebody browsing another parish is reading,
 * not planning to attend, and a notification about a church they
 * looked at once is spam.
 */
import { useEffect, useRef } from 'react'

import { notifyNow, permissionState } from './deviceNotifications'
import { watchParishContent } from './parishContent'
import {
  classifyMassChange,
  massDiff,
  massFingerprint,
  massFingerprintKey,
  massNotice,
```

```
  nextStoredFingerprint,
} from './massScheduleWatch'

function readSeen(parishId: string): string | null {
  try {
    return localStorage.getItem(massFingerprintKey(parishId))
  } catch {
    // Private mode. Without a baseline every change reads as
    // first-seen, which says nothing — the quiet failure, not the
    // noisy one.
    return null
  }
}

function writeSeen(parishId: string, value: string): void {
  try {
    localStorage.setItem(massFingerprintKey(parishId), value)
  } catch {
    /* nothing to do; it will simply compare again next time */
  }
}

export function useMassScheduleNotice(
  parishId: string | null | undefined,
  parishName: string,
): void {
  // The name can change render to render while the parish does not, and
  // it is only needed at the moment something is said.
  const name = useRef(parishName)
  name.current = parishName

  useEffect(() => {
    if (!parishId) return

    let live = true

    const stop = watchParishContent(parishId, content => {
      if (!live) return

      const current = massFingerprint(content?.massSchedule)
      const previous = readSeen(parishId)
      const change = classifyMassChange(previous, current)

      // Written before the notification is sent, not after. If the
      // send throws, the user has still seen this state and should not
      // be told about it again on the next snapshot.
      writeSeen(parishId, nextStoredFingerprint(current))

      // What moved, so the notice can name the Mass rather than
      // saying only that something changed.
      const notice = massNotice(change, name.current, massDiff(previous, current))
      if (!notice) return

      void (async () => {
        // Asked for, never demanded. Somebody who has not granted
        // notifications gets nothing rather than a permission prompt
        // in the middle of reading about a parish.
        if ((await permissionState()) !== 'granted') return
        await notifyNow(notice.title, notice.body)
```

```
      })()
    })

    return () => {
      live = false
      stop()
    }
  }, [parishId])
}
```

1.4.79 APP-149 — useNarration.ts

Code ID	APP-149
File Name	useNarration.ts
File Path	SanctDemo/src/lib/useNarration.ts
Language / Type	TypeScript
Lines	92
Functions / Classes	Narration (exported type); useNarration (exported function)
Related Files (its imports)	./speech, ./language
Where It Is Used	src/components/ArTour.tsx, src/components/ChurchHistory.tsx
How to Recognize It	Open SanctDemo/src/lib/useNarration.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useNarration.ts

```
import { useCallback, useEffect, useRef, useState } from "react";
import { currentEngine, speakAloud, stopAloud, type Engine } from "./speech";
import type { Language } from "./language";

/**
 * Reading a station's words aloud.
 *
 * ## Why the browser's voice and not an audio file
 *
 * `Station.audioUrl` exists in the type and is set on exactly none of
 * them — no narration has been recorded for this parish yet. The app
 * already solved that once, on the Map tab, by handing the text to
 * `speechSynthesis`, and doing the same here keeps one answer to the
 * question rather than two. When real recordings arrive, `speak` takes
 * a url instead and nothing above this module changes.
 *
 * ## Why it tracks `speaking` itself rather than trusting the engine
 *
 * `speechSynthesis.speaking` stays true through a pause and lies for a
 * moment after cancel(), so a button bound to it flickers and sticks.
 * The events are the reliable signal, and this holds the state they
 * report.
 */
export interface Narration {
  speak: (text: string) => void;
  stop: () => void;
  /** Starts this text, or stops if it is already being read. */
  toggle: (text: string) => void;
  speaking: boolean;
  supported: boolean;
}

export function useNarration(language: Language = "en"): Narration {
```

```
  const [speaking, setSpeaking] = useState(false);

  // Read once per mount rather than on every render. On the phone this
  // is always "native"; in a browser it depends on whether any voice is
  // installed, which is not knowable before the page has loaded.
  const [engine, setEngine] = useState<Engine>(() => currentEngine());

  useEffect(() => {
    if (engine !== "none" || typeof window === "undefined") return;
    // Chrome populates its voice list asynchronously, so a "none" read
    // at mount can be wrong. One recheck when the list arrives is
    // enough; without it the first visitor to a cold page gets no
    // narration button at all.
    const recheck = () => setEngine(currentEngine());
    window.speechSynthesis?.addEventListener?.("voiceschanged", recheck);
    return () => {
      window.speechSynthesis?.removeEventListener?.("voiceschanged", recheck);
    };
  }, [engine]);

  const supported = engine !== "none";

  const speakingRef = useRef(false);
  speakingRef.current = speaking;

  const stop = useCallback(() => {
    void stopAloud();
    setSpeaking(false);
  }, []);

  const speak = useCallback(
    (text: string) => {
      if (!supported || !text.trim()) return;
      setSpeaking(true);
      // The promise resolves when the engine has finished reading, which
      // is how the button knows to go back to "play" - the native plugin
      // has no onend event to listen for.
      void speakAloud(text, language).finally(() => setSpeaking(false));
    },
    [supported, language],
  );

  const toggle = useCallback(
    (text: string) => {
      if (speakingRef.current) stop();
      else speak(text);
    },
    [speak, stop],
  );

  // Leaving the screen must silence it. A voice still reading the
  // Baptismal Font while the pilgrim is back on Home is the kind of bug
  // people close the app over.
  useEffect(() => {
    return () => { void stopAloud(); };
  }, []);

  return { speak, stop, toggle, speaking, supported };
}
```

1.4.80 APP-150 — useOpeningNotices.ts

Code ID	APP-150
File Name	useOpeningNotices.ts
File Path	SanctDemo/src/lib/useOpeningNotices.ts
Language / Type	TypeScript
Lines	141
Functions / Classes	seenKey (internal function); readSeen (internal function); writeSeen (internal function); newlyOpened (exported function); openedNotice (exported function); useOpeningNotices (exported function)
Related Files (its imports)	./availability, ./deviceNotifications, ./parishContent, ./openingSubscriptions
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/useOpeningNotices.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Tells the pilgrim when something they asked to be told about opens.
>
> The bells in lib/openingSubscriptions record the wish. This is what honours it: one watcher on the home parish's content, comparing the closed list against the one it last saw, and a notification for every watched item that has just come off it.
>
> ## Why it only watches one parish
>
> The same reason useMassScheduleNotice does. Content streams per parish, so watching all thirty-one would mean thirty-one live Firestore listeners on a phone, and somebody who tapped a bell on a church they passed through once does not want it waking their screen months later. A watch on another parish is kept and simply reported the next time that parish is open on screen.
>
> ## Why the watch is dropped once it fires
>
> The pilgrim asked to be told when it opens. They have been told. A watch that stayed would fire again on the next close-and-reopen, which is a different question nobody asked.

Complete Source Code — useOpeningNotices.ts

```
/**
 * Tells the pilgrim when something they asked to be told about opens.
 *
 * The bells in lib/openingSubscriptions record the wish. This is what
 * honours it: one watcher on the home parish's content, comparing the
 * closed list against the one it last saw, and a notification for every
 * watched item that has just come off it.
 *
```

```
 * ## Why it only watches one parish
 *
 * The same reason useMassScheduleNotice does. Content streams per
 * parish, so watching all thirty-one would mean thirty-one live
 * Firestore listeners on a phone, and somebody who tapped a bell on a
 * church they passed through once does not want it waking their screen
 * months later. A watch on another parish is kept and simply reported
 * the next time that parish is open on screen.
 *
 * ## Why the watch is dropped once it fires
 *
 * The pilgrim asked to be told when it opens. They have been told. A
 * watch that stayed would fire again on the next close-and-reopen,
 * which is a different question nobody asked.
 */
import { useEffect, useRef } from 'react'

import { closedIds } from './availability'
import { notifyNow, permissionState } from './deviceNotifications'
import { watchParishContent } from './parishContent'
import { isWatched, setWatched } from './openingSubscriptions'

function seenKey(parishId: string): string {
  return `sanctiwalk.closed.seen.${parishId}`
}

function readSeen(parishId: string): string[] | null {
  try {
    const raw = localStorage.getItem(seenKey(parishId))
    if (!raw) return null
    const parsed: unknown = JSON.parse(raw)
    return Array.isArray(parsed) ? parsed.filter(v => typeof v === 'string') : null
  } catch {
    return null
  }
}

function writeSeen(parishId: string, ids: string[]): void {
  try {
    localStorage.setItem(seenKey(parishId), JSON.stringify(ids))
  } catch {
    /* private mode; it will compare again next time */
  }
}

/**
 * The ids that were closed and no longer are.
 *
 * `previous` being null is a first run: there is no "was closed" to
 * compare against, so nothing has opened yet and nothing is said. The
 * alternative is announcing every open ministry in the parish the first
 * time the app starts.
 */
export function newlyOpened(previous: string[] | null, current: string[]): string[] {
  if (previous === null) return []
  const nowClosed = new Set(current)
  return previous.filter(id => !nowClosed.has(id))
}
```

```
/** The words. Kept out of the effect so they can be tested. */
export function openedNotice(
  names: string[],
  parishName: string,
): { title: string; body: string } | null {
  if (names.length === 0) return null
  if (names.length === 1) {
    return {
      title: `${names[0]} is open`,
      body: `${parishName} is accepting applications for ${names[0]} again.`,
    }
  }
  return {
    title: 'Now open at your parish',
    body: `${parishName} has reopened ${names.join(', ')}.`,
  }
}

export function useOpeningNotices(
  parishId: string | null | undefined,
  parishName: string,
  /** Turns an id into something a person would recognise. */
  nameFor: (itemId: string) => string,
): void {
  // Both change render to render while the parish does not, and both
  // are only needed at the moment something is said.
  const latest = useRef({ parishName, nameFor })
  latest.current = { parishName, nameFor }

  useEffect(() => {
    if (!parishId) return
    let live = true

    const stop = watchParishContent(parishId, content => {
      if (!live) return

      const current = [...closedIds(content)].sort()
      const previous = readSeen(parishId)

      // Written before anything is sent. If the notification throws,
      // this state has still been seen and must not be reported twice.
      writeSeen(parishId, current)

      const opened = newlyOpened(previous, current).filter(id =>
        // A ministry nobody asked about opening is not news.
        isWatched({ kind: 'ministry', parishId, itemId: id }) ||
        isWatched({ kind: 'sacrament', parishId, itemId: id }),
      )
      if (opened.length === 0) return

      for (const id of opened) {
        setWatched({ kind: 'ministry', parishId, itemId: id }, false)
        setWatched({ kind: 'sacrament', parishId, itemId: id }, false)
      }

      const notice = openedNotice(
        opened.map(id => latest.current.nameFor(id)),
        latest.current.parishName,
      )
```

```
      if (!notice) return

      void (async () => {
        // Asked for by the bell, never demanded here.
        if ((await permissionState()) !== 'granted') return
        await notifyNow(notice.title, notice.body)
      })()
    })

    return () => {
      live = false
      stop()
    }
  }, [parishId])
}
```

1.4.81 APP-151 — useParishContent.ts

Code ID	APP-151
File Name	useParishContent.ts
File Path	SanctDemo/src/lib/useParishContent.ts
Language / Type	TypeScript
Lines	28
Functions / Classes	useParishContent (exported function)
Related Files (its imports)	./parishContent
Where It Is Used	src/admin/AdminAvailability.tsx, src/admin/AdminContent.tsx, src/admin/AdminItems.tsx, src/admin/AdminMassSchedule.tsx, src/admin/ParishStatus.tsx, src/components/ChurchHistory.tsx, src/components/ChurchHistoryCard.tsx, src/components/Dashboard.tsx, src/components/MassScheduleCard.tsx, src/components/MinistriesTab.tsx, src/components/ParishContentEditor.tsx, src/components/SacramentsTab.tsx
How to Recognize It	Open SanctDemo/src/lib/useParishContent.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useParishContent.ts

```
import { useEffect, useState } from "react";
import { watchParishContent, type ParishContent } from "./parishContent";

/**
 * Live admin-managed content for one parish, or null while it loads and for
 * every parish nobody has edited yet.
 *
 * Subscribed rather than fetched, so a photo an admin uploads on one phone
 * appears on every other device without anyone reloading — which is the
 * behaviour a parish office expects of a noticeboard.
 *
 * Callers must treat null as "use the values from data.ts". Nothing here
 * replaces the compiled content; it only overlays it.
 */
export function useParishContent(parishId: string | null | undefined): ParishContent | null {
  const [content, setContent] = useState<ParishContent | null>(null);

  useEffect(() => {
    if (!parishId) {
      setContent(null);
      return;
    }
    setContent(null); // clear the previous parish's content while the new one loads
    return watchParishContent(parishId, setContent);
  }, [parishId]);
```

```
  return content;
}
```

1.4.82 APP-152 — useParishContents.ts

Code ID	APP-152
File Name	useParishContents.ts
File Path	SanctDemo/src/lib/useParishContents.ts
Language / Type	TypeScript
Lines	60
Functions / Classes	useParishContents (exported function)
Related Files (its imports)	./parishContent
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/useParishContents.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useParishContents.ts

```
import { useEffect, useMemo, useRef, useState } from "react";
import { watchParishContent, type ParishContent } from "./parishContent";

/**
 * Admin-managed content for several parishes at once.
 *
 * `useParishContent` watches one parish, which was right while the app
 * only ever cared about the one on screen. Reminders changed that: a
 * pilgrim can follow a second parish, and its Mass times have to be as
 * current as their own — a reminder built from the times compiled into
 * the app would keep sending them to a Mass the parish moved.
 *
 * ## Why the ids are joined into a string
 *
 * The caller builds the array inline, so a new one arrives on every
 * render and an effect keyed on the array itself would tear down and
 * rebuild every listener each time. Keyed on the joined ids, it only
 * reacts when the SET actually changes.
 */
export function useParishContents(
  parishIds: string[],
): Record<string, ParishContent | null> {
  const key = parishIds.join("|");
  const [contents, setContents] = useState<Record<string, ParishContent | null>>({});

  // Read inside the effect so the effect does not depend on the array.
  const idsRef = useRef(parishIds);
  idsRef.current = parishIds;

  useEffect(() => {
    const ids = idsRef.current;
    if (ids.length === 0) {
      setContents({});
```

```
      return;
    }

    const unsubscribes = ids.map(id =>
      watchParishContent(id, content =>
        setContents(current => ({ ...current, [id]: content })),
      ),
    );

    return () => {
      for (const stop of unsubscribes) stop();
      // Drop parishes that are no longer followed, so a stale schedule
      // cannot go on producing reminders after the switch is turned off.
      setContents(current => {
        const next: Record<string, ParishContent | null> = {};
        for (const id of idsRef.current) {
          if (id in current) next[id] = current[id];
        }
        return next;
      });
    };
  }, [key]);

  // Stable while the contents are unchanged, so the scheduler below
  // does not rebuild every alarm on every render.
  return useMemo(() => contents, [contents]);
}
```

1.4.83 APP-153 — useQrScanner.ts

Code ID	APP-153
File Name	useQrScanner.ts
File Path	SanctDemo/src/lib/useQrScanner.ts
Language / Type	TypeScript
Lines	178
Functions / Classes	QrScannerState (exported type); nativeDetector (internal function); useQrScanner (exported function)
Related Files (its imports)	./qr
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/useQrScanner.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useQrScanner.ts

```
import { useEffect, useRef, useState, type RefObject } from 'react'
import jsQR from 'jsqr'
import { resolveScannedQr, type QrScanResult } from './qr'

/**
 * Continuous QR scanning over an already-running camera stream.
 *
 * It deliberately does NOT own a camera. useCamera already holds the stream,
 * the secure-origin check and the permission handling; opening a second
 * MediaStream for the same device is how you get "camera in use" errors and a
 * second green recording dot. This reads frames out of the <video> that is
 * already on screen.
 *
 * Decoding is jsQR, not the browser's BarcodeDetector. BarcodeDetector is
 * faster where it exists but is absent on iOS Safari — the platform this app
 * is being demonstrated on — so it is a fast path, never the implementation.
 */

/** How often to look for a code. */
const SCAN_INTERVAL_MS = 220

/**
 * Frames are downscaled to this width before decoding.
 *
 * jsQR walks every pixel, so a full 1080p frame costs tens of milliseconds
 * several times a second on a mid-range phone — enough to make the camera
 * preview stutter, which reads as the app being broken. A printed poster's
 * code is comfortably legible at this width from normal scanning distance.
 */
const DECODE_WIDTH = 640
```

```
/**
 * How long the same code is ignored after being reported.
 *
 * Without this, a poster held in frame reports the same station five times a
 * second. The point is not throttling for its own sake: each report opens a
 * screen, and re-opening it under the pilgrim's finger makes the app feel
 * possessed.
 */
const REPEAT_SUPPRESSION_MS = 2500

interface UseQrScannerOptions {
  /** Scanning only runs while this is true. */
  active: boolean
  /** Called once per distinct code seen. */
  onResult: (result: QrScanResult, raw: string) => void
}

export interface QrScannerState {
  /** True while a frame is being looked at — for a subtle "looking" hint. */
  scanning: boolean
  /** The most recent raw payload seen, whether or not it was one of ours. */
  lastRaw: string | null
}

interface BarcodeDetectorLike {
  detect(source: CanvasImageSource): Promise<{ rawValue: string }[]>
}

function nativeDetector(): BarcodeDetectorLike | null {
  const ctor = (window as unknown as {
    BarcodeDetector?: new (options: { formats: string[] }) => BarcodeDetectorLike
  }).BarcodeDetector

  if (typeof ctor !== 'function') return null
  try {
    return new ctor({ formats: ['qr_code'] })
  } catch {
    // Constructing throws when the format is unsupported. Not an error worth
    // reporting — jsQR handles it either way.
    return null
  }
}

export function useQrScanner(
  videoRef: RefObject<HTMLVideoElement | null>,
  { active, onResult }: UseQrScannerOptions,
): QrScannerState {
  const [scanning, setScanning] = useState(false)
  const [lastRaw, setLastRaw] = useState<string | null>(null)

  const canvasRef = useRef<HTMLCanvasElement | null>(null)
  // Held in a ref so changing the handler does not restart the scan loop,
  // which would drop the suppression state and re-fire the code in frame.
  const onResultRef = useRef(onResult)
  onResultRef.current = onResult
  const lastReportedRef = useRef<{ code: string; at: number } | null>(null)

  useEffect(() => {
    if (!active) {
```

```
  setScanning(false)
  lastReportedRef.current = null
  return
}

setScanning(true)
const detector = nativeDetector()
let stopped = false
let timer: number | undefined

const readFrame = async () => {
  if (stopped) return
  const video = videoRef.current

  // readyState < 2 means no frame has been decoded yet, so the canvas
  // would hold a blank rectangle rather than the scene.
  if (!video || video.readyState < 2 || !video.videoWidth) {
    timer = window.setTimeout(readFrame, SCAN_INTERVAL_MS)
    return
  }

  const canvas = canvasRef.current ?? (canvasRef.current = document.createElement('canvas'))
  const scale = Math.min(1, DECODE_WIDTH / video.videoWidth)
  canvas.width = Math.round(video.videoWidth * scale)
  canvas.height = Math.round(video.videoHeight * scale)

  const ctx = canvas.getContext('2d', { willReadFrequently: true })
  if (!ctx) {
    console.error('[Scanner] QR: no 2D context — cannot decode')
    stopped = true
    setScanning(false)
    return
  }

  ctx.drawImage(video, 0, 0, canvas.width, canvas.height)

  let raw: string | null = null

  if (detector) {
    try {
      const found = await detector.detect(canvas)
      raw = found[0]?.rawValue ?? null
    } catch {
      // A detector that throws mid-session falls through to jsQR rather
      // than stopping the scan.
      raw = null
    }
  }

  if (!raw) {
    const image = ctx.getImageData(0, 0, canvas.width, canvas.height)
    // "attemptBoth" reads codes printed light-on-dark as well as the
    // usual dark-on-light — a parish is as likely to print white on navy.
    raw = jsQR(image.data, image.width, image.height, {
      inversionAttempts: 'attemptBoth',
    })?.data ?? null
  }

  if (raw && !stopped) {
```

```
        const now = Date.now()
        const last = lastReportedRef.current
        const isRepeat = last?.code === raw && now - last.at < REPEAT_SUPPRESSION_MS

        if (!isRepeat) {
          lastReportedRef.current = { code: raw, at: now }
          setLastRaw(raw)
          const result = resolveScannedQr(raw)
          console.log(
            `[Scanner] QR read: ${JSON.stringify(raw.slice(0, 80))} → ${result.kind}`,
          )
          onResultRef.current(result, raw)
        }
      }

      if (!stopped) timer = window.setTimeout(readFrame, SCAN_INTERVAL_MS)
    }

    void readFrame()

    return () => {
      stopped = true
      if (timer !== undefined) window.clearTimeout(timer)
      setScanning(false)
    }
  }, [active, videoRef])

  return { scanning, lastRaw }
}
```

1.4.84 APP-154 — useReminders.ts

Code ID	APP-154
File Name	useReminders.ts
File Path	SanctDemo/src/lib/useReminders.ts
Language / Type	TypeScript
Lines	134
Functions / Classes	loadSettings (exported function); saveSettings (exported function); useReminders (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/useReminders.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useReminders.ts

```
import { useCallback, useEffect, useRef, useState } from "react";
import {
  buildScheduleForParishes, DEFAULT_REMINDER_SETTINGS,
  type ParishReminders, type ReminderSettings,
} from "./reminderSchedule";
import {
  applySchedule, notificationsSupported, permissionState, requestPermission,
  type PermissionState,
} from "./deviceNotifications";
const STORAGE_KEY = "sanctiwalk.reminders";

/**
 * Reminder settings, kept on the device.
 *
 * Deliberately NOT in Firestore. These are a property of this phone —
 * which alarms it has queued — not of the account. Two people sharing a
 * login should not switch each other's Mass reminders off, and somebody
 * signed out should still get reminded about Mass.
 */
export function loadSettings(): ReminderSettings {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return DEFAULT_REMINDER_SETTINGS;
    // Spread over the defaults rather than trusting the stored shape: a
    // build that adds a category would otherwise read `undefined` for it
    // on every phone that saved settings before.
    return { ...DEFAULT_REMINDER_SETTINGS, ...JSON.parse(raw) };
  } catch {
    return DEFAULT_REMINDER_SETTINGS;
  }
}
```

```
export function saveSettings(settings: ReminderSettings): void {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(settings));
  } catch {
    // Private mode or blocked storage. The settings hold for this run.
  }
}

/**
 * Keeps the phone's queued reminders in step with the parish.
 *
 * ## Why it reschedules wholesale
 *
 * The inputs all change under it: the parish suspends a Mass, posts a
 * feast, archives a notice. Working out the difference between the old
 * queue and the new one would be a second implementation of the
 * scheduling rules, able to disagree with the first. Rebuilding the
 * whole list and replacing it is one rule, and the reminder ids are
 * content-derived so the alarms that did not change are simply
 * re-registered unchanged.
 *
 * ## Why it also runs when the app is reopened
 *
 * Reminders are scheduled a week out. Someone who opens the app once a
 * fortnight would run dry, so every return to the foreground tops the
 * queue back up.
 */
export function useReminders(parishes: ParishReminders[]) {
  const [settings, setSettingsState] = useState<ReminderSettings>(loadSettings);
  const [permission, setPermission] = useState<PermissionState>("prompt");
  const [scheduled, setScheduled] = useState(0);

  // So the effect below can re-run on a resume without being re-created.
  const inputs = useRef({ parishes, settings });
  inputs.current = { parishes, settings };

  useEffect(() => {
    void permissionState().then(setPermission);
  }, []);

  const reschedule = useCallback(async () => {
    const { parishes: list, settings: cfg } = inputs.current;
    if (!notificationsSupported()) return;
    if ((await permissionState()) !== "granted") return;

    const reminders = buildScheduleForParishes(list, new Date(), cfg);
    setScheduled(await applySchedule(reminders));
  }, []);

  /*
   * Whenever the followed parishes, their content, or the settings
   * change.
   *
   * Keyed on a SUMMARY of the parishes rather than the array. The
   * caller builds it inline, so the array is new on every render and
   * depending on it directly would rebuild every alarm continuously.
   * The summary changes only when something that affects a reminder
   * does.
```

```
   */
  const parishKey = parishes
    .map(p => `${p.parishId}:${p.massSchedule.map(e => `${e.day}@${e.time}@${(e.unavailableTimes ?? []).join(",")}`).join(";")}:${p.announcements.length}`)
    .join("|");

  useEffect(() => {
    void reschedule();
  }, [reschedule, parishKey, settings]);

  // And whenever the app comes back to the foreground, to top up the
  // week-long horizon for someone who opens it rarely.
  useEffect(() => {
    const onVisible = () => {
      if (document.visibilityState === "visible") void reschedule();
    };
    document.addEventListener("visibilitychange", onVisible);
    return () => document.removeEventListener("visibilitychange", onVisible);
  }, [reschedule]);

  const update = useCallback((patch: Partial<ReminderSettings>) => {
    setSettingsState(current => {
      const next = { ...current, ...patch };
      saveSettings(next);
      return next;
    });
  }, []);

  const enable = useCallback(async () => {
    const state = await requestPermission();
    setPermission(state);
    if (state === "granted") await reschedule();
    return state;
  }, [reschedule]);

  return {
    settings,
    update,
    permission,
    enable,
    /** How many alarms are queued right now — shown so it is not a black box. */
    scheduled,
    supported: notificationsSupported(),
  };
}
```

1.4.85 APP-155 — useSimulatorMode.ts

Code ID	APP-155
File Name	useSimulatorMode.ts
File Path	SanctDemo/src/lib/useSimulatorMode.ts
Language / Type	TypeScript
Lines	43
Functions / Classes	useSimulatorMode (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/components/ArTour.tsx
How to Recognize It	Open SanctDemo/src/lib/useSimulatorMode.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — useSimulatorMode.ts

```
import { useCallback, useEffect, useState } from 'react'
import {
  SIMULATOR_EVENT,
  SIMULATOR_KEY,
  loadSimulatorMode,
  saveSimulatorMode,
} from './simulatorMode'

/**
 * The simulator setting, shared across every screen that reads it.
 *
 * Same shape as useLanguage, and for the same reason: `storage` only
 * reaches other tabs, so without the custom event the Me screen would
 * update and the scanner would keep the old answer until a reload.
 */
export function useSimulatorMode(): {
  simulatorMode: boolean
  setSimulatorMode: (on: boolean) => void
} {
  const [simulatorMode, setState] = useState<boolean>(loadSimulatorMode)

  useEffect(() => {
    const onStorage = (e: StorageEvent) => {
      if (e.key !== null && e.key !== SIMULATOR_KEY) return
      setState(loadSimulatorMode())
    }
    const onLocal = () => setState(loadSimulatorMode())

    window.addEventListener('storage', onStorage)
    window.addEventListener(SIMULATOR_EVENT, onLocal)
    return () => {
      window.removeEventListener('storage', onStorage)
      window.removeEventListener(SIMULATOR_EVENT, onLocal)
```

```
    }
  }, [])

  const setSimulatorMode = useCallback((on: boolean) => {
    saveSimulatorMode(on)
    setState(on)
  }, [])

  return { simulatorMode, setSimulatorMode }
}
```

1.4.86 APP-156 — userProfile.ts

Code ID	APP-156
File Name	userProfile.ts
File Path	SanctDemo/src/lib/userProfile.ts
Language / Type	TypeScript
Lines	119
Functions / Classes	createProfile (exported function); getProfile (exported function); EditableProfile (exported type); updateProfile (exported function); syncEmailVerified (exported function); isChurchAdmin (exported function)
Related Files (its imports)	./firebase, ../types
Where It Is Used	src/admin/AdminApp.tsx, src/components/EditProfileCard.tsx, src/components/MeTab.tsx, src/lib/applications.ts, src/lib/authFlow.ts, src/routes/FinishSignIn.tsx
How to Recognize It	Open SanctDemo/src/lib/userProfile.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — userProfile.ts

```
import { doc, getDoc, setDoc, serverTimestamp } from "firebase/firestore";
import type { User } from "firebase/auth";
import { db } from "./firebase";
import type { UserProfile } from "../types";

/**
 * The account document at users/{uid}.
 *
 * ## What this module is careful about
 *
 * `role` and `churchId` are what the security rules read to decide what an
 * account may touch. The rules refuse to let an account change either one
 * after creation, and refuse to let it create itself as anything but
 * "user" - so this module can never grant a privilege, only record a choice
 * the rules already allow. That is deliberate: if the only way to become an
 * admin is a write the client is forbidden to make, there is no client bug
 * that can make someone an admin.
 *
 * `emailVerified` is mirrored here for display. It is NOT the source of
 * truth and the rules never read it; Firebase Auth is. A mirror is only as
 * honest as whoever last refreshed it, and the one place that must not be
 * guessed is whether someone proved they own an address.
 */

/** A new account, as the rules will accept it. */
export async function createProfile(
  user: User,
  fullName: string,
  churchId: string,
```

```
  phoneNumber?: string,
  nickname?: string,
): Promise<void> {
  const profile: UserProfile = {
    uid: user.uid,
    email: user.email ?? "",
    fullName,
    churchId,
    // Never true at creation - the rules reject a profile that claims it,
    // and claiming it is the one thing phone verification exists to prevent.
    phoneVerified: false,
    emailVerified: user.emailVerified,
    role: "user",
    createdAt: new Date().toISOString(),
    points: 0,
    steps: 0,
    completedStations: [],
    badges: [],
  };
  if (phoneNumber) profile.phoneNumber = phoneNumber;
  // Falls back to the first word of the full name rather than to the
  // email address, which is what Home used to greet people with.
  profile.nickname = (nickname?.trim() || fullName.trim().split(/\s+/)[0] || "").slice(0, 24);

  await setDoc(doc(db, "users", user.uid), profile);
}

export async function getProfile(uid: string): Promise<UserProfile | null> {
  const snap = await getDoc(doc(db, "users", uid));
  return snap.exists() ? (snap.data() as UserProfile) : null;
}

/**
 * The fields an account may change about itself.
 *
 * Typed as a subset rather than Partial<UserProfile> so that passing `role`
 * or `churchId` is a compile error here, instead of a permission error in
 * front of the user. The rules would refuse it either way; this refuses it
 * earlier and more legibly.
 */
export type EditableProfile = Partial<
  Pick<
    UserProfile,
    | "fullName"
    | "nickname"
    | "photoUrl"
    | "phoneNumber"
    | "points"
    | "steps"
    | "completedStations"
    | "badges"
  >
>;

export async function updateProfile(uid: string, changes: EditableProfile): Promise<void> {
  await setDoc(
    doc(db, "users", uid),
    { ...changes, updatedAt: new Date().toISOString() },
    { merge: true },
```

```
  );
}

/**
 * Brings the mirrored emailVerified flag up to date with Firebase Auth.
 *
 * Called after a reload() of the auth user, because Firebase does not tell
 * the app when someone clicks the link in their inbox - the flag only
 * changes on the next token refresh.
 *
 * Writing this requires the field to be in the rules' editable list, which
 * it deliberately is not. So the write is attempted and allowed to fail:
 * the display falls back to Firebase Auth's own value, which is the truth
 * anyway. Silently wrong UI would be worse than a mirror that lags.
 */
export async function syncEmailVerified(user: User): Promise<boolean> {
  const verified = user.emailVerified;
  try {
    await setDoc(doc(db, "users", user.uid), { emailVerified: verified }, { merge: true });
  } catch {
    /* The rules own this field. Auth remains the source of truth. */
  }
  return verified;
}

/** True when this account administers a church. Display only - rules decide. */
export function isChurchAdmin(profile: UserProfile | null): boolean {
  return profile?.role === "church_admin";
}

export { serverTimestamp };
```

1.4.87 APP-157 — verseBackdrops.ts

Code ID	APP-157
File Name	verseBackdrops.ts
File Path	SanctDemo/src/lib/verseBackdrops.ts
Language / Type	TypeScript
Lines	57
Functions / Classes	Backdrop (exported type); dayIndex (internal function); backdropForDate (exported function); ALL_BACKDROPS (exported constant)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/verseBackdrops.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> The photograph behind the Verse of the Day.
>
> ## Why the parish's own pictures and not stock
>
> The brief points at Unsplash's Catholic photography for the feel, and the feel is right: a verse set over a nave reads as prayer, where the same words in a tinted box read as a notice. But the app already holds photographs of these two churches, and this codebase has a standing rule about that - a picture of some other parish's altar reads as a picture of THIS parish, and it is not. Using the parish's own interiors gets the same effect and tells the truth.
>
> Nothing is downloaded at runtime either, which matters on a phone in a church with one bar of signal.
>
> ## Why it changes by the day and not at random
>
> Random means a new picture on every re-render - the verse would flicker between photographs while you read it. Keyed to the date, the backdrop is steady all day and different tomorrow.

Complete Source Code — verseBackdrops.ts

```
/**
 * The photograph behind the Verse of the Day.
 *
 * ## Why the parish's own pictures and not stock
 *
 * The brief points at Unsplash's Catholic photography for the feel, and
 * the feel is right: a verse set over a nave reads as prayer, where the
 * same words in a tinted box read as a notice. But the app already holds
 * photographs of these two churches, and this codebase has a standing rule
 * about that - a picture of some other parish's altar reads as a picture
 * of THIS parish, and it is not. Using the parish's own interiors gets the
```

```
 * same effect and tells the truth.
 *
 * Nothing is downloaded at runtime either, which matters on a phone in a
 * church with one bar of signal.
 *
 * ## Why it changes by the day and not at random
 *
 * Random means a new picture on every re-render - the verse would flicker
 * between photographs while you read it. Keyed to the date, the backdrop
 * is steady all day and different tomorrow.
 */

export interface Backdrop {
  url: string;
  /**
   * How far down the frame the subject sits, as a CSS background-position.
   * A nave photographed from the back has its altar near the middle; a
   * patron in a niche has a face near the top. Centring everything crops
   * the subject out of a short card.
   */
  position: string;
}

const BACKDROPS: Backdrop[] = [
  { url: "/parish/mass-gospel.jpg", position: "50% 45%" },
  { url: "/parish/mary-help-history.jpg", position: "50% 35%" },
  { url: "/parish/san-roque-church.jpg", position: "50% 40%" },
  { url: "/parish/sacraments-christening.jpg", position: "50% 40%" },
];

/** A stable day number, so the pick holds from midnight to midnight. */
function dayIndex(date: Date): number {
  return Math.floor(
    Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()) / 86_400_000,
  );
}

export function backdropForDate(date: Date = new Date()): Backdrop {
  const n = dayIndex(date);
  // Guard the negative case: dates before 1970 would index backwards off
  // the array and hand back undefined.
  const i = ((n % BACKDROPS.length) + BACKDROPS.length) % BACKDROPS.length;
  return BACKDROPS[i];
}

export const ALL_BACKDROPS = BACKDROPS;
```

1.4.88 APP-158 — verses.ts

Code ID	APP-158
File Name	verses.ts
File Path	SanctDemo/src/lib/verses.ts
Language / Type	TypeScript
Lines	97
Functions / Classes	Verse (exported type); dayNumber (internal function); verseForDate (exported function)
Related Files (its imports)	./liturgical
Where It Is Used	src/components/VerseCard.tsx
How to Recognize It	Open SanctDemo/src/lib/verses.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Verse of the Day, chosen to suit the liturgical season.
>
> Replaces a single hardcoded verse (2 Peter 1:4) that never changed — which stopped reading as "of the day" the moment anyone opened the app twice.
>
> Local, not fetched. A daily-verse API would be one more thing to fail during a defense, in a church basement, on a phone with no signal; the app is offline-first everywhere else and this is no different.
>
> Same verse for everyone, same verse all day: the index comes from the calendar date, not from Math.random(). Two pilgrims standing beside each other must see the same verse, and it must not change while someone is reading it.

Complete Source Code — verses.ts

```
// Verse of the Day, chosen to suit the liturgical season.
//
// Replaces a single hardcoded verse (2 Peter 1:4) that never changed — which
// stopped reading as "of the day" the moment anyone opened the app twice.
//
// Local, not fetched. A daily-verse API would be one more thing to fail
// during a defense, in a church basement, on a phone with no signal; the app
// is offline-first everywhere else and this is no different.
//
// Same verse for everyone, same verse all day: the index comes from the
// calendar date, not from Math.random(). Two pilgrims standing beside each
// other must see the same verse, and it must not change while someone is
// reading it.

import { liturgicalDay, type LiturgicalSeason } from './liturgical'

export interface Verse {
  text: string
  reference: string
```

```
}

/**
 * Verses per season. Deliberately short — a verse that needs scrolling is not
 * a verse anyone reads on a bus.
 *
 * Scripture quoted from the New American Bible, Revised Edition, the
 * translation approved for liturgical use in the Philippines.
 */
const VERSES: Record<LiturgicalSeason, Verse[]> = {
  Advent: [
    { text: 'Prepare the way of the Lord, make straight his paths.', reference: 'Mark 1:3' },
    { text: 'The virgin shall conceive, and bear a son, and shall name him Emmanuel.', reference: 'Isaiah 7:14' },
    { text: 'Behold, I am the handmaid of the Lord. May it be done to me according to your word.', reference: 'Luke 1:38' },
    { text: 'Be patient, therefore, brothers, until the coming of the Lord.', reference: 'James 5:7' },
    { text: 'The people who walked in darkness have seen a great light.', reference: 'Isaiah 9:1' },
  ],
  Christmas: [
    { text: 'For today in the city of David a savior has been born for you who is Messiah and Lord.', reference: 'Luke 2:11' },
    { text: 'And the Word became flesh and made his dwelling among us.', reference: 'John 1:14' },
    { text: 'Glory to God in the highest and on earth peace to those on whom his favor rests.', reference: 'Luke 2:14' },
    { text: 'Mary kept all these things, reflecting on them in her heart.', reference: 'Luke 2:19' },
  ],
  'Ordinary Time': [
    { text: 'Your word is a lamp for my feet, a light for my path.', reference: 'Psalm 119:105' },
    { text: 'Come to me, all you who labor and are burdened, and I will give you rest.', reference: 'Matthew 11:28' },
    { text: 'Whatever you do, do everything for the glory of God.', reference: '1 Corinthians 10:31' },
    { text: 'Love is patient, love is kind. It is not jealous, it is not pompous.', reference: '1 Corinthians 13:4' },
    { text: 'The Lord is my shepherd; there is nothing I lack.', reference: 'Psalm 23:1' },
    { text: 'Be strong and steadfast; do not fear nor be dismayed, for the Lord is with you.', reference: 'Joshua 1:9' },
    { text: 'Do to others whatever you would have them do to you.', reference: 'Matthew 7:12' },
    { text: 'Cast all your worries upon him because he cares for you.', reference: '1 Peter 5:7' },
    { text: 'Blessed are the peacemakers, for they will be called children of God.', reference: 'Matthew 5:9' },
    { text: 'Faith is the realization of what is hoped for and evidence of things not seen.', reference: 'Hebrews 11:1' },
    { text: 'This is the day the Lord has made; let us rejoice in it and be glad.', reference: 'Psalm 118:24' },
    { text: 'He has given us his very great and precious promises.', reference: '2 Peter 1:4' },
  ],
  Lent: [
    { text: 'Return to me with your whole heart, with fasting, and weeping, and mourning.', reference: 'Joel 2:12' },
    { text: 'A clean heart create for me, God; renew within me a steadfast spirit.', reference: 'Psalm 51:12' },
    { text: 'Whoever wishes to come after me must deny himself, take up his cross, and follow me.', reference: 'Matthew 16:24' },
    { text: 'For where your treasure is, there also will your heart be.', reference: 'Matthew 6:21' },
    { text: 'The Lord is gracious and merciful, slow to anger and abounding in mercy.', reference: 'Psalm 145:8' },
  ],
  Triduum: [
    { text: 'Greater love has no one than this, to lay down one’s life for one’s friends.', reference: 'John 15:13' },
    { text: 'Father, into your hands I commend my spirit.', reference: 'Luke 23:46' },
    { text: 'By his stripes we were healed.', reference: 'Isaiah 53:5' },
  ],
  Easter: [
    { text: 'He is not here, for he has been raised just as he said.', reference: 'Matthew 28:6' },
    { text: 'I am the resurrection and the life; whoever believes in me, even if he dies, will live.', reference: 'John 11:25' },
    { text: 'Peace be with you. As the Father has sent me, so I send you.', reference: 'John 20:21' },
    { text: 'Blessed are those who have not seen and have believed.', reference: 'John 20:29' },
    { text: 'Death is swallowed up in victory.', reference: '1 Corinthians 15:54' },
  ],
}

/** Days since the epoch — a stable, timezone-local day number. */
```

```
function dayNumber(date: Date): number {
  return Math.floor(
    Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()) / 86_400_000,
  )
}

/**
 * The verse for a given day.
 *
 * Indexed by the day number so it advances once per calendar day and is
 * identical on every device — and so it cannot change between two renders of
 * the same screen, which a random pick would.
 */
export function verseForDate(date: Date): Verse & { season: LiturgicalSeason } {
  const season = liturgicalDay(date).season
  const pool = VERSES[season]
  const verse = pool[((dayNumber(date) % pool.length) + pool.length) % pool.length]
  return { ...verse, season }
}
```

1.4.89 APP-159 — visits.ts

Code ID	APP-159
File Name	visits.ts
File Path	SanctDemo/src/lib/visits.ts
Language / Type	TypeScript
Lines	159
Functions / Classes	VisitDoc (exported type); VISIT_COOLDOWN_MS (exported constant); ACTIVE_WINDOW_MS (exported constant); visitKey (exported function); shouldRecordVisit (exported function); ActiveLike (exported type); countActive (exported function); ChurchCount (exported type); VisitDay (exported type); VisitSummary (exported type); dateKey (internal function); summariseVisits (exported function); metresFrom (exported function)
Related Files (its imports)	./geo
Where It Is Used	src/lib/recordVisit.ts
How to Recognize It	Open SanctDemo/src/lib/visits.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Estimated church visits, and who is using the app.
>
> ## What a "visit" is here, and what it is not
>
> A visit is recorded when the app's own presence detection says the pilgrim is inside a parish's geofence - 100 metres of its verified coordinates, held for long enough to clear the dwell gate in presence.ts. That is proximity, not entry. A pilgrim standing in the street outside, or in the hall next door, is indistinguishable from one at the altar, and consumer GPS in a built-up area is routinely out by more than the radius.
>
> So the figure is called an ESTIMATED visit everywhere it appears, and nothing in this file should ever be described as proof that somebody entered a church.
>
> ## What is stored
>
> The church, the time, and the uid. No coordinates, no track, no history of where anybody went - only that somebody was near this parish at this time. The uid is there because it is the only way to tell twenty visits from twenty visitors, and because the security rules need it to refuse a record written on somebody else's behalf.

Complete Source Code — visits.ts

```
/**
 * Estimated church visits, and who is using the app.
 *
 * ## What a "visit" is here, and what it is not
 *
 * A visit is recorded when the app's own presence detection says the
```

```
 * pilgrim is inside a parish's geofence - 100 metres of its verified
 * coordinates, held for long enough to clear the dwell gate in
 * presence.ts. That is proximity, not entry. A pilgrim standing in the
 * street outside, or in the hall next door, is indistinguishable from
 * one at the altar, and consumer GPS in a built-up area is routinely
 * out by more than the radius.
 *
 * So the figure is called an ESTIMATED visit everywhere it appears,
 * and nothing in this file should ever be described as proof that
 * somebody entered a church.
 *
 * ## What is stored
 *
 * The church, the time, and the uid. No coordinates, no track, no
 * history of where anybody went - only that somebody was near this
 * parish at this time. The uid is there because it is the only way to
 * tell twenty visits from twenty visitors, and because the security
 * rules need it to refuse a record written on somebody else's behalf.
 */

import { haversineMeters, type Coordinates } from './geo'

export interface VisitDoc {
  /** Who. Needed to separate visits from visitors, and by the rules. */
  uid: string
  churchId: string
  /** ISO 8601. */
  at: string
}

/**
 * How long before the same person at the same church counts again.
 *
 * The geofence re-fires every time GPS drifts across its boundary, and
 * near a wall that can be several times a minute. Without this, one
 * pilgrim praying a rosary would register as a crowd.
 *
 * Two hours: long enough to cover a Mass and the time around it, short
 * enough that somebody returning for an evening devotion is counted
 * again, which they should be.
 */
export const VISIT_COOLDOWN_MS = 2 * 60 * 60 * 1000

/** The window the dashboard calls "active". */
export const ACTIVE_WINDOW_MS = 15 * 60 * 1000

/** The key a device remembers its last visit under. */
export function visitKey(uid: string, churchId: string): string {
  return `sanctiwalk.visit.${uid}.${churchId}`
}

/**
 * Whether to write a visit, given when this device last wrote one.
 *
 * A `lastAt` in the future is treated as "not yet due". A clock that
 * has jumped backwards would otherwise make every crossing count.
 */
export function shouldRecordVisit(lastAt: number | null, now: number): boolean {
  if (lastAt === null) return true
```

```
  if (lastAt > now) return false
  return now - lastAt >= VISIT_COOLDOWN_MS
}

export interface ActiveLike {
  lastActiveAt?: string
}

/** How many of these profiles were seen inside the active window. */
export function countActive(profiles: ActiveLike[], now: Date): number {
  const floor = now.getTime() - ACTIVE_WINDOW_MS
  let n = 0
  for (const profile of profiles) {
    if (!profile.lastActiveAt) continue
    const seen = new Date(profile.lastActiveAt).getTime()
    if (Number.isNaN(seen)) continue
    if (seen >= floor && seen <= now.getTime() + 60_000) n += 1
  }
  return n
}

export interface ChurchCount {
  churchId: string
  count: number
}

export interface VisitDay {
  /** Local calendar date, YYYY-MM-DD. */
  date: string
  count: number
}

export interface VisitSummary {
  /** Every visit on record, not only those in the chart's window. */
  total: number
  /** Distinct people behind those visits. */
  visitors: number
  churchesWithVisits: number
  byChurch: ChurchCount[]
  daily: VisitDay[]
}

function dateKey(date: Date): string {
  const m = `${date.getMonth() + 1}`.padStart(2, '0')
  const d = `${date.getDate()}`.padStart(2, '0')
  return `${date.getFullYear()}-${m}-${d}`
}

/** Everything the dashboard shows, from one pass over the records. */
export function summariseVisits(
  visits: VisitDoc[],
  days: number,
  now: Date,
): VisitSummary {
  const perChurch = new Map<string, number>()
  const people = new Set<string>()

  const order: string[] = []
  const perDay = new Map<string, number>()
```

```
  for (let back = days - 1; back >= 0; back--) {
    const day = new Date(now.getFullYear(), now.getMonth(), now.getDate() - back)
    const key = dateKey(day)
    order.push(key)
    perDay.set(key, 0)
  }

  for (const v of visits) {
    perChurch.set(v.churchId, (perChurch.get(v.churchId) ?? 0) + 1)
    if (v.uid) people.add(v.uid)

    const when = new Date(v.at)
    if (Number.isNaN(when.getTime())) continue
    const key = dateKey(when)
    if (perDay.has(key)) perDay.set(key, (perDay.get(key) ?? 0) + 1)
  }

  return {
    total: visits.length,
    visitors: people.size,
    churchesWithVisits: perChurch.size,
    byChurch: [...perChurch.entries()]
      .map(([churchId, count]) => ({ churchId, count }))
      .sort((a, b) => b.count - a.count || a.churchId.localeCompare(b.churchId)),
    daily: order.map(date => ({ date, count: perDay.get(date) ?? 0 })),
  }
}

/**
 * Distance from a position to a church, for the honesty note on the
 * dashboard. Re-exported so the admin screen does not reach into geo.
 */
export function metresFrom(a: Coordinates, b: Coordinates): number {
  return haversineMeters(a, b)
}
```

1.5 Parish Data

1.5.1 APP-160 — data.ts

Code ID	APP-160
File Name	data.ts
File Path	SanctDemo/src/data.ts
Language / Type	TypeScript
Lines	635
Functions / Classes	RosaryMystery (exported type); Ministry (exported type); Sacrament (exported type); QuizQuestion (exported type); ROSARY_MYSTERIES (exported constant); MINISTRY_IMAGES (exported constant); MINISTRIES (exported constant); ministriesForParish (exported function); parishHasConfirmedMinistries (exported function); SACRAMENT_IMAGES (exported constant); SACRAMENTS (exported constant); PILGRIM_QUIZ (exported constant); ParishMassSchedule (exported type); MASS_SCHEDULES (exported constant); ParishContact (exported type); PARISH_CONTACTS (exported constant); PARISH_PATRON_IMAGES (exported constant); PARISH_HEADER_IMAGES (exported constant); PARISH_HEADER_FACES (exported constant); PARISH_PATRON_SAINTS (exported constant); ROUTES (exported constant); BADGES (exported constant)
Related Files (its imports)	./types
Where It Is Used	server.ts, src/admin/AdminAnalytics.tsx, src/admin/AdminAvailability.tsx, src/admin/AdminContent.tsx, src/admin/AdminItems.tsx, src/admin/AdminMassSchedule.tsx, src/admin/ParishStatus.tsx, src/components/ArTour.tsx, src/components/ChurchHistory.tsx, src/components/ChurchHistoryCard.tsx, src/components/Dashboard.tsx, src/components/DioceseMap.tsx, src/components/DioceseMapLive.tsx, src/components/ExploreTab.tsx …
How to Recognize It	Open SanctDemo/src/data.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — data.ts

```
import { Route, Badge } from "./types";

export interface RosaryMystery {
  id: string;
  category: "Joyful" | "Sorrowful" | "Glorious" | "Luminous";
  title: string;
  days: string;
  prayers: string[];
}

export interface Ministry {
```

```
  id: string;
  name: string;
  description: string;
  requirements: string[];
}

export interface Sacrament {
  id: string;
  name: string;
  description: string;
  requirements: string[];
  scheduleDetails: string;
}

export interface QuizQuestion {
  id: string;
  question: string;
  options: string[];
  correctAnswerIndex: number;
  explanation: string;
}

export const ROSARY_MYSTERIES: RosaryMystery[] = [
  {
    id: "joyful",
    category: "Joyful",
    title: "The Joyful Mysteries",
    days: "Mondays & Saturdays",
    prayers: [
      "1. The Annunciation: Mary accepts her call to be the Mother of God.",
      "2. The Visitation: Mary visits her cousin Elizabeth who is with child.",
      "3. The Nativity: The birth of our Lord Jesus Christ in a humble stable.",
      "4. The Presentation: Jesus is presented in the Temple by Mary and Joseph.",
      "5. The Finding in the Temple: Jesus is found teaching the elders."
    ]
  },
  {
    id: "sorrowful",
    category: "Sorrowful",
    title: "The Sorrowful Mysteries",
    days: "Tuesdays & Fridays",
    prayers: [
      "1. The Agony in the Garden: Jesus prays in Gethsemane.",
      "2. The Scourging at the Pillar: Jesus is brutally whipped by Roman soldiers.",
      "3. The Crowning with Thorns: Jesus is mocked with a crown of thorns.",
      "4. The Carrying of the Cross: Jesus carries His heavy cross to Calvary.",
      "5. The Crucifixion: Jesus dies on the cross for our salvation."
    ]
  },
  {
    id: "glorious",
    category: "Glorious",
    title: "The Glorious Mysteries",
    days: "Wednesdays & Sundays",
    prayers: [
      "1. The Resurrection: Jesus rises from the dead on Easter Sunday.",
      "2. The Ascension: Jesus ascends to Heaven forty days after Easter.",
      "3. The Descent of the Holy Spirit: Pentecost descends upon the Apostles.",
      "4. The Assumption of Mary: Mary is taken body and soul into Heaven.",
```

```
      "5. The Coronation of Mary: Mary is crowned Queen of Heaven and Earth."
    ]
  },
  {
    id: "luminous",
    category: "Luminous",
    title: "The Luminous Mysteries",
    days: "Thursdays",
    prayers: [
      "1. The Baptism in the Jordan: Jesus is baptized by John the Baptist.",
      "2. The Wedding at Cana: Jesus performs His first miracle at Mary's request.",
      "3. The Proclamation of the Kingdom: Jesus calls all to repentance and faith.",
      "4. The Transfiguration: Jesus reveals His divine glory on Mount Tabor.",
      "5. The Institution of the Eucharist: Jesus institutes the Holy Mass."
    ]
  }
];

/**
 * A photograph per ministry, taken from the parish's own PDF.
 *
 * The photograph on each page is taken to belong to the ministry whose text
 * is on that page. That is an inference from the document's order, not
 * something the PDF states - worth an eye over before anyone relies on it.
 */
export const MINISTRY_IMAGES: Record<string, string> = {
  "min-altar-servers": "/ministries/min-altar-servers.jpg",
  "min-emhc": "/ministries/min-emhc.jpg",
  "min-lectors": "/ministries/min-lectors.jpg",
  "min-greeters": "/ministries/min-greeters.jpg",
  "min-soccom": "/ministries/min-soccom.jpg",
  "min-apostolado": "/ministries/min-apostolado.jpg",
  "min-mother-butlers": "/ministries/min-mother-butlers.jpg",
  "min-youth": "/ministries/min-youth.jpg",
  "min-catechetical": "/ministries/min-catechetical.jpg",
  "min-choir-cfc": "/ministries/min-choir-cfc.jpg",
  "min-choir-teatro": "/ministries/min-choir-teatro.jpg",
  "min-choir-singing-servants": "/ministries/min-choir-singing-servants.jpg",
  "min-choir-charismatic": "/ministries/min-choir-charismatic.jpg",
  "min-choir-marian-youth": "/ministries/min-choir-marian-youth.jpg",
  "min-choir-apostleship": "/ministries/min-choir-apostleship.jpg",
};

/**
 * Mary Help of Christians' own ministries, from the list the parish supplied.
 *
 * The words are theirs, verbatim and in Tagalog. They are the ones who get to
 * say what their ministries do, and translating or tightening it here would
 * be putting the app's voice in their mouth.
 *
 * Requirements are EMPTY on purpose. The parish's document states none, and
 * the placeholder list this replaces had entries like "Age 10-25" that nobody
 * at the parish had written. An empty list hides that section rather than
 * inventing conditions for joining somebody else's ministry.
 */
export const MINISTRIES: Ministry[] = [
  {
    id: "min-altar-servers",
    name: "Ministry of Altar Servers (MAS)",
```

```
    description:
      "Ang Ministry of Altar Servers (MAS) ay isang ministeryo na tumutugon sa marangal na tungkulin ng paglilingkod sa dambana ng Panginoon. Binubuo ito ng mga kabataang may bukas na puso at matatag na pananampalataya na kusang-loob na naglilingkod sa maayos at taimtim na pagdiriwang ng Banal na Liturhiya at iba pang sakramental na gawain ng Simbahan. Bilang mga lingkod ng dambana, hinuhubog ng ministeryong ito ang kanilang disiplina, kabanalan, at wastong asal, upang maging buhay na saksi ng kaayusan, paggalang, at kabanalan sa loob ng simbahan.",
    requirements: [],
  },
  {
    id: "min-emhc",
    name: "Extraordinary Ministers of Holy Communion (EMHC)",
    description:
      "Ang Extraordinary Ministers of Holy Communion (EMHC) ay sumusuporta sa pari, lalo na sa mga Misa na may malalaking kongregasyon o kapag kailangang dalhin ang Komunyon sa mga maysakit, matatanda, o nasa bahay. Ang mga EMHC ay maingat na pinipili batay sa pananampalataya, moral na katangian, at aktibong pakikibahagi sa buhay ng Simbahan, at ang kanilang ministeryo ay nakaugat sa pagpapakumbaba at paglilingkod. Bilang kinatawan ng pamayanan, tinutulungan nilang matiyak na matatanggap ng mga mananampalataya ang Katawan at Dugo ni Kristo nang may dignidad at debosyon, at pinalalawak nila ang pastoral na pangangalaga ng Simbahan sa mga hindi makakadalo sa Misa.",
    requirements: [],
  },
  {
    id: "min-lectors",
    name: "Ministry of Lectors and Commentators (MLC)",
    description:
      "Ang Ministry of Lectors and Commentators (MLC) ay nakatuon sa malinaw, marangal, at makahulugang pagpapahayag ng Salita ng Diyos at ng mga paanyaya sa banal na pagdiriwang. Sa pamamagitan ng maayos na pagbasa ng Banal na Kasulatan at disiplinadong pagbibigay-gabay sa sambayanan bilang mga komentarista, tinutulungan ng ministeryong ito ang pamayanan na ganap na makilahok sa liturhiya. Sa kanilang paghahanda, disiplina, at paggalang sa kabanalan ng gawain, nagiging daluyan sila ng kaayusan, pagkaunawa, at mas malalim na pakikinig sa tinig ng Diyos.",
    requirements: [],
  },
  {
    id: "min-greeters",
    name: "Ministry of Greeters and Collectors (MGC)",
    description:
      "Ang Ministry of Greeters and Collectors (MGC) ay nagsisilbing unang mukha ng pamayanang Kristiyano sa pamamagitan ng magalang na pagtanggap, maayos na paggabay, at mapagkalingang presensiya sa bawat mananampalataya. Kasabay nito, sila ay pinagkakatiwalaan sa tapat, disiplinado, at may pananagutang pangangalap ng mga handog, na nagsisilbing kongkretong pagpapahayag ng sakripisyo ng sambayanan para sa patuloy na misyon ng Simbahan. Sa kanilang pagpapakumbaba, integridad, at paggalang sa kabanalan ng gawain, pinagtitibay ng ministeryong ito ang kaayusan, pagkakaisa, at diwa ng paglilingkod sa loob ng pamayanang sumasamba.",
    requirements: [],
  },
  {
    id: "min-soccom",
    name: "Social Communications Ministry (SOCCOM)",
    description:
      "Ang Social Communications Ministry (SOCCOM) ay nagpapalaganap ng mensahe ng Ebanghelyo sa pamamagitan ng wasto, responsable, at makabagong paggamit ng midya at komunikasyon. Sa pagdodokumento, pag-uulat, at pagpapahayag ng mga gawain ng Simbahan, nagsisilbi silang tinig at larawan ng pananampalatayang buhay ng pamayanan. Pinagtitibay ng SOCCOM ang ugnayan ng Simbahan at sambayanan, at ginagawang daluyan ang komunikasyon upang higit na maipahayag ang misyon ng Simbahan sa makabagong panahon.",
    requirements: [],
  },
  {
    id: "min-apostolado",
    name: "Apostolado ng Panalangin (AnP)",
    description:
      "Ang Apostolado ng Panalangin (AnP) ay naglalayong palalimin ang ugnayan ng mananampalataya sa Diyos sa pamamagitan ng taimtim, sama-sama, at tuloy-tuloy na panalangin. Sa pag-aalay ng sarili, gawain, at paghihirap sa Panginoon, nagiging espirituwal na haligi ang ministeryong ito ng Simbahan na sumusuporta sa misyon nito sa pamamagitan ng pananampalataya at sakripisyo.",
    requirements: [],
  },
  {
    id: "min-mother-butlers",
```

```
    name: "Mother Butlers Guild (MBG)",
    description:
      "Ang Mother Butlers Guild (MBG) ay isang ministeryong nakatuon sa masinop at mapagkalingang pangangalaga sa mga banal na kagamitan at sa kaayusan ng altar at simbahan. Sa kanilang tahimik ngunit mahalagang paglilingkod, ipinahahayag nila ang paggalang sa kabanalan ng liturhiya at ang diwa ng inang naglilingkod nang may pagmamahal at pananagutan.",
    requirements: [],
  },
  {
    id: "min-youth",
    name: "Ministry on Youth Affairs",
    description:
      "Ang Ministry on Youth Affairs ay nagsisilbing gabay at katuwang ng kabataan sa kanilang paghubog bilang responsableng Kristiyano at aktibong kasapi ng Simbahan at lipunan. Sa pamamagitan ng pormasyon, pakikilahok, at paglilingkod, inihahanda ng ministeryong ito ang kabataan bilang mga pinuno ng kasalukuyan at hinaharap na Simbahan.",
    requirements: [],
  },
  {
    id: "min-catechetical",
    name: "Catechetical Ministry",
    description:
      "Ang Catechetical Ministry ay may pananagutang ipahayag at ituro ang mga saligan ng pananampalatayang Katoliko sa malinaw, tapat, at sistematikong paraan. Sa kanilang pagtuturo at paggabay, pinalalalim nila ang pagkaunawa ng sambayanan sa doktrina at hinuhubog ang buhay-Kristiyano na nakaugat sa Ebanghelyo.",
    requirements: [],
  },
  {
    id: "min-choir-cfc",
    name: "Couples for Christ Choir",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
  },
  {
    id: "min-choir-teatro",
    name: "Teatro Pilipino Choir",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
  },
  {
    id: "min-choir-singing-servants",
    name: "Singing Servants of Christ",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
  },
  {
    id: "min-choir-charismatic",
    name: "Charismatic Music Ministry",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
```

```
  },
  {
    id: "min-choir-marian-youth",
    name: "Marian Youth Choir",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may
pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at
maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
  },
  {
    id: "min-choir-apostleship",
    name: "Apostleship of Prayer Chorale",
    description:
      "Nangangasiwa sa angkop na paggamit ng musika sa liturhiya upang suportahan ang panalangin at palalimin ang pakikilahok ng mga mananampalataya. Ito ay may
pananagutan sa pagpili at paghahanda ng mga himno at awiting liturhikal para sa mga Banal na Misa at iba pang pagdiriwang ng Simbahan. Sa pamamagitan ng taimtim at
maayos na musika, naglilingkod ito para sa higit na kaluwalhatian ng Diyos.",
    requirements: [],
  },
];

/**
 * A photograph per sacrament, on the same terms as MINISTRY_IMAGES: the
 * parish's own, or the placeholder. A christening photo from some other
 * parish captioned "Holy Baptism" would read as this parish's christening.
 */
/**
 * Which ministries each parish actually has.
 *
 * MINISTRIES below is Mary Help of Christians' own list, supplied by
 * that parish. It was being shown for every parish in the app, so a
 * pilgrim at San Roque was offered fifteen groups the cathedral had
 * never been asked about, and could apply to them - the application
 * would arrive at a parish with no such ministry to receive it.
 *
 * A parish appears here only once it has told us what it runs. San
 * Roque has confirmed its altar servers and nothing else yet, so that
 * is all it lists. A parish not in this table lists nothing, which is
 * the honest answer to "what ministries does it have" when nobody has
 * asked it: better an empty list that says so than fifteen borrowed
 * from somebody else.
 */
const PARISH_MINISTRY_IDS: Record<string, string[] | "all"> = {
  "route-mhcp": "all",
  "route-src": ["min-altar-servers"],
};

/** The ministries this parish runs, in the order they are listed. */
export function ministriesForParish(parishId: string | undefined): Ministry[] {
  const allowed = PARISH_MINISTRY_IDS[parishId ?? ""];
  if (allowed === "all") return MINISTRIES;
  if (!allowed) return [];
  return MINISTRIES.filter(m => allowed.includes(m.id));
}

/** True when the parish has told us its ministries at all. */
export function parishHasConfirmedMinistries(parishId: string | undefined): boolean {
  return PARISH_MINISTRY_IDS[parishId ?? ""] !== undefined;
}
```

```
export const SACRAMENT_IMAGES: Record<string, string> = {
  "sac-baptism": "/parish/sacraments-christening.jpg",
  "sac-confirmation": "/sacraments/sac-confirmation.jpg",
  "sac-matrimony": "/sacraments/sac-matrimony.jpg",
  "sac-eucharist": "/parish/mass-gospel.jpg",
  "sac-confession": "/sacraments/sac-confession.jpg",
};

export const SACRAMENTS: Sacrament[] = [
  {
    id: "sac-baptism",
    name: "Holy Baptism",
    description: "The gateway to life in the Spirit, which washes away original sin and welcomes the soul into the Body of Christ.",
    requirements: [
      "Child's Birth Certificate (PSA copy)",
      "Marriage Contract of Parents",
      "Godparent (Ninong/Ninang) Catholic Certificates",
      "Pre-Baptismal Seminar attendance"
    ],
    scheduleDetails: "Saturdays and Sundays: 10:00 AM (Group Baptism)"
  },
  {
    id: "sac-confirmation",
    name: "Sacrament of Confirmation",
    description: "Imparts the special strength of the Holy Spirit to witness to the Christian faith actively.",
    requirements: [
      "Baptismal Certificate (with 'For Confirmation' annotation)",
      "Parish Catechism seminar certificate",
      "One Sponsor certificate copy"
    ],
    scheduleDetails: "Annual Diocesan Schedule / Parish Feast Day masses"
  },
  {
    id: "sac-matrimony",
    name: "Holy Matrimony",
    description: "A covenant by which a man and a woman establish a lifelong partnership directed toward the good of the spouses.",
    requirements: [
      "PSA Certificate of Singleness (CENOMAR)",
      "PSA Birth Certificates",
      "New copies of Baptismal & Confirmation Certificates",
      "Pre-Cana Seminar completion",
      "Canonical Interview with Father Paul Woo"
    ],
    scheduleDetails: "Tuesday to Saturday: 9:00 AM, 1:30 PM, or 3:30 PM (By advanced booking)"
  },
  {
    id: "sac-eucharist",
    name: "First Holy Communion",
    description: "The first reception of Christ's true Body and Blood, uniting children intimately with Jesus.",
    requirements: [
      "Baptismal Certificate",
      "Active enrollment in Parish Catechesis",
      "First Confession clearance"
    ],
    scheduleDetails: "Usually organized through local Catholic schools & Sunday school batches"
  },
  {
    id: "sac-confession",
```

```
    name: "Sacrament of Reconciliation (Confession)",
    description: "The merciful healing sacrament where sins are forgiven and the soul is fully restored to grace.",
    requirements: [
      "Sincere examination of conscience",
      "Intent to confess and repent of sins"
    ],
    scheduleDetails: "Every first Friday of the month (before Mass) or by special appointment"
  }
];

export const PILGRIM_QUIZ: QuizQuestion[] = [
  {
    id: "q1",
    question: "When was the Mary Help of Christians Parish in Maypajo established?",
    options: ["1942", "1952", "1962", "2003"],
    correctAnswerIndex: 1,
    explanation: "The Mary Help of Christians community in Maypajo, Caloocan, was founded in 1952, establishing a central place for Marian devotion."
  },
  {
    id: "q2",
    question: "Who is the beloved patron saint of Caloocan, invoked against pestilence and plagues?",
    options: ["Saint Augustine", "Saint James", "San Roque (Saint Roch)", "San Lorenzo Ruiz"],
    correctAnswerIndex: 2,
    explanation: "San Roque is the beloved patron saint of the San Roque Cathedral Parish, known as the healer of the sick and plagues."
  },
  {
    id: "q3",
    question: "What does PWA stand for, which solves the problem of cross-platform app development for students on Windows?",
    options: ["Personal Web Application", "Progressive Web App", "Public Wireless Access", "Programmed Web Asset"],
    correctAnswerIndex: 1,
    explanation: "PWA stands for Progressive Web App. It lets you develop single-codebase cross-platform mobile apps easily from a Windows laptop without Apple
Xcode!"
  },
  {
    id: "q4",
    question: "The Diocese of Kalookan was officially established in what year?",
    options: ["1952", "1985", "2003", "2015"],
    correctAnswerIndex: 2,
    explanation: "The Diocese of Kalookan was carved out from the Archdiocese of Manila and officially established in 2003."
  },
  {
    id: "q5",
    question: "What is the parish book called in Mary Help of Christians Parish that recounts the church history?",
    options: ["Agos", "Sinag", "Gabay", "Liwanag"],
    correctAnswerIndex: 0,
    explanation: "The book is called 'Agos' (Flow), which records the beautiful spiritual history, community streams, and Marian miracles of the Maypajo parish."
  }
];

export interface ParishMassSchedule {
  // Mirrors the `coordinatesVerified` pattern already used for parish
  // coordinates elsewhere in this codebase: true only when the times below
  // came from the parish's own published schedule. False means the schedule
  // is a stand-in and must be visibly flagged as such wherever it renders —
  // never presented as if it were the parish's real Mass times.
  scheduleVerified: boolean;
  schedule: { day: string; time: string }[];
}
```

```
export const MASS_SCHEDULES: Record<string, ParishMassSchedule> = {
  "route-mhcp": {
    scheduleVerified: true,
    schedule: [
      { day: "Monday", time: "6:00 AM" },
      { day: "Tuesday", time: "6:00 AM" },
      { day: "Wednesday", time: "6:00 AM, 6:00 PM" },
      { day: "Thursday", time: "6:00 AM, 6:00 PM" },
      { day: "Friday", time: "6:00 AM, 6:00 PM" },
      { day: "Saturday", time: "6:00 AM, 6:00 PM" },
      { day: "Sunday", time: "6:00 AM, 7:30 AM, 9:00 AM, 10:30 AM, 4:30 PM, 6:00 PM" },
    ],
  },
  // PLACEHOLDER SCHEDULE — not from the parish. San Roque Cathedral's real
  // Mass schedule loads dynamically on the diocese site and could not be
  // scraped, so this is a deliberately-different stand-in that exists only
  // so the per-parish switch can be demoed before the parish office
  // supplies real times. scheduleVerified stays false until someone replaces
  // the array below with confirmed times from the parish and flips it to
  // true — do not remove the flag, and do not treat these times as real.
  "route-src": {
    scheduleVerified: false,
    schedule: [
      { day: "Monday", time: "6:00 PM" },
      { day: "Tuesday", time: "6:00 PM" },
      { day: "Wednesday", time: "6:00 PM" },
      { day: "Thursday", time: "6:00 PM" },
      { day: "Friday", time: "6:00 PM" },
      { day: "Saturday", time: "6:00 AM, 5:00 PM" },
      { day: "Sunday", time: "6:30 AM, 8:00 AM, 10:00 AM, 5:00 PM" },
    ],
  },
};

export interface ParishContact {
  address: string;
  phone?: string;
  email?: string;
}

// Only Mary Help of Christians' contact details were ever actually scraped
// from the diocese site. San Roque's are not yet available — MassSchedule
// must say so rather than leaving blank fields or borrowing MHCP's.
export const PARISH_CONTACTS: Record<string, ParishContact> = {
  "route-mhcp": {
    address: "J.P Rizal Street, Maypajo, Caloocan City",
    phone: "(8) 288-7482 / 0945 253 5329",
    email: "maryhelpofchristians@dioceseofkalookan.ph",
  },
  "route-src": {
    address: "A. Mabini St, Caloocan City",
  },
};

// Patron saint per live parish, for the map screen's parish cards. Route
// itself has no dedicated field for this, so it's pulled from the parish's
// own station content above rather than invented: MHCP's "mhcp-patron"
// station names Maria Auxiliadora, and SRC's "src-statue" station names
// San Roque.
```

```
/**
 * A photograph of each parish's patron image, for the collapsing header on
 * Home.
 *
 * Deliberately empty. No parish photography has been collected yet — the
 * field team's sheet asks for it — and the header renders the name alone
 * when a parish is missing from here, which is the same thing it looks like
 * once scrolled. That is the honest empty state.
 *
 * Do NOT fill this with stock imagery. This is the largest element on the
 * home screen, and a generic church photograph presented as *this* parish's
 * patron is the most visible possible version of the mistake the rest of
 * this file avoids (see coordinatesVerified / scheduleVerified). Add an
 * entry only when someone has photographed that parish's own image:
 *
 *   "route-mhcp": "/parish/mary-help-patron.jpg",
 */
export const PARISH_PATRON_IMAGES: Record<string, string | undefined> = {
  // The parish's own image of Maria Auxiliadora in its niche, photographed by
  // the client. Converted from the original iPhone HEIC — which no browser
  // outside Safari can display — and resized from 3024x4032 to 1200px wide.
  "route-mhcp": "/parish/mary-help-patron.jpg",

  // San Roque's patron, extracted from the PSD the parish supplied. Until
  // now this parish had no photograph at all and its history card fell back
  // to the placeholder illustration.
  "route-src": "/parish/san-roque-hero.jpg",
}

/**
 * The image behind the welcome header, which is a different picture from the
 * one on the history card: the header needs something that reads at a glance
 * behind text, the card needs the subject whole.
 */
export const PARISH_HEADER_IMAGES: Record<string, string | undefined> = {
  "route-mhcp": "/parish/mary-help-hero.jpg",
  "route-src": "/parish/san-roque-hero.jpg",
}

/**
 * How far down each header photograph the patron's face sits, as a fraction
 * of the picture's height.
 *
 * The home band collapses to a bar about 90px tall as the page scrolls, and
 * which part of the photograph survives that is decided by the picture's own
 * composition — so no single crop can serve both of these. Mary Help's is a
 * landscape frame with the faces just past the middle; San Roque's is a
 * portrait with his head in the top third. Left to one rule, Mary's face
 * ended up behind the sheet's rounded lip and only the crown and halo showed.
 *
 * Measured by opening each file and reading off where the face is:
 *
 *   mary-help-hero.jpg   1200x900   Mary's face y 455-575, the child's
 *                                   455-600; centred on the pair at y 535
 *   san-roque-hero.jpg   902x1265   head and hat y 200-445, centred at 322
 *
 * ParishWelcomeHeader turns this into the offset that lands the face in the
 * middle of the collapsed bar. A parish with no entry here — including one
 * whose photograph was uploaded through the admin portal, where nobody has
```

```
 * said where the face is — keeps the plain top-aligned crop.
 */
export const PARISH_HEADER_FACES: Record<string, number | undefined> = {
  "route-mhcp": 0.59,
  "route-src": 0.255,
}

export const PARISH_PATRON_SAINTS: Record<string, string> = {
  "route-mhcp": "Maria Auxiliadora (Mary Help of Christians)",
  "route-src": "San Roque",
};

export const ROUTES: Route[] = [
  {
    id: "route-mhcp",
    name: "Mary Help of Christians Parish Guide",
    description: "Pilgrim trail inside the Maypajo Parish, exploring the historical altar, the sacred image of Maria Auxiliadora, and spiritual milestones.",
    location: "Maypajo, Caloocan City",
    distanceKm: 0.5,
    durationMins: 20,
    difficulty: "Easy",
    estimatedSteps: 600,
    category: "Marian Church",
    accentColor: "from-[var(--color-brand-primary)] to-[var(--color-brand-primary-dark)]",
    coordinates: { lat: 14.637702, lng: 120.97344 },
    coordinatesVerified: true,
    geofenceRadius: 100,
    status: "live",
    stations: [
      {
        id: "mhcp-altar",
        name: "Main Altar & Tabernacle",
        description: "The central sanctuary of Mary Help of Christians Parish, designed for deep worship and Eucharist celebration.",
        history: "Consecrated as the spiritual center of the parish. It holds the tabernacle and displays majestic holy figures depicting the Salesian devotions.",
        reflection: "Quiet your heart as you stand before the Altar. In the stillness, listen to God's whisper. What are you most grateful for today?",
        coordinates: { lat: 14.6305, lng: 120.9711 },
        imageUrl: "https://dioceseofkalookan.ph/wp-content/uploads/2020/12/maryhelpPic1-1-1024x680.jpg",
        qrCode: "MHCP-ALTAR"
      },
      {
        id: "mhcp-patron",
        name: "Maria Auxiliadora Patroness Statue",
        description: "The majestic crowned image of Mary Help of Christians holding the Child Jesus with open arms.",
        history: "A title promoted strongly by St. John Bosco. This specific parish image has been a source of spiritual comfort, community miracles, and grace since
the parish's founding.",
        reflection: "Mary holds Jesus, presenting Him to us as our ultimate help. Where in your life do you need a helper today? Sincere prayer is never ignored.",
        coordinates: { lat: 14.6308, lng: 120.9715 },
        qrCode: "MHCP-PATRON"
      },
      {
        id: "mhcp-bapt",
        name: "Baptismal Sanctuary Font",
        description: "The holy font where infants and adults enter into God's family, baptized in water and the Spirit.",
        history: "Located at the church entrance, symbolizing baptism as the doorway to all other sacraments. Built in high-polish granite.",
        reflection: "Water brings life and cleanses. Remember that your life has a divine purpose. How can you bring a clean start and hope to someone who is down
today?",
        coordinates: { lat: 14.6302, lng: 120.9708 },
        qrCode: "MHCP-BAPT"
      }
```

```
    ]
  },
  {
    id: "route-src",
    name: "San Roque Cathedral Parish Tour",
    description: "Pilgrimage of Caloocan's primary cathedral, established in 1815, home to Saint Roch the healer.",
    location: "A. Mabini St, Caloocan City",
    distanceKm: 0.8,
    durationMins: 25,
    difficulty: "Easy",
    estimatedSteps: 1100,
    category: "Historic Heritage",
    accentColor: "from-[var(--color-brand-primary)] to-[var(--color-brand-text)]",
    coordinates: { lat: 14.651647, lng: 120.972648 },
    coordinatesVerified: true,
    geofenceRadius: 100,
    status: "live",
    stations: [
      {
        id: "src-sanctuary",
        name: "Cathedral Central Sanctuary",
        description: "The solemn altar and seat of the Bishop of Kalookan, hosting major religious feasts and historical assemblies.",
        history: "Originally built in 1815, the church witnessed the Philippine Revolution as a stronghold. Consecrated as a Cathedral in 2003 under the first
bishop.",
        reflection: "Reflect on the resilience of this cathedral, standing strong through wars and natural events. How resilient is your spirit in times of trial?",
        coordinates: { lat: 14.6514, lng: 120.9739 },
        qrCode: "SRC-ALTAR"
      },
      {
        id: "src-statue",
        name: "Miraculous Statue of San Roque",
        description: "The highly venerated statue of San Roque, representing his life of sacrifice and healing.",
        history: "Venerated by the community. San Roque was a layman who cared for plague-stricken people in Europe, and is invoked for protection against sickness
and health crises.",
        reflection: "San Roque put his life at risk to nurse others. Who among your sick neighbors, friends, or family members can you pray for or reach out to
today?",
        coordinates: { lat: 14.6518, lng: 120.9742 },
        qrCode: "SRC-ROQUE"
      }
    ]
  }
];

export const BADGES: Badge[] = [
  {
    id: "badge-1",
    name: "Mary Help Disciple",
    description: "Check in at the Mary Help of Christians Parish Maypajo altar.",
    icon: "Footprints"
  },
  {
    id: "badge-2",
    name: "Cathedral Pilgrim",
    description: "Complete all stations of the San Roque Cathedral Parish guide.",
    icon: "Compass"
  },
  {
    id: "badge-3",
    name: "Agos Scholar",
```

```
    description: "Scan pages of the 'Agos' book in the interactive AR viewer.",
    icon: "BookOpen"
  },
  {
    id: "badge-4",
    name: "Faith Defender",
    description: "Complete the Pilgrim Quiz with a perfect 5/5 score!",
    icon: "Award"
  },
  {
    id: "badge-5",
    name: "Community Active",
    description: "Post a spiritual reflection comment on any church station.",
    icon: "Heart"
  }
];
```

1.5.2 APP-161 — types.ts

Code ID	APP-161
File Name	types.ts
File Path	SanctDemo/src/types.ts
Language / Type	TypeScript
Lines	214
Functions / Classes	Station (exported type); Route (exported type); UserProgress (exported type); Badge (exported type); Reflection (exported type); UserRole (exported type); Church (exported type); UserProfile (exported type); ApplicationStatus (exported type); APPLICATION_STATUSES (exported constant); ApplicationEvent (exported type); ApplicationDoc (exported type); NotificationDoc (exported type)
Related Files (its imports)	No project-local imports.
Where It Is Used	src/admin/AdminApp.tsx, src/admin/AdminApplicationDetail.tsx, src/admin/AdminApplications.tsx, src/admin/AdminDashboard.tsx, src/admin/StatusBadge.tsx, src/components/ArTour.tsx, src/components/ChangeParishModal.tsx, src/components/ChurchHistory.tsx, src/components/CompanionTab.tsx, src/components/Dashboard.tsx, src/components/EditProfileCard.tsx, src/components/ExploreTab.tsx, src/components/MapTab.tsx, src/components/MassSchedule.tsx …
How to Recognize It	Open SanctDemo/src/types.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — types.ts

```
export interface Station {
  id: string;
  name: string;
  description: string;
  history: string;
  reflection: string;
  coordinates: { lat: number; lng: number };
  /**
   * A recorded narration for this station, once one exists.
   *
   * This replaces `audioDuration`, which was a string like "1:30" on every
   * station while no audio file had ever been recorded — the app was
   * advertising narration that did not exist. A duration is a property OF a
   * recording, so it is read from the file at playback rather than typed by
   * hand, and with no file there is nothing to claim.
   */
  audioUrl?: string;
  /**
   * A photograph OF THIS STATION. Optional, and absent is the honest state
   * for most: four of the five stations carried Unsplash stock photographs of
   * unrelated churches, which presented another parish's baptismal font as
```

```
   * this one's. Same principle as `scheduleVerified` in data.ts — show the
   * placeholder, never the plausible-looking wrong thing.
   */
  imageUrl?: string;
  qrCode: string;
}

export interface Route {
  id: string;
  name: string;
  description: string;
  location: string;
  distanceKm: number;
  durationMins: number;
  difficulty: "Easy" | "Moderate" | "Challenging";
  stations: Station[];
  estimatedSteps: number;
  category: "Marian Church" | "Historic Heritage" | "Spiritual Retreat" | "Wellness Trail";
  accentColor: string;
  coordinates?: { lat: number; lng: number };
  coordinatesVerified?: boolean;
  geofenceRadius?: number;
  status?: "live" | "coming_soon";
}

export interface UserProgress {
  completedStations: string[]; // station ids
  completedRoutes: string[]; // route ids
  badges: string[]; // badge ids
  steps: number;
  distanceKm: number;
  points: number;
}

export interface Badge {
  id: string;
  name: string;
  description: string;
  icon: string;
  dateEarned?: string;
}

export interface Reflection {
  id: string;
  stationId: string;
  routeId: string;
  date: string;
  text: string;
  mood: string;
}

// ---------------------------------------------------------------------------
// Accounts, churches and applications
//
// These three travel together: a person belongs to exactly one church, a
// church has exactly one administrator, and an application belongs to the
// church its applicant belongs to. The security rules enforce all three; the
// types here are so the app cannot casually write a shape the rules will
// reject at the last moment, in front of the user.
```

```
// ---------------------------------------------------------------------------

/** What a signed-in account is allowed to be. Never chosen by the client. */
export type UserRole = "user" | "church_admin";

/**
 * A parish, as an account belongs to it.
 *
 * `id` is the route id ("route-mhcp"), not a separate key. The parish's
 * admin-managed content already lives at parishContent/{routeId} and the
 * rules authorise that write with the same id, so a second identifier would
 * mean two ways to name one parish and a mapping table to keep in step.
 */
export interface Church {
  id: string;
  name: string;
  location: string;
  contactEmail?: string;
  contactNumber?: string;
  /**
   * The one administrator. Immutable from the client - that immutability IS
   * the one-admin-per-church rule, since nothing in the app can assign or
   * reassign it.
   */
  adminUid?: string;
  createdAt?: string;
  updatedAt?: string;
}

/**
 * The account document at users/{uid}.
 *
 * `role` and `churchId` decide what the security rules will let this account
 * do, and neither is writable by the account itself after creation.
 * `emailVerified` is a mirror of Firebase Auth for display only - the rules
 * never trust it, because a mirror is only as honest as whoever last wrote
 * it.
 */
export interface UserProfile {
  uid: string;
  email: string;
  fullName: string;
  /**
   * What the app calls you. Shown on Home's welcome band.
   *
   * Separate from fullName because the two are for different readers: the
   * parish office needs the name on your baptismal record, and the home
   * screen should greet you the way your family does. Before this, Home
   * greeted people with the first word of their email address.
   */
  nickname?: string;
  /** A small square avatar. See lib/avatar.ts for why it is a data URL. */
  photoUrl?: string;
  phoneNumber?: string;
  phoneVerified: boolean;
  emailVerified: boolean;
  role: UserRole;
  churchId: string;
  createdAt: string;
```

```
  updatedAt?: string;
  points?: number;
  steps?: number;
  completedStations?: string[];
  badges?: string[];
}

/** Where an application has got to. The rules refuse anything outside this. */
export type ApplicationStatus =
  | "pending"
  | "under_review"
  | "approved"
  | "rejected"
  | "completed";

export const APPLICATION_STATUSES: ApplicationStatus[] = [
  "pending", "under_review", "approved", "rejected", "completed",
];

/** One step in an application's audit trail. */
export interface ApplicationEvent {
  status: ApplicationStatus;
  at: string;
  by?: string;
  note?: string;
}

/**
 * A sacrament or ministry application.
 *
 * Owner is `uid`, not `applicantId`: that is the field the documents already
 * in the database carry and the field the existing query filters on, and
 * renaming it would strand every application already submitted.
 */
export interface ApplicationDoc {
  id?: string;
  uid: string;
  applicantName: string;
  applicantEmail: string;
  churchId: string;
  kind: "sacrament" | "ministry";
  /**
   * The ministry's or sacrament's id - "min-altar-servers", "sac-baptism".
   *
   * Optional on the type only because applications submitted before this
   * field existed do not carry one. Every NEW application must have it:
   * the security rules refuse a create without it, because it is what
   * they check against the parish's closed-applications list.
   */
  itemId?: string;
  /** "Baptism", "Confirmation", or the ministry's name. */
  type: string;
  status: ApplicationStatus;
  referenceNumber: string;
  formData?: Record<string, unknown>;
  documentPaths?: string[];
  adminNote?: string;
  reviewedBy?: string;
  reviewedAt?: string;
```

```
  history?: ApplicationEvent[];
  createdAt: string;
  updatedAt?: string;
}

/** An in-app message. Email needs Cloud Functions, which needs Blaze. */
export interface NotificationDoc {
  id?: string;
  userId: string;
  churchId: string;
  title: string;
  body: string;
  applicationId?: string;
  createdAt: string;
  readAt?: string | null;
}
```

1.6 Styling

1.6.1 APP-162 — index.css

Code ID	APP-162
File Name	index.css
File Path	SanctDemo/src/index.css
Language / Type	CSS
Lines	6125
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	src/main.tsx
How to Recognize It	Open SanctDemo/src/index.css. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — index.css

```
@import url('https://fonts.googleapis.com/css2?family=Montserrat:ital,wght@0,300..900;1,300..900&family=DM+Mono:wght@400;500&display=swap');
@import "tailwindcss";

:root {
  /* ─── PALETTE — "warm paper" (Claude Design redesign, direction 1c) ───
     The app used to be navy-on-navy: the client's #151B53 as the whole
     background with cream cards floating on it. The redesign inverts that.
     Paper is now the ground, and navy is demoted to an accent that marks
     exactly three things — your position, your route, and actions you can
     take. That is what gives it its job back; as a full background it
     marked nothing.

     Token names are roles, not colours, which is why re-pointing the values
     here re-skins every screen without touching a component. */

  --color-brand-bg: #E7E4DD;           /* Page canvas — the paper the app sits on */
  --color-brand-card: #FAF8F4;         /* Raised surface: screens, cards, the nav bar */
  --color-brand-card-sunk: #F2EFE9;    /* Recessed surface — search field, inactive rows */
  --color-brand-border: #DED9D0;       /* Dividers and card outlines */
  --color-brand-hairline: rgb(20 23 31 / 0.07); /* The 1px rules between sections */

  --color-brand-text: #14171F;         /* Ink — primary text. 16.4:1 on paper */
  --color-brand-secondary: #63666D;    /* Muted text — 5.2:1 on paper, clears AA */

  --color-brand-primary: #1C2C56;      /* Navy: position, routes, primary actions. 11.9:1 on paper */
  --color-brand-primary-dark: #0F1A36; /* Pressed/hover navy */
  --color-brand-accent: #1C2C56;       /* The accent IS the navy now — the redesign drops
                                          the teal entirely. Kept as its own token so an
                                          accent that is not the primary can return later
                                          without touching every call site. */
  --color-brand-on-accent: #FAF8F4;    /* Text on a solid navy fill — 11.9:1 */
```

```
  /* Gold is decorative and structural, NEVER text on paper: #D9AD63 on
     #FAF8F4 is 1.9:1 and #B98B45 only reaches 3.4:1, both short of AA for
     body text. Use it for markers, rules, fills and large display type on
     dark; use ink or navy for anything that has to be read. */
  --color-brand-gold: #D9AD63;
  --color-brand-gold-deep: #B98B45;

  --color-brand-success: #4C7A5C;      /* "Open now", live parishes — 4.6:1 on paper */
  --color-brand-success-soft: #7F9E8B; /* The same green on dark surfaces */
  --color-brand-error: #B3543F;        /* Camera/permission failures */
  --color-brand-error-soft: #FFB4A2;   /* Error text on a dark overlay */
  /* The study-area polygon.
     Was the client's original orange (#E65100), kept for a long time because
     it was their data and their colour. They have now asked for it in the
     app's blue: the orange read as a warning — the only hot colour on an
     otherwise cool map — for a shape that marks nothing urgent, and at
     street zoom its fill washed the whole view. Navy makes it part of the
     app rather than an alarm sitting on top of it. */
  --color-brand-scope: #1C2C56;

  --color-brand-ink-surface: #12141A;  /* Dark surfaces that survive the inversion:
                                          the AR viewfinder, the phone status bar */

  /* ─── TYPE ─── 
     Montserrat throughout, DM Mono for small labels, per the redesign.

     The mockup's ladder runs 10.5 / 11 / 12.5 / 14.5 / 21 / 27 at true phone
     width. That is below this app's floor — 16px body, 14px absolute
     minimum, set because the congregation includes people in their
     seventies. The ladder below keeps the design's *proportions* and lifts
     the whole thing so nothing lands under 14px. */
  --font-primary: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-secondary: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-mono-label: "DM Mono", ui-monospace, monospace;

  --text-label: 14px;    /* was 10.5–11.5 — mono eyebrows, chips, nav labels */
  --text-meta: 15px;     /* was 12–12.5 — secondary lines under a title */
  --text-body: 16px;     /* was 13–14.5 — body copy and anything tappable */
  --text-subhead: 20px;  /* was 17ish — section headings */
  --text-title: 24px;    /* was 21 — card titles */
  --text-display: 30px;  /* was 27 — the one big number or name per screen */
}

@theme {
  --font-primary: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-secondary: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-sans: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-serif: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-display: "Montserrat", ui-sans-serif, system-ui, sans-serif;
  --font-mono: "DM Mono", ui-monospace, monospace;


  --color-brand-primary: var(--color-brand-primary);
  --color-brand-primary-dark: var(--color-brand-primary-dark);
  --color-brand-secondary: var(--color-brand-secondary);
  --color-brand-accent: var(--color-brand-accent);
  --color-brand-bg: var(--color-brand-bg);
  --color-brand-card: var(--color-brand-card);
```

```
  --color-brand-border: var(--color-brand-border);
  --color-brand-text: var(--color-brand-text);
  --color-brand-success: var(--color-brand-success);
  --color-brand-error: var(--color-brand-error);
  --color-brand-error-soft: var(--color-brand-error-soft);
  --color-brand-scope: var(--color-brand-scope);
  --color-brand-card-sunk: var(--color-brand-card-sunk);
  --color-brand-hairline: var(--color-brand-hairline);
  --color-brand-gold: var(--color-brand-gold);
  --color-brand-gold-deep: var(--color-brand-gold-deep);
  --color-brand-success-soft: var(--color-brand-success-soft);
  --color-brand-ink-surface: var(--color-brand-ink-surface);
  --color-brand-on-accent: var(--color-brand-on-accent);
}

/* ─────────────────────────────────────────────────────────────
   DIOCESE MAP (DioceseMap.tsx)
   A hand-drawn, geographically-calibrated illustration of the
   diocese — not a street map. All colour comes from the brand
   variables above; no literal colours here. The map is displayed
   inside a height-capped card, so its 300x400 viewBox is rendered
   below 1x scale on screen. SVG text scales down with the whole
   graphic, so the font-sizes below are inflated well past the
   14px legibility floor to compensate — see DioceseMap.tsx's
   wrapLabel/PinLabel, which wraps names onto two lines so the
   larger text still fits the frame.
   ──────────────────────────────────────────────────────────── */
.dmap { display: block; width: 100%; height: 100%; }
/* This used to be a DARK map, drawn in whites over navy. The redesign turned
   the ground to paper, which left roads at 14% white and white pin labels
   invisible — on the one map that only ever appears when MapLibre cannot
   start, i.e. offline, i.e. exactly when it has to work. It is a light map
   now: land a shade off the page so the coastline reads without a border,
   water cooler and darker than land, and every line and label in ink. */
.dmap__land { fill: color-mix(in srgb, var(--color-brand-card) 88%, var(--color-brand-border)); }
.dmap__water { fill: color-mix(in srgb, var(--color-brand-primary) 12%, var(--color-brand-bg)); }
.dmap__river { stroke: color-mix(in srgb, var(--color-brand-primary) 35%, transparent); fill: none; }
.dmap__road { stroke: color-mix(in srgb, var(--color-brand-text) 16%, transparent); fill: none; }
.dmap__area-label {
  fill: color-mix(in srgb, var(--color-brand-text) 55%, transparent);
  font-size: 21px;
  font-weight: 600;
  letter-spacing: 1px;
  text-anchor: middle;
}
/* Live parishes are the only saturated thing on the map — everything else is
   context, they are the destinations. */
.dmap__pin--live { fill: var(--color-brand-accent); stroke: var(--color-brand-card); }
.dmap__pin--soon {
  fill: color-mix(in srgb, var(--color-brand-text) 30%, transparent);
  stroke: color-mix(in srgb, var(--color-brand-text) 45%, transparent);
}
.dmap__pin-label { fill: var(--color-brand-text); font-size: 21px; font-weight: 700; text-anchor: middle; }
.dmap__you { fill: var(--color-brand-success); stroke: var(--color-brand-card); }
.dmap__you-ring {
  fill: none;
  stroke: var(--color-brand-success);
  animation: dmap-pulse 2s ease-out infinite;
}
```

```
@keyframes dmap-pulse {
  0% { r: 8; opacity: 0.8; }
  100% { r: 20; opacity: 0; }
}
.dmap__hit { cursor: pointer; }
.dmap__hit:focus-visible { outline: 2px solid var(--color-brand-accent); outline-offset: 2px; }

/* ─────────────────────────────────────────────────────────────
   LIVE DIOCESE MAP (DioceseMapLive.tsx)
   The client's earlier prototype (DioceseMap.jsx/.css), reproduced: a real
   MapLibre GL map on the light `positron` basemap, deliberately plain so
   the church-glyph pins below stay the loudest thing on screen — no more
   runtime restyling of the tile layers, since positron already ships
   light and decluttered. Colours for the parts MapLibre itself draws (the
   scope polygon, the route line) are still read from these custom
   properties at runtime via getComputedStyle in DioceseMapLive.tsx, since
   MapLibre paint properties need literal colour values, not CSS var()
   references; everything below is plain CSS for the HTML pins, popup,
   search UI and container chrome. Falls back to DioceseMap.tsx (the drawn
   SVG map, a different, deliberately dark illustration — see its own block
   above) when tiles are unreachable.
   ──────────────────────────────────────────────────────────── */
.dmap-live { position: relative; width: 100%; height: 100%; border-radius: 1rem; overflow: hidden; }
.dmap-live__canvas { width: 100%; height: 100%; }

/* Church-glyph pins — a real <button> per pin (built in mapMarkers.ts),
   positioned by MapLibre, not SVG children of the map, so they need their
   own reset. Every pin is tappable, including coming-soon ones: tapping is
   how you find out which parish a pin is, so a silent pin would be a dead
   end (the client's prototype makes the same point in its own comments). */
.dmap__pin { display: block; padding: 0; border: 0; background: none; line-height: 0; cursor: pointer; }
.dmap__pin svg { overflow: visible; }
.dmap__pin--live svg { width: 32px; height: 32px; }
/* Stroke is the app's navy, not the cream/white used elsewhere for
   "on-accent" text — on the light positron ground a cream outline nearly
   disappears into the basemap (both are pale near-whites), where navy
   reads clearly as a pin silhouette. */
.dmap__pin--live { fill: var(--color-brand-accent); stroke: var(--color-brand-primary); }
.dmap__pin--live:focus-visible { outline: 2px solid var(--color-brand-accent); outline-offset: 2px; }
/* Every parish is a real destination now, so every pin is the app's navy.
   These used to be grey and 22px — "not yet destinations, read as context" —
   which was true when only two could be routed to. Now all 31 can be, and a
   pilgrim looking for their own parish should not have to work out why it is
   the faint one.

   Slightly smaller than a live pin, and hollow rather than filled, so the two
   with the full programme still lead the eye. That distinction is carried
   properly by the card, which says what is still being collected. */
.dmap__pin--soon svg { width: 27px; height: 27px; }
.dmap__pin--soon {
  fill: color-mix(in srgb, var(--color-brand-accent) 45%, transparent);
  stroke: var(--color-brand-primary);
}

/* "You are here", as every map app draws it: a solid blue dot in a white
   collar, with a soft shadow lifting it off the basemap.

   It was a green dot in a navy ring with a pulsing halo and a translucent
   triangle fanning out of it. Three problems. Green is the colour this
```

```
   app uses for "open now" on a parish, so the dot competed with the pins
   it was supposed to be distinct from; the pulse drew the eye
   continuously to a thing that was not changing; and the triangle read at
   a glance as the marker simply BEING a triangle.

   The white collar is not decoration - it is what keeps a blue dot
   legible over a blue route line and over dark basemap features. */
.dmap-live__marker--you {
  position: relative;
  width: 16px;
  height: 16px;
  border-radius: 50%;
  background: #1A73E8;
  border: 2.5px solid #fff;
  box-shadow: 0 1px 4px rgb(0 0 0 / 0.35);
  pointer-events: none;
}

.dmap-live__fallback-badge {
  position: absolute;
  top: 8px;
  right: 8px;
  z-index: 5;
  font-size: 14px;
  font-weight: 600;
  padding: 2px 8px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--color-brand-text) 70%, transparent);
  color: var(--color-brand-card);
}

/* Recentre-on-me control — sits above the MapLibre canvas, on the right so
   it never collides with the search bar/results on the left. Stacked below
   the NavigationControl's zoom buttons (top-right), in the same 52px-wide
   right-hand margin the search bar is inset to avoid. Disabled (with an
   explanatory title/aria-label set inline) whenever position is null,
   since flying to an unknown position would be meaningless. */
/* While turn-by-turn is running, the instruction card occupies the whole top
   strip of the map — roughly 170px, more when the street name wraps to three
   lines. The search field, the compass and the Recentre button all live in
   that strip and were being covered by it.

   Search is hidden outright: mid-route is not when you look for a different
   parish. The compass and Recentre move below the card, because both stay
   useful while walking — Recentre especially, since the whole point is
   getting back to your own position after panning. */
.dmap-live[data-navigating="true"] .dmap-search { display: none; }
.dmap-live[data-navigating="true"] .dmap-compass-wrap,
.dmap-live[data-navigating="true"] .dmap-live__recentre { top: 196px; }

.dmap-live__recentre {
  position: absolute;
  top: calc(92px + env(safe-area-inset-top, 0px));
  right: 8px;
  z-index: 5;
  font-size: 14px;
  font-weight: 600;
  padding: 6px 10px;
  border-radius: 999px;
```

```
  border: 1px solid color-mix(in srgb, var(--color-brand-card) 40%, transparent);
  background: color-mix(in srgb, var(--color-brand-text) 88%, transparent);
  color: var(--color-brand-on-accent);
  cursor: pointer;
}
.dmap-live__recentre:disabled {
  cursor: not-allowed;
  opacity: 0.5;
}
.dmap-live__recentre:focus-visible {
  outline: 2px solid var(--color-brand-card);
  outline-offset: 2px;
}

/* The north-up control shares the recentre pill's shape and column, sitting
   directly beneath it. Quieter of the two: recentring is the thing you reach
   for mid-walk, resetting the view is the thing you reach for once. */
.dmap-live__northup {
  position: absolute;
  top: 130px;
  right: 8px;
  z-index: 5;
  font-size: 13px;
  font-weight: 600;
  padding: 5px 10px;
  border-radius: 999px;
  border: 1px solid color-mix(in srgb, var(--color-brand-text) 18%, transparent);
  background: color-mix(in srgb, var(--color-brand-card) 92%, transparent);
  color: var(--color-brand-text);
  cursor: pointer;
}
.dmap-live__northup:focus-visible {
  outline: 2px solid var(--color-brand-text);
  outline-offset: 2px;
}
/* Follows the recentre pill down when the turn card takes the top strip. */
.dmap-live[data-navigating="true"] .dmap-live__northup { top: 234px; }

/* Honest status for the you-are-here marker — sits directly under the
   recentre control, in the same right-hand margin. A real GPS fix states
   its own reported accuracy radius rather than a bare, over-precise dot; a
   simulated position is labelled as such so it is never mistaken for one. */
.dmap-live__position-badge {
  position: absolute;
  top: 132px;
  right: 8px;
  z-index: 5;
  font-size: 14px;
  font-weight: 600;
  padding: 4px 10px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--color-brand-text) 82%, transparent);
  color: var(--color-brand-card);
}
.dmap-live__position-badge--sim {
  background: color-mix(in srgb, var(--color-brand-scope) 85%, transparent);
}

/* Floating pill-shaped search bar, top-left, inset on the right to leave
```

```
   room for the NavigationControl's zoom buttons (top-right) — matches the
   client's prototype layout exactly. Chrome uses the cream card surface
   (not MapLibre's default white popup/control styling), since it now sits
   directly over the light basemap and needs its own visual edge. */
.dmap-search {
  /* Below the status bar, the way Google Maps sits below it.
     On a phone with a notch, top:8px put this under the clock and the
     signal bars. env() is 0 in a browser, so the desktop layout is
     unchanged. */
  position: absolute; z-index: 6; top: calc(8px + env(safe-area-inset-top, 0px));
  left: 8px; right: 52px;
  display: flex; align-items: center; gap: 8px;
  padding: 8px 12px; border-radius: 999px;
  background: color-mix(in srgb, var(--color-brand-card) 94%, transparent);
  backdrop-filter: blur(8px);
  box-shadow: 0 4px 16px rgb(0 0 0 / 0.18);
}
.dmap-search__icon {
  width: 16px; height: 16px; flex: none;
  fill: none; stroke: var(--color-brand-secondary); stroke-width: 2; stroke-linecap: round;
}
.dmap-search__input {
  flex: 1; min-width: 0; border: 0; background: none; color: var(--color-brand-text);
  font: inherit; font-size: 16px; outline: none;
}
.dmap-search__input::placeholder { color: var(--color-brand-secondary); }

/* Result list, hanging below the search bar. */
/* Above everything, including the visit planner.
   The planner is a sheet pinned to the bottom at z-index 30, and searching
   is how a church gets added to it - so with the keyboard up on a phone the
   sheet rose to meet the results list and covered it. Measured at a
   420px-tall viewport, which is about what an Android phone has left with
   the keyboard open: 119px of the list was behind the sheet, including the
   first result. You could not see what you were searching for.
   Wider, and lower, than before: it used to stop 52px short of the right
   edge to clear the zoom buttons, which cost a quarter of the width on
   every phone. Starting below them instead buys that back.
   The height is capped against the DYNAMIC viewport, so when the keyboard
   opens the list shortens rather than running underneath it. */
.dmap-results {
  position: absolute; z-index: 40; top: 60px;
  left: 8px; right: 8px;
  margin: 0; padding: 4px; list-style: none;
  max-height: min(44dvh, 260px); overflow-y: auto;
  overscroll-behavior: contain;
  border-radius: 0.75rem;
  background: var(--color-brand-card);
  box-shadow: 0 8px 24px rgb(0 0 0 / 0.22);
}
.dmap-results__item {
  display: block; width: 100%; padding: 8px 10px;
  border: 0; border-radius: 0.5rem;
  background: none; color: var(--color-brand-text); font: inherit; text-align: left;
  cursor: pointer;
}
.dmap-results__item:hover { background: color-mix(in srgb, var(--color-brand-accent) 14%, transparent); }
.dmap-results__name { display: block; font-size: 16px; font-weight: 600; line-height: 1.25; }
.dmap-results__where { display: block; font-size: 15px; color: var(--color-brand-secondary); margin-top: 2px; }
```

```
.dmap-results__empty { padding: 10px; font-size: 15px; color: var(--color-brand-secondary); }

/* Info card shown when a pin (or a search result) is tapped. MapLibre
   supplies the positioning and the little pointer tip; everything inside
   is ours, so it is themed from the app's tokens rather than MapLibre's
   default white popup. */
.dmap-popup .maplibregl-popup-content {
  background: var(--color-brand-card);
  color: var(--color-brand-text);
  border-radius: 0.75rem;
  padding: 0.75rem;
  box-shadow: 0 8px 24px rgb(0 0 0 / 0.22);
  min-width: 168px;
}
.dmap-popup .maplibregl-popup-tip { border-top-color: var(--color-brand-card); }
.dmap-card__name { margin: 0; font-size: 16px; font-weight: 600; line-height: 1.25; color: var(--color-brand-text); }
.dmap-card__where { margin: 4px 0 0; font-size: 15px; color: var(--color-brand-secondary); }
.dmap-card__soon { margin: 8px 0 0; font-size: 15px; color: var(--color-brand-secondary); font-style: italic; }
.dmap-card__action {
  margin-top: 10px; width: 100%; padding: 7px 12px;
  border: 0; border-radius: 0.5rem;
  background: var(--color-brand-accent); color: var(--color-brand-on-accent);
  font: inherit; font-size: 16px; font-weight: 600; cursor: pointer;
}
.dmap-card__action:hover { filter: brightness(1.08); }
.dmap-card__action--secondary {
  margin-top: 6px;
  background: var(--color-brand-card);
  color: var(--color-brand-accent);
  border: 1px solid var(--color-brand-accent);
}
.dmap-card__action--secondary:hover { background: color-mix(in srgb, var(--color-brand-accent) 12%, var(--color-brand-card)); }
.dmap-card__action--secondary:disabled { opacity: 0.6; cursor: not-allowed; }

/* "Start walking" — the live turn-by-turn session, as opposed to "Get
   directions", which only draws a line. Filled rather than outlined because
   it is the more consequential of the two: it takes over the bottom of the
   map and follows the pilgrim until they stop it. */
.dmap-card__action--navigate {
  margin-top: 6px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  border: 1px solid var(--color-brand-primary);
}
.dmap-card__action--navigate:hover {
  background: var(--color-brand-primary-dark);
}
/* Filled in after "Get directions" is tapped: distance/time once routed,
   a straight-line figure clearly labelled "direct" when OSRM couldn't be
   reached, or why it can't route yet (no known position). Empty (no text
   content) collapses to nothing so a card nobody asked directions for shows
   no stray blank line. */
.dmap-card__directions-status:empty { display: none; }
.dmap-card__directions-status {
  margin: 6px 0 0;
  font-size: 15px;
  color: var(--color-brand-secondary);
}
```

```
/* Per-parish distance panel — shown in both live (MapLibre) and offline
   (drawn SVG) modes, since distance needs no network. In live mode it
   overlays the map canvas; in fallback mode it sits below the SVG. */
.dmap-live__panel {
  position: absolute;
  left: 8px;
  bottom: 8px;
  right: 8px;
  z-index: 5;
  display: flex;
  flex-direction: column;
  gap: 4px;
  padding: 8px 10px;
  border-radius: 0.75rem;
  background: color-mix(in srgb, var(--color-brand-text) 82%, transparent);
}
[data-map-mode="fallback"] .dmap-live__panel {
  position: static;
  margin-top: 8px;
  background: color-mix(in srgb, var(--color-brand-text) 88%, transparent);
}
.dmap-live__panel-row {
  display: flex;
  justify-content: space-between;
  gap: 8px;
  font-size: 14px;
  line-height: 1.3;
}
.dmap-live__panel-name {
  font-weight: 600;
  color: var(--color-brand-on-accent);
}
.dmap-live__panel-distance {
  color: color-mix(in srgb, var(--color-brand-on-accent) 85%, transparent);
  text-align: right;
}

/* Caption under the map explaining the shaded polygon — a shaded area with
   no explanation reads as decoration, so this states plainly what it is. */
.dmap-live__legend {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 0 2px;
  font-size: 15px;
  line-height: 1.35;
  color: var(--color-brand-secondary);
}
.dmap-live__legend-swatch {
  flex: none;
  width: 14px;
  height: 14px;
  border-radius: 3px;
  background: color-mix(in srgb, var(--color-brand-scope) 35%, transparent);
  border: 1.5px solid var(--color-brand-scope);
}

.dmap-live__open-link {
  display: inline-flex;
```

```
  align-items: center;
  justify-content: center;
  gap: 6px;
  padding: 0.625rem 1rem;
  border-radius: 0.75rem;
  font-size: 16px;
  font-weight: 700;
  background: var(--color-brand-card);
  color: var(--color-brand-primary);
}

/* ─────────────────────────────────────────────────────────────
   MAP SCREEN — diocese header + parish cards
   Matches the client's earlier prototype (Churches.jsx/.css):
   a centred diocese heading above the map, then one card per
   live parish below it. Card surface uses --color-brand-card-sunk
   (#1E2668, added above) — the prototype's exact hex, one step
   lighter than --color-brand-bg so it reads as a raised surface.
   The "Live" chip reuses --color-brand-success rather than the
   prototype's literal #2FA37A: that literal only clears 3.16:1
   contrast with white text (fails AA's 4.5:1 floor for text this
   size), while --color-brand-success is the same green family
   already darkened for AA (5.29:1) — same fix pattern already
   applied to --color-brand-accent above.
   ──────────────────────────────────────────────────────────── */
/* The map tab is the map: a header strip, then the canvas taking every
   remaining pixel, with the parish rail floating over its lower edge.
   The header stays a real strip rather than an overlay because the map
   already floats a search bar at its top-left and zoom controls at its
   top-right — a third floating element there would collide with both. */
.map-screen {
  display: flex;
  flex-direction: column;
  height: 100%;
  min-height: 0;
  position: relative;
  background: var(--color-brand-card);
}
.map-screen__header {
  text-align: center;
  padding: 0.75rem 0.5rem 0.5rem;
  flex: none;
}
.map-screen__title {
  font-size: 20px;
  font-weight: 700;
  color: var(--color-brand-text);
}
.map-screen__count {
  font-size: 15px;
  color: var(--color-brand-secondary);
  margin-top: 2px;
}
.map-screen__canvas {
  flex: 1;
  min-height: 0;
  padding: 0 0.5rem;
}
```

```
/* The rail overlays the map's lower edge and scrolls horizontally, snapping
   card to card. */
.map-screen__rail {
  position: absolute;
  left: 0;
  right: 0;
  bottom: 8px;
  z-index: 6;
  display: flex;
  gap: 10px;
  padding: 0 12px;
  overflow-x: auto;
  overflow-y: hidden;
  /* Same reason as .card-rail: without this the map underneath claims the
     drag and the cards will not move. */
  touch-action: pan-x;
  overscroll-behavior-x: contain;
  scroll-snap-type: x proximity;
  scroll-padding-left: 12px;
  -webkit-overflow-scrolling: touch;
  scrollbar-width: none;
}
.map-screen__rail::-webkit-scrollbar { display: none; }
.map-screen__rail-item {
  flex: 0 0 72%;
  scroll-snap-align: start;
  touch-action: pan-x;
}
.map-screen__rail .parish-card {
  box-shadow: 0 8px 24px rgb(20 23 31 / 0.18);
}

/* The rail sits exactly where the map puts its own distance panel, and the
   cards already carry each parish's distance. Rather than let the two
   overlap, the panel is suppressed on this screen only — it still shows on
   the dashboard's mini map, where there is no rail. */
.map-screen .dmap-live__panel {
  display: none;
}

.parish-card {
  display: block;
  text-decoration: none;
  background: var(--color-brand-card-sunk);
  border: 1px solid var(--color-brand-border);
  border-radius: 20px;
  padding: 1rem 1.125rem;
  cursor: pointer;
  width: 100%;
  text-align: left;
}
.parish-card__row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 0.5rem;
}
.parish-card__name {
  font-size: 18px;
```

```
  font-weight: 700;
  color: var(--color-brand-text);
  line-height: 1.25;
}
.parish-card__patron {
  font-size: 15px;
  color: var(--color-brand-secondary);
  margin-top: 4px;
}
.parish-card__rule {
  border: none;
  border-top: 1px solid var(--color-brand-border);
  margin: 0.75rem 0;
}
.parish-card__meta {
  font-size: 15px;
  color: var(--color-brand-secondary);
}

.chip--live {
  flex: none;
  display: inline-flex;
  align-items: center;
  font-size: 14px;
  font-weight: 700;
  line-height: 1;
  padding: 6px 12px;
  border-radius: 999px;
  background: var(--color-brand-success);
  color: var(--color-brand-on-accent);
  white-space: nowrap;
}

/* Base Typography Configuration */
body {
  font-family: var(--font-secondary);
}

h1, h2, h3, h4, h5, h6, .font-heading, .font-primary, .font-montserrat {
  font-family: var(--font-primary);
  font-weight: 600; /* matches the diocese's own site: Montserrat headings at weight 600 */
}

.font-secondary, .font-poppins {
  font-family: var(--font-secondary);
}

/* Custom Scrollbar */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: transparent;
}
::-webkit-scrollbar-thumb {
  background: rgba(156, 163, 175, 0.3);
  border-radius: 3px;
}
```

```
::-webkit-scrollbar-thumb:hover {
  background: rgba(156, 163, 175, 0.5);
}

/* ——————————————————————————————————————————————————————
   DYNAMIC COLOR BRIDGING OVERRIDES
   These rules capture Tailwind's generated class names for
   the hardcoded colors in the components and link them
   directly to your CSS theme variables above.
   —————————————————————————————————————————————————————— */

/* Primary Color mappings (#5A5A40) */
.bg-\[\#5A5A40\] { background-color: var(--color-brand-primary) !important; }
.text-\[\#5A5A40\] { color: var(--color-brand-primary) !important; }
.border-\[\#5A5A40\] { border-color: var(--color-brand-primary) !important; }
.stroke-\[\#5A5A40\] { stroke: var(--color-brand-primary) !important; }
.fill-\[\#5A5A40\] { fill: var(--color-brand-primary) !important; }
.from-\[\#5A5A40\] { --tw-gradient-from: var(--color-brand-primary) !important; --tw-gradient-to: var(--color-brand-primary) !important; }
.to-\[\#5A5A40\] { --tw-gradient-to: var(--color-brand-primary) !important; }

/* Primary Dark Color mappings (#4A4A35) */
.bg-\[\#4A4A35\] { background-color: var(--color-brand-primary-dark) !important; }
.text-\[\#4A4A35\] { color: var(--color-brand-primary-dark) !important; }
.border-\[\#4A4A35\] { border-color: var(--color-brand-primary-dark) !important; }
.stroke-\[\#4A4A35\] { stroke: var(--color-brand-primary-dark) !important; }
.fill-\[\#4A4A35\] { fill: var(--color-brand-primary-dark) !important; }
.from-\[\#4A4A35\] { --tw-gradient-from: var(--color-brand-primary-dark) !important; --tw-gradient-to: var(--color-brand-primary-dark) !important; }
.to-\[\#4A4A35\] { --tw-gradient-to: var(--color-brand-primary-dark) !important; }

/* Secondary/Muted Color mappings (#8A8A70) */
.bg-\[\#8A8A70\] { background-color: var(--color-brand-secondary) !important; }
.text-\[\#8A8A70\] { color: var(--color-brand-secondary) !important; }
.border-\[\#8A8A70\] { border-color: var(--color-brand-secondary) !important; }
.stroke-\[\#8A8A70\] { stroke: var(--color-brand-secondary) !important; }
.fill-\[\#8A8A70\] { fill: var(--color-brand-secondary) !important; }

/* Accent Color mappings (#C2A649) */
.bg-\[\#C2A649\] { background-color: var(--color-brand-accent) !important; }
.text-\[\#C2A649\] { color: var(--color-brand-accent) !important; }
.border-\[\#C2A649\] { border-color: var(--color-brand-accent) !important; }
.stroke-\[\#C2A649\] { stroke: var(--color-brand-accent) !important; }
.fill-\[\#C2A649\] { fill: var(--color-brand-accent) !important; }
.from-\[\#C2A649\] { --tw-gradient-from: var(--color-brand-accent) !important; --tw-gradient-to: var(--color-brand-accent) !important; }
.to-\[\#C2A649\] { --tw-gradient-to: var(--color-brand-accent) !important; }

/* Body Background mappings (#F5F5F0) */
.bg-\[\#F5F5F0\] { background-color: var(--color-brand-bg) !important; }
.text-\[\#F5F5F0\] { color: var(--color-brand-bg) !important; }
.border-\[\#F5F5F0\] { border-color: var(--color-brand-bg) !important; }

/* Card/Surface mappings (#EBEBE0) */
.bg-\[\#EBEBE0\] { background-color: var(--color-brand-card) !important; }
.text-\[\#EBEBE0\] { color: var(--color-brand-card) !important; }
.border-\[\#EBEBE0\] { border-color: var(--color-brand-card) !important; }

/* Border/Divider mappings (#D6D6C2) */
.bg-\[\#D6D6C2\] { background-color: var(--color-brand-border) !important; }
.text-\[\#D6D6C2\] { color: var(--color-brand-border) !important; }
.border-\[\#D6D6C2\] { border-color: var(--color-brand-border) !important; }
```

```
/* Body Text mappings (#33332D) */
.text-\[\#33332D\] { color: var(--color-brand-text) !important; }
.bg-\[\#33332D\] { background-color: var(--color-brand-text) !important; }

/* Error mappings (#B3543F / #FFB4A2) — camera/permission error states in
   ArTour.tsx and useCamera.ts. */
.bg-\[\#B3543F\] { background-color: var(--color-brand-error) !important; }
.text-\[\#B3543F\] { color: var(--color-brand-error) !important; }
.border-\[\#B3543F\] { border-color: var(--color-brand-error) !important; }
.text-\[\#FFB4A2\] { color: var(--color-brand-error-soft) !important; }

/* "Clear directions" — only rendered while a route is actually drawn, so it
   sits directly beneath "Recentre on me" rather than reserving space that is
   empty most of the time. Same pill treatment; the accent border marks it as
   acting on the route rather than the camera. */
.dmap-live__clear-route {
  position: absolute;
  top: 130px;
  right: 8px;
  z-index: 5;
  font-size: 14px;
  font-weight: 600;
  padding: 6px 10px;
  border-radius: 999px;
  border: 1px solid var(--color-brand-accent);
  background: color-mix(in srgb, var(--color-brand-text) 82%, transparent);
  color: var(--color-brand-card);
  cursor: pointer;
}
.dmap-live__clear-route:focus-visible {
  outline: 2px solid var(--color-brand-card);
  outline-offset: 2px;
}

/* The direction light on the "you are here" dot.
 *
 * A soft fan, not a triangle. The first version was a hard-edged
 * translucent wedge, and at a glance it read as the DOT being a triangle
 * pointing somewhere rather than as a dot with a beam coming off it - so
 * it was removed, which left the map with nothing saying which way you
 * face. This is the middle version: it fades to nothing at its far edge
 * and at both sides, so the eye reads light rather than a shape.
 *
 * Anchored at the dot's centre and rotated from there, which is why the
 * transform is translate(-50%, -100%) - the fan hangs upward from the
 * centre point and rotate() then swings it around that same point.
 * Changing one without the other makes the beam orbit the dot instead of
 * turning on the spot. */
.dmap-live__beam {
  position: absolute;
  left: 50%;
  top: 50%;
  width: 76px;
  height: 76px;
  transform: translate(-50%, -100%) rotate(0deg);
  transform-origin: 50% 100%;
  pointer-events: none;
  /* Hidden until the compass produces a real bearing; the component sets
```

```
     this to 1. A beam stuck at due north is worse than no beam. */
  opacity: 0;
  transition: opacity 240ms ease;
  /* The fan shape. A cone of about 60 degrees, clipped out of a circle
     so the far edge is round rather than cut straight across. */
  clip-path: polygon(50% 100%, 20% 0%, 80% 0%);
  /* Transparent for the first fifth, so the beam does not wash over the
     blue dot it comes from. MapLibre puts a transform on the marker
     element, which makes it a stacking context - so `z-index: -1` would
     not reliably put this behind the dot, and clearing the centre is the
     version that does not depend on that. */
  /* The dot's own blue (#1A73E8), not the parish brand navy. The beam
     belongs to the dot, and a navy fan coming off a bright blue dot
     reads as two unrelated things - as well as being nearly invisible
     against the map's pale grey. */
  background: radial-gradient(
    circle at 50% 100%,
    rgb(26 115 232 / 0) 0%,
    rgb(26 115 232 / 0) 16%,
    rgb(26 115 232 / 0.55) 32%,
    rgb(26 115 232 / 0.3) 58%,
    rgb(26 115 232 / 0) 84%
  );
  /* Softens the straight sides left by clip-path, so the edges of the
     beam do not read as drawn lines. */
  filter: blur(2.5px);
}

@media (prefers-reduced-motion: reduce) {
  .dmap-live__beam { transition-duration: 1ms; }
}

/* The walking puck.
 *
 * While turn-by-turn is running the camera turns the map to the walker's
 * heading, so "up the screen" IS the way they are facing. The beam, which
 * earns its keep on a north-up map, would then be pinned straight up and
 * never move - so navigation hides it and shows an arrow instead, which is
 * what the direction of travel looks like in every turn-by-turn app anyone
 * has already used.
 *
 * The arrow does not rotate. The map does. */
.dmap-live__marker--you[data-navigating="true"] {
  /* The dot's fill and ring hand over to the arrow entirely, or the arrow
     sits on a blue disc and reads as a badge rather than a heading. */
  background: transparent;
  border-color: transparent;
  box-shadow: none;
}

.dmap-live__marker--you[data-navigating="true"] .dmap-live__beam {
  display: none;
}

.dmap-live__chevron {
  position: absolute;
  left: 50%;
  top: 50%;
  width: 34px;
```

```
  height: 34px;
  transform: translate(-50%, -50%);
  pointer-events: none;
  display: none;
  /* A rounded triangle rather than a hard one: at this size a sharp apex
     aliases into a grey smudge on a phone screen. Drawn as a mask so the
     fill below stays a single colour and the white rim can be a filter. */
  background: #1AC6D9;
  clip-path: polygon(50% 6%, 93% 92%, 50% 72%, 7% 92%);
  filter:
    drop-shadow(0 0 2.5px #fff)
    drop-shadow(0 0 2.5px #fff)
    drop-shadow(0 2px 3px rgb(0 0 0 / 0.4));
}

.dmap-live__marker--you[data-navigating="true"] .dmap-live__chevron {
  display: block;
}
/* ── Compass ──────────────────────────────────────────────────────────
   Sits below the zoom control, above "Recentre on me". Tapping it swaps
   between north-up and heading-up. */
.dmap-compass-wrap {
  position: absolute;
  top: 92px;
  left: 8px;
  z-index: 5;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
}
.dmap-compass {
  width: 56px;
  height: 56px;
  padding: 0;
  border: 0;
  border-radius: 50%;
  background: none;
  cursor: pointer;
  filter: drop-shadow(0 2px 8px rgb(0 0 0 / 0.35));
}
.dmap-compass:disabled { cursor: default; opacity: 0.55; }
.dmap-compass:focus-visible { outline: 2px solid var(--color-brand-accent); outline-offset: 2px; }
.dmap-compass svg { width: 100%; height: 100%; display: block; }
.dmap-compass__face {
  fill: color-mix(in srgb, var(--color-brand-text) 88%, transparent);
  stroke: color-mix(in srgb, var(--color-brand-card) 45%, transparent);
}
/* The dial turns; the transition is what makes a turn read as a sweep
   rather than a step between sensor updates. */
.dmap-compass__dial {
  transform-origin: center;
  transition: transform 220ms ease-out;
}
@media (prefers-reduced-motion: reduce) {
  .dmap-compass__dial { transition: none; }
}
/* 14px is the floor, and these are two-character glyphs on a 56px dial —
   any smaller and the ordinals are unreadable at arm's length. */
```

```
.dmap-compass__cardinal {
  font-size: 14px;
  font-weight: 700;
  fill: var(--color-brand-bg);
}
.dmap-compass__ordinal {
  font-size: 14px;
  font-weight: 600;
  fill: color-mix(in srgb, var(--color-brand-bg) 62%, transparent);
}
.dmap-compass__needle { fill: var(--color-brand-danger, #B9302F); }
.dmap-compass__needle-tail { fill: color-mix(in srgb, var(--color-brand-bg) 35%, transparent); }
/* Heading-up is the non-default state, so it is marked — otherwise there is
   no way to tell which mode the map is in when the pilgrim faces north. */
.dmap-compass[data-mode="heading-up"] .dmap-compass__face {
  stroke: var(--color-brand-accent);
  stroke-width: 2;
}
.dmap-compass__readout {
  margin: 0;
  max-width: 108px;
  text-align: center;
  font-size: 14px;
  font-weight: 600;
  line-height: 1.25;
  color: var(--color-brand-text);
  text-shadow: 0 1px 4px rgb(0 0 0 / 0.55);
}
/* The "why it is off" state carries real sentences, not a two-word
   heading, so it gets a readable card instead of shadowed text floating on
   the map. Wider than the dial, because the remedy (a Settings path) is
   useless if it is truncated. */
.dmap-compass__readout[data-state="off"] {
  max-width: 168px;
  margin-top: 2px;
  padding: 6px 9px;
  border-radius: 12px;
  font-size: 13px;
  font-weight: 500;
  background: color-mix(in srgb, var(--color-brand-card) 94%, transparent);
  border: 1px solid var(--color-brand-border);
  color: var(--color-brand-secondary);
  text-shadow: none;
}
.dmap-compass__degrees {
  display: block;
  font-weight: 500;
  font-variant-numeric: tabular-nums;
  color: color-mix(in srgb, var(--color-brand-text) 78%, transparent);
}
.dmap-compass--prompt {
  width: auto;
  height: auto;
  padding: 8px 12px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 600;
  background: color-mix(in srgb, var(--color-brand-text) 88%, transparent);
  color: var(--color-brand-bg);
```

```
}

/* Search result secondary line: distance first when known, then the
   (inferred) vicariate. Stacked rather than inline so a long parish name
   plus a distance never pushes the location off the row. */
.dmap-results__where {
  display: flex;
  flex-direction: column;
  gap: 1px;
  margin-top: 2px;
  font-size: 15px;
  color: var(--color-brand-secondary);
}
.dmap-results__distance {
  font-weight: 600;
  font-variant-numeric: tabular-nums;
  color: var(--color-brand-accent);
}

/* MapLibre ships its attribution at 12px. The credit is required and stays,
   but this app's floor is 14px with no exceptions — a rule with a carve-out
   for "it's only third-party text" stops being a rule. */
.maplibregl-ctrl-attrib,
.maplibregl-ctrl-attrib-inner,
.maplibregl-ctrl-attrib a {
  font-size: 14px;
}

/* Distance on a parish card. Straight-line, and it says so — the routed
   figure only appears once someone asks for directions. */
.parish-card__distance {
  margin-top: 4px;
  font-size: 15px;
  font-weight: 600;
  font-variant-numeric: tabular-nums;
  color: var(--color-brand-primary);
}
.parish-card__walk {
  font-weight: 500;
  color: var(--color-brand-secondary);
}

/* On the full-bleed map screen the legend and the Google Maps link were
   taking 145px below the canvas, which both shrank the map and left the
   floating rail hanging past its lower edge. The legend becomes a compact
   overlay pinned under the header — it still explains the shaded study area,
   which is the client's own data and must stay explained — and the external
   link moves out of the stack, since the pin popups already offer directions
   and this screen is the map. */
/* The legend sits above the map, in normal flow, rather than floating over
   it. As an overlay it collided with the search bar the map already floats at
   its top-left, and anchoring it to the screen put it over the header
   instead. In flow it costs one line and never overlaps anything, while the
   frame below takes every remaining pixel. */
/* Hidden on the map screen at the client's request, along with the heading
   block above it. The shaded study area is still drawn — it is simply no
   longer captioned here. */
.map-screen .dmap-live__legend {
  display: none;
```

```
}
/* The pin popups already offer directions, and this screen is the map, so
   the "open in Google Maps" link is not worth the height here. */
.map-screen .dmap-live__open-link {
  display: none;
}
.map-screen .dmap-live {
  flex: 1;
  min-height: 0;
}

/* North is the one letter you look for, so it carries the needle's colour
   rather than sitting equal with the other three. */
.dmap-compass__north {
  font-size: 14px;
  font-weight: 700;
  fill: var(--color-brand-error);
}

/* "Mass now" on a map popup. Empty by default and collapsed via :empty, so a
   parish with no Mass under way shows no stray blank line — the same pattern
   the directions status uses. Red for in-progress, green for starting soon:
   the first is a door you can still walk through, the second is a reason to
   hurry. */
.dmap-card__mass:empty { display: none; }
.dmap-card__mass {
  margin: 4px 0 0;
  display: inline-block;
  font-size: 14px;
  font-weight: 600;
  padding: 3px 8px;
  border-radius: 6px;
  color: var(--color-brand-on-accent);
  background: var(--color-brand-success);
}
.dmap-card__mass[data-state="in-progress"] {
  background: var(--color-brand-error);
}

/* ── Horizontal card rail (parish bulletin, map parish cards) ──────────────
   Written as a class rather than a pile of arbitrary Tailwind values because
   what makes a rail swipeable on a phone is four properties that only work
   together, and each one is easy to drop by accident.

   touch-action: pan-x is the important one. Left at `auto`, the browser is
   free to give a mostly-horizontal drag to the vertically-scrolling screen
   above the rail — so the rail simply does not move, which is what "I can't
   swipe it" actually was. Declaring pan-x claims horizontal gestures for the
   rail and lets vertical ones pass through to the page, which is what a
   reader expects from both.

   overflow-y: hidden stops the rail being a vertical scroll container of its
   own. Setting overflow-x alone computes overflow-y to auto, so the rail was
   competing with the page for the same drag.

   Snapping is `proximity`, not `mandatory`. With cards this wide, mandatory
   forces every swipe to complete a full card or spring back — a short flick
   just bounced. Proximity lets a small movement settle where it lands. */
.card-rail {
```

```
  display: flex;
  /* Cards take their natural height. Stretching them to match made the
     one-sentence "nothing posted yet" notice as tall as a card carrying a
     photograph — a lot of empty box for one sentence. */
  align-items: flex-start;
  gap: 12px;
  overflow-x: auto;
  overflow-y: hidden;
  touch-action: pan-x;
  overscroll-behavior-x: contain;
  scroll-snap-type: x proximity;
  scroll-padding-inline: 0;
  -webkit-overflow-scrolling: touch;
  scrollbar-width: none;
  /* Enough room either side for the FIRST and LAST card to reach the centre.
     A centre-snapping rail whose end cards cannot physically get there just
     parks them against the padding, which is the off-centre look this was
     changed to fix. The scrollport is the rail's border box - the section
     plus the 16px it is pulled out to either side - so the slack beside a
     card of width `w` is (100cqw + 32px - w) / 2, and at w = 74cqw that is
     the calc below. Change one and the other has to follow. */
  padding: 0 calc(13cqw + 16px) 4px;
  margin: 0 -16px;
}

/* The width the cqw maths above is measured against. Containment has to sit
   on the section rather than on the rail itself: the rail's own padding is
   what is being solved for, and a container sized by its own content box
   would be defining that padding in terms of itself. */
.card-rail-frame {
  container-type: inline-size;
}
.card-rail::-webkit-scrollbar { display: none; }

/* Snap to the centre. Aligning to the start put every resting card hard
   against the left gutter with the whole of the slack - and the peek of the
   next card - piled on the right, which reads as a rail that has been
   scrolled rather than one sitting where it belongs. Centred, the card being
   read is in the middle of the screen and both neighbours show an edge, so
   which way there is more to swipe is visible in both directions.

   This is only true because of the padding above; see the note there. */
.card-rail > * {
  scroll-snap-align: center;
  flex: 0 0 auto;
  /* The cards are buttons, and a button's default touch-action overrides the
     rail's. Without this a drag that begins on a card is treated as a tap on
     that card rather than a pan of the rail. */
  touch-action: pan-x;
}

/* Narrow enough that the next card always shows an edge. A peeking card is
   what tells someone there is more to swipe — without it the rail looks like
   a single card and nobody tries.

   The section cards are wide because they now carry a photograph AND a
   description; at the old 54% the description broke to two or three words a
   line. The empty-bulletin notice is narrower — it is one sentence, and at
   card width it pushed the ministry card, the useful one, off the screen. */
```

```
/* One width for everything in the rail, in cqw so it is measured against the
   section rather than against the rail's own content box, which the padding
   above already depends on.

   The notice used to be narrower than a card. That was right when cards
   snapped to the left and a wide notice pushed the ministry card off the
   screen; centred, the notice is its own slide with dots under it, and a
   rail whose slides are different widths cannot be centred to one padding. */
.card-rail__card,
.card-rail__note { width: 74cqw; }

/* The section cards match each other's height whatever their copy runs to —
   two photo cards of visibly different heights side by side reads as a
   mistake. They stretch; the notice above does not, which is the whole
   reason the rail aligns to flex-start rather than stretching everything. */
.card-rail__link {
  width: 74cqw;
  align-self: stretch;
}

/* The place card that opens when a map pin is tapped. A bottom sheet rather
   than a pin-anchored popup: the map frame is only ~600px tall, so a card
   with a photograph in it cannot open upward from a pin without covering the
   thing it describes or being clipped by the canvas. */
.map-sheet {
  position: absolute;
  left: 10px;
  right: 10px;
  bottom: 10px;
  z-index: 20;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  border-radius: 20px;
  box-shadow: 0 12px 32px rgb(0 0 0 / 0.18);
  overflow: hidden;
  animation: map-sheet-in 180ms ease-out;
}
@keyframes map-sheet-in {
  from { transform: translateY(14px); opacity: 0; }
  to   { transform: translateY(0);    opacity: 1; }
}
@media (prefers-reduced-motion: reduce) {
  .map-sheet { animation: none; }
}

.map-sheet__close {
  position: absolute;
  top: 8px;
  right: 8px;
  z-index: 2;
  height: 30px;
  width: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  background: color-mix(in srgb, var(--color-brand-card) 88%, transparent);
  border: 1px solid var(--color-brand-border);
  color: var(--color-brand-secondary);
```

```
}
.map-sheet__photo {
  display: block;
  width: 100%;
  height: 110px;
  object-fit: cover;
}
.map-sheet__body { padding: 12px 14px 14px; }
.map-sheet__name {
  font-size: 18px;
  font-weight: 700;
  line-height: 1.25;
  color: var(--color-brand-text);
  /* Room for the close button. */
  padding-right: 34px;
}
.map-sheet__meta,
.map-sheet__distance,
.map-sheet__patron {
  font-size: 15px;
  line-height: 1.4;
  color: var(--color-brand-secondary);
  margin-top: 3px;
}
.map-sheet__actions {
  display: flex;
  gap: 8px;
  margin-top: 12px;
}
.map-sheet__action {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 11px 12px;
  border-radius: 999px;
  font-size: 16px;
  font-weight: 600;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  border: 1px solid var(--color-brand-primary);
}
.map-sheet__action:disabled { opacity: 0.55; }
.map-sheet__action--ghost {
  flex: 0 0 auto;
  background: transparent;
  color: var(--color-brand-primary);
}
.map-sheet__status {
  margin-top: 8px;
  font-size: 15px;
  line-height: 1.4;
  color: var(--color-brand-secondary);
}
.map-sheet__link {
  margin-top: 10px;
  display: flex;
  align-items: center;
```

```
  gap: 2px;
  font-size: 15px;
  font-weight: 600;
  color: var(--color-brand-primary);
  background: none;
  border: 0;
  padding: 0;
}
.map-sheet__soon {
  margin-top: 10px;
  font-size: 15px;
  color: var(--color-brand-secondary);
}

/* Google's travel-mode row: an icon and an estimated time per mode.
   Walk, motorcycle and car — the three ways people actually reach a parish
   in Caloocan. No transit tab: no free routing service carries jeepney data
   (see travelModes.ts), and an invented time would be worse than none. */
.map-sheet__modes {
  display: flex;
  gap: 6px;
  margin-top: 12px;
  border-bottom: 1px solid var(--color-brand-border);
}
.map-sheet__mode {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 3px;
  padding: 8px 4px 9px;
  color: var(--color-brand-secondary);
  background: none;
  border: 0;
  /* The selected tab is marked by a rule directly under it, so the row keeps
     working for anyone who cannot separate the two colours. */
  border-bottom: 2px solid transparent;
  margin-bottom: -1px;
}
.map-sheet__mode[data-selected="true"] {
  color: var(--color-brand-primary);
  border-bottom-color: var(--color-brand-primary);
}
.map-sheet__mode-time {
  font-size: 14px;
  font-weight: 600;
  font-variant-numeric: tabular-nums;
}
.map-sheet__estimate {
  margin-top: 8px;
  font-size: 14px;
  line-height: 1.4;
  color: var(--color-brand-secondary);
}

/* ─── HOME: the "bright" theme ──────────────────────────────────────────
   Scoped to Home, deliberately.

   The palette tokens are ROLES, not colours, so redefining them here rather
```

```
   than in :root re-skins everything Home renders — including shared
   components like BulletinRail — without touching a single component and
   without changing Map, Pray, Me or Scan, which stay warm paper. Rolling
   this theme across the whole app later is a matter of moving the block to
   :root; that is the entire reason not to hardcode hex into components.

   Colours are estimated from a compressed screenshot mockup, not from a
   design file, and are expected to need tuning against the real thing. Every
   pairing below was checked for contrast before being written down. */
.home-bright {
  /* The parish's own three colours, or Mary Help's navy when the element
     above has not set them. Everything below is mixed from these, so a
     parish is three hex values in src/lib/parishTheme.ts and no CSS.

     The mixes replace eight hand-picked navies. For Mary Help they land
     within a few RGB points of the values they replace - the blue cast
     survives because the pale steps are mixed into --parish-tint, which is
     itself a tinted white, rather than into plain white, which would wash
     the hue out of every one of them. */
  --parish-ink: #1B2A6B;
  --parish-band: #101B4A;
  --parish-tint: #EEF3FC;

  --color-brand-bg: var(--parish-tint);
  --color-brand-card: #FFFFFF;         /* Cards and raised surfaces */
  --color-brand-card-sunk: color-mix(in srgb, var(--parish-ink) 4%, var(--parish-tint));
  --color-brand-border: color-mix(in srgb, var(--parish-ink) 8%, var(--parish-tint));
  --color-brand-hairline: color-mix(in srgb, var(--parish-ink) 8%, transparent);

  /* 13.6:1 on the navy ground, 11.0:1 on the cream one. */
  --color-brand-text: color-mix(in srgb, var(--parish-ink) 70%, #0B0F1A);
  /* The weakest pair in the system at 6.1:1, still comfortably past AA. */
  --color-brand-secondary: color-mix(in srgb, var(--parish-ink) 50%, #6B7280);

  --color-brand-primary: var(--parish-ink);   /* 13.2:1 navy, 9.4:1 brown */
  --color-brand-primary-dark: color-mix(in srgb, var(--parish-ink) 78%, #000);
  --color-brand-accent: var(--parish-ink);
  --color-brand-on-accent: #FFFFFF;

  /* Home-only additions. Named for what they are on this screen. */
  --home-heading: var(--parish-ink);
  --home-watermark: color-mix(in srgb, var(--parish-ink) 12%, var(--parish-tint));
  /* Unused since the per-section motifs replaced it, and it cannot follow
     the parish anyway - the stroke colour is baked into the data URI. Kept
     only because removing it is a separate change from recolouring. */
  --home-watermark-svg:
url("data:image/svg+xml,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%22132%22%20height%3D%22132%22%20viewBox%3D%220%200%20132%20132%22%3E%3
Cg%20fill%3D%22none%22%20stroke%3D%22%23D8E2F2%22%20stroke-width%3D%222%22%20stroke-linejoin%3D%22round%22%20stroke-
linecap%3D%22round%22%3E%3Cpath%20d%3D%22M18%2074%20h40%20v26%20h-40%20z%22%2F%3E%3Cpath%20d%3D%22M18%2074%20l20%20-
16%20l20%2016%22%2F%3E%3Cpath%20d%3D%22M34%20100%20v-14%20h8%20v14%22%2F%3E%3Cpath%20d%3D%22M38%2058%20v-
10%20M34%2052%20h8%22%2F%3E%3Cpath%20d%3D%22M100%2030%20v26%20M92%2038%20h16%22%2F%3E%3Cpath%20d%3D%22M86%20108%20h26%20v16%20h-
26%20z%22%2F%3E%3Cpath%20d%3D%22M86%20108%20l13%20-11%20l13%2011%22%2F%3E%3Cpath%20d%3D%22M99%20124%20v-9%22%2F%3E%3C%2Fg%3E%3C%2Fsvg%3E");

  /* The Mass panel's gradient.
     The mockup's green is around #4CAF50, and white body text on that is
     2.78:1 — a clear fail, and what sits on it is the Mass TIMES, the single
     most important thing on the card. Both stops are therefore darkened
     until the LIGHTEST point still carries white body text at 4.7:1. The
     colour still reads as the same vivid green; it is now legible in a
```

```
     church porch in daylight, which the original was not. */
  --home-mass-from: #37833B;
  --home-mass-to: #1F5C24;
}

/* The faint church line-art behind everything. Drawn rather than shipped as
   an image so it recolours with the token and costs no request. */
/* Colour only. The pattern now comes from the per-section motifs below,
   which is what the design does: push-pins behind the bulletin, calendars
   behind the schedule, crosses behind the history. Painting the old
   church line-art here as well put two watermarks on top of each other. */
.home-bright__ground {
  background-color: var(--color-brand-bg);
}

/* The three section titles: centred, heavy, widely tracked, stacked over
   two lines the way the mockup has them. */
.home-section-title {
  margin: 26px 0 14px;
  text-align: center;
  /* Montserrat at its heaviest. The mockup's headings look like a heavier
     geometric face, but the app already loads Montserrat 300-900, and a
     second webfont for three headings is a real cost on parish wifi for a
     difference only a designer would notice. */
  font-family: var(--font-primary);
  font-size: 30px;
  font-weight: 900;
  line-height: 1.04;
  letter-spacing: 0.02em;
  text-transform: uppercase;
  color: var(--home-heading);
  text-wrap: balance;
}

/* The dots under each rail. Purely an indicator — the rail itself is what
   scrolls, and it stays reachable by keyboard through its own buttons. */
.home-rail-dots {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 6px;
  margin-top: 12px;
}
.home-rail-dots span {
  width: 7px;
  height: 7px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--home-heading) 22%, transparent);
}
.home-rail-dots span[data-active="true"] {
  width: 20px;
  background: color-mix(in srgb, var(--home-heading) 55%, transparent);
}

/* ----------------------------------------------------------------------
   The welcome header, from the supplied PSD.

   Dark navy, full bleed, with the parish's patron image coming in from the
   right and the text sitting over it. The measurements are taken from the
```

```
   design at its own scale: the artboard is 1127px wide for a 390px screen,
   so everything from it is divided by 2.89.
   ------------------------------------------------------------------------ */
.parish-welcome {
  /* Scrolls up by exactly the amount that should disappear, then pins. The
     offset is negative, which is the whole trick: the band keeps its full
     layout height, so the page never changes length and the scroll position
     is never clamped, while the part that should go simply travels off the
     top. See ParishWelcomeHeader for the rest. */
  position: sticky;
  top: calc(-1 * var(--pw-collapse, 0px));
  /* Above the content sheet, so the sheet scrolls underneath the collapsed
     bar instead of over it. */
  z-index: 20;

  /* Defaults, so the resting band is correct before any script runs and
     stays correct if none ever does. */
  --pw-t: 0;
  --pw-collapse: 0px;
  --pw-title-dy: 0px;
  --pw-scale-delta: 0;
  --pw-media-dy: 0px;
  --pw-image-dy: 0px;
  /* Height of the rounded lip drawn at this band's bottom edge.
     The sheet's padding-top has to exceed it or the heading ends
     up behind the curve. */
  --pw-lip: 26px;

  isolation: isolate;
  overflow: hidden;
  /* Scan and Pray put the band inside a flex column that is itself the
     scroller, and a flex child with no basis is free to be squashed - the
     band collapsed to 48px on both. Home never showed it because its
     column is not the scroller. The band's height is its content's. */
  flex-shrink: 0;
  background: var(--parish-band, #101B4A);
  /* Paints up into the status bar area rather than stopping below it. */
  margin-top: calc(-1 * env(safe-area-inset-top, 0px));
  padding-top: env(safe-area-inset-top, 0px);
  /* Room for the sheet's curve to sit over this band without landing
     on the street address. 22px of breathing room plus the 26px the
     sheet is pulled up by, below. */
  padding-bottom: 48px;
  /* Pulls the sheet up so its rounded top is drawn OVER this band.
     That overlap is the whole reason the curve is visible: the sheet's
     corners are transparent, so without dark behind them there is a
     straight edge and no curve at all.

     What does NOT come back with it is the lip. The band used to paint
     its own copy of the curve on top at z-index 2, and that copy is
     what sliced the heading in half and left a spare strip under the
     collapsed header. One curve, drawn by the sheet, with the band
     simply showing through behind it. */
  margin-bottom: -26px;
}

/* The curve where the sheet meets the band.

   This lip is what actually draws it. The band sits above the sheet in
```

```
   paint order, so the sheet's own rounded top is hidden behind it -
   deleting this once left a dead straight edge with no curve at all,
   which is not what anybody wanted.

   The bug it used to cause was never the curve itself, it was the
   heading sitting underneath the curve: at 8px of sheet padding,
   PARISH BULLETIN began 12px inside this lip and had the top of every
   letter covered. The sheet now starts its content at 36px - past the
   26px of lip, with ten to spare - so the curve is drawn and the words
   are below it rather than behind it. */
.parish-welcome::after {
  content: "";
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
  height: var(--pw-lip);
  background: color-mix(in srgb, var(--parish-ink, #1B2A6B) 2%, var(--parish-tint, #EEF3FC));
  border-radius: 30px 30px 0 0;
  z-index: 2;
}

/* Drifts up more slowly than the page, so the collapse reads as depth
   rather than as the photograph being dragged away. The factor is how much
   of the scroll the picture absorbs: 0 would move it with the band, 1 would
   nail it to the screen. */
.parish-welcome__media {
  position: absolute;
  inset: 0;
  z-index: -1;
  transform: translate3d(0, calc(var(--pw-t) * var(--pw-media-dy, 0px)), 0);
  will-change: transform;
}

/* Fades while sliding out of the way. Fading rather than letting the screen
   edge clip it is what stops the greeting being cut through mid-letter on
   the way out, and the 1.45 has it gone before the collapse finishes. */
.parish-welcome__intro {
  transform: translate3d(0, calc(var(--pw-t) * -16px), 0);
  opacity: calc(1 - var(--pw-t) * 1.45);
  will-change: transform, opacity;
}

/* Travels DOWN inside the band by as much as the band travels up, which
   holds it still on screen, and shrinks to the collapsed size as it goes.
   Scaling rather than animating font-size keeps it off the layout path —
   font-size would lay the page out again on every frame. */
.parish-welcome__title {
  transform: translate3d(0, calc(var(--pw-t) * var(--pw-title-dy)), 0)
    scale(calc(1 - var(--pw-t) * var(--pw-scale-delta)));
  transform-origin: left top;
  will-change: transform;
}

.parish-welcome__image {
  position: absolute;
  inset: 0 0 0 28%;
  background-size: cover;
  background-position: center top;
```

```
  /* On top of the wrapper's parallax: the small extra shift that lands the
     patron's face in the collapsed bar rather than behind the sheet's lip.
     It is per parish, because each photograph is composed differently — see
     PARISH_HEADER_FACES in data.ts. A transform rather than moving
     background-position, which would repaint the picture every frame. */
  transform: translate3d(0, calc(var(--pw-t) * var(--pw-image-dy, 0px)), 0);
  will-change: transform;
}

/* Darkens everything a little, the text side a lot. */
.parish-welcome__scrim {
  position: absolute;
  inset: 0;
  /* The same three-stop wash as before, written against the parish's band
     colour instead of the navy literal it used to repeat six times. */
  background:
    linear-gradient(
      90deg,
      var(--parish-band, #101B4A) 34%,
      color-mix(in srgb, var(--parish-band, #101B4A) 72%, transparent) 52%,
      color-mix(in srgb, var(--parish-band, #101B4A) 8%, transparent) 78%
    ),
    linear-gradient(
      180deg,
      color-mix(in srgb, var(--parish-band, #101B4A) 55%, transparent),
      color-mix(in srgb, var(--parish-band, #101B4A) 12%, transparent) 38%,
      color-mix(in srgb, var(--parish-band, #101B4A) 45%, transparent)
    );
}

.parish-welcome__content {
  position: relative;
  /* The status bar sits over this band in the packaged app, so the top
     row is inset past it. On the web the inset is zero and this is just
     the design's own 14px. */
  padding: calc(14px + env(safe-area-inset-top, 0px)) 20px 0;
  color: #fff;
}

.parish-welcome__top {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 16px;
}

.parish-welcome__brand {
  display: inline-flex;
  align-items: center;
}

/* Sized by HEIGHT, width following. The lockup is 3.4:1; fixing the width
   instead would let the bar's height move whenever the artwork changed. */
.parish-welcome__lockup {
  height: 28px;
  width: auto;
  display: block;
}
```

```
.parish-welcome__crest {
  /* The submark is a portrait oval, roughly 3:4. A square box with
     object-fit: contain would letterbox it and leave it visually smaller
     than the wordmark beside it, so the box matches the artwork instead.
     No border-radius: the mark is already a closed oval, and rounding its
     bounding box only clips the cross. */
  width: 22px;
  height: 29px;
  object-fit: contain;
}

.parish-welcome__avatar {
  width: 34px;
  height: 34px;
  border-radius: 999px;
  display: grid;
  place-items: center;
  background: rgb(255 255 255 / 0.92);
  color: #5A6B8C;
  border: none;
  cursor: pointer;
  transition: transform 0.12s ease;
}
.parish-welcome__avatar:active { transform: scale(0.94); }

.parish-welcome__greeting {
  font-family: var(--font-primary);
  margin: 0;
  text-shadow: 0 2px 10px rgb(6 12 38 / 0.45);
}

/* The salutation. Small, wide-tracked and slightly held back - it is the
   same word every morning, and at 30px it was the loudest thing on a
   screen whose job is to say which parish you are in. */
.parish-welcome__salutation {
  display: block;
  font-size: 14px;
  font-weight: 700;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  line-height: 1.1;
  color: rgb(255 255 255 / 0.82);
}

/* Standing alone - signed out, with no name to follow - it would look
   stranded at label size, so it keeps some presence. */
.parish-welcome__salutation--alone {
  font-size: 26px;
  letter-spacing: 0.01em;
  color: #fff;
}

.parish-welcome__name {
  display: block;
  margin-top: 3px;
  font-size: 30px;
  font-weight: 800;
  line-height: 1.04;
```

```
  letter-spacing: 0.005em;
  /* A nickname is written the way its owner wrote it. Upper-casing it
     turned "Adrich" into shouting and mangled any name with its own
     capitalisation. */
  text-transform: none;
  color: #fff;
  /* Long nicknames shorten rather than pushing the parish name off the
     band. */
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.parish-welcome__date {
  margin: 8px 0 0;
  font-size: 13px;
  font-weight: 500;
  letter-spacing: 0.06em;
  color: rgb(255 255 255 / 0.9);
  text-shadow: 0 1px 8px rgb(6 12 38 / 0.5);
}

/* The parish name is the one italic serif on the screen, exactly as the
   design has it - it is the title of the page, not a label. */
.parish-welcome__parish {
  margin: 14px 0 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 27px;
  line-height: 1.12;
  text-wrap: balance;
  text-shadow: 0 2px 12px rgb(6 12 38 / 0.5);
}

.parish-welcome__location {
  display: flex;
  align-items: center;
  gap: 6px;
  margin: 8px 0 0;
  font-size: 14.5px;
  color: rgb(255 255 255 / 0.92);
  text-shadow: 0 1px 8px rgb(6 12 38 / 0.55);
}

/* The content sheet below the header: rounded top corners lifted over the
   navy, with the chevron the design puts at its top edge. */
.home-sheet {
  position: relative;
  border-radius: 30px 30px 0 0;
  /* Below the curve, not inside it.

     The sheet is pulled 26px up over the band, so its first 26px are
     the rounded corners themselves. The heading has to start after
     them: at 8px it began inside the curve and the band showed through
     the lettering, which is what made PARISH BULLETIN unreadable on
     San Roque, whose heading ink is a brown close to its own band.

     36px is those 26 plus ten of daylight. Anything under 26 puts the
```

```
     heading back inside the curve. */
  padding-top: 36px;
  /* Clips the section motifs at the screen edge, which is where they were
     always meant to stop.
     They bleed outwards on purpose - `--motif-bleed` is negative so each
     pair runs off both sides - but a bleed to the RIGHT extends the
     scrollable width of every ancestor, and that was the 92px the whole app
     could be dragged sideways into. Clipping here removes the overflow at
     its source rather than forbidding the scroll higher up; the sheet spans
     the full width, so the motifs still disappear off the edge exactly as
     before. overflow-y stays visible, which `clip` on one axis permits. */
  overflow-x: clip;
  /* Slightly deeper at the top, palest in the middle, back to the page
     ground at the foot - the same shape as before, now in the parish's
     own tint. */
  background: linear-gradient(
    180deg,
    color-mix(in srgb, var(--parish-ink) 2%, var(--parish-tint)) 0%,
    color-mix(in srgb, var(--parish-tint) 55%, #fff) 42%,
    var(--parish-tint) 100%
  );
}

/* The sheet had a chevron at its top edge, in the band where it rides
   up over the hero photograph. It was there to say "this panel pulls
   up" - and nothing pulls. There is no drag handler on .home-sheet and
   never was, so it promised a gesture the app does not have, and what
   it actually looked like was a stray line drawn across the parish's
   name. Removed rather than restyled: the honest fix for a control
   that controls nothing is to take it away, and the sheet's rounded
   top already says where it begins. */

/* Each section carries its own faint motif, as the design does: push-pins
   behind the bulletin, calendars behind the schedule, crosses behind the
   history. The images are the PSD's own icons.

   TWO of each, not a repeating tile. This was tiled at 118px in both
   directions, which filled every section with a dozen identical pins and
   read as wallpaper. The PSD has exactly two layers per section - e.g.
   `push-pin` and `push-pin copy` - sitting one on each side of the section
   heading and bleeding off both edges of the screen. Comparing the pairs
   pixel by pixel, each `copy` is the horizontal mirror of its partner
   (difference 0.6 mirrored against 21 un-mirrored), which is why ::after
   is the same image flipped rather than a second file.

   The sizes and offsets are the PSD's own, divided by 3: the artboard is
   1125px across where the phone is 375, which the 1030px history card
   confirms - it is the 343px card on screen. Offsets are in pixels rather
   than percentages because `background-position: left -15%` measures the
   percentage against the container MINUS the image, not the container, so
   a percentage there would not mean what it appears to. Pixels also keep
   the bleed the same at any phone width, which is the point of a bleed. */
.home-motif {
  position: relative;
  /* Makes this a stacking context, so the motifs below sit behind this
     section's own text but still in front of the sheet's gradient. Without
     it a negative z-index escapes to the nearest stacking context and the
     motifs vanish underneath the sheet. */
  isolation: isolate;
```

```
}

/* Masked rather than drawn.
   The three motif PNGs are flat #CEDBEE at 85% alpha - Mary Help's blue,
   baked in - so used as background-images they stayed blue on San Roque's
   warm ground and read as a printing mistake. Taking the shape from the
   file's alpha and the colour from --home-watermark lets one set of
   artwork serve every parish. */
.home-motif::before,
.home-motif::after {
  content: "";
  position: absolute;
  z-index: -1;
  top: var(--motif-top);
  width: var(--motif-width);
  aspect-ratio: var(--motif-ratio);
  background: var(--home-watermark);
  -webkit-mask: var(--motif) center / contain no-repeat;
  mask: var(--motif) center / contain no-repeat;
  pointer-events: none;
}

.home-motif::before { left: var(--motif-bleed); }

/* The design's mirrored copy. */
.home-motif::after {
  right: var(--motif-bleed);
  transform: scaleX(-1);
}

.home-motif--pin {
  --motif: url("/ui/motif-pin.png");
  --motif-ratio: 200 / 259;
  --motif-width: 111px;
  --motif-bleed: -51px;
  --motif-top: -29px;
}

.home-motif--calendar {
  --motif: url("/ui/motif-calendar.png");
  --motif-ratio: 200 / 196;
  --motif-width: 169px;
  --motif-bleed: -108px;
  --motif-top: -26px;
}

.home-motif--cross {
  --motif: url("/ui/motif-cross.png");
  --motif-ratio: 200 / 268;
  --motif-width: 160px;
  --motif-bleed: -73px;
  --motif-top: -28px;
}

/* On a touch device this WebView draws a classic 6px scrollbar and takes
   that width out of the layout, so the full-bleed header band stopped six
   pixels short of the screen edge with a pale strip beside it. Phones
   scroll by touch and expect an overlay indicator, so the gutter is
   removed there; on a desktop pointer the scrollbar stays, because there
```

```
   it is the only way to see or drag the scroll position. */
/* The page must never scroll sideways.
   overflow-y-auto sets only ONE axis, and CSS then computes the other from
   `visible` to `auto` - so the app has been horizontally scrollable all
   along. It had 92px of slack to slide into, which is what produced the
   white column down the right-hand side with the content pushed off to the
   left: one stray sideways drag on the bulletin rail and the whole app was
   scrolled over, with no way to tell what had happened.
   `clip` rather than `hidden`, because hidden makes this a programmatic
   scroll container all over again; clip simply refuses. */
.app-scroll {
  overflow-x: clip;
}

/* Clearance for the raised Scan button.
   The Scan tab is lifted 12px above the bar (-translate-y-3 in App.tsx),
   so its circle sticks up out of the bar and into the scrolling content.
   Anything that comes to rest in that band is covered and cannot be
   tapped - measured on Home, where it sat on top of the "Show less"
   control, and on Ministries, where it sat on a ministry in the list.

   The padding is on the scrollers rather than the bar because the bar
   cannot grow: the circle is positioned relative to it, so making the
   bar taller moves the circle up with it and the overlap travels along.
   Reserving the band at the end of the content is what actually keeps
   the last item clear. 16px: the 12px overhang plus a little daylight,
   so a control is not merely clear of the circle but readably so. */
/* The full-bleed tabs take none of it.

   Map is a map and Pray is the Rosary: both fill their space to the
   bar, and neither ends in a card the floating button could sit on.
   The clearance there was simply a strip of dead background above the
   tab bar - which is what it looked like. */
.app-scroll[data-tab=navigator],
.app-scroll[data-tab=rosary],
.app-scroll[data-tab=ar] {
  padding-bottom: 0;
}

/* History, Ministries and Sacraments do the same as Home, below.

   Each of these three fills its screen with one coloured panel - the
   history gradient, or the card tint the two lists sit on. The 56px of
   clearance was on the SCROLLER, one layer above that panel, so the gap
   showed the shell underneath: a pale strip ruled across the bottom of
   a deep blue page. Moved onto the panel, the panel's own colour simply
   continues down to the tab bar and there is nothing to see.

   Ministries already pads itself through .scroll-safe, so it only needs
   the outer padding removed or the two would add up to 112px. */
.app-scroll[data-tab=history],
.app-scroll[data-tab=ministries],
.app-scroll[data-tab=sacraments] {
  padding-bottom: 0;
}
.app-scroll[data-tab=history] .parish-history {
  padding-bottom: 56px;
}
```

```
/* Home takes its clearance from the sheet instead of the scroller.

   The same 56px, one layer lower. On the scroller the gap showed the
   shell underneath it - a near-white that is not the parish tint the
   sheet ends on - so it read as a white band ruled across the bottom
   of the page. Moved onto the sheet, the sheet's own colour simply
   continues down to the bar and there is nothing to see. */
.app-scroll[data-tab=home] {
  padding-bottom: 0;
}
.app-scroll[data-tab=home] .home-sheet {
  padding-bottom: 56px;
}

.app-scroll,
.scroll-safe {
  /* Clearance for BOTH floating things at the bottom of the app.

     16px was enough for the raised Scan button, which only overhangs
     the bar by 12px. It was never enough for the Ask Sancti button,
     which floats 76px up from the bar and stands 42px tall - measured,
     it reaches 54px above the scroller's bottom edge. So the last
     item on every screen sat underneath it: on Home that was the
     announcement note, with the button painted across the sentence
     telling you there was nothing this week.

     68px is that 54 plus daylight, so the final line clears the button
     rather than merely escaping it. This is the ordinary cost of a
     floating action button, and the reason apps that have one reserve
     room for it. */
  /* 54px is what the Ask Sancti button actually reaches above the
     bar - measured, not guessed - and the extra two keep the last line
     from touching it.

     It cannot go to zero on a page of cards while a button floats
     there: the clearance IS the room that button occupies, and without
     it the last card is covered again. It is switched off below for
     the pages that have no last card to cover. */
  padding-bottom: 56px;
}

/* No scrollbar gutter at phone widths.

   A touch device gives you an overlay scrollbar that takes no space, so
   the coarse-pointer rule was enough while the app only ever ran on a
   phone or filled a laptop window. It is not enough inside the landing
   page's frame: that is a desktop browser rendering a 375px viewport,
   the pointer is a mouse, and the classic 5px gutter appeared as a pale
   stripe down the right edge of the mockup - clearly visible against
   the navy hero.

   Keyed to width as well as pointer, because the question is really
   "is this laid out as a phone", and 768px is the same breakpoint
   isDeviceSurface uses to answer it. */
@media (pointer: coarse), (max-width: 768px) {
  .app-scroll {
    scrollbar-width: none;
  }
```

```
  .app-scroll::-webkit-scrollbar {
    width: 0;
    height: 0;
  }
}

/* The dots under the bulletin rail: one per card, the one in view drawn as
   a pill. Horizontal, because they mirror the axis the rail scrolls on. */
.rail-dots {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 2px;
  margin-top: 10px;
}

/* The visible dot is 7px, which no finger can hit. The button around it is
   the touch target and the dot is only its mark. */
.rail-dots__hit {
  display: grid;
  place-items: center;
  width: 22px;
  height: 24px;
  padding: 0;
  border: none;
  background: none;
  cursor: pointer;
  -webkit-tap-highlight-color: transparent;
}

.rail-dots__dot {
  width: 7px;
  height: 7px;
  border-radius: 999px;
  /* Falls back to the app accent: the rail is a shared component and
     --home-heading only exists inside Home's bright theme. */
  background: color-mix(in srgb, var(--home-heading, var(--color-brand-primary)) 26%, transparent);
  transition: width 0.18s ease, background 0.18s ease;
}

.rail-dots__hit[aria-current="true"] .rail-dots__dot {
  width: 18px;
  background: var(--home-heading, var(--color-brand-primary));
}

.rail-dots__hit:focus-visible {
  outline: 2px solid var(--home-heading, var(--color-brand-primary));
  outline-offset: 1px;
  border-radius: 999px;
}

@media (prefers-reduced-motion: reduce) {
  .rail-dots__dot { transition: none; }
}

/* ── A parish's history ──────────────────────────────────────────
   The "MHCP HISTORY PAGE" / "SRCP HISTORY PAGE" artboards from the client's
   PSD, which are the same layout in two liveries: the patron fills the top
   of the screen, and the history rises over the bottom of that photograph on
```

```
   a panel coloured out of the image.

   The two colours come in as --ph-top and --ph-bottom, set per parish by the
   component. Everything here is written against those variables, so a third
   parish is a pair of hex values and no CSS. */
.parish-history {
  flex: 1;
  min-height: 0;
  overflow-y: auto;
  overscroll-behavior-y: contain;
  background: var(--ph-bottom);
  scrollbar-width: none;
}
.parish-history::-webkit-scrollbar { display: none; }

/* Just under half the screen, as in the artboard. A fraction rather than a
   fixed height: the same 46% frames the patron the same way on a 640px
   phone and on a 900px one, where 320px would fill the first and float in
   the middle of the second. */
.parish-history__hero {
  position: relative;
  height: 46vh;
  min-height: 260px;
  background: var(--ph-bottom);
}

.parish-history__photo {
  width: 100%;
  height: 100%;
  object-fit: cover;
  /* The face, not the middle of the frame, and measured per photograph
     rather than guessed once for all of them - the two patrons are framed
     quite differently, and one rule that suits Maria Auxiliadora crops San
     Roque through the eyes. The component supplies --ph-face. */
  object-position: var(--ph-face, center 30%);
  display: block;
}

.parish-history__photo--empty {
  background: linear-gradient(180deg, var(--ph-top), var(--ph-bottom));
}

/* Over the photograph, where the design has no header at all. Dark enough to
   read against a gilded statue under warm light, which is the one background
   a white button cannot be trusted on. */
.parish-history__back {
  position: absolute;
  top: calc(12px + env(safe-area-inset-top));
  left: 14px;
  width: 42px;
  height: 42px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  color: #fff;
  background: rgb(9 14 32 / 0.55);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
```

```
}

/* Pulled up over the photograph, which is what makes the page one image
   rather than a picture with a caption under it. min-height keeps the colour
   running to the bottom of the screen on a parish whose history is short —
   without it the panel ends mid-screen and the scroll container's own
   background shows through as a band. */
.parish-history__panel {
  position: relative;
  margin-top: -34px;
  min-height: 54vh;
  border-radius: 34px 34px 0 0;
  background: linear-gradient(180deg, var(--ph-top) 0%, var(--ph-bottom) 62%);
  overflow: hidden;
  isolation: isolate;
}

/* The patron again, behind the words. Very low opacity and offset to the
   right, as in the PSD: it is texture, not a second photograph, and anything
   stronger competes with the text it sits under. */
.parish-history__watermark {
  position: absolute;
  top: 0;
  right: -18%;
  width: 92%;
  height: 100%;
  object-fit: cover;
  object-position: center 18%;
  opacity: 0.17;
  mix-blend-mode: soft-light;
  pointer-events: none;
  z-index: -1;
}

.parish-history__body {
  padding: 30px 22px calc(34px + env(safe-area-inset-bottom));
  color: #fff;
}

/* The one italic serif on the page, matching the parish name in the home
   header. The shadow is the design's and it earns its place here: the title
   sits directly over the brightest part of the panel gradient. */
.parish-history__title {
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 40px;
  line-height: 1.04;
  letter-spacing: -0.01em;
  text-wrap: balance;
  text-shadow: 0 3px 16px rgb(6 12 38 / 0.45);
}

/* The listen control, directly under the title.
   Outlined rather than filled: it sits on the patron's photograph, and a
   solid pill there reads as a second, competing subject. It goes solid
   while speaking, which is the only state worth shouting about. */
.parish-history__listen {
```

```
  margin: 14px 0 0;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 8px 15px;
  border-radius: 999px;
  border: 1.5px solid rgb(255 255 255 / 0.55);
  background: rgb(255 255 255 / 0.1);
  color: #fff;
  font-family: inherit;
  font-size: 14px;
  font-weight: 700;
  cursor: pointer;
  -webkit-backdrop-filter: blur(6px);
  backdrop-filter: blur(6px);
  transition: background 140ms ease, border-color 140ms ease, transform 120ms ease;
}
.parish-history__listen:hover { background: rgb(255 255 255 / 0.18); }
.parish-history__listen:active { transform: scale(0.985); }
.parish-history__listen[aria-pressed="true"] {
  background: #fff;
  border-color: #fff;
  color: var(--color-brand-text);
}
.parish-history__listen:focus-visible {
  outline: 2px solid #fff;
  outline-offset: 2px;
}

/* How long the reading takes, so a tap is an informed one. Quieter than
   the verb beside it: it is a caveat, not the action. */
.parish-history__listen-time {
  font-weight: 600;
  opacity: 0.72;
  font-size: 13px;
}

@media (prefers-reduced-motion: reduce) {
  .parish-history__listen { transition: none; }
  .parish-history__listen:active { transform: none; }
}

.parish-history__section { margin-top: 26px; }
.parish-history__section:first-of-type { margin-top: 14px; }

.parish-history__eyebrow {
  margin: 0 0 14px;
  font-size: 15px;
  font-weight: 700;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  text-shadow: 0 2px 10px rgb(6 12 38 / 0.4);
}

.parish-history__paragraph {
  margin: 0 0 14px;
  font-size: 16.5px;
  line-height: 1.62;
  color: rgb(255 255 255 / 0.93);
```

```
}
.parish-history__paragraph:last-child { margin-bottom: 0; }

/* ── Photographs in the parish history ──────────────────────────
   These are scans from a souvenir programme and prints from the 1950s
   onwards: low contrast, paper grain, some of them creased. They are
   given a light mount rather than being dropped straight onto the dark
   band, because a faded sepia print on near-black reads as a stain. */
/* The AR tab, when the tour cannot open here.
   Sits in the scanner sheet over the live camera, so it is a panel
   rather than a page: dark, legible on whatever the camera happens to
   be pointed at, and the same shape as the AR panel it replaces. */
.scan-gate {
  display: flex;
  flex-direction: column;
  align-items: center;
  text-align: center;
  gap: 7px;
  padding: 18px 20px 20px;
}

.scan-gate__icon {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 42px;
  height: 42px;
  border-radius: 999px;
  background: rgb(255 255 255 / 0.13);
  border: 1px solid rgb(255 255 255 / 0.28);
  color: #fff;
}

.scan-gate__title {
  margin: 2px 0 0;
  font-size: 17px;
  font-weight: 700;
  color: #fff;
}

.scan-gate__body {
  margin: 0;
  max-width: 34ch;
  font-size: 14.5px;
  line-height: 1.5;
  color: rgb(255 255 255 / 0.78);
}

.parish-figure {
  margin: 18px 0 22px;
}

.parish-figure__frame {
  border-radius: 14px;
  overflow: hidden;
  background: rgb(255 255 255 / 0.9);
  padding: 7px;
  box-shadow: 0 6px 20px rgb(0 0 0 / 0.3);
}
```

```
/* A caption covering several photographs sets them side by side, which
   is how the parish's own document has them. */
.parish-figure__frame.is-set {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(90px, 1fr));
  gap: 6px;
}

.parish-figure__img {
  display: block;
  width: 100%;
  height: auto;
  border-radius: 8px;
}

.parish-figure__caption {
  display: block;
  margin-top: 9px;
  font-size: 14px;
  line-height: 1.45;
  color: rgb(255 255 255 / 0.84);
  text-align: center;
}

/* The attribution, quieter than the caption but never dropped: these
   photographs belong to the souvenir programme and to the people who
   took them, and the parish credited each one. */
.parish-figure__source {
  display: block;
  margin-top: 3px;
  font-size: 12.5px;
  font-style: italic;
  color: rgb(255 255 255 / 0.6);
}

/* Says, once, that what follows is the parish's own Filipino text. An
   English reader meeting ten sections of Filipino with no explanation
   would reasonably think the app had failed to translate them. */
.parish-history__language-note {
  margin: 26px 0 4px;
  padding: 11px 13px;
  border-radius: 12px;
  background: rgb(255 255 255 / 0.1);
  border: 1px solid rgb(255 255 255 / 0.2);
  font-size: 14px;
  line-height: 1.5;
  color: rgb(255 255 255 / 0.82);
}

/* ── Test Sancti Knowledge ──────────────────────────────────────── */
   Five questions at the foot of the history, on the same dark parish
   band the prose sits on. Deliberately quieter than the history above
   it: this is a way of looking back at what was read, not the reason
   the page exists. */
.history-quiz {
  margin-top: 34px;
  padding-top: 24px;
  border-top: 1px solid rgb(255 255 255 / 0.18);
```

```
}

.history-quiz__title {
  margin: 0 0 6px;
  font-size: 15px;
  font-weight: 700;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  text-shadow: 0 2px 10px rgb(6 12 38 / 0.4);
}

.history-quiz__lede {
  margin: 0 0 18px;
  font-size: 15px;
  line-height: 1.55;
  color: rgb(255 255 255 / 0.72);
}

.history-quiz__q { margin-bottom: 20px; }
/* Named when Submit finds it blank, so the reader is not left hunting. */
.history-quiz__q.is-flagged {
  border-left: 3px solid #F2C14E;
  padding-left: 11px;
  margin-left: -14px;
}

.history-quiz__question {
  margin: 0 0 10px;
  font-size: 16px;
  line-height: 1.5;
  font-weight: 600;
  color: rgb(255 255 255 / 0.95);
}
.history-quiz__number { opacity: 0.6; }

.history-quiz__options { display: grid; gap: 7px; }

.history-quiz__option {
  display: flex;
  align-items: center;
  gap: 9px;
  width: 100%;
  padding: 11px 13px;
  border-radius: 13px;
  border: 1px solid rgb(255 255 255 / 0.22);
  background: rgb(255 255 255 / 0.07);
  color: rgb(255 255 255 / 0.92);
  font-size: 15px;
  line-height: 1.4;
  text-align: left;
  cursor: pointer;
  transition: background-color 140ms ease, border-color 140ms ease;
}
.history-quiz__option:hover:not(:disabled) { border-color: rgb(255 255 255 / 0.45); }
.history-quiz__option.is-picked {
  background: rgb(255 255 255 / 0.17);
  border-color: rgb(255 255 255 / 0.6);
}
/* After submission: the right answer is always marked, whether or not
```

```
   it was the one chosen. Showing only the reader's mistake teaches
   nothing. */
.history-quiz__option.is-answer {
  background: rgb(76 158 108 / 0.30);
  border-color: #7FD6A0;
}
.history-quiz__option.is-wrong {
  background: rgb(190 78 78 / 0.26);
  border-color: #E79A9A;
}
.history-quiz__option:disabled { cursor: default; }

.history-quiz__mark {
  display: inline-flex;
  width: 17px;
  flex-shrink: 0;
}

.history-quiz__explain {
  margin: 9px 0 0;
  padding-left: 11px;
  border-left: 2px solid rgb(255 255 255 / 0.3);
  font-size: 14.5px;
  line-height: 1.55;
  color: rgb(255 255 255 / 0.78);
}
.history-quiz__explain.is-right { border-left-color: #7FD6A0; }

.history-quiz__missing {
  margin: 0 0 10px;
  font-size: 14.5px;
  font-weight: 600;
  color: #F2C14E;
}

/* The way in. Outlined rather than solid: the white Submit button
   below is the page's one emphatic action, and two filled buttons in
   the same section would compete for it. */
.history-quiz__start {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 100%;
  padding: 13px 18px;
  border-radius: 999px;
  border: 1px solid rgb(255 255 255 / 0.45);
  background: rgb(255 255 255 / 0.12);
  color: #fff;
  font-size: 16px;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 140ms ease, border-color 140ms ease;
}
.history-quiz__start:hover {
  background: rgb(255 255 255 / 0.2);
  border-color: rgb(255 255 255 / 0.7);
}

.history-quiz__submit,
```

```
.history-quiz__again {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  width: 100%;
  padding: 13px 18px;
  border-radius: 999px;
  border: 0;
  font-size: 16px;
  font-weight: 700;
  cursor: pointer;
  background: #fff;
  color: var(--color-brand-primary);
}
.history-quiz__again {
  background: rgb(255 255 255 / 0.14);
  color: #fff;
  border: 1px solid rgb(255 255 255 / 0.4);
}

.history-quiz__result { margin-top: 18px; text-align: center; }
.history-quiz__score {
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-size: 30px;
  color: #fff;
}
.history-quiz__verdict {
  margin: 4px 0 14px;
  font-size: 15px;
  line-height: 1.5;
  color: rgb(255 255 255 / 0.76);
}

/* ── The Church History card on Home ──────────────────────────────
   The card's own type, so it matches the history page it opens: the same
   italic Georgia for the patron's name, and a "LEARN MORE" centred under
   the whole card with a short rule beneath it, exactly as the PSD has it. */
.church-history-card__title {
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 26px;
  line-height: 1.1;
  letter-spacing: -0.01em;
  text-wrap: balance;
  text-shadow: 0 2px 12px rgb(6 12 38 / 0.45);
}

/* Six lines, then it stops. The card carries the opening of the history
   rather than a summary written for it, so it has to end somewhere; ending
   mid-sentence under a "Learn more" is the design's own answer to that. */
.church-history-card__body {
  margin-top: 10px;
  font-size: 15px;
  line-height: 1.55;
  color: rgb(255 255 255 / 0.9);
```

```
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 6;
  line-clamp: 6;
  overflow: hidden;
}

.church-history-card__more {
  position: relative;
  display: block;
  width: 100%;
  padding: 4px 20px 18px;
  font-size: 14px;
  font-weight: 700;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  text-align: center;
  color: #fff;
  text-shadow: 0 2px 10px rgb(6 12 38 / 0.55);
}

/* The rule under the words - "Rectangle 8" in the PSD. A pseudo-element so
   it centres on the label itself whatever the label says. */
.church-history-card__more::after {
  content: "";
  display: block;
  width: 44px;
  height: 2px;
  margin: 7px auto 0;
  border-radius: 2px;
  background: rgb(255 255 255 / 0.85);
}

/* ── A parish's page, reached from the map ──────────────────────────
   The same construction as .parish-history, in the same per-parish livery,
   because in the PSD they are one design. What differs is proportion: this
   page is mostly information, so the photograph takes a third of the screen
   rather than half, and the panel carries facts instead of prose. */
.parish-hero {
  position: relative;
  flex-shrink: 0;
  background: var(--ph-bottom);
}

/* Sticky, and deliberately so. The old header was a sticky bar because it
   carried the only way off this page; put the photograph in a sticky frame
   and the two buttons inside it stay reachable at any scroll position while
   the panel and everything below it slide over the image. */
.parish-hero__frame {
  position: sticky;
  top: 0;
  height: 34vh;
  min-height: 190px;
  max-height: 300px;
  overflow: hidden;
}

.parish-hero__photo {
  width: 100%;
```

```
  height: 100%;
  object-fit: cover;
  object-position: var(--ph-face, center 30%);
  display: block;
}

.parish-hero__photo--empty {
  background: linear-gradient(180deg, var(--ph-top), var(--ph-bottom));
}

.parish-hero__control {
  position: absolute;
  top: calc(12px + env(safe-area-inset-top));
  width: 42px;
  height: 42px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  color: #fff;
  background: rgb(9 14 32 / 0.55);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
}
.parish-hero__control--back { left: 14px; }
.parish-hero__control--save { right: 14px; }

/* Relative, so it paints over the sticky frame behind it rather than under
   it. Without this the photograph wins the stacking order and the panel
   disappears the moment the page scrolls. */
.parish-hero__panel {
  position: relative;
  z-index: 1;
  margin-top: -30px;
  border-radius: 30px 30px 0 0;
  background: linear-gradient(180deg, var(--ph-top) 0%, var(--ph-bottom) 78%);
  overflow: hidden;
  isolation: isolate;
}

.parish-hero__watermark {
  position: absolute;
  top: 0;
  right: -20%;
  width: 88%;
  height: 100%;
  object-fit: cover;
  object-position: center 16%;
  opacity: 0.15;
  mix-blend-mode: soft-light;
  pointer-events: none;
  z-index: -1;
}

.parish-hero__body {
  padding: 24px 20px 26px;
  color: #fff;
}
```

```
/* The same italic Georgia as the history page's title and the home header's
   parish name. It is the title of the page, not a label. */
.parish-hero__name {
  margin: 8px 0 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 31px;
  line-height: 1.08;
  letter-spacing: -0.01em;
  text-wrap: balance;
  text-shadow: 0 2px 14px rgb(6 12 38 / 0.45);
}

/* ── Planning a Bisita Iglesia ──────────────────────────────────────
   Seven churches in one evening is the tradition, and the part a map can
   help with is the order. These are the pieces that let a visit be built on
   the map without leaving it: a plus on the search bar, a second action on
   each place card, lettered markers, and the plan itself. */

/* The plus that starts a visit, inside the search pill at its right end.
   It shows the number of churches once there are any, so the plan can be
   found again after it has been closed. */
.dmap-search__add {
  flex: 0 0 auto;
  width: 28px;
  height: 28px;
  margin-left: 6px;
  margin-right: -4px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 700;
  font-variant-numeric: tabular-nums;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

/* The second action on a place card. Outlined rather than filled, because
   Directions is the primary errand and two solid buttons side by side give
   no clue which one the card is for. */
.map-sheet__add {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 11px 12px;
  border-radius: 999px;
  font-size: 16px;
  font-weight: 600;
  background: transparent;
  color: var(--color-brand-primary);
  border: 1px solid var(--color-brand-border);
}
.map-sheet__add--on {
  background: color-mix(in srgb, var(--color-brand-primary) 10%, transparent);
```

```
  border-color: var(--color-brand-primary);
}

/* A, B, C on the map itself. Big enough to read at a glance over a street
   map, which is the whole point of lettering the stops. */
.dmap-live__trip-stop {
  width: 28px;
  height: 28px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  font-size: 15px;
  font-weight: 700;
  line-height: 1;
  border: 2px solid #fff;
  box-shadow: 0 2px 8px rgb(0 0 0 / 0.3);
  cursor: default;
}

/* The plan. A sheet over the bottom of the map rather than a separate
   screen: the churches being chosen are on the map behind it, and the line
   through them redraws as the list changes, which is most of the reason to
   look at it. */
.trip {
  position: absolute;
  left: 10px;
  right: 10px;
  bottom: 10px;
  z-index: 30;
  max-height: calc(100% - 78px);
  display: flex;
  flex-direction: column;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  border-radius: 20px;
  box-shadow: 0 12px 32px rgb(0 0 0 / 0.18);
  overflow: hidden;
  animation: map-sheet-in 180ms ease-out;
}

.trip__head {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 12px 8px;
}

.trip__icon {
  width: 36px;
  height: 36px;
  flex: 0 0 auto;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  color: var(--color-brand-secondary);
```

```
}

.trip__heading { flex: 1; min-width: 0; }

.trip__title {
  margin: 0;
  font-size: 17px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.trip__sub {
  margin: 1px 0 0;
  font-size: 14px;
  color: var(--color-brand-secondary);
}

/* Scrolls, because fourteen churches will not fit over a map and the
   footer carrying the total must stay visible while they are scrolled. */
.trip__stops {
  flex: 1 1 auto;
  min-height: 0;
  overflow-y: auto;
  margin: 0;
  padding: 0 12px;
  list-style: none;
}

.trip__stop {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 7px 0;
}

/* The connecting line down the left, so the rows read as one journey
   rather than as a list. Drawn on the row rather than between rows: a
   border between list items cannot reach the letter's centre. */
.trip__stop::before {
  content: "";
  position: absolute;
  width: 0;
}

.trip__letter {
  flex: 0 0 auto;
  width: 26px;
  height: 26px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  font-size: 14px;
  font-weight: 700;
}

.trip__letter--ghost {
```

```
  background: transparent;
  color: var(--color-brand-secondary);
  border: 1px dashed var(--color-brand-border);
}

/* Where the pilgrim is standing. A dot, not a letter: it is not a church
   and it cannot be removed. */
.trip__pip {
  flex: 0 0 auto;
  width: 26px;
  height: 26px;
  display: block;
  border-radius: 999px;
  background:
    radial-gradient(circle at center,
      var(--color-brand-primary) 0 4px,
      transparent 5px),
    color-mix(in srgb, var(--color-brand-primary) 12%, transparent);
}

.trip__name {
  flex: 1;
  min-width: 0;
  font-size: 16px;
  color: var(--color-brand-text);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.trip__stop--start .trip__name { color: var(--color-brand-secondary); }

.trip__remove {
  flex: 0 0 auto;
  width: 30px;
  height: 30px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  color: var(--color-brand-secondary);
}

.trip__add {
  flex: 1;
  text-align: left;
  font-size: 16px;
  font-weight: 600;
  color: var(--color-brand-primary);
}
.trip__add:disabled {
  color: var(--color-brand-secondary);
  font-weight: 400;
}

.trip__optimise {
  display: flex;
  align-items: center;
  justify-content: center;
```

```
  gap: 8px;
  margin: 4px 12px 0;
  padding: 10px 12px;
  border-radius: 999px;
  font-size: 15px;
  font-weight: 600;
  color: var(--color-brand-primary);
  background: color-mix(in srgb, var(--color-brand-primary) 10%, transparent);
}

.trip__foot {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 10px;
  padding: 10px 12px calc(10px + env(safe-area-inset-bottom, 0px));
  background: color-mix(in srgb, var(--color-brand-primary) 8%, transparent);
}

.trip__total {
  flex: 1;
  min-width: 0;
  display: flex;
  flex-direction: column;
  gap: 1px;
}

.trip__time {
  font-size: 16px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.trip__distance,
.trip__measuring {
  font-size: 14px;
  color: var(--color-brand-secondary);
}

.trip__done {
  flex: 0 0 auto;
  padding: 10px 20px;
  border-radius: 999px;
  font-size: 16px;
  font-weight: 700;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

/* ── Ministries ──────────────────────────────────────────────────
   The list was a stack of text rows that grew downwards as you opened them,
   which reads as a form to fill in rather than as people to join. It is the
   same photo card the bulletin rail uses on Home, and opening one gives it
   the whole screen the way a parish history page does. */
.ministry-card {
  display: block;
  width: 100%;
  text-align: left;
  border-radius: 22px;
```

```
  overflow: hidden;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  box-shadow: 0 1px 2px rgb(16 24 40 / 0.05);
}

.ministry-card__media {
  position: relative;
  display: block;
  height: 116px;
}
.ministry-card__media img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

/* A scrim rather than a shadow: what sits behind the caption is a
   photograph nobody has chosen yet, so its brightness cannot be assumed. */
.ministry-card__scrim {
  position: absolute;
  inset-inline: 0;
  bottom: 0;
  height: 58px;
  background: linear-gradient(to top, rgb(0 0 0 / 0.72), transparent);
}

.ministry-card__caption {
  position: absolute;
  left: 12px;
  right: 12px;
  bottom: 8px;
  display: block;
  font-size: 14px;
  font-weight: 600;
  line-height: 1.25;
  color: #fff;
}

.ministry-card__foot {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 14px 16px;
}

.ministry-card__hint {
  flex: 1;
  min-width: 0;
  font-size: 15px;
  line-height: 1.35;
  color: var(--color-brand-secondary);
  /* Two lines, then it stops. These descriptions run to a full sentence and
     the card is a summary, not the page. */
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 2;
  line-clamp: 2;
```

```
  overflow: hidden;
}

/* The opened ministry's header: the photograph, with its name over it. */
.ministry-hero {
  position: relative;
  height: 210px;
}
.ministry-hero img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}
.ministry-hero__scrim {
  position: absolute;
  inset: 0;
  background: linear-gradient(to top, rgb(9 14 32 / 0.86) 8%, rgb(9 14 32 / 0.15) 60%, rgb(9 14 32 / 0.35));
}

.ministry-hero__name {
  position: absolute;
  left: 18px;
  right: 18px;
  bottom: 14px;
  margin: 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 23px;
  line-height: 1.12;
  color: #fff;
  text-wrap: balance;
  text-shadow: 0 2px 14px rgb(6 12 38 / 0.5);
}

.ministry-hero__back {
  position: absolute;
  top: 14px;
  left: 14px;
  width: 42px;
  height: 42px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 999px;
  color: #fff;
  background: rgb(9 14 32 / 0.55);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
}

/* ── The header on Ministries, Sacraments and pages like them ──────────
   The welcome band from Home, reused. It was a flat navy panel with a large
   faded outline icon in the corner - the same panel for every parish, in
   Mary Help's blue, so San Roque's ministries were introduced in another
   parish's colours with a piece of clip-art where the rest of the app uses
   the parish's own photography. */
.parish-section {
```

```
  position: relative;
  overflow: hidden;
  isolation: isolate;
  flex-shrink: 0;
  border-radius: 0 0 2rem 2rem;
  background: var(--parish-band, #101B4A);
  padding: 22px 20px 26px;
  color: #fff;
}

/* ---- the collapsing variant, as Home's welcome band does it -----------
   Same trick: the band keeps its full layout height and is sticky with a
   NEGATIVE top, so it travels up by exactly the amount that should
   disappear and then pins. Nothing changes layout, so the page never
   changes length and the scroll position is never clamped - shrinking the
   band's own height fights itself on a barely-scrollable page.

   One custom property, --ps-t, runs 0 to 1 and everything derives from it.
   If the script never runs it stays 0 and the band simply scrolls away. */
.parish-section--collapsing {
  --ps-t: 0;
  --ps-collapse: 0px;
  --ps-title-dy: 0px;
  --ps-scale-delta: 0;
  position: sticky;
  top: calc(-1 * var(--ps-collapse, 0px));
  z-index: 20;
}

/* The eyebrow and the blurb are the part that goes: they explain the tab,
   and once you are reading the tab you no longer need explaining to. */
.parish-section--collapsing .parish-section__eyebrow,
.parish-section--collapsing .parish-section__blurb {
  opacity: calc(1 - var(--ps-t) * 1.6);
  transform: translateY(calc(var(--ps-t) * -10px));
}

/* The title stays, and shrinks. It travels down inside the band by as much
   as the band travels up, which is what holds it on screen. */
.parish-section--collapsing .parish-section__title {
  transform-origin: left center;
  transform:
    translateY(calc(var(--ps-t) * var(--ps-title-dy)))
    scale(calc(1 - var(--ps-t) * var(--ps-scale-delta)));
}

/* The photograph travels down inside the band as the band travels up, by
   whatever amount puts the patron's face in the collapsed strip. The
   distance is measured in the component, because it depends on the band's
   height and on where the face sits in that particular frame. */
.parish-section--collapsing {
  --ps-photo-dy: 0px;
}
.parish-section--collapsing .parish-section__photo {
  transform: translateY(calc(var(--ps-t) * var(--ps-photo-dy)));
}

/* From the right, behind the words, as the design has it on Home. */
.parish-section__photo {
```

```
  position: absolute;
  inset: 0 0 0 28%;
  z-index: -2;
  background-size: cover;
  background-position: center var(--psh-face, 30%);
}

.parish-section__scrim {
  position: absolute;
  inset: 0;
  z-index: -1;
  background:
    linear-gradient(
      90deg,
      var(--parish-band, #101B4A) 34%,
      color-mix(in srgb, var(--parish-band, #101B4A) 74%, transparent) 54%,
      color-mix(in srgb, var(--parish-band, #101B4A) 10%, transparent) 80%
    ),
    linear-gradient(
      180deg,
      color-mix(in srgb, var(--parish-band, #101B4A) 45%, transparent),
      color-mix(in srgb, var(--parish-band, #101B4A) 10%, transparent) 40%,
      color-mix(in srgb, var(--parish-band, #101B4A) 45%, transparent)
    );
}

.parish-section__content { position: relative; max-width: 74%; }

.parish-section__eyebrow {
  display: flex;
  align-items: center;
  gap: 7px;
  margin: 0;
  font-family: var(--font-mono-label), ui-monospace, monospace;
  font-size: 13px;
  font-weight: 600;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: rgb(255 255 255 / 0.88);
  text-shadow: 0 1px 8px rgb(6 12 38 / 0.5);
}

/* The one italic serif the design uses for a parish's name - the same face
   as .parish-welcome__parish on Home, which is what "the dashboard's fonts"
   means here. It was Montserrat italic, which is not a serif at all: the
   --font-serif token is mapped to Montserrat, so `font-serif italic` was
   quietly giving a slanted sans. */
.parish-section__title {
  margin: 10px 0 0;
  font-family: Georgia, "Times New Roman", serif;
  font-style: italic;
  font-weight: 700;
  font-size: 27px;
  line-height: 1.12;
  letter-spacing: -0.01em;
  text-wrap: balance;
  text-shadow: 0 2px 12px rgb(6 12 38 / 0.5);
}
```

```
.parish-section__blurb {
  margin: 8px 0 0;
  font-size: 15px;
  line-height: 1.5;
  color: rgb(255 255 255 / 0.9);
  text-shadow: 0 1px 8px rgb(6 12 38 / 0.45);
}

/* ── The bottom tab bar ──────────────────────────────────────────
   The selected tab looked exactly like the unselected ones: both branches of
   the conditional set --color-brand-secondary, so the bar never said where
   you were. */
.tab-item {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 2px;
  flex: 1;
  min-width: 0;
  height: 48px;
  border-radius: 14px;
  color: var(--color-brand-secondary);
  transition: color 140ms ease, background-color 140ms ease;
}

.tab-item span {
  font-size: 16px;
  font-weight: 700;
  line-height: 1;
}

/* The icon fills solid; nothing sits behind the tab.
   The pill that used to be here read as a button someone had left pressed,
   and it boxed in one tab of five. A solid icon beside four outlined ones
   is the stronger signal anyway - it changes the SHAPE, not just the
   colour, so it still reads at arm's length in a bright church and for
   anyone who cannot separate the two greys. */
.tab-item--on {
  color: var(--color-brand-primary);
}

/* Lucide icons are drawn as strokes with no fill, so filling them turns the
   outline into a solid silhouette. The stroke stays the same colour as the
   fill, otherwise the shape gains a visible border it does not have when
   idle and appears to grow on selection. */
.tab-item--on svg {
  fill: var(--color-brand-primary);
  stroke: var(--color-brand-primary);
}

/* ── Searching the ministries ────────────────────────────────────
   In the flow, above the list. An overlay would sit on top of the very
   cards it filters, which are the thing it exists to help you read. */
.ministry-search {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 14px;
```

```
  border-radius: 999px;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
}

.ministry-search input {
  flex: 1;
  min-width: 0;
  border: 0;
  background: none;
  outline: none;
  font: inherit;
  /* 16px exactly, or iOS zooms the whole page when the field is focused. */
  font-size: 16px;
  color: var(--color-brand-text);
}
.ministry-search input::placeholder { color: var(--color-brand-secondary); }

/* The round X a search input draws for itself. Ours is a real button, so
   the browser's would be a second one beside it. */
.ministry-search input::-webkit-search-cancel-button { display: none; }

.ministry-chips {
  display: flex;
  flex-wrap: wrap;
  gap: 7px;
  padding: 0 2px;
}

.ministry-chips button {
  padding: 7px 13px;
  border-radius: 999px;
  font-size: 14px;
  font-weight: 600;
  color: var(--color-brand-primary);
  background: color-mix(in srgb, var(--color-brand-primary) 10%, transparent);
  border: 1px solid color-mix(in srgb, var(--color-brand-primary) 20%, transparent);
}


/* ---------------------------------------------------------------- Scan
   The two ways into the AR experience, offered as a choice rather than as
   one button and a link underneath it. They are the same card as a
   ministry - 22px, paper, hairline border - so the tab reads as part of
   the app rather than as a feature bolted onto it. The photograph is the
   parish's own, which is what makes the choice feel like this church's
   rather than the software's. */
.scan-card {
  display: block;
  width: 100%;
  text-align: left;
  border-radius: 22px;
  overflow: hidden;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  box-shadow: 0 1px 2px rgb(16 24 40 / 0.05);
}

.scan-card__media { position: relative; display: block; height: 132px; }
```

```
.scan-card__media img { width: 100%; height: 100%; object-fit: cover; display: block; }

/* A scrim, not a shadow: these are gilded statues under warm light, so the
   brightness behind the label cannot be assumed. */
.scan-card__scrim {
  position: absolute; inset: 0;
  background: linear-gradient(to top, rgb(12 16 28 / 0.78) 0%, rgb(12 16 28 / 0.22) 52%, transparent 100%);
}

.scan-card__title {
  position: absolute; left: 16px; right: 16px; bottom: 11px;
  color: #FAF8F4;
  font-family: var(--font-serif, Georgia, serif);
  font-style: italic; font-weight: 700;
  font-size: 21px; line-height: 1.15;
  text-shadow: 0 1px 10px rgb(0 0 0 / 0.45);
}

.scan-card__body { padding: 14px 16px 16px; }

.scan-card__blurb {
  margin: 0;
  font-size: 15px; line-height: 1.5;
  color: var(--color-brand-text);
}

.scan-card__note {
  margin: 8px 0 0;
  font-size: 13px; line-height: 1.45;
  color: var(--color-brand-secondary);
}
.scan-card__note--error { color: var(--color-brand-error); font-weight: 600; }

.scan-card__action {
  margin-top: 13px;
  width: 100%;
  display: inline-flex; align-items: center; justify-content: center; gap: 8px;
  padding: 12px 16px;
  border: 0; border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  font-family: inherit; font-size: 15px; font-weight: 700;
  cursor: pointer;
  transition: transform 120ms ease, filter 120ms ease;
}
.scan-card__action:active { transform: scale(0.985); }
.scan-card__action:disabled { opacity: 0.55; cursor: default; }

/* The quieter of the two. Both are real choices, but the scanner is the
   one the tab is named after, so the museum does not compete with it. */
.scan-card__action--ghost {
  background: transparent;
  color: var(--color-brand-primary);
  border: 1.5px solid var(--color-brand-primary);
}


/* ------------------------------------------------- photo parish pins
   The two parishes with a patron photograph wear it on the map, in the
```

```
   app's navy ring over a small stem - a round portrait reads as "this
   particular church" where the glyph can only say "a church".

   Sized for a thumb, not for the picture: 44px is the tap target, and at
   that size the face is all that survives, which is why the crop is top
   -weighted rather than centred. */
.dmap__pin--photo {
  display: block;
  padding: 0; border: 0; background: none;
  line-height: 0;
  cursor: pointer;
  /* Anchored at centre by MapLibre, so the stem is pushed below the disc
     and the whole thing lifted by half the stem - which puts the stem's
     point on the coordinate rather than the middle of the portrait. */
  transform: translateY(-11px);
}

.dmap__pin-disc {
  display: block;
  width: 44px; height: 44px;
  border-radius: 50%;
  overflow: hidden;
  border: 3px solid var(--color-brand-primary);
  background: var(--color-brand-card);
  box-shadow: 0 2px 8px rgb(16 24 40 / 0.35);
}
.dmap__pin-disc img {
  width: 100%; height: 100%;
  object-fit: cover;
  object-position: 50% 22%;
  display: block;
}

/* The point. A triangle rather than a teardrop: at 10px a teardrop's
   curve is two pixels nobody can see, and the triangle keeps a crisp tip
   on the coordinate. */
.dmap__pin-stem {
  display: block;
  width: 0; height: 0;
  margin: -2px auto 0;
  border-left: 7px solid transparent;
  border-right: 7px solid transparent;
  border-top: 11px solid var(--color-brand-primary);
  filter: drop-shadow(0 2px 2px rgb(16 24 40 / 0.3));
}

.dmap__pin--photo:focus-visible .dmap__pin-disc {
  outline: 2px solid var(--color-brand-accent);
  outline-offset: 2px;
}

/* ------------------------------------------------- Verse of the Day card
   The verse in the colour of the liturgical season, flat.

   Every colour here is a custom property set by VerseCard from
   liturgicalColours.ts, which guarantees the ink clears WCAG AA on its own
   wash. Nothing in this block hardcodes green: the green belongs to
   Ordinary Time, and the same rules carry violet, gold, red and rose. */
.verse-card {
```

```
  position: relative;
  overflow: hidden;
  border-radius: 22px;
  min-height: 160px;
  display: flex;
  align-items: stretch;
  background: var(--verse-tint, var(--color-brand-card-sunk));
  border: 1px solid var(--verse-border, var(--color-brand-border));
  /* The season named twice over - once in the chip, once as a firmer rule
     along the top. It is what makes the card read as belonging to the
     calendar rather than being a tinted box. */
  border-top: 3px solid var(--verse-ink, var(--color-brand-primary));
}

.verse-card__body {
  position: relative;
  padding: 18px 18px 16px;
  width: 100%;
}

.verse-card__eyebrow {
  margin: 0 0 8px;
  font-family: var(--font-mono-label, ui-monospace, monospace);
  font-size: 13px;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--verse-ink, var(--color-brand-secondary));
  /* Lighter than the verse itself: it is a label, and at full strength it
     competed with the scripture underneath it. */
  opacity: 0.78;
}

.verse-card__text {
  margin: 0;
  font-size: 17px;
  line-height: 1.52;
  color: var(--color-brand-text);
}

.verse-card__foot {
  margin-top: 12px;
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
}

.verse-card__ref {
  font-size: 15px;
  font-weight: 700;
  font-style: normal;
  color: var(--verse-ink, var(--color-brand-text));
}

/* The season as a chip rather than loose text, as it was over the
   photograph. On the wash it is the ink on a slightly firmer version of
   the same hue, so it reads as part of the card and not as a button. */
.verse-card__season {
  flex-shrink: 0;
```

```
  font-size: 13px;
  font-weight: 700;
  padding: 4px 10px;
  border-radius: 999px;
  color: var(--verse-ink, var(--color-brand-text));
  background: color-mix(in srgb, var(--verse-ink, #000) 10%, transparent);
  border: 1px solid var(--verse-border, var(--color-brand-border));
}

@media (max-width: 380px) {
  .verse-card { min-height: 148px; }
  .verse-card__text { font-size: 16px; }
}

/* The verse at the top of the Parish Bulletin: full width of the sheet,
   and no longer one slide of a rail. */
.home-verse .verse-card {
  /* Plain 100%, not cqw. `cqw` resolves against the nearest container-query
     ancestor, and out here that is a container the width of the page - the
     card escaped the phone by 500px the first time this was tried. */
  width: 100%;
  /* No photograph to give room to any more, so the floor is only what
     stops a short verse becoming a letterbox. */
  min-height: 150px;
}

.home-verse .verse-card__body {
  padding: 16px 17px 15px;
}

.home-verse .verse-card__text {
  font-size: 16.5px;
  line-height: 1.5;
  /* A clamp is still needed: the longest verses in the set run past five
     lines, and the attribution must not be pushed off the card. */
  display: -webkit-box;
  -webkit-line-clamp: 5;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.home-verse .verse-card__foot {
  margin-top: 12px;
  flex-wrap: wrap;
  row-gap: 6px;
}

.home-verse .verse-card__ref { font-size: 15px; }
.home-verse .verse-card__season { font-size: 12.5px; padding: 4px 10px; }
.home-verse .verse-card__eyebrow { font-size: 12.5px; margin-bottom: 8px; }


/* The verse, standing alone above the bulletin rail. Full width of the
   sheet, with the rail's own left/right rhythm. */
.home-verse {
  margin-bottom: 14px;
}

/* The way back to your own parish, in the slot the brand lockup uses
```

```
   when you are already there. Pill-shaped and translucent so it reads as
   a control against a photograph, whatever that photograph is. */
.parish-welcome__back {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 6px 12px 6px 9px;
  border-radius: 999px;
  border: 1px solid rgb(255 255 255 / 0.35);
  background: rgb(255 255 255 / 0.16);
  backdrop-filter: blur(3px);
  color: #fff;
  font-size: 14px;
  font-weight: 700;
  line-height: 1;
  white-space: nowrap;
}

/* ------------------------------------------------- Parish Bulletin
   Below the Verse of the Day: the announcements, then the two section
   cards. Nothing here is a horizontal scroller — see the note at the top
   of BulletinRail.tsx for why both halves stopped being one. */
.bulletin {
  display: flex;
  flex-direction: column;
  gap: 14px;
}

.bulletin-note {
  border-radius: 22px;
  background: var(--color-brand-card-sunk);
  border: 1px solid var(--color-brand-border);
  padding: 16px;
  font-size: 15px;
  line-height: 1.55;
  color: var(--color-brand-secondary);
}

/* Every slide occupies the same grid cell, so the frame is as tall as the
   tallest announcement and the page below it never jumps when the deck
   advances. This is also what makes a crossfade possible at all: two
   siblings in a row cannot fade through each other. */
.bulletin-deck {
  display: grid;
  /* Vertical panning still belongs to the page; the deck only wants the
     horizontal flick, and it handles that itself in JS. */
  touch-action: pan-y;
}

.bulletin-card {
  grid-area: 1 / 1;
  overflow: hidden;
  border-radius: 22px;
  background: var(--color-brand-card-sunk);
  border: 1px solid var(--color-brand-border);
  display: flex;
  flex-direction: column;

  opacity: 0;
```

```
  /* Inert, not merely invisible: without this the hidden slides still take
     tab stops and still answer hit-testing over the visible one. */
  visibility: hidden;
  pointer-events: none;
  transition: opacity 320ms ease, visibility 0s linear 320ms;
}

.bulletin-card.is-current {
  opacity: 1;
  visibility: visible;
  pointer-events: auto;
  transition: opacity 320ms ease, visibility 0s;
}

@media (prefers-reduced-motion: reduce) {
  .bulletin-card { transition-duration: 1ms; }
}

.bulletin-card__photo {
  display: block;
  position: relative;
  /* A fixed ratio rather than a fixed height: the admin's uploads are all
     cropped to 16:9 before they are stored, so this is the shape they
     actually are. */
  aspect-ratio: 16 / 9;
  background: var(--color-brand-border);
}

.bulletin-card__photo img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

.bulletin-card__body {
  display: block;
  padding: 14px 16px 16px;
}

.bulletin-card__chip {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  font-size: 14px;
  font-weight: 600;
  padding: 4px 10px;
  border-radius: 6px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

.bulletin-card__title {
  margin: 10px 0 0;
  font-size: 17px;
  font-weight: 700;
  line-height: 1.3;
  color: var(--color-brand-text);
}
```

```
.bulletin-card__when {
  margin: 4px 0 0;
  font-size: 15px;
  color: var(--color-brand-secondary);
}

.bulletin-card__body-text {
  margin: 8px 0 0;
  font-size: 15px;
  line-height: 1.5;
  color: var(--color-brand-text);
  /* Three lines on the card, the rest in the parish's own announcement
     page. A deck whose height is set by its longest slide should not be
     set by an essay. */
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.bulletin-deck__nav {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 14px;
  margin-top: 10px;
}

.bulletin-deck__arrow {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 30px;
  height: 30px;
  border-radius: 999px;
  border: 1px solid var(--color-brand-border);
  background: var(--color-brand-card);
  color: var(--color-brand-secondary);
}

.bulletin-deck__dots {
  display: inline-flex;
  align-items: center;
  gap: 7px;
}

/* The hit area is 18px even though the dot reads as 7px — a 7px tap
   target is not one. */
.bulletin-deck__dot {
  width: 18px;
  height: 18px;
  display: inline-grid;
  place-items: center;
  background: none;
  border: 0;
}

.bulletin-deck__dot::after {
```

```
  content: "";
  width: 7px;
  height: 7px;
  border-radius: 999px;
  background: var(--color-brand-border);
  transition: background-color 200ms ease, width 200ms ease;
}

.bulletin-deck__dot.is-current::after {
  width: 18px;
  background: var(--color-brand-primary);
}

/* The two section cards. Equal columns at every width a phone has: they
   fit, which is the whole reason they stopped being a rail. */
.bulletin-pair {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  align-items: stretch;
}

.bulletin-section {
  display: flex;
  flex-direction: column;
  overflow: hidden;
  text-align: left;
  border-radius: 22px;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  transition: border-color 160ms ease;
}

.bulletin-section:hover,
.bulletin-section:focus-visible {
  border-color: var(--color-brand-primary);
}

.bulletin-section__photo {
  display: block;
  position: relative;
  aspect-ratio: 4 / 3;
}

.bulletin-section__photo img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  display: block;
}

/* The stand-in for a parish whose photographs have not been collected.
   The mark on the parish tint, centred and small, rather than a photo
   cropped to fill - it should read as a placeholder, not as a picture
   of something. */
.bulletin-section__photo--mark {
  display: grid;
  place-items: center;
  background: color-mix(in srgb, var(--parish-ink, #1B2A6B) 6%, var(--parish-tint, #EEF3FC));
```

```
}
.bulletin-section__photo--mark img {
  width: 44%;
  height: auto;
  object-fit: contain;
  opacity: 0.5;
}

.bulletin-section__scrim {
  position: absolute;
  inset: auto 0 0 0;
  height: 56px;
  background: linear-gradient(to top, rgb(0 0 0 / 0.72), transparent);
}

.bulletin-section__caption {
  position: absolute;
  left: 10px;
  right: 10px;
  bottom: 7px;
  font-size: 13px;
  font-weight: 600;
  line-height: 1.25;
  color: #fff;
  /* Two lines at this width; a third pushed the ministry's own name off
     the bottom of the photograph. */
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.bulletin-section__body {
  display: block;
  padding: 12px 13px 13px;
  /* Pushes both cards' bodies to the same depth whatever their hint runs
     to, so the pair reads as a pair. */
  flex: 1;
}

.bulletin-section__label {
  display: flex;
  align-items: center;
  gap: 7px;
  font-size: 15.5px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.bulletin-section__icon {
  color: var(--color-brand-primary);
  display: inline-flex;
}

.bulletin-section__hint {
  display: block;
  margin-top: 3px;
  font-size: 14px;
  line-height: 1.35;
```

```
  color: var(--color-brand-secondary);
}

/* Narrow phones: the pair still fits, but the label needs the room more
   than the chevron does. */
@media (max-width: 360px) {
  .bulletin-section__label { font-size: 14.5px; gap: 5px; }
  .bulletin-section__hint { font-size: 13px; }
}

/* An announcement with no photograph, in a deck whose height is set by one
   that has. Left at the top it floated in a tall empty box; centred, the
   short slide reads as a card of its own rather than as a card that failed
   to load its picture. */
.bulletin-card--text {
  justify-content: center;
}

/* ------------------------------------------------- Availability badge
   Whether a ministry or sacrament is taking applications. Grey, not red,
   when closed: a parish shutting the choir for Lent is housekeeping, and
   red is the colour this app uses when something has gone wrong for the
   person reading it. */
.availability-badge {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  font-size: 13px;
  font-weight: 700;
  padding: 3px 10px 3px 8px;
  border-radius: 999px;
  white-space: nowrap;
  color: #2F5A41;
  background: #E4F0E8;
  border: 1px solid #BFD6C8;
}

.availability-badge__dot {
  width: 7px;
  height: 7px;
  border-radius: 999px;
  background: currentColor;
  flex-shrink: 0;
}

.availability-badge--closed {
  color: var(--color-brand-secondary);
  background: var(--color-brand-card-sunk);
  border-color: var(--color-brand-border);
}

/* A card whose applications are closed. The content stays fully legible —
   only the photograph is pulled back, because that is the part that was
   selling it. Greying the text would make the requirements, which are
   still the useful part, the hardest thing on the card to read. */
.is-unavailable .ministry-card__photo img,
.is-unavailable .ministry-hero img,
.is-unavailable .sacrament-card__photo img {
  filter: grayscale(1);
```

```
  opacity: 0.72;
}

/* A real <input type="checkbox"> painted as a switch. Built on the native
   control rather than a div with an onClick, so it keeps the keyboard, the
   focus ring and the state a screen reader reads. */
.availability-switch {
  appearance: none;
  -webkit-appearance: none;
  flex-shrink: 0;
  width: 50px;
  height: 30px;
  border-radius: 999px;
  background: var(--color-brand-border);
  border: 1px solid var(--color-brand-border);
  position: relative;
  cursor: pointer;
  transition: background-color 160ms ease;
}

.availability-switch::after {
  content: "";
  position: absolute;
  top: 3px;
  left: 3px;
  width: 22px;
  height: 22px;
  border-radius: 999px;
  background: #fff;
  box-shadow: 0 1px 3px rgb(0 0 0 / 0.25);
  transition: transform 160ms ease;
}

.availability-switch:checked {
  background: var(--color-brand-success, #2F5A41);
  border-color: var(--color-brand-success, #2F5A41);
}

.availability-switch:checked::after {
  transform: translateX(20px);
}

.availability-switch:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 2px;
}

.availability-switch:disabled {
  opacity: 0.55;
  cursor: wait;
}

/* ------------------------------------------------ Parish office nav
   Scrolls sideways on a phone rather than wrapping to three ragged rows.
   The parish office is used on a desktop most of the time, but a priest
   checking applications on a phone should not meet a broken header. */
/* Wraps, rather than scrolling sideways with the scrollbar hidden.
 *
 * Ten chips come to about 1390px. In the office's 900px column that put
```

```
 * four of them past the right edge - Events, Visitor activity, What
 * people read and Archive - and `scrollbar-width: none` removed the one
 * clue that there was anything to scroll to. The parish could not reach
 * its own content editor or its archive, and nobody could see why.
 *
 * Wrapping costs a second row on a wide screen and three on a phone,
 * which is the correct price for every section of the office being
 * visible at once. */
.admin-nav {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  padding-bottom: 2px;
  margin-bottom: 18px;
}

.admin-nav__item {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  flex-shrink: 0;
  padding: 9px 15px;
  border-radius: 999px;
  font-size: 15px;
  font-weight: 700;
  white-space: nowrap;
  color: var(--color-brand-secondary);
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  transition: color 140ms ease, background-color 140ms ease, border-color 140ms ease;
}

.admin-nav__item:hover {
  color: var(--color-brand-text);
  border-color: var(--color-brand-primary);
}

.admin-nav__item.is-on {
  color: var(--color-brand-on-accent);
  background: var(--color-brand-primary);
  border-color: var(--color-brand-primary);
}

/* ------------------------------------------------- Application screens
   The form takes the whole screen, because applying is its own job and
   not a footnote to the description you were reading. */
.apply-screen {
  flex: 1;
  display: flex;
  flex-direction: column;
  min-height: 0;
  background: var(--color-brand-bg);
}

.apply-screen__bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
```

```
  padding: 14px 16px 10px;
}

.apply-screen__back {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  font-size: 15px;
  font-weight: 700;
  color: var(--color-brand-primary);
}

.apply-screen__count {
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}

.apply-screen__body {
  flex: 1;
  min-height: 0;
  overflow-y: auto;
  padding: 4px 18px 32px;
}

.apply-screen__eyebrow {
  font-size: 12.5px;
  font-weight: 700;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}

.apply-screen__title {
  margin-top: 2px;
  font-size: 23px;
  font-weight: 700;
  font-style: italic;
  font-family: var(--font-serif, Georgia, serif);
  line-height: 1.22;
  color: var(--color-brand-text);
}

.apply-screen__parish {
  font-size: 15px;
  color: var(--color-brand-secondary);
}

/* How far through. Four steps is short, but not knowing whether it is
   four or fourteen is what makes people abandon a form on screen one. */
.apply-progress {
  display: flex;
  gap: 5px;
  margin: 14px 0 20px;
}

.apply-progress__seg {
```

```
  flex: 1;
  height: 4px;
  border-radius: 999px;
  background: var(--color-brand-border);
  transition: background-color 200ms ease;
}

.apply-progress__seg.is-done { background: var(--color-brand-primary); }

.apply-step__title {
  font-size: 19px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.apply-step__blurb {
  margin-top: 3px;
  font-size: 15px;
  line-height: 1.5;
  color: var(--color-brand-secondary);
}

.apply-label {
  display: block;
  margin-bottom: 4px;
  font-size: 15px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.apply-optional {
  font-weight: 500;
  color: var(--color-brand-secondary);
}

.apply-help {
  margin: -2px 0 6px;
  font-size: 14px;
  line-height: 1.45;
  color: var(--color-brand-secondary);
}

/* 16px minimum: anything smaller makes iOS Safari zoom the page the
   moment the field is focused, and it never zooms back out. */
.apply-input {
  width: 100%;
  border-radius: 14px;
  border: 1px solid var(--color-brand-border);
  background: var(--color-brand-card);
  padding: 12px 13px;
  font-size: 16px;
  line-height: 1.45;
  color: var(--color-brand-text);
}

.apply-input:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 1px;
}
```

```
.apply-input.is-bad { border-color: var(--color-brand-error); }

.apply-chip {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 9px 14px;
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-border);
  background: var(--color-brand-card);
  font-size: 15px;
  font-weight: 600;
  color: var(--color-brand-text);
}

.apply-chip.is-on {
  border-color: var(--color-brand-primary);
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

.apply-primary {
  width: 100%;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  border-radius: 999px;
  padding: 15px;
  font-size: 16px;
  font-weight: 700;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

.apply-primary:disabled { opacity: 0.6; }

.apply-secondary {
  width: 100%;
  border-radius: 999px;
  padding: 14px;
  font-size: 15px;
  font-weight: 700;
  border: 1.5px solid var(--color-brand-border);
  color: var(--color-brand-text);
}

/* ------------------------------------------------- Scanner
   Chrome for the camera screen: the instruction over the feed, the dark
   sheet at the bottom, the AI/AR switch.

   Scoped to its own tokens rather than the app palette. Everywhere else
   in SanctiWalk sits on warm paper with navy type; here the background
   is a live camera in a building that may be lit by one window, and the
   palette has to answer that instead. */
.scan {
  --scan-sheet: rgb(9 12 22 / 0.94);
  --scan-line: rgb(255 255 255 / 0.13);
```

```
  --scan-ink: #fff;
  --scan-dim: rgb(255 255 255 / 0.62);
  /* The single accent: candle gold. The parish navy vanishes against a
     dark nave, and gilt is already the colour that means "look here" in
     this building. Spent only on the AR states, where the pilgrim is
     being asked to find something. */
  --scan-gild: #e9c77c;
}

/* ---- the instruction over the feed ---- */
.scan-head {
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  z-index: 3;
  display: flex;
  align-items: flex-start;
  gap: 8px;
  padding: 14px 14px 0;
}

.scan-head__text {
  flex: 1;
  min-width: 0;
  text-align: center;
  /* Optically centred against the close button, so the title sits in
     the middle of the SCREEN rather than in the middle of the space the
     button left over. */
  padding-left: 40px;
}

.scan-head__title {
  margin: 0;
  font-size: 17.5px;
  font-weight: 700;
  line-height: 1.25;
  color: var(--scan-ink);
  text-shadow: 0 1px 10px rgb(0 0 0 / 0.65);
}

.scan-head__sub {
  margin: 4px auto 0;
  max-width: 30ch;
  font-size: 13.5px;
  line-height: 1.42;
  color: var(--scan-dim);
  text-shadow: 0 1px 8px rgb(0 0 0 / 0.6);
}

.scan-head__close {
  flex-shrink: 0;
  width: 34px;
  height: 34px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  color: var(--scan-ink);
  background: rgb(0 0 0 / 0.38);
```

```
  backdrop-filter: blur(6px);
}

/* ---- the sheet ---- */
/* The two ways out of the AR gate on a phone that cannot run the tour.
   Without these the notice was a dead end - it named the problem and
   offered nothing to do about it. */
.scan-gate__actions {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  justify-content: center;
  margin-top: 14px;
}
.scan-gate__btn {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  padding: 9px 15px;
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-primary);
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  font-family: inherit;
  font-size: 13.5px;
  font-weight: 700;
  text-decoration: none;
  cursor: pointer;
}
.scan-gate__btn--quiet {
  background: transparent;
  color: var(--color-brand-primary);
}

/* ================================================================
   THE APP ON A DESK
   What /app looks like on a laptop. Shares the landing page's
   masthead and background on purpose: a visitor pressing "Open full
   screen" should not feel they have left the site. One logo, one nav,
   one background, whichever page they are on.
   ================================================================ */
.app-stage {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 16px;
  width: 100%;
  min-height: 100vh;
  padding: 0 0 28px;
  background: var(--color-brand-bg);
}

/* Full bleed, unlike the content below it, so the masthead reads as the
   page's own rather than as a card sitting on it. */
.app-stage__bar {
  width: 100%;
  max-width: 1180px;
  display: flex;
  align-items: center;
```

```
  justify-content: space-between;
  gap: 20px;
  padding: 20px clamp(16px, 4vw, 40px);
}
.app-stage__logo { height: auto; width: clamp(140px, 15vw, 170px); display: block; }

.app-stage__nav {
  display: flex;
  gap: clamp(14px, 2.4vw, 30px);
  font-size: 15px;
  font-weight: 600;
}
.app-stage__nav a {
  color: var(--color-brand-secondary);
  text-decoration: none;
}
.app-stage__nav a:hover { color: var(--color-brand-primary); }

/* ================================================================
   THE LANDING PAGE
   What a laptop sees at the root. See src/components/RootGate.tsx for
   why a phone sees something else.

   The app's own tokens throughout - Montserrat, the navy, the warm
   paper. A promotional page in a different palette reads as a different
   product that happens to share a logo.
   ================================================================ */
.lp {
  min-height: 100vh;
  background: var(--color-brand-bg);
  color: var(--color-brand-text);
  font-family: var(--font-primary);
}

/* The white panel the hero sits on, floating on the warm page. */
.lp__panel {
  max-width: 1180px;
  margin: 0 auto;
  padding: 22px clamp(20px, 4vw, 44px) clamp(28px, 4vw, 48px);
  background: var(--color-brand-card);
  border-radius: 0 0 36px 36px;
  box-shadow: 0 18px 50px rgb(20 23 31 / 0.06);
}

.lp__bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 20px;
  padding-bottom: clamp(24px, 4vw, 52px);
}
.lp__logo { height: auto; width: clamp(150px, 16vw, 190px); }

.lp__nav {
  display: flex;
  gap: clamp(16px, 2.5vw, 34px);
  font-size: 15px;
  font-weight: 600;
}
```

```
.lp__nav a {
  color: var(--color-brand-secondary);
  text-decoration: none;
}
.lp__nav a:hover { color: var(--color-brand-primary); }

/* Copy on the left, the phone on the right - the composition from the
   reference. Collapses to one column on a tablet, where side by side
   would leave the headline three words wide. */
.lp__hero {
  display: grid;
  grid-template-columns: 1.05fr 0.95fr;
  gap: clamp(24px, 4vw, 56px);
  align-items: center;
}
@media (max-width: 900px) {
  .lp__hero { grid-template-columns: 1fr; }
  .lp__shot { order: -1; }
}

.lp__headline {
  margin: 0;
  font-size: clamp(38px, 5.4vw, 66px);
  font-weight: 800;
  line-height: 1.03;
  letter-spacing: -0.025em;
}

.lp__lede {
  margin: 20px 0 0;
  max-width: 34em;
  font-size: clamp(15.5px, 1.5vw, 17.5px);
  line-height: 1.62;
  color: var(--color-brand-secondary);
}

.lp__actions {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 28px;
}

.lp__cta {
  display: inline-flex;
  align-items: center;
  gap: 9px;
  padding: 15px 26px;
  border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  font-size: 16px;
  font-weight: 700;
  text-decoration: none;
  border: 1.5px solid var(--color-brand-primary);
  transition: transform 120ms ease, filter 140ms ease;
}
.lp__cta:hover { filter: brightness(1.08); }
.lp__cta:active { transform: scale(0.99); }
```

```
/* The second action is real, not a ghost of the first: downloading the
   APK is a different decision, not a lesser version of exploring. */
.lp__cta--quiet {
  background: transparent;
  color: var(--color-brand-primary);
}
.lp__cta--quiet:hover { background: var(--color-brand-card-sunk); filter: none; }

.lp__footnote {
  margin: 16px 0 0;
  font-family: var(--font-mono-label);
  font-size: 12.5px;
  letter-spacing: 0.03em;
  color: var(--color-brand-secondary);
}

/* ------------------------------------------------------------ the phone
   A frame around a real screenshot. Deliberately a plain rounded
   rectangle rather than a drawn handset with a notch and a camera
   bump: the picture inside is the product, and a detailed bezel around
   it is the page admiring its own illustration. */
.lp__shot { display: flex; justify-content: center; }

.lp__phone {
  position: relative;
  padding: 11px;
  border-radius: 34px;
  background: var(--color-brand-primary);
  box-shadow: 0 26px 60px rgb(28 44 86 / 0.26);
}
/* The running app, at a real phone's size, scaled to fit.

   --ph-scale is the only thing a breakpoint changes. The iframe stays
   375 x 812 - an iPhone's own viewport, and the width the app's layout
   was built against - so the app inside never has to cope with a width
   no real handset has. The wrapper is the scaled box the page lays out
   against, because a CSS transform does not change the space an
   element occupies and the hero would otherwise reserve the full 812px
   of height for something drawn at 633. */
.lp__phone-viewport {
  --ph-scale: 0.78;
  width: calc(375px * var(--ph-scale));
  height: calc(812px * var(--ph-scale));
  overflow: hidden;
  border-radius: 24px;
  background: var(--color-brand-card);
}

.lp__phone-screen {
  display: block;
  width: 375px;
  height: 812px;
  border: 0;
  background: var(--color-brand-card);
  transform: scale(var(--ph-scale));
  transform-origin: top left;
}
```

```
/* The live app in a frame is a laptop idea. On a phone the screen IS
   the frame, so a phone-inside-a-phone would be a toy version of the
   thing the visitor is holding - and a 300px iframe inside a 375px
   screen is unusable besides. The button above takes them to the real
   one instead. */
@media (max-width: 760px) {
  .lp__cta-wide { display: none; }

  /* The running app, on a phone too.

     It is a phone inside a phone, which sounds silly until you watch
     somebody use it: the frame is what tells a visitor that the thing
     scrolling under their thumb is the actual app and not a picture of
     one. The same reassurance the laptop version gives, and the reason
     both pages now show it.

     Below the buttons here, not above. The tablet rule puts it first
     because at that width the copy and the phone sit side by side and
     the eye should land on the picture; on a phone everything is one
     column, and burying the headline under a 600px frame means the
     visitor scrolls before reading a word. */
  .lp__shot { order: 1; margin-top: 26px; }

  .lp__phone { padding: 9px; border-radius: 30px; }
  /* Scaled to the screen it is sitting on, rather than to a fixed
     number: 0.72 of a 375px phone is 270px, which leaves room for the
     frame and the page gutter on the narrowest handset we test. */
  .lp__phone-viewport {
    --ph-scale: 0.72;
    border-radius: 20px;
  }

  /* The nav wrapped "What it does" onto three lines beside the logo and
     pushed the masthead to twice its height. On a phone those links are
     scrolling past a thumb-flick below anyway; Download is the only one
     that goes somewhere else, so it is the only one that stays. */
  .lp__nav a:not(:last-child) { display: none; }
  .lp__bar { padding-bottom: 26px; }
}
@media (min-width: 761px) {
  .lp__cta-narrow { display: none; }
}

.lp__shot-note {
  margin: 12px 0 0;
  text-align: center;
  font-family: var(--font-mono-label);
  font-size: 12px;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}
.lp__shot { flex-direction: column; align-items: center; }

/* ------------------------------------------------------------ band */
.lp__band {
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  padding: clamp(44px, 6vw, 76px) 20px;
```

```
}
.lp__bandinner { max-width: 1060px; margin: 0 auto; }

.lp__bandtitle {
  margin: 0 0 clamp(22px, 3vw, 40px);
  font-size: clamp(26px, 3.2vw, 38px);
  font-weight: 800;
  letter-spacing: -0.02em;
}

.lp__grid {
  margin: 0;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: clamp(20px, 3vw, 36px);
}
.lp__grid dt {
  font-size: 17.5px;
  font-weight: 700;
  margin-bottom: 7px;
}
.lp__grid dd {
  margin: 0;
  font-size: 15px;
  line-height: 1.62;
  color: rgb(255 255 255 / 0.82);
}

/* ------------------------------------------------------------ closing */
.lp__closing {
  max-width: 720px;
  margin: 0 auto;
  padding: clamp(44px, 6vw, 76px) 20px clamp(30px, 4vw, 48px);
  text-align: center;
}
.lp__closing h2 {
  margin: 0 0 16px;
  font-size: clamp(25px, 3vw, 34px);
  font-weight: 800;
  letter-spacing: -0.02em;
}
.lp__closing p {
  margin: 0 auto 14px;
  max-width: 54ch;
  font-size: 16px;
  line-height: 1.65;
  color: var(--color-brand-secondary);
}
.lp__closingnote {
  font-size: 15px !important;
  font-style: italic;
}
.lp__closing .lp__cta { margin-top: 12px; }

.lp__foot {
  max-width: 720px;
  margin: 0 auto;
  padding: 0 20px 56px;
  text-align: center;
```

```
  font-size: 13.5px;
  line-height: 1.6;
  color: var(--color-brand-secondary);
}
.lp__foot a { color: var(--color-brand-primary); font-weight: 600; }

@media (prefers-reduced-motion: reduce) {
  .lp__cta { transition: none; }
  .lp__cta:active { transform: none; }
}

/* ================================================================
   THE ADMIN BOARD
   A grid of doors, shown above the nav on the dashboard. See
   src/admin/AdminTiles.tsx for why it exists alongside the nav rather
   than replacing it.

   Set in the app's own tokens - Montserrat, the navy, the warm card -
   so the office and the thing it publishes to are recognisably one
   product. An admin panel in some other palette reads as a different
   piece of software that happens to share a database.
   ================================================================ */
.admin-board {
  margin: 0 0 22px;
}

.admin-board__title {
  margin: 4px 0 0;
  text-align: center;
  font-size: clamp(22px, 4.5vw, 30px);
  font-weight: 800;
  letter-spacing: -0.01em;
  color: var(--color-brand-text);
}

/* One field and one button, joined. The parish secretary is looking for
   a section by the word they have in mind, not browsing. */
.admin-board__search {
  display: flex;
  gap: 0;
  max-width: 460px;
  margin: 14px auto 20px;
}
.admin-board__input {
  flex: 1;
  min-width: 0;
  padding: 10px 14px;
  border: 1px solid var(--color-brand-border);
  border-right: 0;
  border-radius: 999px 0 0 999px;
  background: var(--color-brand-card);
  color: var(--color-brand-text);
  font: inherit;
  font-size: 15px;
}
.admin-board__input:focus {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: -1px;
}
```

```
.admin-board__go {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 0 16px;
  border: 0;
  border-radius: 0 999px 999px 0;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  cursor: pointer;
}

.admin-board__empty {
  margin: 0;
  text-align: center;
  font-size: 15px;
  color: var(--color-brand-secondary);
}

/* Three across on a desktop, two on a tablet, one on a phone. The
   parish office works on a laptop, but the priest opens it on a
   phone. */
.admin-board__grid {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(190px, 1fr));
  gap: 14px;
}

.admin-tile {
  position: relative;
  overflow: hidden;
  width: 100%;
  min-height: 150px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 14px;
  padding: 24px 14px 20px;
  border: 1px solid var(--color-brand-border);
  border-radius: 20px;
  background: var(--color-brand-card);
  color: var(--color-brand-text);
  font: inherit;
  cursor: pointer;
  transition: border-color 140ms ease, transform 120ms ease;
}
.admin-tile:hover { border-color: var(--color-brand-primary); }
.admin-tile:active { transform: scale(0.99); }
.admin-tile:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 2px;
}

/* Large enough to be recognised at a lean-back distance. The label
   underneath is what actually names it; the icon is for finding the
```

```
   tile again on the second visit. */
.admin-tile__icon svg {
  width: 44px;
  height: 44px;
  stroke-width: 1.5;
  color: var(--color-brand-primary);
}

.admin-tile__label {
  font-size: 14px;
  font-weight: 700;
  letter-spacing: 0.04em;
  text-transform: uppercase;
  text-align: center;
  line-height: 1.3;
}

/* The corner ribbon. It carries the one fact that changes what you do
   next - whether the parish fills this in, or the app keeps it - rather
   than repeating the word "System" on every tile. */
.admin-tile__ribbon {
  position: absolute;
  top: 12px;
  right: -30px;
  width: 116px;
  transform: rotate(45deg);
  text-align: center;
  padding: 3px 0;
  font-family: var(--font-mono-label, ui-monospace, monospace);
  font-size: 10px;
  font-weight: 500;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  pointer-events: none;
}
.admin-tile__ribbon--parish {
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}
/* Quieter: nothing is asked of the reader for these two. */
.admin-tile__ribbon--automatic {
  background: var(--color-brand-card-sunk);
  color: var(--color-brand-secondary);
  border-bottom: 1px solid var(--color-brand-border);
}

@media (prefers-reduced-motion: reduce) {
  .admin-tile { transition: none; }
  .admin-tile:active { transform: none; }
}

/* The answer, collapsed, at the top of the scanner sheet.
   The name only: a full card the instant the scan returns buries the
   thing the camera is pointed at, so the rest waits for a tap. */
.scan-result-peek {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
```

```
  width: 100%;
  /* Above the switch; the sheet's own padding does the rest. */
  margin: 0 0 9px;
  padding: 10px 13px;
  border-radius: 16px;
  border: 1px solid var(--color-brand-border);
  background: color-mix(in srgb, var(--color-brand-card) 92%, transparent);
  color: var(--color-brand-text);
  font-family: inherit;
  text-align: left;
  cursor: pointer;
  transition: transform 120ms ease;
}
.scan-result-peek:active { transform: scale(0.985); }
.scan-result-peek:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 2px;
}

.scan-result-peek__text { min-width: 0; }

.scan-result-peek__label {
  display: block;
  font-size: 11.5px;
  font-weight: 700;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}

/* One line. A long title wrapping to two would start eating the
   viewfinder, which is what the collapsed state exists to protect. */
.scan-result-peek__title {
  display: block;
  font-size: 15px;
  font-weight: 700;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

@media (prefers-reduced-motion: reduce) {
  .scan-result-peek { transition: none; }
  .scan-result-peek:active { transform: none; }
}

/* Who the saint was, under what the camera saw.
   Set in a tinted panel rather than run on as another paragraph: the two
   are different kinds of knowing, and the card should not let a reader
   mistake the Church's general record for this parish's own. */
/* Read-aloud control on the scan card. Sits above the text, because
   somebody who would rather listen than read should not have to read to
   the bottom to discover they could have listened. */
.scan-listen {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  margin: 0 0 12px;
  padding: 7px 13px;
```

```
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-primary);
  background: transparent;
  color: var(--color-brand-primary);
  font-family: inherit;
  font-size: 13.5px;
  font-weight: 700;
  cursor: pointer;
  transition: background 140ms ease, transform 120ms ease;
}
.scan-listen:active { transform: scale(0.985); }
.scan-listen[aria-pressed="true"] {
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}
.scan-listen:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 2px;
}
@media (prefers-reduced-motion: reduce) {
  .scan-listen { transition: none; }
  .scan-listen:active { transform: none; }
}

.saint-note {
  margin-top: 16px;
  padding: 13px 14px 12px;
  border-radius: 16px;
  border: 1px solid var(--color-brand-border);
  background: color-mix(in srgb, var(--color-brand-accent) 7%, transparent);
}

.saint-note__head {
  margin: 0 0 9px;
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}

/* Feast day and patronage read as a short reference block, not prose -
   they are the two things somebody might act on or repeat. */
.saint-note__facts {
  margin: 0 0 10px;
  display: flex;
  flex-wrap: wrap;
  gap: 8px 18px;
}
.saint-note__fact { min-width: 0; }
.saint-note__fact dt {
  font-size: 11.5px;
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--color-brand-secondary);
}
.saint-note__fact dd {
  margin: 1px 0 0;
```

```
  font-size: 14.5px;
  font-weight: 700;
  color: var(--color-brand-text);
}

.saint-note__body {
  margin: 0;
  font-size: 14.5px;
  line-height: 1.6;
  color: var(--color-brand-text);
}

/* Quiet, but present on every card that carries it. */
.saint-note__source {
  margin: 10px 0 0;
  font-size: 12.5px;
  line-height: 1.5;
  color: var(--color-brand-secondary);
}
.saint-note__link {
  color: var(--color-brand-primary);
  font-weight: 700;
  text-decoration: underline;
  text-underline-offset: 2px;
}

.scan-sheet {
  position: absolute;
  left: 8px;
  right: 8px;
  bottom: 8px;
  z-index: 4;
  /* Tightened. This panel was covering a third of the viewfinder,
     which is the one thing the screen is for. */
  padding: 9px 12px 11px;
  border-radius: 26px;
  background: var(--scan-sheet);
  border: 1px solid var(--scan-line);
  backdrop-filter: blur(18px);
  box-shadow: 0 16px 40px rgb(0 0 0 / 0.45);
  color: var(--scan-ink);
}

/* ---- AI / AR switch ---- */
.scan-tabs {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 4px;
  padding: 4px;
  border-radius: 999px;
  background: rgb(255 255 255 / 0.07);
  border: 1px solid var(--scan-line);
}

.scan-tab {
  padding: 7px 10px;
  border-radius: 999px;
  font-size: 14.5px;
  font-weight: 600;
```

```
  color: var(--scan-dim);
  transition: background-color 180ms ease, color 180ms ease;
}

.scan-tab.is-on {
  background: #fff;
  color: #10131f;
  font-weight: 700;
}

.scan-tab:focus-visible {
  outline: 2px solid var(--scan-gild);
  outline-offset: 2px;
}

/* ---- the target ---- */
.scan-reticle {
  position: absolute;
  /* Above centre: the sheet occupies the lower third, so the optical
     middle of the free space is higher than the middle of the screen. */
  top: 40%;
  left: 50%;
  width: min(56%, 215px);
  aspect-ratio: 4 / 5;
  transform: translate(-50%, -50%);
  pointer-events: none;
  z-index: 2;
  transition: transform 420ms ease;
}

.scan-reticle.is-reading {
  transform: translate(-50%, -50%) scale(0.94);
}

.scan-reticle__c {
  position: absolute;
  width: 30px;
  height: 30px;
  border: 0 solid rgb(255 255 255 / 0.9);
  transition: border-color 300ms ease;
}

.scan-reticle.is-reading .scan-reticle__c {
  border-color: var(--scan-gild);
}

.scan-reticle__c--tl { top: 0; left: 0; border-top-width: 3px; border-left-width: 3px; border-top-left-radius: 10px; }
.scan-reticle__c--tr { top: 0; right: 0; border-top-width: 3px; border-right-width: 3px; border-top-right-radius: 10px; }
.scan-reticle__c--bl { bottom: 0; left: 0; border-bottom-width: 3px; border-left-width: 3px; border-bottom-left-radius: 10px; }
.scan-reticle__c--br { bottom: 0; right: 0; border-bottom-width: 3px; border-right-width: 3px; border-bottom-right-radius: 10px; }

/* ---- the shutter ---- */
.scan-shutter {
  width: 56px;
  height: 56px;
  margin: 9px auto 0;
  display: grid;
  place-items: center;
  border-radius: 999px;
```

```
  border: 3px solid rgb(255 255 255 / 0.9);
  transition: transform 160ms ease;
}

.scan-shutter:active { transform: scale(0.94); }
.scan-shutter:disabled { opacity: 0.55; }

.scan-shutter__core {
  width: 42px;
  height: 42px;
  border-radius: 999px;
  background: #fff;
  transition: width 260ms ease, height 260ms ease, background-color 260ms ease;
}

.scan-shutter__core.is-busy {
  width: 24px;
  height: 24px;
  background: var(--scan-gild);
}

.scan-caption {
  margin: 7px 0 0;
  text-align: center;
  font-size: 12.5px;
  line-height: 1.35;
  color: var(--scan-dim);
}

/* ---- AR panel ---- */
.scan-panel { margin-top: 14px; }

.scan-panel__title {
  margin: 0;
  text-align: center;
  font-size: 16px;
  font-weight: 700;
  color: var(--scan-ink);
}

.scan-panel__body {
  margin: 6px auto 0;
  max-width: 34ch;
  text-align: center;
  font-size: 13.5px;
  line-height: 1.5;
  color: var(--scan-dim);
}

.scan-action {
  width: 100%;
  margin-top: 14px;
  padding: 14px;
  border-radius: 999px;
  font-size: 15.5px;
  font-weight: 700;
  background: #fff;
  color: #10131f;
  display: inline-flex;
```

```
  align-items: center;
  justify-content: center;
  gap: 8px;
}

.scan-action--quiet {
  background: rgb(255 255 255 / 0.1);
  color: var(--scan-ink);
  border: 1px solid var(--scan-line);
}

.scan-action:active { transform: scale(0.99); }

/* The read-in-progress bar. The one piece of non-user-triggered motion
   on this screen, and it earns it: it is the only thing saying the app
   has not frozen while the camera is being held still. */
.scan-progress {
  position: relative;
  height: 3px;
  margin-top: 12px;
  border-radius: 999px;
  background: rgb(255 255 255 / 0.14);
  overflow: hidden;
}

.scan-progress::after {
  content: "";
  position: absolute;
  inset: 0;
  width: 40%;
  border-radius: 999px;
  background: var(--scan-gild);
  animation: scan-sweep 1.15s ease-in-out infinite;
}

@keyframes scan-sweep {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(250%); }
}

@media (prefers-reduced-motion: reduce) {
  .scan-progress::after { animation: none; width: 100%; opacity: 0.75; }
}

/* ---- the stop card, while the AR tour is running ---- */
.scan-stop {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  margin-top: 14px;
  padding: 12px;
  border-radius: 18px;
  background: rgb(255 255 255 / 0.07);
  border: 1px solid var(--scan-line);
}

/* Neutral, not gold. The narration control is already gold while it is
   speaking, and two gold things in one small card fight each other -
   the accent is worth more spent on the one that changes. */
```

```
.scan-stop__n {
  flex-shrink: 0;
  width: 30px;
  height: 30px;
  display: grid;
  place-items: center;
  border-radius: 10px;
  background: rgb(255 255 255 / 0.14);
  border: 1px solid var(--scan-line);
  color: var(--scan-ink);
  font-size: 15px;
  font-weight: 700;
}

.scan-stop__name {
  margin: 0;
  font-size: 15.5px;
  font-weight: 700;
  line-height: 1.3;
  color: var(--scan-ink);
}

.scan-stop__meta {
  margin: 2px 0 0;
  font-size: 13px;
  color: var(--scan-dim);
}

/* Narration. Gold while speaking, so the control states what it is
   doing rather than relying on the icon alone. */
.scan-listen {
  display: inline-flex;
  align-items: center;
  gap: 7px;
  margin-top: 9px;
  padding: 7px 13px 7px 10px;
  border-radius: 999px;
  font-size: 13.5px;
  font-weight: 700;
  color: var(--scan-ink);
  background: rgb(255 255 255 / 0.1);
  border: 1px solid var(--scan-line);
}

.scan-listen.is-speaking {
  background: var(--scan-gild);
  border-color: var(--scan-gild);
  color: #221a06;
}

.scan-row {
  display: grid;
  grid-template-columns: 1fr 1.3fr;
  gap: 9px;
  margin-top: 12px;
}

.scan-row > .scan-action { margin-top: 0; }
```

```
/* ---- the marker hint, when the pilgrim is not at one yet ---- */
.scan-marker {
  position: absolute;
  top: 46%;
  left: 50%;
  transform: translate(-50%, -50%);
  z-index: 2;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  pointer-events: none;
}

.scan-marker__tile {
  width: 62px;
  height: 62px;
  display: grid;
  place-items: center;
  border-radius: 18px;
  background: rgb(255 255 255 / 0.1);
  border: 1px solid rgb(255 255 255 / 0.3);
  backdrop-filter: blur(6px);
  animation: scan-halo 2.4s ease-out infinite;
}

.scan-marker__tile img { width: 34px; height: 34px; object-fit: contain; border-radius: 8px; }

.scan-marker__label {
  padding: 4px 10px;
  border-radius: 999px;
  background: rgb(0 0 0 / 0.55);
  font-size: 12.5px;
  font-weight: 600;
  color: var(--scan-ink);
}

@keyframes scan-halo {
  0%   { box-shadow: 0 0 0 0 rgb(233 199 124 / 0.45); }
  70%  { box-shadow: 0 0 0 18px rgb(233 199 124 / 0); }
  100% { box-shadow: 0 0 0 0 rgb(233 199 124 / 0); }
}

@media (prefers-reduced-motion: reduce) {
  .scan-marker__tile { animation: none; border-color: var(--scan-gild); }
}

/* ---- status chip, while the AR tour is running ---- */
.scan-chip {
  position: absolute;
  top: 62px;
  left: 14px;
  z-index: 3;
  display: inline-flex;
  align-items: center;
  gap: 7px;
  padding: 5px 11px;
  border-radius: 999px;
  background: rgb(0 0 0 / 0.5);
```

```
  border: 1px solid var(--scan-line);
  font-size: 12.5px;
  font-weight: 600;
  color: var(--scan-ink);
  backdrop-filter: blur(6px);
}

.scan-chip__dot {
  width: 7px;
  height: 7px;
  border-radius: 999px;
  background: var(--scan-gild);
}

/* ------------------------------------------------ Spotlight
   A hole punched in a dimmed screen, and a card explaining what is in
   it. Shared by the first-run tutorial and by Sancti, so the assistant
   pointing something out looks like the same app that showed you round
   on day one. */
/* Absolute, inside the app shell - not fixed to the window. On a phone
   those are the same box; in the browser preview the app sits inside a
   phone mock, and fixed positioning put the dimming over the whole
   laptop screen with the card hanging off the side of the mock. */
.spot {
  position: absolute;
  inset: 0;
  z-index: 1200;
  /* The overlay itself catches nothing: the hole is the only part that
     takes a tap, so the element underneath stays visible but inert and
     nobody half-presses a button through the dimming. */
  pointer-events: none;
}

/* The dimming IS this element's shadow, spreading outwards from the
   hole. One paint, and no hairline seams where four separate panels
   around the target would meet on fractional pixels. */
.spot__hole {
  position: absolute;
  border-radius: 16px;
  box-shadow: 0 0 0 9999px rgb(8 11 20 / 0.74);
  /* Just enough edge to read as a cut-out rather than a lighting
     accident, in the parish gold the scanner already uses to mean
     "look here". */
  outline: 2px solid rgb(233 199 124 / 0.85);
  outline-offset: 0;
  pointer-events: auto;
  animation: spot-in 260ms ease-out;
}

@keyframes spot-in {
  from { box-shadow: 0 0 0 9999px rgb(8 11 20 / 0); outline-color: rgb(233 199 124 / 0); }
}

.spot__card {
  position: absolute;
  z-index: 1;
  padding: 16px 16px 14px;
  border-radius: 18px;
  background: #141a2e;
```

```
  color: #fff;
  box-shadow: 0 18px 44px rgb(0 0 0 / 0.5);
  pointer-events: auto;
  animation: spot-card-in 240ms ease-out;
}

@keyframes spot-card-in {
  from { opacity: 0; transform: translateY(6px); }
}

@media (prefers-reduced-motion: reduce) {
  .spot__hole, .spot__card { animation: none; }
}

/* The little triangle. Rotated square rather than a border triangle, so
   it inherits the card's own colour and never drifts a shade off it. */
.spot__arrow {
  position: absolute;
  width: 14px;
  height: 14px;
  background: #141a2e;
  transform: translateX(-50%) rotate(45deg);
}

.spot__card--arrow-top .spot__arrow { top: -6px; }
.spot__card--arrow-bottom .spot__arrow { bottom: -6px; }

.spot__title {
  margin: 0 0 5px;
  font-size: 16.5px;
  font-weight: 700;
  line-height: 1.25;
}

.spot__body {
  margin: 0;
  font-size: 15px;
  line-height: 1.5;
  color: rgb(255 255 255 / 0.88);
}

.spot__foot {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-top: 14px;
}

.spot__dots {
  display: inline-flex;
  align-items: center;
  gap: 5px;
  margin-right: auto;
}

.spot__dot {
  width: 6px;
  height: 6px;
  border-radius: 999px;
```

```
  background: rgb(255 255 255 / 0.3);
  transition: width 200ms ease, background-color 200ms ease;
}

.spot__dot.is-on {
  width: 18px;
  background: #fff;
}

/* The buttons the tutorial puts in the footer. */
.spot__btn {
  padding: 9px 15px;
  border-radius: 999px;
  font-size: 14.5px;
  font-weight: 700;
  background: #fff;
  color: #141a2e;
}

.spot__btn--quiet {
  background: transparent;
  color: rgb(255 255 255 / 0.72);
  padding-inline: 4px;
}

.spot__btn:focus-visible,
.spot__btn--quiet:focus-visible {
  outline: 2px solid #e9c77c;
  outline-offset: 2px;
}

/* The last card of the walkthrough. A plain centred panel rather than
   another spotlight: there is nothing left to point at. */
.tut-done {
  position: fixed;
  inset: 0;
  z-index: 1300;
  display: grid;
  place-items: center;
  padding: 24px;
  background: rgb(8 11 20 / 0.78);
}

.tut-done__card {
  width: 100%;
  max-width: 340px;
  padding: 26px 22px 22px;
  border-radius: 24px;
  background: var(--color-brand-card);
  text-align: center;
  box-shadow: 0 20px 50px rgb(0 0 0 / 0.45);
}

.tut-done__title {
  margin: 0;
  font-family: var(--font-serif, Georgia, serif);
  font-style: italic;
  font-size: 22px;
  font-weight: 700;
```

```
  line-height: 1.22;
  color: var(--color-brand-text);
}

.tut-done__body {
  margin: 10px 0 0;
  font-size: 15px;
  line-height: 1.55;
  color: var(--color-brand-secondary);
}

.tut-done__go {
  width: 100%;
  margin-top: 20px;
  padding: 15px;
  border-radius: 999px;
  font-size: 16px;
  font-weight: 700;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

/* ------------------------------------------------ Sancti
   The assistant: a floating way in, and a sheet over the app.

   Built from the parish's own navy and card colours rather than the
   grey-and-blue every chat UI wears. The brief asked for it not to look
   like a generic assistant bolted on, and the quickest way to fail that
   is to reach for the default chat palette. */

/* ---- the way in ---- */
.sancti-fab {
  position: absolute;
  right: 14px;
  /* Clear of the tab bar, including the home indicator on a phone that
     has one. */
  bottom: calc(76px + env(safe-area-inset-bottom, 0px));
  z-index: 60;
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 10px 16px 10px 12px;
  border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  box-shadow: 0 8px 22px rgb(16 24 40 / 0.3);
  transition: transform 150ms ease;
}

.sancti-fab:active { transform: scale(0.96); }

/* Pushed aside: the mark alone, half off the edge, so the bulletin
   underneath is readable. Still a full-height target, because the
   thing it must never stop being is tappable. */
.sancti-fab--tucked {
  padding: 10px;
  gap: 0;
  right: 6px;
  opacity: 0.9;
```

```
  transition: transform 150ms ease, padding 180ms ease, right 180ms ease,
              opacity 180ms ease;
}

.sancti-fab--tucked .sancti-fab__label {
  display: none;
}

/* The drag is a horizontal gesture on a button, so the browser must not
   claim it for scrolling first. */
.sancti-fab { touch-action: pan-y; }

@media (prefers-reduced-motion: reduce) {
  .sancti-fab, .sancti-fab--tucked { transition: none; }
}

.sancti-fab img {
  width: 22px;
  height: 22px;
  object-fit: contain;
}

/* The label is a passenger on the button, not a target of its own, and
   it is wide. Over the map it came to rest on the OpenStreetMap
   attribution link and swallowed the tap - and that link is not
   decoration, it is the credit the tile licence requires to be
   reachable. The button itself still takes the press; only the text
   stops intercepting it. */
.sancti-fab__label {
  pointer-events: none;
  font-size: 14.5px;
  font-weight: 700;
  white-space: nowrap;
}

/* Narrow phones: the mark alone. The label is the first thing to go
   when it would otherwise crowd the tab bar underneath. */
@media (max-width: 340px) {
  .sancti-fab__label { display: none; }
  .sancti-fab { padding: 12px; }
}

/* ---- the sheet ---- */
.sancti {
  position: absolute;
  inset: 0;
  z-index: 100;
  display: flex;
  flex-direction: column;
  justify-content: flex-end;
}

.sancti__scrim {
  position: absolute;
  inset: 0;
  background: rgb(8 11 20 / 0.45);
  animation: sancti-fade 180ms ease-out;
}
```

```
.sancti__sheet {
  position: relative;
  display: flex;
  flex-direction: column;
  /* Tall enough to hold a Mass schedule without scrolling, short enough
     that the screen underneath is still visibly there. */
  height: 76%;
  background: var(--color-brand-card);
  border-radius: 24px 24px 0 0;
  box-shadow: 0 -10px 36px rgb(0 0 0 / 0.26);
  animation: sancti-up 240ms cubic-bezier(0.22, 1, 0.36, 1);
  overflow: hidden;
}

@keyframes sancti-fade { from { opacity: 0; } }
@keyframes sancti-up { from { transform: translateY(14%); opacity: 0.4; } }

@media (prefers-reduced-motion: reduce) {
  .sancti__scrim, .sancti__sheet { animation: none; }
}

.sancti__head {
  display: flex;
  align-items: center;
  gap: 11px;
  padding: 14px 14px 12px;
  border-bottom: 1px solid var(--color-brand-border);
}

.sancti__avatar {
  width: 38px;
  height: 38px;
  flex-shrink: 0;
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: var(--color-brand-primary);
}

.sancti__avatar img { width: 21px; height: 21px; object-fit: contain; }

.sancti__who { min-width: 0; flex: 1; }

.sancti__name {
  display: block;
  font-size: 17px;
  font-weight: 700;
  font-style: italic;
  font-family: var(--font-serif, Georgia, serif);
  color: var(--color-brand-text);
}

.sancti__sub {
  display: block;
  font-size: 13.5px;
  color: var(--color-brand-secondary);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
```

```
}

.sancti__x {
  flex-shrink: 0;
  width: 34px;
  height: 34px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  color: var(--color-brand-secondary);
}

.sancti__log {
  flex: 1;
  min-height: 0;
  overflow-y: auto;
  padding: 14px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

/* ---- the empty state, which is where a new pilgrim starts ---- */
.sancti__greeting {
  margin: 0;
  font-size: 15.5px;
  line-height: 1.55;
  color: var(--color-brand-text);
}

.sancti__try {
  margin: 16px 0 8px;
  font-size: 14px;
  font-weight: 700;
  color: var(--color-brand-secondary);
}

.sancti__chips { display: flex; flex-wrap: wrap; gap: 8px; }

.sancti__chip {
  padding: 9px 14px;
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-border);
  background: var(--color-brand-card-sunk);
  font-size: 14.5px;
  font-weight: 600;
  color: var(--color-brand-text);
  text-align: left;
}

.sancti__chip:active { transform: scale(0.98); }

/* ---- messages ---- */
.sancti__row { display: flex; flex-direction: column; gap: 6px; }
.sancti__row--you { align-items: flex-end; }
.sancti__row--sancti { align-items: flex-start; }

.sancti__bubble {
  max-width: 86%;
```

```
  padding: 11px 14px;
  border-radius: 18px;
  font-size: 15.5px;
  line-height: 1.5;
  display: flex;
  flex-direction: column;
  gap: 2px;
}

/* The pilgrim in the parish navy, Sancti on the card. The other way
   round reads as the app talking at you. */
.sancti__bubble--you {
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
  border-bottom-right-radius: 6px;
}

.sancti__bubble--sancti {
  background: var(--color-brand-card-sunk);
  border: 1px solid var(--color-brand-border);
  color: var(--color-brand-text);
  border-bottom-left-radius: 6px;
}

/* A blank line in a schedule is a real gap, not a missing line. */
.sancti__line:empty { height: 0.5em; }

.sancti__followup {
  align-self: flex-start;
  padding: 8px 14px;
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-primary);
  font-size: 14px;
  font-weight: 700;
  color: var(--color-brand-primary);
}

/* Three dots, not the word "typing". Nobody is typing. */
.sancti__thinking { flex-direction: row; gap: 5px; padding: 15px 16px; }

.sancti__thinking span {
  width: 7px;
  height: 7px;
  border-radius: 999px;
  background: var(--color-brand-secondary);
  opacity: 0.45;
  animation: sancti-blink 1.1s ease-in-out infinite;
}

.sancti__thinking span:nth-child(2) { animation-delay: 0.16s; }
.sancti__thinking span:nth-child(3) { animation-delay: 0.32s; }

@keyframes sancti-blink {
  0%, 60%, 100% { opacity: 0.35; transform: translateY(0); }
  30% { opacity: 1; transform: translateY(-3px); }
}

@media (prefers-reduced-motion: reduce) {
  .sancti__thinking span { animation: none; }
```

```
}

/* ---- the input ---- */
.sancti__compose {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 12px 14px calc(14px + env(safe-area-inset-bottom, 0px));
  border-top: 1px solid var(--color-brand-border);
  background: var(--color-brand-card);
}

/* 16px minimum, or iOS Safari zooms the page on focus and never zooms
   back. The same floor the application form uses. */
.sancti__input {
  flex: 1;
  min-width: 0;
  padding: 12px 15px;
  border-radius: 999px;
  border: 1px solid var(--color-brand-border);
  background: var(--color-brand-card-sunk);
  font-size: 16px;
  color: var(--color-brand-text);
}

.sancti__input:focus-visible {
  outline: 2px solid var(--color-brand-primary);
  outline-offset: 1px;
}

.sancti__send {
  flex-shrink: 0;
  width: 44px;
  height: 44px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: var(--color-brand-primary);
  color: var(--color-brand-on-accent);
}

.sancti__send:disabled { opacity: 0.4; }

/* ------------------------------------------------ Home quick links
   The four things people open Home to find, directly under the parish
   name. Two columns rather than four: at phone width four would give
   each label about seven characters, and "Mga Sakramento" is fourteen. */
/* Three across, in one line, as drawn. Not a scroller: a row that
   scrolls sideways hides the third button, and the whole reason this
   row exists is that things below the fold go unnoticed. */
.quick {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 8px;
  margin-bottom: 18px;
}

.quick__btn {
  display: flex;
```

```
  flex-direction: column;
  align-items: center;
  gap: 5px;
  padding: 8px 5px 7px;
  border-radius: 14px;
  text-align: center;
  background: var(--color-brand-card);
  border: 1px solid var(--color-brand-border);
  transition: border-color 150ms ease, transform 120ms ease;
}

.quick__btn:active { transform: scale(0.985); }

.quick__btn:hover,
.quick__btn:focus-visible {
  border-color: var(--color-brand-primary);
}

.quick__icon {
  flex-shrink: 0;
  display: grid;
  place-items: center;
  width: 26px;
  height: 26px;
  border-radius: 9px;
  background: var(--color-brand-card-sunk);
  color: var(--color-brand-primary);
  transition: background-color 160ms ease, color 160ms ease;
}

.quick__label {
  min-width: 0;
  font-size: 11.5px;
  font-weight: 700;
  line-height: 1.2;
  color: var(--color-brand-text);
  /* Two lines' worth of room, whether the label uses them or not.
     "History" is one word where the other two are two, so its button
     ended 21px of empty space tall against their 7px - three buttons
     side by side, one of them visibly hollow at the bottom. Reserving
     the height here and centring the text in it keeps the three
     identical, and keeps them identical in Tagalog too, where the word
     lengths are different again. em rather than lh: the latter is not
     safe to assume in an Android WebView. */
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: 2.4em;
  /* The labels carry their own line breaks so "Mass Schedule" always
     splits the same way, rather than wherever the width happens to
     land it. */
  white-space: pre-line;
  /* Wraps rather than truncating. "Ministeryo at Sakramento" is three
     words in a third of a phone, and half a word read is worse than
     two lines. */
  overflow-wrap: anywhere;
}

/* Narrow phones: the label wraps rather than truncating. A half-read
```

```
   "Mga Sakrame…" is worse than two lines. */
/* The chosen one. The parish navy, so it reads as selected rather than
   merely hovered - these three are a choice, and at any moment exactly
   one of them is true. */
.quick__btn.is-on {
  background: var(--color-brand-primary);
  border-color: var(--color-brand-primary);
}

.quick__btn.is-on .quick__label { color: var(--color-brand-on-accent); }

.quick__btn.is-on .quick__icon {
  background: rgb(255 255 255 / 0.18);
  color: var(--color-brand-on-accent);
}

@media (max-width: 340px) {
  .quick__label { font-size: 10.5px; }
  .quick__btn { padding: 7px 3px; gap: 4px; }
}

/* The bulletin's own name, above the chooser.
   Built on .home-section-title rather than beside it: the heavy
   uppercase Montserrat is the voice the rest of Home speaks in, and a
   second, lighter heading here read as a different app. Only the size
   and the margins differ, because this one sits above a row of buttons
   rather than above a card. */
.home-sheet__title {
  margin: 4px 0 14px;
  text-align: center;
  font-family: var(--font-primary);
  font-size: 26px;
  font-weight: 900;
  line-height: 1.04;
  letter-spacing: 0.02em;
  text-transform: uppercase;
  color: var(--home-heading);
  text-wrap: balance;
}

/* The chosen section fades in. Keyed on the choice, so changing it
   restarts the animation and the swap is visible rather than silent. */
.home-panel {
  /* Breathing room under the chosen section, before the verse and the
     announcements that always follow it. The section headings used to
     provide this with their own 26px top margin; they are gone, so the
     panel provides it. */
  margin-bottom: 22px;
  animation: home-panel-in 260ms ease-out;
}

/* The first heading inside a chosen section has the chooser directly
   above it, so it does not need the 26px it carries elsewhere. */
.home-panel .home-section-title { margin-top: 0; }

@keyframes home-panel-in {
  from { opacity: 0; transform: translateY(5px); }
}
```

```
@media (prefers-reduced-motion: reduce) {
  .home-panel { animation: none; }
}

.home-section { margin-top: 0; }

/* The ring that says "this is the part you asked for".
   Scroll alone is easy to miss - the page moves and the thing you
   wanted is now somewhere among three others. It fades by itself, so
   there is nothing to dismiss. */
.home-flash {
  border-radius: 22px;
  animation: home-flash 1.8s ease-out;
}

@keyframes home-flash {
  0%   { box-shadow: 0 0 0 0 rgb(233 199 124 / 0); }
  12%  { box-shadow: 0 0 0 5px rgb(233 199 124 / 0.85); }
  70%  { box-shadow: 0 0 0 5px rgb(233 199 124 / 0.55); }
  100% { box-shadow: 0 0 0 0 rgb(233 199 124 / 0); }
}

@media (prefers-reduced-motion: reduce) {
  .home-flash { animation: none; box-shadow: 0 0 0 4px rgb(233 199 124 / 0.7); }
}


/* ---- Preview: the feature, recorded ---- */

/* On the sheet's right edge rather than in the column of controls:
   it is not part of scanning, and a full-width button there would
   read as another step in it. */
.scan-preview-btn {
  position: absolute;
  /* ABOVE the sheet, not inside it. Sitting in the top-right corner
     put it straight on top of the "AR Scanner" tab label, because the
     switch is the first thing in the sheet and spans its full width. */
  bottom: calc(100% + 8px);
  right: 2px;
  z-index: 5;
  padding: 6px 14px;
  border-radius: 999px;
  font-size: 12.5px;
  font-weight: 700;
  color: var(--scan-ink);
  /* Opaque: it floats over the camera now, not over the panel. */
  background: var(--scan-sheet);
  border: 1px solid var(--scan-line);
  box-shadow: 0 6px 18px rgb(0 0 0 / 0.4);
}

.scan-preview-btn:active { transform: scale(0.96); }

.preview-backdrop {
  position: absolute;
  inset: 0;
  z-index: 60;
  display: grid;
  place-items: center;
```

```
  padding: 16px;
  background: rgb(4 7 14 / 0.86);
  backdrop-filter: blur(6px);
}

.preview-box {
  width: 100%;
  max-width: 560px;
  border-radius: 20px;
  overflow: hidden;
  background: #0b1220;
  border: 1px solid rgb(255 255 255 / 0.14);
  box-shadow: 0 24px 60px rgb(0 0 0 / 0.55);
}

.preview-bar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  padding: 10px 12px;
}

.preview-title {
  font-size: 14.5px;
  font-weight: 700;
  color: #fff;
}

.preview-close {
  height: 30px;
  width: 30px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  color: #fff;
  background: rgb(255 255 255 / 0.12);
}

.preview-video {
  display: block;
  width: 100%;
  max-height: 62vh;
  background: #000;
}

.preview-note {
  margin: 0;
  padding: 9px 12px 12px;
  font-size: 12px;
  line-height: 1.4;
  color: rgb(255 255 255 / 0.62);
}

/* ========================================================================
   "Notify me" bells

   On a closed ministry, a suspended Mass, a sacrament between intakes.
   Small and quiet: it sits beside the badge that says the thing is shut,
```

```
   and it is the answer to that badge rather than a second announcement
   competing with it.
   ========================================================================== */
.watch-bell__wrap {
  display: inline-flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 6px 8px;
}

.watch-bell {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  /* Tall enough to hit. At 6px of padding this came out 27px high,
     which is a control you aim at rather than tap - and it is the only
     way to act on a closed ministry, so missing it is the whole
     interaction. */
  min-height: 40px;
  padding: 8px 14px;
  border-radius: 999px;
  border: 1.5px solid var(--color-brand-border, rgba(27, 42, 107, 0.18));
  background: transparent;
  color: var(--color-brand-primary, #1B2A6B);
  font-size: 13px;
  font-weight: 700;
  line-height: 1;
  /* The row this sits in can get tight on a 360px screen, and a control
     that shrinks until its label is unreadable is worse than one that
     wraps onto its own line. */
  white-space: nowrap;
  cursor: pointer;
  transition: background-color 120ms ease, border-color 120ms ease;
}

.watch-bell:hover {
  background: color-mix(in srgb, var(--color-brand-primary, #1B2A6B) 7%, transparent);
}

.watch-bell.is-on {
  border-color: var(--color-brand-primary, #1B2A6B);
  background: color-mix(in srgb, var(--color-brand-primary, #1B2A6B) 12%, transparent);
}

/* The Mass card is a deep green panel, so the brand navy disappears
   into it. On dark, the bell borrows the panel's own white. */
.watch-bell--onDark {
  border-color: rgba(255, 255, 255, 0.55);
  color: #fff;
}
.watch-bell--onDark:hover { background: rgba(255, 255, 255, 0.14); }
.watch-bell--onDark.is-on {
  border-color: #fff;
  background: rgba(255, 255, 255, 0.2);
}

.watch-bell__note {
  font-size: 13px;
  font-weight: 600;
```

```
  color: var(--color-brand-muted, #5B6478);
}
.watch-bell--onDark + .watch-bell__note,
.watch-bell__wrap:has(.watch-bell--onDark) .watch-bell__note {
  color: rgba(255, 255, 255, 0.85);
}

/* The bell on its own, in the Mass card's header.
   Square so the circle is a circle, and the same 40px tap target the
   labelled form has. */
.watch-bell--icon {
  min-width: 40px;
  padding: 0;
  justify-content: center;
  flex: none;
}

/* ============================================================================
   Update banner

   Sits above the scroller in the installed app only. It is information,
   not an alarm: the parish navy rather than a warning colour, and a
   dismiss that means "not this release" rather than "not ever".
   ============================================================================ */
.update-banner {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 12px;
  padding-top: calc(10px + env(safe-area-inset-top, 0px));
  background: var(--color-brand-primary, #1B2A6B);
  color: #fff;
}

.update-banner__text {
  display: flex;
  flex-direction: column;
  gap: 1px;
  min-width: 0;
  flex: 1;
}

.update-banner__title {
  font-size: 14px;
  font-weight: 700;
  line-height: 1.25;
}

.update-banner__notes {
  font-size: 12.5px;
  line-height: 1.3;
  color: rgba(255, 255, 255, 0.82);
}

.update-banner__go {
  flex: none;
  display: inline-flex;
  align-items: center;
  min-height: 36px;
```

```
  padding: 0 14px;
  border-radius: 999px;
  background: #fff;
  color: var(--color-brand-primary, #1B2A6B);
  font-size: 13px;
  font-weight: 800;
  text-decoration: none;
  white-space: nowrap;
}

.update-banner__close {
  flex: none;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 32px;
  height: 32px;
  border: 0;
  border-radius: 999px;
  background: transparent;
  color: rgba(255, 255, 255, 0.85);
  cursor: pointer;
}
.update-banner__close:hover { background: rgba(255, 255, 255, 0.15); }
```

Part 2 — Backend, Firebase and Administrative Portal

2.1 Backend Server and API

2.1.1 API-001 — server.ts

Code ID	API-001
File Name	server.ts
File Path	SanctDemo/server.ts
Language / Type	TypeScript
Lines	1080
Functions / Classes	getLanIps (internal function); requireAppKey (internal function); clientIp (internal function); devBackstopAllows (internal function); parishName (internal function); requireParish (internal function); getAi (internal function)
Related Files (its imports)	./src/lib/modelRetry, ./src/lib/scanQuota, ./src/lib/parishContentStore, ./src/lib/sanctiGrounding, ./src/data
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/server.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — server.ts

```
import express from "express";
import path from "path";
import fs from "fs";
import os from "os";
import https from "https";
import { createServer as createViteServer } from "vite";
import { GoogleGenAI, Type, ThinkingLevel } from "@google/genai";
import dotenv from "dotenv";
import nodemailer from "nodemailer";
import { ATTEMPT_TIMEOUT_MS, askWithRetry, isModelBusy, isTimeout, retryPlan } from "./src/lib/modelRetry";
import { emptyBudget, isScannableParish, spentMessage, type ScanBudget } from "./src/lib/scanQuota";
import { readParishContent } from "./src/lib/parishContentStore";
import { groundingBlock, isGap, itemFacts, massFact, type Fact, type Gap } from "./src/lib/sanctiGrounding";
import { MASS_SCHEDULES } from "./src/data";
import {
  initQuotaStore, quotaStoreReady, readBudget, refundScan, spendScan, uidFromToken,
} from "./src/lib/scanQuotaStore";

dotenv.config();

const app = express();
const PORT = Number(process.env.PORT) || 3000;

// getUserMedia (the AR Tour camera) only works in a "secure context" — https,
```

```
// or the literal hostname `localhost`. A phone on the church wifi visiting
// http://192.168.x.x:5173 gets `navigator.mediaDevices === undefined` before
// any permission prompt ever shows. HTTPS is how a phone on the LAN gets in.
//
// This is opt-in (HTTPS=1 npm run dev, or `npm run dev:https`) so the default
// `npm run dev` / http://localhost:5173 workflow is unchanged and needs no
// certificate. The cert is self-signed and generated on first run with the
// `selfsigned` package (pure JS, no external binary, no account) — see
// docs/CAMERA-SETUP.md for what the resulting phone warning means.
const USE_HTTPS = process.env.HTTPS === "1" || process.env.HTTPS === "true";
const CERT_DIR = path.join(process.cwd(), ".cert");
const CERT_PATH = path.join(CERT_DIR, "cert.pem");
const KEY_PATH = path.join(CERT_DIR, "key.pem");

async function getOrCreateCert(): Promise<{ key: string; cert: string }> {
  if (fs.existsSync(CERT_PATH) && fs.existsSync(KEY_PATH)) {
    return { key: fs.readFileSync(KEY_PATH, "utf8"), cert: fs.readFileSync(CERT_PATH, "utf8") };
  }

  // Dynamic import: `selfsigned` is a devDependency, only ever touched when a
  // developer opts into HTTPS locally — a production install must not need it.
  const selfsigned = await import("selfsigned");
  const lanIps = getLanIps();
  const altNames: Array<{ type: 2 | 7; value?: string; ip?: string }> = [
    { type: 2, value: "localhost" }, // DNS
    { type: 7, ip: "127.0.0.1" }, // IP
    ...lanIps.map((ip) => ({ type: 7 as const, ip })),
  ];
  const notAfterDate = new Date();
  notAfterDate.setDate(notAfterDate.getDate() + 825);
  const pems = await selfsigned.generate([{ name: "commonName", value: "localhost" }], {
    notAfterDate,
    keySize: 2048,
    extensions: [{ name: "subjectAltName", altNames }],
  });

  fs.mkdirSync(CERT_DIR, { recursive: true });
  fs.writeFileSync(KEY_PATH, pems.private);
  fs.writeFileSync(CERT_PATH, pems.cert);
  return { key: pems.private, cert: pems.cert };
}

function getLanIps(): string[] {
  const ips: string[] = [];
  const interfaces = os.networkInterfaces();
  for (const name of Object.keys(interfaces)) {
    for (const iface of interfaces[name] ?? []) {
      if (iface.family === "IPv4" && !iface.internal) ips.push(iface.address);
    }
  }
  return ips;
}

// 12mb, not the 100kb express.json() defaults to.
//
// The camera posts a base64 JPEG, and base64 inflates a payload by about a
// third. At the old default a 640px frame of a plain subject squeaked
// through at 65kb, while the same 640px frame of a *detailed* scene — a
// flower-covered carroza, stained glass, a crowded nave — compressed worse,
```

```
// crossed 100kb, and was rejected with a 413 that the dev server answered
// with an HTML error page. The client then failed to parse it as JSON and
// reported "Could not reach the server", so a size limit surfaced as a
// network fault, intermittently, depending on what the camera was pointed at.
// Cross-origin access for the packaged app.
//
// On the web SanctiWalk is served BY this server, so every /api call is
// same-origin and CORS never enters into it. Inside the APK it is not:
// Capacitor serves the bundle from https://localhost and this server is a
// different origin, so the browser demands CORS headers and silently
// refuses the request without them. That is why the scanner failed on the
// phone with the endpoint plainly reachable - the request arrived and the
// WebView threw the answer away.
//
// The same applies to the deployed backend, where the app's origin is
// still https://localhost, so this is not tunnel-specific plumbing.
//
// The allowlist is the app's own origins plus localhost and private-LAN
// dev servers. It is not `*`, because these endpoints spend a real Gemini
// quota and the APP_KEY header is what stands between them and anyone who
// finds the URL; a wildcard would let any web page in the world use a
// key lifted from the bundle.
const APP_ORIGINS = new Set([
  "https://localhost",      // Capacitor, Android
  "capacitor://localhost",  // Capacitor, iOS
  "http://localhost",
]);

const isAllowedOrigin = (origin: string) =>
  APP_ORIGINS.has(origin) ||
  /^https?:\/\/localhost:\d+$/.test(origin) ||
  /^https?:\/\/(?:192\.168|10)\.\d{1,3}\.\d{1,3}(?::\d+)?$/.test(origin);

app.use("/api", (req, res, next) => {
  const origin = req.headers.origin;
  if (typeof origin === "string" && isAllowedOrigin(origin)) {
    res.setHeader("Access-Control-Allow-Origin", origin);
    res.setHeader("Vary", "Origin");
    // Authorization belongs here because every scan now carries a
    // Firebase ID token. Leaving it out does not fail loudly: the
    // preflight is refused, the browser discards a response the server
    // happily produced, and the app reports "could not reach the
    // server" for a request the log shows arriving and being answered.
    res.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization, X-SanctiWalk-Key");
    res.setHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
    res.setHeader("Access-Control-Max-Age", "86400");
  }
  // Preflight needs an answer before any body parsing or auth.
  if (req.method === "OPTIONS") {
    res.sendStatus(204);
    return;
  }
  next();
});

app.use(express.json({ limit: "12mb" }));

// Lazy-initialize Gemini API
/**
```

```
 * Recognition is capped per day.
 *
 * Counted here rather than in the browser because what is actually scarce is
 * the free-tier allowance on the single API key this server holds — shared by
 * every device using the app. A per-device counter in localStorage would both
 * over- and under-report, and anyone could clear it.
 *
 * In memory, deliberately: a server restart resets the count. That is the
 * right trade for a capstone demo, and it is stated rather than pretended
 * otherwise.
 */
/**
 * The vision model used for camera recognition.
 *
 * Measured on the same 24KB frame, all returning the same correct
 * identification with the full summary and highlights:
 *
 *   gemini-3.5-flash      + LOW thinking      20.7s
 *   gemini-3.5-flash      + MINIMAL thinking  13.7s
 *   gemini-3.5-flash-lite + MINIMAL thinking   1.5-1.9s
 *
 * A scanner that takes twenty seconds is not a scanner, so flash-lite is the
 * default. It is a smaller model, so if a particular statue or retablo is
 * being misidentified, GEMINI_VISION_MODEL=gemini-3.5-flash in .env switches
 * back and trades the speed for the larger model's judgement.
 *
 * The two models also hold separate free-tier quotas, which is worth knowing
 * when recognition starts refusing: exhausting one does not exhaust the other.
 */
const VISION_MODEL = process.env.GEMINI_VISION_MODEL || "gemini-3.5-flash-lite";

/**
 * Where recognition goes when the primary model is congested.
 *
 * Deliberately NOT another "-lite": a 503 means that model's serving pool is
 * saturated, so retrying inside the same pool is the one thing guaranteed not
 * to help. The two models also hold separate free-tier quotas (see above), so
 * falling back neither borrows nor exhausts the primary's allowance.
 */
const VISION_MODEL_FALLBACK = process.env.GEMINI_VISION_MODEL_FALLBACK || "gemini-3.5-flash";

/**
 * Shared key the app sends on every /api call.
 *
 * Enforced ONLY when APP_KEY is set in this server's environment, so local
 * development needs no configuration and nothing that works today breaks.
 * Set it on the deployed host, where the URL is public and /api/identify
 * spends a real Gemini quota.
 *
 * This is a speed bump, not authentication: the key ships inside the app and
 * can be extracted from the APK. It stops bots and passers-by, which is the
 * realistic threat. The per-IP cap below bounds what one determined person
 * can spend.
 */
const APP_KEY = process.env.APP_KEY || "";
const APP_KEY_HEADER = "x-sanctiwalk-key";

function requireAppKey(req: express.Request, res: express.Response): boolean {
  if (!APP_KEY) return true;
```

```
  const provided = req.get(APP_KEY_HEADER);
  if (provided === APP_KEY) return true;

  console.warn(`[Auth] rejected ${req.method} ${req.path} from ${clientIp(req)} - bad or missing key`);
  res.status(401).json({ error: "This build is not authorised to use the SanctiWalk service." });
  return false;
}

/**
 * The daily allowance, per signed-in user and per parish.
 *
 * This used to count against the caller's IP address, which is not a
 * person. Everyone on a church's Wi-Fi shared one address and therefore
 * one allowance, so the first visitor through the door could spend the
 * whole room's; step outside onto mobile data and you became somebody
 * new with a fresh twenty. Both halves were wrong.
 *
 * It is counted against the Firebase account now, in Firestore, with
 * the read and the write in one transaction so two requests racing
 * cannot both be told they were the last one. The rules themselves live
 * in src/lib/scanQuota.ts, which the app imports too, so what the
 * screen promises and what the server enforces cannot drift apart.
 *
 * The consequence for a visitor is real and worth stating: scanning now
 * requires signing in. Everything else in SanctiWalk still works
 * signed-out.
 */
function clientIp(req: express.Request): string {
  // Render puts the real address in x-forwarded-for; req.ip would be the
  // load balancer. Only used for logging now.
  const forwarded = req.get("x-forwarded-for");
  if (forwarded) return forwarded.split(",")[0].trim();
  return req.ip || "unknown";
}

/**
 * Lets a developer run the scanner with no Firebase credentials.
 *
 * Off unless asked for, and refused outright in production so it can
 * never be the thing that makes a deployed server unlimited. Even when
 * on, the backstop below keeps a hard per-IP ceiling, because "my
 * laptop" and "reachable from the internet" are not always as different
 * as the person running it believes.
 */
const UNCOUNTED_OK =
  process.env.ALLOW_UNCOUNTED_SCANS === "1" && process.env.NODE_ENV !== "production";

const DEV_BACKSTOP_LIMIT = Number(process.env.DEV_SCAN_LIMIT) || 40;
let devBackstopDay = "";
const devBackstopByIp = new Map<string, number>();

function devBackstopAllows(req: express.Request, res: express.Response): boolean {
  const today = new Date().toISOString().slice(0, 10);
  if (devBackstopDay !== today) {
    devBackstopDay = today;
    devBackstopByIp.clear();
  }
  const ip = clientIp(req);
```

```
  const used = devBackstopByIp.get(ip) ?? 0;
  if (used >= DEV_BACKSTOP_LIMIT) {
    res.status(429).json({
      error: "This development server's scan ceiling is spent. Restart it, or configure Firebase.",
      code: "daily_limit",
    });
    return false;
  }
  devBackstopByIp.set(ip, used + 1);
  return true;
}

/**
 * The signed-in user behind a request, or null after answering 401.
 *
 * The uid comes from a token Google's keys verify, never from the body,
 * so a client cannot claim to be somebody else or invent a fresh
 * identity per scan.
 */
async function requireUser(
  req: express.Request,
  res: express.Response,
): Promise<string | null> {
  if (!quotaStoreReady()) {
    // Fail CLOSED. The first version returned a shared "anonymous" uid
    // here, which meant a deploy that simply forgot
    // FIREBASE_SERVICE_ACCOUNT served an unlimited, unauthenticated
    // scanner to anyone who found the URL - and the per-IP cap that
    // used to bound that had just been removed. A missing environment
    // variable is the likeliest mistake of the whole deployment, so it
    // has to be the safe one.
    //
    // Local development opts out explicitly, and production may not:
    // see UNCOUNTED_OK.
    if (!UNCOUNTED_OK) {
      res.status(503).json({
        error: "The scanner is not available right now. Please try again later.",
        code: "quota_unavailable",
      });
      return null;
    }
    if (!devBackstopAllows(req, res)) return null;
    return `dev:${clientIp(req)}`;
  }

  const uid = await uidFromToken(req.get("authorization"));
  if (uid) return uid;

  res.status(401).json({
    error: "Sign in to use the AI scanner. Your daily scans are counted against your account.",
    code: "sign_in_required",
  });
  return null;
}

/** A parish's name, for messages a visitor reads. */
function parishName(parishId: string): string {
  return parishId === "route-src"
    ? "San Roque Cathedral Parish"
```

```
    : "Mary Help of Christians Parish";
}

/** The parish a request is for, or null after answering 400. */
function requireParish(req: express.Request, res: express.Response): string | null {
  const parishId = (req.body?.parishId ?? req.query?.parishId) as unknown;
  if (isScannableParish(parishId)) return parishId;

  res.status(400).json({
    error: "The scanner needs to know which parish you are visiting.",
    code: "parish_required",
  });
  return null;
}

/** Open-ended recognition, used when there is no station shortlist. */
const OPEN_INSTRUCTION = [
  "Identify the main subject of this camera frame.",
  "",
  "Name it as specifically as you can — a particular statue, altar, window,",
  "artwork, or building rather than a generic category. Ignore hands and the",
  "person holding the camera. If the frame is too blurry, too dark, or shows",
  "nothing identifiable, set recognized to false.",
  "",
  "Do not invent parish-specific history, dates, donors, or names. If you do",
  "not know, describe what is visible instead.",
  "",
  "If the subject depicts a saint, a blessed, or Our Lady under a title, also",
  "fill saintName and aboutSaint, and - only when you are certain - feastDay",
  "and patronage. Someone standing in front of a statue can already see the",
  "statue; what they cannot see is who it is of. Leave feastDay and patronage",
  "empty rather than guessing.",
].join("\n");

let aiClient: GoogleGenAI | null = null;
function getAi(): GoogleGenAI {
  if (!aiClient) {
    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey || apiKey === "MY_GEMINI_API_KEY") {
      throw new Error("GEMINI_API_KEY environment variable is not configured. Please add it to Secrets.");
    }
    aiClient = new GoogleGenAI({
      apiKey: apiKey,
      httpOptions: {
        headers: {
          'User-Agent': 'aistudio-build',
        }
      }
    });
  }
  return aiClient;
}

// Full-stack API Endpoint: Generate Custom Walk Guide with Gemini
app.post("/api/generate-walk", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  try {
    const { interest, location, durationMinutes } = req.body;
```

```
    if (!interest || !location) {
      return res.status(400).json({ error: "Missing interest or location parameters." });
    }

    const duration = durationMinutes ? parseInt(durationMinutes) : 30;
    const prompt = `Create a custom historic/heritage/spiritual walking tour guide in ${location} based on the user's interest in "${interest}" that lasts roughly
${duration} minutes. Give it an inspiring name, descriptive intro, and exactly 2-3 stops (stations) with detailed historical significance and a personal
wellness/reflection question at each stop. Return the data structured in the requested JSON format. Ensure all coordinate-based offsets are logical.`;

    const ai = getAi();
    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash",
      contents: prompt,
      config: {
        systemInstruction: "You are an expert cultural guide, historian, and wellness walking coach. You design engaging walks that combine history, heritage, and
mindfulness.",
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            name: {
              type: Type.STRING,
              description: "An inspiring, elegant name for this walk."
            },
            description: {
              type: Type.STRING,
              description: "A summary of what makes this walk unique and relaxing."
            },
            distanceKm: {
              type: Type.NUMBER,
              description: "Estimated distance in kilometers (e.g. 1.8)."
            },
            durationMins: {
              type: Type.INTEGER,
              description: "Estimated walking time in minutes."
            },
            estimatedSteps: {
              type: Type.INTEGER,
              description: "Estimated steps (e.g. 2400)."
            },
            stations: {
              type: Type.ARRAY,
              description: "The stops on this walking guide.",
              items: {
                type: Type.OBJECT,
                properties: {
                  name: {
                    type: Type.STRING,
                    description: "Name of the stop/station."
                  },
                  description: {
                    type: Type.STRING,
                    description: "A friendly 1-2 sentence description of what the user sees here."
                  },
                  history: {
                    type: Type.STRING,
                    description: "The historical context, heritage significance, or architectural details of this spot."
```

```
                },
                reflection: {
                  type: Type.STRING,
                  description: "A mindfulness or spiritual question for the user to contemplate while at this spot."
                },
                audioDuration: {
                  type: Type.STRING,
                  description: "Estimated audio guide duration, e.g., '2:30'"
                }
              },
              required: ["name", "description", "history", "reflection", "audioDuration"]
            }
          }
        },
        required: ["name", "description", "distanceKm", "durationMins", "estimatedSteps", "stations"]
      }
    }
  });

  const resultText = response.text;
  if (!resultText) {
    throw new Error("Empty response from AI.");
  }

  const walkData = JSON.parse(resultText);
  res.json(walkData);
 } catch (error: any) {
  console.error("Gemini Walk Generation failed:", error);
  // Return mock fallback walk data so that it still functions gracefully if key is missing!
  res.json({
    name: `Custom ${req.body.interest || "Peaceful"} Walk`,
    description: `A simulated custom walk created for ${req.body.location || "your location"} focusing on ${req.body.interest || "mindfulness"}. (Note: Connect
Gemini API Key to enable real AI generation).`,
    distanceKm: 1.8,
    durationMins: req.body.durationMinutes ? parseInt(req.body.durationMinutes) : 30,
    estimatedSteps: 2500,
    isFallback: true,
    stations: [
      {
        name: "The Garden of Tranquility",
        description: "A peaceful outdoor green space to settle your thoughts.",
        history: "This location represents the timeless tradition of cultivating spaces of retreat and natural beauty amid academic and community settings.",
        reflection: "Close your eyes and listen to three distinct sounds. What are they telling you about the present moment?",
        audioDuration: "1:45"
      },
      {
        name: "The Archway of Legacy",
        description: "A beautiful structural landmark symbolizing transition, passage, and academic milestone achievements.",
        history: "Historically, stone arches have marked the boundaries of cities and learning institutions, commemorating those who passed through to build the
future.",
        reflection: "As you walk through boundaries today, what old habits are you leaving behind, and what new paths are you embracing?",
        audioDuration: "2:15"
      }
    ]
  });
 }
});

// Full-stack API Endpoint: Identify what the camera is looking at (AR Tour)
```

```
//
// The API key stays on the server — the browser only ever sends an image and
// receives text, so nothing sensitive reaches the client bundle.
//
// `candidates` is the accuracy lever. A vision model is weak at open-ended
// "what specific statue is this", but strong at "which of these five stations
// is this". When the app knows which parish the pilgrim is in, it should send
// that parish's station names and turn recognition into a multiple choice.
app.post("/api/identify", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  const uid = await requireUser(req, res);
  if (!uid) return;
  const parishId = requireParish(req, res);
  if (!parishId) return;

  // Reserved here, before the model is asked. Checking first and
  // charging afterwards lets a client that aborts mid-request scan for
  // ever; the refund below covers the failures that are genuinely ours.
  let spent = false;
  let budget: ScanBudget;

  try {
    const attempt = quotaStoreReady()
      ? await spendScan(uid, parishId)
      // The development path, and only reachable through it, because
      // requireUser has already answered 503 for everybody else. There
      // is no store to count in, so the per-IP ceiling it already
      // charged is what bounds this.
      : { allowed: true, budget: emptyBudget(parishId) };

    budget = attempt.budget;
    if (!attempt.allowed) {
      return res.status(429).json({
        error: spentMessage(parishName(parishId)),
        code: "daily_limit",
        budget,
      });
    }
    spent = quotaStoreReady();
  } catch (err) {
    console.error("[Quota] could not reserve a scan:", (err as Error).message);
    return res.status(503).json({
      error: "The scanner could not check your remaining scans. Please try again.",
      code: "quota_unavailable",
    });
  }

  try {
    const { imageBase64, mimeType, candidates } = req.body ?? {};

    if (typeof imageBase64 !== "string" || imageBase64.length < 512) {
      console.error(
        `[Scanner] Scanner error: rejected payload — imageBase64 was ${
          typeof imageBase64 === "string" ? `${imageBase64.length} chars` : typeof imageBase64
        }`,
      );
      return res.status(400).json({
        error: "The captured image did not arrive properly. Please try scanning again.",
```

```
  });
}

// Base64 is 4 chars per 3 bytes, so this is the decoded size.
const imageBytes = Math.round(imageBase64.length * 0.75);
console.log(`[Scanner] API request: ${(imageBytes / 1024).toFixed(0)} KB image, model ${VISION_MODEL}`);

const shortlist: string[] = Array.isArray(candidates) ? candidates.filter(Boolean) : [];

const instruction = shortlist.length
  ? [
      "You are identifying a feature inside a specific Catholic parish church.",
      "",
      "The pilgrim is at a station that is one of the following:",
      ...shortlist.map((name: string) => `- ${name}`),
      "",
      "Look carefully at the whole frame before answering.",
      "",
      "If the photo shows one of them, name that station and set",
      "matchedStation to true. The list is the strongest evidence you",
      "have about what this object is, so prefer a listed station over a",
      "guess when the image is consistent with it.",
      "",
      "If it shows none of them, DO NOT set recognized to false. Identify the",
      "subject anyway — as specifically as you can — and set matchedStation to",
      "false. A pilgrim pointing a camera at something outside the tour still",
      "deserves an answer.",
      "",
      "Set recognized to false only when the frame is too blurry or dark, or",
      "shows nothing identifiable at all.",
      "",
      "In `summary`, say what you actually saw that led to the answer, so",
      "the pilgrim can check it against what is in front of them. Set",
      "`confidence` honestly, and when the frame is unusable put a short",
      "practical instruction in `advice`.",
      "",
      "Only state facts you can see in the image or that are common knowledge",
      "about the subject. Do not invent parish-specific history, dates, donors,",
      "or names — if you do not know, say what is visible instead.",
      "",
      "If the subject depicts a saint, a blessed, or Our Lady under a title,",
      "also fill saintName, aboutSaint, and - only when you are certain -",
      "feastDay and patronage. `summary` explains what you saw; `aboutSaint`",
      "says who they were. Someone standing in front of a statue can already",
      "see the statue, so a description of it teaches them nothing. Leave",
      "feastDay and patronage empty rather than guessing: a wrong feast day",
      "sends somebody to church on the wrong morning.",
    ].join("\n")
  : OPEN_INSTRUCTION;

const ai = getAi();
const askModel = (prompt: string, model: string = VISION_MODEL, timeoutMs: number = ATTEMPT_TIMEOUT_MS) => ai.models.generateContent({
  model,
  contents: [
    {
      role: "user",
      parts: [
        { inlineData: { mimeType: mimeType || "image/jpeg", data: imageBase64 } },
        { text: prompt },
```

```
          ],
        },
      ],
      config: {
        // Bounds a single attempt. Without it a call that is accepted and
        // then never answered — observed running past two minutes against
        // this very endpoint — holds the request open indefinitely.
        abortSignal: AbortSignal.timeout(timeoutMs),
        // Recognition is perception, not deliberation. MINIMAL, not LOW:
        // measured on the same frame, LOW spent 275 thinking tokens and 7
        // extra seconds to reach the identical answer. Note that omitting
        // thinkingConfig entirely does NOT disable thinking — it re-enables
        // it at the model's own default, which measured worse than MINIMAL.
        thinkingConfig: { thinkingLevel: ThinkingLevel.MINIMAL },
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            recognized: {
              type: Type.BOOLEAN,
              description: "False when the frame shows nothing identifiable.",
            },
            title: {
              type: Type.STRING,
              // This field's own description used to read "the subject's
              // specific name", which pulled against the instruction above
              // and kept producing confident wrong devotions — a schema
              // description is an instruction to the model too.
              description:
                "What the subject is. Use a specific saint or devotion ONLY when the image shows attributes that distinguish it (an inscription, name plate, or
unique emblem). Otherwise use an accurate general description such as 'Crowned Marian image with the Child Jesus'. Empty when not recognized.",
            },
            category: { type: Type.STRING, description: "Short type label: Statue, Altar, Window, Painting, Relic, Architecture." },
            confidence: {
              type: Type.NUMBER,
              description:
                "0 to 1. Be honest. Use 0.9+ only when an inscription or unique emblem confirms the identification; use 0.4-0.6 for a general category recognised
without distinguishing marks.",
            },
            summary: {
              type: Type.STRING,
              description:
                "2-4 sentences for someone standing in front of it. Begin by naming the visual evidence you used — the crown, vestment colour, emblem, inscription —
then what it is. Say plainly if you are unsure.",
            },
            highlights: {
              type: Type.ARRAY,
              items: { type: Type.STRING },
              description: "Three short standalone facts.",
            },
            advice: {
              type: Type.STRING,
              description:
                "When recognized is false: one short, practical instruction for the pilgrim, e.g. move closer or improve the lighting. Empty otherwise.",
            },
            matchedStation: {
              type: Type.BOOLEAN,
              description:
                "True when the subject is one of the listed stations. False when it was identified but is not on the list. Ignored when no list was given.",
```

```
            },
            saintName: {
              type: Type.STRING,
              description:
                "When the subject depicts a saint, blessed, or Our Lady under a title, their full common English name, e.g. 'Saint Francis of Assisi' or 'Our Lady of
Perpetual Help'. Empty string when the subject is not a holy figure.",
            },
            feastDay: {
              type: Type.STRING,
              description:
                "That saint's feast day in the Roman calendar, e.g. 'October 4'. Leave EMPTY if you are not certain - a wrong feast day sends someone to church on
the wrong morning. Never guess.",
            },
            patronage: {
              type: Type.STRING,
              description:
                "What they are the patron of, in a few words, e.g. 'animals, ecology and merchants'. Leave EMPTY if unsure. Never guess.",
            },
            aboutSaint: {
              type: Type.STRING,
              description:
                "3-5 sentences on who this saint actually was and why they are venerated: when and where they lived, what they are known for, and what their usual
emblems mean. This is general Catholic knowledge about the person, NOT a description of the object in the photo - the pilgrim can already see the object. Write it in
your own words. Empty string when the subject is not a holy figure.",
            },
          },
          required: ["recognized", "title", "category", "confidence", "summary", "highlights"],
        },
      },
    });

    const startedAt = Date.now();
    // Retry sequencing lives in src/lib/modelRetry.ts, where it is unit
    // tested — the behaviour that matters (three tries, then a different
    // model, and NO retry for a bad key or a spent quota) is not something
    // to verify by re-reading.
    const { value: response, model: answeringModel, attempt } = await askWithRetry(
      retryPlan(VISION_MODEL, VISION_MODEL_FALLBACK),
      (model, timeoutMs) => askModel(instruction, model, timeoutMs),
      {
        onBusy: (model, n) => console.warn(`[Scanner] ${model} did not answer (attempt ${n}) — retrying`),
      },
    );
    if (attempt > 1) console.log(`[Scanner] Recovered on attempt ${attempt} using ${answeringModel}`);

    // Logged so "the scanner feels slow" can be answered with a number
    // instead of a guess — which is how the 20s-to-1.5s fix was found.
    console.log(`[Scanner] API response: ${answeringModel} answered in ${Date.now() - startedAt}ms`);
    const text = response.text;
    if (!text) {
      return res.status(502).json({ error: "The vision model returned no content." });
    }

    const result = JSON.parse(text);

    // The station shortlist used to instruct the model to answer "none of
    // these" for anything that was not one of the parish's own stations —
    // which is most of what a pilgrim will point a camera at. The identical
    // frame that returned a confident "Our Lady of Grace" with no shortlist
```

```
  // returned recognized:false with one, so the scanner looked broken the
  // moment you were standing at a parish.
  //
  // The prompt now asks for both jobs in one pass: match a station if the
  // photo shows one, otherwise identify the subject anyway and say it isn't
  // on the list. A two-call fallback would also have worked, but it spent
  // two of the day's twenty recognitions on every miss.
  if (!shortlist.length) delete result.matchedStation;

  return res.json({ ...result, budget });
} catch (error: any) {
  const message = String(error?.message ?? error);

  // The free tier allows a small number of requests per model per day. Say so
  // plainly rather than surfacing a raw stack trace to a pilgrim.
  if (message.includes("429") || message.toUpperCase().includes("RESOURCE_EXHAUSTED")) {
    if (spent) await refundScan(uid, parishId);
    return res.status(429).json({
      error: "The recognition service is busy. Please try again in a little while.",
      code: "quota_exhausted",
      budget: await readBudget(uid, parishId),
    });
  }

  // The real error goes to the console; the pilgrim gets something they can
  // act on. An API key problem in particular is a configuration fault, not
  // something a pilgrim can fix by trying again, so it says so.
  console.error("[Scanner] Scanner error:", message);

  // Everything below is the server, the key or the model failing - none
  // of it is the visitor's doing, so they get the scan back. A
  // photograph the model simply could not name is NOT refunded: that is
  // a real recognition, and refunding it would make the allowance
  // unlimited for anyone pointing the camera at a wall.
  if (spent) await refundScan(uid, parishId);

  if (/API key|API_KEY|PERMISSION_DENIED|UNAUTHENTICATED/i.test(message)) {
    return res.status(500).json({
      error: "Recognition is not configured on the server. Check GEMINI_API_KEY.",
      code: "auth",
    });
  }
  if (/deadline|timeout|ETIMEDOUT|ECONNRESET|fetch failed|ENOTFOUND/i.test(message)) {
    return res.status(504).json({
      error: "Recognition timed out. Check your connection and try again.",
      code: "timeout",
    });
  }
  // Congestion, after every retry and the fallback model have been tried.
  // It gets its own code and its own words because the remedy — wait a
  // moment, tap again — is completely different from "your photo was bad",
  // which is what the generic message below implies.
  if (isModelBusy(message) || isTimeout(message)) {
    return res.status(503).json({
      error:
        "The recognition service is busy right now. This is on their side, not yours — wait a few seconds and tap scan again.",
      code: "model_busy",
    });
  }
```

```
    return res.status(500).json({ error: "Recognition failed. Please try again." });
  }
});

// Is this server up? Nothing more.
//
// The host's health check has to reach this without a key: it is an
// anonymous GET from the platform, and a check that 401s reads as a dead
// service - the deploy is then held back and never goes live. render.yaml
// used to point at /api/identify/budget, which is key-guarded, so setting
// APP_KEY would have failed every deploy from then on.
//
// It answers only "yes", with no quota, version or configuration in it, so
// there is nothing here worth guarding.
app.get("/api/health", (_req, res) => {
  res.json({ ok: true });
});

/**
 * Ask Sancti, grounded in the parish's own records.
 *
 * ## Why the model never answers alone
 *
 * A language model asked "what time is Mass at San Roque" will answer.
 * It will produce a time, in the right format, with no sign that it was
 * invented - which for the one question a pilgrim acts on by leaving
 * the house at a particular hour is the worst possible failure. So the
 * model is never the source of a fact here. Every parish fact is read
 * from Firestore first, handed over as a VERIFIED FACTS block, and the
 * model's only job is to put it into a sentence in the pilgrim's own
 * language.
 *
 * Gaps are passed in as explicitly as facts. A model given only facts
 * fills a silence, and what it fills it with is a plausible Mass time;
 * told in writing that the schedule is not published, saying so becomes
 * the expected answer rather than a failure to answer.
 *
 * ## Why failure is a clean 503
 *
 * The client keeps its on-device matcher and falls back to it whenever
 * this endpoint cannot help. An assistant whose first act is to fail at
 * a dead URL is worse than no assistant, which is the reason Sancti was
 * built on-device in the first place; that guarantee is kept by saying
 * so clearly rather than by returning something invented.
 */
const ASK_SYSTEM_INSTRUCTION = [
  "You are Sancti, a warm and respectful guide inside the SanctiWalk app for the Diocese of Kalookan.",
  "",
  "ABSOLUTE RULES:",
  "1. Parish facts - Mass times, schedules, requirements, contact details, fees, policies, events - may ONLY come from the VERIFIED FACTS block. Never infer,
estimate or recall them.",
  "2. If something is listed under NOT AVAILABLE, say plainly that it is not published in the app and suggest contacting the parish office. Never fill the gap.",
  "3. If a fact is marked as SanctiWalk app data rather than the parish office, say it is the app's information and may not be the parish's latest.",
  "4. If a parish fact is old, mention when the parish last updated it.",
  "5. General Catholic teaching, prayers, the Rosary, the meaning of sacraments and traditions you may answer from your own knowledge - but label it as general
Catholic guidance, kept separate from this parish's own information.",
  "6. Never invent a parish's history, a saint's connection to it, a phone number, a price, or an event date.",
  "7. Never claim a parish offers a service unless the VERIFIED FACTS say so.",
```

```
  "8. Never mention buttons, screens or features that were not named in the context you were given.",
  "",
  "STYLE: answer the question first, in two or three sentences, then any useful detail. Reply in the language the user wrote in - English or Filipino. Warm, plain,
never preachy.",
].join("\n");

app.post("/api/ask", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  const question = typeof req.body?.question === "string" ? req.body.question.trim() : "";
  if (!question) return res.status(400).json({ error: "Missing question." });
  // A guard, not a limit on curiosity: the grounding block and the
  // recent conversation share a context window with it.
  if (question.length > 600) return res.status(400).json({ error: "Question is too long." });

  const parishId = typeof req.body?.parishId === "string" ? req.body.parishId : "";
  const parishName =
    typeof req.body?.parishName === "string" && req.body.parishName.trim()
      ? req.body.parishName.trim()
      : "this parish";

  try {
    const ai = getAi();

    // Retrieval first. Everything the answer may assert is gathered
    // here, from the parish's own document where it has one.
    const content = parishId ? await readParishContent(parishId) : null;
    const facts: Fact[] = [];
    const gaps: Gap[] = [];

    if (parishId) {
      const mass = massFact(content, MASS_SCHEDULES[parishId]?.schedule);
      if (isGap(mass)) gaps.push(mass); else facts.push(mass);

      if (content?.description?.trim()) {
        facts.push({ topic: "description", value: content.description.trim(), source: "parish", updatedAt: content.updatedAt });
      }
      if (content?.historyBody?.trim()) {
        facts.push({ topic: "history", value: content.historyBody.trim(), source: "parish", updatedAt: content.updatedAt });
      } else {
        gaps.push({ topic: "history", reason: "This parish has not published its history in the app." });
      }

      // Only the sacraments and ministries the question actually reaches
      // for. Sending all of them would crowd the window and bury the one
      // fact the pilgrim asked about.
      const asked = question.toLowerCase();
      const TOPICS: Array<[string, string, string[]]> = [
        ["sac-baptism", "Baptism", ["bapti", "binyag", "christen"]],
        ["sac-wedding", "Wedding", ["wedding", "marriage", "kasal", "ikakasal"]],
        ["sac-confirmation", "Confirmation", ["confirmation", "kumpil"]],
        ["sac-eucharist", "Holy Communion", ["communion", "eucharist", "komunyon"]],
        ["min-choir", "Choir", ["choir", "koro", "sing"]],
        ["min-altar-servers", "Altar Servers", ["altar server", "sakristan", "acolyte"]],
      ];
      for (const [id, label, words] of TOPICS) {
        if (!words.some(w => asked.includes(w))) continue;
        const { facts: f, gaps: g } = itemFacts(content, id, label);
        facts.push(...f);
```

```
        gaps.push(...g);
      }
    }

    const context = parishId
      ? groundingBlock({ parishId, parishName, facts, gaps })
      : [
          "NO PARISH SELECTED.",
          "You have no verified parish information for this question.",
          "If the question needs parish-specific facts, ask which parish they mean instead of guessing.",
        ].join("\n");

    // Recent turns only, and only as plain text, so the model can
    // resolve "what about tomorrow" without the whole session.
    const history = Array.isArray(req.body?.history) ? req.body.history.slice(-6) : [];
    const transcript = history
      .filter((m: unknown) => typeof (m as { text?: unknown })?.text === "string")
      .map((m: { role?: string; text: string }) => (m.role === "sancti" ? "Sancti: " : "User: ") + m.text)
      .join("\n");

    const prompt = [
      context,
      transcript ? "\nRECENT CONVERSATION:\n" + transcript : "",
      "\nUSER QUESTION: " + question,
    ].join("\n");

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash",
      contents: prompt,
      config: { systemInstruction: ASK_SYSTEM_INSTRUCTION, temperature: 0.2 },
    });

    const answer = (response.text ?? "").trim();
    if (!answer) return res.status(502).json({ error: "The assistant did not answer." });

    res.json({
      answer,
      grounded: facts.length > 0,
      // So the UI can say where this came from without re-deriving it.
      sources: facts.map(f => ({ topic: f.topic, source: f.source, updatedAt: f.updatedAt ?? null })),
      gaps: gaps.map(g => g.topic),
    });
  } catch (err) {
    // Never a fabricated fallback answer: the client has a real
    // on-device assistant to fall back to, and it is better than a
    // guess dressed up as a reply.
    console.warn("[Sancti] ask failed:", (err as Error).message);
    res.status(503).json({ error: "The assistant is unavailable." });
  }
});

// Lets the scanner show what is left before anyone spends one. Guarded:
// how much Gemini quota is left is nobody else's business.
app.get("/api/identify/budget", async (req, res) => {
  if (!requireAppKey(req, res)) return;
  const uid = await requireUser(req, res);
  if (!uid) return;
  const parishId = requireParish(req, res);
  if (!parishId) return;
```

```
  res.json(await readBudget(uid, parishId));
});

// Check if SMTP is configured for real email sending
app.get("/api/smtp-status", (req, res) => {
  const isConfigured = !!(process.env.SMTP_USER && process.env.SMTP_PASS);
  res.json({
    isConfigured,
    user: process.env.SMTP_USER || null,
    senderName: process.env.SMTP_SENDER_NAME || "Mary Help of Christians Parish Office"
  });
});

// Send real email via SMTP
app.post("/api/send-email", async (req, res) => {
  if (!requireAppKey(req, res)) return;

  try {
    const { to, subject, body } = req.body;

    if (!to || !subject || !body) {
      return res.status(400).json({ error: "Missing 'to', 'subject', or 'body' parameter." });
    }

    const smtpHost = process.env.SMTP_HOST || "smtp.gmail.com";
    const smtpPort = parseInt(process.env.SMTP_PORT || "587");
    const smtpUser = process.env.SMTP_USER;
    const smtpPass = process.env.SMTP_PASS;
    const senderName = process.env.SMTP_SENDER_NAME || "Mary Help of Christians Parish Office";

    if (!smtpUser || !smtpPass) {
      return res.status(400).json({
        error: "SMTP_NOT_CONFIGURED",
        message: "SMTP user and password are not configured in environment secrets."
      });
    }

    const transporter = nodemailer.createTransport({
      host: smtpHost,
      port: smtpPort,
      secure: smtpPort === 465,
      auth: {
        user: smtpUser,
        pass: smtpPass,
      },
      tls: {
        rejectUnauthorized: false
      }
    });

    const mailOptions = {
      from: `"${senderName}" <${smtpUser}>`,
      to,
      subject,
      text: body,
      html: body.replace(/\n/g, "<br/>")
    };

    const info = await transporter.sendMail(mailOptions);
```

```
    console.log("Email sent successfully: %s", info.messageId);

    res.json({ success: true, messageId: info.messageId });
  } catch (error: any) {
    console.error("Failed to send email via SMTP:", error);
    res.status(500).json({ error: "SMTP_ERROR", message: error.message });
  }
});

// Setup Vite Dev Server / Static Asset Serving
async function start() {
  if (process.env.NODE_ENV !== "production") {
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    app.use(vite.middlewares);
  } else {
    const distPath = path.join(process.cwd(), "dist");
    app.use(express.static(distPath));
    app.get("*", (req, res) => {
      res.sendFile(path.join(distPath, "index.html"));
    });
  }

  // Unconditional, and at real startup rather than inside the local
  // HTTPS helper where the first version of this hid - on Render that
  // helper never runs, so the store was never initialised and every
  // request would have taken the unavailable path.
  if (initQuotaStore()) {
    console.log("[Quota] Firebase Admin ready — scans counted per user, per parish");
  } else if (UNCOUNTED_OK) {
    console.warn(
      `[Quota] No Firebase credentials. ALLOW_UNCOUNTED_SCANS is on, so scanning works `
      + `with a ${DEV_BACKSTOP_LIMIT}/day per-IP ceiling. Never do this in production.`,
    );
  } else {
    console.error(
      "[Quota] FIREBASE_SERVICE_ACCOUNT is not set — the scanner will answer 503. "
      + "Set it on the host, or set ALLOW_UNCOUNTED_SCANS=1 for local development.",
    );
  }

  const lanIps = getLanIps();

  if (USE_HTTPS) {
    const { key, cert } = await getOrCreateCert();
    https.createServer({ key, cert }, app).listen(PORT, "0.0.0.0", () => {
      console.log(`SanctiWalk Server running on https://localhost:${PORT}`);
      for (const ip of lanIps) {
        console.log(`  On your phone (same wifi): https://${ip}:${PORT}`);
      }
      console.log(
        "  Self-signed certificate — the phone will show a security warning once. " +
          "That is expected; see docs/CAMERA-SETUP.md.",
      );
    });
  } else {
    app.listen(PORT, "0.0.0.0", () => {
```

```
      console.log(`SanctiWalk Server running on http://localhost:${PORT}`);
      for (const ip of lanIps) {
        console.log(`  On your LAN: http://${ip}:${PORT} (camera will NOT work here — no HTTPS)`);
      }
      console.log("  For a working phone camera, run: npm run dev:https");
    });
  }
}

start();
```

2.2 Firebase, Database and Security Rules

2.2.1 API-002 — firebase.ts

Code ID	API-002
File Name	firebase.ts
File Path	SanctDemo/src/lib/firebase.ts
Language / Type	TypeScript
Lines	11
Functions / Classes	auth (exported constant); db (exported constant)
Related Files (its imports)	../../firebase-applet-config.json
Where It Is Used	src/App.tsx, src/admin/AdminAnalytics.tsx, src/admin/AdminAnnouncements.tsx, src/admin/AdminApp.tsx, src/admin/AdminApplicationDetail.tsx, src/admin/AdminApplications.tsx, src/admin/AdminDashboard.tsx, src/admin/ParishStatus.tsx, src/components/EditProfileCard.tsx, src/components/LoginModal.tsx, src/components/MeTab.tsx, src/components/MyApplicationsPage.tsx, src/components/NotificationsPage.tsx, src/lib/activityLog.ts …
How to Recognize It	Open SanctDemo/src/lib/firebase.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — firebase.ts

```
import { initializeApp, getApps, getApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";
import firebaseConfig from "../../firebase-applet-config.json";

// Initialize Firebase
const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApp();
export const auth = getAuth(app);
export const db = getFirestore(app, firebaseConfig.firestoreDatabaseId);

export default app;
```

2.2.2 API-003 — authFlow.ts

Code ID	API-003
File Name	authFlow.ts
File Path	SanctDemo/src/lib/authFlow.ts
Language / Type	TypeScript
Lines	262
Functions / Classes	linkSettings (internal function); AuthError (exported class); friendlyAuthError (exported function); SignUpInput (exported type); signUp (exported function); resendVerificationEmail (exported function); refreshEmailVerified (exported function); SignInResult (exported type); signIn (exported function); sendSecondFactorLink (exported function); isSecondFactorLink (exported function); completeSecondFactor (exported function); resetPassword (exported function); changePassword (exported function); signOutNow (exported function)
Related Files (its imports)	./firebase, ./userProfile
Where It Is Used	src/admin/AdminAnalytics.tsx, src/admin/AdminDashboard.tsx, src/admin/AdminLogin.tsx, src/components/LoginModal.tsx, src/routes/FinishSignIn.tsx, src/routes/SignUpFlow.tsx
How to Recognize It	Open SanctDemo/src/lib/authFlow.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — authFlow.ts

```
import {
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  sendEmailVerification,
  sendPasswordResetEmail,
  sendSignInLinkToEmail,
  isSignInWithEmailLink,
  signInWithEmailLink,
  reauthenticateWithCredential,
  EmailAuthProvider,
  updatePassword,
  signOut,
  type User,
} from "firebase/auth";
import { auth } from "./firebase";
import { createProfile, getProfile } from "./userProfile";

/**
 * Account creation, sign-in, verification and the second factor.
 *
 * ## Why the second factor is an email link and not an SMS code
 *
 * Firebase's own multi-factor authentication sends SMS, and it is only
 * available on projects upgraded to Google Cloud Identity Platform - a
 * billing change, plus a per-message cost. This project is on the Spark
```

```
 * plan, so that door is shut.
 *
 * What is open is the email sign-in link, which Firebase sends itself for
 * free. Requiring one after the password gives the real property a second
 * factor is for: a stolen password alone does not get you in, because the
 * attacker also has to be reading the inbox. It is weaker than SMS against
 * someone who has already taken over the mailbox, and that is an honest
 * trade rather than a hidden one.
 *
 * The flow is deliberately NOT "password, then optionally a link". The
 * challenge is issued before the session is treated as trusted, and
 * `pendingSecondFactor` is what the app checks - not a boolean the login
 * screen sets for itself.
 *
 * ## Why errors are translated
 *
 * Firebase's codes name internals ("auth/invalid-credential") and, worse,
 * distinguish "no such user" from "wrong password", which tells an attacker
 * which addresses are registered. Everything below collapses to messages
 * that are true, useful, and say no more than the person needs.
 */

/** Where the email link returns to. Must be an Authorized Domain in Firebase. */
function linkSettings() {
  return {
    url: `${window.location.origin}/finish-sign-in`,
    handleCodeInApp: true,
  };
}

const PENDING_EMAIL_KEY = "sanctiwalk.pendingSecondFactorEmail";

export class AuthError extends Error {
  constructor(message: string, readonly code?: string) {
    super(message);
    this.name = "AuthError";
  }
}

/**
 * A Firebase error as something worth showing a person.
 *
 * Unknown codes fall through to a generic line rather than the raw message,
 * because the raw message is written for whoever wrote the SDK.
 */
export function friendlyAuthError(err: unknown): AuthError {
  const code = typeof err === "object" && err && "code" in err ? String((err as { code: unknown }).code) : "";

  const map: Record<string, string> = {
    // Deliberately identical for all three: telling someone which half was
    // wrong tells them which addresses have accounts.
    "auth/invalid-credential": "Incorrect email or password.",
    "auth/wrong-password": "Incorrect email or password.",
    "auth/user-not-found": "Incorrect email or password.",
    "auth/invalid-email": "That does not look like an email address.",
    "auth/email-already-in-use": "An account already exists for that email. Try signing in.",
    "auth/weak-password": "Please choose a password of at least 6 characters.",
    "auth/too-many-requests": "Too many attempts. Please wait a few minutes and try again.",
    "auth/network-request-failed": "No connection. Check your internet and try again.",
```

```
    "auth/requires-recent-login": "Please sign in again before changing this.",
    "auth/invalid-action-code": "That link has expired or has already been used.",
    "auth/user-disabled": "This account has been disabled. Please contact your parish.",
    "auth/operation-not-allowed":
      "Email sign-in is not switched on for this app yet. Please contact the parish office.",
    // Firebase Authentication has not been set up on the project at all.
    // Distinct from operation-not-allowed, which means the product exists
    // but this one provider is off.
    "auth/configuration-not-found":
      "Accounts are not set up for this app yet. Please contact the parish office.",
    "auth/admin-restricted-operation":
      "New accounts are currently closed. Please contact the parish office.",
  };

  const known = map[code];
  if (!known) {
    // The generic line is for the person; the code is for whoever has to
    // find out why. Without this, an unmapped failure reads as "Something
    // went wrong" in the UI and leaves nothing at all in the console -
    // which is how a project with Authentication switched off looked
    // exactly like a bug in the form.
    console.error("[auth] unmapped Firebase error:", code || "(no code)", err);
  }

  return new AuthError(known ?? "Something went wrong. Please try again.", code || undefined);
}

export interface SignUpInput {
  fullName: string;
  email: string;
  password: string;
  churchId: string;
  phoneNumber?: string;
  /** What Home will call them. Optional; falls back to their first name. */
  nickname?: string;
}

/**
 * Creates the account, writes the profile, and sends the verification email.
 *
 * The profile write happens before the verification email so that a person
 * who closes the tab immediately still has an account the admin can see,
 * rather than an auth user with no parish and no name attached to it.
 */
export async function signUp(input: SignUpInput): Promise<User> {
  try {
    const cred = await createUserWithEmailAndPassword(auth, input.email, input.password);
    await createProfile(
      cred.user, input.fullName, input.churchId, input.phoneNumber, input.nickname,
    );
    await sendEmailVerification(cred.user);
    return cred.user;
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

export async function resendVerificationEmail(): Promise<void> {
  const user = auth.currentUser;
```

```
  if (!user) throw new AuthError("You are not signed in.");
  try {
    await sendEmailVerification(user);
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

/** Re-reads the account from Firebase, so a just-clicked link is noticed. */
export async function refreshEmailVerified(): Promise<boolean> {
  const user = auth.currentUser;
  if (!user) return false;
  await user.reload();
  return auth.currentUser?.emailVerified ?? false;
}

export interface SignInResult {
  user: User;
  /** True when a second factor is still owed. The session is not trusted yet. */
  pendingSecondFactor: boolean;
}

/**
 * Password sign-in, then the second-factor decision.
 *
 * An unverified email short-circuits this: there is no point sending a
 * one-time link to an address nobody has proved they can read.
 */
export async function signIn(email: string, password: string): Promise<SignInResult> {
  try {
    const cred = await signInWithEmailAndPassword(auth, email, password);
    const profile = await getProfile(cred.user.uid);

    // Administrators always take the second factor. They can change other
    // people's records, so their password alone is worth more.
    const required = profile?.role === "church_admin" || cred.user.emailVerified;

    return { user: cred.user, pendingSecondFactor: required };
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

/** Sends the one-time sign-in link that completes the second factor. */
export async function sendSecondFactorLink(email: string): Promise<void> {
  try {
    await sendSignInLinkToEmail(auth, email, linkSettings());
    // Needed to finish the link on return: Firebase requires the address to
    // be supplied again, and asking for it twice is a worse experience than
    // remembering it on this device.
    window.localStorage.setItem(PENDING_EMAIL_KEY, email);
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

export function isSecondFactorLink(url: string = window.location.href): boolean {
  return isSignInWithEmailLink(auth, url);
}
```

```
/**
 * Finishes the second factor from the link the person clicked.
 *
 * `email` is only asked for when the link is opened on a different device
 * from the one that requested it, which is the case localStorage cannot
 * cover.
 */
export async function completeSecondFactor(email?: string): Promise<User> {
  const stored = email ?? window.localStorage.getItem(PENDING_EMAIL_KEY) ?? "";
  if (!stored) {
    throw new AuthError(
      "Please enter the email address you used, to finish signing in on this device.",
    );
  }
  try {
    const cred = await signInWithEmailLink(auth, stored, window.location.href);
    window.localStorage.removeItem(PENDING_EMAIL_KEY);
    return cred.user;
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

export async function resetPassword(email: string): Promise<void> {
  try {
    await sendPasswordResetEmail(auth, email);
  } catch (err) {
    // A reset request deliberately reports success whatever happened: the
    // error distinguishes registered addresses from unregistered ones, and
    // the screen that calls this says "if an account exists, we've sent a
    // link" for the same reason.
    if ((err as { code?: string })?.code !== "auth/user-not-found") {
      throw friendlyAuthError(err);
    }
  }
}

/**
 * Changes the password, proving the current one first.
 *
 * Firebase would demand a recent login anyway; doing it explicitly means the
 * person is asked for their password at the moment it makes sense, rather
 * than being thrown back to the login screen mid-change.
 */
export async function changePassword(currentPassword: string, newPassword: string): Promise<void> {
  const user = auth.currentUser;
  if (!user?.email) throw new AuthError("You are not signed in.");
  try {
    await reauthenticateWithCredential(user, EmailAuthProvider.credential(user.email, currentPassword));
    await updatePassword(user, newPassword);
  } catch (err) {
    throw friendlyAuthError(err);
  }
}

export async function signOutNow(): Promise<void> {
  window.localStorage.removeItem(PENDING_EMAIL_KEY);
  await signOut(auth);
```

```
}
```

2.2.3 API-004 — parishContent.ts

Code ID	API-004
File Name	parishContent.ts
File Path	SanctDemo/src/lib/parishContent.ts
Language / Type	TypeScript
Lines	186
Functions / Classes	ParishContent (exported type); watchParishContent (exported function); saveParishContent (exported function); MAX_PHOTO_BYTES (exported constant); PhotoUploadResult (exported type); ParishPhotoKind (exported type); uploadParishPhoto (exported function)
Related Files (its imports)	./firebase, ./schedule
Where It Is Used	src/admin/AdminAvailability.tsx, src/admin/AdminContent.tsx, src/admin/AdminItems.tsx, src/admin/AdminMassSchedule.tsx, src/components/ParishContentEditor.tsx, src/components/SanctiHost.tsx, src/lib/availability.ts, src/lib/itemContent.ts, src/lib/parishContentStore.ts, src/lib/sanctiGrounding.ts, src/lib/useMassScheduleNotice.ts, src/lib/useOpeningNotices.ts, src/lib/useParishContent.ts, src/lib/useParishContents.ts
How to Recognize It	Open SanctDemo/src/lib/parishContent.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — parishContent.ts

```
import { doc, onSnapshot, setDoc } from "firebase/firestore";
import { getStorage, ref, uploadBytes, getDownloadURL } from "firebase/storage";
import { db } from "./firebase";
import app from "./firebase";
import type { MassScheduleEntry } from "./schedule";

/**
 * Per-parish content an admin can change without a developer.
 *
 * Everything here is optional and overlays the values compiled into data.ts —
 * it never replaces them wholesale. A parish with no document behaves exactly
 * as before, which matters because 29 of the 31 have no collected content at
 * all and must not become blank the moment this feature exists.
 */
export interface ParishContent {
  /** Download URL of the patron photograph, once an admin has uploaded one. */
  photoUrl?: string;
  /** The parish's own description, shown on Home. */
  description?: string;
  /** Mass times, entered by the parish office rather than hardcoded. */
  massSchedule?: MassScheduleEntry[];
  /** Brand colour. Validated for contrast before it is ever saved. */
  themeColor?: string;
```

```
  /**
   * The Church History card on Home.
   *
   * Every parish's history is its own, so this has to be editable or 29 of
   * the 31 show an empty card. The title falls back to the patron's name when
   * an admin leaves it blank.
   */
  historyTitle?: string;
  historyBody?: string;
  historyPhotoUrl?: string;

  /**
   * What the day commemorates, shown with the Mass card.
   *
   * An OVERRIDE, not the only source. The app already derives the liturgical
   * day from the calendar (lib/liturgical.ts), so a parish with nobody
   * editing still shows something true; this replaces that text when a parish
   * wants to say something more specific — a titular feast, a parish
   * anniversary, a bishop's visit.
   */
  commemoratesText?: string;

  /** The photograph behind the Mass schedule panel on Home. */
  massImageUrl?: string;

  /**
   * Ministries and sacraments this parish is NOT currently accepting
   * applications for, by id.
   *
   * A list of exceptions rather than a flag on each one, so a parish with
   * no document at all is open for everything rather than closed for
   * everything. See lib/availability.ts for the full reasoning - and note
   * that the security rules read this exact field when they decide whether
   * to accept an application, so its shape is not free to change.
   */
  closedApplications?: string[];

  /**
   * What the parish has written about each ministry and sacrament,
   * keyed by id: the about text, requirements, schedule, process and
   * reminders a pilgrim reads before applying.
   *
   * Separate from the application records on purpose - a parish
   * correcting a requirement must not have to touch anybody's
   * submission. See lib/itemContent.ts.
   */
  itemContent?: Record<string, import("./itemContent").ItemContent>;

  /** Who last changed this and when — a parish office needs an audit trail. */
  updatedBy?: string;
  updatedAt?: string;
}

const COLLECTION = "parishContent";

/** Live-subscribes to one parish's admin-managed content. */
export function watchParishContent(
  parishId: string,
```

```
  onChange: (content: ParishContent | null) => void,
): () => void {
  return onSnapshot(
    doc(db, COLLECTION, parishId),
    (snapshot) => onChange(snapshot.exists() ? (snapshot.data() as ParishContent) : null),
    (error) => {
      // Never fatal. Without this the app silently shows nothing when rules
      // are wrong or the network drops, instead of falling back to the
      // content compiled into data.ts.
      console.error("[parishContent] read failed:", error.message);
      onChange(null);
    },
  );
}

export async function saveParishContent(
  parishId: string,
  patch: Partial<ParishContent>,
  editorEmail: string,
): Promise<void> {
  await setDoc(
    doc(db, COLLECTION, parishId),
    { ...patch, updatedBy: editorEmail, updatedAt: new Date().toISOString() },
    // Merge, so editing the description cannot wipe a photo someone else
    // uploaded from another device.
    { merge: true },
  );
}

/** Largest photo accepted, before upload. */
export const MAX_PHOTO_BYTES = 5 * 1024 * 1024;

/**
 * Deliberately one shape with two optional fields rather than a discriminated
 * union on `ok: true | false`. This project compiles with `strict` off, which
 * widens boolean literal types and stops TypeScript narrowing such a union at
 * all — the `else` branch still believed it held the success case.
 */
export interface PhotoUploadResult {
  url?: string;
  error?: string;
}

/**
 * Which of a parish's photographs is being replaced.
 *
 * Each gets its own path, so uploading a history photo cannot silently
 * overwrite the patron image the home header depends on.
 */
export type ParishPhotoKind = "patron" | "history" | "mass";

/**
 * Uploads one of a parish's photographs and returns its public URL.
 *
 * Validated before the network is touched: a phone camera photo is routinely
 * 8-12MB, and letting one upload for a minute on parish wifi before failing
 * is worse than refusing it in a sentence.
 */
export async function uploadParishPhoto(
```

```
  parishId: string,
  file: File,
  kind: ParishPhotoKind = "patron",
): Promise<PhotoUploadResult> {
  if (!file.type.startsWith("image/")) {
    return { error: "That file is not an image. Choose a JPG or PNG photo." };
  }
  if (file.size > MAX_PHOTO_BYTES) {
    return {
      error: `That photo is ${(file.size / 1024 / 1024).toFixed(1)} MB. Please use one under 5 MB — most phones can resize when sharing.`,
    };
  }

  try {
    const storage = getStorage(app);
    // One fixed path per parish PER KIND, so re-uploading replaces rather
    // than accumulating orphaned files nobody will ever clean up — and so a
    // history photo cannot overwrite the patron image.
    //
    // "patron" keeps the original unsuffixed path so any file already
    // uploaded under it stays reachable.
    const extension = file.name.split(".").pop()?.toLowerCase() ?? "jpg";
    const name = kind === "patron" ? parishId : `${parishId}-${kind}`;
    const target = ref(storage, `parish-photos/${name}.${extension}`);
    await uploadBytes(target, file, { contentType: file.type });
    return { url: await getDownloadURL(target) };
  } catch (error) {
    const message = error instanceof Error ? error.message : String(error);
    console.error("[parishContent] upload failed:", message);

    // The overwhelmingly likely cause in a fresh project, said plainly rather
    // than as a raw Firebase error code.
    if (/storage\/unauthorized|permission/i.test(message)) {
      return {
        error: "Upload was refused. Firebase Storage rules need deploying — see docs/admin-parish-content.md.",
      };
    }
    if (/storage\/unknown|bucket/i.test(message)) {
      return {
        error: "Firebase Storage is not set up for this project yet — see docs/admin-parish-content.md.",
      };
    }
    return { error: "The photo could not be uploaded. Check your connection and try again." };
  }
}
```

2.2.4 API-005 — parishContentStore.ts

Code ID	API-005
File Name	parishContentStore.ts
File Path	SanctDemo/src/lib/parishContentStore.ts
Language / Type	TypeScript
Lines	59
Functions / Classes	parishStoreReady (exported function); readParishContent (exported function)
Related Files (its imports)	./parishContent, ./scanQuotaStore
Where It Is Used	server.ts
How to Recognize It	Open SanctDemo/src/lib/parishContentStore.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Reading a parish's admin-managed content on the server.
>
> ## Why this exists at all
>
> Sancti could only ever see the content of the ONE parish the pilgrim had open, because that is the only document the client subscribes to. Ask about any other parish and the answer quietly fell back to the schedule compiled into the bundle - so a parish office could move its Mass in the admin portal and Sancti would keep telling everybody else the old time. Reading from the server removes that limit: the Admin SDK can fetch any parish's document, so an answer about San Roque is as current as an answer about the parish you are standing in.
>
> ## Why it is read-only, and server-only
>
> Server-only because it uses the Admin SDK, whose credentials must never reach a browser - nothing under src/components may import this. Read-only because the assistant answers questions; nothing it does should ever write to a parish's document. The Admin SDK bypasses security rules, so the narrowness is the safeguard.
>
> Reuses the Admin app started by scanQuotaStore rather than starting a second one: two initialisations of the same credential is a warning at best and a leak at worst.

Complete Source Code — parishContentStore.ts

```
/**
 * Reading a parish's admin-managed content on the server.
 *
 * ## Why this exists at all
 *
 * Sancti could only ever see the content of the ONE parish the pilgrim
 * had open, because that is the only document the client subscribes to.
 * Ask about any other parish and the answer quietly fell back to the
 * schedule compiled into the bundle - so a parish office could move its
 * Mass in the admin portal and Sancti would keep telling everybody else
 * the old time. Reading from the server removes that limit: the Admin
```

```
 * SDK can fetch any parish's document, so an answer about San Roque is
 * as current as an answer about the parish you are standing in.
 *
 * ## Why it is read-only, and server-only
 *
 * Server-only because it uses the Admin SDK, whose credentials must
 * never reach a browser - nothing under src/components may import this.
 * Read-only because the assistant answers questions; nothing it does
 * should ever write to a parish's document. The Admin SDK bypasses
 * security rules, so the narrowness is the safeguard.
 *
 * Reuses the Admin app started by scanQuotaStore rather than starting a
 * second one: two initialisations of the same credential is a warning
 * at best and a leak at worst.
 */
import { getFirestore } from 'firebase-admin/firestore'

import type { ParishContent } from './parishContent'
import { quotaStoreReady } from './scanQuotaStore'

const COLLECTION = 'parishContent'

/** True when the server can read Firestore at all. */
export function parishStoreReady(): boolean {
  return quotaStoreReady()
}

/**
 * One parish's content, or null when there is none to be had.
 *
 * Null covers three different situations on purpose - no Admin SDK, no
 * document, or a read that failed - because the caller does the same
 * thing in all three: ground on what it has, and say what it does not.
 * The alternative, throwing, would turn a parish that has simply not
 * filled anything in into an error page.
 */
export async function readParishContent(parishId: string): Promise<ParishContent | null> {
  if (!parishId || !parishStoreReady()) return null

  try {
    const snap = await getFirestore().collection(COLLECTION).doc(parishId).get()
    if (!snap.exists) return null
    return (snap.data() ?? null) as ParishContent | null
  } catch (err) {
    console.warn('[Sancti] could not read parish content:', (err as Error).message)
    return null
  }
}
```

2.2.5 API-006 — firestore.rules

Code ID	API-006
File Name	firestore.rules
File Path	SanctDemo/firestore.rules
Language / Type	Firestore Security Rules
Lines	409
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/firestore.rules. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — firestore.rules

```
rules_version = '2';

// Firestore security rules for SanctiWalk.
//
// ## Why the checks live here and not in the app
//
// Everything below runs on Google's servers, on every request, whatever the
// client is. The app's own filtering is a convenience for the person using
// it; it is not a control. Anyone with the project id can talk to this
// database directly with curl, so a rule that exists only in React is no
// rule at all.
//
// ## Why roles are documents and not custom claims
//
// Custom claims are the usual way to carry `role` and `churchId`, and they
// cost nothing to check. Setting one needs the Admin SDK, which needs Cloud
// Functions, which needs the Blaze plan - and this project is on Spark. So
// the role lives in the user's own document and the rules read it with
// get(). That is still enforced server-side and still unforgeable from the
// client, because the fields that matter are not writable by the user (see
// onlyEditableUserFields below). It costs one document read per evaluation,
// which is the price of staying on the free plan.
//
// Moving to Blaze later means changing myRole() and myChurch() to read
// request.auth.token instead. Nothing else here has to change.
//
// ## On field naming
//
// Applications key their owner as `uid`, not `applicantId`. That is the
// field the existing documents already carry and the existing query filters
// on; renaming it would strand every application already submitted.
//
// Deploy with:  npx firebase deploy --only firestore:rules
```

```
service cloud.firestore {
  match /databases/{database}/documents {

    // A churchId that cannot collide with a real one, for documents written
    // before churches existed. Without it, a missing field reads as "" and
    // an admin whose own churchId was somehow "" would match every orphan.
    function noChurch() {
      return '__none__';
    }

    function isSignedIn() {
      return request.auth != null;
    }

    function me() {
      return get(/databases/$(database)/documents/users/$(request.auth.uid)).data;
    }

    function myRole() {
      return isSignedIn() ? me().get('role', 'user') : 'none';
    }

    function myChurch() {
      return isSignedIn() ? me().get('churchId', noChurch()) : noChurch();
    }

    // A church admin acting on their OWN church. Every church-scoped rule
    // goes through this rather than through the role alone: the role says
    // what someone is, this says what they may touch. Checking only the role
    // is exactly how an admin of one parish ends up reading another's
    // applications.
    // The cheap comparisons come first on purpose. myRole() and myChurch()
    // each cost a document read, and a rule request is allowed ten; an
    // orphaned document or a signed-out caller should cost none of them.
    function isAdminOf(churchId) {
      return isSignedIn()
        && churchId != noChurch()
        && churchId == myChurch()
        && myRole() == 'church_admin';
    }

    // The legacy global administrator, from before churches existed. Kept so
    // console-granted accounts keep working for the diocese-wide collections,
    // and deliberately NOT accepted as a substitute for isAdminOf() - a
    // global flag is the precise thing church isolation has to stop.
    function isSuperAdmin() {
      return isSignedIn()
        && exists(/databases/$(database)/documents/admins/$(request.auth.uid));
    }

    // ------------------------------------------------------------------ users

    // The fields a person may change about themselves. Everything outside
    // this list is set at creation or set by the parish, and letting the
    // client write any of it is the whole attack: `role` makes you an admin,
    // `churchId` moves you into someone else's parish, and `phoneVerified` /
    // `emailVerified` would let you claim a verification you never did.
    function onlyEditableUserFields() {
      return request.resource.data.diff(resource.data).affectedKeys()
```

```
    .hasOnly([
      'fullName', 'nickname', 'phoneNumber', 'photoUrl', 'updatedAt',
      'preferredLanguage', 'notificationsSeenAt',
      // The activity heartbeat. Without it in this list the write is
      // silently denied and the dashboard shows nobody as active -
      // which is exactly how the station check-in was lost once.
      'lastActiveAt',
      // The pilgrimage progress the app saves as you walk. Every field
      // of UserProgress has to be here: the list is exhaustive, so one
      // missing name does not fail loudly - it silently denies the
      // write and the check-in is simply never saved.
      'points', 'steps', 'distanceKm',
      'completedStations', 'completedRoutes', 'badges'
    ]);
}

match /users/{userId} {
  // A church admin may read their own parishioners, for the admin user
  // list. Nobody reads across churches.
  allow read: if isSignedIn()
    && (request.auth.uid == userId
        || isAdminOf(resource.data.get('churchId', noChurch()))
        || isSuperAdmin());

  // At creation a user may declare which church they joined, but not
  // that they run it, and not that they are already verified.
  allow create: if isSignedIn()
    && request.auth.uid == userId
    && request.resource.data.get('role', 'user') == 'user'
    && request.resource.data.get('phoneVerified', false) == false;

  allow update: if isSignedIn()
    && request.auth.uid == userId
    && onlyEditableUserFields();

  // Profiles are never deleted from the client; an account removal is an
  // auth operation with records hanging off it.
  allow delete: if false;
}

// ------------------------------------------------------------- churches

// Public to read: the signup screen lists churches before anyone has
// signed in. Never created or deleted from the client, and `adminUid` is
// immutable - that immutability IS the one-admin-per-church rule, since
// nothing client-side can assign or reassign it.
match /churches/{churchId} {
  allow read: if true;

  allow update: if isAdminOf(churchId)
    && request.resource.data.adminUid == resource.data.adminUid
    && request.resource.data.id == resource.data.id;

  allow create, delete: if false;
}

// ------------------------------------------------------------ applications

// Sacrament and ministry applications share one collection, with `kind`
```

```
// separating them, because every query the parish office runs - pending
// across the parish, by reference, by date - wants both at once.

// What an applicant may assert when submitting. `status` is not among
// them: a client that could write "approved" could approve its own
// baptism.
// Ministries and sacraments the parish has closed to new applications.
//
// Absent document, or absent field, means nothing is closed - which is
// the state 29 of the 31 parishes are in, and they must stay open.
function closedApplications(churchId) {
  return exists(/databases/$(database)/documents/parishContent/$(churchId))
    ? get(/databases/$(database)/documents/parishContent/$(churchId))
        .data.get('closedApplications', [])
    : [];
}

function validNewApplication() {
  return isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.status == 'pending'
    && request.resource.data.kind in ['sacrament', 'ministry']
    // The church you belong to is the church you apply to. Read from the
    // profile rather than trusted from the request, so a crafted
    // churchId cannot drop an application into another parish's queue.
    && request.resource.data.churchId == myChurch()
    && myChurch() != noChurch()
    // Which ministry or sacrament this is, by id. Required, because the
    // check below is worthless if a client can leave it out: an
    // application with no itemId would match nothing in the closed list
    // and sail through.
    && request.resource.data.itemId is string
    && request.resource.data.itemId != ''
    // Closed means closed. The app greys out the button, but the button
    // is not the control - anyone can POST here with curl, and a parish
    // that has shut applications for the choir has shut them.
    && !(request.resource.data.itemId in closedApplications(myChurch()));
}

// What a reviewer may change: the decision and its audit trail. Not the
// applicant, not the church, not the submitted answers.
function onlyStatusFields() {
  return request.resource.data.diff(resource.data).affectedKeys()
    .hasOnly(['status', 'adminNote', 'reviewedBy', 'reviewedAt',
              'updatedAt', 'history']);
}

match /applications/{applicationId} {
  allow create: if validNewApplication();

  allow read: if (isSignedIn() && resource.data.uid == request.auth.uid)
    || isAdminOf(resource.data.get('churchId', noChurch()))
    || isSuperAdmin();

  // An applicant cannot edit a submitted application. It is a record the
  // parish is acting on, and a form that changes under the reviewer is
  // worse than one that cannot be corrected.
  allow update: if isAdminOf(resource.data.get('churchId', noChurch()))
    && onlyStatusFields()
```

```
    && request.resource.data.status in
         ['pending', 'under_review', 'approved', 'rejected', 'completed'];

  // Withdrawal, not deletion.
  //
  // An applicant may take back something the parish has not started
  // on. Once it is under review, approved, rejected or completed it
  // is a record of a decision somebody made, and letting the
  // applicant erase it would take the parish's own work with it -
  // including the queue entry the office is working from.
  //
  // The parish cannot delete either. A rejection that can be made to
  // disappear is worse for the applicant than one that stands.
  allow delete: if isSignedIn()
    && resource.data.uid == request.auth.uid
    && resource.data.status == 'pending';
}

// ------------------------------------------------------------ notifications

match /notifications/{notificationId} {
  allow read: if isSignedIn() && resource.data.userId == request.auth.uid;

  allow create: if isSignedIn()
    && (request.resource.data.userId == request.auth.uid
        || isAdminOf(request.resource.data.get('churchId', noChurch())));

  // The only thing a reader may do to a notification is mark it read.
  allow update: if isSignedIn()
    && resource.data.userId == request.auth.uid
    && request.resource.data.diff(resource.data).affectedKeys()
         .hasOnly(['readAt']);

  allow delete: if isSignedIn() && resource.data.userId == request.auth.uid;
}

// ------------------------------------------------------------------ activity

// Sign-ins, station check-ins and comments. These used to be written
// into `applications`, which put login records in the queue the parish
// works through and forced the application rules to accept rows that
// were not applications.
//
// Append-only by design: a log a client can edit is not a log.
match /activity/{entryId} {
  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid;
  allow read: if (isSignedIn() && resource.data.uid == request.auth.uid)
    || isAdminOf(resource.data.get('churchId', noChurch()))
    || isSuperAdmin();
  allow update, delete: if false;
}

// ------------------------------------------------------------------ visits

// Estimated church visits, for the parish dashboard.
//
// A visit says only that a signed-in pilgrim was inside a parish's
// geofence at a time. No coordinates, no track, nothing that could
```

```
// reconstruct where somebody went.
//
// Written by the pilgrim's own device, so the rule's job is to stop
// it being written on anybody else's behalf, stop it naming a
// church that does not exist, and stop it carrying more than the
// three fields the dashboard counts. A client that could add a
// field here could add a location to it.
//
// Nothing may edit or delete a visit. An admin who could rewrite
// them could manufacture a parish's attendance.
/**
 * Page opens and downloads. See lib/pageEvents.
 *
 * The only collection in this database an UNAUTHENTICATED caller
 * may write to, and it is on purpose: most people who open
 * SanctiWalk never sign in, and the download page is plain HTML
 * with no session at all. A count that only saw signed-in visitors
 * would answer a different question from the one asked.
 *
 * So it is locked down by shape instead of by identity. Three
 * fields and no others, a kind from a fixed pair, a visitor token
 * of bounded length, and the SERVER's timestamp - a caller cannot
 * backdate a row, cannot attach anything to it, and cannot write a
 * field that was not designed for. Nothing here identifies a
 * person, so a forged row costs an inaccurate tally and nothing
 * else.
 *
 * Nobody reads it but a parish admin, and no one may ever change
 * or delete a row: a tally that can be edited is not a record.
 */
match /pageEvents/{eventId} {
  allow create: if request.resource.data.keys().hasOnly(['kind', 'visitorId', 'createdAt'])
    && request.resource.data.kind in ['open', 'download']
    && request.resource.data.visitorId is string
    && request.resource.data.visitorId.size() >= 8
    && request.resource.data.visitorId.size() <= 64
    // A window rather than an exact match on request.time.
    //
    // The app writes this with the SDK's serverTimestamp, which
    // resolves to request.time exactly. The download page cannot:
    // it is plain HTML posting to the REST endpoint, where the only
    // timestamp available is the browser's own. Demanding equality
    // would have silently rejected every download - the one event
    // the parish asked for most.
    //
    // Five minutes either way still makes a row impossible to file
    // into last month, which is the thing worth preventing.
    && request.resource.data.createdAt is timestamp
    && request.resource.data.createdAt > request.time - duration.value(5, 'm')
    && request.resource.data.createdAt < request.time + duration.value(5, 'm');

  allow read: if isSignedIn() && myRole() == 'church_admin';
  allow update, delete: if false;
}

match /visits/{visitId} {
  allow create: if isSignedIn()
    && request.resource.data.uid == request.auth.uid
    && request.resource.data.churchId is string
```

```
    && request.resource.data.churchId != ''
    && exists(/databases/$(database)/documents/churches/$(request.resource.data.churchId))
    && request.resource.data.keys().hasOnly(['uid', 'churchId', 'at', 'createdAt'])
    // The server's clock, not the handset's. A device with a wrong
    // date cannot backdate or postdate its way into another period.
    && request.resource.data.createdAt == request.time;

  // A pilgrim may read their own; an admin may read their parish's.
  // Nobody reads across parishes.
  allow read: if isSignedIn()
    && (resource.data.uid == request.auth.uid
        || isAdminOf(resource.data.get('churchId', noChurch()))
        || isSuperAdmin());

  allow update, delete: if false;
}

// ------------------------------------------------- diocese-wide content

// Split per operation rather than one `write`. A delete carries no
// request.resource at all, so a rule phrased on the incoming document
// can never authorise one - the parish could post an announcement and
// then not be able to take it down.
//
// An update is checked on BOTH sides: the document as it stands and the
// document as it would be. Checking only the incoming one would let a
// parish claim another parish's announcement by rewriting its churchId.
//
// Legacy announcements carry no churchId, so they resolve to noChurch()
// and only a superadmin can touch them. That is deliberate: nobody can
// prove which parish wrote them.
match /announcements/{docId} {
  allow read: if true;
  allow create: if isSuperAdmin()
    || isAdminOf(request.resource.data.get('churchId', noChurch()));
  allow update: if isSuperAdmin()
    || (isAdminOf(resource.data.get('churchId', noChurch()))
        && isAdminOf(request.resource.data.get('churchId', noChurch())));
  allow delete: if isSuperAdmin()
    || isAdminOf(resource.data.get('churchId', noChurch()));
}

// Admin-managed parish content: patron photo, description, Mass times,
// brand colour. Public to read, because a pilgrim browsing the diocese is
// not signed in and the home screen must still fill in. Parish-only to
// write - Mass times in particular are the one thing here someone could
// change to send a congregation to a locked church.
match /parishContent/{parishId} {
  allow read: if true;
  allow write: if isSuperAdmin() || isAdminOf(parishId);
}

// The legacy admin roster. Readable so the client can tell whether to
// offer the portal, never writable from the client.
match /admins/{userId} {
  allow read: if isSignedIn();
  allow write: if false;
}
```

```
    // The scanner's daily allowance, one document per user per parish
    // per day.
    //
    // Closed to clients in both directions, which is the whole point of
    // it. The server writes these with the Admin SDK, which bypasses
    // these rules; a client that could write here could grant itself
    // scans, and one that could read another user's could count their
    // visits. The app never touches this collection - it asks the
    // server, which is the only thing that may be trusted about a
    // number the server enforces.
    match /scanQuotas/{quotaId} {
      allow read, write: if false;
    }

    // Anything not named above is closed. Without this, a collection added
    // later is world-writable until someone remembers to write a rule for it.
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

2.2.6 API-007 — firestore.indexes.json

Code ID	API-007
File Name	firestore.indexes.json
File Path	SanctDemo/firestore.indexes.json
Language / Type	JSON
Lines	5
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/firestore.indexes.json. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — firestore.indexes.json

```
{
  "//": "No composite indexes yet. The app's queries are single-field, which Firestore indexes automatically. This file exists because firebase.json references it;
add entries here if a query ever needs one, and Firestore's error message will tell you exactly what to put.",
  "indexes": [],
  "fieldOverrides": []
}
```

2.3 Administrative Portal

2.3.1 API-008 — AdminAnalytics.tsx

Code ID	API-008
File Name	AdminAnalytics.tsx
File Path	SanctDemo/src/admin/AdminAnalytics.tsx
Language / Type	TypeScript (React, TSX)
Lines	522
Functions / Classes	AdminAnalytics (exported component/function (default)); parishLabel (internal function); useStationNames (internal function); Figure (internal function); Panel (internal function); Bars (internal function); Rows (internal function); ReachPanel (internal function)
Related Files (its imports)	../lib/firebase, ./AdminNav, ../lib/authFlow, ./AdminApp, ../data
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminAnalytics.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminAnalytics.tsx

```
import { useEffect, useMemo, useState } from "react";
import { collection, onSnapshot, query, where } from "firebase/firestore";
import { Loader2 } from "lucide-react";
import { db } from "../lib/firebase";
import AdminNav from "./AdminNav";
import { signOutNow } from "../lib/authFlow";
import type { AdminSession } from "./AdminApp";
import { ROUTES } from "../data";
import {
  formatHour, summariseActivity,
  type ActivityEntry, type AnalyticsSummary,
} from "../lib/analytics";
import {
  ACTIVE_WINDOW_MS, countActive, summariseVisits,
  type VisitDoc, type VisitSummary,
} from "../lib/visits";
import {
  summarisePageEvents, type PageEvent, type PageEventTotals,
} from "../lib/pageEvents";

/**
 * What the parish's visitors did, as opposed to what they asked for.
 *
 * ## Why this screen exists separately from the dashboard
 *
 * The dashboard answers "what is waiting for me" - applications needing
 * a decision. This answers "what is happening in the church", and the
```

```
 * two are read at different times by different people. Putting visitor
 * figures above an application queue would also push the work down the
 * page behind numbers nobody has to act on.
 *
 * ## What it deliberately does not show
 *
 * Nobody is named. The log carries a user id and this screen uses it
 * only to count distinct people - never to list them, never to show
 * what any one person did. "How many came on Sunday" and "who came on
 * Sunday" are different questions, and the parish office has not been
 * given the second one.
 *
 * ## Why the empty state says what it says
 *
 * A table of zeros cannot distinguish "nobody came" from "nothing is
 * being recorded". Until there is at least one entry this screen says
 * which of the two it is, rather than drawing an empty chart that looks
 * like a measurement.
 */

/** How far back the figures go. A fortnight covers two Sundays. */
const WINDOW_DAYS = 14;

/**
 * The radius quoted in the honesty note, read from the parish's own
 * record rather than written here twice. presence.ts is what actually
 * decides; this only has to describe it accurately.
 */
const GEOFENCE_M = ROUTES.find(r => r.geofenceRadius)?.geofenceRadius ?? 100;

export default function AdminAnalytics({ session }: { session: AdminSession }) {
  const [entries, setEntries] = useState<ActivityEntry[] | null>(null);
  const [error, setError] = useState<string | null>(null);
  const churchId = session.profile.churchId;

  // Filtered on churchId here AND in the security rules, like every other
  // admin screen: the filter makes it correct, the rule makes it safe.
  useEffect(() => {
    const q = query(collection(db, "activity"), where("churchId", "==", churchId));
    return onSnapshot(
      q,
      snap => setEntries(snap.docs.map(d => d.data() as ActivityEntry)),
      err => setError(err.message),
    );
  }, [churchId]);

  const summary = useMemo<AnalyticsSummary | null>(
    () => (entries === null ? null : summariseActivity(entries, new Date(), WINDOW_DAYS)),
    [entries],
  );

  // Estimated visits and the parish's own users.
  //
  // Both filtered on churchId here AND in the rules, like every other
  // admin screen: the filter makes it correct, the rule makes it safe.
  const [visits, setVisits] = useState<VisitDoc[] | null>(null);
  const [people, setPeople] = useState<{ role?: string; lastActiveAt?: string }[] | null>(null);

  useEffect(() => {
```

```
  const q = query(collection(db, "visits"), where("churchId", "==", churchId));
  return onSnapshot(
    q,
    snap => setVisits(snap.docs.map(d => d.data() as VisitDoc)),
    err => setError(err.message),
  );
}, [churchId]);

useEffect(() => {
  const q = query(collection(db, "users"), where("churchId", "==", churchId));
  return onSnapshot(
    q,
    snap => setPeople(snap.docs.map(d => d.data() as { role?: string; lastActiveAt?: string })),
    err => setError(err.message),
  );
}, [churchId]);

const visitSummary = useMemo<VisitSummary | null>(
  () => (visits === null ? null : summariseVisits(visits, WINDOW_DAYS, new Date())),
  [visits],
);

/** Ordinary pilgrims. An administrator is not one of the parish's users. */
const registered = useMemo(
  () => (people === null ? null : people.filter(p => (p.role ?? "user") === "user").length),
  [people],
);

const activeNow = useMemo(
  () => (people === null ? null
    : countActive(people.filter(p => (p.role ?? "user") === "user"), new Date())),
  [people],
);

const stationName = useStationNames(churchId);

return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <header className="bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-4 pt-6 pb-5">
      <p className="text-[13px] font-bold uppercase tracking-wider opacity-80">
        Parish office
      </p>
      <h1 className="text-[24px] font-bold font-serif italic">Visitor activity</h1>
      <button
        onClick={() => void signOutNow()}
        className="mt-3 rounded-full border border-white/40 px-4 py-1.5 text-[14px] font-semibold"
      >
        Sign out
      </button>
    </header>

    <main className="px-4 py-5 max-w-[900px] mx-auto">
      <AdminNav />

      {error && (
        <p role="alert" className="mb-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}
```

```
<ReachPanel />

{summary === null && (
  <div className="py-12 text-center">
    <Loader2 className="mx-auto w-6 h-6 animate-spin text-[var(--color-brand-primary)]" />
  </div>
)}

{/* The four figures the parish asked for, above everything
    else. A parish opens this to ask "how many", and the station
    breakdown below answers a different question. */}
{visitSummary !== null && (
  <>
    <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 mb-5">
      <Figure label="Registered users" value={registered ?? "—"}
              hint="pilgrims with an account at this parish" />
      <Figure label="Estimated visits" value={visitSummary.total}
              hint="geolocation-detected, all time" />
      <Figure label="Active users" value={activeNow ?? "—"}
              hint={`in the last ${Math.round(ACTIVE_WINDOW_MS / 60000)} minutes`} />
      <Figure label="Visitors" value={visitSummary.visitors}
              hint="distinct people behind those visits" />
    </div>

    <Panel
      title={`Estimated visits, last ${WINDOW_DAYS} days`}
      note="Counted when a signed-in pilgrim's device reports it is within this parish's geofence."
    >
      {visitSummary.total === 0 ? (
        <p className="text-[15px] text-[var(--color-brand-secondary)]">
          No visits recorded yet. A visit appears here when a
          signed-in pilgrim arrives at the parish with location
          turned on.
        </p>
      ) : (
        <Bars
          data={visitSummary.daily.map(d => ({
            key: d.date,
            label: d.date.slice(8),
            value: d.count,
          }))}
        />
      )}
    </Panel>

    <Panel
      title="Visits by parish"
      note="You see your own parish only. The security rules refuse another parish's records, which is why this list is not diocese-wide."
    >
      {visitSummary.byChurch.length === 0 ? (
        <p className="text-[15px] text-[var(--color-brand-secondary)]">
          Nothing recorded yet.
        </p>
      ) : (
        <Rows
          data={visitSummary.byChurch.map(c => ({
            key: c.churchId,
            label: parishLabel(c.churchId),
```

```
            value: c.count,
          }))}
        />
      )}
    </Panel>

    <p className="mb-6 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
      <strong>Estimated</strong> is the operative word. A visit is
      recorded when a phone reports being within {GEOFENCE_M} metres
      of this parish, which is proximity and not proof of entry:
      somebody passing on the street counts, and satellite accuracy
      in a built-up area is often worse than that radius. Only
      signed-in pilgrims are counted, and the same person is counted
      once per parish every two hours. No location is stored — a
      record holds the parish, the time, and an account identifier,
      and nothing else.
    </p>
  </>
)}

{summary !== null && summary.totalEvents === 0 && (
  <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-5 py-8 text-center">
    <p className="text-[16px] font-semibold text-[var(--color-brand-text)]">
      Nothing recorded yet
    </p>
    <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
      Activity appears here once pilgrims sign in and check in at
      stations on the tour. Visitors who are not signed in are not
      counted.
    </p>
  </div>
)}

{summary !== null && summary.totalEvents > 0 && (
  <>
    <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 mb-5">
      <Figure label="People" value={summary.uniqueVisitors}
              hint="distinct signed-in visitors" />
      <Figure label="Activity" value={summary.totalEvents}
              hint="recorded events" />
      <Figure label="Busiest hour" value={formatHour(summary.peakHour)}
              hint="across all days" />
      <Figure label="Check-ins" value={summary.byKind.station_visit}
              hint="at tour stations" />
    </div>

    <Panel title="When people come"
           note="By hour of the day, all recorded activity.">
      <Bars
        data={summary.byHour.map(h => ({
          key: String(h.hour),
          label: h.hour % 6 === 0 ? formatHour(h.hour) : "",
          value: h.count,
        }))}
      />
    </Panel>

    <Panel title="Which days"
           note="Sunday first. A parish's week is not a working week.">
```

```
  <Rows
    data={summary.byWeekday.map(d => ({
      key: d.label, label: d.label, value: d.count,
    }))}
  />
</Panel>

<Panel
  title={`The last ${WINDOW_DAYS} days`}
  note="Activity, and the number of distinct people behind it."
>
  <table className="w-full text-[15px]">
    <thead>
      <tr className="text-left text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        <th className="py-1.5">Date</th>
        <th className="py-1.5 text-right">Activity</th>
        <th className="py-1.5 text-right">People</th>
      </tr>
    </thead>
    <tbody>
      {summary.daily.map(d => (
        <tr key={d.date} className="border-t border-[var(--color-brand-border)]">
          <td className="py-1.5 tabular-nums">{d.date}</td>
          <td className="py-1.5 text-right tabular-nums">{d.count}</td>
          <td className="py-1.5 text-right tabular-nums">{d.visitors}</td>
        </tr>
      ))}
    </tbody>
  </table>
</Panel>

<Panel
  title="Most visited stations"
  note="Check-ins and comments together: both are someone stopping there."
>
  {summary.topStations.length === 0 ? (
    <p className="text-[15px] text-[var(--color-brand-secondary)]">
      No station check-ins recorded yet.
    </p>
  ) : (
    <Rows
      data={summary.topStations.map(s => ({
        key: s.stationId,
        label: stationName(s.stationId),
        value: s.count,
      }))}
    />
  )}
</Panel>

<Panel title="Which features are used"
       note="Every recorded kind, including those at zero.">
  <Rows
    data={[
      { key: "sign_in", label: "Sign-ins", value: summary.byKind.sign_in },
      { key: "station_visit", label: "Station check-ins", value: summary.byKind.station_visit },
      { key: "station_comment", label: "Station comments", value: summary.byKind.station_comment },
    ]}
  />
```

```
            </Panel>

            <p className="mt-6 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
              These figures count signed-in visitors only, and no individual
              is identified. A pilgrim who browses without signing in is not
              recorded at all, so the numbers are a floor rather than a
              count of everyone who came.
            </p>
          </>
        )}
      </main>
    </div>
  );
}

/**
 * Station ids back into the names the parish knows them by.
 *
 * The log stores an id. "st-altar-mayor" on a report to a parish priest
 * is not an answer to "which part of my church do people stop at".
 */
/** The parish's own name, rather than its id, in the breakdown. */
function parishLabel(churchId: string): string {
  const route = ROUTES.find(r => r.id === churchId);
  return route ? route.name.replace(" Guide", "").replace(" Tour", "") : churchId;
}

function useStationNames(churchId: string): (id: string) => string {
  return useMemo(() => {
    const route = ROUTES.find(r => r.id === churchId);
    const names = new Map<string, string>();
    for (const station of route?.stations ?? []) names.set(station.id, station.name);
    // Falls back to the id rather than hiding the row: a station that has
    // been renamed or removed still had people standing at it.
    return (id: string) => names.get(id) ?? id;
  }, [churchId]);
}

function Figure({ label, value, hint }: { label: string; value: number | string; hint: string }) {
  return (
    <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4">
      <p className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        {label}
      </p>
      <p className="mt-1 text-[26px] font-bold leading-tight text-[var(--color-brand-text)]">
        {value}
      </p>
      <p className="mt-0.5 text-[13px] text-[var(--color-brand-secondary)]">{hint}</p>
    </div>
  );
}

function Panel({ title, note, children }: {
  title: string; note: string; children: React.ReactNode;
}) {
  return (
    <section className="mb-5 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-5">
      <h2 className="text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">
        {title}
```

```
      </h2>
      <p className="mt-0.5 mb-3 text-[14px] text-[var(--color-brand-secondary)]">{note}</p>
      {children}
    </section>
  );
}

interface Datum { key: string; label: string; value: number }

/** A column per bucket. For the hours, where there are twenty-four. */
function Bars({ data }: { data: Datum[] }) {
  const max = Math.max(1, ...data.map(d => d.value));
  return (
    <div className="flex items-end gap-[3px] h-[120px]" role="img"
         aria-label={data.map(d => `${d.key}: ${d.value}`).join(", ")}>
      {data.map(d => (
        <div key={d.key} className="flex-1 flex flex-col items-center gap-1 h-full justify-end">
          <div
            className="w-full rounded-t-[3px] bg-[var(--color-brand-primary)]"
            style={{ height: `${Math.max(d.value === 0 ? 0 : 3, (d.value / max) * 100)}%` }}
            title={`${d.key}: ${d.value}`}
          />
          <span className="text-[10px] text-[var(--color-brand-secondary)] whitespace-nowrap">
            {d.label}
          </span>
        </div>
      ))}
    </div>
  );
}

/** A labelled row per item, for short lists where the name matters. */
function Rows({ data }: { data: Datum[] }) {
  const max = Math.max(1, ...data.map(d => d.value));
  return (
    <ul className="space-y-2">
      {data.map(d => (
        <li key={d.key} className="flex items-center gap-3">
          <span className="w-[42%] shrink-0 truncate text-[15px] text-[var(--color-brand-text)]"
                title={d.label}>
            {d.label}
          </span>
          <span className="flex-1 h-[10px] rounded-full bg-[var(--color-brand-card-sunk)] overflow-hidden">
            <span
              className="block h-full rounded-full bg-[var(--color-brand-primary)]"
              style={{ width: `${(d.value / max) * 100}%` }}
            />
          </span>
          <span className="w-[3ch] shrink-0 text-right text-[15px] font-semibold tabular-nums text-[var(--color-brand-text)]">
            {d.value}
          </span>
        </li>
      ))}
    </ul>
  );
}


/**
```

```
 * How many found the app at all, and how many took it away.
 *
 * Separate from the visit figures above it because it answers a
 * different question: those are about people inside a church, these
 * are about everyone who opened SanctiWalk anywhere. Most of them
 * never sign in and are not standing near a parish, so they appear in
 * nothing else on this page.
 *
 * Counts, not names. The same decision as the rest of this screen:
 * "how many downloaded it" and "who downloaded it" are different
 * questions and only the first is answered here. There is nothing in
 * the record that reaches a person.
 */
function ReachPanel() {
  const [totals, setTotals] = useState<PageEventTotals | null>(null);
  const [since, setSince] = useState<string | null>(null);

  useEffect(() => {
    const stop = onSnapshot(
      collection(db, "pageEvents"),
      snap => {
        const rows: PageEvent[] = [];
        let earliest: number | null = null;
        snap.forEach(doc => {
          const d = doc.data() as {
            kind?: string;
            visitorId?: string;
            createdAt?: { toDate?: () => Date };
          };
          const when = d.createdAt?.toDate?.();
          if (!when) return;
          if (earliest === null || when.getTime() < earliest) earliest = when.getTime();
          if (d.kind !== "open" && d.kind !== "download") return;
          rows.push({ kind: d.kind, visitorId: d.visitorId ?? "", at: when.toISOString() });
        });
        setTotals(summarisePageEvents(rows));
        setSince(earliest === null ? null : new Date(earliest).toLocaleDateString());
      },
      () => setTotals(null),
    );
    return stop;
  }, []);

  if (totals === null) return null;

  return (
    <section className="mb-5">
      <h2 className="text-[18px] font-bold text-[var(--color-brand-text)]">
        Reach
      </h2>
      <p className="text-[14px] leading-snug text-[var(--color-brand-secondary)]">
        {since
          ? `Everyone who opened SanctiWalk or downloaded it, counted since ${since}.`
          : "Everyone who opened SanctiWalk or downloaded it."}{" "}
        Nobody is named.
      </p>

      <div className="mt-3 grid grid-cols-2 md:grid-cols-4 gap-3">
        <Figure
```

```
          label="Opened the app"
          value={totals.opens}
          hint="Every time somebody opened SanctiWalk, on the web or on a phone."
        />
        <Figure
          label="Different people"
          value={totals.people}
          hint="Distinct browsers. Forty opens from four people is not forty people."
        />
        <Figure
          label="Downloads"
          value={totals.downloads}
          hint="Taps on Download for Android. Downloading twice usually means the first one failed."
        />
        <Figure
          label="Different downloaders"
          value={totals.downloaders}
          hint="Distinct browsers that downloaded it at least once."
        />
      </div>
    </section>
  );
}
```

2.3.2 API-009 — AdminAnnouncements.tsx

Code ID	API-009
File Name	AdminAnnouncements.tsx
File Path	SanctDemo/src/admin/AdminAnnouncements.tsx
Language / Type	TypeScript (React, TSX)
Lines	575
Functions / Classes	AdminAnnouncements (exported component/function (default)); BoardList (internal function); ComposeForm (internal function); Row (internal function); StatusChip (internal function); Action (internal function); titleFor (internal function); Field (internal function)
Related Files (its imports)	../lib/firebase, ../lib/announcementImage, ../lib/notifications, ./AdminNav, ./AdminApp
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminAnnouncements.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminAnnouncements.tsx

```
import { useEffect, useMemo, useRef, useState, type ReactNode } from "react";
import { useParams } from "react-router-dom";
import { collection, onSnapshot, query, where } from "firebase/firestore";
import {
  CalendarDays, Eye, FileEdit, ImagePlus, Loader2, MapPin, Megaphone,
  Archive, Trash2, Undo2, X,
} from "lucide-react";
import { db } from "../lib/firebase";
import {
  ANNOUNCEMENT_TYPES, BOARD_LABEL, EVENT_TYPE, FEAST_TYPE, STATUS_LABEL,
  createAnnouncement, deleteAnnouncement, updateAnnouncement,
  formatWhen, onBoard, startOfDay, statusOf,
  type AnnouncementDoc, type AnnouncementStatus, type Board,
} from "../lib/announcements";
import { fileToBannerDataUrl, AnnouncementImageError } from "../lib/announcementImage";
import { notifyParish, MAX_PARISH_NOTIFICATIONS } from "../lib/notifications";
import AdminNav from "./AdminNav";
import type { AdminSession } from "./AdminApp";

/**
 * Announcements, Feast Days and Events — one page, three boards.
 *
 * ## Why not three pages
 *
 * They are the same document with a different chip on it: a title, a
 * date, a picture, and a published/draft/archived life. Three
 * collections would have been three sets of security rules, three admin
 * screens and three places for the next bug to hide. The nav still shows
 * them as three places, because that is how a parish thinks about them.
 *
```

```
 * ## Why archive and not delete
 *
 * Delete is here, but behind a confirmation and only for drafts and
 * archived items. A published announcement that has been on five hundred
 * home screens is a record of what the parish said; "Archive" takes it
 * down without pretending it never happened.
 */
export default function AdminAnnouncements({ session }: { session: AdminSession }) {
  const { board: boardParam } = useParams<{ board?: string }>();
  const board: Board =
    boardParam === "feasts" ? "feasts" : boardParam === "events" ? "events" : "announcements";

  const churchId = session.profile.churchId;
  const [items, setItems] = useState<AnnouncementDoc[] | null>(null);
  const [listError, setListError] = useState<string | null>(null);
  const [editing, setEditing] = useState<AnnouncementDoc | null>(null);

  useEffect(() => {
    const q = query(collection(db, "announcements"), where("churchId", "==", churchId));
    return onSnapshot(
      q,
      snap => setItems(snap.docs.map(d => ({ ...(d.data() as AnnouncementDoc), id: d.id }))),
      err => setListError(err.message),
    );
  }, [churchId]);

  // Leaving the board must not leave an edit form open over a document
  // the new board does not show.
  useEffect(() => { setEditing(null); }, [board]);

  const { current, archived } = useMemo(() => {
    const mine = (items ?? [])
      .filter(a => onBoard(a, board))
      .sort((a, b) => (b.createdAt ?? "").localeCompare(a.createdAt ?? ""));
    return {
      current: mine.filter(a => statusOf(a) !== "archived"),
      archived: mine.filter(a => statusOf(a) === "archived"),
    };
  }, [items, board]);

  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)]">
      <div className="max-w-[820px] mx-auto px-4 py-6">
        <AdminNav />

        <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
          {BOARD_LABEL[board]}
        </h1>
        <p className="text-[15px] text-[var(--color-brand-secondary)]">
          {session.church?.name ?? churchId}
          {board === "announcements"
            ? " — everything you have posted. Published items appear in the Parish Bulletin on Home."
            : board === "feasts"
              ? " — feast days, shown to pilgrims with the date and where they are held."
              : " — parish events, shown to pilgrims with the date, time and place."}
        </p>

        <ComposeForm
          key={editing?.id ?? `new-${board}`}
```

```
          board={board}
          churchId={churchId}
          authorEmail={session.user.email ?? ""}
          editing={editing}
          onDone={() => setEditing(null)}
        />

        {listError && (
          <p role="alert" className="mt-5 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
            {listError}
          </p>
        )}

        {items === null ? (
          <div className="py-12 text-center">
            <Loader2 className="mx-auto w-6 h-6 animate-spin text-[var(--color-brand-primary)]" />
          </div>
        ) : (
          <>
            <BoardList
              title={`Current ${BOARD_LABEL[board].toLowerCase()}`}
              count={current.length}
              empty={
                <>
                  <Megaphone className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
                  <p className="mt-2 text-[15px] text-[var(--color-brand-secondary)]">
                    Nothing posted yet. Whatever you publish above appears on every
                    pilgrim&apos;s home screen straight away.
                  </p>
                </>
              }
            >
              {current.map(a => (
                <Row key={a.id} item={a} authorEmail={session.user.email ?? ""}
                     onEdit={() => setEditing(a)} />
              ))}
            </BoardList>

            {/* Only when there is something in it. An empty "Archived"
                heading on a parish that has never archived anything is a
                question nobody asked. */}
            {archived.length > 0 && (
              <BoardList title="Archived" count={archived.length} tone="muted" empty={null}>
                {archived.map(a => (
                  <Row key={a.id} item={a} authorEmail={session.user.email ?? ""}
                       onEdit={() => setEditing(a)} />
                ))}
              </BoardList>
            )}
          </>
        )}
      </div>
    </div>
  );
}

function BoardList({
  title, count, children, empty, tone,
}: {
```

```
  title: string;
  count: number;
  children: ReactNode;
  empty: ReactNode;
  tone?: "muted";
}) {
  return (
    <section className={`mt-8${tone === "muted" ? " opacity-90" : ""}`}>
      <h2 className="mb-3 text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">
        {title}{" "}
        {count > 0 && (
          <span className="not-italic font-sans text-[15px] text-[var(--color-brand-secondary)]">
            ({count})
          </span>
        )}
      </h2>
      {count === 0 ? (
        <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] py-10 text-center px-6">
          {empty}
        </div>
      ) : (
        <ul className="space-y-3">{children}</ul>
      )}
    </section>
  );
}

function ComposeForm({
  board, churchId, authorEmail, editing, onDone,
}: {
  board: Board;
  churchId: string;
  authorEmail: string;
  editing: AnnouncementDoc | null;
  onDone: () => void;
}) {
  // A feast posted from the Feast Days board is a feast. Making someone
  // pick "Feast" from a dropdown on a page called Feast Days is asking a
  // question whose answer is the page they are on.
  const defaultType =
    board === "feasts" ? FEAST_TYPE : board === "events" ? EVENT_TYPE : ANNOUNCEMENT_TYPES[0];

  const [title, setTitle] = useState(editing?.title ?? "");
  const [type, setType] = useState<string>(editing?.type ?? defaultType);
  const [date, setDate] = useState(editing?.date ?? "");
  const [time, setTime] = useState(editing?.time ?? "");
  const [location, setLocation] = useState(editing?.location ?? "");
  const [body, setBody] = useState(editing?.body ?? "");
  const [imageUrl, setImageUrl] = useState(editing?.imageUrl ?? "");
  const [busy, setBusy] = useState<"photo" | "save" | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [done, setDone] = useState<string | null>(null);
  // Off by default, and asked rather than assumed. The surest way to
  // make people ignore notifications is to send one that did not matter.
  const [notify, setNotify] = useState(false);
  const fileRef = useRef<HTMLInputElement | null>(null);

  async function choosePhoto(file: File) {
    setBusy("photo");
```

```
  setError(null);
  try {
    setImageUrl(await fileToBannerDataUrl(file));
  } catch (err) {
    setError(err instanceof AnnouncementImageError ? err.message : "That photo could not be read.");
  } finally {
    setBusy(null);
  }
}

async function save(status: AnnouncementStatus) {
  if (!title.trim()) { setError("Give it a title."); return; }
  if (!date) { setError("Choose the date this is for."); return; }
  setBusy("save");
  setError(null);
  try {
    if (editing) {
      await updateAnnouncement(
        editing.id,
        { title, body, date, time, type, location, imageUrl, status },
        authorEmail,
      );
      onDone();
      return;
    }
    await createAnnouncement(
      { churchId, title, body, date, time, type, location, imageUrl, status },
      authorEmail,
    );
    setTitle(""); setBody(""); setDate(""); setTime(""); setLocation(""); setImageUrl("");
    setType(defaultType);
    let note = status === "draft"
      ? "Saved as a draft. Nobody can see it until you publish it."
      : "Published. It is on the home screen now.";

    // Only on publish. A draft nobody can see must not ring anyone's
    // phone, whatever the checkbox says.
    if (notify && status === "published") {
      const result = await notifyParish(
        churchId,
        titleFor(board),
        `${title.trim()} — see the Parish Bulletin for details.`,
      );
      note += result.tooManyFor
        ? ` Nobody was notified: this parish has ${result.tooManyFor} members, more than the ${MAX_PARISH_NOTIFICATIONS} this can message at once.`
        : ` ${result.sent} ${result.sent === 1 ? "person was" : "people were"} notified.`;
      setNotify(false);
    }

    setDone(note);
    window.setTimeout(() => setDone(null), 6000);
  } catch {
    setError("That could not be saved. Check your connection and try again.");
  } finally {
    setBusy(null);
  }
}

const noun = board === "feasts" ? "feast day" : board === "events" ? "event" : "announcement";
```

```
return (
  <section className="mt-6 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-5">
    <div className="mb-4 flex items-center justify-between gap-3">
      <h2 className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        {editing ? `Editing "${editing.title}"` : `Add a ${noun}`}
      </h2>
      {editing && (
        <button type="button" onClick={onDone}
                className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-secondary)]">
          <X className="w-3.5 h-3.5" /> Cancel
        </button>
      )}
    </div>

    {error && (
      <p role="alert" className="mb-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
        {error}
      </p>
    )}

    <Field label="Title" htmlFor="ann-title">
      <input id="ann-title" value={title} maxLength={90}
             onChange={e => setTitle(e.target.value)}
             placeholder={board === "feasts" ? "Feast of Mary Help of Christians" : "Sunday Mass schedule update"}
             className={INPUT} />
    </Field>

    <div className="grid sm:grid-cols-3 gap-3">
      <Field label="Kind" htmlFor="ann-type">
        <select id="ann-type" value={type} onChange={e => setType(e.target.value)} className={INPUT}>
          {ANNOUNCEMENT_TYPES.map(t => <option key={t} value={t}>{t}</option>)}
        </select>
      </Field>
      <Field label="Date" htmlFor="ann-date">
        <input id="ann-date" type="date" value={date}
               onChange={e => setDate(e.target.value)} className={INPUT} />
      </Field>
      <Field label="Time (optional)" htmlFor="ann-time">
        <input id="ann-time" type="time" value={time}
               onChange={e => setTime(e.target.value)} className={INPUT} />
      </Field>
    </div>

    <Field label="Where (optional)" htmlFor="ann-where">
      <input id="ann-where" value={location} maxLength={120}
             onChange={e => setLocation(e.target.value)}
             placeholder="Parish church, parish hall, the grounds…"
             className={INPUT} />
    </Field>

    <Field label="Details (optional)" htmlFor="ann-body">
      <textarea id="ann-body" value={body} rows={3} maxLength={600}
                onChange={e => setBody(e.target.value)}
                placeholder="A sentence or two for the pilgrims."
                className={INPUT} />
    </Field>

    <Field label="Photo (optional)" htmlFor="">
```

```
        {imageUrl ? (
          <div className="relative w-full max-w-[360px]">
            <img src={imageUrl} alt="" className="w-full aspect-video object-cover rounded-xl border border-[var(--color-brand-border)]" />
            <button type="button" onClick={() => setImageUrl("")} aria-label="Remove this photo"
                    className="absolute top-2 left-2 w-8 h-8 rounded-full bg-black/60 text-white flex items-center justify-center">
              <X className="w-4 h-4" />
            </button>
          </div>
        ) : (
          <button type="button" onClick={() => fileRef.current?.click()} disabled={busy === "photo"}
                  className="inline-flex items-center gap-2 rounded-full border-[1.5px] border-[var(--color-brand-border)] px-4 py-2.5 text-[15px] font-bold text-
[var(--color-brand-text)]">
            {busy === "photo" ? <Loader2 className="w-4 h-4 animate-spin" /> : <ImagePlus className="w-4 h-4" />}
            Choose a photo
          </button>
        )}
        <input ref={fileRef} type="file" accept="image/*" className="hidden"
               onChange={e => {
                 const file = e.target.files?.[0];
                 e.target.value = "";
                 if (file) void choosePhoto(file);
               }} />
      </Field>

      <label className="mb-4 flex items-start gap-2.5 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
        <input
          type="checkbox"
          checked={notify}
          onChange={e => setNotify(e.target.checked)}
          className="mt-0.5 w-5 h-5 shrink-0 accent-[var(--color-brand-primary)]"
        />
        <span>
          Send this to parish users as a notification
          <span className="block text-[14px] text-[var(--color-brand-secondary)]">
            Only when you publish. Leave it off for small corrections.
          </span>
        </span>
      </label>

      <div className="mt-5 flex items-center gap-2.5 flex-wrap">
        <button type="button" onClick={() => void save("published")} disabled={busy !== null}
                className="inline-flex items-center gap-2 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-5 py-3 text-[15px]
font-bold disabled:opacity-60">
          {busy === "save" ? <Loader2 className="w-4 h-4 animate-spin" /> : <Eye className="w-4 h-4" />}
          {editing ? "Save and publish" : "Publish now"}
        </button>
        {/* Saving a draft is the quieter of the two, so it is the quieter
            button. A parish writing up the fiesta over three days should
            not have to publish it to keep it. */}
        <button type="button" onClick={() => void save("draft")} disabled={busy !== null}
                className="inline-flex items-center gap-2 rounded-full border-[1.5px] border-[var(--color-brand-border)] px-5 py-3 text-[15px] font-bold text-[var(--
color-brand-text)] disabled:opacity-60">
          <FileEdit className="w-4 h-4" /> Save as draft
        </button>
        {done && (
          <span role="status" className="text-[15px] font-semibold text-[var(--color-brand-success)]">
            {done}
          </span>
        )}
```

```
      </div>
    </section>
  );
}

function Row({
  item, authorEmail, onEdit,
}: { item: AnnouncementDoc; authorEmail: string; onEdit: () => void }) {
  const [confirming, setConfirming] = useState(false);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const status = statusOf(item);
  const when = item.date ? new Date(item.date) : null;
  const dated = when && !Number.isNaN(when.getTime()) ? when : null;
  const past = dated !== null && dated.getTime() < startOfDay(new Date());

  async function setStatus(next: AnnouncementStatus) {
    setBusy(true);
    setError(null);
    try {
      await updateAnnouncement(item.id, { status: next }, authorEmail);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(false);
    }
  }

  async function remove() {
    setBusy(true);
    try {
      await deleteAnnouncement(item.id);
    } catch {
      setError("That could not be removed. Check your connection and try again.");
      setBusy(false);
      setConfirming(false);
    }
  }

  return (
    <li className={`rounded-2xl border bg-[var(--color-brand-card)] overflow-hidden ${
      status === "archived"
        ? "border-dashed border-[var(--color-brand-border)]"
        : "border-[var(--color-brand-border)]"
    }`}>
      <div className="flex gap-3 p-4">
        {item.imageUrl && (
          <img src={item.imageUrl} alt=""
               className={`w-[108px] h-[72px] shrink-0 rounded-lg object-cover${
                 status === "archived" ? " grayscale opacity-70" : ""
               }`} />
        )}

        <div className="min-w-0 flex-1">
          <div className="flex flex-wrap items-center gap-2">
            <span className="inline-flex items-center gap-1.5 text-[13px] font-bold px-2.5 py-1 rounded-md bg-[var(--color-brand-primary)] text-[var(--color-brand-
on-accent)]">
              <CalendarDays className="w-3.5 h-3.5" />
```

```
        {item.type}
      </span>
      <StatusChip status={status} />
    </div>

    <h3 className="mt-2 text-[16px] font-bold leading-snug text-[var(--color-brand-text)]">
      {item.title}
    </h3>
    <p className="mt-0.5 text-[14px] text-[var(--color-brand-secondary)]">
      {dated ? formatWhen(dated) : "No date"}
      {item.time ? ` · ${item.time}` : ""}
      {/* Said plainly, so an admin wondering why their announcement
          vanished from Home gets the answer here. */}
      {past && status === "published" && " · past, no longer shown on Home"}
    </p>
    {item.location && (
      <p className="mt-0.5 inline-flex items-center gap-1.5 text-[14px] text-[var(--color-brand-secondary)]">
        <MapPin className="w-3.5 h-3.5" /> {item.location}
      </p>
    )}
    {item.body && (
      <p className="mt-1.5 text-[14px] leading-relaxed text-[var(--color-brand-text)]">{item.body}</p>
    )}
    {error && <p role="alert" className="mt-2 text-[14px] text-[var(--color-brand-error)]">{error}</p>}

    <div className="mt-3 flex flex-wrap gap-x-4 gap-y-2">
      <Action onClick={onEdit} icon={<FileEdit className="w-3.5 h-3.5" />} label="Edit" />

      {status === "draft" && (
        <Action onClick={() => void setStatus("published")} busy={busy}
                icon={<Eye className="w-3.5 h-3.5" />} label="Publish" primary />
      )}
      {status === "published" && (
        <Action onClick={() => void setStatus("archived")} busy={busy}
                icon={<Archive className="w-3.5 h-3.5" />} label="Archive" />
      )}
      {status === "archived" && (
        <Action onClick={() => void setStatus("published")} busy={busy}
                icon={<Undo2 className="w-3.5 h-3.5" />} label="Restore" primary />
      )}

      {/* Delete only once it is off the home screen. Something five
          hundred people have read is a record of what the parish
          said; archive takes it down without erasing it. */}
      {status !== "published" && (
        <Action onClick={() => setConfirming(true)} danger
                icon={<Trash2 className="w-3.5 h-3.5" />} label="Delete" />
      )}
    </div>
  </div>
</div>

{confirming && (
  <div role="dialog" aria-modal="true" aria-label="Delete permanently"
       className="fixed inset-0 z-50 flex items-center justify-center bg-black/40 px-5">
    <div className="w-full max-w-[380px] rounded-[22px] bg-[var(--color-brand-card)] p-6">
      <h3 className="text-[18px] font-bold font-serif italic">Delete this permanently?</h3>
      <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
        <strong>{item.title}</strong> will be gone for good. If you only want it
```

```
              off the home screen, archive it instead — it stays here and you can
              restore it later.
            </p>
            <div className="mt-5 flex gap-2">
              <button onClick={() => setConfirming(false)} disabled={busy}
                      className="flex-1 rounded-full border-[1.5px] border-[var(--color-brand-border)] py-3 text-[15px] font-bold">
                Keep it
              </button>
              <button onClick={() => void remove()} disabled={busy}
                      className="flex-1 rounded-full bg-[var(--color-brand-error)] text-white py-3 text-[15px] font-bold disabled:opacity-60">
                {busy ? "Deleting…" : "Yes, delete"}
              </button>
            </div>
          </div>
        </div>
      )}
    </li>
  );
}

function StatusChip({ status }: { status: AnnouncementStatus }) {
  const tone =
    status === "published" ? { bg: "#E4F0E8", fg: "#2F5A41", br: "#BFD6C8" }
    : status === "draft" ? { bg: "#F6F0E2", fg: "#6E5010", br: "#E0D2AE" }
    : { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-secondary)", br: "var(--color-brand-border)" };

  return (
    <span className="text-[13px] font-bold px-2.5 py-1 rounded-full border"
          style={{ background: tone.bg, color: tone.fg, borderColor: tone.br }}>
      {STATUS_LABEL[status]}
    </span>
  );
}

function Action({
  onClick, icon, label, primary, danger, busy,
}: {
  onClick: () => void;
  icon: ReactNode;
  label: string;
  primary?: boolean;
  danger?: boolean;
  busy?: boolean;
}) {
  return (
    <button type="button" onClick={onClick} disabled={busy}
            className={`inline-flex items-center gap-1.5 text-[14px] font-bold disabled:opacity-50 ${
              danger ? "text-[var(--color-brand-error)]"
              : primary ? "text-[var(--color-brand-primary)]"
              : "text-[var(--color-brand-secondary)]"
            }`}>
      {busy ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : icon}
      {label}
    </button>
  );
}

/** What the notification says it is about. */
function titleFor(board: Board): string {
```

```
  if (board === "feasts") return "Upcoming feast day";
  if (board === "events") return "Parish event";
  return "Parish announcement";
}

const INPUT =
  "w-full rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-bg)] px-3 py-2.5 text-[16px] text-[var(--color-brand-text)]";

function Field({ label, htmlFor, children }: { label: string; htmlFor: string; children: ReactNode }) {
  return (
    <div className="mb-4">
      <label htmlFor={htmlFor || undefined} className="block mb-1.5 text-[14px] font-bold text-[var(--color-brand-secondary)]">
        {label}
      </label>
      {children}
    </div>
  );
}
```

2.3.3 API-010 — AdminApp.tsx

Code ID	API-010
File Name	AdminApp.tsx
File Path	SanctDemo/src/admin/AdminApp.tsx
Language / Type	TypeScript (React, TSX)
Lines	135
Functions / Classes	AdminSession (exported type); AdminApp (exported component/function (default)); Shell (internal function)
Related Files (its imports)	../lib/firebase, ../lib/userProfile, ../lib/churches, ./AdminLogin, ./AdminDashboard, ./AdminApplicationDetail, ./AdminApplications, ./AdminAnnouncements, ./AdminAnalytics, ./AdminItems, ./AdminMassSchedule, ../types
Where It Is Used	src/admin/AdminAnalytics.tsx, src/admin/AdminAnnouncements.tsx, src/admin/AdminApplicationDetail.tsx, src/admin/AdminApplications.tsx, src/admin/AdminAvailability.tsx, src/admin/AdminContent.tsx, src/admin/AdminDashboard.tsx, src/admin/AdminItems.tsx, src/admin/AdminMassSchedule.tsx, src/main.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminApp.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminApp.tsx

```
import { useEffect, useState, type ReactNode } from "react";
import { Routes, Route, Navigate } from "react-router-dom";
import { onAuthStateChanged, type User } from "firebase/auth";
import { Loader2, ShieldAlert } from "lucide-react";
import { auth } from "../lib/firebase";
import { getProfile } from "../lib/userProfile";
import { getChurch } from "../lib/churches";
import AdminLogin from "./AdminLogin";
import AdminDashboard from "./AdminDashboard";
import AdminApplicationDetail from "./AdminApplicationDetail";
import AdminApplications from "./AdminApplications";
import AdminAnnouncements from "./AdminAnnouncements";
import AdminAnalytics from "./AdminAnalytics";
import AdminItems from "./AdminItems";
import AdminMassSchedule from "./AdminMassSchedule";
import type { Church, UserProfile } from "../types";

/**
 * The parish office, at /admin.
 *
 * ## What this gate is and is not
 *
 * It decides what to RENDER. It is not what stops an ordinary pilgrim
 * reading another parish's applications - firestore.rules does that, on
```

```
 * Google's servers, for every request whatever the client is. Someone who
 * reaches /admin/dashboard by typing it gets a shell that cannot load a
 * single document.
 *
 * It exists so the person sees an honest "this account is not an
 * administrator" instead of a dashboard full of permission errors, which
 * looks like the app is broken rather than like it is working.
 */
export interface AdminSession {
  user: User;
  profile: UserProfile;
  church: Church | null;
}

export default function AdminApp() {
  const [session, setSession] = useState<AdminSession | null>(null);
  const [state, setState] = useState<"loading" | "out" | "not-admin" | "in">("loading");

  useEffect(() => {
    return onAuthStateChanged(auth, async user => {
      if (!user) { setSession(null); setState("out"); return; }

      const profile = await getProfile(user.uid);
      if (!profile || profile.role !== "church_admin" || !profile.churchId) {
        setSession(null);
        setState("not-admin");
        return;
      }

      setSession({ user, profile, church: await getChurch(profile.churchId) });
      setState("in");
    });
  }, []);

  if (state === "loading") {
    return (
      <Shell>
        <Loader2 className="mx-auto w-8 h-8 animate-spin text-[var(--color-brand-primary)]" />
        <p className="mt-3 text-[15px] text-[var(--color-brand-secondary)]">Checking your access…</p>
      </Shell>
    );
  }

  if (state === "not-admin") {
    return (
      <Shell>
        <ShieldAlert className="mx-auto w-8 h-8 text-[var(--color-brand-error)]" />
        <h1 className="mt-3 text-[19px] font-bold font-serif italic">This is the parish office</h1>
        <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
          Your account is signed in, but it does not administer a parish.
        </p>
        <p className="mt-2 text-[14px] text-[var(--color-brand-secondary)]">
          Parish administrators are appointed by the diocese. If that should be you,
          ask them to set it on your account.
        </p>
        <a href="/" className="mt-5 inline-block w-full rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] py-3 font-bold text-[15px]">
          Back to SanctiWalk
        </a>
      </Shell>
```

```
    );
  }

  if (state === "out") {
    return (
      <Routes>
        <Route path="login" element={<AdminLogin />} />
        <Route path="*" element={<Navigate to="/parish-office-gav8thhe/login" replace />} />
      </Routes>
    );
  }

  return (
    <Routes>
      <Route path="login" element={<Navigate to="/parish-office-gav8thhe" replace />} />
      <Route path="/" element={<AdminDashboard session={session!} />} />
      {/* "view/:id" rather than ":id", so a detail route can never be
          mistaken for the "ministry" / "sacrament" list routes. */}
      <Route path="applications" element={<AdminApplications session={session!} />} />
      <Route path="applications/view/:id" element={<AdminApplicationDetail session={session!} />} />
      <Route path="applications/:kind" element={<AdminApplications session={session!} />} />
      <Route path="announcements" element={<AdminAnnouncements session={session!} />} />
      {/* Feast Days and Events are the same page on a different board;
          see AdminAnnouncements for why they are not three collections. */}
      <Route path="announcements/:board" element={<AdminAnnouncements session={session!} />} />
      <Route path="mass-schedule" element={<AdminMassSchedule session={session!} />} />
      {/* Ministries and Sacraments: one page each, carrying both the
          description people read and the switch that decides whether
          they can apply. The two pages these replaced - "What people
          read" and "Open & Closed" - each listed both kinds, so doing
          one afternoon's work on the choir meant visiting both. */}
      <Route path="ministries" element={<AdminItems session={session!} kind="ministry" />} />
      <Route path="sacraments" element={<AdminItems session={session!} kind="sacrament" />} />

      {/* Old links, including anything bookmarked by the parish office. */}
      <Route path="content" element={<Navigate to="/parish-office-gav8thhe/ministries" replace />} />
      <Route path="availability" element={<Navigate to="/parish-office-gav8thhe/ministries" replace />} />
      <Route path="analytics" element={<AdminAnalytics session={session!} />} />
      <Route path="*" element={<Navigate to="/parish-office-gav8thhe" replace />} />
    </Routes>
  );
}

function Shell({ children }: { children: ReactNode }) {
  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)] flex items-center justify-center px-5">
      <div className="w-full max-w-[420px] rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-6 text-center">
        {children}
      </div>
    </div>
  );
}
```

2.3.4 API-011 — AdminApplicationDetail.tsx

Code ID	API-011
File Name	AdminApplicationDetail.tsx
File Path	SanctDemo/src/admin/AdminApplicationDetail.tsx
Language / Type	TypeScript (React, TSX)
Lines	307
Functions / Classes	AdminApplicationDetail (exported component/function (default)); Frame (internal function); Section (internal function); Row (internal function); Action (internal function); ConfirmDialog (internal function); humanise (internal function); renderValue (internal function); formatWhen (internal function)
Related Files (its imports)	../lib/firebase, ../lib/applications, ./StatusBadge, ./AdminApp, ../types
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminApplicationDetail.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminApplicationDetail.tsx

```
import { Fragment, useEffect, useState, type ReactNode } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { doc, onSnapshot } from "firebase/firestore";
import { ArrowLeft, Loader2, ShieldAlert, Check, X, Clock, FileCheck } from "lucide-react";
import { db } from "../lib/firebase";
import { setApplicationStatus, STATUS_LABEL, isClosed } from "../lib/applications";
import StatusBadge from "./StatusBadge";
import type { AdminSession } from "./AdminApp";
import type { ApplicationDoc, ApplicationStatus } from "../types";

/**
 * One application, and the decision on it.
 *
 * ## Why a live subscription rather than a single read
 *
 * A parish can have more than one person in the office. Two reviewers on
 * the same application should not each be looking at a snapshot from when
 * they opened it, deciding against a state that has since changed. The
 * document streams, so a decision made elsewhere appears here.
 *
 * ## Why rejection asks and approval does not
 *
 * Approving moves an application forward and can be followed by rejecting
 * it later. Rejecting is what the applicant reads as "no", and it is the
 * one a slip of the thumb should not be able to do. The confirm step is on
 * the irreversible-feeling action only; putting one on every button
 * teaches people to dismiss them.
 */
export default function AdminApplicationDetail({ session }: { session: AdminSession }) {
```

```
const { id } = useParams<{ id: string }>();
const navigate = useNavigate();

const [app, setApp] = useState<ApplicationDoc | null | "missing">(null);
const [error, setError] = useState<string | null>(null);
const [busy, setBusy] = useState(false);
const [confirming, setConfirming] = useState<ApplicationStatus | null>(null);
const [note, setNote] = useState("");

useEffect(() => {
  if (!id) return;
  return onSnapshot(
    doc(db, "applications", id),
    snap => setApp(snap.exists() ? { id: snap.id, ...(snap.data() as ApplicationDoc) } : "missing"),
    // A denial here is the rules refusing another parish's document, which
    // is the system working. Say so plainly rather than showing a stack.
    () => setApp("missing"),
  );
}, [id]);

async function decide(status: ApplicationStatus) {
  // The document, not the union: `app` is also "missing" and null while
  // loading, and neither has an applicant to tell.
  const current = app && app !== "missing" ? app : null;
  if (!id || !current) return;
  setBusy(true);
  setError(null);
  try {
    await setApplicationStatus(id, status, note.trim() || undefined, {
      uid: current.uid,
      churchId: current.churchId,
      what: current.type,
    });
    setNote("");
    setConfirming(null);
  } catch (err) {
    setError(err instanceof Error ? err.message : "Could not save that decision.");
  } finally {
    setBusy(false);
  }
}

if (app === null) {
  return <Frame onBack={() => navigate("/parish-office-gav8thhe")}>
    <div className="py-16 text-center">
      <Loader2 className="mx-auto w-7 h-7 animate-spin text-[var(--color-brand-primary)]" />
    </div>
  </Frame>;
}

if (app === "missing") {
  return <Frame onBack={() => navigate("/parish-office-gav8thhe")}>
    <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] py-12 text-center px-6">
      <ShieldAlert className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
      <p className="mt-2 text-[16px] font-bold">This application is not available</p>
      <p className="mt-1 text-[14px] text-[var(--color-brand-secondary)]">
        It may have been removed, or it belongs to another parish.
      </p>
    </div>
```

```
  </Frame>;
}

const closed = isClosed(app.status);

return (
  <Frame onBack={() => navigate("/parish-office-gav8thhe")}>
    <div className="flex flex-wrap items-center gap-3">
      <h1 className="text-[22px] font-bold font-serif italic text-[var(--color-brand-text)]">
        {app.type}
      </h1>
      <StatusBadge status={app.status} />
    </div>
    <p className="mt-1 font-mono text-[14px] text-[var(--color-brand-secondary)]">
      {app.referenceNumber ?? app.id}
    </p>

    {error && (
      <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
        {error}
      </p>
    )}

    <Section title="Applicant">
      <Row label="Name" value={app.applicantName} />
      <Row label="Email" value={app.applicantEmail} />
      <Row label="Parish" value={session.church?.name ?? app.churchId} />
      <Row label="Submitted" value={formatWhen(app.createdAt)} />
    </Section>

    {app.formData && Object.keys(app.formData).length > 0 && (
      <Section title={app.kind === "sacrament" ? "Sacrament details" : "Ministry details"}>
        {Object.entries(app.formData).map(([key, value]) => (
          // Keyed on a Fragment rather than on Row: React 19's types take
          // `key` out of props, so putting it on a plain component is an
          // error unless the component declares it - and a component that
          // declares `key` is lying about what it receives.
          <Fragment key={key}>
            <Row label={humanise(key)} value={renderValue(value)} />
          </Fragment>
        ))}
      </Section>
    )}

    {app.adminNote && (
      <Section title="Office note">
        <p className="text-[15px] leading-relaxed text-[var(--color-brand-text)]">{app.adminNote}</p>
      </Section>
    )}

    {app.history && app.history.length > 0 && (
      <Section title="History">
        <ol className="space-y-2.5">
          {[...app.history].reverse().map((h, i) => (
            <li key={i} className="flex gap-3 items-start">
              <span className="mt-1.5 h-2 w-2 shrink-0 rounded-full bg-[var(--color-brand-primary)]" />
              <span className="min-w-0">
                <span className="block text-[15px] font-bold">{STATUS_LABEL[h.status] ?? h.status}</span>
                <span className="block text-[13px] text-[var(--color-brand-secondary)]">{formatWhen(h.at)}</span>
```

```
                {h.note && <span className="block text-[14px] mt-0.5">{h.note}</span>}
              </span>
            </li>
          ))}
        </ol>
      </Section>
    )}

    <Section title="Decision">
      {closed && (
        <p className="mb-3 text-[14px] text-[var(--color-brand-secondary)]">
          This application is {STATUS_LABEL[app.status].toLowerCase()}. You can still reopen it
          by putting it back under review.
        </p>
      )}

      <label htmlFor="note" className="block text-[14px] font-bold mb-1.5">
        Note for the record <span className="font-normal text-[var(--color-brand-secondary)]">(optional)</span>
      </label>
      <textarea
        id="note"
        rows={2}
        value={note}
        onChange={e => setNote(e.target.value)}
        placeholder="Why, or what happens next."
        className="w-full rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-4 py-3 text-[15px]"
      />

      <div className="mt-3 flex flex-wrap gap-2">
        <Action icon={<Clock className="w-4 h-4" />} label="Under review"
                disabled={busy || app.status === "under_review"}
                onClick={() => void decide("under_review")} />
        <Action icon={<Check className="w-4 h-4" />} label="Approve" primary
                disabled={busy || app.status === "approved"}
                onClick={() => void decide("approved")} />
        <Action icon={<FileCheck className="w-4 h-4" />} label="Completed"
                disabled={busy || app.status === "completed"}
                onClick={() => void decide("completed")} />
        <Action icon={<X className="w-4 h-4" />} label="Reject" danger
                disabled={busy || app.status === "rejected"}
                onClick={() => setConfirming("rejected")} />
      </div>
    </Section>

    {confirming === "rejected" && (
      <ConfirmDialog
        title="Reject this application?"
        body={`${app.applicantName || "The applicant"} will see this marked as rejected. You can reopen it afterwards, but they may have read it by then.`}
        confirmLabel={busy ? "Rejecting…" : "Yes, reject"}
        busy={busy}
        onCancel={() => setConfirming(null)}
        onConfirm={() => void decide("rejected")}
      />
    )}
  </Frame>
 );
}

function Frame({ children, onBack }: { children: ReactNode; onBack: () => void }) {
```

```
  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)]">
      <div className="max-w-[760px] mx-auto px-4 py-6">
        <button onClick={onBack}
                className="mb-4 inline-flex items-center gap-1.5 text-[15px] font-semibold text-[var(--color-brand-primary)]">
          <ArrowLeft className="w-4 h-4" /> All applications
        </button>
        {children}
      </div>
    </div>
  );
}

function Section({ title, children }: { title: string; children: ReactNode }) {
  return (
    <section className="mt-6 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-5">
      <h2 className="mb-3 text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        {title}
      </h2>
      {children}
    </section>
  );
}

function Row({ label, value }: { label: string; value: string }) {
  return (
    <div className="flex flex-wrap gap-x-3 gap-y-0.5 py-1.5 border-b border-[var(--color-brand-border)] last:border-0">
      <span className="w-[150px] shrink-0 text-[14px] text-[var(--color-brand-secondary)]">{label}</span>
      <span className="min-w-0 flex-1 text-[15px] text-[var(--color-brand-text)] break-words">{value || "—"}</span>
    </div>
  );
}

function Action({
  label, icon, onClick, disabled, primary, danger,
}: {
  label: string; icon: ReactNode; onClick: () => void;
  disabled?: boolean; primary?: boolean; danger?: boolean;
}) {
  const base = "inline-flex items-center gap-1.5 rounded-full px-4 py-2.5 text-[15px] font-bold disabled:opacity-40";
  const style = primary
    ? "bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)]"
    : danger
      ? "border-[1.5px] border-[var(--color-brand-error)] text-[var(--color-brand-error)]"
      : "border-[1.5px] border-[var(--color-brand-primary)] text-[var(--color-brand-primary)]";
  return (
    <button type="button" onClick={onClick} disabled={disabled} className={`${base} ${style}`}>
      {icon}{label}
    </button>
  );
}

function ConfirmDialog({
  title, body, confirmLabel, busy, onCancel, onConfirm,
}: {
  title: string; body: string; confirmLabel: string; busy: boolean;
  onCancel: () => void; onConfirm: () => void;
}) {
  return (
```

```
    <div role="dialog" aria-modal="true" aria-label={title}
         className="fixed inset-0 z-50 flex items-center justify-center bg-black/40 px-5">
      <div className="w-full max-w-[400px] rounded-[22px] bg-[var(--color-brand-card)] p-6">
        <h3 className="text-[18px] font-bold font-serif italic">{title}</h3>
        <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-text)]">{body}</p>
        <div className="mt-5 flex gap-2">
          <button onClick={onCancel} disabled={busy}
                  className="flex-1 rounded-full border-[1.5px] border-[var(--color-brand-border)] py-3 text-[15px] font-bold">
            Cancel
          </button>
          <button onClick={onConfirm} disabled={busy}
                  className="flex-1 rounded-full bg-[var(--color-brand-error)] text-white py-3 text-[15px] font-bold disabled:opacity-60">
            {confirmLabel}
          </button>
        </div>
      </div>
    </div>
  );
}

/** "preferredDate" reads as "Preferred date" rather than as a variable. */
function humanise(key: string): string {
  const spaced = key.replace(/([a-z0-9])([A-Z])/g, "$1 $2").replace(/[_-]+/g, " ");
  return spaced.charAt(0).toUpperCase() + spaced.slice(1).toLowerCase();
}

function renderValue(value: unknown): string {
  if (typeof value === "boolean") return value ? "Yes" : "No";
  if (value === null || value === undefined || value === "") return "—";
  if (Array.isArray(value)) return value.join(", ");
  if (typeof value === "object") return JSON.stringify(value);
  return String(value);
}

function formatWhen(iso?: string): string {
  if (!iso) return "—";
  const d = new Date(iso);
  if (Number.isNaN(d.getTime())) return iso;
  return d.toLocaleString(undefined, {
    year: "numeric", month: "long", day: "numeric", hour: "numeric", minute: "2-digit",
  });
}
```

2.3.5 API-012 — AdminApplications.tsx

Code ID	API-012
File Name	AdminApplications.tsx
File Path	SanctDemo/src/admin/AdminApplications.tsx
Language / Type	TypeScript (React, TSX)
Lines	386
Functions / Classes	AdminApplications (exported component/function (default)); Th (internal function); Select (internal function); DateField (internal function); Action (internal function); shortDate (internal function)
Related Files (its imports)	../lib/firebase, ../lib/applications, ../lib/exportApplications, ./StatusBadge, ./AdminNav, ./AdminApp, ../types
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminApplications.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminApplications.tsx

```
import { useEffect, useMemo, useState, type ReactNode } from "react";
import { useNavigate, useParams } from "react-router-dom";
import { collection, onSnapshot, query, where } from "firebase/firestore";
import { Download, FileSpreadsheet, Inbox, Loader2, Printer, Search, X } from "lucide-react";
import { db } from "../lib/firebase";
import { STATUS_ORDER, STATUS_LABEL } from "../lib/applications";
import {
  applyFilters, sortApplications, typesPresent, yearsPresent, activeFilterCount,
  isArchived, MONTHS, NO_FILTERS, type Filters, type Scope, type SortKey,
} from "../lib/applicationFilters";
import { exportExcel, exportCsv } from "../lib/exportApplications";
import StatusBadge from "./StatusBadge";
import AdminNav from "./AdminNav";
import type { AdminSession } from "./AdminApp";
import type { ApplicationDoc, ApplicationStatus } from "../types";

/**
 * The parish's applications, as a table a parish secretary can work.
 *
 * ## Why everything is loaded and filtered here
 *
 * One query, `where churchId == mine` - which is also what the security
 * rules allow and nothing more. Every filter below runs in the browser.
 *
 * The alternative is a Firestore query per filter combination, and each
 * needs its own composite index: churchId+kind+status+createdAt is a
 * different index from churchId+type+status. A missing index does not
 * degrade, it throws, and the page a parish opens on a Monday morning
 * would be a permissions error. A parish has hundreds of applications,
```

```
 * not millions.
 *
 * ## The three views are one component
 *
 * /admin/applications, /ministry and /sacrament differ by a single locked
 * filter. Three components would be three places to fix the next bug.
 */
export default function AdminApplications({ session }: { session: AdminSession }) {
  const { kind } = useParams<{ kind?: string }>();
  const navigate = useNavigate();
  const churchId = session.profile.churchId;

  const locked: Filters["kind"] =
    kind === "ministry" ? "ministry" : kind === "sacrament" ? "sacrament" : null;

  // The archive is a fourth tab on the same route segment rather than a
  // query string, so it can be linked to from the dashboard and bookmarked
  // by a secretary who lives in it.
  const routeScope: Scope = kind === "archive" ? "archived" : "open";

  const [apps, setApps] = useState<ApplicationDoc[] | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [filters, setFilters] = useState<Filters>({ ...NO_FILTERS, kind: locked, scope: routeScope });
  const [sortKey, setSortKey] = useState<SortKey>("submitted");
  const [sortDir, setSortDir] = useState<"asc" | "desc">("desc");

  // The locked filter follows the route, so switching tabs does not carry
  // "Ministry" into the Sacraments view.
  useEffect(() => {
    setFilters(f => ({ ...f, kind: locked, type: null, scope: routeScope }));
  }, [locked, routeScope]);

  useEffect(() => {
    const q = query(collection(db, "applications"), where("churchId", "==", churchId));
    return onSnapshot(
      q,
      snap => setApps(snap.docs.map(d => ({ id: d.id, ...(d.data() as ApplicationDoc) }))),
      err => setError(err.message),
    );
  }, [churchId]);

  const visible = useMemo(() => {
    if (!apps) return [];
    return sortApplications(applyFilters(apps, filters), sortKey, sortDir);
  }, [apps, filters, sortKey, sortDir]);

  const typeOptions = useMemo(
    () => typesPresent(apps ?? [], locked ?? undefined),
    [apps, locked],
  );
  const yearOptions = useMemo(() => yearsPresent(apps ?? []), [apps]);
  // The locked kind is the route, not a filter the admin chose - and on the
  // Archive tab neither is the scope.
  const activeCount = activeFilterCount({
    ...filters, kind: null, scope: routeScope === "archived" ? "open" : filters.scope,
  });

  // What the archive holds, so the tab can say so and an admin can see at
  // a glance that the finished work has gone somewhere rather than away.
```

```
const archivedCount = (apps ?? []).filter(a => isArchived(a.status)).length;

// The heading says which set is on screen, because the Show control can
// change it without changing the tab. "All Applications" over a list
// that was quietly hiding the finished ones was the original complaint.
const title = locked === "ministry" ? "Ministry Applications"
  : locked === "sacrament" ? "Sacrament Applications"
  : filters.scope === "archived" ? "Archive"
  : filters.scope === "all" ? "All Applications"
  : "Open Applications";

function sortBy(key: SortKey) {
  if (key === sortKey) setSortDir(d => (d === "asc" ? "desc" : "asc"));
  else { setSortKey(key); setSortDir(key === "submitted" ? "desc" : "asc"); }
}

return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <div className="max-w-[1100px] mx-auto px-4 py-6">

      <AdminNav />

      <nav className="mb-5 flex flex-wrap gap-2 print:hidden">
        {[
          { to: "/parish-office-gav8thhe/applications", label: "Open", on: locked === null && routeScope === "open" },
          { to: "/parish-office-gav8thhe/applications/ministry", label: "Ministries", on: locked === "ministry" },
          { to: "/parish-office-gav8thhe/applications/sacrament", label: "Sacraments", on: locked === "sacrament" },
          {
            to: "/parish-office-gav8thhe/applications/archive",
            label: archivedCount > 0 ? `Archive (${archivedCount})` : "Archive",
            on: routeScope === "archived",
          },
        ].map(t => (
          <button
            key={t.to}
            onClick={() => navigate(t.to)}
            className={`rounded-full px-4 py-2 text-[15px] font-bold ${
              t.on
                ? "bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)]"
                : "border border-[var(--color-brand-border)] text-[var(--color-brand-secondary)]"
            }`}
          >
            {t.label}
          </button>
        ))}
      </nav>

      <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">{title}</h1>
      <p className="text-[15px] text-[var(--color-brand-secondary)]">
        {session.church?.name ?? churchId}
      </p>

      {error && (
        <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}

      {/* ---- filters ---- */}
```

```
<div className="mt-5 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4 print:hidden">
  <div className="flex flex-wrap gap-2.5 items-end">
    <Select label="Month" value={filters.month ?? ""} onChange={v =>
      setFilters(f => ({ ...f, month: v === "" ? null : Number(v) }))}>
      <option value="">Any month</option>
      {MONTHS.map((m, i) => <option key={m} value={i + 1}>{m}</option>)}
    </Select>

    <Select label="Year" value={filters.year ?? ""} onChange={v =>
      setFilters(f => ({ ...f, year: v === "" ? null : Number(v) }))}>
      <option value="">Any year</option>
      {yearOptions.map(y => <option key={y} value={y}>{y}</option>)}
    </Select>

    <Select
      label={locked === "sacrament" ? "Sacrament" : locked === "ministry" ? "Ministry" : "Ministry / Sacrament"}
      value={filters.type ?? ""}
      onChange={v => setFilters(f => ({ ...f, type: v || null }))}
    >
      <option value="">All</option>
      {typeOptions.map(t => <option key={t} value={t}>{t}</option>)}
    </Select>

    {/* Shown on every tab, not just the archive one: an admin
        looking at Ministries needs the finished ones too, and
        without this the only way to them was a different tab that
        lost the ministry they were looking at. */}
    <Select label="Show" value={filters.scope} onChange={v =>
      setFilters(f => ({ ...f, scope: v as Scope }))}>
      <option value="open">Still open</option>
      <option value="archived">Archived</option>
      <option value="all">Everything</option>
    </Select>

    <Select label="Status" value={filters.status ?? ""} onChange={v =>
      setFilters(f => ({ ...f, status: (v || null) as ApplicationStatus | null }))}>
      <option value="">Any status</option>
      {STATUS_ORDER.map(s => <option key={s} value={s}>{STATUS_LABEL[s]}</option>)}
    </Select>

    <div className="flex-1 min-w-[200px]">
      <label htmlFor="q" className="block text-[13px] font-bold text-[var(--color-brand-secondary)] mb-1">
        Search
      </label>
      <div className="relative">
        <Search className="w-4 h-4 absolute left-3 top-1/2 -translate-y-1/2 text-[var(--color-brand-secondary)]" />
        <input
          id="q"
          value={filters.search}
          onChange={e => setFilters(f => ({ ...f, search: e.target.value }))}
          placeholder="Name, email or reference"
          className="w-full rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] pl-9 pr-3 py-2 text-[15px]"
        />
      </div>
    </div>
  </div>

  <div className="mt-3 flex flex-wrap gap-2.5 items-end">
    <DateField label="From" value={filters.from ?? ""} onChange={v => setFilters(f => ({ ...f, from: v || null }))} />
```

```
            <DateField label="To" value={filters.to ?? ""} onChange={v => setFilters(f => ({ ...f, to: v || null }))} />
            {activeCount > 0 && (
              <button
                onClick={() => setFilters({ ...NO_FILTERS, kind: locked, scope: routeScope })}
                className="inline-flex items-center gap-1.5 rounded-full border border-[var(--color-brand-border)] px-3.5 py-2 text-[14px] font-bold text-[var(--
color-brand-secondary)]"
              >
                <X className="w-3.5 h-3.5" /> Clear {activeCount} filter{activeCount === 1 ? "" : "s"}
              </button>
            )}
          </div>
        </div>

        {/* ---- count + actions ---- */}
        <div className="mt-4 mb-2 flex flex-wrap items-center gap-2">
          <p className="text-[15px] font-bold text-[var(--color-brand-text)]">
            {apps === null ? "Loading…" : `Showing ${visible.length} application${visible.length === 1 ? "" : "s"}`}
            {apps && visible.length !== apps.length && (
              <span className="font-normal text-[var(--color-brand-secondary)]"> of {apps.length}</span>
            )}
          </p>

          <div className="ml-auto flex flex-wrap gap-2 print:hidden">
            <Action icon={<FileSpreadsheet className="w-4 h-4" />} label="Export Excel"
                    disabled={visible.length === 0}
                    onClick={() => exportExcel(visible, filters)} />
            <Action icon={<Download className="w-4 h-4" />} label="Export CSV"
                    disabled={visible.length === 0}
                    onClick={() => exportCsv(visible, filters)} />
            <Action icon={<Printer className="w-4 h-4" />} label="Print"
                    disabled={visible.length === 0}
                    onClick={() => window.print()} />
          </div>
        </div>

        {/* ---- table ---- */}
        {apps === null ? (
          <div className="py-14 text-center">
            <Loader2 className="mx-auto w-6 h-6 animate-spin text-[var(--color-brand-primary)]" />
          </div>
        ) : visible.length === 0 ? (
          <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] py-12 text-center">
            <Inbox className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
            <p className="mt-2 text-[16px] font-bold">
              {apps.length === 0 ? "No applications yet"
                : filters.scope === "archived" ? "Nothing archived yet"
                : activeCount === 0 && filters.scope === "open" ? "Nothing waiting on you"
                : "Nothing matches these filters"}
            </p>
            <p className="mt-1 text-[14px] text-[var(--color-brand-secondary)]">
              {apps.length === 0
                ? "Applications from your parishioners appear here as they are submitted."
                : filters.scope === "archived"
                  ? "Applications move here once you mark them completed or rejected."
                  : activeCount === 0 && filters.scope === "open"
                    // The common case now that the archive exists, and the
                    // one most likely to be misread as a broken page.
                    ? `Every application has been dealt with. ${archivedCount} ${archivedCount === 1 ? "is" : "are"} in the archive.`
                    : "Try widening the month, the status, or clearing the search."}
```

```
          </p>
        </div>
      ) : (
        <div className="overflow-x-auto rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)]">
          <table className="w-full text-left text-[14px]">
            <thead className="text-[13px] uppercase tracking-wider text-[var(--color-brand-secondary)]">
              <tr>
                <Th label="Reference" sortKey="reference" active={sortKey} dir={sortDir} onSort={sortBy} />
                <Th label="Applicant" sortKey="name" active={sortKey} dir={sortDir} onSort={sortBy} />
                <Th label={locked === "sacrament" ? "Sacrament" : locked === "ministry" ? "Ministry" : "Ministry / Sacrament"}
                    sortKey="type" active={sortKey} dir={sortDir} onSort={sortBy} />
                <Th label="Applied" sortKey="submitted" active={sortKey} dir={sortDir} onSort={sortBy} />
                <Th label="Status" sortKey="status" active={sortKey} dir={sortDir} onSort={sortBy} />
              </tr>
            </thead>
            <tbody className="divide-y divide-[var(--color-brand-border)]">
              {visible.map(a => (
                <tr
                  key={a.id}
                  tabIndex={0}
                  role="link"
                  aria-label={`Open ${a.type} from ${a.applicantName}`}
                  onClick={() => navigate(`/admin/applications/view/${a.id}`)}
                  onKeyDown={e => {
                    if (e.key === "Enter" || e.key === " ") {
                      e.preventDefault();
                      navigate(`/admin/applications/view/${a.id}`);
                    }
                  }}
                  className="cursor-pointer hover:bg-[var(--color-brand-card-sunk)] focus:bg-[var(--color-brand-card-sunk)] outline-none"
                >
                  <td className="px-4 py-3 font-mono text-[13px] whitespace-nowrap">{a.referenceNumber ?? a.id}</td>
                  <td className="px-4 py-3">
                    <span className="block font-semibold">{a.applicantName || "—"}</span>
                    <span className="block text-[13px] text-[var(--color-brand-secondary)]">{a.applicantEmail}</span>
                  </td>
                  <td className="px-4 py-3">{a.type}</td>
                  <td className="px-4 py-3 whitespace-nowrap">{shortDate(a.createdAt)}</td>
                  <td className="px-4 py-3"><StatusBadge status={a.status} /></td>
                </tr>
              ))}
            </tbody>
          </table>
        </div>
      )}
    </div>
  </div>
  );
}

function Th({ label, sortKey: key, active, dir, onSort }: {
  label: string; sortKey: SortKey; active: SortKey; dir: "asc" | "desc"; onSort: (k: SortKey) => void;
}) {
  const on = active === key;
  return (
    <th className="px-4 py-3 font-bold">
      <button
        type="button"
        onClick={() => onSort(key)}
```

```
        aria-sort={on ? (dir === "asc" ? "ascending" : "descending") : "none"}
        className="inline-flex items-center gap-1 uppercase tracking-wider"
      >
        {label}
        {/* An arrow only on the sorted column. One on every header is four
            arrows saying nothing. */}
        <span aria-hidden className={on ? "opacity-100" : "opacity-0"}>
          {dir === "asc" ? "▲" : "▼"}
        </span>
      </button>
    </th>
  );
}

function Select({ label, value, onChange, children }: {
  label: string; value: string | number; onChange: (v: string) => void; children: ReactNode;
}) {
  const id = `f-${label.replace(/\W+/g, "-").toLowerCase()}`;
  return (
    <div>
      <label htmlFor={id} className="block text-[13px] font-bold text-[var(--color-brand-secondary)] mb-1">
        {label}
      </label>
      <select
        id={id}
        value={value}
        onChange={e => onChange(e.target.value)}
        className="rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-3 py-2 text-[15px] max-w-[230px]"
      >
        {children}
      </select>
    </div>
  );
}

function DateField({ label, value, onChange }: { label: string; value: string; onChange: (v: string) => void }) {
  const id = `d-${label.toLowerCase()}`;
  return (
    <div>
      <label htmlFor={id} className="block text-[13px] font-bold text-[var(--color-brand-secondary)] mb-1">{label}</label>
      <input id={id} type="date" value={value} onChange={e => onChange(e.target.value)}
             className="rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] px-3 py-2 text-[15px]" />
    </div>
  );
}

function Action({ icon, label, onClick, disabled }: {
  icon: ReactNode; label: string; onClick: () => void; disabled?: boolean;
}) {
  return (
    <button type="button" onClick={onClick} disabled={disabled}
            className="inline-flex items-center gap-1.5 rounded-full border-[1.5px] border-[var(--color-brand-primary)] px-3.5 py-2 text-[14px] font-bold text-[var(-
-color-brand-primary)] disabled:opacity-40">
      {icon}{label}
    </button>
  );
}

function shortDate(iso?: string): string {
```

```
  if (!iso) return "—";
  const d = new Date(iso);
  if (Number.isNaN(d.getTime())) return iso;
  return d.toLocaleDateString(undefined, { year: "numeric", month: "short", day: "numeric" });
}
```

2.3.6 API-013 — AdminAvailability.tsx

Code ID	API-013
File Name	AdminAvailability.tsx
File Path	SanctDemo/src/admin/AdminAvailability.tsx
Language / Type	TypeScript (React, TSX)
Lines	189
Functions / Classes	AdminAvailability (exported component/function (default)); Group (internal function); Row (internal function)
Related Files (its imports)	../data, ../lib/useParishContent, ../lib/parishContent, ../lib/availability, ./AdminApp, ./AdminNav
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/admin/AdminAvailability.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminAvailability.tsx

```
import { useState } from "react";
import { Loader2 } from "lucide-react";
import { SACRAMENTS, ministriesForParish } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { saveParishContent } from "../lib/parishContent";
import { closedIds, isOpenForApplications, withAvailability } from "../lib/availability";
import type { AdminSession } from "./AdminApp";
import AdminNav from "./AdminNav";

/**
 * Turning applications on and off, ministry by ministry.
 *
 * ## Why this is not a "settings" page
 *
 * A parish secretary closing the choir because it is full is doing the
 * single most ordinary thing in this admin. It is one switch per line,
 * worded the way they would say it - "Accepting applications" - and it
 * saves the moment it is pressed. No Save button: there is nothing to
 * compose here, one switch is one decision, and a page of switches with
 * a Save at the bottom is a page people leave without pressing it.
 *
 * ## Why it writes a list and not a flag per ministry
 *
 * See lib/availability.ts. Short version: absent has to mean open, and
 * the security rules need to answer "is this closed" in one expression
 * over one field.
 */
export default function AdminAvailability({ session }: { session: AdminSession }) {
  const churchId = session.profile.churchId;
  const managed = useParishContent(churchId);
```

```
// This parish's ministries, not every parish's.
const parishMinistries = ministriesForParish(churchId);

// Which row is mid-save, so only that switch shows a spinner. A single
// page-wide "saving" flag made every row flicker when one was pressed.
const [saving, setSaving] = useState<string | null>(null);
const [error, setError] = useState<string | null>(null);

async function setOpen(itemId: string, open: boolean) {
  setSaving(itemId);
  setError(null);
  try {
    await saveParishContent(
      churchId,
      { closedApplications: withAvailability(closedIds(managed), itemId, open) },
      session.user.email ?? "",
    );
  } catch {
    setError("That could not be saved. Check your connection and try again.");
  } finally {
    setSaving(null);
  }
}

const ministriesOpen = parishMinistries.filter(m => isOpenForApplications(managed, m.id)).length;
const sacramentsOpen = SACRAMENTS.filter(s => isOpenForApplications(managed, s.id)).length;

return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <div className="max-w-[760px] mx-auto px-4 py-6">
      <AdminNav />

      <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
        What people can apply for
      </h1>
      <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
        Switch something off and it stays visible in the app, with its
        description and requirements, but nobody can apply until you switch
        it back on.
      </p>

      {error && (
        <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}

      <Group
        title="Ministries"
        summary={`${ministriesOpen} of ${parishMinistries.length} accepting applications`}
      >
        {parishMinistries.map(m => (
          <Row
            key={m.id}
            name={m.name}
            open={isOpenForApplications(managed, m.id)}
            busy={saving === m.id}
            openLabel="Accepting applications"
```

```
              closedLabel="Not accepting applications"
              onChange={next => void setOpen(m.id, next)}
            />
          ))}
        </Group>

        <Group
          title="Sacraments"
          summary={`${sacramentsOpen} of ${SACRAMENTS.length} open for applications`}
        >
          {SACRAMENTS.map(s => (
            <Row
              key={s.id}
              name={s.name}
              open={isOpenForApplications(managed, s.id)}
              busy={saving === s.id}
              openLabel="Applications open"
              closedLabel="Not available"
              onChange={next => void setOpen(s.id, next)}
            />
          ))}
        </Group>

        <p className="mt-6 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
          Applications already submitted are not affected. They stay in your
          list and you can still approve or complete them.
        </p>
      </div>
    </div>
  );
}

function Group({
  title, summary, children,
}: { title: string; summary: string; children: React.ReactNode }) {
  return (
    <section className="mt-6">
      <h2 className="text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">{title}</h2>
      <p className="mb-2 text-[14px] text-[var(--color-brand-secondary)]">{summary}</p>
      <ul className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] divide-y divide-[var(--color-brand-border)]">
        {children}
      </ul>
    </section>
  );
}

/**
 * One switch.
 *
 * A real checkbox underneath the styling, not a div with an onClick. It
 * is what gives the row a keyboard, a focus ring and a state a screen
 * reader can read, none of which would be worth rebuilding by hand.
 */
function Row({
  name, open, busy, openLabel, closedLabel, onChange,
}: {
  name: string;
  open: boolean;
  busy: boolean;
```

```
  openLabel: string;
  closedLabel: string;
  onChange: (open: boolean) => void;
}) {
  return (
    <li>
      <label className="flex items-center gap-3 p-4 cursor-pointer">
        <span className="min-w-0 flex-1">
          <span className="block text-[16px] font-bold leading-snug text-[var(--color-brand-text)]">
            {name}
          </span>
          <span className={`mt-0.5 inline-flex items-center gap-1.5 text-[14px] font-semibold ${
            open ? "text-[var(--color-brand-success)]" : "text-[var(--color-brand-secondary)]"
          }`}>
            <span
              aria-hidden
              className="w-2 h-2 rounded-full"
              style={{ background: "currentColor" }}
            />
            {open ? openLabel : closedLabel}
          </span>
        </span>

        {busy && <Loader2 className="w-4 h-4 animate-spin text-[var(--color-brand-primary)]" />}

        <input
          type="checkbox"
          className="availability-switch"
          // The wrapping label holds the ministry's name AND its current
          // state, so the computed name came out as the word "on". Said
          // explicitly instead, or a screen reader reaches a page of
          // fifteen switches that are all called the same thing.
          aria-label={`Accepting applications for ${name}`}
          checked={open}
          disabled={busy}
          onChange={e => onChange(e.target.checked)}
        />
      </label>
    </li>
  );
}
```

2.3.7 API-014 — AdminContent.tsx

Code ID	API-014
File Name	AdminContent.tsx
File Path	SanctDemo/src/admin/AdminContent.tsx
Language / Type	TypeScript (React, TSX)
Lines	318
Functions / Classes	AdminContent (exported component/function (default)); Group (internal function); Item (exported function); Box (internal function)
Related Files (its imports)	../data, ../lib/useParishContent, ../lib/parishContent, ./AdminNav, ./AdminApp
Where It Is Used	src/admin/AdminItems.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminContent.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminContent.tsx

```
import { useState } from "react";
import { ChevronDown, Loader2 } from "lucide-react";
import { SACRAMENTS, ministriesForParish } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { saveParishContent } from "../lib/parishContent";
import {
  hasParishContent, linesToList, listToLines, mergeItemContent,
  resolveMinistry, resolveSacrament, type ItemContent,
} from "../lib/itemContent";
import AdminNav from "./AdminNav";
import type { AdminSession } from "./AdminApp";

/**
 * What pilgrims read about each ministry and sacrament.
 *
 * ## Why editing here changes nothing about applications
 *
 * This is the page somebody reads before they apply. Their application
 * is a separate record, and nothing typed here touches one — a parish
 * correcting a requirement in March does not alter what somebody
 * submitted in February.
 *
 * ## Why every box starts blank
 *
 * Blank means "use what the app already shows", which for most of these
 * is text the parish supplied in the first place. Pre-filling the boxes
 * with it would invite someone to save it back as though they had
 * written it, and then an edit to the shared text would stop reaching
 * this parish. The placeholder shows what is currently displayed; the
 * box stays empty until the parish wants something different.
 */
```

```
export default function AdminContent({ session }: { session: AdminSession }) {
  const churchId = session.profile.churchId;
  const managed = useParishContent(churchId);

  // This parish's ministries, not every parish's.
  const parishMinistries = ministriesForParish(churchId);

  const [openId, setOpenId] = useState<string | null>(null);
  const [busy, setBusy] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState<string | null>(null);

  async function save(itemId: string, patch: ItemContent) {
    setBusy(itemId);
    setError(null);
    try {
      await saveParishContent(
        churchId,
        { itemContent: mergeItemContent(managed, itemId, patch) },
        session.user.email ?? "",
      );
      setSaved(itemId);
      window.setTimeout(() => setSaved(null), 2600);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(null);
    }
  }

  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)]">
      <div className="max-w-[820px] mx-auto px-4 py-6">
        <AdminNav />

        <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
          What people read
        </h1>
        <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          {session.church?.name ?? churchId} — the information shown before someone
          applies. Leave a box empty to keep what the app already shows.
        </p>

        {error && (
          <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
            {error}
          </p>
        )}

        <Group title="Sacraments">
          {SACRAMENTS.map(s => (
            <Item
              key={s.id}
              id={s.id}
              name={s.name}
              kind="sacrament"
              edited={hasParishContent(managed, s.id)}
              open={openId === s.id}
              busy={busy === s.id}
```

```
              saved={saved === s.id}
              current={resolveSacrament(managed, s.id)}
              existing={managed?.itemContent?.[s.id] ?? {}}
              onToggle={() => setOpenId(openId === s.id ? null : s.id)}
              onSave={patch => void save(s.id, patch)}
            />
          ))}
        </Group>

        <Group title="Ministries">
          {parishMinistries.map(m => (
            <Item
              key={m.id}
              id={m.id}
              name={m.name}
              kind="ministry"
              edited={hasParishContent(managed, m.id)}
              open={openId === m.id}
              busy={busy === m.id}
              saved={saved === m.id}
              current={resolveMinistry(managed, m.id)}
              existing={managed?.itemContent?.[m.id] ?? {}}
              onToggle={() => setOpenId(openId === m.id ? null : m.id)}
              onSave={patch => void save(m.id, patch)}
            />
          ))}
        </Group>
      </div>
    </div>
  );
}

function Group({ title, children }: { title: string; children: React.ReactNode }) {
  return (
    <section className="mt-7">
      <h2 className="mb-3 text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">
        {title}
      </h2>
      <ul className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] divide-y divide-[var(--color-brand-border)] overflow-
hidden">
        {children}
      </ul>
    </section>
  );
}

export function Item({
  id, name, kind, edited, open, busy, saved, current, existing, onToggle, onSave,
  availability,
}: {
  id: string;
  name: string;
  kind: "ministry" | "sacrament";
  edited: boolean;
  open: boolean;
  busy: boolean;
  saved: boolean;
  current: ReturnType<typeof resolveMinistry>;
  existing: ItemContent;
```

```
  onToggle: () => void;
  onSave: (patch: ItemContent) => void;
  /**
   * The "accepting applications" switch, when the page wants one.
   *
   * Editing what a ministry says and deciding whether anybody can join
   * it used to be two separate pages, so a secretary closing the choir
   * had to find one screen, and correcting its description another.
   * They are the same thought about the same ministry, so the switch
   * rides along with the row. Left out where a page only edits.
   */
  availability?: React.ReactNode;
}) {
  // Keyed off `open` by the parent remounting, so reopening a row always
  // shows what is saved rather than a half-finished edit from earlier.
  const [about, setAbout] = useState(existing.about ?? "");
  const [requirements, setRequirements] = useState(listToLines(existing.requirements));
  const [schedule, setSchedule] = useState(existing.schedule ?? "");
  const [responsibilities, setResponsibilities] = useState(listToLines(existing.responsibilities));
  const [process, setProcess] = useState(listToLines(existing.process));
  const [reminders, setReminders] = useState(listToLines(existing.reminders));
  const [contact, setContact] = useState(existing.contact ?? "");

  return (
    <li>
      {/* The switch sits BESIDE the expander, not inside it: one button
          cannot contain another, and a switch that also opened the
          editor would make closing a ministry a two-step accident. */}
      <div className="flex items-center">
      <button
        type="button"
        onClick={onToggle}
        aria-expanded={open}
        className="min-w-0 flex-1 flex items-center gap-3 p-4 text-left"
      >
        <span className="min-w-0 flex-1">
          <span className="block text-[16px] font-bold leading-snug text-[var(--color-brand-text)]">
            {name}
          </span>
          <span className="text-[14px] text-[var(--color-brand-secondary)]">
            {edited ? "You have written your own information" : "Using the app's information"}
          </span>
        </span>
        {saved && (
          <span role="status" className="text-[14px] font-bold text-[var(--color-brand-success)]">
            Saved
          </span>
        )}
        <ChevronDown className={`w-5 h-5 shrink-0 text-[var(--color-brand-secondary)] transition-transform ${open ? "rotate-180" : ""}`} />
      </button>

      {availability && <div className="shrink-0 pr-4">{availability}</div>}
      </div>

      {open && (
        <div className="px-4 pb-5 space-y-4">
          <Box
            label="About"
            value={about}
```

```
  onChange={setAbout}
  rows={4}
  placeholder={current?.about}
  help="Leave empty to keep what is shown now."
/>

<Box
  label="Requirements"
  value={requirements}
  onChange={setRequirements}
  rows={5}
  placeholder={current?.requirements.join("\n")}
  help="One per line. Leave empty to keep the list shown now."
/>

<Box
  label="Schedule"
  value={schedule}
  onChange={setSchedule}
  rows={2}
  placeholder={current?.schedule || "Saturdays, 9:00 AM"}
  help="In your own words."
/>

{kind === "ministry" && (
  <Box
    label="What members do"
    value={responsibilities}
    onChange={setResponsibilities}
    rows={4}
    help="One per line. Shown as a list."
  />
)}

<Box
  label="How applying works"
  value={process}
  onChange={setProcess}
  rows={5}
  help="One step per line. Shown numbered, in this order."
/>

<Box
  label="Important reminders"
  value={reminders}
  onChange={setReminders}
  rows={4}
  help="One per line."
/>

<Box
  label="Who to contact"
  value={contact}
  onChange={setContact}
  rows={2}
  help="A number, an email, or the office hours."
/>

<button
```

```
            type="button"
            disabled={busy}
            onClick={() =>
              onSave({
                about,
                requirements: linesToList(requirements),
                schedule,
                responsibilities: linesToList(responsibilities),
                process: linesToList(process),
                reminders: linesToList(reminders),
                contact,
              })
            }
            className="inline-flex items-center gap-2 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-5 py-3 text-[15px] font-
bold disabled:opacity-60"
          >
            {busy && <Loader2 className="w-4 h-4 animate-spin" />}
            Save
          </button>
        </div>
      )}
    </li>
  );
}

function Box({
  label, value, onChange, rows, help, placeholder,
}: {
  label: string;
  value: string;
  onChange: (value: string) => void;
  rows: number;
  help?: string;
  placeholder?: string;
}) {
  const id = `c-${label.replace(/\s+/g, "-").toLowerCase()}`;
  return (
    <div>
      <label htmlFor={id} className="block text-[14px] font-bold text-[var(--color-brand-secondary)]">
        {label}
      </label>
      {help && <p className="mb-1.5 text-[13px] text-[var(--color-brand-secondary)]">{help}</p>}
      <textarea
        id={id}
        rows={rows}
        value={value}
        onChange={e => onChange(e.target.value)}
        placeholder={placeholder}
        className="w-full rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-bg)] px-3 py-2.5 text-[16px] text-[var(--color-brand-text)]"
      />
    </div>
  );
}
```

2.3.8 API-015 — AdminDashboard.tsx

Code ID	API-015
File Name	AdminDashboard.tsx
File Path	SanctDemo/src/admin/AdminDashboard.tsx
Language / Type	TypeScript (React, TSX)
Lines	261
Functions / Classes	AdminDashboard (exported component/function (default)); Tile (internal function)
Related Files (its imports)	../lib/firebase, ./StatusBadge, ../lib/authFlow, ./AdminApp, ../lib/applicationFilters, ./ParishStatus, ./AdminTiles, ../types
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminDashboard.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminDashboard.tsx

```
import { useEffect, useMemo, useState } from "react";
import { collection, onSnapshot, query, where } from "firebase/firestore";
import { Loader2, Inbox } from "lucide-react";
import { useNavigate } from "react-router-dom";
import { db } from "../lib/firebase";
import StatusBadge from "./StatusBadge";
import { signOutNow } from "../lib/authFlow";
import type { AdminSession } from "./AdminApp";
import { isArchived } from "../lib/applicationFilters";
import ParishStatus from "./ParishStatus";
import AdminTiles from "./AdminTiles";
import type { ApplicationDoc, ApplicationStatus } from "../types";

/**
 * The parish office dashboard.
 *
 * The query filters on churchId, and so do the security rules. Both, on
 * purpose: the filter is what makes the screen correct, the rule is what
 * makes it safe. Without the rule a changed filter would leak another
 * parish; without the filter the rule would simply deny the read and the
 * admin would see an error instead of their own work.
 */
const STATUS_LABEL: Record<ApplicationStatus, string> = {
  pending: "Pending",
  under_review: "Under review",
  approved: "Approved",
  rejected: "Rejected",
  completed: "Completed",
};

export default function AdminDashboard({ session }: { session: AdminSession }) {
```

```
const navigate = useNavigate();
const [apps, setApps] = useState<ApplicationDoc[] | null>(null);
const [error, setError] = useState<string | null>(null);
const churchId = session.profile.churchId;

useEffect(() => {
  const q = query(collection(db, "applications"), where("churchId", "==", churchId));
  return onSnapshot(
    q,
    snap => setApps(snap.docs.map(d => ({ id: d.id, ...(d.data() as ApplicationDoc) }))),
    err => setError(err.message),
  );
}, [churchId]);

const counts = useMemo(() => {
  const base: Record<ApplicationStatus, number> = {
    pending: 0, under_review: 0, approved: 0, rejected: 0, completed: 0,
  };
  for (const a of apps ?? []) base[a.status] = (base[a.status] ?? 0) + 1;
  return base;
}, [apps]);

/**
 * The totals a parish office actually asks about.
 *
 * Counted over OPEN applications only. These tiles answer "how much
 * work is in", and a completed baptism is not work that is in - it was
 * inflating every figure on this screen, so a parish that had dealt
 * with everything still saw the same four it started the month with
 * and had no way to tell the difference.
 *
 * "This month" is the exception and stays over everything: it is a
 * record of what arrived, not of what is outstanding, and a month's
 * intake that shrank as the office worked through it would be useless.
 */
const totals = useMemo(() => {
  const now = new Date();
  let open = 0, ministry = 0, sacrament = 0, thisMonth = 0, archived = 0;
  for (const a of apps ?? []) {
    const d = a.createdAt ? new Date(a.createdAt) : null;
    if (d && !Number.isNaN(d.getTime())
        && d.getFullYear() === now.getFullYear() && d.getMonth() === now.getMonth()) {
      thisMonth++;
    }

    if (isArchived(a.status)) { archived++; continue; }

    open++;
    if (a.kind === "ministry") ministry++;
    else if (a.kind === "sacrament") sacrament++;
  }
  return { open, ministry, sacrament, thisMonth, archived };
}, [apps]);

const openApps = useMemo(
  () => (apps ?? []).filter(a => !isArchived(a.status)),
  [apps],
);
```

```
return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <header className="bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-5 py-5">
      <p className="text-[13px] font-bold uppercase tracking-wider opacity-80">Parish office</p>
      <h1 className="text-[24px] font-bold font-serif italic">
        {session.church?.name ?? churchId}
      </h1>
      <p className="mt-0.5 text-[14px] opacity-90">{session.profile.fullName || session.user.email}</p>
      {/* Just Sign out here now. Announcements and the rest moved into
          the labelled nav below, where they read as sections of the
          office rather than as two buttons beside the way out. */}
      <button
        onClick={() => void signOutNow()}
        className="mt-3 rounded-full border border-white/40 px-4 py-1.5 text-[14px] font-semibold"
      >
        Sign out
      </button>
    </header>

    <main className="px-4 py-5 max-w-[900px] mx-auto">
      {/* The whole office at a glance, for somebody arriving. The nav
          below is for moving around once you know the place. */}
      <AdminTiles parishName={session.church?.name ?? "Parish"} />

      {/* No pill row here.

          Every pill on it is a tile directly above it, so on this one
          page it says everything twice. It stays on every OTHER page,
          where there are no tiles and it is the only way to move
          between sections. */}

      {error && (
        <p role="alert" className="mb-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}

      {/* Volume first - what came in and of what kind - then the
          status breakdown below. A parish asks "how many" before it
          asks "how many pending". */}
      <div className="grid grid-cols-2 sm:grid-cols-4 gap-3 mb-3">
        <Tile label="Still open" value={apps === null ? "—" : totals.open}
              onClick={() => navigate("/parish-office-gav8thhe/applications")} />
        <Tile label="Ministries" value={apps === null ? "—" : totals.ministry}
              onClick={() => navigate("/parish-office-gav8thhe/applications/ministry")} />
        <Tile label="Sacraments" value={apps === null ? "—" : totals.sacrament}
              onClick={() => navigate("/parish-office-gav8thhe/applications/sacrament")} />
        <Tile label="This month" value={apps === null ? "—" : totals.thisMonth}
              onClick={() => navigate("/parish-office-gav8thhe/applications")} />
      </div>

      <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
        {(["pending", "under_review", "approved"] as ApplicationStatus[]).map(s => (
          <div key={s} className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4">
            <p className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
              {STATUS_LABEL[s]}
            </p>
            <p className="mt-1 text-[28px] font-bold text-[var(--color-brand-text)]">
              {apps === null ? "—" : counts[s]}
```

```
      </p>
    </div>
  ))}

  {/* Completed and rejected together, and a way in. They are off
      every other figure on this screen, so there has to be
      somewhere they visibly went - a number that simply dropped
      reads as data lost. */}
  <Tile label="Archived" value={apps === null ? "—" : totals.archived}
        onClick={() => navigate("/parish-office-gav8thhe/applications/archive")} />
</div>

{/* What the parish currently looks like to a pilgrim, as opposed
    to how much work is in the tray. An admin who has closed the
    choir and suspended the 8am Mass should be able to see both
    without opening two pages. */}
<ParishStatus churchId={churchId} />

<div className="mt-7 mb-3 flex items-baseline justify-between gap-3">
  {/* "Needing attention", not "Recent". The list under it is now
      the open ones, and a finished application is still recent. */}
  <h2 className="text-[17px] font-bold font-serif italic">Needing attention</h2>
  <button
    onClick={() => navigate("/parish-office-gav8thhe/applications")}
    className="text-[15px] font-bold text-[var(--color-brand-primary)]"
  >
    Open the full table
  </button>
</div>

{apps === null && (
  <div className="py-10 text-center">
    <Loader2 className="mx-auto w-6 h-6 animate-spin text-[var(--color-brand-primary)]" />
  </div>
)}

{apps !== null && openApps.length === 0 && (
  <div className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] py-10 text-center">
    <Inbox className="mx-auto w-7 h-7 text-[var(--color-brand-secondary)]" />
    <p className="mt-2 text-[15px] font-bold">Nothing waiting</p>
    <p className="mt-1 text-[14px] text-[var(--color-brand-secondary)]">
      {apps.length === 0
        ? "Applications from your parishioners appear here as they are submitted."
        : `Everything has been dealt with. ${totals.archived} ${totals.archived === 1 ? "application is" : "applications are"} in the archive.`}
    </p>
  </div>
)}

{apps !== null && openApps.length > 0 && (
  <div className="overflow-x-auto rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)]">
    <table className="w-full text-left text-[14px]">
      <thead className="text-[13px] uppercase tracking-wider text-[var(--color-brand-secondary)]">
        <tr>
          <th className="px-4 py-3 font-bold">Reference</th>
          <th className="px-4 py-3 font-bold">Applicant</th>
          <th className="px-4 py-3 font-bold">Type</th>
          <th className="px-4 py-3 font-bold">Status</th>
        </tr>
      </thead>
```

```
              <tbody className="divide-y divide-[var(--color-brand-border)]">
                {openApps.map(a => (
                  <tr
                    key={a.id}
                    tabIndex={0}
                    role="link"
                    aria-label={`Open ${a.type ?? a.kind} from ${a.applicantName ?? "applicant"}`}
                    onClick={() => navigate(`/admin/applications/view/${a.id}`)}
                    onKeyDown={e => {
                      // The row is the target, so it has to answer the
                      // keyboard the way a link would.
                      if (e.key === "Enter" || e.key === " ") {
                        e.preventDefault();
                        navigate(`/admin/applications/view/${a.id}`);
                      }
                    }}
                    className="cursor-pointer hover:bg-[var(--color-brand-card-sunk)] focus:bg-[var(--color-brand-card-sunk)] outline-none"
                  >
                    <td className="px-4 py-3 font-mono text-[13px]">{a.referenceNumber ?? a.id}</td>
                    <td className="px-4 py-3">{a.applicantName ?? "—"}</td>
                    <td className="px-4 py-3">{a.type ?? a.kind}</td>
                    <td className="px-4 py-3"><StatusBadge status={a.status} /></td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </main>
    </div>
  );
}

/**
 * One headline number, and a way into the list behind it.
 *
 * Clickable because a count a parish cannot act on is trivia: "12
 * pending" should be one tap from the twelve.
 */
function Tile({ label, value, onClick }: { label: string; value: number | string; onClick: () => void }) {
  return (
    <button
      type="button"
      onClick={onClick}
      className="text-left rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4 hover:bg-[var(--color-brand-card-sunk)]"
    >
      <p className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        {label}
      </p>
      <p className="mt-1 text-[28px] font-bold text-[var(--color-brand-text)]">{value}</p>
    </button>
  );
}
```

2.3.9 API-016 — AdminItems.tsx

Code ID	API-016
File Name	AdminItems.tsx
File Path	SanctDemo/src/admin/AdminItems.tsx
Language / Type	TypeScript (React, TSX)
Lines	201
Functions / Classes	AdminItems (exported component/function (default)); AvailabilitySwitch (internal function)
Related Files (its imports)	../data, ../lib/useParishContent, ../lib/parishContent, ../lib/availability, ./AdminNav, ./AdminContent, ./AdminApp
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminItems.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminItems.tsx

```
import { useState } from "react";
import { Loader2 } from "lucide-react";

import { SACRAMENTS, ministriesForParish } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { saveParishContent } from "../lib/parishContent";
import { closedIds, isOpenForApplications, withAvailability } from "../lib/availability";
import {
  hasParishContent, mergeItemContent, resolveMinistry, resolveSacrament,
  type ItemContent,
} from "../lib/itemContent";
import AdminNav from "./AdminNav";
import { Item } from "./AdminContent";
import type { AdminSession } from "./AdminApp";

/**
 * One ministry or sacrament, start to finish, on one page.
 *
 * ## Why this replaced two pages
 *
 * "What people read" edited the descriptions; "Open & Closed" switched
 * applications on and off. Both listed every ministry and every
 * sacrament, in the same order, from the same record - so a secretary
 * closing the choir and then correcting its requirements had to find
 * two different screens to do one afternoon's work, and the dashboard
 * carried two tiles for what the parish thinks of as one thing.
 *
 * They are now a page per kind: Ministries, and Sacraments. Each row
 * carries its own switch and opens its own editor.
 *
 * ## Why Mass is not here
```

```
 *
 * A Mass is not applied for. Suspending the 6:00 AM is a change to a
 * schedule, which is what the Mass Schedule page is, and the bell that
 * tells people about it is already wired to the schedule's own record.
 * Putting it on a page about applications would be filing it by the
 * screen it resembles rather than by what it is.
 */
export default function AdminItems({
  session,
  kind,
}: {
  session: AdminSession;
  kind: "ministry" | "sacrament";
}) {
  const churchId = session.profile.churchId;
  const managed = useParishContent(churchId);

  // This parish's ministries, not every parish's. Sacraments are the
  // same seven everywhere, which is why they are not filtered.
  const items =
    kind === "ministry"
      ? ministriesForParish(churchId).map(m => ({ id: m.id, name: m.name }))
      : SACRAMENTS.map(s => ({ id: s.id, name: s.name }));

  const [openId, setOpenId] = useState<string | null>(null);
  const [busy, setBusy] = useState<string | null>(null);
  const [switching, setSwitching] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState<string | null>(null);

  async function saveContent(itemId: string, patch: ItemContent) {
    setBusy(itemId);
    setError(null);
    try {
      await saveParishContent(
        churchId,
        { itemContent: mergeItemContent(managed, itemId, patch) },
        session.user.email ?? "",
      );
      setSaved(itemId);
      window.setTimeout(() => setSaved(null), 2600);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(null);
    }
  }

  async function setOpen(itemId: string, open: boolean) {
    setSwitching(itemId);
    setError(null);
    try {
      await saveParishContent(
        churchId,
        { closedApplications: withAvailability(closedIds(managed), itemId, open) },
        session.user.email ?? "",
      );
    } catch {
      setError("That could not be saved. Check your connection and try again.");
```

```
    } finally {
      setSwitching(null);
    }
  }

  const openCount = items.filter(i => isOpenForApplications(managed, i.id)).length;
  const title = kind === "ministry" ? "Ministries" : "Sacraments";

  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)]">
      <div className="max-w-[760px] mx-auto px-4 py-6">
        <AdminNav />

        <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
          {title}
        </h1>
        <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          Open a row to change what people read about it. Use the switch to
          decide whether they can apply. Switched off, it stays visible in the
          app with everything you have written, and the Apply button goes.
        </p>

        <p className="mt-3 text-[14px] font-semibold text-[var(--color-brand-secondary)]">
          {openCount} of {items.length} accepting applications
        </p>

        {error && (
          <p role="alert" className="mt-3 text-[14px] font-semibold text-[var(--color-brand-danger)]">
            {error}
          </p>
        )}

        <ul className="mt-4 divide-y divide-[var(--color-brand-border)] rounded-2xl border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] overflow-
hidden">
          {items.map(item => {
            const open = isOpenForApplications(managed, item.id);
            return (
              <Item
                key={`${item.id}-${openId === item.id}`}
                id={item.id}
                name={item.name}
                kind={kind}
                edited={hasParishContent(managed, item.id)}
                open={openId === item.id}
                busy={busy === item.id}
                saved={saved === item.id}
                current={
                  kind === "ministry"
                    ? resolveMinistry(managed, item.id, "en")
                    : resolveSacrament(managed, item.id, "en")
                }
                existing={managed?.itemContent?.[item.id] ?? {}}
                onToggle={() => setOpenId(openId === item.id ? null : item.id)}
                onSave={patch => void saveContent(item.id, patch)}
                availability={
                  <AvailabilitySwitch
                    name={item.name}
                    open={open}
                    busy={switching === item.id}
```

```
                  onChange={next => void setOpen(item.id, next)}
                />
              }
            />
          );
        })}
      </ul>
    </div>
  </div>
  );
}

/**
 * One switch, saved the moment it is pressed.
 *
 * A real checkbox under the styling, as on the page this replaced: it
 * is what gives the row a keyboard, a focus ring and a state a screen
 * reader can read, none of which is worth rebuilding by hand.
 *
 * No Save button either. One switch is one decision, and a page of
 * switches with a single Save at the bottom invites someone to flip
 * four things and lose three of them.
 */
function AvailabilitySwitch({
  name,
  open,
  busy,
  onChange,
}: {
  name: string;
  open: boolean;
  busy: boolean;
  onChange: (next: boolean) => void;
}) {
  return (
    <span className="flex items-center gap-2">
      {busy && (
        <Loader2 className="w-4 h-4 animate-spin text-[var(--color-brand-primary)]" aria-hidden />
      )}
      <input
        type="checkbox"
        className="availability-switch"
        // Said explicitly: the row around it already carries the name
        // and the state, so the computed label came out as "on" for
        // every switch on the page.
        aria-label={`Accepting applications for ${name}`}
        checked={open}
        disabled={busy}
        onChange={e => onChange(e.target.checked)}
      />
    </span>
  );
}
```

2.3.10 API-017 — AdminLogin.tsx

Code ID	API-017
File Name	AdminLogin.tsx
File Path	SanctDemo/src/admin/AdminLogin.tsx
Language / Type	TypeScript (React, TSX)
Lines	114
Functions / Classes	AdminLogin (exported component/function (default)); Card (internal function)
Related Files (its imports)	../lib/authFlow
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminLogin.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminLogin.tsx

```
import { useState, type FormEvent, type ReactNode } from "react";
import { Loader2, Mail, ShieldCheck } from "lucide-react";
import { signIn, sendSecondFactorLink, AuthError } from "../lib/authFlow";

/**
 * Parish office sign-in.
 *
 * Administrators always take the second factor, whatever their email
 * verification state: they can change other people's records, so their
 * password on its own is worth more than a pilgrim's. signIn() decides
 * that, not this screen - a login form that chose its own security level
 * would be a login form an attacker could ask nicely.
 */
export default function AdminLogin() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [sent, setSent] = useState(false);
  const [notice, setNotice] = useState<string | null>(null);

  async function submit(e: FormEvent) {
    e.preventDefault();
    setBusy(true);
    setError(null);
    try {
      const result = await signIn(email.trim(), password);
      if (result.pendingSecondFactor) {
        await sendSecondFactorLink(email.trim());
        setSent(true);
      }
    } catch (err) {
      setError(err instanceof AuthError ? err.message : "Could not sign in.");
```

```
    } finally {
      setBusy(false);
    }
  }

  if (sent) {
    return (
      <Card>
        <ShieldCheck className="mx-auto w-8 h-8 text-[var(--color-brand-primary)]" />
        <h1 className="mt-3 text-[19px] font-bold font-serif italic">One more step</h1>
        <p className="mt-2 text-[15px] leading-relaxed text-[var(--color-brand-text)]">
          We sent a one-time sign-in link to <strong>{email}</strong>. Open it on this
          device to finish.
        </p>
        <p className="mt-2 text-[14px] text-[var(--color-brand-secondary)]">
          Your password alone does not open the parish office. The link expires and
          can only be used once.
        </p>
      </Card>
    );
  }

  return (
    <Card>
      <h1 className="text-[22px] font-bold font-serif italic text-[var(--color-brand-text)]">Parish office</h1>
      <p className="mt-1 mb-5 text-[15px] text-[var(--color-brand-secondary)]">
        Sign in to review your parish's applications.
      </p>

      {error && (
        <p role="alert" className="mb-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-left text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}
      {notice && (
        <p className="mb-4 text-[14px] text-[var(--color-brand-secondary)]">{notice}</p>
      )}

      <form onSubmit={submit} className="text-left space-y-3.5">
        <div>
          <label htmlFor="ae" className="block text-[14px] font-bold mb-1.5">Email address</label>
          <input id="ae" type="email" required autoComplete="email" value={email}
                 onChange={e => setEmail(e.target.value)}
                 className="w-full rounded-2xl border border-[var(--color-brand-border)] px-4 py-3 text-[15px]" />
        </div>
        <div>
          <label htmlFor="ap" className="block text-[14px] font-bold mb-1.5">Password</label>
          <input id="ap" type="password" required autoComplete="current-password" value={password}
                 onChange={e => setPassword(e.target.value)}
                 className="w-full rounded-2xl border border-[var(--color-brand-border)] px-4 py-3 text-[15px]" />
        </div>

        <button type="submit" disabled={busy}
                className="w-full rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] py-3 font-bold text-[15px] inline-flex items-
center justify-center gap-2 disabled:opacity-50">
          {busy ? <><Loader2 className="w-4 h-4 animate-spin" /> Signing in…</> : <><Mail className="w-4 h-4" /> Continue</>}
        </button>

        {/* Not a self-service reset.
```

```
            A parish office account is handed out by whoever runs
            SanctiWalk for the diocese, and getting back into one is
            their decision rather than something an email link should
            settle. Saying so is also more honest than a button that
            sends a message to an inbox the secretary may not have. */}
          <p className="w-full pt-2 text-[13px] leading-snug text-[var(--color-brand-muted)]">
            Forgot your password? Contact the SanctiWalk administrator to have it
            reset.
          </p>
        </form>
      </Card>
    );
}

function Card({ children }: { children: ReactNode }) {
  return (
    <div className="min-h-screen bg-[var(--color-brand-bg)] flex items-center justify-center px-5">
      <div className="w-full max-w-[420px] rounded-[22px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-6 text-center">
        {children}
      </div>
    </div>
  );
}
```

2.3.11 API-018 — AdminMassSchedule.tsx

Code ID	API-018
File Name	AdminMassSchedule.tsx
File Path	SanctDemo/src/admin/AdminMassSchedule.tsx
Language / Type	TypeScript (React, TSX)
Lines	348
Functions / Classes	AdminMassSchedule (exported component/function (default)); DayRow (internal function)
Related Files (its imports)	../lib/useParishContent, ../lib/parishContent, ../data, ../lib/notifications, ./AdminNav, ./AdminApp
Where It Is Used	src/admin/AdminApp.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminMassSchedule.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminMassSchedule.tsx

```
import { useState } from "react";
import { Loader2, Plus, Trash2, X } from "lucide-react";
import { useParishContent } from "../lib/useParishContent";
import { saveParishContent } from "../lib/parishContent";
import {
  isTimeAvailable, parseTimes, withTimeAvailability, type MassScheduleEntry,
} from "../lib/schedule";
import { MASS_SCHEDULES } from "../data";
import { notifyParish, MAX_PARISH_NOTIFICATIONS } from "../lib/notifications";
import AdminNav from "./AdminNav";
import type { AdminSession } from "./AdminApp";

const DAYS = [
  "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday",
];

/**
 * The parish's Mass times.
 *
 * ## This was already in Firebase
 *
 * `parishContent.massSchedule` has always been the source the app reads —
 * nothing was ever hard-coded into the phone. What was missing was a way
 * to edit it from the parish office: the only editor lived inside the old
 * in-app admin, behind a tab most parishes never found.
 *
 * ## Why suspending is not deleting
 *
 * Removing the 8:00 AM Mass takes it off the schedule as though it had
 * never existed, and someone who comes every week at 8 finds no mention
 * of it and assumes they misremembered. Suspended, the app still lists it
```

```
 * and says "Not available" — which is the information they actually need.
 */
export default function AdminMassSchedule({ session }: { session: AdminSession }) {
  const churchId = session.profile.churchId;
  const managed = useParishContent(churchId);

  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [saved, setSaved] = useState(false);

  // Telling the parish is a separate, deliberate act - not something
  // every edit does. A secretary fixing a typo in a time must not send
  // four hundred phones a notification.
  const [telling, setTelling] = useState(false);
  const [message, setMessage] = useState("");
  const [told, setTold] = useState<string | null>(null);

  // The compiled schedule is the starting point for a parish that has
  // never edited one. Without it the office opens an empty page and has
  // to retype times the app is already showing.
  const compiled = MASS_SCHEDULES[churchId]?.schedule ?? [];
  const schedule: MassScheduleEntry[] = managed?.massSchedule ?? compiled;

  async function save(next: MassScheduleEntry[]) {
    setBusy(true);
    setError(null);
    try {
      await saveParishContent(churchId, { massSchedule: next }, session.user.email ?? "");
      setSaved(true);
      window.setTimeout(() => setSaved(false), 2400);
    } catch {
      setError("That could not be saved. Check your connection and try again.");
    } finally {
      setBusy(false);
    }
  }

  async function tellParish() {
    setTelling(true);
    setTold(null);
    setError(null);
    try {
      const result = await notifyParish(churchId, "Mass schedule updated", message.trim());
      setTold(result.tooManyFor
        ? `Nobody was notified: this parish has ${result.tooManyFor} members, more than the ${MAX_PARISH_NOTIFICATIONS} this can message at once. Post an
announcement instead.`
        : `Sent to ${result.sent} ${result.sent === 1 ? "person" : "people"}.`);
      if (!result.tooManyFor) setMessage("");
    } catch {
      setError("That could not be sent. Check your connection and try again.");
    } finally {
      setTelling(false);
    }
  }

  function replaceDay(day: string, entry: MassScheduleEntry | null) {
    const without = schedule.filter(e => e.day !== day);
    const next = entry ? [...without, entry] : without;
    // Kept in week order whatever order they were added in, so the saved
```

```
  // document reads the way the app displays it.
  next.sort((a, b) => DAYS.indexOf(a.day) - DAYS.indexOf(b.day));
  void save(next);
}

return (
  <div className="min-h-screen bg-[var(--color-brand-bg)]">
    <div className="max-w-[760px] mx-auto px-4 py-6">
      <AdminNav />

      <h1 className="text-[24px] font-bold font-serif italic text-[var(--color-brand-text)]">
        Mass Schedule
      </h1>
      <p className="text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
        {session.church?.name ?? churchId} — this is what the app shows on Home.
        Changes appear on every phone straight away.
      </p>

      {error && (
        <p role="alert" className="mt-4 rounded-2xl bg-[#FBE9E4] border border-[#E7C3B8] px-4 py-3 text-[15px] text-[var(--color-brand-error)]">
          {error}
        </p>
      )}
      {saved && (
        <p role="status" className="mt-4 text-[15px] font-semibold text-[var(--color-brand-success)]">
          Saved.
        </p>
      )}

      <ul className="mt-5 space-y-3">
        {DAYS.map(day => (
          <DayRow
            key={day}
            day={day}
            entry={schedule.find(e => e.day === day) ?? null}
            busy={busy}
            onChange={entry => replaceDay(day, entry)}
          />
        ))}
      </ul>

      <p className="mt-6 text-[14px] leading-relaxed text-[var(--color-brand-secondary)]">
        Write times as <strong>6:00 AM</strong> or <strong>4:30 PM</strong>. A day with
        no times is simply not shown in the app. A Mass switched off stays on the
        list marked <strong>Not available</strong>, so nobody turns up for one that
        has been called off.
      </p>

      <section className="mt-6 rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-5">
        <h2 className="text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
          Tell the parish
        </h2>
        <p className="mt-1.5 text-[15px] leading-relaxed text-[var(--color-brand-secondary)]">
          Your changes are already live on everyone&apos;s phone. Send a notification
          as well only when it is something people need to know before Sunday.
        </p>

        {told && (
          <p role="status" className="mt-3 text-[15px] font-semibold text-[var(--color-brand-success)]">
```

```
              {told}
            </p>
          )}

          <div className="mt-3 flex flex-wrap gap-2.5 items-start">
            <input
              value={message}
              maxLength={160}
              aria-label="What to tell the parish"
              placeholder="The Sunday 8:00 AM Mass is suspended this month."
              onChange={e => setMessage(e.target.value)}
              className="flex-1 min-w-[240px] rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-bg)] px-3 py-2.5 text-[16px]"
            />
            <button
              type="button"
              disabled={telling || !message.trim()}
              onClick={() => void tellParish()}
              className="inline-flex items-center gap-2 rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-5 py-3 text-[15px] font-
bold disabled:opacity-50"
            >
              {telling && <Loader2 className="w-4 h-4 animate-spin" />}
              Send to parish users
            </button>
          </div>
        </section>
      </div>
    </div>
  );
}

function DayRow({
  day, entry, busy, onChange,
}: {
  day: string;
  entry: MassScheduleEntry | null;
  busy: boolean;
  onChange: (entry: MassScheduleEntry | null) => void;
}) {
  const [adding, setAdding] = useState(false);
  const [draft, setDraft] = useState("");
  const [note, setNote] = useState(entry?.note ?? "");
  const [editingNote, setEditingNote] = useState(false);

  const times = entry ? parseTimes(entry.time) : [];

  function addTime() {
    const value = draft.trim();
    if (!value) return;
    const next = [...times, value];
    onChange({ ...(entry ?? { day, time: "" }), day, time: next.join(", ") });
    setDraft("");
    setAdding(false);
  }

  function removeTime(time: string) {
    const next = times.filter(t => t !== time);
    if (next.length === 0) { onChange(null); return; }
    const base = entry ?? { day, time: "" };
    onChange({
```

```
    ...base,
    day,
    time: next.join(", "),
    // A suspension for a time that no longer exists would sit in the
    // document for ever.
    unavailableTimes: (base.unavailableTimes ?? []).filter(t => t !== time),
  });
}

function toggleTime(time: string, available: boolean) {
  if (!entry) return;
  onChange(withTimeAvailability(entry, time, available));
}

return (
  <li className="rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4">
    <div className="flex items-baseline justify-between gap-3">
      <h2 className="text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">{day}</h2>
      {times.length === 0 && (
        <span className="text-[14px] text-[var(--color-brand-secondary)]">No Mass</span>
      )}
    </div>

    {times.length > 0 && (
      <ul className="mt-2.5 space-y-2">
        {times.map(time => {
          const on = entry ? isTimeAvailable(entry, time) : true;
          return (
            <li key={time} className="flex items-center gap-3">
              <span className={`text-[16px] font-bold tabular-nums ${
                on ? "text-[var(--color-brand-text)]"
                   : "text-[var(--color-brand-secondary)] line-through"
              }`}>
                {time}
              </span>

              <span className={`text-[13px] font-bold ${
                on ? "text-[var(--color-brand-success)]" : "text-[var(--color-brand-secondary)]"
              }`}>
                {on ? "Available" : "Not available"}
              </span>

              <span className="ml-auto flex items-center gap-3">
                <input
                  type="checkbox"
                  className="availability-switch"
                  aria-label={`${day} ${time} Mass is going ahead`}
                  checked={on}
                  disabled={busy}
                  onChange={e => toggleTime(time, e.target.checked)}
                />
                <button
                  type="button"
                  onClick={() => removeTime(time)}
                  disabled={busy}
                  aria-label={`Remove the ${day} ${time} Mass`}
                  className="text-[var(--color-brand-error)]"
                >
                  <Trash2 className="w-4 h-4" />
```

```
            </button>
          </span>
        </li>
      );
    })}
  </ul>
)}

{entry?.note && !editingNote && (
  <p className="mt-2.5 rounded-xl bg-[var(--color-brand-card-sunk)] px-3 py-2 text-[14px] text-[var(--color-brand-text)]">
    {entry.note}
  </p>
)}

<div className="mt-3 flex flex-wrap items-center gap-3">
  {adding ? (
    <span className="flex items-center gap-2">
      <input
        value={draft}
        autoFocus
        placeholder="8:00 AM"
        aria-label={`Time of a new Mass on ${day}`}
        onChange={e => setDraft(e.target.value)}
        onKeyDown={e => {
          if (e.key === "Enter") addTime();
          if (e.key === "Escape") { setAdding(false); setDraft(""); }
        }}
        className="w-[120px] rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-bg)] px-3 py-2 text-[16px]"
      />
      <button type="button" onClick={addTime} disabled={busy}
              className="rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-4 py-2 text-[14px] font-bold">
        Add
      </button>
      <button type="button" onClick={() => { setAdding(false); setDraft(""); }}
              aria-label="Cancel adding a Mass"
              className="text-[var(--color-brand-secondary)]">
        <X className="w-4 h-4" />
      </button>
    </span>
  ) : (
    <button type="button" onClick={() => setAdding(true)} disabled={busy}
            className="inline-flex items-center gap-1.5 text-[14px] font-bold text-[var(--color-brand-primary)]">
      <Plus className="w-4 h-4" /> Add a Mass
    </button>
  )}

  {editingNote ? (
    <span className="flex items-center gap-2 flex-1 min-w-[220px]">
      <input
        value={note}
        autoFocus
        maxLength={120}
        placeholder="No 6pm Mass during the renovation"
        aria-label={`Note for ${day}`}
        onChange={e => setNote(e.target.value)}
        className="flex-1 rounded-xl border border-[var(--color-brand-border)] bg-[var(--color-brand-bg)] px-3 py-2 text-[16px]"
      />
      <button
        type="button"
```

```
              disabled={busy || times.length === 0}
              onClick={() => {
                if (!entry) return;
                const { note: _old, ...rest } = entry;
                onChange(note.trim() ? { ...rest, note: note.trim() } : rest);
                setEditingNote(false);
              }}
              className="rounded-full bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-4 py-2 text-[14px] font-bold"
            >
              Save
            </button>
          </span>
        ) : times.length > 0 && (
          <button type="button" onClick={() => setEditingNote(true)}
                  className="text-[14px] font-bold text-[var(--color-brand-secondary)]">
            {entry?.note ? "Edit note" : "Add a note"}
          </button>
        )}

        {busy && <Loader2 className="w-4 h-4 animate-spin text-[var(--color-brand-primary)]" />}
      </div>
    </li>
  );
}
```

2.3.12 API-019 — AdminNav.tsx

Code ID	API-019
File Name	AdminNav.tsx
File Path	SanctDemo/src/admin/AdminNav.tsx
Language / Type	TypeScript (React, TSX)
Lines	112
Functions / Classes	NavItem (exported type); AdminNav (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/admin/AdminAnalytics.tsx, src/admin/AdminAnnouncements.tsx, src/admin/AdminApplications.tsx, src/admin/AdminAvailability.tsx, src/admin/AdminContent.tsx, src/admin/AdminItems.tsx, src/admin/AdminMassSchedule.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminNav.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminNav.tsx

```
import { useLocation, useNavigate } from "react-router-dom";
import {
  ClipboardList, LayoutDashboard, Megaphone, ToggleLeft, Archive,
  Church, PartyPopper, CalendarDays, BookOpen, BarChart3,
} from "lucide-react";

/**
 * The parish office's own navigation.
 *
 * ## Why every item is a word and an icon, never an icon alone
 *
 * The person using this is a parish secretary, not a developer. An icon
 * is a guess until you have clicked it once; a word is not. The previous
 * version had Announcements and Archive as two unlabelled-looking
 * buttons tucked beside Sign out in the header, and the client could not
 * tell what either did.
 *
 * ## Why only the sections that exist are here
 *
 * Mass Schedule, Feast Days and Notifications are coming, and a nav item
 * that opens nothing is worse than a missing one - it teaches people the
 * admin is broken. They go in when the pages behind them do.
 */

export interface NavItem {
  to: string;
  label: string;
  icon: React.ReactNode;
  /** Matched as a prefix, so a detail page keeps its section highlighted. */
```

```
  match?: string;
}

const ITEMS: NavItem[] = [
  { to: "/parish-office-gav8thhe", label: "Dashboard", icon: <LayoutDashboard className="w-4 h-4" /> },
  {
    to: "/parish-office-gav8thhe/applications",
    label: "Applications",
    icon: <ClipboardList className="w-4 h-4" />,
    match: "/parish-office-gav8thhe/applications",
  },
  {
    to: "/parish-office-gav8thhe/ministries",
    label: "Ministries",
    icon: <BookOpen className="w-4 h-4" />,
  },
  {
    to: "/parish-office-gav8thhe/sacraments",
    label: "Sacraments",
    icon: <ToggleLeft className="w-4 h-4" />,
  },
  {
    to: "/parish-office-gav8thhe/announcements",
    label: "Announcements",
    icon: <Megaphone className="w-4 h-4" />,
  },
  {
    to: "/parish-office-gav8thhe/mass-schedule",
    label: "Mass Schedule",
    icon: <Church className="w-4 h-4" />,
  },
  {
    to: "/parish-office-gav8thhe/analytics",
    label: "Visitor activity",
    icon: <BarChart3 className="w-4 h-4" />,
  },
  {
    to: "/parish-office-gav8thhe/applications/archive",
    label: "Archive",
    icon: <Archive className="w-4 h-4" />,
  },
];

export default function AdminNav() {
  const navigate = useNavigate();
  const { pathname } = useLocation();

  // Three of these are prefixes of another, so a plain startsWith would
  // light up two or three chips at once. Exact wins, and the two
  // sub-boards are checked before the section they live under.
  function isOn(item: NavItem): boolean {
    if (pathname === item.to) return true;
    if (item.to === "/parish-office-gav8thhe/applications") {
      return pathname.startsWith("/parish-office-gav8thhe/applications/")
        && !pathname.startsWith("/parish-office-gav8thhe/applications/archive");
    }
    if (item.to === "/parish-office-gav8thhe/announcements") {
      // The board routes have their own chips.
      return false;
```

```
    }
    return false;
  }

  return (
    <nav aria-label="Parish office" className="admin-nav print:hidden">
      {ITEMS.map(item => {
        const on = isOn(item);
        return (
          <button
            key={item.to}
            type="button"
            onClick={() => navigate(item.to)}
            aria-current={on ? "page" : undefined}
            className={`admin-nav__item${on ? " is-on" : ""}`}
          >
            {item.icon}
            {item.label}
          </button>
        );
      })}
    </nav>
  );
}
```

2.3.13 API-020 — AdminTiles.tsx

Code ID	API-020
File Name	AdminTiles.tsx
File Path	SanctDemo/src/admin/AdminTiles.tsx
Language / Type	TypeScript (React, TSX)
Lines	178
Functions / Classes	AdminTiles (exported component/function (default))
Related Files (its imports)	No project-local imports.
Where It Is Used	src/admin/AdminDashboard.tsx
How to Recognize It	Open SanctDemo/src/admin/AdminTiles.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminTiles.tsx

```
import { useMemo, useState } from "react";
import { useNavigate } from "react-router-dom";
import {
  ClipboardList, Megaphone, ToggleLeft, Archive, Church,
  PartyPopper, CalendarDays, BookOpen, BarChart3, Search,
} from "lucide-react";

/**
 * The office, as a board of doors.
 *
 * ## Why a grid of tiles and not just the nav
 *
 * AdminNav is a row of small chips. It is fine once you know the place,
 * and poor on the first morning: nine words in a line, all the same
 * size, with nothing to say which one you probably want. A parish
 * secretary opening this for the first time needs to see the whole
 * office at once and recognise a door by its shape.
 *
 * The nav stays. This is for arriving; the nav is for moving around
 * once you have.
 *
 * ## Why icons AND words, at this size
 *
 * The same rule AdminNav already states: an icon is a guess until you
 * have clicked it once, a word is not. So every tile carries both, and
 * the icon is large enough to be recognised from a lean-back distance
 * rather than decorating the label.
 *
 * ## Why the ribbon says what it says
 *
 * The reference design this follows puts a ribbon on every tile reading
 * "System", which tells a user nothing. Here it carries the one fact
 * that changes what you do next: whether a section is something the
```

```
 * parish fills in itself, or something the app keeps on its own.
 */

interface TileDef {
  to: string;
  label: string;
  icon: React.ReactNode;
  /** What the parish actually does here - used by the search box. */
  keywords: string;
  ribbon: "Parish" | "Automatic";
}

const TILES: TileDef[] = [
  {
    to: "/parish-office-gav8thhe/applications",
    label: "Applications",
    icon: <ClipboardList />,
    keywords: "applications ministry sacrament baptism wedding approve reject pending binyag kasal aplikasyon ministeryo",
    ribbon: "Parish",
  },
  {
    to: "/parish-office-gav8thhe/mass-schedule",
    label: "Mass Schedule",
    icon: <Church />,
    keywords: "mass schedule times sunday weekday misa oras linggo iskedyul",
    ribbon: "Parish",
  },
  {
    to: "/parish-office-gav8thhe/announcements",
    label: "Announcements",
    icon: <Megaphone />,
    keywords: "announcements bulletin notice post feast days events calendar patalastas abiso kapistahan pista kaganapan",
    ribbon: "Parish",
  },
  // Ministries and Sacraments replace three tiles: "What People Read",
  // which edited the descriptions of both, and "Open & Closed", which
  // switched applications for both. Those were two screens describing
  // one thing, so the parish had to visit both to finish a job.
  //
  // Feast Days and Events are gone from here too. They are boards on
  // the Announcements page, reachable from it, and three tiles for one
  // page is three places to look for the same thing.
  {
    to: "/parish-office-gav8thhe/ministries",
    label: "Ministries",
    icon: <BookOpen />,
    keywords: "ministries ministry open closed accepting applications description requirements choir lector ministeryo",
    ribbon: "Parish",
  },
  {
    to: "/parish-office-gav8thhe/sacraments",
    label: "Sacraments",
    icon: <ToggleLeft />,
    keywords: "sacraments baptism wedding confirmation open closed accepting applications binyag kasal kumpil",
    ribbon: "Parish",
  },
  {
    to: "/parish-office-gav8thhe/analytics",
    label: "Visitor Activity",
```

```
    icon: <BarChart3 />,
    keywords: "analytics visitors activity statistics numbers",
    ribbon: "Automatic",
  },
  {
    to: "/parish-office-gav8thhe/applications/archive",
    label: "Archive",
    icon: <Archive />,
    keywords: "archive old finished completed past",
    ribbon: "Automatic",
  },
];

export default function AdminTiles({ parishName }: { parishName: string }) {
  const navigate = useNavigate();
  const [query, setQuery] = useState("");

  const shown = useMemo(() => {
    const q = query.trim().toLowerCase();
    if (!q) return TILES;
    // Matched against the keywords as well as the label, so "binyag" or
    // "oras" finds the right door for somebody who does not think in the
    // app's English section names.
    return TILES.filter(
      t => t.label.toLowerCase().includes(q) || t.keywords.includes(q),
    );
  }, [query]);

  return (
    <section className="admin-board">
      <h2 className="admin-board__title">{parishName} Dashboard</h2>

      <form
        className="admin-board__search"
        role="search"
        onSubmit={e => {
          e.preventDefault();
          // Enter on a single result opens it. With several, or none,
          // the list on screen is already the answer.
          if (shown.length === 1) navigate(shown[0].to);
        }}
      >
        <input
          type="search"
          value={query}
          onChange={e => setQuery(e.target.value)}
          placeholder="Search the office, e.g. Mass"
          aria-label="Search admin sections"
          className="admin-board__input"
        />
        <button type="submit" className="admin-board__go" aria-label="Search">
          <Search className="w-4 h-4" />
        </button>
      </form>

      {shown.length === 0 ? (
        <p className="admin-board__empty">
          Nothing here matches “{query}”. Try “mass”, “applications” or “history”.
        </p>
```

```
      ) : (
        <ul className="admin-board__grid">
          {shown.map(tile => (
            <li key={tile.to}>
              <button
                type="button"
                onClick={() => navigate(tile.to)}
                className="admin-tile"
              >
                <span
                  className={`admin-tile__ribbon admin-tile__ribbon--${tile.ribbon.toLowerCase()}`}
                  aria-hidden
                >
                  {tile.ribbon}
                </span>
                <span className="admin-tile__icon" aria-hidden>
                  {tile.icon}
                </span>
                <span className="admin-tile__label">{tile.label}</span>
              </button>
            </li>
          ))}
        </ul>
      )}
    </section>
  );
}
```

2.3.14 API-021 — ParishStatus.tsx

Code ID	API-021
File Name	ParishStatus.tsx
File Path	SanctDemo/src/admin/ParishStatus.tsx
Language / Type	TypeScript (React, TSX)
Lines	147
Functions / Classes	ParishStatus (exported component/function (default)); Card (internal function); Dot (internal function)
Related Files (its imports)	../lib/firebase, ../data, ../lib/useParishContent, ../lib/availability, ../lib/schedule
Where It Is Used	src/admin/AdminDashboard.tsx
How to Recognize It	Open SanctDemo/src/admin/ParishStatus.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ParishStatus.tsx

```
import { useEffect, useState } from "react";
import { collection, onSnapshot, query, where } from "firebase/firestore";
import { useNavigate } from "react-router-dom";
import { CalendarDays, Church, Sparkles, Users } from "lucide-react";
import { db } from "../lib/firebase";
import { SACRAMENTS, ministriesForParish } from "../data";
import { useParishContent } from "../lib/useParishContent";
import { summarise } from "../lib/availability";
import { availableTimes } from "../lib/schedule";
import {
  FEAST_TYPE, publishedOnly, upcomingAnnouncements, formatWhen,
  type AnnouncementDoc,
} from "../lib/announcements";

/**
 * What the parish looks like to a pilgrim right now.
 *
 * ## Why this is separate from the application counts above it
 *
 * Those answer "how much work is in". This answers "what have we got
 * switched on" — and the two get confused. An admin who closed the choir
 * three weeks ago and forgot has no way to notice from a list of
 * applications, because the symptom is applications that never arrive.
 *
 * Every tile links to the page that changes it, so noticing and fixing
 * are one click apart.
 */
export default function ParishStatus({ churchId }: { churchId: string }) {
  const navigate = useNavigate();
  const managed = useParishContent(churchId);
  const [announcements, setAnnouncements] = useState<AnnouncementDoc[]>([]);
```

```
useEffect(() => {
  const q = query(collection(db, "announcements"), where("churchId", "==", churchId));
  return onSnapshot(
    q,
    snap => setAnnouncements(snap.docs.map(d => ({ ...(d.data() as AnnouncementDoc), id: d.id }))),
    // Not fatal. This panel is a summary; an admin whose announcements
    // fail to load should still see their availability.
    () => setAnnouncements([]),
  );
}, [churchId]);

const parishMinistries = summarise(managed, ministriesForParish(churchId).map(m => m.id));
const sacraments = summarise(managed, SACRAMENTS.map(s => s.id));

const schedule = managed?.massSchedule ?? [];
const suspended = schedule.reduce(
  (n, entry) => n + (entry.unavailableTimes?.length ?? 0), 0,
);
const running = schedule.reduce((n, entry) => n + availableTimes(entry).length, 0);

const nextFeast = upcomingAnnouncements(
  publishedOnly(announcements).filter(a => a.type === FEAST_TYPE),
)[0];

return (
  <section className="mt-7">
    <h2 className="mb-3 text-[17px] font-bold font-serif italic text-[var(--color-brand-text)]">
      Parish status
    </h2>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-3">
      <Card
        icon={<Users className="w-4 h-4" />}
        label="Ministries"
        onClick={() => navigate("/parish-office-gav8thhe/availability")}
        good={`${parishMinistries.open} accepting applications`}
        bad={parishMinistries.closed > 0 ? `${parishMinistries.closed} not available` : null}
      />
      <Card
        icon={<Sparkles className="w-4 h-4" />}
        label="Sacraments"
        onClick={() => navigate("/parish-office-gav8thhe/availability")}
        good={`${sacraments.open} open for applications`}
        bad={sacraments.closed > 0 ? `${sacraments.closed} not available` : null}
      />
      <Card
        icon={<Church className="w-4 h-4" />}
        label="Mass schedule"
        onClick={() => navigate("/parish-office-gav8thhe/mass-schedule")}
        good={
          schedule.length === 0
            ? "Using the times built into the app"
            : `${running} Mass${running === 1 ? "" : "es"} a week`
        }
        bad={suspended > 0 ? `${suspended} suspended` : null}
      />
      <Card
        icon={<CalendarDays className="w-4 h-4" />}
```

```
          label="Next feast day"
          onClick={() => navigate("/parish-office-gav8thhe/announcements/feasts")}
          good={
            nextFeast
              ? `${nextFeast.title}${nextFeast.when ? ` — ${formatWhen(nextFeast.when)}` : ""}`
              : "None published"
          }
          bad={null}
        />
      </div>
    </section>
  );
}

function Card({
  icon, label, good, bad, onClick,
}: {
  icon: React.ReactNode;
  label: string;
  good: string;
  /** The part an admin needs to notice. Absent when there is nothing off. */
  bad: string | null;
  onClick: () => void;
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      className="text-left rounded-[18px] border border-[var(--color-brand-border)] bg-[var(--color-brand-card)] p-4 transition-colors hover:border-[var(--color-
brand-primary)]"
    >
      <span className="flex items-center gap-2 text-[13px] font-bold uppercase tracking-wider text-[var(--color-brand-secondary)]">
        <span className="text-[var(--color-brand-primary)]">{icon}</span>
        {label}
      </span>
      <span className="mt-1.5 flex items-center gap-2 text-[16px] font-bold text-[var(--color-brand-text)]">
        <Dot on /> {good}
      </span>
      {/* Only rendered when something is off. A permanent "0 not
          available" line trains people to stop reading this row. */}
      {bad && (
        <span className="mt-1 flex items-center gap-2 text-[15px] font-bold text-[var(--color-brand-secondary)]">
          <Dot on={false} /> {bad}
        </span>
      )}
    </button>
  );
}

function Dot({ on }: { on: boolean }) {
  return (
    <span
      aria-hidden
      className="w-2 h-2 rounded-full shrink-0"
      style={{ background: on ? "var(--color-brand-success)" : "var(--color-brand-border)" }}
    />
  );
}
```

2.3.15 API-022 — StatusBadge.tsx

Code ID	API-022
File Name	StatusBadge.tsx
File Path	SanctDemo/src/admin/StatusBadge.tsx
Language / Type	TypeScript (React, TSX)
Lines	30
Functions / Classes	StatusBadge (exported component/function (default))
Related Files (its imports)	../lib/applications, ../types
Where It Is Used	src/admin/AdminApplicationDetail.tsx, src/admin/AdminApplications.tsx, src/admin/AdminDashboard.tsx
How to Recognize It	Open SanctDemo/src/admin/StatusBadge.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StatusBadge.tsx

```
import { STATUS_LABEL } from "../lib/applications";
import type { ApplicationStatus } from "../types";

/**
 * A status, as a word and a colour.
 *
 * The word is not decoration. Colour alone fails for anyone who cannot
 * separate the hues, and it fails again when the table is printed, which
 * a parish office does. The tint narrows the search; the word is the
 * answer.
 */
const TONE: Record<ApplicationStatus, { bg: string; fg: string }> = {
  pending:      { bg: "var(--color-brand-card-sunk)", fg: "var(--color-brand-text)" },
  under_review: { bg: "#E8EEF8", fg: "#1C2C56" },
  approved:     { bg: "#E4F0E8", fg: "#2F5A41" },
  rejected:     { bg: "#FBE9E4", fg: "#8E3F2C" },
  completed:    { bg: "#EDE7F6", fg: "#4A3B70" },
};

export default function StatusBadge({ status }: { status: ApplicationStatus }) {
  const tone = TONE[status] ?? TONE.pending;
  return (
    <span
      className="inline-block rounded-full px-2.5 py-1 text-[13px] font-bold whitespace-nowrap"
      style={{ background: tone.bg, color: tone.fg }}
    >
      {STATUS_LABEL[status] ?? status}
    </span>
  );
}
```

Part 3 — Maps and Map Placer

3.1 In-App Map and Navigation

3.1.1 MAP-001 — DioceseMapLive.tsx

Code ID	MAP-001
File Name	DioceseMapLive.tsx
File Path	SanctDemo/src/components/DioceseMapLive.tsx
Language / Type	TypeScript (React, TSX)
Lines	1702
Functions / Classes	OPEN_IN_GOOGLE_MAPS_URL (exported constant); TILE_HOST (exported constant); resolveColor (internal function); shortLabel (internal function); vicariateLabel (internal function); routeLineId (internal function); routeFeature (internal function); upsertRouteLayer (internal function); upsertAccuracyCircle (internal function); removeAccuracyCircle (internal function); upsertTripLayer (internal function); removeTripLayer (internal function); removeRouteLayer (internal function); isTileHostError (internal function); isNonFatalStyleError (internal function); DioceseMapLive (exported component/function (default))
Related Files (its imports)	../context/PresenceContext, ../lib/project, ../lib/geo, ../lib/routing, ../lib/mapSearch, ../lib/mapMarkers, ../data, ./MapPlaceSheet, ../lib/heading, ../lib/mapAccuracy, ../lib/useDeviceHeading, ./CompassControl, ./TripPlanner, ./NavigationOverlay, ../data/diocese-parishes.json, ./DioceseMap
Where It Is Used	src/components/CustomDioceseMap.tsx
How to Recognize It	Open SanctDemo/src/components/DioceseMapLive.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — DioceseMapLive.tsx

```
import { useEffect, useMemo, useRef, useState } from "react";
import { Map as MapLibreMap, Marker as MapLibreMarker, NavigationControl, LngLatBounds, type GeoJSONSource, type DataDrivenPropertyValueSpecification } from
"maplibre-gl";
import "maplibre-gl/dist/maplibre-gl.css";
import { usePresence } from "../context/PresenceContext";
import { DIOCESE_BOUNDS } from "../lib/project";
import { haversineMeters, type Coordinates } from "../lib/geo";
import { formatDistance, formatWalkingMinutes, getWalkingDirections, type WalkingRoute } from "../lib/routing";
import { searchParishesScored, type SearchableParish } from "../lib/mapSearch";
import { buildChurchPinElement } from "../lib/mapMarkers";
import { PARISH_PATRON_IMAGES } from "../data";
import MapPlaceSheet from "./MapPlaceSheet";
import { shortestAngleDelta } from "../lib/heading";
import { accuracyRadiusExpression } from "../lib/mapAccuracy";
```

```
import {
  FOLLOW_DURATION_MS, FOLLOW_ZOOM, followCamera, needsCameraMove, needsZoomChange,
  shouldFollow, zoomForTurn,
} from "../lib/followCamera";
import { useDeviceHeading } from "../lib/useDeviceHeading";
import { Plus } from "lucide-react";
import { type MapOrientationMode } from "./CompassControl";
import TripPlanner from "./TripPlanner";
import {
  MAX_TRIP_STOPS,
  isOptimalOrder,
  measureTrip,
  orderStops,
  stopLabel,
  type TripRoute,
  type TripStop,
} from "../lib/pilgrimage";
import NavigationOverlay from "./NavigationOverlay";
import parishData from "../data/diocese-parishes.json";
import DioceseMap from "./DioceseMap";

// Beyond this, a position change is a relocation (first fix, or the demo
// simulator being switched) rather than walking, and easing across it looks
// like the map flying somewhere. 120m is far more than GPS noise and far
// less than a walk between parishes.
const MARKER_SNAP_METERS = 120;

// One GPS tick is roughly a second; easing a little under that keeps the
// marker continuously in motion without lagging visibly behind the fix.
const MARKER_EASE_MS = 800;

// Below this the map is already pointing where it should be, and issuing a
// fresh easeTo would restart the animation on every sensor event.
const MAP_BEARING_EPSILON = 1;

interface DioceseMapLiveProps {
  onSelectParish: (parishId: string) => void;
  /**
   * A diocese parish id handed over by Home's "Get directions". The map draws the
   * walking route to it as soon as it is ready, then calls onWalkToConsumed
   * so returning to this tab later does not silently redraw a route nobody
   * asked for a second time.
   */
  walkToParishId?: string | null;
  onWalkToConsumed?: () => void;
  // When set, the map surface itself is given this fixed pixel height and
  // the legend/open-link stack below it at their natural height, instead of
  // the map flexing to fill a fixed-height ancestor (the h-72/h-64 cards on
  // the dashboard and church selector). Used by the dedicated map screen so
  // the map keeps ~374px regardless of how tall the legend/link end up.
  heightPx?: number;
}

// The client's own Google My Map ("SanctDemoMap") — a fully public link that
// costs nothing to keep around and is genuinely useful (e.g. sharing outside
// the app), even though the map itself is now reproduced here instead of
// embedded.
const MY_MAP_ID = "1gNkblHn4JSJoWLb6zP4D_h6E6WIZomg";
export const OPEN_IN_GOOGLE_MAPS_URL = `https://www.google.com/maps/d/viewer?mid=${MY_MAP_ID}`;
```

```
// The study-area polygon is gone at the client's request.
//
// It was their own KML scope shape, drawn as a shaded overlay with a caption.
// Removed rather than recoloured: it annotated the CAPSTONE's coverage area,
// which is a fact about the project and not about any parish, and on the map
// it read as though the shaded strip meant something to a pilgrim. Its fill
// also dominated the view at street zoom. The scope belongs in the write-up,
// not on the map a congregation uses.

interface ParishRecord {
  id: string;
  name: string;
  vicariate: string;
  coordinates: { lat: number; lng: number };
  status: "live" | "coming_soon";
}

const PARISHES = parishData.parishes as ParishRecord[];

// Free, keyless vector tiles — OpenFreeMap (openfreemap.org), an unlimited
// no-signup host of OpenMapTiles-schema data (MIT/ODbL). No API key, no
// billing account, nothing that can expire mid-defense — the client's own
// requirement. `positron` is the client's preferred design, taken from
// their own earlier prototype (DioceseMap.jsx): a light, deliberately plain
// basemap, so the church pins stay the loudest thing on screen. It ships
// pre-styled and pre-decluttered, unlike the "liberty" style this replaces,
// which needed hand-tuned layer-by-layer restyling to reach the same
// result — that restyling is gone with it.
const STYLE_URL = "https://tiles.openfreemap.org/styles/positron";
export const TILE_HOST = "tiles.openfreemap.org";

// diocese-parishes.json carries all 31 parishes, including the 2 that are
// already "live" in ROUTES (src/data.ts) — but under different ids
// (parish-mary-help-of-christians-parish / parish-san-roque-cathedral
// rather than route-mhcp / route-src). Tapping a live pin's popup action
// must open the tour via the route id App.tsx already wires up, so this
// maps the JSON id to the route id for just those two instead of drawing
// duplicate pins.
const LIVE_PARISH_TO_ROUTE_ID: Record<string, string> = {
  "parish-mary-help-of-christians-parish": "route-mhcp",
  "parish-san-roque-cathedral": "route-src",
};

interface LiveParish {
  routeId: string;
  name: string;
  coordinates: Coordinates;
}

// The subset of PARISHES that are actually navigable (status "live" and
// mapped to a route id) — both the distance panel and the walking-route
// layers only ever target these, never a "coming soon" pin.
const LIVE_PARISHES: LiveParish[] = PARISHES.filter(
  p => p.status === "live" && LIVE_PARISH_TO_ROUTE_ID[p.id],
).map(p => ({
  routeId: LIVE_PARISH_TO_ROUTE_ID[p.id],
  name: p.name,
  coordinates: p.coordinates,
```

```
}));

// Reads a CSS custom property (optionally through a color-mix() expression)
// by letting the browser resolve it on a detached element, rather than
// hardcoding any hex value here. MapLibre paint properties need a literal
// color string, not a live var() reference, so this is the runtime-read
// equivalent of what plain CSS classes do for the HTML pin/popup/search
// elements below.
function resolveColor(expr: string): string {
  const probe = document.createElement("div");
  probe.style.color = expr;
  document.body.appendChild(probe);
  const resolved = getComputedStyle(probe).color;
  document.body.removeChild(probe);
  return resolved || "var(--color-brand-primary)";
}

function shortLabel(name: string): string {
  return name.replace(/ Guide$/, "").replace(/ Tour$/, "").replace(/ Parish$/, "");
}

// Every parish's vicariate in diocese-parishes.json is flagged
// `vicariateVerified: false` — each one was assigned to its nearest
// vicariate seat, not sourced from the diocese as official. It's still
// shown (the rule this codebase holds itself to: unverified data is
// labelled, never hidden — see coordinatesVerified/scheduleVerified), but
// never as a bare fact, and — per mapSearch.ts — never matched against.
function vicariateLabel(vicariate: string, parishId?: string): string {
  // The two implemented parishes have been through the parish office:
  // their vicariate, coordinates and schedule are all confirmed, so the
  // caveat is simply wrong on them now. It stays on the other
  // twenty-nine, where each vicariate is still our nearest-seat guess
  // rather than the diocese's own word.
  if (parishId && CONFIRMED_PARISHES.has(parishId)) return vicariate;
  return `${vicariate} (unconfirmed)`;
}

/** Parishes whose details have been confirmed with the parish itself. */
const CONFIRMED_PARISHES = new Set([
  "parish-mary-help-of-christians-parish",
  "parish-san-roque-cathedral",
]);

// Search shows a parish's vicariate as its secondary "location" line — the
// same role `location` plays in the client's prototype data
// (churches.json), which this dataset doesn't carry a dedicated field for
// — but, unlike the prototype, does not match against it (see
// mapSearch.ts's searchParishes for why).
interface SearchParish extends SearchableParish {
  routeId?: string;
  isLive: boolean;
}

const SEARCHABLE_PARISHES: SearchParish[] = PARISHES.map(p => ({
  id: p.id,
  name: shortLabel(p.name),
  location: vicariateLabel(p.vicariate, p.id),
  coordinates: p.coordinates,
  routeId: LIVE_PARISH_TO_ROUTE_ID[p.id],
```

```
  isLive: p.status === "live" && Boolean(LIVE_PARISH_TO_ROUTE_ID[p.id]),
}));

function routeLineId(routeId: string): { sourceId: string; layerId: string } {
  return { sourceId: `route-line-src-${routeId}`, layerId: `route-line-${routeId}` };
}

function routeFeature(route: WalkingRoute): GeoJSON.Feature<GeoJSON.LineString> {
  return {
    type: "Feature",
    properties: {},
    geometry: {
      type: "LineString",
      coordinates: route.path.map(p => [p.lng, p.lat]),
    },
  };
}

// Draws (or updates) one parish's walking-route line. A 'direct' route
// (OSRM unreachable/timed out — see routing.ts) is dashed so it reads
// visually distinct from a real routed street path, matching the
// distance-panel label ("direct" vs a walking time).
function upsertRouteLayer(map: MapLibreMap, routeId: string, route: WalkingRoute, accentColor: string) {
  const { sourceId, layerId } = routeLineId(routeId);
  const data = routeFeature(route);
  const dasharray = route.kind === "direct" ? [2, 2] : [1, 0];

  const existingSource = map.getSource(sourceId) as GeoJSONSource | undefined;
  if (existingSource) {
    existingSource.setData(data);
  } else {
    map.addSource(sourceId, { type: "geojson", data });
  }

  if (map.getLayer(layerId)) {
    map.setPaintProperty(layerId, "line-dasharray", dasharray);
  } else {
    map.addLayer({
      id: layerId,
      type: "line",
      source: sourceId,
      layout: { "line-join": "round", "line-cap": "round" },
      paint: {
        "line-color": accentColor,
        "line-width": 3,
        "line-dasharray": dasharray,
      },
    });
  }
}


/** Google's location blue, which is what the reference shows. */
const YOU_BLUE = "#1A73E8";

const ACCURACY_SOURCE = "you-accuracy-src";
const ACCURACY_FILL = "you-accuracy-fill";
const ACCURACY_EDGE = "you-accuracy-edge";
```

```
/**
 * The translucent disc around the blue dot, showing how sure the fix is.
 *
 * ## Why this is a map layer and not a CSS circle
 *
 * The radius is a distance on the ground - "somewhere within 20 metres" -
 * so it has to grow as you zoom in and shrink as you zoom out, the way the
 * streets under it do. A div sized in pixels would claim 20 m at one zoom
 * and 200 m at the next while looking identical, which is worse than
 * drawing nothing: it would be a confident picture of the wrong thing.
 *
 * MapLibre only takes `circle-radius` in pixels, so the metres are
 * converted in an expression evaluated per frame at the current zoom:
 *
 *     pixels = metres / (156543.03392 * cos(latitude) / 2^zoom)
 *
 * cos(latitude) is folded in as a constant because the map never spans
 * enough latitude for it to drift, and MapLibre expressions have no cos().
 */
function upsertAccuracyCircle(
  map: MapLibreMap,
  position: Coordinates,
  accuracyMeters: number,
  colour: string,
) {
  const data: GeoJSON.Feature = {
    type: "Feature",
    properties: {},
    geometry: { type: "Point", coordinates: [position.lng, position.lat] },
  };

  const radius = accuracyRadiusExpression(accuracyMeters, position.lat);

  const existing = map.getSource(ACCURACY_SOURCE) as GeoJSONSource | undefined;
  if (existing) {
    existing.setData(data);
  } else {
    map.addSource(ACCURACY_SOURCE, { type: "geojson", data });
  }

  if (map.getLayer(ACCURACY_FILL)) {
    map.setPaintProperty(ACCURACY_FILL, "circle-radius", radius);
    map.setPaintProperty(ACCURACY_EDGE, "circle-radius", radius);
    return;
  }

  map.addLayer({
    id: ACCURACY_FILL,
    type: "circle",
    source: ACCURACY_SOURCE,
    paint: {
      "circle-radius": radius,
      "circle-color": colour,
      "circle-opacity": 0.16,
      // No pitch scaling: the disc is a footprint on the ground, and
      // letting it shrink with pitch would understate the uncertainty.
      "circle-pitch-alignment": "map",
    },
  });
```

```
  // A faint edge. Without it the fill dissolves into a pale basemap and
  // the disc reads as a smudge rather than as a boundary.
  map.addLayer({
    id: ACCURACY_EDGE,
    type: "circle",
    source: ACCURACY_SOURCE,
    paint: {
      "circle-radius": radius,
      "circle-color": "rgba(0,0,0,0)",
      "circle-stroke-color": colour,
      "circle-stroke-width": 1,
      "circle-stroke-opacity": 0.35,
      "circle-pitch-alignment": "map",
    },
  });
}

function removeAccuracyCircle(map: MapLibreMap) {
  for (const id of [ACCURACY_EDGE, ACCURACY_FILL]) {
    if (map.getLayer(id)) map.removeLayer(id);
  }
  if (map.getSource(ACCURACY_SOURCE)) map.removeSource(ACCURACY_SOURCE);
}

const TRIP_SOURCE = "trip-line-src";
const TRIP_LAYER = "trip-line";

/**
 * The whole visit as one line.
 *
 * Every leg concatenated rather than a layer per hop: a Bisita Iglesia is
 * one journey, and fourteen sources with fourteen layers to add and remove
 * is fourteen chances to leak one. The dash tells the same story the single
 * route line tells - solid where the streets were routed, dashed where the
 * router could not answer and the line is a straight one.
 */
function upsertTripLayer(map: MapLibreMap, trip: TripRoute, accentColor: string) {
  const coordinates: [number, number][] = [];
  for (const leg of trip.legs) {
    if (!leg.route) continue;
    for (const point of leg.route.path) {
      const last = coordinates[coordinates.length - 1];
      // The end of one leg is the start of the next; drawn twice it is a
      // zero-length segment that MapLibre renders as a blob at every church.
      if (last && last[0] === point.lng && last[1] === point.lat) continue;
      coordinates.push([point.lng, point.lat]);
    }
  }

  if (coordinates.length < 2) {
    removeTripLayer(map);
    return;
  }

  const data: GeoJSON.Feature<GeoJSON.LineString> = {
    type: "Feature",
    properties: {},
    geometry: { type: "LineString", coordinates },
```

```
  };
  const dasharray = trip.hasDirectLeg ? [2, 2] : [1, 0];

  const source = map.getSource(TRIP_SOURCE) as GeoJSONSource | undefined;
  if (source) {
    source.setData(data);
  } else {
    map.addSource(TRIP_SOURCE, { type: "geojson", data });
  }

  if (map.getLayer(TRIP_LAYER)) {
    map.setPaintProperty(TRIP_LAYER, "line-dasharray", dasharray);
    map.setPaintProperty(TRIP_LAYER, "line-color", accentColor);
  } else {
    map.addLayer({
      id: TRIP_LAYER,
      type: "line",
      source: TRIP_SOURCE,
      layout: { "line-join": "round", "line-cap": "round" },
      paint: { "line-color": accentColor, "line-width": 4, "line-dasharray": dasharray },
    });
  }
}

function removeTripLayer(map: MapLibreMap) {
  if (map.getLayer(TRIP_LAYER)) map.removeLayer(TRIP_LAYER);
  if (map.getSource(TRIP_SOURCE)) map.removeSource(TRIP_SOURCE);
}

function removeRouteLayer(map: MapLibreMap, routeId: string) {
  const { sourceId, layerId } = routeLineId(routeId);
  if (map.getLayer(layerId)) map.removeLayer(layerId);
  if (map.getSource(sourceId)) map.removeSource(sourceId);
}

// Detects whether a request the browser attempted actually reached the
// tile host — being "online" per navigator.onLine but unable to resolve or
// reach the tile host (captive portal, DNS failure, firewalled venue
// Wi-Fi) is the more common failure in a defense room than a literal
// offline flag, so this is checked in addition to it, not instead of it.
function isTileHostError(error: unknown): boolean {
  const message = error instanceof Error ? error.message : String(error);
  return message.includes(TILE_HOST) || /Failed to fetch|NetworkError|ERR_/.test(message);
}

/**
 * A complaint about the STYLE, not about the map being usable.
 *
 * MapLibre validates paint and layout properties and reports anything it
 * dislikes through the same `error` event it uses for a failed tile
 * fetch. The two could not be less alike: a malformed `circle-radius`
 * means one layer draws wrong, while a failed tile fetch means there is
 * no map. Treating them the same is what sent this screen to the
 * schematic fallback the moment GPS produced a fix - two validation
 * messages per update, three updates, and the live map was gone.
 *
 * The map keeps rendering through every one of these, so none of them
 * counts towards falling back.
 */
```

```
function isNonFatalStyleError(error: unknown): boolean {
  const message = error instanceof Error ? error.message : String(error);
  return (
    /may only be used as input to/.test(message) ||
    /Expected value to be of type/.test(message) ||
    /Unknown property|does not exist in the set of known/.test(message) ||
    /^layers?[.[]/.test(message)
  );
}

export default function DioceseMapLive({ onSelectParish, heightPx, walkToParishId, onWalkToConsumed }: DioceseMapLiveProps) {
  const { position, accuracyMeters, gpsStatus, simulation } = usePresence();

  // The position on screen is the last one the device gave us, which is
  // not the same as where the pilgrim is standing now. When the watch
  // stops producing fixes the dot silently freezes, and the honest
  // reading of "Recentre does nothing after I move" is that it flew to a
  // stale point exactly as asked. Say so rather than let the badge keep
  // asserting a metre figure for a fix that has stopped arriving.
  const positionIsStale = simulation === "off" && (gpsStatus === "lost" || gpsStatus === "searching");
  const containerRef = useRef<HTMLDivElement | null>(null);
  const mapRef = useRef<MapLibreMap | null>(null);
  // Parish markers keyed by parish id, so a search result can fly to and
  // open the matching pin's popup — mirrors the client's prototype, which
  // keeps the same lookup for the same reason.
  const markersRef = useRef<Map<string, MapLibreMarker>>(new Map());
  const youMarkerRef = useRef<MapLibreMarker | null>(null);
  /** The marker's own element, so navigation can restyle it without a render. */
  const youElementRef = useRef<HTMLDivElement | null>(null);
  // The direction light inside the "you are here" dot, held directly so
  // the compass can be written to its transform at sensor rate without
  // re-rendering this component. The sensor fires several times a second;
  // a setState per reading would re-render the whole map screen.
  const beamRef = useRef<HTMLSpanElement | null>(null);
  // Where the marker currently *appears*, which lags the latest fix while
  // the ease runs — the start point for the next ease.
  const animatedPositionRef = useRef<Coordinates | null>(null);
  const moveFrameRef = useRef<number | null>(null);
  const [mode, setMode] = useState<"loading" | "live" | "fallback">("loading");
  const [offlineFlagged, setOfflineFlagged] = useState(false);
  const [routes, setRoutes] = useState<Record<string, WalkingRoute>>({});

  // A Bisita Iglesia in progress: the churches chosen, in the order they
  // will be visited, and the measured walk between them.
  const [tripStops, setTripStops] = useState<TripStop[]>([]);
  const [tripOpen, setTripOpen] = useState(false);
  const [trip, setTrip] = useState<TripRoute | null>(null);
  const [tripMeasuring, setTripMeasuring] = useState(false);
  /** Markers keyed by stop id, so only the letters that changed are redrawn. */
  const tripMarkersRef = useRef<Map<string, MapLibreMarker>>(new Map());
  /** So "Add a church" can put the cursor where the next church is named. */
  const searchInputRef = useRef<HTMLInputElement | null>(null);
  const [query, setQuery] = useState("");
  // North-up, and no longer switchable. The dial that used to toggle it has
  // been taken off the map: the direction cone on the you-are-here dot
  // already shows which way the pilgrim faces, so a second compass beside it
  // was saying the same thing twice while covering the map to do it.
  //
  // The mode is kept as a value rather than inlined because the cone and the
```

```
// map bearing both branch on it, and a rotating map is the obvious next
// thing to want back.
//
// The cast is load-bearing: TypeScript narrows a const declared straight
// from a string literal down to that literal, and then reads every
// `=== "heading-up"` below as a comparison that cannot hold and reports it
// as a mistake. Widening back to the union keeps both branches legal.
const orientationMode = "north-up" as MapOrientationMode;
// The parish currently being navigated to, if any. Separate from a drawn
// route: a route is a line on the map, navigation is a live session that
// follows the pilgrim and reroutes.
const [navigatingTo, setNavigatingTo] = useState<{ id: string; name: string; coordinates: Coordinates } | null>(null);
/**
 * Distance to the next manoeuvre, reported by the navigation overlay.
 * A ref rather than state: it changes on every GPS fix and only the
 * camera reads it, so re-rendering the whole map for it would be a
 * repaint per fix for nothing visible.
 */
const metersToTurnRef = useRef<number | null>(null);
// True once the camera has closed in for the current navigation session, so
// a reroute mid-walk does not yank it back.
const didZoomForNavRef = useRef(false);
/**
 * Set once the walker pinches the map themselves. Automatic zoom stands
 * down from then on, the same bargain `userPannedRef` strikes for
 * panning: being overruled a second after you pinched out to see the
 * rest of the route is the complaint either way. Recentring clears it.
 */
const userZoomedRef = useRef(false);
/**
 * Whether the map is currently turned off north or tilted. Drives the
 * north-up control's visibility: a "reset the view" button on a view
 * that is already reset is a button that does nothing, and on this map
 * that is the normal state.
 */
const [viewRotated, setViewRotated] = useState(false);
/**
 * True once the walker has dragged the map during a navigation session.
 *
 * Panning mid-route is a deliberate act - checking a side street, or how
 * far is left - and a camera that hauls you back a second later makes it
 * impossible. Following stops until Recentre is tapped. A ref, not state:
 * it is read inside a camera effect that must not re-run because of it.
 */
const userPannedRef = useRef(false);


// The tapped pin, shown as a place sheet over the map. Held as an id
// rather than the parish object so a re-render always reads current data.
const [selectedParishId, setSelectedParishId] = useState<string | null>(null);

/**
 * Routes we have MEASURED but not drawn, keyed by parish id.
 *
 * This exists to stop the travel times changing under the pilgrim's finger.
 * The card used to show times from the straight-line distance until a route
 * was drawn, then switch to the road distance — and a road is always longer
 * than a straight line, so tapping Directions always made the journey
 * jump. Measured on the same parish: "8 min · 699 m direct" became
```

```
 * "10 min · 874 m" the instant you tapped, for a walk that had not changed.
 *
 * So the route is fetched when the card OPENS and the times come from the
 * road distance immediately. Nothing is drawn until Directions is tapped,
 * which is the rule the client set after unasked-for blue lines kept
 * appearing across the map — measuring and drawing are now separate acts.
 *
 * Directions then draws this already-measured route rather than asking
 * again, so the number cannot change and the line appears instantly.
 */
const [measuredRoutes, setMeasuredRoutes] = useState<Record<string, WalkingRoute>>({});

// The parish whose route is being measured right now, if any. The card
// shows "Measuring…" rather than a straight-line number that is about to be
// replaced — closing the last window in which the times could still visibly
// change under the pilgrim.
const [measuringParishId, setMeasuringParishId] = useState<string | null>(null);
const [directionsState, setDirectionsState] = useState<{ parishId: string; status: string; busy: boolean } | null>(null);
const selectedParish = selectedParishId ? PARISHES.find(p => p.id === selectedParishId) : null;

// Measure the road route as soon as a card opens, so its times are the real
// ones from the first render. Never draws; see measuredRoutes above.
useEffect(() => {
  const parish = selectedParish;
  const from = positionRef.current;
  if (!parish || !from || measuredRoutes[parish.id]) return;

  let live = true;
  setMeasuringParishId(parish.id);

  void getWalkingDirections(from, parish.coordinates).then(result => {
    if (!live) return;
    setMeasuringParishId(null);
    if (result.status === "no-position") return;
    // A 'direct' result means OSRM could not be reached and the distance is
    // a straight line. Storing it would replace one honest label with a
    // silently wrong one, so it is left out and the card falls back to
    // showing its straight-line figure, labelled "direct".
    if (result.route.kind !== "routed") return;
    setMeasuredRoutes(prev => ({ ...prev, [parish.id]: result.route }));
  });

  return () => {
    live = false;
    setMeasuringParishId(null);
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [selectedParishId]);
// Only the bearing is read here now. The status and the permission request
// were for the compass dial's own UI; the hook asks for the sensor itself
// on the first touch, so nothing on the map has to drive it.
const { heading } = useDeviceHeading();
// Held in a ref so the marker-building effect doesn't need `onSelectParish`
// in its dependency array — App.tsx passes a fresh function each render,
// and re-running that effect on every render would tear down and rebuild
// every marker for no reason.
const onSelectParishRef = useRef(onSelectParish);
onSelectParishRef.current = onSelectParish;
```

```
// Read by the popup's "Get directions" click handler, which is wired once
// when markers are built (see the marker-building effect) and must always
// see the pilgrim's *current* position, not whatever it was when the
// popup happened to be constructed.
const positionRef = useRef<Coordinates | null>(position);
positionRef.current = position;

// Invalidates an in-flight OSRM request when a newer "Get directions" tap
// lands first, so a slow response can never overwrite a fresher route.
const directionsRequestRef = useRef(0);

// Fetches and draws the walking route to ANY parish in the diocese, for the
// place sheet's Directions button. This is the only path that ever draws a
// route line — nothing is drawn merely because a position became known.
//
// Keyed by parish id, which every one of the 31 has. It used to take a tour
// route id as well, which only two parishes have, and that is what made the
// other 29 unreachable.
function getDirectionsFor(parishId: string, coords: Coordinates, alreadyMeasured?: WalkingRoute) {
  const from = positionRef.current;
  const requestId = ++directionsRequestRef.current;

  if (!from) {
    setDirectionsState({
      parishId,
      busy: false,
      status: "Your position is unknown — enable GPS or the location simulator to get directions.",
    });
    return;
  }

  // The card already measured this route when it opened. Draw that one
  // rather than asking again: the line appears with no wait, and — the
  // point of all this — the distance on screen cannot change, because it
  // is the same route the times were computed from.
  if (alreadyMeasured) {
    drawRoute(parishId, alreadyMeasured);
    setDirectionsState({ parishId, busy: false, status: describeRoute(alreadyMeasured) });
    return;
  }

  setDirectionsState({ parishId, busy: true, status: "" });

  getWalkingDirections(from, coords).then(result => {
    // A newer Directions tap (this parish or another) landed first — this
    // response is stale and must not clobber it.
    if (directionsRequestRef.current !== requestId) return;
    if (result.status === "no-position") return; // from was truthy above; kept for narrowing

    const { route, label } = result;
    drawRoute(parishId, route);
    setDirectionsState({ parishId, busy: false, status: label });
  });
}

/** Paints a route on the map and frames it. Shared by both paths above. */
function drawRoute(parishId: string, route: WalkingRoute) {
  const map = mapRef.current;
  if (!map) return;
```

```
  upsertRouteLayer(map, parishId, route, resolveColor("var(--color-brand-accent)"));
  setRoutes(prev => ({ ...prev, [parishId]: route }));

  const [first, ...rest] = route.path;
  if (!first) return;
  const bounds = rest.reduce(
    (b, p) => b.extend([p.lng, p.lat]),
    new LngLatBounds([first.lng, first.lat], [first.lng, first.lat]),
  );
  // Bottom padding keeps the route clear of the place sheet, which covers
  // the lower third of the frame.
  map.fitBounds(bounds, {
    padding: { top: 64, left: 64, right: 64, bottom: 260 },
    // Fitting all thirty-one parishes pulls back far enough that
    // the streets disappear. Past this the map stops being worth
    // looking at, so it stops here and lets the pilgrim pan.
    maxZoom: 13.4,
    duration: 500,
  });
}

/** The one-line summary the card shows under a drawn route. */
function describeRoute(route: WalkingRoute): string {
  return `${formatDistance(route.distanceMeters)} walk · ${formatWalkingMinutes(route.durationMinutes)}`;
}

const getDirectionsRef = useRef(getDirectionsFor);
getDirectionsRef.current = getDirectionsFor;

// Scored rather than filtered, and ranked against the pilgrim's own
// position so "churches near me" can be answered with real distances and
// equally-relevant names break ties by which is closer.
const results = useMemo(
  () => searchParishesScored(query, SEARCHABLE_PARISHES, { origin: position }),
  [query, position?.lat, position?.lng],
);

// Falls back the instant the browser reports offline, even mid-session —
// a live map needs network and this app must never be caught showing an
// empty grey box because Wi-Fi dropped after the map had already loaded.
useEffect(() => {
  const goOffline = () => {
    setOfflineFlagged(true);
    setMode("fallback");
  };
  window.addEventListener("offline", goOffline);
  if (!navigator.onLine) goOffline();
  return () => window.removeEventListener("offline", goOffline);
}, []);

useEffect(() => {
  if (mode === "fallback" || !containerRef.current) return;
  if (mapRef.current) return; // StrictMode double-invoke guard

  let cancelled = false;
  const toFallback = () => {
    if (cancelled) return;
    cancelled = true;
```

```
  setMode("fallback");
};

const map = new MapLibreMap({
  container: containerRef.current,
  style: STYLE_URL,
  center: [DIOCESE_BOUNDS.lngMin + (DIOCESE_BOUNDS.lngMax - DIOCESE_BOUNDS.lngMin) / 2, 14.655],
  // Close enough to read street names. 12.3 fitted the whole
  // diocese on screen but rendered it as a grey mass with no
  // streets in it, which is not a map anybody can use.
  zoom: 13.4,
  attributionControl: { compact: true },
});
mapRef.current = map;

map.addControl(new NavigationControl({ showCompass: false }), "top-right");

// A drag by the walker suspends the follow camera. `dragstart` and not
// `movestart`: the camera's own easeTo fires movestart too, so listening
// for that would have the map stop following itself on the first fix.
map.on("dragstart", () => { userPannedRef.current = true; });
// Only a real gesture. Guard on originalEvent, or the camera's own
// easeTo below would register as the walker zooming and switch
// automatic zoom off on the first fix.
map.on("zoomstart", e => { if ((e as { originalEvent?: unknown }).originalEvent) userZoomedRef.current = true; });
const syncRotated = () => setViewRotated(Math.abs(map.getBearing()) > 1 || map.getPitch() > 1);
map.on("rotate", syncRotated);
map.on("pitch", syncRotated);
map.on("moveend", syncRotated);

// Tapping the map itself dismisses the place sheet. Pin taps stop
// propagation, so this only ever fires on empty map.
map.on("click", () => setSelectedParishId(null));

// MapLibre's error event fires for style/sprite/glyph/tile fetch
// failures alike — this is the primary offline-detection path, since a
// venue can be "online" (navigator.onLine === true) while unable to
// reach the tile host specifically. A map that keeps erroring on every
// frame (observed in one constrained test environment, where repeated
// render-loop errors were frequent enough to starve the JS timer queue
// and delay the setTimeout safety net below) is just as unusable as one
// that never loads, so any error is also counted — several in quick
// succession fall back even when the individual message doesn't name
// the tile host. The counter check runs synchronously inside the event
// handler itself, so it isn't subject to that same starvation risk.
// Only failures that make the map UNUSABLE count. A style complaint
// does not: the map carries on drawing, and counting those is what
// used to throw the whole live map away over a bad paint property.
//
// The threshold is also higher than it was. A phone on mobile data
// drops the odd tile request as a matter of course; three of those in
// a session is ordinary, not a reason to give up on the map - and
// isTileHostError below still falls back immediately when the tile
// host itself is unreachable, which is the case that matters.
let errorCount = 0;
map.on("error", e => {
  const message = e.error instanceof Error ? e.error.message : String(e.error);

  if (isNonFatalStyleError(e.error)) {
```

```
    // Still worth seeing - it means a layer is drawing wrong - but it
    // is a developer's problem, not a reason to change what the
    // pilgrim is looking at.
    console.warn("[map] style problem, map still usable:", message);
    return;
  }

  errorCount += 1;
  console.error(`[map] error ${errorCount}:`, message);
  if (isTileHostError(e.error) || errorCount >= 8) toFallback();
});

// A render-loop failure inside MapLibre's own renderer can throw as a
// plain uncaught exception rather than a MapLibre ErrorEvent (observed
// in one constrained test environment lacking full WebGL/Worker
// support), which map.on("error") above never sees. Falling back on
// that too, rather than only on MapLibre's own event, is what keeps
// this from ever being stuck rendering a broken canvas — the specific
// failure mode matters less than "the map isn't usable, and we have a
// known-good drawing to show instead".
const isFromMapLibre = (stack: unknown) => typeof stack === "string" && stack.includes("maplibre-gl");
const onWindowError = (e: ErrorEvent) => {
  if (e.filename?.includes("maplibre-gl") || isFromMapLibre(e.error?.stack)) {
    e.preventDefault(); // already switching to the known-good fallback; no need to also surface this in the console
    toFallback();
  }
};
// Some renderer failures surface as a rejected promise rather than a
// thrown exception, which "error" above never sees.
const onRejection = (e: PromiseRejectionEvent) => {
  const reason = e.reason;
  if (isFromMapLibre(reason?.stack) || isFromMapLibre(String(reason))) {
    e.preventDefault();
    toFallback();
  }
};
window.addEventListener("error", onWindowError);
window.addEventListener("unhandledrejection", onRejection);

const loadTimeout = window.setTimeout(toFallback, 8000);

map.on("load", () => {
  window.clearTimeout(loadTimeout);
  if (cancelled) return;
  map.fitBounds(
    [
      [DIOCESE_BOUNDS.lngMin, DIOCESE_BOUNDS.latMin],
      [DIOCESE_BOUNDS.lngMax, DIOCESE_BOUNDS.latMax],
    ],
    { padding: 24, duration: 0 },
  );
  setMode("live");
});

return () => {
  cancelled = true;
  window.removeEventListener("error", onWindowError);
  window.removeEventListener("unhandledrejection", onRejection);
  window.clearTimeout(loadTimeout);
```

```
    markersRef.current.forEach(m => m.remove());
    markersRef.current.clear();
    youMarkerRef.current?.remove();
    youMarkerRef.current = null;
    map.remove();
    mapRef.current = null;
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [mode === "fallback"]);

// Parish pins are built once, when the map finishes loading — unlike the
// old dot markers, every pin here (including coming-soon ones) carries its
// own popup, so there is nothing about a pin that needs to change when the
// pilgrim's own GPS position updates. Rebuilding on every position tick
// would also tear down any popup the pilgrim currently has open.
useEffect(() => {
  const map = mapRef.current;
  if (mode !== "live" || !map || markersRef.current.size > 0) return;

  for (const parish of PARISHES) {
    const routeId = LIVE_PARISH_TO_ROUTE_ID[parish.id];
    const isLive = parish.status === "live" && Boolean(routeId);
    const displayName = shortLabel(parish.name);

    // The parish's own patron, where one has been photographed. Keyed by
    // route id because that is what the photo maps use.
    const el = buildChurchPinElement({
      name: displayName,
      isLive,
      photoUrl: routeId ? PARISH_PATRON_IMAGES[routeId] : undefined,
    });
    const lngLat: [number, number] = [parish.coordinates.lng, parish.coordinates.lat];

    // Tapping a pin opens the React place sheet rather than a MapLibre
    // popup. The popup was capped at 220px and anchored to the pin, which
    // is why it could only ever be a name and a stack of buttons — the
    // client asked for the parish's details on tap, and details need room.
    el.addEventListener("click", event => {
      event.stopPropagation();
      setSelectedParishId(parish.id);
      // Nudge the pin above the sheet, which covers the lower third.
      mapRef.current?.easeTo({ center: lngLat, offset: [0, -90], duration: 300 });
    });

    const marker = new MapLibreMarker({ element: el, anchor: "center" })
      .setLngLat(lngLat)
      .addTo(map);
    markersRef.current.set(parish.id, marker);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [mode]);

// The pilgrim's own position gets its own marker, separate from the
// parish pins above, so GPS updates only ever touch this one element
// instead of rebuilding all 31 parish markers (and closing whatever
// popup the pilgrim currently has open) on every tick.
//
// The marker is created once and then moved. An earlier version removed
// and re-added it on every fix, which is what made the dot jump: each
```

```
// update tore the element out of the DOM and dropped a new one at the new
// coordinates, so there was nothing left to animate between.
useEffect(() => {
  const map = mapRef.current;
  if (mode !== "live" || !map) return;

  if (!position) {
    youMarkerRef.current?.remove();
    youMarkerRef.current = null;
    animatedPositionRef.current = null;
    removeAccuracyCircle(map);
    return;
  }

  // The accuracy disc, under the dot.
  //
  // Only drawn when the fix actually reports a radius, and only when
  // that radius is worth drawing: a sub-metre disc is smaller than the
  // dot itself, and the simulator reports no accuracy at all. Inventing
  // a default would be inventing a confidence nobody measured.
  if (accuracyMeters != null && accuracyMeters > 1) {
    upsertAccuracyCircle(map, position, accuracyMeters, YOU_BLUE);
  } else {
    removeAccuracyCircle(map);
  }

  if (!youMarkerRef.current) {
    const you = document.createElement("div");
    you.className = "dmap-live__marker dmap-live__marker--you";
    you.title = "You are here";

    // Which way the pilgrim is facing.
    //
    // This was removed once for being a hard-edged translucent
    // triangle: at a glance it read as the DOT being a triangle
    // pointing somewhere, rather than as a dot with a beam. Removing
    // it went too far - without it there is nothing on the map saying
    // which way you are facing, which is most of the point of a
    // compass. It is back as a soft fan that fades out at its far
    // edge, so the dot still reads as a dot and the light reads as
    // light. See .dmap-live__beam.
    //
    // Hidden until the compass actually produces a bearing. A beam
    // locked at due north on a phone with no magnetometer is worse
    // than no beam: it is confidently wrong.
    const beam = document.createElement("span");
    beam.className = "dmap-live__beam";
    beam.setAttribute("aria-hidden", "true");
    you.appendChild(beam);
    beamRef.current = beam;

    // The walking puck, for turn-by-turn only.
    //
    // A dot says where you are; it cannot say which way you are
    // pointing, and while you are following a route that is the thing
    // you check at every corner. The beam answers it on a north-up
    // map, but the navigation camera turns the map to your heading -
    // and a beam that always points straight up a map that is always
    // turned to your heading is a beam that never moves.
```

```
    //
    // So navigation swaps the dot for an arrow, the way every
    // turn-by-turn app people have already used does it. CSS decides
    // which is visible; both exist from the start so no element has
    // to be built mid-walk.
    const chevron = document.createElement("span");
    chevron.className = "dmap-live__chevron";
    chevron.setAttribute("aria-hidden", "true");
    you.appendChild(chevron);
    youElementRef.current = you;

    youMarkerRef.current = new MapLibreMarker({ element: you, anchor: "center" })
      .setLngLat([position.lng, position.lat])
      .addTo(map);
    animatedPositionRef.current = position;
    return;
  }

  // Ease from wherever the marker currently is to the new fix, rather
  // than teleporting. GPS delivers a fix every second or so and each one
  // carries several metres of noise; without this the dot twitches
  // constantly even when the pilgrim is standing still.
  const marker = youMarkerRef.current;
  const from = animatedPositionRef.current ?? position;
  const to = position;

  if (moveFrameRef.current !== null) cancelAnimationFrame(moveFrameRef.current);

  // A large jump is a genuine relocation (first fix, or the simulator
  // being switched) rather than walking, and easing across a kilometre
  // looks like the map is flying somewhere. Snap those instead.
  if (haversineMeters(from, to) > MARKER_SNAP_METERS) {
    marker.setLngLat([to.lng, to.lat]);
    animatedPositionRef.current = to;
    return;
  }

  const startedAt = performance.now();
  const step = (now: number) => {
    const t = Math.min(1, (now - startedAt) / MARKER_EASE_MS);
    // easeOutQuad — quick off the mark, settling gently, which reads as
    // movement rather than as a slide.
    const eased = 1 - (1 - t) * (1 - t);
    const lat = from.lat + (to.lat - from.lat) * eased;
    const lng = from.lng + (to.lng - from.lng) * eased;
    marker.setLngLat([lng, lat]);
    animatedPositionRef.current = { lat, lng };
    moveFrameRef.current = t < 1 ? requestAnimationFrame(step) : null;
  };
  moveFrameRef.current = requestAnimationFrame(step);

  return () => {
    if (moveFrameRef.current !== null) {
      cancelAnimationFrame(moveFrameRef.current);
      moveFrameRef.current = null;
    }
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [mode, position?.lat, position?.lng, accuracyMeters]);
```

```
// Point the direction light.
//
// Written straight to the element's style rather than through state:
// the orientation sensor fires several times a second, and a render of
// this component per reading would re-run every map effect below it.
useEffect(() => {
  const beam = beamRef.current;
  if (!beam) return;

  if (heading === null) {
    // No compass on this device, or no absolute bearing yet.
    beam.style.opacity = "0";
    return;
  }

  beam.style.opacity = "1";
  // Minus the map's own bearing, so the beam keeps pointing at true
  // north-relative heading even when the map itself has been rotated.
  const mapBearing = mapRef.current?.getBearing() ?? 0;
  beam.style.transform = `translate(-50%, -100%) rotate(${heading - mapBearing}deg)`;
}, [heading]);

// Walk with the walker.
//
// Navigation used to close in once, at the start, and then leave the
// camera where it was: fifty metres later you were off the edge of your
// own route, looking at the corner you set out from. This keeps the
// walker pinned low on the screen with the road ahead above them, and
// turns the map so the direction of travel is up.
//
// Every decision in here is in lib/followCamera.ts, where it can be
// tested. Neither a GPS fix nor a magnetometer reading exists in a
// desktop browser, so maths left in this file could only ever be checked
// by walking outside with a handset.
useEffect(() => {
  const map = mapRef.current;
  if (!map || mode !== "live") return;
  if (!shouldFollow(navigatingTo !== null, userPannedRef.current)) return;
  if (!position) return;

  const current = {
    center: { lat: map.getCenter().lat, lng: map.getCenter().lng },
    bearing: map.getBearing(),
    zoom: map.getZoom(),
  };
  // GPS jitters by metres while standing still and the compass wavers by
  // a degree; without this the camera restarts its ease on every reading
  // and the map shivers in the hand.
  if (!needsCameraMove(current, position, heading)) return;

  // The walker's own zoom is kept. Pinching out to see the rest of the
  // route and being overruled a second later is the same complaint as
  // being dragged back after a pan.
  // Closer as a turn arrives, wider on open road - the behaviour the
  // Waze reference shows. Only after the initial fit, and only while
  // the walker has not taken the zoom over themselves.
  const target = userZoomedRef.current
    ? current.zoom
```

```
    : didZoomForNavRef.current
      ? zoomForTurn(metersToTurnRef.current)
      : FOLLOW_ZOOM;
  // zoomForTurn returns a slightly different number on every fix; acting
  // on each one makes the map breathe in and out while standing still.
  const zoom = needsZoomChange(current.zoom, target) ? target : current.zoom;

  map.easeTo({
    ...followCamera(position, heading, current.bearing, zoom),
    duration: FOLLOW_DURATION_MS,
    // Linear. An ease-in-out that is still accelerating when the next
    // fix arrives reads as the map lurching once a second.
    easing: t => t,
  });
}, [mode, navigatingTo, position?.lat, position?.lng, heading]);

// The dot becomes an arrow while a route is being followed, and the map
// goes back to flat north-up when it ends.
useEffect(() => {
  const you = youElementRef.current;
  if (you) you.dataset.navigating = navigatingTo ? "true" : "false";

  if (navigatingTo) return;
  userPannedRef.current = false;
  userZoomedRef.current = false;
  const map = mapRef.current;
  if (!map) return;
  if (map.getPitch() > 0.5 || Math.abs(map.getBearing()) > 0.5) {
    map.easeTo({ pitch: 0, bearing: 0, duration: 500 });
  }
}, [navigatingTo]);


// `heading` still drives heading-up mode below, which turns the MAP
// rather than drawing an arrow on it.


// Heading-up mode: the map turns so the way the pilgrim faces is up.
//
// MapLibre keeps point labels viewport-aligned by default and flips
// street labels rather than letting them run upside-down, so place names
// stay readable at any bearing without extra work here.
useEffect(() => {
  const map = mapRef.current;
  if (mode !== "live" || !map) return;

  // Not while navigating. The follow camera deliberately turns the map
  // course-up so a route that bends left on screen means a left turn in
  // the street; this effect re-runs on every heading update, so without
  // this guard the two fought each other - followCamera eased toward the
  // walker's bearing and was snapped back to north a frame later, for as
  // long as the walk lasted.
  if (navigatingTo) return;

  if (orientationMode === "north-up") {
    if (Math.abs(map.getBearing()) > 0.5) map.easeTo({ bearing: 0, duration: 400 });
    return;
  }
  if (heading === null) return;
```

```
  // `heading` is already smoothed and deadbanded upstream, so this only
  // fires on real turns. easeTo's own interpolation then covers the gap
  // between updates, which is what keeps the rotation from stepping.
  if (Math.abs(shortestAngleDelta(map.getBearing(), heading)) < MAP_BEARING_EPSILON) return;
  map.easeTo({ bearing: heading, duration: 300, easing: t => t });
}, [mode, heading, orientationMode, navigatingTo]);

// No route is drawn automatically. Directions appear only when the pilgrim
// asks for them from a parish's popup — the same contract as Google Maps or
// Waze, where turning location on shows you where you are and nothing more.
// An earlier version quietly drew a line to every live parish the moment a
// position was known, which read as the app having already decided where you
// were going.

// Dismisses a drawn route. Directions are a thing the pilgrim asked for,
// so there has to be a way to take them back off the map without hunting
// for the popup that produced them.
function clearDirections() {
  const map = mapRef.current;
  directionsRequestRef.current++; // any in-flight response is now stale
  if (map) {
    for (const routeId of Object.keys(routes)) removeRouteLayer(map, routeId);
  }
  setRoutes({});
}

// Honours Home's "Get directions". Waits for `mode === "live"` because a route
// layer cannot be added to a map that has not finished loading, and for a
// position because a route needs somewhere to start.
useEffect(() => {
  if (!walkToParishId || mode !== "live") return;
  const map = mapRef.current;
  const from = positionRef.current;
  const parish = PARISHES.find(p => p.id === walkToParishId);
  if (!map || !parish) return;
  if (!from) {
    // No fix yet — drop the request rather than holding it forever, since
    // the pilgrim is already looking at the map and can ask again.
    onWalkToConsumed?.();
    return;
  }

  let live = true;
  const requestId = ++directionsRequestRef.current;
  void getWalkingDirections(from, parish.coordinates).then(result => {
    if (!live || directionsRequestRef.current !== requestId) return;
    if (result.status === "no-position") return;
    const currentMap = mapRef.current;
    if (currentMap) {
      upsertRouteLayer(currentMap, parish.id, result.route, resolveColor("var(--color-brand-accent)"));
      setRoutes(prev => ({ ...prev, [parish.id]: result.route }));
      const [first, ...rest] = result.route.path;
      const bounds = rest.reduce(
        (b, pt) => b.extend([pt.lng, pt.lat]),
        new LngLatBounds([first.lng, first.lat], [first.lng, first.lat]),
      );
      currentMap.fitBounds(bounds, { padding: 64, duration: 500 });
    }
```

```
    onWalkToConsumed?.();
  });
  return () => {
    live = false;
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [walkToParishId, mode]);

function recentreOnMe() {
  const map = mapRef.current;
  if (!map || !position) return;

  // Mid-route this is how following is resumed after a pan - which is
  // the button's whole job once a walk is under way, and the reason it
  // stays on screen while the instruction card covers everything else.
  if (navigatingTo) {
    userPannedRef.current = false;
    // Recentring is the walker asking for the guided camera back, zoom
    // included, not just the position.
    userZoomedRef.current = false;
    map.easeTo({
      ...followCamera(position, heading, map.getBearing(), Math.max(map.getZoom(), FOLLOW_ZOOM)),
      duration: 600,
    });
    return;
  }

  map.flyTo({ center: [position.lng, position.lat], zoom: Math.max(map.getZoom(), 16.5) });
}

// Selecting a search result flies to the parish and opens its card, so
// search and tapping a pin directly end in exactly the same state —
// including for coming-soon parishes, which never navigate.
function selectSearchResult(parish: SearchParish) {
  const map = mapRef.current;
  const marker = markersRef.current.get(parish.id);
  if (!map || !marker) return;

  setQuery("");

  // While a visit is being planned, searching for a church is how you add
  // it. Opening its place card instead would mean a tap, a card, and a
  // second tap for every church on the list.
  if (tripOpen && parish.coordinates) {
    toggleTripStop({ id: parish.id, name: parish.name, coordinates: parish.coordinates });
    map.flyTo({ center: marker.getLngLat(), zoom: 14, duration: 600 });
    return;
  }

  map.flyTo({
    center: marker.getLngLat(),
    zoom: 16,
    offset: [0, -90],
    duration: 800,
  });
  setSelectedParishId(parish.id);
}

// ---- the Bisita Iglesia planner --------------------------------------
```

```
/** Adds a church to the visit, or takes it out if it is already in. */
function toggleTripStop(parish: { id: string; name: string; coordinates: Coordinates }) {
  setTripStops(prev => {
    if (prev.some(stop => stop.id === parish.id)) {
      return prev.filter(stop => stop.id !== parish.id);
    }
    if (prev.length >= MAX_TRIP_STOPS) return prev;
    return [
      ...prev,
      {
        id: parish.id,
        name: shortLabel(parish.name),
        coordinates: parish.coordinates,
        routeId: LIVE_PARISH_TO_ROUTE_ID[parish.id] ?? null,
      },
    ];
  });
  setTripOpen(true);
}

function clearTrip() {
  setTripStops([]);
  setTrip(null);
}

// Measures the visit whenever the churches or their order change.
//
// Debounced, because adding four churches in four taps would otherwise
// fire four full trips at OSRM's public demo server - the last three of
// which are already obsolete when they are sent. The sequence counter is
// the same guard the single-route request uses: a slow answer to an old
// question must never overwrite a fast answer to the current one.
const tripRequestRef = useRef(0);
useEffect(() => {
  if (tripStops.length === 0) {
    setTrip(null);
    setTripMeasuring(false);
    return;
  }

  const ticket = ++tripRequestRef.current;
  setTripMeasuring(true);
  const timer = window.setTimeout(() => {
    void measureTrip(positionRef.current, tripStops).then(result => {
      if (tripRequestRef.current !== ticket) return;
      setTrip(result);
      setTripMeasuring(false);
    });
  }, 450);

  return () => window.clearTimeout(timer);
  // positionRef is read at call time on purpose: re-measuring the whole
  // visit on every GPS tick would be a request a second.
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [tripStops]);

// Draws the visit: one line through every church, and a lettered marker
// at each one.
```

```
useEffect(() => {
  const map = mapRef.current;
  if (mode !== "live" || !map) return;

  const markers = tripMarkersRef.current;
  const wanted = new Set(tripStops.map(stop => stop.id));
  for (const [id, marker] of markers) {
    if (!wanted.has(id)) {
      marker.remove();
      markers.delete(id);
    }
  }

  tripStops.forEach((stop, index) => {
    const existing = markers.get(stop.id);
    if (existing) {
      // Only the letter can have changed - reordering does not move a
      // church - so the element is relabelled rather than rebuilt.
      const el = existing.getElement();
      if (el.textContent !== stopLabel(index)) el.textContent = stopLabel(index);
      return;
    }
    const el = document.createElement("div");
    el.className = "dmap-live__trip-stop";
    el.textContent = stopLabel(index);
    el.title = stop.name;
    const marker = new MapLibreMarker({ element: el, anchor: "center" })
      .setLngLat([stop.coordinates.lng, stop.coordinates.lat])
      .addTo(map);
    markers.set(stop.id, marker);
  });

  if (trip) {
    upsertTripLayer(map, trip, resolveColor("var(--color-brand-accent)"));
  } else {
    removeTripLayer(map);
  }
}, [tripStops, trip, mode]);

// The markers and the line outlive any single render, so they need taking
// down when the map itself goes - the map teardown effect above owns the
// parish pins, and this owns these.
useEffect(() => {
  const markers = tripMarkersRef.current;
  return () => {
    markers.forEach(marker => marker.remove());
    markers.clear();
  };
}, []);

const tripOptimal = isOptimalOrder(position, tripStops);

// Straight-line distance only, and labelled "direct" so it is never
// mistaken for a walking distance. A real routed figure needs an OSRM
// round-trip per parish, and firing those off just to populate a caption
// is what used to draw an unasked-for line across the map.
const distancePanel = (
  <div className="dmap-live__panel" data-testid="parish-distances">
    {LIVE_PARISHES.map(parish => {
```

```
      const straightLine = position ? haversineMeters(position, parish.coordinates) : null;
      const route = routes[parish.routeId];
      return (
        <div className="dmap-live__panel-row" key={parish.routeId}>
          <span className="dmap-live__panel-name">{shortLabel(parish.name)}</span>
          <span className="dmap-live__panel-distance">
            {route
              ? route.kind === "routed"
                ? `${formatDistance(route.distanceMeters)} walk · ${formatWalkingMinutes(route.durationMinutes)}`
                : `${formatDistance(route.distanceMeters)} direct (no route)`
              : straightLine === null
                ? "Distance unknown"
                : `${formatDistance(straightLine)} direct`}
          </span>
        </div>
      );
    })}
  </div>
);

// With heightPx set, the map surface gets that fixed height directly and
// the wrapper stops relying on flex-1/h-full against an ancestor's fixed
// height — so the legend/link below it get their own natural height
// instead of being squeezed out of a shared budget.
const frameStyle = heightPx != null ? { height: heightPx } : undefined;
const frameClassName = heightPx != null ? "dmap-live" : "dmap-live flex-1 min-h-0";
const wrapClassName = heightPx != null ? "flex flex-col gap-2" : "flex flex-col gap-2 h-full min-h-0";

if (mode === "fallback") {
  return (
    <div className={wrapClassName}>
      <div className={frameClassName} style={frameStyle} data-map-mode="fallback">
        {offlineFlagged && <div className="dmap-live__fallback-badge">Offline map</div>}
        <DioceseMap onSelectParish={onSelectParish} />
        {distancePanel}
      </div>
    </div>
  );
}

return (
  <div className={wrapClassName}>
    <div className={frameClassName} style={frameStyle} data-map-mode={mode} data-navigating={navigatingTo ? "true" : undefined}>
      <div ref={containerRef} className="dmap-live__canvas" role="img" aria-label="Map of the Diocese of Kalookan" />

      {/* Floating pill-shaped search bar, top-left, inset so it never
          overlaps the NavigationControl's zoom buttons (top-right). */}
      <div className="dmap-search">
        <svg className="dmap-search__icon" viewBox="0 0 20 20" aria-hidden="true">
          <circle cx="9" cy="9" r="6" />
          <line x1="13.5" y1="13.5" x2="18" y2="18" />
        </svg>
        <input
          ref={searchInputRef}
          type="search"
          className="dmap-search__input"
          placeholder={tripOpen ? "Search a church to add" : "Search parish or place"}
          aria-label={tripOpen ? "Search a church to add to this visit" : "Search parish or place"}
          value={query}
```

```
    onChange={e => setQuery(e.target.value)}
  />

  {/* The plus the client asked for, on the search bar rather than
      off in a corner: the errand it starts is "another church", and
      searching is how you name one. It carries the count once a
      visit is under way, so the planner is findable again after it
      has been closed. */}
  <button
    type="button"
    className="dmap-search__add"
    onClick={() => setTripOpen(true)}
    aria-label={
      tripStops.length > 0
        ? `Open this visit (${tripStops.length} churches)`
        : "Add a church to visit"
    }
    title="Add church"
  >
    {tripStops.length > 0 ? tripStops.length : <Plus className="w-4 h-4" />}
  </button>
</div>

{/* Hidden while a search is being typed.
    Raising the results above the sheet stops them being covered;
    this stops them being crowded. The sheet is where the churches
    being chosen are listed, and choosing one is exactly when it is
    in the way - so it stands aside for as long as there is a query,
    and comes back the moment one is picked, because selecting a
    result clears the query. */}
{tripOpen && query.trim() === "" && (
  <TripPlanner
    stops={tripStops}
    trip={trip}
    measuring={tripMeasuring}
    optimal={tripOptimal}
    hasPosition={position !== null}
    onAddStop={() => searchInputRef.current?.focus()}
    onRemoveStop={id => setTripStops(prev => prev.filter(stop => stop.id !== id))}
    onOptimise={() => setTripStops(prev => orderStops(positionRef.current, prev))}
    onClear={clearTrip}
    onClose={() => setTripOpen(false)}
  />
)}

{query.trim() !== "" && (
  <ul className="dmap-results">
    {results.length === 0 && <li className="dmap-results__empty">No parish found</li>}
    {results.map(({ parish, distanceMeters }) => (
      <li key={parish.id}>
        <button
          type="button"
          className="dmap-results__item"
          onClick={() => selectSearchResult(parish)}
        >
          <span className="dmap-results__name">{parish.name}</span>
          <span className="dmap-results__where">
            {/* Distance leads when it is known — it is the more
                useful discriminator between two similarly-named
```

```
                parishes, and the vicariate behind it is an inferred
                guess, so it is labelled as one. */}
            {distanceMeters !== null && (
              <span className="dmap-results__distance">{formatDistance(distanceMeters)} away</span>
            )}
            {/* parish.location already carries the caveat where it still
                applies - see vicariateLabel - so appending a second
                one printed it twice on the unconfirmed parishes and
                wrongly on the two that are confirmed. */}
            {parish.location && <span>{parish.location}</span>}
          </span>
        </button>
      </li>
    ))}
  </ul>
)}

{/* Hidden while navigating. The place sheet answers "what is this
    place and how far"; once you are walking, the turn is the only
    question, and the two sat on top of each other at the bottom of
    the screen — the travel-mode row showing through behind the
    navigation card. */}
{selectedParish && !navigatingTo && (
  <MapPlaceSheet
    name={shortLabel(selectedParish.name)}
    location={vicariateLabel(selectedParish.vicariate, selectedParish.id)}
    routeId={LIVE_PARISH_TO_ROUTE_ID[selectedParish.id] ?? null}
    straightLineMetres={position ? haversineMeters(position, selectedParish.coordinates) : null}
    directionsStatus={
      directionsState?.parishId === selectedParish.id ? directionsState.status : ""
    }
    directionsBusy={
      directionsState?.parishId === selectedParish.id && directionsState.busy
    }
    // Routes are keyed by the PARISH id, not the tour route id. Only
    // two parishes have a tour, but all 31 have verified coordinates —
    // keying the drawn route off the tour is what made the other 29
    // impossible to route to.
    //
    // The MEASURED route feeds the distance, so the times are the road
    // ones from the moment the card opens and cannot change when
    // Directions is tapped. Only a DRAWN route flips the button to
    // Start, which is why the two are kept apart.
    routeMetres={
      measuredRoutes[selectedParish.id]?.distanceMeters ??
      routes[selectedParish.id]?.distanceMeters ??
      null
    }
    hasRoute={Boolean(routes[selectedParish.id])}
    measuring={measuringParishId === selectedParish.id}
    onDirections={() =>
      getDirectionsFor(
        selectedParish.id,
        selectedParish.coordinates,
        measuredRoutes[selectedParish.id],
      )
    }
    onStartWalking={() =>
      setNavigatingTo({
```

```
        id: selectedParish.id,
        name: shortLabel(selectedParish.name),
        coordinates: selectedParish.coordinates,
      })
    }
    onOpenParish={() => {
      const routeId = LIVE_PARISH_TO_ROUTE_ID[selectedParish.id];
      if (routeId) onSelectParishRef.current(routeId);
    }}
    onClose={() => setSelectedParishId(null)}
    inTrip={tripStops.some(stop => stop.id === selectedParish.id)}
    onToggleTrip={() =>
      toggleTripStop({
        id: selectedParish.id,
        name: selectedParish.name,
        coordinates: selectedParish.coordinates,
      })
    }
  />
)}

<button
  type="button"
  className="dmap-live__recentre"
  onClick={recentreOnMe}
  disabled={!position}
  title={
    !position
      ? "Your position is unknown — enable GPS or the location simulator"
      : simulation !== "off"
        ? "Recentre the map on your simulated position"
        : positionIsStale
          ? "Recentre on your last known position — GPS has stopped updating"
          : accuracyMeters != null
            ? `Recentre the map on your position (accurate to ±${Math.round(accuracyMeters)} m)`
            : "Recentre the map on your position"
  }
  aria-label="Recentre map on my location"
>
  Recentre
</button>

{/* Back to the plain north-up, flat view.
    Navigation turns and tilts the map on purpose, and MapLibre
    lets a two-finger twist do the same by accident; either way the
    way out should not be "rotate it back by hand until it looks
    straight". Shown only when there is something to undo. */}
{viewRotated && (
  <button
    type="button"
    className="dmap-live__northup"
    onClick={() => {
      const map = mapRef.current;
      if (!map) return;
      // Also stops the follow camera re-tilting a frame later: north-up
      // is a request for the plain map, not for one frame of it.
      userPannedRef.current = true;
      map.easeTo({ bearing: 0, pitch: 0, duration: 450 });
    }}
```

```
    title="Face the map north and remove the tilt"
    aria-label="Reset the map to north-up"
  >
    North up
  </button>
)}

{/* Not while navigating: it sits in the top strip under the turn
    instruction, and Stop already clears the route. Two ways to undo
    the same thing, one of them half-hidden. */}
{Object.keys(routes).length > 0 && !navigatingTo && (
  <button
    type="button"
    className="dmap-live__clear-route"
    onClick={clearDirections}
    aria-label="Clear directions from the map"
  >
    Clear directions
  </button>
)}

{/* Honest status for the you-are-here marker: a simulated position
    is clearly labelled as such (never mistaken for a real fix), and
    a real GPS fix shows its own reported accuracy radius rather than
    asserting a precise dot — wifi-based geolocation on a desktop is
    routinely only accurate to within a few hundred metres, and this
    says so instead of pretending otherwise. */}
{/* Hidden while navigating: it sits in the top strip, which is now
    where the turn instruction lives, and accuracy is a thing you
    check before setting off rather than at every corner. */}
{position && !navigatingTo && (
  <div
    className={simulation !== "off" ? "dmap-live__position-badge dmap-live__position-badge--sim" : "dmap-live__position-badge"}
    data-testid="position-accuracy-badge"
  >
    {simulation !== "off"
      ? "Simulated location"
      : positionIsStale
        ? "Last known position — still looking for GPS"
        : accuracyMeters != null
          ? `Accurate to ±${Math.round(accuracyMeters)} m`
          : "Accuracy unknown"}
  </div>
)}
{distancePanel}

{navigatingTo && (
  <NavigationOverlay
    destination={navigatingTo.coordinates}
    destinationName={navigatingTo.name}
    onStop={() => {
      const map = mapRef.current;
      if (map) removeRouteLayer(map, navigatingTo.id);
      setRoutes(prev => {
        const next = { ...prev };
        delete next[navigatingTo.id];
        return next;
      });
      setNavigatingTo(null);
```

```
            }}
            onProgress={meters => { metersToTurnRef.current = meters; }}
            onRoute={route => {
              // Redraw on every reroute, so the line on the map is always the
              // route currently being followed rather than the original one.
              const map = mapRef.current;
              if (!map) return;
              upsertRouteLayer(map, navigatingTo.id, route, resolveColor("var(--color-brand-accent)"));
              setRoutes(prev => ({ ...prev, [navigatingTo.id]: route }));

              // Close in on the walker when navigation starts.
              //
              // The map was still showing the whole route fitted to the
              // screen, which is the right view for CHOOSING a route and the
              // wrong one for walking it — at that zoom a street corner is a
              // few pixels wide. Only on the first route, so a reroute
              // mid-walk does not yank the camera back.
              if (!didZoomForNavRef.current) {
                didZoomForNavRef.current = true;
                const from = positionRef.current ?? route.path[0];
                if (from) {
                  map.easeTo({
                    center: [from.lng, from.lat],
                    zoom: 17.5,
                    // Leaves room for the instruction card at the top.
                    offset: [0, 60],
                    duration: 900,
                  });
                }
              }
            }}
          />
        )}
      </div>
    </div>
  );
}
```

3.1.2 MAP-002 — NavigationOverlay.tsx

Code ID	MAP-002
File Name	NavigationOverlay.tsx
File Path	SanctDemo/src/components/NavigationOverlay.tsx
Language / Type	TypeScript (React, TSX)
Lines	244
Functions / Classes	NavigationOverlay (exported component/function (default)); BandMessage (internal function); maneuverTransform (internal function)
Related Files (its imports)	../context/PresenceContext, ../lib/geo, ../lib/routing
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/components/NavigationOverlay.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — NavigationOverlay.tsx

```
import { useEffect, useRef, useState, type ReactNode } from "react";
import { X, Navigation2, CornerUpLeft, RotateCw, Flag } from "lucide-react";
import { usePresence } from "../context/PresenceContext";
import type { Coordinates } from "../lib/geo";
import { fetchWalkingRoute, formatDistance, formatWalkingMinutes, WALK_SPEED_MPS, type WalkingRoute } from "../lib/routing";
import {
  announcementFor,
  instructionFor,
  nextOffRouteState,
  progressAlong,
  type NavigationProgress,
  type OffRouteState,
} from "../lib/navigation";

interface NavigationOverlayProps {
  destination: Coordinates;
  destinationName: string;
  onStop: () => void;
  /** Called whenever a route is (re)calculated, so the map can draw it. */
  onRoute?: (route: WalkingRoute) => void;
  /**
   * Called on every fix with how far the next turn is, so the camera can
   * close in on it. Null once there is no live guidance - routing,
   * arrived, or stopped - which the map reads as "no turn to approach"
   * rather than "a turn very far away".
   */
  onProgress?: (metersToManeuver: number | null) => void;
}

type Phase = "routing" | "navigating" | "rerouting" | "arrived" | "no-route";
```

```
/**
 * Turn-by-turn walking navigation.
 *
 * The maths lives in lib/navigation.ts and is tested there; this component
 * owns only what a pure function cannot: the GPS subscription, the reroute
 * request, and what the pilgrim sees.
 *
 * Two honesty constraints carry over from the rest of the app. A 'direct'
 * route — OSRM unreachable, or the request timed out — has no turns, so
 * navigation refuses to start rather than reading out "head north, then
 * arrive" over a straight line drawn through buildings. And walking time is
 * always derived at WALK_SPEED_MPS, never from OSRM's own duration, which
 * assumes a vehicle.
 */
export default function NavigationOverlay({
  destination,
  destinationName,
  onStop,
  onRoute,
  onProgress,
}: NavigationOverlayProps) {
  const { position } = usePresence();

  // Held in a ref so a parent that re-creates this callback inline does
  // not re-run the per-fix effect below on every render.
  const onProgressRef = useRef(onProgress);
  onProgressRef.current = onProgress;
  const [route, setRoute] = useState<WalkingRoute | null>(null);
  const [phase, setPhase] = useState<Phase>("routing");
  const [progress, setProgress] = useState<NavigationProgress | null>(null);
  const [reroutes, setReroutes] = useState(0);

  const offRouteRef = useRef<OffRouteState>({ strikes: 0, offRoute: false });
  const stepIndexRef = useRef(0);
  const requestRef = useRef(0);
  const onRouteRef = useRef(onRoute);
  onRouteRef.current = onRoute;

  // Held in a ref so the routing effect does not re-run on every GPS tick —
  // it needs the position at the moment it fires, not as a dependency.
  const positionRef = useRef<Coordinates | null>(position);
  positionRef.current = position;

  const requestRoute = async (isReroute: boolean) => {
    const from = positionRef.current;
    if (!from) return;

    const id = ++requestRef.current;
    setPhase(isReroute ? "rerouting" : "routing");

    const result = await fetchWalkingRoute(from, destination, { withSteps: true });
    if (requestRef.current !== id) return; // superseded by a newer request

    // A direct route has no turns to give. Navigation cannot honestly run on
    // one, so it says so instead of pretending.
    if (result.kind !== "routed" || result.steps.length === 0) {
      setRoute(result);
      setPhase("no-route");
      return;
```

```
  }

  stepIndexRef.current = 0;
  offRouteRef.current = { strikes: 0, offRoute: false };
  setRoute(result);
  setPhase("navigating");
  if (isReroute) setReroutes(n => n + 1);
  onRouteRef.current?.(result);
};

// First route, once.
useEffect(() => {
  void requestRoute(false);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [destination.lat, destination.lng]);

// Every fix: advance the instruction, and decide whether the route is lost.
useEffect(() => {
  if (!position || !route || phase !== "navigating") return;

  const next = progressAlong(position, route.steps, destination, stepIndexRef.current);
  stepIndexRef.current = next.stepIndex;
  setProgress(next);
  onProgressRef.current?.(next.arrived ? null : next.metersToManeuver);

  if (next.arrived) {
    setPhase("arrived");
    return;
  }

  offRouteRef.current = nextOffRouteState(position, route.path, offRouteRef.current);
  if (offRouteRef.current.offRoute) void requestRoute(true);
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [position?.lat, position?.lng, route, phase]);

const step = route && progress ? route.steps[progress.stepIndex] : null;
const upcoming = route && progress ? route.steps[progress.stepIndex + 1] : null;

return (
  <>
    {/* The instruction goes at the TOP and the summary at the bottom, the
        way every turn-by-turn app does it, and for the same reason: the
        next turn is what you glance at while walking, and a glance goes to
        the top of the screen. Both used to sit in one card at the bottom,
        which also put them on top of the place sheet. z-30 keeps them
        above that sheet whatever else is open. */}
    <div className="absolute inset-x-0 top-0 z-30 p-3">
      {/* Navy, because in this palette navy marks position, route and
          action — and this is all three. */}
      <div className="rounded-[22px] bg-[var(--color-brand-primary)] text-[var(--color-brand-on-accent)] px-5 py-4 shadow-2xl">
        {phase === "routing" && <BandMessage icon={<RotateCw className="w-5 h-5 animate-spin" />} text="Finding a walking route…" />}

        {phase === "rerouting" && (
          <BandMessage icon={<RotateCw className="w-5 h-5 animate-spin" />} text="You've left the route — finding a new one…" />
        )}

        {phase === "no-route" && (
          <BandMessage
            icon={<Navigation2 className="w-5 h-5" />}
```

```
            text="No walking directions available right now. The route line shows the direct line only — follow the map rather than turn-by-turn."
          />
        )}

        {phase === "arrived" && (
          <BandMessage icon={<Flag className="w-5 h-5" />} text={`You have arrived at ${destinationName}.`} />
        )}

        {phase === "navigating" && step && progress && (
          <div className="flex items-start gap-3.5">
            <CornerUpLeft
              className="w-6 h-6 shrink-0 mt-0.5"
              style={{ transform: maneuverTransform(step.maneuver.modifier) }}
              aria-hidden
            />
            <div className="min-w-0">
              <p className="text-[20px] font-bold leading-tight">{instructionFor(step)}</p>
              <p className="mt-1 text-[16px] opacity-85 tabular-nums">
                {announcementFor(step, progress.metersToManeuver)}
              </p>
            </div>
          </div>
        )}
      </div>
    </div>

    {/* Distance, time and the way out — the things you check occasionally
        rather than continuously, so they sit at the bottom under the thumb. */}
    <div className="absolute inset-x-0 bottom-0 z-30 p-3">
      <div className="flex items-center justify-between gap-3 rounded-[22px] bg-[var(--color-brand-card)] border border-[var(--color-brand-border)] shadow-2xl px-5
py-3">
        <div className="min-w-0">
          {progress && phase !== "no-route" ? (
            <p className="text-[16px] font-semibold text-[var(--color-brand-text)] tabular-nums">
              {formatDistance(progress.metersRemaining)} ·{" "}
              {formatWalkingMinutes(progress.metersRemaining / WALK_SPEED_MPS / 60)}
            </p>
          ) : (
            <p className="text-[16px] font-semibold text-[var(--color-brand-text)] truncate">{destinationName}</p>
          )}
          <p className="text-[15px] text-[var(--color-brand-secondary)] truncate">
            {upcoming ? `Then ${instructionFor(upcoming).toLowerCase()}` : `to ${destinationName}`}
            {reroutes > 0 && ` · rerouted ${reroutes}×`}
          </p>
        </div>

        <button
          type="button"
          onClick={onStop}
          className="shrink-0 flex items-center gap-1.5 px-4 py-2.5 rounded-full text-[16px] font-semibold bg-[var(--color-brand-card-sunk)] border border-[var(--
color-brand-border)] text-[var(--color-brand-text)]"
        >
          <X className="w-4 h-4" />
          Stop
        </button>
      </div>
    </div>
  </>
);
```

```
}

function BandMessage({ icon, text }: { icon: ReactNode; text: string }) {
  return (
    <div className="flex items-start gap-3.5">
      <span className="shrink-0 mt-0.5">{icon}</span>
      <p className="text-[16px] leading-snug font-semibold">{text}</p>
    </div>
  );
}

/**
 * Points the single arrow glyph in the direction of the manoeuvre.
 *
 * One icon rotated beats importing eight near-identical ones, and it keeps
 * the arrow's weight and size identical between instructions — a set of
 * separate glyphs visibly jitters as it swaps.
 */
function maneuverTransform(modifier?: string): string {
  switch (modifier) {
    case "right":
      return "scaleX(-1)";
    case "slight right":
      return "scaleX(-1) rotate(25deg)";
    case "sharp right":
      return "scaleX(-1) rotate(-30deg)";
    case "slight left":
      return "rotate(25deg)";
    case "sharp left":
      return "rotate(-30deg)";
    case "uturn":
      return "rotate(90deg)";
    case "straight":
      return "rotate(90deg) scaleX(-1)";
    default:
      return "none";
  }
}
```

3.1.3 MAP-003 — CompassControl.tsx

Code ID	MAP-003
File Name	CompassControl.tsx
File Path	SanctDemo/src/components/CompassControl.tsx
Language / Type	TypeScript (React, TSX)
Lines	180
Functions / Classes	MapOrientationMode (exported type); rosePosition (internal function); CompassControl (exported component/function (default)); unavailableLabel (internal function)
Related Files (its imports)	../lib/heading, ../lib/useDeviceHeading
Where It Is Used	src/components/DioceseMapLive.tsx
How to Recognize It	Open SanctDemo/src/components/CompassControl.tsx. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — CompassControl.tsx

```
import { COMPASS_POINTS_4, COMPASS_POINTS_8, classifyHeading, formatHeading } from "../lib/heading";
import { headingNeedsPermission, type HeadingStatus } from "../lib/useDeviceHeading";

export type MapOrientationMode = "north-up" | "heading-up";

interface CompassControlProps {
  /** Smoothed bearing in degrees, or null when no heading is available. */
  heading: number | null;
  status: HeadingStatus;
  mode: MapOrientationMode;
  onToggleMode: () => void;
  /** iOS only — shown as an enable button while status is "prompt". */
  onRequestPermission: () => void;
}

// The rose is drawn once, upright, and the whole dial is counter-rotated by
// the current heading. That is what makes the letters behave like a real
// compass: turning the phone east swings the dial so E climbs to the top.
//
// Four cardinals, not eight. The type floor is 14px and the dial is 56px
// across, which leaves about 119px of circumference — enough for four labels
// with air around them, and not nearly enough for eight, which collided into
// an unreadable smear. The eight-point rose still drives the readout below
// ("Facing Northeast"), so no direction is lost; only the dial is simplified.
// The client's spec asks for N/E/S/W with the ordinals explicitly optional.
const ROSE_RADIUS = 19;
const DIAL_POINTS = COMPASS_POINTS_4;

function rosePosition(centre: number) {
  // -90 because SVG angles start at 3 o'clock and bearings start at 12.
  const radians = ((centre - 90) * Math.PI) / 180;
```

```
  return {
    x: Math.cos(radians) * ROSE_RADIUS,
    y: Math.sin(radians) * ROSE_RADIUS,
  };
}

/**
 * The map's compass, modelled on Google Maps': a dial that turns with the
 * pilgrim, and a tap target that puts the map back to north.
 *
 * It renders in every sensor state rather than disappearing when there is no
 * heading. A compass that vanishes on a laptop — or on a phone that refused
 * the permission — looks like a broken feature; one that says why it is
 * inert is merely honest about the hardware.
 */
export default function CompassControl({
  heading,
  status,
  mode,
  onToggleMode,
  onRequestPermission,
}: CompassControlProps) {
  if (status === "prompt") {
    // Inside the wrap, like every other state.
    //
    // This branch used to return the bare button, and ALL of the positioning
    // lives on .dmap-compass-wrap (absolute, top 92px, left 8px). Without it
    // the "Enable compass" pill had no position at all and fell to the bottom
    // of the map — nowhere near the Recentre control it belongs beside, and
    // on iOS this is the only state most users ever see, because iOS always
    // starts at "prompt".
    return (
      <div className="dmap-compass-wrap">
        <button type="button" className="dmap-compass dmap-compass--prompt" onClick={onRequestPermission}>
          Enable compass
        </button>
      </div>
    );
  }

  const hasHeading = heading !== null && status === "granted";
  const direction = hasHeading ? classifyHeading(heading, COMPASS_POINTS_8) : null;

  // In heading-up mode the map itself is already rotated, so the dial sits
  // still and the pilgrim's direction is always at the top. In north-up the
  // map is fixed and the dial does the turning.
  const dialRotation = !hasHeading ? 0 : mode === "heading-up" ? 0 : -heading;

  const label = !hasHeading
    ? unavailableLabel(status)
    : `${formatHeading(heading, COMPASS_POINTS_8)} — ${
        mode === "heading-up" ? "map follows your heading" : "map is north-up"
      }. Tap to switch to ${mode === "heading-up" ? "north-up" : "heading-up"}.`;

  return (
    <div className="dmap-compass-wrap">
      <button
        type="button"
        className="dmap-compass"
```

```
  onClick={onToggleMode}
  disabled={!hasHeading}
  title={label}
  aria-label={label}
  data-mode={mode}
>
  <svg viewBox="-28 -28 56 56" aria-hidden="true">
    <circle className="dmap-compass__face" cx="0" cy="0" r="25" />

    <g style={{ transform: `rotate(${dialRotation}deg)` }} className="dmap-compass__dial">
      {DIAL_POINTS.map(point => {
        const { x, y } = rosePosition(point.centre);
        const cardinal = point.abbreviation.length === 1;
        const isNorth = point.centre === 0;
        return (
          <text
            key={point.abbreviation}
            x={x}
            y={y}
            // Counter-rotate each letter by the dial's own rotation so
            // the text stays upright while its position orbits — a
            // rotating "S" that ends up upside-down is unreadable.
            transform={`rotate(${-dialRotation} ${x} ${y})`}
            className={
              isNorth ? "dmap-compass__north" : cardinal ? "dmap-compass__cardinal" : "dmap-compass__ordinal"
            }
            textAnchor="middle"
            dominantBaseline="central"
          >
            {point.abbreviation}
          </text>
        );
      })}

      {/* The north needle, so north is findable at a glance without
          reading the letters. */}
      <polygon className="dmap-compass__needle" points="0,-14 4,-2 -4,-2" />
      <polygon className="dmap-compass__needle-tail" points="0,14 4,2 -4,2" />
    </g>
  </svg>
</button>

{/* The readout is shown in EVERY state, not only when a heading
    exists. It used to put the "why" in a tooltip and aria-label —
    neither of which a phone can display — so on the device the
    compass simply sat there inert with no way to tell whether the
    permission was refused, the sensor was missing, or it was still
    warming up. On a phone the explanation has to be on the glass. */}
<p className="dmap-compass__readout" data-state={hasHeading ? "ok" : "off"}>
  {hasHeading && direction ? (
    <>
      Facing {direction.name}
      <span className="dmap-compass__degrees">{Math.round(heading)}°</span>
    </>
  ) : (
    unavailableLabel(status)
  )}
</p>
```

```
    </div>
  );
}

// Each of these is a genuinely different situation with a different remedy,
// so they get different words rather than one catch-all "compass
// unavailable" that leaves the pilgrim nothing to act on.
function unavailableLabel(status: HeadingStatus): string {
  switch (status) {
    case "insecure":
      return "Compass needs a secure (https) connection";
    case "denied":
      return "Compass permission was denied";
    case "unsupported":
      return "No compass on this device";
    case "unavailable":
      // Distinct from "unsupported": the events were listened for and
      // nothing absolute arrived within three seconds.
      //
      // The remedy is platform-specific, so the message has to be too.
      // Only a device that gates the sensor behind a permission call is an
      // iOS device, and only there is the Settings path the actual fix —
      // handing that instruction to someone on a laptop with no
      // magnetometer sends them hunting for a switch that isn't there.
      return headingNeedsPermission()
        ? "No compass reading — turn on Settings › Safari › Motion & Orientation Access"
        : "No compass on this device";
    default:
      return "Finding north…";
  }
}
```

3.1.4 MAP-004 — followCamera.ts

Code ID	MAP-004
File Name	followCamera.ts
File Path	SanctDemo/src/lib/followCamera.ts
Language / Type	TypeScript
Lines	190
Functions / Classes	FOLLOW_ZOOM (exported constant); FOLLOW_PITCH (exported constant); FOLLOW_OFFSET_Y (exported constant); POSITION_EPSILON_M (exported constant); BEARING_EPSILON_DEG (exported constant); CameraState (exported type); FollowCamera (exported type); followCamera (exported function); needsCameraMove (exported function); FOLLOW_DURATION_MS (exported constant); shouldFollow (exported function); APPROACH_ZOOM (exported constant); CRUISE_ZOOM (exported constant); TURN_NEAR_M (exported constant); TURN_FAR_M (exported constant); zoomForTurn (exported function); ZOOM_EPSILON (exported constant); needsZoomChange (exported function)
Related Files (its imports)	./geo, ./heading
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/followCamera.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — followCamera.ts

```
import { haversineMeters, type Coordinates } from './geo'
import { shortestAngleDelta } from './heading'

/**
 * The camera that walks with you.
 *
 * ## What was missing
 *
 * Navigation closed in on the walker once, at 17.5, and then left the
 * camera where it was. Walk fifty metres and you are off the edge of your
 * own route; the map is still showing the corner you started from. Every
 * turn-by-turn app people have actually used - Waze, Google Maps - keeps
 * the walker pinned to the same spot on the screen and moves the world
 * underneath them instead.
 *
 * ## Why the maths is out here
 *
 * None of it can be exercised in a browser. It is driven by a GPS fix and
 * a magnetometer, neither of which a laptop has, so a camera decision made
 * inside the map component can only be tested by walking outside with a
 * handset. Out here it is four pure functions over numbers.
 *
 * ## The three rules
```

```
 *
 * 1. The walker sits low on the screen, not in the middle, so most of the
 *    map shows where they are GOING. That is the `offset`.
 * 2. The map turns so the direction of travel is up. A route that bends
 *    left on screen then means a left turn in the street, which is the
 *    whole reason course-up exists.
 * 3. Nothing moves unless something has actually changed. GPS jitters by a
 *    few metres while standing still and the compass wavers by a degree or
 *    two; re-issuing an eased camera move on every one of those restarts
 *    the animation and the map shivers.
 */

/** Close enough to read street names and see the next corner. */
export const FOLLOW_ZOOM = 17.5

/**
 * A modest tilt. Enough to give the road ahead some depth without the
 * horizon swallowing the top of the screen - past about 60 the far end of
 * the route stretches to a vanishing point and the labels pile up.
 */
export const FOLLOW_PITCH = 50

/**
 * How far down the screen the walker sits, in pixels below centre.
 *
 * Positive moves the CAMERA's target down, which puts the walker lower in
 * the frame and hands the upper two-thirds to the road ahead.
 */
export const FOLLOW_OFFSET_Y = 90

/** Below this, a new fix is indistinguishable from standing still. */
export const POSITION_EPSILON_M = 3

/** Below this, a bearing change is the compass wavering, not a turn. */
export const BEARING_EPSILON_DEG = 2

export interface CameraState {
  center: Coordinates
  bearing: number
  zoom: number
}

export interface FollowCamera {
  center: [number, number]
  bearing: number
  zoom: number
  pitch: number
  offset: [number, number]
}

/**
 * Where the camera should be for a walker at `position` facing `heading`.
 *
 * `heading` of null means no compass: the map keeps whatever bearing it
 * has rather than snapping to north, because a map that spins to north the
 * moment the magnetometer drops out is worse than one that simply stops
 * turning.
 */
export function followCamera(
```

```
  position: Coordinates,
  heading: number | null,
  currentBearing: number,
  zoom: number = FOLLOW_ZOOM,
): FollowCamera {
  return {
    center: [position.lng, position.lat],
    bearing: heading ?? currentBearing,
    zoom,
    pitch: FOLLOW_PITCH,
    offset: [0, FOLLOW_OFFSET_Y],
  }
}

/**
 * Whether the camera has drifted far enough from the walker to be worth
 * moving.
 *
 * Both thresholds have to be crossed for nothing to happen - a turn on the
 * spot still re-aims the camera even though the walker has not moved, and
 * walking in a straight line still follows them even though the bearing is
 * unchanged.
 */
export function needsCameraMove(
  current: CameraState,
  position: Coordinates,
  heading: number | null,
): boolean {
  if (haversineMeters(current.center, position) >= POSITION_EPSILON_M) return true
  if (heading === null) return false
  return Math.abs(shortestAngleDelta(current.bearing, heading)) >= BEARING_EPSILON_DEG
}

/**
 * How long the camera should take to get there.
 *
 * Matched to the GPS, not chosen for looks. Fixes arrive about once a
 * second, so an ease that outlasts the gap is still running when the next
 * one arrives and gets cancelled mid-flight - which is the stutter people
 * describe as the map "fighting itself". Anything shorter than the gap
 * leaves the camera parked between fixes, so this aims just under it.
 */
export const FOLLOW_DURATION_MS = 800

/**
 * Whether the app should still be driving the camera.
 *
 * Panning the map during navigation is a deliberate act - checking what is
 * down a side street, or how far the destination still is - and a camera
 * that drags you back a second later makes that impossible. So a drag
 * suspends following until the walker asks to be recentred.
 */
export function shouldFollow(navigating: boolean, userPanned: boolean): boolean {
  return navigating && !userPanned
}

/**
 * How close the camera gets as a turn arrives, and how far it pulls back
 * on open road.
```

```
 *
 * A fixed zoom is wrong at both ends of a walk. Held at 17.5 the long
 * straight stretch between two corners shows almost nothing of the road
 * ahead, so the next instruction arrives with no warning; and at the
 * corner itself the two streets you are choosing between are a few pixels
 * apart. Waze and Google Maps both close in on a manoeuvre and open out
 * between them, which is the behaviour being matched here.
 */
export const APPROACH_ZOOM = 18.4

/** Pulled back between turns, to show more of the road ahead. */
export const CRUISE_ZOOM = 16.6

/** At or inside this, the camera is fully closed in on the turn. */
export const TURN_NEAR_M = 45

/** Beyond this, the turn is far enough away to be cruising. */
export const TURN_FAR_M = 260

/**
 * The zoom for a walker `metersToManeuver` from their next turn.
 *
 * Interpolated rather than switched between two values: a step change in
 * zoom reads as the map jumping, and the jump would land exactly when the
 * walker is looking for a street sign. Null - no active guidance, so no
 * turn to approach - keeps the plain follow zoom.
 */
export function zoomForTurn(metersToManeuver: number | null): number {
  if (metersToManeuver === null || !Number.isFinite(metersToManeuver)) return FOLLOW_ZOOM
  if (metersToManeuver <= TURN_NEAR_M) return APPROACH_ZOOM
  if (metersToManeuver >= TURN_FAR_M) return CRUISE_ZOOM

  const t = (metersToManeuver - TURN_NEAR_M) / (TURN_FAR_M - TURN_NEAR_M)
  return APPROACH_ZOOM + (CRUISE_ZOOM - APPROACH_ZOOM) * t
}

/**
 * Whether a zoom change is worth issuing.
 *
 * The same reasoning as needsCameraMove: zoomForTurn returns a slightly
 * different number on every GPS fix, and re-easing the camera for a
 * hundredth of a zoom level restarts the animation and makes the map
 * breathe in and out while the walker stands still.
 */
export const ZOOM_EPSILON = 0.15

export function needsZoomChange(currentZoom: number, targetZoom: number): boolean {
  return Math.abs(currentZoom - targetZoom) >= ZOOM_EPSILON
}
```

3.1.5 MAP-005 — geo.ts

Code ID	MAP-005
File Name	geo.ts
File Path	SanctDemo/src/lib/geo.ts
Language / Type	TypeScript
Lines	42
Functions / Classes	Coordinates (exported type); toRad (internal function); haversineMeters (exported function); nearestLiveParish (exported function)
Related Files (its imports)	./presence
Where It Is Used	src/components/DioceseMapLive.tsx, src/components/HomeHero.tsx, src/components/NavigationOverlay.tsx, src/components/Onboarding.tsx, src/components/ParishCard.tsx, src/components/ParishListRow.tsx, src/components/SanctiHost.tsx, src/components/SearchScreen.tsx, src/context/PresenceContext.tsx, src/lib/followCamera.ts, src/lib/mapSearch.ts, src/lib/navigation.ts, src/lib/pilgrimage.ts, src/lib/presence.ts …
How to Recognize It	Open SanctDemo/src/lib/geo.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — geo.ts

```
import type { GeoParish } from './presence'

export interface Coordinates {
  lat: number
  lng: number
}

const EARTH_RADIUS_M = 6371000

function toRad(deg: number): number {
  return (deg * Math.PI) / 180
}

export function haversineMeters(a: Coordinates, b: Coordinates): number {
  const dLat = toRad(b.lat - a.lat)
  const dLng = toRad(b.lng - a.lng)
  const lat1 = toRad(a.lat)
  const lat2 = toRad(b.lat)

  const h =
    Math.sin(dLat / 2) ** 2 +
    Math.cos(lat1) * Math.cos(lat2) * Math.sin(dLng / 2) ** 2

  return 2 * EARTH_RADIUS_M * Math.asin(Math.sqrt(h))
}
```

```
export function nearestLiveParish(
  pos: Coordinates | null,
  parishes: GeoParish[],
): { parish: GeoParish; distance: number } | null {
  if (!pos) return null

  let best: { parish: GeoParish; distance: number } | null = null
  for (const parish of parishes) {
    if (parish.status !== 'live') continue
    const distance = haversineMeters(pos, parish.coordinates)
    if (!best || distance < best.distance) {
      best = { parish, distance }
    }
  }
  return best
}
```

3.1.6 MAP-006 — travelModes.ts

Code ID	MAP-006
File Name	travelModes.ts
File Path	SanctDemo/src/lib/travelModes.ts
Language / Type	TypeScript
Lines	94
Functions / Classes	TravelModeId (exported type); TravelMode (exported type); TRAVEL_MODES (exported constant); travelMode (exported function); minutesFor (exported function); formatModeDuration (exported function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/src/lib/travelModes.ts. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Travel modes for the place sheet — walking, motorcycle, car.
>
> ALL THREE TIMES COME FROM ONE DISTANCE, and that is a deliberate limitation rather than an oversight. The free OSRM demo server this app routes against serves a single profile and ignores the one it is asked for — probed directly, /foot, /driving, /walking and /cycling for the same pair returned byte-identical results (1102 m, 158 s, 25.1 km/h). There is therefore exactly one route available, and it is a road route.
>
> What follows honestly from that:
>
> - The distance is a ROAD distance. It is right for a car or a motorcycle and slightly long for a walker, who can cut through alleys in Maypajo that no vehicle can use. - The times are estimates from average speeds, not routed durations. OSRM's own duration is never used anywhere in this app; it reports a ~25 km/h vehicle speed no matter which profile is requested.
>
> Both facts are surfaced in the UI rather than hidden, because a Mass time missed on the strength of a confident-looking wrong number is a real cost to a real person.
>
> Jeepney and other public transit are deliberately absent: no free routing service carries Caloocan jeepney data, and a fabricated transit time would be worse than none.

Complete Source Code — travelModes.ts

```
/**
 * Travel modes for the place sheet — walking, motorcycle, car.
 *
 * ALL THREE TIMES COME FROM ONE DISTANCE, and that is a deliberate
 * limitation rather than an oversight. The free OSRM demo server this app
 * routes against serves a single profile and ignores the one it is asked
 * for — probed directly, /foot, /driving, /walking and /cycling for the same
 * pair returned byte-identical results (1102 m, 158 s, 25.1 km/h). There is
```

```
 * therefore exactly one route available, and it is a road route.
 *
 * What follows honestly from that:
 *
 *  - The distance is a ROAD distance. It is right for a car or a motorcycle
 *    and slightly long for a walker, who can cut through alleys in Maypajo
 *    that no vehicle can use.
 *  - The times are estimates from average speeds, not routed durations.
 *    OSRM's own duration is never used anywhere in this app; it reports a
 *    ~25 km/h vehicle speed no matter which profile is requested.
 *
 * Both facts are surfaced in the UI rather than hidden, because a Mass time
 * missed on the strength of a confident-looking wrong number is a real cost
 * to a real person.
 *
 * Jeepney and other public transit are deliberately absent: no free routing
 * service carries Caloocan jeepney data, and a fabricated transit time would
 * be worse than none.
 */

export type TravelModeId = 'walk' | 'motorcycle' | 'car'

export interface TravelMode {
  id: TravelModeId
  label: string
  /** Average door-to-door speed in metres per second. */
  speedMps: number
  /** Shown under the row, so the estimate is never mistaken for a routed time. */
  note: string
}

/**
 * Speeds are door-to-door averages for inner Caloocan, not free-flow speeds.
 *
 * A motorcycle filters through traffic a car sits in, which is why it is
 * quicker here than the car despite the same roads — that is the everyday
 * reality of the route, not a modelling error. The car figure is deliberately
 * low for the same reason: 18 km/h is what C-3 and A. Mabini actually move at
 * for most of the day, and quoting a 40 km/h free-flow speed would make the
 * app confidently early.
 */
export const TRAVEL_MODES: TravelMode[] = [
  {
    id: 'walk',
    label: 'Walk',
    speedMps: 5000 / 3600, // 5 km/h
    note: 'Estimated at a 5 km/h walking pace. The route follows roads, so a shortcut on foot may be shorter.',
  },
  {
    id: 'motorcycle',
    label: 'Motorcycle',
    speedMps: 22000 / 3600, // 22 km/h door to door, filtering through traffic
    note: 'Estimated at 22 km/h — a motorcycle or tricycle filtering through traffic.',
  },
  {
    id: 'car',
    label: 'Car',
    speedMps: 18000 / 3600, // 18 km/h, inner-city Caloocan traffic
    note: 'Estimated at 18 km/h, the everyday pace of inner Caloocan traffic. Expect longer at rush hour.',
```

```
  },
]

export function travelMode(id: TravelModeId): TravelMode {
  // Non-null: the ids are a closed union and every one has an entry above.
  return TRAVEL_MODES.find(m => m.id === id)!
}

/** Minutes for one mode over a given distance. Never below one minute. */
export function minutesFor(distanceMeters: number, mode: TravelMode): number {
  return distanceMeters / mode.speedMps / 60
}

/**
 * "21 min", "1 hr 5 min" — the compact form the mode row needs.
 *
 * Rounded to whole minutes because the underlying figure is an average-speed
 * estimate; a decimal on it would imply a precision that is not there.
 */
export function formatModeDuration(minutes: number): string {
  if (minutes < 1) return '<1 min'
  const whole = Math.round(minutes)
  if (whole < 60) return `${whole} min`
  const hours = Math.floor(whole / 60)
  const rest = whole % 60
  return rest === 0 ? `${hours} hr` : `${hours} hr ${rest} min`
}
```

3.2 Station Placer

3.2.1 MAP-007 — place.html

Code ID	MAP-007
File Name	place.html
File Path	SanctiWalkAR/Tools/viewer/place.html
Language / Type	HTML
Lines	864
Functions / Classes	resize (function); labelSprite (function); rebuildPins (function); loadModels (function); markerItem (function); renderList (function); nextId (function); addStation (function); removeStation (function); rebuildItems (function); renderEditor (function); bindEditor (function); refresh (function); toPointer (function); pick (function); nudge (function); turn (function); walk (function); loop (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/viewer/place.html. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — place.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Station Placer</title>
<style>
  :root {
    --bg:#14171f; --panel:#1b202b; --line:#2b3240; --ink:#eef1f6;
    --muted:#98a1b3; --gold:#d9ad63; --blue:#4ea3ff; --good:#45c07a;
  }
  * { box-sizing:border-box; }
  html,body { margin:0; height:100%; background:var(--bg); color:var(--ink);
    font:14px/1.5 "Segoe UI",system-ui,sans-serif; overflow:hidden; }
  #view { position:fixed; inset:0 360px 0 0; }
  #panel { position:fixed; top:0; right:0; width:360px; height:100%;
    background:var(--panel); border-left:1px solid var(--line);
    padding:16px; overflow-y:auto; }
  h1 { font-size:16px; margin:0 0 2px; }
  .sub { color:var(--muted); font-size:12px; margin-bottom:14px; }
  .item { display:flex; align-items:center; gap:10px; padding:9px 10px;
    border:1px solid var(--line); border-radius:9px; margin-bottom:7px;
    cursor:pointer; background:#222834; }
  .item.on { border-color:var(--blue); background:#1d2a3d; }
```

```
  .dot { width:13px; height:13px; border-radius:50%; flex:0 0 auto; }
  .nm { flex:1; min-width:0; }
  .nm b { display:block; font-size:13px; }
  .nm span { display:block; color:var(--muted); font-size:11px;
    font-variant-numeric:tabular-nums; }
  .grid { display:grid; grid-template-columns:1fr 1fr; gap:6px; margin:10px 0; }
  button { font:inherit; cursor:pointer; border-radius:8px;
    border:1px solid var(--line); background:#232936; color:var(--ink); padding:9px; }
  button.primary { background:var(--gold); color:#14171f; border-color:var(--gold); font-weight:700; }
  button.good { background:var(--good); color:#0d1a12; border-color:var(--good); font-weight:700; }
  .note { color:var(--muted); font-size:12px; margin:12px 0; }
  .note b { color:var(--ink); }
  #live,#status { position:fixed; background:rgba(20,23,31,.9);
    border:1px solid var(--line); border-radius:8px; padding:8px 11px;
    font-variant-numeric:tabular-nums; font-size:12.5px; }
  #live { left:12px; bottom:12px; }
  #status { left:12px; top:12px; }
  hr { border:0; border-top:1px solid var(--line); margin:14px 0; }
  .x { flex:0 0 auto; width:24px; height:24px; padding:0; line-height:1;
       border-radius:6px; color:var(--muted); background:transparent;
       border:1px solid transparent; font-size:16px; }
  .item:hover .x { border-color:var(--line); }
  .x:hover { color:#ff8b8b; border-color:#ff8b8b; }
  .x.armed { color:#0f1116; background:#ff8b8b; border-color:#ff8b8b;
             width:auto; padding:0 8px; font-size:11px; font-weight:700; }
  .field { display:block; margin:8px 0 4px; color:var(--muted); font-size:11px;
           text-transform:uppercase; letter-spacing:.06em; }
  input, select { width:100%; font:inherit; padding:8px 9px; border-radius:8px;
    border:1px solid var(--line); background:#1a1f29; color:var(--ink); }
  input:focus, select:focus { outline:none; border-color:var(--blue); }
  .row2 { display:grid; grid-template-columns:1fr 1fr; gap:6px; }
  #pad { position:fixed; left:12px; bottom:56px; display:grid; gap:5px;
    grid-template-columns:repeat(3,42px); grid-template-rows:repeat(3,38px); }
  #pad button { padding:0; font-size:13px; background:rgba(27,32,43,.92);
    border-color:#39415260; }
  #pad button:active { background:var(--blue); color:#0d1420; }
  #pad .wide { grid-column:span 3; height:30px; font-size:11px; }
  #panel textarea {
    width:100%; box-sizing:border-box; margin-bottom:6px;
    background:#11141b; color:var(--ink); border:1px solid #2a3040;
    border-radius:6px; padding:6px 8px; font:inherit; font-size:12px;
    resize:vertical;
  }
  #panel textarea:focus { outline:1px solid #4a7cff; }
  kbd { background:#262d3b; border:1px solid var(--line); border-bottom-width:2px;
    border-radius:4px; padding:1px 5px; font:inherit; font-size:11px; }
</style>
</head>
<body>
<div id="view"></div>
<div id="status">loading the scan…</div>
<div id="live">hover the scan to read a position</div>

<div id="pad" title="You can also use W A S D, and R / F for up and down">
  <button data-mv="up" title="Up (R)">up</button>
  <button data-mv="fwd" title="Forward (W)">&#9650;</button>
  <button data-mv="dn" title="Down (F)">dn</button>
  <button data-mv="left" title="Left (A)">&#9668;</button>
  <button data-mv="back" title="Back (S)">&#9660;</button>
```

```
  <button data-mv="right" title="Right (D)">&#9658;</button>
  <button class="wide" data-mv="boost" title="Hold with a direction to move faster (Shift)">faster</button>
</div>

<div id="panel">
  <h1>Station Placer</h1>
  <div class="sub">Click a thing below, then click where it really is.</div>
  <div id="list"></div>
  <button id="add" style="width:100%; margin-top:4px">+ Add station</button>

  <hr />
  <div id="editor">
    <label class="field" for="f-name">Station name</label>
    <input id="f-name" placeholder="What the app shows over it" />
    <label class="field" for="f-kind">Kind</label>
    <select id="f-kind">
      <option value="history">History — card, tap to read</option>
      <option value="image">AR image — picture in the room</option>
      <option value="model">3D object — stands in the room</option>
    </select>
    <div class="row2" id="model-row" style="display:none">
      <div>
        <label class="field" for="f-scale">Size &times;</label>
        <input id="f-scale" type="number" step="0.05" min="0.05" />
      </div>
      <div>
        <label class="field" for="f-yaw">Facing (&deg;)</label>
        <input id="f-yaw" type="number" step="15" />
      </div>
    </div>
    <div class="row2">
      <div>
        <label class="field" for="f-side">Side of the aisle</label>
        <select id="f-side">
          <option value="left">left</option>
          <option value="right">right</option>
          <option value="altar">centre / altar</option>
        </select>
      </div>
      <div>
        <label class="field" for="f-radius">Trigger radius (m)</label>
        <input id="f-radius" type="number" step="0.1" min="0.5" />
      </div>
    </div>
    <label class="field" for="f-caption">Caption (one line, shown under the title)</label>
    <textarea id="f-caption" rows="2"
      placeholder="A short line, or leave blank"></textarea>

    <label class="field" for="f-description">Description (the card the visitor reads)</label>
    <textarea id="f-description" rows="6"
      placeholder="The full text, same as a history station"></textarea>

    <div class="field" id="f-id"></div>
  </div>

  <div class="note" id="hint"></div>

  <div class="grid">
    <button id="nudge-xm">&larr; X</button><button id="nudge-xp">X &rarr;</button>
```

```
    <button id="nudge-zm">&darr; Z</button><button id="nudge-zp">&uarr; Z</button>
    <button id="nudge-ym">Lower</button><button id="nudge-yp">Raise</button>
  </div>

  <div class="grid">
    <button id="turn-l">Turn &#8630;</button><button id="turn-r">Turn &#8631;</button>
  </div>

  <hr />
  <button class="good" id="save" style="width:100%">Save placements</button>
  <button id="download" style="width:100%; margin-top:4px">Download a copy</button>
  <div class="note" id="count"></div>
  <div class="note" id="saved"></div>
  <div class="note">
    <b>Getting around.</b> <kbd>W</kbd><kbd>A</kbd><kbd>S</kbd><kbd>D</kbd> walks,
    <kbd>R</kbd>/<kbd>F</kbd> goes up and down, <kbd>Shift</kbd> speeds up.
    Left-drag looks around &middot; wheel zooms.
    Or use the arrows at the bottom left.
  </div>
  <button id="goto" style="width:100%">Fly to the selected station</button>
  <div class="note">
    Left-drag orbits &middot; right-drag pans &middot; wheel zooms.
    Everything is metres in MultiSet map space, exactly as stored.
  </div>
</div>

<script type="importmap">
{ "imports": {
  "three": "./lib/three/three.module.js",
  "three/addons/": "./lib/three/examples/jsm/"
} }
</script>

<script type="module">
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { DRACOLoader } from 'three/addons/loaders/DRACOLoader.js';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

const NUDGE = 0.05;                // metres per nudge tap
const TURN  = 5;                   // degrees per turn tap
const $ = id => document.getElementById(id);

// ---------------------------------------------------------------- 3D setup
const view = $('view');
const renderer = new THREE.WebGLRenderer({ antialias:true });
renderer.setPixelRatio(devicePixelRatio);
view.appendChild(renderer.domElement);

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x14171f);
const camera = new THREE.PerspectiveCamera(55, 1, 0.05, 500);
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;

scene.add(new THREE.AmbientLight(0xffffff, 2.4));
const key = new THREE.DirectionalLight(0xffffff, 1.2);
key.position.set(3, 6, 4);
scene.add(key);
```

```
function resize() {
  const w = view.clientWidth, h = view.clientHeight;
  renderer.setSize(w, h);
  camera.aspect = w / h;
  camera.updateProjectionMatrix();
}
addEventListener('resize', resize);

// ------------------------------------------------------------ the objects
//
// One entry per thing that can be placed. The marker is in this list too,
// deliberately: it is the thing every station is measured from, so being
// able to move it here is the whole point of the page.
let data = null;          // the placed.json contents
let items = [];           // { id, name, colour, get/set position, isMarker }
let selected = 0;
let mesh = null;

const pins = new THREE.Group();
scene.add(pins);

function labelSprite(text, colour) {
  const c = document.createElement('canvas');
  const ctx = c.getContext('2d');
  ctx.font = 'bold 44px Segoe UI, system-ui, sans-serif';
  c.width = ctx.measureText(text).width + 36;
  c.height = 66;
  const g = c.getContext('2d');
  g.font = 'bold 44px Segoe UI, system-ui, sans-serif';
  g.fillStyle = 'rgba(12,15,22,.86)';
  g.roundRect(0, 0, c.width, c.height, 14); g.fill();
  g.fillStyle = colour; g.textBaseline = 'middle';
  g.fillText(text, 18, c.height / 2 + 2);
  const s = new THREE.Sprite(new THREE.SpriteMaterial({
    map: new THREE.CanvasTexture(c), depthTest:false, transparent:true }));
  s.scale.set(c.width / 150, c.height / 150, 1);
  return s;
}

function rebuildPins() {
  pins.clear();
  items.forEach((it, i) => {
    const on = i === selected;
    const group = new THREE.Group();
    group.position.copy(it.get());

    if (it.isMarker) {
      // The printed sheet at its real size, plus an arrow out of the wall
      // so the facing is something you can see rather than a number.
      const sheet = new THREE.Mesh(
        new THREE.PlaneGeometry(0.18, 0.24),
        new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0xd9ad63,
          side: THREE.DoubleSide, transparent:true, opacity:0.95 }));
      const n = it.normal();
      sheet.lookAt(n.clone().add(new THREE.Vector3(0,0,0)));
      sheet.quaternion.setFromUnitVectors(new THREE.Vector3(0,0,1), n);
      group.add(sheet);
```

```
  const dir = new THREE.ArrowHelper(n, new THREE.Vector3(0,0,0), 0.6,
    on ? 0x4ea3ff : 0xd9ad63, 0.14, 0.08);
  group.add(dir);
} else if (it.kind === 'model') {
  // The real object, at its real size, standing where it would stand.
  // A ball or a panel would tell you nothing about whether a life-size
  // statue actually FITS where you just clicked, which is the only
  // question worth asking about a 3D station.
  const loaded = models[it.id];
  if (loaded) {
    const inst = loaded.clone(true);
    inst.rotation.y = (it.yaw || 0) * Math.PI / 180;
    const sc = it.scale || 1;
    inst.scale.setScalar(sc);
    // Standing ON the point, not centred on it: a station marks a spot
    // on the floor and the statue's feet belong there.
    inst.position.y = 0;
    group.add(inst);
    if (on) {
      const box = new THREE.Box3().setFromObject(inst);
      group.add(new THREE.Box3Helper(box, 0x4ea3ff));
    }
  } else {
    const ph = new THREE.Mesh(
      new THREE.CylinderGeometry(0.25, 0.25, 1.7, 12),
      new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0x9a6bd4,
        transparent:true, opacity:0.45 }));
    ph.position.y = 0.85;
    group.add(ph);
  }
} else if (it.kind === 'image') {
  // An AR image is a picture on the wall, so it is drawn as one: a
  // panel at roughly the size it will hang at. A history and an image
  // that both look like a green ball are impossible to tell apart on
  // the scan, and they are the two halves of every group.
  const panel = new THREE.Mesh(
    new THREE.PlaneGeometry(0.75, 0.55),
    new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0xd9ad63,
      side: THREE.DoubleSide, transparent:true, opacity:0.92 }));
  group.add(panel);
  // Turned to face the aisle, so a left-hand and a right-hand image
  // read as facing each other rather than both facing the same way.
  panel.rotation.y = it.side === 'right' ? -Math.PI / 2 : Math.PI / 2;
  const floorY = data.floorY ?? -1.587;
  const h = Math.max(0.01, it.get().y - floorY);
  const stem = new THREE.Mesh(
    new THREE.CylinderGeometry(0.008, 0.008, h, 8),
    new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0x8a6d34 }));
  stem.position.y = -h / 2;
  group.add(stem);
} else {
  const ball = new THREE.Mesh(
    new THREE.SphereGeometry(on ? 0.13 : 0.09, 20, 14),
    new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0x45c07a }));
  group.add(ball);
  // A stem to the floor, so depth is readable from any angle - a ball
  // floating in a photogrammetry mesh is impossible to judge otherwise.
  const floorY = data.floorY ?? -1.587;
  const h = Math.max(0.01, it.get().y - floorY);
```

```
      const stem = new THREE.Mesh(
        new THREE.CylinderGeometry(0.008, 0.008, h, 8),
        new THREE.MeshBasicMaterial({ color: on ? 0x4ea3ff : 0x2f7f54 }));
      stem.position.y = -h / 2;
      group.add(stem);
    }

    const tag = labelSprite(it.name, on ? '#9ccdff' : '#e6ecf5');
    tag.position.y = 0.3;
    group.add(tag);
    pins.add(group);
  });
}

// Loaded GLBs, by station id. Kept out of rebuildPins because that runs on
// every nudge and a 26 MB statue cannot be re-fetched sixty times a second.
const models = {};

function loadModels() {
  for (const st of data.stations) {
    if (st.kind !== 'model' || !st.model || models[st.id] === undefined) {
      if (st.kind !== 'model' || !st.model) continue;
    }
    if (models[st.id]) continue;
    models[st.id] = null;
    $('status').textContent = `loading ${st.model}…`;
    loader.load(`./models/${st.model}.glb`, g => {
      models[st.id] = g.scene;
      // Drop it so its lowest point sits at y=0, then the pin's own
      // position is where the feet go. Scans arrive centred on their own
      // bounding box as often as not, which buries half the statue.
      const box = new THREE.Box3().setFromObject(g.scene);
      g.scene.position.y -= box.min.y;
      const size = box.getSize(new THREE.Vector3());
      console.log(`${st.model}: ${size.x.toFixed(2)} x ${size.y.toFixed(2)} x ${size.z.toFixed(2)} m`);
      $('status').textContent = `${meshFile} — ${st.model} loaded (${size.y.toFixed(2)} m tall)`;
      rebuildPins();
    }, undefined, err => {
      $('status').textContent = `could not load models/${st.model}.glb`;
      console.warn(err);
    });
  }
}

const KIND_LABEL = { history: 'history', image: 'AR image', model: '3D object' };

function markerItem() {
  return {
    id: 'trigger', name: 'Printed marker', isMarker: true,
    get: () => new THREE.Vector3(...data.trigger.map),
    set: v => {
      data.trigger.map = [r(v.x), r(v.y), r(v.z)];
      data.trigger.heightAboveFloor = r(v.y - (data.floorY ?? -1.587));
    },
    normal: () => new THREE.Vector3(...data.trigger.normal).normalize(),
    setNormal: n => {
      data.trigger.normal = [r4(n.x), 0, r4(n.z)];
      data.trigger.facingDeg = r((Math.atan2(n.x, n.z) * 180 / Math.PI + 360) % 360);
    },
```

```
  };
}

// ------------------------------------------------------------------ the UI
function renderList() {
  $('list').innerHTML = '';
  items.forEach((it, i) => {
    const p = it.get();
    const row = document.createElement('div');
    row.className = 'item' + (i === selected ? ' on' : '');
    row.innerHTML =
      `<span class="dot" style="background:${i===selected?'#4ea3ff':(it.isMarker?'#d9ad63':'#45c07a')}"></span>` +
      `<span class="nm"><b>${it.name}</b>` +
      (it.side ? `<span>${KIND_LABEL[it.kind] || 'history'} · ${it.side}</span>` : '') +
      `<span>${p.x.toFixed(3)}, ${p.y.toFixed(3)}, ${p.z.toFixed(3)}</span></span>`;
    row.onclick = () => { selected = i; armed = null; refresh(); };

    if (!it.isMarker) {
      const x = document.createElement('button');
      x.className = 'x' + (armed === it.id ? ' armed' : '');
      x.textContent = armed === it.id ? 'delete?' : '\u00d7';
      x.title = 'Remove this station';
      // Two taps, never one. A single-click delete in a list you are also
      // clicking to SELECT is a station gone by accident, and the only way
      // back is the last save.
      x.onclick = ev => {
        ev.stopPropagation();
        if (armed !== it.id) { armed = it.id; refresh(); return; }
        armed = null;
        removeStation(it.id);
      };
      row.appendChild(x);
    }
    $('list').appendChild(row);
  });
}

// Which row is asking to be confirmed. One at a time, cleared by any other
// interaction, so an armed x cannot sit forgotten and fire on a later click.
let armed = null;

function nextId() {
  const prefix = site || 'station';
  let n = data.stations.length + 1;
  const taken = new Set(data.stations.map(s => s.id));
  let id;
  do { id = `${prefix}-${String(n).padStart(2, '0')}`; n++; } while (taken.has(id));
  return id;
}

function addStation() {
  // Born at whatever the camera is looking at, which is the one place on
  // the scan you can be sure is on screen. Dropping it at the origin puts
  // new stations behind you, off the mesh, or inside a wall.
  const at = controls.target.clone();
  const id = nextId();
  data.stations.push({
    id, name: 'New station', kind: 'history', side: 'left',
    map: [r(at.x), r(at.y), r(at.z)], radius: 2.0,
```

```
  });
  rebuildItems();
  selected = items.length - 1;      // select it, so the next click places it
  armed = null;
  refresh();
  $('f-name').focus();
  $('f-name').select();
}

function removeStation(id) {
  const at = data.stations.findIndex(s => s.id === id);
  if (at < 0) return;
  data.stations.splice(at, 1);
  rebuildItems();
  // Keep the selection on something that still exists, and never on a
  // hole: after deleting the last row, select the one now last.
  selected = Math.min(selected, items.length - 1);
  refresh();
}

// The items list is a view over data.stations, so it is rebuilt from
// scratch whenever that array changes length. The closures capture an
// INDEX, and after a splice every index past the hole means a different
// station - stale ones would quietly edit the wrong row.
function rebuildItems() {
  items = [markerItem()];
  data.stations.forEach((s, i) => items.push({
    id: s.id, name: s.name, side: s.side, kind: s.kind, isMarker: false,
    model: s.model, scale: s.modelScale, yaw: s.yawDegrees,
    get: () => new THREE.Vector3(...data.stations[i].map),
    set: v => { data.stations[i].map = [r(v.x), r(v.y), r(v.z)]; },
  }));
}

function renderEditor() {
  const it = items[selected];
  const isStation = it && !it.isMarker;
  $('editor').style.display = isStation ? '' : 'none';
  if (!isStation) return;

  const s = data.stations.find(x => x.id === it.id);
  $('f-name').value = s.name || '';
  $('f-kind').value = s.kind || 'history';
  $('model-row').style.display = s.kind === 'model' ? '' : 'none';
  if (s.kind === 'model') {
    $('f-scale').value = s.modelScale ?? 1;
    $('f-yaw').value = s.yawDegrees ?? 0;
  }
  $('f-caption').value = s.caption || '';
  $('f-description').value = s.description || '';
  $('f-side').value = s.side || 'left';
  $('f-radius').value = s.radius ?? 2.0;
  $('f-id').textContent = 'id ' + s.id;
}

function bindEditor() {
  const selectedStation = () => {
    const it = items[selected];
    return it && !it.isMarker ? data.stations.find(x => x.id === it.id) : null;
```

```
  };
  // 'input', not 'change': the name is also the pin's label in the 3D view,
  // so it should follow what is being typed rather than wait for a blur.
  $('f-name').oninput = () => {
    const s = selectedStation(); if (!s) return;
    s.name = $('f-name').value;
    items[selected].name = s.name;
    renderList(); rebuildPins();
  };
  $('f-kind').onchange = () => {
    const s = selectedStation(); if (!s) return;
    s.kind = $('f-kind').value;
    items[selected].kind = s.kind;
    renderList(); rebuildPins();
  };
  $('f-side').onchange = () => {
    const s = selectedStation(); if (!s) return;
    s.side = $('f-side').value;
    items[selected].side = s.side;
    renderList(); rebuildPins();
  };
  $('f-radius').onchange = () => {
    const s = selectedStation(); if (!s) return;
    const v = parseFloat($('f-radius').value);
    if (isFinite(v) && v > 0) s.radius = Math.round(v * 10) / 10;
    $('f-radius').value = s.radius;
  };
  $('f-scale').onchange = () => {
    const s = selectedStation(); if (!s) return;
    const v = parseFloat($('f-scale').value);
    if (isFinite(v) && v > 0) s.modelScale = Math.round(v * 100) / 100;
    $('f-scale').value = s.modelScale;
    items[selected].scale = s.modelScale;
    rebuildPins();
  };
  $('f-yaw').onchange = () => {
    const s = selectedStation(); if (!s) return;
    const v = parseFloat($('f-yaw').value);
    s.yawDegrees = isFinite(v) ? Math.round(v) : 0;
    $('f-yaw').value = s.yawDegrees;
    items[selected].yaw = s.yawDegrees;
    rebuildPins();
  };
  // 'input', not 'change': long text is easy to lose to a stray click if it
  // is only committed on blur.
  $('f-caption').oninput = () => {
    const s = selectedStation(); if (!s) return;
    s.caption = $('f-caption').value;
  };
  $('f-description').oninput = () => {
    const s = selectedStation(); if (!s) return;
    s.description = $('f-description').value;
    renderList();
  };
  $('add').onclick = addStation;
}

function refresh() {
  renderEditor();
```

```
  $('count').textContent = `${data.stations.length} station(s) in this plan.`;
  renderList();
  rebuildPins();
  const it = items[selected];
  const above = it.isMarker
    ? (items[0].get().y - (data.floorY ?? -1.587)).toFixed(2) : null;
  $('hint').innerHTML = it.isMarker
    ? `<b>${it.name}</b> — <b>${above} m</b> above the floor. ` +
      `Click the wall where the real printed sheet hangs. ` +
      `Then use <b>Turn</b> until the arrow points into the room, the way the sheet faces.`
    : `<b>${it.name}</b> selected. Click the scan where the real object is. ` +
      `Fine-tune with the buttons below.`;
}

// ------------------------------------------------------------- picking
const ray = new THREE.Raycaster();
const pointer = new THREE.Vector2();
let hovered = null;

function toPointer(e) {
  const r = renderer.domElement.getBoundingClientRect();
  pointer.x = ((e.clientX - r.left) / r.width) * 2 - 1;
  pointer.y = -((e.clientY - r.top) / r.height) * 2 + 1;
}

function pick() {
  if (!mesh) return null;
  ray.setFromCamera(pointer, camera);
  const hit = ray.intersectObject(mesh, true)[0];
  return hit ? hit.point.clone() : null;
}

renderer.domElement.addEventListener('pointermove', e => {
  toPointer(e);
  hovered = pick();
  $('live').textContent = hovered
    ? `x ${hovered.x.toFixed(3)}   y ${hovered.y.toFixed(3)}   z ${hovered.z.toFixed(3)}`
    : 'hover the scan to read a position';
});

// A click that follows a drag is an orbit, not a placement. Without this
// every camera move would fling the selected object across the room.
let downAt = null;
renderer.domElement.addEventListener('pointerdown', e => downAt = { x:e.clientX, y:e.clientY });
renderer.domElement.addEventListener('pointerup', e => {
  if (!downAt) return;
  const moved = Math.hypot(e.clientX - downAt.x, e.clientY - downAt.y);
  downAt = null;
  if (moved > 5) return;

  toPointer(e);
  const p = pick();
  if (!p) return;
  items[selected].set(p);
  refresh();
});

// ------------------------------------------------------------- nudging
function nudge(dx, dy, dz) {
```

```
  const it = items[selected];
  const p = it.get();
  it.set(new THREE.Vector3(p.x + dx, p.y + dy, p.z + dz));
  refresh();
}
$('nudge-xm').onclick = () => nudge(-NUDGE,0,0);
$('nudge-xp').onclick = () => nudge( NUDGE,0,0);
$('nudge-zm').onclick = () => nudge(0,0,-NUDGE);
$('nudge-zp').onclick = () => nudge(0,0, NUDGE);
$('nudge-ym').onclick = () => nudge(0,-NUDGE,0);
$('nudge-yp').onclick = () => nudge(0, NUDGE,0);

function turn(deg) {
  const it = items[selected];
  if (it.kind === 'model') {
    const s = data.stations.find(x => x.id === it.id);
    s.yawDegrees = ((s.yawDegrees || 0) + deg) % 360;
    it.yaw = s.yawDegrees;
    $('f-yaw').value = s.yawDegrees;
    rebuildPins();
    return;
  }
  if (!it.isMarker) return;
  const n = it.normal();
  const a = Math.atan2(n.x, n.z) + deg * Math.PI / 180;
  it.setNormal(new THREE.Vector3(Math.sin(a), 0, Math.cos(a)).normalize());
  refresh();
}
$('turn-l').onclick = () => turn(-TURN);
$('turn-r').onclick = () => turn( TURN);

// ------------------------------------------------------------------ saving
$('save').onclick = async () => {
  // Written to out/, which is the only place the little server accepts
  // writes to. The import step on the Unity side reads it from there.
  const body = JSON.stringify(data, null, 2);
  try {
    const res = await fetch(saveFile, { method:'PUT', body });
    $('saved').textContent = res.ok
      ? `Saved to Tools/viewer${saveFile.slice(1)} — tell Claude and it goes into the app.`
      : `Could not save: HTTP ${res.status}`;
  } catch (err) {
    $('saved').textContent = 'Could not save: ' + err.message;
  }
};

// Save writes through the local server; download does not need it, so the
// plan can still be carried away if that server is not running.
$('download').onclick = () => {
  const name = (saveFile.split('/').pop()) || 'placements.json';
  const blob = new Blob([JSON.stringify(data, null, 2)], { type: 'application/json' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url; a.download = name;
  document.body.appendChild(a); a.click(); a.remove();
  URL.revokeObjectURL(url);
  $('saved').textContent = 'Downloaded ' + name + ' to your Downloads folder.';
};
```

```
// ---------------------------------------------------------------- load
const params = new URLSearchParams(location.search);

// Which plan is being placed.
//
// No ?site= is the sala itself, reading placed.json exactly as before -
// those are the four positions that took a week to get right, and nothing
// here is allowed to alter how they load or where they save.
//
// ?site=mhcp reads sites/mhcp.json and saves to out/mhcp.json: a parish
// laid out on whatever scan that plan names. Today that is the sala scan,
// so the walk can be rehearsed before the church is ever scanned. When the
// church scan arrives, only the coordinates change - the list, the names,
// the sides and the order are already decided by then.
const site = params.get('site');
const planFile = site ? `./sites/${site}.json` : './placed.json';
const saveFile = site ? `./out/${site}.json`   : './out/placed.json';

const loader = new GLTFLoader();
const draco = new DRACOLoader();
draco.setDecoderPath('./lib/three/examples/jsm/libs/draco/gltf/');
loader.setDRACOLoader(draco);

let meshFile = 'sala-textured.glb';
let loadedFrom = '';

async function start() {
  // Your saved work wins over the generated plan.
  //
  // Saving writes out/<site>.json; the generator writes sites/<site>.json.
  // Reading only the generated one meant every reload silently threw away
  // everything that had been placed - the file was still on disk, the page
  // just never looked at it.
  //
  // Cache-busted because both files are rewritten between reloads, and a
  // cached copy looks exactly like the save having failed.
  let res = await fetch(`${saveFile}?t=${Date.now()}`);
  let from = saveFile;
  if (!res.ok) { res = await fetch(`${planFile}?t=${Date.now()}`); from = planFile; }
  if (!res.ok) {
    $('status').textContent = `No plan at ${planFile} (HTTP ${res.status}).`;
    return;
  }
  data = await res.json();
  loadedFrom = from;

  // The plan names its own scan, so a plan and a mesh cannot drift apart by
  // someone opening the page with a stale bookmark. ?mesh= still wins, for
  // trying a re-scan against an existing plan.
  meshFile = params.get('mesh') || data.scan || meshFile;
  if (data.title) {
    document.querySelector('h1').textContent = data.title;
    document.title = data.title + ' - Station Placer';
  }

  rebuildItems();
  bindEditor();
  loadModels();
```

```
  loader.load(meshFile, gltf => {
    mesh = gltf.scene;
    scene.add(mesh);

    const box = new THREE.Box3().setFromObject(mesh);
    const centre = box.getCenter(new THREE.Vector3());
    const size = box.getSize(new THREE.Vector3()).length();
    controls.target.copy(centre);
    camera.position.copy(centre).add(new THREE.Vector3(size * 0.4, size * 0.3, size * 0.4));
    camera.near = size / 400; camera.far = size * 6;
    camera.updateProjectionMatrix();

    // A handle for measuring the scan from the console. The mesh arrives
    // Draco-compressed and nothing outside a browser in this project can
    // decode it, so this page is also the only place its floor height and
    // extents can be read.
    window.__placer = { mesh, data, controls, camera, THREE };

    const which = loadedFrom.startsWith('./out/')
      ? 'your saved placements'
      : 'the generated plan (nothing saved yet)';
    $('status').textContent = `${meshFile} — ${which}`;
    resize();
    refresh();
  }, undefined, err => {
    $('status').textContent = 'Could not load ' + meshFile + ': ' + err;
  });
}

const r  = n => Math.round(n * 1000) / 1000;
const r4 = n => Math.round(n * 10000) / 10000;

// ------------------------------------------------------------ walking
//
// OrbitControls alone is unusable in a 37 m nave: it circles one point, so
// getting from the door to the sanctuary means zooming out until the church
// is a smudge and back in somewhere else. These move the camera AND its
// target together, which is what makes it feel like walking rather than
// orbiting - the view keeps its heading and simply goes where you send it.
const held = new Set();
const SPEED = 6.0;            // metres per second; Shift trebles it
let lastFrame = performance.now();

const KEYS = {
  KeyW:'fwd', KeyS:'back', KeyA:'left', KeyD:'right',
  ArrowUp:'fwd', ArrowDown:'back', ArrowLeft:'left', ArrowRight:'right',
  KeyR:'up', KeyF:'dn', KeyQ:'dn', KeyE:'up', Space:'up',
};

addEventListener('keydown', e => {
  // Never while a name is being typed, or walking the church renames a
  // station to "wwwdddsss".
  const el = document.activeElement;
  if (el && (el.tagName === 'INPUT' || el.tagName === 'SELECT')) return;
  const dir = KEYS[e.code];
  if (!dir) return;
  held.add(dir);
  e.preventDefault();
});
```

```
addEventListener('keyup', e => { const d = KEYS[e.code]; if (d) held.delete(d); });
// Losing the window with a key down would otherwise leave it held forever.
addEventListener('blur', () => held.clear());

for (const b of document.querySelectorAll('#pad button')) {
  const dir = b.dataset.mv;
  if (!dir) continue;
  const on = e => { e.preventDefault(); held.add(dir); };
  const off = () => held.delete(dir);
  b.addEventListener('pointerdown', on);
  b.addEventListener('pointerup', off);
  b.addEventListener('pointerleave', off);
}

function walk(dt) {
  if (!held.size) return;
  const step = SPEED * dt * (held.has('boost') ? 3 : 1);
  const fwd = new THREE.Vector3();
  camera.getWorldDirection(fwd);
  const flat = new THREE.Vector3(fwd.x, 0, fwd.z);
  // Looking straight down leaves nothing to walk along, so fall back to
  // the world axis rather than freezing.
  if (flat.lengthSq() < 1e-6) flat.set(0, 0, 1);
  flat.normalize();
  const right = new THREE.Vector3().crossVectors(flat, new THREE.Vector3(0,1,0)).normalize();

  const d = new THREE.Vector3();
  if (held.has('fwd'))   d.add(flat);
  if (held.has('back'))  d.sub(flat);
  if (held.has('right')) d.add(right);
  if (held.has('left'))  d.sub(right);
  if (held.has('up'))    d.y += 1;
  if (held.has('dn'))    d.y -= 1;
  if (!d.lengthSq()) return;

  d.normalize().multiplyScalar(step);
  camera.position.add(d);
  controls.target.add(d);
}

addEventListener('keydown', e => { if (e.shiftKey) held.add('boost'); });
addEventListener('keyup',   e => { if (!e.shiftKey) held.delete('boost'); });

// Standing in front of the selected station, at head height, looking at it.
// Hunting for station 12 in a long nave by dragging is the other half of
// the problem this page had.
$('goto').onclick = () => {
  const it = items[selected];
  if (!it) return;
  const at = it.get();
  const floorY = data.floorY ?? -1.587;
  const back = new THREE.Vector3(0, 0, 1).multiplyScalar(2.5);
  camera.position.set(at.x + back.x, floorY + 1.55, at.z + back.z);
  controls.target.copy(at);
  controls.update();
};

function loop() {
  const now = performance.now();
```

```
  const dt = Math.min(0.1, (now - lastFrame) / 1000);
  lastFrame = now;
  requestAnimationFrame(loop);
  walk(dt);
  controls.update();
  renderer.render(scene, camera);
}

start();
resize();
loop();
</script>
</body>
</html>
```

3.2.2 MAP-008 — index.html

Code ID	MAP-008
File Name	index.html
File Path	SanctiWalkAR/Tools/viewer/index.html
Language / Type	HTML
Lines	383
Functions / Classes	labelSprite (function); hit (function); record (function); draw (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/viewer/index.html. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>MapSpace Point Picker</title>
<style>
  :root {
    --bg: #14171f; --panel: #1c212c; --line: #2b3240;
    --ink: #eef1f6; --muted: #98a1b3; --accent: #d9ad63; --pick: #4ea3ff;
  }
  * { box-sizing: border-box; }
  html, body { margin: 0; height: 100%; background: var(--bg); color: var(--ink);
    font: 14px/1.5 "Segoe UI", system-ui, sans-serif; overflow: hidden; }
  #view { position: fixed; inset: 0; }
  #panel { position: fixed; top: 0; right: 0; width: 330px; height: 100%;
    background: var(--panel); border-left: 1px solid var(--line);
    padding: 14px; overflow-y: auto; }
  h1 { font-size: 15px; margin: 0 0 2px; letter-spacing: .01em; }
  .sub { color: var(--muted); font-size: 12px; margin-bottom: 12px; }
  .row { display: flex; gap: 6px; margin-bottom: 8px; }
  input, button, select { font: inherit; border-radius: 7px; border: 1px solid var(--line);
    background: #232936; color: var(--ink); padding: 7px 9px; }
  input { flex: 1; min-width: 0; }
  button { cursor: pointer; }
  button.primary { background: var(--accent); color: #14171f; border-color: var(--accent); font-weight: 700; }
  #hint { color: var(--muted); font-size: 12px; margin: 10px 0; }
  table { width: 100%; border-collapse: collapse; font-size: 12px; }
  th, td { text-align: left; padding: 5px 4px; border-bottom: 1px solid var(--line);
    font-variant-numeric: tabular-nums; }
  th { color: var(--muted); font-weight: 600; }
  td.num { text-align: right; }
  .del { color: #ff8b7a; cursor: pointer; }
```

```
  #live { position: fixed; left: 12px; bottom: 12px; background: rgba(20,23,31,.88);
    border: 1px solid var(--line); border-radius: 8px; padding: 8px 11px;
    font-variant-numeric: tabular-nums; font-size: 13px; }
  #status { position: fixed; left: 12px; top: 12px; background: rgba(20,23,31,.88);
    border: 1px solid var(--line); border-radius: 8px; padding: 8px 11px; font-size: 12px; }
  textarea { width: 100%; height: 120px; background: #232936; color: var(--ink);
    border: 1px solid var(--line); border-radius: 7px; padding: 8px;
    font: 12px/1.4 ui-monospace, Consolas, monospace; resize: vertical; }
</style>
</head>
<body>
<div id="view"></div>
<div id="status">loading mesh…</div>
<div id="live">hover a surface to read its map-space position</div>

<div id="panel">
  <h1>MapSpace Point Picker</h1>
  <div class="sub" id="mapname">—</div>

  <div class="row">
    <input id="label" placeholder="name this point, e.g. door" />
    <button class="primary" id="add">Pick</button>
  </div>
  <div id="hint">
    Type a name, then <b>click the spot on the mesh</b>. Left-drag orbits,
    right-drag pans, wheel zooms. Coordinates are metres in MultiSet map
    space, exactly as stored.
  </div>

  <table>
    <thead><tr><th>Point</th><th class="num">X</th><th class="num">Y</th><th class="num">Z</th><th></th></tr></thead>
    <tbody id="rows"></tbody>
  </table>

  <div class="row" style="margin-top:12px">
    <button id="copy">Copy JSON</button>
    <button id="clear">Clear all</button>
  </div>
  <textarea id="out" readonly></textarea>
</div>

<script type="importmap">
{ "imports": {
  "three": "./lib/three/three.module.js",
  "three/addons/": "./lib/three/examples/jsm/"
} }
</script>

<script type="module">
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { DRACOLoader } from 'three/addons/loaders/DRACOLoader.js';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

const params = new URLSearchParams(location.search);
const file = params.get('mesh') || 'mesh.glb';
document.getElementById('mapname').textContent = file;

const view = document.getElementById('view');
```

```
const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setPixelRatio(Math.min(devicePixelRatio, 2));
renderer.setSize(innerWidth, innerHeight);
view.appendChild(renderer.domElement);

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x14171f);
const camera = new THREE.PerspectiveCamera(55, innerWidth / innerHeight, 0.05, 500);
const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;

// A textured scan carries its own photographic lighting, so this only
// lifts the shadows enough to read geometry in the dark corners.
scene.add(new THREE.AmbientLight(0xffffff, 2.2));
const key = new THREE.DirectionalLight(0xffffff, 1.1);
key.position.set(3, 8, 5);
scene.add(key);

// Map-space origin marker: three 1 m axes at (0,0,0). Everything is
// measured from here, so it helps to see where "here" is.
const axes = new THREE.AxesHelper(1);
scene.add(axes);

let target = null;
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
const picks = [];
const markers = new THREE.Group();
scene.add(markers);

// What the APP will do with the dots, drawn over the scan.
//
// The picker shows where you clicked; it cannot show what is then made of
// those clicks. This draws placed.json - the project's own trigger and
// stations, read back out of StationMarker.cs and MuseumStation.cs by
// Tools/show-placement.py - so the arrangement can be checked here rather
// than discovered in the room.
//
// A circle on the floor per station, the radius a visitor has to be inside
// before the card appears. Two circles that swallow each other is the
// failure the picker hides: the dots look well spread, and in the room the
// wrong card keeps opening.
const placed = new THREE.Group();
scene.add(placed);

function labelSprite(text, colour) {
  const pad = 12, font = 48;
  const c = document.createElement('canvas');
  const g = c.getContext('2d');
  g.font = `${font}px sans-serif`;
  c.width = g.measureText(text).width + pad * 2;
  c.height = font + pad * 2;
  const g2 = c.getContext('2d');
  g2.font = `${font}px sans-serif`;
  g2.fillStyle = 'rgba(12,16,26,0.82)';
  g2.fillRect(0, 0, c.width, c.height);
  g2.fillStyle = colour;
  g2.textBaseline = 'middle';
  g2.fillText(text, pad, c.height / 2);
```

```
  const sprite = new THREE.Sprite(new THREE.SpriteMaterial({
    map: new THREE.CanvasTexture(c), depthTest: false, transparent: true,
  }));
  sprite.scale.set(c.width / c.height * 0.20, 0.20, 1);
  return sprite;
}

fetch('./placed.json')
  .then(r => (r.ok ? r.json() : null))
  .then(data => {
    if (!data) return;
    const floorY = data.floorY ?? -1.6;

    // The trigger, and which way it looks. Its facing aims everything, so
    // an arrow that points into the wall rather than the room is worth
    // seeing before a rebuild.
    const t = data.trigger.map;
    const marker = new THREE.Mesh(
      new THREE.BoxGeometry(0.18, 0.24, 0.02),
      new THREE.MeshBasicMaterial({ color: 0xffc85c }),
    );
    marker.position.set(t[0], t[1], t[2]);
    placed.add(marker);

    const n = data.trigger.normal;
    const arrow = new THREE.ArrowHelper(
      new THREE.Vector3(n[0], n[1], n[2]).normalize(),
      new THREE.Vector3(t[0], t[1], t[2]), 0.8, 0xffc85c, 0.18, 0.1,
    );
    placed.add(arrow);

    const tl = labelSprite(`trigger  ${data.trigger.facingDeg}\u00b0`, '#ffc85c');
    tl.position.set(t[0], t[1] + 0.3, t[2]);
    placed.add(tl);

    for (const st of data.stations) {
      const [x, y, z] = st.map;

      const ball = new THREE.Mesh(
        new THREE.SphereGeometry(0.09, 20, 14),
        new THREE.MeshBasicMaterial({ color: 0x4ea3ff }),
      );
      ball.position.set(x, y, z);
      placed.add(ball);

      // Down to the floor, so a dot floating in mid air is obvious.
      placed.add(new THREE.Line(
        new THREE.BufferGeometry().setFromPoints([
          new THREE.Vector3(x, y, z), new THREE.Vector3(x, floorY, z),
        ]),
        new THREE.LineBasicMaterial({ color: 0x4ea3ff, transparent: true, opacity: 0.5 }),
      ));

      const ring = new THREE.Mesh(
        new THREE.RingGeometry(st.radius - 0.03, st.radius, 64),
        new THREE.MeshBasicMaterial({
          color: 0x4ea3ff, side: THREE.DoubleSide, transparent: true, opacity: 0.75,
        }),
      );
```

```
      ring.rotation.x = -Math.PI / 2;
      ring.position.set(x, floorY + 0.01, z);
      placed.add(ring);

      const lab = labelSprite(st.name, '#cfe4ff');
      lab.position.set(x, y + 0.28, z);
      placed.add(lab);
    }

    document.getElementById('hint').insertAdjacentHTML('beforeend',
      `<br><b>Gold</b> is the trigger and its facing. <b>Blue</b> is where each `
      + `description sits, with the circle a visitor must be inside for it to open.`);
  })
  .catch(() => {});

// MultiSet's meshes are Draco compressed, which is why their Unity SDK
// pulls in a Draco package. Without this the loader throws outright.
const draco = new DRACOLoader();
draco.setDecoderPath('./lib/three/examples/jsm/libs/draco/gltf/');
const loader = new GLTFLoader();
loader.setDRACOLoader(draco);

loader.load(file, (gltf) => {
  target = gltf.scene;
  scene.add(target);

  const box = new THREE.Box3().setFromObject(target);
  const size = box.getSize(new THREE.Vector3());
  const mid = box.getCenter(new THREE.Vector3());
  controls.target.copy(mid);
  camera.position.set(mid.x + size.x * 0.7, box.max.y + size.y * 0.9, mid.z + size.z * 0.7);
  camera.lookAt(mid);
  controls.update();

  document.getElementById('status').textContent =
    `${size.x.toFixed(2)} × ${size.z.toFixed(2)} m footprint, ${size.y.toFixed(2)} m tall`;
}, (e) => {
  if (e.total) document.getElementById('status').textContent =
    `loading ${(100 * e.loaded / e.total).toFixed(0)}%`;
}, (err) => {
  document.getElementById('status').textContent = 'failed to load: ' + err;
});

function hit(event) {
  if (!target) return null;
  const r = renderer.domElement.getBoundingClientRect();
  pointer.x = ((event.clientX - r.left) / r.width) * 2 - 1;
  pointer.y = -((event.clientY - r.top) / r.height) * 2 + 1;
  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObject(target, true);
  if (!hits.length) return null;

  // The surface normal comes back too. For the marker that is the whole
  // point: it says which way the printed sheet faces, which is what turns
  // a position into a full pose. Read in world space, not the triangle's
  // local space, or a rotated node reports the wrong direction.
  const first = hits[0];
  let normal = null;
  if (first.face) {
```

```
    normal = first.face.normal.clone()
      .applyNormalMatrix(new THREE.Matrix3().getNormalMatrix(first.object.matrixWorld))
      .normalize();
  }
  return { point: first.point, normal };
}

const live = document.getElementById('live');
renderer.domElement.addEventListener('pointermove', (e) => {
  const h = hit(e);
  live.textContent = h
    ? `x ${h.point.x.toFixed(3)}   y ${h.point.y.toFixed(3)}   z ${h.point.z.toFixed(3)}`
    : 'hover a surface to read its map-space position';
});

// A click only records a point when the camera did not move, so orbiting
// never drops a stray marker.
let downAt = null;
renderer.domElement.addEventListener('pointerdown', (e) => { downAt = { x: e.clientX, y: e.clientY }; });
renderer.domElement.addEventListener('pointerup', (e) => {
  if (!downAt) return;
  const moved = Math.hypot(e.clientX - downAt.x, e.clientY - downAt.y);
  downAt = null;
  if (moved > 4 || e.button !== 0) return;
  const h = hit(e);
  if (h) record(h.point, h.normal);
});

function record(p, normal) {
  const input = document.getElementById('label');
  const name = (input.value || `point-${picks.length + 1}`).trim();
  const entry = { name, x: +p.x.toFixed(3), y: +p.y.toFixed(3), z: +p.z.toFixed(3) };
  if (normal) {
    entry.nx = +normal.x.toFixed(3);
    entry.ny = +normal.y.toFixed(3);
    entry.nz = +normal.z.toFixed(3);
    // Compass bearing of the surface in the map's XZ plane, so a wall's
    // facing can be read without interpreting three components by eye.
    entry.facingDeg = +(((Math.atan2(normal.x, normal.z) * 180 / Math.PI) + 360) % 360).toFixed(1);
  }
  picks.push(entry);
  input.value = '';

  const dot = new THREE.Mesh(
    new THREE.SphereGeometry(0.06, 16, 12),
    new THREE.MeshBasicMaterial({ color: 0x4ea3ff }),
  );
  dot.position.copy(p);
  markers.add(dot);
  draw();
}

function draw() {
  const rows = document.getElementById('rows');
  rows.innerHTML = '';
  picks.forEach((p, i) => {
    const tr = document.createElement('tr');
    tr.innerHTML = `<td>${p.name}</td><td class="num">${p.x.toFixed(2)}</td>` +
      `<td class="num">${p.y.toFixed(2)}</td><td class="num">${p.z.toFixed(2)}</td>` +
```

```
      `<td class="del" data-i="${i}">✕</td>`;
    rows.appendChild(tr);
  });
  rows.querySelectorAll('.del').forEach(cell => {
    cell.onclick = () => {
      const i = +cell.dataset.i;
      picks.splice(i, 1);
      markers.remove(markers.children[i]);
      draw();
    };
  });
  document.getElementById('out').value = JSON.stringify(picks, null, 2);
}

document.getElementById('add').onclick = () =>
  document.getElementById('label').focus();
document.getElementById('clear').onclick = () => {
  picks.length = 0;
  while (markers.children.length) markers.remove(markers.children[0]);
  draw();
};
document.getElementById('copy').onclick = async () => {
  try { await navigator.clipboard.writeText(JSON.stringify(picks, null, 2)); } catch {}
};

// Exposed so the picking and the camera can be driven from the console as
// well as by hand - useful for framing a consistent set of views.
window.mapPicks = picks;
window.recordAt = (x, y, z) => record(new THREE.Vector3(x, y, z), null);
window.view = { THREE, scene, camera, controls, get target() { return target; } };
window.look = (px, py, pz, tx, ty, tz) => {
  camera.position.set(px, py, pz);
  controls.target.set(tx, ty, tz);
  controls.update();
};
window.bounds = () => {
  if (!target) return null;
  const b = new THREE.Box3().setFromObject(target);
  return { min: b.min.toArray(), max: b.max.toArray() };
};

addEventListener('resize', () => {
  camera.aspect = innerWidth / innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(innerWidth, innerHeight);
});

(function loop() {
  requestAnimationFrame(loop);
  controls.update();
  renderer.render(scene, camera);
})();
</script>
</body>
</html>
```

3.2.3 MAP-009 — export.html

Code ID	MAP-009
File Name	export.html
File Path	SanctiWalkAR/Tools/viewer/export.html
Language / Type	HTML
Lines	172
Functions / Classes	say (function); BOUNDS (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/viewer/export.html. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — export.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<title>Sala Reference Export</title>
<style>
  body { margin:0; background:#14171f; color:#eef1f6;
         font:14px/1.6 "Segoe UI", system-ui, sans-serif; padding:24px; }
  h1 { font-size:17px; margin:0 0 4px; }
  p { color:#98a1b3; margin:0 0 18px; }
  pre { background:#1c212c; border:1px solid #2b3240; border-radius:8px;
        padding:12px; white-space:pre-wrap; font:12px/1.5 ui-monospace, Consolas, monospace; }
</style>
</head>
<body>
<h1>Sala reference export</h1>
<p>Decodes the MultiSet scan, cuts the ceiling away, and writes an OBJ Unity can import.</p>
<pre id="log">starting…</pre>

<script type="importmap">
{ "imports": { "three": "./lib/three/three.module.js", "three/addons/": "./lib/three/examples/jsm/" } }
</script>

<script type="module">
import * as THREE from 'three';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { DRACOLoader } from 'three/addons/loaders/DRACOLoader.js';

const out = document.getElementById('log');
const say = (m) => { out.textContent += '\n' + m; console.log(m); };

// Everything above this height is ceiling. The floor sits at y = -1.587
// and the ceiling band at about +1.4, so this cuts below the ceiling and
```

```
// above head height, which is what makes the room readable from above.
const CEILING_CUT = 0.95;

// The scan frays at its edges into disconnected shards that are not room
// and only make the view harder to read. Every station sits within
// x -0.07..3.40 and z 0.77..5.06, and the marker at z 5.35, so this keeps
// a wide margin around all of them while cutting the debris.
// These were measured for the SALA and crop to its walls. Applied to a
// church they crop the nave down to a living-room-sized box: the first
// Mary Help export kept 45k of 848k triangles, all of them inside the
// sala's old bounding box.
//
// ?bounds=none keeps everything but the ceiling, which is what a church
// wants - its own walls already bound it.
const SALA_BOUNDS = { minX: -2.6, maxX: 5.4, minZ: -1.4, maxZ: 7.6, minY: -1.95 };
const INFINITE = { minX: -1e9, maxX: 1e9, minZ: -1e9, maxZ: 1e9, minY: -1e9 };
const BOUNDS = (new URLSearchParams(location.search).get('bounds') === 'none')
  ? INFINITE : SALA_BOUNDS;

const draco = new DRACOLoader();
draco.setDecoderPath('./lib/three/examples/jsm/libs/draco/gltf/');
const loader = new GLTFLoader();
loader.setDRACOLoader(draco);

const params = new URLSearchParams(location.search);
const MESH = params.get('mesh') || 'sala-textured.glb';
const OUTNAME = params.get('as') || 'sala-reference';
document.querySelector('h1').textContent = OUTNAME + ' export';
say('decoding ' + MESH + ' -> ' + OUTNAME + '.obj');

loader.load(MESH, async (gltf) => {
  say('decoded glb');

  const kept = [];
  let total = 0, dropped = 0;
  const a = new THREE.Vector3(), b = new THREE.Vector3(), c = new THREE.Vector3();
  const uvA = new THREE.Vector2(), uvB = new THREE.Vector2(), uvC = new THREE.Vector2();

  gltf.scene.updateMatrixWorld(true);
  gltf.scene.traverse(o => {
    if (!o.isMesh) return;
    const g = o.geometry;
    const pos = g.getAttribute('position');
    const uv = g.getAttribute('uv');
    const idx = g.index;
    const n = idx ? idx.count : pos.count;

    for (let i = 0; i < n; i += 3) {
      const i0 = idx ? idx.getX(i) : i, i1 = idx ? idx.getX(i + 1) : i + 1, i2 = idx ? idx.getX(i + 2) : i + 2;
      a.fromBufferAttribute(pos, i0).applyMatrix4(o.matrixWorld);
      b.fromBufferAttribute(pos, i1).applyMatrix4(o.matrixWorld);
      c.fromBufferAttribute(pos, i2).applyMatrix4(o.matrixWorld);
      total++;

      // Drop a triangle if any corner is above the cut or outside the room.
      const bad = [a, b, c].some(p =>
        p.y > CEILING_CUT || p.y < BOUNDS.minY ||
        p.x < BOUNDS.minX || p.x > BOUNDS.maxX ||
        p.z < BOUNDS.minZ || p.z > BOUNDS.maxZ);
```

```
    if (bad) { dropped++; continue; }

    uvA.fromBufferAttribute(uv, i0);
    uvB.fromBufferAttribute(uv, i1);
    uvC.fromBufferAttribute(uv, i2);
    kept.push([a.clone(), b.clone(), c.clone(), uvA.clone(), uvB.clone(), uvC.clone()]);
  }
});

say(`triangles: ${total} total, ${dropped} removed (ceiling + fringe), ${kept.length} kept`);

// Write the OBJ by hand. OBJExporter would re-emit the whole scene
// including the geometry just filtered out.
const lines = [];
lines.push('# SanctiWalk sala reference, exported from MAP_HXPCAEUNK15O');
lines.push('# Ceiling cut at y > ' + CEILING_CUT + ' so the room can be read from above.');
lines.push('# Editor reference only. The original glb remains the source of truth.');
lines.push('mtllib ' + OUTNAME + '.mtl');
lines.push('o SalaReference');

const vs = [], vts = [], fs = [];
let v = 1;
for (const [p0, p1, p2, t0, t1, t2] of kept) {
  // Unity is left-handed; OBJ and glTF are right-handed. Negating X here
  // means the imported mesh sits in Unity at the same coordinates the
  // picker reported, so the surveyed numbers keep their meaning.
  for (const p of [p0, p1, p2]) vs.push(`v ${(-p.x).toFixed(4)} ${p.y.toFixed(4)} ${p.z.toFixed(4)}`);
  // V is flipped. glTF puts the texture origin top-left and this atlas
  // ships with flipY = false; OBJ and Unity both take it bottom-left.
  // Written straight through, every UV lands in the atlas's unused white
  // gaps - measured during diagnosis as 75% of triangles sampling white,
  // against 0.2% with the correct orientation.
  for (const t of [t0, t1, t2]) vts.push(`vt ${t.x.toFixed(5)} ${(1 - t.y).toFixed(5)}`);
  fs.push(`f ${v}/${v} ${v + 2}/${v + 2} ${v + 1}/${v + 1}`);   // winding flipped with X
  v += 3;
}
// Not lines.push(...vs, ...vts, ...fs): spreading roughly 670k elements
// into one call exhausts the argument stack and throws RangeError.
const obj = lines.join('\n') + '\n'
  + vs.join('\n') + '\n'
  + vts.join('\n') + '\n'
  + 'usemtl SalaTexture\n'
  + fs.join('\n') + '\n';
say(`obj built: ${(obj.length / 1e6).toFixed(1)} MB`);

const mtl = [
  'newmtl SalaTexture',
  'Ka 1.000 1.000 1.000',
  'Kd 1.000 1.000 1.000',
  'Ks 0.000 0.000 0.000',
  'illum 1',
  'map_Kd ' + OUTNAME + '.jpg',
].join('\n');

await put(OUTNAME + '.obj', obj);
await put(OUTNAME + '.mtl', mtl);

// The atlas, re-encoded from the decoded texture.
const tex = gltf.scene.getObjectByProperty('isMesh', true)?.material?.map;
```

```
  if (tex && tex.image) {
    const cv = document.createElement('canvas');
    cv.width = tex.image.width; cv.height = tex.image.height;
    cv.getContext('2d').drawImage(tex.image, 0, 0);
    const blob = await new Promise(r => cv.toBlob(r, 'image/jpeg', 0.92));
    await put(OUTNAME + '.jpg', blob);
    say(`texture written: ${cv.width}x${cv.height}, ${(blob.size / 1e6).toFixed(1)} MB jpg`);
  } else {
    say('WARNING: no texture found');
  }

  say('DONE');
}, undefined, (e) => say('load failed: ' + e));

async function put(name, body) {
  const res = await fetch('/save/' + name, { method: 'PUT', body });
  say(`PUT ${name} -> ${res.status}`);
}
</script>
</body>
</html>
```

3.2.4 MAP-010 — save_server.py

Code ID	MAP-010
File Name	save_server.py
File Path	SanctiWalkAR/Tools/viewer/save_server.py
Language / Type	Python
Lines	77
Functions / Classes	Handler (class); Server (class)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/viewer/save_server.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Serves the viewer and accepts PUT so the page can write files to disk.
>
> The reference mesh arrives from MultiSet as Draco-compressed glTF, which Unity cannot import without adding packages to a project that currently builds cleanly. Rather than risk that, the decoding happens in the browser - which already has a Draco decoder for the picker - and the result is PUT back here as a plain OBJ that Unity imports natively.
>
> Writes are confined to Tools/viewer/out.

Complete Source Code — save_server.py

```
"""Serves the viewer and accepts PUT so the page can write files to disk.

The reference mesh arrives from MultiSet as Draco-compressed glTF, which
Unity cannot import without adding packages to a project that currently
builds cleanly. Rather than risk that, the decoding happens in the browser
- which already has a Draco decoder for the picker - and the result is
PUT back here as a plain OBJ that Unity imports natively.

Writes are confined to Tools/viewer/out.
"""

import http.server
import os
import posixpath
import socketserver

PORT = 8777
HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(HERE, "out")


class Handler(http.server.SimpleHTTPRequestHandler):
```

```
    def __init__(self, *args, **kwargs):
        super().__init__(*args, directory=HERE, **kwargs)

    def do_PUT(self):
        name = posixpath.basename(self.path)
        if not name or name.startswith("."):
            self.send_error(400, "bad name")
            return

        length = int(self.headers.get("Content-Length", 0))
        os.makedirs(OUT, exist_ok=True)
        target = os.path.join(OUT, name)

        remaining = length
        with open(target, "wb") as handle:
            while remaining > 0:
                chunk = self.rfile.read(min(1 << 20, remaining))
                if not chunk:
                    break
                handle.write(chunk)
                remaining -= len(chunk)

        print(f"wrote {target} ({os.path.getsize(target)} bytes)", flush=True)
        self.send_response(200)
        self.send_header("Content-Length", "2")
        self.end_headers()
        self.wfile.write(b"ok")

    def end_headers(self):
        # Never cache anything here.
        #
        # This is a dev tool whose files are rewritten constantly - the page
        # itself while it is being worked on, and the plan JSON every time
        # the generator runs. A cached copy is indistinguishable from the
        # edit or the save having failed, and that cost several rounds of
        # "it did not change" before anyone suspected the browser.
        self.send_header("Cache-Control", "no-store, must-revalidate")
        self.send_header("Pragma", "no-cache")
        self.send_header("Expires", "0")
        super().end_headers()

    def log_message(self, fmt, *args):
        pass


class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True


if __name__ == "__main__":
    os.makedirs(OUT, exist_ok=True)
    with Server(("127.0.0.1", PORT), Handler) as httpd:
        print(f"serving {HERE} on http://127.0.0.1:{PORT}  (PUT -> {OUT})", flush=True)
        httpd.serve_forever()
```

3.3 Placement Pipeline and Survey Tools

3.3.1 MAP-011 — deploy-stations.py

Code ID	MAP-011
File Name	deploy-stations.py
File Path	SanctiWalkAR/Tools/deploy-stations.py
Language / Type	Python
Lines	302
Functions / Classes	mirror (function); cs (function); load_i18n (function); emit (function); main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/deploy-stations.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turns a saved station plan into the C# table the app compiles in.
>
> ## Why this is generated rather than typed
>
> The placer writes Tools/viewer/out/<site>.json; the app reads a C# array. Between those two sits a transcription of fifteen positions, three floats each, and a transcription error does not fail to compile - it puts one station somewhere plausible and wrong, which is only discovered by standing in the church.
>
> So the array is generated, and the JSON stays the single source of truth. Re-run this after every save and the two cannot drift.
>
> ## The frame
>
> Positions are written in MultiSet MAP space, verbatim, because that is what the VPS route consumes: ChurchOriginTest sets m_ToSession = MapSpaceIdentity when the origin comes from MultiSet, so a map coordinate IS a station coordinate. Nothing is converted here, and a plan recorded in any other frame is refused rather than reinterpreted.
>
> python Tools/deploy-stations.py # show what would change python Tools/deploy-stations.py --write

Complete Source Code — deploy-stations.py

```
"""Turns a saved station plan into the C# table the app compiles in.

## Why this is generated rather than typed

The placer writes Tools/viewer/out/<site>.json; the app reads a C# array.
Between those two sits a transcription of fifteen positions, three floats
each, and a transcription error does not fail to compile - it puts one
station somewhere plausible and wrong, which is only discovered by standing
```

```
in the church.

So the array is generated, and the JSON stays the single source of truth.
Re-run this after every save and the two cannot drift.

## The frame

Positions are written in MultiSet MAP space, verbatim, because that is
what the VPS route consumes: ChurchOriginTest sets m_ToSession =
MapSpaceIdentity when the origin comes from MultiSet, so a map coordinate
IS a station coordinate. Nothing is converted here, and a plan recorded in
any other frame is refused rather than reinterpreted.

    python Tools/deploy-stations.py            # show what would change
    python Tools/deploy-stations.py --write
"""
import io
import json
import os
import sys

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
OUT = os.path.join(HERE, "viewer", "out")
TARGET = os.path.join(ROOT, "Assets", "Scripts", "MuseumStationsGenerated.cs")

SITES = [
    ("mhcp", "MHCP", "route-mhcp", "Mary Help of Christians Parish"),
    ("srcp", "SRCP", "route-src", "San Roque Cathedral Parish"),
    # The same San Roque tour laid out on the SALA scan, so the walk can be
    # seen at home before going to the cathedral. Sala map coordinates, and
    # it pairs with the sala map code - not the cathedral's.
    ("srcp-sala", "SRCP_IN_SALA", "route-src-preview",
     "San Roque tour, previewed in the sala"),
    # The two Mary Help statues stood up in the sala, so the models can be
    # looked at properly at home before carrying the phone to Maypajo. Only
    # the models - none of the parish's history or image stations.
    ("mhcp-sala", "MHCP_IN_SALA", "route-mhcp-preview",
     "Mary Help statues, previewed in the sala"),
]

# The only frame these numbers mean anything in.
# Sideways correction, applied to every number this file writes.
#
# The placer reports a position in the frame three.js reads out of the
# scan GLB. The app consumes MultiSet map space. The two agree on up and
# on into-the-room, and disagree on sideways: a station picked on the
# left of the scan arrived on the right of the room, in all three sites,
# which is a whole-frame mirror rather than three mistakes.
#
# Corrected here, at the one boundary both sides cross, rather than in
# the placer - the saved JSON for every site already holds the numbers
# as picked, and negating on save would have meant re-placing every
# station in every church to make the files agree again.
#
# Applied to the marker position and the marker normal as well as to the
# stations. Mirroring the stations alone would leave them correct
# relative to the room and wrong relative to the marker they are
# measured from, which is worse than the bug: everything would be out by
```

```
# twice the marker offset instead of reflected about the centre line.
#
# If left and right are STILL swapped after this, flip it back to 1.0 -
# and if the error is front-to-back instead, the axis is wrong, not the
# sign: move the negation to z. Nothing else in this file needs touching.
# Per site, because handedness is a property of the SCAN, not of the
# project. Each church is its own MultiSet reconstruction, and two of
# them can come back with opposite senses of "left" - which is exactly
# what happened. One global constant quietly assumed otherwise.
#
# San Roque: a station placed on the right of the scan arrived on the
# left of the cathedral, reported standing in the building. Mary Help,
# generated by the same script on the same day, is correct - so this is
# not a missed correction, it is the correction being applied where it
# was not wanted. Its marker also sits at the opposite end of its
# station spread from Mary Help's, which is the same fact seen in the
# numbers.
#
# -1.0 flips left and right; 1.0 leaves the scan as it was placed. If a
# church is still swapped after a change here, flip its entry. If the
# error is front-to-back instead, the axis is wrong rather than the
# sign, and the negation belongs on z.
MIRROR_X = {
    "mhcp": -1.0,
    # Back to -1.0, where it started.
    #
    # It was set to 1.0 on a report that San Roque was mirrored, without
    # checking, and that was wrong twice over: it did not fix the one
    # station being complained about, and it mirrored the fourteen that
    # had been right. The sign is not a per-church preference at all.
    #
    # The placer is three.js, right-handed: facing -Z, right is +X. Unity
    # is left-handed: facing -Z, right is -X. So map X is negated on the
    # way into Unity for EVERY parish, which is why Mary Help - the same
    # placer, the same runtime - has always needed -1.0 and has always
    # been correct with it.
    "srcp": -1.0,
}

# Unknown sites keep the old global value, so the sala preview and
# anything else not listed above is untouched.
MIRROR_X_DEFAULT = -1.0


def mirror(v, site):
    """A map-space triple, with this site's sideways axis correction."""
    return (MIRROR_X.get(site, MIRROR_X_DEFAULT) * v[0], v[1], v[2])


REQUIRED_FRAME = "map-v1"

# Emitted beside each parish's stations: where its printed marker hangs on
# that parish's own scan. Scanning the sheet gives ARCore the same marker in
# session space, and the two together place the whole map - which is what
# makes a printed marker work in a church without a VPS call.
MARKER_BLOCK = """    /// <summary>
    /// Where the printed %s marker hangs, in this parish's map space, and
    /// the direction it faces. Measured by placing it on the scan, not
    /// assumed.
```

```
    /// </summary>
    public static readonly Vector3 %s_MarkerInMap =
        new Vector3(%.3ff, %.3ff, %.3ff);
    public static readonly Vector3 %s_MarkerNormal =
        new Vector3(%.4ff, %.4ff, %.4ff);"""



def cs(text):
    """A C# string literal. Tagalog content carries quotes and newlines."""
    if not text:
        return '""'
    out = (text.replace("\\", "\\\\").replace('"', '\\"')
               .replace("\r", "").replace("\n", "\\n"))
    return '"%s"' % out


def load_i18n(site):
    """The tl/en text for a site, if someone has written it.

    Optional on purpose: a site with no i18n file still deploys, and its
    stations simply have one language. The file lives beside the plan in
    Tools/viewer/sites/<site>.i18n.json and is keyed by station id.
    """
    path = os.path.join(HERE, "viewer", "sites", site + ".i18n.json")
    if not os.path.exists(path):
        return {}
    with io.open(path, encoding="utf-8") as f:
        return json.load(f).get("stations", {})


def emit(site, field, parish_id, title, plan):
    lines = []
    lines.append("    /// <summary>")
    lines.append("    /// %s, %d stations." % (title, len(plan["stations"])))
    lines.append("    ///")
    lines.append("    /// Generated by Tools/deploy-stations.py from")
    lines.append("    /// Tools/viewer/out/%s.json. Do not edit by hand - the next" % site)
    lines.append("    /// save from the placer overwrites it.")
    lines.append("    ///")
    lines.append("    /// Positions are MultiSet map coordinates for %s," % plan.get("scan", "?"))
    lines.append("    /// used verbatim on the VPS route.")
    lines.append("    /// </summary>")
    lines.append("    public static readonly MuseumStation[] %s =" % field)
    lines.append("    {")

    i18n = load_i18n(site)
    stations = plan["stations"]
    for i, st in enumerate(stations):
        nxt = stations[i + 1]["id"] if i + 1 < len(stations) else ""
        x, y, z = mirror(st["map"], site)
        kind = st.get("kind", "history")
        lines.append("        new MuseumStation")
        lines.append("        {")
        lines.append("            stationId = %s," % cs(st["id"]))
        lines.append("            stationName = %s," % cs(st["name"]))
        lines.append("            subtitle = %s," % cs("%d ng %d" % (i + 1, len(stations))))
        lines.append("            kind = MuseumStationKind.%s," % kind.capitalize())
        lines.append("            position = new Vector3(%.3ff, %.3ff, %.3ff)," % (x, y, z))
```

```
        lines.append("            triggerRadius = %.2ff," % st.get("radius", 2.0))
        if kind == "model":
            lines.append("            model = %s," % cs(st.get("model", "")))
            lines.append("            modelScale = %.2ff," % st.get("modelScale", 1.0))
            lines.append("            yawDegrees = %.0ff," % st.get("yawDegrees", 0))
        else:
            # Resources.Load takes a path RELATIVE TO Resources/, so the
            # subfolder is part of the name. Without it every image silently
            # fails to load and the card falls back to "artwork is missing".
            img = st.get("image", "")
            lines.append("            image = %s," % cs("ArImages/" + img if img else ""))
        if st.get("caption"):
            lines.append("            shortDescription = %s," % cs(st["caption"]))
        if st.get("description"):
            lines.append("            detailedDescription = %s," % cs(st["description"]))

        # The same text in both languages, where it has been translated.
        # Only non-empty sides are written, so MuseumStation.Pick can fall
        # back rather than show a blank card.
        tr = i18n.get(st["id"], {})
        for key, tl_field, en_field in (("name", "nameTl", "nameEn"),
                                        ("caption", "shortTl", "shortEn"),
                                        ("description", "detailedTl", "detailedEn")):
            pair = tr.get(key) or {}
            if pair.get("tl"):
                lines.append("            %s = %s," % (tl_field, cs(pair["tl"])))
            if pair.get("en"):
                lines.append("            %s = %s," % (en_field, cs(pair["en"])))
        lines.append("            nextStationId = %s," % cs(nxt))
        lines.append("        },")
    lines.append("    };")
    return "\n".join(lines)


def main():
    write = "--write" in sys.argv
    blocks, summary = [], []

    for site, field, parish_id, title in SITES:
        path = os.path.join(OUT, site + ".json")
        if not os.path.exists(path):
            # A preview is generated, never placed by hand, so it has no
            # saved file - fall back to the plan rather than skipping it.
            generated = os.path.join(HERE, "viewer", "sites", site + ".json")
            if os.path.exists(generated):
                path = generated
            else:
                print("  %s: nothing saved yet (%s)" % (site, path))
                continue
        plan = json.load(io.open(path, encoding="utf-8"))

        frame = plan.get("frame")
        if frame != REQUIRED_FRAME:
            # Three floats mean completely different things in each frame.
            # Writing marker-frame numbers into a map-frame table compiles
            # and runs and is wrong everywhere.
            print("  REFUSED %s: frame is %r, expected %r." % (site, frame, REQUIRED_FRAME))
            return 1
```

```
        ids = [s["id"] for s in plan["stations"]]
        if len(set(ids)) != len(ids):
            print("  REFUSED %s: duplicate station ids." % site)
            return 1

        blocks.append(emit(site, field, parish_id, title, plan))

        t = plan.get("trigger") or {}
        if t.get("map"):
            n = t.get("normal") or [0, 0, 1]
            mx, my, mz = mirror(t["map"], site)
            nx, ny, nz = mirror(n, site)
            blocks.append(MARKER_BLOCK % (
                site, field, mx, my, mz,
                field, nx, ny, nz))
        kinds = {}
        for s in plan["stations"]:
            kinds[s.get("kind", "history")] = kinds.get(s.get("kind", "history"), 0) + 1
        summary.append("  %-5s %-34s %2d stations  %s  (%s)"
                       % (site, title, len(plan["stations"]),
                          ", ".join("%d %s" % (v, k) for k, v in sorted(kinds.items())),
                          plan.get("scan", "?")))

    if not blocks:
        print("Nothing to deploy.")
        return 1

    source = (
        "// Generated file - do not edit.\n"
        "//\n"
        "// Written by Tools/deploy-stations.py from the plans saved out of\n"
        "// the station placer (Tools/viewer/out/*.json). Edit the positions\n"
        "// in the placer and re-run that script; anything typed here is lost\n"
        "// on the next run.\n"
        "using UnityEngine;\n"
        "\n"
        "public static partial class MuseumStations\n"
        "{\n"
        + "\n\n".join(blocks) + "\n"
        "}\n"
    )

    print("Would write %s:" % os.path.relpath(TARGET, ROOT))
    for line in summary:
        print(line)

    if not write:
        print()
        print("Nothing written. Re-run with --write to apply.")
        return 0

    io.open(TARGET, "w", encoding="utf-8", newline="\n").write(source)
    print()
    print("Wrote %s (%d lines)." % (os.path.relpath(TARGET, ROOT), source.count("\n")))
    return 0


sys.exit(main())
```

3.3.2 MAP-012 — make-parish-marker.py

Code ID	MAP-012
File Name	make-parish-marker.py
File Path	SanctiWalkAR/Tools/make-parish-marker.py
Language / Type	Python
Lines	216
Functions / Classes	font (function); fit_photo (function); draw_band (function); build (function); report (function); main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/make-parish-marker.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Draws a printable AR marker for one parish.
>
> ## What a marker has to be, before it is anything else
>
> ARCore matches a printed sheet by its FEATURES - corners, edges, texture - not by how it looks to a person. So the two jobs pull in opposite directions: a clean brand panel is a flat panel, and flat is exactly what a tracker has nothing to hold on to. The existing Sala marker is the way it is for that reason, and it works: stddev 74 grey levels, asymmetric geometry, nothing repeating.
>
> This keeps that and adds the parish, rather than replacing it with a pretty sheet that tracks badly:
>
> * The patron's photograph fills the upper two thirds. A photograph is the most feature-dense thing available and it is what makes this parish's marker look like this parish. * A high-contrast geometric band sits under it, DIFFERENT for each parish. Two markers that are both "photo over a navy bar" can be confused with each other by the matcher; the band is what keeps them apart, and it is drawn from each parish's own motif rather than picked at random. * The name is set in italic Georgia, the one serif the app uses for a parish's name anywhere it appears.
>
> ## Printing
>
> Exported at 180 mm wide because that is what the app is told the marker measures (k_MarkerWidthMetres in ARProjectSetup). Print at 100%, not "fit to page": scale the sheet and every distance the app derives from it is wrong by the same factor, which reads as the stations being placed badly.
>
> python Tools/make-parish-marker.py route-mhcp python Tools/make-parish-marker.py route-src

Complete Source Code — make-parish-marker.py

```
"""Draws a printable AR marker for one parish.
```

```
## What a marker has to be, before it is anything else

ARCore matches a printed sheet by its FEATURES - corners, edges, texture -
not by how it looks to a person. So the two jobs pull in opposite
directions: a clean brand panel is a flat panel, and flat is exactly what a
tracker has nothing to hold on to. The existing Sala marker is the way it is
for that reason, and it works: stddev 74 grey levels, asymmetric geometry,
nothing repeating.

This keeps that and adds the parish, rather than replacing it with a pretty
sheet that tracks badly:

  * The patron's photograph fills the upper two thirds. A photograph is the
    most feature-dense thing available and it is what makes this parish's
    marker look like this parish.
  * A high-contrast geometric band sits under it, DIFFERENT for each parish.
    Two markers that are both "photo over a navy bar" can be confused with
    each other by the matcher; the band is what keeps them apart, and it is
    drawn from each parish's own motif rather than picked at random.
  * The name is set in italic Georgia, the one serif the app uses for a
    parish's name anywhere it appears.

## Printing

Exported at 180 mm wide because that is what the app is told the marker
measures (k_MarkerWidthMetres in ARProjectSetup). Print at 100%, not "fit to
page": scale the sheet and every distance the app derives from it is wrong
by the same factor, which reads as the stations being placed badly.

    python Tools/make-parish-marker.py route-mhcp
    python Tools/make-parish-marker.py route-src
"""
import io
import math
import os
import sys

from PIL import Image, ImageDraw, ImageFont, ImageStat

DPI = 300
MM = DPI / 25.4
W_MM, H_MM = 180.0, 240.0
W, H = int(W_MM * MM), int(H_MM * MM)

FONTS = r"C:\Windows\Fonts"
GEORGIA_I = os.path.join(FONTS, "georgiai.ttf")
GEORGIA_B = os.path.join(FONTS, "georgiab.ttf")
ARIAL_B = os.path.join(FONTS, "arialbd.ttf")

PARISHES = {
    "route-mhcp": {
        "name": "Mary Help of Christians",
        "sub": "PARISH  \u00b7  MAYPAJO, CALOOCAN",
        "code": "MHCP-ENTRANCE",
        "photo": r"D:\SanctDemo\public\parish\mary-help-hero.jpg",
        "ink": (16, 27, 74),
        "accent": (201, 162, 77),
        # Each parish gets a different band so the matcher cannot confuse
```

```
        # one marker for the other. Chevrons here, diamonds for San Roque.
        "band": "chevrons",
    },
    "route-src": {
        "name": "San Roque Cathedral",
        "sub": "PARISH  \u00b7  POBLACION, CALOOCAN",
        "code": "SRCP-ENTRANCE",
        "photo": r"D:\SanctDemo\public\parish\san-roque-hero.jpg",
        "ink": (58, 31, 12),
        "accent": (211, 121, 49),
        "band": "diamonds",
    },
}

CREAM = (242, 239, 232)


def font(path, px):
    return ImageFont.truetype(path, px)


def fit_photo(path, box_w, box_h):
    im = Image.open(path).convert("RGB")
    scale = max(box_w / im.width, box_h / im.height)
    im = im.resize((int(im.width * scale), int(im.height * scale)), Image.LANCZOS)
    left = (im.width - box_w) // 2
    top = int((im.height - box_h) * 0.18)     # faces sit high in both photos
    return im.crop((left, top, left + box_w, top + box_h))


def draw_band(d, x0, y0, x1, y1, kind, ink, accent):
    """High-contrast geometry, and a different shape per parish."""
    d.rectangle([x0, y0, x1, y1], fill=ink)
    h = y1 - y0
    step = int(h * 0.92)

    if kind == "chevrons":
        # Nested chevrons: many corners, all asymmetric about the centre.
        x = x0 + step // 2
        flip = False
        while x < x1 - step // 4:
            for k, col in ((0, CREAM), (1, accent)):
                off = int(h * 0.20 * k)
                top = y0 + int(h * 0.18) + off
                bot = y1 - int(h * 0.18) + off
                mid = (top + bot) // 2
                w = int(h * 0.16)
                pts = [(x, top), (x + step // 3, mid), (x, bot),
                       (x - w, bot), (x + step // 3 - w, mid), (x - w, top)]
                if flip:
                    pts = [(2 * x - px, py) for px, py in pts]
                d.polygon(pts, fill=col)
            x += step
            flip = not flip
    else:
        # Diamonds on a broken baseline: no two gaps the same.
        x = x0 + int(step * 0.55)
        i = 0
        while x < x1 - step // 4:
```

```
            r = int(h * (0.30 if i % 2 == 0 else 0.20))
            cy = y0 + int(h * (0.44 if i % 3 else 0.58))
            col = CREAM if i % 2 == 0 else accent
            d.polygon([(x, cy - r), (x + r, cy), (x, cy + r), (x - r, cy)], fill=col)
            if i % 3 == 1:
                d.rectangle([x - r // 3, cy + r + 10, x + r // 3, cy + r + 10 + r // 2], fill=accent)
            x += int(step * (0.62 if i % 2 else 0.86))
            i += 1


def build(parish_id):
    cfg = PARISHES[parish_id]
    img = Image.new("RGB", (W, H), CREAM)
    d = ImageDraw.Draw(img)

    margin = int(6 * MM)
    # A heavy asymmetric border: four long edges the tracker can lock to,
    # thicker on the left so the sheet has an unambiguous orientation.
    d.rectangle([0, 0, W - 1, H - 1], outline=cfg["ink"], width=int(3.2 * MM))
    d.rectangle([0, 0, int(9 * MM), H - 1], fill=cfg["ink"])

    inner_x0 = margin + int(7 * MM)
    inner_x1 = W - margin
    photo_y0 = margin + int(4 * MM)
    photo_h = int(H * 0.52)

    photo = fit_photo(cfg["photo"], inner_x1 - inner_x0, photo_h)
    img.paste(photo, (inner_x0, photo_y0))
    d.rectangle([inner_x0, photo_y0, inner_x1, photo_y0 + photo_h],
                outline=cfg["ink"], width=int(1.4 * MM))

    band_y0 = photo_y0 + photo_h + int(5 * MM)
    band_y1 = band_y0 + int(20 * MM)
    draw_band(d, inner_x0, band_y0, inner_x1, band_y1, cfg["band"], cfg["ink"], cfg["accent"])

    # The name, in the app's own italic serif.
    name_font = font(GEORGIA_I, int(11.5 * MM))
    ty = band_y1 + int(9 * MM)
    words = cfg["name"].split()
    line1 = " ".join(words[:2]) if len(words) > 2 else cfg["name"]
    line2 = " ".join(words[2:]) if len(words) > 2 else ""
    d.text((inner_x0, ty), line1, font=name_font, fill=cfg["ink"])
    if line2:
        ty += int(13 * MM)
        d.text((inner_x0, ty), line2, font=name_font, fill=cfg["ink"])

    rule_y = ty + int(15 * MM)
    d.rectangle([inner_x0, rule_y, inner_x1, rule_y + int(1.2 * MM)], fill=cfg["accent"])

    sub_font = font(ARIAL_B, int(3.6 * MM))
    d.text((inner_x0, rule_y + int(5 * MM)), cfg["sub"], font=sub_font, fill=cfg["ink"])

    code_font = font(ARIAL_B, int(3.0 * MM))
    d.text((inner_x0, H - margin - int(9 * MM)), cfg["code"], font=code_font,
           fill=cfg["accent"])

    return img
```

```
def report(img, label):
    g = img.convert("L")
    sd = ImageStat.Stat(g).stddev[0]
    # Detail density: how much of the sheet has a strong local gradient.
    from PIL import ImageFilter
    edges = g.filter(ImageFilter.FIND_EDGES)
    busy = sum(1 for v in edges.getdata() if v > 40) / (img.width * img.height)
    print("  %-22s contrast %.1f   detail %.1f%%" % (label, sd, busy * 100))
    return sd, busy


def main():
    which = sys.argv[1:] or list(PARISHES)
    out_dir = r"D:\SanctiWalkAR\Assets\AR"
    os.makedirs(out_dir, exist_ok=True)

    for pid in which:
        if pid not in PARISHES:
            print("unknown parish:", pid)
            continue
        img = build(pid)
        png = os.path.join(out_dir, "marker-%s.png" % pid.replace("route-", ""))
        img.save(png, dpi=(DPI, DPI))

        pdf = r"D:\SanctiWalkAR\Builds\SanctiWalk-marker-%s-A4.pdf" % pid.replace("route-", "")
        os.makedirs(os.path.dirname(pdf), exist_ok=True)
        # Centred on A4 at exactly 180 mm wide, so "print at 100%" is true.
        a4 = Image.new("RGB", (int(210 * MM), int(297 * MM)), "white")
        a4.paste(img, ((a4.width - img.width) // 2, (a4.height - img.height) // 2))
        a4.save(pdf, "PDF", resolution=DPI)

        print("%s" % pid)
        print("  %s  %dx%d px  (%.0f x %.0f mm)" % (os.path.basename(png), img.width, img.height, W_MM, H_MM))
        print("  %s" % os.path.basename(pdf))
        report(img, "tracking")


main()
```

3.3.3 MAP-013 — make-marker-pdf.py

Code ID	MAP-013
File Name	make-marker-pdf.py
File Path	SanctiWalkAR/Tools/make-marker-pdf.py
Language / Type	Python
Lines	68
Functions / Classes	main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/make-marker-pdf.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Printable A4 sheets for the parish trigger posters.
>
> The physical size matters more than anything else here. ARCore works out how far the phone is from how large a known image appears in the frame, so the sheet has to come off the printer at exactly the size declared in ChurchMarkers.asset - 210 x 297 mm. Printed at "fit to page" a few millimetres smaller, the app believes the camera is further away than it is and every station lands short.
>
> So the page is exactly A4 and the poster fills it edge to edge, with no margin to tempt a printer into scaling.
>
> Usage: python Tools/make-marker-pdf.py

Complete Source Code — make-marker-pdf.py

```
"""Printable A4 sheets for the parish trigger posters.

The physical size matters more than anything else here. ARCore works
out how far the phone is from how large a known image appears in the
frame, so the sheet has to come off the printer at exactly the size
declared in ChurchMarkers.asset - 210 x 297 mm. Printed at "fit to
page" a few millimetres smaller, the app believes the camera is
further away than it is and every station lands short.

So the page is exactly A4 and the poster fills it edge to edge, with
no margin to tempt a printer into scaling.

Usage:  python Tools/make-marker-pdf.py
"""
import os

from PIL import Image

HERE = os.path.dirname(os.path.abspath(__file__))
```

```
AR = os.path.join(HERE, "..", "Assets", "AR")
OUT = os.path.join(HERE, "..", "Builds")

DPI = 300
A4_MM = (210, 297)
A4_PX = (round(A4_MM[0] / 25.4 * DPI), round(A4_MM[1] / 25.4 * DPI))

SHEETS = [
    ("marker-mhcp", "SanctiWalk-marker-mhcp-A4.pdf"),
    ("marker-src", "SanctiWalk-marker-src-A4.pdf"),
]


def main():
    os.makedirs(OUT, exist_ok=True)
    for key, name in SHEETS:
        src = os.path.join(AR, key + ".png")
        if not os.path.exists(src):
            print(f"  --   {key}: missing")
            continue

        im = Image.open(src).convert("RGB")
        # The posters are already A-series proportion; this guards
        # against a future one that is not, rather than silently
        # stretching it.
        want = A4_MM[0] / A4_MM[1]
        got = im.width / im.height
        if abs(got - want) > 0.01:
            print(f"  !!   {key}: ratio {got:.3f}, A4 is {want:.3f} — it will be cropped")
            if got > want:
                w = round(im.height * want)
                im = im.crop(((im.width - w) // 2, 0, (im.width + w) // 2, im.height))
            else:
                h = round(im.width / want)
                im = im.crop((0, (im.height - h) // 2, im.width, (im.height + h) // 2))

        im = im.resize(A4_PX, Image.LANCZOS)
        dst = os.path.join(OUT, name)
        im.save(dst, "PDF", resolution=DPI)
        print(f"  OK   {name}  ({A4_MM[0]}x{A4_MM[1]} mm at {DPI} dpi, "
              f"{os.path.getsize(dst)/1048576:.1f} MB)")

    print("\n  Print at 100% / Actual size. NOT 'fit to page' — that")
    print("  shrinks the sheet by a few millimetres and the AR content")
    print("  then sits at the wrong distance.")


if __name__ == "__main__":
    main()
```

3.3.4 MAP-014 — marker-pose.py

Code ID	MAP-014
File Name	marker-pose.py
File Path	SanctiWalkAR/Tools/marker-pose.py
Language / Type	Python
Lines	223
Functions / Classes	sub (function); add (function); scale (function); length (function); normalise (function); cross (function); dot (function); bearing (function); find (function); wall_normal (function); main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/marker-pose.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turn two picked corners into the marker's pose constants.
>
> WHY TWO POINTS AND NOT ONE ---------- StationMarker.MarkerInMapSpace is the CENTRE of the printed marker, because that is where ARCore reports a tracked image's pose. The centre of a blank rectangle on a wall is not a thing you can point at, so picking it directly means guessing - and the print is 180 x 240 mm, so guessing the centre puts you up to 150 mm out before any other error.
>
> Its corners, though, can be aligned against something real: the corner of a television, the edge of a shelf, a socket. Pick where the two TOP corners will physically sit and the centre is their midpoint, dropped half the print's height. A midpoint also AVERAGES the two clicking errors, where a single pick would carry its error whole.
>
> WHERE THE FACING COMES FROM, AND WHERE IT DOES NOT ---------- Not from the line between the corners. That line is only 180 mm long, so it magnifies error savagely: a 30 mm slip on one corner tilts it by 9.5 degrees, which is 83 cm of drift five metres away - worse than the offset this is meant to fix.
>
> The viewer reports the mesh's own surface normal at each click, measured from the scanned wall itself over an area far larger than 180 mm. That is the facing. The corner-to-corner line is then used only to CHECK the picks, never to aim anything.
>
> USAGE ----- In Tools/viewer (open-viewer.cmd), pick exactly two points, named:
>
> marker-tl where the print's TOP-LEFT corner will sit marker-tr where the print's TOP-RIGHT corner will sit
>
> Both must be on the wall the print goes on. Left and right do not have to be the right way round - the centre is their midpoint and the facing comes from the wall - but keeping them straight makes the report easier to read. Copy JSON, save it, then:

Complete Source Code — marker-pose.py

```
"""Turn two picked corners into the marker's pose constants.

WHY TWO POINTS AND NOT ONE
--------------------------
StationMarker.MarkerInMapSpace is the CENTRE of the printed marker, because
that is where ARCore reports a tracked image's pose. The centre of a blank
rectangle on a wall is not a thing you can point at, so picking it directly
means guessing - and the print is 180 x 240 mm, so guessing the centre puts
you up to 150 mm out before any other error.

Its corners, though, can be aligned against something real: the corner of a
television, the edge of a shelf, a socket. Pick where the two TOP corners
will physically sit and the centre is their midpoint, dropped half the
print's height. A midpoint also AVERAGES the two clicking errors, where a
single pick would carry its error whole.

WHERE THE FACING COMES FROM, AND WHERE IT DOES NOT
--------------------------------------------------
Not from the line between the corners. That line is only 180 mm long, so it
magnifies error savagely: a 30 mm slip on one corner tilts it by 9.5
degrees, which is 83 cm of drift five metres away - worse than the offset
this is meant to fix.

The viewer reports the mesh's own surface normal at each click, measured
from the scanned wall itself over an area far larger than 180 mm. That is
the facing. The corner-to-corner line is then used only to CHECK the picks,
never to aim anything.

USAGE
-----
In Tools/viewer (open-viewer.cmd), pick exactly two points, named:

    marker-tl     where the print's TOP-LEFT corner will sit
    marker-tr     where the print's TOP-RIGHT corner will sit

Both must be on the wall the print goes on. Left and right do not have to be
the right way round - the centre is their midpoint and the facing comes from
the wall - but keeping them straight makes the report easier to read. Copy
JSON, save it, then:

    python Tools/marker-pose.py picks.json
"""

import json
import math
import sys

# The printed marker, metres. Measured off the print: the 100 mm calibration
# line came out exact, so these are the real dimensions and not the nominal
# ones. ARProjectSetup.k_MarkerWidthMetres and ChurchMarkers.asset agree.
MARKER_WIDTH = 0.180
MARKER_HEIGHT = 0.240

# How far the span between the picks may stray from MARKER_WIDTH. Clicking a
# corner on a photogrammetry mesh is good to a centimetre or two, not to a
# millimetre, and the span is only a check here - the facing no longer hangs
# off it - so this is loose enough to accept honest clicks and tight enough
# to catch a different corner or a pick that missed the wall.
SPAN_TOLERANCE = 0.030
```

```
# The corners should sit level with each other and flat against the wall.
# More than this and a pick has landed on something that is not the wall.
LEVEL_TOLERANCE = 0.030
SQUARE_TOLERANCE_DEG = 12.0


def sub(a, b):
    return [a[i] - b[i] for i in range(3)]


def add(a, b):
    return [a[i] + b[i] for i in range(3)]


def scale(a, k):
    return [v * k for v in a]


def length(a):
    return math.sqrt(sum(v * v for v in a))


def normalise(a):
    n = length(a)
    if n == 0:
        raise SystemExit("A direction came out zero length; the picks are degenerate.")
    return [v / n for v in a]


def cross(a, b):
    return [a[1] * b[2] - a[2] * b[1],
            a[2] * b[0] - a[0] * b[2],
            a[0] * b[1] - a[1] * b[0]]


def dot(a, b):
    return sum(a[i] * b[i] for i in range(3))


def bearing(v):
    """The XZ compass bearing the viewer reports, so the two can be compared."""
    return (math.degrees(math.atan2(v[0], v[2])) + 360) % 360


def find(picks, name):
    for p in picks:
        if p.get("name", "").strip().lower() == name:
            return p
    raise SystemExit(
        f'No pick named "{name}". Found: '
        + ", ".join(repr(p.get("name")) for p in picks)
    )


def wall_normal(*picks):
    """The wall's facing, from the mesh normals the viewer measured.

    Flattened into the XZ plane because the marker hangs on a vertical wall
```

```
    and StationMarker's frame takes Y as up; any Y in the normal is the
    scan's surface noise, not the wall leaning over.
    """
    total = [0.0, 0.0, 0.0]
    used = 0
    for p in picks:
        n = [p.get("nx"), p.get("ny"), p.get("nz")]
        if any(v is None for v in n):
            continue
        flat = [n[0], 0.0, n[2]]
        if length(flat) < 1e-6:
            continue                        # a floor or ceiling pick
        total = add(total, normalise(flat))
        used += 1
    if used == 0:
        raise SystemExit(
            "Neither pick carries a surface normal, so the wall's facing cannot\n"
            "be measured. Re-pick in the viewer, which records one per click."
        )
    return normalise(total), used


def main():
    if len(sys.argv) < 2:
        raise SystemExit(__doc__)

    picks = json.load(open(sys.argv[1], encoding="utf-8"))
    if isinstance(picks, dict):
        picks = [picks]

    tl_pick = find(picks, "marker-tl")
    tr_pick = find(picks, "marker-tr")
    tl = [tl_pick["x"], tl_pick["y"], tl_pick["z"]]
    tr = [tr_pick["x"], tr_pick["y"], tr_pick["z"]]

    print(f"picked top-left   {tl[0]:+.3f} {tl[1]:+.3f} {tl[2]:+.3f}")
    print(f"picked top-right  {tr[0]:+.3f} {tr[1]:+.3f} {tr[2]:+.3f}")

    # --- the facing, from the wall rather than from the picks -----------
    normal, used = wall_normal(tl_pick, tr_pick)
    print(f"\nwall facing       {bearing(normal):.1f} degrees "
          f"(from {used} measured surface normal{'s' if used > 1 else ''})")

    # --- checks on the picks --------------------------------------------
    edge = sub(tr, tl)
    span = length(edge)
    drift = span - MARKER_WIDTH
    print(f"span between them {span:.3f} m, off by {drift * 1000:+.0f} mm "
          f"(the print is {MARKER_WIDTH:.3f} m wide)")

    problems = []
    if abs(drift) > SPAN_TOLERANCE:
        problems.append(
            f"The picks are {span:.3f} m apart but the print is {MARKER_WIDTH:.3f} m "
            f"wide.\n  They are not its two top corners: a different corner, or a "
            f"click that\n  missed the wall and landed on something behind it.")

    if abs(edge[1]) > LEVEL_TOLERANCE:
        problems.append(
```

```
            f"The two corners differ by {abs(edge[1]) * 1000:.0f} mm in height, so "
            f"they are not\n  level. The print would hang crooked, and the centre "
            f"computed here\n  would be wrong.")

    if span > 1e-6:
        off_wall = abs(dot(normalise(edge), normal))
        angle = math.degrees(math.asin(min(1.0, off_wall)))
        if angle > SQUARE_TOLERANCE_DEG:
            problems.append(
                f"The line between the picks runs {angle:.0f} degrees out of the "
                f"wall plane.\n  One of them is probably on a different surface.")

    if problems:
        print("\nREFUSED:\n")
        for p in problems:
            print(f"- {p}\n")
        print("Nothing has been changed. Re-pick and run this again.")
        return 1

    # --- the pose ------------------------------------------------------
    up = [0.0, 1.0, 0.0]
    # Midpoint averages the two clicking errors instead of carrying one whole.
    centre = add(scale(add(tl, tr), 0.5), scale(up, -MARKER_HEIGHT / 2))

    print(f"centre            {centre[0]:+.3f} {centre[1]:+.3f} {centre[2]:+.3f}")

    # For the record: what the short baseline WOULD have said, and by how
    # much it disagrees. This is why it is not used to aim anything.
    if span > 1e-6:
        from_edge = normalise(cross(normalise(edge), up))
        gap = abs((bearing(from_edge) - bearing(normal) + 180) % 360 - 180)
        print(f"\n(for reference: the 180 mm corner-to-corner line would have put "
              f"the facing\n at {bearing(from_edge):.1f} degrees, {gap:.1f} degrees "
              f"from the wall's own. At 5 m that is\n {math.tan(math.radians(gap)) * 5:.2f} m "
              f"of drift, which is why the wall is used instead.)")

    print("\n--- paste into Assets/Scripts/StationMarker.cs ---\n")
    print(f"    public static readonly Vector3 MarkerInMapSpace = "
          f"new Vector3({centre[0]:.3f}f, {centre[1]:.3f}f, {centre[2]:.3f}f);")
    print(f"    public static readonly Vector3 MarkerNormal = "
          f"new Vector3({normal[0]:.4f}f, {normal[1]:.4f}f, {normal[2]:.4f}f);")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

3.3.5 MAP-015 — glb-to-obj-indexed.py

Code ID	MAP-015
File Name	glb-to-obj-indexed.py
File Path	SanctiWalkAR/Tools/glb-to-obj-indexed.py
Language / Type	Python
Lines	200
Functions / Classes	texture_of (function); decimate (function); main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/glb-to-obj-indexed.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Turn a .glb into an INDEXED .obj + .mtl + texture that Unity imports.
>
> ## Why this exists beside glb-to-obj.py
>
> glb-to-obj.py writes one vertex per triangle corner: a 400,000 triangle mesh leaves as 1,200,000 vertices with nothing shared. OBJ allows shared vertices and the mesh already has them - the scanned statues are about 245,000 unique vertices - so that is five times the vertex data for the same shape.
>
> On a phone that is the difference between fitting and not. Two statues, a room scan and a logo at 1.2M vertices each is what was running the device out of memory and killing the app mid-walk. It also made a 129 MB file out of a 30 MB one.
>
> Shared vertices additionally mean shared normals, which is what makes a scanned surface read as smooth rather than faceted.
>
> ## What it does beyond the format swap
>
> --ground drops the mesh so its lowest point sits at y = 0, because a station position is where the object STANDS. A pivot anywhere else buries it in the floor or hangs it in the air.
>
> --centre centres x and z, so "put it here" means the object is there.

Complete Source Code — glb-to-obj-indexed.py

```
"""Turn a .glb into an INDEXED .obj + .mtl + texture that Unity imports.

## Why this exists beside glb-to-obj.py

glb-to-obj.py writes one vertex per triangle corner: a 400,000 triangle mesh
leaves as 1,200,000 vertices with nothing shared. OBJ allows shared vertices
and the mesh already has them - the scanned statues are about 245,000 unique
```

```
vertices - so that is five times the vertex data for the same shape.

On a phone that is the difference between fitting and not. Two statues, a
room scan and a logo at 1.2M vertices each is what was running the device out
of memory and killing the app mid-walk. It also made a 129 MB file out of a
30 MB one.

Shared vertices additionally mean shared normals, which is what makes a
scanned surface read as smooth rather than faceted.

## What it does beyond the format swap

  --ground   drops the mesh so its lowest point sits at y = 0, because a
             station position is where the object STANDS. A pivot anywhere
             else buries it in the floor or hangs it in the air.

  --centre   centres x and z, so "put it here" means the object is there.

  --faces N  quadric decimation to roughly N triangles, carrying UVs and
             recomputing smooth normals afterwards. Skipped if the mesh is
             already under N.

  --tex N    longest texture edge, default 2048. Photogrammetry arrives at
             8192, which is 16x the pixels for detail no phone shows.

Usage:
  python glb-to-obj-indexed.py in.glb out.obj [--ground] [--centre]
                               [--faces 400000] [--tex 2048]
"""
import os
import sys

import numpy as np
import trimesh
from PIL import Image


def texture_of(material):
    """The base colour image, whichever material class glTF produced."""
    for attr in ("baseColorTexture", "image", "emissiveTexture"):
        img = getattr(material, attr, None)
        if img is not None:
            return img
    return None


def decimate(mesh, target_faces):
    """Quadric decimation that keeps UVs.

    fast_simplification works on positions and faces alone, so the UVs are
    carried across with the index map it reports: replay_simplification maps
    each ORIGINAL vertex to its surviving index, and the UV goes with it.
    Mapping the other way round silently produces a UV array of the wrong
    length, which exporters then drop - that is how the statues first
    shipped with no texture at all.
    """
    import fast_simplification

    if len(mesh.faces) <= target_faces:
```

```
        return mesh

    uv0 = np.asarray(mesh.visual.uv)
    ratio = 1.0 - (target_faces / len(mesh.faces))
    _, _, collapses = fast_simplification.simplify(
        mesh.vertices, mesh.faces, ratio, return_collapses=True)
    verts, faces, index_map = fast_simplification.replay_simplification(
        mesh.vertices, mesh.faces, collapses)

    index_map = np.asarray(index_map)
    kept = index_map >= 0
    uv = np.zeros((len(verts), 2), dtype=uv0.dtype)
    uv[index_map[kept]] = uv0[np.nonzero(kept)[0]]

    out = trimesh.Trimesh(vertices=verts, faces=faces, process=False)
    out.visual = trimesh.visual.TextureVisuals(uv=uv, material=mesh.visual.material)
    return out


def main():
    if len(sys.argv) < 3:
        print(__doc__)
        return 1

    src, dst = sys.argv[1], sys.argv[2]
    args = sys.argv[3:]
    ground = "--ground" in args
    centre = "--centre" in args
    # glTF is right-handed, Unity is left-handed. On a statue
    # nobody can tell; the logo is lettering and arrived on the
    # wall reading backwards.
    flip_x = "--flip-x" in args

    def opt(name, default):
        return int(args[args.index(name) + 1]) if name in args else default

    target_faces = opt("--faces", 400_000)
    max_tex = opt("--tex", 2048)

    mesh = trimesh.load(src, force="mesh", process=False)
    print("  in      %s tris, %s verts" % (f"{len(mesh.faces):,}", f"{len(mesh.vertices):,}"))

    mesh = decimate(mesh, target_faces)

    # Normals AFTER decimation and on the welded mesh, so neighbouring
    # triangles share them and the surface shades smoothly.
    normals = mesh.vertex_normals
    uv = np.asarray(mesh.visual.uv)

    verts = np.array(mesh.vertices, dtype=np.float64)
    faces_out = np.asarray(mesh.faces)

    if flip_x:
        # Negating an axis also reverses which way every triangle
        # faces, so the winding is reversed to match - otherwise
        # the model renders inside out.
        verts = verts * np.array([-1.0, 1.0, 1.0])
        normals = normals * np.array([-1.0, 1.0, 1.0])
        faces_out = faces_out[:, ::-1]
```

```
    print("  flipped X, winding reversed")

if ground or centre:
    lo = verts.min(axis=0)
    hi = verts.max(axis=0)
    shift = np.zeros(3)
    if ground:
        shift[1] = -lo[1]
    if centre:
        shift[0] = -(lo[0] + hi[0]) / 2.0
        shift[2] = -(lo[2] + hi[2]) / 2.0
    verts = verts + shift
    print("  moved   %.3f, %.3f, %.3f  (base to y=0, centred)" % tuple(shift))

size = verts.max(axis=0) - verts.min(axis=0)
print("  size    %.2f x %.2f x %.2f m" % tuple(size))

out_dir = os.path.dirname(os.path.abspath(dst)) or "."
os.makedirs(out_dir, exist_ok=True)
stem = os.path.splitext(os.path.basename(dst))[0]

img = texture_of(mesh.visual.material)
tex_name = None
if img is not None:
    if max(img.size) > max_tex:
        img = img.convert("RGB").resize((max_tex, max_tex), Image.LANCZOS)
    tex_name = stem + "_tex0.jpg"
    img.convert("RGB").save(os.path.join(out_dir, tex_name), quality=90)
    print("  texture -> %s (%d KB)"
          % (tex_name, os.path.getsize(os.path.join(out_dir, tex_name)) // 1024))

mtl_name = stem + ".mtl"
with open(os.path.join(out_dir, mtl_name), "w", encoding="utf-8") as f:
    f.write("newmtl mat0\nKd 1.000 1.000 1.000\nKa 0 0 0\nKs 0 0 0\nd 1.0\nillum 1\n")
    if tex_name:
        f.write("map_Kd %s\n" % tex_name)

# One v, one vt and one vn per UNIQUE vertex; faces reference them. This
# is the whole point of the file.
faces = faces_out
with open(dst, "w", encoding="utf-8") as f:
    f.write("# generated by Tools/glb-to-obj-indexed.py - shared vertices\n")
    f.write("mtllib %s\n" % mtl_name)
    for v in verts:
        f.write("v %.6f %.6f %.6f\n" % (v[0], v[1], v[2]))
    for t in uv:
        # V is written exactly as trimesh reports it.
        #
        # This used to write 1.0 - v, on the reasoning that glTF puts
        # the texture origin at the top and OBJ at the bottom. That is
        # true of the FILE, but not of what arrives here: trimesh's glTF
        # loader has already converted the UVs to its own bottom-left
        # convention on load. Flipping again was a second flip that
        # cancelled the first, so the OBJ went out carrying glTF-space
        # UVs and every texture was sampled upside down.
        #
        # On a photogrammetry atlas - thousands of small islands - an
        # upside-down lookup does not read as 'flipped', it reads as
        # camouflage, which is why it looked like broken geometry
```

```
            # rather than a broken texture coordinate.
            f.write("vt %.6f %.6f\n" % (t[0], t[1]))
        for n in normals:
            f.write("vn %.6f %.6f %.6f\n" % (n[0], n[1], n[2]))
        f.write("usemtl mat0\n")
        for a, b, c in faces + 1:          # OBJ indices are 1-based
            f.write("f %d/%d/%d %d/%d/%d %d/%d/%d\n" % (a, a, a, b, b, b, c, c, c))

    print("  %s  %s tris, %s verts, %.1f MB"
          % (os.path.basename(dst), f"{len(faces):,}", f"{len(verts):,}",
             os.path.getsize(dst) / 1e6))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

3.3.6 MAP-016 — optimise-scan.py

Code ID	MAP-016
File Name	optimise-scan.py
File Path	SanctiWalkAR/Tools/optimise-scan.py
Language / Type	Python
Lines	151
Functions / Classes	mb (function); shrink_textures (function); main (function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Tools/optimise-scan.py. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Make a photogrammetry scan small enough for a phone.
>
> A scan comes off the capture pipeline at two to three million triangles with full-resolution textures, because that is what the photographs support. A phone cannot hold it. The Infinix handset this project is tested on is already killed by Android's low-memory killer on ordinary screens, and a three-million-triangle mesh with 8K textures decodes to far more memory than the file size suggests.
>
> The target here is the model that already ships: san-roque at 200k triangles and 24.7 MB. That one works on the phone, so it is the budget, not a guess.
>
> Two reductions, both lossy and both chosen because they are invisible at the distance a visitor stands from a station:
>
> Triangles. A statue seen from two metres on a 6-inch screen cannot resolve a two-million-triangle surface. Quadric decimation keeps the silhouette and the large forms and spends the budget there.
>
> Textures. The colour is what a visitor actually sees, so textures are cut less aggressively than geometry - down to a maximum edge, not to a fixed small size.
>
> Usage: python optimise-scan.py in.glb out.glb [--faces 200000] [--tex 2048]

Complete Source Code — optimise-scan.py

```
"""Make a photogrammetry scan small enough for a phone.

A scan comes off the capture pipeline at two to three million triangles
with full-resolution textures, because that is what the photographs
support. A phone cannot hold it. The Infinix handset this project is
tested on is already killed by Android's low-memory killer on ordinary
screens, and a three-million-triangle mesh with 8K textures decodes to
far more memory than the file size suggests.

The target here is the model that already ships: san-roque at 200k
```

```
triangles and 24.7 MB. That one works on the phone, so it is the
budget, not a guess.

Two reductions, both lossy and both chosen because they are invisible
at the distance a visitor stands from a station:

  Triangles. A statue seen from two metres on a 6-inch screen cannot
  resolve a two-million-triangle surface. Quadric decimation keeps the
  silhouette and the large forms and spends the budget there.

  Textures. The colour is what a visitor actually sees, so textures are
  cut less aggressively than geometry - down to a maximum edge, not to
  a fixed small size.

Usage:
  python optimise-scan.py in.glb out.glb [--faces 200000] [--tex 2048]
"""
import argparse
import io
import os
import sys

import fast_simplification
import numpy as np
import trimesh
from PIL import Image


def mb(path):
    return os.path.getsize(path) / 1048576


def shrink_textures(mesh, max_edge):
    """Resizes every texture in place, longest edge to max_edge."""
    shrunk = []
    visual = getattr(mesh, "visual", None)
    material = getattr(visual, "material", None)
    if material is None:
        return shrunk

    for attr in ("image", "baseColorTexture", "emissiveTexture",
                 "normalTexture", "occlusionTexture",
                 "metallicRoughnessTexture"):
        img = getattr(material, attr, None)
        if not isinstance(img, Image.Image):
            continue
        if max(img.size) <= max_edge:
            continue
        before = img.size
        scale = max_edge / max(img.size)
        img = img.resize(
            (max(1, round(img.width * scale)), max(1, round(img.height * scale))),
            Image.LANCZOS,
        )
        if img.mode not in ("RGB", "RGBA"):
            img = img.convert("RGB")
        setattr(material, attr, img)
        shrunk.append(f"{attr} {before[0]}x{before[1]} -> {img.size[0]}x{img.size[1]}")
    return shrunk
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("src")
    ap.add_argument("dst")
    ap.add_argument("--faces", type=int, default=200_000,
                    help="triangle budget; the shipped model is 200k")
    ap.add_argument("--tex", type=int, default=2048,
                    help="longest texture edge")
    args = ap.parse_args()

    print(f"loading {args.src} ({mb(args.src):.1f} MB)")
    scene = trimesh.load(args.src, process=False)

    meshes = ([scene] if isinstance(scene, trimesh.Trimesh)
              else list(scene.geometry.values()))
    total = sum(len(m.faces) for m in meshes)
    print(f"  {len(meshes)} mesh(es), {total:,} triangles")

    # The budget is shared out in proportion to size, so a model made of
    # several parts does not have its small parts decimated into nothing
    # while one large part keeps everything.
    for mesh in meshes:
        share = max(1, round(args.faces * len(mesh.faces) / total))
        if len(mesh.faces) > share:
            before = len(mesh.faces)
            reduction = 1.0 - (share / len(mesh.faces))

            uv = getattr(mesh.visual, "uv", None)
            uv = None if uv is None else np.asarray(uv)

            # simplify(..., return_collapses=True) then replay, rather than
            # trimesh's own decimation, ONLY because of the UVs. The plain
            # call returns a new mesh with no texture coordinates at all,
            # and the model then arrives on the phone as a grey blob -
            # which looks like a loading failure and is hard to trace back
            # to here. replay hands back a mapping from every original
            # vertex to its surviving one, which is what carries them.
            _, _, collapses = fast_simplification.simplify(
                np.asarray(mesh.vertices, dtype=np.float32),
                np.asarray(mesh.faces, dtype=np.int32),
                reduction,
                return_collapses=True,
            )
            new_pts, new_faces, mapping = fast_simplification.replay_simplification(
                np.asarray(mesh.vertices, dtype=np.float32),
                np.asarray(mesh.faces, dtype=np.int32),
                collapses,
            )

            new_uv = None
            if uv is not None:
                new_uv = np.zeros((len(new_pts), 2), dtype=np.float64)
                keep = mapping >= 0
                new_uv[mapping[keep]] = uv[: len(mapping)][keep]

            material = getattr(mesh.visual, "material", None)
            mesh.vertices = new_pts
```

```
            mesh.faces = new_faces
            if new_uv is not None and material is not None:
                mesh.visual = trimesh.visual.TextureVisuals(
                    uv=new_uv, material=material)

            print(f"  triangles {before:,} -> {len(mesh.faces):,}"
                  f"{' (UVs kept)' if new_uv is not None else ' (no UVs in source)'}")

        for line in shrink_textures(mesh, args.tex):
            print(f"  texture {line}")

    out = scene if not isinstance(scene, trimesh.Trimesh) else scene
    data = trimesh.exchange.gltf.export_glb(
        out if isinstance(out, trimesh.Scene) else trimesh.Scene(out))
    with open(args.dst, "wb") as f:
        f.write(data)

    print(f"wrote {args.dst} ({mb(args.dst):.1f} MB) "
          f"from {mb(args.src):.1f} MB")


if __name__ == "__main__":
    sys.exit(main())
```

Part 4 — Unity AR Project

4.1 AR Runtime Scripts

4.1.1 AR-001 — AdminAccess.cs

Code ID	AR-001
File Name	AdminAccess.cs
File Path	SanctiWalkAR/Assets/Scripts/AdminAccess.cs
Language / Type	C#
Lines	110
Functions / Classes	AdminAccess (public class)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/AdminAccess.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AdminAccess.cs

```
using UnityEngine;

/// <summary>
/// Which phones may set a church up.
///
/// One APK ships to everybody. AR SETUP writes the positions every later
/// visitor will see, so it cannot be a button on the screen a visitor is
/// looking at - and it cannot be behind a code either, because a code
/// compiled into an APK anyone can unzip is not a secret, it is a
/// formality. The devices allowed to do it are named instead.
///
/// The cost is honest and worth stating: adding a second admin phone needs
/// a rebuild. That is the right trade for a capstone with one setup phone,
/// and the day it stops being the right trade, this is the one place that
/// changes - a list here becomes a list fetched from the backend, and
/// nothing else in the app has to know.
///
/// ANDROID_ID rather than the serial number: the serial has needed a
/// privileged permission since Android 10 and comes back as "unknown" to an
/// ordinary app, so a serial allowlist would have quietly matched nobody.
/// ANDROID_ID is readable, is stable for the life of an install, and is
/// per-app-signing-key, so it identifies this phone running THIS app.
/// </summary>
public static class AdminAccess
{
    /// <summary>
```

```
    /// Phones that may run AR SETUP.
    ///
    /// Infinix X6887 - the setup phone.
    ///
    /// Read it from the APP's own log, NOT from adb:
    ///   adb logcat -s Unity:* | grep AdminAccess
    ///
    /// `adb shell settings get secure android_id` reports a DIFFERENT value
    /// and putting that one here matches nobody. Since Android 8 the id is
    /// scoped per app signing key, so the shell sees the shell's id and this
    /// app sees its own. That was worth one wasted build to learn: the
    /// allowlist looked correct, compiled, installed, and silently made the
    /// setup phone a visitor.
    /// </summary>
    static readonly string[] k_AdminDevices =
    {
        "cc66541fc50f5ed1",
    };

    static bool? s_IsAdmin;
    static string s_DeviceId;

    /// <summary>This phone's ANDROID_ID, or "" where there is no Android to ask.</summary>
    public static string DeviceId
    {
        get
        {
            if (s_DeviceId != null)
                return s_DeviceId;

            s_DeviceId = "";
#if UNITY_ANDROID && !UNITY_EDITOR
            try
            {
                using var player = new AndroidJavaClass("com.unity3d.player.UnityPlayer");
                using var activity = player.GetStatic<AndroidJavaObject>("currentActivity");
                using var resolver = activity.Call<AndroidJavaObject>("getContentResolver");
                using var secure = new AndroidJavaClass("android.provider.Settings$Secure");
                s_DeviceId = secure.CallStatic<string>("getString", resolver, "android_id") ?? "";
            }
            catch (System.Exception e)
            {
                Debug.LogWarning($"[AdminAccess] could not read ANDROID_ID: {e.Message}");
            }
#endif
            return s_DeviceId;
        }
    }

    /// <summary>
    /// True on a setup phone.
    ///
    /// True in the Editor as well, unconditionally: the desktop test build
    /// is how the flow gets walked without a phone, and locking the editor
    /// out of it would mean every change to the setup screens could only be
    /// checked by making an APK.
    /// </summary>
    public static bool IsAdmin
    {
```

```
        get
        {
            if (s_IsAdmin.HasValue)
                return s_IsAdmin.Value;

#if UNITY_EDITOR
            s_IsAdmin = true;
#else
            var id = DeviceId;
            s_IsAdmin = false;
            foreach (var allowed in k_AdminDevices)
            {
                if (!string.IsNullOrEmpty(id) &&
                    string.Equals(id, allowed, System.StringComparison.OrdinalIgnoreCase))
                {
                    s_IsAdmin = true;
                    break;
                }
            }
            Debug.Log($"[AdminAccess] device {id} -> {(s_IsAdmin.Value ? "ADMIN" : "visitor")}");
#endif
            return s_IsAdmin.Value;
        }
    }
}
```

4.1.2 AR-002 — AppLanguage.cs

Code ID	AR-002
File Name	AppLanguage.cs
File Path	SanctiWalkAR/Assets/Scripts/AppLanguage.cs
Language / Type	C#
Lines	71
Functions / Classes	AppLanguage (public class); Toggle (public method); Pick (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/AppLanguage.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AppLanguage.cs

```
using UnityEngine;

/// <summary>
/// Which language the walk is being read and spoken in.
///
/// The parish wrote its history in Tagalog and the two statue descriptions
/// in English; both now exist in both languages (sites/mhcp.i18n.json). A
/// visitor picks one and everything follows it - the cards, the bottom bar,
/// and what the phone reads aloud.
///
/// Stored in PlayerPrefs so the choice survives the walk being restarted,
/// which happens often: the AR app is entered by deep link and killed
/// rather than backgrounded.
/// </summary>
public static class AppLanguage
{
    const string k_Key = "sanctiwalk.lang";

    /// <summary>"tl" or "en".</summary>
    public static string Current
    {
        get => PlayerPrefs.GetString(k_Key, "tl");
        set
        {
            if (value != "tl" && value != "en") return;
            PlayerPrefs.SetString(k_Key, value);
            PlayerPrefs.Save();
            Changed?.Invoke();
        }
    }

    public static bool IsTagalog => Current == "tl";
```

```
    /// <summary>Raised after a change, so open UI can re-read its text.</summary>
    public static event System.Action Changed;

    public static void Toggle() => Current = IsTagalog ? "en" : "tl";

    /// <summary>
    /// The language being spoken right now.
    ///
    /// The button used to name the language it would switch TO, on the
    /// reasoning that a button should say what it does. In the church it
    /// read as a label: the narration was in English and the top of the
    /// screen said "Tagalog", so the only way to find out what you were
    /// listening to was to listen to it - and the control meant to make
    /// the language clear was the thing making it unclear.
    ///
    /// It names the current language now. Tapping it still swaps, and a
    /// label that changes when you tap it teaches that in one tap.
    /// </summary>
    public static string CurrentLabel => IsTagalog ? "Tagalog" : "English";

    /// <summary>The name of the other language.</summary>
    public static string OtherLabel => IsTagalog ? "English" : "Tagalog";

    /// <summary>
    /// Picks the right one of a pair, falling back to whatever exists.
    ///
    /// A station with no translation yet must still show something: an
    /// empty card is worse than a card in the wrong language.
    /// </summary>
    public static string Pick(string tl, string en, string fallback = null)
    {
        var wanted = IsTagalog ? tl : en;
        if (!string.IsNullOrEmpty(wanted)) return wanted;
        var other = IsTagalog ? en : tl;
        if (!string.IsNullOrEmpty(other)) return other;
        return fallback ?? string.Empty;
    }
}
```

4.1.3 AR-003 — ArFlowUI.cs

Code ID	AR-003
File Name	ArFlowUI.cs
File Path	SanctiWalkAR/Assets/Scripts/ArFlowUI.cs
Language / Type	C#
Lines	1423
Functions / Classes	ArFlowUI (public class); Create (public method); Build (method); Show (public method); Refresh (public method); Update (method); BuildSplash (method); ScanFrame (method); TapWordmark (method); BuildDetected (method); LeaveAr (method); BuildExplore (method); BuildTutorial (method); BuildTutorialDots (method); OpenTutorial (method); AdvanceTutorial (method); Greet (method); CloseTutorial (method); ReplayTutorial (public method); RefreshTutorial (method); BuildSetupScan (method); BuildSetupChoose (method); BuildObjectRows (method); BuildSetupPlace (method); BuildSetupSaved (method); BuildSetupComplete (method); BeginPlacing (method); Move (method); Turn (method); ResetOne (method); SaveOne (method); SetupHeader (method); Gap (method); Height (method); InColumn (method); AutoNarrate (method); SpeakCurrent (method); SetSpeakerPlaying (method); RefreshStationTitle (method); Tick (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/ArFlowUI.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ArFlowUI.cs

```
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// The screens a person actually sees: three steps to set a church up, and
/// four to walk one.
///
/// This replaces the IMGUI overlay for everything except debugging. That
/// overlay was a developer readout with the controls buried in it - session
/// state, library counts, tracking flags, coordinates - and it was the first
/// thing a visitor met. What matters more than the styling is that the
/// ORDER is now fixed: scan, choose, place, save, next. The old calibrator
/// put every control on screen at once and left the operator to work out
/// which mattered, which is why calibrating felt like operating a debugger.
///
/// Nothing about localization or coordinates lives here. The origin is
/// still ChurchOriginTest's, the maths is still StationCalibrator's, and
/// this only decides what is on screen and which of their methods a button
```

```
/// calls. That separation is deliberate: a UI rewrite must not be able to
/// move a station by a millimetre.
/// </summary>
[RequireComponent(typeof(Canvas))]
public class ArFlowUI : MonoBehaviour
{
    public enum Screen
    {
        Splash,          // SANCTIWALK AR - scan the marker
        Detected,        // Church detected - start AR
        Explore,         // the camera, with the bottom bar
        SetupScan,       // step 1 of 3
        SetupChoose,     // step 2 of 3
        SetupPlace,      // step 3 of 3
        SetupSaved,      // position saved
        SetupComplete,   // setup complete
    }

    // The mockup's palette. Named rather than inlined so a change is one
    // edit and so the intent of each colour is legible.
    static readonly Color Navy      = new Color32(0x14, 0x2C, 0x56, 0xFF);
    static readonly Color NavyDeep  = new Color32(0x0C, 0x1B, 0x38, 0xFF);
    static readonly Color Blue      = new Color32(0x1B, 0x74, 0xE4, 0xFF);
    static readonly Color Ink       = new Color32(0x14, 0x17, 0x1F, 0xFF);
    static readonly Color Muted     = new Color32(0x6B, 0x72, 0x80, 0xFF);
    static readonly Color Paper     = new Color32(0xFF, 0xFF, 0xFF, 0xFF);
    static readonly Color Hairline  = new Color32(0xE2, 0xE8, 0xF0, 0xFF);
    static readonly Color Good      = new Color32(0x22, 0xA5, 0x5A, 0xFF);

    ChurchOriginTest m_Host;
    Font m_Heading, m_Body;
    RectTransform m_Root;

    Screen m_Screen = Screen.Splash;
    readonly Dictionary<Screen, GameObject> m_Panels = new Dictionary<Screen, GameObject>();

    // Live text that changes between renders.
    Text m_ScanStatusWaiting, m_ScanStatusFound, m_DetectedName;
    Text m_PlaceName, m_PlaceReadout, m_SavedName, m_StepLabel;
    Text m_SplashStatus, m_SavedProof;
    GameObject m_ObjectListContent;
    GameObject m_AdminEntry;
    GameObject m_VisitorBar;
    GameObject m_Tutorial, m_TutButton;
    Text m_TutTitle, m_TutBody, m_TutButtonLabel;
    readonly List<Image> m_TutDots = new List<Image>();
    int m_TutStep;
    string m_SpokenStationId;
    string m_PendingStationId;
    float m_DetectedSince;
    int m_WordmarkTaps;
    float m_LastWordmarkTap;
    bool m_Greeted;
    float m_PendingSince;
    Text m_StationTitle, m_LangLabel;
    Image m_SpeakerIcon;
    Image m_SpeakerBg;

    /// <summary>Which station the operator is placing, as an index into the host's list.</summary>
```

```
int m_Placing;

public Screen Current => m_Screen;

/// <summary>True while a setup step owns the screen, so taps are not drags.</summary>
public bool InSetup =>
    m_Screen == Screen.SetupScan || m_Screen == Screen.SetupChoose ||
    m_Screen == Screen.SetupPlace || m_Screen == Screen.SetupSaved ||
    m_Screen == Screen.SetupComplete;

/// <summary>True only on the one screen where dragging a station is meant.</summary>
public bool IsPlacing => m_Screen == Screen.SetupPlace;

public static ArFlowUI Create(ChurchOriginTest host)
{
    var go = new GameObject("ArFlowUI",
        typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRaycaster));
    var ui = go.AddComponent<ArFlowUI>();
    ui.m_Host = host;
    ui.Build();
    return ui;
}

Font LoadFont(string name)
{
    var f = Resources.Load<Font>(name);
    if (f != null) return f;
    return Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
}

void Build()
{
    m_Heading = LoadFont("Montserrat-SemiBold");
    m_Body = LoadFont("Poppins-Regular");

    var canvas = GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    // Above MuseumUI's canvas, so a setup screen covers the station
    // cards rather than fighting them for the same pixels.
    canvas.sortingOrder = 10;

    var scaler = GetComponent<CanvasScaler>();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(390, 844);
    scaler.matchWidthOrHeight = 0.5f;

    if (FindAnyObjectByType<UnityEngine.EventSystems.EventSystem>() == null)
    {
        new GameObject("EventSystem",
            typeof(UnityEngine.EventSystems.EventSystem),
            typeof(UnityEngine.EventSystems.StandaloneInputModule));
    }

    m_Root = (RectTransform)transform;

    BuildSplash();
    BuildDetected();
    BuildExplore();
    BuildSetupScan();
```

```
    BuildSetupChoose();
    BuildSetupPlace();
    BuildSetupSaved();
    BuildSetupComplete();
    BuildTutorial();

    Show(Screen.Splash);
}

// ------------------------------------------------------------ plumbing

public void Show(Screen screen)
{
    m_Screen = screen;
    foreach (var kv in m_Panels)
        if (kv.Value != null) kv.Value.SetActive(kv.Key == screen);

    // The station cards belong to the explore screen alone. On a setup
    // step they would sit behind the panel and reappear at the edges.
    m_Host.SetMuseumVisible(screen == Screen.Explore);

    // And dragging belongs to the place screen alone. Every exit from
    // setup goes through here, so there is no route that can leave the
    // calibrator holding a station.
    m_Host.SetCalibrating(screen == Screen.SetupPlace);

    if (screen == Screen.SetupPlace)
        BeginPlacing(m_Placing);

    // The tutorial belongs to the camera screen and to nobody else. It
    // is opened here rather than from Update so it cannot reappear
    // behind a setup step, and only the first time, because a notice
    // you have already read is in the way of the church.
    if (screen == Screen.Detected) m_DetectedSince = Time.unscaledTime;

    if (screen == Screen.Explore && !TutorialSeen) OpenTutorial();
    else if (m_Tutorial != null) m_Tutorial.SetActive(false);

    // The greeting. A visitor arrives here facing the printed sheet
    // with the whole room behind them, so the first thing the tour
    // says is which way to look. Spoken once per run, and only when
    // no notice is covering the screen.
    if (screen == Screen.Explore) Greet();

    Refresh();
}

/// <summary>Per-frame text: statuses, distances, coordinates.</summary>
public void Refresh()
{
    var found = m_Host.MarkerDetected;

    if (m_ScanStatusWaiting != null)
    {
        m_ScanStatusWaiting.color = found ? Muted : Ink;
        m_ScanStatusFound.color = found ? Good : Hairline;
    }

    if (m_DetectedName != null)
```

```
        m_DetectedName.text = m_Host.ParishDisplayName;

    // On every non-setup screen, not just Explore.
    //
    // It used to appear on Explore alone, which is only reachable once a
    // marker has been read - so on the one screen where the operator was
    // stuck, the button that would have got them out was hidden. An
    // admin control that is only there once everything already works is
    // not much of a control.
    if (m_AdminEntry != null) m_AdminEntry.SetActive(false);

    // The visitor bar is the one thing every visitor gets, so it is
    // shown on Explore regardless of who is holding the phone.
    if (m_VisitorBar != null)
        m_VisitorBar.SetActive(m_Screen == Screen.Explore);

    if (m_Screen == Screen.SetupPlace && m_PlaceReadout != null)
    {
        var station = m_Host.StationAt(m_Placing);
        if (station != null)
        {
            var p = m_Host.Calibrator.PositionOf(station.stationId);
            m_PlaceName.text = $"Place: {station.stationName}";
            m_PlaceReadout.text =
                $"{p.x:F2},  {p.y:F2},  {p.z:F2}   ·   {m_Host.MetresToStation(station):F1} m away";
        }
    }

    if (m_Screen == Screen.Explore)
    {
        RefreshStationTitle();
        // The engine finishes on its own; nothing calls back, so the
        // icon is driven from its state rather than from the tap.
        SetSpeakerPlaying(DeviceSpeech.IsSpeaking());
        AutoNarrate();
    }

    if (m_Tutorial != null && m_Tutorial.activeSelf)
        RefreshTutorial();

    if (m_Screen == Screen.Splash && m_SplashStatus != null)
        m_SplashStatus.text = m_Host.MultiSetBusy ? "Looking for the church…" : "";
}

void Update()
{
    if (m_Screen == Screen.Splash && m_Host.OriginSet)
    {
        Show(Screen.Detected);
        return;
    }

    // Hold still, then go. A minimum dwell so "Church detected" is
    // actually read rather than flashing past, and no button.
    if (m_Screen == Screen.Detected && m_Host.OriginSet
        && Time.unscaledTime - m_DetectedSince > 1.4f)
    {
        Show(Screen.Explore);
        return;
```

```
    }

    if (m_Screen == Screen.Splash && m_Host.MarkerDetected)
    {
        // The marker is read by ARCore continuously, so "scanning" is
        // just waiting. Moving on by itself is what the mockup shows,
        // and asking for a tap to confirm something the phone already
        // knows would be ceremony.
        Show(Screen.Detected);
        return;
    }

    Refresh();
}

// ------------------------------------------------------- the screens

void BuildSplash()
{
    var panel = Sheet(Screen.Splash, NavyDeep);
    var col = Column(panel, 300, 210);

    var wordmark = Title(col, "SANCTIWALK", 28, Paper);
    Clickable(wordmark.gameObject, TapWordmark);
    Gap(col, 8);
    var museum = Body(col, "AR MUSEUM", 12, new Color(1, 1, 1, 0.55f));
    museum.fontStyle = FontStyle.Bold;

    Gap(col, 34);
    ScanFrame(col);
    Gap(col, 30);

    Body(col, "Point your camera at the church marker\nto start AR.", 14, new Color(1, 1, 1, 0.82f));
    Gap(col, 12);
    m_SplashStatus = Body(col, "", 12, new Color(1, 1, 1, 0.6f));

    // "Scan Marker" is gone. It went to the operator's setup flow, but
    // it sat where a visitor reads "the button that starts this", and
    // there was never anything to press: ARCore reads the marker
    // continuously and Update moves the screen on by itself the moment
    // it lands. A button for something the phone is already doing is
    // one more thing in the way of the church.
    Gap(col, 22);
    var fallback = Body(col, "No marker? Start here", 13, new Color(1, 1, 1, 0.75f));
    fallback.fontStyle = FontStyle.Bold;
    Height(fallback.gameObject, 34);
    Clickable(fallback.gameObject, () => m_Host.LocalizeWithMultiSet());
}

/// <summary>
/// Four corner brackets, the proportions of the printed sheet.
///
/// A viewfinder rather than a picture: it says what shape to look for
/// and where to put it, which is the one thing this screen is for.
/// </summary>
void ScanFrame(RectTransform col)
{
    var row = new GameObject("ScanFrame", typeof(RectTransform));
    InColumn(col, row, 150);
```

```
    var rt = (RectTransform)row.transform;

    const float w = 114f, h = 150f, len = 30f, thick = 3f;
    var ink = new Color(1, 1, 1, 0.5f);
    foreach (var sx in new[] { -1f, 1f })
    foreach (var sy in new[] { -1f, 1f })
    {
        Box(rt, new Vector2(0.5f, 0.5f),
            new Vector2(sx * (w / 2 - len / 2), sy * (h / 2 - thick / 2)),
            new Vector2(len, thick), ink);
        Box(rt, new Vector2(0.5f, 0.5f),
            new Vector2(sx * (w / 2 - thick / 2), sy * (h / 2 - len / 2)),
            new Vector2(thick, len), ink);
    }
}

/// <summary>
/// Three taps on the wordmark opens the setup flow.
///
/// The taps have to be quick - the count resets after a second and a
/// half - so a pilgrim prodding the screen while they wait for the
/// marker cannot fall into it.
/// </summary>
void TapWordmark()
{
    if (Time.unscaledTime - m_LastWordmarkTap > 1.5f) m_WordmarkTaps = 0;
    m_LastWordmarkTap = Time.unscaledTime;

    if (++m_WordmarkTaps < 3) return;
    m_WordmarkTaps = 0;

    if (!AdminAccess.IsAdmin) return;
    m_Placing = 0;
    Show(m_Host.OriginSet ? Screen.SetupChoose : Screen.SetupScan);
}

void BuildDetected()
{
    var panel = Sheet(Screen.Detected, NavyDeep);
    var col = Column(panel, 300, 300);

    Tick(col, Good);
    Gap(col, 16);
    Title(col, "Church detected", 22, Paper);
    Gap(col, 10);
    m_DetectedName = Body(col, "", 16, Paper);
    m_DetectedName.fontStyle = FontStyle.Bold;

    // No "Start AR" button. The marker is read, the origin is being
    // resolved, and there is nothing for the visitor to decide - so
    // this says what to do with the phone while that finishes and
    // moves on by itself. Pressing a button to continue something
    // already underway is ceremony, and it is the step people stood
    // still at wondering whether it had worked.
    Gap(col, 22);
    Body(col, "Hold still while the tour loads", 14, new Color(1, 1, 1, 0.75f));
}

/// <summary>
```

```
    /// Back out of the AR tour, all the way out.
    ///
    /// The AR app has no launcher icon: a visitor arrives here from
    /// SanctiWalk and expects Back to take them to SanctiWalk, the way any
    /// other screen in that app would. Returning to our own splash leaves
    /// them on a screen for an app they do not know they are in, with no
    /// way out but the system gesture.
    ///
    /// Quitting pops this activity off the task, and Android shows whatever
    /// was underneath it - which is the app that launched us.
    /// </summary>
    void LeaveAr()
    {
#if UNITY_EDITOR
        Show(Screen.Splash);
#else
        Application.Quit();
#endif
    }

    void BuildExplore()
    {
        // Transparent: the camera IS the screen here.
        var panel = Panel(Screen.Explore, new Color(0, 0, 0, 0));

        var back = Round(Box(panel, new Vector2(0f, 1f), new Vector2(46, -54),
                             new Vector2(46, 46), Navy), 23);
        var backGlyph = Body((RectTransform)back.transform, "<", 20, Paper);
        Stretch((RectTransform)backGlyph.transform);
        backGlyph.alignment = TextAnchor.MiddleCenter;
        Clickable(back, LeaveAr);

        // Started here rather than on the first tap: the engine takes a
        // second or two to come up, and a speaker button that does nothing
        // the first time you press it reads as broken.
        DeviceSpeech.Init();

        // The bar is the whole of the visitor's UI: what you are looking
        // at, and a speaker to hear it.
        //
        // It used to carry History / Stations / Navigation on a second bar
        // above - three ways to jump around a walk that is meant to be
        // walked - and an AR SETUP button for one phone in the parish, all
        // stacked over the bottom third of the church. Those were the
        // operator's, and the operator has the setup flow.
        //
        // Navy, like the controls at the top, because a white slab was the
        // brightest thing on a dim camera feed and the first thing the eye
        // went to.
        var bar = Round(Box(panel, new Vector2(0.5f, 0f), new Vector2(0, 40),
                            new Vector2(344, 72), Navy), 24);
        m_VisitorBar = bar;

        var titleGo = Body((RectTransform)bar.transform, "", 14, Paper);
        titleGo.fontStyle = FontStyle.Bold;
        titleGo.alignment = TextAnchor.MiddleCenter;
        titleGo.horizontalOverflow = HorizontalWrapMode.Wrap;
        var trt = (RectTransform)titleGo.transform;
        trt.anchorMin = new Vector2(0f, 0f); trt.anchorMax = new Vector2(1f, 1f);
```

```
    trt.offsetMin = new Vector2(18f, 6f); trt.offsetMax = new Vector2(-62f, -6f);
    m_StationTitle = titleGo;

    // Tapping reads the station in the chosen language; tapping again
    // while it talks stops it, because a visitor who walks on wants it
    // to stop, not to finish.
    var spk = Round(Box(bar, new Vector2(1f, 0.5f), new Vector2(-38, 0),
                        new Vector2(44, 44), Blue), 22);
    m_SpeakerBg = spk.GetComponent<Image>();
    m_SpeakerIcon = SpeakerIcon((RectTransform)spk.transform, 26);
    Clickable(spk, SpeakCurrent);
    SetSpeakerPlaying(false);

    var help = Round(Box(panel, new Vector2(1f, 1f), new Vector2(-128, -54),
                         new Vector2(40, 40), Navy), 20);
    var helpGlyph = Body((RectTransform)help.transform, "?", 18, Paper);
    Stretch((RectTransform)helpGlyph.transform);
    helpGlyph.alignment = TextAnchor.MiddleCenter;
    Clickable(help, OpenTutorial);

    // One button that names the language it will switch TO, so it
    // reads as an action rather than a state.
    var lang = Round(Box(panel, new Vector2(1f, 1f), new Vector2(-54, -54),
                         new Vector2(84, 40), Navy), 20);
    m_LangLabel = Body((RectTransform)lang.transform, AppLanguage.CurrentLabel, 12, Paper);
    Stretch((RectTransform)m_LangLabel.transform);
    m_LangLabel.alignment = TextAnchor.MiddleCenter;
    Clickable(lang, () =>
    {
        DeviceSpeech.Stop();
        // The station the visitor is standing at is worth hearing
        // again in the language they just chose.
        m_SpokenStationId = null;
        AppLanguage.Toggle();
        m_LangLabel.text = AppLanguage.CurrentLabel;
        RefreshStationTitle();
        SetSpeakerPlaying(false);
        m_Host.RebuildStationsForLanguage();
    });

    // The operator's door is not on the screen at all any more.
    //
    // It was a navy pill on the splash, then a quieter one, and it
    // was still the loudest thing a visitor met: a button labelled
    // AR SETUP, for one phone in the parish, on the first screen of
    // a church tour. Moving it was not enough - nothing a visitor
    // can see should belong to the operator.
    //
    // It is three taps on the wordmark now. Invisible to a pilgrim,
    // and the one person who needs it knows where it is.
}

// ------------------------------------------------------------ tutorial

const string TutorialKey = "sanctiwalk.ar.tutorialSeen";

static bool TutorialSeen => PlayerPrefs.GetInt(TutorialKey, 0) == 1;

/// <summary>
```

```
/// Three cards telling a visitor where to touch, over the live camera.
///
/// It is here and not in SanctiWalk because the thing being explained
/// is this screen: a bar that fills itself in as you walk, a speaker,
/// and a language button. Read in the pew five minutes earlier none of
/// that has anything to point at.
///
/// The last card is the only one that commits to anything, so it is the
/// only one that waits: "Get started" stays dim until the church origin
/// is actually set, because until then there are no stations to walk to
/// and the tour would open onto an empty room.
/// </summary>
void BuildTutorial()
{
    // Parented to the root rather than to the Explore panel: Show()
    // deactivates panels, and the overlay has to outlive that so its
    // own rule about when it appears is the only one acting on it.
    var scrim = new GameObject("Tutorial", typeof(Image));
    scrim.transform.SetParent(m_Root, false);
    Stretch((RectTransform)scrim.transform);
    scrim.GetComponent<Image>().color = new Color(0.02f, 0.06f, 0.14f, 0.74f);
    m_Tutorial = scrim;

    var card = Box((RectTransform)scrim.transform, new Vector2(0.5f, 0f),
                   new Vector2(0, 36), new Vector2(344, 300), Paper);
    Round(card, 28);
    var cardRt = (RectTransform)card.transform;

    // Laid out at fixed offsets from the card's top edge, NOT with a
    // VerticalLayoutGroup. The first attempt used one and the slack
    // was shared out between the children, which pushed the dots, the
    // button and Skip off the bottom of the card - leaving a notice
    // over the camera with nothing on it to press. A tutorial you
    // cannot dismiss is worse than no tutorial, so the one thing this
    // layout guarantees is that the button is where it was put.
    m_TutTitle = TutText(cardRt, 26, 38, 20, Navy, m_Heading);
    m_TutTitle.fontStyle = FontStyle.Bold;

    m_TutBody = TutText(cardRt, 74, 96, 14, Muted, m_Body);
    m_TutBody.alignment = TextAnchor.UpperCenter;

    BuildTutorialDots(cardRt, 182);

    var button = Box(cardRt, new Vector2(0.5f, 1f), new Vector2(0, -206),
                     new Vector2(248, 46), Blue);
    var brt = (RectTransform)button.transform;
    Round(button, 23);
    m_TutButton = button;
    m_TutButtonLabel = Body(brt, "Next", 15, Paper);
    m_TutButtonLabel.fontStyle = FontStyle.Bold;
    Stretch((RectTransform)m_TutButtonLabel.transform);
    m_TutButtonLabel.alignment = TextAnchor.MiddleCenter;
    Clickable(button, AdvanceTutorial);

    var skip = TutText(cardRt, 260, 28, 13, Muted, m_Body);
    skip.text = "Skip";
    Clickable(skip.gameObject, CloseTutorial);

    scrim.SetActive(false);
```

```
}

/// <summary>A line of text pinned a fixed distance below the card's top.</summary>
Text TutText(RectTransform card, float top, float height, int size, Color colour, Font font)
{
    var t = Make(card, "", font, size, colour);
    t.alignment = TextAnchor.MiddleCenter;
    var rt = (RectTransform)t.transform;
    rt.anchorMin = new Vector2(0f, 1f);
    rt.anchorMax = new Vector2(1f, 1f);
    rt.pivot = new Vector2(0.5f, 1f);
    rt.anchoredPosition = new Vector2(0f, -top);
    rt.sizeDelta = new Vector2(-48f, height);
    return t;
}

void BuildTutorialDots(RectTransform card, float top)
{
    m_TutDots.Clear();
    var n = TutorialSteps.Length;
    const float spacing = 16f;
    for (var i = 0; i < n; i++)
    {
        var x = (i - (n - 1) / 2f) * spacing;
        var dot = Box(card, new Vector2(0.5f, 1f), new Vector2(x, -top),
                      new Vector2(8, 8), Hairline);
        Round(dot, 4);
        m_TutDots.Add(dot.GetComponent<Image>());
    }
}

// Each card names one thing on this screen and where it is. Nothing
// here describes a control that does not exist: the bar, the speaker
// and the language button are all built in BuildExplore above.
static readonly (string Title, string Body)[] TutorialSteps =
{
    ("Walk up to a station",
     "Stations stand where the statue, altar or image really is. "
     + "Walk towards one and its card appears on its own."),
    ("Tap a card to read it",
     "A card opens the full history of what you are looking at. "
     + "Tap anywhere outside it to close it again."),
    ("Listen, in your language",
     "The bar at the bottom names what is in front of you. Tap the "
     + "speaker to hear it read aloud, and EN / FIL at the top to "
     + "change the language."),
};

void OpenTutorial()
{
    m_TutStep = 0;
    m_Tutorial.SetActive(true);
    RefreshTutorial();
}

void AdvanceTutorial()
{
    if (m_TutStep < TutorialSteps.Length - 1)
    {
```

```
        m_TutStep++;
        RefreshTutorial();
        return;
    }

    // The last card. Nothing happens until the church is actually
    // located, and RefreshTutorial has already dimmed the button to
    // say so - this is the guard for a tap that lands anyway.
    if (!m_Host.OriginSet) return;
    CloseTutorial();
}

/// <summary>
/// The welcome, once per run.
///
/// Held back while the tutorial is up - the visitor is reading, and
/// two sets of instructions at once is neither - so CloseTutorial
/// calls this as well. Whichever of the two happens last is the one
/// that speaks.
/// </summary>
void Greet()
{
    if (m_Greeted) return;
    if (m_Tutorial != null && m_Tutorial.activeSelf) return;
    if (m_Screen != Screen.Explore) return;

    m_Greeted = true;
    m_GreetingSpeaking = true;
    DeviceSpeech.Speak(WelcomeLine(), AppLanguage.Current);
    SetSpeakerPlaying(true);
}

/// <summary>
/// True while the welcome is still being said.
///
/// The welcome was being spoken and then cut off mid-sentence. A
/// visitor who has just located is, by definition, standing
/// somewhere - usually right next to the first station - so
/// AutoNarrate found one immediately, called Stop, and started
/// reading it over the greeting. The welcome is the one line that
/// explains what the app is about to do, and it never finished.
///
/// It yields to a deliberate tap: StationNarration and the speaker
/// button both clear this, because a visitor who asks for something
/// has overruled the introduction.
/// </summary>
bool m_GreetingSpeaking;

void CloseTutorial()
{
    PlayerPrefs.SetInt(TutorialKey, 1);
    PlayerPrefs.Save();
    if (m_Tutorial != null) m_Tutorial.SetActive(false);
    Greet();
}

/// <summary>Re-opens the tutorial from the "?" button, once asked for.</summary>
public void ReplayTutorial()
{
```

```
    if (m_Screen != Screen.Explore) Show(Screen.Explore);
    OpenTutorial();
}

void RefreshTutorial()
{
    var step = TutorialSteps[m_TutStep];
    m_TutTitle.text = step.Title;
    m_TutBody.text = step.Body;

    for (var i = 0; i < m_TutDots.Count; i++)
        m_TutDots[i].color = i == m_TutStep ? Blue : Hairline;

    var last = m_TutStep == TutorialSteps.Length - 1;
    var ready = !last || m_Host.OriginSet;

    m_TutButtonLabel.text = !last ? "Next"
                          : ready ? "Get started"
                          : "Finding the church…";
    m_TutButton.GetComponent<Image>().color = ready ? Blue : Hairline;
    m_TutButtonLabel.color = ready ? Paper : Muted;
}

void BuildSetupScan()
{
    var panel = Sheet(Screen.SetupScan, Paper);
    SetupHeader(panel, "Step 1 of 3", true, () => Show(Screen.Splash));

    var col = Column(panel, 320, 96);
    Title(col, "Scan the church marker", 19, Ink);
    Gap(col, 8);
    Body(col, "Point your camera at the church marker\nto start the setup.", 13, Muted);
    Gap(col, 22);

    var card = Box(col, new Vector2(0.5f, 1f), Vector2.zero, new Vector2(300, 132), new Color32(0xF3, 0xF6, 0xFB, 0xFF));
    Round(card, 14);
    var hint = Body((RectTransform)card.transform, "Hold the marker in view", 13, Muted);
    Stretch((RectTransform)hint.transform);
    hint.alignment = TextAnchor.MiddleCenter;
    InColumn(col, card, 132);

    Gap(col, 18);

    var status = Box(col, new Vector2(0.5f, 1f), Vector2.zero, new Vector2(300, 84), new Color32(0xF7, 0xF9, 0xFC, 0xFF));
    Round(status, 12);
    var sc = Column((RectTransform)status.transform, 268, 12);
    m_ScanStatusWaiting = Body(sc, "Waiting for marker", 13, Ink);
    Gap(sc, 6);
    m_ScanStatusFound = Body(sc, "Marker detected", 13, Hairline);
    m_ScanStatusFound.fontStyle = FontStyle.Bold;
    InColumn(col, status, 84);

    Gap(col, 22);
    Pill(col, "Continue", Blue, Paper, () =>
    {
        if (m_Host.MarkerDetected || m_Host.OriginSet) Show(Screen.SetupChoose);
    });
}
```

```
void BuildSetupChoose()
{
    var panel = Sheet(Screen.SetupChoose, Paper);
    SetupHeader(panel, "Step 2 of 3", true, () => Show(Screen.Explore));

    var col = Column(panel, 330, 96);
    Title(col, "Choose what you want to place", 18, Ink);
    Gap(col, 6);
    Body(col, "Select an object from the list below.", 13, Muted);
    Gap(col, 16);

    m_ObjectListContent = col.gameObject;
    // Rows are built on first show, because the station list is only
    // known once an origin exists.
}

void BuildObjectRows()
{
    if (m_ObjectListContent == null) return;
    var col = (RectTransform)m_ObjectListContent.transform;

    // Clear anything from a previous origin.
    for (var i = col.childCount - 1; i >= 0; i--)
    {
        var child = col.GetChild(i);
        if (child.name.StartsWith("Row_")) Destroy(child.gameObject);
    }

    var stations = m_Host.StationList;
    if (stations == null) return;

    for (var i = 0; i < stations.Length; i++)
    {
        var index = i;
        var station = stations[i];

        var row = Box(col, new Vector2(0.5f, 1f), Vector2.zero, new Vector2(306, 54), Paper);
        row.name = $"Row_{station.stationId}";
        Round(row, 12);
        Outline(row, Hairline);

        var name = Body((RectTransform)row.transform, station.stationName, 15, Ink);
        var nrt = (RectTransform)name.transform;
        nrt.anchorMin = new Vector2(0, 0);
        nrt.anchorMax = new Vector2(1, 1);
        nrt.offsetMin = new Vector2(18, 0);
        nrt.offsetMax = new Vector2(-34, 0);
        name.alignment = TextAnchor.MiddleLeft;

        var chev = Body((RectTransform)row.transform, ">", 16, Muted);
        var crt = (RectTransform)chev.transform;
        crt.anchorMin = crt.anchorMax = new Vector2(1f, 0.5f);
        crt.anchoredPosition = new Vector2(-18, 0);
        crt.sizeDelta = new Vector2(20, 20);

        Clickable(row, () => { m_Placing = index; Show(Screen.SetupPlace); });
        InColumn(col, row, 54);
        Gap(col, 8);
    }
```

```
}

void BuildSetupPlace()
{
    // Transparent above, controls below: the whole point of this screen
    // is seeing the real object through the camera.
    var panel = Panel(Screen.SetupPlace, new Color(0, 0, 0, 0));
    SetupHeader(panel, "Step 3 of 3", false, () => Show(Screen.SetupChoose));

    var pill = Box(panel, new Vector2(0.5f, 1f), new Vector2(0, -108), new Vector2(220, 40), NavyDeep);
    Round(pill, 20);
    m_PlaceName = Body((RectTransform)pill.transform, "Place:", 14, Paper);
    Stretch((RectTransform)m_PlaceName.transform);
    m_PlaceName.alignment = TextAnchor.MiddleCenter;

    var pad = Box(panel, new Vector2(0.5f, 0f), new Vector2(0, 26), new Vector2(348, 262), new Color(0.05f, 0.07f, 0.12f, 0.92f));
    Round(pad, 18);

    var grid = pad.AddComponent<GridLayoutGroup>();
    grid.cellSize = new Vector2(158, 44);
    grid.spacing = new Vector2(8, 8);
    grid.padding = new RectOffset(10, 10, 34, 10);
    grid.constraint = GridLayoutGroup.Constraint.FixedColumnCount;
    grid.constraintCount = 2;
    grid.childAlignment = TextAnchor.UpperCenter;

    m_PlaceReadout = Body((RectTransform)pad.transform, "", 11, new Color(1, 1, 1, 0.7f));
    var rrt = (RectTransform)m_PlaceReadout.transform;
    rrt.anchorMin = rrt.anchorMax = new Vector2(0.5f, 1f);
    rrt.anchoredPosition = new Vector2(0, -18);
    rrt.sizeDelta = new Vector2(320, 18);
    m_PlaceReadout.alignment = TextAnchor.MiddleCenter;
    m_PlaceReadout.transform.SetAsLastSibling();

    // The eight moves from the mockup, each one a direction in the room
    // rather than an axis in a file.
    PadButton(pad, "< Left",        () => Move(Vector3.left));
    PadButton(pad, "Right >",       () => Move(Vector3.right));
    PadButton(pad, "^ Forward",     () => Move(Vector3.forward));
    PadButton(pad, "v Back",        () => Move(Vector3.back));
    PadButton(pad, "+ Height",      () => Move(Vector3.up));
    PadButton(pad, "- Height",      () => Move(Vector3.down));
    PadButton(pad, "Rotate Left",   () => Turn(-1f));
    PadButton(pad, "Rotate Right",  () => Turn(1f));
    PadButton(pad, "Reset",         ResetOne, new Color32(0x3A, 0x41, 0x52, 0xFF));
    PadButton(pad, "Save Position", SaveOne, Blue);

    // The readout must not be pushed into the grid's flow.
    m_PlaceReadout.GetComponent<LayoutElement>();
    var ignore = m_PlaceReadout.gameObject.AddComponent<LayoutElement>();
    ignore.ignoreLayout = true;
}

void BuildSetupSaved()
{
    var panel = Sheet(Screen.SetupSaved, NavyDeep);
    var col = Column(panel, 300, 30);

    Tick(col, Good);
```

```
    Gap(col, 14);
    Title(col, "Position saved!", 22, Paper);
    Gap(col, 8);
    m_SavedName = Body(col, "", 15, new Color(1, 1, 1, 0.8f));
    Gap(col, 8);
    m_SavedProof = Body(col, "", 12, new Color(1, 1, 1, 0.6f));
    Gap(col, 22);

    Pill(col, "Next Object", Blue, Paper, () =>
    {
        var stations = m_Host.StationList;
        if (stations != null && m_Placing + 1 < stations.Length)
        {
            m_Placing++;
            Show(Screen.SetupPlace);
        }
        else
        {
            Show(Screen.SetupComplete);
        }
    });
    Gap(col, 10);
    Pill(col, "Back to Objects", new Color(0, 0, 0, 0), Paper, () => Show(Screen.SetupChoose), Paper);
}

void BuildSetupComplete()
{
    var panel = Sheet(Screen.SetupComplete, Paper);
    var col = Column(panel, 300, 30);

    Tick(col, Good);
    Gap(col, 14);
    Title(col, "Setup Complete!", 22, Ink);
    Gap(col, 10);
    Body(col, "All objects have been placed\nand saved for this church.", 13, Muted);
    Gap(col, 26);
    Pill(col, "Done", Blue, Paper, () =>
    {
        m_Host.SaveAllCalibration();
        Show(Screen.Explore);
    });
}

// ------------------------------------------------------------ the actions

void BeginPlacing(int index)
{
    var station = m_Host.StationAt(index);
    if (station == null) return;
    m_Host.BeginCalibrating(station.stationId);
}

void Move(Vector3 direction) { m_Host.NudgeSelected(direction); Refresh(); }
void Turn(float sign) { m_Host.TurnSelected(sign); Refresh(); }
void ResetOne() { m_Host.ResetSelected(); Refresh(); }

void SaveOne()
{
    m_Host.SaveOneCalibration();
```

```
    var station = m_Host.StationAt(m_Placing);
    if (m_SavedName != null)
        m_SavedName.text = station != null ? station.stationName : "";
    if (m_SavedProof != null)
        m_SavedProof.text = m_Host.LastSaveResult;
    Show(Screen.SetupSaved);
}

// ------------------------------------------------------------ toolkit
//
// Small on purpose. A handful of primitives that compose, rather than a
// prefab per screen: every screen here is a column of text and pills,
// and eight of them is less code than eight prefabs to keep in step.

GameObject Panel(Screen screen, Color background)
{
    var go = new GameObject($"Panel_{screen}", typeof(Image));
    go.transform.SetParent(m_Root, false);
    var rt = (RectTransform)go.transform;
    Stretch(rt);
    var img = go.GetComponent<Image>();
    img.color = background;
    img.raycastTarget = background.a > 0.01f;
    m_Panels[screen] = go;
    return go;
}

/// <summary>A full-screen opaque panel - the non-camera screens.</summary>
GameObject Sheet(Screen screen, Color background) => Panel(screen, background);

void SetupHeader(GameObject panel, string step, bool closes, Action onLeft)
{
    var bar = Box((RectTransform)panel.transform, new Vector2(0.5f, 1f), Vector2.zero,
                  new Vector2(390, 78), Navy);
    var rt = (RectTransform)bar.transform;
    rt.anchorMin = new Vector2(0, 1);
    rt.anchorMax = new Vector2(1, 1);
    rt.offsetMin = new Vector2(0, -78);
    rt.offsetMax = Vector2.zero;

    var title = Body(rt, "AR SETUP", 15, Paper);
    title.fontStyle = FontStyle.Bold;
    var trt = (RectTransform)title.transform;
    trt.anchorMin = trt.anchorMax = new Vector2(0.5f, 0.5f);
    trt.anchoredPosition = new Vector2(0, 6);
    trt.sizeDelta = new Vector2(240, 22);
    title.alignment = TextAnchor.MiddleCenter;

    m_StepLabel = Body(rt, step, 12, new Color(1, 1, 1, 0.75f));
    var srt = (RectTransform)m_StepLabel.transform;
    srt.anchorMin = srt.anchorMax = new Vector2(0.5f, 0.5f);
    srt.anchoredPosition = new Vector2(0, -12);
    srt.sizeDelta = new Vector2(240, 18);
    m_StepLabel.alignment = TextAnchor.MiddleCenter;

    var left = Box(rt, new Vector2(0f, 0.5f), new Vector2(30, 0), new Vector2(40, 40), new Color(0, 0, 0, 0));
    var glyph = Body((RectTransform)left.transform, closes ? "X" : "<", 18, Paper);
    Stretch((RectTransform)glyph.transform);
    glyph.alignment = TextAnchor.MiddleCenter;
```

```
    Clickable(left, onLeft);
}

RectTransform Column(GameObject parent, float width, float topInset)
{
    var go = new GameObject("Column", typeof(RectTransform));
    go.transform.SetParent(parent.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = new Vector2(0.5f, 1f);
    rt.anchorMax = new Vector2(0.5f, 1f);
    rt.pivot = new Vector2(0.5f, 1f);
    rt.anchoredPosition = new Vector2(0, -topInset);
    rt.sizeDelta = new Vector2(width, 0);

    var layout = go.AddComponent<VerticalLayoutGroup>();
    layout.childAlignment = TextAnchor.UpperCenter;
    layout.childForceExpandWidth = true;

    // childControlHeight MUST be true, and ForceExpand false.
    //
    // With control off the group never reads a child's LayoutElement,
    // so every Height() and every Gap() in this file was ignored and
    // each child counted as its raw 100x100 default instead. The
    // column then came out far taller than its contents and - with
    // ForceExpandHeight defaulting to true - shared the slack out
    // between the children, which is why Scan Marker was five times
    // its 46px and why the tutorial's own button ended up below the
    // bottom edge of its card.
    layout.childControlHeight = true;
    layout.childForceExpandHeight = false;

    var fit = go.AddComponent<ContentSizeFitter>();
    fit.verticalFit = ContentSizeFitter.FitMode.PreferredSize;
    return rt;
}

RectTransform Column(RectTransform parent, float width, float topInset) =>
    Column(parent.gameObject, width, topInset);

Text Title(RectTransform parent, string text, int size, Color colour)
{
    var t = Make(parent, text, m_Heading, size, colour);
    t.fontStyle = FontStyle.Bold;
    t.alignment = TextAnchor.MiddleCenter;
    Height(t.gameObject, size + 14);
    return t;
}

Text Body(RectTransform parent, string text, int size, Color colour)
{
    var t = Make(parent, text, m_Body, size, colour);
    t.alignment = TextAnchor.MiddleCenter;
    Height(t.gameObject, (size + 8) * Mathf.Max(1, text.Split('\n').Length));
    return t;
}

Text Make(RectTransform parent, string text, Font font, int size, Color colour)
{
    var go = new GameObject("Text", typeof(Text));
```

```
    go.transform.SetParent(parent, false);
    var t = go.GetComponent<Text>();
    t.text = text;
    t.font = font;
    t.fontSize = size;
    t.color = colour;
    t.horizontalOverflow = HorizontalWrapMode.Wrap;
    t.verticalOverflow = VerticalWrapMode.Overflow;
    return t;
}

void Gap(RectTransform parent, float pixels)
{
    var go = new GameObject("Gap", typeof(RectTransform));
    go.transform.SetParent(parent, false);
    Height(go, pixels);
}

void Height(GameObject go, float pixels)
{
    var le = go.GetComponent<LayoutElement>() ?? go.AddComponent<LayoutElement>();
    le.minHeight = le.preferredHeight = pixels;
}

void InColumn(RectTransform column, GameObject child, float height)
{
    child.transform.SetParent(column, false);
    var rt = (RectTransform)child.transform;
    rt.anchorMin = new Vector2(0.5f, 0.5f);
    rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.pivot = new Vector2(0.5f, 0.5f);
    Height(child, height);
}

GameObject Box(RectTransform parent, Vector2 anchor, Vector2 offset, Vector2 size, Color colour)
{
    var go = new GameObject("Box", typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = anchor;
    rt.pivot = new Vector2(0.5f, anchor.y);
    rt.anchoredPosition = offset;
    rt.sizeDelta = size;
    go.GetComponent<Image>().color = colour;
    return go;
}

GameObject Box(GameObject parent, Vector2 anchor, Vector2 offset, Vector2 size, Color colour) =>
    Box((RectTransform)parent.transform, anchor, offset, size, colour);

GameObject Pill(RectTransform parent, string label, Color fill, Color ink,
                Action onClick, Color? border = null)
{
    var go = new GameObject($"Pill_{label}", typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    go.GetComponent<Image>().color = fill;
    Round(go, 22);
    if (border.HasValue) Outline(go, border.Value);
```

```
    var text = Body((RectTransform)go.transform, label, 15, ink);
    text.fontStyle = FontStyle.Bold;
    Stretch((RectTransform)text.transform);
    text.alignment = TextAnchor.MiddleCenter;
    text.GetComponent<LayoutElement>().ignoreLayout = true;

    go.GetComponent<Button>().onClick.AddListener(() => onClick());
    Height(go, 46);
    return go;
}

GameObject PadButton(GameObject pad, string label, Action onClick, Color? fill = null)
{
    var go = new GameObject($"Pad_{label}", typeof(Image), typeof(Button));
    go.transform.SetParent(pad.transform, false);
    go.GetComponent<Image>().color = fill ?? new Color32(0x1E, 0x25, 0x33, 0xFF);
    Round(go, 10);

    var text = Body((RectTransform)go.transform, label, 13, Paper);
    Stretch((RectTransform)text.transform);
    text.alignment = TextAnchor.MiddleCenter;
    text.GetComponent<LayoutElement>().ignoreLayout = true;

    go.GetComponent<Button>().onClick.AddListener(() => onClick());
    return go;
}

/// <summary>
/// Narrates whichever station the visitor is nearest, and changes
/// when that changes.
///
/// The rule is the bar's rule. The title at the bottom already names
/// the nearest station and swaps the moment a nearer one exists, so
/// the narration follows the same nearest-station test rather than a
/// radius of its own: first it only spoke inside triggerRadius, which
/// meant walking the room changed the title and left the audio on
/// whatever it had last said - or silent.
///
/// A new station interrupts the old one. Letting it finish sounds
/// right in principle and is wrong here: a visitor who has walked on
/// is being read about something behind them.
///
/// The dwell is the one piece of restraint. Standing between two
/// statues the nearest flickers between them frame to frame, and
/// without it the phone would stutter the first syllable of each
/// forever, so a station has to stay nearest for a moment before it
/// is worth starting.
/// </summary>
///
/// Raised from 0.7s. With the sticky margin now doing the work of
/// rejecting jitter, this is no longer the only guard and can be
/// what it should always have been: long enough that walking PAST a
/// statue does not start reading it aloud. Two seconds is about the
/// difference between passing a thing and stopping at it.
const float NarrationDwellSeconds = 2.0f;

void AutoNarrate()
{
    // Not over the tutorial: the visitor is reading, and the phone
```

```
    // talking over it would be two instructions at once.
    if (m_Tutorial != null && m_Tutorial.activeSelf) return;

    // Nor over the welcome. See m_GreetingSpeaking.
    if (m_GreetingSpeaking)
    {
        // Pending as well as speaking. The welcome is the FIRST line
        // of a run, so it is almost always still queued behind the
        // engine starting up - not yet speaking, and Stop() would
        // discard it. See DeviceSpeech.Pending.
        if (DeviceSpeech.IsSpeaking() || DeviceSpeech.Pending) return;
        m_GreetingSpeaking = false;
    }

    var st = m_Host.NearestStation();
    if (st == null) return;

    if (st.stationId == m_SpokenStationId)
    {
        m_PendingStationId = null;
        return;
    }

    if (st.stationId != m_PendingStationId)
    {
        m_PendingStationId = st.stationId;
        m_PendingSince = Time.unscaledTime;
        return;
    }

    if (Time.unscaledTime - m_PendingSince < NarrationDwellSeconds) return;

    m_SpokenStationId = st.stationId;
    m_PendingStationId = null;

    m_GreetingSpeaking = false;
    DeviceSpeech.Stop();
    DeviceSpeech.Speak(st.Narration, AppLanguage.Current);
    SetSpeakerPlaying(true);
}

/// <summary>
/// Reads the nearest station aloud, or stops if it is already reading.
/// </summary>
void SpeakCurrent()
{
    if (DeviceSpeech.IsSpeaking())
    {
        DeviceSpeech.Stop();
        SetSpeakerPlaying(false);
        // Silencing it counts as having heard this one, or AutoNarrate
        // would start it again on the very next frame.
        var here = m_Host.NearestStation();
        if (here != null) m_SpokenStationId = here.stationId;
        return;
    }

    var st = m_Host.NearestStation();
    // A deliberate tap overrules the introduction; see m_GreetingSpeaking.
```

```
    m_GreetingSpeaking = false;
    // Before any station is in range the bar carries the welcome line
    // from the mockup, so the speaker reads that instead of nothing.
    var text = st != null ? st.Narration : WelcomeLine();
    DeviceSpeech.Speak(text, AppLanguage.Current);
    SetSpeakerPlaying(true);
}

/// <summary>
/// White while idle, blue while speaking.
///
/// A flat icon gave the visitor no way to tell whether a tap had done
/// anything - and on a phone held at arm's length in a church, with the
/// narration quiet, "did that work?" is the whole question.
/// </summary>
void SetSpeakerPlaying(bool playing)
{
    if (m_SpeakerBg == null || m_SpeakerIcon == null) return;

    // Lit means talking. It was the other way round: a white button
    // on a white bar while silent, going dark navy while it spoke -
    // so the loud state looked switched off and the silent one
    // looked active, and the only way to know which was which was to
    // listen. Dim is off, blue is on, the same way every play button
    // the visitor has ever used behaves.
    m_SpeakerBg.color = playing ? Blue : new Color(1f, 1f, 1f, 0.16f);
    m_SpeakerIcon.color = playing ? Paper : new Color(1f, 1f, 1f, 0.75f);
}

static string WelcomeLine() =>
    AppLanguage.IsTagalog
        ? "Maligayang pagdating sa SanctiWalk AR Museum. Pakilingon po."
        : "Welcome to the SanctiWalk AR Museum. Please turn around.";

/// <summary>
/// Puts the nearest station's name on the bar, or the welcome line
/// when there is no station nearby yet - which is what the visitor
/// sees while still facing the printed marker.
/// </summary>
void RefreshStationTitle()
{
    if (m_StationTitle == null) return;
    var st = m_Host.NearestStation();
    // At the marker the sign carries the branding, so the bar only has
    // to say what to do next.
    m_StationTitle.text = st != null
        ? st.Name
        : (AppLanguage.IsTagalog ? "Lumingon" : "Turn Around");
}


void Tick(RectTransform parent, Color colour)
{
    var go = new GameObject("Tick", typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(64, 64);
    go.GetComponent<Image>().color = colour;
    Round(go, 32);
```

```
    var glyph = Body(rt, "OK", 18, Paper);
    glyph.fontStyle = FontStyle.Bold;
    Stretch((RectTransform)glyph.transform);
    glyph.alignment = TextAnchor.MiddleCenter;
    glyph.GetComponent<LayoutElement>().ignoreLayout = true;

    Height(go, 64);
    var le = go.GetComponent<LayoutElement>();
    le.preferredWidth = le.minWidth = 64;
}

/// <summary>
/// Rounded corners without a sprite atlas.
///
/// Unity's Image needs a sliced sprite to round anything, and there is
/// no sprite in this project to slice. Generating one white rounded
/// texture per radius and reusing it costs a few kilobytes and keeps the
/// cards looking like the mockup instead of like rectangles.
/// </summary>
static readonly Dictionary<int, Sprite> s_Rounded = new Dictionary<int, Sprite>();

static Sprite s_Speaker;

/// <summary>
/// A speaker: the cone, then two arcs.
///
/// Drawn rather than typed. The old glyph was the three characters
/// "<triangle>))", which is punctuation doing an icon's job - the
/// triangle and the brackets are different weights, and they sit on
/// a text baseline instead of the centre of the button.
///
/// Generated into a texture like the rounded-rectangle sprite
/// above, so there is no asset to import and nothing to keep in
/// step with a .meta file.
/// </summary>
Image SpeakerIcon(RectTransform parent, int size)
{
    if (s_Speaker == null)
    {
        const int N = 128;                    // drawn big, drawn once
        var tex = new Texture2D(N, N, TextureFormat.RGBA32, false);
        var clear = new Color(1, 1, 1, 0);
        var pix = new Color[N * N];
        for (var i = 0; i < pix.Length; i++) pix[i] = clear;

        // Everything below is in units of the texture, so the
        // proportions hold whatever N is.
        float cx = N * 0.40f, cy = N * 0.5f;
        float neckW = N * 0.11f, neckH = N * 0.22f;
        float coneW = N * 0.17f, coneH = N * 0.52f;

        for (var y = 0; y < N; y++)
        for (var x = 0; x < N; x++)
        {
            float dx = x - cx, dy = y - cy;
            var on = false;

            // The neck: a small block on the left.
```

```
            if (dx >= -coneW - neckW && dx <= -coneW * 0.15f
                && Mathf.Abs(dy) <= neckH * 0.5f) on = true;

            // The cone: a triangle widening to the right.
            if (!on && dx >= -coneW && dx <= 0f)
            {
                var t = (dx + coneW) / coneW;             // 0 at the neck
                if (Mathf.Abs(dy) <= coneH * 0.5f * t) on = true;
            }

            // Two arcs, open to the right, centred on the cone's mouth.
            if (!on && dx > 0f)
            {
                var r = Mathf.Sqrt(dx * dx + dy * dy);
                var ang = Mathf.Atan2(Mathf.Abs(dy), dx);  // 0 = straight out
                if (ang < 0.95f)
                {
                    float t1 = N * 0.055f;                 // stroke width
                    if (Mathf.Abs(r - N * 0.20f) < t1 * 0.5f) on = true;
                    if (Mathf.Abs(r - N * 0.33f) < t1 * 0.5f) on = true;
                }
            }

            if (on) pix[y * N + x] = Color.white;
        }

        tex.SetPixels(pix);
        tex.Apply();
        tex.filterMode = FilterMode.Bilinear;
        s_Speaker = Sprite.Create(tex, new Rect(0, 0, N, N), new Vector2(0.5f, 0.5f));
    }

    var go = new GameObject("SpeakerIcon", typeof(Image));
    go.transform.SetParent(parent, false);
    var img = go.GetComponent<Image>();
    img.sprite = s_Speaker;
    img.raycastTarget = false;             // the button beneath takes the tap
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = Vector2.zero;
    rt.sizeDelta = new Vector2(size, size);
    return img;
}

GameObject Round(GameObject go, int radius)
{
    var img = go.GetComponent<Image>();
    if (img == null) return go;

    if (!s_Rounded.TryGetValue(radius, out var sprite))
    {
        var size = radius * 2 + 2;
        var tex = new Texture2D(size, size, TextureFormat.RGBA32, false);
        for (var y = 0; y < size; y++)
        for (var x = 0; x < size; x++)
        {
            var cx = Mathf.Min(x, size - 1 - x);
            var cy = Mathf.Min(y, size - 1 - y);
            var inside = true;
```

```
            if (cx < radius && cy < radius)
            {
                var dx = radius - cx;
                var dy = radius - cy;
                inside = dx * dx + dy * dy <= radius * radius;
            }
            tex.SetPixel(x, y, inside ? Color.white : new Color(1, 1, 1, 0));
        }
        tex.Apply();
        sprite = Sprite.Create(tex, new Rect(0, 0, size, size), new Vector2(0.5f, 0.5f),
                               100f, 0, SpriteMeshType.FullRect,
                               new Vector4(radius, radius, radius, radius));
        s_Rounded[radius] = sprite;
    }

    img.sprite = sprite;
    img.type = Image.Type.Sliced;
    return go;
}

void Outline(GameObject go, Color colour)
{
    var o = go.AddComponent<Outline>();
    o.effectColor = colour;
    o.effectDistance = new Vector2(1, 1);
}

void Clickable(GameObject go, Action onClick)
{
    var img = go.GetComponent<Image>();
    if (img != null) img.raycastTarget = true;
    var btn = go.GetComponent<Button>() ?? go.AddComponent<Button>();
    btn.onClick.AddListener(() => onClick());
}

static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero;
    rt.offsetMax = Vector2.zero;
}

/// <summary>Rebuilds the object list when the station table changes.</summary>
public void OnStationsChanged() => BuildObjectRows();
}
```

4.1.4 AR-004 — ChurchFrame.cs

Code ID	AR-004
File Name	ChurchFrame.cs
File Path	SanctiWalkAR/Assets/Scripts/ChurchFrame.cs
Language / Type	C#
Lines	124
Functions / Classes	ChurchFrame (public class); ExpectedErrorAt (public method); FromFloorAndForward (public method); ToSession (public method); ToChurch (public method); IsBaselineUsable (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/ChurchFrame.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChurchFrame.cs

```
using UnityEngine;

/// <summary>
/// The church's own coordinate frame: X across, Y up, Z into the nave,
/// metres, with the origin wherever the tour starts.
///
/// ## Why this class exists, and why it is this short
///
/// Station positions were previously carried into the room by a printed
/// marker, and four separate things had to be right for that to work. None
/// of them reliably were:
///
///   - The marker's pose is reported at the CENTRE of the printed sheet,
///     and the centre of a blank rectangle is not something anyone can
///     point at. Picking it on the scan guessed by up to 150 mm.
///   - The wall's facing was read off one triangle of a photogrammetry
///     mesh. At the spot chosen it swung between 20 and 339 degrees
///     depending on how much of the surface was averaged.
///   - The sideways direction was cross(up, normal) - a handedness
///     convention assumed to match what ARCore reports for a tracked image
///     and never tested. Backwards, the room mirrors.
///   - The numbers were copied by hand between three tools.
///
/// A floor plane removes all four at once, because it supplies three of the
/// six degrees of freedom for free and gravity-aligned: pitch, roll, and
/// height. What is left is a position on the floor and a heading - and both
/// of those CAN be pointed at. Two taps on the floor give them.
///
/// So the frame is just a Pose, and converting is just the Pose. There is
/// no sign to get backwards, no normal to average, no convention to assume,
```

```
/// because the rotation is built from two points that were physically
/// indicated rather than derived. That is the whole fix, and the reason
/// this file is twenty lines of arithmetic rather than a hundred.
///
/// MuseumStation.position is already in these terms, so nothing about the
/// station data has to change.
/// </summary>
public static class ChurchFrame
{
    /// <summary>
    /// How far apart the two floor taps must be, in metres.
    ///
    /// The heading is the line between them, so a short baseline magnifies
    /// tap error into rotation error - the same trap that made the 180 mm
    /// marker useless for facing, where a 30 mm slip became 9.5 degrees.
    ///
    /// This was 1 m, and Tools/check-floor-accuracy.py says that is not
    /// good enough. Against the sala's furthest station, 4.93 m out:
    ///
    ///     tap B off by   baseline 1 m   baseline 3 m   baseline 5 m
    ///            5 cm         0.25 m         0.08 m         0.05 m
    ///           10 cm         0.49 m         0.16 m         0.10 m
    ///           20 cm         0.97 m         0.33 m         0.20 m
    ///
    /// A 10 cm slip is not a careless tap - it is an ordinary one, on a
    /// floor plane, at arm's length. At a metre it costs half a metre at
    /// the far end, which is the size of error this whole change exists to
    /// remove. At three metres the same tap costs 16 cm.
    ///
    /// So the floor is 2.5 m, and the picker says what the baseline
    /// actually bought rather than leaving it to be discovered by walking.
    /// </summary>
    public const float MinBaselineMetres = 2.5f;

    /// <summary>
    /// What a baseline is worth: the error at `distance` metres from the
    /// origin caused by an ordinary tap slip. Used to tell the operator
    /// what they just bought, in metres rather than degrees.
    /// </summary>
    public const float TypicalTapSlipMetres = 0.05f;

    public static float ExpectedErrorAt(float distance, float baseline)
        => baseline <= 0.01f ? float.PositiveInfinity
                             : distance * Mathf.Atan2(TypicalTapSlipMetres, baseline);

    /// <summary>
    /// The frame, from a point on the floor and a direction to look in.
    ///
    /// `forwardHorizontal` is flattened and normalised here rather than by
    /// the caller, so a tap picked slightly up a step or on a sloped bit of
    /// mesh cannot tilt the whole church.
    /// </summary>
    public static Pose FromFloorAndForward(Vector3 floorPoint, Vector3 forwardHorizontal)
    {
        var forward = forwardHorizontal;
        forward.y = 0f;

        // Degenerate input would make LookRotation return identity with a
        // console warning, silently giving a frame rotated to whatever the
```

```
        // session happened to start at. Better to say so.
        if (forward.sqrMagnitude < 1e-6f)
        {
            Debug.LogWarning("[ChurchFrame] The two points are vertically above each other; " +
                             "heading cannot be derived. Using session forward.");
            forward = Vector3.forward;
        }

        return new Pose(floorPoint, Quaternion.LookRotation(forward.normalized, Vector3.up));
    }

    /// <summary>Church metres to where they are in this AR session.</summary>
    public static Vector3 ToSession(Pose origin, Vector3 church)
        => origin.position + origin.rotation * church;

    /// <summary>Where something in this session is, in church metres.</summary>
    public static Vector3 ToChurch(Pose origin, Vector3 session)
        => Quaternion.Inverse(origin.rotation) * (session - origin.position);

    /// <summary>
    /// Whether two floor taps are far enough apart to take a heading from.
    /// `why` is written for the person holding the phone, not the log.
    /// </summary>
    public static bool IsBaselineUsable(Vector3 a, Vector3 b, out string why)
    {
        var apart = new Vector2(b.x - a.x, b.z - a.z).magnitude;
        if (apart < MinBaselineMetres)
        {
            why = $"Too close together ({apart:F2} m). Walk a few steps further and tap again.";
            return false;
        }
        why = null;
        return true;
    }
}
```

4.1.5 AR-005 — ChurchOriginTest.cs

Code ID	AR-005
File Name	ChurchOriginTest.cs
File Path	SanctiWalkAR/Assets/Scripts/ChurchOriginTest.cs
Language / Type	C#
Lines	2602
Functions / Classes	ChurchOriginTest (public class); Awake (method); OnDestroy (method); StationAt (public method); SetMuseumVisible (public method); MetresToStation (public method); RebuildStationsForLanguage (public method); NearestStation (public method); NextStationLine (public method); OpenNearestCard (public method); CycleToNextStation (public method); SetCalibrating (public method); BeginCalibrating (public method); NudgeSelected (public method); TurnSelected (public method); ResetSelected (public method); LocalizeWithMultiSet (public method); SaveOneCalibration (public method); SaveAllCalibration (public method); ApplyCalibrationLive (method); SaveCalibration (method); BindWorld (method); InvalidateStationsForFrameChange (method); ReadLaunchUrl (method); Update (method); LateUpdate (method); OnEnable (method); OnDisable (method); OnImagesChanged (method); RecordMarkerSighting (method); MarkerBelongsHere (method); TryOrigin (method); ReportPlacements (method); SimulateScan (public method); OnFloorOrigin (method); OnVpsLocalized (method); DrawVpsMarkerProof (method); SpawnStationPoster (method); NormaliseStationsToMapSpace (method); ParishMarker (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/ChurchOriginTest.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChurchOriginTest.cs

```
using System.Text;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Phase 0 proof of concept: does ARCore's world tracking hold a virtual object
/// on its real surveyed position after the entrance marker leaves the camera?
///
/// The whole test rests on one line — images.enabled = false, in SetOrigin().
/// Once the marker has given us the origin we stop tracking it entirely, so
/// everything the app knows about where it is from that moment on comes from
/// SLAM alone. Leave the image manager running and you are measuring marker
/// tracking, which was never in doubt.
///
```

```
/// Two cubes are placed at every surveyed point:
///   - a plain one, parented to nothing, showing raw visual-inertial drift
///   - an anchored one, which ARCore refines as it learns the room
/// Comparing them answers a real Phase 1 question: do anchors actually help
/// inside this church, or do they not?
/// </summary>
public class ChurchOriginTest : MonoBehaviour
{
    [Header("Surveyed checkpoints — METRES, in CHURCH terms")]
    [Tooltip("X across the nave, Y up, Z into the nave. Measure with a tape from the marker. " +
             "ChurchToMarker() converts these into the marker's own axes, which are not the same.")]
    /// <summary>
    /// House-scale by default, so the dry run at home actually shows
    /// something. These started at 6/12/18 m — church scale — which meant the
    /// cubes were placed through the wall and out of the building during
    /// indoor testing, and the app looked broken when it was working.
    /// Replace with the real surveyed values before the church test.
    /// </summary>
    public Vector3[] surveyed =
    {
        // Surveyed off the MultiSet scan of the sala (map MAP_HXPCAEUNK15O),
        // not paced out with a tape measure. Each point was picked on the
        // textured mesh in Tools/viewer, which reports map-space metres, then
        // converted here by subtracting the marker pose.
        //
        // The marker sits on the wall behind the TV, map-space
        // (0.137, -0.180, 5.348), 1.407 m above the floor, wall facing 135.0
        // degrees. The TV lands at X = 0.000 below, which is the arithmetic
        // agreeing with itself: the marker was placed directly behind it.
        new Vector3(1.513f, -0.753f, 1.218f),  // sofa
        new Vector3(1.947f, -0.431f, 4.533f),  // santo nino
        new Vector3(-1.913f, -0.297f, 2.705f), // alak
        new Vector3(0.000f, -0.397f, 0.401f),  // tv
    };

    ARTrackedImageManager m_Images;
    ARAnchorManager m_Anchors;

    /// <summary>
    /// The museum experience layered over this drift test. It is a separate
    /// component so the test keeps working on its own: the cubes, poles and
    /// on-screen numbers are what prove the survey, and the cards are what
    /// the pilgrim actually sees.
    /// </summary>
    MuseumUI m_Museum;
    Camera m_Camera;

    bool m_OriginSet;
    Pose m_Origin;
    int m_NextCheckpoint;

    // Furthest the camera has ever been from the origin, and how far it has
    // travelled in total. Two screenshots taken while the phone is rotating
    // on the spot look identical, which makes a live x/z readout impossible
    // to interpret remotely — it is genuinely ambiguous whether tracking is
    // frozen or the person simply has not walked anywhere. These two numbers
    // only ever go up, so one screenshot settles it.
    /// <summary>The info card, kept turned toward the viewer each frame.</summary>
    /// <summary>
```

```
/// The Santo Nino card, re-placeable by aiming at the statue.
///
/// Typing coordinates is the wrong way to position AR content: the number
/// you can read off a phone is where YOU are standing, not where the
/// object is, and the two differ by however far away you were. Aiming and
/// tapping puts the card where the thing actually is.
/// </summary>
Transform m_NinoPanel;

/// <summary>How far ahead of the camera a placed card lands.</summary>
const float k_PlaceDistance = 1.2f;

readonly System.Collections.Generic.List<Transform> m_Panels = new System.Collections.Generic.List<Transform>();

/// <summary>
/// One card per station, by station index. This is the shape the real
/// feature has - each station carries its own content - so the demo is
/// built that way rather than special-casing a single panel.
/// </summary>
static readonly string[] k_PanelForStation =
{
    null,                 // sofa - no artwork for it
    "panel-santo-nino",   // the sala really does have a Santo Nino
    null,                 // alak
    null,                 // tv
};

float m_MaxDistance;
float m_PathLength;
Vector3 m_LastSample;
bool m_HasSample;

/// <summary>
/// The scan-the-room gate and the two-tap origin, which together are
/// now the normal way the frame is established. The printed marker is
/// the fallback for when plane detection cannot get going - somewhere
/// too dark, or a floor with nothing on it to track.
/// </summary>
SiteReadiness m_Readiness;
OriginPicker m_Picker;

/// <summary>
/// MultiSet's own map query. Unlike the marker and the floor picker it
/// does not reconstruct the map's alignment from something measured -
/// it asks the map directly where the phone is, and returns a pose in
/// the same coordinates the POIs were picked in.
/// </summary>
MultiSetVps m_Vps;

/// <summary>
/// Which MultiSet map to localize against, per parish.
///
/// The map code says WHICH building; the credentials in
/// Resources/multiset-credentials.txt say who is asking. Both have to
/// match the scan the POIs were picked on, because a pose returned by
/// the wrong map is still a valid-looking pose - it just puts every
/// station somewhere plausible in the wrong building.
///
/// MAP_AIF6H1PDFOTS is the scan the Mary Help stations were placed on
```

```
/// (Tools/viewer/mhcp-textured.glb, the same export), and
/// MAP_0F1RHM0L37Q8 the one San Roque's were placed on
/// (Tools/viewer/srcp-textured.glb). An earlier note here named
/// MAP_UC18HZ93A6VK for Mary Help; that is a different, older scan and
/// the current stations do not belong to it.
///
/// One set of credentials covers all of them: the clientId and secret
/// identify the ACCOUNT, and the map code picks the building.
/// </summary>
const string k_SalaMapCode  = "MAP_HXPCAEUNK15O";
const string k_MhcpMapCode  = "MAP_AIF6H1PDFOTS";
const string k_SrcpMapCode  = "MAP_0F1RHM0L37Q8";

/// <summary>
/// The map for whichever parish this run is for.
///
/// Falls back to the sala rather than to a church: the sala is where
/// this is tested, and a wrong-building localization is harder to
/// notice than no localization at all.
/// </summary>
string MapCodeForParish =>
    ParishId == "route-mhcp" ? k_MhcpMapCode
    : ParishId == "route-src" ? k_SrcpMapCode
    // The San Roque PREVIEW runs in the sala, so it localizes against
    // the sala map. Its stations are sala coordinates; pairing them
    // with the cathedral's map would put the tour in a building that
    // is not there.
    : k_SalaMapCode;

/// <summary>True when the origin came from MultiSet rather than the marker or the floor.</summary>
bool m_FromVps;

/// <summary>Drawn where the printed marker should be, as the accuracy check.</summary>
GameObject m_VpsMarkerProof;

/// <summary>
/// True when the frame came from the floor rather than the marker.
/// Worth carrying because the two are not equally trustworthy and the
/// overlay should say which one is in use.
/// </summary>
bool m_FromFloor;

/// <summary>
/// Whether the printed marker is allowed to set the origin.
///
/// It was not gated at all to begin with, and that made a nonsense of
/// calling it a fallback: OnImagesChanged fires the moment the image is
/// recognised, which is long before a floor plane can be found, so the
/// marker won every single time and the floor path never ran. The
/// screen said "MARKER - reduced accuracy" and the whole change looked
/// like it had done nothing.
///
/// A fallback has to be something you fall back TO. So the marker is
/// off while the room is being scanned, and is offered as a button only
/// once the scan has had a fair chance and not managed it - which is
/// also the honest moment to offer it, because that is when the room
/// has told you it is too dark or too bare for the better method.
/// </summary>
bool m_MarkerAllowed = true;
```

```
/// <summary>When the readiness gate started trying, for the offer below.</summary>
float m_ScanningSince = -1f;

/// <summary>How long to let the scan work before offering the marker.</summary>
const float k_OfferMarkerAfterSeconds = 20f;

readonly StringBuilder m_Log = new StringBuilder();
string m_Status = "Point the camera at the entrance marker";
GUIStyle m_Text, m_Button;

void Awake()
{
    m_Images = FindAnyObjectByType<ARTrackedImageManager>();
    m_Anchors = FindAnyObjectByType<ARAnchorManager>();
    m_Museum = gameObject.AddComponent<MuseumUI>();

    // Prefer the XR Origin's own camera over Camera.main. Camera.main is
    // a tag lookup and will happily return some other camera if the scene
    // ever gains one, which would report a position that never moves.
    var origin = FindAnyObjectByType<Unity.XR.CoreUtils.XROrigin>();
    m_Camera = origin != null && origin.Camera != null ? origin.Camera : Camera.main;

    Debug.Log($"[ChurchOriginTest] camera = {(m_Camera != null ? m_Camera.name : "NULL")}");

    // The screens. From here on the IMGUI overlay is a debugging tool
    // only - three taps in the corner - and every control a person is
    // meant to use lives in ArFlowUI.
    m_Flow = ArFlowUI.Create(this);
    Debug.Log($"[ChurchOriginTest] admin device: {AdminAccess.IsAdmin} ({AdminAccess.DeviceId})");

    // MultiSet localization. Independent of the marker and the floor
    // picker: all three produce the same thing, a Pose for the map's
    // origin in this session, so they can coexist and be compared.
    var cameraManager = FindAnyObjectByType<ARCameraManager>();
    if (cameraManager != null && m_Camera != null)
    {
        m_Vps = gameObject.AddComponent<MultiSetVps>();
        m_Vps.Bind(cameraManager, m_Camera, MapCodeForParish);
        Debug.Log($"[ChurchOriginTest] MultiSet map {MapCodeForParish} for {ParishId}");
        m_Vps.Localized += OnVpsLocalized;
    }

    // The readiness gate lives on whatever object carries the plane
    // manager, which BuildScene puts on the XR Origin. Absent - an old
    // scene, or a build without the managers - the marker path still
    // works, so this degrades rather than throwing.
    var planes = FindAnyObjectByType<ARPlaneManager>();
    if (planes != null)
    {
        // Nothing of the museum UI until the room has been scanned.
        m_Museum.SetVisible(false);

        // Marker tracking stays on. It is the default route again: it
        // is the one that was closest to working, and swapping it out
        // for the floor origin before the floor origin was proven put
        // the whole thing backwards. The floor origin is offered below
        // for whoever wants to try it.
```

```
        m_Readiness = planes.GetComponent<SiteReadiness>()
                      ?? planes.gameObject.AddComponent<SiteReadiness>();
        m_Readiness.Bind(m_Camera);

        var raycasts = FindAnyObjectByType<ARRaycastManager>();
        m_Picker = gameObject.AddComponent<OriginPicker>();
        m_Picker.Bind(m_Camera, raycasts);
        m_Picker.Established += OnFloorOrigin;
    }
    else
    {
        Debug.LogWarning("[ChurchOriginTest] No ARPlaneManager in the scene; " +
                         "falling back to the printed marker for the origin.");
    }

    // Never let the screen sleep.
    //
    // This is not a convenience. The phone's display timed out mid-test,
    // Android paused the activity, ARCore tore down the session, and on
    // resume it had to start tracking from nothing — which read as
    // "tracking keeps failing" when the real cause was a screen timeout.
    // Walking a church takes minutes of holding a phone without touching
    // it, which is exactly the condition that triggers it.
    Screen.sleepTimeout = SleepTimeout.NeverSleep;

    // How we were launched. SanctiWalk opens us with
    // sanctiwalkar://tour?church=route-mhcp, so the AR app knows which
    // parish's stations to show. Phase 1 loads the tour by this id; for
    // now it is recorded and displayed, which proves the handoff works
    // end to end before anything depends on it.
    // A remembered parish first, so an icon launch returns to where you
    // actually are. A link still wins: ReadLaunchUrl overwrites this
    // whenever one arrives.
    m_LaunchedForChurch = PlayerPrefs.GetString(k_LastParishKey, null);
    if (!string.IsNullOrEmpty(m_LaunchedForChurch))
        Debug.Log($"[ChurchOriginTest] resuming last parish: {m_LaunchedForChurch}");

    ReadLaunchUrl(Application.absoluteURL);
    Application.deepLinkActivated += ReadLaunchUrl;

    // Anything recorded in this room on a previous visit wins over the
    // surveyed table. Loaded before the origin is set, so the first
    // frame after the scan already has the right positions.
    m_Recording = StationRecording.Load(ParishId);
}

void OnDestroy()
{
    Application.deepLinkActivated -= ReadLaunchUrl;
}

/// <summary>Parish id from the launch URL, or null when opened directly.</summary>
string m_LaunchedForChurch;

/// <summary>Where the last parish is remembered between launches.</summary>
const string k_LastParishKey = "sanctiwalk.lastParish";

/// <summary>
/// Which parish's stations and recording this run is for.
```

```
///
/// Opened straight from the launcher - with no deep link from
/// SanctiWalk and nothing remembered - this used to fall back to
/// Maypajo, so tapping the icon at home put the phone in a church it
/// was nowhere near and drew that church's plan on the sala. The
/// fallback is now the sala itself, which is the room this is
/// actually tested in and the only one the models are standing in.
///
/// A launch FROM SanctiWalk still carries its own parish, so this
/// changes nothing a visitor at either church will see.
/// </summary>
string ParishId => string.IsNullOrEmpty(m_LaunchedForChurch) ? k_MhcpPreviewId : m_LaunchedForChurch;

/// <summary>
/// Positions recorded by walking the room, if this phone has any for
/// this parish. Null means fall back to the surveyed table.
/// </summary>
RecordedStationSet m_Recording;

/// <summary>
/// The stations for this run, recorded positions already applied.
/// Resolved once rather than per use, so the bubbles, the museum UI and
/// the placement report cannot end up describing different rooms.
/// </summary>
MuseumStation[] m_Stations;

MuseumStation[] Stations => m_Stations ??= StationRecording.Apply(
    TableForParish, m_Recording, CurrentFrameId);

/// <summary>
/// The compiled station table for whichever parish this run is for.
///
/// These are generated into MuseumStationsGenerated.cs from the plans
/// saved out of the station placer, in MultiSet map coordinates - which
/// is why they pair with MapCodeForParish and not with the sala. Both
/// have to name the same building or the app localizes in one place and
/// draws the stations of another.
///
/// Falls back to the sala, because the sala is where this is tested and
/// a table with no stations looks like the AR having failed.
/// </summary>
/// <summary>
/// Whether the compiled table is already in MultiSet map coordinates.
///
/// The parish tables are generated from the station placer, which picks
/// points on the MultiSet scan, so they are map space by construction.
/// The sala's table predates that and is metres from the printed
/// marker. The two are three floats either way and nothing in the type
/// says which, so it is said here.
/// </summary>
bool TableIsMapSpace =>
    ParishId == "route-mhcp" || ParishId == "route-src"
    || ParishId == k_SrcPreviewId || ParishId == k_MhcpPreviewId;

/// <summary>
/// Walking the San Roque tour in the sala, to see it before the church.
///
/// Reached with  adb shell am start -a android.intent.action.VIEW
///   -d "sanctiwalkar://ar?church=route-src-preview"
```

```
///
/// Same fifteen stations, same order, same images, same sides - laid
/// out on the sala scan instead of the cathedral. The cathedral's own
/// plan is untouched, so this cannot be confused for it: the positions
/// are metres apart in different maps.
/// </summary>
const string k_SrcPreviewId = "route-src-preview";

/// <summary>
/// The two Mary Help statues stood up in the sala, to judge the models
/// themselves before carrying the phone to Maypajo.
///
/// Reached with  adb shell am start -a android.intent.action.VIEW
///   -d "sanctiwalkar://ar?church=route-mhcp-preview"
///
/// Models only - none of the parish's history or image stations - and
/// laid out on the sala scan, so it cannot be mistaken for the parish's
/// own plan.
/// </summary>
const string k_MhcpPreviewId = "route-mhcp-preview";

MuseumStation[] TableForParish =>
    ParishId == "route-mhcp" ? MuseumStations.MHCP
    : ParishId == "route-src" ? MuseumStations.SRCP
    : ParishId == k_SrcPreviewId ? MuseumStations.SRCP_IN_SALA
    : ParishId == k_MhcpPreviewId ? MuseumStations.MHCP_IN_SALA
    : MuseumStations.Sala;

/// <summary>
/// The conversion from a stored station position into the session, for
/// whichever origin is actually in use.
///
/// There are three, and they are NOT interchangeable: the marker route
/// permutes axes, the floor route passes X and Z straight through, and
/// the VPS route works in map space. Every one of them was already
/// chosen correctly at its own Bind call - and then ReportPlacements
/// and SpawnStationPoster both went and used ChurchToMarker regardless.
///
/// For the poster that put it in a different place from the station it
/// belongs beside. For the report it was worse than a misplacement: the
/// log is how this project has been JUDGING whether a placement is
/// right, and on the floor and VPS routes it was describing a position
/// nothing was ever drawn at. Holding the converter in one field is
/// what stops a fourth caller doing it again.
/// </summary>
System.Func<Vector3, Vector3> m_ToSession;

string m_CalibrationNote = "";

/// <summary>The screens. Null only before Awake has run.</summary>
ArFlowUI m_Flow;

// ------------------------------------------------------------------
// What ArFlowUI is allowed to ask for.
//
// Deliberately a narrow surface. The UI can read state and ask for an
// action; it cannot reach the origin, the converter or the recording
// directly. A screen rewrite must not be able to move a station by a
// millimetre, and the only way to guarantee that is to not hand it the
```

```
// means.
// ------------------------------------------------------------------

public bool OriginSet => m_OriginSet;

/// <summary>True once ARCore has actually read the printed marker.</summary>
public bool MarkerDetected => m_MarkerSeenValid || m_OriginSet;

public string ParishDisplayName =>
    ParishId == "route-mhcp" ? "Mary Help of Christians Parish"
    : ParishId == "route-src" ? "San Roque Cathedral Parish"
    : ParishId == k_SrcPreviewId ? "San Roque (preview)"
    : ParishId;

public MuseumStation[] StationList => Stations;

public MuseumStation StationAt(int index)
{
    var all = Stations;
    return all != null && index >= 0 && index < all.Length ? all[index] : null;
}

public StationCalibrator Calibrator => m_Calibrator;

public void SetMuseumVisible(bool visible)
{
    if (m_Museum != null) m_Museum.SetVisible(visible);
}

/// <summary>How far the phone is from a station right now.</summary>
public float MetresToStation(MuseumStation station)
{
    if (station == null || m_Camera == null) return 0f;
    return Vector3.Distance(m_Camera.transform.position, SessionPositionOf(station));
}

/// <summary>
/// Redraws the floating cards after the visitor changes language.
/// </summary>
public void RebuildStationsForLanguage()
{
    if (m_Museum != null) m_Museum.RebuildForLanguage();
}

/// <summary>
/// Whichever station the visitor is closest to, or null before the
/// origin is set. The bottom bar names it and reads it aloud.
/// </summary>
/// <summary>
/// How much closer a rival has to be before it takes over.
///
/// Plain nearest-wins is unusable where stations are close together.
/// San Roque's are about 1.2 m apart and carry 2.2 m radii, so their
/// zones overlap almost completely: standing between two of them, a
/// shift of a few centimetres - or ARCore's own tracking jitter,
/// which moves the camera without the visitor moving at all - was
/// enough to change which one was nearest. The title changed and the
/// narration restarted, over and over, while the person stood still.
///
```

```
/// So the station a visitor is already on keeps it until somebody
/// else is CLEARLY closer. Three quarters of a metre is further than
/// jitter and shorter than a step, which is the gap between noise
/// and an intention to move on.
/// </summary>
const float k_StickyMarginMetres = 0.75f;

/// <summary>The station last handed out, so it can be kept. See above.</summary>
MuseumStation m_StickyStation;

public MuseumStation NearestStation()
{
    var all = Stations;
    if (all == null || all.Length == 0 || m_Camera == null) return null;

    MuseumStation nearest = null;
    var best = float.MaxValue;
    foreach (var s in all)
    {
        var d = MetresToStation(s);
        if (d < best) { best = d; nearest = s; }
    }

    // Still on the one we were on? Keep it unless beaten clearly.
    if (m_StickyStation != null)
    {
        var held = float.MaxValue;
        foreach (var s in all)
            if (s.stationId == m_StickyStation.stationId) { held = MetresToStation(s); break; }

        // Out of range entirely is not a tie to break - it is gone.
        var stillHere = held <= m_StickyStation.triggerRadius + k_StickyMarginMetres;
        if (stillHere && best > held - k_StickyMarginMetres)
            return m_StickyStation;
    }

    m_StickyStation = nearest;
    return nearest;
}

/// <summary>"Next: Santo Nino - 4 m", for the bottom bar.</summary>
public string NextStationLine()
{
    var all = Stations;
    if (all == null || all.Length == 0 || m_Camera == null) return "Navigation";

    MuseumStation nearest = null;
    var best = float.MaxValue;
    foreach (var s in all)
    {
        var d = MetresToStation(s);
        if (d < best) { best = d; nearest = s; }
    }
    return nearest == null ? "Navigation" : $"{nearest.stationName}  {best:F0} m";
}

/// <summary>Opens the card for whichever station the visitor is nearest.</summary>
public void OpenNearestCard()
{
```

```
    var all = Stations;
    if (all == null || m_Museum == null) return;

    MuseumStation nearest = null;
    var best = float.MaxValue;
    foreach (var s in all)
    {
        var d = MetresToStation(s);
        if (d < best) { best = d; nearest = s; }
    }
    if (nearest != null) m_Museum.OpenCardFor(nearest);
}

/// <summary>Points the camera's attention at the next station along.</summary>
public void CycleToNextStation()
{
    var all = Stations;
    if (all == null || all.Length == 0) return;
    m_BarStation = (m_BarStation + 1) % all.Length;
    if (m_Museum != null) m_Museum.OpenCardFor(all[m_BarStation]);
}

int m_BarStation;

// ---- calibration, driven by the setup screens ---------------------

/// <summary>
/// Turns the calibrator on or off as the screens come and go.
///
/// Called by ArFlowUI on every screen change, so leaving setup by ANY
/// route - save, back, the X, or finishing - puts it down. Relying on
/// each exit to remember is how it was left on in the first place.
/// </summary>
public void SetCalibrating(bool active)
{
    if (m_Calibrator.Active == active) return;

    m_Calibrator.SetActive(active);
    if (active)
        m_Calibrator.Bind(Stations, m_Camera);
}

public void BeginCalibrating(string stationId)
{
    if (!m_Calibrator.Active)
    {
        m_Calibrator.SetActive(true);
        m_Calibrator.Bind(Stations, m_Camera);
    }

    var all = Stations;
    for (var i = 0; i < all.Length; i++)
        if (all[i].stationId == stationId) { m_Calibrator.Select(i); return; }
}

public void NudgeSelected(Vector3 screenDirection)
{
    m_Calibrator.NudgeFromWhereIStand(screenDirection, m_Origin, m_ToSession);
    ApplyCalibrationLive();
```

```
}

public void TurnSelected(float sign)
{
    m_Calibrator.NudgeYaw(sign);
    ApplyCalibrationLive();
}

public void ResetSelected()
{
    m_Calibrator.ResetSelected(Surveyed);
    ApplyCalibrationLive();
}

/// <summary>
/// Ask MultiSet where we are, from the flow UI.
///
/// The printed marker is the intended route and this is the way out of
/// a dead end: with no marker in view there is no origin, and with no
/// origin there is nothing for AR SETUP to place a station against - so
/// the splash screen was a screen you could not leave.
/// </summary>
public void LocalizeWithMultiSet()
{
    if (m_Vps != null && !m_Vps.Busy) m_Vps.Localize();
}

public bool MultiSetBusy => m_Vps != null && m_Vps.Busy;
public string MultiSetStatus => m_Vps != null ? m_Vps.Status : "MultiSet unavailable";

public void SaveOneCalibration() => SaveCalibration(false);

/// <summary>What the save actually did, for the confirmation screen.</summary>
public string LastSaveResult => m_CalibrationNote;

/// <summary>How many positions are on disk for this parish right now.</summary>
public int SavedCount => m_Recording != null && m_Recording.stations != null
    ? m_Recording.stations.Count : 0;

public void SaveAllCalibration()
{
    SaveCalibration(true);
    m_Calibrator.SetActive(false);
}

/// <summary>Shows every station's numbers, for checking a calibration.</summary>
bool m_ShowCoordinateDump;

/// <summary>
/// Pushes an edit into the world immediately, so a button press is
/// visible in the room rather than only in the readout.
/// </summary>
void ApplyCalibrationLive()
{
    m_Calibrator.ApplyTo(Stations);
    if (m_Museum != null)
        m_Museum.Reposition(Stations, m_Origin, m_ToSession);
}
```

```
/// <summary>
/// Writes what the calibrator is holding to the parish's file.
///
/// The table is then rebuilt FROM that file rather than from the
/// calibrator's own copy, so what is drawn next is what was actually
/// written down. If those two ever disagree it shows up here, in the
/// room, rather than on somebody else's phone next week.
/// </summary>
void SaveCalibration(bool everything)
{
    var name = m_Calibrator.Selected != null ? m_Calibrator.Selected.stationName : "station";

    m_Recording = everything
        ? m_Calibrator.ToRecording(ParishId, CurrentFrameId)
        : m_Calibrator.ToRecording(ParishId, CurrentFrameId, m_Recording);

    if (!StationRecording.Save(m_Recording, out var path))
    {
        m_CalibrationNote = "COULD NOT SAVE - see the log.";
        return;
    }

    m_Stations = null;
    if (m_Museum != null)
        m_Museum.Bind(Stations, m_Origin, m_ToSession);
    m_Calibrator.Bind(Stations, m_Camera);
    ReportPlacements();

    // Read back from disk and check the number that was just written is
    // the number that comes back.
    //
    // "I do not know if it saved" is a fair complaint about a button
    // whose only feedback was the word Saved. Worse, a save CAN be
    // written and then refused on load - StationRecording.Apply throws
    // out a recording made in another frame - and the station would
    // silently snap back to its compiled position with nothing said.
    // This is the one moment both halves can be compared.
    var verified = "";
    var check = StationRecording.Load(ParishId);
    if (check == null)
    {
        verified = "  BUT IT DID NOT READ BACK";
    }
    else if (m_Calibrator.Selected != null &&
             check.TryGet(m_Calibrator.Selected.stationId, out var readBack, out _))
    {
        var slip = Vector3.Distance(readBack, m_Calibrator.PositionOf(m_Calibrator.Selected.stationId));
        verified = slip < 0.002f ? "  verified on disk" : $"  READ BACK {slip:F3} m OFF";
    }

    m_CalibrationNote = everything
        ? $"Saved all {m_Recording.stations.Count} ({CurrentFrameId}){verified}"
        : $"Saved {name} ({CurrentFrameId}){verified}";
    Debug.Log($"[ChurchOriginTest] {m_CalibrationNote} -> {path}");
}

/// <summary>
/// Re-binds everything that depends on the origin: the museum UI, and
/// the calibrator's working copy. Called by each route once it has set
```

```
/// m_Origin and m_ToSession, so there is one place that knows the order
/// these have to happen in.
/// </summary>
/// <summary>
/// Where the floor is, in session metres.
///
/// The surveyed floor is a height in MAP space, and the origin's
/// rotation is a yaw about world up on every route - so heights pass
/// through it unchanged and this is a subtraction rather than a full
/// transform. Worth stating, because if a route ever tilts the origin
/// this stops being true and cards would sit at a height that leans.
/// </summary>
float SessionFloorY =>
    m_Origin.position.y + (StationMarker.FloorY - StationMarker.MarkerInMapSpace.y);

void BindWorld()
{
    if (m_Museum != null)
        m_Museum.SessionFloorY = SessionFloorY;

    InvalidateStationsForFrameChange();
    if (m_Museum != null)
        m_Museum.Bind(Stations, m_Origin, m_ToSession);
    m_Calibrator.Bind(Stations, m_Camera);
    SpawnStationPoster("sala-santo-nino", "padre-speaks", 0.75f);
    ReportPlacements();

    // The setup screen's object list is built from this table, so it is
    // rebuilt here rather than guessing when the table settled.
    if (m_Flow != null)
        m_Flow.OnStationsChanged();
}

/// <summary>Where a stored station position actually lands in the session.</summary>
Vector3 SessionPositionOf(MuseumStation station)
{
    var convert = m_ToSession ?? ChurchToMarker;
    return m_Origin.position + m_Origin.rotation * convert(station.position);
}

/// <summary>
/// Which frame this session is working in. A recording is only valid in
/// the frame it was taken in, and the two are not interchangeable.
/// </summary>
/// <summary>
/// Which frame this session is working in.
///
/// Three routes, three frames, and they are not interchangeable: the
/// numbers are three floats in all of them and mean completely
/// different things. The VPS route was missing here, so a calibration
/// done under MultiSet would have been written down as marker-frame and
/// replayed as marker-frame on the next visit, putting every station
/// somewhere plausible and wrong with nothing in the log.
/// </summary>
string CurrentFrameId =>
    m_FromVps ? StationRecording.MapFrameId
    : m_FromFloor ? StationRecording.ChurchFrameId
    : StationRecording.MarkerFrameId;
```

```
/// <summary>
/// Throws away the station cache when the frame changes under it.
///
/// Stations are resolved once, lazily, and a recording is only applied
/// when it was taken in the frame currently in use. Both of those are
/// right on their own and wrong together: the first thing to touch
/// Stations fixes the answer for the session, and establishing a floor
/// origin afterwards changes CurrentFrameId without changing the cache.
/// A marker-frame recording would then still be applied while running
/// in the church frame - the exact reinterpretation StationRecording
/// refuses to do, arrived at by the back door.
/// </summary>
void InvalidateStationsForFrameChange()
{
    m_Stations = null;
}

/// <summary>
/// The admin position editor. Owns the working copy of every station's
/// position while it is being adjusted; nothing is written to the file
/// until SAVE.
/// </summary>
readonly StationCalibrator m_Calibrator = new StationCalibrator();

/// <summary>
/// The positions compiled into the project, kept untouched so RESET has
/// something true to go back to. Stations themselves are mutated by the
/// calibrator, so a copy has to be taken before any of that happens.
/// </summary>
MuseumStation[] m_Surveyed;

MuseumStation[] Surveyed
{
    get
    {
        if (m_Surveyed == null)
        {
            // The same table Stations comes from. This used to be
            // MuseumStations.Sala unconditionally, which made RESET put
            // living-room positions into a church.
            var table = TableForParish;
            m_Surveyed = new MuseumStation[table.Length];
            for (var i = 0; i < m_Surveyed.Length; i++)
                m_Surveyed[i] = table[i].Copy();
        }
        return m_Surveyed;
    }
}

/// <summary>Which station the recorder is waiting to be walked to.</summary>
int m_RecordIndex;

/// <summary>Last thing the recorder did, shown under the button.</summary>
string m_RecordNote = "";

/// <summary>Whether "Start over" has been armed by a first tap.</summary>
bool m_ConfirmClear;

void ReadLaunchUrl(string url)
```

```
{
    if (string.IsNullOrEmpty(url))
        return;

    Debug.Log($"[ChurchOriginTest] launched with: {url}");

    const string key = "church=";
    var at = url.IndexOf(key, System.StringComparison.Ordinal);
    if (at < 0)
        return;

    var value = url.Substring(at + key.Length);
    var amp = value.IndexOf('&');
    if (amp >= 0)
        value = value.Substring(0, amp);

    var previous = m_LaunchedForChurch;
    m_LaunchedForChurch = System.Uri.UnescapeDataString(value);

    // Remember it. Opening the app from the HOME SCREEN ICON carries no
    // link, so the parish fell back to the default - and the default is
    // Mary Help. Standing in San Roque, backgrounding the app and
    // tapping the icon to come back silently moved you to the other
    // parish, with its stations and its marker.
    PlayerPrefs.SetString(k_LastParishKey, m_LaunchedForChurch);
    PlayerPrefs.Save();
    Debug.Log($"[ChurchOriginTest] church = {m_LaunchedForChurch}");

    if (previous == m_LaunchedForChurch)
        return;

    // Everything derived from the parish has to go.
    //
    // On Android, Application.absoluteURL is often empty at Awake and
    // the link arrives later through deepLinkActivated - by which time
    // the station table has already been resolved and CACHED, using the
    // default parish. The screen then said "San Roque" while every
    // station, every photograph and the marker it was hunting for were
    // Mary Help's. The label reads ParishId live; the table did not.
    m_Stations = null;
    m_Surveyed = null;
    m_Recording = StationRecording.Load(ParishId);

    // The marker library is searched for this parish's sheet, and the
    // map code names its building, so both follow from the same change.
    if (m_Vps != null)
        m_Vps.Bind(FindAnyObjectByType<ARCameraManager>(), m_Camera, MapCodeForParish);

    Debug.Log($"[ChurchOriginTest] rebound to {ParishId}: " +
              $"map {MapCodeForParish}, {TableForParish.Length} stations");

    // If an origin was already established under the WRONG parish, it is
    // meaningless now: it placed another building's stations. Start over
    // rather than leave a confident, wrong room on screen.
    if (m_OriginSet)
    {
        m_OriginSet = false;
        m_FromVps = false;
        m_FromFloor = false;
```

```
        if (m_Images != null) m_Images.enabled = true;
        if (m_Museum != null) m_Museum.SetVisible(false);
        m_Status = "Parish changed - scan again";
    }
}

void Update()
{
    // Only the two-tap origin consumes taps, and only once the room has
    // been understood well enough to trust a raycast against its floor.
    if (m_OriginSet || m_Picker == null || m_Readiness == null)
        return;

    // The museum UI stays out of the way until there is an origin, so
    // its scan panel does not draw through the checklist.
    if (m_Museum != null)
        m_Museum.SetVisible(m_Readiness.Current.IsReady);

    if (m_ScanningSince < 0f)
        m_ScanningSince = Time.time;

    if (m_Readiness.Current.HasFloor)
        m_Picker.SetFloor(m_Readiness.Current.FloorY);

    if (!m_Readiness.Current.IsReady)
        return;

    if (m_Picker.TryGetTapThisFrame(out var screenPoint))
        m_Picker.Tap(screenPoint);
}

void LateUpdate()
{
    // Before the early return: the developer gesture is most needed on
    // the screen that exists before an origin does.
    PollDevGesture();

    if (!m_OriginSet || m_Camera == null)
        return;

    m_MarkerSeenAgeSeconds += Time.deltaTime;

    // Dragging a station, if the calibrator is open. Here rather than in
    // Update so the camera pose the drag is measured against is the one
    // this frame actually rendered with.
    // Dragging happens on ONE screen, and only while it is on screen.
    //
    // This used to be gated on m_Calibrator.Active alone, and Active
    // stayed true after leaving setup - so back on the AR screen every
    // swipe was still dragging whichever station had last been
    // selected. The cards wandered around the room and no swipe did
    // what it looked like it should, which is exactly what was
    // reported. Being "in calibration mode" and "looking at the screen
    // where you place things" are not the same state and must not share
    // one flag.
    if (m_Calibrator.Active && m_Flow != null && m_Flow.IsPlacing)
    {
        m_Calibrator.HandleTouch(m_Origin, m_ToSession);
        if (m_Calibrator.Dirty)
```

```
        {
            // Live, while the finger is still down - aligning a card you
            // cannot see move is not aligning it with anything.
            // Reposition rather than Bind: Bind destroys and rebuilds
            // every bubble, which would delete the card out from under
            // the finger holding it, sixty times a second.
            m_Calibrator.ApplyTo(Stations);
            if (m_Museum != null)
                m_Museum.Reposition(Stations, m_Origin, m_ToSession);
        }
    }

    // Billboard the card. Rotated about the world up axis only, so it
    // turns to follow you but never tips over or lies back - a panel that
    // pitches to match a phone held at chest height reads as broken.
    foreach (var panel in m_Panels)
    {
        if (panel == null)
            continue;

        var toCamera = m_Camera.transform.position - panel.position;
        toCamera.y = 0f;
        if (toCamera.sqrMagnitude > 0.0001f)
            panel.rotation = Quaternion.LookRotation(-toCamera.normalized, Vector3.up);
    }

    var local = LocalPose();
    var flat = new Vector3(local.x, 0f, local.z);

    m_MaxDistance = Mathf.Max(m_MaxDistance, flat.magnitude);

    if (m_HasSample)
        m_PathLength += Vector3.Distance(flat, m_LastSample);

    m_LastSample = flat;
    m_HasSample = true;
}

void OnEnable()
{
    if (m_Images != null)
        m_Images.trackablesChanged.AddListener(OnImagesChanged);
}

void OnDisable()
{
    if (m_Images != null)
        m_Images.trackablesChanged.RemoveListener(OnImagesChanged);
}

/// <summary>
/// Where ARCore currently believes the printed marker is, whether or not
/// it set the origin.
///
/// Recorded purely so a VPS localization can be checked against it. The
/// two are independent measurements of the same physical sheet of paper:
/// ARCore reads it optically, MultiSet derives it from the map. The gap
/// between them is the localization error in metres, measured, with no
/// eyeballing and no assumption about where anything "should" be.
```

```
///
/// Nothing downstream reads this. It cannot set or move an origin.
/// </summary>
Pose m_MarkerSeen;
bool m_MarkerSeenValid;
float m_MarkerSeenAgeSeconds;

void OnImagesChanged(ARTrackablesChangedEventArgs<ARTrackedImage> args)
{
    // Before the early return below, so the marker keeps being observed
    // after an origin exists. Observation only.
    RecordMarkerSighting(args.added);
    RecordMarkerSighting(args.updated);

    if (m_OriginSet)
        return;

    // Not until it has been chosen. See m_MarkerAllowed.
    if (m_Readiness != null && !m_MarkerAllowed)
        return;

    // Both lists, deliberately.
    //
    // The first version only looked at `added`, and skipped any image
    // whose state was not yet Tracking. That is almost always the case on
    // the very first report: ARCore adds the image as Limited while it is
    // still resolving the pose, then promotes it to Tracking in a later
    // UPDATE. So the one callback we listened to was the one callback that
    // could never succeed — the app sat there saying "point at the marker"
    // while the marker was, in fact, already detected.
    if (TryOrigin(args.added))
        return;

    TryOrigin(args.updated);
}

void RecordMarkerSighting(System.Collections.Generic.IEnumerable<ARTrackedImage> images)
{
    if (images == null)
        return;

    foreach (var image in images)
    {
        if (image == null || image.trackingState != TrackingState.Tracking)
            continue;

        m_MarkerSeen = new Pose(image.transform.position, image.transform.rotation);
        m_MarkerSeenValid = true;
        m_MarkerSeenAgeSeconds = 0f;
        return;
    }
}

/// <summary>
/// Rejects a sheet that belongs to the other church.
///
/// One APK carries both parishes, so both printed markers are in the
/// same reference image library and ARCore will happily recognise
/// either one wherever it is held up. Nothing used to check which.
```

```
///
/// That is a worse failure than it sounds, because it does not look
/// like a failure. Both sheets are printed A4 at 0.21 x 0.297, so a
/// pose read off the wrong one is geometrically perfectly valid -
/// SetOrigin simply falls past its surveyed branch (the name does not
/// match the parish) into the raw-pose fallback and lays San Roque's
/// fifteen stations out from a Mary Help sheet. The app says
/// "located", draws cards, and every one of them is in the wrong
/// place. There is no error anywhere to find afterwards.
///
/// So the sheet has to name the parish being toured. When it does not,
/// the scan is ignored and the screen says which sheet is wanted,
/// which is a thing somebody standing in a church can act on.
///
/// Where the parish has no surveyed marker - the sala, and the legacy
/// "entrance" sheet - ParishMarker returns false, there is no name to
/// insist on, and any marker is accepted exactly as before.
/// </summary>
bool MarkerBelongsHere(ARTrackedImage image)
{
    if (!ParishMarker(out _, out _, out var expected) || string.IsNullOrEmpty(expected))
        return true;

    var seen = image.referenceImage.name;
    if (seen == expected)
        return true;

    Debug.LogWarning(
        $"[ChurchOriginTest] ignoring '{seen}' marker: {ParishId} is located " +
        $"from the '{expected}' sheet. Wrong church's print.");
    m_Status = $"That is the {seen} marker. Point at the {expected} one.";
    return false;
}

bool TryOrigin(System.Collections.Generic.IEnumerable<ARTrackedImage> images)
{
    foreach (var image in images)
    {
        if (image.trackingState != TrackingState.Tracking)
            continue;

        if (!MarkerBelongsHere(image))
            continue;

        SetOrigin(FromTrackedImage(image));
        return true;
    }

    return false;
}

/// <summary>
/// +1 or -1: which way the marker's +Y (its surface normal) faces.
/// Resolved at detection time from where the camera was standing.
/// </summary>
float m_IntoRoomSign = 1f;

/// <summary>
/// Which way along the wall the survey's X runs, +1 or -1.
```

```
///
/// The into-room axis is worked out on device, by asking which side of
/// the marker the camera was on when it was detected. The sideways one
/// never was. Converting a picked point into the marker's frame needs a
/// "right" direction, and that was taken as cross(up, normal) - a
/// handedness convention that was assumed and never checked against what
/// ARCore actually reports for a tracked image.
///
/// Get it backwards and the room mirrors along the wall. A station near
/// the marker's centre line barely moves, so the nearest one still looks
/// about right, while a station two metres to one side lands four metres
/// away on the other - which is exactly how this failed.
///
/// So it is a switch rather than a guess. Stand in the room, flip it,
/// and see which way puts the cards on the real objects; the answer is
/// then a fact about this marker and can be written down.
///
/// It has now been written down: -1. A station picked on the left of
/// the scan arrived on the right of the room, on the marker route as
/// well as on the map route, which is one whole-frame mirror rather
/// than two faults. The map route is corrected where its numbers are
/// generated (Tools/deploy-stations.py, MIRROR_X); this is the same
/// correction for the route that does not go through that file.
///
/// The button in OnGUI still flips it, so the comparison can still be
/// made in the room. If it turns out to want +1 again, that is the one
/// number to change back.
/// </summary>
float m_SidewaysSign = -1f;

/// <summary>
/// Converts a survey written in natural church terms into the marker's
/// own axes.
///
/// Survey input:  X across the nave, Y up, Z into the nave.
/// Marker frame:  X along the wall, Y out of the wall, Z up.
///                (measured on device, not assumed - see AxisReport)
///
/// So Y and Z swap. Writing the survey in the marker's raw frame would
/// mean measuring "up" into Z and "forwards" into Y, which is exactly the
/// kind of thing that gets transposed on a clipboard in a dim church.
/// Measure it the natural way; let this do the swap.
/// </summary>
Vector3 ChurchToMarker(Vector3 church)
{
    return new Vector3(church.x * m_SidewaysSign, church.z * m_IntoRoomSign, church.y);
}

/// <summary>
/// Reads an ARCore tracked image into the scan record. Everything the
/// camera contributes is extracted here and nowhere else, which is what
/// lets a simulated scan drive the identical downstream code.
/// </summary>
MarkerScan FromTrackedImage(ARTrackedImage image)
{
    return new MarkerScan
    {
        trackedImageName = image.referenceImage.name,
        referenceGuid = image.referenceImage.guid.ToString(),
```

```
        pose = new Pose(image.transform.position, image.transform.rotation),
        trackingState = image.trackingState,
        // Configuration, from ChurchMarkers.asset - not measured by the
        // camera, though the pose above depends on it being right.
        physicalSize = image.referenceImage.size,
        simulated = false,
        cameraPoseAtDetection = m_Camera == null
            ? default
            : new Pose(m_Camera.transform.position, m_Camera.transform.rotation),
    };
}

/// <summary>
/// Drives the whole flow from a scan the camera never took. Same struct,
/// same SetOrigin, same everything after it.
/// </summary>
/// <summary>
/// Runs the simulated scan a moment after start when the player is
/// launched with -simulateScan, so the whole flow can be exercised
/// from a command line with no clicking and no camera.
/// </summary>
System.Collections.IEnumerator Start()
{
    var auto = false;
    foreach (var arg in System.Environment.GetCommandLineArgs())
        if (arg == "-simulateScan") auto = true;
    if (!auto) yield break;

    // One second, so the AR session has finished failing to start and
    // the log is not interleaved with its warnings.
    yield return new WaitForSeconds(1f);
    SimulateScan();
    ReportPlacements();
}

/// <summary>Prints where every station ended up, for the log.</summary>
void ReportPlacements()
{
    var log = new StringBuilder();
    // Named, because the same station lands in three different places
    // depending on which origin produced the frame, and a log that does
    // not say which one it is describing cannot be compared with the
    // log from the run before it.
    log.AppendLine($"[ChurchOriginTest] station placements via {OriginLabel}:");
    foreach (var station in Stations)
    {
        var world = SessionPositionOf(station);
        log.AppendLine(
            $"  {station.stationId,-18} surveyed({station.position.x,7:F3},{station.position.y,7:F3},{station.position.z,7:F3})" +
            $"  ->  session({world.x,7:F3},{world.y,7:F3},{world.z,7:F3})  r={station.triggerRadius:F1}m  next={station.nextStationId}");
    }
    Debug.Log(log.ToString());
}

public void SimulateScan()
{
    if (m_OriginSet)
        return;
```

```
    var scan = ScanSimulator.Sala(m_Camera == null ? null : m_Camera.transform);
    Debug.Log("[ChurchOriginTest] simulated scan input:" + System.Environment.NewLine + scan);
    SetOrigin(scan);
}

/// <summary>
/// The frame, from two taps on the floor.
///
/// Everything the marker path has to work out - which way is up, which
/// side is the room, which way X runs - is already settled here. The
/// floor gave pitch, roll and height; the two taps gave position and
/// heading; and a station's coordinates are then simply local
/// coordinates of this pose, so the conversion is the identity and
/// there is no sign anywhere that can be backwards.
/// </summary>
void OnFloorOrigin(Pose origin)
{
    m_Origin = origin;
    m_OriginSet = true;
    m_FromFloor = true;

    // The marker is not needed and should not keep scanning: it costs
    // frames, and a late detection would otherwise re-set the origin
    // out from under a frame the operator has just established.
    if (m_Images != null)
        m_Images.enabled = false;

    if (m_Readiness != null)
        m_Readiness.StopScanning();

    m_Status = "Located from the floor";
    m_Museum.SetVisible(true);
    m_ToSession = FloorConverter;
    BindWorld();
}

/// <summary>
/// Station coordinates into the floor origin's frame.
///
/// X and Z pass through untouched: both frames measure across and into
/// the room in metres, and the floor origin's pose IS that frame, so
/// there is nothing to permute and no sign that can be backwards. This
/// is the whole point of the change.
///
/// Y is the one thing that does not line up, and only because of where
/// the two frames put their zero. The existing survey measures height
/// from the PRINTED MARKER, which hangs 1.407 m up; a floor origin
/// measures it from the floor. Left alone every card would float a
/// marker's height too high.
///
/// This shim exists only while the station data is still the marker
/// survey. Re-record the stations by walking them - the recorder tags
/// them "church-v1" - and it becomes the identity and can go.
/// </summary>
static Vector3 MarkerSurveyToChurch(Vector3 surveyed)
{
    var markerHeight = StationMarker.MarkerInMapSpace.y - StationMarker.FloorY;
    return new Vector3(surveyed.x, surveyed.y + markerHeight, surveyed.z);
}
```

```
/// <summary>
/// The converter for whichever data is in play: recorded stations are
/// already in the church's frame and need nothing doing to them.
/// </summary>
System.Func<Vector3, Vector3> FloorConverter =>
    m_Recording != null && m_Recording.frame == StationRecording.ChurchFrameId
        ? (System.Func<Vector3, Vector3>)(v => v)
        : MarkerSurveyToChurch;

/// <summary>
/// The map's origin, straight from MultiSet.
///
/// The same Pose the other two routes produce, so everything
/// downstream is unchanged. What is different is where it came from:
/// nothing was hung on a wall, nothing was tapped, and no handedness
/// was assumed - isRightHanded is a documented parameter and is set.
///
/// The proof is drawn rather than asserted. A quad the size of the
/// printed sheet is placed at the marker's KNOWN map-space position. If
/// the localization is right, that quad lands on the real printed
/// marker on the wall, and any gap is the error, in the room, at a
/// glance. Nothing about the POIs is touched to do it.
/// </summary>
void OnVpsLocalized(Pose sessionFromMap, float confidence)
{
    m_Origin = sessionFromMap;
    m_OriginSet = true;
    m_FromVps = true;
    m_FromFloor = false;

    // Image tracking deliberately LEFT ON here.
    //
    // It used to be switched off the moment MultiSet answered, on the
    // grounds that the marker had been superseded. That also destroyed
    // the only independent measurement of the same physical object: the
    // printed sheet is the one thing in the room whose position both
    // systems can state - ARCore optically, MultiSet from the map - and
    // the gap between the two IS the localization error, in metres, with
    // nothing eyeballed. With tracking off there is nothing to compare
    // against.
    //
    // It cannot take the origin back: OnImagesChanged returns as soon as
    // m_OriginSet is true, and the sighting is recorded before that
    // return purely to be read.
    if (m_Readiness != null)
        m_Readiness.StopScanning();

    DrawVpsMarkerProof();

    // Stations too, now that there is a map origin to hang them from.
    // Their stored positions are in the MARKER's frame, so they are
    // converted back into map space - the exact inverse of how they
    // were derived from the picks, which round-trips to 0.3 mm
    // (Tools/check-floor-accuracy.py). The picks are used as picked.
    // Everything into map space FIRST, then the map-to-session step is
    // the identity.
    //
    // The converter used to be MarkerFrameToMap for every station on
```

```
    // this route, which is right for the compiled positions - those are
    // metres from the printed marker - and wrong for anything the
    // calibrator saved here. A station calibrated under MultiSet is
    // stored as map-v1: it is ALREADY in map space, and converting it
    // again adds the marker's own map position on top, putting it out by
    // exactly that vector, every session, for ever.
    //
    // One converter per route cannot express this, because on this route
    // the two kinds of coordinate live side by side in the same array.
    // Normalising each station by where its number came from removes the
    // question instead of answering it per-caller.
    NormaliseStationsToMapSpace();
    m_ToSession = MapSpaceIdentity;
    BindWorld();

    m_Status = $"Located by MultiSet, confidence {confidence:F2}";
    if (m_Museum != null)
        m_Museum.SetVisible(true);
}

/// <summary>
/// A quad the size of the printed marker, at the marker's map-space
/// position, facing out of the wall it hangs on.
/// </summary>
void DrawVpsMarkerProof()
{
    if (m_VpsMarkerProof != null)
        Destroy(m_VpsMarkerProof);

    m_VpsMarkerProof = new GameObject("VPS marker proof",
        typeof(MeshFilter), typeof(MeshRenderer));
    m_VpsMarkerProof.GetComponent<MeshFilter>().sharedMesh =
        Resources.GetBuiltinResource<Mesh>("Quad.fbx");

    // Map space straight through: the VPS origin IS the map's origin,
    // so a map-space point needs no conversion at all.
    m_VpsMarkerProof.transform.position =
        m_Origin.position + m_Origin.rotation * StationMarker.MarkerInMapSpace;

    var normal = m_Origin.rotation * StationMarker.MarkerNormal;
    m_VpsMarkerProof.transform.rotation = Quaternion.LookRotation(-normal, Vector3.up);
    m_VpsMarkerProof.transform.localScale = new Vector3(0.18f, 0.24f, 1f);

    Paint(m_VpsMarkerProof, new Color(1f, 0.85f, 0.2f));

    Debug.Log($"[ChurchOriginTest] marker proof at {m_VpsMarkerProof.transform.position} " +
              $"(map {StationMarker.MarkerInMapSpace})");
}

/// <summary>
/// A poster hung in the room beside one station, life-sized.
///
/// Not a thumbnail on a card - a panel standing in the air next to the
/// thing it is about, which is the point of doing this in AR at all.
/// Drawn only when the artwork is actually present, so a missing file
/// leaves the station as it was rather than an empty white rectangle.
/// </summary>
void SpawnStationPoster(string stationId, string textureName, float metresTall)
{
```

```
    var art = Resources.Load<Texture2D>(textureName);
    if (art == null)
        return;

    MuseumStation station = null;
    foreach (var s in Stations)
        if (s.stationId == stationId) { station = s; break; }
    if (station == null)
        return;

    var world = SessionPositionOf(station);

    var poster = GameObject.CreatePrimitive(PrimitiveType.Quad);
    Destroy(poster.GetComponent<Collider>());
    poster.name = $"Poster {textureName}";

    // Beside the station rather than on top of it, and at reading
    // height rather than at the floor.
    poster.transform.position = world + m_Origin.rotation * new Vector3(0.55f, 0.45f, 0f);
    poster.transform.localScale =
        new Vector3(metresTall * art.width / art.height, metresTall, 1f);

    Paint(poster, Color.white);
    poster.GetComponent<Renderer>().material.mainTexture = art;
    m_Panels.Add(poster.transform);

    Debug.Log($"[ChurchOriginTest] poster '{textureName}' beside {stationId} at {poster.transform.position}");
}

/// <summary>
/// A stored station position, which is metres from the printed marker,
/// expressed in MultiSet map space.
///
/// This is the inverse of the conversion that produced those numbers
/// from the picked points in the first place, so it returns the pick.
/// Under a VPS origin the map IS the frame, so that is all that is
/// needed - no sideways sign, no into-room sign, nothing to have
/// backwards.
/// </summary>
/// <summary>
/// Map space is the frame, so a stored position needs nothing doing to
/// it. Named rather than passed as null so the intent is on the page.
/// </summary>
static Vector3 MapSpaceIdentity(Vector3 mapSpace) => mapSpace;

/// <summary>
/// Rewrites every station's position into MultiSet map space.
///
/// A position that came from a map-v1 recording is left exactly as it
/// was saved - that is the whole promise of calibration, that what was
/// saved is what comes back. A position that came from the compiled
/// table is metres from the printed marker and is converted once.
///
/// Safe to call repeatedly: it rebuilds from the sources each time
/// rather than converting whatever happens to be in the array, so a
/// second call cannot convert an already-converted number.
/// </summary>
void NormaliseStationsToMapSpace()
{
```

```
    var recorded = m_Recording != null && m_Recording.frame == StationRecording.MapFrameId
        ? m_Recording
        : null;

    m_Stations = null;                    // rebuild from the table
    var stations = Stations;

    // The SAME table Stations was built from. This indexed
    // MuseumStations.Sala, which was the only table there was; with a
    // fifteen-station parish that is an index past the end of a
    // four-station array on the fifth station.
    var table = TableForParish;

    for (var i = 0; i < stations.Length; i++)
    {
        var source = table[i];
        if (recorded != null && recorded.TryGet(source.stationId, out var saved, out var yaw))
        {
            stations[i].position = saved;   // already map space
            stations[i].yawDegrees = yaw;
        }
        else
        {
            // A parish table is ALREADY map space - the placer picks
            // straight off the MultiSet scan. Only the sala's table is
            // metres from the printed marker and needs converting.
            // Converting an already-converted number moves every
            // station by the marker's own offset, twice.
            stations[i].position = TableIsMapSpace
                ? source.position
                : MarkerFrameToMap(source.position);
            stations[i].yawDegrees = source.yawDegrees;
        }
    }
}

static Vector3 MarkerFrameToMap(Vector3 markerFrame)
{
    var up = Vector3.up;
    var right = Vector3.Cross(up, StationMarker.MarkerNormal).normalized;
    return StationMarker.MarkerInMapSpace
         + right * markerFrame.x
         + up * markerFrame.y
         + StationMarker.MarkerNormal * markerFrame.z;
}

/// <summary>
/// This parish's printed marker on its own scan, or null for the sala.
///
/// The sala predates the placer: its stations are surveyed as metres
/// from the sheet, so its marker needs no map pose. A parish's stations
/// are map coordinates, so its marker's map pose is what connects the
/// two.
/// </summary>
bool ParishMarker(out Vector3 inMap, out Vector3 normal, out string expectedName)
{
    if (ParishId == "route-mhcp")
    {
        inMap = MuseumStations.MHCP_MarkerInMap;
```

```
        normal = MuseumStations.MHCP_MarkerNormal;
        expectedName = "mhcp";
        return true;
    }
    if (ParishId == "route-src")
    {
        inMap = MuseumStations.SRCP_MarkerInMap;
        normal = MuseumStations.SRCP_MarkerNormal;
        expectedName = "src";
        return true;
    }

    // The sala previews were missing from this list, and that is what
    // mirrored them.
    //
    // Their stations are map-frame, like every other parish's, and the
    // placer saves a marker pose for them too. But with no entry here
    // the scan fell through to the legacy route below, which hands the
    // numbers to ChurchToMarker - a converter for a CHURCH-frame survey
    // (X across, Y up, Z into the room), which permutes the axes and
    // then applies a sideways sign that was "assumed and never checked".
    // Map coordinates are not that frame, so the room came out flipped
    // along the wall: a statue placed to the left of the sheet appeared
    // to its right.
    //
    // Listed here, a preview takes the same path as its parish -
    // MapFromMarker plus MapSpaceIdentity - which is a yaw and a shift
    // and therefore cannot mirror anything.
    //
    // Same printed sheet as the parish it previews, so the same
    // reference image name.
    if (ParishId == k_MhcpPreviewId)
    {
        inMap = MuseumStations.MHCP_IN_SALA_MarkerInMap;
        normal = MuseumStations.MHCP_IN_SALA_MarkerNormal;
        expectedName = "mhcp";
        return true;
    }
    if (ParishId == k_SrcPreviewId)
    {
        inMap = MuseumStations.SRCP_IN_SALA_MarkerInMap;
        normal = MuseumStations.SRCP_IN_SALA_MarkerNormal;
        expectedName = "src";
        return true;
    }

    inMap = default;
    normal = default;
    expectedName = null;
    return false;
}

/// <summary>
/// The marker's pose in map space, built the way ARCore reports a
/// tracked image: +Y out of the sheet, +Z along the sheet's own up.
///
/// That convention is not invented here - it is the one ChurchToMarker
/// already encodes (church up lands in the marker's Z, church "into the
/// room" in its Y) and which was measured on the device rather than
```

```
/// assumed.
/// </summary>
/// <summary>
/// The map's pose in this session, from one printed sheet.
///
/// Both frames are gravity-aligned, so what separates them is a yaw and
/// a shift - nothing else. The yaw comes from one horizontal direction
/// known in both: the way the sheet faces. In map space that is the
/// normal saved with it in the placer; in session space it is the
/// tracked image's up axis, which is how ARCore reports the direction
/// out of a printed image.
///
/// `observer` is where the camera was when the sheet was seen. ARCore
/// can report the normal pointing into the wall rather than out of it,
/// and the reader is unambiguously in the room, so that settles it.
/// Without this the map is placed 180 degrees out and every station
/// lands behind the wall it should be in front of.
/// </summary>
static Pose MapFromMarker(Vector3 markerInMap, Vector3 markerNormal,
                          Pose markerInSession, Vector3 observer)
{
    var inMap = Flatten(markerNormal, Vector3.forward);

    var inSession = Flatten(markerInSession.rotation * Vector3.up, Vector3.forward);
    var toCamera = observer - markerInSession.position;
    toCamera.y = 0f;
    if (toCamera.sqrMagnitude > 1e-6f && Vector3.Dot(inSession, toCamera) < 0f)
        inSession = -inSession;

    var yaw = Vector3.SignedAngle(inMap, inSession, Vector3.up);
    var rotation = Quaternion.AngleAxis(yaw, Vector3.up);

    Debug.Log($"[ChurchOriginTest] marker yaw {yaw:F1} deg  " +
              $"(map normal {inMap}, session normal {inSession})");

    return new Pose(markerInSession.position - rotation * markerInMap, rotation);
}

/// <summary>A direction flattened onto the horizontal plane.</summary>
static Vector3 Flatten(Vector3 v, Vector3 fallback)
{
    v.y = 0f;
    return v.sqrMagnitude < 1e-6f ? fallback : v.normalized;
}

void SetOrigin(MarkerScan scan)
{
    // A parish marker carries the whole map with it.
    if (ParishMarker(out var markerInMap, out var markerNormal, out var expected)
        && scan.trackedImageName == expected)
    {
        var seenFrom = scan.cameraPoseAtDetection.position;
        if (seenFrom == Vector3.zero && m_Camera != null)
            seenFrom = m_Camera.transform.position;

        var sessionFromMap = MapFromMarker(
            markerInMap, markerNormal, scan.pose, seenFrom);

        Debug.Log($"[ChurchOriginTest] marker '{expected}' places {ParishId} " +
```

```
            $"from map {markerInMap} -> session {sessionFromMap.position}");

    // The sign hangs on the sheet, so the museum needs the sheet's
    // own map pose - not the map origin, which is where its
    // coordinates start and is nowhere in particular.
    if (m_Museum != null) m_Museum.SetMarkerSign(markerInMap, markerNormal);

    m_Origin = sessionFromMap;
    m_OriginSet = true;
    m_FromVps = true;        // map-v1: the frame is the map, as with VPS
    m_FromFloor = false;
    m_ToSession = MapSpaceIdentity;

    if (m_Images != null) m_Images.enabled = false;
    if (m_Readiness != null) m_Readiness.StopScanning();

    m_Status = $"Located from the {expected} marker";
    if (m_Museum != null) m_Museum.SetVisible(true);
    BindWorld();
    return;
}

m_Origin = scan.pose;
m_OriginSet = true;

// The marker route skips the checklist, so the UI has to come back.
if (m_Museum != null)
    m_Museum.SetVisible(true);

// Which side of the marker is "the room"? The normal can face either
// way depending on how ARCore resolves the image, and guessing wrong
// puts every station inside the wall. The camera is unambiguously in
// the room at the moment of detection, so use that.
var observer = scan.cameraPoseAtDetection.position;
if (observer == Vector3.zero && m_Camera != null)
    observer = m_Camera.transform.position;
var toCamera = observer - m_Origin.position;
var normal = m_Origin.rotation * Vector3.up;
m_IntoRoomSign = Vector3.Dot(normal, toCamera) < 0f ? -1f : 1f;
Debug.Log($"[ChurchOriginTest] into-room sign = {m_IntoRoomSign:+0;-0}");

// The three coloured bars are gone.
//
// They were a survey instrument: 30 cm of red, green and blue along
// the marker's local axes, read once through the phone to find out
// which way the frame actually pointed before anything was placed.
// That question is answered and the answer is in the tables. What
// was left was three saturated bars across the middle of a visitor's
// first view of the church.
//
// The marker already carries the SanctiWalk sign, spawned by
// MuseumUI.SetMarkerSign below, which is the thing worth seeing
// when a scan lands.

// Nothing is drawn per checkpoint any more. A cyan cube for raw
// tracking beside an orange one for the ARCore anchor showed drift
// at a glance, and their separation answered a real Phase 1
// question - but two saturated 20 cm boxes at every station is what
// the room looked like, and each already has a card on a stalk
```

```
    // saying what it is.
    //
    // The measurement is untouched: LogCheckpoint compares the camera's
    // pose against surveyed[] and never read the cubes. What is lost is
    // seeing raw against anchored, which is one revert away.

    // The pre-rendered panel quads are gone. This method's own comment
    // said "Phase 4 builds the real version of this, driven by content"
    // - StationBubble is that version, spawned per station by
    // MuseumUI.Bind below. Drawing both would put a texture of a card
    // and a real card in the same spot.
    //
    // SpawnInfoPanel is kept rather than deleted: it is the fallback if
    // the world-space canvases turn out to misbehave on this hardware,
    // which is not yet tested.

    // Hand the stations to the UI in the same frame the cubes use, so a
    // card can never disagree with the marker it is standing next to.
    m_ToSession = ChurchToMarker;
    BindWorld();

    // See the class comment. This is what makes it a SLAM test.
    m_Images.enabled = false;

    m_Status = "Origin set. Walk the route.";
}

/// <summary>
/// The station information panel, anchored beside the marker.
///
/// This is the first thing in the project that looks like the actual
/// product rather than a diagnostic: a card about Maria Auxiliadora that
/// stays on the wall as you walk past it, instead of a rectangle stuck to
/// the screen. Phase 4 builds the real version of this, driven by
/// Firestore content; here it is one pre-rendered texture on one quad,
/// which is enough to prove the anchoring works and to see it.
///
/// Placed AT checkpoint 1, at eye height, and billboarded so it always
/// turns to face the viewer.
///
/// It started flat on the wall beside the marker, which was wrong twice
/// over: you had to already be facing the marker to find it, and it was
/// invisible from the side. A pilgrim walking a church is not going to
/// hunt for a card. Putting it on the first station and letting it turn
/// toward the reader is both easier to find and what the real feature
/// will do — station content should be readable from wherever you are
/// standing when you arrive.
/// </summary>
void SpawnInfoPanel(string textureName, int stationIndex)
{
    var texture = Resources.Load<Texture2D>(textureName);
    if (texture == null)
    {
        Debug.LogWarning($"[ChurchOriginTest] panel texture '{textureName}' missing from Resources.");
        return;
    }

    const float widthMetres = 0.80f;
    var heightMetres = widthMetres * texture.height / texture.width;
```

```
    var panel = GameObject.CreatePrimitive(PrimitiveType.Quad);
    panel.name = "InfoPanel";
    Object.Destroy(panel.GetComponent<Collider>());

    // Just above the station cubes.
    //
    // This was +1.4 m "to eye height", which was wrong: the survey's Y is
    // measured FROM THE MARKER, not from the floor. With the marker
    // mounted high on a cabinet the cards ended up near the ceiling. The
    // cubes sit at the station height and are comfortable to look at, so
    // the card sits a little above them and inherits whatever height the
    // marker was mounted at.
    var station = surveyed[stationIndex];
    panel.transform.position = m_Origin.position
        + m_Origin.rotation * ChurchToMarker(new Vector3(station.x, station.y + 0.30f, station.z));

    panel.transform.localScale = new Vector3(widthMetres, heightMetres, 1f);
    m_Panels.Add(panel.transform);
    if (textureName == "panel-santo-nino")
        m_NinoPanel = panel.transform;

    // Unlit and double-sided would be ideal; Unlit/Texture is unlit and
    // single-sided, which is correct here - the card should not be
    // readable through the wall from behind.
    var shader = Shader.Find("Unlit/Texture") ?? Shader.Find("Unlit/Color");
    panel.GetComponent<Renderer>().material = new Material(shader) { mainTexture = texture };

    Debug.Log($"[ChurchOriginTest] panel '{textureName}' at station {stationIndex + 1}, {widthMetres:F2} x {heightMetres:F2} m");
}



static Shader s_Unlit;

static void Paint(GameObject go, Color colour)
{
    // Unlit, deliberately.
    //
    // The scene is built from an EMPTY scene — there is no light in it,
    // and there should not be: this is an AR overlay, not a lit 3D world.
    // Unity's default material for a runtime primitive uses the *lit*
    // Standard shader, so every cube and pole rendered pure black and was
    // invisible against a dark camera feed. They were being placed
    // correctly the entire time; nobody could see them.
    //
    // Unlit/Color ignores lighting entirely, so a marker is the colour it
    // is told to be, in a dim church as much as in a bright room. It is
    // registered in Always Included Shaders by ARProjectSetup so the build
    // cannot strip it.
    if (s_Unlit == null)
        s_Unlit = Shader.Find("Unlit/Color");

    var renderer = go.GetComponent<Renderer>();
    renderer.material = s_Unlit != null
        ? new Material(s_Unlit) { color = colour }
        : renderer.material;
    renderer.material.color = colour;
```

```
    Object.Destroy(go.GetComponent<Collider>());
}

/// <summary>Device position expressed in the marker's frame.</summary>
/// <summary>
/// Where the camera is, in CHURCH terms: X across, Y up, Z into the
/// nave. Everything that reads a position - the on-screen x and z, the
/// checkpoint error, and what the recorder writes down - comes through
/// here, so it has to be right for whichever origin is in use.
///
/// The two origins do NOT need the same arithmetic, and assuming they
/// did produced a recording with the height and the depth swapped: Z
/// sat at 1.4 m for all four stations while Y swung three metres, when
/// Y is a phone held at a steady height and Z is how far someone
/// walked. Obvious in the data, invisible in the code.
///
/// A tracked image's frame has its local Z up the wall and its local Y
/// out of it, so the marker path has to permute them, and the signs
/// exist because ARCore can resolve the image either way round. A floor
/// origin is built by LookRotation(forward, up): its local Y is already
/// up and its local Z is already forward. There is nothing to permute
/// and no sign to apply - which is the entire reason the floor origin
/// is more trustworthy, so applying the marker's corrections to it
/// threw away the thing that made it better.
/// </summary>
Vector3 LocalPose()
{
    if (m_FromFloor)
        return ChurchFrame.ToChurch(m_Origin, m_Camera.transform.position);

    var raw = Quaternion.Inverse(m_Origin.rotation) * (m_Camera.transform.position - m_Origin.position);
    return new Vector3(raw.x * m_SidewaysSign, raw.z, raw.y * m_IntoRoomSign);
}

void PlaceNinoHere()
{
    if (m_NinoPanel == null || m_Camera == null)
        return;

    var cam = m_Camera.transform;
    var ahead = cam.position + cam.forward * k_PlaceDistance;

    // Keep it upright regardless of how the phone was tilted when aimed.
    ahead.y = cam.position.y;
    m_NinoPanel.position = ahead;

    var church = Quaternion.Inverse(m_Origin.rotation) * (ahead - m_Origin.position);
    var placed = m_FromFloor
        ? church
        : new Vector3(church.x * m_SidewaysSign, church.z, church.y * m_IntoRoomSign);

    m_Log.AppendLine($"Santo Nino placed at  x {placed.x:F2}  y {placed.y:F2}  z {placed.z:F2}");
    Debug.Log($"[ChurchOriginTest] Santo Nino station: " +
              $"new Vector3({placed.x:F2}f, {placed.y:F2}f, {placed.z:F2}f)");
}

/// <summary>
/// Write down where we are standing as the next station's position.
///
```

```
/// LocalPose is already in the frame MuseumStation.position uses - X
/// across, Y up, Z into the room, metres from the marker - so what the
/// phone reports is the value that belongs in the table, with no
/// conversion and nothing to copy by hand.
/// </summary>
void RecordStationHere()
{
    if (!m_OriginSet || m_RecordIndex >= Stations.Length)
        return;

    var station = Stations[m_RecordIndex];
    var here = LocalPose();

    m_Recording ??= new RecordedStationSet { parishId = ParishId, frame = CurrentFrameId };
    m_Recording.frame = CurrentFrameId;
    m_Recording.Set(station.stationId, here);

    // Drop the cache so the bubbles and the museum UI pick the new
    // position up on the next bind rather than the one recorded over.
    m_Stations = null;

    m_ConfirmClear = false;
    m_RecordNote = $"{station.stationName}: x {here.x:F2}  z {here.z:F2}";
    Debug.Log($"[ChurchOriginTest] recorded {station.stationId} at " +
              $"({here.x:F3}, {here.y:F3}, {here.z:F3})");

    m_RecordIndex++;
}

/// <summary>
/// Put the stations back where the current axis signs say they go.
///
/// Bind rebuilds the bubbles from the conversion, so calling it again
/// after a sign changes is what moves them. Without this the flip would
/// change the readout and leave every card where it was.
/// </summary>
void RebindStations()
{
    if (!m_OriginSet || m_Museum == null)
        return;

    // Only the marker route is re-pointed at ChurchToMarker. Setting it
    // unconditionally meant that tapping the sideways-flip button while
    // running on a map origin did not flip anything - it swapped the
    // whole converter, and ChurchToMarker also exchanges y and z. Every
    // card moved, none of them to the right place, and the button
    // looked like it had corrupted the layout rather than mirrored it.
    if (!m_FromVps)
        m_ToSession = ChurchToMarker;

    BindWorld();
}

void SaveRecording()
{
    if (m_Recording == null || m_Recording.stations.Count == 0)
    {
        m_RecordNote = "Nothing recorded yet.";
        return;
```

```
    }

    m_RecordNote = StationRecording.Save(m_Recording, out var path)
        ? $"Saved {m_Recording.stations.Count} to {System.IO.Path.GetFileName(path)}"
        : "Could not save - see the log.";
}

void LogCheckpoint()
{
    if (!m_OriginSet || m_NextCheckpoint >= surveyed.Length)
        return;

    var actual = LocalPose();
    var expected = surveyed[m_NextCheckpoint];

    // Horizontal error only. Height is not what navigation cares about and
    // holding a phone at a different height would pollute the number.
    var error = new Vector2(actual.x - expected.x, actual.z - expected.z).magnitude;

    m_Log.AppendLine(
        $"CP{m_NextCheckpoint + 1}  want({expected.x:F2},{expected.z:F2})  " +
        $"got({actual.x:F2},{actual.z:F2})  ERR {error:F2} m");

    Debug.Log($"[ChurchOriginTest] {m_Log}");
    m_NextCheckpoint++;
}

/// <summary>
/// Session diagnostics, shown at all times.
///
/// The first version only reported state once the marker had been found,
/// which made a failure impossible to diagnose: the screen said "point the
/// camera at the marker" whether the marker was unreadable, the library was
/// missing, or — as actually happened — ARCore had never achieved world
/// tracking at all and image detection was never going to run. The device
/// log said "VisualInertialState is kNotTracking" while the app said
/// nothing. Never make the person holding the phone guess.
/// </summary>
/// <summary>
/// Everything needed to attribute a VPS error to a stage, on one screen.
///
/// Deliberately raw. Each line is a measurement, not a conclusion, and
/// the point is to find WHICH of these is wrong before changing any of
/// them - the previous rounds of this problem were lost to adjusting a
/// sign, re-testing, and learning nothing because two things had moved.
///
/// Reading it:
///
/// * SKEW should be 0.000. Anything else means the uploaded picture and
///   the camera pose it was paired with are from different frames, and
///   the map pose is being attached to the wrong place. Turning the
///   phone at 30 deg/s for one frame at 30 fps is a degree of error on
///   its own.
/// * MARKER ERR is the headline. ARCore sees the printed marker
///   optically; MultiSet predicts where it should be from the map. If
///   this is small the localization is good and the error is downstream;
///   if it is large the localization itself is wrong and nothing
///   downstream can fix it.
/// * MARKER ANG separates the two kinds of wrong: a position error with
```

```
///   a small angle is an offset, and calibration absorbs it. A large
///   angle is a rotation, which grows with distance and cannot be
///   calibrated away with a single offset.
/// * MOVED is how far the phone travelled while the query was in
///   flight. It does not enter the maths, but it says how much ARCore
///   tracking had to be right about for the answer to still apply.
/// </summary>
string VpsDiagnostics()
{
    if (m_Vps == null || !m_Vps.PoseFound)
        return "";

    var sb = new StringBuilder();
    var mapRotEuler = m_Vps.MapRotation.eulerAngles;
    var camEuler = m_Vps.QueryCameraRotation.eulerAngles;
    var originEuler = m_Origin.rotation.eulerAngles;

    sb.AppendLine("VPS FOUND");
    sb.AppendLine($"  conf {m_Vps.Confidence:F3}   {m_Vps.LastResponseMs} ms   " +
                  $"sent {m_Vps.SentSize.x}x{m_Vps.SentSize.y}");
    sb.AppendLine($"  Map        {m_Vps.MapPosition.x,8:F3}{m_Vps.MapPosition.y,8:F3}{m_Vps.MapPosition.z,8:F3}");
    sb.AppendLine($"  Map Rot    {mapRotEuler.x,8:F1}{mapRotEuler.y,8:F1}{mapRotEuler.z,8:F1}");
    sb.AppendLine($"  Cam Pos    {m_Vps.QueryCameraPosition.x,8:F3}{m_Vps.QueryCameraPosition.y,8:F3}{m_Vps.QueryCameraPosition.z,8:F3}");
    sb.AppendLine($"  Cam Rot    {camEuler.x,8:F1}{camEuler.y,8:F1}{camEuler.z,8:F1}");
    sb.AppendLine($"  VPS Origin {m_Origin.position.x,8:F3}{m_Origin.position.y,8:F3}{m_Origin.position.z,8:F3}");
    sb.AppendLine($"  Origin Rot {originEuler.x,8:F1}{originEuler.y,8:F1}{originEuler.z,8:F1}");
    sb.AppendLine($"  SKEW       {m_Vps.FrameSkewSeconds,8:F3} s      MOVED {m_Vps.MovedDuringQuery:F2} m");
    sb.AppendLine($"  fx fy px py  {m_Vps.SentIntrinsics.x:F1} {m_Vps.SentIntrinsics.y:F1} " +
                  $"{m_Vps.SentIntrinsics.z:F1} {m_Vps.SentIntrinsics.w:F1}" +
                  $"   sensor {m_Vps.SensorSize.x:F0}x{m_Vps.SensorSize.y:F0}");

    // The measured error, when both estimates of the marker exist.
    var predicted = m_Origin.position + m_Origin.rotation * StationMarker.MarkerInMapSpace;
    sb.AppendLine($"  Marker pred{predicted.x,8:F3}{predicted.y,8:F3}{predicted.z,8:F3}");

    if (m_MarkerSeenValid)
    {
        var metres = Vector3.Distance(predicted, m_MarkerSeen.position);

        // The marker's outward normal as the map knows it, put into the
        // session by the VPS origin, against the one ARCore reports. An
        // ARCore tracked image's +Z points out of its face.
        var predictedNormal = m_Origin.rotation * StationMarker.MarkerNormal;
        var seenNormal = m_MarkerSeen.rotation * Vector3.forward;
        var degrees = Vector3.Angle(predictedNormal, seenNormal);

        sb.AppendLine($"  Marker seen{m_MarkerSeen.position.x,8:F3}{m_MarkerSeen.position.y,8:F3}{m_MarkerSeen.position.z,8:F3}" +
                      $"   ({m_MarkerSeenAgeSeconds:F0}s ago)");
        sb.AppendLine($"  MARKER ERR {metres,8:F2} m     MARKER ANG {degrees,6:F1} deg");
    }
    else
    {
        sb.AppendLine("  Marker seen  -  not tracked this session, so no measured error.");
        sb.AppendLine("  Point at the printed marker to get one.");
    }

    return sb.ToString();
}
```

```
string OriginLabel => m_FromVps ? "MULTISET VPS"
                    : m_FromFloor ? "floor"
                    : "MARKER - reduced accuracy";

/// <summary>
/// Everything the localization test is judged on, in one block: whether
/// a pose was found, how confident it was, where MultiSet says the
/// phone is in the map, and what that became in Unity. Kept on screen
/// after success so it can be read while walking.
/// </summary>
string VpsReadout()
{
    if (m_Vps == null || !m_Vps.PoseFound)
        return "";

    var e = m_Origin.rotation.eulerAngles;
    return $"VPS FOUND   confidence {m_Vps.Confidence:F3}   {m_Vps.LastResponseMs} ms\n" +
           $"MultiSet XYZ   {m_Vps.MapPosition.x:F3}  {m_Vps.MapPosition.y:F3}  {m_Vps.MapPosition.z:F3}\n" +
           $"Unity position {m_Origin.position.x:F3}  {m_Origin.position.y:F3}  {m_Origin.position.z:F3}\n" +
           $"Unity rotation {e.x:F1}  {e.y:F1}  {e.z:F1}\n";
}

string Diagnostics()
{
    var state = ARSession.state;
    var reason = ARSession.notTrackingReason;
    var images = m_Images != null ? m_Images.trackables.count : -1;
    var libCount = m_Images != null && m_Images.referenceLibrary != null
        ? m_Images.referenceLibrary.count
        : -1;

    var line = m_LaunchedForChurch != null
        ? $"from SanctiWalk: {m_LaunchedForChurch}\nsession: {state}"
        : $"session: {state}";
    if (state != ARSessionState.SessionTracking)
        line += $"  ({reason})";

    line += $"\nlibrary: {libCount} image(s)   detected: {images}";

    if (m_OriginSet)
    {
        line += $"\nmax dist {m_MaxDistance:F2} m   walked {m_PathLength:F2} m";
        line += "\n" + AxisReport();
    }

    if (state != ARSessionState.SessionTracking)
        line += "\n\nTRACKING NOT ESTABLISHED.\nPoint at a well-lit area with texture\nand 3D shape, and move slowly sideways.";
    else if (!m_OriginSet)
        line += "\n\nTracking OK. Now point at the marker.";

    return line;
}

/// <summary>
/// Which way the marker's axes actually point, measured against gravity.
///
/// Reading this off a photograph of three coloured bars is guesswork, and
/// I guessed wrong once already: I assumed +Z ran into the nave, when on
/// a wall-mounted marker it runs straight up the wall. That put every
```

```
/// test checkpoint metres up in the air. Unity's world +Y is gravity-up
/// on a tracked device, so a dot product settles it in one line and the
/// survey can be written down with confidence.
/// </summary>
string AxisReport()
{
    var x = m_Origin.rotation * Vector3.right;
    var y = m_Origin.rotation * Vector3.up;
    var z = m_Origin.rotation * Vector3.forward;

    string Describe(Vector3 axis)
    {
        var up = Vector3.Dot(axis, Vector3.up);
        if (up > 0.7f) return "UP";
        if (up < -0.7f) return "DOWN";
        return "horizontal";
    }

    // The horizontal axis pointing away from the wall is the one to use
    // for "into the nave" when surveying.
    var report = $"axes:  +X {Describe(x)}   +Y {Describe(y)}   +Z {Describe(z)}";

    var intoRoom = Mathf.Abs(Vector3.Dot(y, Vector3.up)) < 0.7f ? "+Y"
                 : Mathf.Abs(Vector3.Dot(z, Vector3.up)) < 0.7f ? "+Z"
                 : "+X";
    report += $"\nsurvey 'into the nave' along: {intoRoom}";
    return report;
}

/// <summary>
/// Whether the developer overlay is on screen. Off by default.
///
/// Everything technical lives behind this: the readiness checklist,
/// session state, library counts, tracking status, coordinates, the VPS
/// diagnostic block. None of it means anything to someone visiting a
/// church, and all of it was the first thing they saw.
///
/// It is hidden rather than deleted, because it is how this project is
/// debugged and the VPS error is still being chased.
/// </summary>
bool m_DevOverlay;

int m_DevTaps;
float m_DevTapWindow;

/// <summary>
/// Three taps in the top-right corner within two seconds.
///
/// A corner rather than a visible button: a control labelled
/// "developer" on a church wall is a thing visitors press. Three taps so
/// it cannot happen while steadying the phone, and a window so two
/// accidental taps an hour apart never add up to it.
/// </summary>
void PollDevGesture()
{
    if (Input.touchCount != 1) return;
    var touch = Input.GetTouch(0);
    if (touch.phase != TouchPhase.Began) return;
```

```
    if (touch.position.x < Screen.width * 0.8f || touch.position.y < Screen.height * 0.88f)
        return;

    if (Time.unscaledTime > m_DevTapWindow) m_DevTaps = 0;
    m_DevTaps++;
    m_DevTapWindow = Time.unscaledTime + 2f;

    if (m_DevTaps >= 3)
    {
        m_DevTaps = 0;
        m_DevOverlay = !m_DevOverlay;
        Debug.Log($"[ChurchOriginTest] developer overlay {(m_DevOverlay ? "on" : "off")}");
    }
}

/// <summary>
/// What a visitor sees: an instruction, and nothing else.
///
/// The clean screen is NOT an empty one. Every control this app has
/// lived in the developer overlay, so hiding that without putting
/// something in its place would leave a camera with no way to localize,
/// no way to calibrate and no way back - a prettier dead end. This is
/// the smallest thing that keeps the app operable while the technical
/// readout is gone.
/// </summary>
void DrawVisitorUI()
{
    GUILayout.BeginArea(new Rect(24, Screen.height - 440, Screen.width - 48, 400));

    if (!m_OriginSet)
    {
        GUI.backgroundColor = new Color(0f, 0f, 0f, 0.55f);
        GUILayout.Label("Point your camera at the church marker", m_Text);
        GUILayout.Space(14);

        if (m_Vps != null && GUILayout.Button(
                m_Vps.Busy ? "Finding the church…" : "Start AR here",
                m_Button, GUILayout.Height(120), GUILayout.Width(640)))
        {
            if (!m_Vps.Busy) m_Vps.Localize();
        }
    }
    else
    {
        GUILayout.Label("Walk up to a station to read about it.", m_Text);
        GUILayout.Space(14);

        if (GUILayout.Button("AR SETUP", m_Button, GUILayout.Height(110), GUILayout.Width(420)))
        {
            m_Calibrator.SetActive(true);
            m_Calibrator.Bind(Stations, m_Camera);
            m_CalibrationNote = "";
        }
    }

    GUILayout.EndArea();
}

void OnGUI()
```

```
{
    if (m_Text == null)
    {
        m_Text = new GUIStyle(GUI.skin.label) { fontSize = 34 };
        m_Text.normal.textColor = Color.white;
        m_Button = new GUIStyle(GUI.skin.button) { fontSize = 38 };
    }

    // ArFlowUI owns every screen a person is meant to use now, so the
    // overlay draws only when it is explicitly asked for. It is still
    // here because the VPS error is still being chased and this is what
    // the numbers are read off.
    if (!m_DevOverlay)
        return;

    if (m_Text == null)
    {
        m_Text = new GUIStyle(GUI.skin.label) { fontSize = 34 };
        m_Text.normal.textColor = Color.white;
        m_Button = new GUIStyle(GUI.skin.button) { fontSize = 38 };
    }

    var pose = m_OriginSet ? LocalPose() : Vector3.zero;

    GUILayout.BeginArea(new Rect(24, 24, Screen.width - 48, Screen.height - 48));

    GUI.backgroundColor = Color.black;

    // Before an origin exists, the screen is the checklist and one
    // instruction. The rows are now information rather than a barrier:
    // they say how well the room is understood, and the marker can set
    // the origin at any point regardless - it is the default route and
    // waiting on the floor was what stalled it.
    if (!m_OriginSet && m_Readiness != null)
    {
        GUILayout.Label("Getting your bearings", m_Text);
        GUILayout.Space(10);
        foreach (var (label, done) in m_Readiness.Checklist())
            GUILayout.Label($"   {(done ? "[x]" : "[  ]")}  {label}", m_Text);

        GUILayout.Space(14);
        GUILayout.Label(m_Picker != null && m_Readiness.Current.IsReady
            ? m_Picker.Note
            : m_Readiness.Current.Instruction, m_Text);

        // MultiSet is offered from the start: it needs a camera frame
        // and a network, not a floor plane, so waiting on the
        // checklist would be waiting for something it does not use.
        if (m_Vps != null)
        {
            GUILayout.Space(16);
            if (GUILayout.Button(m_Vps.Busy ? "Asking MultiSet…" : "Localize with MultiSet (Sala)",
                    m_Button, GUILayout.Height(110), GUILayout.Width(900)))
            {
                if (!m_Vps.Busy) m_Vps.Localize();
            }
            GUILayout.Space(8);
            GUILayout.Label($"MultiSet: {m_Vps.Status}" +
                           (m_Vps.Attempts > 0 ? $"   (tries: {m_Vps.Attempts})" : ""), m_Text);
```

```
    }

    // The floor origin is the option now, not the default. Shown
    // only once the room is understood enough for a floor raycast
    // to mean anything, because offering it before then is
    // offering something that cannot work yet.
    if (m_Readiness.Current.IsReady)
    {
        GUILayout.Space(16);
        GUILayout.Label(m_Picker != null ? m_Picker.Note : "", m_Text);
        GUILayout.Space(8);
        GUILayout.Label("— or point at the printed marker —", m_Text);
    }

    GUILayout.Space(16);
    GUILayout.Label(Diagnostics(), m_Text);
    GUILayout.EndArea();
    return;
}

GUILayout.Label(m_OriginSet
    ? $"x {pose.x:F2}   z {pose.z:F2}   [{OriginLabel}]\n{VpsDiagnostics()}{Diagnostics()}\n" +
      $"next: CP{Mathf.Min(m_NextCheckpoint + 1, surveyed.Length)} of {surveyed.Length}"
    : Diagnostics(), m_Text);

GUILayout.Space(18);

if (m_OriginSet && m_NinoPanel != null)
{
    if (GUILayout.Button($"Put Santo Nino card here ({k_PlaceDistance:F1} m ahead)",
            m_Button, GUILayout.Height(110), GUILayout.Width(760)))
    {
        PlaceNinoHere();
    }

    GUILayout.Space(14);
}

// Re-query while located. Walking to another part of the room and
// asking again is the actual test - does MultiSet still know where
// the phone is - and it has to be possible without restarting.
if (m_OriginSet && m_Vps != null)
{
    if (GUILayout.Button(m_Vps.Busy ? "ASKING MULTISET…" : ">> USE MULTISET VPS (Sala) <<",
            m_Button, GUILayout.Height(120), GUILayout.Width(900)))
    {
        if (!m_Vps.Busy) m_Vps.Localize();
    }
    GUILayout.Label($"MultiSet: {m_Vps.Status}", m_Text);
    GUILayout.Space(12);
}

// ------------------------------------------------------ calibration
//
// The admin position editor. Off by default and unmistakable when
// on, because everything in it writes to the file the whole parish
// will be reading tomorrow.
if (m_OriginSet)
{
```

```
    if (GUILayout.Button(m_Calibrator.Active ? "CLOSE CALIBRATION" : "CALIBRATE STATIONS",
            m_Button, GUILayout.Height(110), GUILayout.Width(640)))
    {
        m_Calibrator.SetActive(!m_Calibrator.Active);
        if (m_Calibrator.Active)
        {
            m_Calibrator.Bind(Stations, m_Camera);
            m_CalibrationNote = "";
        }
    }
    GUILayout.Space(10);
}

if (m_OriginSet && m_Calibrator.Active)
{
    var station = m_Calibrator.Selected;

    GUILayout.Label($"CHURCH:  {ParishId}", m_Text);
    GUILayout.Label($"ORIGIN:  {OriginLabel}    FRAME: {CurrentFrameId}", m_Text);
    GUILayout.Label(
        station != null ? $"STATION: {station.stationName}" : "STATION: none", m_Text);
    GUILayout.Space(6);

    // The warning the spec asks for, and it earns its place: this
    // writes the position every later visit will use.
    GUILayout.Label("Move the card until it is aligned with the real", m_Text);
    GUILayout.Label("object. Save only when it is correctly placed.", m_Text);
    GUILayout.Space(10);

    GUILayout.BeginHorizontal();
    if (GUILayout.Button("< PREV", m_Button, GUILayout.Height(96), GUILayout.Width(280)))
        m_Calibrator.SelectPrevious();
    if (GUILayout.Button("NEXT >", m_Button, GUILayout.Height(96), GUILayout.Width(280)))
        m_Calibrator.SelectNext();
    GUILayout.EndHorizontal();
    GUILayout.Space(8);

    if (station != null)
    {
        var p = m_Calibrator.PositionOf(station.stationId);
        GUILayout.Label(
            $"POSITION  X {p.x:F3}   Y {p.y:F3}   Z {p.z:F3}", m_Text);
        GUILayout.Label(
            $"ROTATION  Y {m_Calibrator.YawOf(station.stationId):F0} deg" +
            $"    from origin {p.magnitude:F2} m", m_Text);
        GUILayout.Space(8);

        // Typed entry, for when a number is known rather than eyeballed.
        m_Calibrator.DrawNumberFields(m_Text, 200);
        if (GUILayout.Button("APPLY TYPED", m_Button, GUILayout.Height(84), GUILayout.Width(360)))
        {
            m_Calibrator.CommitNumberFields();
            ApplyCalibrationLive();
        }
        GUILayout.Space(8);

        // Nudges, in the frame's own axes, for the last few
        // centimetres that a finger drag cannot land.
        // Named for what they do in the room, and mapped to where
```

```
        // the admin is standing rather than to the map's axes.
        GUILayout.BeginHorizontal();
        if (GUILayout.Button("<- LEFT", m_Button, GUILayout.Height(84), GUILayout.Width(200)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.left, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        if (GUILayout.Button("RIGHT ->", m_Button, GUILayout.Height(84), GUILayout.Width(200)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.right, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        if (GUILayout.Button("FORWARD", m_Button, GUILayout.Height(84), GUILayout.Width(220)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.forward, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        if (GUILayout.Button("BACK", m_Button, GUILayout.Height(84), GUILayout.Width(200)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.back, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        GUILayout.EndHorizontal();

        GUILayout.BeginHorizontal();
        if (GUILayout.Button("HEIGHT -", m_Button, GUILayout.Height(84), GUILayout.Width(220)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.down, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        if (GUILayout.Button("HEIGHT +", m_Button, GUILayout.Height(84), GUILayout.Width(220)))
        { m_Calibrator.NudgeFromWhereIStand(Vector3.up, m_Origin, m_ToSession); ApplyCalibrationLive(); }
        if (GUILayout.Button("ROTATE L", m_Button, GUILayout.Height(84), GUILayout.Width(210)))
        { m_Calibrator.NudgeYaw(-1f); ApplyCalibrationLive(); }
        if (GUILayout.Button("ROTATE R", m_Button, GUILayout.Height(84), GUILayout.Width(210)))
        { m_Calibrator.NudgeYaw(1f); ApplyCalibrationLive(); }
        GUILayout.EndHorizontal();
        GUILayout.Space(8);

        // Standing at the object beats dragging it whenever you can
        // actually stand there, so it stays available here too.
        if (GUILayout.Button("PUT IT WHERE I AM STANDING",
                m_Button, GUILayout.Height(96), GUILayout.Width(760)))
        {
            m_Calibrator.PutWhereIAmStanding(m_Origin, m_ToSession);
            ApplyCalibrationLive();
        }
        GUILayout.Space(8);

        GUILayout.BeginHorizontal();
        if (GUILayout.Button("RESET", m_Button, GUILayout.Height(96), GUILayout.Width(250)))
        { m_Calibrator.ResetSelected(Surveyed); ApplyCalibrationLive(); }
        if (GUILayout.Button("SAVE POSITION", m_Button, GUILayout.Height(96), GUILayout.Width(420)))
            SaveCalibration(false);
        GUILayout.EndHorizontal();

        if (GUILayout.Button("SAVE ALL POSITIONS", m_Button, GUILayout.Height(96), GUILayout.Width(680)))
            SaveCalibration(true);

        GUILayout.Space(6);
        GUILayout.Label(m_Calibrator.Note, m_Text);
        if (!string.IsNullOrEmpty(m_CalibrationNote))
            GUILayout.Label(m_CalibrationNote, m_Text);

        GUILayout.Space(8);
        if (GUILayout.Button(m_ShowCoordinateDump ? "HIDE COORDINATES" : "SHOW COORDINATES",
                m_Button, GUILayout.Height(84), GUILayout.Width(520)))
            m_ShowCoordinateDump = !m_ShowCoordinateDump;
        if (m_ShowCoordinateDump)
            GUILayout.Label(m_Calibrator.DebugReadout(m_Origin, m_ToSession), m_Text);
    }

    GUILayout.Space(14);
}
```

```
if (m_OriginSet && m_NextCheckpoint < surveyed.Length)
{
    if (GUILayout.Button($"Log checkpoint {m_NextCheckpoint + 1}",
            m_Button, GUILayout.Height(110), GUILayout.Width(520)))
    {
        LogCheckpoint();
    }
}

// Recording: walk to a station, stand where the visitor should
// stand, tap. One button that names the station it is waiting for,
// so there is nothing to read and nothing to type while holding a
// phone at arm's length in a church.
if (m_OriginSet)
{
    GUILayout.Space(14);

    // One tap to settle a question that cannot be settled by
    // reading code: does the survey's sideways axis run the same way
    // as the marker's? Flipping re-binds, so the cards move while
    // you watch and the right answer is whichever puts them on the
    // real objects.
    if (GUILayout.Button($"Sideways axis: {(m_SidewaysSign > 0 ? "normal" : "FLIPPED")}  -  tap to swap",
            m_Button, GUILayout.Height(96), GUILayout.Width(860)))
    {
        m_SidewaysSign = -m_SidewaysSign;
        RebindStations();
        m_RecordNote = m_SidewaysSign > 0
            ? "Sideways axis normal."
            : "Sideways axis flipped. Which puts the cards on the real objects?";
    }

    GUILayout.Space(14);

    // The record button keeps its box whether or not there is a
    // station left to record. It used to be replaced by a label when
    // the last one was taken, which let the row below - carrying
    // "Start over" - jump up into the space the button had just
    // occupied. Tap once more out of habit and every position you
    // had just walked the building to collect was gone. A
    // destructive control must never land where a safe one was a
    // moment ago.
    var done = m_RecordIndex >= Stations.Length;
    if (done)
    {
        GUILayout.Label($"All {Stations.Length} recorded. Save them.", m_Text,
            GUILayout.Height(110), GUILayout.Width(860));
    }
    else
    {
        var next = Stations[m_RecordIndex];
        if (GUILayout.Button($"Stand here = {next.stationName}  ({m_RecordIndex + 1}/{Stations.Length})",
                m_Button, GUILayout.Height(110), GUILayout.Width(860)))
        {
            RecordStationHere();
        }
    }
```

```
            if (m_Recording != null && m_Recording.stations.Count > 0)
            {
                GUILayout.Space(10);
                GUILayout.BeginHorizontal();

                if (GUILayout.Button("Save", m_Button, GUILayout.Height(96), GUILayout.Width(280)))
                {
                    SaveRecording();
                    m_ConfirmClear = false;
                }

                // Two taps, and the first one says what the second will
                // destroy. Walking a church to collect these is an hour of
                // someone's day; one stray tap should not be able to spend
                // it.
                var clearLabel = m_ConfirmClear
                    ? $"Tap again to erase {m_Recording.stations.Count}"
                    : "Start over";
                if (GUILayout.Button(clearLabel, m_Button, GUILayout.Height(96), GUILayout.Width(520)))
                {
                    if (m_ConfirmClear)
                    {
                        m_Recording = null;
                        m_Stations = null;
                        m_RecordIndex = 0;
                        m_ConfirmClear = false;
                        m_RecordNote = "Cleared. The saved file is untouched until you Save again.";
                    }
                    else
                    {
                        m_ConfirmClear = true;
                        m_RecordNote = "Start over will erase this session's positions.";
                    }
                }

                GUILayout.EndHorizontal();
            }

            if (!string.IsNullOrEmpty(m_RecordNote))
            {
                GUILayout.Space(8);
                GUILayout.Label(m_RecordNote, m_Text);
            }
        }

        GUILayout.Space(18);
        GUILayout.Label(m_Log.ToString(), m_Text);
        GUILayout.EndArea();
    }
}
```

4.1.6 AR-006 — DeviceSpeech.cs

Code ID	AR-006
File Name	DeviceSpeech.cs
File Path	SanctiWalkAR/Assets/Scripts/DeviceSpeech.cs
Language / Type	C#
Lines	194
Functions / Classes	DeviceSpeech (public class); CanSpeak (public method); Init (public method); Speak (public method); Stop (public method); IsSpeaking (public method); Shutdown (public method); onInit (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/DeviceSpeech.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — DeviceSpeech.cs

```
using System;
using UnityEngine;

/// <summary>
/// Narration, spoken by the phone itself.
///
/// ## Why the device and not an audio file
///
/// The English narration could be generated ahead of time with Piper and
/// shipped as clips. The Tagalog could not: Piper has no Filipino voice, and
/// an English or Spanish voice reading Tagalog mispronounces most of it.
///
/// Android has shipped a Filipino voice for years through Google
/// Text-to-Speech, it is free, it is already on the phone, and once its
/// voice data is downloaded it speaks offline - which matters inside a
/// church with no signal. So the walk speaks its own text in whichever
/// language the visitor has chosen, and no audio is shipped at all.
///
/// ## What this does NOT do
///
/// It does not translate. The Tagalog and English text are both authored,
/// in sites/mhcp.i18n.json; this only reads aloud whichever one is showing.
///
/// Everything here is a no-op off-device, so the editor and the Windows
/// test build behave as if the phone simply had no voice installed.
/// </summary>
public static class DeviceSpeech
{
    // android.speech.tts.TextToSpeech constants.
    const int k_Success = 0;              // TextToSpeech.SUCCESS
```

```
    const int k_QueueFlush = 0;           // TextToSpeech.QUEUE_FLUSH
    const int k_LangMissingData = -1;     // LANG_MISSING_DATA
    const int k_LangNotSupported = -2;    // LANG_NOT_SUPPORTED

    static AndroidJavaObject s_Tts;
    static bool s_Ready;
    static bool s_Tried;
    static string s_PendingText;
    static string s_PendingLang;

    /// <summary>True once the engine has reported itself initialised.</summary>
    public static bool Ready => s_Ready;

    /// <summary>
    /// Whether the phone can actually speak this language.
    ///
    /// Worth asking before drawing a speaker button: a button that does
    /// nothing when tapped is worse than no button, and on a phone with no
    /// Filipino voice data that is exactly what it would be.
    /// </summary>
    public static bool CanSpeak(string lang)
    {
#if UNITY_ANDROID && !UNITY_EDITOR
        if (!s_Ready) return false;
        try
        {
            using (var locale = LocaleFor(lang))
            {
                var r = s_Tts.Call<int>("isLanguageAvailable", locale);
                return r != k_LangMissingData && r != k_LangNotSupported;
            }
        }
        catch (Exception e) { Debug.LogWarning("[DeviceSpeech] " + e.Message); return false; }
#else
        return false;
#endif
    }

    /// <summary>
    /// Starts the engine. Safe to call more than once; only the first call
    /// does anything. Initialisation is asynchronous, so a Speak that
    /// arrives first is held and replayed when the engine reports ready.
    /// </summary>
    public static void Init()
    {
        if (s_Tried) return;
        s_Tried = true;
#if UNITY_ANDROID && !UNITY_EDITOR
        try
        {
            using (var player = new AndroidJavaClass("com.unity3d.player.UnityPlayer"))
            using (var activity = player.GetStatic<AndroidJavaObject>("currentActivity"))
            {
                s_Tts = new AndroidJavaObject("android.speech.tts.TextToSpeech",
                                              activity, new InitListener());
            }
        }
        catch (Exception e)
        {
```

```
            Debug.LogWarning("[DeviceSpeech] could not start the engine: " + e.Message);
        }
#endif
    }

    /// <summary>
    /// Speaks text, interrupting whatever was being said.
    ///
    /// `lang` is "tl" or "en". Interrupting rather than queueing is
    /// deliberate: walking to the next station should start that station's
    /// narration, not wait out the previous one.
    /// </summary>
    public static void Speak(string text, string lang)
    {
        if (string.IsNullOrEmpty(text)) return;
#if UNITY_ANDROID && !UNITY_EDITOR
        Init();
        if (!s_Ready) { s_PendingText = text; s_PendingLang = lang; return; }
        try
        {
            using (var locale = LocaleFor(lang)) s_Tts.Call<int>("setLanguage", locale);
            s_Tts.Call<int>("speak", text, k_QueueFlush, null, "sanctiwalk");
        }
        catch (Exception e) { Debug.LogWarning("[DeviceSpeech] speak failed: " + e.Message); }
#endif
    }

    public static void Stop()
    {
#if UNITY_ANDROID && !UNITY_EDITOR
        s_PendingText = null;
        if (s_Ready) { try { s_Tts.Call<int>("stop"); } catch { } }
#endif
    }

    /// <summary>
    /// True when something is waiting for the engine to finish starting.
    ///
    /// Android's TextToSpeech is ready some time after Init, so the first
    /// line of a run is usually queued rather than spoken, and
    /// IsSpeaking() is false for it. Anything deciding whether it may
    /// talk has to ask this as well, or it will conclude that nothing is
    /// happening and call Stop - which throws the queued line away. That
    /// is exactly what silenced the welcome.
    /// </summary>
    public static bool Pending
    {
        get
        {
#if UNITY_ANDROID && !UNITY_EDITOR
            return !string.IsNullOrEmpty(s_PendingText);
#else
            return false;
#endif
        }
    }

    public static bool IsSpeaking()
    {
```

```
#if UNITY_ANDROID && !UNITY_EDITOR
        if (!s_Ready) return false;
        try { return s_Tts.Call<bool>("isSpeaking"); } catch { return false; }
#else
        return false;
#endif
    }

    public static void Shutdown()
    {
#if UNITY_ANDROID && !UNITY_EDITOR
        if (s_Tts != null) { try { s_Tts.Call("shutdown"); } catch { } s_Tts = null; }
        s_Ready = false; s_Tried = false;
#endif
    }

#if UNITY_ANDROID && !UNITY_EDITOR
    /// <summary>
    /// Tagalog is "fil" to Android, not "tl" - the ISO code changed and the
    /// platform followed the new one. Asking for "tl" finds nothing on most
    /// phones even when the Filipino voice is installed.
    /// </summary>
    static AndroidJavaObject LocaleFor(string lang) =>
        lang == "tl"
            ? new AndroidJavaObject("java.util.Locale", "fil", "PH")
            : new AndroidJavaObject("java.util.Locale", "en", "US");

    class InitListener : AndroidJavaProxy
    {
        public InitListener() : base("android.speech.tts.TextToSpeech$OnInitListener") { }

        // Called by Android on its own thread once the engine is up.
        public void onInit(int status)
        {
            s_Ready = status == k_Success;
            Debug.Log("[DeviceSpeech] engine " + (s_Ready ? "ready" : "failed, status " + status));
            if (s_Ready && !string.IsNullOrEmpty(s_PendingText))
            {
                var t = s_PendingText; var l = s_PendingLang;
                s_PendingText = null;
                Speak(t, l);
            }
        }
    }
#endif
}
```

4.1.7 AR-007 — FirstPersonWalkthrough.cs

Code ID	AR-007
File Name	FirstPersonWalkthrough.cs
File Path	SanctiWalkAR/Assets/Scripts/FirstPersonWalkthrough.cs
Language / Type	C#
Lines	166
Functions / Classes	FirstPersonWalkthrough (public class); Awake (method); Update (method); HandleCursor (method); Capture (method); Look (method); Move (method); OnGUI (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/FirstPersonWalkthrough.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — FirstPersonWalkthrough.cs

```
using UnityEngine;

/// <summary>
/// Walk the scanned room on a desktop, for authoring and review.
///
/// This exists because pressing Play on the authoring scene rendered
/// nothing: the scene was built for the Scene view and had no camera, no
/// audio listener and no collision at all, so there was nothing to look
/// through and nothing to stand on.
///
/// Deliberately plain. A CharacterController, a camera, and the legacy
/// Input class - which works here because Active Input Handling is set to
/// "Both". No FPS framework, no new packages, no input action assets.
///
/// It lives only in StationAuthoring.unity, and the APK builds
/// DriftTest.unity alone, so none of this can reach a device. On the
/// phone the camera is the AR camera and the room is the real room.
///
///   WASD / arrows  walk
///   mouse          look
///   Shift          walk faster
///   Space          step up onto something
///   Escape         release the cursor
///   click          capture it again
/// </summary>
[RequireComponent(typeof(CharacterController))]
public class FirstPersonWalkthrough : MonoBehaviour
{
    [Header("Movement")]
    public float walkSpeed = 1.6f;      // a slow indoor walk, m/s
    public float sprintSpeed = 4.0f;
```

```
public float jumpSpeed = 2.4f;
public float gravity = 9.81f;

[Header("Look")]
public float mouseSensitivity = 2.2f;
public float pitchLimit = 85f;

[Header("Eye")]
[Tooltip("Metres from the controller's feet to the camera.")]
public float eyeHeight = 1.6f;

CharacterController m_Controller;
Camera m_Camera;
float m_Yaw, m_Pitch;
float m_VerticalSpeed;
bool m_Looking = true;

void Awake()
{
    m_Controller = GetComponent<CharacterController>();
    m_Camera = GetComponentInChildren<Camera>();

    if (m_Camera == null)
    {
        // Built here rather than assumed, so dropping this component on
        // a bare GameObject is enough to get a working player.
        var go = new GameObject("Eye", typeof(Camera), typeof(AudioListener));
        go.transform.SetParent(transform, false);
        m_Camera = go.GetComponent<Camera>();
        m_Camera.nearClipPlane = 0.05f;
        m_Camera.farClipPlane = 120f;
        m_Camera.fieldOfView = 60f;
    }

    // The controller's origin is its centre, so the eye sits above it by
    // the difference between eye height and half the capsule.
    m_Camera.transform.localPosition = new Vector3(0f, eyeHeight - m_Controller.height * 0.5f, 0f);

    m_Yaw = transform.eulerAngles.y;
    m_Pitch = 0f;
    Capture(true);
}

void Update()
{
    HandleCursor();
    if (m_Looking) Look();
    Move();
}

void HandleCursor()
{
    if (Input.GetKeyDown(KeyCode.Escape)) Capture(false);
    else if (!m_Looking && Input.GetMouseButtonDown(0)) Capture(true);
}

void Capture(bool on)
{
    m_Looking = on;
```

```
    Cursor.lockState = on ? CursorLockMode.Locked : CursorLockMode.None;
    Cursor.visible = !on;
}

void Look()
{
    m_Yaw += Input.GetAxisRaw("Mouse X") * mouseSensitivity;
    m_Pitch -= Input.GetAxisRaw("Mouse Y") * mouseSensitivity;
    m_Pitch = Mathf.Clamp(m_Pitch, -pitchLimit, pitchLimit);

    // Yaw on the body, pitch on the camera only. Pitching the body
    // would tilt the capsule and make it climb walls.
    transform.rotation = Quaternion.Euler(0f, m_Yaw, 0f);
    m_Camera.transform.localRotation = Quaternion.Euler(m_Pitch, 0f, 0f);
}

void Move()
{
    var input = new Vector3(Input.GetAxisRaw("Horizontal"), 0f, Input.GetAxisRaw("Vertical"));
    if (input.sqrMagnitude > 1f) input.Normalize();

    var speed = (Input.GetKey(KeyCode.LeftShift) || Input.GetKey(KeyCode.RightShift))
        ? sprintSpeed
        : walkSpeed;

    var horizontal = transform.TransformDirection(input) * speed;

    if (m_Controller.isGrounded)
    {
        // A small downward bias keeps isGrounded true on the uneven
        // floor of a scan, which is never perfectly flat.
        m_VerticalSpeed = -1f;
        if (Input.GetKeyDown(KeyCode.Space)) m_VerticalSpeed = jumpSpeed;
    }
    else
    {
        m_VerticalSpeed -= gravity * Time.deltaTime;
    }

    var motion = horizontal;
    motion.y = m_VerticalSpeed;
    m_Controller.Move(motion * Time.deltaTime);
}

void OnGUI()
{
    var style = new GUIStyle(GUI.skin.label)
    {
        fontSize = 13,
        normal = { textColor = Color.white },
    };

    var feet = transform.position.y - m_Controller.height * 0.5f;
    GUI.Label(new Rect(14, 10, 640, 22),
        "WASD walk   ·   mouse look   ·   Shift faster   ·   Space step up   ·   Esc release cursor",
        style);
    GUI.Label(new Rect(14, 30, 640, 22),
        $"map space   X {transform.position.x:F2}   Y {transform.position.y:F2}   Z {transform.position.z:F2}" +
        $"      {feet - StationMarker.FloorY:+0.00;-0.00} m above floor",
```

```
            style);

        // Which station you are standing in, and how far to the others.
        var markers = FindObjectsByType<StationMarker>(FindObjectsInactive.Exclude);
        var y = 54f;
        foreach (var marker in markers)
        {
            var d = Vector3.Distance(transform.position, marker.transform.position);
            var inside = d <= marker.triggerRadius;
            style.normal.textColor = inside ? new Color(0.55f, 1f, 0.6f) : new Color(0.75f, 0.78f, 0.84f);
            GUI.Label(new Rect(14, y, 640, 20),
                $"{(inside ? "IN  " : "    ")}station {marker.number}  {marker.stationName}   {d:F2} m" +
                $"   (radius {marker.triggerRadius:F1} m)", style);
            y += 19f;
        }
    }
}
```

4.1.8 AR-008 — MarkerScan.cs

Code ID	AR-008
File Name	MarkerScan.cs
File Path	SanctiWalkAR/Assets/Scripts/MarkerScan.cs
Language / Type	C#
Lines	83
Functions / Classes	MarkerScan (public struct); ToString (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/MarkerScan.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MarkerScan.cs

```
using System;
using UnityEngine;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Everything a successful trigger-image scan produces, and nothing else.
///
/// The separation this type exists to enforce:
///
///   FROM THE SCAN          the name of the reference image that matched,
///                          and the pose ARCore computed for it in session
///                          space. That is all. A scan knows *which* image
///                          it saw and *where that image is*. It knows
///                          nothing about parishes, stations or history.
///
///   FROM THE LIBRARY       the physical size of the print. This is carried
///                          here because the pose is meaningless without it
///                          - ARCore scales the pose by the declared size -
///                          but it is configuration, authored in
///                          ChurchMarkers.asset, not something measured
///                          from the camera frame.
///
///   FROM POI CONFIG        every station: its id, name, subtitle, surveyed
///                          offset, trigger radius, artwork, descriptions,
///                          metadata and next stop. See MuseumStation.
///                          None of it comes from the scan.
///
///   DERIVED AT RUNTIME     each station's world position, which is the
///                          marker pose composed with the surveyed offset,
///                          and the into-room sign, which is resolved from
///                          where the camera stood at detection.
///
/// Mixing these is how AR projects end up unable to answer "why is the
```

```
/// altar in the wrong place" - the answer is always in exactly one of the
/// four, and keeping them apart is what makes that findable.
/// </summary>
[Serializable]
public struct MarkerScan
{
    // ---- from the scan ------------------------------------------------
    /// <summary>Reference image name that matched, e.g. "entrance".</summary>
    public string trackedImageName;

    /// <summary>The library entry's guid, as ARCore reports it.</summary>
    public string referenceGuid;

    /// <summary>
    /// The marker's pose in SESSION space - ARCore's own frame, whose origin
    /// is wherever the app happened to start. Not map space, not world
    /// coordinates, and not comparable between runs.
    /// </summary>
    public Pose pose;

    /// <summary>Tracking, Limited or None at the moment of the report.</summary>
    public TrackingState trackingState;

    // ---- from the reference image library -----------------------------
    /// <summary>Printed size in metres. Configuration, not measurement.</summary>
    public Vector2 physicalSize;

    // ---- provenance ---------------------------------------------------
    /// <summary>True when this came from ScanSimulator rather than a camera.</summary>
    public bool simulated;

    /// <summary>Where the camera was when the image was reported.</summary>
    public Pose cameraPoseAtDetection;

    public override string ToString()
    {
        var e = pose.rotation.eulerAngles;
        return
            $"MarkerScan {{\n" +
            $"  source            : {(simulated ? "SIMULATED" : "ARCore camera")}\n" +
            $"  trackedImageName  : \"{trackedImageName}\"       <- from scan\n" +
            $"  referenceGuid     : {referenceGuid}              <- from scan\n" +
            $"  trackingState     : {trackingState}              <- from scan\n" +
            $"  pose.position     : ({pose.position.x:F3}, {pose.position.y:F3}, {pose.position.z:F3})  <- from scan, SESSION space, metres\n" +
            $"  pose.rotation     : euler ({e.x:F1}, {e.y:F1}, {e.z:F1})   <- from scan\n" +
            $"  physicalSize      : {physicalSize.x:F3} x {physicalSize.y:F3} m   <- from ChurchMarkers.asset, NOT the scan\n" +
            $"  cameraAtDetection : ({cameraPoseAtDetection.position.x:F3}, {cameraPoseAtDetection.position.y:F3}, {cameraPoseAtDetection.position.z:F3})\n" +
            $"}}";
    }
}
```

4.1.9 AR-009 — MultiSetVps.cs

Code ID	AR-009
File Name	MultiSetVps.cs
File Path	SanctiWalkAR/Assets/Scripts/MultiSetVps.cs
Language / Type	C#
Lines	501
Functions / Classes	MultiSetVps (public class); Bind (public method); OnFrameReceived (method); OnDestroy (method); LoadCredentials (method); Localize (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/MultiSetVps.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MultiSetVps.cs

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.Networking;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Asks MultiSet where the phone is inside the scanned map, and turns the
/// answer into the AR session's origin for that map.
///
/// ## Why this is the right way round
///
/// Every earlier attempt reconstructed the map's alignment by hand: hang a
/// printed sheet where the scan says it is, or tap two points on the floor,
/// and derive a frame from that. Each needed something to be measured or
/// assumed, and each of those was wrong at least once - the sheet's
/// unpointable centre, a wall normal taken from one mesh triangle, a
/// handedness convention nobody had checked.
///
/// MultiSet's own map query does the alignment directly. It takes a camera
/// frame and returns the device's 6-DoF pose IN MAP SPACE - the same
/// coordinates the points were picked in. Nothing is measured, nothing is
/// transcribed, and the handedness is a documented request parameter rather
/// than a guess.
///
/// ## The arithmetic, which is the only real step
///
/// The query gives T_map_device: where the phone is, in the map. ARCore
/// already knows T_session_device: where the phone is, in this session. The
```

```
/// map's origin in session space is then
///
///     T_session_map = T_session_device * inverse(T_map_device)
///
/// and that is exactly the Pose the rest of this project already consumes,
/// so VPS slots in beside the marker and the floor picker rather than
/// replacing anything.
///
/// Both poses must be sampled at the same instant, so the camera transform
/// is read in the same frame the image is captured. Reading it after the
/// round trip would be reading it about two seconds and several steps
/// later, and the error would look exactly like bad localization.
/// </summary>
public class MultiSetVps : MonoBehaviour
{
    const string k_Api = "https://api.multiset.ai/v1";

    /// <summary>
    /// Whether the pose should come back right-handed. Unity is
    /// left-handed, and MultiSet's own Unity example passes false, so false
    /// is what a Unity client wants. This is the question that was
    /// previously guessed at with a cross product and a coin flip; it is a
    /// documented parameter, so it is set rather than inferred.
    /// </summary>
    const string k_IsRightHanded = "false";

    /// <summary>
    /// The query image is capped at 1280 px on its long side by the API.
    /// Sending more is refused; sending much less loses the detail the
    /// match depends on.
    /// </summary>
    const int k_MaxEdge = 1280;

    [Serializable] class TokenReply { public string token; public string accessToken; }
    [Serializable] class Vec3Json { public float x, y, z; }
    [Serializable] class QuatJson { public float x, y, z, w; }

    [Serializable]
    class QueryReply
    {
        public bool poseFound;
        public Vec3Json position;
        public QuatJson rotation;
        public float confidence;
        public string[] mapCodes;
        public int responseTime;
        public string error;
    }

    // ---- what the overlay reads ----------------------------------
    public bool Busy { get; private set; }
    public bool PoseFound { get; private set; }
    public float Confidence { get; private set; }
    public Vector3 MapPosition { get; private set; }
    public Quaternion MapRotation { get; private set; }
    public Pose SessionFromMap { get; private set; }
    public string Status { get; private set; } = "Not tried yet";
    public int LastResponseMs { get; private set; }
    public int Attempts { get; private set; }
```

```
// ------------------------------------------------------------ diagnostics
//
// Everything the last query actually saw and sent, kept so the error can
// be attributed rather than guessed at. None of it is used to compute
// anything; it exists to be read.

/// <summary>The camera pose the returned map pose is paired with.</summary>
public Vector3 QueryCameraPosition { get; private set; }
public Quaternion QueryCameraRotation { get; private set; }

/// <summary>Timestamp of the CPU image that was uploaded, in seconds.</summary>
public double ImageTimestamp { get; private set; }

/// <summary>
/// Timestamp of the AR frame whose camera transform was paired with it.
///
/// These two must be the same frame. TryAcquireLatestCpuImage hands back
/// the most recent image ARCore has finished with, and the camera
/// transform is whatever Unity last applied - which is not automatically
/// the same instant. If they differ by a frame and the phone is turning
/// at even a slow walking pace, the pose being solved for is not the
/// pose the picture was taken from, and every metre of the resulting
/// origin error is real and unexplainable from the numbers alone.
/// </summary>
public double PoseTimestamp { get; private set; }

/// <summary>Image seconds minus pose seconds. Zero is what is wanted.</summary>
public double FrameSkewSeconds => ImageTimestamp - PoseTimestamp;

/// <summary>What was actually posted as intrinsics, after scaling.</summary>
public Vector4 SentIntrinsics { get; private set; }
public Vector2Int SentSize { get; private set; }
public Vector2 SensorSize { get; private set; }

/// <summary>The reply, verbatim, trimmed to something printable.</summary>
public string LastReplyBody { get; private set; } = "";

/// <summary>How far the phone moved between the query and the answer.</summary>
public float MovedDuringQuery { get; private set; }

/// <summary>Raised with the map's origin in session space.</summary>
public event Action<Pose, float> Localized;

ARCameraManager m_CameraManager;
Camera m_Camera;
string m_MapCode;
string m_Token;
string m_ClientId, m_ClientSecret;

public bool HasCredentials => !string.IsNullOrEmpty(m_ClientId) && !string.IsNullOrEmpty(m_ClientSecret);

/// <summary>The timestamp of the most recent AR frame Unity has applied.</summary>
double m_LatestFrameSeconds;

public void Bind(ARCameraManager cameraManager, Camera cam, string mapCode)
{
    if (m_CameraManager != null)
        m_CameraManager.frameReceived -= OnFrameReceived;
```

```
    m_CameraManager = cameraManager;
    m_Camera = cam;
    m_MapCode = mapCode;

    // Subscribed purely so the diagnostic can say whether the image and
    // the camera transform came from the same frame.
    if (m_CameraManager != null)
        m_CameraManager.frameReceived += OnFrameReceived;

    LoadCredentials();
}

void OnFrameReceived(ARCameraFrameEventArgs args)
{
    if (args.timestampNs.HasValue)
        m_LatestFrameSeconds = args.timestampNs.Value / 1e9;
}

void OnDestroy()
{
    if (m_CameraManager != null)
        m_CameraManager.frameReceived -= OnFrameReceived;
}

/// <summary>
/// Credentials come from a Resources text asset that the editor writes
/// from Tools/multiset.env at build time, so the secret is not typed
/// into a source file and not committed. It IS inside the APK, which is
/// acceptable for a test build on the owner's own account and would not
/// be for a released one - that wants a server holding the secret and
/// the app calling the server.
/// </summary>
void LoadCredentials()
{
    var asset = Resources.Load<TextAsset>("multiset-credentials");
    if (asset == null)
    {
        Status = "No credentials in the build. Run SanctiWalk > Stage MultiSet credentials, then rebuild.";
        Debug.LogWarning("[MultiSetVps] " + Status);
        return;
    }

    foreach (var raw in asset.text.Split('\n'))
    {
        var line = raw.Trim();
        if (line.Length == 0 || line.StartsWith("#") || !line.Contains("=")) continue;
        var at = line.IndexOf('=');
        var key = line.Substring(0, at).Trim();
        var value = line.Substring(at + 1).Trim().Trim('"').Trim('\'');
        if (key == "MULTISET_CLIENT_ID") m_ClientId = value;
        else if (key == "MULTISET_CLIENT_SECRET") m_ClientSecret = value;
    }

    Status = HasCredentials ? "Ready" : "Credentials file present but incomplete";
}

public void Localize()
{
```

```
    if (Busy) return;
    StartCoroutine(Run());
}

IEnumerator Run()
{
    Busy = true;
    Attempts++;
    PoseFound = false;

    if (!HasCredentials)
    {
        Status = "No credentials.";
        Busy = false;
        yield break;
    }

    if (string.IsNullOrEmpty(m_Token))
    {
        Status = "Signing in…";
        yield return GetToken();
        if (string.IsNullOrEmpty(m_Token))
        {
            Busy = false;
            yield break;
        }
    }

    // Wait for the end of the frame so the camera image for THIS frame
    // is available, then take the image and the pose together.
    yield return new WaitForEndOfFrame();

    if (!m_CameraManager.TryAcquireLatestCpuImage(out var cpu))
    {
        Status = "No camera image yet. Give ARCore a moment.";
        Busy = false;
        yield break;
    }

    byte[] jpg;
    int w, h;
    var imageSeconds = cpu.timestamp;
    try
    {
        jpg = Encode(cpu, out w, out h);
    }
    finally
    {
        cpu.Dispose();
    }

    if (jpg == null)
    {
        Status = "Could not encode the camera image.";
        Busy = false;
        yield break;
    }

    if (!m_CameraManager.TryGetIntrinsics(out var intr))
```

```
{
    Status = "No camera intrinsics yet.";
    Busy = false;
    yield break;
}

// Sampled here, in the same frame as the image above. This pairing
// is the whole basis of the alignment.
var sessionPos = m_Camera.transform.position;
var sessionRot = m_Camera.transform.rotation;

QueryCameraPosition = sessionPos;
QueryCameraRotation = sessionRot;
ImageTimestamp = imageSeconds;
PoseTimestamp = m_LatestFrameSeconds;

// Intrinsics are for the full sensor image; the upload is scaled, so
// the focal length and principal point scale with it or the match is
// solved for the wrong camera.
var scale = (float)w / intr.resolution.x;

SentIntrinsics = new Vector4(
    intr.focalLength.x * scale, intr.focalLength.y * scale,
    intr.principalPoint.x * scale, intr.principalPoint.y * scale);
SentSize = new Vector2Int(w, h);
SensorSize = new Vector2(intr.resolution.x, intr.resolution.y);

var form = new List<IMultipartFormSection>
{
    new MultipartFormDataSection("mapCode", m_MapCode),
    new MultipartFormDataSection("isRightHanded", k_IsRightHanded),
    new MultipartFormDataSection("width", w.ToString()),
    new MultipartFormDataSection("height", h.ToString()),
    new MultipartFormDataSection("fx", (intr.focalLength.x * scale).ToString("R")),
    new MultipartFormDataSection("fy", (intr.focalLength.y * scale).ToString("R")),
    new MultipartFormDataSection("px", (intr.principalPoint.x * scale).ToString("R")),
    new MultipartFormDataSection("py", (intr.principalPoint.y * scale).ToString("R")),
    new MultipartFormFileSection("queryImage", jpg, "query.jpg", "image/jpeg"),
};

// Keep the exact bytes that went to the server. When a query comes
// back "no pose found" the first question is whether the image was
// any good - upside down, mirrored, black, or pointed at a wall -
// and that is answerable in one adb pull instead of by guessing.
try
{
    System.IO.File.WriteAllBytes(
        System.IO.Path.Combine(Application.persistentDataPath, "last-vps-query.jpg"), jpg);
}
catch (Exception e)
{
    Debug.LogWarning($"[MultiSetVps] could not save the query image: {e.Message}");
}

Status = $"Querying the map… ({w}x{h}, {jpg.Length / 1024} KB)";
using var req = UnityWebRequest.Post($"{k_Api}/vps/map/query-form", form);
req.SetRequestHeader("Authorization", "Bearer " + m_Token);
req.timeout = 30;
yield return req.SendWebRequest();
```

```
if (req.result != UnityWebRequest.Result.Success)
{
    // A stale token shows up as 401; get a new one and let the next
    // press try again rather than retrying silently here.
    if (req.responseCode == 401)
    {
        m_Token = null;
        Status = "Session expired. Tap again.";
    }
    else
    {
        Status = $"HTTP {req.responseCode}: {Trim(req.downloadHandler.text)}";
    }
    Debug.LogWarning($"[MultiSetVps] query failed {req.responseCode} {req.downloadHandler.text}");
    Busy = false;
    yield break;
}

QueryReply reply;
try
{
    reply = JsonUtility.FromJson<QueryReply>(req.downloadHandler.text);
}
catch (Exception e)
{
    Status = "Could not read the reply: " + e.Message;
    Busy = false;
    yield break;
}

LastResponseMs = reply.responseTime;
LastReplyBody = Trim(req.downloadHandler.text);

// Not used in the maths - the pose below is paired with the pose
// sampled at query time, which is correct. This only says how much
// the phone moved while waiting, because an origin computed from a
// two-second-old pose is only as good as ARCore's tracking over
// those two seconds.
MovedDuringQuery = Vector3.Distance(m_Camera.transform.position, sessionPos);

if (reply == null || !reply.poseFound || reply.position == null || reply.rotation == null)
{
    Status = string.IsNullOrEmpty(reply?.error)
        ? "No pose found here. Point at something the scan covers and try again."
        : reply.error;
    Busy = false;
    yield break;
}

MapPosition = new Vector3(reply.position.x, reply.position.y, reply.position.z);
MapRotation = new Quaternion(reply.rotation.x, reply.rotation.y, reply.rotation.z, reply.rotation.w);
Confidence = reply.confidence;
PoseFound = true;

// T_session_map = T_session_device * inverse(T_map_device)
var originRot = sessionRot * Quaternion.Inverse(MapRotation);
var originPos = sessionPos - originRot * MapPosition;
SessionFromMap = new Pose(originPos, originRot);
```

```
    Status = $"VPS FOUND  conf {Confidence:F2}  {LastResponseMs} ms";
    Debug.Log($"[MultiSetVps] map ({MapPosition.x:F3},{MapPosition.y:F3},{MapPosition.z:F3}) " +
              $"conf {Confidence:F3} -> origin {originPos} {originRot.eulerAngles}");

    Localized?.Invoke(SessionFromMap, Confidence);
    Busy = false;
}

IEnumerator GetToken()
{
    var basic = Convert.ToBase64String(
        System.Text.Encoding.UTF8.GetBytes($"{m_ClientId}:{m_ClientSecret}"));

    // A POST with no body: UnityWebRequest.Post with an empty form would
    // add one, so the handler is set explicitly.
    using var req = new UnityWebRequest($"{k_Api}/m2m/token", "POST")
    {
        downloadHandler = new DownloadHandlerBuffer(),
        timeout = 20,
    };
    req.SetRequestHeader("Authorization", "Basic " + basic);
    yield return req.SendWebRequest();

    if (req.result != UnityWebRequest.Result.Success)
    {
        Status = $"Sign-in failed: HTTP {req.responseCode}";
        Debug.LogWarning($"[MultiSetVps] token {req.responseCode} {req.downloadHandler.text}");
        yield break;
    }

    var reply = JsonUtility.FromJson<TokenReply>(req.downloadHandler.text);
    m_Token = !string.IsNullOrEmpty(reply?.token) ? reply.token : reply?.accessToken;
    if (string.IsNullOrEmpty(m_Token))
        Status = "Signed in but no token in the reply.";
}

/// <summary>
/// The camera frame as JPEG, scaled to the API's limit.
///
/// Converted to RGBA32 first because EncodeToJPG needs a Texture2D in a
/// format it understands, and ARCore hands over YUV.
/// </summary>
byte[] Encode(XRCpuImage cpu, out int width, out int height)
{
    width = cpu.width;
    height = cpu.height;

    var longest = Mathf.Max(width, height);
    if (longest > k_MaxEdge)
    {
        var k = (float)k_MaxEdge / longest;
        width = Mathf.RoundToInt(width * k);
        height = Mathf.RoundToInt(height * k);
    }

    var parms = new XRCpuImage.ConversionParams
    {
        inputRect = new RectInt(0, 0, cpu.width, cpu.height),
```

```
            outputDimensions = new Vector2Int(width, height),
            outputFormat = TextureFormat.RGBA32,
            // No transformation, deliberately.
            //
            // MirrorY was here because a CPU image arrives upside down
            // relative to a Unity texture - which is true, and irrelevant,
            // because this image is not going into a texture, it is going
            // to a feature matcher along with the intrinsics that describe
            // it. fx, fy, px, py are for the sensor image as the sensor
            // produced it. Flip the pixels and leave the principal point
            // where it was and the two no longer describe the same camera,
            // so the match either fails or solves for a pose that is not
            // the phone's. Sending it unaltered is the only version that
            // agrees with the numbers sent beside it.
            transformation = XRCpuImage.Transformation.None,
        };

        Texture2D tex = null;
        try
        {
            // Convert wants a raw pointer, and the obvious way to get one
            // from a NativeArray needs `unsafe`, which this project does not
            // compile with. Pinning a managed array hands over the same
            // address without it - and without turning unsafe code on
            // project-wide to read one camera frame.
            var size = cpu.GetConvertedDataSize(parms);
            var managed = new byte[size];
            var pin = System.Runtime.InteropServices.GCHandle.Alloc(
                managed, System.Runtime.InteropServices.GCHandleType.Pinned);
            try
            {
                cpu.Convert(parms, pin.AddrOfPinnedObject(), managed.Length);
            }
            finally
            {
                pin.Free();
            }

            tex = new Texture2D(width, height, TextureFormat.RGBA32, false);
            tex.LoadRawTextureData(managed);
            tex.Apply();
            return tex.EncodeToJPG(85);
        }
        catch (Exception e)
        {
            Debug.LogWarning($"[MultiSetVps] encode failed: {e.Message}");
            return null;
        }
        finally
        {
            if (tex != null) Destroy(tex);
        }
    }

    static string Trim(string s)
        => string.IsNullOrEmpty(s) ? "" : (s.Length > 120 ? s.Substring(0, 120) : s);
}
```

4.1.10 AR-010 — MuseumStation.cs

Code ID	AR-010
File Name	MuseumStation.cs
File Path	SanctiWalkAR/Assets/Scripts/MuseumStation.cs
Language / Type	C#
Lines	242
Functions / Classes	MuseumStation (public class); Copy (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/MuseumStation.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MuseumStation.cs

```
using System;
using UnityEngine;

/// <summary>
/// One stop on the tour: where it is, what it is, and what to say about it.
///
/// The fields are the ones the brief asked to store per station. Position
/// is kept in the same frame as ChurchOriginTest.surveyed - metres from the
/// entrance marker, X across the wall, Y up, Z into the room - because that
/// is the frame the runtime already works in. The MultiSet map is where
/// these numbers were measured, not where they live.
/// </summary>
/// <summary>
/// What a station puts in the room.
///
/// From the station plan: a history is a CARD you tap to read, and its
/// picture lives on a separate AR Image station rather than on the card.
/// Keeping them apart is the parish's decision, not an implementation
/// detail - "In Histories there will be no pic, only description; the
/// picture will be in AR images."
///
/// Model is the same idea one step further: the AR images are meant to
/// become 3D objects, so the content type is a field rather than something
/// inferred from whether an image file happens to be set.
/// </summary>
public enum MuseumStationKind
{
    History,
    Image,
    Model,
}

[Serializable]
```

```
public class MuseumStation
{
    public string stationId;

    /// <summary>Card, picture, or object. See MuseumStationKind.</summary>
    public MuseumStationKind kind = MuseumStationKind.History;
    public string stationName;

    /// <summary>The small grey line under the title, e.g. "Parish Patroness".</summary>
    public string subtitle;

    /// <summary>Metres from the marker. X across, Y up, Z into the room.</summary>
    public Vector3 position;

    /// <summary>
    /// Which way this station's card faces, in degrees about the frame's up
    /// axis. Zero faces the way the frame faces. Set by the calibrator.
    /// </summary>
    public float yawDegrees;

    /// <summary>How close the pilgrim must be before the bubble appears.</summary>
    public float triggerRadius = 2.5f;

    /// <summary>Texture name under Resources/, or empty for no artwork.</summary>
    public string image;

    /// <summary>
    /// A 3D object to stand at this station, by name under
    /// Assets/Resources/Models. Empty means no object, which stays the
    /// normal case - most stations are a card about something that is
    /// already there.
    ///
    /// Under Resources rather than anywhere tidier because it is loaded by
    /// NAME at runtime, and Resources is the only folder Unity keeps
    /// loadable that way in a build. A model outside it compiles, imports,
    /// previews correctly in the editor and then silently is not there on
    /// the phone.
    /// </summary>
    public string model;

    /// <summary>
    /// Multiplies the model's own size. 1 means the file is already in
    /// metres, which is what Unity and the AR session both assume; anything
    /// authored in centimetres arrives a hundred times too big and this is
    /// the escape hatch rather than re-exporting it.
    /// </summary>
    public float modelScale = 1f;

    /// <summary>
    /// Where the model stands relative to the station, in metres, in the
    /// station's own frame: x across, y up, z into the room.
    ///
    /// A station marks a thing that is already in the room, so an object
    /// placed AT one stands inside whatever it is labelling. This is how a
    /// statue stands next to the Santo Nino rather than in it.
    /// </summary>
    public Vector3 modelOffset = Vector3.zero;

    /// <summary>One or two sentences, shown on the bubble.</summary>
```

```
public string shortDescription;

/// <summary>The full text, shown on the museum card.</summary>
public string detailedDescription;

// ---- the same three, in both languages --------------------------
//
// The parish wrote its history in Tagalog and the statue descriptions
// in English; sites/mhcp.i18n.json now carries both for every station.
// The plain fields above stay as the author wrote them, so nothing
// that already reads them changes; these are what the language switch
// actually picks between. Read them through Name/Short/Detailed rather
// than directly, so a station with no translation still shows text.
public string nameTl, nameEn;
public string shortTl, shortEn;
public string detailedTl, detailedEn;

/// <summary>Station name in the language the visitor chose.</summary>
public string Name => AppLanguage.Pick(nameTl, nameEn, stationName);

/// <summary>Bubble line in the language the visitor chose.</summary>
public string Short => AppLanguage.Pick(shortTl, shortEn, shortDescription);

/// <summary>Card text, and what the phone reads aloud.</summary>
public string Detailed => AppLanguage.Pick(detailedTl, detailedEn, detailedDescription);

/// <summary>
/// What the speaker button should say. The card text if there is any,
/// otherwise the one-line caption, otherwise the name - so the button
/// never does nothing when tapped.
/// </summary>
public string Narration
{
    get
    {
        var d = Detailed;
        if (!string.IsNullOrEmpty(d)) return d;
        var s2 = Short;
        if (!string.IsNullOrEmpty(s2)) return s2;
        return Name;
    }
}

// The three metadata rows on the card. Blank rows are not drawn, which
// is better than printing "Unknown" for a thing nobody has researched
// yet - the parish office can fill these in later.
public string yearInstalled;
public string artist;
public string material;

/// <summary>stationId of the next stop, or empty if this is the last.</summary>
public string nextStationId;

// ---- content still to be supplied -------------------------------
// Left blank on purpose. The parish history is not ours to invent, so
// these stay empty until real text and recordings arrive; the UI
// already omits an empty metadata row rather than printing a
// placeholder, and the audio button simply does not appear.
```

```
    /// <summary>Narration clip name under Resources/, or empty for none.</summary>
    public string audioClip;

    /// <summary>What kind of thing this is: statue, window, altar, artifact.</summary>
    public string poiType;

    /// <summary>Longer background for the museum card, beyond the description.</summary>
    public string historyNotes;

    /// <summary>
    /// A copy, so a caller can change one field without changing the shared
    /// table. This is a class, not a struct: MuseumStations.Sala hands out
    /// references, and writing a recorded position straight onto one would
    /// edit the entry every other caller sees, for the rest of the run.
    /// Every field is a string, a float or a Vector3, so a shallow copy is a
    /// complete one.
    /// </summary>
    public MuseumStation Copy() => (MuseumStation)MemberwiseClone();
}

/// <summary>
/// The stations for a given building.
///
/// Home first, exactly as the plan says: prove the flow in the sala, then
/// the same structure carries the real churches. The sala entries are
/// labelled as test content rather than dressed up as history, because
/// inventing a provenance for a sofa would make the demo a lie.
/// </summary>
public static partial class MuseumStations
{
    public static readonly MuseumStation[] Sala =
    {
        new MuseumStation
        {
            stationId = "sala-sofa",
            stationName = "Pagkakatatag, 1956",
            subtitle = "3 ng 4",
            position = new Vector3(-0.700f, -0.862f, 3.072f),
            triggerRadius = 2.0f,
            image = "station-sofa",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-alak",
        },
        new MuseumStation
        {
            stationId = "sala-santo-nino",
            stationName = "Ang Kapilya, 1951",
            subtitle = "1 ng 4",
            position = new Vector3(0.605f, -0.583f, 1.202f),
            triggerRadius = 2.0f,
            image = "station-santo-nino",
            // Converted from the supplied GLB by Tools/glb-to-obj.py, which
            // also moved it: the file had its base 20.6 m above its own
            // origin and was 14 m and 71 m off in x and z, so placed as it
            // came it would have stood in the sky, two rooms away.
            model = "st-cyril",
            modelScale = 1f,
            // 0.7 m to one side, so it stands NEXT to the Santo Nino rather
```

```
            // than inside it. Life-size already - 0.84 x 2.08 x 0.86 m.
            modelOffset = new Vector3(0.7f, 0f, 0f),
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-tv",
        },
        new MuseumStation
        {
            stationId = "sala-alak",
            stationName = "Ang Parokya, 1956",
            subtitle = "4 ng 4",
            position = new Vector3(-0.041f, -0.357f, 5.472f),
            triggerRadius = 2.0f,
            image = "station-shelf",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "",
        },
        new MuseumStation
        {
            stationId = "sala-tv",
            stationName = "Pd. Albert Klaeser",
            subtitle = "2 ng 4",
            position = new Vector3(-2.761f, -0.587f, 4.187f),
            triggerRadius = 2.0f,
            image = "station-tv",
            shortDescription = "Pindutin upang basahin",
            detailedDescription = "",
            nextStationId = "sala-sofa",
        },
    };
}
```

4.1.11 AR-011 — MuseumStationsGenerated.cs

Code ID	AR-011
File Name	MuseumStationsGenerated.cs
File Path	SanctiWalkAR/Assets/Scripts/MuseumStationsGenerated.cs
Language / Type	C#
Lines	710
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/MuseumStationsGenerated.cs. The file begins with its own explanatory header comment.

Purpose and Explanation

Quoted from the file's own header comment:

> Generated file - do not edit.
>
> Written by Tools/deploy-stations.py from the plans saved out of the station placer (Tools/viewer/out/*.json). Edit the positions in the placer and re-run that script; anything typed here is lost on the next run.

Complete Source Code — MuseumStationsGenerated.cs

```
// Generated file - do not edit.
//
// Written by Tools/deploy-stations.py from the plans saved out of
// the station placer (Tools/viewer/out/*.json). Edit the positions
// in the placer and re-run that script; anything typed here is lost
// on the next run.
using UnityEngine;

public static partial class MuseumStations
{
    /// <summary>
    /// Mary Help of Christians Parish, 17 stations.
    ///
    /// Generated by Tools/deploy-stations.py from
    /// Tools/viewer/out/mhcp.json. Do not edit by hand - the next
    /// save from the placer overwrites it.
    ///
    /// Positions are MultiSet map coordinates for mhcp-textured.glb,
    /// used verbatim on the VPS route.
    /// </summary>
    public static readonly MuseumStation[] MHCP =
    {
        new MuseumStation
        {
            stationId = "mhcp-01",
            stationName = "Mula sa Kapilya Patungo sa Pagpapatayo ng Isang Parokya",
```

```
            subtitle = "1 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(1.300f, -0.180f, 0.000f),
            triggerRadius = 2.26f,
            image = "",
            detailedDescription = "Nagsimula ang Parokya ng Maria Auxiliadora bilang isang maliit na kapilya sa Maypajo. Noong 1951, naging pagmamay-ari ng Kapisanan
ang lupang dating inuupahan sa J.P. Rizal at Taytay. Nabuo rin ang Young Ladies Association of Charity (YLAC) upang tumulong sa mga gawain ng kapilya. Noong
Disyembre 15, 1952, binuksan ang kapilya at nagsimula ang regular na Banal na Eukaristiya tuwing Linggo. Sa pagtutulungan ng mga residente at mga samahang
panrelihiyon, unti-unting nabuo ang pangarap na magkaroon ng sariling parokya.",
            nameTl = "Mula sa Kapilya Patungo sa Pagpapatayo ng Isang Parokya",
            nameEn = "From a Chapel to the Founding of a Parish",
            detailedTl = "Nagsimula ang Parokya ng Maria Auxiliadora bilang isang maliit na kapilya sa Maypajo. Noong 1951, naging pagmamay-ari ng Kapisanan ang
lupang dating inuupahan sa J.P. Rizal at Taytay. Nabuo rin ang Young Ladies Association of Charity (YLAC) upang tumulong sa mga gawain ng kapilya. Noong Disyembre
15, 1952, binuksan ang kapilya at nagsimula ang regular na Banal na Eukaristiya tuwing Linggo. Sa pagtutulungan ng mga residente at mga samahang panrelihiyon, unti-
unting nabuo ang pangarap na magkaroon ng sariling parokya.",
            detailedEn = "The Parish of Maria Auxiliadora began as a small chapel in Maypajo. In 1951, the association came to own the land it had previously rented
at J.P. Rizal and Taytay. The Young Ladies Association of Charity (YLAC) was also formed to help with the work of the chapel. On December 15, 1952, the chapel opened
and the regular Sunday celebration of the Holy Eucharist began. Through the cooperation of the residents and the religious associations, the dream of having their
own parish slowly took shape.",
            nextStationId = "mhcp-02",
        },
        new MuseumStation
        {
            stationId = "mhcp-02",
            stationName = "Ang Pagbabasbas ng Marian Playground, ang Ikalawang Proyekto",
            subtitle = "2 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.100f, -0.030f, -1.200f),
            triggerRadius = 2.23f,
            image = "ArImages/ar-image-1",
            nameTl = "Ang Pagbabasbas ng Marian Playground, ang Ikalawang Proyekto",
            nameEn = "The Blessing of the Marian Playground, the Second Project",
            nextStationId = "mhcp-03",
        },
        new MuseumStation
        {
            stationId = "mhcp-03",
            stationName = "Pagpapatatag ng Kapilya at mga Samahan",
            subtitle = "3 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(-2.300f, -0.180f, -3.500f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "Dahil sa kakulangan ng mga paring secular, inimbitahan ang mga paring Heswita upang mangasiwa sa mga misa sa kapilya. Kabilang
dito sina Pd. Albert Klaeser, Pd. J. Seffert, at Pd. Jean Desautel. Itinatag ni Pd. Albert Klaeser ang Holy Family Confraternity upang palakasin ang ugnayan ng
simbahan at pamayanan. Naging aktibo rin ang YLAC sa pagtulong sa mga pangangailangan ng kapilya. Sa pamamagitan ng pagkakaisa ng mga samahan, lalong naging matatag
ang pananampalataya ng mga taga-Maypajo.",
            nameTl = "Pagpapatatag ng Kapilya at mga Samahan",
            nameEn = "Strengthening the Chapel and Its Associations",
            detailedTl = "Dahil sa kakulangan ng mga paring secular, inimbitahan ang mga paring Heswita upang mangasiwa sa mga misa sa kapilya. Kabilang dito sina
Pd. Albert Klaeser, Pd. J. Seffert, at Pd. Jean Desautel. Itinatag ni Pd. Albert Klaeser ang Holy Family Confraternity upang palakasin ang ugnayan ng simbahan at
pamayanan. Naging aktibo rin ang YLAC sa pagtulong sa mga pangangailangan ng kapilya. Sa pamamagitan ng pagkakaisa ng mga samahan, lalong naging matatag ang
pananampalataya ng mga taga-Maypajo.",
            detailedEn = "Because of a shortage of secular priests, Jesuit priests were invited to oversee the Masses at the chapel. Among them were Fr. Albert
Klaeser, Fr. J. Seffert, and Fr. Jean Desautel. Fr. Albert Klaeser founded the Holy Family Confraternity to strengthen the bond between the church and the community.
The YLAC also remained active in meeting the needs of the chapel. Through the unity of these associations, the faith of the people of Maypajo grew still stronger.",
            nextStationId = "mhcp-04",
        },
        new MuseumStation
```

```
        {
            stationId = "mhcp-04",
            stationName = "Pd. Albert Klaeser",
            subtitle = "4 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(-3.100f, -0.030f, -4.700f),
            triggerRadius = 2.23f,
            image = "ArImages/ar-image-2",
            nameTl = "Pd. Albert Klaeser",
            nameEn = "Fr. Albert Klaeser",
            nextStationId = "mhcp-05",
        },
        new MuseumStation
        {
            stationId = "mhcp-05",
            stationName = "Mula sa Pangarap Patungo sa Pagkakatatag ng Parokya",
            subtitle = "5 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(1.300f, -0.180f, -7.000f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "Nangailangan ng malaking tulong at sapat na pondo upang maisakatuparan ang pagtatayo ng bagong simbahan. Nagkaisa ang mga pamilya
sa Maypajo upang suportahan ang mga kinakailangang gastusin, kabilang ang upa sa lupa, kuryente, kandila, at mga bulaklak. Isang bakanteng lote sa likod ng
kasalukuyang Maypajo Integrated School ang inilaan para sa pagtatayo ng simbahan. Noong Abril 27, 1956, nilagdaan ang mga dokumento para sa opisyal na pagtatatag ng
parokya, at noong Mayo 20, 1956, sinimulan ang konstruksyon sa ilalim ng pamumuno ni Kanyang Kabunyian Rufino J. Cardinal Santos.",
            nameTl = "Mula sa Pangarap Patungo sa Pagkakatatag ng Parokya",
            nameEn = "From a Dream to the Establishment of the Parish",
            detailedTl = "Nangailangan ng malaking tulong at sapat na pondo upang maisakatuparan ang pagtatayo ng bagong simbahan. Nagkaisa ang mga pamilya sa
Maypajo upang suportahan ang mga kinakailangang gastusin, kabilang ang upa sa lupa, kuryente, kandila, at mga bulaklak. Isang bakanteng lote sa likod ng kasalukuyang
Maypajo Integrated School ang inilaan para sa pagtatayo ng simbahan. Noong Abril 27, 1956, nilagdaan ang mga dokumento para sa opisyal na pagtatatag ng parokya, at
noong Mayo 20, 1956, sinimulan ang konstruksyon sa ilalim ng pamumuno ni Kanyang Kabunyian Rufino J. Cardinal Santos.",
            detailedEn = "Great help and sufficient funds were needed to carry out the building of the new church. The families of Maypajo came together to support
the necessary expenses, including the rent for the land, electricity, candles, and flowers. A vacant lot behind what is now Maypajo Integrated School was set aside
for the construction of the church. On April 27, 1956, the documents for the official establishment of the parish were signed, and on May 20, 1956, construction
began under the leadership of His Eminence Rufino J. Cardinal Santos.",
            nextStationId = "mhcp-06",
        },
        new MuseumStation
        {
            stationId = "mhcp-06",
            stationName = "Kanyang Kabunyian Rufino J. Cardinal Santos",
            subtitle = "6 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.100f, -0.030f, -8.200f),
            triggerRadius = 2.23f,
            image = "ArImages/ar-image-3",
            nameTl = "Kanyang Kabunyian Rufino J. Cardinal Santos",
            nameEn = "His Eminence Rufino J. Cardinal Santos",
            nextStationId = "mhcp-07",
        },
        new MuseumStation
        {
            stationId = "mhcp-07",
            stationName = "Ang Parokya ng Maria Auxiliadora, 1956",
            subtitle = "7 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(-2.300f, -0.180f, -10.500f),
            triggerRadius = 2.23f,
            image = "",
```

```
            detailedDescription = "Noong 1956, naging mahalagang yugto sa kasaysayan ng Maypajo ang pormal na pagtatatag ng Parokya ng Maria Auxiliadora. Mula sa dating kapilya, nagsimula itong magkaroon ng mas organisadong mga gawain at pamamahala. Naging sentro ito ng panalangin, pagsamba, at pagkakaisa ng mga mamamayan. Patuloy na nakibahagi ang mga layko sa pagpapaunlad ng parokya. Dahil dito, lalong naging matatag ang buhay pananampalataya ng pamayanan.",
            nameTl = "Ang Parokya ng Maria Auxiliadora, 1956",
            nameEn = "The Parish of Maria Auxiliadora, 1956",
            detailedTl = "Noong 1956, naging mahalagang yugto sa kasaysayan ng Maypajo ang pormal na pagtatatag ng Parokya ng Maria Auxiliadora. Mula sa dating kapilya, nagsimula itong magkaroon ng mas organisadong mga gawain at pamamahala. Naging sentro ito ng panalangin, pagsamba, at pagkakaisa ng mga mamamayan. Patuloy na nakibahagi ang mga layko sa pagpapaunlad ng parokya. Dahil dito, lalong naging matatag ang buhay pananampalataya ng pamayanan.",
            detailedEn = "In 1956, the formal establishment of the Parish of Maria Auxiliadora became an important chapter in the history of Maypajo. From the former chapel, it began to take on more organised activities and governance. It became a centre of prayer, worship, and unity for the people. The laity continued to take part in the growth of the parish. Because of this, the life of faith in the community grew firmer still.",
            nextStationId = "mhcp-08",
        },
        new MuseumStation
        {
            stationId = "mhcp-08",
            stationName = "Ang Parokya ng Maria Auxiliadora, 1956",
            subtitle = "8 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(-3.100f, -0.030f, -11.700f),
            triggerRadius = 2.50f,
            image = "ArImages/ar-image-4",
            nameTl = "Ang Parokya ng Maria Auxiliadora, 1956",
            nameEn = "The Parish of Maria Auxiliadora, 1956",
            nextStationId = "mhcp-09",
        },
        new MuseumStation
        {
            stationId = "mhcp-09",
            stationName = "Pagtatatag ng mga Programang Liturhikal at mga Samahan",
            subtitle = "9 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.100f, -0.030f, -14.000f),
            triggerRadius = 2.50f,
            image = "ArImages/ar-image-5",
            nameTl = "Pagtatatag ng mga Programang Liturhikal at mga Samahan",
            nameEn = "The Founding of Liturgical Programmes and Associations",
            nextStationId = "mhcp-10",
        },
        new MuseumStation
        {
            stationId = "mhcp-10",
            stationName = "Sa Ilalim ng Bagong Diyosesis",
            subtitle = "10 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(-2.300f, -0.180f, -17.500f),
            triggerRadius = 2.26f,
            image = "",
            detailedDescription = "Noong Hunyo 28, 2003, itinatag ang Diyosesis ng Kalookan sa ilalim ni Obispo Deogracias S. Iñiguez Jr., D.D. Sa panahong ito, pinangunahan ni Reb. Pd. Lauro E. Toledo ang patuloy na pag-unlad ng parokya. Isinagawa ang Diocesan Rosary Congress na pinangunahan ng Parish Youth Ministry. Nagkaroon din ng mga pagsasaayos sa simbahan tulad ng bagong retablo ng altar, kisame, at pagpapataas ng sahig. Patuloy na umunlad ang mga programa at gawain ng parokya bilang bahagi ng bagong diyosesis.",
            nameTl = "Sa Ilalim ng Bagong Diyosesis",
            nameEn = "Under the New Diocese",
            detailedTl = "Noong Hunyo 28, 2003, itinatag ang Diyosesis ng Kalookan sa ilalim ni Obispo Deogracias S. Iñiguez Jr., D.D. Sa panahong ito, pinangunahan ni Reb. Pd. Lauro E. Toledo ang patuloy na pag-unlad ng parokya. Isinagawa ang Diocesan Rosary Congress na pinangunahan ng Parish Youth Ministry. Nagkaroon din ng mga pagsasaayos sa simbahan tulad ng bagong retablo ng altar, kisame, at pagpapataas ng sahig. Patuloy na umunlad ang mga programa at gawain ng parokya bilang bahagi ng bagong diyosesis.",
```

```
            detailedEn = "On June 28, 2003, the Diocese of Kalookan was established under Bishop Deogracias S. Iñiguez Jr., D.D. During this time, Rev. Fr. Lauro E. Toledo led the continuing growth of the parish. The Diocesan Rosary Congress was held, spearheaded by the Parish Youth Ministry. Improvements were also made to the church, such as a new altar retablo, the ceiling, and the raising of the floor. The programmes and activities of the parish continued to develop as part of the new diocese.",
            nextStationId = "mhcp-11",
        },
        new MuseumStation
        {
            stationId = "mhcp-11",
            stationName = "Diocesan Rosary Congress, 2002",
            subtitle = "11 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(-3.100f, -0.030f, -18.700f),
            triggerRadius = 2.23f,
            image = "ArImages/ar-image-6",
            shortDescription = "Sanggunian: G. Richard Ricafrente",
            nameTl = "Diocesan Rosary Congress, 2002",
            nameEn = "Diocesan Rosary Congress, 2002",
            shortTl = "Sanggunian: G. Richard Ricafrente",
            shortEn = "Source: Mr. Richard Ricafrente",
            nextStationId = "mhcp-12",
        },
        new MuseumStation
        {
            stationId = "mhcp-12",
            stationName = "Ang Parokya sa Ilalim ng Pandemya: Kwento ng Pagbangon",
            subtitle = "12 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(1.300f, -0.180f, -21.000f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "Malaki ang naging epekto ng COVID-19 pandemic sa mga gawain at pagsamba ng Parokya ng Maria Auxiliadora. Sa kabila nito, nanatiling matatag ang pananampalataya at debosyon ng mga taga-Maypajo kay Maria Auxiliadora. Gumamit ang parokya ng social media upang magsagawa ng online Masses at iba pang programang espirituwal. Nang unti-unting bumalik ang mga pisikal na gawain, nagsimulang muling bumangon ang pamayanan. Naging simbolo ng pagbangon ang mga bagong proyekto at pagdiriwang ng parokya pagkatapos ng pandemya.",
            nameTl = "Ang Parokya sa Ilalim ng Pandemya: Kwento ng Pagbangon",
            nameEn = "The Parish Under the Pandemic: A Story of Rising Again",
            detailedTl = "Malaki ang naging epekto ng COVID-19 pandemic sa mga gawain at pagsamba ng Parokya ng Maria Auxiliadora. Sa kabila nito, nanatiling matatag ang pananampalataya at debosyon ng mga taga-Maypajo kay Maria Auxiliadora. Gumamit ang parokya ng social media upang magsagawa ng online Masses at iba pang programang espirituwal. Nang unti-unting bumalik ang mga pisikal na gawain, nagsimulang muling bumangon ang pamayanan. Naging simbolo ng pagbangon ang mga bagong proyekto at pagdiriwang ng parokya pagkatapos ng pandemya.",
            detailedEn = "The COVID-19 pandemic had a great effect on the activities and worship of the Parish of Maria Auxiliadora. In spite of this, the faith and devotion of the people of Maypajo to Maria Auxiliadora remained firm. The parish used social media to hold online Masses and other spiritual programmes. As in-person activities slowly returned, the community began to rise again. The new projects and celebrations of the parish after the pandemic became a symbol of that rising.",
            nextStationId = "mhcp-13",
        },
        new MuseumStation
        {
            stationId = "mhcp-13",
            stationName = "Pagbabasbas at Pagpapasinaya ng Prayer Sanctuary ng Parokya, 2021",
            subtitle = "13 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.100f, -0.030f, -22.200f),
            triggerRadius = 2.23f,
            image = "ArImages/ar-image-7",
            nameTl = "Pagbabasbas at Pagpapasinaya ng Prayer Sanctuary ng Parokya, 2021",
            nameEn = "The Blessing and Inauguration of the Parish Prayer Sanctuary, 2021",
            nextStationId = "mhcp-14",
        },
```

```
        new MuseumStation
        {
            stationId = "mhcp-14",
            stationName = "Ang Parokya sa Kasalukuyang Panahon",
            subtitle = "14 ng 17",
            kind = MuseumStationKind.History,
            position = new Vector3(-4.400f, -0.180f, -24.500f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "Patuloy na isinasagawa ng Parokya ng Maria Auxiliadora ang mga gawaing panrelihiyon at paglilingkod sa komunidad. Noong Pebrero
10, 2025, itinalaga si Reb. Pd. Kennedy A. Neral bilang parish priest at si Reb. Pd. Paul Nicklaus C. Woo bilang parochial vicar. Sa kanilang pamumuno, patuloy na
ibinabahagi ng parokya ang Mabuting Balita at inilalapit ang simbahan sa pamayanan. Patuloy rin ang mga debosyon at pagdiriwang bilang parangal kay Maria
Auxiliadora. Sa halos pitumpung taon, nananatili siyang mahalagang Ina at Patrona ng Maypajo.",
            nameTl = "Ang Parokya sa Kasalukuyang Panahon",
            nameEn = "The Parish in the Present Day",
            detailedTl = "Patuloy na isinasagawa ng Parokya ng Maria Auxiliadora ang mga gawaing panrelihiyon at paglilingkod sa komunidad. Noong Pebrero 10, 2025,
itinalaga si Reb. Pd. Kennedy A. Neral bilang parish priest at si Reb. Pd. Paul Nicklaus C. Woo bilang parochial vicar. Sa kanilang pamumuno, patuloy na ibinabahagi
ng parokya ang Mabuting Balita at inilalapit ang simbahan sa pamayanan. Patuloy rin ang mga debosyon at pagdiriwang bilang parangal kay Maria Auxiliadora. Sa halos
pitumpung taon, nananatili siyang mahalagang Ina at Patrona ng Maypajo.",
            detailedEn = "The Parish of Maria Auxiliadora continues to carry out its religious work and its service to the community. On February 10, 2025, Rev. Fr.
Kennedy A. Neral was appointed parish priest and Rev. Fr. Paul Nicklaus C. Woo parochial vicar. Under their leadership, the parish continues to share the Good News
and to bring the church closer to the community. The devotions and celebrations in honour of Maria Auxiliadora also continue. For nearly seventy years she has
remained the beloved Mother and Patroness of Maypajo.",
            nextStationId = "mhcp-15",
        },
        new MuseumStation
        {
            stationId = "mhcp-15",
            stationName = "Rev. Fr. Kennedy Neral at Rev. Fr. Paul Nicklaus Woo",
            subtitle = "15 ng 17",
            kind = MuseumStationKind.Image,
            position = new Vector3(-4.800f, -0.030f, -25.050f),
            triggerRadius = 2.50f,
            image = "ArImages/ar-image-8",
            nameTl = "Rev. Fr. Kennedy Neral at Rev. Fr. Paul Nicklaus Woo",
            nameEn = "Rev. Fr. Kennedy Neral and Rev. Fr. Paul Nicklaus Woo",
            nextStationId = "mhcp-16",
        },
        new MuseumStation
        {
            stationId = "mhcp-16",
            stationName = "Mary Help of Christians",
            subtitle = "16 ng 17",
            kind = MuseumStationKind.Model,
            position = new Vector3(-1.774f, -1.630f, -25.013f),
            triggerRadius = 3.00f,
            model = "mary-help",
            modelScale = 0.87f,
            yawDegrees = 5f,
            detailedDescription = "Mary Help of Christians is the patroness of the Mary Help of Christians Parish in Maypajo, Caloocan. She is honored as a loving
and protective mother who guides, strengthens, and brings hope to the faithful. Her devotion inspires the parish community to remain united in faith and service.",
            nameTl = "Maria Auxiliadora",
            nameEn = "Mary Help of Christians",
            detailedTl = "Si Maria Auxiliadora ang Patrona ng Parokya ng Mary Help of Christians sa Maypajo, Caloocan. Pinararangalan siya bilang isang mapagmahal at
mapagtanggol na ina na gumagabay, nagpapalakas, at nagdudulot ng pag-asa sa mga mananampalataya. Ang debosyon sa kanya ay nagbibigay-inspirasyon sa pamayanan ng
parokya upang manatiling nagkakaisa sa pananampalataya at paglilingkod.",
            detailedEn = "Mary Help of Christians is the patroness of the Mary Help of Christians Parish in Maypajo, Caloocan. She is honored as a loving and
protective mother who guides, strengthens, and brings hope to the faithful. Her devotion inspires the parish community to remain united in faith and service.",
            nextStationId = "mhcp-17",
```

```
        },
        new MuseumStation
        {
            stationId = "mhcp-17",
            stationName = "Sto. Niño de Pajotan",
            subtitle = "17 ng 17",
            kind = MuseumStationKind.Model,
            position = new Vector3(4.075f, -1.480f, -25.367f),
            triggerRadius = 3.00f,
            model = "santo-nino-pajotan",
            modelScale = 0.86f,
            yawDegrees = 25f,
            shortDescription = "Through a colorful religious procession, traditional Ati-Atihan dancing, decorated floats, and lively performances. Its name comes
from the local variety of small, sweet mangoes called “Pajotan,” which is associated with Maypajo. The festival represents the community's deep faith, unity, and
devotion to the Child Jesus.",
            detailedDescription = "Sto. Niño de Pajotan is the patron of the Pajotan Sto. Niño Festival. The antique heirloom image, originally belonging to the
Navarro family of Maypajo, is now enshrined at the side altar of the Mary Help of Christians Parish. Devotees honor the Child Jesus through prayer, dancing,
processions, and joyful celebration, expressing their faith, gratitude, and devotion.",
            nameTl = "Sto. Niño de Pajotan",
            nameEn = "Sto. Niño de Pajotan",
            shortTl = "Sa pamamagitan ng makulay na prusisyon, tradisyonal na sayaw na Ati-Atihan, mga pinalamutiang karosa, at masisiglang pagtatanghal. Ang
pangalan nito ay mula sa katutubong uri ng maliliit at matatamis na mangga na tinatawag na \"Pajotan,\" na iniuugnay sa Maypajo. Ang pista ay kumakatawan sa malalim
na pananampalataya, pagkakaisa, at debosyon ng pamayanan sa Banal na Sanggol.",
            shortEn = "Through a colorful religious procession, traditional Ati-Atihan dancing, decorated floats, and lively performances. Its name comes from the
local variety of small, sweet mangoes called \"Pajotan,\" which is associated with Maypajo. The festival represents the community's deep faith, unity, and devotion
to the Child Jesus.",
            detailedTl = "Ang Sto. Niño de Pajotan ang patron ng Pajotan Sto. Niño Festival. Ang lumang imaheng minana, na dating pag-aari ng pamilyang Navarro ng
Maypajo, ay nakaluklok na ngayon sa gilid na altar ng Parokya ng Mary Help of Christians. Pinararangalan ng mga deboto ang Banal na Sanggol sa pamamagitan ng
panalangin, sayaw, prusisyon, at masayang pagdiriwang, bilang pagpapahayag ng kanilang pananampalataya, pasasalamat, at debosyon.",
            detailedEn = "Sto. Niño de Pajotan is the patron of the Pajotan Sto. Niño Festival. The antique heirloom image, originally belonging to the Navarro
family of Maypajo, is now enshrined at the side altar of the Mary Help of Christians Parish. Devotees honor the Child Jesus through prayer, dancing, processions, and
joyful celebration, expressing their faith, gratitude, and devotion.",
            nextStationId = "",
        },
    };

    /// <summary>
    /// Where the printed mhcp marker hangs, in this parish's map space, and
    /// the direction it faces. Measured by placing it on the scan, not
    /// assumed.
    /// </summary>
    public static readonly Vector3 MHCP_MarkerInMap =
        new Vector3(-6.513f, -0.144f, 1.829f);
    public static readonly Vector3 MHCP_MarkerNormal =
        new Vector3(-0.0000f, 0.0000f, -1.0000f);

    /// <summary>
    /// San Roque Cathedral Parish, 15 stations.
    ///
    /// Generated by Tools/deploy-stations.py from
    /// Tools/viewer/out/srcp.json. Do not edit by hand - the next
    /// save from the placer overwrites it.
    ///
    /// Positions are MultiSet map coordinates for srcp-textured.glb,
    /// used verbatim on the VPS route.
    /// </summary>
    public static readonly MuseumStation[] SRCP =
    {
        new MuseumStation
```

```
        {
            stationId = "srcp-01",
            stationName = "The Beginning",
            subtitle = "1 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(2.050f, -0.170f, 3.000f),
            triggerRadius = 2.26f,
            image = "",
            detailedDescription = "San Roque de Caloocan began as a visita of the Parish of Tondo. In 1762, Fr. Antonio de San Prospero, O.S.A. became the first
missionary assigned to the community. The area was originally known as Aromahan or Espina, near the lagoon called Dagat-Dagatan.",
            nameTl = "Ang Simula",
            nameEn = "The Beginning",
            detailedTl = "Nagsimula ang San Roque de Caloocan bilang isang visita ng Parokya ng Tondo. Noong 1762, si Fr. Antonio de San Prospero, O.S.A. ang naging
kauna-unahang misyonerong itinalaga sa pamayanan. Ang lugar ay dating kilala bilang Aromahan o Espina, malapit sa lawa na tinatawag na Dagat-Dagatan.",
            detailedEn = "San Roque de Caloocan began as a visita of the Parish of Tondo. In 1762, Fr. Antonio de San Prospero, O.S.A. became the first missionary
assigned to the community. The area was originally known as Aromahan or Espina, near the lagoon called Dagat-Dagatan.",
            nextStationId = "srcp-02",
        },
        new MuseumStation
        {
            stationId = "srcp-02",
            stationName = "The Beginning",
            subtitle = "2 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.850f, -0.020f, 1.800f),
            triggerRadius = 2.23f,
            image = "ArImages/src-image-1",
            nameTl = "Ang Simula",
            nameEn = "The Beginning",
            nextStationId = "srcp-03",
        },
        new MuseumStation
        {
            stationId = "srcp-03",
            stationName = "Becoming a Parish",
            subtitle = "3 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(-1.550f, -0.170f, -0.500f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "As the community grew, the town and church were moved to their present location in 1802. In 1815, Kalookan became an independent
town and parish separate from Tondo. A larger church was later constructed, with the work completed in 1847.",
            nameTl = "Pagiging Isang Parokya",
            nameEn = "Becoming a Parish",
            detailedTl = "Habang lumalaki ang pamayanan, inilipat ang bayan at ang simbahan sa kasalukuyang kinaroroonan nito noong 1802. Noong 1815, naging malayang
bayan at parokya ang Kalookan, hiwalay na sa Tondo. Nang maglaon ay nagpatayo ng mas malaking simbahan, at natapos ang gawain noong 1847.",
            detailedEn = "As the community grew, the town and church were moved to their present location in 1802. In 1815, Kalookan became an independent town and
parish separate from Tondo. A larger church was later constructed, with the work completed in 1847.",
            nextStationId = "srcp-04",
        },
        new MuseumStation
        {
            stationId = "srcp-04",
            stationName = "Becoming a Parish",
            subtitle = "4 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(-2.350f, -0.020f, -1.700f),
            triggerRadius = 2.23f,
            image = "ArImages/src-image-2",
```

```
            nameTl = "Pagiging Isang Parokya",
            nameEn = "Becoming a Parish",
            nextStationId = "srcp-05",
        },
        new MuseumStation
        {
            stationId = "srcp-05",
            stationName = "During the Revolution",
            subtitle = "5 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(2.050f, -0.170f, -4.000f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "San Roque Parish became connected to the Filipino struggle for freedom. During the events leading to the Cry of Balintawak in
1896, Katipuneros used homes in Caloocan as places of shelter. One famous incident during the San Roque fiesta became known as the Pistang Takbuhan.",
            nameTl = "Noong Panahon ng Himagsikan",
            nameEn = "During the Revolution",
            detailedTl = "Naging kaugnay ang Parokya ng San Roque sa pakikibaka ng mga Pilipino para sa kalayaan. Sa mga pangyayaring humantong sa Sigaw ng
Balintawak noong 1896, ginamit ng mga Katipunero ang mga bahay sa Caloocan bilang mga taguan. Isang tanyag na pangyayari noong pista ng San Roque ang nakilala bilang
Pistang Takbuhan.",
            detailedEn = "San Roque Parish became connected to the Filipino struggle for freedom. During the events leading to the Cry of Balintawak in 1896,
Katipuneros used homes in Caloocan as places of shelter. One famous incident during the San Roque fiesta became known as the Pistang Takbuhan.",
            nextStationId = "srcp-06",
        },
        new MuseumStation
        {
            stationId = "srcp-06",
            stationName = "Pistang Takbuhan",
            subtitle = "6 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.850f, -0.020f, -5.200f),
            triggerRadius = 2.23f,
            image = "ArImages/src-image-3",
            nameTl = "Pistang Takbuhan",
            nameEn = "Pistang Takbuhan",
            nextStationId = "srcp-07",
        },
        new MuseumStation
        {
            stationId = "srcp-07",
            stationName = "The Philippine-American War",
            subtitle = "7 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(-1.550f, -0.170f, -7.500f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "During the Philippine-American War, the church became a refuge for General Antonio Luna and served as part of the Filipino defense
during the Battle of Kalookan on February 10, 1899. The church was severely damaged and was later used by American forces as a stable and field hospital.",
            nameTl = "Ang Digmaang Pilipino-Amerikano",
            nameEn = "The Philippine-American War",
            detailedTl = "Noong Digmaang Pilipino-Amerikano, naging kanlungan ang simbahan ni Heneral Antonio Luna at naging bahagi ng depensa ng mga Pilipino sa
Labanan sa Kalookan noong ika-10 ng Pebrero 1899. Malubhang nasira ang simbahan at ginamit itong kuwadra at pansamantalang ospital ng mga puwersang Amerikano.",
            detailedEn = "During the Philippine-American War, the church became a refuge for General Antonio Luna and served as part of the Filipino defense during
the Battle of Kalookan on February 10, 1899. The church was severely damaged and was later used by American forces as a stable and field hospital.",
            nextStationId = "srcp-08",
        },
        new MuseumStation
        {
            stationId = "srcp-08",
```

```
            stationName = "The Battle of Kalookan, 1899",
            subtitle = "8 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(-2.350f, -0.020f, -8.700f),
            triggerRadius = 2.23f,
            image = "ArImages/src-image-4",
            nameTl = "Ang Labanan sa Kalookan, 1899",
            nameEn = "The Battle of Kalookan, 1899",
            nextStationId = "srcp-09",
        },
        new MuseumStation
        {
            stationId = "srcp-09",
            stationName = "Ruin and Reconstruction",
            subtitle = "9 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(2.050f, -0.170f, -11.000f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "After the destruction caused by the war, reconstruction began in 1914. Repairs and improvements continued over the following
decades. The church also survived the Japanese Occupation during World War II, which residents attributed to the intercession of San Roque.",
            nameTl = "Pagkawasak at Muling Pagtatayo",
            nameEn = "Ruin and Reconstruction",
            detailedTl = "Matapos ang pagkawasak na dulot ng digmaan, nagsimula ang muling pagtatayo noong 1914. Nagpatuloy ang pagkukumpuni at pagpapaganda sa mga
sumunod na dekada. Nalagpasan din ng simbahan ang Pananakop ng mga Hapones noong Ikalawang Digmaang Pandaigdig, na iniugnay ng mga residente sa pamamagitan ni San
Roque.",
            detailedEn = "After the destruction caused by the war, reconstruction began in 1914. Repairs and improvements continued over the following decades. The
church also survived the Japanese Occupation during World War II, which residents attributed to the intercession of San Roque.",
            nextStationId = "srcp-10",
        },
        new MuseumStation
        {
            stationId = "srcp-10",
            stationName = "The Old San Roque Church",
            subtitle = "10 ng 15",
            kind = MuseumStationKind.Model,
            position = new Vector3(1.512f, -0.686f, -11.276f),
            triggerRadius = 2.23f,
            model = "old-cathedral",
            modelScale = 1.00f,
            yawDegrees = 0f,
            nameTl = "Ang Lumang Simbahan ng San Roque",
            nameEn = "The Old San Roque Church",
            nextStationId = "srcp-11",
        },
        new MuseumStation
        {
            stationId = "srcp-11",
            stationName = "From Parish to Cathedral",
            subtitle = "11 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(-1.550f, -0.170f, -14.500f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "In 2003, Pope John Paul II established the Diocese of Kalookan through the Apostolic Letter Quoniam Quae Libet. With the creation
of the diocese, San Roque Parish was elevated to San Roque Cathedral Parish, becoming the mother church of the Diocese.",
            nameTl = "Mula Parokya Tungo sa Katedral",
            nameEn = "From Parish to Cathedral",
```

```
            detailedTl = "Noong 2003, itinatag ni Papa Juan Pablo II ang Diyosesis ng Kalookan sa pamamagitan ng Apostolikong Liham na Quoniam Quae Libet. Sa
pagkakatatag ng diyosesis, itinaas ang Parokya ng San Roque bilang San Roque Cathedral Parish, at naging inang simbahan ito ng Diyosesis.",
            detailedEn = "In 2003, Pope John Paul II established the Diocese of Kalookan through the Apostolic Letter Quoniam Quae Libet. With the creation of the
diocese, San Roque Parish was elevated to San Roque Cathedral Parish, becoming the mother church of the Diocese.",
            nextStationId = "srcp-12",
        },
        new MuseumStation
        {
            stationId = "srcp-12",
            stationName = "From Parish to Cathedral, 2003",
            subtitle = "12 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(-2.350f, -0.020f, -15.700f),
            triggerRadius = 2.23f,
            image = "ArImages/src-image-6",
            nameTl = "Mula Parokya Tungo sa Katedral, 2003",
            nameEn = "From Parish to Cathedral, 2003",
            nextStationId = "srcp-13",
        },
        new MuseumStation
        {
            stationId = "srcp-13",
            stationName = "The Cathedral Today",
            subtitle = "13 ng 15",
            kind = MuseumStationKind.History,
            position = new Vector3(2.050f, -0.170f, -18.000f),
            triggerRadius = 2.23f,
            image = "",
            detailedDescription = "In 2015, San Roque Cathedral celebrated the bicentennial of its canonical establishment. The altar and sanctuary were renovated,
followed by the solemn dedication of the Cathedral on December 12, 2015. A relic of the bone of San Roque was also gifted by Pope Francis and placed in the
consecrated altar.",
            nameTl = "Ang Katedral Ngayon",
            nameEn = "The Cathedral Today",
            detailedTl = "Noong 2015, ipinagdiwang ng San Roque Cathedral ang ikalawang daantaon ng kanonikal na pagkakatatag nito. Isinaayos ang altar at ang
santuwaryo, at sinundan ito ng solemneng pagtatalaga ng Katedral noong ika-12 ng Disyembre 2015. Iginawad din ni Papa Francisco ang isang relikya ng buto ni San
Roque, na inilagay sa konsagradong altar.",
            detailedEn = "In 2015, San Roque Cathedral celebrated the bicentennial of its canonical establishment. The altar and sanctuary were renovated, followed
by the solemn dedication of the Cathedral on December 12, 2015. A relic of the bone of San Roque was also gifted by Pope Francis and placed in the consecrated
altar.",
            nextStationId = "srcp-14",
        },
        new MuseumStation
        {
            stationId = "srcp-14",
            stationName = "The Dedication, 2015",
            subtitle = "14 ng 15",
            kind = MuseumStationKind.Image,
            position = new Vector3(2.850f, -0.020f, -19.200f),
            triggerRadius = 2.50f,
            image = "ArImages/src-image-7",
            nameTl = "Ang Pagtatalaga, 2015",
            nameEn = "The Dedication, 2015",
            nextStationId = "srcp-15",
        },
        new MuseumStation
        {
            stationId = "srcp-15",
            stationName = "San Roque",
            subtitle = "15 ng 15",
```

```
            kind = MuseumStationKind.Model,
            position = new Vector3(-10.092f, -1.632f, -18.444f),
            triggerRadius = 3.00f,
            model = "san-roque",
            modelScale = 1.00f,
            yawDegrees = -10f,
            nameTl = "San Roque",
            nameEn = "San Roque",
            nextStationId = "",
        },
    };

    /// <summary>
    /// Where the printed srcp marker hangs, in this parish's map space, and
    /// the direction it faces. Measured by placing it on the scan, not
    /// assumed.
    /// </summary>
    public static readonly Vector3 SRCP_MarkerInMap =
        new Vector3(3.753f, 0.117f, 4.003f);
    public static readonly Vector3 SRCP_MarkerNormal =
        new Vector3(-0.0000f, 0.0000f, -1.0000f);

    /// <summary>
    /// San Roque tour, previewed in the sala, 5 stations.
    ///
    /// Generated by Tools/deploy-stations.py from
    /// Tools/viewer/out/srcp-sala.json. Do not edit by hand - the next
    /// save from the placer overwrites it.
    ///
    /// Positions are MultiSet map coordinates for sala-textured.glb,
    /// used verbatim on the VPS route.
    /// </summary>
    public static readonly MuseumStation[] SRCP_IN_SALA =
    {
        new MuseumStation
        {
            stationId = "srcp-01",
            stationName = "The Beginning",
            subtitle = "1 ng 5",
            kind = MuseumStationKind.History,
            position = new Vector3(-1.953f, -0.727f, 0.761f),
            triggerRadius = 1.50f,
            image = "",
            detailedDescription = "San Roque de Caloocan began as a visita of the Parish of Tondo. In 1762, Fr. Antonio de San Prospero, O.S.A. became the first
missionary assigned to the community. The area was originally known as Aromahan or Espina, near the lagoon called Dagat-Dagatan.",
            nextStationId = "srcp-02",
        },
        new MuseumStation
        {
            stationId = "srcp-02",
            stationName = "The Beginning",
            subtitle = "2 ng 5",
            kind = MuseumStationKind.Image,
            position = new Vector3(-0.302f, -0.731f, 4.946f),
            triggerRadius = 1.50f,
            image = "ArImages/src-image-1",
            nextStationId = "srcp-03",
        },
        new MuseumStation
```

```
        {
            stationId = "srcp-03",
            stationName = "Becoming a Parish",
            subtitle = "3 ng 5",
            kind = MuseumStationKind.History,
            position = new Vector3(-1.636f, -1.006f, 3.019f),
            triggerRadius = 1.50f,
            image = "",
            detailedDescription = "As the community grew, the town and church were moved to their present location in 1802. In 1815, Kalookan became an independent
town and parish separate from Tondo. A larger church was later constructed, with the work completed in 1847.",
            nextStationId = "srcp-04",
        },
        new MuseumStation
        {
            stationId = "srcp-04",
            stationName = "Becoming a Parish",
            subtitle = "4 ng 5",
            kind = MuseumStationKind.Image,
            position = new Vector3(-3.309f, -0.501f, 4.862f),
            triggerRadius = 1.50f,
            image = "ArImages/src-image-2",
            nextStationId = "srcp-15",
        },
        new MuseumStation
        {
            stationId = "srcp-15",
            stationName = "San Roque",
            subtitle = "5 ng 5",
            kind = MuseumStationKind.Model,
            position = new Vector3(-1.000f, -1.587f, 2.000f),
            triggerRadius = 3.00f,
            model = "san-roque",
            modelScale = 1.00f,
            yawDegrees = 0f,
            nextStationId = "",
        },
    };

    /// <summary>
    /// Where the printed srcp-sala marker hangs, in this parish's map space, and
    /// the direction it faces. Measured by placing it on the scan, not
    /// assumed.
    /// </summary>
    public static readonly Vector3 SRCP_IN_SALA_MarkerInMap =
        new Vector3(-0.870f, -0.144f, -0.037f);
    public static readonly Vector3 SRCP_IN_SALA_MarkerNormal =
        new Vector3(-0.7066f, 0.0000f, -0.7076f);

    /// <summary>
    /// Mary Help statues, previewed in the sala, 4 stations.
    ///
    /// Generated by Tools/deploy-stations.py from
    /// Tools/viewer/out/mhcp-sala.json. Do not edit by hand - the next
    /// save from the placer overwrites it.
    ///
    /// Positions are MultiSet map coordinates for sala-textured.glb,
    /// used verbatim on the VPS route.
    /// </summary>
    public static readonly MuseumStation[] MHCP_IN_SALA =
```

```
    {
        new MuseumStation
        {
            stationId = "mhcp-sala-1",
            stationName = "Mary Help of Christians",
            subtitle = "1 ng 4",
            kind = MuseumStationKind.Model,
            position = new Vector3(-0.723f, -0.515f, 1.365f),
            triggerRadius = 1.50f,
            model = "mary-help",
            modelScale = 0.87f,
            yawDegrees = -240f,
            detailedDescription = "Mary Help of Christians is the patroness of the Mary Help of Christians Parish in Maypajo, Caloocan. She is honored as a loving
and protective mother who guides, strengthens, and brings hope to the faithful. Her devotion inspires the parish community to remain united in faith and service.",
            nameTl = "Maria Auxiliadora",
            nameEn = "Mary Help of Christians",
            detailedTl = "Si Maria Auxiliadora ang Patrona ng Parokya ng Mary Help of Christians sa Maypajo, Caloocan. Pinararangalan siya bilang isang mapagmahal at
mapagtanggol na ina na gumagabay, nagpapalakas, at nagdudulot ng pag-asa sa mga mananampalataya. Ang debosyon sa kanya ay nagbibigay-inspirasyon sa pamayanan ng
parokya upang manatiling nagkakaisa sa pananampalataya at paglilingkod.",
            detailedEn = "Mary Help of Christians is the patroness of the Mary Help of Christians Parish in Maypajo, Caloocan. She is honored as a loving and
protective mother who guides, strengthens, and brings hope to the faithful. Her devotion inspires the parish community to remain united in faith and service.",
            nextStationId = "mhcp-sala-2",
        },
        new MuseumStation
        {
            stationId = "mhcp-sala-2",
            stationName = "Sto. Niño de Pajotan",
            subtitle = "2 ng 4",
            kind = MuseumStationKind.Model,
            position = new Vector3(-1.253f, -1.037f, 1.811f),
            triggerRadius = 1.50f,
            model = "santo-nino-pajotan",
            modelScale = 0.86f,
            yawDegrees = -270f,
            shortDescription = "Through a colorful religious procession, traditional Ati-Atihan dancing, decorated floats, and lively performances.",
            detailedDescription = "Sto. Niño de Pajotan is the patron of the Pajotan Sto. Niño Festival. The antique heirloom image, originally belonging to the
Navarro family of Maypajo, is now enshrined at the side altar of the Mary Help of Christians Parish. Devotees honor the Child Jesus through prayer, dancing,
processions, and joyful celebration, expressing their faith, gratitude, and devotion.",
            nameTl = "Sto. Niño de Pajotan",
            nameEn = "Sto. Niño de Pajotan",
            shortTl = "Sa pamamagitan ng makulay na prusisyon, tradisyonal na sayaw na Ati-Atihan, mga pinalamutiang karosa, at masisiglang pagtatanghal. Ang
pangalan nito ay mula sa katutubong uri ng maliliit at matatamis na mangga na tinatawag na \"Pajotan,\" na iniuugnay sa Maypajo. Ang pista ay kumakatawan sa malalim
na pananampalataya, pagkakaisa, at debosyon ng pamayanan sa Banal na Sanggol.",
            shortEn = "Through a colorful religious procession, traditional Ati-Atihan dancing, decorated floats, and lively performances. Its name comes from the
local variety of small, sweet mangoes called \"Pajotan,\" which is associated with Maypajo. The festival represents the community's deep faith, unity, and devotion
to the Child Jesus.",
            detailedTl = "Ang Sto. Niño de Pajotan ang patron ng Pajotan Sto. Niño Festival. Ang lumang imaheng minana, na dating pag-aari ng pamilyang Navarro ng
Maypajo, ay nakaluklok na ngayon sa gilid na altar ng Parokya ng Mary Help of Christians. Pinararangalan ng mga deboto ang Banal na Sanggol sa pamamagitan ng
panalangin, sayaw, prusisyon, at masayang pagdiriwang, bilang pagpapahayag ng kanilang pananampalataya, pasasalamat, at debosyon.",
            detailedEn = "Sto. Niño de Pajotan is the patron of the Pajotan Sto. Niño Festival. The antique heirloom image, originally belonging to the Navarro
family of Maypajo, is now enshrined at the side altar of the Mary Help of Christians Parish. Devotees honor the Child Jesus through prayer, dancing, processions, and
joyful celebration, expressing their faith, gratitude, and devotion.",
            nextStationId = "mhcp-sala-4",
        },
        new MuseumStation
        {
            stationId = "mhcp-sala-4",
            stationName = "San Roque",
            subtitle = "3 ng 4",
```

```
            kind = MuseumStationKind.Model,
            position = new Vector3(-2.000f, -0.937f, 0.650f),
            triggerRadius = 1.50f,
            model = "san-roque",
            modelScale = 1.00f,
            yawDegrees = 0f,
            detailedDescription = "San Roque, patron of the San Roque Cathedral Parish in Caloocan.",
            nextStationId = "mhcp-sala-5",
        },
        new MuseumStation
        {
            stationId = "mhcp-sala-5",
            stationName = "St Cyril",
            subtitle = "4 ng 4",
            kind = MuseumStationKind.Model,
            position = new Vector3(-3.050f, -1.587f, 2.300f),
            triggerRadius = 1.50f,
            model = "st-cyril",
            modelScale = 1.00f,
            yawDegrees = 0f,
            detailedDescription = "St Cyril.",
            nextStationId = "",
        },
    };

    /// <summary>
    /// Where the printed mhcp-sala marker hangs, in this parish's map space, and
    /// the direction it faces. Measured by placing it on the scan, not
    /// assumed.
    /// </summary>
    public static readonly Vector3 MHCP_IN_SALA_MarkerInMap =
        new Vector3(-0.870f, -0.144f, -0.037f);
    public static readonly Vector3 MHCP_IN_SALA_MarkerNormal =
        new Vector3(-0.4524f, 0.0000f, 0.8918f);
}
```

4.1.12 AR-012 — MuseumUI.cs

Code ID	AR-012
File Name	MuseumUI.cs
File Path	SanctiWalkAR/Assets/Scripts/MuseumUI.cs
Language / Type	C#
Lines	1154
Functions / Classes	MuseumUI (public class); Awake (method); SpawnModel (method); Bind (public method); SetMarkerSign (public method); SpawnMarkerLogo (method); RebuildForLanguage (public method); Reposition (public method); BuildCanvas (method); BuildScanPanel (method); BuildPreviewButton (method); PreviewNext (method); BuildBubblePanel (method); BuildCardPanel (method); BuildNextPill (method); SetVisible (public method); ShowScanning (method); OpenCardFor (public method); ShowBubble (method); ShowCard (method); ShowPanelCard (method); MetaRow (method); FitCardToContent (method); HandleBubbleTap (method); CloseCard (method); UpdateNextPill (method); Update (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/MuseumUI.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MuseumUI.cs

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// The museum experience UI, from the SanctiWalk AR design.
///
/// Five states, in the order the design lays them out:
///   1. Scan      - reticle and "point your camera at a marker"
///   2. Bubble    - thumbnail, name, one line, "View details"
///   3. Card      - full text plus the year / artist / material rows
///   4. Next stop - the pill with the following station and its distance
///   5. Discover  - the bubble reappearing for whatever you walk up to next
///
/// Built entirely in C#. Nothing here is a prefab, because a prefab has to
/// be assembled by clicking in the editor and this project is driven from
/// the command line. It also means the layout is reviewable as text.
///
/// Legacy uGUI rather than TextMeshPro: TMP needs its Essential Resources
/// imported before it will render anything at all, and they are not in this
/// project. uGUI Text with a dynamic TTF needs no such step.
/// </summary>
public class MuseumUI : MonoBehaviour
```

```
{
    // ------------------------------------------------------------ design
    // Sampled from the palette in the design sheet.
    static readonly Color Cream      = new Color32(0xFB, 0xF8, 0xF3, 0xFF);
    static readonly Color CardWhite  = new Color32(0xFF, 0xFF, 0xFF, 0xFA);
    static readonly Color Ink        = new Color32(0x1A, 0x1D, 0x23, 0xFF);
    static readonly Color Muted      = new Color32(0x6B, 0x72, 0x80, 0xFF);
    static readonly Color Navy       = new Color32(0x14, 0x26, 0x4A, 0xFF);
    static readonly Color Accent     = new Color32(0x1D, 0x4E, 0xD8, 0xFF);
    static readonly Color Sand       = new Color32(0xE8, 0xDC, 0xC8, 0xFF);
    static readonly Color Scrim      = new Color32(0x00, 0x00, 0x00, 0x66);

    const int PadX = 18;

    Font m_Heading;   // Montserrat
    Font m_Body;      // Poppins
    Font m_BodyMed;

    // ------------------------------------------------------------ state
    public enum Phase { Scanning, Bubble, Card }

    Phase m_Phase = Phase.Scanning;
    MuseumStation m_Current;
    MuseumStation[] m_Stations;

    /// <summary>Marker-relative station positions, resolved into world space.</summary>
    readonly Dictionary<string, Vector3> m_World = new Dictionary<string, Vector3>();
    readonly HashSet<string> m_Visited = new HashSet<string>();
    readonly List<StationBubble> m_Bubbles = new List<StationBubble>();

    Camera m_Camera;
    bool m_Located;

    // panels
    GameObject m_ScanPanel, m_BubblePanel, m_CardPanel, m_NextPill;
    RectTransform m_Root;
    Text m_BubbleTitle, m_BubbleSub, m_BubbleBody;
    RawImage m_BubbleThumb;
    Text m_CardTitle, m_CardSub, m_CardBody;
    RectTransform m_CardMetaHolder;
    Text m_NextText;

    // ------------------------------------------------------------ build
    void Awake()
    {
        m_Camera = Camera.main;
        m_Heading = Resources.Load<Font>("Montserrat-SemiBold") ?? LoadFont("Montserrat-SemiBold");
        m_Body = Resources.Load<Font>("Poppins-Regular") ?? LoadFont("Poppins-Regular");
        m_BodyMed = Resources.Load<Font>("Poppins-Medium") ?? LoadFont("Poppins-Medium");

        BuildCanvas();
        ShowScanning();
    }

    static Font LoadFont(string name)
    {
        // Fonts live in Assets/Fonts and are copied to Resources by the
        // build step; this fallback keeps the UI legible if that is missed
        // rather than rendering nothing.
```

```
    var font = Resources.Load<Font>("Fonts/" + name);
    if (font != null) return font;
    Debug.LogWarning($"[MuseumUI] font '{name}' not found, using the built-in face.");
    return Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
}

/// <summary>The floor in session metres, so cards can sit at reading height.</summary>
public float SessionFloorY
{
    get => StationBubble.SessionFloorY;
    set => StationBubble.SessionFloorY = value;
}

/// <summary>Models standing in the room, keyed by station, so they can be moved and cleared.</summary>
readonly Dictionary<string, Transform> m_Models = new Dictionary<string, Transform>();

/// <summary>
/// Puts a station's 3D object in the room, if it has one.
///
/// Placed at the station's own position and turned by its saved yaw,
/// which is the same pair of numbers the card uses - so an object and
/// the card about it can never end up describing different places.
///
/// A missing model is not an error and draws nothing. Naming a model
/// that has not been added yet is the normal state of a museum being
/// built, and a magenta error cube in the middle of a church is a worse
/// answer than an empty space.
/// </summary>
void SpawnModel(MuseumStation station, Vector3 world, Quaternion facing)
{
    if (string.IsNullOrEmpty(station.model)) return;

    var prefab = Resources.Load<GameObject>("Models/" + station.model);
    if (prefab == null)
    {
        Debug.LogWarning($"[MuseumUI] no model 'Models/{station.model}' for {station.stationId} " +
                         "- it must be under Assets/Resources/Models to load by name in a build.");
        return;
    }

    // The offset is in the station's own frame, so it has to be turned
    // by the same facing the model gets - otherwise "0.7 m to the side"
    // means a different side depending on which way the room is.
    var go = Instantiate(prefab, world + facing * station.modelOffset, facing);
    go.name = $"Model_{station.stationId}";
    var scale = station.modelScale <= 0f ? 1f : station.modelScale;
    go.transform.localScale = prefab.transform.localScale * scale;

    // Nothing in the room should block a tap meant for a card.
    foreach (var col in go.GetComponentsInChildren<Collider>())
        Destroy(col);

    m_Models[station.stationId] = go.transform;
}

public void Bind(MuseumStation[] stations, Pose markerOrigin, System.Func<Vector3, Vector3> toMarkerAxes)
{
    m_Stations = stations;
    m_World.Clear();
```

```
    foreach (var b in m_Bubbles) if (b != null) Destroy(b.gameObject);
    m_Bubbles.Clear();
    foreach (var m in m_Models.Values) if (m != null) Destroy(m.gameObject);
    m_Models.Clear();
    if (m_MarkerLogo != null) { Destroy(m_MarkerLogo); m_MarkerLogo = null; }

    SpawnMarkerLogo(markerOrigin);

    foreach (var s in stations)
    {
        var world = markerOrigin.position + markerOrigin.rotation * toMarkerAxes(s.position);
        m_World[s.stationId] = world;

        // The card that hangs on the object itself, from screens 2 and
        // 5 of the design. The bottom sheet is the reading view; this
        // is what tells you there is something here at all.
        m_Bubbles.Add(StationBubble.Create(s, world, m_Heading, m_Body, m_Camera));

        SpawnModel(s, world, Quaternion.Euler(0f, markerOrigin.rotation.eulerAngles.y + s.yawDegrees, 0f));
    }
    m_Located = true;
    m_LastStations = stations;
    m_LastOrigin = markerOrigin;
    m_LastAxes = toMarkerAxes;
    ShowScanning();
}

GameObject m_MarkerLogo;

/// <summary>
/// The SanctiWalk sign, standing on the printed marker.
///
/// The marker is where the walk begins and, until the visitor turns
/// round, the only thing on screen - so it carries the branding rather
/// than leaving them looking at a sheet of paper with a bare caption
/// under it.
///
/// Placed at the marker's own pose and pushed a little way out along
/// its normal, so it stands off the wall instead of z-fighting with it,
/// and dropped by half its height so it centres on the sheet rather
/// than standing on top of it - the model is grounded, with its pivot
/// at its base.
/// </summary>
/// <summary>Where the printed sheet is in map space, and which way it faces.</summary>
Vector3 m_SignInMap;
Vector3 m_SignNormal;
bool m_SignKnown;

/// <summary>
/// Tells the sign where the printed sheet actually is.
///
/// Called from the marker route, which is the only place that knows
/// which parish's marker was recognised.
/// </summary>
public void SetMarkerSign(Vector3 inMap, Vector3 normal)
{
    m_SignInMap = inMap;
    m_SignNormal = normal;
    m_SignKnown = true;
```

```
}

void SpawnMarkerLogo(Pose markerOrigin)
{
    if (!m_SignKnown) return;

    var prefab = Resources.Load<GameObject>("Models/sanctiwalk-logo");
    if (prefab == null)
    {
        Debug.LogWarning("[MuseumUI] no model 'Models/sanctiwalk-logo' - the marker " +
                         "shows no sign. It must be under Assets/Resources/Models.");
        return;
    }

    // The printed sheet, in the session.
    //
    // markerOrigin is the pose of MAP SPACE, not of the marker: its
    // position is where map (0,0,0) lands. The sign was being put
    // there, which is wherever the scan's origin happens to be - meters
    // from the sheet and facing whichever way map +Z points. That is
    // what made it look laid over and in the wrong place.
    var signPos = markerOrigin.position + markerOrigin.rotation * m_SignInMap;
    var outward = markerOrigin.rotation * m_SignNormal;
    outward.y = 0f;
    if (outward.sqrMagnitude < 1e-6f) outward = markerOrigin.rotation * Vector3.forward;
    outward.Normalize();

    // Upright and square to the wall: the model's own +Y is up and its
    // face looks down +Z, so pointing +Z out of the wall with world up
    // as the up vector stands it on end facing the reader.
    var facing = Quaternion.LookRotation(outward, Vector3.up);

    // Sized to cover the sheet rather than to any fixed number. The
    // printed marker is 0.18 x 0.24 m; the sign is 3.5 times wider than
    // it is tall, so matching HEIGHT is what hides the sheet - matching
    // its width would leave a sign 5 cm tall that nobody can read.
    const float sheetHeight = 0.24f;
    const float coverage = 1.25f;          // a little proud of the sheet's edges
    var modelHeight = prefab.GetComponentInChildren<MeshFilter>() != null
        ? Mathf.Max(0.01f, prefab.GetComponentInChildren<MeshFilter>().sharedMesh.bounds.size.y)
        : 0.543f;
    var scale = sheetHeight * coverage / modelHeight;

    // Centred on the sheet and a little in front of the wall, so it
    // covers the marker instead of z-fighting with it. The model is
    // grounded - pivot at its base - so half its scaled height brings
    // its middle onto the sheet's middle.
    var pos = signPos + outward * 0.03f - Vector3.up * (modelHeight * scale * 0.5f);

    m_MarkerLogo = Instantiate(prefab, pos, facing);
    m_MarkerLogo.name = "MarkerLogo";
    m_MarkerLogo.transform.localScale = prefab.transform.localScale * scale;
    foreach (var col in m_MarkerLogo.GetComponentsInChildren<Collider>())
        Destroy(col);

    StartCoroutine(FadeInSign(m_MarkerLogo));
}

/// <summary>
```

```
/// Flashes the sign in: a bright burst that settles into the logo.
///
/// This is the moment the scan lands, and it used to be marked by
/// three coloured survey bars across the room. The sign is what
/// should mark it instead, so it does not merely appear - it over-
/// brightens and overshoots its size for a beat, then settles. On a
/// dim camera feed a plain fade is easy to miss, and missing it is
/// missing the one confirmation the visitor gets that the church was
/// recognised.
///
/// The marker lock is abrupt - one frame there is nothing, the next the
/// whole thing is there - and a sign that appears that way reads as a
/// glitch. Materials are instanced per spawn (the renderer's `material`,
/// not `sharedMaterial`) so the fade cannot leak into the imported
/// asset and leave it half transparent for every later spawn.
/// </summary>
System.Collections.IEnumerator FadeInSign(GameObject sign)
{
    var renderers = sign.GetComponentsInChildren<Renderer>();
    var mats = new System.Collections.Generic.List<Material>();
    foreach (var r in renderers)
        foreach (var m in r.materials)
        {
            // Built-in Standard shader, transparent setup. Left in this
            // mode once opaque: switching back mid-frame is a second
            // chance to get it wrong for no visible gain.
            m.SetFloat("_Mode", 2f);
            m.SetInt("_SrcBlend", (int)UnityEngine.Rendering.BlendMode.SrcAlpha);
            m.SetInt("_DstBlend", (int)UnityEngine.Rendering.BlendMode.OneMinusSrcAlpha);
            m.SetInt("_ZWrite", 0);
            m.DisableKeyword("_ALPHATEST_ON");
            m.EnableKeyword("_ALPHABLEND_ON");
            m.DisableKeyword("_ALPHAPREMULTIPLY_ON");
            m.renderQueue = 3000;
            mats.Add(m);
        }
    if (mats.Count == 0) yield break;

    var settled = sign.transform.localScale;

    const float seconds = 0.9f;
    for (var t = 0f; t < seconds; t += Time.deltaTime)
    {
        if (sign == null) yield break;
        var k = Mathf.Clamp01(t / seconds);
        var a = Mathf.SmoothStep(0f, 1f, k);

        // Over-bright at the start, falling to the texture's own
        // colour. White multiplies to the texture, so 1 is normal and
        // anything above it blows the sign out for the instant the
        // scan lands.
        var burst = Mathf.Lerp(2.6f, 1f, Mathf.SmoothStep(0f, 1f, Mathf.Clamp01(k * 1.6f)));

        foreach (var m in mats)
        {
            var c = m.color;
            c.r = c.g = c.b = burst;
            c.a = a;
            m.color = c;
```

```
        }

        // Overshoots to 1.12x and eases back, so it arrives rather
        // than switches on.
        var pop = 1f + 0.12f * Mathf.Sin(Mathf.PI * Mathf.Clamp01(k * 1.35f));
        sign.transform.localScale = settled * pop;

        yield return null;
    }

    if (sign == null) yield break;
    sign.transform.localScale = settled;
    foreach (var m in mats)
        m.color = new Color(1f, 1f, 1f, 1f);
}

MuseumStation[] m_LastStations;
Pose m_LastOrigin;
System.Func<Vector3, Vector3> m_LastAxes;

/// <summary>
/// Rebuilds the floating cards in the newly chosen language.
///
/// The cards are drawn once, with their height measured from the text
/// they contain, so there is nothing to re-point at a different string
/// - a Tagalog paragraph and its English translation are not the same
/// number of lines. Rebuilding from the stored pose is cheaper to get
/// right than teaching every label to re-measure itself.
/// </summary>
public void RebuildForLanguage()
{
    if (!m_Located || m_LastStations == null) return;
    Bind(m_LastStations, m_LastOrigin, m_LastAxes);
}

/// <summary>
/// Moves the stations that are already on screen, without rebuilding
/// them.
///
/// Bind destroys every bubble and makes new ones, which is right when
/// the origin changes and wrong sixty times a second while a finger is
/// dragging one: the card being aligned would be destroyed and recreated
/// under the finger every frame, which flickers and throws away a
/// GameObject per station per frame. This just puts them somewhere else.
/// </summary>
public void Reposition(MuseumStation[] stations, Pose origin, System.Func<Vector3, Vector3> toMarkerAxes)
{
    if (!m_Located) return;

    foreach (var s in stations)
    {
        var world = origin.position + origin.rotation * toMarkerAxes(s.position);
        m_World[s.stationId] = world;
    }

    foreach (var bubble in m_Bubbles)
    {
        if (bubble == null) continue;
        if (m_World.TryGetValue(bubble.StationId, out var world))
```

```
            bubble.MoveTo(world);
    }

    // The models follow, or an object being calibrated would stay put
    // while its card slid away from it.
    foreach (var pair in m_Models)
    {
        if (pair.Value == null || !m_World.TryGetValue(pair.Key, out var world))
            continue;
        var station = Find(pair.Key);
        pair.Value.position = station != null
            ? world + pair.Value.rotation * station.modelOffset
            : world;
    }
}

void BuildCanvas()
{
    var canvasGo = new GameObject("MuseumCanvas",
        typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false);

    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;

    var scaler = canvasGo.GetComponent<CanvasScaler>();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(390, 844);   // the design's frame
    scaler.matchWidthOrHeight = 0.5f;

    if (FindAnyObjectByType<UnityEngine.EventSystems.EventSystem>() == null)
    {
        new GameObject("EventSystem",
            typeof(UnityEngine.EventSystems.EventSystem),
            typeof(UnityEngine.EventSystems.StandaloneInputModule));
    }

    var root = (RectTransform)canvasGo.transform;
    m_Root = root;
    BuildScanPanel(root);
    // BuildBubblePanel is gone.
    //
    // It drew a card at the bottom of the screen for whichever station
    // you were standing nearest - the same title, the same thumbnail,
    // the same line of text as the card already floating on the object
    // itself. Two copies of one station, the lower one landing under the
    // AR SETUP button and the bottom bar, three things stacked in the
    // same strip of screen.
    //
    // The floating card is the one worth keeping: it is attached to the
    // thing it describes, which is the entire point of doing this in AR.
    // Tapping it opens the full panel directly now, with nothing in
    // between.
    BuildCardPanel(root);
    // BuildNextPill is gone too.
    //
    // It floated a "Next Stop - <name> - <n> m" bar across the top of
    // the camera, over whatever the visitor had raised the phone to
    // look at. The bottom bar already carries the same station and the
```

```
    // same distance, so this was a second copy of it in the worst
    // possible place: across the middle of the view.
    //
    // Nothing about the tour changes - the next station is still
    // chained, still walked in order, and still shown. Only this
    // duplicate is no longer drawn over the room.
    BuildPreviewButton(root);
}

// ------------------------------------------------------------ 1. scanning
void BuildScanPanel(RectTransform parent)
{
    m_ScanPanel = Panel(parent, "ScanPanel");
    var rt = (RectTransform)m_ScanPanel.transform;
    Stretch(rt);

    // The corner brackets of the design's reticle, drawn as four L
    // shapes so the middle stays clear of the thing being scanned.
    var size = 150f;
    var arm = 34f;
    var thick = 3f;
    foreach (var (cx, cy) in new[] { (-1, 1), (1, 1), (-1, -1), (1, -1) })
    {
        var ox = cx * size / 2f;
        var oy = cy * size / 2f + 40f;
        Bar(rt, new Vector2(ox - cx * arm / 2f, oy), new Vector2(arm, thick), Color.white);
        Bar(rt, new Vector2(ox, oy - cy * arm / 2f), new Vector2(thick, arm), Color.white);
    }

    var caption = Label(rt, "Point your camera at a marker or\nartwork to explore its story.",
        m_Body, 13, Color.white, TextAnchor.MiddleCenter);
    var crt = (RectTransform)caption.transform;
    crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 0.5f);
    crt.anchoredPosition = new Vector2(0, -80);
    crt.sizeDelta = new Vector2(300, 48);
    Shadow(caption.gameObject);

    // Until the marker is printed and mounted, nothing past this screen
    // can be reached, so none of the cards can be checked or shown to
    // anyone. This walks them with no AR at all. It says what it is
    // rather than pretending a station has been found, and it is gone
    // the moment a real localization happens.
}

void BuildPreviewButton(RectTransform parent)
{
    m_PreviewButton = Button(parent, "Preview the cards (no AR)", new Vector2(240, 38), Sand, Ink);
    var prt = (RectTransform)m_PreviewButton.transform;
    // Top of the screen, not the bottom: at the bottom it landed on top
    // of the very card it opens, covering the subtitle and half the
    // description.
    prt.anchorMin = prt.anchorMax = new Vector2(0.5f, 1f);
    prt.pivot = new Vector2(0.5f, 1f);
    prt.anchoredPosition = new Vector2(0, -150);
    m_PreviewButton.GetComponent<Button>().onClick.AddListener(PreviewNext);

    // The real thing, minus the camera: runs a scan through the same
    // SetOrigin the tracked-image handler calls, so the stations are
    // placed, the bubbles spawn and the trigger radii arm exactly as
```

```
    // they will on the phone.
    m_SimulateButton = Button(parent, "Simulate Scan", new Vector2(240, 38), Navy, Color.white);
    var srt = (RectTransform)m_SimulateButton.transform;
    srt.anchorMin = srt.anchorMax = new Vector2(0.5f, 1f);
    srt.pivot = new Vector2(0.5f, 1f);
    srt.anchoredPosition = new Vector2(0, -104);
    m_SimulateButton.GetComponent<Button>().onClick.AddListener(() =>
    {
        var origin = GetComponent<ChurchOriginTest>();
        if (origin != null) origin.SimulateScan();
    });
}

GameObject m_PreviewButton;
GameObject m_SimulateButton;
int m_PreviewIndex = -1;

/// <summary>Steps through the stations with no marker and no tracking.</summary>
void PreviewNext()
{
    m_Stations = m_Stations ?? MuseumStations.Sala;
    m_PreviewIndex = (m_PreviewIndex + 1) % m_Stations.Length;
    ShowBubble(m_Stations[m_PreviewIndex]);
}

// ------------------------------------------------------------ 2. bubble
void BuildBubblePanel(RectTransform parent)
{
    m_BubblePanel = Panel(parent, "BubblePanel");
    var rt = (RectTransform)m_BubblePanel.transform;
    Stretch(rt);

    var card = Card(rt, new Vector2(354, 196));
    var crt = (RectTransform)card.transform;
    crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 0f);
    crt.pivot = new Vector2(0.5f, 0f);
    crt.anchoredPosition = new Vector2(0, 26);

    m_BubbleThumb = Thumb(crt, new Vector2(PadX, -PadX), 54);

    m_BubbleTitle = Label(crt, "", m_Heading, 16, Ink, TextAnchor.UpperLeft);
    Place(m_BubbleTitle, new Vector2(PadX + 66, -PadX - 1), new Vector2(252, 42));

    m_BubbleSub = Label(crt, "", m_Body, 11, Muted, TextAnchor.UpperLeft);
    Place(m_BubbleSub, new Vector2(PadX + 66, -PadX - 44), new Vector2(252, 16));

    m_BubbleBody = Label(crt, "", m_Body, 12, Ink, TextAnchor.UpperLeft);
    Place(m_BubbleBody, new Vector2(PadX, -86), new Vector2(318, 54));

    var btn = Button(crt, "View details", new Vector2(318, 42), Navy, Color.white);
    var brt = (RectTransform)btn.transform;
    brt.anchorMin = brt.anchorMax = new Vector2(0f, 0f);
    brt.pivot = new Vector2(0f, 0f);
    brt.anchoredPosition = new Vector2(PadX, PadX);
    btn.GetComponent<Button>().onClick.AddListener(ShowCard);

    var arrow = Label((RectTransform)btn.transform, "→", m_BodyMed, 16, Color.white, TextAnchor.MiddleRight);
    var art = (RectTransform)arrow.transform;
    Stretch(art);
```

```
    art.offsetMin = new Vector2(0, 0);
    art.offsetMax = new Vector2(-16, 0);
}

// ------------------------------------------------------------ 3. card
void BuildCardPanel(RectTransform parent)
{
    m_CardPanel = Panel(parent, "CardPanel");
    var rt = (RectTransform)m_CardPanel.transform;
    Stretch(rt);

    var scrim = Bar(rt, Vector2.zero, Vector2.zero, Scrim);
    Stretch((RectTransform)scrim.transform);
    scrim.transform.SetAsFirstSibling();

    var card = Card(rt, new Vector2(354, 430));
    var crt = (RectTransform)card.transform;
    crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 0f);
    crt.pivot = new Vector2(0.5f, 0f);
    crt.anchoredPosition = new Vector2(0, 26);

    m_CardTitle = Label(crt, "", m_Heading, 19, Ink, TextAnchor.UpperLeft);
    Place(m_CardTitle, new Vector2(PadX, -20), new Vector2(318, 52));

    m_CardSub = Label(crt, "", m_Body, 11, Muted, TextAnchor.UpperLeft);
    Place(m_CardSub, new Vector2(PadX, -70), new Vector2(318, 16));

    m_CardBody = Label(crt, "", m_Body, 12.5f, Ink, TextAnchor.UpperLeft);
    Place(m_CardBody, new Vector2(PadX, -96), new Vector2(318, 150));

    var meta = new GameObject("Meta", typeof(RectTransform));
    meta.transform.SetParent(crt, false);
    m_CardMetaHolder = (RectTransform)meta.transform;
    m_CardMetaHolder.anchorMin = new Vector2(0, 1);
    m_CardMetaHolder.anchorMax = new Vector2(0, 1);
    m_CardMetaHolder.pivot = new Vector2(0, 1);
    m_CardMetaHolder.anchoredPosition = new Vector2(PadX, -252);
    m_CardMetaHolder.sizeDelta = new Vector2(318, 110);

    // The panel view.
    //
    // These stations are scanned pages from the parish's souvenir
    // programme: each one already carries its own title, its paragraph
    // and its captioned photograph. Laying a title and a description
    // around it would print the same words twice and shrink the page
    // until the Tagalog body text could not be read - so when a station
    // is a panel, the card IS the panel and everything else gets out of
    // the way.
    //
    // Sized to the screen rather than to the card, because the page is
    // 1400px wide and the phone is 1080: shown full width it lands near
    // its native resolution and stays legible. Inside the 354pt card it
    // would be less than a third of that.
    m_PanelImage = new GameObject("Panel", typeof(RawImage)).GetComponent<RawImage>();
    m_PanelImage.transform.SetParent(rt, false);
    m_PanelImage.raycastTarget = false;
    var prt = (RectTransform)m_PanelImage.transform;
    prt.anchorMin = prt.anchorMax = new Vector2(0.5f, 0.5f);
    prt.anchoredPosition = Vector2.zero;
```

```
    m_PanelImage.gameObject.SetActive(false);

    var close = Button(crt, "Close", new Vector2(318, 44), Sand, Ink);
    var clrt = (RectTransform)close.transform;
    clrt.anchorMin = clrt.anchorMax = new Vector2(0f, 0f);
    clrt.pivot = new Vector2(0f, 0f);
    clrt.anchoredPosition = new Vector2(PadX, PadX);
    close.GetComponent<Button>().onClick.AddListener(CloseCard);
}

// ------------------------------------------------------------ 4. next stop
void BuildNextPill(RectTransform parent)
{
    m_NextPill = Panel(parent, "NextPill");
    var rt = (RectTransform)m_NextPill.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 1f);
    rt.pivot = new Vector2(0.5f, 1f);
    rt.anchoredPosition = new Vector2(0, -74);
    rt.sizeDelta = new Vector2(214, 52);

    var bg = rt.gameObject.AddComponent<Image>();
    bg.color = Navy;
    Round(bg);

    m_NextText = Label(rt, "", m_BodyMed, 12, Color.white, TextAnchor.MiddleCenter);
    Stretch((RectTransform)m_NextText.transform);
}

// ------------------------------------------------------------------ states
/// <summary>
/// Hide or show the whole museum UI.
///
/// While the room is still being scanned the screen belongs to the
/// readiness checklist, and this UI's own scan panel was drawn straight
/// through it - "Simulate Scan" and "Preview the cards" sitting on top
/// of the checklist text, both unreadable. Neither is wrong to exist;
/// they just cannot both own the screen at once.
/// </summary>
public void SetVisible(bool visible)
{
    if (m_Root != null)
        m_Root.gameObject.SetActive(visible);
}

void ShowScanning()
{
    m_Phase = Phase.Scanning;
    m_Current = null;
    if (m_ScanPanel) m_ScanPanel.SetActive(!m_Located);
    if (m_BubblePanel) m_BubblePanel.SetActive(false);
    if (m_CardPanel) m_CardPanel.SetActive(false);
    if (m_NextPill) m_NextPill.SetActive(false);
    if (m_PreviewButton) m_PreviewButton.SetActive(!m_Located);
    if (m_SimulateButton) m_SimulateButton.SetActive(!m_Located);
}

/// <summary>
/// Opens a station's card directly, whichever one is asked for.
///
```

```
/// The bottom bar needs this: History opens whatever the visitor is
/// standing nearest, and Stations steps through them. Both are choices
/// made outside this component, so the proximity rule in Update cannot
/// express them - it always picks the nearest, which is the right
/// default and the wrong answer when someone has asked for a specific
/// one.
/// </summary>
public void OpenCardFor(MuseumStation station)
{
    if (station == null) return;
    ShowBubble(station);
    ShowCard();
}

/// <summary>
/// Clips a caption to the bubble's three visible lines, breaking on a
/// word so the cut does not land mid-syllable.
/// </summary>
const int k_TeaserChars = 110;

static string Teaser(string text)
{
    if (string.IsNullOrEmpty(text) || text.Length <= k_TeaserChars) return text;
    var cut = text.LastIndexOf(' ', Mathf.Min(k_TeaserChars, text.Length - 1));
    if (cut < k_TeaserChars / 2) cut = k_TeaserChars;
    return text.Substring(0, cut).TrimEnd(',', '.', ';', ' ') + "…";
}

void ShowBubble(MuseumStation station)
{
    m_Phase = Phase.Bubble;
    m_Current = station;

    m_BubbleTitle.text = station.Name;
    m_BubbleSub.text = station.subtitle;
    // The bubble body is a fixed 318x54 box: roughly three lines at 12pt.
    // A caption written as a paragraph - and some are, the placer accepts
    // any length - used to overflow it and cover the "View details"
    // button underneath, so the station became unreadable AND unopenable.
    // The teaser is clipped to what the box can hold; the full text is on
    // the card that button opens, so nothing is lost.
    m_BubbleBody.text = Teaser(station.Short);

    // No artwork, no frame. This used to keep the 54 px thumbnail in
    // place and just paint it Sand, which put a blank beige square
    // beside the title of every station whose picture has not been
    // taken yet - and that is most of them. An empty box reads as a
    // picture that failed to load, so the box goes and the title and
    // subtitle take the column back.
    var art = string.IsNullOrEmpty(station.image) ? null : Resources.Load<Texture2D>(station.image);
    var hasArt = art != null;

    m_BubbleThumb.gameObject.SetActive(hasArt);
    if (hasArt)
    {
        m_BubbleThumb.texture = art;
        m_BubbleThumb.color = Color.white;
    }
```

```
    var indent = hasArt ? PadX + 66f : PadX;
    var headWidth = hasArt ? 252f : 318f;
    Place(m_BubbleTitle, new Vector2(indent, -PadX - 1), new Vector2(headWidth, 42));
    Place(m_BubbleSub, new Vector2(indent, -PadX - 44), new Vector2(headWidth, 16));

    m_ScanPanel.SetActive(false);
    // The world card now carries the title and the whole history, so an
    // on-screen copy of the same thing is a second card covering the
    // first. Admins keep it: it is how a station being placed is
    // identified without reading the room.
    if (m_BubblePanel) m_BubblePanel.SetActive(AdminAccess.IsAdmin);
    m_CardPanel.SetActive(false);
    if (m_PreviewButton) m_PreviewButton.SetActive(!m_Located);
    if (m_SimulateButton) m_SimulateButton.SetActive(!m_Located);
    UpdateNextPill();
}

/// <summary>The scanned page, when a station is one.</summary>
RawImage m_PanelImage;

/// <summary>
/// A station whose whole content is a single scanned page.
///
/// Recognised by having artwork and no written description, rather than
/// by a flag: that is exactly the condition under which a title and a
/// body would have nothing to say that the image does not already say.
/// </summary>
static bool IsPanel(MuseumStation station) =>
    !string.IsNullOrEmpty(station.image) &&
    string.IsNullOrEmpty(station.Detailed);

void ShowCard()
{
    if (m_Current == null) return;
    m_Phase = Phase.Card;

    if (IsPanel(m_Current))
    {
        ShowPanelCard(m_Current);
        return;
    }

    m_PanelImage.gameObject.SetActive(false);
    m_CardTitle.gameObject.SetActive(true);
    m_CardSub.gameObject.SetActive(true);
    m_CardBody.gameObject.SetActive(true);
    m_CardMetaHolder.gameObject.SetActive(true);

    m_CardTitle.text = m_Current.Name;
    m_CardSub.text = m_Current.subtitle;
    m_CardBody.text = m_Current.Detailed;

    for (var i = m_CardMetaHolder.childCount - 1; i >= 0; i--)
        Destroy(m_CardMetaHolder.GetChild(i).gameObject);

    var row = 0;
    row = MetaRow(row, "Year Installed", m_Current.yearInstalled);
    row = MetaRow(row, "Artist", m_Current.artist);
    MetaRow(row, "Material", m_Current.material);
```

```
    FitCardToContent(row);

    if (m_BubblePanel) m_BubblePanel.SetActive(false);
    m_CardPanel.SetActive(true);
    if (m_NextPill) m_NextPill.SetActive(false);
    if (m_PreviewButton) m_PreviewButton.SetActive(false);
    if (m_SimulateButton) m_SimulateButton.SetActive(false);
}

/// <summary>The page, as large as the screen allows, and nothing else.</summary>
void ShowPanelCard(MuseumStation station)
{
    var art = Resources.Load<Texture2D>(station.image);
    if (art == null)
    {
        // Fall back to the ordinary card rather than showing a blank
        // screen: the station still has a name worth reading.
        m_PanelImage.gameObject.SetActive(false);
        m_CardTitle.text = station.Name;
        m_CardSub.text = station.subtitle;
        m_CardBody.text = "This panel's artwork is missing from the build.";
        m_CardPanel.SetActive(true);
        if (m_BubblePanel) m_BubblePanel.SetActive(false);
        if (m_NextPill) m_NextPill.SetActive(false);
        return;
    }

    m_PanelImage.texture = art;
    m_PanelImage.gameObject.SetActive(true);

    // Fit inside the screen with a margin, keeping the page's own shape.
    var rt = (RectTransform)m_PanelImage.transform;
    var maxW = m_Root.rect.width - 24f;
    var maxH = m_Root.rect.height - 150f;     // room for Close underneath
    var scale = Mathf.Min(maxW / art.width, maxH / art.height);
    rt.sizeDelta = new Vector2(art.width * scale, art.height * scale);
    rt.anchoredPosition = new Vector2(0f, 26f);

    // The card behind it carries only the Close button now.
    m_CardTitle.gameObject.SetActive(false);
    m_CardSub.gameObject.SetActive(false);
    m_CardBody.gameObject.SetActive(false);
    m_CardMetaHolder.gameObject.SetActive(false);

    var card = (RectTransform)m_CardTitle.transform.parent;
    card.sizeDelta = new Vector2(354, 84);

    if (m_BubblePanel) m_BubblePanel.SetActive(false);
    m_CardPanel.SetActive(true);
    if (m_NextPill) m_NextPill.SetActive(false);
    if (m_PreviewButton) m_PreviewButton.SetActive(false);
    if (m_SimulateButton) m_SimulateButton.SetActive(false);
}

int MetaRow(int index, string label, string value)
{
    // A blank row is left out rather than printed as "Unknown". The
    // parish has not researched every object, and saying so by omission
```

```
    // is more honest than filling the card with placeholders.
    if (string.IsNullOrEmpty(value)) return index;

    var y = -index * 34f;
    var l = Label(m_CardMetaHolder, label, m_Body, 11, Muted, TextAnchor.UpperLeft);
    Place(l, new Vector2(0, y), new Vector2(150, 16));
    var v = Label(m_CardMetaHolder, value, m_BodyMed, 12, Ink, TextAnchor.UpperLeft);
    Place(v, new Vector2(150, y), new Vector2(168, 16));
    return index + 1;
}

/// <summary>
/// Grows or shrinks the card to whatever this station actually has to
/// say, so a stop with no metadata does not leave a blank half-card.
/// </summary>
void FitCardToContent(int metaRows)
{
    var card = (RectTransform)m_CardTitle.transform.parent;
    var body = (RectTransform)m_CardBody.transform;

    var bodyHeight = Mathf.Max(40f, m_CardBody.preferredHeight);
    body.sizeDelta = new Vector2(body.sizeDelta.x, bodyHeight);

    var metaTop = 96f + bodyHeight + 18f;
    m_CardMetaHolder.anchoredPosition = new Vector2(PadX, -metaTop);

    var height = metaTop + metaRows * 34f + 20f + 44f + PadX;
    card.sizeDelta = new Vector2(card.sizeDelta.x, Mathf.Min(height, 620f));
}

/// <summary>
/// A tap on a world bubble opens that station's description, so you can
/// read a stop you can see across the room without walking into its
/// trigger radius first.
/// </summary>
void HandleBubbleTap()
{
    // Station cards carry no collider any more, so this raycast can
    // never hit one. Kept because the admin screens still place and
    // select stations, and returning early is cheaper than a raycast
    // against a scene full of scanned geometry on every touch.
    if (!AdminAccess.IsAdmin) return;
    if (Input.touchCount == 0) return;
    var touch = Input.GetTouch(0);
    if (touch.phase != TouchPhase.Began) return;

    if (!Physics.Raycast(m_Camera.ScreenPointToRay(touch.position), out var hit, 30f)) return;

    var bubble = hit.collider.GetComponentInParent<StationBubble>();
    if (bubble == null) return;

    // Straight to the panel. ShowBubble used to be the step in between,
    // and with nothing left for it to show it would only set a phase
    // that ShowCard immediately replaces.
    m_Current = bubble.Station;
    ShowCard();
}

void CloseCard()
```

```
{
    if (m_Current != null) m_Visited.Add(m_Current.stationId);
    m_CardPanel.SetActive(false);
    if (m_BubblePanel) m_BubblePanel.SetActive(true);
    m_Phase = Phase.Bubble;
    UpdateNextPill();
}

void UpdateNextPill()
{
    // Nothing to update while the pill is not built. The station it
    // would have named is still chosen, and the bottom bar still shows
    // it - this only stops the write to a label that is not there.
    if (m_NextPill == null || m_NextText == null) return;

    if (m_Current == null || string.IsNullOrEmpty(m_Current.nextStationId))
    {
        if (m_NextPill) m_NextPill.SetActive(false);
        return;
    }
    var next = Find(m_Current.nextStationId);
    if (next == null || !m_World.TryGetValue(next.stationId, out var world))
    {
        if (m_NextPill) m_NextPill.SetActive(false);
        return;
    }
    var metres = FloorDistance(m_Camera.transform.position, world);
    m_NextText.text = $"Next Stop   ·   {next.Name}   ·   {metres:0} m";
    if (m_NextPill) m_NextPill.SetActive(true);
}

MuseumStation Find(string id)
{
    if (m_Stations == null) return null;
    foreach (var s in m_Stations)
        if (s.stationId == id) return s;
    return null;
}

/// <summary>
/// How much closer a rival station must be to take the card over.
/// Kept the same as ChurchOriginTest's, so the card and the voice
/// never disagree about which station the visitor is on.
/// </summary>
const float k_StickyMarginMetres = 0.75f;

// ------------------------------------------------------------------ tick
void Update()
{
    if (!m_Located || m_Stations == null || m_Camera == null) return;

    HandleBubbleTap();
    if (m_Phase == Phase.Card) return;

    // Nearest station inside its own radius wins, so standing between
    // two of them shows the one you are actually closest to rather than
    // whichever happens to come first in the array.
    MuseumStation best = null;
    var bestDistance = float.MaxValue;
```

```
    var here = m_Camera.transform.position;

    foreach (var s in m_Stations)
    {
        if (!m_World.TryGetValue(s.stationId, out var world)) continue;
        var d = FloorDistance(here, world);
        if (d <= s.triggerRadius && d < bestDistance)
        {
            best = s;
            bestDistance = d;
        }
    }

    if (best == null)
    {
        if (m_Phase == Phase.Bubble) ShowScanning();
        if (m_Located && m_ScanPanel.activeSelf) m_ScanPanel.SetActive(false);
        return;
    }

    // The card keeps the station it is on until another is clearly
    // closer, for the reason set out on k_StickyMarginMetres in
    // ChurchOriginTest: where stations overlap, nearest-wins changes
    // its mind on tracking jitter alone and the bubble flickers
    // between two titles while the visitor stands still.
    if (best != m_Current && m_Current != null
        && m_World.TryGetValue(m_Current.stationId, out var currentWorld))
    {
        var heldDistance = FloorDistance(here, currentWorld);
        var stillHere = heldDistance <= m_Current.triggerRadius + k_StickyMarginMetres;
        if (stillHere && bestDistance > heldDistance - k_StickyMarginMetres)
        {
            UpdateNextPill();
            return;
        }
    }

    if (best != m_Current) ShowBubble(best);
    else UpdateNextPill();
}

// ------------------------------------------------------------ plumbing
GameObject Panel(RectTransform parent, string name)
{
    var go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    return go;
}

static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero;
    rt.offsetMax = Vector2.zero;
}

static void Place(Graphic g, Vector2 topLeft, Vector2 size)
{
```

```
    var rt = (RectTransform)g.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0, 1);
    rt.pivot = new Vector2(0, 1);
    rt.anchoredPosition = topLeft;
    rt.sizeDelta = size;
}

GameObject Card(RectTransform parent, Vector2 size)
{
    var go = new GameObject("Card", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    ((RectTransform)go.transform).sizeDelta = size;
    var img = go.GetComponent<Image>();
    img.color = CardWhite;
    Round(img);
    Shadow(go);
    return go;
}

GameObject Bar(RectTransform parent, Vector2 pos, Vector2 size, Color colour)
{
    var go = new GameObject("Bar", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    go.GetComponent<Image>().color = colour;
    return go;
}

/// <summary>
/// How far apart two points are on the floor plan, ignoring height.
///
/// A station marks where a visitor should be STANDING, and whether they
/// are standing there is a question about the floor. Measuring through
/// the air instead charges the radius for the difference between the
/// height the station was recorded at and the height the next person
/// holds their phone - a head taller and a 2 m radius is really 1.97,
/// crouching to look at something and it shrinks further. It is the same
/// reason LogCheckpoint takes only the horizontal error.
/// </summary>
static float FloorDistance(Vector3 a, Vector3 b)
{
    a.y = 0f;
    b.y = 0f;
    return Vector3.Distance(a, b);
}

RawImage Thumb(RectTransform parent, Vector2 topLeft, float size)
{
    var go = new GameObject("Thumb", typeof(RectTransform), typeof(RawImage));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0, 1);
    rt.pivot = new Vector2(0, 1);
    rt.anchoredPosition = topLeft;
    rt.sizeDelta = new Vector2(size, size);
    var raw = go.GetComponent<RawImage>();
    raw.color = Sand;
```

```
    return raw;
}

Text Label(RectTransform parent, string text, Font font, float size, Color colour, TextAnchor anchor)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var t = go.GetComponent<Text>();
    t.text = text;
    t.font = font;
    t.fontSize = Mathf.RoundToInt(size);
    t.color = colour;
    t.alignment = anchor;
    t.horizontalOverflow = HorizontalWrapMode.Wrap;
    t.verticalOverflow = VerticalWrapMode.Truncate;
    t.raycastTarget = false;
    return t;
}

GameObject Button(RectTransform parent, string caption, Vector2 size, Color fill, Color ink)
{
    var go = new GameObject("Button", typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    ((RectTransform)go.transform).sizeDelta = size;
    var img = go.GetComponent<Image>();
    img.color = fill;
    Round(img);

    var label = Label((RectTransform)go.transform, caption, m_BodyMed, 13, ink, TextAnchor.MiddleCenter);
    Stretch((RectTransform)label.transform);
    return go;
}

/// <summary>
/// Rounded corners without shipping a sprite: a 1x1 white texture on a
/// sliced Image cannot round, so this builds a small rounded-rect
/// texture once and slices it. Cheaper than an asset nobody can review.
/// </summary>
static Sprite s_Rounded;
static void Round(Image img)
{
    if (s_Rounded == null)
    {
        const int s = 48, r = 14;
        var tex = new Texture2D(s, s, TextureFormat.RGBA32, false);
        for (var y = 0; y < s; y++)
        for (var x = 0; x < s; x++)
        {
            var dx = Mathf.Max(r - x, x - (s - 1 - r), 0);
            var dy = Mathf.Max(r - y, y - (s - 1 - r), 0);
            var inside = Mathf.Sqrt(dx * dx + dy * dy) <= r;
            tex.SetPixel(x, y, inside ? Color.white : Color.clear);
        }
        tex.Apply();
        tex.wrapMode = TextureWrapMode.Clamp;
        s_Rounded = Sprite.Create(tex, new Rect(0, 0, s, s), new Vector2(0.5f, 0.5f),
            100f, 0, SpriteMeshType.FullRect, new Vector4(r, r, r, r));
    }
    img.sprite = s_Rounded;
```

```
        img.type = Image.Type.Sliced;
    }

    static void Shadow(GameObject go)
    {
        var shadow = go.AddComponent<Shadow>();
        shadow.effectColor = new Color(0, 0, 0, 0.28f);
        shadow.effectDistance = new Vector2(0, -2);
    }
}
```

4.1.13 AR-013 — OriginPicker.cs

Code ID	AR-013
File Name	OriginPicker.cs
File Path	SanctiWalkAR/Assets/Scripts/OriginPicker.cs
Language / Type	C#
Lines	154
Functions / Classes	OriginPicker (public class); Bind (public method); SetFloor (public method); Reset (public method); Tap (public method); TryGetTapThisFrame (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/OriginPicker.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — OriginPicker.cs

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Establishes the church's frame from two taps on the real floor.
///
/// Tap A is the origin - where the tour starts. Tap B is a few metres
/// further along the direction the tour runs. The line between them is the
/// heading, the floor supplies everything else, and that is the whole
/// survey. No printed sheet to hang, nothing measured against a scan,
/// nothing typed into a file.
///
/// What makes this accurate where the marker was not is that both inputs
/// are things a person can physically point at. The marker's centre was
/// not, its facing came from mesh noise, and its sideways direction was a
/// convention nobody had checked. Here the operator indicates two spots on
/// the ground and the frame follows from them.
///
/// Y is taken from the floor plane rather than from the raycast hit, so a
/// tap that catches the edge of a rug or a slightly domed bit of mesh does
/// not tilt or lift the church.
/// </summary>
public class OriginPicker : MonoBehaviour
{
    public enum Step { WaitingForA, WaitingForB, Done }

    public Step Current { get; private set; } = Step.WaitingForA;
    public Pose Origin { get; private set; }
    public bool HasOrigin { get; private set; }
```

```
/// <summary>Said to the operator: what to do, or why the last tap was refused.</summary>
public string Note { get; private set; } = "Tap the floor where the tour starts";

public System.Action<Pose> Established;

ARRaycastManager m_Raycasts;
Camera m_Camera;
readonly List<ARRaycastHit> m_Hits = new List<ARRaycastHit>();

/// <summary>
/// Roughly how far the furthest station is, for the quality estimate.
/// It only has to be the right size - it is scaling a rule of thumb,
/// not computing a result.
/// </summary>
public float FarStationMetres = 5f;

Vector3 m_PointA;
float m_FloorY;
bool m_HasFloorY;

public void Bind(Camera cam, ARRaycastManager raycasts)
{
    m_Camera = cam;
    m_Raycasts = raycasts;
}

/// <summary>The floor height the readiness gate settled on.</summary>
public void SetFloor(float floorY)
{
    m_FloorY = floorY;
    m_HasFloorY = true;
}

public void Reset()
{
    Current = Step.WaitingForA;
    HasOrigin = false;
    Note = "Tap the floor where the tour starts";
}

/// <summary>
/// A tap at a screen point. Returns true when the tap was used; false
/// when it was refused, with Note saying why.
/// </summary>
public bool Tap(Vector2 screenPoint)
{
    if (m_Raycasts == null || Current == Step.Done)
        return false;

    // PlaneWithinPolygon, not PlaneWithinInfinity: a plane's polygon is
    // the part ARCore has actually seen. Casting against the infinite
    // plane would happily return a point out in the car park.
    if (!m_Raycasts.Raycast(screenPoint, m_Hits, TrackableType.PlaneWithinPolygon) || m_Hits.Count == 0)
    {
        Note = "No floor there. Aim at the ground you can see.";
        return false;
    }
```

```
    var hit = m_Hits[0].pose.position;
    if (m_HasFloorY)
        hit.y = m_FloorY;

    if (Current == Step.WaitingForA)
    {
        m_PointA = hit;
        Current = Step.WaitingForB;
        Note = $"Now walk at least {ChurchFrame.MinBaselineMetres:F0} m along the tour " +
               "and tap the floor again. Further is better.";
        return true;
    }

    if (!ChurchFrame.IsBaselineUsable(m_PointA, hit, out var why))
    {
        Note = why;
        return false;
    }

    Origin = ChurchFrame.FromFloorAndForward(m_PointA, hit - m_PointA);
    HasOrigin = true;
    Current = Step.Done;

    var apart = new Vector2(hit.x - m_PointA.x, hit.z - m_PointA.z).magnitude;

    // Say what it bought. A baseline is the one thing the operator
    // controls that changes the accuracy, and "3.2 m" means nothing on
    // its own - "about 8 cm at the far end" is the same fact in the
    // units the result will be judged in.
    var atFar = ChurchFrame.ExpectedErrorAt(FarStationMetres, apart);
    Note = $"Origin set. {apart:F1} m baseline ≈ {atFar * 100:F0} cm at the far station."
         + (apart < 3.5f ? "  A longer walk between taps would do better." : "");
    Debug.Log($"[OriginPicker] origin {Origin.position} heading " +
              $"{Origin.rotation.eulerAngles.y:F1} deg, baseline {apart:F2} m");

    Established?.Invoke(Origin);
    return true;
}

/// <summary>
/// The first touch of a new press, or a mouse click in the editor.
/// Returns false when nothing was pressed this frame.
/// </summary>
public bool TryGetTapThisFrame(out Vector2 screenPoint)
{
    if (Input.touchCount > 0)
    {
        var touch = Input.GetTouch(0);
        if (touch.phase == TouchPhase.Began)
        {
            screenPoint = touch.position;
            return true;
        }
    }
    else if (Input.GetMouseButtonDown(0))
    {
        screenPoint = Input.mousePosition;
        return true;
    }
```

```
        screenPoint = default;
        return false;
    }
}
```

4.1.14 AR-014 — POIDirectory.cs

Code ID	AR-014
File Name	POIDirectory.cs
File Path	SanctiWalkAR/Assets/Scripts/POIDirectory.cs
Language / Type	C#
Lines	159
Functions / Classes	POIDirectory (public class); Start (method); Refresh (method); Update (method); GoTo (method); OnGUI (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/POIDirectory.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — POIDirectory.cs

```
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

/// <summary>
/// A testing aid: lists every point of interest in the scene and teleports
/// the walkthrough player to whichever one you click.
///
/// Not navigation, and not meant to become it. Walking the room end to end
/// to check a trigger is slow, and this makes each one reachable in a
/// click. Press TAB to show or hide it.
///
/// It is IMGUI on purpose. A uGUI list would need its own canvas, layout
/// groups and event system in a scene whose only screen-space UI is the
/// walkthrough readout; OnGUI costs nothing here and disappears with the
/// component.
/// </summary>
public class POIDirectory : MonoBehaviour
{
    public KeyCode toggleKey = KeyCode.Tab;
    public bool visible = true;

    /// <summary>Where the player is put, relative to the POI.</summary>
    public float standOffDistance = 1.4f;

    readonly List<PointOfInterest> m_Pois = new List<PointOfInterest>();
    FirstPersonWalkthrough m_Player;
    CharacterController m_Controller;

    void Start()
    {
```

```
    Refresh();
    m_Player = FindFirstObjectByType<FirstPersonWalkthrough>();
    if (m_Player != null) m_Controller = m_Player.GetComponent<CharacterController>();

    foreach (var arg in System.Environment.GetCommandLineArgs())
        if (arg == "-poiSelfTest") { StartCoroutine(SelfTest()); break; }
}

/// <summary>
/// Visits every point of interest and reports whether its trigger
/// actually fired and a panel actually appeared.
///
/// Driving the walkthrough by hand to check four triggers is slow, and
/// a screenshot of a game window is easily mistaken for one of another
/// window. This states the result in the log instead.
/// </summary>
System.Collections.IEnumerator SelfTest()
{
    yield return new WaitForSeconds(1f);
    Debug.Log($"[POISelfTest] {m_Pois.Count} points of interest found");

    var passed = 0;
    foreach (var poi in m_Pois)
    {
        GoTo(poi);

        // Physics runs on the fixed step, so the trigger cannot be
        // read on the same frame the player is moved.
        yield return new WaitForFixedUpdate();
        yield return new WaitForSeconds(0.4f);

        var panel = GameObject.Find($"POIPanel_{poi.id}");
        var panelUp = panel != null && panel.activeInHierarchy;
        var distance = Vector3.Distance(m_Player.transform.position, poi.transform.position);
        var ok = poi.IsOccupied && panelUp;
        if (ok) passed++;

        Debug.Log($"[POISelfTest] {poi.id,-18} {distance:F2} m  " +
                  $"inside={poi.IsOccupied}  panel={(panel == null ? "none" : panelUp ? "shown" : "hidden")}  " +
                  $"-> {(ok ? "PASS" : "FAIL")}");
    }

    // And that leaving actually hides it again.
    var away = m_Pois[0].transform.position + Vector3.right * 12f;
    m_Controller.enabled = false;
    m_Player.transform.position = new Vector3(away.x, m_Player.transform.position.y, away.z);
    m_Controller.enabled = true;
    yield return new WaitForFixedUpdate();
    yield return new WaitForSeconds(0.4f);

    var stillOccupied = m_Pois.FindAll(p => p.IsOccupied).Count;
    Debug.Log($"[POISelfTest] after walking away, {stillOccupied} still occupied (want 0)");
    Debug.Log($"[POISelfTest] RESULT {passed}/{m_Pois.Count} triggers fired and showed a panel");

    Application.Quit();
}

void Refresh()
{
```

```
    m_Pois.Clear();
    m_Pois.AddRange(FindObjectsByType<PointOfInterest>(FindObjectsInactive.Exclude)
        .OrderBy(p => p.id));
}

void Update()
{
    if (Input.GetKeyDown(toggleKey)) visible = !visible;
}

void GoTo(PointOfInterest poi)
{
    if (m_Player == null || m_Controller == null) return;

    // Stand off to one side rather than inside the object, and at the
    // floor height the rest of the system uses. Moving a
    // CharacterController by assigning transform.position needs it
    // disabled first, or the controller overwrites the move.
    var here = m_Player.transform.position;
    var away = here - poi.transform.position;
    away.y = 0f;
    if (away.sqrMagnitude < 0.01f) away = Vector3.forward;
    away = away.normalized;

    var target = poi.transform.position + away * standOffDistance;
    target.y = StationMarker.FloorY + m_Controller.height * 0.5f + 0.05f;

    m_Controller.enabled = false;
    m_Player.transform.position = target;
    m_Player.transform.rotation = Quaternion.LookRotation(-away, Vector3.up);
    m_Controller.enabled = true;
}

void OnGUI()
{
    if (!visible)
    {
        GUI.Label(new Rect(Screen.width - 210, 10, 200, 22), "TAB  point of interest list");
        return;
    }

    const float w = 230f;
    var h = 54f + m_Pois.Count * 30f;
    var box = new Rect(Screen.width - w - 12f, 10f, w, h);

    GUI.Box(box, "Points of interest");

    var y = box.y + 26f;
    foreach (var poi in m_Pois)
    {
        var distance = m_Player == null
            ? 0f
            : Vector3.Distance(m_Player.transform.position, poi.transform.position);

        var label = $"{poi.title}   {distance:F1} m";
        var inside = poi.IsOccupied;

        var previous = GUI.color;
        if (inside) GUI.color = new Color(0.55f, 1f, 0.6f);
```

```
            if (GUI.Button(new Rect(box.x + 8f, y, w - 16f, 26f), label)) GoTo(poi);

            GUI.color = previous;
            y += 30f;
        }

        GUI.Label(new Rect(box.x + 8f, y, w - 16f, 22f), "click to stand at it");
    }
}
```

4.1.15 AR-015 — POIPanel.cs

Code ID	AR-015
File Name	POIPanel.cs
File Path	SanctiWalkAR/Assets/Scripts/POIPanel.cs
Language / Type	C#
Lines	112
Functions / Classes	POIPanel (public class); Create (public method); Build (method); LateUpdate (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/POIPanel.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — POIPanel.cs

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// The plain prototype panel: a world-space card that appears beside a
/// point of interest and turns to face the reader.
///
/// Deliberately unstyled. StationBubble is the designed AR card and reads
/// the same PointOfInterest fields; this one exists so the trigger system
/// can be walked and tested before any of the real content or artwork is
/// written. Replacing it later means pointing PointOfInterest at a
/// different presentation component - the data does not move.
/// </summary>
public class POIPanel : MonoBehaviour
{
    /// <summary>Metres per canvas pixel. 480 px of card is just under half a metre.</summary>
    const float Scale = 0.001f;
    const float Width = 480f;
    const float Height = 230f;

    PointOfInterest m_Poi;
    Transform m_Card;
    Camera m_Camera;

    public static POIPanel Create(PointOfInterest poi)
    {
        var go = new GameObject($"POIPanel_{poi.id}");
        go.transform.position = poi.PanelAnchor;
        var panel = go.AddComponent<POIPanel>();
        panel.Build(poi);
        return panel;
    }
```

```
void Build(PointOfInterest poi)
{
    m_Poi = poi;
    m_Camera = Camera.main;

    var canvasGo = new GameObject("Card", typeof(Canvas), typeof(CanvasRenderer));
    canvasGo.transform.SetParent(transform, false);
    m_Card = canvasGo.transform;

    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.WorldSpace;
    canvas.worldCamera = m_Camera;

    var rt = (RectTransform)canvasGo.transform;
    rt.sizeDelta = new Vector2(Width, Height);
    rt.localScale = Vector3.one * Scale;

    var bg = canvasGo.AddComponent<Image>();
    bg.color = new Color32(0xFA, 0xF8, 0xF4, 0xF2);

    var heading = Resources.Load<Font>("Montserrat-SemiBold");
    var body = Resources.Load<Font>("Poppins-Regular");

    Label(rt, "POINT OF INTEREST", body, 17, new Color32(0x8A, 0x8F, 0x99, 0xFF),
          new Vector2(22f, -20f), new Vector2(Width - 44f, 24f));

    Label(rt, poi.title, heading, 30, new Color32(0x1A, 0x1D, 0x23, 0xFF),
          new Vector2(22f, -46f), new Vector2(Width - 44f, 42f));

    var descriptionTop = -92f;
    if (!string.IsNullOrEmpty(poi.subtitle))
    {
        Label(rt, poi.subtitle, body, 19, new Color32(0x6B, 0x72, 0x80, 0xFF),
              new Vector2(22f, -90f), new Vector2(Width - 44f, 26f));
        descriptionTop = -120f;
    }

    Label(rt, poi.description, body, 19, new Color32(0x2C, 0x30, 0x38, 0xFF),
          new Vector2(22f, descriptionTop), new Vector2(Width - 44f, Height + descriptionTop - 20f));
}

static void Label(RectTransform parent, string text, Font font, int size, Color colour,
                  Vector2 topLeft, Vector2 size2)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);

    var t = go.GetComponent<Text>();
    t.text = text;
    t.font = font != null ? font : Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
    t.fontSize = size;
    t.color = colour;
    t.alignment = TextAnchor.UpperLeft;
    t.horizontalOverflow = HorizontalWrapMode.Wrap;
    t.verticalOverflow = VerticalWrapMode.Truncate;
    t.raycastTarget = false;

    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0f, 1f);
```

```
        rt.pivot = new Vector2(0f, 1f);
        rt.anchoredPosition = topLeft;
        rt.sizeDelta = size2;
    }

    void LateUpdate()
    {
        if (m_Camera == null) m_Camera = Camera.main;
        if (m_Camera == null || m_Card == null || m_Poi == null) return;

        transform.position = m_Poi.PanelAnchor;

        // Yaw toward the reader, never pitch. A card that tips back when
        // you crouch reads as broken - the same rule the AR bubble uses.
        var toCamera = m_Camera.transform.position - m_Card.position;
        toCamera.y = 0f;
        if (toCamera.sqrMagnitude > 0.0001f)
            m_Card.rotation = Quaternion.LookRotation(-toCamera.normalized, Vector3.up);
    }
}
```

4.1.16 AR-016 — PointOfInterest.cs

Code ID	AR-016
File Name	PointOfInterest.cs
File Path	SanctiWalkAR/Assets/Scripts/PointOfInterest.cs
Language / Type	C#
Lines	195
Functions / Classes	PointOfInterest (public class); Awake (method); EnsureTrigger (method); OnValidate (method); OnTriggerEnter (method); OnTriggerExit (method); Update (method); Leave (method); ShowPanel (method); HidePanel (method); OnDrawGizmos (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/PointOfInterest.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — PointOfInterest.cs

```
using UnityEngine;

/// <summary>
/// One point of interest: what it is, where its trigger is, and what to
/// say when someone reaches it.
///
/// This is the single owner of POI data. StationMarker used to carry id,
/// name and radius too; it now reads them from here, so there is one
/// authoritative copy and no chance of the pin and the trigger disagreeing.
///
/// Data is deliberately kept apart from presentation. This component knows
/// nothing about canvases or AR: it decides only whether the visitor is
/// inside the radius, and raises Entered / Exited. POIPanel draws the plain
/// prototype panel from it; StationBubble draws the designed AR card from
/// the same fields. Swapping one for the other later changes no data.
/// </summary>
[DisallowMultipleComponent]
public class PointOfInterest : MonoBehaviour
{
    [Header("Identity")]
    [Tooltip("Stable id, e.g. sala-santo-nino. Matches ChurchOriginTest and MuseumStations.")]
    public string id = "sala-station";

    [Tooltip("Shown as the heading on the panel.")]
    public string title = "Station";

    [Tooltip("Small grey line under the title. Optional.")]
    public string subtitle = "";

    [TextArea(3, 8)]
```

```
[Tooltip("Panel body. Replace this with the real text when you have it.")]
public string description = "Church information will be added here.";

[Tooltip("Statue, window, altar, artifact - whatever this stop is. Optional.")]
public string poiType = "";

[Header("Trigger")]
[Tooltip("The panel appears when the player is within this many metres.")]
public float triggerRadius = 2f;

[Tooltip("Placeholder artwork marking the trigger in the Scene view. Swap freely.")]
public Texture2D triggerImage;

[Header("Panel")]
[Tooltip("Metres above this point that the panel floats.")]
public float panelHeight = 0.55f;

[Tooltip("Sideways offset, if the panel would otherwise sit inside the object.")]
public float panelSideOffset = 0f;

[Header("Content (optional, fill in later)")]
public Texture2D image;
public AudioClip audioClip;

[Header("Tour")]
[Tooltip("id of the next stop, or blank if this is the last.")]
public string nextId = "";

/// <summary>Raised the moment the visitor comes inside the radius.</summary>
public event System.Action<PointOfInterest> Entered;

/// <summary>Raised when they leave it.</summary>
public event System.Action<PointOfInterest> Exited;

public bool IsOccupied { get; private set; }

SphereCollider m_Trigger;
POIPanel m_Panel;

void Awake()
{
    EnsureTrigger();
}

void EnsureTrigger()
{
    m_Trigger = GetComponent<SphereCollider>();
    if (m_Trigger == null) m_Trigger = gameObject.AddComponent<SphereCollider>();
    m_Trigger.isTrigger = true;
    m_Trigger.radius = triggerRadius;
}

void OnValidate()
{
    // Keep the collider honest when the radius is edited in the
    // Inspector, so the gizmo and the actual trigger never disagree.
    var sphere = GetComponent<SphereCollider>();
    if (sphere != null)
    {
```

```
        sphere.isTrigger = true;
        sphere.radius = triggerRadius;
    }
    if (string.IsNullOrEmpty(title)) title = name;
}

void OnTriggerEnter(Collider other)
{
    if (!IsPlayer(other) || IsOccupied) return;
    IsOccupied = true;
    ShowPanel();
    Entered?.Invoke(this);
}

void OnTriggerExit(Collider other)
{
    if (!IsPlayer(other)) return;
    Leave();
}

/// <summary>
/// A safety net for exits the collider never reports.
///
/// OnTriggerExit does not fire when a CharacterController is disabled
/// and teleported, which is exactly what the POI directory does - the
/// self test caught all four stations still reading as occupied after
/// the player had been moved 12 m away, with every panel left on
/// screen. Walking out normally still fires the callback; this only
/// corrects the state when it does not.
/// </summary>
void Update()
{
    if (!IsOccupied) return;

    var player = m_Player != null ? m_Player : m_Player = FindPlayer();
    if (player == null) return;

    if (Vector3.Distance(player.position, transform.position) > triggerRadius * 1.05f)
        Leave();
}

void Leave()
{
    if (!IsOccupied) return;
    IsOccupied = false;
    HidePanel();
    Exited?.Invoke(this);
}

Transform m_Player;

static Transform FindPlayer()
{
    var walker = FindFirstObjectByType<FirstPersonWalkthrough>();
    if (walker != null) return walker.transform;
    var cam = Camera.main;
    return cam != null ? cam.transform : null;
}
```

```
    /// <summary>
    /// The walkthrough player, or the AR camera rig later. Matched by
    /// component rather than tag, so nothing depends on tag setup being
    /// remembered.
    /// </summary>
    static bool IsPlayer(Collider other)
    {
        return other.GetComponentInParent<FirstPersonWalkthrough>() != null
            || other.GetComponentInParent<CharacterController>() != null;
    }

    void ShowPanel()
    {
        if (m_Panel == null) m_Panel = POIPanel.Create(this);
        m_Panel.gameObject.SetActive(true);
    }

    void HidePanel()
    {
        if (m_Panel != null) m_Panel.gameObject.SetActive(false);
    }

    /// <summary>Where the panel hangs, in world space.</summary>
    public Vector3 PanelAnchor =>
        transform.position
        + Vector3.up * panelHeight
        + transform.right * panelSideOffset;

#if UNITY_EDITOR
    void OnDrawGizmos()
    {
        // The trigger volume. Drawn here rather than on StationMarker so a
        // POI is visible even on an object that has no authoring pin.
        var colour = IsOccupied ? new Color(0.35f, 0.95f, 0.45f) : new Color(0.25f, 0.6f, 1f);

        Gizmos.color = new Color(colour.r, colour.g, colour.b, 0.10f);
        Gizmos.DrawSphere(transform.position, triggerRadius);
        Gizmos.color = new Color(colour.r, colour.g, colour.b, 0.9f);
        Gizmos.DrawWireSphere(transform.position, triggerRadius);

        // Where the panel will appear.
        UnityEditor.Handles.color = new Color(1f, 0.85f, 0.45f, 0.8f);
        UnityEditor.Handles.DrawDottedLine(transform.position, PanelAnchor, 3f);
        UnityEditor.Handles.DrawWireCube(PanelAnchor, new Vector3(0.5f, 0.28f, 0.01f));
    }
#endif
}
```

4.1.17 AR-017 — ScanSimulator.cs

Code ID	AR-017
File Name	ScanSimulator.cs
File Path	SanctiWalkAR/Assets/Scripts/ScanSimulator.cs
Language / Type	C#
Lines	69
Functions / Classes	ScanSimulator (public class); Sala (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/ScanSimulator.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ScanSimulator.cs

```
using UnityEngine;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Produces a MarkerScan identical in shape to one ARCore would deliver,
/// without a camera, a phone or a printed marker.
///
/// This is not a second code path. It builds the same struct the real
/// tracked-image handler builds, and hands it to the same SetOrigin, so
/// everything downstream - the into-room sign, the surveyed offsets, the
/// bubbles, the trigger radii, the next-stop chain - runs exactly as it
/// will on device. When the phone comes back, deleting the button is the
/// whole of the change.
///
/// The numbers below are chosen to match what a real scan of the sala
/// marker looks like: someone standing about 1.2 m back from the wall
/// behind the TV, holding the phone at chest height, pointing at the
/// print.
/// </summary>
public static class ScanSimulator
{
    /// <summary>The reference image name in ChurchMarkers.asset.</summary>
    public const string MarkerName = "entrance";

    /// <summary>Printed size, matching the library and the physical print.</summary>
    public static readonly Vector2 PhysicalSize = new Vector2(0.18f, 0.24f);

    /// <summary>
    /// A scan as ARCore would report it.
    ///
    /// Session space has its origin where the AR session started, which for
    /// a simulated run is simply the camera's starting transform. The
    /// marker is placed 1.2 m in front of that, at the height the print is
```

```
    /// actually mounted relative to a chest-height phone: the marker centre
    /// sits 1.407 m above the floor and the phone about 1.45 m, so the
    /// marker is a few centimetres below the camera.
    /// </summary>
    public static MarkerScan Sala(Transform camera)
    {
        var cameraPose = camera == null
            ? new Pose(Vector3.zero, Quaternion.identity)
            : new Pose(camera.position, camera.rotation);

        // Straight ahead of the camera, level, and facing back at it.
        var forward = camera == null ? Vector3.forward : camera.forward;
        forward.y = 0f;
        forward = forward.sqrMagnitude < 0.0001f ? Vector3.forward : forward.normalized;

        var markerCentre = cameraPose.position + forward * 1.2f + Vector3.down * 0.04f;

        // An ARCore tracked image's +Y is the normal coming out of the
        // print, and its +Z is "up" along the printed page. Built here the
        // same way, so the into-room test and ChurchToMarker behave
        // identically to a real detection.
        var normal = -forward;                       // out of the wall, toward the camera
        var rotation = Quaternion.LookRotation(Vector3.up, normal);

        return new MarkerScan
        {
            trackedImageName = MarkerName,
            referenceGuid = "9f2c1d64-0a77-4b1e-9d53-6c0b8e4f21aa",
            pose = new Pose(markerCentre, rotation),
            trackingState = TrackingState.Tracking,
            physicalSize = PhysicalSize,
            simulated = true,
            cameraPoseAtDetection = cameraPose,
        };
    }
}
```

4.1.18 AR-018 — SceneGizmos.cs

Code ID	AR-018
File Name	SceneGizmos.cs
File Path	SanctiWalkAR/Assets/Scripts/SceneGizmos.cs
Language / Type	C#
Lines	57
Functions / Classes	MarkerGizmo (public class); OnDrawGizmos (method); FloorGridGizmo (public class)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/SceneGizmos.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SceneGizmos.cs

```
using UnityEngine;

// These two draw the marker and the floor grid in the Scene view. They
// used to live in StationEditorScene.cs, inside Assets/Editor, which is
// not compiled into builds - so every build reported "the referenced
// script on this Behaviour is missing". The drawing is already guarded by
// UNITY_EDITOR; only the class needed to be loadable at runtime.

/// <summary>Draws the marker's axes and its outward normal.</summary>
[ExecuteAlways]
public class MarkerGizmo : MonoBehaviour
{
#if UNITY_EDITOR
    void OnDrawGizmos()
    {
        var p = transform.position;
        UnityEditor.Handles.color = new Color(1f, 0.82f, 0.35f);
        UnityEditor.Handles.ArrowHandleCap(0, p, Quaternion.LookRotation(StationMarker.MarkerNormal), 0.8f, EventType.Repaint);
        UnityEditor.Handles.DrawDottedLine(p, new Vector3(p.x, StationMarker.FloorY, p.z), 3f);

        var style = new GUIStyle { normal = { textColor = new Color(1f, 0.85f, 0.45f) }, fontSize = 11, fontStyle = FontStyle.Bold };
        UnityEditor.Handles.Label(p + Vector3.up * 0.22f,
            $"ENTRANCE MARKER  (origin)\nmap X {p.x:F3}  Y {p.y:F3}  Z {p.z:F3}\n" +
            $"{p.y - StationMarker.FloorY:F3} m above floor   180 x 240 mm", style);
    }
#endif
}

/// <summary>A one-metre grid on the floor plane, for scale.</summary>
[ExecuteAlways]
public class FloorGridGizmo : MonoBehaviour
{
    public int halfExtent = 7;
```

```
#if UNITY_EDITOR
    void OnDrawGizmos()
    {
        var y = StationMarker.FloorY;
        UnityEditor.Handles.color = new Color(1f, 1f, 1f, 0.10f);
        for (var i = -halfExtent; i <= halfExtent + 4; i++)
        {
            UnityEditor.Handles.DrawLine(new Vector3(i, y, -halfExtent), new Vector3(i, y, halfExtent + 4));
            UnityEditor.Handles.DrawLine(new Vector3(-halfExtent, y, i), new Vector3(halfExtent + 4, y, i));
        }

        // Map-space origin, which is 1.6 m above the floor where the scan
        // started - worth seeing, because it is not on the ground.
        UnityEditor.Handles.color = Color.red;
        UnityEditor.Handles.DrawLine(Vector3.zero, Vector3.right);
        UnityEditor.Handles.color = Color.green;
        UnityEditor.Handles.DrawLine(Vector3.zero, Vector3.up);
        UnityEditor.Handles.color = Color.blue;
        UnityEditor.Handles.DrawLine(Vector3.zero, Vector3.forward);
        UnityEditor.Handles.Label(Vector3.up * 1.1f, "map space (0,0,0)");
    }
#endif
}
```

4.1.19 AR-019 — SiteReadiness.cs

Code ID	AR-019
File Name	SiteReadiness.cs
File Path	SanctiWalkAR/Assets/Scripts/SiteReadiness.cs
Language / Type	C#
Lines	274
Functions / Classes	SiteReadiness (public class); Bind (public method); StopScanning (public method); Update (method); FloorHeld (method); AccumulateMotion (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/SiteReadiness.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — SiteReadiness.cs

```
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

/// <summary>
/// Decides when the room is understood well enough to put anything in it.
///
/// Apps that place content well do not start placing it the moment the
/// camera opens. They ask you to move the phone about, find the floor, and
/// only then show you anything - because ARCore's idea of the room in the
/// first second is a guess that it then revises, and content placed into
/// that guess visibly slides as the guess improves.
///
/// Every gate here is something that was silently assumed before, and each
/// one has an instruction attached, because a checklist that says "not
/// ready" without saying what to do is just a spinner.
///
/// ## The floor has to be found geometrically
///
/// ARCore does not classify planes: ARCorePlaneSubsystem sets
/// supportsClassificationDelegate to false, so ARPlane.classifications is
/// always None and PlaneClassifications.Floor can never be tested for.
/// What is available is the alignment, the size, and where it sits relative
/// to the camera - a large horizontal-up plane about a person's height
/// below the phone is the floor, and a table is not, because a table is
/// neither that big nor that far down.
/// </summary>
[RequireComponent(typeof(ARPlaneManager))]
public class SiteReadiness : MonoBehaviour
{
```

```
public enum Gate
{
    Session,     // the AR session is running at all
    Steady,      // tracking is not complaining
    Floor,       // a floor plane has been found
    Coverage,    // enough of the room has been seen to trust it
    Ready,
}

/// <summary>What the phone should be told to do, and how far along it is.</summary>
public readonly struct Report
{
    public readonly Gate Stage;
    public readonly string Instruction;
    public readonly float FloorY;
    public readonly bool HasFloor;

    public Report(Gate stage, string instruction, float floorY, bool hasFloor)
    {
        Stage = stage;
        Instruction = instruction;
        FloorY = floorY;
        HasFloor = hasFloor;
    }

    public bool IsReady => Stage == Gate.Ready;
}

// Held, not instantaneous. Tracking flickers for a frame all the time,
// and a checklist that ticks and unticks reads as broken even when it
// is telling the truth.
const float k_SteadyForSeconds = 2.0f;
const float k_FloorForSeconds = 1.0f;

// A floor is big. A table is not, and neither is a chair seat.
const float k_MinFloorArea = 1.5f;

// How far below the phone the floor is allowed to be. Below 1.1 m is
// furniture; beyond 2.0 m is a stairwell or a bad plane.
const float k_MinDrop = 1.1f;
const float k_MaxDrop = 2.0f;

// Parallax. ARCore triangulates depth from motion, so a phone that has
// only rotated has seen the room from one point and knows least about
// the far end of it.
const float k_NeedWalked = 1.5f;
const float k_NeedSweptDeg = 90f;

public Report Current { get; private set; } =
    new Report(Gate.Session, "Starting the camera…", 0f, false);

public System.Action<Report> Changed;

ARPlaneManager m_Planes;
Camera m_Camera;

float m_SteadySince = -1f;
float m_FloorSince = -1f;
float m_FloorY;
```

```
bool m_HasFloor;

Vector3 m_LastPosition;
float m_Walked;
float m_SweptDeg;
float m_LastYaw;
bool m_MotionStarted;

public void Bind(Camera cam)
{
    m_Camera = cam;
    m_Planes = GetComponent<ARPlaneManager>();
    if (m_Planes != null)
    {
        // Arbitrary planes are not supported by ARCore; horizontal and
        // vertical are. Vertical is requested because walls give ARCore
        // extra structure to track against even though only the floor
        // is read here.
        m_Planes.requestedDetectionMode = PlaneDetectionMode.Horizontal | PlaneDetectionMode.Vertical;
        m_Planes.enabled = true;
    }
}

/// <summary>
/// Stop looking for planes. Three trackable managers running is real
/// frame time on this hardware, and ARCore needs a steady 30 fps here -
/// GLES3 collapsing to 1.93 fps is what stopped tracking working at
/// all, so headroom is not a luxury on this device.
/// </summary>
public void StopScanning()
{
    if (m_Planes != null)
        m_Planes.enabled = false;
}

void Update()
{
    if (m_Camera == null)
        return;

    var now = Time.time;
    AccumulateMotion();

    var stage = Gate.Session;
    string instruction;

    if (ARSession.state != ARSessionState.SessionTracking)
    {
        m_SteadySince = -1f;
        instruction = "Starting the camera…";
    }
    else if (ARSession.notTrackingReason != NotTrackingReason.None)
    {
        m_SteadySince = -1f;
        stage = Gate.Steady;
        instruction = InstructionFor(ARSession.notTrackingReason);
    }
    else
    {
```

```
        if (m_SteadySince < 0f) m_SteadySince = now;

        if (now - m_SteadySince < k_SteadyForSeconds)
        {
            stage = Gate.Steady;
            instruction = "Hold still for a moment…";
        }
        else if (!FloorHeld(now))
        {
            stage = Gate.Floor;
            instruction = "Point the camera at the floor a few steps ahead";
        }
        else if (m_Walked < k_NeedWalked && m_SweptDeg < k_NeedSweptDeg)
        {
            stage = Gate.Coverage;
            instruction = "Walk a couple of steps, or look slowly around the room";
        }
        else
        {
            stage = Gate.Ready;
            instruction = "Ready";
        }
    }

    var next = new Report(stage, instruction, m_FloorY, m_HasFloor);
    if (next.Stage != Current.Stage || next.Instruction != Current.Instruction)
    {
        Current = next;
        Changed?.Invoke(next);
    }
    else
    {
        Current = next;
    }
}

/// <summary>ARCore's own reason, said to the person holding the phone.</summary>
static string InstructionFor(NotTrackingReason reason) => reason switch
{
    NotTrackingReason.InsufficientLight => "Too dark here - find more light",
    NotTrackingReason.InsufficientFeatures => "Not enough detail - point at something patterned",
    NotTrackingReason.ExcessiveMotion => "Moving too fast - slow down",
    NotTrackingReason.Relocalizing => "Finding the room again…",
    NotTrackingReason.Initializing => "Getting your bearings…",
    NotTrackingReason.CameraUnavailable => "The camera is in use by another app",
    _ => "Move the phone slowly side to side",
};

/// <summary>
/// The largest horizontal-up plane at a plausible height below the
/// phone, held long enough to believe. Largest rather than first: a big
/// plane is a floor, and the first one reported is often a small patch
/// that later merges into something bigger.
/// </summary>
bool FloorHeld(float now)
{
    if (m_Planes == null)
        return false;
```

```
    var best = float.NegativeInfinity;
    var bestY = 0f;
    var found = false;

    foreach (var plane in m_Planes.trackables)
    {
        if (plane.alignment != PlaneAlignment.HorizontalUp) continue;
        if (plane.trackingState != TrackingState.Tracking) continue;

        var area = plane.size.x * plane.size.y;
        if (area < k_MinFloorArea) continue;

        var drop = m_Camera.transform.position.y - plane.center.y;
        if (drop < k_MinDrop || drop > k_MaxDrop) continue;

        if (area > best)
        {
            best = area;
            bestY = plane.center.y;
            found = true;
        }
    }

    if (!found)
    {
        m_FloorSince = -1f;
        m_HasFloor = false;
        return false;
    }

    m_FloorY = bestY;
    m_HasFloor = true;
    if (m_FloorSince < 0f) m_FloorSince = now;
    return now - m_FloorSince >= k_FloorForSeconds;
}

void AccumulateMotion()
{
    var p = m_Camera.transform.position;
    var yaw = m_Camera.transform.eulerAngles.y;

    if (!m_MotionStarted)
    {
        m_LastPosition = p;
        m_LastYaw = yaw;
        m_MotionStarted = true;
        return;
    }

    m_Walked += Vector3.Distance(p, m_LastPosition);
    m_LastPosition = p;

    m_SweptDeg += Mathf.Abs(Mathf.DeltaAngle(m_LastYaw, yaw));
    m_LastYaw = yaw;
}

/// <summary>For the overlay: the checklist, in order, with its state.</summary>
public IEnumerable<(string label, bool done)> Checklist()
{
```

```
        var s = Current.Stage;
        yield return ("camera ready", s > Gate.Session);
        yield return ("holding steady", s > Gate.Steady);
        yield return ("floor found", s > Gate.Floor);
        yield return ("room seen", s > Gate.Coverage);
    }
}
```

4.1.20 AR-020 — StationBubble.cs

Code ID	AR-020
File Name	StationBubble.cs
File Path	SanctiWalkAR/Assets/Scripts/StationBubble.cs
Language / Type	C#
Lines	426
Functions / Classes	StationBubble (public class); MoveTo (public method); Create (public method); PlaceCardAtReadingHeight (method); Build (method); BuildHistoryCard (method); BuildFramedImage (method); LateUpdate (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/StationBubble.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StationBubble.cs

```
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// The information bubble that hangs in the air beside a station.
///
/// This is the thing in screens 2 and 5 of the design: a small card sitting
/// on the statue or the window itself, not a rectangle stuck to the phone
/// screen. Phase 0 proved the anchoring with a single pre-rendered texture
/// on a quad and left a note saying the real version would be built from
/// content. This is that version - the same card for every station, filled
/// from MuseumStation.
///
/// Built as a world-space Canvas so the text is real text at any distance,
/// rather than a texture that turns to mush when you walk up to it.
/// </summary>
public class StationBubble : MonoBehaviour
{
    /// <summary>Metres per canvas pixel. 500 px of card is half a metre.</summary>
    const float Scale = 0.001f;

    // 0.8 m of card at Scale = 0.001, which is what a whole paragraph needs
    // to be readable standing at the station rather than pressed against it.
    // The height is computed per station now: a history carries its whole
    // text and they are not the same length.
    const float CardWidth = 800f;
    const float CardHeight = 240f;

    /// <summary>Widest an AR image may be drawn, in canvas pixels.</summary>
    const float FrameWidth = 700f;
```

```
// The frame, from the mockup: a dark wood outer, a gold line inside it,
// then the photograph. Drawn rather than loaded so it stretches to each
// photograph's own shape - they run from 263x192 to 1704x1278 and a
// fixed frame would letterbox most of them.
const float FrameOuter = 26f;     // wood
const float FrameInner = 7f;      // gold

static readonly Color CardWhite = new Color32(0xFF, 0xFF, 0xFF, 0xF7);
static readonly Color Ink       = new Color32(0x1A, 0x1D, 0x23, 0xFF);
static readonly Color Muted     = new Color32(0x6B, 0x72, 0x80, 0xFF);
static readonly Color Sand      = new Color32(0xE8, 0xDC, 0xC8, 0xFF);
static readonly Color Navy      = new Color32(0x14, 0x26, 0x4A, 0xFF);
static readonly Color Wood      = new Color32(0x4A, 0x33, 0x22, 0xFF);
static readonly Color Gold      = new Color32(0xC9, 0xA2, 0x4D, 0xFF);
static readonly Color Caption   = new Color32(0xF2, 0xEF, 0xE8, 0xFF);

public MuseumStation Station { get; private set; }

/// <summary>Which station this card belongs to, for matching one by id.</summary>
public string StationId => Station != null ? Station.stationId : null;

/// <summary>
/// Moves the whole bubble - stalk, anchor and card - somewhere else.
///
/// The card's own orientation is left alone: it is turned to face the
/// viewer every frame anyway, so setting it here would be overwritten
/// before anyone saw it.
/// </summary>
public void MoveTo(Vector3 worldPosition)
{
    transform.position = worldPosition;
    // The anchor moved, so the lift needed to reach reading height did
    // too - without this a station dragged downwards keeps a card that
    // now floats well above where it should.
    PlaceCardAtReadingHeight();
}

Camera m_Camera;
Transform m_Card;
Transform m_Stalk;

/// <summary>
/// How high the card floats above the floor, in metres.
///
/// Reading height, not object height. The anchor stays ON the thing it
/// describes - that is what makes it a label rather than a sign in the
/// air - but the CARD was pinned 22cm above the anchor, so a card about
/// a sofa sat at shin level and had to be read by crouching. Everything
/// in a room is at a different height and none of those heights is a
/// comfortable one to read at.
///
/// 1.45m is a little below average eye level, which is deliberate: it
/// reads as being in front of you rather than above you, and it does
/// not put the card in the ceiling for a shorter visitor.
/// </summary>
public const float CardAboveFloor = 1.45f;

/// <summary>The floor, in session metres. Set by MuseumUI when it binds.</summary>
```

```
public static float SessionFloorY;

public static StationBubble Create(MuseumStation station, Vector3 worldPosition,
                                   Font heading, Font body, Camera camera)
{
    var go = new GameObject($"Bubble_{station.stationId}");
    go.transform.position = worldPosition;
    var bubble = go.AddComponent<StationBubble>();
    bubble.Build(station, heading, body, camera);
    return bubble;
}

/// <summary>
/// Puts the card at reading height and stretches the stalk to reach it.
///
/// Never BELOW the anchor: an object already above head height - a
/// statue in a niche, a window - would otherwise have its card dragged
/// down through it. For those the old 22cm offset is still right, so it
/// is the floor of this calculation rather than something it replaces.
/// </summary>
/// <summary>
/// How far a card steps aside from the thing it describes.
///
/// A card is 0.8 m wide and hangs at reading height, centred on the
/// station. Over a history marker that is fine - there is nothing there
/// to hide. Over a life-size statue standing on the floor it is not: the
/// card lands in the statue's chest and the stalk runs up through the
/// text. Models therefore get their card beside them, clear of both.
/// </summary>
const float ModelSideStep = 0.85f;

float m_Lift;
float m_SideStep;

void PlaceCardAtReadingHeight()
{
    if (m_Card == null) return;

    var anchorY = transform.position.y;
    var wanted = SessionFloorY + CardAboveFloor;
    var lift = Mathf.Max(0.22f + CardHeight * Scale * 0.5f, wanted - anchorY);
    m_Lift = lift;

    m_Card.localPosition = new Vector3(0f, lift, 0f);

    if (m_Stalk != null)
    {
        // The stalk is a unit cylinder: 0.5 tall at scale 1, so the
        // scale IS half the height, and its centre sits at half again.
        var reach = Mathf.Max(0.02f, lift - CardHeight * Scale * 0.5f);
        m_Stalk.localScale = new Vector3(0.004f, reach * 0.5f, 0.004f);
        m_Stalk.localPosition = new Vector3(0f, reach * 0.5f, 0f);
    }
}

void Build(MuseumStation station, Font heading, Font body, Camera camera)
{
    Station = station;
    m_Camera = camera;
```

```
    // A short stalk from the object up to the card, so the card floats
    // clear of whatever it is describing instead of intersecting it.
    //
    // A model station gets neither stalk nor dot. The station IS the
    // statue, so the stalk rose from its feet straight up through its
    // face - a white line across the thing you came to look at - and the
    // card now sits beside the statue anyway, where a stem pointing at
    // its own base explains nothing.
    var isModel = station.kind == MuseumStationKind.Model;
    if (!isModel)
    {
        var stalk = Shape("Stalk", "Cylinder.fbx", new Vector3(0.004f, 0.11f, 0.004f),
                          new Vector3(0f, 0.11f, 0f), new Color(1f, 1f, 1f, 0.75f));
        m_Stalk = stalk.transform;
        Shape("Anchor", "Sphere.fbx", Vector3.one * 0.035f, Vector3.zero, Navy);
    }

    // The card itself.
    var canvasGo = new GameObject("Card", typeof(Canvas), typeof(GraphicRaycaster));
    canvasGo.transform.SetParent(transform, false);
    canvasGo.transform.localPosition = new Vector3(0f, 0.22f + CardHeight * Scale * 0.5f, 0f);
    m_Card = canvasGo.transform;
    PlaceCardAtReadingHeight();

    var canvas = canvasGo.GetComponent<Canvas>();
    canvas.renderMode = RenderMode.WorldSpace;
    canvas.worldCamera = camera;

    // Only models displace their card. History and image stations have
    // nothing standing at the station to collide with, and moving those
    // would change fifteen cards that are already correct in the room.
    // Always to the same hand. MuseumStation carries no aisle side - that
    // field stays in the plan JSON and never reaches the app - and a
    // consistent side is easier to read on a walk than an alternating one.
    m_SideStep = station.kind == MuseumStationKind.Model ? ModelSideStep : 0f;

    var rt = (RectTransform)canvasGo.transform;
    rt.localScale = Vector3.one * Scale;

    if (station.kind == MuseumStationKind.Image)
        BuildFramedImage(rt, station, body);
    else
        BuildHistoryCard(rt, station, heading, body);
}

/// <summary>
/// Title and the whole history, floating. Nothing else.
///
/// No thumbnail: the plan is explicit that a history has no picture and
/// its picture lives on its own AR Image station. No "Tap to know more"
/// either - there is nothing left to tap for, because the text that
/// prompt used to lead to is now on the card.
/// </summary>
void BuildHistoryCard(RectTransform rt, MuseumStation station, Font heading, Font body)
{
    var text = string.IsNullOrEmpty(station.Detailed)
        ? station.Short
        : station.Detailed;
```

```
    // Height from the text, not a constant. These run from one sentence
    // to about six hundred characters and a fixed card either clips the
    // long ones or leaves the short ones mostly empty.
    const float pad = 34f;
    const float titleSize = 34f;
    const float bodySize = 23f;
    var titleLines = Mathf.Ceil(Measure(station.Name, titleSize * 0.52f, CardWidth - pad * 2f));
    var bodyLines  = Mathf.Ceil(Measure(text, bodySize * 0.50f, CardWidth - pad * 2f));
    var titleH = titleLines * (titleSize + 8f);
    var bodyH  = bodyLines * (bodySize + 9f);
    var height = pad + titleH + 18f + bodyH + pad;

    rt.sizeDelta = new Vector2(CardWidth, height);

    var bg = rt.gameObject.AddComponent<Image>();
    bg.color = CardWhite;
    Rounded(bg);

    var title = Label(rt, station.Name, heading, (int)titleSize, Ink);
    Place(title, new Vector2(pad, -pad), new Vector2(CardWidth - pad * 2f, titleH));

    var para = Label(rt, text, body, (int)bodySize, Ink);
    Place(para, new Vector2(pad, -(pad + titleH + 18f)),
          new Vector2(CardWidth - pad * 2f, bodyH));
}

/// <summary>
/// The photograph, framed, with its credit underneath.
///
/// The frame is three nested rectangles - wood, gold, photograph - which
/// is enough to read as a picture on a wall and costs no asset. Sized to
/// the photograph's own aspect so nothing is letterboxed or stretched.
/// </summary>
void BuildFramedImage(RectTransform rt, MuseumStation station, Font body)
{
    var art = string.IsNullOrEmpty(station.image)
        ? null
        : Resources.Load<Texture2D>(station.image);

    // Aspect from the photograph. Missing artwork falls back to a
    // portrait rectangle rather than a zero-height frame, so a station
    // with a broken path is visible and obviously wrong.
    var aspect = art != null && art.height > 0 ? (float)art.width / art.height : 0.75f;
    var photoW = FrameWidth;
    var photoH = photoW / aspect;

    // A very tall photograph would be a two-metre frame. Cap the height
    // and take the width back from the aspect instead.
    const float maxPhotoH = 760f;
    if (photoH > maxPhotoH)
    {
        photoH = maxPhotoH;
        photoW = photoH * aspect;
    }

    var edge = FrameOuter + FrameInner;
    var frameW = photoW + edge * 2f;
    var frameH = photoH + edge * 2f;
```

```
    // The credit sits under the frame, not on it.
    var caption = !string.IsNullOrEmpty(station.Short)
        ? station.Short
        : station.Name;
    const float capSize = 20f;
    var capLines = Mathf.Ceil(Measure(caption, capSize * 0.48f, frameW - 20f));
    var capH = capLines * (capSize + 6f);

    rt.sizeDelta = new Vector2(frameW, frameH + 12f + capH);

    // wood
    var wood = Panel(rt, Wood);
    Place(wood, Vector2.zero, new Vector2(frameW, frameH));
    // gold
    var gold = Panel(rt, Gold);
    Place(gold, new Vector2(FrameOuter, -FrameOuter),
          new Vector2(frameW - FrameOuter * 2f, frameH - FrameOuter * 2f));
    // photograph
    var photo = new GameObject("Photo", typeof(RectTransform), typeof(RawImage));
    photo.transform.SetParent(rt, false);
    var prt = (RectTransform)photo.transform;
    prt.anchorMin = prt.anchorMax = new Vector2(0f, 1f);
    prt.pivot = new Vector2(0f, 1f);
    prt.anchoredPosition = new Vector2(edge, -edge);
    prt.sizeDelta = new Vector2(photoW, photoH);
    var raw = photo.GetComponent<RawImage>();
    raw.texture = art;
    if (art == null) raw.color = new Color(0.1f, 0.1f, 0.1f, 0.85f);

    var cap = Label(rt, caption, body, (int)capSize, Caption);
    cap.alignment = TextAnchor.UpperCenter;
    cap.fontStyle = FontStyle.Italic;
    Place(cap, new Vector2(10f, -(frameH + 12f)), new Vector2(frameW - 20f, capH));
}

/// <summary>
/// Roughly how many lines of text at this width.
///
/// Unity has no cheap way to ask a Text how tall it wants to be before
/// a layout pass, and the card's height has to be known when it is
/// built. Average glyph width is close enough: the cost of being a
/// little generous is whitespace, the cost of being short is clipped
/// history.
/// </summary>
static float Measure(string text, float glyphWidth, float width)
{
    if (string.IsNullOrEmpty(text)) return 1f;
    var perLine = Mathf.Max(1f, width / Mathf.Max(1f, glyphWidth));
    return Mathf.Max(1f, text.Length / perLine + 0.999f);
}

static Image Panel(RectTransform parent, Color colour)
{
    var go = new GameObject("Panel", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    go.GetComponent<Image>().color = colour;
    return go.GetComponent<Image>();
}
```

```
void LateUpdate()
{
    if (m_Camera == null || m_Card == null) return;

    // Turn to face the reader but never tip over: a card that pitches
    // back when you crouch reads as broken. Yaw only, same rule the
    // Phase 0 panel used.
    var toCamera = m_Camera.transform.position - m_Card.position;
    toCamera.y = 0f;
    if (toCamera.sqrMagnitude > 0.0001f)
        m_Card.rotation = Quaternion.LookRotation(-toCamera.normalized, Vector3.up);

    // The offset follows the card's own right rather than a world axis,
    // so the card stays beside the statue as the reader walks around it
    // instead of swinging in front of it from half the angles.
    m_Card.position = transform.position
                    + Vector3.up * m_Lift
                    + m_Card.right * m_SideStep;
}

// ------------------------------------------------------------ helpers
/// <summary>
/// A decorative shape, built from a built-in mesh rather than
/// GameObject.CreatePrimitive.
///
/// CreatePrimitive always attaches a collider, and on Android the
/// managed stripper had removed CapsuleCollider - nothing in the built
/// scene referenced it, and a CreatePrimitive call is invisible to
/// static analysis. The result was "Can't add component because class
/// 'CapsuleCollider' doesn't exist!" thrown on the device only, every
/// time a bubble was built. These shapes never wanted colliders in the
/// first place, so this asks for none.
/// </summary>
GameObject Shape(string name, string mesh, Vector3 scale, Vector3 localPosition, Color colour)
{
    var go = new GameObject(name, typeof(MeshFilter), typeof(MeshRenderer));
    go.transform.SetParent(transform, false);
    go.transform.localScale = scale;
    go.transform.localPosition = localPosition;
    go.GetComponent<MeshFilter>().sharedMesh = Resources.GetBuiltinResource<Mesh>(mesh);
    Tint(go, colour);
    return go;
}

static void Tint(GameObject go, Color colour)
{
    // Unlit, because the AR scene has no lights and a lit shader renders
    // everything black - a fault that cost a whole test run in Phase 0.
    var material = new Material(Shader.Find("Unlit/Color")) { color = colour };
    go.GetComponent<Renderer>().material = material;
}

static Text Label(RectTransform parent, string text, Font font, int size, Color colour)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var t = go.GetComponent<Text>();
    t.text = text;
```

```
        t.font = font;
        t.fontSize = size;
        t.color = colour;
        t.alignment = TextAnchor.UpperLeft;
        t.horizontalOverflow = HorizontalWrapMode.Wrap;
        t.verticalOverflow = VerticalWrapMode.Truncate;
        t.raycastTarget = false;
        return t;
    }

    static void Place(Graphic g, Vector2 topLeft, Vector2 size)
    {
        var rt = (RectTransform)g.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0f, 1f);
        rt.pivot = new Vector2(0f, 1f);
        rt.anchoredPosition = topLeft;
        rt.sizeDelta = size;
    }

    static Sprite s_Rounded;
    static void Rounded(Image img)
    {
        if (s_Rounded == null)
        {
            const int s = 64, r = 18;
            var tex = new Texture2D(s, s, TextureFormat.RGBA32, false);
            for (var y = 0; y < s; y++)
            for (var x = 0; x < s; x++)
            {
                var dx = Mathf.Max(r - x, x - (s - 1 - r), 0);
                var dy = Mathf.Max(r - y, y - (s - 1 - r), 0);
                tex.SetPixel(x, y, Mathf.Sqrt(dx * dx + dy * dy) <= r ? Color.white : Color.clear);
            }
            tex.Apply();
            tex.wrapMode = TextureWrapMode.Clamp;
            s_Rounded = Sprite.Create(tex, new Rect(0, 0, s, s), new Vector2(0.5f, 0.5f),
                100f, 0, SpriteMeshType.FullRect, new Vector4(r, r, r, r));
        }
        img.sprite = s_Rounded;
        img.type = Image.Type.Sliced;
    }
}
```

4.1.21 AR-021 — StationCalibrator.cs

Code ID	AR-021
File Name	StationCalibrator.cs
File Path	SanctiWalkAR/Assets/Scripts/StationCalibrator.cs
Language / Type	C#
Lines	454
Functions / Classes	StationCalibrator (public class); Bind (public method); SetActive (public method); PositionOf (public method); YawOf (public method); ApplyTo (public method); Select (public method); SelectNext (public method); SelectPrevious (public method); HandleTouch (public method); SetPosition (public method); Nudge (public method); NudgeFromWhereIStand (public method); NudgeYaw (public method); PutWhereIAmStanding (public method); ResetSelected (public method); ResetAll (public method); ToRecording (public method); SyncEditFields (method); DrawNumberFields (public method); CommitNumberFields (public method); DebugReadout (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/StationCalibrator.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StationCalibrator.cs

```
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// Putting each station exactly where the real object is, on site, by hand.
///
/// ## Why this exists, and what it can and cannot fix
///
/// Every station position in this project was surveyed on the MultiSet scan
/// and typed in. The survey itself is sound - the picks round-trip through
/// the frame to 0.3 mm - but the frame those numbers are measured in has to
/// be rebuilt at runtime from a printed marker or two taps on the floor, and
/// THAT is where the error is: measured against the recorded checkpoints,
/// the marker frame put stations 1.18 m to 4.20 m from the real objects.
///
/// Calibrating in the room removes that error, and it is worth being precise
/// about why, because it is not the obvious reason. It does NOT make the
/// origin correct. It makes the origin IRRELEVANT: the admin drags the card
/// until it sits on the real statue while the app is running under whatever
/// origin it produces, and what gets stored is the offset from that origin.
/// Any constant error in the origin is inside that offset, so the next
/// session cancels it out exactly.
///
/// The condition is repeatability, not accuracy. An origin that is three
```

```
/// metres off but three metres off the same way every time calibrates
/// perfectly; an origin that is half a metre off in a different direction
/// each time cannot be calibrated against at all. That is the whole reason
/// the frame id below is stored with the numbers and checked on load.
///
/// ## What is stored
///
/// A position and a yaw per station, in the frame that was in use when SAVE
/// was pressed, written to the same per-parish file the walk-up recorder
/// already writes (StationRecording). Nothing about the station's title,
/// description or artwork is touched - this owns WHERE, and MuseumStations
/// owns WHAT, which is the split the rest of the project already uses.
///
/// Screen coordinates are never stored, and cannot be: everything here is
/// in metres in the session's own frame. Screen size, aspect, resolution
/// and field of view are used only to turn a finger drag into a distance,
/// and none of them survive into the file.
/// </summary>
public class StationCalibrator
{
    /// <summary>How far a full-screen drag moves a station, in metres.</summary>
    const float k_DragMetresPerScreen = 3.0f;

    /// <summary>One tap of a nudge button.</summary>
    const float k_NudgeMetres = 0.05f;
    const float k_NudgeDegrees = 5f;

    readonly Dictionary<string, Vector3> m_Position = new Dictionary<string, Vector3>();
    readonly Dictionary<string, float> m_Yaw = new Dictionary<string, float>();

    MuseumStation[] m_Stations;
    Camera m_Camera;
    int m_Selected;

    /// <summary>True while a finger is dragging the selected station.</summary>
    bool m_Dragging;
    Vector2 m_DragFrom;
    Vector3 m_DragStartedAt;

    /// <summary>Text being typed into the X/Y/Z boxes, kept as text while it is half-typed.</summary>
    string m_EditX = "", m_EditY = "", m_EditZ = "";
    bool m_Editing;

    public bool Active { get; private set; }

    /// <summary>Set when something changed and the world needs rebuilding.</summary>
    public bool Dirty { get; private set; }

    public string Note { get; private set; } = "";

    public MuseumStation Selected =>
        m_Stations != null && m_Stations.Length > 0
            ? m_Stations[Mathf.Clamp(m_Selected, 0, m_Stations.Length - 1)]
            : null;

    /// <summary>
    /// Takes the stations as they currently stand. Called every time the
    /// origin is (re)established, because the numbers being edited are
    /// offsets from that origin and a new origin makes the old edits mean
```

```
/// something different.
/// </summary>
public void Bind(MuseumStation[] stations, Camera camera)
{
    // Which station was being worked on survives a re-bind, matched by
    // id rather than by index. Re-binding happens whenever the origin
    // moves or a save round-trips, and dropping back to station 1 each
    // time would make adjusting the fourth one a fight.
    var was = Selected?.stationId;

    m_Stations = stations;
    m_Camera = camera;
    m_Position.Clear();
    m_Yaw.Clear();
    foreach (var s in stations)
    {
        m_Position[s.stationId] = s.position;
        m_Yaw[s.stationId] = s.yawDegrees;
    }

    m_Selected = 0;
    if (was != null)
    {
        for (var i = 0; i < stations.Length; i++)
            if (stations[i].stationId == was) { m_Selected = i; break; }
    }

    Dirty = false;
    SyncEditFields();
}

public void SetActive(bool active)
{
    Active = active;
    m_Dragging = false;
    m_Editing = false;
    Note = active
        ? "Move the card until it sits on the real object, then SAVE."
        : "";
}

public Vector3 PositionOf(string stationId) =>
    m_Position.TryGetValue(stationId, out var p) ? p : Vector3.zero;

public float YawOf(string stationId) =>
    m_Yaw.TryGetValue(stationId, out var y) ? y : 0f;

/// <summary>Writes the edited values back onto the station table.</summary>
public void ApplyTo(MuseumStation[] stations)
{
    foreach (var s in stations)
    {
        if (m_Position.TryGetValue(s.stationId, out var p)) s.position = p;
        if (m_Yaw.TryGetValue(s.stationId, out var y)) s.yawDegrees = y;
    }
    Dirty = false;
}

public void Select(int index)
```

```
{
    if (m_Stations == null || m_Stations.Length == 0) return;
    m_Selected = ((index % m_Stations.Length) + m_Stations.Length) % m_Stations.Length;
    m_Editing = false;
    Note = $"Editing {Selected.stationName}.";
}

public void SelectNext() => Select(m_Selected + 1);
public void SelectPrevious() => Select(m_Selected - 1);

/// <summary>
/// Drag, in the horizontal plane, in the direction the ADMIN is facing.
///
/// Not in the frame's axes. The frame's X may point any which way in the
/// room, so "drag right moves it along +X" means the card goes sideways,
/// backwards or towards you depending on where you happen to be standing
/// - which is unusable. Mapping the drag to the camera's own right and
/// forward makes it behave the way every 3D tool does: it goes where you
/// pushed it. The numbers stored are still the frame's, because that is
/// what gets converted back here from the camera basis.
/// </summary>
public void HandleTouch(Pose origin, System.Func<Vector3, Vector3> toSession)
{
    if (!Active || m_Camera == null || Selected == null) return;

    // A tap that starts in the top of the screen is the control panel's,
    // not the canvas's. Without this every button press also dragged the
    // station out from under the finger that pressed it.
    const float panelFraction = 0.62f;

    if (Input.touchCount == 1)
    {
        var touch = Input.GetTouch(0);
        var fromTop = Screen.height - touch.position.y;

        if (touch.phase == TouchPhase.Began)
        {
            if (fromTop < Screen.height * panelFraction) return;
            m_Dragging = true;
            m_DragFrom = touch.position;
            m_DragStartedAt = PositionOf(Selected.stationId);
        }
        else if (m_Dragging &&
                 (touch.phase == TouchPhase.Moved || touch.phase == TouchPhase.Stationary))
        {
            var delta = touch.position - m_DragFrom;

            // The camera's own horizontal basis. Projected flat so that
            // looking up or down at the object does not make a sideways
            // drag also change its height.
            var right = Vector3.ProjectOnPlane(m_Camera.transform.right, Vector3.up).normalized;
            var forward = Vector3.ProjectOnPlane(m_Camera.transform.forward, Vector3.up).normalized;
            if (right.sqrMagnitude < 0.01f || forward.sqrMagnitude < 0.01f) return;

            var metresRight = delta.x / Screen.width * k_DragMetresPerScreen;
            var metresForward = delta.y / Screen.height * k_DragMetresPerScreen;
            var sessionShift = right * metresRight + forward * metresForward;

            // Back out of the session and into the frame the numbers are
```

```
            // stored in. Rotation only - a shift is a direction, not a
            // place, so the origin's position must not be subtracted.
            var frameShift = Quaternion.Inverse(origin.rotation) * sessionShift;
            var stored = FromSessionShift(frameShift, toSession);

            SetPosition(Selected.stationId, m_DragStartedAt + stored);
        }
        else if (touch.phase == TouchPhase.Ended || touch.phase == TouchPhase.Canceled)
        {
            m_Dragging = false;
        }
    }
    else
    {
        m_Dragging = false;
    }
}

/// <summary>
/// A shift expressed in the origin's axes, turned into the same shift in
/// the stored frame's axes.
///
/// The three routes store their numbers in three different axis
/// conventions - the marker permutes them, the floor does not, the map
/// has its own - and the converter that expresses that is handed in. It
/// is applied to a DIFFERENCE of two converted points rather than to the
/// shift directly, which is what makes this correct for a converter that
/// translates as well as rotates (the map one does): the translation
/// cancels, leaving only the part that acts on a direction.
/// </summary>
static Vector3 FromSessionShift(Vector3 originAxesShift, System.Func<Vector3, Vector3> toSession)
{
    if (toSession == null) return originAxesShift;

    // Probe the converter along each stored axis to recover how a unit
    // step in the file maps into the origin's axes, then solve for the
    // combination that produces the shift asked for. Three probes, so it
    // costs nothing and needs no knowledge of which converter this is.
    var zero = toSession(Vector3.zero);
    var ex = toSession(Vector3.right) - zero;
    var ey = toSession(Vector3.up) - zero;
    var ez = toSession(Vector3.forward) - zero;

    var basis = new Matrix4x4(ex, ey, ez, new Vector4(0, 0, 0, 1));
    if (Mathf.Abs(basis.determinant) < 1e-6f) return originAxesShift;
    return basis.inverse.MultiplyVector(originAxesShift);
}

public void SetPosition(string stationId, Vector3 position)
{
    m_Position[stationId] = position;
    Dirty = true;
    if (!m_Editing) SyncEditFields();
}

public void Nudge(Vector3 frameAxes)
{
    if (Selected == null) return;
    SetPosition(Selected.stationId, PositionOf(Selected.stationId) + frameAxes * k_NudgeMetres);
```

```
}

/// <summary>
/// LEFT, RIGHT, FORWARD, BACK - from where the admin is standing.
///
/// This is the difference between a usable control and an unusable one,
/// and it is worth spelling out. The stored frame's X and Z point
/// wherever the map happens to point, which has nothing to do with which
/// way the person holding the phone is facing. A button labelled "+X"
/// therefore moves the object left, right, towards you or away depending
/// on where you are standing - so aligning anything becomes trial and
/// error, pressing buttons to see which one happens to help today.
///
/// Mapped to the camera's own horizontal basis, RIGHT always moves it
/// right on screen. The stored number is still the frame's, because the
/// shift is converted back through the same probe the drag uses.
///
/// `screenDirection` is in camera terms: x is right, y is up, z is away.
/// </summary>
public void NudgeFromWhereIStand(Vector3 screenDirection, Pose origin,
                                 System.Func<Vector3, Vector3> toSession)
{
    if (Selected == null || m_Camera == null) return;

    // Height is the one axis that must NOT follow the camera: tilting the
    // phone down to look at something should not turn "up" into
    // "towards me". Gravity is up, always.
    var right = Vector3.ProjectOnPlane(m_Camera.transform.right, Vector3.up).normalized;
    var forward = Vector3.ProjectOnPlane(m_Camera.transform.forward, Vector3.up).normalized;
    if (right.sqrMagnitude < 0.01f || forward.sqrMagnitude < 0.01f) return;

    var sessionShift =
        (right * screenDirection.x + forward * screenDirection.z + Vector3.up * screenDirection.y)
        * k_NudgeMetres;

    var originAxes = Quaternion.Inverse(origin.rotation) * sessionShift;
    var stored = FromSessionShift(originAxes, toSession);

    SetPosition(Selected.stationId, PositionOf(Selected.stationId) + stored);
}

public void NudgeYaw(float sign)
{
    if (Selected == null) return;
    m_Yaw[Selected.stationId] = Mathf.Repeat(YawOf(Selected.stationId) + sign * k_NudgeDegrees, 360f);
    Dirty = true;
}

/// <summary>
/// Puts the station where the admin is standing.
///
/// Kept because for anything you can stand in front of it is both faster
/// and more accurate than dragging - you are placing it by walking to it,
/// with no eyeballing at all. Dragging is for the things you cannot stand
/// inside: an altar, a niche, a statue against a wall.
/// </summary>
public void PutWhereIAmStanding(Pose origin, System.Func<Vector3, Vector3> toSession)
{
    if (Selected == null || m_Camera == null) return;
```

```
    var here = m_Camera.transform.position;
    var inOriginAxes = Quaternion.Inverse(origin.rotation) * (here - origin.position);

    // Same probe as the drag, but applied to a position rather than a
    // direction, so the converter's own offset has to come out too.
    var zero = toSession != null ? toSession(Vector3.zero) : Vector3.zero;
    var stored = FromSessionShift(inOriginAxes - zero, toSession);

    SetPosition(Selected.stationId, stored);
    Note = $"{Selected.stationName} put where you are standing.";
}

/// <summary>Back to the value compiled into the project.</summary>
public void ResetSelected(MuseumStation[] surveyed)
{
    if (Selected == null) return;
    foreach (var s in surveyed)
    {
        if (s.stationId != Selected.stationId) continue;
        m_Position[s.stationId] = s.position;
        m_Yaw[s.stationId] = s.yawDegrees;
        Dirty = true;
        SyncEditFields();
        Note = $"{s.stationName} reset to the surveyed position.";
        return;
    }
}

public void ResetAll(MuseumStation[] surveyed)
{
    foreach (var s in surveyed)
    {
        m_Position[s.stationId] = s.position;
        m_Yaw[s.stationId] = s.yawDegrees;
    }
    Dirty = true;
    SyncEditFields();
    Note = "Every station reset to the surveyed positions.";
}

/// <summary>Everything currently being edited, as a saveable set.</summary>
public RecordedStationSet ToRecording(string parishId, string frameId)
{
    var set = new RecordedStationSet { parishId = parishId, frame = frameId };
    foreach (var s in m_Stations)
        set.Set(s.stationId, PositionOf(s.stationId), YawOf(s.stationId));
    return set;
}

/// <summary>Just the selected one, merged into whatever is already saved.</summary>
public RecordedStationSet ToRecording(string parishId, string frameId, RecordedStationSet existing)
{
    if (Selected == null) return existing;

    // A recording from another frame cannot be merged into - its numbers
    // mean something else, and half a file in each frame is worse than
    // either. Start again rather than mix them.
    var set = existing != null && existing.frame == frameId
```

```
        ? existing
        : new RecordedStationSet { parishId = parishId, frame = frameId };

    set.parishId = parishId;
    set.frame = frameId;
    set.Set(Selected.stationId, PositionOf(Selected.stationId), YawOf(Selected.stationId));
    return set;
}

// ------------------------------------------------------------ numbers

void SyncEditFields()
{
    if (Selected == null) return;
    var p = PositionOf(Selected.stationId);
    m_EditX = p.x.ToString("F3");
    m_EditY = p.y.ToString("F3");
    m_EditZ = p.z.ToString("F3");
}

/// <summary>
/// The three text boxes. Kept as strings while they are being typed:
/// parsing on every keystroke turns "-" and "1." into 0 and snatches the
/// field back from under the finger, which makes a negative number
/// impossible to enter.
/// </summary>
public void DrawNumberFields(GUIStyle field, float width)
{
    if (Selected == null) return;
    if (string.IsNullOrEmpty(m_EditX)) SyncEditFields();

    GUILayout.BeginHorizontal();
    GUILayout.Label("X", field, GUILayout.Width(40));
    m_EditX = GUILayout.TextField(m_EditX, field, GUILayout.Width(width));
    GUILayout.Label("Y", field, GUILayout.Width(40));
    m_EditY = GUILayout.TextField(m_EditY, field, GUILayout.Width(width));
    GUILayout.Label("Z", field, GUILayout.Width(40));
    m_EditZ = GUILayout.TextField(m_EditZ, field, GUILayout.Width(width));
    GUILayout.EndHorizontal();

    m_Editing = GUI.GetNameOfFocusedControl() != "";
}

/// <summary>Commits whatever is in the boxes, ignoring anything unparseable.</summary>
public void CommitNumberFields()
{
    if (Selected == null) return;
    var p = PositionOf(Selected.stationId);
    if (float.TryParse(m_EditX, out var x)) p.x = x;
    if (float.TryParse(m_EditY, out var y)) p.y = y;
    if (float.TryParse(m_EditZ, out var z)) p.z = z;
    m_Editing = false;
    SetPosition(Selected.stationId, p);
    Note = "Typed position applied.";
}

/// <summary>The line the debug readout asks for: name, XYZ, distance.</summary>
public string DebugReadout(Pose origin, System.Func<Vector3, Vector3> toSession)
{
```

```
        if (m_Stations == null) return "";
        var sb = new System.Text.StringBuilder();
        foreach (var s in m_Stations)
        {
            var p = PositionOf(s.stationId);
            var world = origin.position + origin.rotation * (toSession != null ? toSession(p) : p);
            var fromMe = m_Camera != null
                ? Vector3.Distance(m_Camera.transform.position, world)
                : 0f;
            var mark = s == Selected ? ">" : " ";
            sb.AppendLine(
                $"{mark} {s.stationName,-16} x{p.x,7:F3} y{p.y,7:F3} z{p.z,7:F3}" +
                $"   |origin| {p.magnitude,5:F2} m   you {fromMe,5:F2} m");
        }
        return sb.ToString();
    }
}
```

4.1.22 AR-022 — StationMarker.cs

Code ID	AR-022
File Name	StationMarker.cs
File Path	SanctiWalkAR/Assets/Scripts/StationMarker.cs
Language / Type	C#
Lines	120
Functions / Classes	StationMarker (public class); OnDrawGizmos (method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/StationMarker.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StationMarker.cs

```
using UnityEngine;

/// <summary>
/// One AR station, shown in the Scene view as a pin you can drag.
///
/// The transform IS the station position - move the object and the
/// coordinates move with it, which is the whole point. Everything drawn
/// here is a gizmo, so none of it exists in a build: no meshes, no
/// materials, no draw calls, nothing in the APK.
///
/// Coordinates are reported in two frames, because the two are needed for
/// different jobs and confusing them is how stations end up inside walls:
///
///   MAP SPACE     what the MultiSet scan and the reference mesh use, and
///                 what Tools/viewer reports when you click a surface.
///                 This is the Unity world position of this object.
///
///   MARKER SPACE  metres from the printed entrance marker, X across the
///                 wall, Y up, Z into the room. This is what goes into
///                 ChurchOriginTest.surveyed and what the phone uses.
/// </summary>
[ExecuteAlways]
public class StationMarker : MonoBehaviour
{
    [Header("Identity")]
    [Tooltip("Ordering only. Everything else comes from the PointOfInterest.")]
    public int number = 1;

    [Tooltip("Draws an arrow showing which way the visitor is expected to look.")]
    public bool showFacing = true;

    /// <summary>
    /// The POI this pin reports on. StationMarker used to carry its own
```

```
    /// copies of id, name and radius, which meant the pin and the trigger
    /// could quietly disagree. There is now one copy, and it lives there.
    /// </summary>
    public PointOfInterest Poi => m_Poi != null ? m_Poi : m_Poi = GetComponent<PointOfInterest>();
    PointOfInterest m_Poi;

    public string stationId => Poi != null ? Poi.id : "(no PointOfInterest)";
    public string stationName => Poi != null ? Poi.title : name;
    public float triggerRadius => Poi != null ? Poi.triggerRadius : 0f;

    public Color pinColour = new Color(0.16f, 0.42f, 0.90f);

    /// <summary>
    /// Where the printed marker sits in map space. Surveyed on the scan:
    /// the wall behind the TV, 1.407 m above a floor measured at -1.587.
    /// </summary>
    public static readonly Vector3 MarkerInMapSpace = new Vector3(0.870f, -0.144f, -0.037f);

    /// <summary>The wall's facing, 135 degrees, as a direction in the XZ plane.</summary>
    public static readonly Vector3 MarkerNormal = new Vector3(0.4524f, 0.0000f, 0.8918f);

    /// <summary>Floor height in map space, measured off the mesh.</summary>
    public const float FloorY = -1.587f;

    public Vector3 MapSpacePosition => transform.position;

    /// <summary>
    /// The value that belongs in ChurchOriginTest.surveyed: metres from the
    /// marker, X across the wall, Y up, Z into the room.
    /// </summary>
    public Vector3 MarkerSpacePosition
    {
        get
        {
            var up = Vector3.up;
            var right = Vector3.Cross(up, MarkerNormal).normalized;
            var d = transform.position - MarkerInMapSpace;
            return new Vector3(Vector3.Dot(d, right), d.y, Vector3.Dot(d, MarkerNormal));
        }
    }

    public float HeightAboveFloor => transform.position.y - FloorY;

#if UNITY_EDITOR
    void OnDrawGizmos()
    {
        var p = transform.position;

        // The trigger volume, as a translucent sphere plus a floor ring so
        // the footprint is readable from directly above.
        Gizmos.color = new Color(pinColour.r, pinColour.g, pinColour.b, 0.12f);
        Gizmos.DrawSphere(p, triggerRadius);

        UnityEditor.Handles.color = new Color(pinColour.r, pinColour.g, pinColour.b, 0.85f);
        UnityEditor.Handles.DrawWireDisc(new Vector3(p.x, FloorY + 0.01f, p.z), Vector3.up, triggerRadius);

        // A stalk down to the floor, so the pin's actual footprint is
        // unambiguous in a perspective view.
        UnityEditor.Handles.color = new Color(pinColour.r, pinColour.g, pinColour.b, 0.5f);
```

```
        UnityEditor.Handles.DrawDottedLine(p, new Vector3(p.x, FloorY, p.z), 3f);

        Gizmos.color = pinColour;
        Gizmos.DrawSphere(p, 0.08f);

        if (showFacing)
        {
            UnityEditor.Handles.color = pinColour;
            UnityEditor.Handles.ArrowHandleCap(0, p, transform.rotation, 0.6f, EventType.Repaint);
        }

        var marker = MarkerSpacePosition;
        var style = new GUIStyle
        {
            normal = { textColor = Color.white },
            fontSize = 11,
            fontStyle = FontStyle.Bold,
            alignment = TextAnchor.UpperLeft,
        };
        UnityEditor.Handles.Label(p + Vector3.up * 0.30f,
            $"STATION {number}  {stationName.ToUpperInvariant()}\n" +
            $"map    X {p.x:F3}  Y {p.y:F3}  Z {p.z:F3}\n" +
            $"marker X {marker.x:F3}  Y {marker.y:F3}  Z {marker.z:F3}\n" +
            $"floor  +{HeightAboveFloor:F2} m    radius {triggerRadius:F1} m",
            style);
    }
#endif
}
```

4.1.23 AR-023 — StationRecording.cs

Code ID	AR-023
File Name	StationRecording.cs
File Path	SanctiWalkAR/Assets/Scripts/StationRecording.cs
Language / Type	C#
Lines	250
Functions / Classes	RecordedStation (public class); RecordedStationSet (public class); TryGet (public method); Set (public method); StationRecording (public class); PathFor (public method); Load (public method); Save (public method); Apply (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Scripts/StationRecording.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StationRecording.cs

```
using System;
using System.Collections.Generic;
using System.IO;
using UnityEngine;

/// <summary>
/// Where a station's card should appear, recorded by standing there.
///
/// ## Why this exists
///
/// Every station position until now was surveyed on the MultiSet scan and
/// typed into the project, which meant the printed marker had to hang in
/// exactly the place the scan said it was. It never did. The centre of the
/// sheet is not a thing you can point at, so picking it guesses by up to
/// 150 mm on a 180 x 240 mm print; the facing came from one triangle of a
/// photogrammetry mesh, which at the spot chosen swung fifty degrees
/// depending on how much of it you averaged; and every number was copied by
/// hand between three tools. At a church all of that would have to be done
/// again, on a ladder.
///
/// Recording in the room removes the whole chain. The phone writes down
/// where IT is, in the marker's own frame, using the same sensors that will
/// later put the card back. The scan's coordinates stop mattering and the
/// marker can hang wherever is convenient, because nothing is measured
/// against the scan any more - only against the marker, by the device that
/// reads it.
///
/// ## What a recorded position means
///
/// Where the visitor should be STANDING, not where the object is. The
```

```
/// stations trigger on a proximity radius, so the useful question is "where
/// is someone when this card should appear", and that is answerable by
/// walking there. Pointing at an object instead would need ARCore to find a
/// surface on it, which it cannot reliably do on a plain wall or in a dark
/// corner - which is most of a church.
///
/// ## What is recorded and what is not
///
/// Positions only. Titles, descriptions and images stay in
/// MuseumStations, which is the same split the authoring scene and the
/// runtime table already use: one owner for WHERE, another for WHAT.
/// Recording at a church should never risk the copy someone wrote.
/// </summary>
[Serializable]
public class RecordedStation
{
    public string id;
    public Vector3 position;

    /// <summary>
    /// Which way the card faces, in degrees about the frame's up axis.
    ///
    /// Only yaw is stored. A card that is tilted or rolled is a card that
    /// is harder to read, and there is no object in a church whose
    /// signage wants to lean; the two remaining axes would be three more
    /// numbers to get wrong for no gain. Zero means "as the frame faces",
    /// which is what every recording written before this field existed
    /// meant, so old files need no migration.
    /// </summary>
    public float yawDegrees;
}

[Serializable]
public class RecordedStationSet
{
    public string parishId;
    public string recordedUtc;

    /// <summary>
    /// Which coordinate frame these positions are in.
    ///
    /// They used to be metres from the printed marker; they are now metres
    /// from a floor origin set by two taps. The numbers look identical -
    /// three floats - and mean completely different things, so a recording
    /// made under one and replayed under the other lands everything in the
    /// wrong place, with no error and nothing in the log. Which is exactly
    /// the failure this whole change exists to remove, so it is refused
    /// rather than trusted.
    ///
    /// Empty means a file written before this field existed, which can only
    /// be the marker frame.
    /// </summary>
    public string frame = StationRecording.ChurchFrameId;
    public List<RecordedStation> stations = new List<RecordedStation>();

    public bool TryGet(string id, out Vector3 position) => TryGet(id, out position, out _);

    public bool TryGet(string id, out Vector3 position, out float yawDegrees)
    {
```

```
        foreach (var s in stations)
        {
            if (s.id == id)
            {
                position = s.position;
                yawDegrees = s.yawDegrees;
                return true;
            }
        }
        position = default;
        yawDegrees = 0f;
        return false;
    }

    public void Set(string id, Vector3 position) => Set(id, position, 0f);

    public void Set(string id, Vector3 position, float yawDegrees)
    {
        foreach (var s in stations)
        {
            if (s.id == id)
            {
                s.position = position;
                return;
            }
        }
        stations.Add(new RecordedStation { id = id, position = position });
    }
}

public static class StationRecording
{
    /// <summary>Metres from a floor origin, set by two taps.</summary>
    public const string ChurchFrameId = "church-v1";

    /// <summary>Metres from the printed marker. Everything before the floor origin.</summary>
    public const string MarkerFrameId = "marker-v1";

    /// <summary>
    /// Metres from the MultiSet map's own origin.
    ///
    /// The only one of the three that means the same thing twice. A marker
    /// frame is rebuilt from wherever the printed sheet is detected and a
    /// floor frame from wherever two taps land, so both are session-local:
    /// they are repeatable enough to calibrate against and use on the same
    /// phone in the same visit, and they are not the same frame on another
    /// phone tomorrow. The map's origin is fixed in the scan, so a position
    /// stored against it is the one thing here that genuinely transfers.
    /// </summary>
    public const string MapFrameId = "map-v1";

    /// <summary>
    /// One file per parish, under persistentDataPath so it survives
    /// reinstalling the app and can be pulled off with adb:
    ///   adb pull /sdcard/Android/data/com.sanctiwalk.artest/files/
    /// </summary>
    public static string PathFor(string parishId)
    {
        var safe = string.IsNullOrEmpty(parishId) ? "unknown" : parishId;
```

```
    foreach (var c in Path.GetInvalidFileNameChars())
        safe = safe.Replace(c, '-');
    return Path.Combine(Application.persistentDataPath, $"stations-{safe}.json");
}

public static RecordedStationSet Load(string parishId)
{
    var path = PathFor(parishId);
    if (!File.Exists(path))
        return null;

    try
    {
        var set = JsonUtility.FromJson<RecordedStationSet>(File.ReadAllText(path));
        // An empty or malformed file is worse than none: it would silently
        // put every station on the marker.
        if (set == null || set.stations == null || set.stations.Count == 0)
            return null;
        Debug.Log($"[StationRecording] loaded {set.stations.Count} from {path}");
        return set;
    }
    catch (Exception e)
    {
        Debug.LogWarning($"[StationRecording] could not read {path}: {e.Message}");
        return null;
    }
}

public static bool Save(RecordedStationSet set, out string path)
{
    path = PathFor(set.parishId);
    try
    {
        set.recordedUtc = DateTime.UtcNow.ToString("o");
        File.WriteAllText(path, JsonUtility.ToJson(set, true));
        Debug.Log($"[StationRecording] wrote {set.stations.Count} to {path}");
        return true;
    }
    catch (Exception e)
    {
        Debug.LogError($"[StationRecording] could not write {path}: {e.Message}");
        return false;
    }
}

/// <summary>
/// The stations, with any recorded positions swapped in.
///
/// Returns the originals untouched when nothing has been recorded, so a
/// phone that has never been to this church still shows the surveyed
/// positions rather than nothing at all.
/// </summary>
public static MuseumStation[] Apply(MuseumStation[] stations, RecordedStationSet recording,
                                    string expectedFrame = ChurchFrameId)
{
    if (recording == null || stations == null)
        return stations;

    // A file with no frame predates the field, so it is the marker's.
```

```
        var was = string.IsNullOrEmpty(recording.frame) ? MarkerFrameId : recording.frame;
        if (was != expectedFrame)
        {
            Debug.LogWarning(
                $"[StationRecording] ignoring a recording made in the {was} frame while " +
                $"running in {expectedFrame}. Re-record it here; reinterpreting the numbers " +
                "would put every station somewhere plausible and wrong.");
            return stations;
        }

        var applied = new MuseumStation[stations.Length];
        var used = 0;
        for (var i = 0; i < stations.Length; i++)
        {
            if (recording.TryGet(stations[i].stationId, out var where, out var yaw))
            {
                // Copy, never write through. These are references into the
                // shared table.
                applied[i] = stations[i].Copy();
                applied[i].position = where;
                applied[i].yawDegrees = yaw;
                used++;
            }
            else
            {
                // Copied too, not shared.
                //
                // This used to hand back the reference straight out of
                // MuseumStations, so the returned array was part shallow
                // copy and part window onto the compiled table. Anything
                // that edited a position - which is now the whole point of
                // the calibrator - was editing the constant it would later
                // be asked to RESET to, and the reset had nothing true left
                // to go back to. The array a caller is given is the
                // caller's.
                applied[i] = stations[i].Copy();
            }
        }

        Debug.Log($"[StationRecording] {used} of {stations.Length} positions came from the recording");
        return applied;
    }
}
```

4.2 Unity Editor Tooling

4.2.1 AR-024 — ARProjectSetup.cs

Code ID	AR-024
File Name	ARProjectSetup.cs
File Path	SanctiWalkAR/Assets/Editor/ARProjectSetup.cs
Language / Type	C#
Lines	587
Functions / Classes	ARProjectSetup (public class); ConfigureProject (public method); BuildScene (public method); RebuildMarkerLibrary (public method); BuildApk (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/ARProjectSetup.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ARProjectSetup.cs

```
using System.IO;
using UnityEditor;
using UnityEditor.Build;
using UnityEditor.SceneManagement;
using UnityEditor.XR.ARSubsystems;
using UnityEditor.XR.Management;
using UnityEditor.XR.Management.Metadata;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;
using UnityEngine.XR.Management;

/// <summary>
/// One-time project automation for the Phase 0 drift test, callable from the
/// command line so the whole setup is reproducible rather than a list of
/// things somebody clicked once and cannot repeat.
///
/// Run in order:
///   -executeMethod ARProjectSetup.ConfigureProject
///   -executeMethod ARProjectSetup.BuildScene
///   -executeMethod ARProjectSetup.BuildApk
/// </summary>
public static class ARProjectSetup
{
    const string k_ScenePath = "Assets/Scenes/DriftTest.unity";
    const string k_LibraryPath = "Assets/AR/ChurchMarkers.asset";
    const string k_MarkerPath = "Assets/AR/entrance-marker.png";
```

```
/// <summary>
/// Every printed marker the app can recognise, and the name ARCore
/// reports when it sees one.
///
/// The library used to hold only the sala's. A parish marker could be
/// designed, printed and taken to the church, and the app would sit on
/// "Point your camera at the church marker" forever, because the sheet
/// in front of it was not a sheet it had ever been told about.
///
/// The name is what ChurchOriginTest matches on to decide WHICH
/// building it is looking at, so these strings are load-bearing.
/// </summary>
static readonly (string Name, string Path)[] k_Markers =
{
    ("entrance", "Assets/AR/entrance-marker.png"),
    ("mhcp",     "Assets/AR/marker-mhcp.png"),
    ("src",      "Assets/AR/marker-src.png"),
};
const string k_ApkPath = "Builds/SanctiWalkAR-drifttest.apk";

/// <summary>
/// Physical printed width of the marker, in metres. A3 landscape is 0.420.
/// This single number sets the scale of the entire coordinate system — a
/// 5% error here becomes a 5% error at every station. Re-measure the actual
/// print with a ruler and correct it before the church test.
/// </summary>
/// <summary>
/// Physical printed width of the marker, in metres.
///
/// 0.180 m = the 180 mm marker on SanctiWalk-entrance-marker-A4.pdf.
/// This MUST match the sheet actually pinned to the wall. It was 0.420
/// (A3 landscape) while the plan was to print A3; the generated A4 sheet
/// is 180 mm, so this moved with it. If the print scales and the ruler on
/// page 2 does not read 100 mm, change this to the measured width rather
/// than reprinting - either is correct, but they have to agree.
/// </summary>
const float k_MarkerWidthMetres = 0.180f;

/// <summary>
/// THE survey. Metres in the marker's frame: X across, Y up, Z away from
/// the marker into the room or nave.
///
/// This lives here, not on the component, because Unity serialises a
/// component's fields into the scene the moment it is created — after
/// which editing the C# default changes nothing, and the build silently
/// keeps using the old numbers. That already caught us once: the app kept
/// reporting checkpoints at 6/12/18 m after the defaults had been changed
/// to 1/2/3. One source of truth, applied by BuildScene.
///
/// Home dry run: 1, 2, 3 m in a line.
/// Church: replace with the real tape-measured positions, then re-run
/// BuildScene followed by BuildApk.
/// </summary>
/// <summary>
/// The four real stations, in the same order as MuseumStations.Sala.
///
/// This held three placeholders - (0,0,1), (-0.31,0,0.52), (0,0,3) -
/// long after the four real ones existed in C#, and because BuildScene
```

```
/// writes this array into the scene, and a serialized value beats a C#
/// initialiser, those placeholders were what the device actually used.
/// The readout said "CP1 of 3" for weeks and every checkpoint error was
/// measured against a dummy a metre in front of the marker. That is the
/// exact trap the comment above warns about, and it was live in this
/// file while warning about itself.
///
/// Keep in step with MuseumStations.Sala. Tools/show-placement.py
/// prints both and the distances between them.
/// </summary>
static readonly Vector3[] k_Surveyed =
{
    new Vector3(0.585f, -0.826f, 2.707f),   // the sofa
    new Vector3(1.956f, -0.547f, 4.529f),   // santo nino
    new Vector3(-1.901f, -0.321f, 2.585f),  // the shelf
    new Vector3(0.167f, -0.551f, 0.401f),   // the television
};

[MenuItem("SanctiWalk/1 · Configure project for ARCore")]
public static void ConfigureProject()
{
    EditorUserBuildSettings.SwitchActiveBuildTarget(BuildTargetGroup.Android, BuildTarget.Android);

    PlayerSettings.companyName = "SanctiWalk";
    PlayerSettings.productName = "SanctiWalk AR Drift Test";
    PlayerSettings.SetApplicationIdentifier(NamedBuildTarget.Android, "com.sanctiwalk.artest");

    // Android 10. ARCore's own floor is 7.0, but an "ARCore Required" app
    // built with Vulkan is refused below API 29 — the build fails outright
    // with that message. No practical cost: ARCore-certified devices are
    // overwhelmingly Android 10 or newer.
    PlayerSettings.Android.minSdkVersion = AndroidSdkVersions.AndroidApiLevel29;
    PlayerSettings.Android.targetArchitectures = AndroidArchitecture.ARM64;
    PlayerSettings.SetScriptingBackend(NamedBuildTarget.Android, ScriptingImplementation.IL2CPP);

    // Vulkan first, OpenGLES3 as fallback.
    //
    // This started as "OpenGLES3 only, fewest surprises". That was wrong
    // for this hardware and it cost us a diagnosis. On the Dimensity 7020
    // test device the GLES3 path threw GL_INVALID_ENUM every single frame
    // — 549 errors in one short run — and the frame rate collapsed from 30
    // fps to as low as 1.93. ARCore needs a steady 30 fps; when frames
    // arrive erratically the camera has moved too far between them for
    // RANSAC to match features, which showed up as ~200 features found but
    // only 3-13% inliers, and world tracking never started at all.
    //
    // Vulkan removes the erroring GL layer entirely. GLES3 stays second so
    // a device without Vulkan support still has a path.
    PlayerSettings.SetUseDefaultGraphicsAPIs(BuildTarget.Android, false);
    PlayerSettings.SetGraphicsAPIs(BuildTarget.Android, new[]
    {
        UnityEngine.Rendering.GraphicsDeviceType.Vulkan,
        UnityEngine.Rendering.GraphicsDeviceType.OpenGLES3,
    });

    EnableArCoreLoader();
    EnsureUnlitShaderIsIncluded();
    EnableNewInputSystem();
```

```
    AssetDatabase.SaveAssets();
    Debug.Log("[ARProjectSetup] Player settings configured for ARCore.");
}

/// <summary>
/// Guarantee Unlit/Color survives into the build.
///
/// The debug markers are created at runtime, so no scene asset references
/// their shader and Unity is entitled to strip it. A stripped shader in a
/// player renders magenta or nothing at all — another silent failure of
/// exactly the kind that cost us an afternoon with the marker aspect.
/// </summary>
static void EnsureUnlitShaderIsIncluded()
{
    // Unlit/Texture too: the information panel uses it, and it is created
    // at runtime so nothing in the scene keeps it out of the stripper.
    foreach (var name in new[] { "Unlit/Color", "Unlit/Texture" })
        AddAlwaysIncluded(name);
}

static void AddAlwaysIncluded(string shaderName)
{
    var shader = Shader.Find(shaderName);
    if (shader == null)
    {
        Debug.LogWarning($"[ARProjectSetup] {shaderName} not found; markers may not render.");
        return;
    }

    var graphics = AssetDatabase.LoadAllAssetsAtPath("ProjectSettings/GraphicsSettings.asset");
    if (graphics.Length == 0)
        return;

    var so = new SerializedObject(graphics[0]);
    var list = so.FindProperty("m_AlwaysIncludedShaders");
    if (list == null)
        return;

    for (var i = 0; i < list.arraySize; i++)
    {
        if (list.GetArrayElementAtIndex(i).objectReferenceValue == shader)
        {
            Debug.Log($"[ARProjectSetup] {shaderName} already in Always Included Shaders.");
            return;
        }
    }

    list.InsertArrayElementAtIndex(list.arraySize);
    list.GetArrayElementAtIndex(list.arraySize - 1).objectReferenceValue = shader;
    so.ApplyModifiedProperties();
    Debug.Log($"[ARProjectSetup] Added {shaderName} to Always Included Shaders.");
}

/// <summary>
/// Turn the new Input System on. Without it the AR camera never moves.
///
/// This was THE bug. XR Origin ships an Input-System TrackedPoseDriver to
/// apply ARCore's pose to the camera transform, but `-createProject`
/// leaves Active Input Handling on "Input Manager (Old)". Opening the
```

```
/// project in the GUI would have prompted to switch; batch mode never
/// asks, so it silently stayed off.
///
/// The result looked like a tracking failure and was not one: ARCore
/// reported SessionTracking the whole time and produced correct poses,
/// while Unity applied none of them. The camera sat at the origin, the
/// position readout never changed, and every AR object appeared welded to
/// the screen instead of to the room.
///
/// 2 = Both, so legacy Input.* still works alongside it.
///
/// NOTE: Unity applies this on editor restart, so ConfigureProject must
/// run as its own invocation BEFORE the build invocation.
/// </summary>
/// <summary>
/// Force sane import settings on a texture we ship.
///
/// Unity's default NPOT rescale resizes each axis independently to the
/// nearest power of two, which silently changes the aspect ratio. It
/// turned the 1200x1600 marker into 1024x2048, and then the 1600x1000
/// information panel into 2048x1024 - a card built at 8:5 rendering as
/// 2:1. Same defect twice, so it is a helper now rather than a one-off.
/// </summary>
static Texture2D ImportUndistorted(string assetPath, bool readable)
{
    var importer = AssetImporter.GetAtPath(assetPath) as TextureImporter;
    if (importer == null)
        return null;

    if (importer.npotScale != TextureImporterNPOTScale.None
        || importer.isReadable != readable
        || importer.mipmapEnabled
        || importer.textureCompression != TextureImporterCompression.Uncompressed)
    {
        importer.npotScale = TextureImporterNPOTScale.None;
        importer.isReadable = readable;
        importer.mipmapEnabled = false;
        importer.textureCompression = TextureImporterCompression.Uncompressed;
        importer.maxTextureSize = 2048;
        importer.SaveAndReimport();
    }

    return AssetDatabase.LoadAssetAtPath<Texture2D>(assetPath);
}

static void EnableNewInputSystem()
{
    var settings = AssetDatabase.LoadAllAssetsAtPath("ProjectSettings/ProjectSettings.asset");
    if (settings.Length == 0)
        return;

    var so = new SerializedObject(settings[0]);
    var prop = so.FindProperty("activeInputHandler");
    if (prop == null)
    {
        Debug.LogWarning("[ARProjectSetup] activeInputHandler not found.");
        return;
    }
```

```
    if (prop.intValue == 2)
    {
        Debug.Log("[ARProjectSetup] Active Input Handling already 'Both'.");
        return;
    }

    Debug.Log($"[ARProjectSetup] Active Input Handling {prop.intValue} -> 2 (Both). " +
              "Restart required; run the build as a separate invocation.");
    prop.intValue = 2;
    so.ApplyModifiedPropertiesWithoutUndo();
    AssetDatabase.SaveAssets();
}

static void EnableArCoreLoader()
{
    if (!EditorBuildSettings.TryGetConfigObject(XRGeneralSettings.k_SettingsKey,
            out XRGeneralSettingsPerBuildTarget perTarget) || perTarget == null)
    {
        Directory.CreateDirectory("Assets/XR");
        perTarget = ScriptableObject.CreateInstance<XRGeneralSettingsPerBuildTarget>();
        AssetDatabase.CreateAsset(perTarget, "Assets/XR/XRGeneralSettings.asset");
        EditorBuildSettings.AddConfigObject(XRGeneralSettings.k_SettingsKey, perTarget, true);
    }

    perTarget.CreateDefaultManagerSettingsForBuildTarget(BuildTargetGroup.Android);
    var settings = perTarget.SettingsForBuildTarget(BuildTargetGroup.Android);

    if (settings.Manager == null)
    {
        settings.Manager = ScriptableObject.CreateInstance<XRManagerSettings>();
        AssetDatabase.AddObjectToAsset(settings.Manager, perTarget);
    }

    settings.InitManagerOnStart = true;

    var assigned = XRPackageMetadataStore.AssignLoader(
        settings.Manager, "UnityEngine.XR.ARCore.ARCoreLoader", BuildTargetGroup.Android);

    Debug.Log($"[ARProjectSetup] ARCore loader assigned: {assigned}");
    EditorUtility.SetDirty(perTarget);
}

[MenuItem("SanctiWalk/2 · Build the drift-test scene")]
public static void BuildScene()
{
    Directory.CreateDirectory("Assets/Scenes");
    Directory.CreateDirectory("Assets/AR");

    // EVERY panel in Resources, not one named file.
    //
    // Naming a single path meant the second card (Santo Nino) skipped the
    // fix and imported at 1600x800 instead of 1600x1000, so it rendered
    // squashed. The same NPOT defect has now bitten three separate
    // textures; enumerate the folder so the next one cannot slip through.
    foreach (var guid in AssetDatabase.FindAssets("t:Texture2D", new[] { "Assets/Resources" }))
    {
        var path = AssetDatabase.GUIDToAssetPath(guid);
        var tex = ImportUndistorted(path, false);
        if (tex != null)
```

```
        Debug.Log($"[ARProjectSetup] {System.IO.Path.GetFileName(path)}  {tex.width}x{tex.height}");
}

var library = CreateImageLibrary();

var scene = EditorSceneManager.NewScene(NewSceneSetup.EmptyScene, NewSceneMode.Single);

// Unity's own creation code, rather than hand-assembling the hierarchy.
// The XR Origin has a camera offset, a tracked pose driver and input
// wiring that are tedious and easy to get subtly wrong by hand.
EditorApplication.ExecuteMenuItem("GameObject/XR/AR Session");
EditorApplication.ExecuteMenuItem("GameObject/XR/XR Origin (Mobile AR)");

var origin = Object.FindAnyObjectByType<Unity.XR.CoreUtils.XROrigin>();
if (origin == null)
{
    Debug.LogError("[ARProjectSetup] XR Origin was not created — cannot continue.");
    return;
}

var images = origin.gameObject.GetComponent<ARTrackedImageManager>()
             ?? origin.gameObject.AddComponent<ARTrackedImageManager>();
images.referenceLibrary = library;
images.requestedMaxNumberOfMovingImages = 0;   // the marker is fixed to a wall

if (origin.gameObject.GetComponent<ARAnchorManager>() == null)
    origin.gameObject.AddComponent<ARAnchorManager>();

// Plane detection, raycasting and the point cloud: what lets the
// app understand the room before it puts anything in it, and what
// the two-tap floor origin casts against.
//
// ARCore supports horizontal and vertical planes but not arbitrary
// ones, and never classifies them - so the floor is found by
// geometry (a large horizontal-up plane about a person's height
// below the phone) rather than by asking which plane is the floor.
var planes = origin.gameObject.AddComponent<ARPlaneManager>();
planes.requestedDetectionMode = PlaneDetectionMode.Horizontal | PlaneDetectionMode.Vertical;
origin.gameObject.AddComponent<ARRaycastManager>();
origin.gameObject.AddComponent<ARPointCloudManager>();

// SiteReadiness sits with the plane manager it reads. Thresholds
// are const in that file rather than serialized fields, because a
// serialized default written into the scene silently beats whatever
// the C# initialiser says - the same trap that left surveyed[]
// holding three placeholder checkpoints while the code declared
// four real ones.
origin.gameObject.AddComponent<SiteReadiness>();

var runner = new GameObject("DriftTest");
var test = runner.AddComponent<ChurchOriginTest>();
test.surveyed = (Vector3[])k_Surveyed.Clone();
EditorUtility.SetDirty(test);

Debug.Log($"[ARProjectSetup] Survey applied: {k_Surveyed.Length} checkpoints at " +
          string.Join(", ", System.Array.ConvertAll(k_Surveyed, v => $"({v.x},{v.z})")));

EditorSceneManager.SaveScene(scene, k_ScenePath);
EditorBuildSettings.scenes = new[] { new EditorBuildSettingsScene(k_ScenePath, true) };
```

```
    AssetDatabase.SaveAssets();
    Debug.Log($"[ARProjectSetup] Scene written to {k_ScenePath}");
}

static XRReferenceImageLibrary CreateImageLibrary()
{
    var texture = AssetDatabase.LoadAssetAtPath<Texture2D>(k_MarkerPath);
    if (texture == null)
    {
        Debug.LogError($"[ARProjectSetup] No marker image at {k_MarkerPath}.");
        return null;
    }

    // Two importer settings decide whether this works at all.
    //
    // isReadable: ARCore has to read the pixels to extract features.
    // Without it the library builds and simply never recognises anything,
    // with no error to explain why.
    //
    // npotScale: THIS ONE ALREADY BIT US. Unity's default rescales a
    // non-power-of-two image to the nearest powers of two independently
    // per axis — it turned the 1200x1600 marker into 1024x2048, changing
    // the aspect from 3:4 to 1:2. The library then declared the print
    // 0.840 m tall when it is really 0.560 m. Nothing errors; tracking
    // just fails, or succeeds at the wrong scale, and it looks like AR
    // is broken rather than like an import setting.
    var importer = (TextureImporter)AssetImporter.GetAtPath(k_MarkerPath);
    if (!importer.isReadable
        || importer.npotScale != TextureImporterNPOTScale.None
        || importer.mipmapEnabled
        || importer.textureCompression != TextureImporterCompression.Uncompressed)
    {
        importer.isReadable = true;
        importer.npotScale = TextureImporterNPOTScale.None;
        importer.mipmapEnabled = false;
        importer.textureCompression = TextureImporterCompression.Uncompressed;
        importer.maxTextureSize = 2048;
        importer.SaveAndReimport();
        texture = AssetDatabase.LoadAssetAtPath<Texture2D>(k_MarkerPath);
    }

    // Guard, not a comment. If the imported texture's aspect ever drifts
    // from the source file's again, stop rather than print a marker whose
    // declared size is a lie.
    var sourceAspect = SourceAspect(k_MarkerPath);
    var importedAspect = (float)texture.width / texture.height;
    if (sourceAspect > 0f && Mathf.Abs(sourceAspect - importedAspect) > 0.01f)
    {
        Debug.LogError(
            $"[ARProjectSetup] Marker aspect corrupted on import: source {sourceAspect:F4}, " +
            $"imported {importedAspect:F4} ({texture.width}x{texture.height}). " +
            "The declared physical size would be wrong. Fix the importer before printing.");
        return null;
    }

    var library = AssetDatabase.LoadAssetAtPath<XRReferenceImageLibrary>(k_LibraryPath);
    if (library == null)
    {
```

```
        library = ScriptableObject.CreateInstance<XRReferenceImageLibrary>();
        AssetDatabase.CreateAsset(library, k_LibraryPath);
    }

    while (library.count > 0)
        library.RemoveAt(0);

    var added = 0;
    foreach (var marker in k_Markers)
    {
        var tex = marker.Path == k_MarkerPath
            ? texture
            : Prepare(marker.Path);
        if (tex == null)
        {
            // A missing parish marker is not fatal - the sala's is what
            // the test loop needs - but it must be loud, because the
            // failure it causes is "the app ignores my printed sheet"
            // and that is discovered standing in a church.
            Debug.LogWarning($"[ARProjectSetup] No marker image at {marker.Path}; " +
                             $"'{marker.Name}' will NOT be recognised.");
            continue;
        }

        library.Add();
        library.SetTexture(added, tex, true);
        library.SetName(added, marker.Name);
        library.SetSpecifySize(added, true);

        // Every marker is printed at the same width, so its height
        // follows from its own aspect rather than being assumed square.
        var h = k_MarkerWidthMetres * tex.height / tex.width;
        library.SetSize(added, new Vector2(k_MarkerWidthMetres, h));

        Debug.Log($"[ARProjectSetup] marker '{marker.Name}': {tex.width}x{tex.height} px, " +
                  $"{k_MarkerWidthMetres:F3} x {h:F3} m");
        added++;
    }

    EditorUtility.SetDirty(library);
    AssetDatabase.SaveAssets();

    Debug.Log($"[ARProjectSetup] Image library: {added} marker(s) registered.");
    return library;
}

/// <summary>
/// Loads a marker image with the import settings the reference library
/// requires. Same settings the sala's marker already gets - readable, no
/// NPOT scaling, no mipmaps, uncompressed - because a library rejects
/// anything else and does it quietly.
/// </summary>
static Texture2D Prepare(string path)
{
    var tex = AssetDatabase.LoadAssetAtPath<Texture2D>(path);
    if (tex == null) return null;

    var importer = AssetImporter.GetAtPath(path) as TextureImporter;
    if (importer == null) return tex;
```

```
    if (!importer.isReadable
        || importer.npotScale != TextureImporterNPOTScale.None
        || importer.mipmapEnabled
        || importer.textureCompression != TextureImporterCompression.Uncompressed)
    {
        importer.isReadable = true;
        importer.npotScale = TextureImporterNPOTScale.None;
        importer.mipmapEnabled = false;
        importer.textureCompression = TextureImporterCompression.Uncompressed;
        importer.maxTextureSize = 2048;
        importer.SaveAndReimport();
        tex = AssetDatabase.LoadAssetAtPath<Texture2D>(path);
    }
    return tex;
}

/// <summary>
/// Width/height read straight out of the PNG header, so the check above
/// compares against the file on disk rather than against anything Unity
/// has already had a chance to alter.
/// </summary>
static float SourceAspect(string assetPath)
{
    try
    {
        using var stream = File.OpenRead(assetPath);
        var header = new byte[24];
        if (stream.Read(header, 0, 24) < 24)
            return 0f;

        // PNG IHDR: width and height are big-endian uint32 at bytes 16 and 20.
        int width = (header[16] << 24) | (header[17] << 16) | (header[18] << 8) | header[19];
        int height = (header[20] << 24) | (header[21] << 16) | (header[22] << 8) | header[23];
        return height == 0 ? 0f : (float)width / height;
    }
    catch
    {
        return 0f;
    }
}

/// <summary>
/// Rebuilds the printed-marker library and nothing else.
///
/// The library is built by step 2, which also rebuilds the whole scene.
/// So adding a marker meant either rebuilding the scene - and risking
/// everything else in it - or, as actually happened, changing the marker
/// code and shipping an APK where the change had never run. The library
/// asset is edited in place, so the scene's reference to it survives.
/// </summary>
[MenuItem("SanctiWalk/Rebuild marker library")]
public static void RebuildMarkerLibrary()
{
    var library = CreateImageLibrary();
    if (library == null)
    {
        Debug.LogError("[ARProjectSetup] Marker library was NOT rebuilt.");
        return;
```

```
        }
        AssetDatabase.SaveAssets();
        AssetDatabase.Refresh();
        Debug.Log($"[ARProjectSetup] Marker library rebuilt: {library.count} image(s).");
    }

    [MenuItem("SanctiWalk/3 · Build APK")]
    public static void BuildApk()
    {
        Directory.CreateDirectory("Builds");

        var report = BuildPipeline.BuildPlayer(new BuildPlayerOptions
        {
            scenes = new[] { k_ScenePath },
            locationPathName = k_ApkPath,
            target = BuildTarget.Android,
            targetGroup = BuildTargetGroup.Android,
            options = BuildOptions.None,
        });

        var summary = report.summary;
        Debug.Log($"[ARProjectSetup] Build {summary.result}: " +
                  $"{summary.totalSize / (1024 * 1024)} MB in {summary.totalTime.TotalSeconds:F0}s → {k_ApkPath}");

        if (summary.result != UnityEditor.Build.Reporting.BuildResult.Succeeded)
            EditorApplication.Exit(1);
    }
}
```

4.2.2 AR-025 — ArDeepLink.cs

Code ID	AR-025
File Name	ArDeepLink.cs
File Path	SanctiWalkAR/Assets/Editor/ArDeepLink.cs
Language / Type	C#
Lines	179
Functions / Classes	ArDeepLink (public class); OnPostGenerateGradleAndroidProject (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/ArDeepLink.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ArDeepLink.cs

```
using System.IO;
using System.Xml;
using UnityEditor.Android;

/// <summary>
/// Registers the sanctiwalkar:// URL scheme on the Unity activity, so the
/// SanctiWalk web app can launch the AR tour with a link.
///
/// Done by editing the manifest Unity generates, rather than by dropping an
/// Assets/Plugins/Android/AndroidManifest.xml into the project. That file is
/// a wholesale override: it would replace the manifest ARCore's own build
/// processor writes, silently losing the camera permission and the
/// com.google.ar.core metadata, and the app would install and then fail to
/// start AR. Editing the generated file leaves all of that intact.
/// </summary>
public class ArDeepLink : IPostGenerateGradleAndroidProject
{
    // After ARCore's own manifest processor, so its additions are already in.
    public int callbackOrder => 500;

    const string k_Scheme = "sanctiwalkar";

    /// <summary>
    /// Whether the AR app keeps its own icon in the app drawer.
    ///
    /// False is what ships: SanctiWalk and the AR tour are two APKs for a
    /// technical reason - the browser cannot anchor content to a place the
    /// way ARCore can - but that is our problem, not the visitor's. Two
    /// icons invite someone to open the AR app directly, where it has no
    /// parish, no idea which church they are in, and nothing useful to do.
    ///
    /// Set true for a development build when starting the AR app by hand is
    /// more convenient than typing an adb deep link.
```

```
/// </summary>
const bool k_KeepLauncherIcon = false;
const string k_Host = "tour";
const string k_AndroidNs = "http://schemas.android.com/apk/res/android";

public void OnPostGenerateGradleAndroidProject(string projectPath)
{
    var manifestPath = Path.Combine(projectPath, "src", "main", "AndroidManifest.xml");
    if (!File.Exists(manifestPath))
    {
        UnityEngine.Debug.LogWarning($"[ArDeepLink] No manifest at {manifestPath}");
        return;
    }

    var doc = new XmlDocument();
    doc.Load(manifestPath);

    var activity = FindLauncherActivity(doc);
    if (activity == null)
    {
        UnityEngine.Debug.LogWarning("[ArDeepLink] Launcher activity not found; no deep link added.");
        return;
    }

    // Two independent changes, and the manifest is saved once at the
    // end. This used to return early when the deep link was already
    // present, which also skipped everything after it - so a second
    // change added here would silently never run on most builds.
    var changed = false;

    if (HasOurFilter(activity))
    {
        UnityEngine.Debug.Log("[ArDeepLink] Deep link already present.");
    }
    else
    {
        var filter = doc.CreateElement("intent-filter");

        AppendWithName(doc, filter, "action", "android.intent.action.VIEW");
        AppendWithName(doc, filter, "category", "android.intent.category.DEFAULT");
        AppendWithName(doc, filter, "category", "android.intent.category.BROWSABLE");

        var data = doc.CreateElement("data");
        data.SetAttribute("scheme", k_AndroidNs, k_Scheme);
        data.SetAttribute("host", k_AndroidNs, k_Host);
        filter.AppendChild(data);

        activity.AppendChild(filter);
        changed = true;
        UnityEngine.Debug.Log($"[ArDeepLink] Registered {k_Scheme}://{k_Host}");
    }

    if (!k_KeepLauncherIcon && HideFromLauncher(activity))
    {
        changed = true;
        UnityEngine.Debug.Log("[ArDeepLink] Launcher icon removed; the AR app is " +
                              "reachable only through SanctiWalk's deep link.");
    }
```

```
    if (changed)
        doc.Save(manifestPath);
}

static void AppendWithName(XmlDocument doc, XmlElement parent, string tag, string name)
{
    var element = doc.CreateElement(tag);
    element.SetAttribute("name", k_AndroidNs, name);
    parent.AppendChild(element);
}

/// <summary>
/// Takes the launcher category off the activity, so Android stops
/// listing the AR app as its own app.
///
/// Only the CATEGORY goes. The filter itself and its MAIN action stay,
/// as does our sanctiwalkar:// filter, so the activity is still
/// perfectly launchable - just not from the app drawer. Removing the
/// whole filter would also work but is a bigger change to a generated
/// manifest than it needs to be.
/// </summary>
static bool HideFromLauncher(XmlElement activity)
{
    foreach (XmlNode child in activity.ChildNodes)
    {
        if (child.Name != "intent-filter")
            continue;

        XmlNode launcher = null;
        foreach (XmlNode entry in child.ChildNodes)
        {
            var name = entry.Attributes?["android:name"]?.Value;
            if (entry.Name == "category" && name == "android.intent.category.LAUNCHER")
            {
                launcher = entry;
                break;
            }
        }

        if (launcher != null)
        {
            child.RemoveChild(launcher);
            return true;
        }
    }

    return false;
}

static XmlElement FindLauncherActivity(XmlDocument doc)
{
    foreach (XmlNode activity in doc.GetElementsByTagName("activity"))
    {
        foreach (XmlNode child in activity.ChildNodes)
        {
            if (child.Name != "intent-filter")
                continue;

            foreach (XmlNode entry in child.ChildNodes)
```

```
                {
                    var name = entry.Attributes?["android:name"]?.Value;
                    if (entry.Name == "category" && name == "android.intent.category.LAUNCHER")
                        return activity as XmlElement;
                }
            }
        }

        return null;
    }

    static bool HasOurFilter(XmlElement activity)
    {
        foreach (XmlNode child in activity.ChildNodes)
        {
            if (child.Name != "intent-filter")
                continue;

            foreach (XmlNode entry in child.ChildNodes)
            {
                if (entry.Name == "data" && entry.Attributes?["android:scheme"]?.Value == k_Scheme)
                    return true;
            }
        }

        return false;
    }
}
```

4.2.3 AR-026 — AuthoringPreview.cs

Code ID	AR-026
File Name	AuthoringPreview.cs
File Path	SanctiWalkAR/Assets/Editor/AuthoringPreview.cs
Language / Type	C#
Lines	123
Functions / Classes	AuthoringPreview (public class); Render (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/AuthoringPreview.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AuthoringPreview.cs

```
using System.IO;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

/// <summary>
/// Renders the authoring scene to PNGs, so the visualisation can be
/// checked without opening the editor by hand.
/// </summary>
public static class AuthoringPreview
{
    const int W = 1400, H = 950;

    [MenuItem("SanctiWalk/Stations/Render Authoring Preview")]
    public static void Render()
    {
        EditorSceneManager.OpenScene("Assets/Scenes/StationAuthoring.unity", OpenSceneMode.Single);

        var camGo = new GameObject("PreviewCam", typeof(Camera));
        var cam = camGo.GetComponent<Camera>();
        cam.clearFlags = CameraClearFlags.SolidColor;
        cam.backgroundColor = new Color32(0x1B, 0x1E, 0x25, 0xFF);
        cam.fieldOfView = 45f;
        cam.nearClipPlane = 0.05f;
        cam.farClipPlane = 200f;

        var dir = "Builds/UIShots";
        Directory.CreateDirectory(dir);

        // Down into the room, which is the view the ceiling cut makes possible.
        Shot(cam, new Vector3(2.4f, 7.2f, -2.4f), new Vector3(2.2f, -1.2f, 3.6f), dir, "editor-1-overview");

        // Low, over the floor, roughly where a visitor stands.
```

```
    Shot(cam, new Vector3(4.6f, 0.4f, -0.4f), new Vector3(0.8f, -0.9f, 4.6f), dir, "editor-2-eye-level");

    // The TV wall with the marker on it.
    Shot(cam, new Vector3(1.9f, 0.2f, 2.6f), new Vector3(0.14f, -0.35f, 5.35f), dir, "editor-3-marker-wall");

    // What the player actually sees on Play, from the spawn, plus the
    // checks that say whether that spawn is standable.
    VerifySpawn(cam, dir);

    Object.DestroyImmediate(camGo);
    Debug.Log($"[AuthoringPreview] wrote previews to {dir}");
}

/// <summary>
/// Confirms the spawn is legal, rather than assuming it. Three
/// questions: is the capsule overlapping the scan, is there floor
/// underneath it, and does the camera see the room from there.
/// </summary>
static void VerifySpawn(Camera cam, string dir)
{
    var player = Object.FindFirstObjectByType<FirstPersonWalkthrough>();
    if (player == null) { Debug.LogError("[AuthoringPreview] no player in the scene"); return; }

    var controller = player.GetComponent<CharacterController>();
    var centre = player.transform.position;
    // Inside the controller's own volume and lifted off the floor, or
    // the floor itself counts as an obstruction.
    var feetY = centre.y - controller.height * 0.5f;
    var bottom = new Vector3(centre.x, feetY + controller.radius + 0.06f, centre.z);
    var top = new Vector3(centre.x, feetY + controller.height - controller.radius, centre.z);

    // OverlapCapsule rather than CheckCapsule, and the player's own
    // CharacterController is filtered out: the spawn search runs before
    // the player exists, so a bare check here always found the capsule
    // itself and reported every position as blocked.
    var hits = Physics.OverlapCapsule(bottom, top, controller.radius * 0.95f,
                                      ~0, QueryTriggerInteraction.Ignore);
    var foreign = System.Array.FindAll(hits, c => c.transform != player.transform);
    if (foreign.Length == 0)
    {
        Debug.Log("[VerifySpawn] spawn is clear - capsule touches no scan geometry");
    }
    else
    {
        foreach (var c in foreign)
            Debug.LogWarning($"[VerifySpawn] BLOCKED by '{c.name}' ({c.GetType().Name})");
    }

    var feet = centre - Vector3.up * controller.height * 0.5f;
    if (Physics.Raycast(feet + Vector3.up * 0.05f, Vector3.down, out var hit, 6f,
                        ~0, QueryTriggerInteraction.Ignore))
        Debug.Log($"[VerifySpawn] floor {hit.distance:F3} m below the feet, at y {hit.point.y:F3} " +
                  $"(measured floor {StationMarker.FloorY:F3})");
    else
        Debug.LogError("[VerifySpawn] NO FLOOR under the spawn - the player would fall");

    var eye = player.GetComponentInChildren<Camera>();
    var eyePos = eye.transform.position;
    Debug.Log($"[VerifySpawn] eye at ({eyePos.x:F2}, {eyePos.y:F2}, {eyePos.z:F2}), " +
```

```
                $"{eyePos.y - StationMarker.FloorY:F2} m above floor");

        foreach (var m in Object.FindObjectsByType<StationMarker>(FindObjectsInactive.Exclude))
            Debug.Log($"[VerifySpawn] station {m.number} {m.stationName}: " +
                      $"{Vector3.Distance(eyePos, m.transform.position):F2} m away, radius {m.triggerRadius:F1}");

        Shot(cam, eyePos, eyePos + StationMarker.MarkerNormal * 4f, dir, "editor-4-play-spawn");
    }

    static void Shot(Camera cam, Vector3 from, Vector3 lookAt, string dir, string name)
    {
        cam.transform.position = from;
        cam.transform.LookAt(lookAt);

        var rt = new RenderTexture(W, H, 24, RenderTextureFormat.ARGB32) { antiAliasing = 4 };
        cam.targetTexture = rt;
        cam.Render();

        var previous = RenderTexture.active;
        RenderTexture.active = rt;
        var shot = new Texture2D(W, H, TextureFormat.RGBA32, false);
        shot.ReadPixels(new Rect(0, 0, W, H), 0, 0);
        shot.Apply();
        RenderTexture.active = previous;

        File.WriteAllBytes(Path.Combine(dir, name + ".png"), shot.EncodeToPNG());
        cam.targetTexture = null;
        Object.DestroyImmediate(rt);
        Object.DestroyImmediate(shot);
    }
}
```

4.2.4 AR-027 — DesktopTestBuild.cs

Code ID	AR-027
File Name	DesktopTestBuild.cs
File Path	SanctiWalkAR/Assets/Editor/DesktopTestBuild.cs
Language / Type	C#
Lines	60
Functions / Classes	DesktopTestBuild (public class); BuildWalkthrough (public method); Build (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/DesktopTestBuild.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — DesktopTestBuild.cs

```
using System.IO;
using UnityEditor;
using UnityEditor.Build.Reporting;
using UnityEngine;

/// <summary>
/// Builds the AR scene as a Windows application so the flow can be walked
/// on a laptop with no phone, no marker and no ARCore.
///
/// The AR session simply never starts on a desktop, which is why the scene
/// falls straight to the scan screen with the Simulate Scan button on it.
/// Everything downstream of SetOrigin is platform-independent, so the
/// stations, bubbles, trigger radii and next-stop chain behave exactly as
/// they do on the phone.
/// </summary>
public static class DesktopTestBuild
{
    [MenuItem("SanctiWalk/Build Windows Walkthrough (sala)")]
    public static void BuildWalkthrough()
    {
        Build("Assets/Scenes/StationAuthoring.unity", "Builds/SalaWalkthrough", "SalaWalkthrough.exe");
    }

    [MenuItem("SanctiWalk/Build Windows Test App")]
    public static void Build()
    {
        Build("Assets/Scenes/DriftTest.unity", "Builds/WindowsTest", "SanctiWalkTest.exe");
    }

    static void Build(string scene, string folder, string exe)
    {
        var dir = folder;
        Directory.CreateDirectory(dir);
```

```
        var options = new BuildPlayerOptions
        {
            scenes = new[] { scene },
            locationPathName = Path.Combine(dir, exe),
            target = BuildTarget.StandaloneWindows64,
            targetGroup = BuildTargetGroup.Standalone,
            options = BuildOptions.Development,
        };

        // Windowed and phone-shaped, so the layout is judged at roughly the
        // proportions it will actually be read at.
        PlayerSettings.defaultScreenWidth = 450;
        PlayerSettings.defaultScreenHeight = 950;
        PlayerSettings.fullScreenMode = FullScreenMode.Windowed;
        PlayerSettings.resizableWindow = true;
        PlayerSettings.runInBackground = true;

        var report = BuildPipeline.BuildPlayer(options);
        var summary = report.summary;

        if (summary.result == BuildResult.Succeeded)
            Debug.Log($"[DesktopTestBuild] Succeeded: {summary.totalSize / 1048576} MB -> {options.locationPathName}");
        else
            Debug.LogError($"[DesktopTestBuild] {summary.result}: {summary.totalErrors} error(s)");
    }
}
```

4.2.5 AR-028 — ExportPois.cs

Code ID	AR-028
File Name	ExportPois.cs
File Path	SanctiWalkAR/Assets/Editor/ExportPois.cs
Language / Type	C#
Lines	247
Functions / Classes	ExportPois (public class); ExportMarker (public method); Export (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/ExportPois.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ExportPois.cs

```
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Text.RegularExpressions;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;
using UnityEngine.SceneManagement;

/// <summary>
/// Copies the POI positions out of StationAuthoring.unity and into the
/// runtime table in MuseumStation.cs.
///
/// ## Why this exists
///
/// It was the missing link. StationAuthoring.unity is where POIs are placed,
/// and it is the only place you can see them against the scanned room - but
/// it is not in the build, and nothing read it. The device gets its stations
/// from MuseumStations.Sala, a hand-written array. The two matched only
/// because someone once typed one to agree with the other, and nothing kept
/// them that way: drag a POI in the scene, rebuild, and the phone showed the
/// old position with no warning that it had ignored you.
///
/// Now the scene is the source of truth for WHERE, and the array stays the
/// source of truth for WHAT - the titles, the descriptions, the images. This
/// rewrites the position lines and touches nothing else, so writing station
/// copy and placing stations stay separate jobs.
///
/// ## Coordinates
///
/// The scene holds map space, straight off the MultiSet scan. The runtime
/// wants metres from the printed marker. The conversion is
/// StationMarker.MarkerSpacePosition's, used through the same constants, so
```

```
/// there is no second copy of that arithmetic to fall out of step.
///
/// It refuses rather than half-writing: an id in the scene with no entry in
/// the table, or an entry no POI claims, stops the whole export. A silent
/// partial write is how the two drifted apart in the first place.
/// </summary>
public static class ExportPois
{
    const string k_ScenePath = "Assets/Scenes/StationAuthoring.unity";
    const string k_TablePath = "Assets/Scripts/MuseumStation.cs";

    const string k_MarkerPath = "Assets/Scripts/StationMarker.cs";

    /// <summary>
    /// Writes the marker's pose back out of the authoring scene.
    ///
    /// ## Why this is the important one
    ///
    /// MarkerInMapSpace and MarkerNormal were surveyed by picking points on
    /// the scan in a web viewer, and both picks are weak in a way that
    /// cannot be improved by picking more carefully. The position is the
    /// CENTRE of a 180x240mm printed sheet - there is nothing at the centre
    /// to aim at, so the pick is a guess of up to 150mm. The facing came
    /// from the normal of one triangle of a photogrammetry mesh, and at that
    /// spot the surface normal swings 20.9, 33.0, 10.8 and 338.8 degrees
    /// depending on how much of it you average.
    ///
    /// Every station is measured FROM this pose, so its error is added to
    /// all of them. Dragging each station until it looks right hides that,
    /// one station at a time, until the sheet is re-hung and they are all
    /// wrong again.
    ///
    /// Placing the marker object on the scanned wall in the editor fixes it
    /// at the source: you can see the wall, orbit it, and put the sheet
    /// where the real one hangs, with the mesh to line it up against rather
    /// than a single click in a viewer.
    ///
    /// The normal is taken from the object's own forward, flattened into the
    /// XZ plane and re-normalised. Flattened because the wall is vertical and
    /// a marker tilted a degree or two by hand would otherwise write a
    /// normal that tips the whole room's frame; the runtime treats this as a
    /// horizontal facing, so storing anything else is storing a number that
    /// will not be used as written.
    /// </summary>
    [MenuItem("SanctiWalk/Export the MARKER pose from the authoring scene")]
    public static void ExportMarker()
    {
        var scene = EditorSceneManager.GetActiveScene();
        if (!scene.path.EndsWith("StationAuthoring.unity"))
        {
            EditorUtility.DisplayDialog("Wrong scene",
                "Open Assets/Scenes/StationAuthoring.unity first.", "OK");
            return;
        }

        var root = GameObject.Find("EntranceMarker (origin)");
        if (root == null)
        {
            EditorUtility.DisplayDialog("No marker",
```

```
            "The scene has no 'EntranceMarker (origin)'. Rebuild it from\n" +
            "SanctiWalk > Stations > Build Authoring Scene.", "OK");
        return;
    }

    var position = root.transform.position;

    var normal = root.transform.forward;
    normal.y = 0f;
    if (normal.sqrMagnitude < 1e-6f)
    {
        EditorUtility.DisplayDialog("Marker is facing straight up or down",
            "Rotate it so it faces out of the wall, then export again.", "OK");
        return;
    }
    normal.Normalize();

    var source = File.ReadAllText(k_MarkerPath);

    var pos = $"new Vector3({position.x:0.###}f, {position.y:0.###}f, {position.z:0.###}f)";
    var nor = $"new Vector3({normal.x:0.####}f, {normal.y:0.####}f, {normal.z:0.####}f)";

    var before = source;
    source = Regex.Replace(source,
        @"MarkerInMapSpace\s*=\s*new Vector3\([^)]*\)",
        "MarkerInMapSpace = " + pos);
    source = Regex.Replace(source,
        @"MarkerNormal\s*=\s*new Vector3\([^)]*\)",
        "MarkerNormal = " + nor);

    if (source == before)
    {
        EditorUtility.DisplayDialog("Nothing written",
            "Could not find MarkerInMapSpace / MarkerNormal in StationMarker.cs.", "OK");
        return;
    }

    File.WriteAllText(k_MarkerPath, source);
    AssetDatabase.Refresh();

    Debug.Log($"[ExportPois] marker -> {pos}  normal {nor}");
    EditorUtility.DisplayDialog("Marker exported",
        $"Position {position}\nFacing   {normal}\n\n" +
        "Now run SanctiWalk > Export POIs, because every station is\n" +
        "measured from this pose and they all moved with it.", "OK");
}

[MenuItem("SanctiWalk/Export POIs from the authoring scene")]
public static void Export()
{
    var scene = SceneManager.GetActiveScene();
    var reopen = scene.path != k_ScenePath;
    if (reopen)
    {
        if (!EditorSceneManager.SaveCurrentModifiedScenesIfUserWantsTo())
        {
            Debug.Log("[ExportPois] Cancelled.");
            return;
        }
```

```
    EditorSceneManager.OpenScene(k_ScenePath, OpenSceneMode.Single);
}
else if (scene.isDirty)
{
    // Unsaved drags are exactly what you meant to export, so take
    // them - but the file has to agree with what was exported or the
    // next person reading the scene sees something else.
    EditorSceneManager.SaveScene(scene);
}

var pois = Object.FindObjectsByType<PointOfInterest>(FindObjectsSortMode.None);
if (pois.Length == 0)
{
    Debug.LogError($"[ExportPois] No PointOfInterest in {k_ScenePath}.");
    return;
}

var source = File.ReadAllText(k_TablePath);
var ids = new Regex("stationId\\s*=\\s*\"([^\"]+)\"")
    .Matches(source).Cast<Match>().Select(m => m.Groups[1].Value).ToList();

// --- refuse a half-write ------------------------------------------
var sceneIds = pois.Select(p => p.id).ToList();
var orphans = sceneIds.Where(id => !ids.Contains(id)).ToList();
var missing = ids.Where(id => !sceneIds.Contains(id)).ToList();
if (orphans.Count > 0 || missing.Count > 0)
{
    if (orphans.Count > 0)
        Debug.LogError("[ExportPois] In the scene but not in the table: " +
                       string.Join(", ", orphans));
    if (missing.Count > 0)
        Debug.LogError("[ExportPois] In the table but no POI placed for it: " +
                       string.Join(", ", missing));
    Debug.LogError("[ExportPois] Nothing written. Make the ids agree and run again.");
    return;
}

// --- rewrite the position lines -----------------------------------
var changed = new List<string>();
foreach (var poi in pois.OrderBy(p => ids.IndexOf(p.id)))
{
    var marker = MarkerSpace(poi.transform.position);
    var written = $"position = new Vector3({marker.x:0.000}f, {marker.y:0.000}f, {marker.z:0.000}f)";

    // Bound the search to this entry, so a replacement cannot walk
    // forward into the next station's position line.
    var start = source.IndexOf($"stationId = \"{poi.id}\"", System.StringComparison.Ordinal);
    if (start < 0)
    {
        Debug.LogError($"[ExportPois] Could not find stationId \"{poi.id}\". Nothing written.");
        return;
    }
    var next = source.IndexOf("stationId = \"", start + 1, System.StringComparison.Ordinal);
    var end = next < 0 ? source.Length : next;

    var slice = source.Substring(start, end - start);

    // Look for the line, rather than inferring it from whether the
    // text changed: a POI that has not moved rewrites to the same
```

```
            // characters, and "nothing changed" would then be read as
            // "nothing found" and refuse a perfectly good export.
            var found = Regex.Match(slice, @"position\s*=\s*new Vector3\(([^)]*)\)");
            if (!found.Success)
            {
                Debug.LogError($"[ExportPois] No position line inside \"{poi.id}\". Nothing written.");
                return;
            }

            var replaced = Regex.Replace(
                slice, @"position\s*=\s*new Vector3\([^)]*\)", written, RegexOptions.None);
            var was = found.Groups[1].Value;
            changed.Add($"  {poi.id,-18} {was}  ->  {marker.x:0.000}f, {marker.y:0.000}f, {marker.z:0.000}f");

            source = source.Substring(0, start) + replaced + source.Substring(end);
        }

        File.WriteAllText(k_TablePath, source);
        AssetDatabase.ImportAsset(k_TablePath);

        Debug.Log($"[ExportPois] Wrote {pois.Length} positions to {k_TablePath}:\n" +
                  string.Join("\n", changed) +
                  "\n\nRebuild the APK for these to reach the phone.");
    }

    /// <summary>
    /// Map space to metres from the marker, through StationMarker's own
    /// constants so this cannot disagree with MarkerSpacePosition.
    /// </summary>
    static Vector3 MarkerSpace(Vector3 mapSpace)
    {
        var up = Vector3.up;
        var right = Vector3.Cross(up, StationMarker.MarkerNormal).normalized;
        var d = mapSpace - StationMarker.MarkerInMapSpace;
        return new Vector3(Vector3.Dot(d, right), d.y, Vector3.Dot(d, StationMarker.MarkerNormal));
    }
}
```

4.2.6 AR-029 — ModelCheck.cs

Code ID	AR-029
File Name	ModelCheck.cs
File Path	SanctiWalkAR/Assets/Editor/ModelCheck.cs
Language / Type	C#
Lines	66
Functions / Classes	ModelCheck (public class); Run (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/ModelCheck.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ModelCheck.cs

```
using UnityEditor;
using UnityEngine;

/// <summary>
/// Reports what Unity actually made of the station models.
///
/// The statues shipped twice looking like broken glass, and both times the
/// file on disk looked correct - once because the OBJ carried no normals at
/// all, once because it carried one vertex per triangle corner so no two
/// faces could share one. Neither was visible from the file size or from a
/// three.js preview; both were plain in the imported mesh.
///
/// So this asks Unity directly, the same way the app does
/// (Resources.Load by name), and prints what came back.
///
///   Unity.exe -quit -batchmode -projectPath . -executeMethod ModelCheck.Run
/// </summary>
public static class ModelCheck
{
    static readonly string[] k_Models =
    {
        "mary-help", "santo-nino-pajotan", "san-roque", "sanctiwalk-logo",
    };

    public static void Run()
    {
        foreach (var name in k_Models)
        {
            var prefab = Resources.Load<GameObject>("Models/" + name);
            if (prefab == null)
            {
                Debug.Log($"[ModelCheck] {name}: NOT FOUND under Resources/Models");
                continue;
```

```
            }

            var filters = prefab.GetComponentsInChildren<MeshFilter>();
            int verts = 0, tris = 0, withNormals = 0, withUv = 0;
            var bounds = new Bounds();
            var first = true;

            foreach (var f in filters)
            {
                var m = f.sharedMesh;
                if (m == null) continue;
                verts += m.vertexCount;
                tris += m.triangles.Length / 3;
                if (m.normals != null && m.normals.Length == m.vertexCount) withNormals++;
                if (m.uv != null && m.uv.Length == m.vertexCount) withUv++;
                if (first) { bounds = m.bounds; first = false; }
                else bounds.Encapsulate(m.bounds);
            }

            // A mesh whose vertex count is three times its triangle count has
            // no shared vertices: every face stands alone, which is both the
            // faceted look and five times the memory.
            var ratio = tris > 0 ? verts / (float)tris : 0f;
            var shared = ratio < 2.0f ? "SHARED (good)" : "UNWELDED (every face alone)";

            Debug.Log($"[ModelCheck] {name}: {verts:N0} verts, {tris:N0} tris, " +
                      $"verts/tris {ratio:F2} -> {shared}; " +
                      $"normals on {withNormals}/{filters.Length} meshes, " +
                      $"uv on {withUv}/{filters.Length}; " +
                      $"size {bounds.size.x:F2} x {bounds.size.y:F2} x {bounds.size.z:F2} m");
        }
    }
}
```

4.2.7 AR-030 — MultiSetCredentials.cs

Code ID	AR-030
File Name	MultiSetCredentials.cs
File Path	SanctiWalkAR/Assets/Editor/MultiSetCredentials.cs
Language / Type	C#
Lines	65
Functions / Classes	MultiSetCredentials (public class); Stage (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/MultiSetCredentials.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MultiSetCredentials.cs

```
using System.IO;
using UnityEditor;
using UnityEngine;

/// <summary>
/// Copies Tools/multiset.env into Assets/Resources so the player can read
/// it, because Resources is the only place a plain file reliably survives
/// into an APK.
///
/// Tools/multiset.env is gitignored and stays the single source. The copy
/// is gitignored too - the .gitignore entry goes in at the same time as the
/// copy, so it cannot be committed by someone who did not know it appears.
///
/// Be clear about what this does and does not protect. It keeps the secret
/// out of source control and out of any source file, which is worth doing.
/// It does NOT keep it out of the APK: anyone who unpacks the build can
/// read it. That is acceptable for a test build on the owner's own account
/// and is not acceptable for a released one, where the secret belongs on a
/// server and the app should call the server.
/// </summary>
public static class MultiSetCredentials
{
    const string k_Source = "Tools/multiset.env";
    const string k_Dest = "Assets/Resources/multiset-credentials.txt";
    const string k_Ignore = ".gitignore";
    const string k_IgnoreLine = "Assets/Resources/multiset-credentials.txt";

    [MenuItem("SanctiWalk/Stage MultiSet credentials into the build")]
    public static void Stage()
    {
        var root = Directory.GetParent(Application.dataPath)!.FullName;
        var source = Path.Combine(root, k_Source);
```

```
        if (!File.Exists(source))
        {
            Debug.LogError($"[MultiSetCredentials] No {k_Source}. " +
                           "It should contain MULTISET_CLIENT_ID and MULTISET_CLIENT_SECRET.");
            return;
        }

        var dest = Path.Combine(root, k_Dest);
        Directory.CreateDirectory(Path.GetDirectoryName(dest)!);
        File.Copy(source, dest, true);

        EnsureIgnored(root);

        AssetDatabase.ImportAsset(k_Dest);
        Debug.Log($"[MultiSetCredentials] staged into {k_Dest} (gitignored). " +
                  "It is readable by anyone who unpacks the APK - test builds only.");
    }

    static void EnsureIgnored(string root)
    {
        var path = Path.Combine(root, k_Ignore);
        var text = File.Exists(path) ? File.ReadAllText(path) : "";
        if (text.Contains(k_IgnoreLine))
            return;

        File.AppendAllText(path,
            "\n# Staged MultiSet credentials. Written by MultiSetCredentials.Stage\n" +
            "# from Tools/multiset.env, which is itself ignored. Never commit either.\n" +
            k_IgnoreLine + "\n" + k_IgnoreLine + ".meta\n");
        Debug.Log("[MultiSetCredentials] added the staged copy to .gitignore");
    }
}
```

4.2.8 AR-031 — MuseumUICapture.cs

Code ID	AR-031
File Name	MuseumUICapture.cs
File Path	SanctiWalkAR/Assets/Editor/MuseumUICapture.cs
Language / Type	C#
Lines	140
Functions / Classes	MuseumUICapture (public class); Capture (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/MuseumUICapture.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — MuseumUICapture.cs

```
using System.Reflection;
using System.IO;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// Renders the museum UI to PNGs without a phone.
///
/// The interface can otherwise only be seen by installing an APK on an
/// ARCore device and standing in the right room, which makes checking a
/// layout change a twenty-minute round trip. This drives the real
/// MuseumUI - the same code that ships - into a RenderTexture and writes
/// the frames out.
///
/// It is a capture rig, not a mock. If the layout is wrong here it is
/// wrong on the phone, which is the entire point.
/// </summary>
public static class MuseumUICapture
{
    const int Width = 390 * 2;
    const int Height = 844 * 2;

    [MenuItem("SanctiWalk/Capture Museum UI")]
    public static void Capture()
    {
        var scene = EditorSceneManager.NewScene(NewSceneSetup.EmptyScene, NewSceneMode.Single);

        // A camera for the canvas to render through, and a dark backdrop
        // standing in for the camera feed, which does not exist off-device.
        var camGo = new GameObject("CaptureCamera", typeof(Camera));
        var cam = camGo.GetComponent<Camera>();
```

```
    cam.clearFlags = CameraClearFlags.SolidColor;
    cam.backgroundColor = new Color32(0x24, 0x22, 0x1E, 0xFF);
    cam.orthographic = true;

    var host = new GameObject("MuseumUIHost");
    var ui = host.AddComponent<MuseumUI>();

    // Awake does the building and does not run in edit mode.
    typeof(MuseumUI).GetMethod("Awake", BindingFlags.NonPublic | BindingFlags.Instance)
        ?.Invoke(ui, null);

    var canvas = host.GetComponentInChildren<Canvas>();
    canvas.renderMode = RenderMode.ScreenSpaceCamera;
    canvas.worldCamera = cam;
    canvas.planeDistance = 1f;

    var dir = "Builds/UIShots";
    Directory.CreateDirectory(dir);

    Shoot(ui, cam, canvas, dir, "1-scan", null);

    var stations = MuseumStations.Sala;
    Invoke(ui, "ShowBubble", stations[1]);            // Santo Nino
    Shoot(ui, cam, canvas, dir, "2-bubble", null);

    Invoke(ui, "ShowCard");
    Shoot(ui, cam, canvas, dir, "3-card", null);

    Invoke(ui, "ShowBubble", stations[3]);            // the TV, no artwork
    Shoot(ui, cam, canvas, dir, "4-bubble-no-image", null);

    // The world-anchored bubble, which is a 3D object rather than a
    // screen overlay, so it needs its own perspective shot.
    CaptureWorldBubble(dir);

    Debug.Log($"[MuseumUICapture] wrote frames to {dir}");
    EditorSceneManager.CloseScene(scene, true);
}

static void CaptureWorldBubble(string dir)
{
    var camGo = new GameObject("WorldCam", typeof(Camera));
    var cam = camGo.GetComponent<Camera>();
    cam.clearFlags = CameraClearFlags.SolidColor;
    cam.backgroundColor = new Color32(0x2A, 0x26, 0x21, 0xFF);
    cam.fieldOfView = 55f;
    camGo.transform.position = new Vector3(0f, 1.45f, -1.25f);
    camGo.transform.LookAt(new Vector3(0f, 1.32f, 0f));

    var heading = Resources.Load<Font>("Montserrat-SemiBold");
    var body = Resources.Load<Font>("Poppins-Regular");
    StationBubble.Create(MuseumStations.Sala[1], new Vector3(0f, 1.05f, 0f), heading, body, cam);

    Canvas.ForceUpdateCanvases();

    var rt = new RenderTexture(Width, Height, 24, RenderTextureFormat.ARGB32) { antiAliasing = 4 };
    cam.targetTexture = rt;
    cam.Render();
```

```
        var previous = RenderTexture.active;
        RenderTexture.active = rt;
        var shot = new Texture2D(Width, Height, TextureFormat.RGBA32, false);
        shot.ReadPixels(new Rect(0, 0, Width, Height), 0, 0);
        shot.Apply();
        RenderTexture.active = previous;

        File.WriteAllBytes(Path.Combine(dir, "5-world-bubble.png"), shot.EncodeToPNG());
        cam.targetTexture = null;
        Object.DestroyImmediate(rt);
        Object.DestroyImmediate(shot);
    }

    static void Invoke(MuseumUI ui, string method, params object[] args)
    {
        var m = typeof(MuseumUI).GetMethod(method, BindingFlags.NonPublic | BindingFlags.Instance);
        if (m == null) { Debug.LogError($"[MuseumUICapture] no method {method}"); return; }
        m.Invoke(ui, args);
    }

    static void Shoot(MuseumUI ui, Camera cam, Canvas canvas, string dir, string name, object _)
    {
        var rt = new RenderTexture(Width, Height, 24, RenderTextureFormat.ARGB32)
        {
            antiAliasing = 4,
        };
        cam.targetTexture = rt;

        Canvas.ForceUpdateCanvases();
        canvas.GetComponent<CanvasScaler>().referenceResolution = new Vector2(390, 844);
        LayoutRebuilder.ForceRebuildLayoutImmediate((RectTransform)canvas.transform);
        Canvas.ForceUpdateCanvases();

        cam.Render();

        var previous = RenderTexture.active;
        RenderTexture.active = rt;
        var shot = new Texture2D(Width, Height, TextureFormat.RGBA32, false);
        shot.ReadPixels(new Rect(0, 0, Width, Height), 0, 0);
        shot.Apply();
        RenderTexture.active = previous;

        File.WriteAllBytes(Path.Combine(dir, name + ".png"), shot.EncodeToPNG());
        cam.targetTexture = null;
        Object.DestroyImmediate(rt);
        Object.DestroyImmediate(shot);
    }
}
```

4.2.9 AR-032 — ScanFlowDump.cs

Code ID	AR-032
File Name	ScanFlowDump.cs
File Path	SanctiWalkAR/Assets/Editor/ScanFlowDump.cs
Language / Type	C#
Lines	115
Functions / Classes	ScanFlowDump (public class); Dump (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/ScanFlowDump.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ScanFlowDump.cs

```
using System.IO;
using System.Text;
using UnityEditor;
using UnityEngine;

/// <summary>
/// Runs a simulated scan and writes out every value at every stage, so the
/// flow can be read as data rather than inferred from behaviour.
///
/// Deliberately prints the four sources separately - what the scan gave,
/// what the library gave, what the POI configuration gave, and what was
/// derived from combining them - because the question that matters when a
/// station lands in the wrong place is which of those was wrong.
/// </summary>
public static class ScanFlowDump
{
    [MenuItem("SanctiWalk/Dump Scan Flow")]
    public static void Dump()
    {
        var sb = new StringBuilder();

        // A camera standing where a pilgrim would be: 1.45 m up, looking at
        // the wall. In a real run this is wherever the session started.
        var rig = new GameObject("SimCamera");
        rig.transform.position = new Vector3(0f, 1.45f, 0f);
        rig.transform.rotation = Quaternion.LookRotation(Vector3.forward, Vector3.up);

        var scan = ScanSimulator.Sala(rig.transform);

        sb.AppendLine("================================================================");
        sb.AppendLine(" STAGE 1 - SCAN IMAGE  ->  what the scan produces");
        sb.AppendLine("================================================================");
        sb.AppendLine(scan.ToString());
```

```
sb.AppendLine();
sb.AppendLine("  The scan answers two questions and no others:");
sb.AppendLine("    WHICH image was seen   -> trackedImageName / referenceGuid");
sb.AppendLine("    WHERE that image is    -> pose, in session space");
sb.AppendLine("  It does not know the parish, the station, or the history.");
sb.AppendLine();

sb.AppendLine("================================================================");
sb.AppendLine(" STAGE 2 - MATCHED POI SET  ->  from configuration, not the scan");
sb.AppendLine("================================================================");
sb.AppendLine($"  trigger \"{scan.trackedImageName}\" is configured as the entrance");
sb.AppendLine($"  marker for the sala, so it selects MuseumStations.Sala");
sb.AppendLine($"  ({MuseumStations.Sala.Length} stations).");
sb.AppendLine();

// The same conversion the runtime uses: survey is X across, Y up,
// Z into the room; the marker's own frame swaps Y and Z.
var intoRoom = IntoRoomSign(scan);
sb.AppendLine($"  into-room sign, derived from where the camera stood: {intoRoom:+0;-0}");
sb.AppendLine();

foreach (var station in MuseumStations.Sala)
{
    var local = new Vector3(station.position.x, station.position.z * intoRoom, station.position.y);
    var world = scan.pose.position + scan.pose.rotation * local;
    var range = Vector3.Distance(rig.transform.position, world);

    sb.AppendLine($"  ---- {station.stationId} ----------------------------------------");
    sb.AppendLine($"    FROM POI CONFIG");
    sb.AppendLine($"      stationName      : {station.stationName}");
    sb.AppendLine($"      subtitle         : {station.subtitle}");
    sb.AppendLine($"      surveyed offset  : X {station.position.x,7:F3}  Y {station.position.y,7:F3}  Z {station.position.z,7:F3}   (metres from marker)");
    sb.AppendLine($"      height above floor: {station.position.y + 1.407f:F3} m   (marker centre is 1.407 m up)");
    sb.AppendLine($"      triggerRadius    : {station.triggerRadius:F2} m");
    sb.AppendLine($"      image            : {(string.IsNullOrEmpty(station.image) ? "(none)" : station.image)}");
    sb.AppendLine($"      yearInstalled    : {Or(station.yearInstalled)}");
    sb.AppendLine($"      artist           : {Or(station.artist)}");
    sb.AppendLine($"      material         : {Or(station.material)}");
    sb.AppendLine($"      nextStationId    : {Or(station.nextStationId)}");
    sb.AppendLine($"      shortDescription : {station.shortDescription}");
    sb.AppendLine($"    DERIVED (scan pose x surveyed offset)");
    sb.AppendLine($"      marker-frame     : ({local.x:F3}, {local.y:F3}, {local.z:F3})");
    sb.AppendLine($"      session-space    : ({world.x:F3}, {world.y:F3}, {world.z:F3})");
    sb.AppendLine($"      range from camera: {range:F2} m  -> bubble {(range <= station.triggerRadius ? "SHOWS" : "hidden")} at this position");
    sb.AppendLine();
}

sb.AppendLine("================================================================");
sb.AppendLine(" STAGE 3 - NAVIGATION CHAIN");
sb.AppendLine("================================================================");
var current = MuseumStations.Sala[0];
var step = 1;
while (current != null)
{
    sb.AppendLine($"  {step}. {current.stationName}  ({current.stationId})");
    if (string.IsNullOrEmpty(current.nextStationId)) { sb.AppendLine("     -> end of tour"); break; }
    var next = System.Array.Find(MuseumStations.Sala, x => x.stationId == current.nextStationId);
    if (next == null) { sb.AppendLine($"     -> BROKEN LINK: no station \"{current.nextStationId}\""); break; }
    sb.AppendLine($"     -> next: {next.stationName}");
```

```
            current = next;
            step++;
        }

        Object.DestroyImmediate(rig);

        Directory.CreateDirectory("Builds");
        var path = "Builds/scan-flow.txt";
        File.WriteAllText(path, sb.ToString());
        Debug.Log(sb.ToString());
        Debug.Log($"[ScanFlowDump] written to {path}");
    }

    static string Or(string value) => string.IsNullOrEmpty(value) ? "(not set)" : value;

    /// <summary>Mirrors ChurchOriginTest: which side of the print the room is on.</summary>
    static float IntoRoomSign(MarkerScan scan)
    {
        var normal = scan.pose.rotation * Vector3.up;
        var toCamera = scan.cameraPoseAtDetection.position - scan.pose.position;
        return Vector3.Dot(normal, toCamera) < 0f ? -1f : 1f;
    }
}
```

4.2.10 AR-033 — StationEditorScene.cs

Code ID	AR-033
File Name	StationEditorScene.cs
File Path	SanctiWalkAR/Assets/Editor/StationEditorScene.cs
Language / Type	C#
Lines	403
Functions / Classes	StationEditorScene (public class); BuildScene (public method); MarkerToMap (public method); CopySurveyed (public method)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/Editor/StationEditorScene.cs. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — StationEditorScene.cs

```
using System.Linq;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

/// <summary>
/// Builds and maintains the station-authoring scene.
///
/// This scene is a workbench, not a deliverable. It holds the MultiSet
/// reference mesh, the marker, and one draggable pin per station, so a
/// station can be positioned by looking at the room rather than by typing
/// numbers. Nothing in it ships, because the APK build list contains
/// DriftTest.unity alone - this scene is never built.
///
/// Menu:
///   SanctiWalk / Stations / Build Authoring Scene
///   SanctiWalk / Stations / Copy surveyed[] To Clipboard
/// </summary>
public static class StationEditorScene
{
    const string ScenePath = "Assets/Scenes/StationAuthoring.unity";
    const string MeshPath = "Assets/EditorOnly/SalaReference/sala-reference.obj";

    [MenuItem("SanctiWalk/Stations/Build Authoring Scene")]
    public static void BuildScene()
    {
        var scene = EditorSceneManager.NewScene(NewSceneSetup.EmptyScene, NewSceneMode.Single);

        BuildLighting();
        BuildReference();
```

```
    BuildMarker();
    BuildStations();
    BuildFloorGrid();
    BuildCollision();
    BuildPlayer();

    EditorSceneManager.SaveScene(scene, ScenePath);
    Debug.Log($"[StationEditorScene] built {ScenePath}. Open it and look down into the room.");
    FrameSceneView();
}

static void BuildLighting()
{
    // The scan's own material is unlit, which is why it reads as flat
    // noise. The reference is re-materialled below and actually lit, so
    // depth is visible; these are the lights that do it.
    var keyGo = new GameObject("Key Light");
    var key = keyGo.AddComponent<Light>();
    key.type = LightType.Directional;
    key.intensity = 1.15f;
    key.shadows = LightShadows.Soft;
    keyGo.transform.rotation = Quaternion.Euler(52f, 150f, 0f);

    var fillGo = new GameObject("Fill Light");
    var fill = fillGo.AddComponent<Light>();
    fill.type = LightType.Directional;
    fill.intensity = 0.45f;
    fill.shadows = LightShadows.None;
    fillGo.transform.rotation = Quaternion.Euler(28f, -40f, 0f);

    RenderSettings.ambientMode = UnityEngine.Rendering.AmbientMode.Trilight;
    RenderSettings.ambientSkyColor = new Color(0.62f, 0.64f, 0.68f);
    RenderSettings.ambientEquatorColor = new Color(0.48f, 0.47f, 0.45f);
    RenderSettings.ambientGroundColor = new Color(0.30f, 0.28f, 0.26f);
}

static void BuildReference()
{
    var prefab = AssetDatabase.LoadAssetAtPath<GameObject>(MeshPath);
    if (prefab == null)
    {
        Debug.LogError($"[StationEditorScene] reference mesh missing at {MeshPath}");
        return;
    }

    var go = (GameObject)PrefabUtility.InstantiatePrefab(prefab);
    go.name = "SalaReference";
    go.transform.position = Vector3.zero;   // map space origin. Do not move.
    go.isStatic = true;

    // Not tagged EditorOnly. That tag strips an object from every
    // build, including the desktop walkthrough that exists to walk
    // this very mesh. The APK exclusion comes from the build scene
    // list, which contains DriftTest.unity alone - a stronger
    // guarantee, since it keeps out the whole scene rather than
    // relying on a tag being remembered on each object.

    // Lit, so the room has shading. The imported OBJ material is fine
    // but defaults can vary, so this pins it.
```

```
    foreach (var renderer in go.GetComponentsInChildren<MeshRenderer>())
    {
        var mat = renderer.sharedMaterial;
        if (mat != null && mat.HasProperty("_Glossiness")) mat.SetFloat("_Glossiness", 0.05f);
        if (mat != null && mat.HasProperty("_Smoothness")) mat.SetFloat("_Smoothness", 0.05f);
        renderer.shadowCastingMode = UnityEngine.Rendering.ShadowCastingMode.Off;
    }
}

static void BuildMarker()
{
    var root = new GameObject("EntranceMarker (origin)");
    root.transform.position = StationMarker.MarkerInMapSpace;
    root.transform.rotation = Quaternion.LookRotation(StationMarker.MarkerNormal, Vector3.up);

    // The printed sheet, at its real size, so its place on the wall is
    // unmistakable and its scale can be eyeballed against the room.
    var sheet = GameObject.CreatePrimitive(PrimitiveType.Quad);
    sheet.name = "PrintedMarker 180x240mm";
    sheet.transform.SetParent(root.transform, false);
    sheet.transform.localScale = new Vector3(0.18f, 0.24f, 1f);
    Object.DestroyImmediate(sheet.GetComponent<Collider>());

    var texture = AssetDatabase.LoadAssetAtPath<Texture2D>("Assets/AR/entrance-marker.png");
    var mat = new Material(Shader.Find("Unlit/Texture"));
    if (texture != null) mat.mainTexture = texture;
    sheet.GetComponent<MeshRenderer>().sharedMaterial = mat;

    root.AddComponent<MarkerGizmo>();
}

static void BuildStations()
{
    var parent = new GameObject("Stations").transform;

    // Stand-in artwork so each trigger is visible in the Scene view.
    // Replace per station via PointOfInterest.triggerImage.
    var placeholder = AssetDatabase.LoadAssetAtPath<Texture2D>("Assets/AR/entrance-marker.png");

    var colours = new[]
    {
        new Color(0.16f, 0.42f, 0.90f),
        new Color(0.85f, 0.55f, 0.15f),
        new Color(0.25f, 0.65f, 0.40f),
        new Color(0.72f, 0.26f, 0.55f),
    };

    for (var i = 0; i < MuseumStations.Sala.Length; i++)
    {
        var data = MuseumStations.Sala[i];
        var go = new GameObject($"{i + 1}. {data.stationName}");
        go.transform.SetParent(parent, false);

        // Convert the stored marker-space offset back into map space,
        // so the pins land exactly where the survey put them.
        go.transform.position = MarkerToMap(data.position);
        go.transform.rotation = Quaternion.LookRotation(
            Vector3.ProjectOnPlane(StationMarker.MarkerInMapSpace - go.transform.position, Vector3.up).normalized
                is var look && look.sqrMagnitude > 0.001f ? look : Vector3.forward,
```

```
            Vector3.up);

        // The POI owns the data; the pin only reports on it.
        var poi = go.AddComponent<PointOfInterest>();
        poi.id = data.stationId;
        poi.title = data.stationName;
        poi.subtitle = data.subtitle;
        poi.triggerRadius = data.triggerRadius;
        poi.nextId = data.nextStationId;
        poi.triggerImage = placeholder;

        // Placeholder text only. The parish history is not ours to
        // write, and a plausible-sounding invention is worse than an
        // obvious blank.
        poi.description = "Church information will be added here.";

        var sphere = go.AddComponent<SphereCollider>();
        sphere.isTrigger = true;
        sphere.radius = data.triggerRadius;

        var marker = go.AddComponent<StationMarker>();
        marker.number = i + 1;
        marker.pinColour = colours[i % colours.Length];

        BuildTriggerMarker(go.transform, placeholder);
    }
}

/// <summary>
/// A small quad at the trigger showing its placeholder artwork, so a
/// trigger is something you can see and drag rather than an invisible
/// radius. Authoring-scene only, like everything else here.
/// </summary>
static void BuildTriggerMarker(Transform parent, Texture2D texture)
{
    var quad = GameObject.CreatePrimitive(PrimitiveType.Quad);
    quad.name = "TriggerMarker";
    quad.transform.SetParent(parent, false);
    quad.transform.localPosition = Vector3.zero;
    quad.transform.localScale = new Vector3(0.18f, 0.24f, 1f);
    Object.DestroyImmediate(quad.GetComponent<Collider>());

    var mat = new Material(Shader.Find("Unlit/Texture"));
    if (texture != null) mat.mainTexture = texture;
    quad.GetComponent<MeshRenderer>().sharedMaterial = mat;
}

static void BuildFloorGrid()
{
    var go = new GameObject("Floor Reference");
    go.transform.position = new Vector3(0f, StationMarker.FloorY, 0f);
    go.AddComponent<FloorGridGizmo>();
}

/// <summary>
/// Collision, so Play does not drop you through the world.
///
/// Two layers of it. The scan gets mesh colliders, which is what makes
/// walls and furniture solid; but a photogrammetric floor has holes in
```

```
/// it, and falling through one is indistinguishable from the feature
/// being broken. So an invisible slab sits at the measured floor
/// height as a backstop. Neither changes the visible scan.
/// </summary>
static void BuildCollision()
{
    // Ask the importer for colliders on the reference mesh. Done here
    // rather than by hand so rebuilding the scene cannot forget it.
    var importer = AssetImporter.GetAtPath(MeshPath) as ModelImporter;
    if (importer != null && !importer.addCollider)
    {
        importer.addCollider = true;
        importer.SaveAndReimport();
        Debug.Log("[StationEditorScene] enabled colliders on the reference mesh");
    }

    var floor = GameObject.CreatePrimitive(PrimitiveType.Cube);
    floor.name = "Floor Backstop";
    floor.isStatic = true;
    floor.transform.position = new Vector3(1.4f, StationMarker.FloorY - 0.15f, 3.1f);
    floor.transform.localScale = new Vector3(14f, 0.3f, 14f);

    // Invisible but solid: the renderer goes, the collider stays.
    Object.DestroyImmediate(floor.GetComponent<MeshRenderer>());
}

/// <summary>
/// The walkthrough player, spawned on open floor in front of the marker.
///
/// Position is derived rather than guessed: 1.5 m out along the
/// marker's own normal, which is by construction inside the room, then
/// dropped onto the floor height measured off the mesh. Facing the way
/// the marker faces means you start looking into the room rather than
/// at the wall you are standing against.
/// </summary>
static void BuildPlayer()
{
    var go = new GameObject("Player");

    var controller = go.AddComponent<CharacterController>();
    controller.height = 1.7f;
    controller.radius = 0.28f;
    controller.center = Vector3.zero;
    controller.slopeLimit = 50f;
    controller.stepOffset = 0.35f;
    controller.skinWidth = 0.02f;

    // 1.5 m out along the marker's normal is inside the room but it is
    // also inside the furniture - a capsule test there reports an
    // overlap. So the spawn is searched for rather than calculated.
    var spawn = FindOpenSpawn(controller);
    go.transform.position = spawn;
    go.transform.rotation = Quaternion.LookRotation(StationMarker.MarkerNormal, Vector3.up);

    var eye = new GameObject("Eye", typeof(Camera), typeof(AudioListener));
    eye.transform.SetParent(go.transform, false);
    var cam = eye.GetComponent<Camera>();
    cam.nearClipPlane = 0.05f;
    cam.farClipPlane = 120f;
```

```
    cam.fieldOfView = 60f;
    cam.clearFlags = CameraClearFlags.SolidColor;
    cam.backgroundColor = new Color32(0x1B, 0x1E, 0x25, 0xFF);
    eye.transform.localPosition = new Vector3(0f, 1.6f - controller.height * 0.5f, 0f);

    go.AddComponent<FirstPersonWalkthrough>();

    // The POI list, for reaching a trigger without walking there.
    var directory = new GameObject("POI Directory");
    directory.AddComponent<POIDirectory>();

    Debug.Log($"[StationEditorScene] player spawned at " +
              $"({go.transform.position.x:F2}, {go.transform.position.y:F2}, {go.transform.position.z:F2}), " +
              $"eye {1.6f:F2} m above floor");
}

/// <summary>
/// Hunts for somewhere the player can actually stand.
///
/// Samples the room on a quarter-metre grid, drops each candidate onto
/// whatever floor is beneath it, and rejects any where the capsule
/// intersects the scan. Among the survivors it prefers the one nearest
/// the entrance marker, so the walkthrough still begins where a visitor
/// would come in. Returns the capsule centre, not the feet.
/// </summary>
static Vector3 FindOpenSpawn(CharacterController controller)
{
    Physics.SyncTransforms();

    // The test capsule must sit strictly INSIDE the controller's own
    // volume, lifted clear of the floor. A generous probe reported an
    // overlap at every one of 1085 sampled positions - it was dipping
    // 1.4 cm into the floor slab, so "solid ground" read as "blocked".
    var probe = controller.radius * 0.95f;
    var lift = 0.06f;

    var best = Vector3.zero;
    var bestDistance = float.MaxValue;
    var tested = 0;
    var open = 0;

    for (var x = -2.4f; x <= 5.2f; x += 0.25f)
    for (var z = -1.2f; z <= 7.4f; z += 0.25f)
    {
        tested++;

        // Must be on the room side of the marker's wall. Nearest-to-the
        // -marker alone picked a spot 0.35 m BEHIND the wall plane,
        // which has floor under it in the scan but is not the room.
        var acrossWall = Vector3.Dot(
            new Vector3(x, 0f, z) - new Vector3(StationMarker.MarkerInMapSpace.x, 0f, StationMarker.MarkerInMapSpace.z),
            StationMarker.MarkerNormal);
        if (acrossWall < 1.0f) continue;

        // Where is the floor here? Cast from head height downwards.
        // Triggers ignored throughout: the station trigger spheres are
        // not obstacles, and counting them cut the clear positions from
        // 303 to 117 and pushed the spawn across the room.
        if (!Physics.Raycast(new Vector3(x, 0.9f, z), Vector3.down, out var hit, 4f,
```

```
                            ~0, QueryTriggerInteraction.Ignore)) continue;
        if (Mathf.Abs(hit.point.y - StationMarker.FloorY) > 0.25f) continue;   // a table top is not a floor

        var centre = new Vector3(x, hit.point.y + controller.height * 0.5f, z);
        var bottom = new Vector3(x, hit.point.y + controller.radius + lift, z);
        var top = new Vector3(x, hit.point.y + controller.height - controller.radius, z);
        if (Physics.CheckCapsule(bottom, top, probe, ~0, QueryTriggerInteraction.Ignore)) continue;

        open++;
        var d = Vector3.Distance(new Vector3(x, 0f, z),
            new Vector3(StationMarker.MarkerInMapSpace.x, 0f, StationMarker.MarkerInMapSpace.z));
        if (d < bestDistance) { bestDistance = d; best = centre; }
    }

    Debug.Log($"[StationEditorScene] spawn search: {open} clear of {tested} sampled positions");

    if (open == 0)
    {
        // Nothing stood up to the test. Rather than spawn inside a wall,
        // start above the floor backstop and fall the last few
        // centimetres, which at least leaves the player standing.
        Debug.LogWarning("[StationEditorScene] no clear spawn found; falling back to the room centre");
        return new Vector3(1.4f, StationMarker.FloorY + controller.height * 0.5f + 0.4f, 3.1f);
    }

    // A hair above the floor so the controller settles rather than
    // starting interpenetrated.
    return best + Vector3.up * 0.04f;
}

/// <summary>Marker space (X across, Y up, Z into room) back into map space.</summary>
public static Vector3 MarkerToMap(Vector3 markerSpace)
{
    var right = Vector3.Cross(Vector3.up, StationMarker.MarkerNormal).normalized;
    return StationMarker.MarkerInMapSpace
         + right * markerSpace.x
         + Vector3.up * markerSpace.y
         + StationMarker.MarkerNormal * markerSpace.z;
}

static void FrameSceneView()
{
    var view = SceneView.lastActiveSceneView;
    if (view == null) return;
    view.pivot = new Vector3(2.4f, -0.9f, 3.4f);
    view.rotation = Quaternion.Euler(55f, 25f, 0f);
    view.size = 8.5f;
    view.Repaint();
}

[MenuItem("SanctiWalk/Stations/Copy surveyed[] To Clipboard")]
public static void CopySurveyed()
{
    var markers = Object.FindObjectsByType<StationMarker>(FindObjectsSortMode.None)
        .OrderBy(m => m.number)
        .ToArray();

    if (markers.Length == 0)
    {
```

```
            Debug.LogWarning("[StationEditorScene] no StationMarker in the scene. Build the authoring scene first.");
            return;
        }

        var sb = new StringBuilder();
        sb.AppendLine("    public Vector3[] surveyed =");
        sb.AppendLine("    {");
        foreach (var m in markers)
        {
            var p = m.MarkerSpacePosition;
            sb.AppendLine($"        new Vector3({p.x:F3}f, {p.y:F3}f, {p.z:F3}f),   // {m.stationName}");
        }
        sb.AppendLine("    };");

        EditorGUIUtility.systemCopyBuffer = sb.ToString();
        Debug.Log("[StationEditorScene] copied to clipboard:\n" + sb);
    }
}
```

Part 5 — Build, Configuration and Deployment

5.1 Web Build and Package Configuration

5.1.1 CFG-001 — package.json

Code ID	CFG-001
File Name	package.json
File Path	SanctDemo/package.json
Language / Type	JSON
Lines	76
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/package.json. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — package.json

```
{
  "name": "react-example",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx server.ts",
    "dev:https": "cross-env HTTPS=1 tsx server.ts",
    "build": "vite build && esbuild server.ts --bundle --platform=node --format=cjs --packages=external --sourcemap --outfile=dist/server.cjs",
    "start": "node dist/server.cjs",
    "clean": "rm -rf dist",
    "lint": "tsc --noEmit",
    "test": "vitest run",
    "test:watch": "vitest",
    "qr:posters": "node scripts/generate-qr-posters.mjs",
    "build:app": "vite build",
    "apk": "node tools/write-latest-version.mjs && vite build && node -e \"require('fs').rmSync('dist/download',{recursive:true,force:true})\" && cap sync android &&
node -e \"const
p=require('path').resolve('android',process.platform==='win32'?'gradlew.bat':'gradlew');require('child_process').execFileSync(p,['assembleRelease'],{cwd:'android',st
dio:'inherit',shell:true})\"",
    "apk:lan": "cross-env CAP_LAN_TEST=1 npm run apk",
    "test:rules": "firebase emulators:exec --only firestore --project sanctiwalk-rules-test \"vitest run -c vitest.rules.config.ts\""
  },
  "dependencies": {
    "@capacitor-community/speech-recognition": "^7.0.1",
    "@capacitor-community/text-to-speech": "^8.0.2",
    "@capacitor-firebase/authentication": "^8.5.2",
```

```
    "@capacitor/local-notifications": "^8.3.1",
    "@google/genai": "^2.4.0",
    "@tailwindcss/vite": "^4.1.14",
    "@vitejs/plugin-react": "^5.0.4",
    "dotenv": "^17.2.3",
    "express": "^4.21.2",
    "firebase": "^12.16.0",
    "firebase-admin": "^14.5.0",
    "jsqr": "^1.4.0",
    "lucide-react": "^0.546.0",
    "maplibre-gl": "^6.5.0",
    "mind-ar": "^1.2.5",
    "motion": "^12.23.24",
    "nodemailer": "^9.0.3",
    "react": "^19.0.1",
    "react-dom": "^19.0.1",
    "react-router-dom": "^7.18.4",
    "three": "^0.186.0",
    "vite": "^6.2.3",
    "xlsx": "^0.18.5"
  },
  "devDependencies": {
    "@capacitor/android": "^8.5.2",
    "@capacitor/cli": "^8.5.2",
    "@capacitor/core": "^8.5.2",
    "@firebase/rules-unit-testing": "^5.0.2",
    "@testing-library/jest-dom": "^7.0.1",
    "@testing-library/react": "^16.3.2",
    "@types/express": "^4.17.21",
    "@types/node": "^22.14.0",
    "@types/nodemailer": "^8.0.1",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "autoprefixer": "^10.4.21",
    "cross-env": "^10.1.0",
    "esbuild": "^0.25.0",
    "firebase-tools": "^15.30.2",
    "jsdom": "^30.0.1",
    "pngjs": "^7.0.0",
    "puppeteer-core": "^25.13.0",
    "qrcode": "^1.5.4",
    "selfsigned": "^5.5.0",
    "tailwindcss": "^4.1.14",
    "tsx": "^4.21.0",
    "typescript": "~5.8.2",
    "vite": "^6.2.3",
    "vitest": "^3.2.7"
  },
  "engines": {
    "node": ">=20"
  }
}
```

5.1.2 CFG-002 — vite.config.ts

Code ID	CFG-002
File Name	vite.config.ts
File Path	SanctDemo/vite.config.ts
Language / Type	TypeScript
Lines	119
Functions / Classes	copyMapLibreWorker (internal function)
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/vite.config.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — vite.config.ts

```
import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';
import path from 'path';
import fs from 'fs';
import {defineConfig} from 'vite';

/**
 * Copies MapLibre's Web Worker into the build output.
 *
 * MapLibre 6 does not import its worker statically. It computes the URL at
 * runtime - `new URL('./maplibre-gl-worker.mjs', import.meta.url)` - which
 * Vite cannot see, so the file is never emitted. The bundle then requests
 * /assets/maplibre-gl-worker.mjs, gets a 404, and the map renders nothing at
 * all: no tiles, no pins, no error the user can act on.
 *
 * It worked in the dev server, which serves straight from node_modules, and
 * failed only in a production build - so it reached a packaged APK before
 * anyone saw it.
 */
function copyMapLibreWorker() {
  // BOTH files. The worker is not self-contained - it does
  // `import ... from "./maplibre-gl-shared.mjs"`, so emitting the worker
  // alone gets it fetched and then failing on a missing import, which looks
  // identical to the original bug.
  const files = ['maplibre-gl-worker.mjs', 'maplibre-gl-shared.mjs'];

  return {
    name: 'copy-maplibre-worker',
    apply: 'build' as const,
    generateBundle() {
      for (const file of files) {
        const source = `node_modules/maplibre-gl/dist/${file}`;
        if (!fs.existsSync(source)) {
          this.warn(`MapLibre ${file} not found; the map will not render.`);
```

```
          continue;
        }
        this.emitFile({
          type: 'asset',
          fileName: `assets/${file}`,
          source: fs.readFileSync(source, 'utf8'),
        });
      }
    },
  };
}

// One version, from package.json, baked into the bundle.
//
// The Android build reads the same field for versionName and derives
// versionCode from it, so the running app and the published release
// cannot disagree about which build this is. See lib/appUpdate.
const APP_VERSION = JSON.parse(
  fs.readFileSync(path.resolve(__dirname, 'package.json'), 'utf-8'),
).version as string

export default defineConfig(({command}) => {
  return {
    define: {
      __APP_VERSION__: JSON.stringify(APP_VERSION),
    },
    plugins: [react(), tailwindcss(), copyMapLibreWorker()],
    resolve: {
      alias: {
        '@': path.resolve(__dirname, '.'),
      },
    },
    // The exclusion below is DEV-ONLY.
    //
    // Excluding maplibre-gl fixes a dev-server HMR problem, but applied to a
    // production build it also stops Vite emitting the library's Web Worker:
    // the bundle then requests /assets/maplibre-gl-worker.mjs at runtime, gets
    // a 404, and the map renders nothing. That reached a packaged APK before
    // anyone noticed, because the dev server had the file and the build did
    // not.
    optimizeDeps: {
      // maplibre-gl loads its tile-processing code as a separate module
      // Worker (`new Worker(new URL(...), { type: 'module' })`), which
      // esbuild's dependency pre-bundler can't see statically. Pre-bundling
      // the package anyway leaves the worker chunk out of the optimized
      // deps directory, which desyncs Vite's dev-time module graph (stale
      // "does not provide an export named 'default'" errors after HMR).
      // Excluding it serves the package straight from node_modules, where
      // the worker file resolves correctly.
      // Dev only - see above. `command` is 'serve' for the dev server and
      // 'build' for a production build.
      exclude: command === 'serve' ? ['maplibre-gl'] : [],
    },
    server: {
      // HMR is disabled in AI Studio via DISABLE_HMR env var.
      // Do not modify - file watching is disabled to prevent flickering during agent edits.
      //
      // The HMR websocket needs its own port, and Vite's default (24678) is
      // fixed, so a second dev server on this machine collides with the
```

```
      // first and loses hot reload even after autoPort has moved its HTTP
      // port out of the way. Deriving it from PORT keeps concurrent
      // sessions independent.
      hmr:
        process.env.DISABLE_HMR === 'true'
          ? false
          : process.env.PORT
            ? {port: Number(process.env.PORT) + 1}
            : true,
      // Disable file watching when DISABLE_HMR is true to save CPU during agent edits.
      watch:
        process.env.DISABLE_HMR === 'true'
          ? null
          : {
              // Never watch the Android build output.
              //
              // Adding Capacitor put ~160 MB of Gradle artifacts and APKs
              // under android/, and Vite watched all of it. Copying an APK
              // out of that folder locked the file on Windows, the watcher
              // threw EBUSY, and the dev server died - after 45 hours of
              // uptime, for no reason connected to the web app at all.
              ignored: ['**/android/**', '**/dist/**', '**/.gradle/**'],
            },
    },
  };
});
```

5.1.3 CFG-003 — tsconfig.json

Code ID	CFG-003
File Name	tsconfig.json
File Path	SanctDemo/tsconfig.json
Language / Type	JSON
Lines	27
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/tsconfig.json. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "experimentalDecorators": true,
    "useDefineForClassFields": false,
    "module": "ESNext",
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "moduleDetection": "force",
    "allowJs": true,
    "jsx": "react-jsx",
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}
```

5.1.4 CFG-004 — vitest.config.ts

Code ID	CFG-004
File Name	vitest.config.ts
File Path	SanctDemo/vitest.config.ts
Language / Type	TypeScript
Lines	25
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/vitest.config.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — vitest.config.ts

```
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    // Vitest's default is 5s, and a handful of the component tests sit
    // close enough to it that they time out when the suite runs flat
    // out beside a Gradle or Unity build - a different file each run,
    // every one of them passing on its own. That is the machine being
    // busy, not the test being wrong, and a suite that cries wolf is a
    // suite people stop reading. A genuinely broken test still fails on
    // its assertion, in milliseconds, and is unaffected by this.
    testTimeout: 20000,
    hookTimeout: 20000,
    // The Firestore rules suite needs the emulator and refuses to run
    // without it, so it is not part of `npm test` - a default run that
    // fails on every machine without a Java emulator teaches people to
    // ignore the result. Run it with `npm run test:rules`, which starts
    // the emulator around it.
    exclude: ['**/node_modules/**', '**/dist/**', 'src/lib/firestoreRules.test.ts'],
  },
})
```

5.1.5 CFG-005 — vercel.json

Code ID	CFG-005
File Name	vercel.json
File Path	SanctDemo/vercel.json
Language / Type	JSON
Lines	43
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/vercel.json. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "installCommand": "npm install --ignore-scripts",
  "buildCommand": "npm run build:app",
  "outputDirectory": "dist",
  "framework": "vite",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
        { "key": "X-Frame-Options", "value": "SAMEORIGIN" }
      ]
    },
    {
      "source": "/assets/(.*)",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }
      ]
    },
    {
      "source": "/sw.js",
      "headers": [
        { "key": "Cache-Control", "value": "public, max-age=0, must-revalidate" },
        { "key": "Service-Worker-Allowed", "value": "/" }
      ]
    },
```

```
    {
      "source": "/manifest.webmanifest",
      "headers": [
        { "key": "Content-Type", "value": "application/manifest+json" },
        { "key": "Cache-Control", "value": "public, max-age=0, must-revalidate" }
      ]
    }
  ]
}
```

5.1.6 CFG-006 — capacitor.config.ts

Code ID	CFG-006
File Name	capacitor.config.ts
File Path	SanctDemo/capacitor.config.ts
Language / Type	TypeScript
Lines	69
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/capacitor.config.ts. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — capacitor.config.ts

```
import type { CapacitorConfig } from '@capacitor/cli'

// Packages the SanctiWalk web app as a real Android application.
//
// The paper calls SanctiWalk "a mobile app tour guide system" and says it
// must "operate on mobile devices with or without internet connectivity".
// Served from a browser it was neither: not installable, and dead without a
// connection. Capacitor bundles the built web app inside an APK, so it gets
// its own launcher icon, opens without a browser, and its shell, parish data,
// station content and Rosary all work with no signal.
//
// It does not make the AI recognition work offline - that reaches Gemini
// through the Express backend and always needed the internet. See
// src/lib/apiBase.ts for how the packaged app is told where that backend is.
// Set CAP_LAN_TEST=1 when the backend is a laptop on the same WiFi rather
// than a deployed HTTPS server. It is off unless asked for, so the ordinary
// build keeps both of Android's protections intact.
//
// Pair it with VITE_API_BASE pointing at that laptop:
//   CAP_LAN_TEST=1 VITE_API_BASE=http://192.168.1.20:3000 npm run apk:lan
const LAN_TEST = process.env.CAP_LAN_TEST === '1'

const config: CapacitorConfig = {
  appId: 'ph.edu.sti.sanctiwalk',
  appName: 'SanctiWalk',
  // `npx vite build` writes here. Use that rather than `npm run build`,
  // which also esbuilds server.cjs into the same folder - a Node bundle has
  // no business inside an Android APK.
  webDir: 'dist',

  android: {
    // The scanner and the AR tour both need the camera, and a WebView only
    // grants getUserMedia on a secure origin. Capacitor serves the bundle
    // over https://localhost, which qualifies.
```

```
    //
    // The page is https, so a plain-http backend is mixed content and the
    // WebView blocks it. That is the correct default: it is exactly the
    // protection that stops anyone on the same WiFi reading the photographs
    // being scanned. It is relaxed only for a deliberate LAN test build.
    allowMixedContent: LAN_TEST,
  },

  plugins: {
    FirebaseAuthentication: {
      // Only the Google account picker is wanted from the native side.
      // The credential it returns is signed into the JavaScript SDK by
      // src/lib/googleAuth.ts, and that is the SDK every Firestore rule
      // reads. Letting the plugin sign in natively as well would leave
      // two sessions that can disagree about who is signed in.
      skipNativeAuth: true,
      // Without this the Google SDK is not loaded on Android at all and
      // the picker never opens. The web client id it needs comes from
      // google-services.json, as R.string.default_web_client_id, so
      // there is nothing to paste here.
      providers: ['google.com'],
    },
  },

  server: {
    androidScheme: 'https',
    // Android has refused cleartext traffic by default since API 28, on top
    // of the mixed-content rule above. Both have to give way for a backend
    // running on a laptop at http://192.168.x.x, and neither should give way
    // for the APK that gets handed to anyone else.
    cleartext: LAN_TEST,
  },
}

export default config
```

5.2 Android Application Configuration

5.2.1 CFG-007 — build.gradle

Code ID	CFG-007
File Name	build.gradle
File Path	SanctDemo/android/app/build.gradle
Language / Type	Gradle (Groovy)
Lines	97
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/android/app/build.gradle. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — build.gradle

```
apply plugin: 'com.android.application'

// Release signing, and the version the app reports.
//
// Android identifies an app by the key that signed it. While SanctiWalk was
// signed with the debug key, every build carried whatever debug keystore the
// build machine happened to have - so an "update" built elsewhere could not
// install over an existing copy at all, and the only way out for the user was
// to uninstall and lose their saved parish, reminders and subscriptions.
//
// keystore.properties and the .keystore beside it are deliberately NOT in
// git. They must be backed up: lose them and no future build can ever update
// an installed SanctiWalk again.
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('keystore.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

// The version comes from package.json, so the web app and the APK can never
// disagree about which release this is. versionCode has to be an integer that
// only ever increases, which is what the in-app update check compares.
def packageJson = new groovy.json.JsonSlurper().parse(rootProject.file('../package.json'))
def semver = (packageJson.version ?: '1.0.0').toString().tokenize('.')
def appVersionName = semver.join('.')
def appVersionCode = semver[0].toInteger() * 10000 + semver[1].toInteger() * 100 + semver[2].toInteger()

android {
    namespace = "ph.edu.sti.sanctiwalk"
```

```
    compileSdk = rootProject.ext.compileSdkVersion
    defaultConfig {
        applicationId "ph.edu.sti.sanctiwalk"
        minSdkVersion rootProject.ext.minSdkVersion
        targetSdkVersion rootProject.ext.targetSdkVersion
        versionCode appVersionCode
        versionName appVersionName
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
        aaptOptions {
             // Files and dirs to omit from the packaged assets dir, modified to accommodate modern web apps.
             // Default: https://android.googlesource.com/platform/frameworks/base/+/282e181b58cf72b6ca770dc7ca5f91f135444502/tools/aapt/AaptAssets.cpp#61
            ignoreAssetsPattern = '!.svn:!.git:!.ds_store:!*.scc:.*:!CVS:!thumbs.db:!picasa.ini:!*~'
        }
    }
    signingConfigs {
        release {
            if (keystorePropertiesFile.exists()) {
                storeFile rootProject.file(keystoreProperties['storeFile'])
                storePassword keystoreProperties['storePassword']
                keyAlias keystoreProperties['keyAlias']
                keyPassword keystoreProperties['keyPassword']
            }
        }
    }

    buildTypes {
        release {
            // Left off deliberately. The app is a WebView around a built web
            // bundle; there is almost no Java to shrink, and R8 has broken
            // Capacitor plugin reflection here before. The APK is the same
            // size either way.
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
            if (keystorePropertiesFile.exists()) {
                signingConfig signingConfigs.release
            }
        }
    }
}

repositories {
    flatDir{
        dirs '../capacitor-cordova-android-plugins/src/main/libs', 'libs'
    }
}

dependencies {
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation "androidx.appcompat:appcompat:$androidxAppCompatVersion"
    implementation "androidx.coordinatorlayout:coordinatorlayout:$androidxCoordinatorLayoutVersion"
    implementation "androidx.core:core-splashscreen:$coreSplashScreenVersion"
    implementation project(':capacitor-android')
    testImplementation "junit:junit:$junitVersion"
    androidTestImplementation "androidx.test.ext:junit:$androidxJunitVersion"
    androidTestImplementation "androidx.test.espresso:espresso-core:$androidxEspressoCoreVersion"
    implementation project(':capacitor-cordova-android-plugins')
}

apply from: 'capacitor.build.gradle'
```

```
try {
    def servicesJSON = file('google-services.json')
    if (servicesJSON.text) {
        apply plugin: 'com.google.gms.google-services'
    }
} catch(Exception e) {
    logger.info("google-services.json not found, google-services plugin not applied. Push Notifications won't work")
}
```

5.2.2 CFG-008 — variables.gradle

Code ID	CFG-008
File Name	variables.gradle
File Path	SanctDemo/android/variables.gradle
Language / Type	Gradle (Groovy)
Lines	16
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/android/variables.gradle. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — variables.gradle

```
ext {
    minSdkVersion = 24
    compileSdkVersion = 36
    targetSdkVersion = 36
    androidxActivityVersion = '1.11.0'
    androidxAppCompatVersion = '1.7.1'
    androidxCoordinatorLayoutVersion = '1.3.0'
    androidxCoreVersion = '1.17.0'
    androidxFragmentVersion = '1.8.9'
    coreSplashScreenVersion = '1.2.0'
    androidxWebkitVersion = '1.14.0'
    junitVersion = '4.13.2'
    androidxJunitVersion = '1.3.0'
    androidxEspressoCoreVersion = '3.7.0'
    cordovaAndroidVersion = '14.0.1'
}
```

5.2.3 CFG-009 — AndroidManifest.xml

Code ID	CFG-009
File Name	AndroidManifest.xml
File Path	SanctDemo/android/app/src/main/AndroidManifest.xml
Language / Type	XML
Lines	95
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctDemo/android/app/src/main/AndroidManifest.xml. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <!-- Portrait only. Every screen here is laid out as a single phone
             column - the parish band, the bulletin rail, the rosary - and
             rotated there is no second column for the extra width to become,
             so landscape only made each screen shorter. That broke the
             modals outright: the rosary settings card is about 530dp tall,
             which in a 360dp-high landscape window overflowed by 84dp at
             each end and took its own close button off the top of the
             screen with nothing left to scroll. The modals are height-safe
             now as well, because this lock does not apply to the same app
             opened in a browser. -->
        <activity
            android:screenOrientation="portrait"
            android:configChanges="orientation|keyboardHidden|keyboard|screenSize|locale|smallestScreenSize|screenLayout|uiMode|navigation|density"
            android:name=".MainActivity"
            android:label="@string/title_activity_main"
            android:theme="@style/AppTheme.NoActionBarLaunch"
            android:launchMode="singleTask"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
```

```
        </intent-filter>

    </activity>

    <provider
        android:name="androidx.core.content.FileProvider"
        android:authorities="${applicationId}.fileprovider"
        android:exported="false"
        android:grantUriPermissions="true">
        <meta-data
            android:name="android.support.FILE_PROVIDER_PATHS"
            android:resource="@xml/file_paths"></meta-data>
    </provider>
</application>

<!-- Permissions -->

<uses-permission android:name="android.permission.INTERNET" />

<!-- The Scan tab opens the camera for AI recognition and QR codes. A
     WebView only grants getUserMedia when the app also holds the OS
     permission, so declaring INTERNET alone is not enough. -->
<uses-permission android:name="android.permission.CAMERA" />

<!-- The Rosary's glow responds to the voice of whoever is praying, which
     it measures with a loudness meter over the microphone
     (public/rosary/index.html, startMeter). Leaving these undeclared did
     not merely disable that: Capacitor maps an AUDIO_CAPTURE request from
     the WebView onto RECORD_AUDIO and launches it, the framework refuses
     an undeclared permission outright, and the result came back to
     BridgeWebChromeClient twice - the second call to deny() threw
     IllegalStateException on the main thread and took the whole app down.
     That was the Rosary "crashing": a FATAL EXCEPTION, not a slow page.
     MODIFY_AUDIO_SETTINGS is requested alongside RECORD_AUDIO by that
     same Capacitor code path, so both have to be here. -->
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />

<!-- Mass reminders, and the "tell me when this opens" bells on the
     schedule, ministries and sacraments.

     Android 13 made posting a notification a runtime permission. The
     app already asked for it through the Capacitor plugin and the
     answer was always no - silently, with no system dialog, because
     a permission the manifest does not declare cannot be granted and
     requestPermissions() just returns denied. So notifications were
     the one thing the app never prompted for on install, and every
     Mass change notice went nowhere. It has to be declared here for
     the prompt to exist at all. -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<!-- Parish detection and walking directions. Coarse is listed too so the
     app still installs and degrades gracefully on a device that refuses
     precise location. -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<!-- Optional, deliberately. Marking the camera required would stop the
     app installing on any device without one, and every camera feature
```

```
        already has a working fallback (manual station picking). -->
    <uses-feature android:name="android.hardware.camera" android:required="false" />
</manifest>
```

5.3 Unity Project Configuration

5.3.1 CFG-010 — ChurchMarkers.asset

Code ID	CFG-010
File Name	ChurchMarkers.asset
File Path	SanctiWalkAR/Assets/AR/ChurchMarkers.asset
Language / Type	ASSET
Lines	49
Functions / Classes	None detected by the parser for this file type.
Related Files (its imports)	No project-local imports.
Where It Is Used	Not imported by another file (entry point, config, or standalone).
How to Recognize It	Open SanctiWalkAR/Assets/AR/ChurchMarkers.asset. The file begins with code rather than a comment.

Purpose and Explanation

This file carries no header comment, so no description is quoted here. Its behaviour is visible in the listing below.

Complete Source Code — ChurchMarkers.asset

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_ObjectHideFlags: 0
  m_CorrespondingSourceObject: {fileID: 0}
  m_PrefabInstance: {fileID: 0}
  m_PrefabAsset: {fileID: 0}
  m_GameObject: {fileID: 0}
  m_Enabled: 1
  m_EditorHideFlags: 0
  m_Script: {fileID: 11500000, guid: 848409ee8ba454f42a5ddff137688a27, type: 3}
  m_Name: ChurchMarkers
  m_EditorClassIdentifier: Unity.XR.ARSubsystems::UnityEngine.XR.ARSubsystems.XRReferenceImageLibrary
  m_GuidLow: 5221858799135266310
  m_GuidHigh: 845307247789671090
  m_DataStore:
    m_Storage: []
  m_Images:
  - m_SerializedGuid:
      m_GuidLow: 5067246456670324555
      m_GuidHigh: 4342390500712006807
    m_SerializedTextureGuid:
      m_GuidLow: 41743421549136862
      m_GuidHigh: 16457455228540139099
    m_Size: {x: 0.18, y: 0.24000001}
    m_SpecifySize: 1
    m_Name: entrance
    m_Texture: {fileID: 2800000, guid: b5254bde4d6b00945bb607c455a064e4, type: 3}
```

```
- m_SerializedGuid:
    m_GuidLow: 5182273698385290287
    m_GuidHigh: 419889082416516269
  m_SerializedTextureGuid:
    m_GuidLow: 4099435998160304822
    m_GuidHigh: 18023605617680242794
  m_Size: {x: 0.21, y: 0.297}
  m_SpecifySize: 1
  m_Name: mhcp
  m_Texture: {fileID: 2800000, guid: 11cb3ab61f5238e46ad890edd0b520fa, type: 3}
- m_SerializedGuid:
    m_GuidLow: 5170556117758646684
    m_GuidHigh: 2755764151916527804
  m_SerializedTextureGuid:
    m_GuidLow: 13412928489686837802
    m_GuidHigh: 6418539862898045656
  m_Size: {x: 0.21, y: 0.297}
  m_SpecifySize: 1
  m_Name: src
  m_Texture: {fileID: 2800000, guid: 89802a2a4b65ba24d8f2b898ff3b1359, type: 3}
```

Master Source Code Index

Code ID	File Name	File Path	Type	Lines
APP-001	main.tsx	SanctDemo/src/main.tsx	TypeScript (React, TSX)	120
APP-002	App.tsx	SanctDemo/src/App.tsx	TypeScript (React, TSX)	1664
APP-003	RootGate.tsx	SanctDemo/src/components/RootGate.tsx	TypeScript (React, TSX)	70
APP-004	PhoneContainer.tsx	SanctDemo/src/components/PhoneContainer.tsx	TypeScript (React, TSX)	193
APP-005	Landing.tsx	SanctDemo/src/components/Landing.tsx	TypeScript (React, TSX)	228
APP-006	AdminPortal.tsx	SanctDemo/src/components/AdminPortal.tsx	TypeScript (React, TSX)	980
APP-007	ApplicationForm.tsx	SanctDemo/src/components/ApplicationForm.tsx	TypeScript (React, TSX)	376
APP-008	ArPlaceholder.tsx	SanctDemo/src/components/ArPlaceholder.tsx	TypeScript (React, TSX)	75
APP-009	ArScannerPanel.tsx	SanctDemo/src/components/ArScannerPanel.tsx	TypeScript (React, TSX)	83
APP-010	ArTour.tsx	SanctDemo/src/components/ArTour.tsx	TypeScript (React, TSX)	1392
APP-011	AvailabilityBadge.tsx	SanctDemo/src/components/AvailabilityBadge.tsx	TypeScript (React, TSX)	38
APP-012	BulletinRail.tsx	SanctDemo/src/components/BulletinRail.tsx	TypeScript (React, TSX)	232
APP-013	ChangeParishModal.tsx	SanctDemo/src/components/ChangeParishModal.tsx	TypeScript (React, TSX)	80
APP-014	ChurchHistory.tsx	SanctDemo/src/components/ChurchHistory.tsx	TypeScript (React, TSX)	423
APP-015	ChurchHistoryCard.tsx	SanctDemo/src/components/ChurchHistoryCard.tsx	TypeScript (React, TSX)	115
APP-016	CompanionTab.tsx	SanctDemo/src/components/CompanionTab.tsx	TypeScript (React, TSX)	248
APP-017	CustomDioceseMap.tsx	SanctDemo/src/components/CustomDioceseMap.tsx	TypeScript (React, TSX)	54
APP-018	DailyRosary.tsx	SanctDemo/src/components/DailyRosary.tsx	TypeScript (React, TSX)	203
APP-019	Dashboard.tsx	SanctDemo/src/components/Dashboard.tsx	TypeScript (React, TSX)	271
APP-020	DioceseMap.tsx	SanctDemo/src/components/DioceseMap.tsx	TypeScript (React, TSX)	200
APP-021	EditProfileCard.tsx	SanctDemo/src/components/EditProfileCard.tsx	TypeScript (React, TSX)	234
APP-022	ExploreTab.tsx	SanctDemo/src/components/ExploreTab.tsx	TypeScript (React, TSX)	156
APP-023	HistoryQuiz.tsx	SanctDemo/src/components/HistoryQuiz.tsx	TypeScript (React, TSX)	200
APP-024	HomeHero.tsx	SanctDemo/src/components/HomeHero.tsx	TypeScript (React, TSX)	211
APP-025	HomeQuickLinks.tsx	SanctDemo/src/components/HomeQuickLinks.tsx	TypeScript (React, TSX)	86
APP-026	ItemDetailSections.tsx	SanctDemo/src/components/ItemDetailSections.tsx	TypeScript (React, TSX)	112
APP-027	LoginModal.tsx	SanctDemo/src/components/LoginModal.tsx	TypeScript (React, TSX)	491
APP-028	MapPlaceSheet.tsx	SanctDemo/src/components/MapPlaceSheet.tsx	TypeScript (React, TSX)	237
APP-029	MapTab.tsx	SanctDemo/src/components/MapTab.tsx	TypeScript (React, TSX)	440
APP-030	MassNowBadge.tsx	SanctDemo/src/components/MassNowBadge.tsx	TypeScript (React, TSX)	63
APP-031	MassSchedule.tsx	SanctDemo/src/components/MassSchedule.tsx	TypeScript (React, TSX)	209
APP-032	MassScheduleCard.tsx	SanctDemo/src/components/MassScheduleCard.tsx	TypeScript (React, TSX)	309
APP-033	MeTab.tsx	SanctDemo/src/components/MeTab.tsx	TypeScript (React, TSX)	429
APP-034	MinistriesTab.tsx	SanctDemo/src/components/MinistriesTab.tsx	TypeScript (React, TSX)	412
APP-035	MinistrySacramentPair.tsx	SanctDemo/src/components/MinistrySacramentPair.tsx	TypeScript (React, TSX)	117
APP-036	MyApplications.tsx	SanctDemo/src/components/MyApplications.tsx	TypeScript (React, TSX)	124
APP-037	MyApplicationsPage.tsx	SanctDemo/src/components/MyApplicationsPage.tsx	TypeScript (React, TSX)	203
APP-038	NotificationsCard.tsx	SanctDemo/src/components/NotificationsCard.tsx	TypeScript (React, TSX)	93
APP-039	NotificationsPage.tsx	SanctDemo/src/components/NotificationsPage.tsx	TypeScript (React, TSX)	128
APP-040	Onboarding.tsx	SanctDemo/src/components/Onboarding.tsx	TypeScript (React, TSX)	154
APP-041	ParishCard.tsx	SanctDemo/src/components/ParishCard.tsx	TypeScript (React, TSX)	64
APP-042	ParishContentEditor.tsx	SanctDemo/src/components/ParishContentEditor.tsx	TypeScript (React, TSX)	307
APP-043	ParishHero.tsx	SanctDemo/src/components/ParishHero.tsx	TypeScript (React, TSX)	220
APP-044	ParishListRow.tsx	SanctDemo/src/components/ParishListRow.tsx	TypeScript (React, TSX)	118

APP-045	ParishSectionHeader.tsx	SanctDemo/src/components/ParishSectionHeader.tsx	TypeScript (React, TSX)	201
APP-046	ParishWelcomeHeader.tsx	SanctDemo/src/components/ParishWelcomeHeader.tsx	TypeScript (React, TSX)	394
APP-047	PilgrimQuiz.tsx	SanctDemo/src/components/PilgrimQuiz.tsx	TypeScript (React, TSX)	220
APP-048	PrayScreen.tsx	SanctDemo/src/components/PrayScreen.tsx	TypeScript (React, TSX)	42
APP-049	PresenceBanner.tsx	SanctDemo/src/components/PresenceBanner.tsx	TypeScript (React, TSX)	26
APP-050	PresenceSheet.tsx	SanctDemo/src/components/PresenceSheet.tsx	TypeScript (React, TSX)	142
APP-051	PreviewVideo.tsx	SanctDemo/src/components/PreviewVideo.tsx	TypeScript (React, TSX)	81
APP-052	PwaBanner.tsx	SanctDemo/src/components/PwaBanner.tsx	TypeScript (React, TSX)	182
APP-053	RailDots.tsx	SanctDemo/src/components/RailDots.tsx	TypeScript (React, TSX)	122
APP-054	ReminderSettingsCard.tsx	SanctDemo/src/components/ReminderSettingsCard.tsx	TypeScript (React, TSX)	241
APP-055	RosarySettingsModal.tsx	SanctDemo/src/components/RosarySettingsModal.tsx	TypeScript (React, TSX)	202
APP-056	SacramentsTab.tsx	SanctDemo/src/components/SacramentsTab.tsx	TypeScript (React, TSX)	245
APP-057	SanctiHost.tsx	SanctDemo/src/components/SanctiHost.tsx	TypeScript (React, TSX)	490
APP-058	SanctiSheet.tsx	SanctDemo/src/components/SanctiSheet.tsx	TypeScript (React, TSX)	249
APP-059	ScannerChrome.tsx	SanctDemo/src/components/ScannerChrome.tsx	TypeScript (React, TSX)	130
APP-060	SearchScreen.tsx	SanctDemo/src/components/SearchScreen.tsx	TypeScript (React, TSX)	250
APP-061	SettingDisclosure.tsx	SanctDemo/src/components/SettingDisclosure.tsx	TypeScript (React, TSX)	73
APP-062	SignInFirst.tsx	SanctDemo/src/components/SignInFirst.tsx	TypeScript (React, TSX)	53
APP-063	SimulatorPanel.tsx	SanctDemo/src/components/SimulatorPanel.tsx	TypeScript (React, TSX)	133
APP-064	Spotlight.tsx	SanctDemo/src/components/Spotlight.tsx	TypeScript (React, TSX)	269
APP-065	TripPlanner.tsx	SanctDemo/src/components/TripPlanner.tsx	TypeScript (React, TSX)	160
APP-066	TutorialRunner.tsx	SanctDemo/src/components/TutorialRunner.tsx	TypeScript (React, TSX)	157
APP-067	UpdateBanner.tsx	SanctDemo/src/components/UpdateBanner.tsx	TypeScript (React, TSX)	105
APP-068	VerseCard.tsx	SanctDemo/src/components/VerseCard.tsx	TypeScript (React, TSX)	54
APP-069	WatchBell.tsx	SanctDemo/src/components/WatchBell.tsx	TypeScript (React, TSX)	93
APP-070	PresenceContext.tsx	SanctDemo/src/context/PresenceContext.tsx	TypeScript (React, TSX)	213
APP-071	activityLog.ts	SanctDemo/src/lib/activityLog.ts	TypeScript	47
APP-072	analytics.ts	SanctDemo/src/lib/analytics.ts	TypeScript	243
APP-073	announcementImage.ts	SanctDemo/src/lib/announcementImage.ts	TypeScript	85
APP-074	announcements.ts	SanctDemo/src/lib/announcements.ts	TypeScript	235
APP-075	apiBase.ts	SanctDemo/src/lib/apiBase.ts	TypeScript	41
APP-076	appKey.ts	SanctDemo/src/lib/appKey.ts	TypeScript	37
APP-077	appUpdate.ts	SanctDemo/src/lib/appUpdate.ts	TypeScript	134
APP-078	applicationFilters.ts	SanctDemo/src/lib/applicationFilters.ts	TypeScript	225
APP-079	applicationForm.ts	SanctDemo/src/lib/applicationForm.ts	TypeScript	358
APP-080	applicationView.ts	SanctDemo/src/lib/applicationView.ts	TypeScript	146
APP-081	applications.ts	SanctDemo/src/lib/applications.ts	TypeScript	190
APP-082	arApp.ts	SanctDemo/src/lib/arApp.ts	TypeScript	136
APP-083	askSancti.ts	SanctDemo/src/lib/askSancti.ts	TypeScript	84
APP-084	availability.ts	SanctDemo/src/lib/availability.ts	TypeScript	102
APP-085	avatar.ts	SanctDemo/src/lib/avatar.ts	TypeScript	119
APP-086	churches.ts	SanctDemo/src/lib/churches.ts	TypeScript	91
APP-087	contentTranslations.ts	SanctDemo/src/lib/contentTranslations.ts	TypeScript	184
APP-088	contrast.ts	SanctDemo/src/lib/contrast.ts	TypeScript	130
APP-089	deviceNotifications.ts	SanctDemo/src/lib/deviceNotifications.ts	TypeScript	194
APP-090	displayMode.ts	SanctDemo/src/lib/displayMode.ts	TypeScript	74
APP-091	exportApplications.ts	SanctDemo/src/lib/exportApplications.ts	TypeScript	211

APP-092	findScroller.ts	SanctDemo/src/lib/findScroller.ts	TypeScript	36
APP-093	followedParishes.ts	SanctDemo/src/lib/followedParishes.ts	TypeScript	100
APP-094	googleAuth.ts	SanctDemo/src/lib/googleAuth.ts	TypeScript	151
APP-095	heading.ts	SanctDemo/src/lib/heading.ts	TypeScript	154
APP-096	historyNarration.ts	SanctDemo/src/lib/historyNarration.ts	TypeScript	114
APP-097	historyQuiz.ts	SanctDemo/src/lib/historyQuiz.ts	TypeScript	183
APP-098	homeParish.ts	SanctDemo/src/lib/homeParish.ts	TypeScript	38
APP-099	idToken.ts	SanctDemo/src/lib/idToken.ts	TypeScript	34
APP-100	itemContent.ts	SanctDemo/src/lib/itemContent.ts	TypeScript	204
APP-101	language.ts	SanctDemo/src/lib/language.ts	TypeScript	145
APP-102	liturgical.ts	SanctDemo/src/lib/liturgical.ts	TypeScript	195
APP-103	liturgicalColours.ts	SanctDemo/src/lib/liturgicalColours.ts	TypeScript	53
APP-104	mapAccuracy.ts	SanctDemo/src/lib/mapAccuracy.ts	TypeScript	97
APP-105	mapMarkers.ts	SanctDemo/src/lib/mapMarkers.ts	TypeScript	72
APP-106	mapSearch.ts	SanctDemo/src/lib/mapSearch.ts	TypeScript	250
APP-107	massScheduleWatch.ts	SanctDemo/src/lib/massScheduleWatch.ts	TypeScript	243
APP-108	ministryApplication.ts	SanctDemo/src/lib/ministryApplication.ts	TypeScript	149
APP-109	ministrySearch.ts	SanctDemo/src/lib/ministrySearch.ts	TypeScript	83
APP-110	modelRetry.ts	SanctDemo/src/lib/modelRetry.ts	TypeScript	180
APP-111	mysteries.ts	SanctDemo/src/lib/mysteries.ts	TypeScript	39
APP-112	navigation.ts	SanctDemo/src/lib/navigation.ts	TypeScript	255
APP-113	notifications.ts	SanctDemo/src/lib/notifications.ts	TypeScript	211
APP-114	openingSubscriptions.ts	SanctDemo/src/lib/openingSubscriptions.ts	TypeScript	118
APP-115	pageEvents.ts	SanctDemo/src/lib/pageEvents.ts	TypeScript	185
APP-116	parishIds.ts	SanctDemo/src/lib/parishIds.ts	TypeScript	74
APP-117	parishPhotos.ts	SanctDemo/src/lib/parishPhotos.ts	TypeScript	56
APP-118	parishSelection.ts	SanctDemo/src/lib/parishSelection.ts	TypeScript	49
APP-119	parishTheme.ts	SanctDemo/src/lib/parishTheme.ts	TypeScript	90
APP-120	pilgrimage.ts	SanctDemo/src/lib/pilgrimage.ts	TypeScript	203
APP-121	presence.ts	SanctDemo/src/lib/presence.ts	TypeScript	114
APP-122	project.ts	SanctDemo/src/lib/project.ts	TypeScript	43
APP-123	qr.ts	SanctDemo/src/lib/qr.ts	TypeScript	122
APP-124	recordPageEvent.ts	SanctDemo/src/lib/recordPageEvent.ts	TypeScript	34
APP-125	recordVisit.ts	SanctDemo/src/lib/recordVisit.ts	TypeScript	127
APP-126	registerServiceWorker.ts	SanctDemo/src/lib/registerServiceWorker.ts	TypeScript	46
APP-127	reminderSchedule.ts	SanctDemo/src/lib/reminderSchedule.ts	TypeScript	337
APP-128	rosarySettings.ts	SanctDemo/src/lib/rosarySettings.ts	TypeScript	71
APP-129	routing.ts	SanctDemo/src/lib/routing.ts	TypeScript	195
APP-130	sancti.ts	SanctDemo/src/lib/sancti.ts	TypeScript	749
APP-131	sanctiAnswers.ts	SanctDemo/src/lib/sanctiAnswers.ts	TypeScript	370
APP-132	sanctiGrounding.ts	SanctDemo/src/lib/sanctiGrounding.ts	TypeScript	244
APP-133	scanNarration.ts	SanctDemo/src/lib/scanNarration.ts	TypeScript	102
APP-134	scanQuota.ts	SanctDemo/src/lib/scanQuota.ts	TypeScript	104
APP-135	scanQuotaStore.ts	SanctDemo/src/lib/scanQuotaStore.ts	TypeScript	177
APP-136	schedule.ts	SanctDemo/src/lib/schedule.ts	TypeScript	177
APP-137	simulatorMode.ts	SanctDemo/src/lib/simulatorMode.ts	TypeScript	103
APP-138	speech.ts	SanctDemo/src/lib/speech.ts	TypeScript	227

APP-139	spotlight.ts	SanctDemo/src/lib/spotlight.ts	TypeScript	137
APP-140	sundayMasses.ts	SanctDemo/src/lib/sundayMasses.ts	TypeScript	199
APP-141	tutorial.ts	SanctDemo/src/lib/tutorial.ts	TypeScript	151
APP-142	ui.ts	SanctDemo/src/lib/ui.ts	TypeScript	147
APP-143	useAuthUid.ts	SanctDemo/src/lib/useAuthUid.ts	TypeScript	19
APP-144	useCamera.ts	SanctDemo/src/lib/useCamera.ts	TypeScript	322
APP-145	useDeviceHeading.ts	SanctDemo/src/lib/useDeviceHeading.ts	TypeScript	231
APP-146	useDragSafeClicks.ts	SanctDemo/src/lib/useDragSafeClicks.ts	TypeScript	58
APP-147	useLanguage.ts	SanctDemo/src/lib/useLanguage.ts	TypeScript	53
APP-148	useMassScheduleNotice.ts	SanctDemo/src/lib/useMassScheduleNotice.ts	TypeScript	90
APP-149	useNarration.ts	SanctDemo/src/lib/useNarration.ts	TypeScript	92
APP-150	useOpeningNotices.ts	SanctDemo/src/lib/useOpeningNotices.ts	TypeScript	141
APP-151	useParishContent.ts	SanctDemo/src/lib/useParishContent.ts	TypeScript	28
APP-152	useParishContents.ts	SanctDemo/src/lib/useParishContents.ts	TypeScript	60
APP-153	useQrScanner.ts	SanctDemo/src/lib/useQrScanner.ts	TypeScript	178
APP-154	useReminders.ts	SanctDemo/src/lib/useReminders.ts	TypeScript	134
APP-155	useSimulatorMode.ts	SanctDemo/src/lib/useSimulatorMode.ts	TypeScript	43
APP-156	userProfile.ts	SanctDemo/src/lib/userProfile.ts	TypeScript	119
APP-157	verseBackdrops.ts	SanctDemo/src/lib/verseBackdrops.ts	TypeScript	57
APP-158	verses.ts	SanctDemo/src/lib/verses.ts	TypeScript	97
APP-159	visits.ts	SanctDemo/src/lib/visits.ts	TypeScript	159
APP-160	data.ts	SanctDemo/src/data.ts	TypeScript	635
APP-161	types.ts	SanctDemo/src/types.ts	TypeScript	214
APP-162	index.css	SanctDemo/src/index.css	CSS	6125
API-001	server.ts	SanctDemo/server.ts	TypeScript	1080
API-002	firebase.ts	SanctDemo/src/lib/firebase.ts	TypeScript	11
API-003	authFlow.ts	SanctDemo/src/lib/authFlow.ts	TypeScript	262
API-004	parishContent.ts	SanctDemo/src/lib/parishContent.ts	TypeScript	186
API-005	parishContentStore.ts	SanctDemo/src/lib/parishContentStore.ts	TypeScript	59
API-006	firestore.rules	SanctDemo/firestore.rules	Firestore Security Rules	409
API-007	firestore.indexes.json	SanctDemo/firestore.indexes.json	JSON	5
API-008	AdminAnalytics.tsx	SanctDemo/src/admin/AdminAnalytics.tsx	TypeScript (React, TSX)	522
API-009	AdminAnnouncements.tsx	SanctDemo/src/admin/AdminAnnouncements.tsx	TypeScript (React, TSX)	575
API-010	AdminApp.tsx	SanctDemo/src/admin/AdminApp.tsx	TypeScript (React, TSX)	135
API-011	AdminApplicationDetail.tsx	SanctDemo/src/admin/AdminApplicationDetail.tsx	TypeScript (React, TSX)	307
API-012	AdminApplications.tsx	SanctDemo/src/admin/AdminApplications.tsx	TypeScript (React, TSX)	386
API-013	AdminAvailability.tsx	SanctDemo/src/admin/AdminAvailability.tsx	TypeScript (React, TSX)	189
API-014	AdminContent.tsx	SanctDemo/src/admin/AdminContent.tsx	TypeScript (React, TSX)	318
API-015	AdminDashboard.tsx	SanctDemo/src/admin/AdminDashboard.tsx	TypeScript (React, TSX)	261
API-016	AdminItems.tsx	SanctDemo/src/admin/AdminItems.tsx	TypeScript (React, TSX)	201
API-017	AdminLogin.tsx	SanctDemo/src/admin/AdminLogin.tsx	TypeScript (React, TSX)	114
API-018	AdminMassSchedule.tsx	SanctDemo/src/admin/AdminMassSchedule.tsx	TypeScript (React, TSX)	348
API-019	AdminNav.tsx	SanctDemo/src/admin/AdminNav.tsx	TypeScript (React, TSX)	112
API-020	AdminTiles.tsx	SanctDemo/src/admin/AdminTiles.tsx	TypeScript (React, TSX)	178
API-021	ParishStatus.tsx	SanctDemo/src/admin/ParishStatus.tsx	TypeScript (React, TSX)	147
API-022	StatusBadge.tsx	SanctDemo/src/admin/StatusBadge.tsx	TypeScript (React, TSX)	30
MAP-001	DioceseMapLive.tsx	SanctDemo/src/components/DioceseMapLive.tsx	TypeScript (React, TSX)	1702

MAP-002	NavigationOverlay.tsx	SanctDemo/src/components/NavigationOverlay.tsx	TypeScript (React, TSX)	244
MAP-003	CompassControl.tsx	SanctDemo/src/components/CompassControl.tsx	TypeScript (React, TSX)	180
MAP-004	followCamera.ts	SanctDemo/src/lib/followCamera.ts	TypeScript	190
MAP-005	geo.ts	SanctDemo/src/lib/geo.ts	TypeScript	42
MAP-006	travelModes.ts	SanctDemo/src/lib/travelModes.ts	TypeScript	94
MAP-007	place.html	SanctiWalkAR/Tools/viewer/place.html	HTML	864
MAP-008	index.html	SanctiWalkAR/Tools/viewer/index.html	HTML	383
MAP-009	export.html	SanctiWalkAR/Tools/viewer/export.html	HTML	172
MAP-010	save_server.py	SanctiWalkAR/Tools/viewer/save_server.py	Python	77
MAP-011	deploy-stations.py	SanctiWalkAR/Tools/deploy-stations.py	Python	302
MAP-012	make-parish-marker.py	SanctiWalkAR/Tools/make-parish-marker.py	Python	216
MAP-013	make-marker-pdf.py	SanctiWalkAR/Tools/make-marker-pdf.py	Python	68
MAP-014	marker-pose.py	SanctiWalkAR/Tools/marker-pose.py	Python	223
MAP-015	glb-to-obj-indexed.py	SanctiWalkAR/Tools/glb-to-obj-indexed.py	Python	200
MAP-016	optimise-scan.py	SanctiWalkAR/Tools/optimise-scan.py	Python	151
AR-001	AdminAccess.cs	SanctiWalkAR/Assets/Scripts/AdminAccess.cs	C#	110
AR-002	AppLanguage.cs	SanctiWalkAR/Assets/Scripts/AppLanguage.cs	C#	71
AR-003	ArFlowUI.cs	SanctiWalkAR/Assets/Scripts/ArFlowUI.cs	C#	1423
AR-004	ChurchFrame.cs	SanctiWalkAR/Assets/Scripts/ChurchFrame.cs	C#	124
AR-005	ChurchOriginTest.cs	SanctiWalkAR/Assets/Scripts/ChurchOriginTest.cs	C#	2602
AR-006	DeviceSpeech.cs	SanctiWalkAR/Assets/Scripts/DeviceSpeech.cs	C#	194
AR-007	FirstPersonWalkthrough.cs	SanctiWalkAR/Assets/Scripts/FirstPersonWalkthrough.cs	C#	166
AR-008	MarkerScan.cs	SanctiWalkAR/Assets/Scripts/MarkerScan.cs	C#	83
AR-009	MultiSetVps.cs	SanctiWalkAR/Assets/Scripts/MultiSetVps.cs	C#	501
AR-010	MuseumStation.cs	SanctiWalkAR/Assets/Scripts/MuseumStation.cs	C#	242
AR-011	MuseumStationsGenerated.cs	SanctiWalkAR/Assets/Scripts/MuseumStationsGenerated.cs	C#	710
AR-012	MuseumUI.cs	SanctiWalkAR/Assets/Scripts/MuseumUI.cs	C#	1154
AR-013	OriginPicker.cs	SanctiWalkAR/Assets/Scripts/OriginPicker.cs	C#	154
AR-014	POIDirectory.cs	SanctiWalkAR/Assets/Scripts/POIDirectory.cs	C#	159
AR-015	POIPanel.cs	SanctiWalkAR/Assets/Scripts/POIPanel.cs	C#	112
AR-016	PointOfInterest.cs	SanctiWalkAR/Assets/Scripts/PointOfInterest.cs	C#	195
AR-017	ScanSimulator.cs	SanctiWalkAR/Assets/Scripts/ScanSimulator.cs	C#	69
AR-018	SceneGizmos.cs	SanctiWalkAR/Assets/Scripts/SceneGizmos.cs	C#	57
AR-019	SiteReadiness.cs	SanctiWalkAR/Assets/Scripts/SiteReadiness.cs	C#	274
AR-020	StationBubble.cs	SanctiWalkAR/Assets/Scripts/StationBubble.cs	C#	426
AR-021	StationCalibrator.cs	SanctiWalkAR/Assets/Scripts/StationCalibrator.cs	C#	454
AR-022	StationMarker.cs	SanctiWalkAR/Assets/Scripts/StationMarker.cs	C#	120
AR-023	StationRecording.cs	SanctiWalkAR/Assets/Scripts/StationRecording.cs	C#	250
AR-024	ARProjectSetup.cs	SanctiWalkAR/Assets/Editor/ARProjectSetup.cs	C#	587
AR-025	ArDeepLink.cs	SanctiWalkAR/Assets/Editor/ArDeepLink.cs	C#	179
AR-026	AuthoringPreview.cs	SanctiWalkAR/Assets/Editor/AuthoringPreview.cs	C#	123
AR-027	DesktopTestBuild.cs	SanctiWalkAR/Assets/Editor/DesktopTestBuild.cs	C#	60
AR-028	ExportPois.cs	SanctiWalkAR/Assets/Editor/ExportPois.cs	C#	247
AR-029	ModelCheck.cs	SanctiWalkAR/Assets/Editor/ModelCheck.cs	C#	66
AR-030	MultiSetCredentials.cs	SanctiWalkAR/Assets/Editor/MultiSetCredentials.cs	C#	65
AR-031	MuseumUICapture.cs	SanctiWalkAR/Assets/Editor/MuseumUICapture.cs	C#	140
AR-032	ScanFlowDump.cs	SanctiWalkAR/Assets/Editor/ScanFlowDump.cs	C#	115

AR-033	StationEditorScene.cs	SanctiWalkAR/Assets/Editor/StationEditorScene.cs	C#	403
CFG-001	package.json	SanctDemo/package.json	JSON	76
CFG-002	vite.config.ts	SanctDemo/vite.config.ts	TypeScript	119
CFG-003	tsconfig.json	SanctDemo/tsconfig.json	JSON	27
CFG-004	vitest.config.ts	SanctDemo/vitest.config.ts	TypeScript	25
CFG-005	vercel.json	SanctDemo/vercel.json	JSON	43
CFG-006	capacitor.config.ts	SanctDemo/capacitor.config.ts	TypeScript	69
CFG-007	build.gradle	SanctDemo/android/app/build.gradle	Gradle (Groovy)	97
CFG-008	variables.gradle	SanctDemo/android/variables.gradle	Gradle (Groovy)	16
CFG-009	AndroidManifest.xml	SanctDemo/android/app/src/main/AndroidManifest.xml	XML	95
CFG-010	ChurchMarkers.asset	SanctiWalkAR/Assets/AR/ChurchMarkers.asset	ASSET	49